Document Number: Dxxxx
Date: 2015-07-23
Revises: N4527
Reply to: Richard Smith
Google Inc
cxxeditor@gmail.com

Working Draft, Standard for Programming Language C++

Note: this is an early draft. It's known to be incomplet and incorrekt, and it has lots of bad formatting.

Contents

Contents ii

List of Tables x

List of Figures xiv

1 General 1
1.1 Scope . . . 1
1.2 Normative references . . . 1
1.3 Terms and definitions . . . 2
1.4 Implementation compliance . . . 4
1.5 Structure of this International Standard . . . 5
1.6 Syntax notation . . . 5
1.7 The C++ memory model . . . 6
1.8 The C++ object model . . . 7
1.9 Program execution . . . 7
1.10 Multi-threaded executions and data races . . . 11
1.11 Acknowledgments . . . 15

2 Lexical conventions 16
2.1 Separate translation . . . 16
2.2 Phases of translation . . . 16
2.3 Character sets . . . 17
2.4 Preprocessing tokens . . . 18
2.5 Alternative tokens . . . 19
2.6 Tokens . . . 20
2.7 Comments . . . 20
2.8 Header names . . . 20
2.9 Preprocessing numbers . . . 21
2.10 Identifiers . . . 21
2.11 Keywords . . . 22
2.12 Operators and punctuators . . . 22
2.13 Literals . . . 23

3 Basic concepts 33
3.1 Declarations and definitions . . . 33
3.2 One-definition rule . . . 35
3.3 Scope . . . 38
3.4 Name lookup . . . 44
3.5 Program and linkage . . . 58
3.6 Start and termination . . . 61
3.7 Storage duration . . . 64
3.8 Object lifetime . . . 68
3.9 Types . . . 72
3.10 Lvalues and rvalues . . . 77
3.11 Alignment . . . 79

4 Standard conversions **81**
4.1 Lvalue-to-rvalue conversion . . . 82
4.2 Array-to-pointer conversion . . . 82
4.3 Function-to-pointer conversion . . . 82
4.4 Qualification conversions . . . 83
4.5 Integral promotions . . . 83
4.6 Floating point promotion . . . 84
4.7 Integral conversions . . . 84
4.8 Floating point conversions . . . 84
4.9 Floating-integral conversions . . . 85
4.10 Pointer conversions . . . 85
4.11 Pointer to member conversions . . . 85
4.12 Boolean conversions . . . 86
4.13 Integer conversion rank . . . 86

5 Expressions **87**
5.1 Primary expressions . . . 90
5.2 Postfix expressions . . . 101
5.3 Unary expressions . . . 112
5.4 Explicit type conversion (cast notation) . . . 121
5.5 Pointer-to-member operators . . . 122
5.6 Multiplicative operators . . . 123
5.7 Additive operators . . . 123
5.8 Shift operators . . . 125
5.9 Relational operators . . . 125
5.10 Equality operators . . . 126
5.11 Bitwise AND operator . . . 127
5.12 Bitwise exclusive OR operator . . . 127
5.13 Bitwise inclusive OR operator . . . 127
5.14 Logical AND operator . . . 128
5.15 Logical OR operator . . . 128
5.16 Conditional operator . . . 128
5.17 Throwing an exception . . . 129
5.18 Assignment and compound assignment operators . . . 130
5.19 Comma operator . . . 131
5.20 Constant expressions . . . 131

6 Statements **136**
6.1 Labeled statement . . . 136
6.2 Expression statement . . . 136
6.3 Compound statement or block . . . 136
6.4 Selection statements . . . 137
6.5 Iteration statements . . . 138
6.6 Jump statements . . . 141
6.7 Declaration statement . . . 142
6.8 Ambiguity resolution . . . 144

7 Declarations **146**
7.1 Specifiers . . . 148
7.2 Enumeration declarations . . . 165
7.3 Namespaces . . . 168

7.4 The `asm` declaration . . . 181
7.5 Linkage specifications . . . 181
7.6 Attributes . . . 184

8 Declarators **190**
8.1 Type names . . . 191
8.2 Ambiguity resolution . . . 192
8.3 Meaning of declarators . . . 193
8.4 Function definitions . . . 206
8.5 Initializers . . . 209

9 Classes **225**
9.1 Class names . . . 228
9.2 Class members . . . 229
9.3 Member functions . . . 232
9.4 Static members . . . 235
9.5 Unions . . . 237
9.6 Bit-fields . . . 239
9.7 Nested class declarations . . . 239
9.8 Local class declarations . . . 241
9.9 Nested type names . . . 241

10 Derived classes **242**
10.1 Multiple base classes . . . 243
10.2 Member name lookup . . . 245
10.3 Virtual functions . . . 248
10.4 Abstract classes . . . 253

11 Member access control **255**
11.1 Access specifiers . . . 256
11.2 Accessibility of base classes and base class members . . . 258
11.3 Friends . . . 260
11.4 Protected member access . . . 263
11.5 Access to virtual functions . . . 264
11.6 Multiple access . . . 265
11.7 Nested classes . . . 265

12 Special member functions **266**
12.1 Constructors . . . 266
12.2 Temporary objects . . . 269
12.3 Conversions . . . 271
12.4 Destructors . . . 274
12.5 Free store . . . 277
12.6 Initialization . . . 278
12.7 Construction and destruction . . . 285
12.8 Copying and moving class objects . . . 287
12.9 Inheriting constructors . . . 295

13 Overloading **299**
13.1 Overloadable declarations . . . 299
13.2 Declaration matching . . . 301

13.3 Overload resolution . . . 302
13.4 Address of overloaded function . . . 322
13.5 Overloaded operators . . . 324
13.6 Built-in operators . . . 328

14 Templates **332**
14.1 Template parameters . . . 333
14.2 Names of template specializations . . . 336
14.3 Template arguments . . . 338
14.4 Type equivalence . . . 343
14.5 Template declarations . . . 344
14.6 Name resolution . . . 363
14.7 Template instantiation and specialization . . . 378
14.8 Function template specializations . . . 391

15 Exception handling **413**
15.1 Throwing an exception . . . 414
15.2 Constructors and destructors . . . 416
15.3 Handling an exception . . . 416
15.4 Exception specifications . . . 418
15.5 Special functions . . . 423

16 Preprocessing directives **425**
16.1 Conditional inclusion . . . 426
16.2 Source file inclusion . . . 428
16.3 Macro replacement . . . 429
16.4 Line control . . . 434
16.5 Error directive . . . 434
16.6 Pragma directive . . . 434
16.7 Null directive . . . 435
16.8 Predefined macro names . . . 435
16.9 Pragma operator . . . 436

17 Library introduction **437**
17.1 General . . . 437
17.2 The C standard library . . . 438
17.3 Definitions . . . 438
17.4 Additional definitions . . . 441
17.5 Method of description (Informative) . . . 441
17.6 Library-wide requirements . . . 446

18 Language support library **467**
18.1 General . . . 467
18.2 Types . . . 467
18.3 Implementation properties . . . 468
18.4 Integer types . . . 477
18.5 Start and termination . . . 478
18.6 Dynamic memory management . . . 480
18.7 Type identification . . . 486
18.8 Exception handling . . . 488
18.9 Initializer lists . . . 492

18.10 Other runtime support . . . 493

19 Diagnostics library **496**
19.1 General . . . 496
19.2 Exception classes . . . 496
19.3 Assertions . . . 500
19.4 Error numbers . . . 500
19.5 System error support . . . 500

20 General utilities library **512**
20.1 General . . . 512
20.2 Utility components . . . 512
20.3 Pairs . . . 517
20.4 Tuples . . . 521
20.5 Compile-time integer sequences . . . 531
20.6 Class template `bitset` . . . 532
20.7 Memory . . . 539
20.8 Smart pointers . . . 554
20.9 Function objects . . . 580
20.10 Metaprogramming and type traits . . . 601
20.11 Compile-time rational arithmetic . . . 620
20.12 Time utilities . . . 623
20.13 Class template `scoped_allocator_adaptor` . . . 638
20.14 Class `type_index` . . . 644

21 Strings library **646**
21.1 General . . . 646
21.2 Character traits . . . 646
21.3 String classes . . . 652
21.4 Class template `basic_string` . . . 656
21.5 Numeric conversions . . . 682
21.6 Hash support . . . 683
21.7 Suffix for `basic_string` literals . . . 684
21.8 Null-terminated sequence utilities . . . 684

22 Localization library **688**
22.1 General . . . 688
22.2 Header `<locale>` synopsis . . . 688
22.3 Locales . . . 689
22.4 Standard `locale` categories . . . 701
22.5 Standard code conversion facets . . . 740
22.6 C library locales . . . 741

23 Containers library **743**
23.1 General . . . 743
23.2 Container requirements . . . 743
23.3 Sequence containers . . . 772
23.4 Associative containers . . . 803
23.5 Unordered associative containers . . . 822
23.6 Container adaptors . . . 841

24 Iterators library **850**
24.1 General . . . 850
24.2 Iterator requirements . . . 850
24.3 Header `<iterator>` synopsis . . . 855
24.4 Iterator primitives . . . 858
24.5 Iterator adaptors . . . 862
24.6 Stream iterators . . . 875
24.7 Range access . . . 882
24.8 Container access . . . 883

25 Algorithms library **884**
25.1 General . . . 884
25.2 Non-modifying sequence operations . . . 895
25.3 Mutating sequence operations . . . 900
25.4 Sorting and related operations . . . 908
25.5 C library algorithms . . . 921

26 Numerics library **922**
26.1 General . . . 922
26.2 Numeric type requirements . . . 922
26.3 The floating-point environment . . . 923
26.4 Complex numbers . . . 924
26.5 Random number generation . . . 934
26.6 Numeric arrays . . . 978
26.7 Generalized numeric operations . . . 999
26.8 C library . . . 1002

27 Input/output library **1007**
27.1 General . . . 1007
27.2 Iostreams requirements . . . 1008
27.3 Forward declarations . . . 1008
27.4 Standard iostream objects . . . 1010
27.5 Iostreams base classes . . . 1012
27.6 Stream buffers . . . 1030
27.7 Formatting and manipulators . . . 1039
27.8 String-based streams . . . 1067
27.9 File-based streams . . . 1077

28 Regular expressions library **1092**
28.1 General . . . 1092
28.2 Definitions . . . 1092
28.3 Requirements . . . 1093
28.4 Header `<regex>` synopsis . . . 1095
28.5 Namespace `std::regex_constants` . . . 1102
28.6 Class `regex_error` . . . 1105
28.7 Class template `regex_traits` . . . 1105
28.8 Class template `basic_regex` . . . 1109
28.9 Class template `sub_match` . . . 1114
28.10 Class template `match_results` . . . 1120
28.11 Regular expression algorithms . . . 1125
28.12 Regular expression iterators . . . 1130

28.13 Modified ECMAScript regular expression grammar . . . 1136

29 Atomic operations library 1139
29.1 General . . . 1139
29.2 Header `<atomic>` synopsis . . . 1139
29.3 Order and consistency . . . 1142
29.4 Lock-free property . . . 1144
29.5 Atomic types . . . 1144
29.6 Operations on atomic types . . . 1148
29.7 Flag type and operations . . . 1153
29.8 Fences . . . 1154

30 Thread support library 1156
30.1 General . . . 1156
30.2 Requirements . . . 1156
30.3 Threads . . . 1159
30.4 Mutual exclusion . . . 1164
30.5 Condition variables . . . 1184
30.6 Futures . . . 1192

A Grammar summary 1208
A.1 Keywords . . . 1208
A.2 Lexical conventions . . . 1208
A.3 Basic concepts . . . 1212
A.4 Expressions . . . 1212
A.5 Statements . . . 1216
A.6 Declarations . . . 1217
A.7 Declarators . . . 1221
A.8 Classes . . . 1223
A.9 Derived classes . . . 1223
A.10 Special member functions . . . 1224
A.11 Overloading . . . 1224
A.12 Templates . . . 1224
A.13 Exception handling . . . 1225
A.14 Preprocessing directives . . . 1226

B Implementation quantities 1228

C Compatibility 1230
C.1 C++ and ISO C . . . 1230
C.2 C++ and ISO C++ 2003 . . . 1239
C.3 C++ and ISO C++ 2011 . . . 1246
C.4 C++ and ISO C++ 2014 . . . 1247
C.5 C standard library . . . 1247

D Compatibility features 1251
D.1 Increment operator with `bool` operand . . . 1251
D.2 `register` keyword . . . 1251
D.3 Implicit declaration of copy functions . . . 1251
D.4 Dynamic exception specifications . . . 1251
D.5 C standard library headers . . . 1251

D.6 Old iostreams members . . . 1252
D.7 `char*` streams . . . 1253
D.8 Violating *exception-specifications* . . . 1262
D.9 `uncaught_exception` . . . 1262

E Universal character names for identifier characters **1263**
E.1 Ranges of characters allowed . . . 1263
E.2 Ranges of characters disallowed initially . . . 1263

F Cross references **1264**

Index **1282**

Index of grammar productions **1311**

Index of library names **1314**

Index of implementation-defined behavior **1351**

List of Tables

1 Alternative tokens . . . 19
2 Identifiers with special meaning . . . 21
3 Keywords . . . 22
4 Alternative representations . . . 22
5 Types of integer literals . . . 24
6 Escape sequences . . . 26
7 String literal concatenations . . . 29

8 Relations on `const` and `volatile` . . . 77

9 *simple-type-specifier*s and the types they specify . . . 159

10 Relationship between operator and function call notation . . . 307
11 Conversions . . . 315

12 Value of folding empty sequences . . . 351

13 Library categories . . . 437
14 C++ library headers . . . 447
15 C++ headers for C library facilities . . . 448
16 C++ headers for freestanding implementations . . . 449
17 `EqualityComparable` requirements . . . 450
18 `LessThanComparable` requirements . . . 450
19 `DefaultConstructible` requirements . . . 450
20 `MoveConstructible` requirements . . . 450
21 `CopyConstructible` requirements (in addition to `MoveConstructible`) . . . 450
22 `MoveAssignable` requirements . . . 451
23 `CopyAssignable` requirements (in addition to `MoveAssignable`) . . . 451
24 `Destructible` requirements . . . 451
25 `NullablePointer` requirements . . . 453
26 `Hash` requirements . . . 453
27 Descriptive variable definitions . . . 454
28 Allocator requirements . . . 454

29 Language support library summary . . . 467
30 Header `<cstddef>` synopsis . . . 467
31 Header `<climits>` synopsis . . . 477
32 Header `<cfloat>` synopsis . . . 477
33 Header `<cstdlib>` synopsis . . . 478
34 Header `<csetjmp>` synopsis . . . 494
35 Header `<csignal>` synopsis . . . 494
36 Header `<cstdalign>` synopsis . . . 495
37 Header `<cstdarg>` synopsis . . . 495
38 Header `<cstdbool>` synopsis . . . 495
39 Header `<cstdlib>` synopsis . . . 495
40 Header `<ctime>` synopsis . . . 495

41 Diagnostics library summary . . . 496
42 Header `<cassert>` synopsis . . . 500
43 Header `<cerrno>` synopsis . . . 501

44 General utilities library summary . . . 512
45 Header `<cstdlib>` synopsis . . . 553
46 Header `<cstring>` synopsis . . . 553
47 Primary type category predicates . . . 605
48 Composite type category predicates . . . 606
49 Type property predicates . . . 607
50 Type property queries . . . 613
51 Type relationship predicates . . . 614
52 Const-volatile modifications . . . 615
53 Reference modifications . . . 616
54 Sign modifications . . . 616
55 Array modifications . . . 617
56 Pointer modifications . . . 617
57 Other transformations . . . 618
58 Expressions used to perform ratio arithmetic . . . 622
59 Clock requirements . . . 626
60 Header `<ctime>` synopsis . . . 638

61 Strings library summary . . . 646
62 Character traits requirements . . . 647
63 `basic_string(const Allocator&)` effects . . . 661
64 `basic_string(const basic_string&)` effects . . . 661
65 `basic_string(const basic_string&, size_type, size_type, const Allocator&)` effects . . . 661
66 `basic_string(const charT*, size_type, const Allocator&)` effects . . . 662
67 `basic_string(const charT*, const Allocator&)` effects . . . 662
68 `basic_string(size_t, charT, const Allocator&)` effects . . . 662
69 `basic_string(const basic_string&, const Allocator&)` and `basic_string(basic_string&&, const Allocator&)` effects . . . 663
70 `operator=(const basic_string&)` effects . . . 663
71 `compare()` results . . . 676
72 Potential `mbstate_t` data races . . . 685
73 Header `<cctype>` synopsis . . . 685
74 Header `<cwctype>` synopsis . . . 686
75 Header `<cstring>` synopsis . . . 686
76 Header `<cwchar>` synopsis . . . 686
77 Header `<cstdlib>` synopsis . . . 686
78 Header `<cuchar>` synopsis . . . 687

79 Localization library summary . . . 688
80 Locale category facets . . . 692
81 Required specializations . . . 693
82 `do_in/do_out` result values . . . 710
83 `do_unshift` result values . . . 711
84 Integer conversions . . . 714
85 Length modifier . . . 715
86 Integer conversions . . . 719
87 Floating-point conversions . . . 719

88 Length modifier . . . 719
89 Numeric conversions . . . 719
90 Fill padding . . . 720
91 `do_get_date` effects . . . 727
92 Header `<clocale>` synopsis . . . 742
93 Potential `setlocale` data races . . . 742

94 Containers library summary . . . 743
95 Container requirements . . . 744
96 Reversible container requirements . . . 746
97 Optional container operations . . . 748
98 Allocator-aware container requirements . . . 749
99 Sequence container requirements (in addition to container) . . . 751
100 Optional sequence container operations . . . 753
101 Associative container requirements (in addition to container) . . . 756
102 Unordered associative container requirements (in addition to container) . . . 764

103 Iterators library summary . . . 850
104 Relations among iterator categories . . . 850
105 Iterator requirements . . . 852
106 Input iterator requirements (in addition to Iterator) . . . 852
107 Output iterator requirements (in addition to Iterator) . . . 853
108 Forward iterator requirements (in addition to input iterator) . . . 854
109 Bidirectional iterator requirements (in addition to forward iterator) . . . 854
110 Random access iterator requirements (in addition to bidirectional iterator) . . . 855

111 Algorithms library summary . . . 884
112 Header `<cstdlib>` synopsis . . . 921

113 Numerics library summary . . . 922
114 Seed sequence requirements . . . 936
115 Uniform random number generator requirements . . . 937
116 Random number engine requirements . . . 938
117 Random number distribution requirements . . . 941
118 Header `<cmath>` synopsis . . . 1002
119 Header `<cstdlib>` synopsis . . . 1003

120 Input/output library summary . . . 1007
121 `fmtflags` effects . . . 1017
122 `fmtflags` constants . . . 1017
123 `iostate` effects . . . 1017
124 `openmode` effects . . . 1017
125 `seekdir` effects . . . 1018
126 Position type requirements . . . 1022
127 `basic_ios::init()` effects . . . 1024
128 `basic_ios::copyfmt()` effects . . . 1026
129 `seekoff` positioning . . . 1071
130 `newoff` values . . . 1071
131 File open modes . . . 1081
132 `seekoff` effects . . . 1083
133 Header `<cstdio>` synopsis . . . 1091

134 Header `<cinttypes>` synopsis . . . 1091

135 Regular expressions library summary . . . 1092
136 Regular expression traits class requirements . . . 1093
137 `syntax_option_type` effects . . . 1103
138 `regex_constants::match_flag_type` effects when obtaining a match against a character container sequence `[first,last)`. . . . 1103
139 `error_type` values in the C locale . . . 1104
140 Character class names and corresponding `ctype` masks . . . 1108
141 `match_results` assignment operator effects . . . 1122
142 Effects of `regex_match` algorithm . . . 1125
143 Effects of `regex_search` algorithm . . . 1127

144 Atomics library summary . . . 1139
145 `atomic` integral typedefs . . . 1148
146 `atomic <inttypes.h>` typedefs . . . 1149
147 Atomic arithmetic computations . . . 1152

148 Thread support library summary . . . 1156

149 Standard macros . . . 1247
150 Standard values . . . 1248
151 Standard types . . . 1248
152 Standard structs . . . 1248
153 Standard functions . . . 1249

154 C headers . . . 1251
155 `strstreambuf(streamsize)` effects . . . 1255
156 `strstreambuf(void* (*)(size_t), void (*)(void*))` effects . . . 1255
157 `strstreambuf(charT*, streamsize, charT*)` effects . . . 1255
158 `seekoff` positioning . . . 1258
159 `newoff` values . . . 1258

List of Figures

1 Expression category taxonomy 77

2 Directed acyclic graph 243
3 Non-virtual base 244
4 Virtual base 245
5 Virtual and non-virtual base 245
6 Name lookup 247

7 Stream position, offset, and size types [non-normative] 1007

1 General [intro]

1.1 Scope [intro.scope]

1 This International Standard specifies requirements for implementations of the C++ programming language. The first such requirement is that they implement the language, and so this International Standard also defines C++. Other requirements and relaxations of the first requirement appear at various places within this International Standard.

2 C++ is a general purpose programming language based on the C programming language as described in ISO/IEC 9899:1999 *Programming languages — C* (hereinafter referred to as the *C standard*). In addition to the facilities provided by C, C++ provides additional data types, classes, templates, exceptions, namespaces, operator overloading, function name overloading, references, free store management operators, and additional library facilities.

1.2 Normative references [intro.refs]

1 The following referenced documents are indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

(1.1) — Ecma International, *ECMAScript Language Specification*, Standard Ecma-262, third edition, 1999.

(1.2) — ISO/IEC 2382 (all parts), *Information technology — Vocabulary*

(1.3) — ISO/IEC 9899:1999, *Programming languages — C*

(1.4) — ISO/IEC 9899:1999/Cor.1:2001(E), *Programming languages — C, Technical Corrigendum 1*

(1.5) — ISO/IEC 9899:1999/Cor.2:2004(E), *Programming languages — C, Technical Corrigendum 2*

(1.6) — ISO/IEC 9899:1999/Cor.3:2007(E), *Programming languages — C, Technical Corrigendum 3*

(1.7) — ISO/IEC 9945:2003, *Information Technology — Portable Operating System Interface (POSIX)*

(1.8) — ISO/IEC 10646-1:1993, *Information technology — Universal Multiple-Octet Coded Character Set (UCS) — Part 1: Architecture and Basic Multilingual Plane*

(1.9) — ISO/IEC TR 19769:2004, *Information technology — Programming languages, their environments and system software interfaces — Extensions for the programming language C to support new character data types*

2 The library described in Clause 7 of ISO/IEC 9899:1999 and Clause 7 of ISO/IEC 9899:1999/Cor.1:2001 and Clause 7 of ISO/IEC 9899:1999/Cor.2:2003 is hereinafter called the *C standard library*.[1]

3 The library described in ISO/IEC TR 19769:2004 is hereinafter called the *C Unicode TR*.

4 The operating system interface described in ISO/IEC 9945:2003 is hereinafter called *POSIX*.

5 The ECMAScript Language Specification described in Standard Ecma-262 is hereinafter called *ECMA-262*.

1) With the qualifications noted in Clauses 18 through 30 and in C.5, the C standard library is a subset of the C++ standard library.

1.3 Terms and definitions [intro.defs]

1 For the purposes of this document, the following definitions apply.

2 17.3 defines additional terms that are used only in Clauses 17 through 30 and Annex D.

3 Terms that are used only in a small portion of this International Standard are defined where they are used and italicized where they are defined.

1.3.1 [defns.access]

access
<execution-time action> to read or modify the value of an object

1.3.2 [defns.argument]

argument
<function call expression> expression in the comma-separated list bounded by the parentheses (5.2.2)

1.3.3 [defns.argument.macro]

argument
<function-like macro> sequence of preprocessing tokens in the comma-separated list bounded by the parentheses (16.3)

1.3.4 [defns.argument.throw]

argument
<throw expression> the operand of `throw` (5.17)

1.3.5 [defns.argument.templ]

argument
<template instantiation> *constant-expression*, *type-id*, or *id-expression* in the comma-separated list bounded by the angle brackets (14.3)

1.3.6 [defns.cond.supp]

conditionally-supported
program construct that an implementation is not required to support
[*Note:* Each implementation documents all conditionally-supported constructs that it does not support. — *end note*]

1.3.7 [defns.diagnostic]

diagnostic message
message belonging to an implementation-defined subset of the implementation's output messages

1.3.8 [defns.dynamic.type]

dynamic type
<glvalue> type of the most derived object (1.8) to which the glvalue refers
[*Example:* if a pointer (8.3.1) `p` whose static type is "pointer to class `B`" is pointing to an object of class `D`, derived from `B` (Clause 10), the dynamic type of the expression `*p` is "`D`". References (8.3.2) are treated similarly. — *end example*]

1.3.9 [defns.dynamic.type.prvalue]

dynamic type
<prvalue> static type of the prvalue expression

1.3.10 [defns.ill.formed]

ill-formed program
program that is not well-formed (1.3.27)

1.3.11 [defns.impl.defined]

implementation-defined behavior

behavior, for a well-formed program construct and correct data, that depends on the implementation and that each implementation documents

1.3.12 [defns.impl.limits]

implementation limits

restrictions imposed upon programs by the implementation

1.3.13 [defns.locale.specific]

locale-specific behavior

behavior that depends on local conventions of nationality, culture, and language that each implementation documents

1.3.14 [defns.multibyte]

multibyte character

sequence of one or more bytes representing a member of the extended character set of either the source or the execution environment

[*Note:* The extended character set is a superset of the basic character set (2.3). — *end note*]

1.3.15 [defns.parameter]

parameter

<function or catch clause> object or reference declared as part of a function declaration or definition or in the catch clause of an exception handler that acquires a value on entry to the function or handler

1.3.16 [defns.parameter.macro]

parameter

<function-like macro> identifier from the comma-separated list bounded by the parentheses immediately following the macro name

1.3.17 [defns.parameter.templ]

parameter

<template> *template-parameter*

1.3.18 [defns.signature]

signature

<function> name, parameter type list (8.3.5), and enclosing namespace (if any)

[*Note:* Signatures are used as a basis for name mangling and linking. — *end note*]

1.3.19 [defns.signature.templ]

signature

<function template> name, parameter type list (8.3.5), enclosing namespace (if any), return type, and template parameter list

1.3.20 [defns.signature.spec]

signature

<function template specialization> signature of the template of which it is a specialization and its template arguments (whether explicitly specified or deduced)

1.3.21 [defns.signature.member]

signature

<class member function> name, parameter type list (8.3.5), class of which the function is a member, *cv*-qualifiers (if any), and *ref-qualifier* (if any)

1.3.22 [defns.signature.member.templ]

signature

<class member function template> name, parameter type list (8.3.5), class of which the function is a member, *cv*-qualifiers (if any), *ref-qualifier* (if any), return type, and template parameter list

1.3.23 [defns.signature.member.spec]

signature

<class member function template specialization> signature of the member function template of which it is a specialization and its template arguments (whether explicitly specified or deduced)

1.3.24 [defns.static.type]

static type

type of an expression (3.9) resulting from analysis of the program without considering execution semantics [*Note:* The static type of an expression depends only on the form of the program in which the expression appears, and does not change while the program is executing. — *end note*]

1.3.25 [defns.undefined]

undefined behavior

behavior for which this International Standard imposes no requirements

[*Note:* Undefined behavior may be expected when this International Standard omits any explicit definition of behavior or when a program uses an erroneous construct or erroneous data. Permissible undefined behavior ranges from ignoring the situation completely with unpredictable results, to behaving during translation or program execution in a documented manner characteristic of the environment (with or without the issuance of a diagnostic message), to terminating a translation or execution (with the issuance of a diagnostic message). Many erroneous program constructs do not engender undefined behavior; they are required to be diagnosed. — *end note*]

1.3.26 [defns.unspecified]

unspecified behavior

behavior, for a well-formed program construct and correct data, that depends on the implementation

[*Note:* The implementation is not required to document which behavior occurs. The range of possible behaviors is usually delineated by this International Standard. — *end note*]

1.3.27 [defns.well.formed]

well-formed program

C++ program constructed according to the syntax rules, diagnosable semantic rules, and the one-definition rule (3.2).

1.4 Implementation compliance [intro.compliance]

1 The set of *diagnosable rules* consists of all syntactic and semantic rules in this International Standard except for those rules containing an explicit notation that "no diagnostic is required" or which are described as resulting in "undefined behavior."

2 Although this International Standard states only requirements on C++ implementations, those requirements are often easier to understand if they are phrased as requirements on programs, parts of programs, or execution of programs. Such requirements have the following meaning:

(2.1) — If a program contains no violations of the rules in this International Standard, a conforming implementation shall, within its resource limits, accept and correctly execute[2] that program.

2) "Correct execution" can include undefined behavior, depending on the data being processed; see 1.3 and 1.9.

(2.2) — If a program contains a violation of any diagnosable rule or an occurrence of a construct described in this Standard as "conditionally-supported" when the implementation does not support that construct, a conforming implementation shall issue at least one diagnostic message.

(2.3) — If a program contains a violation of a rule for which no diagnostic is required, this International Standard places no requirement on implementations with respect to that program.

3 For classes and class templates, the library Clauses specify partial definitions. Private members (Clause 11) are not specified, but each implementation shall supply them to complete the definitions according to the description in the library Clauses.

4 For functions, function templates, objects, and values, the library Clauses specify declarations. Implementations shall supply definitions consistent with the descriptions in the library Clauses.

5 The names defined in the library have namespace scope (7.3). A C++ translation unit (2.2) obtains access to these names by including the appropriate standard library header (16.2).

6 The templates, classes, functions, and objects in the library have external linkage (3.5). The implementation provides definitions for standard library entities, as necessary, while combining translation units to form a complete C++ program (2.2).

7 Two kinds of implementations are defined: a *hosted implementation* and a *freestanding implementation*. For a hosted implementation, this International Standard defines the set of available libraries. A freestanding implementation is one in which execution may take place without the benefit of an operating system, and has an implementation-defined set of libraries that includes certain language-support libraries (17.6.1.3).

8 A conforming implementation may have extensions (including additional library functions), provided they do not alter the behavior of any well-formed program. Implementations are required to diagnose programs that use such extensions that are ill-formed according to this International Standard. Having done so, however, they can compile and execute such programs.

9 Each implementation shall include documentation that identifies all conditionally-supported constructs that it does not support and defines all locale-specific characteristics.[3]

1.5 Structure of this International Standard [intro.structure]

1 Clauses 2 through 16 describe the C++ programming language. That description includes detailed syntactic specifications in a form described in 1.6. For convenience, Annex A repeats all such syntactic specifications.

2 Clauses 18 through 30 and Annex D (the *library clauses*) describe the Standard C++ library. That description includes detailed descriptions of the templates, classes, functions, constants, and macros that constitute the library, in a form described in Clause 17.

3 Annex B recommends lower bounds on the capacity of conforming implementations.

4 Annex C summarizes the evolution of C++ since its first published description, and explains in detail the differences between C++ and C. Certain features of C++ exist solely for compatibility purposes; Annex D describes those features.

5 Throughout this International Standard, each example is introduced by "[*Example:*" and terminated by " — *end example*]". Each note is introduced by "[*Note:*" and terminated by " — *end note*]". Examples and notes may be nested.

1.6 Syntax notation [syntax]

1 In the syntax notation used in this International Standard, syntactic categories are indicated by *italic* type, and literal words and characters in `constant width` type. Alternatives are listed on separate lines except in a few cases where a long set of alternatives is marked by the phrase "one of." If the text of an alternative is

3) This documentation also defines implementation-defined behavior; see 1.9.

too long to fit on a line, the text is continued on subsequent lines indented from the first one. An optional terminal or non-terminal symbol is indicated by the subscript "$_{opt}$", so

> `{` *expression*$_{opt}$`}`

indicates an optional expression enclosed in braces.

2 Names for syntactic categories have generally been chosen according to the following rules:

(2.1) — *X-name* is a use of an identifier in a context that determines its meaning (e.g., *class-name*, *typedef-name*).

(2.2) — *X-id* is an identifier with no context-dependent meaning (e.g., *qualified-id*).

(2.3) — *X-seq* is one or more *X*'s without intervening delimiters (e.g., *declaration-seq* is a sequence of declarations).

(2.4) — *X-list* is one or more *X*'s separated by intervening commas (e.g., *expression-list* is a sequence of expressions separated by commas).

1.7 The C++ memory model [intro.memory]

1 The fundamental storage unit in the C++ memory model is the *byte*. A byte is at least large enough to contain any member of the basic execution character set (2.3) and the eight-bit code units of the Unicode UTF-8 encoding form and is composed of a contiguous sequence of bits, the number of which is implementation-defined. The least significant bit is called the *low-order bit*; the most significant bit is called the *high-order bit*. The memory available to a C++ program consists of one or more sequences of contiguous bytes. Every byte has a unique address.

2 [*Note:* The representation of types is described in 3.9. — *end note*]

3 A *memory location* is either an object of scalar type or a maximal sequence of adjacent bit-fields all having non-zero width. [*Note:* Various features of the language, such as references and virtual functions, might involve additional memory locations that are not accessible to programs but are managed by the implementation. — *end note*] Two or more threads of execution (1.10) can update and access separate memory locations without interfering with each other.

4 [*Note:* Thus a bit-field and an adjacent non-bit-field are in separate memory locations, and therefore can be concurrently updated by two threads of execution without interference. The same applies to two bit-fields, if one is declared inside a nested struct declaration and the other is not, or if the two are separated by a zero-length bit-field declaration, or if they are separated by a non-bit-field declaration. It is not safe to concurrently update two bit-fields in the same struct if all fields between them are also bit-fields of non-zero width. — *end note*]

5 [*Example:* A structure declared as

```
struct {
  char a;
  int b:5,
  c:11,
  :0,
  d:8;
  struct {int ee:8;} e;
}
```

contains four separate memory locations: The field `a` and bit-fields `d` and `e.ee` are each separate memory locations, and can be modified concurrently without interfering with each other. The bit-fields `b` and `c` together constitute the fourth memory location. The bit-fields `b` and `c` cannot be concurrently modified, but `b` and `a`, for example, can be. — *end example*]

1.8 The C++ object model [intro.object]

1 The constructs in a C++ program create, destroy, refer to, access, and manipulate objects. An *object* is a region of storage. [*Note:* A function is not an object, regardless of whether or not it occupies storage in the way that objects do. — *end note*] An object is created by a *definition* (3.1), by a *new-expression* (5.3.4) or by the implementation (12.2) when needed. The properties of an object are determined when the object is created. An object can have a *name* (Clause 3). An object has a *storage duration* (3.7) which influences its *lifetime* (3.8). An object has a *type* (3.9). The term *object type* refers to the type with which the object is created. Some objects are *polymorphic* (10.3); the implementation generates information associated with each such object that makes it possible to determine that object's type during program execution. For other objects, the interpretation of the values found therein is determined by the type of the *expression*s (Clause 5) used to access them.

2 Objects can contain other objects, called *subobjects*. A subobject can be a *member subobject* (9.2), a *base class subobject* (Clause 10), or an array element. An object that is not a subobject of any other object is called a *complete object*.

3 For every object `x`, there is some object called the *complete object of* `x`, determined as follows:

(3.1) — If `x` is a complete object, then `x` is the complete object of `x`.

(3.2) — Otherwise, the complete object of `x` is the complete object of the (unique) object that contains `x`.

4 If a complete object, a data member (9.2), or an array element is of class type, its type is considered the *most derived class*, to distinguish it from the class type of any base class subobject; an object of a most derived class type or of a non-class type is called a *most derived object*.

5 Unless it is a bit-field (9.6), a most derived object shall have a non-zero size and shall occupy one or more bytes of storage. Base class subobjects may have zero size. An object of trivially copyable or standard-layout type (3.9) shall occupy contiguous bytes of storage.

6 Unless an object is a bit-field or a base class subobject of zero size, the address of that object is the address of the first byte it occupies. Two objects that are not bit-fields may have the same address if one is a subobject of the other, or if at least one is a base class subobject of zero size and they are of different types; otherwise, they shall have distinct addresses.[4]

[*Example:*

```
static const char test1 = 'x';
static const char test2 = 'x';
const bool b = &test1 != &test2;      // always true
```

— *end example*]

7 [*Note:* C++ provides a variety of fundamental types and several ways of composing new types from existing types (3.9). — *end note*]

1.9 Program execution [intro.execution]

1 The semantic descriptions in this International Standard define a parameterized nondeterministic abstract machine. This International Standard places no requirement on the structure of conforming implementations. In particular, they need not copy or emulate the structure of the abstract machine. Rather, conforming implementations are required to emulate (only) the observable behavior of the abstract machine as explained below.[5]

4) Under the "as-if" rule an implementation is allowed to store two objects at the same machine address or not store an object at all if the program cannot observe the difference (1.9).

5) This provision is sometimes called the "as-if" rule, because an implementation is free to disregard any requirement of this International Standard as long as the result is *as if* the requirement had been obeyed, as far as can be determined from the observable behavior of the program. For instance, an actual implementation need not evaluate part of an expression if it can deduce that its value is not used and that no side effects affecting the observable behavior of the program are produced.

2 Certain aspects and operations of the abstract machine are described in this International Standard as implementation-defined (for example, `sizeof(int)`). These constitute the parameters of the abstract machine. Each implementation shall include documentation describing its characteristics and behavior in these respects.[6] Such documentation shall define the instance of the abstract machine that corresponds to that implementation (referred to as the "corresponding instance" below).

3 Certain other aspects and operations of the abstract machine are described in this International Standard as unspecified (for example, evaluation of expressions in a *new-initializer* if the allocation function fails to allocate memory (5.3.4)). Where possible, this International Standard defines a set of allowable behaviors. These define the nondeterministic aspects of the abstract machine. An instance of the abstract machine can thus have more than one possible execution for a given program and a given input.

4 Certain other operations are described in this International Standard as undefined (for example, the effect of attempting to modify a `const` object). [*Note:* This International Standard imposes no requirements on the behavior of programs that contain undefined behavior. — *end note*]

5 A conforming implementation executing a well-formed program shall produce the same observable behavior as one of the possible executions of the corresponding instance of the abstract machine with the same program and the same input. However, if any such execution contains an undefined operation, this International Standard places no requirement on the implementation executing that program with that input (not even with regard to operations preceding the first undefined operation).

6 If a signal handler is executed as a result of a call to the `raise` function, then the execution of the handler is sequenced after the invocation of the `raise` function and before its return. [*Note:* When a signal is received for another reason, the execution of the signal handler is usually unsequenced with respect to the rest of the program. — *end note*]

7 An instance of each object with automatic storage duration (3.7.3) is associated with each entry into its block. Such an object exists and retains its last-stored value during the execution of the block and while the block is suspended (by a call of a function or receipt of a signal).

8 The least requirements on a conforming implementation are:

(8.1) — Access to volatile objects are evaluated strictly according to the rules of the abstract machine.

(8.2) — At program termination, all data written into files shall be identical to one of the possible results that execution of the program according to the abstract semantics would have produced.

(8.3) — The input and output dynamics of interactive devices shall take place in such a fashion that prompting output is actually delivered before a program waits for input. What constitutes an interactive device is implementation-defined.

These collectively are referred to as the *observable behavior* of the program. [*Note:* More stringent correspondences between abstract and actual semantics may be defined by each implementation. — *end note*]

9 [*Note:* Operators can be regrouped according to the usual mathematical rules only where the operators really are associative or commutative.[7] For example, in the following fragment

```
int a, b;
/* ... */
a = a + 32760 + b + 5;
```

the expression statement behaves exactly the same as

```
a = (((a + 32760) + b) + 5);
```

6) This documentation also includes conditionally-supported constructs and locale-specific behavior. See 1.4.
7) Overloaded operators are never assumed to be associative or commutative.

due to the associativity and precedence of these operators. Thus, the result of the sum `(a + 32760)` is next added to `b`, and that result is then added to 5 which results in the value assigned to `a`. On a machine in which overflows produce an exception and in which the range of values representable by an `int` is `[-32768,+32767]`, the implementation cannot rewrite this expression as

```
a = ((a + b) + 32765);
```

since if the values for `a` and `b` were, respectively, -32754 and -15, the sum `a + b` would produce an exception while the original expression would not; nor can the expression be rewritten either as

```
a = ((a + 32765) + b);
```

or

```
a = (a + (b + 32765));
```

since the values for `a` and `b` might have been, respectively, 4 and -8 or -17 and 12. However on a machine in which overflows do not produce an exception and in which the results of overflows are reversible, the above expression statement can be rewritten by the implementation in any of the above ways because the same result will occur. — *end note*]

10 A *full-expression* is an expression that is not a subexpression of another expression. [*Note:* in some contexts, such as unevaluated operands, a syntactic subexpression is considered a full-expression (Clause 5). — *end note*] If a language construct is defined to produce an implicit call of a function, a use of the language construct is considered to be an expression for the purposes of this definition. A call to a destructor generated at the end of the lifetime of an object other than a temporary object is an implicit full-expression. Conversions applied to the result of an expression in order to satisfy the requirements of the language construct in which the expression appears are also considered to be part of the full-expression.

[*Example:*

```
struct S {
  S(int i): I(i) { }
  int& v() { return I; }
private:
  int I;
};

 S s1(1);            // full-expression is call of S::S(int)
 S s2 = 2;           // full-expression is call of S::S(int)

void f() {
  if (S(3).v())      // full-expression includes lvalue-to-rvalue and
                     // int to bool conversions, performed before
                     // temporary is deleted at end of full-expression
  { }
}
```

— *end example*]

11 [*Note:* The evaluation of a full-expression can include the evaluation of subexpressions that are not lexically part of the full-expression. For example, subexpressions involved in evaluating default arguments (8.3.6) are considered to be created in the expression that calls the function, not the expression that defines the default argument. — *end note*]

12 Reading an object designated by a `volatile` glvalue (3.10), modifying an object, calling a library I/O function, or calling a function that does any of those operations are all *side effects*, which are changes in the state of the execution environment. *Evaluation* of an expression (or a sub-expression) in general includes

both value computations (including determining the identity of an object for glvalue evaluation and fetching a value previously assigned to an object for prvalue evaluation) and initiation of side effects. When a call to a library I/O function returns or an access to a `volatile` object is evaluated the side effect is considered complete, even though some external actions implied by the call (such as the I/O itself) or by the `volatile` access may not have completed yet.

13 *Sequenced before* is an asymmetric, transitive, pair-wise relation between evaluations executed by a single thread (1.10), which induces a partial order among those evaluations. Given any two evaluations A and B, if A is sequenced before B, then the execution of A shall precede the execution of B. If A is not sequenced before B and B is not sequenced before A, then A and B are *unsequenced*. [*Note:* The execution of unsequenced evaluations can overlap. — *end note*] Evaluations A and B are *indeterminately sequenced* when either A is sequenced before B or B is sequenced before A, but it is unspecified which. [*Note:* Indeterminately sequenced evaluations cannot overlap, but either could be executed first. — *end note*]

14 Every value computation and side effect associated with a full-expression is sequenced before every value computation and side effect associated with the next full-expression to be evaluated.[8]

15 Except where noted, evaluations of operands of individual operators and of subexpressions of individual expressions are unsequenced. [*Note:* In an expression that is evaluated more than once during the execution of a program, unsequenced and indeterminately sequenced evaluations of its subexpressions need not be performed consistently in different evaluations. — *end note*] The value computations of the operands of an operator are sequenced before the value computation of the result of the operator. If a side effect on a scalar object is unsequenced relative to either another side effect on the same scalar object or a value computation using the value of the same scalar object, and they are not potentially concurrent (1.10), the behavior is undefined. [*Note:* The next section imposes similar, but more complex restrictions on potentially concurrent computations. — *end note*]

[*Example:*

```
void f(int, int);
void g(int i, int* v) {
  i = v[i++];          // the behavior is undefined
  i = 7, i++, i++;     // i becomes 9

  i = i++ + 1;         // the behavior is undefined
  i = i + 1;           // the value of i is incremented

  f(i = -1, i = -1);   // the behavior is undefined
}
```

— *end example*]

When calling a function (whether or not the function is inline), every value computation and side effect associated with any argument expression, or with the postfix expression designating the called function, is sequenced before execution of every expression or statement in the body of the called function. [*Note:* Value computations and side effects associated with different argument expressions are unsequenced. — *end note*] Every evaluation in the calling function (including other function calls) that is not otherwise specifically sequenced before or after the execution of the body of the called function is indeterminately sequenced with respect to the execution of the called function.[9] Several contexts in C++ cause evaluation of a function call, even though no corresponding function call syntax appears in the translation unit. [*Example:* Evaluation of a *new-expression* invokes one or more allocation and constructor functions; see 5.3.4. For another example, invocation of a conversion function (12.3.2) can arise in contexts in which no function call syntax appears.

8) As specified in 12.2, after a full-expression is evaluated, a sequence of zero or more invocations of destructor functions for temporary objects takes place, usually in reverse order of the construction of each temporary object.

9) In other words, function executions do not interleave with each other.

— *end example*] The sequencing constraints on the execution of the called function (as described above) are features of the function calls as evaluated, whatever the syntax of the expression that calls the function might be.

1.10 Multi-threaded executions and data races [intro.multithread]

1 A *thread of execution* (also known as a *thread*) is a single flow of control within a program, including the initial invocation of a specific top-level function, and recursively including every function invocation subsequently executed by the thread. [*Note:* When one thread creates another, the initial call to the top-level function of the new thread is executed by the new thread, not by the creating thread. — *end note*] Every thread in a program can potentially access every object and function in a program.[10] Under a hosted implementation, a C++ program can have more than one thread running concurrently. The execution of each thread proceeds as defined by the remainder of this standard. The execution of the entire program consists of an execution of all of its threads. [*Note:* Usually the execution can be viewed as an interleaving of all its threads. However, some kinds of atomic operations, for example, allow executions inconsistent with a simple interleaving, as described below. — *end note*] Under a freestanding implementation, it is implementation-defined whether a program can have more than one thread of execution.

2 A signal handler that is executed as a result of a call to the `raise` function belongs to the same thread of execution as the call to the `raise` function. Otherwise it is unspecified which thread of execution contains a signal handler invocation.

3 Implementations should ensure that all unblocked threads eventually make progress. [*Note:* Standard library functions may silently block on I/O or locks. Factors in the execution environment, including externally-imposed thread priorities, may prevent an implementation from making certain guarantees of forward progress. — *end note*]

4 Executions of atomic functions that are either defined to be lock-free (29.7) or indicated as lock-free (29.4) are *lock-free executions*.

(4.1) — If there is only one unblocked thread, a lock-free execution in that thread shall complete. [*Note:* Concurrently executing threads may prevent progress of a lock-free execution. For example, this situation can occur with load-locked store-conditional implementations. This property is sometimes termed obstruction-free. — *end note*]

(4.2) — When one or more lock-free executions run concurrently, at least one should complete. [*Note:* It is difficult for some implementations to provide absolute guarantees to this effect, since repeated and particularly inopportune interference from other threads may prevent forward progress, e.g., by repeatedly stealing a cache line for unrelated purposes between load-locked and store-conditional instructions. Implementations should ensure that such effects cannot indefinitely delay progress under expected operating conditions, and that such anomalies can therefore safely be ignored by programmers. Outside this International Standard, this property is sometimes termed lock-free. — *end note*]

5 The value of an object visible to a thread T at a particular point is the initial value of the object, a value assigned to the object by T, or a value assigned to the object by another thread, according to the rules below. [*Note:* In some cases, there may instead be undefined behavior. Much of this section is motivated by the desire to support atomic operations with explicit and detailed visibility constraints. However, it also implicitly supports a simpler view for more restricted programs. — *end note*]

6 Two expression evaluations *conflict* if one of them modifies a memory location (1.7) and the other one reads or modifies the same memory location.

10) An object with automatic or thread storage duration (3.7) is associated with one specific thread, and can be accessed by a different thread only indirectly through a pointer or reference (3.9.2).

7 The library defines a number of atomic operations (Clause 29) and operations on mutexes (Clause 30) that are specially identified as synchronization operations. These operations play a special role in making assignments in one thread visible to another. A synchronization operation on one or more memory locations is either a consume operation, an acquire operation, a release operation, or both an acquire and release operation. A synchronization operation without an associated memory location is a fence and can be either an acquire fence, a release fence, or both an acquire and release fence. In addition, there are relaxed atomic operations, which are not synchronization operations, and atomic read-modify-write operations, which have special characteristics. [*Note:* For example, a call that acquires a mutex will perform an acquire operation on the locations comprising the mutex. Correspondingly, a call that releases the same mutex will perform a release operation on those same locations. Informally, performing a release operation on A forces prior side effects on other memory locations to become visible to other threads that later perform a consume or an acquire operation on A. "Relaxed" atomic operations are not synchronization operations even though, like synchronization operations, they cannot contribute to data races. — *end note*]

8 All modifications to a particular atomic object M occur in some particular total order, called the *modification order* of M. If A and B are modifications of an atomic object M and A happens before (as defined below) B, then A shall precede B in the modification order of M, which is defined below. [*Note:* This states that the modification orders must respect the "happens before" relationship. — *end note*] [*Note:* There is a separate order for each atomic object. There is no requirement that these can be combined into a single total order for all objects. In general this will be impossible since different threads may observe modifications to different objects in inconsistent orders. — *end note*]

9 A *release sequence* headed by a release operation A on an atomic object M is a maximal contiguous subsequence of side effects in the modification order of M, where the first operation is `A`, and every subsequent operation

(9.1) — is performed by the same thread that performed `A`, or

(9.2) — is an atomic read-modify-write operation.

10 Certain library calls *synchronize with* other library calls performed by another thread. For example, an atomic store-release synchronizes with a load-acquire that takes its value from the store (29.3). [*Note:* Except in the specified cases, reading a later value does not necessarily ensure visibility as described below. Such a requirement would sometimes interfere with efficient implementation. — *end note*] [*Note:* The specifications of the synchronization operations define when one reads the value written by another. For atomic objects, the definition is clear. All operations on a given mutex occur in a single total order. Each mutex acquisition "reads the value written" by the last mutex release. — *end note*]

11 An evaluation A *carries a dependency* to an evaluation B if

(11.1) — the value of A is used as an operand of B, unless:

(11.1.1) — B is an invocation of any specialization of `std::kill_dependency` (29.3), or

(11.1.2) — A is the left operand of a built-in logical AND (`&&`, see 5.14) or logical OR (`||`, see 5.15) operator, or

(11.1.3) — A is the left operand of a conditional (`?:`, see 5.16) operator, or

(11.1.4) — A is the left operand of the built-in comma (`,`) operator (5.19);

or

(11.2) — A writes a scalar object or bit-field M, B reads the value written by A from M, and A is sequenced before B, or

(11.3) — for some evaluation X, A carries a dependency to X, and X carries a dependency to B.

[*Note:* "Carries a dependency to" is a subset of "is sequenced before", and is similarly strictly intra-thread. — *end note*]

12 An evaluation A is *dependency-ordered before* an evaluation B if

(12.1) — A performs a release operation on an atomic object M, and, in another thread, B performs a consume operation on M and reads a value written by any side effect in the release sequence headed by A, or

(12.2) — for some evaluation X, A is dependency-ordered before X and X carries a dependency to B.

[*Note:* The relation "is dependency-ordered before" is analogous to "synchronizes with", but uses release/-consume in place of release/acquire. — *end note*]

13 An evaluation A *inter-thread happens before* an evaluation B if

(13.1) — A synchronizes with B, or

(13.2) — A is dependency-ordered before B, or

(13.3) — for some evaluation X

(13.3.1) — A synchronizes with X and X is sequenced before B, or

(13.3.2) — A is sequenced before X and X inter-thread happens before B, or

(13.3.3) — A inter-thread happens before X and X inter-thread happens before B.

[*Note:* The "inter-thread happens before" relation describes arbitrary concatenations of "sequenced before", "synchronizes with" and "dependency-ordered before" relationships, with two exceptions. The first exception is that a concatenation is not permitted to end with "dependency-ordered before" followed by "sequenced before". The reason for this limitation is that a consume operation participating in a "dependency-ordered before" relationship provides ordering only with respect to operations to which this consume operation actually carries a dependency. The reason that this limitation applies only to the end of such a concatenation is that any subsequent release operation will provide the required ordering for a prior consume operation. The second exception is that a concatenation is not permitted to consist entirely of "sequenced before". The reasons for this limitation are (1) to permit "inter-thread happens before" to be transitively closed and (2) the "happens before" relation, defined below, provides for relationships consisting entirely of "sequenced before". — *end note*]

14 An evaluation A *happens before* an evaluation B if:

(14.1) — A is sequenced before B, or

(14.2) — A inter-thread happens before B.

The implementation shall ensure that no program execution demonstrates a cycle in the "happens before" relation. [*Note:* This cycle would otherwise be possible only through the use of consume operations. — *end note*]

15 A *visible side effect* A on a scalar object or bit-field M with respect to a value computation B of M satisfies the conditions:

(15.1) — A happens before B and

(15.2) — there is no other side effect X to M such that A happens before X and X happens before B.

The value of a non-atomic scalar object or bit-field M, as determined by evaluation B, shall be the value stored by the visible side effect A. [*Note:* If there is ambiguity about which side effect to a non-atomic object or bit-field is visible, then the behavior is either unspecified or undefined. — *end note*] [*Note:* This states that operations on ordinary objects are not visibly reordered. This is not actually detectable without data

races, but it is necessary to ensure that data races, as defined below, and with suitable restrictions on the use of atomics, correspond to data races in a simple interleaved (sequentially consistent) execution. — *end note*]

16 The value of an atomic object M, as determined by evaluation B, shall be the value stored by some side effect A that modifies M, where B does not happen before A. [*Note:* The set of such side effects is also restricted by the rest of the rules described here, and in particular, by the coherence requirements below. — *end note*]

17 If an operation A that modifies an atomic object M happens before an operation B that modifies M, then A shall be earlier than B in the modification order of M. [*Note:* This requirement is known as write-write coherence. — *end note*]

18 If a value computation A of an atomic object M happens before a value computation B of M, and A takes its value from a side effect X on M, then the value computed by B shall either be the value stored by X or the value stored by a side effect Y on M, where Y follows X in the modification order of M. [*Note:* This requirement is known as read-read coherence. — *end note*]

19 If a value computation A of an atomic object M happens before an operation B that modifies M, then A shall take its value from a side effect X on M, where X precedes B in the modification order of M. [*Note:* This requirement is known as read-write coherence. — *end note*]

20 If a side effect X on an atomic object M happens before a value computation B of M, then the evaluation B shall take its value from X or from a side effect Y that follows X in the modification order of M. [*Note:* This requirement is known as write-read coherence. — *end note*]

21 [*Note:* The four preceding coherence requirements effectively disallow compiler reordering of atomic operations to a single object, even if both operations are relaxed loads. This effectively makes the cache coherence guarantee provided by most hardware available to C++ atomic operations. — *end note*]

22 [*Note:* The value observed by a load of an atomic depends on the "happens before" relation, which depends on the values observed by loads of atomics. The intended reading is that there must exist an association of atomic loads with modifications they observe that, together with suitably chosen modification orders and the "happens before" relation derived as described above, satisfy the resulting constraints as imposed here. — *end note*]

23 Two actions are *potentially concurrent* if

(23.1) — they are performed by different threads, or

(23.2) — they are unsequenced, and at least one is performed by a signal handler.

The execution of a program contains a *data race* if it contains two potentially concurrent conflicting actions, at least one of which is not atomic, and neither happens before the other, except for the special case for signal handlers described below. Any such data race results in undefined behavior. [*Note:* It can be shown that programs that correctly use mutexes and `memory_order_seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as "sequential consistency". However, this applies only to data-race-free programs, and data-race-free programs cannot observe most program transformations that do not change single-threaded program semantics. In fact, most single-threaded program transformations continue to be allowed, since any program that behaves differently as a result must perform an undefined operation. — *end note*]

24 Two accesses to the same object of type `volatile sig_atomic_t` do not result in a data race if both occur in the same thread, even if one or more occurs in a signal handler. For each signal handler invocation, evaluations performed by the thread invoking a signal handler can be divided into two groups A and B,

such that no evaluations in B happen before evaluations in A, and the evaluations of such `volatile sig_atomic_t` objects take values as though all evaluations in A happened before the execution of the signal handler and the execution of the signal handler happened before all evaluations in B.

25 [*Note:* Compiler transformations that introduce assignments to a potentially shared memory location that would not be modified by the abstract machine are generally precluded by this standard, since such an assignment might overwrite another assignment by a different thread in cases in which an abstract machine execution would not have encountered a data race. This includes implementations of data member assignment that overwrite adjacent members in separate memory locations. Reordering of atomic loads in cases in which the atomics in question may alias is also generally precluded, since this may violate the coherence rules. — *end note*]

26 [*Note:* Transformations that introduce a speculative read of a potentially shared memory location may not preserve the semantics of the C++ program as defined in this standard, since they potentially introduce a data race. However, they are typically valid in the context of an optimizing compiler that targets a specific machine with well-defined semantics for data races. They would be invalid for a hypothetical machine that is not tolerant of races or provides hardware race detection. — *end note*]

27 The implementation may assume that any thread will eventually do one of the following:

(27.1) — terminate,

(27.2) — make a call to a library I/O function,

(27.3) — read or modify a volatile object, or

(27.4) — perform a synchronization operation or an atomic operation.

[*Note:* This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. — *end note*]

28 An implementation should ensure that the last value (in modification order) assigned by an atomic or synchronization operation will become visible to all other threads in a finite period of time.

1.11 Acknowledgments [intro.ack]

1 The C++ programming language as described in this International Standard is based on the language as described in Chapter R (Reference Manual) of Stroustrup: *The C++ Programming Language* (second edition, Addison-Wesley Publishing Company, ISBN 0-201-53992-6, copyright ©1991 AT&T). That, in turn, is based on the C programming language as described in Appendix A of Kernighan and Ritchie: *The C Programming Language* (Prentice-Hall, 1978, ISBN 0-13-110163-3, copyright ©1978 AT&T).

2 Portions of the library Clauses of this International Standard are based on work by P.J. Plauger, which was published as *The Draft Standard C++ Library* (Prentice-Hall, ISBN 0-13-117003-1, copyright ©1995 P.J. Plauger).

3 POSIX® is a registered trademark of the Institute of Electrical and Electronic Engineers, Inc.

4 All rights in these originals are reserved.

2 Lexical conventions [lex]

2.1 Separate translation [lex.separate]

1 The text of the program is kept in units called *source files* in this International Standard. A source file together with all the headers (17.6.1.2) and source files included (16.2) via the preprocessing directive `#include`, less any source lines skipped by any of the conditional inclusion (16.1) preprocessing directives, is called a *translation unit*. [*Note:* A C++ program need not all be translated at the same time. — *end note*]

2 [*Note:* Previously translated translation units and instantiation units can be preserved individually or in libraries. The separate translation units of a program communicate (3.5) by (for example) calls to functions whose identifiers have external linkage, manipulation of objects whose identifiers have external linkage, or manipulation of data files. Translation units can be separately translated and then later linked to produce an executable program (3.5). — *end note*]

2.2 Phases of translation [lex.phases]

1 The precedence among the syntax rules of translation is specified by the following phases.[11]

1. Physical source file characters are mapped, in an implementation-defined manner, to the basic source character set (introducing new-line characters for end-of-line indicators) if necessary. The set of physical source file characters accepted is implementation-defined. Any source file character not in the basic source character set (2.3) is replaced by the universal-character-name that designates that character. (An implementation may use any internal encoding, so long as an actual extended character encountered in the source file, and the same extended character expressed in the source file as a universal-character-name (e.g., using the `\uXXXX` notation), are handled equivalently except where this replacement is reverted in a raw string literal.)

2. Each instance of a backslash character (\) immediately followed by a new-line character is deleted, splicing physical source lines to form logical source lines. Only the last backslash on any physical source line shall be eligible for being part of such a splice. Except for splices reverted in a raw string literal, if a splice results in a character sequence that matches the syntax of a universal-character-name, the behavior is undefined. A source file that is not empty and that does not end in a new-line character, or that ends in a new-line character immediately preceded by a backslash character before any such splicing takes place, shall be processed as if an additional new-line character were appended to the file.

3. The source file is decomposed into preprocessing tokens (2.4) and sequences of white-space characters (including comments). A source file shall not end in a partial preprocessing token or in a partial comment.[12] Each comment is replaced by one space character. New-line characters are retained. Whether each nonempty sequence of white-space characters other than new-line is retained or replaced by one space character is unspecified. The process of dividing a source file's characters into preprocessing tokens is context-dependent. [*Example:* see the handling of `<` within a `#include` preprocessing directive. — *end example*]

4. Preprocessing directives are executed, macro invocations are expanded, and `_Pragma` unary operator expressions are executed. If a character sequence that matches the syntax of a universal-character-name

11) Implementations must behave as if these separate phases occur, although in practice different phases might be folded together.

12) A partial preprocessing token would arise from a source file ending in the first portion of a multi-character token that requires a terminating sequence of characters, such as a *header-name* that is missing the closing `"` or `>`. A partial comment would arise from a source file ending with an unclosed `/*` comment.

is produced by token concatenation (16.3.3), the behavior is undefined. A `#include` preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.

5. Each source character set member in a character literal or a string literal, as well as each escape sequence and universal-character-name in a character literal or a non-raw string literal, is converted to the corresponding member of the execution character set (2.13.3, 2.13.5); if there is no corresponding member, it is converted to an implementation-defined member other than the null (wide) character.[13]

6. Adjacent string literal tokens are concatenated.

7. White-space characters separating tokens are no longer significant. Each preprocessing token is converted into a token. (2.6). The resulting tokens are syntactically and semantically analyzed and translated as a translation unit. [*Note:* The process of analyzing and translating the tokens may occasionally result in one token being replaced by a sequence of other tokens (14.2). — *end note*] [*Note:* Source files, translation units and translated translation units need not necessarily be stored as files, nor need there be any one-to-one correspondence between these entities and any external representation. The description is conceptual only, and does not specify any particular implementation. — *end note*]

8. Translated translation units and instantiation units are combined as follows: [*Note:* Some or all of these may be supplied from a library. — *end note*] Each translated translation unit is examined to produce a list of required instantiations. [*Note:* This may include instantiations which have been explicitly requested (14.7.2). — *end note*] The definitions of the required templates are located. It is implementation-defined whether the source of the translation units containing these definitions is required to be available. [*Note:* An implementation could encode sufficient information into the translated translation unit so as to ensure the source is not required here. — *end note*] All the required instantiations are performed to produce *instantiation units*. [*Note:* These are similar to translated translation units, but contain no references to uninstantiated templates and no template definitions. — *end note*] The program is ill-formed if any instantiation fails.

9. All external entity references are resolved. Library components are linked to satisfy external references to entities not defined in the current translation. All such translator output is collected into a program image which contains information needed for execution in its execution environment.

2.3 Character sets [lex.charset]

1 The *basic source character set* consists of 96 characters: the space character, the control characters representing horizontal tab, vertical tab, form feed, and new-line, plus the following 91 graphical characters:[14]

```
a b c d e f g h i j k l m n o p q r s t u v w x y z

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

0 1 2 3 4 5 6 7 8 9

_ { } [ ] # ( ) < > % : ; . ? * + - / ^ & | ~ ! = , \ " '
```

2 The *universal-character-name* construct provides a way to name other characters.

hex-quad:
hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

13) An implementation need not convert all non-corresponding source characters to the same execution character.

14) The glyphs for the members of the basic source character set are intended to identify characters from the subset of ISO/IEC 10646 which corresponds to the ASCII character set. However, because the mapping from source file characters to the source character set (described in translation phase 1) is specified as implementation-defined, an implementation is required to document how the basic source characters are represented in source files.

universal-character-name:
`\u` *hex-quad*
`\U` *hex-quad hex-quad*

The character designated by the universal-character-name `\UNNNNNNNN` is that character whose character short name in ISO/IEC 10646 is `NNNNNNNN`; the character designated by the universal-character-name `\uNNNN` is that character whose character short name in ISO/IEC 10646 is `0000NNNN`. If the hexadecimal value for a universal-character-name corresponds to a surrogate code point (in the range 0xD800–0xDFFF, inclusive), the program is ill-formed. Additionally, if the hexadecimal value for a universal-character-name outside the *c-char-sequence*, *s-char-sequence*, or *r-char-sequence* of a character or string literal corresponds to a control character (in either of the ranges 0x00–0x1F or 0x7F–0x9F, both inclusive) or to a character in the basic source character set, the program is ill-formed.[15]

3 The *basic execution character set* and the *basic execution wide-character set* shall each contain all the members of the basic source character set, plus control characters representing alert, backspace, and carriage return, plus a *null character* (respectively, *null wide character*), whose value is 0. For each basic execution character set, the values of the members shall be non-negative and distinct from one another. In both the source and execution basic character sets, the value of each character after `0` in the above list of decimal digits shall be one greater than the value of the previous. The *execution character set* and the *execution wide-character set* are implementation-defined supersets of the basic execution character set and the basic execution wide-character set, respectively. The values of the members of the execution character sets and the sets of additional members are locale-specific.

2.4 Preprocessing tokens [lex.pptoken]

preprocessing-token:
header-name
identifier
pp-number
character-literal
user-defined-character-literal
string-literal
user-defined-string-literal
preprocessing-op-or-punc
each non-white-space character that cannot be one of the above

1 Each preprocessing token that is converted to a token (2.6) shall have the lexical form of a keyword, an identifier, a literal, an operator, or a punctuator.

2 A preprocessing token is the minimal lexical element of the language in translation phases 3 through 6. The categories of preprocessing token are: header names, identifiers, preprocessing numbers, character literals (including user-defined character literals), string literals (including user-defined string literals), preprocessing operators and punctuators, and single non-white-space characters that do not lexically match the other preprocessing token categories. If a `'` or a `"` character matches the last category, the behavior is undefined. Preprocessing tokens can be separated by white space; this consists of comments (2.7), or white-space characters (space, horizontal tab, new-line, vertical tab, and form-feed), or both. As described in Clause 16, in certain circumstances during translation phase 4, white space (or the absence thereof) serves as more than preprocessing token separation. White space can appear within a preprocessing token only as part of a header name or between the quotation characters in a character literal or string literal.

3 If the input stream has been parsed into preprocessing tokens up to a given character:

(3.1) — If the next character begins a sequence of characters that could be the prefix and initial double quote of a raw string literal, such as `R"`, the next preprocessing token shall be a raw string literal. Between

15) A sequence of characters resembling a universal-character-name in an *r-char-sequence* (2.13.5) does not form a universal-character-name.

the initial and final double quote characters of the raw string, any transformations performed in phases 1 and 2 (universal-character-names and line splicing) are reverted; this reversion shall apply before any *d-char*, *r-char*, or delimiting parenthesis is identified. The raw string literal is defined as the shortest sequence of characters that matches the raw-string pattern

encoding-prefix$_{opt}$`R` *raw-string*

(3.2) — Otherwise, if the next three characters are `<::` and the subsequent character is neither `:` nor `>`, the `<` is treated as a preprocessor token by itself and not as the first character of the alternative token `<:`.

(3.3) — Otherwise, the next preprocessing token is the longest sequence of characters that could constitute a preprocessing token, even if that would cause further lexical analysis to fail.

[*Example:*

```
#define R "x"
const char* s = R"y";           // ill-formed raw string, not "x" "y"
```

— *end example*]

4 [*Example:* The program fragment `0xe+foo` is parsed as a preprocessing number token (one that is not a valid floating or integer literal token), even though a parse as three preprocessing tokens `0xe`, `+`, and `foo` might produce a valid expression (for example, if `foo` were a macro defined as `1`). Similarly, the program fragment `1E1` is parsed as a preprocessing number (one that is a valid floating literal token), whether or not `E` is a macro name. — *end example*]

5 [*Example:* The program fragment `x+++++y` is parsed as `x ++ ++ + y`, which, if `x` and `y` have integral types, violates a constraint on increment operators, even though the parse `x ++ + ++ y` might yield a correct expression. — *end example*]

2.5 Alternative tokens [lex.digraph]

1 Alternative token representations are provided for some operators and punctuators.[16]

2 In all respects of the language, each alternative token behaves the same, respectively, as its primary token, except for its spelling.[17] The set of alternative tokens is defined in Table 1.

Table 1 — Alternative tokens

Alternative	Primary	Alternative	Primary	Alternative	Primary
`<%`	`{`	`and`	`&&`	`and_eq`	`&=`
`%>`	`}`	`bitor`	`|`	`or_eq`	`|=`
`<:`	`[`	`or`	`||`	`xor_eq`	`^=`
`:>`	`]`	`xor`	`^`	`not`	`!`
`%:`	`#`	`compl`	`~`	`not_eq`	`!=`
`%:%:`	`##`	`bitand`	`&`		

16) These include "digraphs" and additional reserved words. The term "digraph" (token consisting of two characters) is not perfectly descriptive, since one of the alternative preprocessing-tokens is `%:%:` and of course several primary tokens contain two characters. Nonetheless, those alternative tokens that aren't lexical keywords are colloquially known as "digraphs".

17) Thus the "stringized" values (16.3.2) of `[` and `<:` will be different, maintaining the source spelling, but the tokens can otherwise be freely interchanged.

2.6 Tokens [lex.token]

token:
identifier
keyword
literal
operator
punctuator

1 There are five kinds of tokens: identifiers, keywords, literals,[18] operators, and other separators. Blanks, horizontal and vertical tabs, newlines, formfeeds, and comments (collectively, "white space"), as described below, are ignored except as they serve to separate tokens. [*Note:* Some white space is required to separate otherwise adjacent identifiers, keywords, numeric literals, and alternative tokens containing alphabetic characters. — *end note*]

2.7 Comments [lex.comment]

1 The characters `/*` start a comment, which terminates with the characters `*/`. These comments do not nest. The characters `//` start a comment, which terminates immediately before the next new-line character. If there is a form-feed or a vertical-tab character in such a comment, only white-space characters shall appear between it and the new-line that terminates the comment; no diagnostic is required. [*Note:* The comment characters `//`, `/*`, and `*/` have no special meaning within a `//` comment and are treated just like other characters. Similarly, the comment characters `//` and `/*` have no special meaning within a `/*` comment. — *end note*]

2.8 Header names [lex.header]

header-name:
`<` *h-char-sequence* `>`
`"` *q-char-sequence* `"`

h-char-sequence:
h-char
h-char-sequence h-char

h-char:
any member of the source character set except new-line and `>`

q-char-sequence:
q-char
q-char-sequence q-char

q-char:
any member of the source character set except new-line and `"`

1 Header name preprocessing tokens shall only appear within a `#include` preprocessing directive (16.2). The sequences in both forms of *header-name*s are mapped in an implementation-defined manner to headers or to external source file names as specified in 16.2.

2 The appearance of either of the characters `'` or `\` or of either of the character sequences `/*` or `//` in a *q-char-sequence* or an *h-char-sequence* is conditionally-supported with implementation-defined semantics, as is the appearance of the character `"` in an *h-char-sequence*.[19]

18) Literals include strings and character and numeric literals.
19) Thus, a sequence of characters that resembles an escape sequence might result in an error, be interpreted as the character corresponding to the escape sequence, or have a completely different meaning, depending on the implementation.

2.9 Preprocessing numbers [lex.ppnumber]

pp-number:
digit
. digit
pp-number digit
pp-number identifier-nondigit
pp-number ' digit
pp-number ' nondigit
pp-number `e` *sign*
pp-number `E` *sign*
pp-number .

1 Preprocessing number tokens lexically include all integer literal tokens (2.13.2) and all floating literal tokens (2.13.4).

2 A preprocessing number does not have a type or a value; it acquires both after a successful conversion to an integer literal token or a floating literal token.

2.10 Identifiers [lex.name]

identifier:
identifier-nondigit
identifier identifier-nondigit
identifier digit

identifier-nondigit:
nondigit
universal-character-name

nondigit: one of
`a b c d e f g h i j k l m`
`n o p q r s t u v w x y z`
`A B C D E F G H I J K L M`
`N O P Q R S T U V W X Y Z _`

digit: one of
`0 1 2 3 4 5 6 7 8 9`

1 An identifier is an arbitrarily long sequence of letters and digits. Each universal-character-name in an identifier shall designate a character whose encoding in ISO 10646 falls into one of the ranges specified in E.1. The initial element shall not be a universal-character-name designating a character whose encoding falls into one of the ranges specified in E.2. Upper- and lower-case letters are different. All characters are significant.[20]

2 The identifiers in Table 2 have a special meaning when appearing in a certain context. When referred to in the grammar, these identifiers are used explicitly rather than using the *identifier* grammar production. Unless otherwise specified, any ambiguity as to whether a given *identifier* has a special meaning is resolved to interpret the token as a regular *identifier*.

Table 2 — Identifiers with special meaning

`override`	`final`

3 In addition, some identifiers are reserved for use by C++ implementations and shall not be used otherwise; no diagnostic is required.

20) On systems in which linkers cannot accept extended characters, an encoding of the universal-character-name may be used in forming valid external identifiers. For example, some otherwise unused character or sequence of characters may be used to encode the `\u` in a universal-character-name. Extended characters may produce a long external identifier, but C++ does not place a translation limit on significant characters for external identifiers. In C++, upper- and lower-case letters are considered different for all identifiers, including external identifiers.

(3.1) — Each identifier that contains a double underscore `__` or begins with an underscore followed by an uppercase letter is reserved to the implementation for any use.

(3.2) — Each identifier that begins with an underscore is reserved to the implementation for use as a name in the global namespace.

2.11 Keywords [lex.key]

1 The identifiers shown in Table 3 are reserved for use as keywords (that is, they are unconditionally treated as keywords in phase 7) except in an *attribute-token* (7.6.1) [*Note:* The `export` keyword is unused but is reserved for future use. — *end note*]:

Table 3 — Keywords

`alignas`	`continue`	`friend`	`register`	`true`
`alignof`	`decltype`	`goto`	`reinterpret_cast`	`try`
`asm`	`default`	`if`	`return`	`typedef`
`auto`	`delete`	`inline`	`short`	`typeid`
`bool`	`do`	`int`	`signed`	`typename`
`break`	`double`	`long`	`sizeof`	`union`
`case`	`dynamic_cast`	`mutable`	`static`	`unsigned`
`catch`	`else`	`namespace`	`static_assert`	`using`
`char`	`enum`	`new`	`static_cast`	`virtual`
`char16_t`	`explicit`	`noexcept`	`struct`	`void`
`char32_t`	`export`	`nullptr`	`switch`	`volatile`
`class`	`extern`	`operator`	`template`	`wchar_t`
`const`	`false`	`private`	`this`	`while`
`constexpr`	`float`	`protected`	`thread_local`	
`const_cast`	`for`	`public`	`throw`	

2 Furthermore, the alternative representations shown in Table 4 for certain operators and punctuators (2.5) are reserved and shall not be used otherwise:

Table 4 — Alternative representations

`and`	`and_eq`	`bitand`	`bitor`	`compl`	`not`
`not_eq`	`or`	`or_eq`	`xor`	`xor_eq`	

2.12 Operators and punctuators [lex.operators]

1 The lexical representation of C++ programs includes a number of preprocessing tokens which are used in the syntax of the preprocessor or are converted into tokens for operators and punctuators:

preprocessing-op-or-punc: one of

`{`	`}`	`[`	`]`	`#`	`##`	`(`	`)`	
`<:`	`:>`	`<%`	`%>`	`%:`	`%:%:`	`;`	`:`	`...`
`new`	`delete`	`?`	`::`	`.`	`.*`			
`+`	`-`	`*`	`/`	`%`	`^`	`&`	`|`	`~`
`!`	`=`	`<`	`>`	`+=`	`-=`	`*=`	`/=`	`%=`
`^=`	`&=`	`|=`	`<<`	`>>`	`>>=`	`<<=`	`==`	`!=`
`<=`	`>=`	`&&`	`||`	`++`	`--`	`,`	`->*`	`->`
`and`	`and_eq`	`bitand`	`bitor`	`compl`	`not`	`not_eq`		
`or`	`or_eq`	`xor`	`xor_eq`					

Each *preprocessing-op-or-punc* is converted to a single token in translation phase 7 (2.2).

2.13 Literals [lex.literal]

2.13.1 Kinds of literals [lex.literal.kinds]

1 There are several kinds of literals.[21]

literal:
- *integer-literal*
- *character-literal*
- *floating-literal*
- *string-literal*
- *boolean-literal*
- *pointer-literal*
- *user-defined-literal*

2.13.2 Integer literals [lex.icon]

integer-literal:
- *binary-literal integer-suffix*$_{opt}$
- *octal-literal integer-suffix*$_{opt}$
- *decimal-literal integer-suffix*$_{opt}$
- *hexadecimal-literal integer-suffix*$_{opt}$

binary-literal:
- `0b` *binary-digit*
- `0B` *binary-digit*
- *binary-literal* `'`$_{opt}$ *binary-digit*

octal-literal:
- `0`
- *octal-literal* `'`$_{opt}$ *octal-digit*

decimal-literal:
- *nonzero-digit*
- *decimal-literal* `'`$_{opt}$ *digit*

hexadecimal-literal:
- `0x` *hexadecimal-digit*
- `0X` *hexadecimal-digit*
- *hexadecimal-literal* `'`$_{opt}$ *hexadecimal-digit*

binary-digit:
- `0`
- `1`

octal-digit: one of
- `0 1 2 3 4 5 6 7`

nonzero-digit: one of
- `1 2 3 4 5 6 7 8 9`

hexadecimal-digit: one of
- `0 1 2 3 4 5 6 7 8 9`
- `a b c d e f`
- `A B C D E F`

integer-suffix:
- *unsigned-suffix long-suffix*$_{opt}$
- *unsigned-suffix long-long-suffix*$_{opt}$
- *long-suffix unsigned-suffix*$_{opt}$
- *long-long-suffix unsigned-suffix*$_{opt}$

unsigned-suffix: one of
- `u U`

21) The term "literal" generally designates, in this International Standard, those tokens that are called "constants" in ISO C.

long-suffix: one of
`l L`
long-long-suffix: one of
`ll LL`

1 An *integer literal* is a sequence of digits that has no period or exponent part, with optional separating single quotes that are ignored when determining its value. An integer literal may have a prefix that specifies its base and a suffix that specifies its type. The lexically first digit of the sequence of digits is the most significant. A *binary* integer literal (base two) begins with `0b` or `0B` and consists of a sequence of binary digits. An *octal* integer literal (base eight) begins with the digit `0` and consists of a sequence of octal digits.[22] A *decimal* integer literal (base ten) begins with a digit other than `0` and consists of a sequence of decimal digits. A *hexadecimal* integer literal (base sixteen) begins with `0x` or `0X` and consists of a sequence of hexadecimal digits, which include the decimal digits and the letters `a` through `f` and `A` through `F` with decimal values ten through fifteen. [*Example:* The number twelve can be written `12`, `014`, `0XC`, or `0b1100`. The literals `1048576`, `1'048'576`, `0X100000`, `0x10'0000`, and `0'004'000'000` all have the same value. — *end example*]

2 The type of an integer literal is the first of the corresponding list in Table 5 in which its value can be represented.

Table 5 — Types of integer literals

Suffix	Decimal literal	Binary, octal, or hexadecimal literal

none	`int`
	`long int`
	`long long int`
		`int`
		`unsigned int`
		`long int`
		`unsigned long int`
		`long long int`
		`unsigned long long int`

`u` or `U`	`unsigned int`
	`unsigned long int`
	`unsigned long long int`
		`unsigned int`
		`unsigned long int`
		`unsigned long long int`

`l` or `L`	`long int`
	`long long int`
		`long int`
		`unsigned long int`
		`long long int`
		`unsigned long long int`

Both `u` or `U` and `l` or `L`	`unsigned long int` `unsigned long long int`	`unsigned long int` `unsigned long long int`
`ll` or `LL`	`long long int`	`long long int` `unsigned long long int`
Both `u` or `U` and `ll` or `LL`	`unsigned long long int`	`unsigned long long int`

3 If an integer literal cannot be represented by any type in its list and an extended integer type (3.9.1) can represent its value, it may have that extended integer type. If all of the types in the list for the literal are signed, the extended integer type shall be signed. If all of the types in the list for the literal are unsigned, the extended integer type shall be unsigned. If the list contains both signed and unsigned types, the extended integer type may be signed or unsigned. A program is ill-formed if one of its translation units contains an integer literal that cannot be represented by any of the allowed types.

22) The digits `8` and `9` are not octal digits.

2.13.3 Character literals [lex.ccon]

character-literal:
encoding-prefix$_{opt}$ `'` *c-char-sequence* `'`

encoding-prefix: one of
`u8` `u` `U` `L`

c-char-sequence:
c-char
c-char-sequence c-char

c-char:
any member of the source character set except
the single-quote `'`, backslash `\`, or new-line character
escape-sequence
universal-character-name

escape-sequence:
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence

simple-escape-sequence: one of
`\'` `\"` `\?` `\\`
`\a` `\b` `\f` `\n` `\r` `\t` `\v`

octal-escape-sequence:
`\` *octal-digit*
`\` *octal-digit octal-digit*
`\` *octal-digit octal-digit octal-digit*

hexadecimal-escape-sequence:
`\x` *hexadecimal-digit*
hexadecimal-escape-sequence hexadecimal-digit

[1] A character literal is one or more characters enclosed in single quotes, as in `'x'`, optionally preceded by `u8`, `u`, `U`, or `L`, as in `u8'w'`, `u'x'`, `U'y'`, or `L'z'`, respectively.

[2] A character literal that does not begin with `u8`, `u`, `U`, or `L` is an *ordinary character literal*. An ordinary character literal that contains a single *c-char* representable in the execution character set has type `char`, with value equal to the numerical value of the encoding of the *c-char* in the execution character set. An ordinary character literal that contains more than one *c-char* is a *multicharacter literal*. A multicharacter literal, or an ordinary character literal containing a single *c-char* not representable in the execution character set, is conditionally-supported, has type `int`, and has an implementation-defined value.

[3] A character literal that begins with `u8`, such as `u8'w'`, is a character literal of type `char`, known as a *UTF-8 character literal*. The value of a UTF-8 character literal is equal to its ISO 10646 code point value, provided that the code point value is representable with a single UTF-8 code unit (that is, provided it is in the C0 Controls and Basic Latin Unicode block). If the value is not representable with a single UTF-8 code unit, the program is ill-formed. A UTF-8 character literal containing multiple *c-char*s is ill-formed.

[4] A character literal that begins with the letter `u`, such as `u'x'`, is a character literal of type `char16_t`. The value of a `char16_t` literal containing a single *c-char* is equal to its ISO 10646 code point value, provided that the code point is representable with a single 16-bit code unit. (That is, provided it is a basic multi-lingual plane code point.) If the value is not representable within 16 bits, the program is ill-formed. A `char16_t` literal containing multiple *c-char*s is ill-formed.

[5] A character literal that begins with the letter `U`, such as `U'y'`, is a character literal of type `char32_t`. The value of a `char32_t` literal containing a single *c-char* is equal to its ISO 10646 code point value. A `char32_t` literal containing multiple *c-char*s is ill-formed.

[6] A character literal that begins with the letter `L`, such as `L'z'`, is a wide-character literal. A wide-character

literal has type `wchar_t`.[23] The value of a wide-character literal containing a single *c-char* has value equal to the numerical value of the encoding of the *c-char* in the execution wide-character set, unless the *c-char* has no representation in the execution wide-character set, in which case the value is implementation-defined. [*Note:* The type `wchar_t` is able to represent all members of the execution wide-character set (see 3.9.1). — *end note*]. The value of a wide-character literal containing multiple *c-char*s is implementation-defined.

7 Certain nongraphic characters, the single quote `'`, the double quote `"`, the question mark `?`,[24] and the backslash `\`, can be represented according to Table 6. The double quote `"` and the question mark `?`, can be represented as themselves or by the escape sequences `\"` and `\?` respectively, but the single quote `'` and the backslash `\` shall be represented by the escape sequences `\'` and `\\` respectively. Escape sequences in which the character following the backslash is not listed in Table 6 are conditionally-supported, with implementation-defined semantics. An escape sequence specifies a single character.

Table 6 — Escape sequences

new-line	NL(LF)	`\n`
horizontal tab	HT	`\t`
vertical tab	VT	`\v`
backspace	BS	`\b`
carriage return	CR	`\r`
form feed	FF	`\f`
alert	BEL	`\a`
backslash	\	`\\`
question mark	?	`\?`
single quote	'	`\'`
double quote	"	`\"`
octal number	*ooo*	`\`*`ooo`*
hex number	*hhh*	`\x`*`hhh`*

8 The escape `\`*`ooo`* consists of the backslash followed by one, two, or three octal digits that are taken to specify the value of the desired character. The escape `\x`*`hhh`* consists of the backslash followed by `x` followed by one or more hexadecimal digits that are taken to specify the value of the desired character. There is no limit to the number of digits in a hexadecimal sequence. A sequence of octal or hexadecimal digits is terminated by the first character that is not an octal digit or a hexadecimal digit, respectively. The value of a character literal is implementation-defined if it falls outside of the implementation-defined range defined for `char` (for literals with no prefix) or `wchar_t` (for literals prefixed by `L`). [*Note:* If the value of a character literal prefixed by `u`, `u8`, or `U` is outside the range defined for its type, the program is ill-formed. — *end note*]

9 A universal-character-name is translated to the encoding, in the appropriate execution character set, of the character named. If there is no such encoding, the universal-character-name is translated to an implementation-defined encoding. [*Note:* In translation phase 1, a universal-character-name is introduced whenever an actual extended character is encountered in the source text. Therefore, all extended characters are described in terms of universal-character-names. However, the actual compiler implementation may use its own native character set, so long as the same results are obtained. — *end note*]

2.13.4 Floating literals [lex.fcon]

floating-literal:
fractional-constant exponent-part$_{opt}$ *floating-suffix*$_{opt}$
digit-sequence exponent-part floating-suffix$_{opt}$

23) They are intended for character sets where a character does not fit into a single byte.
24) Using an escape sequence for a question mark is supported for compatibility with ISO C++14 and ISO C.

fractional-constant:
digit-sequence$_{opt}$ `.` *digit-sequence*
digit-sequence `.`

exponent-part:
`e` *sign*$_{opt}$ *digit-sequence*
`E` *sign*$_{opt}$ *digit-sequence*

sign: one of
`+ -`

digit-sequence:
digit
digit-sequence `'`$_{opt}$ *digit*

floating-suffix: one of
`f l F L`

1 A floating literal consists of an integer part, a decimal point, a fraction part, an `e` or `E`, an optionally signed integer exponent, and an optional type suffix. The integer and fraction parts both consist of a sequence of decimal (base ten) digits. Optional separating single quotes in a *digit-sequence* are ignored when determining its value. [*Example:* The literals `1.602'176'565e-19` and `1.602176565e-19` have the same value. — *end example*] Either the integer part or the fraction part (not both) can be omitted; either the decimal point or the letter `e` (or `E`) and the exponent (not both) can be omitted. The integer part, the optional decimal point and the optional fraction part form the *significant part* of the floating literal. The exponent, if present, indicates the power of 10 by which the significant part is to be scaled. If the scaled value is in the range of representable values for its type, the result is the scaled value if representable, else the larger or smaller representable value nearest the scaled value, chosen in an implementation-defined manner. The type of a floating literal is `double` unless explicitly specified by a suffix. The suffixes `f` and `F` specify `float`, the suffixes `l` and `L` specify `long double`. If the scaled value is not in the range of representable values for its type, the program is ill-formed.

2.13.5 String literals [lex.string]

string-literal:
encoding-prefix$_{opt}$ `"` *s-char-sequence*$_{opt}$ `"`
encoding-prefix$_{opt}$ `R` *raw-string*

s-char-sequence:
s-char
s-char-sequence s-char

s-char:
any member of the source character set except
the double-quote `"`, backslash `\`, or new-line character
escape-sequence
universal-character-name

raw-string:
`"` *d-char-sequence*$_{opt}$ `(` *r-char-sequence*$_{opt}$ `)` *d-char-sequence*$_{opt}$ `"`

r-char-sequence:
r-char
r-char-sequence r-char

r-char:
any member of the source character set, except
a right parenthesis `)` followed by the initial *d-char-sequence*
(which may be empty) followed by a double quote `"`.

d-char-sequence:
d-char
d-char-sequence d-char

d-char:
any member of the basic source character set except:
space, the left parenthesis `(`, the right parenthesis `)`, the backslash `\`,
and the control characters representing horizontal tab,
vertical tab, form feed, and newline.

1 A *string-literal* is a sequence of characters (as defined in 2.13.3) surrounded by double quotes, optionally prefixed by `R`, `u8`, `u8R`, `u`, `uR`, `U`, `UR`, `L`, or `LR`, as in `"..."`, `R"(...)"`, `u8"..."`, `u8R"**(...)**"`, `u"..."`, `uR"*~(...)*~"`, `U"..."`, `UR"zzz(...)zzz"`, `L"..."`, or `LR"(...)"`, respectively.

2 A *string-literal* that has an `R` in the prefix is a *raw string literal*. The *d-char-sequence* serves as a delimiter. The terminating *d-char-sequence* of a *raw-string* is the same sequence of characters as the initial *d-char-sequence*. A *d-char-sequence* shall consist of at most 16 characters.

3 [*Note:* The characters `'('` and `')'` are permitted in a *raw-string*. Thus, `R"delimiter((a|b))delimiter"` is equivalent to `"(a|b)"`. — *end note*]

4 [*Note:* A source-file new-line in a raw string literal results in a new-line in the resulting execution *string-literal*. Assuming no whitespace at the beginning of lines in the following example, the assert will succeed:

```
const char* p = R"(a\
b
c)";
assert(std::strcmp(p, "a\\\nb\nc") == 0);
```

— *end note*]

5 [*Example:* The raw string

```
R"a(
)\
a"
)a"
```

is equivalent to `"\n)\\\na\"\n"`. The raw string

```
R"(??)"
```

is equivalent to `"\?\?"`. The raw string

```
R"#(
)??="
)#"
```

is equivalent to `"\n)\?\?=\"\n"`. — *end example*]

6 After translation phase 6, a *string-literal* that does not begin with an *encoding-prefix* is an ordinary string literal, and is initialized with the given characters.

7 A *string-literal* that begins with `u8`, such as `u8"asdf"`, is a UTF-8 string literal.

8 Ordinary string literals and UTF-8 string literals are also referred to as narrow string literals. A narrow string literal has type "array of *n* `const char`", where *n* is the size of the string as defined below, and has static storage duration (3.7).

9 For a UTF-8 string literal, each successive element of the object representation (3.9) has the value of the corresponding code unit of the UTF-8 encoding of the string.

10 A *string-literal* that begins with `u`, such as `u"asdf"`, is a `char16_t` string literal. A `char16_t` string literal has type "array of *n* `const char16_t`", where *n* is the size of the string as defined below; it is initialized with the given characters. A single *c-char* may produce more than one `char16_t` character in the form of surrogate pairs.

11 A *string-literal* that begins with `U`, such as `U"asdf"`, is a `char32_t` string literal. A `char32_t` string literal has type "array of *n* `const char32_t`", where *n* is the size of the string as defined below; it is initialized with the given characters.

12 A *string-literal* that begins with `L`, such as `L"asdf"`, is a wide string literal. A wide string literal has type "array of *n* `const wchar_t`", where *n* is the size of the string as defined below; it is initialized with the given characters.

13 In translation phase 6 (2.2), adjacent *string-literal*s are concatenated. If both *string-literal*s have the same *encoding-prefix*, the resulting concatenated string literal has that *encoding-prefix*. If one *string-literal* has no *encoding-prefix*, it is treated as a *string-literal* of the same *encoding-prefix* as the other operand. If a UTF-8 string literal token is adjacent to a wide string literal token, the program is ill-formed. Any other concatenations are conditionally-supported with implementation-defined behavior. [*Note:* This concatenation is an interpretation, not a conversion. Because the interpretation happens in translation phase 6 (after each character from a literal has been translated into a value from the appropriate character set), a *string-literal*'s initial rawness has no effect on the interpretation or well-formedness of the concatenation. — *end note*] Table 7 has some examples of valid concatenations.

Table 7 — String literal concatenations

Source		Means	Source		Means	Source		Means
`u"a"`	`u"b"`	`u"ab"`	`U"a"`	`U"b"`	`U"ab"`	`L"a"`	`L"b"`	`L"ab"`
`u"a"`	`"b"`	`u"ab"`	`U"a"`	`"b"`	`U"ab"`	`L"a"`	`"b"`	`L"ab"`
`"a"`	`u"b"`	`u"ab"`	`"a"`	`U"b"`	`U"ab"`	`"a"`	`L"b"`	`L"ab"`

Characters in concatenated strings are kept distinct.

[*Example:*

```
"\xA" "B"
```

contains the two characters `'\xA'` and `'B'` after concatenation (and not the single hexadecimal character `'\xAB'`). — *end example*]

14 After any necessary concatenation, in translation phase 7 (2.2), `'\0'` is appended to every string literal so that programs that scan a string can find its end.

15 Escape sequences and universal-character-names in non-raw string literals have the same meaning as in character literals (2.13.3), except that the single quote `'` is representable either by itself or by the escape sequence `\'`, and the double quote `"` shall be preceded by a `\`, and except that a universal-character-name in a `char16_t` string literal may yield a surrogate pair. In a narrow string literal, a universal-character-name may map to more than one `char` element due to *multibyte encoding*. The size of a `char32_t` or wide string literal is the total number of escape sequences, universal-character-names, and other characters, plus one for the terminating `U'\0'` or `L'\0'`. The size of a `char16_t` string literal is the total number of escape sequences, universal-character-names, and other characters, plus one for each character requiring a surrogate pair, plus one for the terminating `u'\0'`. [*Note:* The size of a `char16_t` string literal is the number of code units, not the number of characters. — *end note*] Within `char32_t` and `char16_t` literals, any universal-character-names shall be within the range `0x0` to `0x10FFFF`. The size of a narrow string literal is the total number of escape sequences and other characters, plus at least one for the multibyte encoding of each universal-character-name, plus one for the terminating `'\0'`.

16 Evaluating a *string-literal* results in a string literal object with static storage duration, initialized from the given characters as specified above. Whether all string literals are distinct (that is, are stored in nonoverlapping objects) and whether successive evaluations of a *string-literal* yield the same or a different object is unspecified. [*Note:* The effect of attempting to modify a string literal is undefined. — *end note*]

2.13.6 Boolean literals [lex.bool]

boolean-literal:
`false`
`true`

1 The Boolean literals are the keywords `false` and `true`. Such literals are prvalues and have type `bool`.

2.13.7 Pointer literals [lex.nullptr]

pointer-literal:
`nullptr`

1 The pointer literal is the keyword `nullptr`. It is a prvalue of type `std::nullptr_t`. [*Note:* `std::nullptr_t` is a distinct type that is neither a pointer type nor a pointer to member type; rather, a prvalue of this type is a null pointer constant and can be converted to a null pointer value or null member pointer value. See 4.10 and 4.11. — *end note*]

2.13.8 User-defined literals [lex.ext]

user-defined-literal:
user-defined-integer-literal
user-defined-floating-literal
user-defined-string-literal
user-defined-character-literal

user-defined-integer-literal:
decimal-literal ud-suffix
octal-literal ud-suffix
hexadecimal-literal ud-suffix
binary-literal ud-suffix

user-defined-floating-literal:
fractional-constant exponent-part$_{opt}$ *ud-suffix*
digit-sequence exponent-part ud-suffix

user-defined-string-literal:
string-literal ud-suffix

user-defined-character-literal:
character-literal ud-suffix

ud-suffix:
identifier

1 If a token matches both *user-defined-literal* and another literal kind, it is treated as the latter. [*Example:* `123_km` is a *user-defined-literal*, but `12LL` is an *integer-literal*. — *end example*] The syntactic non-terminal preceding the *ud-suffix* in a *user-defined-literal* is taken to be the longest sequence of characters that could match that non-terminal.

2 A *user-defined-literal* is treated as a call to a literal operator or literal operator template (13.5.8). To determine the form of this call for a given *user-defined-literal* L with *ud-suffix* X, the *literal-operator-id* whose literal suffix identifier is X is looked up in the context of L using the rules for unqualified name lookup (3.4.1). Let S be the set of declarations found by this lookup. S shall not be empty.

3 If L is a *user-defined-integer-literal*, let n be the literal without its *ud-suffix*. If S contains a literal operator with parameter type `unsigned long long`, the literal L is treated as a call of the form

```
operator "" X(nULL)
```

Otherwise, S shall contain a raw literal operator or a literal operator template (13.5.8) but not both. If S contains a raw literal operator, the literal L is treated as a call of the form

```
operator "" X("n")
```

Otherwise (S contains a literal operator template), L is treated as a call of the form

```
operator "" X<'c1', 'c2', ... 'ck'>()
```

where n is the source character sequence $c_1c_2...c_k$. [*Note:* The sequence $c_1c_2...c_k$ can only contain characters from the basic source character set. — *end note*]

4 If L is a *user-defined-floating-literal*, let f be the literal without its *ud-suffix*. If S contains a literal operator with parameter type `long double`, the literal L is treated as a call of the form

```
operator "" X(fL)
```

Otherwise, S shall contain a raw literal operator or a literal operator template (13.5.8) but not both. If S contains a raw literal operator, the *literal* L is treated as a call of the form

```
operator "" X("f")
```

Otherwise (S contains a literal operator template), L is treated as a call of the form

```
operator "" X<'c1', 'c2', ... 'ck'>()
```

where f is the source character sequence $c_1c_2...c_k$. [*Note:* The sequence $c_1c_2...c_k$ can only contain characters from the basic source character set. — *end note*]

5 If L is a *user-defined-string-literal*, let *str* be the literal without its *ud-suffix* and let *len* be the number of code units in *str* (i.e., its length excluding the terminating null character). The literal L is treated as a call of the form

```
operator "" X(str, len)
```

6 If L is a *user-defined-character-literal*, let *ch* be the literal without its *ud-suffix*. S shall contain a literal operator (13.5.8) whose only parameter has the type of *ch* and the literal L is treated as a call of the form

```
operator "" X(ch)
```

7 [*Example:*

```
long double operator "" _w(long double);
std::string operator "" _w(const char16_t*, std::size_t);
unsigned operator "" _w(const char*);
int main() {
  1.2_w;      // calls operator "" _w(1.2L)
  u"one"_w;   // calls operator "" _w(u"one", 3)
  12_w;       // calls operator "" _w("12")
  "two"_w;    // error: no applicable literal operator
}
```

— *end example*]

8 In translation phase 6 (2.2), adjacent string literals are concatenated and *user-defined-string-literal*s are considered string literals for that purpose. During concatenation, *ud-suffix*es are removed and ignored and the concatenation process occurs as described in 2.13.5. At the end of phase 6, if a string literal is the result of a concatenation involving at least one *user-defined-string-literal*, all the participating *user-defined-string-literal*s shall have the same *ud-suffix* and that suffix is applied to the result of the concatenation.

9 [*Example:*

```
int main() {
  L"A" "B" "C"_x; // OK: same as L"ABC"_x
  "P"_x "Q" "R"_y;// error: two different ud-suffixes
}
```

— *end example*]

3 Basic concepts [basic]

1 [*Note:* This Clause presents the basic concepts of the C++ language. It explains the difference between an *object* and a *name* and how they relate to the value categories for expressions. It introduces the concepts of a *declaration* and a *definition* and presents C++'s notion of *type*, *scope*, *linkage*, and *storage duration*. The mechanisms for starting and terminating a program are discussed. Finally, this Clause presents the *fundamental* types of the language and lists the ways of constructing *compound* types from these. — *end note*]

2 [*Note:* This Clause does not cover concepts that affect only a single part of the language. Such concepts are discussed in the relevant Clauses. — *end note*]

3 An *entity* is a value, object, reference, function, enumerator, type, class member, bit-field, template, template specialization, namespace, parameter pack, or `this`.

4 A *name* is a use of an *identifier* (2.10), *operator-function-id* (13.5), *literal-operator-id* (13.5.8), *conversion-function-id* (12.3.2), or *template-id* (14.2) that denotes an entity or *label* (6.6.4, 6.1).

5 Every name that denotes an entity is introduced by a *declaration*. Every name that denotes a label is introduced either by a `goto` statement (6.6.4) or a *labeled-statement* (6.1).

6 A *variable* is introduced by the declaration of a reference other than a non-static data member or of an object. The variable's name, if any, denotes the reference or object.

7 Some names denote types or templates. In general, whenever a name is encountered it is necessary to determine whether that name denotes one of these entities before continuing to parse the program that contains it. The process that determines this is called *name lookup* (3.4).

8 Two names are *the same* if

(8.1) — they are *identifier*s composed of the same character sequence, or

(8.2) — they are *operator-function-id*s formed with the same operator, or

(8.3) — they are *conversion-function-id*s formed with the same type, or

(8.4) — they are *template-id*s that refer to the same class, function, or variable (14.4), or

(8.5) — they are the names of literal operators (13.5.8) formed with the same literal suffix identifier.

9 A name used in more than one translation unit can potentially refer to the same entity in these translation units depending on the linkage (3.5) of the name specified in each translation unit.

3.1 Declarations and definitions [basic.def]

1 A declaration (Clause 7) may introduce one or more names into a translation unit or redeclare names introduced by previous declarations. If so, the declaration specifies the interpretation and attributes of these names. A declaration may also have effects including:

(1.1) — a static assertion (Clause 7),

(1.2) — controlling template instantiation (14.7.2),

(1.3) — use of attributes (Clause 7), and

(1.4) — nothing (in the case of an *empty-declaration*).

2 A declaration is a *definition* unless it declares a function without specifying the function's body (8.4), it contains the `extern` specifier (7.1.1) or a *linkage-specification*[25] (7.5) and neither an *initializer* nor a *function-body*, it declares a static data member in a class definition (9.2, 9.4), it is a class name declaration (9.1), it is an *opaque-enum-declaration* (7.2), it is a *template-parameter* (14.1), it is a *parameter-declaration* (8.3.5) in a function declarator that is not the *declarator* of a *function-definition*, or it is a `typedef` declaration (7.1.3), an *alias-declaration* (7.1.3), a *using-declaration* (7.3.3), a *static_assert-declaration* (Clause 7), an *attribute-declaration* (Clause 7), an *empty-declaration* (Clause 7), a *using-directive* (7.3.4), an explicit instantiation declaration (14.7.2), or an explicit specialization (14.7.3) whose *declaration* is not a definition.

[*Example:* all but one of the following are definitions:

```
int a;                          // defines a
extern const int c = 1;         // defines c
int f(int x) { return x+a; }    // defines f and defines x
struct S { int a; int b; };     // defines S, S::a, and S::b
struct X {                      // defines X
  int x;                        // defines non-static data member x
  static int y;                 // declares static data member y
  X(): x(0) { }                 // defines a constructor of X
};
int X::y = 1;                   // defines X::y
enum { up, down };              // defines up and down
namespace N { int d; }          // defines N and N::d
namespace N1 = N;               // defines N1
X anX;                          // defines anX
```

whereas these are just declarations:

```
extern int a;                   // declares a
extern const int c;             // declares c
int f(int);                     // declares f
struct S;                       // declares S
typedef int Int;                // declares Int
extern X anotherX;              // declares anotherX
using N::d;                     // declares d
```

— *end example*]

3 [*Note:* In some circumstances, C++ implementations implicitly define the default constructor (12.1), copy constructor (12.8), move constructor (12.8), copy assignment operator (12.8), move assignment operator (12.8), or destructor (12.4) member functions. — *end note*] [*Example:* given

```
#include <string>

struct C {
  std::string s;                // std::string is the standard library class (Clause 21)
};

int main() {
  C a;
  C b = a;
  b = a;
}
```

the implementation will implicitly define functions to make the definition of `C` equivalent to

25) Appearing inside the braced-enclosed *declaration-seq* in a *linkage-specification* does not affect whether a declaration is a definition.

```
struct C {
  std::string s;
  C() : s() { }
  C(const C& x): s(x.s) { }
  C(C&& x): s(static_cast<std::string&&>(x.s)) { }
    // :  s(std::move(x.s)) { }
  C& operator=(const C& x) { s = x.s; return *this; }
  C& operator=(C&& x) { s = static_cast<std::string&&>(x.s); return *this; }
    // { s = std::move(x.s); return *this; }
  ~C() { }
};
```

— *end example*]

4 [*Note:* A class name can also be implicitly declared by an *elaborated-type-specifier* (7.1.6.3). — *end note*]

5 A program is ill-formed if the definition of any object gives the object an incomplete type (3.9).

3.2 One-definition rule [basic.def.odr]

1 No translation unit shall contain more than one definition of any variable, function, class type, enumeration type, or template.

2 An expression is *potentially evaluated* unless it is an unevaluated operand (Clause 5) or a subexpression thereof. The set of *potential results* of an expression `e` is defined as follows:

(2.1) — If `e` is an *id-expression* (5.1.1), the set contains only `e`.

(2.2) — If `e` is a subscripting operation (5.2.1) with an array operand, the set contains that operand.

(2.3) — If `e` is a class member access expression (5.2.5), the set contains the potential results of the object expression.

(2.4) — If `e` is a pointer-to-member expression (5.5) whose second operand is a constant expression, the set contains the potential results of the object expression.

(2.5) — If `e` has the form `(e1)`, the set contains the potential results of `e1`.

(2.6) — If `e` is a glvalue conditional expression (5.16), the set is the union of the sets of potential results of the second and third operands.

(2.7) — If `e` is a comma expression (5.19), the set contains the potential results of the right operand.

(2.8) — Otherwise, the set is empty.

[*Note:* This set is a (possibly-empty) set of *id-expressions*, each of which is either `e` or a subexpression of `e`. [*Example:* In the following example, the set of potential results of the initializer of `n` contains the first `S::x` subexpression, but not the second `S::x` subexpression.

```
struct S { static const int x = 0; };
const int &f(const int &r);
int n = b ? (1, S::x)  // S::x is not odr-used here
          : f(S::x);   // S::x is odr-used here, so
                       // a definition is required
```

— *end example*] — *end note*]

3 A variable `x` whose name appears as a potentially-evaluated expression `ex` is *odr-used* by `ex` unless applying the lvalue-to-rvalue conversion (4.1) to `x` yields a constant expression (5.20) that does not invoke any non-trivial functions and, if `x` is an object, `ex` is an element of the set of potential results of an expression `e`,

where either the lvalue-to-rvalue conversion (4.1) is applied to `e`, or `e` is a discarded-value expression (Clause 5). `this` is odr-used if it appears as a potentially-evaluated expression (including as the result of the implicit transformation in the body of a non-static member function (9.3.1)). A virtual member function is odr-used if it is not pure. A function whose name appears as a potentially-evaluated expression is odr-used if it is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4), unless it is a pure virtual function and either its name is not explicitly qualified or the expression forms a pointer to member (5.3.1). [*Note:* This covers calls to named functions (5.2.2), operator overloading (Clause 13), user-defined conversions (12.3.2), allocation function for placement new (5.3.4), as well as non-default initialization (8.5). A constructor selected to copy or move an object of class type is odr-used even if the call is actually elided by the implementation (12.8). — *end note*] An allocation or deallocation function for a class is odr-used by a *new-expression* appearing in a potentially-evaluated expression as specified in 5.3.4 and 12.5. A deallocation function for a class is odr-used by a delete expression appearing in a potentially-evaluated expression as specified in 5.3.5 and 12.5. A non-placement allocation or deallocation function for a class is odr-used by the definition of a constructor of that class. A non-placement deallocation function for a class is odr-used by the definition of the destructor of that class, or by being selected by the lookup at the point of definition of a virtual destructor (12.4).[26] An assignment operator function in a class is odr-used by an implicitly-defined copy-assignment or move-assignment function for another class as specified in 12.8. A constructor for a class is odr-used as specified in 8.5. A destructor for a class is odr-used if it is potentially invoked (12.4).

4 Every program shall contain exactly one definition of every non-inline function or variable that is odr-used in that program; no diagnostic required. The definition can appear explicitly in the program, it can be found in the standard or a user-defined library, or (when appropriate) it is implicitly defined (see 12.1, 12.4 and 12.8). An inline function shall be defined in every translation unit in which it is odr-used.

5 Exactly one definition of a class is required in a translation unit if the class is used in a way that requires the class type to be complete. [*Example:* the following complete translation unit is well-formed, even though it never defines `X`:

```
struct X;                       // declare X as a struct type
struct X* x1;                   // use X in pointer formation
X* x2;                          // use X in pointer formation
```

— *end example*] [*Note:* The rules for declarations and expressions describe in which contexts complete class types are required. A class type `T` must be complete if:

(5.1) — an object of type `T` is defined (3.1), or

(5.2) — a non-static class data member of type `T` is declared (9.2), or

(5.3) — `T` is used as the object type or array element type in a *new-expression* (5.3.4), or

(5.4) — an lvalue-to-rvalue conversion is applied to a glvalue referring to an object of type `T` (4.1), or

(5.5) — an expression is converted (either implicitly or explicitly) to type `T` (Clause 4, 5.2.3, 5.2.7, 5.2.9, 5.4), or

(5.6) — an expression that is not a null pointer constant, and has type other than *cv* `void*`, is converted to the type pointer to `T` or reference to `T` using a standard conversion (Clause 4), a `dynamic_cast` (5.2.7) or a `static_cast` (5.2.9), or

(5.7) — a class member access operator is applied to an expression of type `T` (5.2.5), or

(5.8) — the `typeid` operator (5.2.8) or the `sizeof` operator (5.3.3) is applied to an operand of type `T`, or

26) An implementation is not required to call allocation and deallocation functions from constructors or destructors; however, this is a permissible implementation technique.

(5.9) — a function with a return type or argument type of type `T` is defined (3.1) or called (5.2.2), or

(5.10) — a class with a base class of type `T` is defined (Clause 10), or

(5.11) — an lvalue of type `T` is assigned to (5.18), or

(5.12) — the type `T` is the subject of an `alignof` expression (5.3.6), or

(5.13) — an *exception-declaration* has type `T`, reference to `T`, or pointer to `T` (15.3).

— *end note*]

6 There can be more than one definition of a class type (Clause 9), enumeration type (7.2), inline function with external linkage (7.1.2), class template (Clause 14), non-static function template (14.5.6), static data member of a class template (14.5.1.3), member function of a class template (14.5.1.1), or template specialization for which some template parameters are not specified (14.7, 14.5.5) in a program provided that each definition appears in a different translation unit, and provided the definitions satisfy the following requirements. Given such an entity named `D` defined in more than one translation unit, then

(6.1) — each definition of `D` shall consist of the same sequence of tokens; and

(6.2) — in each definition of `D`, corresponding names, looked up according to 3.4, shall refer to an entity defined within the definition of `D`, or shall refer to the same entity, after overload resolution (13.3) and after matching of partial template specialization (14.8.3), except that a name can refer to a non-volatile `const` object with internal or no linkage if the object has the same literal type in all definitions of `D`, and the object is initialized with a constant expression (5.20), and the object is not odr-used, and the object has the same value in all definitions of `D`; and

(6.3) — in each definition of `D`, corresponding entities shall have the same language linkage; and

(6.4) — in each definition of `D`, the overloaded operators referred to, the implicit calls to conversion functions, constructors, operator new functions and operator delete functions, shall refer to the same function, or to a function defined within the definition of `D`; and

(6.5) — in each definition of `D`, a default argument used by an (implicit or explicit) function call is treated as if its token sequence were present in the definition of `D`; that is, the default argument is subject to the three requirements described above (and, if the default argument has sub-expressions with default arguments, this requirement applies recursively).[27]

(6.6) — if `D` is a class with an implicitly-declared constructor (12.1), it is as if the constructor was implicitly defined in every translation unit where it is odr-used, and the implicit definition in every translation unit shall call the same constructor for a base class or a class member of `D`. [*Example:*

```
//translation unit 1:
struct X {
  X(int);
  X(int, int);
};
X::X(int = 0) { }
class D: public X { };
D d2;                           // X(int) called by D()

//translation unit 2:
struct X {
  X(int);
```

27) 8.3.6 describes how default argument names are looked up.

```
  X(int, int);
};
X::X(int = 0, int = 0) { }
class D: public X { };          // X(int, int) called by D();
                                // D()'s implicit definition
                                // violates the ODR
```

— *end example*]

If D is a template and is defined in more than one translation unit, then the preceding requirements shall apply both to names from the template's enclosing scope used in the template definition (14.6.3), and also to dependent names at the point of instantiation (14.6.2). If the definitions of D satisfy all these requirements, then the behavior is as if there were a single definition of D. If the definitions of D do not satisfy these requirements, then the behavior is undefined.

3.3 Scope [basic.scope]

3.3.1 Declarative regions and scopes [basic.scope.declarative]

1 Every name is introduced in some portion of program text called a *declarative region*, which is the largest part of the program in which that name is *valid*, that is, in which that name may be used as an unqualified name to refer to the same entity. In general, each particular name is valid only within some possibly discontiguous portion of program text called its *scope*. To determine the scope of a declaration, it is sometimes convenient to refer to the *potential scope* of a declaration. The scope of a declaration is the same as its potential scope unless the potential scope contains another declaration of the same name. In that case, the potential scope of the declaration in the inner (contained) declarative region is excluded from the scope of the declaration in the outer (containing) declarative region.

2 [*Example:* in

```
int j = 24;
int main() {
  int i = j, j;
  j = 42;
}
```

the identifier j is declared twice as a name (and used twice). The declarative region of the first j includes the entire example. The potential scope of the first j begins immediately after that j and extends to the end of the program, but its (actual) scope excludes the text between the , and the }. The declarative region of the second declaration of j (the j immediately before the semicolon) includes all the text between { and }, but its potential scope excludes the declaration of i. The scope of the second declaration of j is the same as its potential scope. — *end example*]

3 The names declared by a declaration are introduced into the scope in which the declaration occurs, except that the presence of a **friend** specifier (11.3), certain uses of the *elaborated-type-specifier* (7.1.6.3), and *using-directive*s (7.3.4) alter this general behavior.

4 Given a set of declarations in a single declarative region, each of which specifies the same unqualified name,

(4.1) — they shall all refer to the same entity, or all refer to functions and function templates; or

(4.2) — exactly one declaration shall declare a class name or enumeration name that is not a typedef name and the other declarations shall all refer to the same variable or enumerator, or all refer to functions and function templates; in this case the class name or enumeration name is hidden (3.3.10). [*Note:* A namespace name or a class template name must be unique in its declarative region (7.3.2, Clause 14). — *end note*]

[*Note:* These restrictions apply to the declarative region into which a name is introduced, which is not necessarily the same as the region in which the declaration occurs. In particular, *elaborated-type-specifier*s (7.1.6.3) and friend declarations (11.3) may introduce a (possibly not visible) name into an enclosing namespace; these restrictions apply to that region. Local extern declarations (3.5) may introduce a name into the declarative region where the declaration appears and also introduce a (possibly not visible) name into an enclosing namespace; these restrictions apply to both regions. — *end note*]

5 [*Note:* The name lookup rules are summarized in 3.4. — *end note*]

3.3.2 Point of declaration [basic.scope.pdecl]

1 The *point of declaration* for a name is immediately after its complete declarator (Clause 8) and before its *initializer* (if any), except as noted below. [*Example:*

```
unsigned char x = 12;
{ unsigned char x = x; }
```

Here the second `x` is initialized with its own (indeterminate) value. — *end example*]

2 [*Note:* a name from an outer scope remains visible up to the point of declaration of the name that hides it.[*Example:*

```
const int  i = 2;
{ int  i[i]; }
```

declares a block-scope array of two integers. — *end example*] — *end note*]

3 The point of declaration for a class or class template first declared by a *class-specifier* is immediately after the *identifier* or *simple-template-id* (if any) in its *class-head* (Clause 9). The point of declaration for an enumeration is immediately after the *identifier* (if any) in either its *enum-specifier* (7.2) or its first *opaque-enum-declaration* (7.2), whichever comes first. The point of declaration of an alias or alias template immediately follows the *type-id* to which the alias refers.

4 The point of declaration of a *using-declaration* that does not name a constructor is immediately after the *using-declaration* (7.3.3).

5 The point of declaration for an enumerator is immediately after its *enumerator-definition*.[*Example:*

```
const int x = 12;
{ enum { x = x }; }
```

Here, the enumerator `x` is initialized with the value of the constant `x`, namely 12. — *end example*]

6 After the point of declaration of a class member, the member name can be looked up in the scope of its class. [*Note:* this is true even if the class is an incomplete class. For example,

```
struct X {
  enum E { z = 16 };
  int b[X::z];      // OK
};
```

— *end note*]

7 The point of declaration of a class first declared in an *elaborated-type-specifier* is as follows:

(7.1) — for a declaration of the form

class-key attribute-specifier-seq$_{opt}$ *identifier* ;

the *identifier* is declared to be a *class-name* in the scope that contains the declaration, otherwise

(7.2) — for an *elaborated-type-specifier* of the form

class-key identifier

if the *elaborated-type-specifier* is used in the *decl-specifier-seq* or *parameter-declaration-clause* of a function defined in namespace scope, the *identifier* is declared as a *class-name* in the namespace that contains the declaration; otherwise, except as a friend declaration, the *identifier* is declared in the smallest namespace or block scope that contains the declaration. [*Note:* These rules also apply within templates. — *end note*] [*Note:* Other forms of *elaborated-type-specifier* do not declare a new name, and therefore must refer to an existing *type-name*. See 3.4.4 and 7.1.6.3. — *end note*]

8 The point of declaration for an *injected-class-name* (Clause 9) is immediately following the opening brace of the class definition.

9 The point of declaration for a function-local predefined variable (8.4) is immediately before the *function-body* of a function definition.

10 The point of declaration for a template parameter is immediately after its complete *template-parameter*. [*Example:*

```
typedef unsigned char T;
template<class T
  = T      // lookup finds the typedef name of unsigned char
  , T      // lookup finds the template parameter
    N = 0> struct A { };
```

— *end example*]

11 [*Note:* Friend declarations refer to functions or classes that are members of the nearest enclosing namespace, but they do not introduce new names into that namespace (7.3.1.2). Function declarations at block scope and variable declarations with the `extern` specifier at block scope refer to declarations that are members of an enclosing namespace, but they do not introduce new names into that scope. — *end note*]

12 [*Note:* For point of instantiation of a template, see 14.6.4.1. — *end note*]

3.3.3 Block scope [basic.scope.block]

1 A name declared in a block (6.3) is local to that block; it has *block scope*. Its potential scope begins at its point of declaration (3.3.2) and ends at the end of its block. A variable declared at block scope is a *local variable*.

2 The potential scope of a function parameter name (including one appearing in a *lambda-declarator*) or of a function-local predefined variable in a function definition (8.4) begins at its point of declaration. If the function has a *function-try-block* the potential scope of a parameter or of a function-local predefined variable ends at the end of the last associated handler, otherwise it ends at the end of the outermost block of the function definition. A parameter name shall not be redeclared in the outermost block of the function definition nor in the outermost block of any handler associated with a *function-try-block*.

3 The name declared in an *exception-declaration* is local to the *handler* and shall not be redeclared in the outermost block of the *handler*.

4 Names declared in the *for-init-statement*, the *for-range-declaration*, and in the *condition* of `if`, `while`, `for`, and `switch` statements are local to the `if`, `while`, `for`, or `switch` statement (including the controlled statement), and shall not be redeclared in a subsequent condition of that statement nor in the outermost block (or, for the `if` statement, any of the outermost blocks) of the controlled statement; see 6.4.

3.3.4 Function prototype scope [basic.scope.proto]

1 In a function declaration, or in any function declarator except the declarator of a function definition (8.4), names of parameters (if supplied) have function prototype scope, which terminates at the end of the nearest

enclosing function declarator.

3.3.5 Function scope [basic.funscope]

1 Labels (6.1) have *function scope* and may be used anywhere in the function in which they are declared. Only labels have function scope.

3.3.6 Namespace scope [basic.scope.namespace]

1 The declarative region of a *namespace-definition* is its *namespace-body*. Entities declared in a *namespace-body* are said to be *members* of the namespace, and names introduced by these declarations into the declarative region of the namespace are said to be *member names* of the namespace. A namespace member name has namespace scope. Its potential scope includes its namespace from the name's point of declaration (3.3.2) onwards; and for each *using-directive* (7.3.4) that nominates the member's namespace, the member's potential scope includes that portion of the potential scope of the *using-directive* that follows the member's point of declaration. [*Example:*

```
namespace N {
  int i;
  int g(int a) { return a; }
  int j();
  void q();
}
namespace { int l=1; }
// the potential scope of l is from its point of declaration
// to the end of the translation unit

namespace N {
  int g(char a) {   // overloads N::g(int)
    return l+a;     // l is from unnamed namespace
  }

  int i;            // error: duplicate definition
  int j();          // OK: duplicate function declaration

  int j() {         // OK: definition of N::j()
    return g(i);    // calls N::g(int)
  }
  int q();          // error: different return type
}
```

— *end example*]

2 A namespace member can also be referred to after the `::` scope resolution operator (5.1) applied to the name of its namespace or the name of a namespace which nominates the member's namespace in a *using-directive;* see 3.4.3.2.

3 The outermost declarative region of a translation unit is also a namespace, called the *global namespace*. A name declared in the global namespace has *global namespace scope* (also called *global scope*). The potential scope of such a name begins at its point of declaration (3.3.2) and ends at the end of the translation unit that is its declarative region. A name with global namespace scope is said to be a *global name*.

3.3.7 Class scope [basic.scope.class]

1 The following rules describe the scope of names declared in classes.

1) The potential scope of a name declared in a class consists not only of the declarative region following the name's point of declaration, but also of all function bodies, default arguments, *exception-specification*s,

and *brace-or-equal-initializers* of non-static data members in that class (including such things in nested classes).

2) A name `N` used in a class `S` shall refer to the same declaration in its context and when re-evaluated in the completed scope of `S`. No diagnostic is required for a violation of this rule.

3) A name declared within a member function hides a declaration of the same name whose scope extends to or past the end of the member function's class.

4) The potential scope of a declaration that extends to or past the end of a class definition also extends to the regions defined by its member definitions, even if the members are defined lexically outside the class (this includes static data member definitions, nested class definitions, and member function definitions, including the member function body and any portion of the declarator part of such definitions which follows the *declarator-id*, including a *parameter-declaration-clause* and any default arguments (8.3.6)).[*Example:*

```
typedef int  c;
enum { i = 1 };

class X {
  char  v[i];                       // error: i refers to ::i
                                    // but when reevaluated is X::i
  int  f() { return sizeof(c); }    // OK: X::c
  char  c;
  enum { i = 2 };
};

typedef char*  T;
struct Y {
  T  a;                             // error: T refers to ::T
                                    // but when reevaluated is Y::T
  typedef long  T;
  T  b;
};

typedef int I;
class D {
  typedef I I;                      // error, even though no reordering involved
};
```

— *end example*]

2 The name of a class member shall only be used as follows:

(2.1) — in the scope of its class (as described above) or a class derived (Clause 10) from its class,

(2.2) — after the `.` operator applied to an expression of the type of its class (5.2.5) or a class derived from its class,

(2.3) — after the `->` operator applied to a pointer to an object of its class (5.2.5) or a class derived from its class,

(2.4) — after the `::` scope resolution operator (5.1) applied to the name of its class or a class derived from its class.

3.3.8 Enumeration scope [basic.scope.enum]

1 The name of a scoped enumerator (7.2) has *enumeration scope*. Its potential scope begins at its point of declaration and terminates at the end of the *enum-specifier*.

3.3.9 Template parameter scope [basic.scope.temp]

1 The declarative region of the name of a template parameter of a template *template-parameter* is the smallest *template-parameter-list* in which the name was introduced.

2 The declarative region of the name of a template parameter of a template is the smallest *template-declaration* in which the name was introduced. Only template parameter names belong to this declarative region; any other kind of name introduced by the *declaration* of a *template-declaration* is instead introduced into the same declarative region where it would be introduced as a result of a non-template declaration of the same name. [*Example:*

```
namespace N {
  template<class T> struct A { };               // #1
  template<class U> void f(U) { }               // #2
  struct B {
    template<class V> friend int g(struct C*);  // #3
  };
}
```

The declarative regions of `T`, `U` and `V` are the *template-declaration*s on lines `#1`, `#2` and `#3`, respectively. But the names `A`, `f`, `g` and `C` all belong to the same declarative region — namely, the *namespace-body* of `N`. (`g` is still considered to belong to this declarative region in spite of its being hidden during qualified and unqualified name lookup.) — *end example*]

3 The potential scope of a template parameter name begins at its point of declaration (3.3.2) and ends at the end of its declarative region. [*Note:* This implies that a *template-parameter* can be used in the declaration of subsequent *template-parameter*s and their default arguments but cannot be used in preceding *template-parameter*s or their default arguments. For example,

```
template<class T, T* p, class U = T> class X { /* ... */ };
template<class T> void f(T* p = new T);
```

This also implies that a *template-parameter* can be used in the specification of base classes. For example,

```
template<class T> class X : public Array<T> { /* ... */ };
template<class T> class Y : public T { /* ... */ };
```

The use of a template parameter as a base class implies that a class used as a template argument must be defined and not just declared when the class template is instantiated. — *end note*]

4 The declarative region of the name of a template parameter is nested within the immediately-enclosing declarative region. [*Note:* As a result, a *template-parameter* hides any entity with the same name in an enclosing scope (3.3.10). [*Example:*

```
typedef int N;
template<N X, typename N, template<N Y> class T> struct A;
```

Here, `X` is a non-type template parameter of type `int` and `Y` is a non-type template parameter of the same type as the second template parameter of `A`. — *end example*] — *end note*]

5 [*Note:* Because the name of a template parameter cannot be redeclared within its potential scope (14.6.1), a template parameter's scope is often its potential scope. However, it is still possible for a template parameter name to be hidden; see 14.6.1. — *end note*]

3.3.10 Name hiding [basic.scope.hiding]

1 A name can be hidden by an explicit declaration of that same name in a nested declarative region or derived class (10.2).

2 A class name (9.1) or enumeration name (7.2) can be hidden by the name of a variable, data member, function, or enumerator declared in the same scope. If a class or enumeration name and a variable, data member, function, or enumerator are declared in the same scope (in any order) with the same name, the class or enumeration name is hidden wherever the variable, data member, function, or enumerator name is visible.

3 In a member function definition, the declaration of a name at block scope hides the declaration of a member of the class with the same name; see 3.3.7. The declaration of a member in a derived class (Clause 10) hides the declaration of a member of a base class of the same name; see 10.2.

4 During the lookup of a name qualified by a namespace name, declarations that would otherwise be made visible by a *using-directive* can be hidden by declarations with the same name in the namespace containing the *using-directive;* see (3.4.3.2).

5 If a name is in scope and is not hidden it is said to be *visible*.

3.4 Name lookup [basic.lookup]

1 The name lookup rules apply uniformly to all names (including *typedef-name*s (7.1.3), *namespace-name*s (7.3), and *class-name*s (9.1)) wherever the grammar allows such names in the context discussed by a particular rule. Name lookup associates the use of a name with a declaration (3.1) of that name. Name lookup shall find an unambiguous declaration for the name (see 10.2). Name lookup may associate more than one declaration with a name if it finds the name to be a function name; the declarations are said to form a set of overloaded functions (13.1). Overload resolution (13.3) takes place after name lookup has succeeded. The access rules (Clause 11) are considered only once name lookup and function overload resolution (if applicable) have succeeded. Only after name lookup, function overload resolution (if applicable) and access checking have succeeded are the attributes introduced by the name's declaration used further in expression processing (Clause 5).

2 A name "looked up in the context of an expression" is looked up as an unqualified name in the scope where the expression is found.

3 The injected-class-name of a class (Clause 9) is also considered to be a member of that class for the purposes of name hiding and lookup.

4 [*Note:* 3.5 discusses linkage issues. The notions of scope, point of declaration and name hiding are discussed in 3.3. — *end note*]

3.4.1 Unqualified name lookup [basic.lookup.unqual]

1 In all the cases listed in 3.4.1, the scopes are searched for a declaration in the order listed in each of the respective categories; name lookup ends as soon as a declaration is found for the name. If no declaration is found, the program is ill-formed.

2 The declarations from the namespace nominated by a *using-directive* become visible in a namespace enclosing the *using-directive*; see 7.3.4. For the purpose of the unqualified name lookup rules described in 3.4.1, the declarations from the namespace nominated by the *using-directive* are considered members of that enclosing namespace.

3 The lookup for an unqualified name used as the *postfix-expression* of a function call is described in 3.4.2. [*Note:* For purposes of determining (during parsing) whether an expression is a *postfix-expression* for a function call, the usual name lookup rules apply. The rules in 3.4.2 have no effect on the syntactic interpretation of an expression. For example,

```
typedef int f;
namespace N {
  struct A {
    friend void f(A &);
    operator int();
```

```
    void g(A a) {
      int i = f(a);          // f is the typedef, not the friend
                             // function: equivalent to int(a)
    }
  };
}
```

Because the expression is not a function call, the argument-dependent name lookup (3.4.2) does not apply and the friend function `f` is not found. — *end note*]

4 A name used in global scope, outside of any function, class or user-declared namespace, shall be declared before its use in global scope.

5 A name used in a user-declared namespace outside of the definition of any function or class shall be declared before its use in that namespace or before its use in a namespace enclosing its namespace.

6 A name used in the definition of a function following the function's *declarator-id*[28] that is a member of namespace `N` (where, only for the purpose of exposition, `N` could represent the global scope) shall be declared before its use in the block in which it is used or in one of its enclosing blocks (6.3) or, shall be declared before its use in namespace `N` or, if `N` is a nested namespace, shall be declared before its use in one of `N`'s enclosing namespaces. [*Example:*

```
namespace A {
  namespace N {
    void f();
  }
}
void A::N::f() {
  i = 5;
  // The following scopes are searched for a declaration of i:
  // 1) outermost block scope of A::N::f, before the use of i
  // 2) scope of namespace N
  // 3) scope of namespace A
  // 4) global scope, before the definition of A::N::f
}
```

— *end example*]

7 A name used in the definition of a class `X` outside of a member function body, default argument, *exception-specification*, *brace-or-equal-initializer* of a non-static data member, or nested class definition[29] shall be declared in one of the following ways:

(7.1) — before its use in class `X` or be a member of a base class of `X` (10.2), or

(7.2) — if `X` is a nested class of class `Y` (9.7), before the definition of `X` in `Y`, or shall be a member of a base class of `Y` (this lookup applies in turn to `Y` 's enclosing classes, starting with the innermost enclosing class),[30] or

(7.3) — if `X` is a local class (9.8) or is a nested class of a local class, before the definition of class `X` in a block enclosing the definition of class `X`, or

28) This refers to unqualified names that occur, for instance, in a type or default argument in the *parameter-declaration-clause* or used in the function body.

29) This refers to unqualified names following the class name; such a name may be used in the *base-clause* or may be used in the class definition.

30) This lookup applies whether the definition of `X` is nested within `Y`'s definition or whether `X`'s definition appears in a namespace scope enclosing `Y` 's definition (9.7).

(7.4) — if `X` is a member of namespace `N`, or is a nested class of a class that is a member of `N`, or is a local class or a nested class within a local class of a function that is a member of `N`, before the definition of class `X` in namespace `N` or in one of `N` 's enclosing namespaces.

[*Example:*

```
namespace M {
  class B { };
}

namespace N {
  class Y : public M::B {
    class X {
      int a[i];
    };
  };
}

// The following scopes are searched for a declaration of i:
// 1) scope of class N::Y::X, before the use of i
// 2) scope of class N::Y, before the definition of N::Y::X
// 3) scope of N::Y's base class M::B
// 4) scope of namespace N, before the definition of N::Y
// 5) global scope, before the definition of N
```

— *end example*] [*Note:* When looking for a prior declaration of a class or function introduced by a `friend` declaration, scopes outside of the innermost enclosing namespace scope are not considered; see 7.3.1.2. — *end note*] [*Note:* 3.3.7 further describes the restrictions on the use of names in a class definition. 9.7 further describes the restrictions on the use of names in nested class definitions. 9.8 further describes the restrictions on the use of names in local class definitions. — *end note*]

8 For the members of a class `X`, a name used in a member function body, in a default argument, in an *exception-specification*, in the *brace-or-equal-initializer* of a non-static data member (9.2), or in the definition of a class member outside of the definition of `X`, following the member's *declarator-id*[31], shall be declared in one of the following ways:

(8.1) — before its use in the block in which it is used or in an enclosing block (6.3), or

(8.2) — shall be a member of class `X` or be a member of a base class of `X` (10.2), or

(8.3) — if `X` is a nested class of class `Y` (9.7), shall be a member of `Y`, or shall be a member of a base class of `Y` (this lookup applies in turn to `Y`'s enclosing classes, starting with the innermost enclosing class),[32] or

(8.4) — if `X` is a local class (9.8) or is a nested class of a local class, before the definition of class `X` in a block enclosing the definition of class `X`, or

(8.5) — if `X` is a member of namespace `N`, or is a nested class of a class that is a member of `N`, or is a local class or a nested class within a local class of a function that is a member of `N`, before the use of the name, in namespace `N` or in one of `N` 's enclosing namespaces.

[*Example:*

31) That is, an unqualified name that occurs, for instance, in a type in the *parameter-declaration-clause* or in the *exception-specification*.

32) This lookup applies whether the member function is defined within the definition of class `X` or whether the member function is defined in a namespace scope enclosing `X`'s definition.

```
class B { };
namespace M {
  namespace N {
    class X : public B {
      void f();
    };
  }
}
void M::N::X::f() {
  i = 16;
}

// The following scopes are searched for a declaration of i:
// 1) outermost block scope of M::N::X::f, before the use of i
// 2) scope of class M::N::X
// 3) scope of M::N::X's base class B
// 4) scope of namespace M::N
// 5) scope of namespace M
// 6) global scope, before the definition of M::N::X::f
```

— *end example*] [*Note:* 9.3 and 9.4 further describe the restrictions on the use of names in member function definitions. 9.7 further describes the restrictions on the use of names in the scope of nested classes. 9.8 further describes the restrictions on the use of names in local class definitions. — *end note*]

9 Name lookup for a name used in the definition of a `friend` function (11.3) defined inline in the class granting friendship shall proceed as described for lookup in member function definitions. If the `friend` function is not defined in the class granting friendship, name lookup in the `friend` function definition shall proceed as described for lookup in namespace member function definitions.

10 In a `friend` declaration naming a member function, a name used in the function declarator and not part of a *template-argument* in the *declarator-id* is first looked up in the scope of the member function's class (10.2). If it is not found, or if the name is part of a *template-argument* in the *declarator-id*, the look up is as described for unqualified names in the definition of the class granting friendship. [*Example:*

```
struct A {
  typedef int AT;
  void f1(AT);
  void f2(float);
  template <class T> void f3();
};
struct B {
  typedef char AT;
  typedef float BT;
  friend void A::f1(AT);       // parameter type is A::AT
  friend void A::f2(BT);       // parameter type is B::BT
  friend void A::f3<AT>();     // template argument is B::AT
};
```

— *end example*]

11 During the lookup for a name used as a default argument (8.3.6) in a function *parameter-declaration-clause* or used in the *expression* of a *mem-initializer* for a constructor (12.6.2), the function parameter names are visible and hide the names of entities declared in the block, class or namespace scopes containing the function declaration. [*Note:* 8.3.6 further describes the restrictions on the use of names in default arguments. 12.6.2 further describes the restrictions on the use of names in a *ctor-initializer*. — *end note*]

12 During the lookup of a name used in the *constant-expression* of an *enumerator-definition*, previously declared

*enumerator*s of the enumeration are visible and hide the names of entities declared in the block, class, or namespace scopes containing the *enum-specifier*.

13 A name used in the definition of a `static` data member of class `X` (9.4.2) (after the *qualified-id* of the static member) is looked up as if the name was used in a member function of `X`. [*Note:* 9.4.2 further describes the restrictions on the use of names in the definition of a `static` data member. — *end note*]

14 If a variable member of a namespace is defined outside of the scope of its namespace then any name that appears in the definition of the member (after the *declarator-id*) is looked up as if the definition of the member occurred in its namespace. [*Example:*

```
namespace N {
  int i = 4;
  extern int j;
}

int i = 2;

int N::j = i;                   // N::j == 4
```

— *end example*]

15 A name used in the handler for a *function-try-block* (Clause 15) is looked up as if the name was used in the outermost block of the function definition. In particular, the function parameter names shall not be redeclared in the *exception-declaration* nor in the outermost block of a handler for the *function-try-block*. Names declared in the outermost block of the function definition are not found when looked up in the scope of a handler for the *function-try-block*. [*Note:* But function parameter names are found. — *end note*]

16 [*Note:* The rules for name lookup in template definitions are described in 14.6. — *end note*]

3.4.2 Argument-dependent name lookup [basic.lookup.argdep]

1 When the *postfix-expression* in a function call (5.2.2) is an *unqualified-id*, other namespaces not considered during the usual unqualified lookup (3.4.1) may be searched, and in those namespaces, namespace-scope friend function or function template declarations (11.3) not otherwise visible may be found. These modifications to the search depend on the types of the arguments (and for template template arguments, the namespace of the template argument). [*Example:*

```
namespace N {
  struct S { };
  void f(S);
}

void g() {
  N::S s;
  f(s);                         // OK: calls N::f
  (f)(s);                       // error: N::f not considered; parentheses
                                // prevent argument-dependent lookup
}
```

— *end example*]

2 For each argument type `T` in the function call, there is a set of zero or more associated namespaces and a set of zero or more associated classes to be considered. The sets of namespaces and classes is determined entirely by the types of the function arguments (and the namespace of any template template argument). Typedef names and *using-declaration*s used to specify the types do not contribute to this set. The sets of namespaces and classes are determined in the following way:

(2.1) — If `T` is a fundamental type, its associated sets of namespaces and classes are both empty.

(2.2) — If `T` is a class type (including unions), its associated classes are: the class itself; the class of which it is a member, if any; and its direct and indirect base classes. Its associated namespaces are the innermost enclosing namespaces of its associated classes. Furthermore, if `T` is a class template specialization, its associated namespaces and classes also include: the namespaces and classes associated with the types of the template arguments provided for template type parameters (excluding template template parameters); the namespaces of which any template template arguments are members; and the classes of which any member templates used as template template arguments are members. [*Note:* Non-type template arguments do not contribute to the set of associated namespaces. — *end note*]

(2.3) — If `T` is an enumeration type, its associated namespace is the innermost enclosing namespace of its declaration. If it is a class member, its associated class is the member's class; else it has no associated class.

(2.4) — If `T` is a pointer to `U` or an array of `U`, its associated namespaces and classes are those associated with `U`.

(2.5) — If `T` is a function type, its associated namespaces and classes are those associated with the function parameter types and those associated with the return type.

(2.6) — If `T` is a pointer to a member function of a class `X`, its associated namespaces and classes are those associated with the function parameter types and return type, together with those associated with `X`.

(2.7) — If `T` is a pointer to a data member of class `X`, its associated namespaces and classes are those associated with the member type together with those associated with `X`.

If an associated namespace is an inline namespace (7.3.1), its enclosing namespace is also included in the set. If an associated namespace directly contains inline namespaces, those inline namespaces are also included in the set. In addition, if the argument is the name or address of a set of overloaded functions and/or function templates, its associated classes and namespaces are the union of those associated with each of the members of the set, i.e., the classes and namespaces associated with its parameter types and return type. Additionally, if the aforementioned set of overloaded functions is named with a *template-id*, its associated classes and namespaces also include those of its type *template-argument*s and its template *template-argument*s.

3 Let X be the lookup set produced by unqualified lookup (3.4.1) and let Y be the lookup set produced by argument dependent lookup (defined as follows). If X contains

(3.1) — a declaration of a class member, or

(3.2) — a block-scope function declaration that is not a *using-declaration*, or

(3.3) — a declaration that is neither a function or a function template

then Y is empty. Otherwise Y is the set of declarations found in the namespaces associated with the argument types as described below. The set of declarations found by the lookup of the name is the union of X and Y. [*Note:* The namespaces and classes associated with the argument types can include namespaces and classes already considered by the ordinary unqualified lookup. — *end note*] [*Example:*

```
namespace NS {
  class T { };
  void f(T);
  void g(T, int);
}
NS::T parm;
void g(NS::T, float);
int main() {
  f(parm);                      // OK: calls NS::f
```

```
  extern void g(NS::T, float);
  g(parm, 1);                   // OK: calls g(NS::T, float)
}
```

— *end example*]

4 When considering an associated namespace, the lookup is the same as the lookup performed when the associated namespace is used as a qualifier (3.4.3.2) except that:

(4.1) — Any *using-directive*s in the associated namespace are ignored.

(4.2) — Any namespace-scope friend functions or friend function templates declared in associated classes are visible within their respective namespaces even if they are not visible during an ordinary lookup (11.3).

(4.3) — All names except those of (possibly overloaded) functions and function templates are ignored.

3.4.3 Qualified name lookup [basic.lookup.qual]

1 The name of a class or namespace member or enumerator can be referred to after the `::` scope resolution operator (5.1) applied to a *nested-name-specifier* that denotes its class, namespace, or enumeration. If a `::` scope resolution operator in a *nested-name-specifier* is not preceded by a *decltype-specifier*, lookup of the name preceding that `::` considers only namespaces, types, and templates whose specializations are types. If the name found does not designate a namespace or a class, enumeration, or dependent type, the program is ill-formed.[*Example:*

```
class A {
public:
  static int n;
};
int main() {
  int A;
  A::n = 42;        // OK
  A b;              // ill-formed: A does not name a type
}
```

— *end example*]

2 [*Note:* Multiply qualified names, such as `N1::N2::N3::n`, can be used to refer to members of nested classes (9.7) or members of nested namespaces. — *end note*]

3 In a declaration in which the *declarator-id* is a *qualified-id*, names used before the *qualified-id* being declared are looked up in the defining namespace scope; names following the *qualified-id* are looked up in the scope of the member's class or namespace. [*Example:*

```
class X { };
class C {
  class X { };
  static const int number = 50;
  static X arr[number];
};
X C::arr[number];   // ill-formed:
                    // equivalent to: ::X C::arr[C::number];
                    // not to: C::X C::arr[C::number];
```

— *end example*]

4 A name prefixed by the unary scope operator `::` (5.1) is looked up in global scope, in the translation unit where it is used. The name shall be declared in global namespace scope or shall be a name whose declaration

is visible in global scope because of a *using-directive* (3.4.3.2). The use of `::` allows a global name to be referred to even if its identifier has been hidden (3.3.10).

5 A name prefixed by a *nested-name-specifier* that nominates an enumeration type shall represent an *enumerator* of that enumeration.

6 If a *pseudo-destructor-name* (5.2.4) contains a *nested-name-specifier*, the *type-name*s are looked up as types in the scope designated by the *nested-name-specifier*. Similarly, in a *qualified-id* of the form:

nested-name-specifier$_{opt}$ *class-name* `::` `~` *class-name*

the second *class-name* is looked up in the same scope as the first. [*Example:*

```
struct C {
  typedef int I;
};
typedef int I1, I2;
extern int* p;
extern int* q;
p->C::I::~I();       // I is looked up in the scope of C
q->I1::~I2();        // I2 is looked up in the scope of
                     // the postfix-expression

struct A {
  ~A();
};
typedef A AB;
int main() {
  AB* p;
  p->AB::~AB();      // explicitly calls the destructor for A
}
```

— *end example*] [*Note:* 3.4.5 describes how name lookup proceeds after the `.` and `->` operators. — *end note*]

3.4.3.1 Class members [class.qual]

1 If the *nested-name-specifier* of a *qualified-id* nominates a class, the name specified after the *nested-name-specifier* is looked up in the scope of the class (10.2), except for the cases listed below. The name shall represent one or more members of that class or of one of its base classes (Clause 10). [*Note:* A class member can be referred to using a *qualified-id* at any point in its potential scope (3.3.7). — *end note*] The exceptions to the name lookup rule above are the following:

(1.1) — a destructor name is looked up as specified in 3.4.3;

(1.2) — a *conversion-type-id* of a *conversion-function-id* is looked up in the same manner as a *conversion-type-id* in a class member access (see 3.4.5);

(1.3) — the names in a *template-argument* of a *template-id* are looked up in the context in which the entire *postfix-expression* occurs.

(1.4) — the lookup for a name specified in a *using-declaration* (7.3.3) also finds class or enumeration names hidden within the same scope (3.3.10).

2 In a lookup in which function names are not ignored[33] and the *nested-name-specifier* nominates a class `C`:

33) Lookups in which function names are ignored include names appearing in a *nested-name-specifier*, an *elaborated-type-specifier*, or a *base-specifier*.

(2.1) — if the name specified after the *nested-name-specifier*, when looked up in `C`, is the injected-class-name of `C` (Clause 9), or

(2.2) — in a *using-declaration* (7.3.3) that is a *member-declaration*, if the name specified after the *nested-name-specifier* is the same as the *identifier* or the *simple-template-id*'s *template-name* in the last component of the *nested-name-specifier*,

the name is instead considered to name the constructor of class `C`. [*Note:* For example, the constructor is not an acceptable lookup result in an *elaborated-type-specifier* so the constructor would not be used in place of the injected-class-name. — *end note*] Such a constructor name shall be used only in the *declarator-id* of a declaration that names a constructor or in a *using-declaration*. [*Example:*

```
struct A { A(); };
struct B: public A { B(); };

A::A() { }
B::B() { }

B::A ba;            // object of type A
A::A a;             // error, A::A is not a type name
struct A::A a2;     // object of type A
```

— *end example*]

3 A class member name hidden by a name in a nested declarative region or by the name of a derived class member can still be found if qualified by the name of its class followed by the `::` operator.

3.4.3.2 Namespace members [namespace.qual]

1 If the *nested-name-specifier* of a *qualified-id* nominates a namespace (including the case where the *nested-name-specifier* is `::`, i.e., nominating the global namespace), the name specified after the *nested-name-specifier* is looked up in the scope of the namespace. The names in a *template-argument* of a *template-id* are looked up in the context in which the entire *postfix-expression* occurs.

2 For a namespace `X` and name `m`, the namespace-qualified lookup set $S(X, m)$ is defined as follows: Let $S'(X, m)$ be the set of all declarations of `m` in `X` and the inline namespace set of `X` (7.3.1). If $S'(X, m)$ is not empty, $S(X, m)$ is $S'(X, m)$; otherwise, $S(X, m)$ is the union of $S(N_i, m)$ for all namespaces N_i nominated by *using-directives* in `X` and its inline namespace set.

3 Given `X::m` (where `X` is a user-declared namespace), or given `::m` (where X is the global namespace), if $S(X, m)$ is the empty set, the program is ill-formed. Otherwise, if $S(X, m)$ has exactly one member, or if the context of the reference is a *using-declaration* (7.3.3), $S(X, m)$ is the required set of declarations of `m`. Otherwise if the use of `m` is not one that allows a unique declaration to be chosen from $S(X, m)$, the program is ill-formed. [*Example:*

```
int x;
namespace Y {
  void f(float);
  void h(int);
}

namespace Z {
  void h(double);
}

namespace A {
  using namespace Y;
  void f(int);
```

```
  void g(int);
  int i;
}

namespace B {
  using namespace Z;
  void f(char);
  int i;
}

namespace AB {
  using namespace A;
  using namespace B;
  void g();
}

void h()
{
  AB::g();          // g is declared directly in AB,
                    // therefore S is { AB::g() } and AB::g() is chosen
  AB::f(1);         // f is not declared directly in AB so the rules are
                    // applied recursively to A and B;
                    // namespace Y is not searched and Y::f(float)
                    // is not considered;
                    // S is { A::f(int), B::f(char) } and overload
                    // resolution chooses A::f(int)
  AB::f('c');       // as above but resolution chooses B::f(char)

  AB::x++;          // x is not declared directly in AB, and
                    // is not declared in A or B , so the rules are
                    // applied recursively to Y and Z,
                    // S is { } so the program is ill-formed
  AB::i++;          // i is not declared directly in AB so the rules are
                    // applied recursively to A and B,
                    // S is { A::i , B::i } so the use is ambiguous
                    // and the program is ill-formed
  AB::h(16.8);      // h is not declared directly in AB and
                    // not declared directly in A or B so the rules are
                    // applied recursively to Y and Z,
                    // S is { Y::h(int), Z::h(double) } and overload
                    // resolution chooses Z::h(double)
}
```

4 The same declaration found more than once is not an ambiguity (because it is still a unique declaration). For example:

```
namespace A {
  int a;
}

namespace B {
  using namespace A;
}

namespace C {
  using namespace A;
```

```
}

namespace BC {
  using namespace B;
  using namespace C;
}

void f()
{
  BC::a++;          // OK: S is { A::a, A::a }
}

namespace D {
  using A::a;
}

namespace BD {
  using namespace B;
  using namespace D;
}

void g()
{
  BD::a++;          // OK: S is {  A::a,  A::a }
}
```

5 Because each referenced namespace is searched at most once, the following is well-defined:

```
namespace B {
  int b;
}

namespace A {
  using namespace B;
  int a;
}

namespace B {
  using namespace A;
}

void f()
{
  A::a++;           // OK: a declared directly in A, S is {A::a}
  B::a++;           // OK: both A and B searched (once), S is {A::a}
  A::b++;           // OK: both A and B searched (once), S is {B::b}
  B::b++;           // OK: b declared directly in B, S is {B::b}
}
```

— *end example*]

6 During the lookup of a qualified namespace member name, if the lookup finds more than one declaration of the member, and if one declaration introduces a class name or enumeration name and the other declarations either introduce the same variable, the same enumerator or a set of functions, the non-type name hides the class or enumeration name if and only if the declarations are from the same namespace; otherwise (the declarations are from different namespaces), the program is ill-formed. [*Example:*

```
namespace A {
  struct x { };
  int x;
  int y;
}

namespace B {
  struct y { };
}

namespace C {
  using namespace A;
  using namespace B;
  int i = C::x;      // OK, A::x (of type int )
  int j = C::y;      // ambiguous, A::y or B::y
}
```

— *end example*]

7 In a declaration for a namespace member in which the *declarator-id* is a *qualified-id*, given that the *qualified-id* for the namespace member has the form

nested-name-specifier unqualified-id

the *unqualified-id* shall name a member of the namespace designated by the *nested-name-specifier* or of an element of the inline namespace set (7.3.1) of that namespace. [*Example:*

```
namespace A {
  namespace B {
    void f1(int);
  }
  using namespace B;
}
void A::f1(int){ }  // ill-formed, f1 is not a member of A
```

— *end example*] However, in such namespace member declarations, the *nested-name-specifier* may rely on *using-directive*s to implicitly provide the initial part of the *nested-name-specifier*. [*Example:*

```
namespace A {
  namespace B {
    void f1(int);
  }
}

namespace C {
  namespace D {
    void f1(int);
  }
}

using namespace A;
using namespace C::D;
void B::f1(int){ }  // OK, defines A::B::f1(int)
```

— *end example*]

3.4.4 Elaborated type specifiers [basic.lookup.elab]

1 An *elaborated-type-specifier* (7.1.6.3) may be used to refer to a previously declared *class-name* or *enum-name* even though the name has been hidden by a non-type declaration (3.3.10).

2 If the *elaborated-type-specifier* has no *nested-name-specifier*, and unless the *elaborated-type-specifier* appears in a declaration with the following form:

class-key attribute-specifier-seq$_{opt}$ *identifier* ;

the *identifier* is looked up according to 3.4.1 but ignoring any non-type names that have been declared. If the *elaborated-type-specifier* is introduced by the `enum` keyword and this lookup does not find a previously declared *type-name*, the *elaborated-type-specifier* is ill-formed. If the *elaborated-type-specifier* is introduced by the *class-key* and this lookup does not find a previously declared *type-name*, or if the *elaborated-type-specifier* appears in a declaration with the form:

class-key attribute-specifier-seq$_{opt}$ *identifier* ;

the *elaborated-type-specifier* is a declaration that introduces the *class-name* as described in 3.3.2.

3 If the *elaborated-type-specifier* has a *nested-name-specifier*, qualified name lookup is performed, as described in 3.4.3, but ignoring any non-type names that have been declared. If the name lookup does not find a previously declared *type-name*, the *elaborated-type-specifier* is ill-formed. [*Example:*

```
struct Node {
  struct Node* Next;            // OK: Refers to Node at global scope
  struct Data* Data;            // OK: Declares type Data
                                // at global scope and member Data
};

struct Data {
  struct Node* Node;            // OK: Refers to Node at global scope
  friend struct ::Glob;         // error: Glob is not declared
                                // cannot introduce a qualified type (7.1.6.3)
  friend struct Glob;           // OK: Refers to (as yet) undeclared Glob
                                // at global scope.
  /* ... */
};

struct Base {
  struct Data;                  // OK: Declares nested Data
  struct ::Data*     thatData;  // OK: Refers to ::Data
  struct Base::Data* thisData;  // OK: Refers to nested Data
  friend class ::Data;          // OK: global Data is a friend
  friend class Data;            // OK: nested Data is a friend
  struct Data { /* ...  */ };   // Defines nested Data
};

struct Data;                    // OK: Redeclares Data at global scope
struct ::Data;                  // error: cannot introduce a qualified type (7.1.6.3)
struct Base::Data;              // error: cannot introduce a qualified type (7.1.6.3)
struct Base::Datum;             // error: Datum undefined
struct Base::Data* pBase;       // OK: refers to nested Data
```

— *end example*]

3.4.5 Class member access [basic.lookup.classref]

1 In a class member access expression (5.2.5), if the `.` or `->` token is immediately followed by an *identifier* followed by a `<`, the identifier must be looked up to determine whether the `<` is the beginning of a template argument list (14.2) or a less-than operator. The identifier is first looked up in the class of the object expression. If the identifier is not found, it is then looked up in the context of the entire *postfix-expression* and shall name a class template.

2 If the *id-expression* in a class member access (5.2.5) is an *unqualified-id*, and the type of the object expression is of a class type `C`, the *unqualified-id* is looked up in the scope of class `C`. For a pseudo-destructor call (5.2.4), the *unqualified-id* is looked up in the context of the complete *postfix-expression*.

3 If the *unqualified-id* is `~`*type-name*, the *type-name* is looked up in the context of the entire *postfix-expression*. If the type `T` of the object expression is of a class type `C`, the *type-name* is also looked up in the scope of class `C`. At least one of the lookups shall find a name that refers to (possibly cv-qualified) `T`. [*Example:*

```
struct A { };

struct B {
  struct A { };
  void f(::A* a);
};

void B::f(::A* a) {
  a->~A();                    // OK: lookup in *a finds the injected-class-name
}
```

— *end example*]

4 If the *id-expression* in a class member access is a *qualified-id* of the form

```
class-name-or-namespace-name::...
```

the *class-name-or-namespace-name* following the `.` or `->` operator is first looked up in the class of the object expression and the name, if found, is used. Otherwise it is looked up in the context of the entire *postfix-expression*. [*Note:* See 3.4.3, which describes the lookup of a name before `::`, which will only find a type or namespace name. — *end note*]

5 If the *qualified-id* has the form

```
::class-name-or-namespace-name::...
```

the *class-name-or-namespace-name* is looked up in global scope as a *class-name* or *namespace-name*.

6 If the *nested-name-specifier* contains a *simple-template-id* (14.2), the names in its *template-argument*s are looked up in the context in which the entire *postfix-expression* occurs.

7 If the *id-expression* is a *conversion-function-id*, its *conversion-type-id* is first looked up in the class of the object expression and the name, if found, is used. Otherwise it is looked up in the context of the entire *postfix-expression*. In each of these lookups, only names that denote types or templates whose specializations are types are considered. [*Example:*

```
struct A { };
namespace N {
  struct A {
    void g() { }
    template <class T> operator T();
  };
}
```

```
int main() {
  N::A a;
  a.operator A();               // calls N::A::operator N::A
}
```

— *end example*]

3.4.6 Using-directives and namespace aliases [basic.lookup.udir]

1 In a *using-directive* or *namespace-alias-definition*, during the lookup for a *namespace-name* or for a name in a *nested-name-specifier* only namespace names are considered.

3.5 Program and linkage [basic.link]

1 A *program* consists of one or more *translation units* (Clause 2) linked together. A translation unit consists of a sequence of declarations.

translation-unit:
declaration-seq$_{opt}$

2 A name is said to have *linkage* when it might denote the same object, reference, function, type, template, namespace or value as a name introduced by a declaration in another scope:

(2.1) — When a name has *external linkage* , the entity it denotes can be referred to by names from scopes of other translation units or from other scopes of the same translation unit.

(2.2) — When a name has *internal linkage* , the entity it denotes can be referred to by names from other scopes in the same translation unit.

(2.3) — When a name has *no linkage* , the entity it denotes cannot be referred to by names from other scopes.

3 A name having namespace scope (3.3.6) has internal linkage if it is the name of

(3.1) — a variable, function or function template that is explicitly declared `static`; or,

(3.2) — a variable of non-volatile const-qualified type that is neither explicitly declared `extern` nor previously declared to have external linkage; or

(3.3) — a data member of an anonymous union.

4 An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal linkage. All other namespaces have external linkage. A name having namespace scope that has not been given internal linkage above has the same linkage as the enclosing namespace if it is the name of

(4.1) — a variable; or

(4.2) — a function; or

(4.3) — a named class (Clause 9), or an unnamed class defined in a typedef declaration in which the class has the typedef name for linkage purposes (7.1.3); or

(4.4) — a named enumeration (7.2), or an unnamed enumeration defined in a typedef declaration in which the enumeration has the typedef name for linkage purposes (7.1.3); or

(4.5) — an enumerator belonging to an enumeration with linkage; or

(4.6) — a template.

5 In addition, a member function, static data member, a named class or enumeration of class scope, or an unnamed class or enumeration defined in a class-scope typedef declaration such that the class or enumeration has the typedef name for linkage purposes (7.1.3), has the same linkage, if any, as the name of the class of which it is a member.

6 The name of a function declared in block scope and the name of a variable declared by a block scope `extern` declaration have linkage. If there is a visible declaration of an entity with linkage having the same name and type, ignoring entities declared outside the innermost enclosing namespace scope, the block scope declaration declares that same entity and receives the linkage of the previous declaration. If there is more than one such matching entity, the program is ill-formed. Otherwise, if no matching entity is found, the block scope entity receives external linkage.[*Example:*

```
static void f();
static int i = 0;               // #1
void g() {
  extern void f();              // internal linkage
  int i;                        // #2 i has no linkage
  {
    extern void f();            // internal linkage
    extern int i;               // #3 external linkage
  }
}
```

There are three objects named `i` in this program. The object with internal linkage introduced by the declaration in global scope (line `#1`), the object with automatic storage duration and no linkage introduced by the declaration on line `#2`, and the object with static storage duration and external linkage introduced by the declaration on line `#3`. — *end example*]

7 When a block scope declaration of an entity with linkage is not found to refer to some other declaration, then that entity is a member of the innermost enclosing namespace. However such a declaration does not introduce the member name in its namespace scope. [*Example:*

```
namespace X {
  void p() {
    q();                        // error: q not yet declared
    extern void q();            // q is a member of namespace X
  }

  void middle() {
    q();                        // error: q not yet declared
  }

  void q() { /* ... */ }        // definition of X::q
}

void q() { /* ... */ }          // some other, unrelated q
```

— *end example*]

8 Names not covered by these rules have no linkage. Moreover, except as noted, a name declared at block scope (3.3.3) has no linkage. A type is said to have linkage if and only if:

(8.1) — it is a class or enumeration type that is named (or has a name for linkage purposes (7.1.3)) and the name has linkage; or

(8.2) — it is an unnamed class or enumeration member of a class with linkage; or

(8.3) — it is a specialization of a class template (Clause 14)[34]; or

(8.4) — it is a fundamental type (3.9.1); or

(8.5) — it is a compound type (3.9.2) other than a class or enumeration, compounded exclusively from types that have linkage; or

(8.6) — it is a cv-qualified (3.9.3) version of a type that has linkage.

A type without linkage shall not be used as the type of a variable or function with external linkage unless

(8.7) — the entity has C language linkage (7.5), or

(8.8) — the entity is declared within an unnamed namespace (7.3.1), or

(8.9) — the entity is not odr-used (3.2) or is defined in the same translation unit.

[*Note:* In other words, a type without linkage contains a class or enumeration that cannot be named outside its translation unit. An entity with external linkage declared using such a type could not correspond to any other entity in another translation unit of the program and thus must be defined in the translation unit if it is odr-used. Also note that classes with linkage may contain members whose types do not have linkage, and that typedef names are ignored in the determination of whether a type has linkage. — *end note*]

[*Example:*

```
template <class T> struct B {
  void g(T) { }
  void h(T);
  friend void i(B, T) { }
};

void f() {
  struct A { int x; };  // no linkage
  A a = { 1 };
  B<A> ba;              // declares B<A>::g(A) and B<A>::h(A)
  ba.g(a);              // OK
  ba.h(a);              // error: B<A>::h(A) not defined in the translation unit
  i(ba, a);             // OK
}
```

— *end example*]

9 Two names that are the same (Clause 3) and that are declared in different scopes shall denote the same variable, function, type, enumerator, template or namespace if

(9.1) — both names have external linkage or else both names have internal linkage and are declared in the same translation unit; and

(9.2) — both names refer to members of the same namespace or to members, not by inheritance, of the same class; and

(9.3) — when both names denote functions, the parameter-type-lists of the functions (8.3.5) are identical; and

(9.4) — when both names denote function templates, the signatures (14.5.6.1) are the same.

34) A class template has the linkage of the innermost enclosing class or namespace in which it is declared.

10 After all adjustments of types (during which typedefs (7.1.3) are replaced by their definitions), the types specified by all declarations referring to a given variable or function shall be identical, except that declarations for an array object can specify array types that differ by the presence or absence of a major array bound (8.3.4). A violation of this rule on type identity does not require a diagnostic.

11 [*Note:* Linkage to non-C++ declarations can be achieved using a *linkage-specification* (7.5). — *end note*]

3.6 Start and termination [basic.start]

3.6.1 Main function [basic.start.main]

1 A program shall contain a global function called `main`, which is the designated start of the program. It is implementation-defined whether a program in a freestanding environment is required to define a `main` function. [*Note:* In a freestanding environment, start-up and termination is implementation-defined; start-up contains the execution of constructors for objects of namespace scope with static storage duration; termination contains the execution of destructors for objects with static storage duration. — *end note*]

2 An implementation shall not predefine the `main` function. This function shall not be overloaded. Its type shall have C++ language linkage and it shall have a declared return type of type `int`, but otherwise its type is implementation-defined. An implementation shall allow both

(2.1) — a function of `()` returning `int` and

(2.2) — a function of `(int`, pointer to pointer to `char)` returning `int`

as the type of `main` (8.3.5). In the latter form, for purposes of exposition, the first function parameter is called `argc` and the second function parameter is called `argv`, where `argc` shall be the number of arguments passed to the program from the environment in which the program is run. If `argc` is nonzero these arguments shall be supplied in `argv[0]` through `argv[argc-1]` as pointers to the initial characters of null-terminated multibyte strings (NTMBS s) (17.5.2.1.4.2) and `argv[0]` shall be the pointer to the initial character of a NTMBS that represents the name used to invoke the program or `""`. The value of `argc` shall be non-negative. The value of `argv[argc]` shall be 0. [*Note:* It is recommended that any further (optional) parameters be added after `argv`. — *end note*]

3 The function `main` shall not be used within a program. The linkage (3.5) of `main` is implementation-defined. A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed. The `main` function shall not be declared with a *linkage-specification* (7.5). A program that declares a variable `main` at global scope or that declares the name `main` with C language linkage (in any namespace) is ill-formed. The name `main` is not otherwise reserved. [*Example:* member functions, classes, and enumerations can be called `main`, as can entities in other namespaces. — *end example*]

4 Terminating the program without leaving the current block (e.g., by calling the function `std::exit(int)` (18.5)) does not destroy any objects with automatic storage duration (12.4). If `std::exit` is called to end a program during the destruction of an object with static or thread storage duration, the program has undefined behavior.

5 A return statement in `main` has the effect of leaving the main function (destroying any objects with automatic storage duration) and calling `std::exit` with the return value as the argument. If control reaches the end of `main` without encountering a `return` statement, the effect is that of executing

```
return 0;
```

3.6.2 Initialization of non-local variables [basic.start.init]

1 There are two broad classes of named non-local variables: those with static storage duration (3.7.1) and those with thread storage duration (3.7.2). Non-local variables with static storage duration are initialized as a consequence of program initiation. Non-local variables with thread storage duration are initialized as a consequence of thread execution. Within each of these phases of initiation, initialization occurs as follows.

2 Variables with static storage duration (3.7.1) or thread storage duration (3.7.2) shall be zero-initialized (8.5) before any other initialization takes place. A *constant initializer* for an object `o` is an expression that is a constant expression, except that it may also invoke `constexpr` constructors for `o` and its subobjects even if those objects are of non-literal class types [*Note:* such a class may have a non-trivial destructor — *end note*]. *Constant initialization* is performed:

(2.1) — if each full-expression (including implicit conversions) that appears in the initializer of a reference with static or thread storage duration is a constant expression (5.20) and the reference is bound to a glvalue designating an object with static storage duration, to a temporary object (see 12.2) or subobject thereof, or to a function;

(2.2) — if an object with static or thread storage duration is initialized by a constructor call, and if the initialization full-expression is a constant initializer for the object;

(2.3) — if an object with static or thread storage duration is not initialized by a constructor call and if either the object is value-initialized or every full-expression that appears in its initializer is a constant expression.

Together, zero-initialization and constant initialization are called *static initialization*; all other initialization is *dynamic initialization*. Static initialization shall be performed before any dynamic initialization takes place. Dynamic initialization of a non-local variable with static storage duration is *unordered* if the variable is an implicitly or explicitly instantiated specialization, and otherwise is *ordered* [*Note:* an explicitly specialized static data member or variable template specialization has ordered initialization. — *end note*]. Variables with ordered initialization defined within a single translation unit shall be initialized in the order of their definitions in the translation unit. If a program starts a thread (30.3), the subsequent initialization of a variable is unsequenced with respect to the initialization of a variable defined in a different translation unit. Otherwise, the initialization of a variable is indeterminately sequenced with respect to the initialization of a variable defined in a different translation unit. If a program starts a thread, the subsequent unordered initialization of a variable is unsequenced with respect to every other dynamic initialization. Otherwise, the unordered initialization of a variable is indeterminately sequenced with respect to every other dynamic initialization. [*Note:* This definition permits initialization of a sequence of ordered variables concurrently with another sequence. — *end note*] [*Note:* The initialization of local static variables is described in 6.7. — *end note*]

3 An implementation is permitted to perform the initialization of a non-local variable with static storage duration as a static initialization even if such initialization is not required to be done statically, provided that

(3.1) — the dynamic version of the initialization does not change the value of any other object of namespace scope prior to its initialization, and

(3.2) — the static version of the initialization produces the same value in the initialized variable as would be produced by the dynamic initialization if all variables not required to be initialized statically were initialized dynamically.

[*Note:* As a consequence, if the initialization of an object `obj1` refers to an object `obj2` of namespace scope potentially requiring dynamic initialization and defined later in the same translation unit, it is unspecified whether the value of `obj2` used will be the value of the fully initialized `obj2` (because `obj2` was statically initialized) or will be the value of `obj2` merely zero-initialized. For example,

```
inline double fd() { return 1.0; }
extern double d1;
double d2 = d1;      // unspecified:
                     // may be statically initialized to 0.0 or
                     // dynamically initialized to 0.0 if d1 is
                     // dynamically initialized, or 1.0 otherwise
double d1 = fd();    // may be initialized statically or dynamically to 1.0
```

— *end note*]

4 It is implementation-defined whether the dynamic initialization of a non-local variable with static storage duration is done before the first statement of `main`. If the initialization is deferred to some point in time after the first statement of `main`, it shall occur before the first odr-use (3.2) of any function or variable defined in the same translation unit as the variable to be initialized.[35] [*Example:*

```
// - File 1 -
#include "a.h"
#include "b.h"
B b;
A::A(){
  b.Use();
}

// - File 2 -
#include "a.h"
A a;

// - File 3 -
#include "a.h"
#include "b.h"
extern A a;
extern B b;

int main() {
  a.Use();
  b.Use();
}
```

It is implementation-defined whether either `a` or `b` is initialized before `main` is entered or whether the initializations are delayed until `a` is first odr-used in `main`. In particular, if `a` is initialized before `main` is entered, it is not guaranteed that `b` will be initialized before it is odr-used by the initialization of `a`, that is, before `A::A` is called. If, however, `a` is initialized at some point after the first statement of `main`, `b` will be initialized prior to its use in `A::A`. — *end example*]

5 It is implementation-defined whether the dynamic initialization of a non-local variable with static or thread storage duration is done before the first statement of the initial function of the thread. If the initialization is deferred to some point in time after the first statement of the initial function of the thread, it shall occur before the first odr-use (3.2) of any variable with thread storage duration defined in the same translation unit as the variable to be initialized.

6 If the initialization of a non-local variable with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1).

3.6.3 Termination [basic.start.term]

1 Destructors (12.4) for initialized objects (that is, objects whose lifetime (3.8) has begun) with static storage duration are called as a result of returning from `main` and as a result of calling `std::exit` (18.5). Destructors for initialized objects with thread storage duration within a given thread are called as a result of returning from the initial function of that thread and as a result of that thread calling `std::exit`. The completions of the destructors for all initialized objects with thread storage duration within that thread are sequenced before the initiation of the destructors of any object with static storage duration. If the completion of the constructor or dynamic initialization of an object with thread storage duration is sequenced before that of

35) A non-local variable with static storage duration having initialization with side-effects must be initialized even if it is not odr-used (3.2, 3.7.1).

another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first. If the completion of the constructor or dynamic initialization of an object with static storage duration is sequenced before that of another, the completion of the destructor of the second is sequenced before the initiation of the destructor of the first. [*Note:* This definition permits concurrent destruction. — *end note*] If an object is initialized statically, the object is destroyed in the same order as if the object was dynamically initialized. For an object of array or class type, all subobjects of that object are destroyed before any block-scope object with static storage duration initialized during the construction of the subobjects is destroyed. If the destruction of an object with static or thread storage duration exits via an exception, `std::terminate` is called (15.5.1).

2 If a function contains a block-scope object of static or thread storage duration that has been destroyed and the function is called during the destruction of an object with static or thread storage duration, the program has undefined behavior if the flow of control passes through the definition of the previously destroyed block-scope object. Likewise, the behavior is undefined if the block-scope object is used indirectly (i.e., through a pointer) after its destruction.

3 If the completion of the initialization of an object with static storage duration is sequenced before a call to `std::atexit` (see `<cstdlib>`, 18.5), the call to the function passed to `std::atexit` is sequenced before the call to the destructor for the object. If a call to `std::atexit` is sequenced before the completion of the initialization of an object with static storage duration, the call to the destructor for the object is sequenced before the call to the function passed to `std::atexit`. If a call to `std::atexit` is sequenced before another call to `std::atexit`, the call to the function passed to the second `std::atexit` call is sequenced before the call to the function passed to the first `std::atexit` call.

4 If there is a use of a standard library object or function not permitted within signal handlers (18.10) that does not happen before (1.10) completion of destruction of objects with static storage duration and execution of `std::atexit` registered functions (18.5), the program has undefined behavior. [*Note:* If there is a use of an object with static storage duration that does not happen before the object's destruction, the program has undefined behavior. Terminating every thread before a call to `std::exit` or the exit from `main` is sufficient, but not necessary, to satisfy these requirements. These requirements permit thread managers as static-storage-duration objects. — *end note*]

5 Calling the function `std::abort()` declared in `<cstdlib>` terminates the program without executing any destructors and without calling the functions passed to `std::atexit()` or `std::at_quick_exit()`.

3.7 Storage duration [basic.stc]

1 Storage duration is the property of an object that defines the minimum potential lifetime of the storage containing the object. The storage duration is determined by the construct used to create the object and is one of the following:

(1.1) — static storage duration

(1.2) — thread storage duration

(1.3) — automatic storage duration

(1.4) — dynamic storage duration

2 Static, thread, and automatic storage durations are associated with objects introduced by declarations (3.1) and implicitly created by the implementation (12.2). The dynamic storage duration is associated with objects created with `operator new` (5.3.4).

3 The storage duration categories apply to references as well. The lifetime of a reference is its storage duration.

3.7.1 Static storage duration [basic.stc.static]

1 All variables which do not have dynamic storage duration, do not have thread storage duration, and are not local have *static storage duration*. The storage for these entities shall last for the duration of the program (3.6.2, 3.6.3).

2 If a variable with static storage duration has initialization or a destructor with side effects, it shall not be eliminated even if it appears to be unused, except that a class object or its copy/move may be eliminated as specified in 12.8.

3 The keyword `static` can be used to declare a local variable with static storage duration. [*Note:* 6.7 describes the initialization of local `static` variables; 3.6.3 describes the destruction of local `static` variables. — *end note*]

4 The keyword `static` applied to a class data member in a class definition gives the data member static storage duration.

3.7.2 Thread storage duration [basic.stc.thread]

1 All variables declared with the `thread_local` keyword have *thread storage duration*. The storage for these entities shall last for the duration of the thread in which they are created. There is a distinct object or reference per thread, and use of the declared name refers to the entity associated with the current thread.

2 A variable with thread storage duration shall be initialized before its first odr-use (3.2) and, if constructed, shall be destroyed on thread exit.

3.7.3 Automatic storage duration [basic.stc.auto]

1 Block-scope variables explicitly declared `register` or not explicitly declared `static`, `thread_local`, or `extern` have *automatic storage duration*. The storage for these entities lasts until the block in which they are created exits.

2 [*Note:* These variables are initialized and destroyed as described in 6.7. — *end note*]

3 If a variable with automatic storage duration has initialization or a destructor with side effects, an implementation shall not destroy it before the end of its block nor eliminate it as an optimization, even if it appears to be unused, except that a class object or its copy/move may be eliminated as specified in 12.8.

3.7.4 Dynamic storage duration [basic.stc.dynamic]

1 Objects can be created dynamically during program execution (1.9), using *new-expressions* (5.3.4), and destroyed using *delete-expressions* (5.3.5). A C++ implementation provides access to, and management of, dynamic storage via the global *allocation functions* `operator new` and `operator new[]` and the global *deallocation functions* `operator delete` and `operator delete[]`.

2 The library provides default definitions for the global allocation and deallocation functions. Some global allocation and deallocation functions are replaceable (18.6.1). A C++ program shall provide at most one definition of a replaceable allocation or deallocation function. Any such function definition replaces the default version provided in the library (17.6.4.6). The following allocation and deallocation functions (18.6) are implicitly declared in global scope in each translation unit of a program.

```
void* operator new(std::size_t);
void* operator new[](std::size_t);
void operator delete(void*) noexcept;
void operator delete[](void*) noexcept;
void operator delete(void*, std::size_t) noexcept;
void operator delete[](void*, std::size_t) noexcept;
```

These implicit declarations introduce only the function names `operator new`, `operator new[]`, `operator delete`, and `operator delete[]`. [*Note:* The implicit declarations do not introduce the names `std`,

`std::size_t`, or any other names that the library uses to declare these names. Thus, a *new-expression*, *delete-expression* or function call that refers to one of these functions without including the header `<new>` is well-formed. However, referring to `std` or `std::size_t` is ill-formed unless the name has been declared by including the appropriate header. — *end note*] Allocation and/or deallocation functions can also be declared and defined for any class (12.5).

3 Any allocation and/or deallocation functions defined in a C++ program, including the default versions in the library, shall conform to the semantics specified in 3.7.4.1 and 3.7.4.2.

3.7.4.1 Allocation functions [basic.stc.dynamic.allocation]

1 An allocation function shall be a class member function or a global function; a program is ill-formed if an allocation function is declared in a namespace scope other than global scope or declared static in global scope. The return type shall be `void*`. The first parameter shall have type `std::size_t` (18.2). The first parameter shall not have an associated default argument (8.3.6). The value of the first parameter shall be interpreted as the requested size of the allocation. An allocation function can be a function template. Such a template shall declare its return type and first parameter as specified above (that is, template parameter types shall not be used in the return type and first parameter type). Template allocation functions shall have two or more parameters.

2 The allocation function attempts to allocate the requested amount of storage. If it is successful, it shall return the address of the start of a block of storage whose length in bytes shall be at least as large as the requested size. There are no constraints on the contents of the allocated storage on return from the allocation function. The order, contiguity, and initial value of storage allocated by successive calls to an allocation function are unspecified. The pointer returned shall be suitably aligned so that it can be converted to a pointer of any complete object type with a fundamental alignment requirement (3.11) and then used to access the object or array in the storage allocated (until the storage is explicitly deallocated by a call to a corresponding deallocation function). Even if the size of the space requested is zero, the request can fail. If the request succeeds, the value returned shall be a non-null pointer value (4.10) `p0` different from any previously returned value `p1`, unless that value `p1` was subsequently passed to an `operator delete`. Furthermore, for the library allocation functions in 18.6.1.1 and 18.6.1.2, `p0` shall point to a block of storage disjoint from the storage for any other object accessible to the caller. The effect of indirecting through a pointer returned as a request for zero size is undefined.[36]

3 An allocation function that fails to allocate storage can invoke the currently installed new-handler function (18.6.2.3), if any. [*Note:* A program-supplied allocation function can obtain the address of the currently installed `new_handler` using the `std::get_new_handler` function (18.6.2.4). — *end note*] If an allocation function that has a non-throwing exception specification (15.4) fails to allocate storage, it shall return a null pointer. Any other allocation function that fails to allocate storage shall indicate failure only by throwing an exception (15.1) of a type that would match a handler (15.3) of type `std::bad_alloc` (18.6.2.1).

4 A global allocation function is only called as the result of a new expression (5.3.4), or called directly using the function call syntax (5.2.2), or called indirectly through calls to the functions in the C++ standard library. [*Note:* In particular, a global allocation function is not called to allocate storage for objects with static storage duration (3.7.1), for objects or references with thread storage duration (3.7.2), for objects of type `std::type_info` (5.2.8), or for an exception object (15.1). — *end note*]

3.7.4.2 Deallocation functions [basic.stc.dynamic.deallocation]

1 Deallocation functions shall be class member functions or global functions; a program is ill-formed if deallocation functions are declared in a namespace scope other than global scope or declared static in global scope.

36) The intent is to have `operator new()` implementable by calling `std::malloc()` or `std::calloc()`, so the rules are substantially the same. C++ differs from C in requiring a zero request to return a non-null pointer.

2 Each deallocation function shall return `void` and its first parameter shall be `void*`. A deallocation function can have more than one parameter. The global `operator delete` with exactly one parameter is a usual (non-placement) deallocation function. The global `operator delete` with exactly two parameters, the second of which has type `std::size_t`, is a usual deallocation function. Similarly, the global `operator delete[]` with exactly one parameter is a usual deallocation function. The global `operator delete[]` with exactly two parameters, the second of which has type `std::size_t`, is a usual deallocation function.[37] If a class `T` has a member deallocation function named `operator delete` with exactly one parameter, then that function is a usual deallocation function. If class `T` does not declare such an `operator delete` but does declare a member deallocation function named `operator delete` with exactly two parameters, the second of which has type `std::size_t`, then this function is a usual deallocation function. Similarly, if a class `T` has a member deallocation function named `operator delete[]` with exactly one parameter, then that function is a usual (non-placement) deallocation function. If class `T` does not declare such an `operator delete[]` but does declare a member deallocation function named `operator delete[]` with exactly two parameters, the second of which has type `std::size_t`, then this function is a usual deallocation function. A deallocation function can be an instance of a function template. Neither the first parameter nor the return type shall depend on a template parameter. [*Note:* That is, a deallocation function template shall have a first parameter of type `void*` and a return type of `void` (as specified above). — *end note*] A deallocation function template shall have two or more function parameters. A template instance is never a usual deallocation function, regardless of its signature.

3 If a deallocation function terminates by throwing an exception, the behavior is undefined. The value of the first argument supplied to a deallocation function may be a null pointer value; if so, and if the deallocation function is one supplied in the standard library, the call has no effect. Otherwise, the behavior is undefined if the value supplied to `operator delete(void*)` in the standard library is not one of the values returned by a previous invocation of either `operator new(std::size_t)` or `operator new(std::size_t, const std::nothrow_t&)` in the standard library, and the behavior is undefined if the value supplied to `operator delete[](void*)` in the standard library is not one of the values returned by a previous invocation of either `operator new[](std::size_t)` or `operator new[](std::size_t, const std::nothrow_t&)` in the standard library.

4 If the argument given to a deallocation function in the standard library is a pointer that is not the null pointer value (4.10), the deallocation function shall deallocate the storage referenced by the pointer, rendering invalid all pointers referring to any part of the deallocated storage. Indirection through an invalid pointer value and passing an invalid pointer value to a deallocation function have undefined behavior. Any other use of an invalid pointer value has implementation-defined behavior.[38]

3.7.4.3 Safely-derived pointers [basic.stc.dynamic.safety]

1 A *traceable pointer object* is

(1.1) — an object of an object pointer type (3.9.2), or

(1.2) — an object of an integral type that is at least as large as `std::intptr_t`, or

(1.3) — a sequence of elements in an array of narrow character type (3.9.1), where the size and alignment of the sequence match those of some object pointer type.

2 A pointer value is a *safely-derived pointer* to a dynamic object only if it has an object pointer type and it is one of the following:

37) This deallocation function precludes use of an allocation function `void operator new(std::size_t, std::size_t)` as a placement allocation function (C.3.2).

38) Some implementations might define that copying an invalid pointer value causes a system-generated runtime fault.

(2.1) — the value returned by a call to the C++ standard library implementation of `::operator new(std::size_t)`;[39]

(2.2) — the result of taking the address of an object (or one of its subobjects) designated by an lvalue resulting from indirection through a safely-derived pointer value;

(2.3) — the result of well-defined pointer arithmetic (5.7) using a safely-derived pointer value;

(2.4) — the result of a well-defined pointer conversion (4.10, 5.4) of a safely-derived pointer value;

(2.5) — the result of a `reinterpret_cast` of a safely-derived pointer value;

(2.6) — the result of a `reinterpret_cast` of an integer representation of a safely-derived pointer value;

(2.7) — the value of an object whose value was copied from a traceable pointer object, where at the time of the copy the source object contained a copy of a safely-derived pointer value.

3 An integer value is an *integer representation of a safely-derived pointer* only if its type is at least as large as `std::intptr_t` and it is one of the following:

(3.1) — the result of a `reinterpret_cast` of a safely-derived pointer value;

(3.2) — the result of a valid conversion of an integer representation of a safely-derived pointer value;

(3.3) — the value of an object whose value was copied from a traceable pointer object, where at the time of the copy the source object contained an integer representation of a safely-derived pointer value;

(3.4) — the result of an additive or bitwise operation, one of whose operands is an integer representation of a safely-derived pointer value `P`, if that result converted by `reinterpret_cast<void*>` would compare equal to a safely-derived pointer computable from `reinterpret_cast<void*>(P)`.

4 An implementation may have *relaxed pointer safety*, in which case the validity of a pointer value does not depend on whether it is a safely-derived pointer value. Alternatively, an implementation may have *strict pointer safety*, in which case a pointer value referring to an object with dynamic storage duration that is not a safely-derived pointer value is an invalid pointer value unless the referenced complete object has previously been declared reachable (20.7.4). [*Note:* the effect of using an invalid pointer value (including passing it to a deallocation function) is undefined, see 3.7.4.2. This is true even if the unsafely-derived pointer value might compare equal to some safely-derived pointer value. — *end note*] It is implementation defined whether an implementation has relaxed or strict pointer safety.

3.7.5 Duration of subobjects [basic.stc.inherit]

1 The storage duration of member subobjects, base class subobjects and array elements is that of their complete object (1.8).

3.8 Object lifetime [basic.life]

1 The *lifetime* of an object is a runtime property of the object. An object is said to have *non-vacuous initialization* if it is of a class or aggregate type and it or one of its members is initialized by a constructor other than a trivial default constructor. [*Note:* initialization by a trivial copy/move constructor is non-vacuous initialization. — *end note*] The lifetime of an object of type `T` begins when:

(1.1) — storage with the proper alignment and size for type `T` is obtained, and

39) This section does not impose restrictions on indirection through pointers to memory not allocated by `::operator new`. This maintains the ability of many C++ implementations to use binary libraries and components written in other languages. In particular, this applies to C binaries, because indirection through pointers to memory allocated by `std::malloc` is not restricted.

(1.2) — if the object has non-vacuous initialization, its initialization is complete.

The lifetime of an object of type `T` ends when:

(1.3) — if `T` is a class type with a non-trivial destructor (12.4), the destructor call starts, or

(1.4) — the storage which the object occupies is reused or released.

2 [*Note:* The lifetime of an array object starts as soon as storage with proper size and alignment is obtained, and its lifetime ends when the storage which the array occupies is reused or released. 12.6.2 describes the lifetime of base and member subobjects. — *end note*]

3 The properties ascribed to objects throughout this International Standard apply for a given object only during its lifetime. [*Note:* In particular, before the lifetime of an object starts and after its lifetime ends there are significant restrictions on the use of the object, as described below, in 12.6.2 and in 12.7. Also, the behavior of an object under construction and destruction might not be the same as the behavior of an object whose lifetime has started and not ended. 12.6.2 and 12.7 describe the behavior of objects during the construction and destruction phases. — *end note*]

4 A program may end the lifetime of any object by reusing the storage which the object occupies or by explicitly calling the destructor for an object of a class type with a non-trivial destructor. For an object of a class type with a non-trivial destructor, the program is not required to call the destructor explicitly before the storage which the object occupies is reused or released; however, if there is no explicit call to the destructor or if a *delete-expression* (5.3.5) is not used to release the storage, the destructor shall not be implicitly called and any program that depends on the side effects produced by the destructor has undefined behavior.

5 Before the lifetime of an object has started but after the storage which the object will occupy has been allocated[40] or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any pointer that refers to the storage location where the object will be or was located may be used but only in limited ways. For an object under construction or destruction, see 12.7. Otherwise, such a pointer refers to allocated storage (3.7.4.2), and using the pointer as if the pointer were of type `void*`, is well-defined. Indirection through such a pointer is permitted but the resulting lvalue may only be used in limited ways, as described below. The program has undefined behavior if:

(5.1) — the object will be or was of a class type with a non-trivial destructor and the pointer is used as the operand of a *delete-expression*,

(5.2) — the pointer is used to access a non-static data member or call a non-static member function of the object, or

(5.3) — the pointer is implicitly converted (4.10) to a pointer to a virtual base class, or

(5.4) — the pointer is used as the operand of a `static_cast` (5.2.9), except when the conversion is to pointer to *cv* `void`, or to pointer to *cv* `void` and subsequently to pointer to either *cv* `char` or *cv* `unsigned char`, or

(5.5) — the pointer is used as the operand of a `dynamic_cast` (5.2.7). [*Example:*

```
#include <cstdlib>

struct B {
  virtual void f();
  void mutate();
  virtual ~B();
};
```

40) For example, before the construction of a global object of non-POD class type (12.7).

```
struct D1 : B { void f(); };
struct D2 : B { void f(); };

void B::mutate() {
  new (this) D2;     // reuses storage — ends the lifetime of *this
  f();               // undefined behavior
  ... = this;        // OK, this points to valid memory
}

void g() {
  void* p = std::malloc(sizeof(D1) + sizeof(D2));
  B* pb = new (p) D1;
  pb->mutate();
  &pb;               // OK: pb points to valid memory
  void* q = pb;      // OK: pb points to valid memory
  pb->f();           // undefined behavior, lifetime of *pb has ended
}
```

— *end example*]

6 Similarly, before the lifetime of an object has started but after the storage which the object will occupy has been allocated or, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, any glvalue that refers to the original object may be used but only in limited ways. For an object under construction or destruction, see 12.7. Otherwise, such a glvalue refers to allocated storage (3.7.4.2), and using the properties of the glvalue that do not depend on its value is well-defined. The program has undefined behavior if:

(6.1) — an lvalue-to-rvalue conversion (4.1) is applied to such a glvalue,

(6.2) — the glvalue is used to access a non-static data member or call a non-static member function of the object, or

(6.3) — the glvalue is bound to a reference to a virtual base class (8.5.3), or

(6.4) — the glvalue is used as the operand of a `dynamic_cast` (5.2.7) or as the operand of `typeid`.

7 If, after the lifetime of an object has ended and before the storage which the object occupied is reused or released, a new object is created at the storage location which the original object occupied, a pointer that pointed to the original object, a reference that referred to the original object, or the name of the original object will automatically refer to the new object and, once the lifetime of the new object has started, can be used to manipulate the new object, if:

(7.1) — the storage for the new object exactly overlays the storage location which the original object occupied, and

(7.2) — the new object is of the same type as the original object (ignoring the top-level cv-qualifiers), and

(7.3) — the type of the original object is not const-qualified, and, if a class type, does not contain any non-static data member whose type is const-qualified or a reference type, and

(7.4) — the original object was a most derived object (1.8) of type `T` and the new object is a most derived object of type `T` (that is, they are not base class subobjects). [*Example:*

```
struct C {
  int i;
  void f();
  const C& operator=( const C& );
```

```
};

const C& C::operator=( const C& other) {
  if ( this != &other ) {
    this->~C();                 // lifetime of *this ends
    new (this) C(other);        // new object of type C created
    f();                        // well-defined
  }
  return *this;
}

C c1;
C c2;
c1 = c2;                        // well-defined
c1.f();                         // well-defined; c1 refers to a new object of type C
```

— *end example*]

8 If a program ends the lifetime of an object of type `T` with static (3.7.1), thread (3.7.2), or automatic (3.7.3) storage duration and if `T` has a non-trivial destructor,[41] the program must ensure that an object of the original type occupies that same storage location when the implicit destructor call takes place; otherwise the behavior of the program is undefined. This is true even if the block is exited with an exception. [*Example:*

```
class T { };
struct B {
   ~B();
};

void h() {
   B b;
   new (&b) T;
}                               // undefined behavior at block exit
```

— *end example*]

9 Creating a new object at the storage location that a `const` object with static, thread, or automatic storage duration occupies or, at the storage location that such a `const` object used to occupy before its lifetime ended results in undefined behavior. [*Example:*

```
struct B {
  B();
  ~B();
};

const B b;

void h() {
  b.~B();
  new (const_cast<B*>(&b)) const B;             // undefined behavior
}
```

— *end example*]

41) That is, an object for which a destructor will be called implicitly—upon exit from the block for an object with automatic storage duration, upon exit from the thread for an object with thread storage duration, or upon exit from the program for an object with static storage duration.

10 In this section, "before" and "after" refer to the "happens before" relation (1.10). [*Note:* Therefore, undefined behavior results if an object that is being constructed in one thread is referenced from another thread without adequate synchronization. — *end note*]

3.9 Types [basic.types]

1 [*Note:* 3.9 and the subclauses thereof impose requirements on implementations regarding the representation of types. There are two kinds of types: fundamental types and compound types. Types describe objects (1.8), references (8.3.2), or functions (8.3.5). — *end note*]

2 For any object (other than a base-class subobject) of trivially copyable type `T`, whether or not the object holds a valid value of type `T`, the underlying bytes (1.7) making up the object can be copied into an array of `char` or `unsigned char`.[42] If the content of the array of `char` or `unsigned char` is copied back into the object, the object shall subsequently hold its original value. [*Example:*

```
#define N sizeof(T)
char buf[N];
T obj;                          // obj initialized to its original value
std::memcpy(buf, &obj, N);      // between these two calls to std::memcpy,
                                // obj might be modified
std::memcpy(&obj, buf, N);      // at this point, each subobject of obj of scalar type
                                // holds its original value
```

— *end example*]

3 For any trivially copyable type `T`, if two pointers to `T` point to distinct `T` objects `obj1` and `obj2`, where neither `obj1` nor `obj2` is a base-class subobject, if the underlying bytes (1.7) making up `obj1` are copied into `obj2`,[43] `obj2` shall subsequently hold the same value as `obj1`. [*Example:*

```
T* t1p;
T* t2p;
    // provided that t2p points to an initialized object ...
std::memcpy(t1p, t2p, sizeof(T));
    // at this point, every subobject of trivially copyable type in *t1p contains
    // the same value as the corresponding subobject in *t2p
```

— *end example*]

4 The *object representation* of an object of type `T` is the sequence of N `unsigned char` objects taken up by the object of type `T`, where N equals `sizeof(T)`. The *value representation* of an object is the set of bits that hold the value of type `T`. For trivially copyable types, the value representation is a set of bits in the object representation that determines a *value*, which is one discrete element of an implementation-defined set of values.[44]

5 A class that has been declared but not defined, an enumeration type in certain contexts (7.2), or an array of unknown size or of incomplete element type, is an *incompletely-defined object type*.[45] Incompletely-defined object types and the void types are *incomplete types* (3.9.1). Objects shall not be defined to have an incomplete type.

6 A class type (such as "`class X`") might be incomplete at one point in a translation unit and complete later on; the type "`class X`" is the same type at both points. The declared type of an array object might be an array of incomplete class type and therefore incomplete; if the class type is completed later on in the translation unit, the array type becomes complete; the array type at those two points is the same type. The

42) By using, for example, the library functions (17.6.1.2) `std::memcpy` or `std::memmove`.
43) By using, for example, the library functions (17.6.1.2) `std::memcpy` or `std::memmove`.
44) The intent is that the memory model of C++ is compatible with that of ISO/IEC 9899 Programming Language C.
45) The size and layout of an instance of an incompletely-defined object type is unknown.

declared type of an array object might be an array of unknown bound and therefore be incomplete at one point in a translation unit and complete later on; the array types at those two points ("array of unknown bound of `T`" and "array of `N T`") are different types. The type of a pointer to array of unknown size, or of a type defined by a `typedef` declaration to be an array of unknown size, cannot be completed. [*Example:*

```
class X;                        // X is an incomplete type
extern X* xp;                   // xp is a pointer to an incomplete type
extern int arr[];               // the type of arr is incomplete
typedef int UNKA[];             // UNKA is an incomplete type
UNKA* arrp;                     // arrp is a pointer to an incomplete type
UNKA** arrpp;

void foo() {
  xp++;                         // ill-formed: X is incomplete
  arrp++;                       // ill-formed: incomplete type
  arrpp++;                      // OK: sizeof UNKA* is known
}

struct X { int i; };            // now X is a complete type
int  arr[10];                   // now the type of arr is complete

X x;
void bar() {
  xp = &x;                      // OK; type is "pointer to X"
  arrp = &arr;                  // ill-formed: different types
  xp++;                         // OK: X is complete
  arrp++;                       // ill-formed: UNKA can't be completed
}
```

— *end example*]

7 [*Note:* The rules for declarations and expressions describe in which contexts incomplete types are prohibited. — *end note*]

8 An *object type* is a (possibly cv-qualified) type that is not a function type, not a reference type, and not a void type.

9 Arithmetic types (3.9.1), enumeration types, pointer types, pointer to member types (3.9.2), `std::nullptr_t`, and cv-qualified versions of these types (3.9.3) are collectively called *scalar types*. Scalar types, POD classes (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *POD types*. Cv-unqualified scalar types, trivially copyable class types (Clause 9), arrays of such types, and non-volatile const-qualified versions of these types (3.9.3) are collectively called *trivially copyable types*. Scalar types, trivial class types (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *trivial types*. Scalar types, standard-layout class types (Clause 9), arrays of such types and cv-qualified versions of these types (3.9.3) are collectively called *standard-layout types*.

10 A type is a *literal type* if it is:

(10.1) — possibly cv-qualified `void`; or

(10.2) — a scalar type; or

(10.3) — a reference type; or

(10.4) — an array of literal type; or

(10.5) — a possibly cv-qualified class type (Clause 9) that has all of the following properties:

(10.5.1) — it has a trivial destructor,

(10.5.2) — it is an aggregate type (8.5.1) or has at least one `constexpr` constructor or constructor template that is not a copy or move constructor, and

(10.5.3) — all of its non-static data members and base classes are of non-volatile literal types.

11 Two types *cv1* `T1` and *cv2* `T2` are *layout-compatible* types if `T1` and `T2` are the same type, layout-compatible enumerations (7.2), or layout-compatible standard-layout class types (9.2).

3.9.1 Fundamental types [basic.fundamental]

1 Objects declared as characters (`char`) shall be large enough to store any member of the implementation's basic character set. If a character from this set is stored in a character object, the integral value of that character object is equal to the value of the single character literal form of that character. It is implementation-defined whether a `char` object can hold negative values. Characters can be explicitly declared `unsigned` or `signed`. Plain `char`, `signed char`, and `unsigned char` are three distinct types, collectively called *narrow character types*. A `char`, a `signed char`, and an `unsigned char` occupy the same amount of storage and have the same alignment requirements (3.11); that is, they have the same object representation. For narrow character types, all bits of the object representation participate in the value representation. For unsigned narrow character types, each possible bit pattern of the value representation represents a distinct number. These requirements do not hold for other types. In any particular implementation, a plain `char` object can take on either the same values as a `signed char` or an `unsigned char`; which one is implementation-defined. For each value i of type `unsigned char` in the range 0 to 255 inclusive, there exists a value j of type `char` such that the result of an integral conversion (4.7) from i to `char` is j, and the result of an integral conversion from j to `unsigned char` is i.

2 There are five *standard signed integer types* : "`signed char`", "`short int`", "`int`", "`long int`", and "`long long int`". In this list, each type provides at least as much storage as those preceding it in the list. There may also be implementation-defined *extended signed integer types*. The standard and extended signed integer types are collectively called *signed integer types*. Plain `int`s have the natural size suggested by the architecture of the execution environment[46]; the other signed integer types are provided to meet special needs.

3 For each of the standard signed integer types, there exists a corresponding (but different) *standard unsigned integer type*: "`unsigned char`", "`unsigned short int`", "`unsigned int`", "`unsigned long int`", and "`unsigned long long int`", each of which occupies the same amount of storage and has the same alignment requirements (3.11) as the corresponding signed integer type[47]; that is, each signed integer type has the same object representation as its corresponding unsigned integer type. Likewise, for each of the extended signed integer types there exists a corresponding *extended unsigned integer type* with the same amount of storage and alignment requirements. The standard and extended unsigned integer types are collectively called *unsigned integer types*. The range of non-negative values of a *signed integer* type is a subrange of the corresponding *unsigned integer* type, and the value representation of each corresponding signed/unsigned type shall be the same. The standard signed integer types and standard unsigned integer types are collectively called the *standard integer types*, and the extended signed integer types and extended unsigned integer types are collectively called the *extended integer types*. The signed and unsigned integer types shall satisfy the constraints given in the C standard, section 5.2.4.2.1.

4 Unsigned integers shall obey the laws of arithmetic modulo 2^n where n is the number of bits in the value representation of that particular size of integer.[48]

46) `int` must also be large enough to contain any value in the range `[INT_MIN,INT_MAX]`, as defined in the header `<climits>`.
47) See 7.1.6.2 regarding the correspondence between types and the sequences of *type-specifier*s that designate them.
48) This implies that unsigned arithmetic does not overflow because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting unsigned integer type.

5 Type `wchar_t` is a distinct type whose values can represent distinct codes for all members of the largest extended character set specified among the supported locales (22.3.1). Type `wchar_t` shall have the same size, signedness, and alignment requirements (3.11) as one of the other integral types, called its *underlying type*. Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`, called the underlying types.

6 Values of type `bool` are either `true` or `false`.[49] [*Note:* There are no `signed`, `unsigned`, `short`, or `long bool` types or values. — *end note*] Values of type `bool` participate in integral promotions (4.5).

7 Types `bool`, `char`, `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively called *integral* types.[50] A synonym for integral type is *integer type*. The representations of integral types shall define values by use of a pure binary numeration system.[51] [*Example:* this International Standard permits 2's complement, 1's complement and signed magnitude representations for integral types. — *end example*]

8 There are three *floating point* types: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The value representation of floating-point types is implementation-defined. *Integral* and *floating* types are collectively called *arithmetic* types. Specializations of the standard template `std::numeric_limits` (18.3) shall specify the maximum and minimum values of each arithmetic type for an implementation.

9 The `void` type has an empty set of values. The `void` type is an incomplete type that cannot be completed. It is used as the return type for functions that do not return a value. Any expression can be explicitly converted to type *cv* `void` (5.4). An expression of type `void` shall be used only as an expression statement (6.2), as an operand of a comma expression (5.19), as a second or third operand of `?:` (5.16), as the operand of `typeid`, `noexcept`, or `decltype`, as the expression in a return statement (6.6.3) for a function with the return type `void`, or as the operand of an explicit conversion to type *cv* `void`.

10 A value of type `std::nullptr_t` is a null pointer constant (4.10). Such values participate in the pointer and the pointer to member conversions (4.10, 4.11). `sizeof(std::nullptr_t)` shall be equal to `sizeof(void*)`.

11 [*Note:* Even if the implementation defines two or more basic types to have the same value representation, they are nevertheless different types. — *end note*]

3.9.2 Compound types [basic.compound]

1 Compound types can be constructed in the following ways:

(1.1) — *arrays* of objects of a given type, 8.3.4;

(1.2) — *functions*, which have parameters of given types and return `void` or references or objects of a given type, 8.3.5;

(1.3) — *pointers* to `void` or objects or functions (including static members of classes) of a given type, 8.3.1;

(1.4) — *references* to objects or functions of a given type, 8.3.2. There are two types of references:

(1.4.1) — *lvalue reference*

(1.4.2) — *rvalue reference*

49) Using a `bool` value in ways described by this International Standard as "undefined," such as by examining the value of an uninitialized automatic object, might cause it to behave as if it is neither `true` nor `false`.

50) Therefore, enumerations (7.2) are not integral; however, enumerations can be promoted to integral types as specified in 4.5.

51) A positional representation for integers that uses the binary digits 0 and 1, in which the values represented by successive bits are additive, begin with 1, and are multiplied by successive integral power of 2, except perhaps for the bit with the highest position. (Adapted from the *American National Dictionary for Information Processing Systems*.)

(1.5) — *classes* containing a sequence of objects of various types (Clause 9), a set of types, enumerations and functions for manipulating these objects (9.3), and a set of restrictions on the access to these entities (Clause 11);

(1.6) — *unions*, which are classes capable of containing objects of different types at different times, 9.5;

(1.7) — *enumerations*, which comprise a set of named constant values. Each distinct enumeration constitutes a different *enumerated type*, 7.2;

(1.8) — *pointers to non-static* [52] *class members*, which identify members of a given type within objects of a given class, 8.3.3.

2 These methods of constructing types can be applied recursively; restrictions are mentioned in 8.3.1, 8.3.4, 8.3.5, and 8.3.2. Constructing a type such that the number of bytes in its object representation exceeds the maximum value representable in the type `std::size_t` (18.2) is ill-formed.

3 The type of a pointer to `void` or a pointer to an object type is called an *object pointer type*. [*Note:* A pointer to `void` does not have a pointer-to-object type, however, because `void` is not an object type. — *end note*] The type of a pointer that can designate a function is called a *function pointer type*. A pointer to objects of type `T` is referred to as a "pointer to `T`". [*Example:* a pointer to an object of type `int` is referred to as "pointer to `int` " and a pointer to an object of class `X` is called a "pointer to `X`". — *end example*] Except for pointers to static members, text referring to "pointers" does not apply to pointers to members. Pointers to incomplete types are allowed although there are restrictions on what can be done with them (3.11). A valid value of an object pointer type represents either the address of a byte in memory (1.7) or a null pointer (4.10). If an object of type `T` is located at an address `A`, a pointer of type *cv* `T*` whose value is the address `A` is said to *point to* that object, regardless of how the value was obtained. [*Note:* For instance, the address one past the end of an array (5.7) would be considered to point to an unrelated object of the array's element type that might be located at that address. There are further restrictions on pointers to objects with dynamic storage duration; see 3.7.4.3. — *end note*] The value representation of pointer types is implementation-defined. Pointers to layout-compatible types shall have the same value representation and alignment requirements (3.11). [*Note:* Pointers to over-aligned types (3.11) have no special representation, but their range of valid values is restricted by the extended alignment requirement. This International Standard specifies only two ways of obtaining such a pointer: taking the address of a valid object with an over-aligned type, and using one of the runtime pointer alignment functions. An implementation may provide other means of obtaining a valid pointer value for an over-aligned type. — *end note*]

4 A pointer to *cv*-qualified (3.9.3) or *cv*-unqualified `void` can be used to point to objects of unknown type. Such a pointer shall be able to hold any object pointer. An object of type *cv* `void*` shall have the same representation and alignment requirements as *cv* `char*`.

3.9.3 CV-qualifiers [basic.type.qualifier]

1 A type mentioned in 3.9.1 and 3.9.2 is a *cv-unqualified type*. Each type which is a cv-unqualified complete or incomplete object type or is `void` (3.9) has three corresponding cv-qualified versions of its type: a *const-qualified* version, a *volatile-qualified* version, and a *const-volatile-qualified* version. The term *object type* (1.8) includes the cv-qualifiers specified in the *decl-specifier-seq* (7.1), *declarator* (Clause 8), *type-id* (8.1), or *new-type-id* (5.3.4) when the object is created.

(1.1) — A *const object* is an object of type `const T` or a non-mutable subobject of such an object.

(1.2) — A *volatile object* is an object of type `volatile T`, a subobject of such an object, or a mutable subobject of a const volatile object.

(1.3) — A *const volatile object* is an object of type `const volatile T`, a non-mutable subobject of such an object, a const subobject of a volatile object, or a non-mutable volatile subobject of a const object.

52) Static class members are objects or functions, and pointers to them are ordinary pointers to objects or functions.

The cv-qualified or cv-unqualified versions of a type are distinct types; however, they shall have the same representation and alignment requirements (3.11).[53]

2 A compound type (3.9.2) is not cv-qualified by the cv-qualifiers (if any) of the types from which it is compounded. Any cv-qualifiers applied to an array type affect the array element type, not the array type (8.3.4).

3 See 8.3.5 and 9.3.2 regarding function types that have *cv-qualifier*s.

4 There is a partial ordering on cv-qualifiers, so that a type can be said to be *more cv-qualified* than another. Table 8 shows the relations that constitute this ordering.

Table 8 — Relations on `const` and `volatile`

no cv-qualifier	<	`const`
no cv-qualifier	<	`volatile`
no cv-qualifier	<	`const volatile`
`const`	<	`const volatile`
`volatile`	<	`const volatile`

5 In this International Standard, the notation *cv* (or *cv1*, *cv2*, etc.), used in the description of types, represents an arbitrary set of cv-qualifiers, i.e., one of {`const`}, {`volatile`}, {`const`, `volatile`}, or the empty set. For a type *cv* `T`, the *top-level cv-qualifiers* of that type are those denoted by *cv*. [*Example:* The type corresponding to the *type-id* `const int&` has no top-level cv-qualifiers. The type corresponding to the *type-id* `volatile int * const` has the top-level cv-qualifier `const`. For a class type `C`, the type corresponding to the *type-id* `void (C::* volatile)(int) const` has the top-level cv-qualifier `volatile`. — *end example*]

6 Cv-qualifiers applied to an array type attach to the underlying element type, so the notation "*cv* `T`", where `T` is an array type, refers to an array whose elements are so-qualified. An array type whose elements are cv-qualified is also considered to have the same cv-qualifications as its elements. [*Example:*

```
typedef char CA[5];
typedef const char CC;
CC arr1[5] = { 0 };
const CA arr2 = { 0 };
```

The type of both `arr1` and `arr2` is "array of 5 `const char`", and the array type is considered to be `const`-qualified. — *end example*]

3.10 Lvalues and rvalues [basic.lval]

1 Expressions are categorized according to the taxonomy in Figure 1.

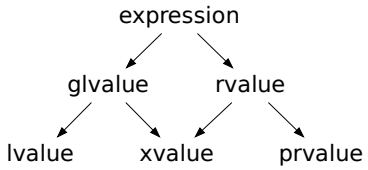


Figure 1 — Expression category taxonomy

53) The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and non-static data members of unions.

(1.1) — An *lvalue* (so called, historically, because lvalues could appear on the left-hand side of an assignment expression) designates a function or an object. [*Example:* If `E` is an expression of pointer type, then `*E` is an lvalue expression referring to the object or function to which `E` points. As another example, the result of calling a function whose return type is an lvalue reference is an lvalue. — *end example*]

(1.2) — An *xvalue* (an "eXpiring" value) also refers to an object, usually near the end of its lifetime (so that its resources may be moved, for example). Certain kinds of expressions involving rvalue references (8.3.2) yield xvalues. [*Example:* The result of calling a function whose return type is an rvalue reference to an object type is an xvalue (5.2.2). — *end example*]

(1.3) — A *glvalue* ("generalized" lvalue) is an lvalue or an xvalue.

(1.4) — An *rvalue* (so called, historically, because rvalues could appear on the right-hand side of an assignment expression) is an xvalue, a temporary object (12.2) or subobject thereof, or a value that is not associated with an object.

(1.5) — A *prvalue* ("pure" rvalue) is an rvalue that is not an xvalue. [*Example:* The result of calling a function whose return type is not a reference is a prvalue. The value of a literal such as `12`, `7.3e5`, or `true` is also a prvalue. — *end example*]

Every expression belongs to exactly one of the fundamental classifications in this taxonomy: lvalue, xvalue, or prvalue. This property of an expression is called its *value category*. [*Note:* The discussion of each built-in operator in Clause 5 indicates the category of the value it yields and the value categories of the operands it expects. For example, the built-in assignment operators expect that the left operand is an lvalue and that the right operand is a prvalue and yield an lvalue as the result. User-defined operators are functions, and the categories of values they expect and yield are determined by their parameter and return types. — *end note*]

2 Whenever a glvalue appears in a context where a prvalue is expected, the glvalue is converted to a prvalue; see 4.1, 4.2, and 4.3. [*Note:* An attempt to bind an rvalue reference to an lvalue is not such a context; see 8.5.3. — *end note*] [*Note:* There are no prvalue bit-fields; if a bit-field is converted to a prvalue (4.1), a prvalue of the type of the bit-field is created, which might then be promoted (4.5). — *end note*]

3 The discussion of reference initialization in 8.5.3 and of temporaries in 12.2 indicates the behavior of lvalues and rvalues in other significant contexts.

4 Unless otherwise indicated (5.2.2), prvalues shall always have complete types or the `void` type; in addition to these types, glvalues can also have incomplete types. [*Note:* class and array prvalues can have cv-qualified types; other prvalues always have cv-unqualified types. See Clause 5. — *end note*]

5 An lvalue for an object is necessary in order to modify the object except that an rvalue of class type can also be used to modify its referent under certain circumstances. [*Example:* a member function called for an object (9.3) can modify the object. — *end example*]

6 Functions cannot be modified, but pointers to functions can be modifiable.

7 A pointer to an incomplete type can be modifiable. At some point in the program when the pointed to type is complete, the object at which the pointer points can also be modified.

8 The referent of a `const`-qualified expression shall not be modified (through that expression), except that if it is of class type and has a `mutable` component, that component can be modified (7.1.6.1).

9 If an expression can be used to modify the object to which it refers, the expression is called *modifiable*. A program that attempts to modify an object through a nonmodifiable lvalue or rvalue expression is ill-formed.

10 If a program attempts to access the stored value of an object through a glvalue of other than one of the following types the behavior is undefined:[54]

54) The intent of this list is to specify those circumstances in which an object may or may not be aliased.

(10.1) — the dynamic type of the object,

(10.2) — a cv-qualified version of the dynamic type of the object,

(10.3) — a type similar (as defined in 4.4) to the dynamic type of the object,

(10.4) — a type that is the signed or unsigned type corresponding to the dynamic type of the object,

(10.5) — a type that is the signed or unsigned type corresponding to a cv-qualified version of the dynamic type of the object,

(10.6) — an aggregate or union type that includes one of the aforementioned types among its elements or non-static data members (including, recursively, an element or non-static data member of a subaggregate or contained union),

(10.7) — a type that is a (possibly cv-qualified) base class type of the dynamic type of the object,

(10.8) — a `char` or `unsigned char` type.

3.11 Alignment [basic.align]

1 Object types have *alignment requirements* (3.9.1, 3.9.2) which place restrictions on the addresses at which an object of that type may be allocated. An *alignment* is an implementation-defined integer value representing the number of bytes between successive addresses at which a given object can be allocated. An object type imposes an alignment requirement on every object of that type; stricter alignment can be requested using the alignment specifier (7.6.2).

2 A *fundamental alignment* is represented by an alignment less than or equal to the greatest alignment supported by the implementation in all contexts, which is equal to `alignof(std::max_align_t)` (18.2). The alignment required for a type might be different when it is used as the type of a complete object and when it is used as the type of a subobject. [*Example:*

```
struct B { long double d; };
struct D : virtual B { char c; };
```

When `D` is the type of a complete object, it will have a subobject of type `B`, so it must be aligned appropriately for a `long double`. If `D` appears as a subobject of another object that also has `B` as a virtual base class, the `B` subobject might be part of a different subobject, reducing the alignment requirements on the `D` subobject. — *end example*] The result of the `alignof` operator reflects the alignment requirement of the type in the complete-object case.

3 An *extended alignment* is represented by an alignment greater than `alignof(std::max_align_t)`. It is implementation-defined whether any extended alignments are supported and the contexts in which they are supported (7.6.2). A type having an extended alignment requirement is an *over-aligned type*. [*Note:* every over-aligned type is or contains a class type to which extended alignment applies (possibly through a non-static data member). — *end note*]

4 Alignments are represented as values of the type `std::size_t`. Valid alignments include only those values returned by an `alignof` expression for the fundamental types plus an additional implementation-defined set of values, which may be empty. Every alignment value shall be a non-negative integral power of two.

5 Alignments have an order from *weaker* to *stronger* or *stricter* alignments. Stricter alignments have larger alignment values. An address that satisfies an alignment requirement also satisfies any weaker valid alignment requirement.

6 The alignment requirement of a complete type can be queried using an `alignof` expression (5.3.6). Furthermore, the narrow character types (3.9.1) shall have the weakest alignment requirement. [*Note:* This enables

the narrow character types to be used as the underlying type for an aligned memory area (7.6.2). — *end note*]

7 Comparing alignments is meaningful and provides the obvious results:

(7.1) — Two alignments are equal when their numeric values are equal.

(7.2) — Two alignments are different when their numeric values are not equal.

(7.3) — When an alignment is larger than another it represents a stricter alignment.

8 [*Note:* The runtime pointer alignment function (20.7.5) can be used to obtain an aligned pointer within a buffer; the aligned-storage templates in the library (20.10.7.6) can be used to obtain aligned storage. — *end note*]

9 If a request for a specific extended alignment in a specific context is not supported by an implementation, the program is ill-formed. Additionally, a request for runtime allocation of dynamic storage for which the requested alignment cannot be honored shall be treated as an allocation failure.

4 Standard conversions [conv]

1 Standard conversions are implicit conversions with built-in meaning. Clause 4 enumerates the full set of such conversions. A *standard conversion sequence* is a sequence of standard conversions in the following order:

(1.1) — Zero or one conversion from the following set: lvalue-to-rvalue conversion, array-to-pointer conversion, and function-to-pointer conversion.

(1.2) — Zero or one conversion from the following set: integral promotions, floating point promotion, integral conversions, floating point conversions, floating-integral conversions, pointer conversions, pointer to member conversions, and boolean conversions.

(1.3) — Zero or one qualification conversion.

[*Note:* A standard conversion sequence can be empty, i.e., it can consist of no conversions. — *end note*] A standard conversion sequence will be applied to an expression if necessary to convert it to a required destination type.

2 [*Note:* expressions with a given type will be implicitly converted to other types in several contexts:

(2.1) — When used as operands of operators. The operator's requirements for its operands dictate the destination type (Clause 5).

(2.2) — When used in the condition of an `if` statement or iteration statement (6.4, 6.5). The destination type is `bool`.

(2.3) — When used in the expression of a `switch` statement. The destination type is integral (6.4).

(2.4) — When used as the source expression for an initialization (which includes use as an argument in a function call and use as the expression in a `return` statement). The type of the entity being initialized is (generally) the destination type. See 8.5, 8.5.3.

— *end note*]

3 An expression `e` can be *implicitly converted* to a type `T` if and only if the declaration `T t=e;` is well-formed, for some invented temporary variable `t` (8.5).

4 Certain language constructs require that an expression be converted to a Boolean value. An expression `e` appearing in such a context is said to be *contextually converted to* `bool` and is well-formed if and only if the declaration `bool t(e);` is well-formed, for some invented temporary variable `t` (8.5).

5 Certain language constructs require conversion to a value having one of a specified set of types appropriate to the construct. An expression `e` of class type `E` appearing in such a context is said to be *contextually implicitly converted to* a specified type `T` and is well-formed if and only if `e` can be implicitly converted to a type `T` that is determined as follows: `E` is searched for conversion functions whose return type is *cv* `T` or reference to *cv* `T` such that `T` is allowed by the context. There shall be exactly one such `T`.

6 The effect of any implicit conversion is the same as performing the corresponding declaration and initialization and then using the temporary variable as the result of the conversion. The result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type (8.3.2), an xvalue if `T` is an rvalue reference to object type, and a prvalue otherwise. The expression `e` is used as a glvalue if and only if the initialization uses it as a glvalue.

7 [*Note:* For class types, user-defined conversions are considered as well; see 12.3. In general, an implicit conversion sequence (13.3.3.1) consists of a standard conversion sequence followed by a user-defined conversion followed by another standard conversion sequence. — *end note*]

8 [*Note:* There are some contexts where certain conversions are suppressed. For example, the lvalue-to-rvalue conversion is not done on the operand of the unary `&` operator. Specific exceptions are given in the descriptions of those operators and contexts. — *end note*]

4.1 Lvalue-to-rvalue conversion [conv.lval]

1 A glvalue (3.10) of a non-function, non-array type `T` can be converted to a prvalue.[55] If `T` is an incomplete type, a program that necessitates this conversion is ill-formed. If `T` is a non-class type, the type of the prvalue is the cv-unqualified version of `T`. Otherwise, the type of the prvalue is `T`.[56]

2 When an lvalue-to-rvalue conversion is applied to an expression `e`, and either

(2.1) — `e` is not potentially evaluated, or

(2.2) — the evaluation of `e` results in the evaluation of a member `ex` of the set of potential results of `e`, and `ex` names a variable `x` that is not odr-used by `ex` (3.2),

the value contained in the referenced object is not accessed. [*Example:*

```
struct S { int n; };
auto f() {
  S x { 1 };
  constexpr S y { 2 };
  return [&](bool b) { return (b ? y : x).n; };
}
auto g = f();
int m = g(false); // undefined behavior due to access of x.n outside its lifetime
int n = g(true);  // OK, does not access y.n
```

— *end example*] In all other cases, the result of the conversion is determined according to the following rules:

(2.3) — If `T` is (possibly cv-qualified) `std::nullptr_t`, the result is a null pointer constant (4.10).

(2.4) — Otherwise, if `T` has a class type, the conversion copy-initializes a temporary of type `T` from the glvalue and the result of the conversion is a prvalue for the temporary.

(2.5) — Otherwise, if the object to which the glvalue refers contains an invalid pointer value (3.7.4.2, 3.7.4.3), the behavior is implementation-defined.

(2.6) — Otherwise, the value contained in the object indicated by the glvalue is the prvalue result.

3 [*Note:* See also 3.10. — *end note*]

4.2 Array-to-pointer conversion [conv.array]

1 An lvalue or rvalue of type "array of `N T`" or "array of unknown bound of `T`" can be converted to a prvalue of type "pointer to `T`". The result is a pointer to the first element of the array.

4.3 Function-to-pointer conversion [conv.func]

1 An lvalue of function type `T` can be converted to a prvalue of type "pointer to `T`". The result is a pointer to the function.[57]

2 [*Note:* See 13.4 for additional rules for the case where the function is overloaded. — *end note*]

55) For historical reasons, this conversion is called the "lvalue-to-rvalue" conversion, even though that name does not accurately reflect the taxonomy of expressions described in 3.10.

56) In C++ class prvalues can have cv-qualified types (because they are objects). This differs from ISO C, in which non-lvalues never have cv-qualified types.

57) This conversion never applies to non-static member functions because an lvalue that refers to a non-static member function cannot be obtained.

4.4 Qualification conversions [conv.qual]

1 A *cv-decomposition* of a type `T` is a sequence of cv_i and P_i such that `T` is

"$cv_0\ P_0\ cv_1\ P_1 \cdots\ cv_{n-1}\ P_{n-1}\ cv_n$ `U`" for $n > 0$,

where each cv_i is a set of cv-qualifiers (3.9.3), and each P_i is "pointer to" (8.3.1), "pointer to member of class C_i of type" (8.3.3), "array of N_i", or "array of unknown bound of" (8.3.4). If P_i designates an array, the cv-qualifiers cv_{i+1} on the element type are also taken as the cv-qualifiers cv_i of the array. [*Example:* The type denoted by the *type-id* `const int **` has two cv-decompositions, taking `U` as "`int`" and as "pointer to `const int`". — *end example*] The n-tuple of cv-qualifiers after the first one in the longest cv-decomposition of `T`, that is, cv_1, cv_2, $\cdots$, cv_n, is called the *cv-qualification signature* of `T`.

2 Two types `T1` and `T2` are *similar* if they have cv-decompositions with the same n such that corresponding P_i components are the same and the types denoted by `U` are the same.

3 A prvalue expression of type `T1` can be converted to type `T2` if the following conditions are satisfied, where cv_i^j denotes the cv-qualifiers in the cv-qualification signature of T_j[58]:

(3.1) — `T1` and `T2` are similar.

(3.2) — For every $i > 0$, if `const` is in cv_i^1 then `const` is in cv_i^2, and similarly for `volatile`.

(3.3) — If the cv_i^1 and cv_i^2 are different, then `const` is in every cv_k^2 for $0 < k < i$.

[*Note:* if a program could assign a pointer of type `T**` to a pointer of type `const T**` (that is, if line `#1` below were allowed), a program could inadvertently modify a `const` object (as it is done on line `#2`). For example,

```
int main() {
  const char c = 'c';
  char* pc;
  const char** pcc = &pc;       // #1: not allowed
  *pcc = &c;
  *pc = 'C';                    // #2: modifies a const object
}
```

— *end note*]

4 [*Note:* A prvalue of type "pointer to *cv1* `T`" can be converted to a prvalue of type "pointer to *cv2* `T`" if "*cv2* `T`" is more cv-qualified than "*cv1* `T`". A prvalue of type "pointer to member of `X` of type *cv1* `T`" can be converted to a prvalue of type "pointer to member of `X` of type *cv2* `T`" if "*cv2* `T`" is more cv-qualified than "*cv1* `T`". — *end note*]

5 [*Note:* Function types (including those used in pointer to member function types) are never cv-qualified (8.3.5). — *end note*]

4.5 Integral promotions [conv.prom]

1 A prvalue of an integer type other than `bool`, `char16_t`, `char32_t`, or `wchar_t` whose integer conversion rank (4.13) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

2 A prvalue of type `char16_t`, `char32_t`, or `wchar_t` (3.9.1) can be converted to a prvalue of the first of the following types that can represent all the values of its underlying type: `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can

58) These rules ensure that const-safety is preserved by the conversion.

represent all the values of its underlying type, a prvalue of type `char16_t`, `char32_t`, or `wchar_t` can be converted to a prvalue of its underlying type.

3 A prvalue of an unscoped enumeration type whose underlying type is not fixed (7.2) can be converted to a prvalue of the first of the following types that can represent all the values of the enumeration (i.e., the values in the range b_{min} to b_{max} as described in 7.2): `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of the enumeration, a prvalue of an unscoped enumeration type can be converted to a prvalue of the extended integer type with lowest integer conversion rank (4.13) greater than the rank of `long long` in which all the values of the enumeration can be represented. If there are two such extended types, the signed one is chosen.

4 A prvalue of an unscoped enumeration type whose underlying type is fixed (7.2) can be converted to a prvalue of its underlying type. Moreover, if integral promotion can be applied to its underlying type, a prvalue of an unscoped enumeration type whose underlying type is fixed can also be converted to a prvalue of the promoted underlying type.

5 A prvalue for an integral bit-field (9.6) can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field. If the bit-field is larger yet, no integral promotion applies to it. If the bit-field has an enumerated type, it is treated as any other value of that type for promotion purposes.

6 A prvalue of type `bool` can be converted to a prvalue of type `int`, with `false` becoming zero and `true` becoming one.

7 These conversions are called *integral promotions*.

4.6 Floating point promotion [conv.fpprom]

1 A prvalue of type `float` can be converted to a prvalue of type `double`. The value is unchanged.

2 This conversion is called *floating point promotion*.

4.7 Integral conversions [conv.integral]

1 A prvalue of an integer type can be converted to a prvalue of another integer type. A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.

2 If the destination type is unsigned, the resulting value is the least unsigned integer congruent to the source integer (modulo 2^n where n is the number of bits used to represent the unsigned type). [*Note:* In a two's complement representation, this conversion is conceptual and there is no change in the bit pattern (if there is no truncation). — *end note*]

3 If the destination type is signed, the value is unchanged if it can be represented in the destination type; otherwise, the value is implementation-defined.

4 If the destination type is `bool`, see 4.12. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.

5 The conversions allowed as integral promotions are excluded from the set of integral conversions.

4.8 Floating point conversions [conv.double]

1 A prvalue of floating point type can be converted to a prvalue of another floating point type. If the source value can be exactly represented in the destination type, the result of the conversion is that exact representation. If the source value is between two adjacent destination values, the result of the conversion is an implementation-defined choice of either of those values. Otherwise, the behavior is undefined.

2 The conversions allowed as floating point promotions are excluded from the set of floating point conversions.

4.9 Floating-integral conversions [conv.fpint]

1 A prvalue of a floating point type can be converted to a prvalue of an integer type. The conversion truncates; that is, the fractional part is discarded. The behavior is undefined if the truncated value cannot be represented in the destination type. [*Note:* If the destination type is `bool`, see 4.12. — *end note*]

2 A prvalue of an integer type or of an unscoped enumeration type can be converted to a prvalue of a floating point type. The result is exact if possible. If the value being converted is in the range of values that can be represented but the value cannot be represented exactly, it is an implementation-defined choice of either the next lower or higher representable value. [*Note:* Loss of precision occurs if the integral value cannot be represented exactly as a value of the floating type. — *end note*] If the value being converted is outside the range of values that can be represented, the behavior is undefined. If the source type is `bool`, the value `false` is converted to zero and the value `true` is converted to one.

4.10 Pointer conversions [conv.ptr]

1 A *null pointer constant* is an integer literal (2.13.2) with value zero or a prvalue of type `std::nullptr_t`. A null pointer constant can be converted to a pointer type; the result is the *null pointer value* of that type and is distinguishable from every other value of object pointer or function pointer type. Such a conversion is called a *null pointer conversion*. Two null pointer values of the same type shall compare equal. The conversion of a null pointer constant to a pointer to cv-qualified type is a single conversion, and not the sequence of a pointer conversion followed by a qualification conversion (4.4). A null pointer constant of integral type can be converted to a prvalue of type `std::nullptr_t`. [*Note:* The resulting prvalue is not a null pointer value. — *end note*]

2 A prvalue of type "pointer to *cv* `T`", where `T` is an object type, can be converted to a prvalue of type "pointer to *cv* `void`". The result of converting a non-null pointer value of a pointer to object type to a "pointer to *cv* `void`" represents the address of the same byte in memory as the original pointer value. The null pointer value is converted to the null pointer value of the destination type.

3 A prvalue of type "pointer to *cv* `D`", where `D` is a class type, can be converted to a prvalue of type "pointer to *cv* `B`", where `B` is a base class (Clause 10) of `D`. If `B` is an inaccessible (Clause 11) or ambiguous (10.2) base class of `D`, a program that necessitates this conversion is ill-formed. The result of the conversion is a pointer to the base class subobject of the derived class object. The null pointer value is converted to the null pointer value of the destination type.

4.11 Pointer to member conversions [conv.mem]

1 A null pointer constant (4.10) can be converted to a pointer to member type; the result is the *null member pointer value* of that type and is distinguishable from any pointer to member not created from a null pointer constant. Such a conversion is called a *null member pointer conversion*. Two null member pointer values of the same type shall compare equal. The conversion of a null pointer constant to a pointer to member of cv-qualified type is a single conversion, and not the sequence of a pointer to member conversion followed by a qualification conversion (4.4).

2 A prvalue of type "pointer to member of `B` of type *cv* `T`", where `B` is a class type, can be converted to a prvalue of type "pointer to member of `D` of type *cv* `T`", where `D` is a derived class (Clause 10) of `B`. If `B` is an inaccessible (Clause 11), ambiguous (10.2), or virtual (10.1) base class of `D`, or a base class of a virtual base class of `D`, a program that necessitates this conversion is ill-formed. The result of the conversion refers to the same member as the pointer to member before the conversion took place, but it refers to the base class member as if it were a member of the derived class. The result refers to the member in `D`'s instance of `B`. Since the result has type "pointer to member of `D` of type *cv* `T`", indirection through it with a `D` object is valid. The result is the same as if indirecting through the pointer to member of `B` with the `B` subobject of `D`. The null member pointer value is converted to the null member pointer value of the destination type.[59]

59) The rule for conversion of pointers to members (from pointer to member of base to pointer to member of derived) appears

4.12 Boolean conversions [conv.bool]

1 A prvalue of arithmetic, unscoped enumeration, pointer, or pointer to member type can be converted to a prvalue of type `bool`. A zero value, null pointer value, or null member pointer value is converted to `false`; any other value is converted to `true`. For direct-initialization (8.5), a prvalue of type `std::nullptr_t` can be converted to a prvalue of type `bool`; the resulting value is `false`.

4.13 Integer conversion rank [conv.rank]

1 Every integer type has an *integer conversion rank* defined as follows:

(1.1) — No two signed integer types other than `char` and `signed char` (if `char` is signed) shall have the same rank, even if they have the same representation.

(1.2) — The rank of a signed integer type shall be greater than the rank of any signed integer type with a smaller size.

(1.3) — The rank of `long long int` shall be greater than the rank of `long int`, which shall be greater than the rank of `int`, which shall be greater than the rank of `short int`, which shall be greater than the rank of `signed char`.

(1.4) — The rank of any unsigned integer type shall equal the rank of the corresponding signed integer type.

(1.5) — The rank of any standard integer type shall be greater than the rank of any extended integer type with the same size.

(1.6) — The rank of `char` shall equal the rank of `signed char` and `unsigned char`.

(1.7) — The rank of `bool` shall be less than the rank of all other standard integer types.

(1.8) — The ranks of `char16_t`, `char32_t`, and `wchar_t` shall equal the ranks of their underlying types (3.9.1).

(1.9) — The rank of any extended signed integer type relative to another extended signed integer type with the same size is implementation-defined, but still subject to the other rules for determining the integer conversion rank.

(1.10) — For all integer types `T1`, `T2`, and `T3`, if `T1` has greater rank than `T2` and `T2` has greater rank than `T3`, then `T1` shall have greater rank than `T3`.

[*Note:* The integer conversion rank is used in the definition of the integral promotions (4.5) and the usual arithmetic conversions (Clause 5). — *end note*]

inverted compared to the rule for pointers to objects (from pointer to derived to pointer to base) (4.10, Clause 10). This inversion is necessary to ensure type safety. Note that a pointer to member is not an object pointer or a function pointer and the rules for conversions of such pointers do not apply to pointers to members. In particular, a pointer to member cannot be converted to a `void*`.

5 Expressions [expr]

1 [*Note:* Clause 5 defines the syntax, order of evaluation, and meaning of expressions.[60] An expression is a sequence of operators and operands that specifies a computation. An expression can result in a value and can cause side effects. — *end note*]

2 [*Note:* Operators can be overloaded, that is, given meaning when applied to expressions of class type (Clause 9) or enumeration type (7.2). Uses of overloaded operators are transformed into function calls as described in 13.5. Overloaded operators obey the rules for syntax specified in Clause 5, but the requirements of operand type, value category, and evaluation order are replaced by the rules for function call. Relations between operators, such as `++a` meaning `a+=1`, are not guaranteed for overloaded operators (13.5), and are not guaranteed for operands of type `bool`. — *end note*]

3 Clause 5 defines the effects of operators when applied to types for which they have not been overloaded. Operator overloading shall not modify the rules for the *built-in operators*, that is, for operators applied to types for which they are defined by this Standard. However, these built-in operators participate in overload resolution, and as part of that process user-defined conversions will be considered where necessary to convert the operands to types appropriate for the built-in operator. If a built-in operator is selected, such conversions will be applied to the operands before the operation is considered further according to the rules in Clause 5; see 13.3.1.2, 13.6.

4 If during the evaluation of an expression, the result is not mathematically defined or not in the range of representable values for its type, the behavior is undefined. [*Note:* most existing implementations of C++ ignore integer overflows. Treatment of division by zero, forming a remainder using a zero divisor, and all floating point exceptions vary among machines, and is usually adjustable by a library function. — *end note*]

5 If an expression initially has the type "reference to `T`" (8.3.2, 8.5.3), the type is adjusted to `T` prior to any further analysis. The expression designates the object or function denoted by the reference, and the expression is an lvalue or an xvalue, depending on the expression.

6 If a prvalue initially has the type "*cv* `T`", where `T` is a cv-unqualified non-class, non-array type, the type of the expression is adjusted to `T` prior to any further analysis.

7 [*Note:* An expression is an xvalue if it is:

(7.1) — the result of calling a function, whether implicitly or explicitly, whose return type is an rvalue reference to object type,

(7.2) — a cast to an rvalue reference to object type,

(7.3) — a class member access expression designating a non-static data member of non-reference type in which the object expression is an xvalue, or

(7.4) — a `.*` pointer-to-member expression in which the first operand is an xvalue and the second operand is a pointer to data member.

In general, the effect of this rule is that named rvalue references are treated as lvalues and unnamed rvalue references to objects are treated as xvalues; rvalue references to functions are treated as lvalues whether named or not. — *end note*]

[*Example:*

60) The precedence of operators is not directly specified, but it can be derived from the syntax.

```
struct A {
  int m;
};
A&& operator+(A, A);
A&& f();

A a;
A&& ar = static_cast<A&&>(a);
```

The expressions `f()`, `f().m`, `static_cast<A&&>(a)`, and `a + a` are xvalues. The expression `ar` is an lvalue. — *end example*]

8 In some contexts, *unevaluated operands* appear (5.2.8, 5.3.3, 5.3.7, 7.1.6.2). An unevaluated operand is not evaluated. An unevaluated operand is considered a full-expression. [*Note:* In an unevaluated operand, a non-static class member may be named (5.1) and naming of objects or functions does not, by itself, require that a definition be provided (3.2). — *end note*]

9 Whenever a glvalue expression appears as an operand of an operator that expects a prvalue for that operand, the lvalue-to-rvalue (4.1), array-to-pointer (4.2), or function-to-pointer (4.3) standard conversions are applied to convert the expression to a prvalue. [*Note:* because cv-qualifiers are removed from the type of an expression of non-class type when the expression is converted to a prvalue, an lvalue expression of type `const int` can, for example, be used where a prvalue expression of type `int` is required. — *end note*]

10 Many binary operators that expect operands of arithmetic or enumeration type cause conversions and yield result types in a similar way. The purpose is to yield a common type, which is also the type of the result. This pattern is called the *usual arithmetic conversions*, which are defined as follows:

(10.1) — If either operand is of scoped enumeration type (7.2), no conversions are performed; if the other operand does not have the same type, the expression is ill-formed.

(10.2) — If either operand is of type `long double`, the other shall be converted to `long double`.

(10.3) — Otherwise, if either operand is `double`, the other shall be converted to `double`.

(10.4) — Otherwise, if either operand is `float`, the other shall be converted to `float`.

(10.5) — Otherwise, the integral promotions (4.5) shall be performed on both operands.[61] Then the following rules shall be applied to the promoted operands:

(10.5.1) — If both operands have the same type, no further conversion is needed.

(10.5.2) — Otherwise, if both operands have signed integer types or both have unsigned integer types, the operand with the type of lesser integer conversion rank shall be converted to the type of the operand with greater rank.

(10.5.3) — Otherwise, if the operand that has unsigned integer type has rank greater than or equal to the rank of the type of the other operand, the operand with signed integer type shall be converted to the type of the operand with unsigned integer type.

(10.5.4) — Otherwise, if the type of the operand with signed integer type can represent all of the values of the type of the operand with unsigned integer type, the operand with unsigned integer type shall be converted to the type of the operand with signed integer type.

(10.5.5) — Otherwise, both operands shall be converted to the unsigned integer type corresponding to the type of the operand with signed integer type.

61) As a consequence, operands of type `bool`, `char16_t`, `char32_t`, `wchar_t`, or an enumerated type are converted to some integral type.

11 In some contexts, an expression only appears for its side effects. Such an expression is called a *discarded-value expression*. The expression is evaluated and its value is discarded. The array-to-pointer (4.2) and function-to-pointer (4.3) standard conversions are not applied. The lvalue-to-rvalue conversion (4.1) is applied if and only if the expression is a glvalue of volatile-qualified type and it is one of the following:

(11.1) — `(` *expression* `)`, where *expression* is one of these expressions,

(11.2) — *id-expression* (5.1.1),

(11.3) — subscripting (5.2.1),

(11.4) — class member access (5.2.5),

(11.5) — indirection (5.3.1),

(11.6) — pointer-to-member operation (5.5),

(11.7) — conditional expression (5.16) where both the second and the third operands are one of these expressions, or

(11.8) — comma expression (5.19) where the right operand is one of these expressions.

[*Note:* Using an overloaded operator causes a function call; the above covers only operators with built-in meaning. If the lvalue is of class type, it must have a volatile copy constructor to initialize the temporary that is the result of the lvalue-to-rvalue conversion. — *end note*]

12 The values of the floating operands and the results of floating expressions may be represented in greater precision and range than that required by the type; the types are not changed thereby.[62]

13 The *cv-combined type* of two types `T1` and `T2` is a type `T3` similar to `T1` whose cv-qualification signature (4.4) is:

(13.1) — for every $j > 0$, $cv_{3,j}$ is the union of $cv_{1,j}$ and $cv_{2,j}$;

(13.2) — if the resulting $cv_{3,j}$ is different from $cv_{1,j}$ or $cv_{2,j}$, then `const` is added to every $cv_{3,k}$ for $0 < k < j$.

[*Note:* Given similar types `T1` and `T2`, this construction ensures that both can be converted to `T3`. — *end note*] The *composite pointer type* of two operands `p1` and `p2` having types `T1` and `T2`, respectively, where at least one is a pointer or pointer to member type or `std::nullptr_t`, is:

(13.3) — if both `p1` and `p2` are null pointer constants, `std::nullptr_t`;

(13.4) — if either `p1` or `p2` is a null pointer constant, `T2` or `T1`, respectively;

(13.5) — if `T1` or `T2` is "pointer to *cv1* `void`" and the other type is "pointer to *cv2* T", "pointer to *cv12* `void`", where *cv12* is the union of *cv1* and *cv2*;

(13.6) — if `T1` is "pointer to *cv1* `C1`" and `T2` is "pointer to *cv2* `C2`", where `C1` is reference-related to `C2` or `C2` is reference-related to `C1` (8.5.3), the cv-combined type of `T1` and `T2` or the cv-combined type of `T2` and `T1`, respectively;

(13.7) — if `T1` is "pointer to member of `C1` of type *cv1* `U1`" and `T2` is "pointer to member of `C2` of type *cv2* `U2`" where `C1` is reference-related to `C2` or `C2` is reference-related to `C1` (8.5.3), the cv-combined type of `T2` and `T1` or the cv-combined type of `T1` and `T2`, respectively;

(13.8) — if `T1` and `T2` are similar types (4.4), the cv-combined type of `T1` and `T2`;

62) The cast and assignment operators must still perform their specific conversions as described in 5.4, 5.2.9 and 5.18.

(13.9) — otherwise, a program that necessitates the determination of a composite pointer type is ill-formed.

[*Example:*

```
typedef void *p;
typedef const int *q;
typedef int **pi;
typedef const int **pci;
```

The composite pointer type of `p` and `q` is "pointer to `const void`"; the composite pointer type of `pi` and `pci` is "pointer to `const` pointer to `const int`". — *end example*]

5.1 Primary expressions [expr.prim]

5.1.1 General [expr.prim.general]

primary-expression:
literal
`this`
`(` *expression* `)`
id-expression
lambda-expression
fold-expression

id-expression:
unqualified-id
qualified-id

unqualified-id:
identifier
operator-function-id
conversion-function-id
literal-operator-id
`~` *class-name*
`~` *decltype-specifier*
template-id

1 A *literal* is a primary expression. Its type depends on its form (2.13). A string literal is an lvalue; all other literals are prvalues.

2 The keyword `this` names a pointer to the object for which a non-static member function (9.3.2) is invoked or a non-static data member's initializer (9.2) is evaluated.

3 If a declaration declares a member function or member function template of a class `X`, the expression `this` is a prvalue of type "pointer to *cv-qualifier-seq* `X`" between the optional *cv-qualifer-seq* and the end of the *function-definition*, *member-declarator*, or *declarator*. It shall not appear before the optional *cv-qualifier-seq* and it shall not appear within the declaration of a static member function (although its type and value category are defined within a static member function as they are within a non-static member function). [*Note:* this is because declaration matching does not occur until the complete declarator is known. — *end note*] Unlike the object expression in other contexts, `*this` is not required to be of complete type for purposes of class member access (5.2.5) outside the member function body. [*Note:* only class members declared prior to the declaration are visible. — *end note*] [*Example:*

```
struct A {
  char g();
  template<class T> auto f(T t) -> decltype(t + g())
    { return t + g(); }
};
template auto A::f(int t) -> decltype(t + g());
```

— *end example*]

4 Otherwise, if a *member-declarator* declares a non-static data member (9.2) of a class `X`, the expression `this` is a prvalue of type "pointer to `X`" within the optional *brace-or-equal-initializer*. It shall not appear elsewhere in the *member-declarator*.

5 The expression `this` shall not appear in any other context. [*Example:*

```
class Outer {
  int a[sizeof(*this)];                // error: not inside a member function
  unsigned int sz = sizeof(*this);     // OK: in brace-or-equal-initializer

  void f() {
    int b[sizeof(*this)];              // OK

    struct Inner {
      int c[sizeof(*this)];            // error: not inside a member function of Inner
    };
  }
};
```

— *end example*]

6 A parenthesized expression is a primary expression whose type and value are identical to those of the enclosed expression. The presence of parentheses does not affect whether the expression is an lvalue. The parenthesized expression can be used in exactly the same contexts as those where the enclosed expression can be used, and with the same meaning, except as otherwise indicated.

7 An *id-expression* is a restricted form of a *primary-expression*. [*Note:* an *id-expression* can appear after `.` and `->` operators (5.2.5). — *end note*]

8 An *identifier* is an *id-expression* provided it has been suitably declared (Clause 7). [*Note:* for *operator-function-id*s, see 13.5; for *conversion-function-id*s, see 12.3.2; for *literal-operator-id*s, see 13.5.8; for *template-id*s, see 14.2. A *class-name* or *decltype-specifier* prefixed by `~` denotes a destructor; see 12.4. Within the definition of a non-static member function, an *identifier* that names a non-static member is transformed to a class member access expression (9.3.1). — *end note*] The type of the expression is the type of the *identifier*. The result is the entity denoted by the identifier. The result is an lvalue if the entity is a function, variable, or data member and a prvalue otherwise.

qualified-id:
nested-name-specifier `template`$_{opt}$ *unqualified-id*
nested-name-specifier:
`::`
type-name `::`
namespace-name `::`
decltype-specifier `::`
nested-name-specifier identifier `::`
nested-name-specifier `template`$_{opt}$ *simple-template-id* `::`

The type denoted by a *decltype-specifier* in a *nested-name-specifier* shall be a class or enumeration type.

9 A *nested-name-specifier* that denotes a class, optionally followed by the keyword `template` (14.2), and then followed by the name of a member of either that class (9.2) or one of its base classes (Clause 10), is a *qualified-id*; 3.4.3.1 describes name lookup for class members that appear in *qualified-ids*. The result is the member. The type of the result is the type of the member. The result is an lvalue if the member is a static member function or a data member and a prvalue otherwise. [*Note:* a class member can be referred to using a *qualified-id* at any point in its potential scope (3.3.7). — *end note*] Where *class-name* `::~` *class-name* is used, the two *class-name*s shall refer to the same class; this notation names the destructor (12.4). The form `~` *decltype-specifier* also denotes the destructor, but it shall not be used as the *unqualified-id* in a *qualified-id*. [*Note:* a *typedef-name* that names a class is a *class-name* (9.1). — *end note*]

10 The *nested-name-specifier* `::` names the global namespace. A *nested-name-specifier* that names a namespace (7.3), optionally followed by the keyword `template` (14.2), and then followed by the name of a member of that namespace (or the name of a member of a namespace made visible by a *using-directive*), is a *qualified-id*; 3.4.3.2 describes name lookup for namespace members that appear in *qualified-ids*. The result is the member. The type of the result is the type of the member. The result is an lvalue if the member is a function or a variable and a prvalue otherwise.

11 A *nested-name-specifier* that denotes an enumeration (7.2), followed by the name of an enumerator of that enumeration, is a *qualified-id* that refers to the enumerator. The result is the enumerator. The type of the result is the type of the enumeration. The result is a prvalue.

12 In a *qualified-id*, if the *unqualified-id* is a *conversion-function-id*, its *conversion-type-id* shall denote the same type in both the context in which the entire *qualified-id* occurs and in the context of the class denoted by the *nested-name-specifier*.

13 An *id-expression* that denotes a non-static data member or non-static member function of a class can only be used:

(13.1) — as part of a class member access (5.2.5) in which the object expression refers to the member's class[63] or a class derived from that class, or

(13.2) — to form a pointer to member (5.3.1), or

(13.3) — if that *id-expression* denotes a non-static data member and it appears in an unevaluated operand. [*Example:*

```
struct S {
  int m;
};
int i = sizeof(S::m);           // OK
int j = sizeof(S::m + 42);      // OK
```

— *end example*]

63) This also applies when the object expression is an implicit `(*this)` (9.3.1).

5.1.2 Lambda expressions [expr.prim.lambda]

1 Lambda expressions provide a concise way to create simple function objects. [*Example:*

```
#include <algorithm>
#include <cmath>
void abssort(float* x, unsigned N) {
  std::sort(x, x + N,
    [](float a, float b) {
      return std::abs(a) < std::abs(b);
    });
}
```

— *end example*]

lambda-expression:
lambda-introducer *lambda-declarator*$_{opt}$ *compound-statement*

lambda-introducer:
`[` *lambda-capture*$_{opt}$ `]`

lambda-capture:
capture-default
capture-list
capture-default `,` *capture-list*

capture-default:
`&`
`=`

capture-list:
capture `...`$_{opt}$
capture-list `,` *capture* `...`$_{opt}$

capture:
simple-capture
init-capture

simple-capture:
identifier
`&` *identifier*
`this`

init-capture:
identifier *initializer*
`&` *identifier* *initializer*

lambda-declarator:
`(` *parameter-declaration-clause* `)` `mutable`$_{opt}$
exception-specification$_{opt}$ *attribute-specifier-seq*$_{opt}$ *trailing-return-type*$_{opt}$

2 The evaluation of a *lambda-expression* results in a prvalue temporary (12.2). This temporary is called the *closure object*. A *lambda-expression* shall not appear in an unevaluated operand (Clause 5), in a *template-argument*, in an *alias-declaration*, in a typedef declaration, or in the declaration of a function or function template outside its function body and default arguments. [*Note:* The intention is to prevent lambdas from appearing in a signature. — *end note*] [*Note:* A closure object behaves like a function object (20.9). — *end note*]

3 The type of the *lambda-expression* (which is also the type of the closure object) is a unique, unnamed non-union class type — called the *closure type* — whose properties are described below. This class type is neither an aggregate (8.5.1) nor a literal type (3.9). The closure type is declared in the smallest block scope, class scope, or namespace scope that contains the corresponding *lambda-expression*. [*Note:* This determines the set of namespaces and classes associated with the closure type (3.4.2). The parameter types of a *lambda-declarator* do not affect these associated namespaces and classes. — *end note*] An implementation may

define the closure type differently from what is described below provided this does not alter the observable behavior of the program other than by changing:

(3.1) — the size and/or alignment of the closure type,

(3.2) — whether the closure type is trivially copyable (Clause 9),

(3.3) — whether the closure type is a standard-layout class (Clause 9), or

(3.4) — whether the closure type is a POD class (Clause 9).

An implementation shall not add members of rvalue reference type to the closure type.

4 If a *lambda-expression* does not include a *lambda-declarator*, it is as if the *lambda-declarator* were `()`. The lambda return type is `auto`, which is replaced by the type specified by the *trailing-return-type* if provided and/or deduced from `return` statements as described in 7.1.6.4. [*Example:*

```
auto x1 = [](int i){ return i; };      // OK: return type is int
auto x2 = []{ return { 1, 2 }; };      // error: deducing return type from braced-init-list
int j;
auto x3 = []()->auto&& { return j; }; // OK: return type is int&
```

— *end example*]

5 The closure type for a non-generic *lambda-expression* has a public inline function call operator (13.5.4) whose parameters and return type are described by the *lambda-expression*'s *parameter-declaration-clause* and *trailing-return-type* respectively. For a generic lambda, the closure type has a public inline function call operator member template (14.5.2) whose *template-parameter-list* consists of one invented type *template-parameter* for each occurrence of `auto` in the lambda's *parameter-declaration-clause*, in order of appearance. The invented type *template-parameter* is a parameter pack if the corresponding *parameter-declaration* declares a function parameter pack (8.3.5). The return type and function parameters of the function call operator template are derived from the *lambda-expression*'s *trailing-return-type* and *parameter-declaration-clause* by replacing each occurrence of `auto` in the *decl-specifier*s of the *parameter-declaration-clause* with the name of the corresponding invented *template-parameter*. [*Example:*

```
auto glambda = [](auto a, auto&& b) { return a < b; };
bool b = glambda(3, 3.14);                                  // OK
auto vglambda = [](auto printer) {
   return [=](auto&& ... ts) {                              // OK: ts is a function parameter pack
       printer(std::forward<decltype(ts)>(ts)...);

       return [=]() {
         printer(ts ...);
       };
   };
};
auto p = vglambda( [](auto v1, auto v2, auto v3)
                     { std::cout << v1 << v2 << v3; } );
auto q = p(1, 'a', 3.14);                                   // OK: outputs 1a3.14
q();                                                        // OK: outputs 1a3.14
```

— *end example*] This function call operator or operator template is declared `const` (9.3.1) if and only if the *lambda-expression*'s *parameter-declaration-clause* is not followed by `mutable`. It is neither virtual nor declared `volatile`. Any *exception-specification* specified on a *lambda-expression* applies to the corresponding function call operator or operator template. An *attribute-specifier-seq* in a *lambda-declarator* appertains to the type of the corresponding function call operator or operator template. [*Note:* Names referenced in the *lambda-declarator* are looked up in the context in which the *lambda-expression* appears. — *end note*]

6 The closure type for a non-generic *lambda-expression* with no *lambda-capture* has a public non-virtual non-explicit const conversion function to pointer to function with C++ language linkage (7.5) having the same parameter and return types as the closure type's function call operator. The value returned by this conversion function shall be the address of a function that, when invoked, has the same effect as invoking the closure type's function call operator. For a generic lambda with no *lambda-capture*, the closure type has a public non-virtual non-explicit const conversion function template to pointer to function. The conversion function template has the same invented *template-parameter-list*, and the pointer to function has the same parameter types, as the function call operator template. The return type of the pointer to function shall behave as if it were a *decltype-specifier* denoting the return type of the corresponding function call operator template specialization. [*Note:* If the generic lambda has no *trailing-return-type* or the *trailing-return-type* contains a placeholder type, return type deduction of the corresponding function call operator template specialization has to be done. The corresponding specialization is that instantiation of the function call operator template with the same template arguments as those deduced for the conversion function template. Consider the following:

```
auto glambda = [](auto a) { return a; };
int (*fp)(int) = glambda;
```

The behavior of the conversion function of `glambda` above is like that of the following conversion function:

```
struct Closure {
  template<class T> auto operator()(T t) const { ... }
  template<class T> static auto lambda_call_operator_invoker(T a) {
    // forwards execution to operator()(a) and therefore has
    // the same return type deduced
    ...
  }
  template<class T> using fptr_t =
     decltype(lambda_call_operator_invoker(declval<T>())) (*)(T);

  template<class T> operator fptr_t<T>() const
    { return &lambda_call_operator_invoker; }
};
```

— *end note*] [*Example:*

```
void f1(int (*)(int))   { }
void f2(char (*)(int))  { }

void g(int (*)(int))    { }  // #1
void g(char (*)(char))  { }  // #2

void h(int (*)(int))    { }  // #3
void h(char (*)(int))   { }  // #4

auto glambda = [](auto a) { return a; };
f1(glambda);  // OK
f2(glambda);  // error: ID is not convertible
g(glambda);   // error: ambiguous
h(glambda);   // OK: calls #3 since it is convertible from ID
int& (*fpi)(int*) = [](auto* a) -> auto& { return *a; }; // OK
```

— *end example*] The value returned by any given specialization of this conversion function template shall be the address of a function that, when invoked, has the same effect as invoking the generic lambda's corresponding function call operator template specialization. [*Note:* This will result in the implicit instantiation

of the generic lambda's body. The instantiated generic lambda's return type and parameter types shall match the return type and parameter types of the pointer to function. — *end note*] [*Example:*

```
auto GL = [](auto a) { std::cout << a; return a; };
int (*GL_int)(int) = GL;  // OK: through conversion function template
GL_int(3);                // OK: same as GL(3)
```

— *end example*]

7 The *lambda-expression*'s *compound-statement* yields the *function-body* (8.4) of the function call operator, but for purposes of name lookup (3.4), determining the type and value of `this` (9.3.2) and transforming *id-expressions* referring to non-static class members into class member access expressions using `(*this)` (9.3.1), the *compound-statement* is considered in the context of the *lambda-expression*. [*Example:*

```
struct S1 {
  int x, y;
  int operator()(int);
  void f() {
    [=]()->int {
      return operator()(this->x + y); // equivalent to S1::operator()(this->x + (*this).y)
                                      // this has type S1*
    };
  }
};
```

— *end example*] Further, a variable `__func__` is implicitly defined at the beginning of the *compound-statement* of the *lambda-expression*, with semantics as described in 8.4.1.

8 If a *lambda-capture* includes a *capture-default* that is `&`, no identifier in a *simple-capture* of that *lambda-capture* shall be preceded by `&`. If a *lambda-capture* includes a *capture-default* that is `=`, each *simple-capture* of that *lambda-capture* shall be of the form "`&` *identifier*". Ignoring appearances in *initializer*s of *init-capture*s, an identifier or `this` shall not appear more than once in a *lambda-capture*. [*Example:*

```
struct S2 { void f(int i); };
void S2::f(int i) {
  [&, i]{ };    // OK
  [&, &i]{ };   // error: i preceded by & when & is the default
  [=, this]{ }; // error: this when = is the default
  [i, i]{ };    // error: i repeated
}
```

— *end example*]

9 A *lambda-expression* whose smallest enclosing scope is a block scope (3.3.3) is a *local lambda expression*; any other *lambda-expression* shall not have a *capture-default* or *simple-capture* in its *lambda-introducer*. The *reaching scope* of a local lambda expression is the set of enclosing scopes up to and including the innermost enclosing function and its parameters. [*Note:* This reaching scope includes any intervening *lambda-expressions*. — *end note*]

10 The *identifier* in a *simple-capture* is looked up using the usual rules for unqualified name lookup (3.4.1); each such lookup shall find an entity. An entity that is designated by a *simple-capture* is said to be *explicitly captured*, and shall be `this` or a variable with automatic storage duration declared in the reaching scope of the local lambda expression.

11 An *init-capture* behaves as if it declares and explicitly captures a variable of the form "`auto` *init-capture* `;`" whose declarative region is the *lambda-expression*'s *compound-statement*, except that:

(11.1) — if the capture is by copy (see below), the non-static data member declared for the capture and the variable are treated as two different ways of referring to the same object, which has the lifetime of the non-static data member, and no additional copy and destruction is performed, and

(11.2) — if the capture is by reference, the variable's lifetime ends when the closure object's lifetime ends.

[*Note:* This enables an *init-capture* like "`x = std::move(x)`"; the second "`x`" must bind to a declaration in the surrounding context. — *end note*] [*Example:*

```
int x = 4;
auto y = [&r = x, x = x+1]()->int {
            r += 2;
            return x+2;
         }();  // Updates ::x to 6, and initializes y to 7.
```

— *end example*]

12 A *lambda-expression* with an associated *capture-default* that does not explicitly capture `this` or a variable with automatic storage duration (this excludes any *id-expression* that has been found to refer to an *init-capture*'s associated non-static data member), is said to *implicitly capture* the entity (i.e., `this` or a variable) if the *compound-statement*:

(12.1) — odr-uses (3.2) the entity, or

(12.2) — names the entity in a potentially-evaluated expression (3.2) where the enclosing full-expression depends on a generic lambda parameter declared within the reaching scope of the *lambda-expression*.

[*Example:*

```
void f(int, const int (&)[2] = {})    { }   // #1
void f(const int&, const int (&)[1])  { }   // #2
void test() {
  const int x = 17;
  auto g = [](auto a) {
    f(x);  // OK: calls #1, does not capture x
  };

  auto g2 = [=](auto a) {
    int selector[sizeof(a) == 1 ? 1 : 2]{};
    f(x, selector);  // OK: is a dependent expression, so captures x
  };
}
```

— *end example*] All such implicitly captured entities shall be declared within the reaching scope of the lambda expression. [*Note:* The implicit capture of an entity by a nested *lambda-expression* can cause its implicit capture by the containing *lambda-expression* (see below). Implicit odr-uses of `this` can result in implicit capture. — *end note*]

13 An entity is *captured* if it is captured explicitly or implicitly. An entity captured by a *lambda-expression* is odr-used (3.2) in the scope containing the *lambda-expression*. If `this` is captured by a local lambda expression, its nearest enclosing function shall be a non-static member function. If a *lambda-expression* or an instantiation of the function call operator template of a generic lambda odr-uses (3.2) `this` or a variable with automatic storage duration from its reaching scope, that entity shall be captured by the *lambda-expression*. If a *lambda-expression* captures an entity and that entity is not defined or captured in the immediately enclosing lambda expression or function, the program is ill-formed. [*Example:*

```
void f1(int i) {
  int const N = 20;
  auto m1 = [=]{
    int const M = 30;
    auto m2 = [i]{
      int x[N][M];              // OK: N and M are not odr-used
      x[0][0] = i;              // OK: i is explicitly captured by m2
                                // and implicitly captured by m1
    };
  };
  struct s1 {
    int f;
    void work(int n) {
      int m = n*n;
      int j = 40;
      auto m3 = [this,m] {
        auto m4 = [&,j] {       // error: j not captured by m3
          int x = n;            // error: n implicitly captured by m4
                                // but not captured by m3
          x += m;               // OK: m implicitly captured by m4
                                // and explicitly captured by m3
          x += i;               // error: i is outside of the reaching scope
          x += f;               // OK: this captured implicitly by m4
                                // and explicitly by m3
        };
      };
    }
  };
}
```

— *end example*]

14 A *lambda-expression* appearing in a default argument shall not implicitly or explicitly capture any entity. [*Example:*

```
void f2() {
  int i = 1;
  void g1(int = ([i]{ return i; })());        // ill-formed
  void g2(int = ([i]{ return 0; })());        // ill-formed
  void g3(int = ([=]{ return i; })());        // ill-formed
  void g4(int = ([=]{ return 0; })());        // OK
  void g5(int = ([]{ return sizeof i; })());  // OK
}
```

— *end example*]

15 An entity is *captured by copy* if it is implicitly captured and the *capture-default* is `=` or if it is explicitly captured with a capture that is not of the form `&` *identifier* or `&` *identifier initializer*. For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The declaration order of these members is unspecified. The type of such a data member is the type of the corresponding captured entity if the entity is not a reference to an object, or the referenced type otherwise. [*Note:* If the captured entity is a reference to a function, the corresponding data member is also a reference to a function. — *end note*] A member of an anonymous union shall not be captured by copy.

16 An entity is *captured by reference* if it is implicitly or explicitly captured but not captured by copy. It is unspecified whether additional unnamed non-static data members are declared in the closure type for entities captured by reference. A member of an anonymous union shall not be captured by reference.

17 If a *lambda-expression* `m2` captures an entity and that entity is captured by an immediately enclosing *lambda-expression* `m1`, then `m2`'s capture is transformed as follows:

(17.1) — if `m1` captures the entity by copy, `m2` captures the corresponding non-static data member of `m1`'s closure type;

(17.2) — if `m1` captures the entity by reference, `m2` captures the same entity captured by `m1`.

[*Example:* the nested lambda expressions and invocations below will output `123234`.

```
int a = 1, b = 1, c = 1;
auto m1 = [a, &b, &c]() mutable {
  auto m2 = [a, b, &c]() mutable {
    std::cout << a << b << c;
    a = 4; b = 4; c = 4;
  };
  a = 3; b = 3; c = 3;
  m2();
};
a = 2; b = 2; c = 2;
m1();
std::cout << a << b << c;
```

— *end example*]

18 Every *id-expression* within the *compound-statement* of a *lambda-expression* that is an odr-use (3.2) of an entity captured by copy is transformed into an access to the corresponding unnamed data member of the closure type. [*Note:* An *id-expression* that is not an odr-use refers to the original entity, never to a member of the closure type. Furthermore, such an *id-expression* does not cause the implicit capture of the entity. — *end note*] If `this` is captured, each odr-use of `this` is transformed into an access to the corresponding unnamed data member of the closure type, cast (5.4) to the type of `this`. [*Note:* The cast ensures that the transformed expression is a prvalue. — *end note*] [*Example:*

```
void f(const int*);
void g() {
  const int N = 10;
  [=] {
    int arr[N];              // OK: not an odr-use, refers to automatic variable
    f(&N);                   // OK: causes N to be captured; &N points to the
                             // corresponding member of the closure type
  };
}
```

— *end example*]

19 Every occurrence of `decltype((x))` where `x` is a possibly parenthesized *id-expression* that names an entity of automatic storage duration is treated as if `x` were transformed into an access to a corresponding data member of the closure type that would have been declared if `x` were an odr-use of the denoted entity. [*Example:*

```
void f3() {
  float x, &r = x;
  [=] {                      // x and r are not captured (appearance in a decltype operand is not an odr-use)
    decltype(x) y1;          // y1 has type float
    decltype((x)) y2 = y1;   // y2 has type float const& because this lambda
                             // is not mutable and x is an lvalue
    decltype(r) r1 = y1;     // r1 has type float& (transformation not considered)
    decltype((r)) r2 = y2;   // r2 has type float const&
```

```
  };
}
```

— *end example*]

20 The closure type associated with a *lambda-expression* has no default constructor and a deleted copy assignment operator. It has a defaulted copy constructor and a defaulted move constructor (12.8). [*Note:* These special member functions are implicitly defined as usual, and might therefore be defined as deleted. — *end note*]

21 The closure type associated with a *lambda-expression* has an implicitly-declared destructor (12.4).

22 A member of a closure type shall not be explicitly instantiated (14.7.2), explicitly specialized (14.7.3), or named in a `friend` declaration (11.3).

23 When the *lambda-expression* is evaluated, the entities that are captured by copy are used to direct-initialize each corresponding non-static data member of the resulting closure object, and the non-static data members corresponding to the *init-capture*s are initialized as indicated by the corresponding *initializer* (which may be copy- or direct-initialization). (For array members, the array elements are direct-initialized in increasing subscript order.) These initializations are performed in the (unspecified) order in which the non-static data members are declared. [*Note:* This ensures that the destructions will occur in the reverse order of the constructions. — *end note*]

24 [*Note:* If an entity is implicitly or explicitly captured by reference, invoking the function call operator of the corresponding *lambda-expression* after the lifetime of the entity has ended is likely to result in undefined behavior. — *end note*]

25 A *simple-capture* followed by an ellipsis is a pack expansion (14.5.3). An *init-capture* followed by an ellipsis is ill-formed. [*Example:*

```
template<class... Args>
void f(Args... args) {
  auto lm = [&, args...] { return g(args...); };
  lm();
}
```

— *end example*]

5.1.3 Fold expressions [expr.prim.fold]

1 A fold expression performs a fold of a template parameter pack (14.5.3) over a binary operator.

fold-expression:
`(` *cast-expression fold-operator* `...` `)`
`(` `...` *fold-operator cast-expression* `)`
`(` *cast-expression fold-operator* `...` *fold-operator cast-expression* `)`

fold-operator: one of

```
+    -    *    /    %    ^    &    |    <<   >>
+=   -=   *=   /=   %=   ^=   &=   |=   <<=  >>=  =
==   !=   <    >    <=   >=   &&   ||   ,    .*   ->*
```

2 An expression of the form `(...` *op* `e)` where *op* is a *fold-operator* is called a *unary left fold*. An expression of the form `(e` *op* `...)` where *op* is a *fold-operator* is called a *unary right fold*. Unary left folds and unary right folds are collectively called *unary folds*. In a unary fold, the *cast-expression* shall contain an unexpanded parameter pack (14.5.3).

3 An expression of the form `(e1` *op1* `...` *op2* `e2)` where *op1* and *op2* are *fold-operator*s is called a *binary fold*. In a binary fold, *op1* and *op2* shall be the same *fold-operator*, and either `e1` shall contain an unexpanded parameter pack or `e2` shall contain an unexpanded parameter pack, but not both. If `e2` contains

an unexpanded parameter pack, the expression is called a *binary left fold*. If `e1` contains an unexpanded parameter pack, the expression is called a *binary right fold*. [*Example:*

```
template<typename ...Args>
bool f(Args ...args) {
  return (true && ... && args); // OK
}

template<typename ...Args>
bool f(Args ...args) {
  return (args + ... + args); // error: both operands contain unexpanded parameter packs
}
```

— *end example*]

5.2 Postfix expressions [expr.post]

1 Postfix expressions group left-to-right.

postfix-expression:
primary-expression
postfix-expression `[` *expression* `]`
postfix-expression `[` *braced-init-list* `]`
postfix-expression `(` *expression-list*$_{opt}$ `)`
simple-type-specifier `(` *expression-list*$_{opt}$ `)`
typename-specifier `(` *expression-list*$_{opt}$ `)`
simple-type-specifier braced-init-list
typename-specifier braced-init-list
postfix-expression `.` `template`$_{opt}$ *id-expression*
postfix-expression `->` `template`$_{opt}$ *id-expression*
postfix-expression `.` *pseudo-destructor-name*
postfix-expression `->` *pseudo-destructor-name*
postfix-expression `++`
postfix-expression `--`
`dynamic_cast` `<` *type-id* `>` `(` *expression* `)`
`static_cast` `<` *type-id* `>` `(` *expression* `)`
`reinterpret_cast` `<` *type-id* `>` `(` *expression* `)`
`const_cast` `<` *type-id* `>` `(` *expression* `)`
`typeid` `(` *expression* `)`
`typeid` `(` *type-id* `)`

expression-list:
initializer-list

pseudo-destructor-name:
nested-name-specifier$_{opt}$ *type-name* `::` `~` *type-name*
nested-name-specifier `template` *simple-template-id* `::` `~` *type-name*
`~` *type-name*
`~` *decltype-specifier*

2 [*Note:* The `>` token following the *type-id* in a `dynamic_cast`, `static_cast`, `reinterpret_cast`, or `const_cast` may be the product of replacing a `>>` token by two consecutive `>` tokens (14.2). — *end note*]

5.2.1 Subscripting [expr.sub]

1 A postfix expression followed by an expression in square brackets is a postfix expression. One of the expressions shall have the type "array of `T`" or "pointer to `T`" and the other shall have unscoped enumeration or integral type. The result is of type "`T`". The type "`T`" shall be a completely-defined object type.[64] The

64) This is true even if the subscript operator is used in the following common idiom: `&x[0]`.

expression `E1[E2]` is identical (by definition) to `*((E1)+(E2))` [*Note:* see 5.3 and 5.7 for details of `*` and `+` and 8.3.4 for details of arrays. — *end note*], except that in the case of an array operand, the result is an lvalue if that operand is an lvalue and an xvalue otherwise.

2 A *braced-init-list* shall not be used with the built-in subscript operator.

5.2.2 Function call [expr.call]

1 A function call is a postfix expression followed by parentheses containing a possibly empty, comma-separated list of *initializer-clause*s which constitute the arguments to the function. The postfix expression shall have function type or pointer to function type. For a call to a non-member function or to a static member function, the postfix expression shall be either an lvalue that refers to a function (in which case the function-to-pointer standard conversion (4.3) is suppressed on the postfix expression), or it shall have pointer to function type. Calling a function through an expression whose function type has a language linkage that is different from the language linkage of the function type of the called function's definition is undefined (7.5). For a call to a non-static member function, the postfix expression shall be an implicit (9.3.1, 9.4) or explicit class member access (5.2.5) whose *id-expression* is a function member name, or a pointer-to-member expression (5.5) selecting a function member; the call is as a member of the class object referred to by the object expression. In the case of an implicit class member access, the implied object is the one pointed to by `this`. [*Note:* a member function call of the form `f()` is interpreted as `(*this).f()` (see 9.3.1). — *end note*] If a function or member function name is used, the name can be overloaded (Clause 13), in which case the appropriate function shall be selected according to the rules in 13.3. If the selected function is non-virtual, or if the *id-expression* in the class member access expression is a *qualified-id*, that function is called. Otherwise, its final overrider (10.3) in the dynamic type of the object expression is called; such a call is referred to as a *virtual function call*. [*Note:* the dynamic type is the type of the object referred to by the current value of the object expression. 12.7 describes the behavior of virtual function calls when the object expression refers to an object under construction or destruction. — *end note*]

2 [*Note:* If a function or member function name is used, and name lookup (3.4) does not find a declaration of that name, the program is ill-formed. No function is implicitly declared by such a call. — *end note*]

3 If the *postfix-expression* designates a destructor (12.4), the type of the function call expression is `void`; otherwise, the type of the function call expression is the return type of the statically chosen function (i.e., ignoring the `virtual` keyword), even if the type of the function actually called is different. This return type shall be an object type, a reference type or *cv* `void`.

4 When a function is called, each parameter (8.3.5) shall be initialized (8.5, 12.8, 12.1) with its corresponding argument. [*Note:* Such initializations are indeterminately sequenced with respect to each other (1.9) — *end note*] If the function is a non-static member function, the `this` parameter of the function (9.3.2) shall be initialized with a pointer to the object of the call, converted as if by an explicit type conversion (5.4). [*Note:* There is no access or ambiguity checking on this conversion; the access checking and disambiguation are done as part of the (possibly implicit) class member access operator. See 10.2, 11.2, and 5.2.5. — *end note*] When a function is called, the parameters that have object type shall have completely-defined object type. [*Note:* this still allows a parameter to be a pointer or reference to an incomplete class type. However, it prevents a passed-by-value parameter to have an incomplete class type. — *end note*] During the initialization of a parameter, an implementation may avoid the construction of extra temporaries by combining the conversions on the associated argument and/or the construction of temporaries with the initialization of the parameter (see 12.2). The lifetime of a parameter ends when the function in which it is defined returns. The initialization and destruction of each parameter occurs within the context of the calling function. [*Example:* the access of the constructor, conversion functions or destructor is checked at the point of call in the calling function. If a constructor or destructor for a function parameter throws an exception, the search for a handler starts in the scope of the calling function; in particular, if the function called has a *function-try-block* (Clause 15) with a handler that could handle the exception, this handler is not considered. — *end example*] The value of a function call is the value returned by the called function

except in a virtual function call if the return type of the final overrider is different from the return type of the statically chosen function, the value returned from the final overrider is converted to the return type of the statically chosen function.

5 [*Note:* a function can change the values of its non-const parameters, but these changes cannot affect the values of the arguments except where a parameter is of a reference type (8.3.2); if the reference is to a const-qualified type, `const_cast` is required to be used to cast away the constness in order to modify the argument's value. Where a parameter is of `const` reference type a temporary object is introduced if needed (7.1.6, 2.13, 2.13.5, 8.3.4, 12.2). In addition, it is possible to modify the values of nonconstant objects through pointer parameters. — *end note*]

6 A function can be declared to accept fewer arguments (by declaring default arguments (8.3.6)) or more arguments (by using the ellipsis, `...`, or a function parameter pack (8.3.5)) than the number of parameters in the function definition (8.4). [*Note:* this implies that, except where the ellipsis (`...`) or a function parameter pack is used, a parameter is available for each argument. — *end note*]

7 When there is no parameter for a given argument, the argument is passed in such a way that the receiving function can obtain the value of the argument by invoking `va_arg` (18.10). [*Note:* This paragraph does not apply to arguments passed to a function parameter pack. Function parameter packs are expanded during template instantiation (14.5.3), thus each such argument has a corresponding parameter when a function template specialization is actually called. — *end note*] The lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are performed on the argument expression. An argument that has (possibly cv-qualified) type `std::nullptr_t` is converted to type `void*` (4.10). After these conversions, if the argument does not have arithmetic, enumeration, pointer, pointer to member, or class type, the program is ill-formed. Passing a potentially-evaluated argument of class type (Clause 9) having a non-trivial copy constructor, a non-trivial move constructor, or a non-trivial destructor, with no corresponding parameter, is conditionally-supported with implementation-defined semantics. If the argument has integral or enumeration type that is subject to the integral promotions (4.5), or a floating point type that is subject to the floating point promotion (4.6), the value of the argument is converted to the promoted type before the call. These promotions are referred to as the *default argument promotions*.

8 [*Note:* The evaluations of the postfix expression and of the arguments are all unsequenced relative to one another. All side effects of argument evaluations are sequenced before the function is entered (see 1.9). — *end note*]

9 Recursive calls are permitted, except to the `main` function (3.6.1).

10 A function call is an lvalue if the result type is an lvalue reference type or an rvalue reference to function type, an xvalue if the result type is an rvalue reference to object type, and a prvalue otherwise.

11 If a function call is a prvalue of object type:

(11.1) — if the function call is either

(11.1.1) — the operand of a *decltype-specifier* or

(11.1.2) — the right operand of a comma operator that is the operand of a *decltype-specifier*,

a temporary object is not introduced for the prvalue. The type of the prvalue may be incomplete. [*Note:* as a result, storage is not allocated for the prvalue and it is not destroyed; thus, a class type is not instantiated as a result of being the type of a function call in this context. This is true regardless of whether the expression uses function call notation or operator notation (13.3.1.2). — *end note*] [*Note:* unlike the rule for a *decltype-specifier* that considers whether an *id-expression* is parenthesized (7.1.6.2), parentheses have no special meaning in this context. — *end note*]

(11.2) — otherwise, the type of the prvalue shall be complete.

5.2.3 Explicit type conversion (functional notation) [expr.type.conv]

1 A *simple-type-specifier* (7.1.6.2) or *typename-specifier* (14.6) followed by a parenthesized *expression-list* constructs a value of the specified type given the expression list. If the expression list is a single expression, the type conversion expression is equivalent (in definedness, and if defined in meaning) to the corresponding cast expression (5.4). If the type specified is a class type, the class type shall be complete. If the expression list specifies more than a single value, the type shall be a class with a suitably declared constructor (8.5, 12.1), and the expression `T(x1, x2, ...)` is equivalent in effect to the declaration `T t(x1, x2, ...);` for some invented temporary variable `t`, with the result being the value of `t` as a prvalue.

2 The expression `T()`, where `T` is a *simple-type-specifier* or *typename-specifier* for a non-array complete object type or the (possibly cv-qualified) `void` type, creates a prvalue of the specified type, whose value is that produced by value-initializing (8.5) an object of type `T`; no initialization is done for the `void()` case. [*Note:* if `T` is a non-class type that is cv-qualified, the *cv-qualifier*s are discarded when determining the type of the resulting prvalue (Clause 5). — *end note*]

3 Similarly, a *simple-type-specifier* or *typename-specifier* followed by a *braced-init-list* creates a temporary object of the specified type direct-list-initialized (8.5.4) with the specified *braced-init-list*, and its value is that temporary object as a prvalue.

5.2.4 Pseudo destructor call [expr.pseudo]

1 The use of a *pseudo-destructor-name* after a dot `.` or arrow `->` operator represents the destructor for the non-class type denoted by *type-name* or *decltype-specifier*. The result shall only be used as the operand for the function call operator `()`, and the result of such a call has type `void`. The only effect is the evaluation of the *postfix-expression* before the dot or arrow.

2 The left-hand side of the dot operator shall be of scalar type. The left-hand side of the arrow operator shall be of pointer to scalar type. This scalar type is the object type. The *cv*-unqualified versions of the object type and of the type designated by the *pseudo-destructor-name* shall be the same type. Furthermore, the two *type-name*s in a *pseudo-destructor-name* of the form

nested-name-specifier$_{opt}$ *type-name* `::` `~` *type-name*

shall designate the same scalar type (ignoring cv-qualification).

5.2.5 Class member access [expr.ref]

1 A postfix expression followed by a dot `.` or an arrow `->`, optionally followed by the keyword `template` (14.2), and then followed by an *id-expression*, is a postfix expression. The postfix expression before the dot or arrow is evaluated;[65] the result of that evaluation, together with the *id-expression*, determines the result of the entire postfix expression.

2 For the first option (dot) the first expression shall have complete class type. For the second option (arrow) the first expression shall have pointer to complete class type. The expression `E1->E2` is converted to the equivalent form `(*(E1)).E2`; the remainder of 5.2.5 will address only the first option (dot).[66] In either case, the *id-expression* shall name a member of the class or of one of its base classes. [*Note:* because the name of a class is inserted in its class scope (Clause 9), the name of a class is also considered a nested member of that class. — *end note*] [*Note:* 3.4.5 describes how names are looked up after the `.` and `->` operators. — *end note*]

3 Abbreviating *postfix-expression.id-expression* as `E1.E2`, `E1` is called the *object expression*. The type and value category of `E1.E2` are determined as follows. In the remainder of 5.2.5, *cq* represents either `const` or the absence of `const` and *vq* represents either `volatile` or the absence of `volatile`. *cv* represents an arbitrary set of cv-qualifiers, as defined in 3.9.3.

65) If the class member access expression is evaluated, the subexpression evaluation happens even if the result is unnecessary to determine the value of the entire postfix expression, for example if the *id-expression* denotes a static member.

66) Note that `(*(E1))` is an lvalue.

4 If `E2` is declared to have type "reference to `T`", then `E1.E2` is an lvalue; the type of `E1.E2` is `T`. Otherwise, one of the following rules applies.

(4.1) — If `E2` is a static data member and the type of `E2` is `T`, then `E1.E2` is an lvalue; the expression designates the named member of the class. The type of `E1.E2` is `T`.

(4.2) — If `E2` is a non-static data member and the type of `E1` is "*cq1 vq1* `X`", and the type of `E2` is "*cq2 vq2* `T`", the expression designates the named member of the object designated by the first expression. If `E1` is an lvalue, then `E1.E2` is an lvalue; otherwise `E1.E2` is an xvalue. Let the notation *vq12* stand for the "union" of *vq1* and *vq2*; that is, if *vq1* or *vq2* is `volatile`, then *vq12* is `volatile`. Similarly, let the notation *cq12* stand for the "union" of *cq1* and *cq2*; that is, if *cq1* or *cq2* is `const`, then *cq12* is `const`. If `E2` is declared to be a `mutable` member, then the type of `E1.E2` is "*vq12* `T`". If `E2` is not declared to be a `mutable` member, then the type of `E1.E2` is "*cq12 vq12* `T`".

(4.3) — If `E2` is a (possibly overloaded) member function, function overload resolution (13.3) is used to determine whether `E1.E2` refers to a static or a non-static member function.

(4.3.1) — If it refers to a static member function and the type of `E2` is "function of parameter-type-list returning `T`", then `E1.E2` is an lvalue; the expression designates the static member function. The type of `E1.E2` is the same type as that of `E2`, namely "function of parameter-type-list returning `T`".

(4.3.2) — Otherwise, if `E1.E2` refers to a non-static member function and the type of `E2` is "function of parameter-type-list *cv ref-qualifier*$_{opt}$ returning `T`", then `E1.E2` is a prvalue. The expression designates a non-static member function. The expression can be used only as the left-hand operand of a member function call (9.3). [*Note:* Any redundant set of parentheses surrounding the expression is ignored (5.1). — *end note*] The type of `E1.E2` is "function of parameter-type-list *cv* returning `T`".

(4.4) — If `E2` is a nested type, the expression `E1.E2` is ill-formed.

(4.5) — If `E2` is a member enumerator and the type of `E2` is `T`, the expression `E1.E2` is a prvalue. The type of `E1.E2` is `T`.

5 If `E2` is a non-static data member or a non-static member function, the program is ill-formed if the class of which `E2` is directly a member is an ambiguous base (10.2) of the naming class (11.2) of `E2`. [*Note:* The program is also ill-formed if the naming class is an ambiguous base of the class type of the object expression; see 11.2. — *end note*]

5.2.6 Increment and decrement [expr.post.incr]

1 The value of a postfix `++` expression is the value of its operand. [*Note:* the value obtained is a copy of the original value — *end note*] The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type or a pointer to a complete object type. The value of the operand object is modified by adding `1` to it, unless the object is of type `bool`, in which case it is set to `true`. [*Note:* this use is deprecated, see Annex D. — *end note*] The value computation of the `++` expression is sequenced before the modification of the operand object. With respect to an indeterminately-sequenced function call, the operation of postfix `++` is a single evaluation. [*Note:* Therefore, a function call shall not intervene between the lvalue-to-rvalue conversion and the side effect associated with any single postfix ++ operator. — *end note*] The result is a prvalue. The type of the result is the cv-unqualified version of the type of the operand. If the operand is a bit-field that cannot represent the incremented value, the resulting value of the bit-field is implementation-defined. See also 5.7 and 5.18.

2 The operand of postfix `--` is decremented analogously to the postfix `++` operator, except that the operand shall not be of type `bool`. [*Note:* For prefix increment and decrement, see 5.3.2. — *end note*]

5.2.7 Dynamic cast [expr.dynamic.cast]

1 The result of the expression `dynamic_cast<T>(v)` is the result of converting the expression `v` to type `T`. `T` shall be a pointer or reference to a complete class type, or "pointer to *cv* `void`". The `dynamic_cast` operator shall not cast away constness (5.2.11).

2 If `T` is a pointer type, `v` shall be a prvalue of a pointer to complete class type, and the result is a prvalue of type `T`. If `T` is an lvalue reference type, `v` shall be an lvalue of a complete class type, and the result is an lvalue of the type referred to by `T`. If `T` is an rvalue reference type, `v` shall be an expression having a complete class type, and the result is an xvalue of the type referred to by `T`.

3 If the type of `v` is the same as `T`, or it is the same as `T` except that the class object type in `T` is more cv-qualified than the class object type in `v`, the result is `v` (converted if necessary).

4 If the value of `v` is a null pointer value in the pointer case, the result is the null pointer value of type `T`.

5 If `T` is "pointer to *cv1* `B`" and `v` has type "pointer to *cv2* `D`" such that `B` is a base class of `D`, the result is a pointer to the unique `B` subobject of the `D` object pointed to by `v`. Similarly, if `T` is "reference to *cv1* `B`" and `v` has type *cv2* `D` such that `B` is a base class of `D`, the result is the unique `B` subobject of the `D` object referred to by `v`. [67] The result is an lvalue if `T` is an lvalue reference, or an xvalue if `T` is an rvalue reference. In both the pointer and reference cases, the program is ill-formed if *cv2* has greater cv-qualification than *cv1* or if `B` is an inaccessible or ambiguous base class of `D`. [*Example:*

```
struct B { };
struct D : B { };
void foo(D* dp) {
  B*  bp = dynamic_cast<B*>(dp);     // equivalent to B* bp = dp;
}
```

— *end example*]

6 Otherwise, `v` shall be a pointer to or a glvalue of a polymorphic type (10.3).

7 If `T` is "pointer to *cv* `void`", then the result is a pointer to the most derived object pointed to by `v`. Otherwise, a run-time check is applied to see if the object pointed or referred to by `v` can be converted to the type pointed or referred to by `T`.

8 If `C` is the class type to which `T` points or refers, the run-time check logically executes as follows:

(8.1) — If, in the most derived object pointed (referred) to by `v`, `v` points (refers) to a `public` base class subobject of a `C` object, and if only one object of type `C` is derived from the subobject pointed (referred) to by `v` the result points (refers) to that `C` object.

(8.2) — Otherwise, if `v` points (refers) to a `public` base class subobject of the most derived object, and the type of the most derived object has a base class, of type `C`, that is unambiguous and `public`, the result points (refers) to the `C` subobject of the most derived object.

(8.3) — Otherwise, the run-time check *fails*.

9 The value of a failed cast to pointer type is the null pointer value of the required result type. A failed cast to reference type throws an exception (15.1) of a type that would match a handler (15.3) of type `std::bad_cast` (18.7.2).

[*Example:*

```
class A { virtual void f(); };
class B { virtual void g(); };
class D : public virtual A, private B { };
void g() {
```

67) The most derived object (1.8) pointed or referred to by `v` can contain other `B` objects as base classes, but these are ignored.

```
  D   d;
  B*  bp = (B*)&d;                  // cast needed to break protection
  A*  ap = &d;                      // public derivation, no cast needed
  D&  dr = dynamic_cast<D&>(*bp);   // fails
  ap = dynamic_cast<A*>(bp);        // fails
  bp = dynamic_cast<B*>(ap);        // fails
  ap = dynamic_cast<A*>(&d);        // succeeds
  bp = dynamic_cast<B*>(&d);        // ill-formed (not a run-time check)
}

class E : public D, public B { };
class F : public E, public D { };
void h() {
  F   f;
  A*  ap  = &f;                     // succeeds: finds unique A
  D*  dp  = dynamic_cast<D*>(ap);   // fails: yields 0
                                    // f has two D subobjects
  E*  ep  = (E*)ap;                 // ill-formed: cast from virtual base
  E*  ep1 = dynamic_cast<E*>(ap);   // succeeds
}
```

— *end example*] [*Note:* 12.7 describes the behavior of a `dynamic_cast` applied to an object under construction or destruction. — *end note*]

5.2.8 Type identification [expr.typeid]

1 The result of a `typeid` expression is an lvalue of static type `const std::type_info` (18.7.1) and dynamic type `const std::type_info` or `const` *name* where *name* is an implementation-defined class publicly derived from `std::type_info` which preserves the behavior described in 18.7.1.[68] The lifetime of the object referred to by the lvalue extends to the end of the program. Whether or not the destructor is called for the `std::type_info` object at the end of the program is unspecified.

2 When `typeid` is applied to a glvalue expression whose type is a polymorphic class type (10.3), the result refers to a `std::type_info` object representing the type of the most derived object (1.8) (that is, the dynamic type) to which the glvalue refers. If the glvalue expression is obtained by applying the unary `*` operator to a pointer[69] and the pointer is a null pointer value (4.10), the `typeid` expression throws an exception (15.1) of a type that would match a handler of type `std::bad_typeid` exception (18.7.3).

3 When `typeid` is applied to an expression other than a glvalue of a polymorphic class type, the result refers to a `std::type_info` object representing the static type of the expression. Lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) conversions are not applied to the expression. If the type of the expression is a class type, the class shall be completely-defined. The expression is an unevaluated operand (Clause 5).

4 When `typeid` is applied to a *type-id*, the result refers to a `std::type_info` object representing the type of the *type-id*. If the type of the *type-id* is a reference to a possibly *cv*-qualified type, the result of the `typeid` expression refers to a `std::type_info` object representing the *cv*-unqualified referenced type. If the type of the *type-id* is a class type or a reference to a class type, the class shall be completely-defined.

5 If the type of the expression or *type-id* is a cv-qualified type, the result of the `typeid` expression refers to a `std::type_info` object representing the cv-unqualified type. [*Example:*

```
class D { /* ...  */ };
D d1;
```

68) The recommended name for such a class is `extended_type_info`.
69) If `p` is an expression of pointer type, then `*p`, `(*p)`, `*(p)`, `((*p))`, `*((p))`, and so on all meet this requirement.

```
const D d2;

typeid(d1) == typeid(d2);       // yields true
typeid(D)  == typeid(const D);  // yields true
typeid(D)  == typeid(d2);       // yields true
typeid(D)  == typeid(const D&); // yields true
```

— *end example*]

6 If the header `<typeinfo>` (18.7.1) is not included prior to a use of `typeid`, the program is ill-formed.

7 [*Note:* 12.7 describes the behavior of `typeid` applied to an object under construction or destruction. — *end note*]

5.2.9 Static cast [expr.static.cast]

1 The result of the expression `static_cast<T>(v)` is the result of converting the expression `v` to type `T`. If `T` is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if `T` is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue. The `static_cast` operator shall not cast away constness (5.2.11).

2 An lvalue of type "*cv1* `B`", where `B` is a class type, can be cast to type "reference to *cv2* `D`", where `D` is a class derived (Clause 10) from `B`, if a valid standard conversion from "pointer to `D`" to "pointer to `B`" exists (4.10), *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*, and `B` is neither a virtual base class of `D` nor a base class of a virtual base class of `D`. The result has type "*cv2* `D`". An xvalue of type "*cv1* `B`" may be cast to type "rvalue reference to *cv2* `D`" with the same constraints as for an lvalue of type "*cv1* `B`". If the object of type "*cv1* `B`" is actually a subobject of an object of type `D`, the result refers to the enclosing object of type `D`. Otherwise, the behavior is undefined. [*Example:*

```
struct B { };
struct D : public B { };
D d;
B &br = d;

static_cast<D&>(br);            // produces lvalue to the original d object
```

— *end example*]

3 A glvalue, class prvalue, or array prvalue of type "*cv1* `T1`" can be cast to type "rvalue reference to *cv2* `T2`" if "*cv2* `T2`" is reference-compatible with "*cv1* `T1`" (8.5.3). If the value is not a bit-field, the result refers to the object or the specified base class subobject thereof; otherwise, the lvalue-to-rvalue conversion (4.1) is applied to the bit-field and the resulting prvalue is used as the *expression* of the `static_cast` for the remainder of this section. If `T2` is an inaccessible (Clause 11) or ambiguous (10.2) base class of `T1`, a program that necessitates such a cast is ill-formed.

4 An expression `e` can be explicitly converted to a type `T` using a `static_cast` of the form `static_cast<T>(e)` if the declaration `T t(e);` is well-formed, for some invented temporary variable `t` (8.5). The effect of such an explicit conversion is the same as performing the declaration and initialization and then using the temporary variable as the result of the conversion. The expression `e` is used as a glvalue if and only if the initialization uses it as a glvalue.

5 Otherwise, the `static_cast` shall perform one of the conversions listed below. No other conversion shall be performed explicitly using a `static_cast`.

6 Any expression can be explicitly converted to type *cv* `void`, in which case it becomes a discarded-value expression (Clause 5). [*Note:* however, if the value is in a temporary object (12.2), the destructor for that object is not executed until the usual time, and the value of the object is preserved for the purpose of executing the destructor. — *end note*]

7 The inverse of any standard conversion sequence (Clause 4) not containing an lvalue-to-rvalue (4.1), array-to-pointer (4.2), function-to-pointer (4.3), null pointer (4.10), null member pointer (4.11), or boolean (4.12) conversion, can be performed explicitly using `static_cast`. A program is ill-formed if it uses `static_cast` to perform the inverse of an ill-formed standard conversion sequence. [*Example:*

```
struct B { };
struct D : private B { };
void f() {
  static_cast<D*>((B*)0);                // Error: B is a private base of D.
  static_cast<int B::*>((int D::*)0);    // Error: B is a private base of D.
}
```

— *end example*]

8 The lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) conversions are applied to the operand. Such a `static_cast` is subject to the restriction that the explicit conversion does not cast away constness (5.2.11), and the following additional rules for specific cases:

9 A value of a scoped enumeration type (7.2) can be explicitly converted to an integral type. When that type is *cv* `bool`, the resulting value is `false` if the original value is zero and `true` for all other values. For the remaining integral types, the value is unchanged if the original value can be represented by the specified type. Otherwise, the resulting value is unspecified. A value of a scoped enumeration type can also be explicitly converted to a floating-point type; the result is the same as that of converting from the original value to the floating-point type.

10 A value of integral or enumeration type can be explicitly converted to a complete enumeration type. The value is unchanged if the original value is within the range of the enumeration values (7.2). Otherwise, the behavior is undefined. A value of floating-point type can also be explicitly converted to an enumeration type. The resulting value is the same as converting the original value to the underlying type of the enumeration (4.9), and subsequently to the enumeration type.

11 A prvalue of type "pointer to *cv1* `B`", where `B` is a class type, can be converted to a prvalue of type "pointer to *cv2* `D`", where `D` is a class derived (Clause 10) from `B`, if a valid standard conversion from "pointer to `D`" to "pointer to `B`" exists (4.10), *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*, and `B` is neither a virtual base class of `D` nor a base class of a virtual base class of `D`. The null pointer value (4.10) is converted to the null pointer value of the destination type. If the prvalue of type "pointer to *cv1* `B`" points to a `B` that is actually a subobject of an object of type `D`, the resulting pointer points to the enclosing object of type `D`. Otherwise, the behavior is undefined.

12 A prvalue of type "pointer to member of `D` of type *cv1* `T`" can be converted to a prvalue of type "pointer to member of `B`" of type *cv2* `T`, where `B` is a base class (Clause 10) of `D`, if a valid standard conversion from "pointer to member of `B` of type `T`" to "pointer to member of `D` of type `T`" exists (4.11), and *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*.[70] The null member pointer value (4.11) is converted to the null member pointer value of the destination type. If class `B` contains the original member, or is a base or derived class of the class containing the original member, the resulting pointer to member points to the original member. Otherwise, the behavior is undefined. [*Note:* although class `B` need not contain the original member, the dynamic type of the object with which indirection through the pointer to member is performed must contain the original member; see 5.5. — *end note*]

13 A prvalue of type "pointer to *cv1* `void`" can be converted to a prvalue of type "pointer to *cv2* `T`", where `T` is an object type and *cv2* is the same cv-qualification as, or greater cv-qualification than, *cv1*. The null pointer value is converted to the null pointer value of the destination type. If the original pointer value represents the address `A` of a byte in memory and `A` satisfies the alignment requirement of `T`, then the resulting pointer value represents the same address as the original pointer value, that is, `A`. The result of any other such

70) Function types (including those used in pointer to member function types) are never cv-qualified; see 8.3.5.

pointer conversion is unspecified. A value of type pointer to object converted to "pointer to *cv* `void`" and back, possibly with different cv-qualification, shall have its original value. [*Example:*

```
T* p1 = new T;
const T* p2 = static_cast<const T*>(static_cast<void*>(p1));
bool b = p1 == p2;  // b will have the value true.
```

— *end example*]

5.2.10 Reinterpret cast [expr.reinterpret.cast]

1 The result of the expression `reinterpret_cast<T>(v)` is the result of converting the expression `v` to type `T`. If `T` is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if `T` is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are performed on the expression `v`. Conversions that can be performed explicitly using `reinterpret_cast` are listed below. No other conversion can be performed explicitly using `reinterpret_cast`.

2 The `reinterpret_cast` operator shall not cast away constness (5.2.11). An expression of integral, enumeration, pointer, or pointer-to-member type can be explicitly converted to its own type; such a cast yields the value of its operand.

3 [*Note:* The mapping performed by `reinterpret_cast` might, or might not, produce a representation different from the original value. — *end note*]

4 A pointer can be explicitly converted to any integral type large enough to hold it. The mapping function is implementation-defined. [*Note:* It is intended to be unsurprising to those who know the addressing structure of the underlying machine. — *end note*] A value of type `std::nullptr_t` can be converted to an integral type; the conversion has the same meaning and validity as a conversion of `(void*)0` to the integral type. [*Note:* A `reinterpret_cast` cannot be used to convert a value of any type to the type `std::nullptr_t`. — *end note*]

5 A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value; mappings between pointers and integers are otherwise implementation-defined. [*Note:* Except as described in 3.7.4.3, the result of such a conversion will not be a safely-derived pointer value. — *end note*]

6 A function pointer can be explicitly converted to a function pointer of a different type. The effect of calling a function through a pointer to a function type (8.3.5) that is not the same as the type used in the definition of the function is undefined. Except that converting a prvalue of type "pointer to `T1`" to the type "pointer to `T2`" (where `T1` and `T2` are function types) and back to its original type yields the original pointer value, the result of such a pointer conversion is unspecified. [*Note:* see also 4.10 for more details of pointer conversions. — *end note*]

7 An object pointer can be explicitly converted to an object pointer of a different type.[71] When a prvalue `v` of object pointer type is converted to the object pointer type "pointer to *cv* `T`", the result is `static_cast<`*cv* `T*>(static_cast<`*cv* `void*>(v))`. Converting a prvalue of type "pointer to `T1`" to the type "pointer to `T2`" (where `T1` and `T2` are object types and where the alignment requirements of `T2` are no stricter than those of `T1`) and back to its original type yields the original pointer value.

8 Converting a function pointer to an object pointer type or vice versa is conditionally-supported. The meaning of such a conversion is implementation-defined, except that if an implementation supports conversions in both directions, converting a prvalue of one type to the other type and back, possibly with different cv-qualification, shall yield the original pointer value.

71) The types may have different *cv*-qualifiers, subject to the overall restriction that a `reinterpret_cast` cannot cast away constness.

9 The null pointer value (4.10) is converted to the null pointer value of the destination type. [*Note:* A null pointer constant of type `std::nullptr_t` cannot be converted to a pointer type, and a null pointer constant of integral type is not necessarily converted to a null pointer value. — *end note*]

10 A prvalue of type "pointer to member of `X` of type `T1`" can be explicitly converted to a prvalue of a different type "pointer to member of `Y` of type `T2`" if `T1` and `T2` are both function types or both object types.[72] The null member pointer value (4.11) is converted to the null member pointer value of the destination type. The result of this conversion is unspecified, except in the following cases:

(10.1) — converting a prvalue of type "pointer to member function" to a different pointer to member function type and back to its original type yields the original pointer to member value.

(10.2) — converting a prvalue of type "pointer to data member of `X` of type `T1`" to the type "pointer to data member of `Y` of type `T2`" (where the alignment requirements of `T2` are no stricter than those of `T1`) and back to its original type yields the original pointer to member value.

11 A glvalue expression of type `T1` can be cast to the type "reference to `T2`" if an expression of type "pointer to `T1`" can be explicitly converted to the type "pointer to `T2`" using a `reinterpret_cast`. The result refers to the same object as the source glvalue, but with the specified type. [*Note:* That is, for lvalues, a reference cast `reinterpret_cast<T&>(x)` has the same effect as the conversion `*reinterpret_cast<T*>(&x)` with the built-in `&` and `*` operators (and similarly for `reinterpret_cast<T&&>(x)`). — *end note*] No temporary is created, no copy is made, and constructors (12.1) or conversion functions (12.3) are not called.[73]

5.2.11 Const cast [expr.const.cast]

1 The result of the expression `const_cast<T>(v)` is of type `T`. If `T` is an lvalue reference to object type, the result is an lvalue; if `T` is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are performed on the expression `v`. Conversions that can be performed explicitly using `const_cast` are listed below. No other conversion shall be performed explicitly using `const_cast`.

2 [*Note:* Subject to the restrictions in this section, an expression may be cast to its own type using a `const_cast` operator. — *end note*]

3 For two similar types `T1` and `T2` (4.4), a prvalue of type `T1` may be explicitly converted to the type `T2` using a `const_cast`. The result of a `const_cast` refers to the original entity. [*Example:*

```
typedef int *A[3];                // array of 3 pointer to int
typedef const int *const CA[3];   // array of 3 const pointer to const int

CA &&r = A{}; // OK, reference binds to temporary array object after qualification conversion to type CA
A &&r1 = const_cast<A>(CA{});    // error: temporary array decayed to pointer
A &&r2 = const_cast<A&&>(CA{}); // OK
```

— *end example*]

4 For two object types `T1` and `T2`, if a pointer to `T1` can be explicitly converted to the type "pointer to `T2`" using a `const_cast`, then the following conversions can also be made:

(4.1) — an lvalue of type `T1` can be explicitly converted to an lvalue of type `T2` using the cast `const_cast<T2&>`;

(4.2) — a glvalue of type `T1` can be explicitly converted to an xvalue of type `T2` using the cast `const_cast<T2&&>`; and

72) `T1` and `T2` may have different *cv*-qualifiers, subject to the overall restriction that a `reinterpret_cast` cannot cast away constness.

73) This is sometimes referred to as a *type pun*.

(4.3) — if `T1` is a class type, a prvalue of type `T1` can be explicitly converted to an xvalue of type `T2` using the cast `const_cast<T2&&>`.

The result of a reference `const_cast` refers to the original object.

5 A null pointer value (4.10) is converted to the null pointer value of the destination type. The null member pointer value (4.11) is converted to the null member pointer value of the destination type.

6 [*Note:* Depending on the type of the object, a write operation through the pointer, lvalue or pointer to data member resulting from a `const_cast` that casts away a const-qualifier[74] may produce undefined behavior (7.1.6.1). — *end note*]

7 A conversion from a type `T1` to a type `T2` *casts away constness* if `T1` and `T2` are different, there is a cv-decomposition (4.4) of `T1` yielding n such that `T2` has a cv-decomposition of the form

$$cv_0^2\ P_0^2\ cv_1^2\ P_1^2\ \cdots\ cv_{n-1}^2\ P_{n-1}^2\ cv_n^2\ \texttt{U}_2,$$

and there is no qualification conversion that converts `T1` to

$$cv_0^2\ P_0^1\ cv_1^2\ P_1^1\ \cdots\ cv_{n-1}^2\ P_{n-1}^1\ cv_n^2\ \texttt{U}_1.$$

8 Casting from an lvalue of type `T1` to an lvalue of type `T2` using an lvalue reference cast or casting from an expression of type `T1` to an xvalue of type `T2` using an rvalue reference cast casts away constness if a cast from a prvalue of type "pointer to `T1`" to the type "pointer to `T2`" casts away constness.

9 [*Note:* some conversions which involve only changes in cv-qualification cannot be done using `const_cast`. For instance, conversions between pointers to functions are not covered because such conversions lead to values whose use causes undefined behavior. For the same reasons, conversions between pointers to member functions, and in particular, the conversion from a pointer to a const member function to a pointer to a non-const member function, are not covered. — *end note*]

5.3 Unary expressions [expr.unary]

1 Expressions with unary operators group right-to-left.

unary-expression:
- *postfix-expression*
- `++` *cast-expression*
- `--` *cast-expression*
- *unary-operator cast-expression*
- `sizeof` *unary-expression*
- `sizeof (` *type-id* `)`
- `sizeof ... (` *identifier* `)`
- `alignof (` *type-id* `)`
- *noexcept-expression*
- *new-expression*
- *delete-expression*

unary-operator: one of
`* & + - ! ~`

5.3.1 Unary operators [expr.unary.op]

1 The unary `*` operator performs *indirection*: the expression to which it is applied shall be a pointer to an object type, or a pointer to a function type and the result is an lvalue referring to the object or function to which the expression points. If the type of the expression is "pointer to `T`", the type of the result is "`T`". [*Note:* indirection through a pointer to an incomplete type (other than *cv* `void`) is valid. The lvalue thus obtained can be used in limited ways (to initialize a reference, for example); this lvalue must not be converted to a prvalue, see 4.1. — *end note*]

74) `const_cast` is not limited to conversions that cast away a const-qualifier.

2 The result of each of the following unary operators is a prvalue.

3 The result of the unary `&` operator is a pointer to its operand. The operand shall be an lvalue or a *qualified-id*. If the operand is a *qualified-id* naming a non-static or variant member `m` of some class `C` with type `T`, the result has type "pointer to member of class `C` of type `T`" and is a prvalue designating `C::m`. Otherwise, if the type of the expression is `T`, the result has type "pointer to `T`" and is a prvalue that is the address of the designated object (1.7) or a pointer to the designated function. [*Note:* In particular, the address of an object of type "*cv* `T`" is "pointer to *cv* `T`", with the same cv-qualification. — *end note*] For purposes of pointer arithmetic (5.7) and comparison (5.9, 5.10), an object that is not an array element whose address is taken in this way is considered to belong to an array with one element of type `T`. [*Example:*

```
struct A { int i; };
struct B : A { };
... &B::i ...        // has type int A::*
int a;
int* p1 = &a;
int* p2 = p1 + 1;    // defined behavior
bool b = p2 > p1;    // defined behavior, with value true
```

— *end example*] [*Note:* a pointer to member formed from a `mutable` non-static data member (7.1.1) does not reflect the `mutable` specifier associated with the non-static data member. — *end note*]

4 A pointer to member is only formed when an explicit `&` is used and its operand is a *qualified-id* not enclosed in parentheses. [*Note:* that is, the expression `&(qualified-id)`, where the *qualified-id* is enclosed in parentheses, does not form an expression of type "pointer to member". Neither does `qualified-id`, because there is no implicit conversion from a *qualified-id* for a non-static member function to the type "pointer to member function" as there is from an lvalue of function type to the type "pointer to function" (4.3). Nor is `&unqualified-id` a pointer to member, even within the scope of the *unqualified-id*'s class. — *end note*]

5 If `&` is applied to an lvalue of incomplete class type and the complete type declares `operator&()`, it is unspecified whether the operator has the built-in meaning or the operator function is called. The operand of `&` shall not be a bit-field.

6 The address of an overloaded function (Clause 13) can be taken only in a context that uniquely determines which version of the overloaded function is referred to (see 13.4). [*Note:* since the context might determine whether the operand is a static or non-static member function, the context can also affect whether the expression has type "pointer to function" or "pointer to member function". — *end note*]

7 The operand of the unary `+` operator shall have arithmetic, unscoped enumeration, or pointer type and the result is the value of the argument. Integral promotion is performed on integral or enumeration operands. The type of the result is the type of the promoted operand.

8 The operand of the unary `-` operator shall have arithmetic or unscoped enumeration type and the result is the negation of its operand. Integral promotion is performed on integral or enumeration operands. The negative of an unsigned quantity is computed by subtracting its value from 2^n, where n is the number of bits in the promoted operand. The type of the result is the type of the promoted operand.

9 The operand of the logical negation operator `!` is contextually converted to `bool` (Clause 4); its value is `true` if the converted operand is `false` and `false` otherwise. The type of the result is `bool`.

10 The operand of `~` shall have integral or unscoped enumeration type; the result is the one's complement of its operand. Integral promotions are performed. The type of the result is the type of the promoted operand. There is an ambiguity in the grammar when `~` is followed by a *class-name* or *decltype-specifier*. The ambiguity is resolved by treating `~` as the unary complement operator rather than as the start of an *unqualified-id* naming a destructor. [*Note:* Because the grammar does not permit an operator to follow the `.`, `->`, or `::` tokens, a `~` followed by a *class-name* or *decltype-specifier* in a member access expression or *qualified-id* is unambiguously parsed as a destructor name. — *end note*]

5.3.2 Increment and decrement [expr.pre.incr]

1 The operand of prefix `++` is modified by adding `1`, or set to `true` if it is `bool` (this use is deprecated). The operand shall be a modifiable lvalue. The type of the operand shall be an arithmetic type or a pointer to a completely-defined object type. The result is the updated operand; it is an lvalue, and it is a bit-field if the operand is a bit-field. If `x` is not of type `bool`, the expression `++x` is equivalent to `x+=1` [*Note:* See the discussions of addition (5.7) and assignment operators (5.18) for information on conversions. — *end note*]

2 The operand of prefix `--` is modified by subtracting `1`. The operand shall not be of type `bool`. The requirements on the operand of prefix `--` and the properties of its result are otherwise the same as those of prefix `++`. [*Note:* For postfix increment and decrement, see 5.2.6. — *end note*]

5.3.3 Sizeof [expr.sizeof]

1 The `sizeof` operator yields the number of bytes in the object representation of its operand. The operand is either an expression, which is an unevaluated operand (Clause 5), or a parenthesized *type-id*. The `sizeof` operator shall not be applied to an expression that has function or incomplete type, to the parenthesized name of such types, or to a glvalue that designates a bit-field. `sizeof(char)`, `sizeof(signed char)` and `sizeof(unsigned char)` are `1`. The result of `sizeof` applied to any other fundamental type (3.9.1) is implementation-defined. [*Note:* in particular, `sizeof(bool)`, `sizeof(char16_t)`, `sizeof(char32_t)`, and `sizeof(wchar_t)` are implementation-defined.[75] — *end note*] [*Note:* See 1.7 for the definition of *byte* and 3.9 for the definition of *object representation*. — *end note*]

2 When applied to a reference or a reference type, the result is the size of the referenced type. When applied to a class, the result is the number of bytes in an object of that class including any padding required for placing objects of that type in an array. The size of a most derived class shall be greater than zero (1.8). The result of applying `sizeof` to a base class subobject is the size of the base class type.[76] When applied to an array, the result is the total number of bytes in the array. This implies that the size of an array of n elements is n times the size of an element.

3 The `sizeof` operator can be applied to a pointer to a function, but shall not be applied directly to a function.

4 The lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are not applied to the operand of `sizeof`.

5 The identifier in a `sizeof...` expression shall name a parameter pack. The `sizeof...` operator yields the number of arguments provided for the parameter pack *identifier*. A `sizeof...` expression is a pack expansion (14.5.3). [*Example:*

```
template<class... Types>
struct count {
  static const std::size_t value = sizeof...(Types);
};
```

— *end example*]

6 The result of `sizeof` and `sizeof...` is a constant of type `std::size_t`. [*Note:* `std::size_t` is defined in the standard header `<cstddef>` (18.2). — *end note*]

5.3.4 New [expr.new]

1 The *new-expression* attempts to create an object of the *type-id* (8.1) or *new-type-id* to which it is applied. The type of that object is the *allocated type*. This type shall be a complete object type, but not an abstract class type or array thereof (1.8, 3.9, 10.4). It is implementation-defined whether over-aligned types

75) `sizeof(bool)` is not required to be `1`.

76) The actual size of a base class subobject may be less than the result of applying `sizeof` to the subobject, due to virtual base classes and less strict padding requirements on base class subobjects.

are supported (3.11). [*Note:* because references are not objects, references cannot be created by *new-expression*s. — *end note*] [*Note:* the *type-id* may be a cv-qualified type, in which case the object created by the *new-expression* has a cv-qualified type. — *end note*]

new-expression:
`::`$_{opt}$ `new` *new-placement*$_{opt}$ *new-type-id* *new-initializer*$_{opt}$
`::`$_{opt}$ `new` *new-placement*$_{opt}$ `(` *type-id* `)` *new-initializer*$_{opt}$

new-placement:
`(` *expression-list* `)`

new-type-id:
type-specifier-seq *new-declarator*$_{opt}$

new-declarator:
ptr-operator *new-declarator*$_{opt}$
noptr-new-declarator

noptr-new-declarator:
`[` *expression* `]` *attribute-specifier-seq*$_{opt}$
noptr-new-declarator `[` *constant-expression* `]` *attribute-specifier-seq*$_{opt}$

new-initializer:
`(` *expression-list*$_{opt}$ `)`
braced-init-list

Entities created by a *new-expression* have dynamic storage duration (3.7.4). [*Note:* the lifetime of such an entity is not necessarily restricted to the scope in which it is created. — *end note*] If the entity is a non-array object, the *new-expression* returns a pointer to the object created. If it is an array, the *new-expression* returns a pointer to the initial element of the array.

2 If a placeholder type (7.1.6.4) appears in the *type-specifier-seq* of a *new-type-id* or *type-id* of a *new-expression*, the *new-expression* shall contain a *new-initializer* of the form

`(` *assignment-expression* `)`

The allocated type is deduced from the *new-initializer* as follows: Let `e` be the *assignment-expression* in the *new-initializer* and `T` be the *new-type-id* or *type-id* of the *new-expression*, then the allocated type is the type deduced for the variable `x` in the invented declaration (7.1.6.4):

```
T x(e);
```

[*Example:*

```
new auto(1);                    // allocated type is int
auto x = new auto('a');         // allocated type is char, x is of type char*
```

— *end example*]

3 The *new-type-id* in a *new-expression* is the longest possible sequence of *new-declarator*s. [*Note:* this prevents ambiguities between the declarator operators `&`, `&&`, `*`, and `[]` and their expression counterparts. — *end note*] [*Example:*

```
new int * i;                    // syntax error: parsed as (new int*) i, not as (new int)*i
```

The `*` is the pointer declarator and not the multiplication operator. — *end example*]

4 [*Note:* parentheses in a *new-type-id* of a *new-expression* can have surprising effects. [*Example:*

```
new int(*[10])();               // error
```

is ill-formed because the binding is

```
(new int) (*[10])();            // error
```

Instead, the explicitly parenthesized version of the `new` operator can be used to create objects of compound types (3.9.2):

```
new (int (*[10])());
```

allocates an array of `10` pointers to functions (taking no argument and returning `int`. — *end example*] — *end note*]

5 When the allocated object is an array (that is, the *noptr-new-declarator* syntax is used or the *new-type-id* or *type-id* denotes an array type), the *new-expression* yields a pointer to the initial element (if any) of the array. [*Note:* both `new int` and `new int[10]` have type `int*` and the type of `new int[i][10]` is `int (*)[10]` — *end note*] The *attribute-specifier-seq* in a *noptr-new-declarator* appertains to the associated array type.

6 Every *constant-expression* in a *noptr-new-declarator* shall be a converted constant expression (5.20) of type `std::size_t` and shall evaluate to a strictly positive value. The *expression* in a *noptr-new-declarator* is implicitly converted to `std::size_t`. [*Example:* given the definition `int n = 42`, `new float[n][5]` is well-formed (because `n` is the *expression* of a *noptr-new-declarator*), but `new float[5][n]` is ill-formed (because `n` is not a constant expression). — *end example*]

7 The *expression* in a *noptr-new-declarator* is erroneous if:

(7.1) — the expression is of non-class type and its value before converting to `std::size_t` is less than zero;

(7.2) — the expression is of class type and its value before application of the second standard conversion (13.3.3.1.2)[77] is less than zero;

(7.3) — its value is such that the size of the allocated object would exceed the implementation-defined limit (annex B); or

(7.4) — the *new-initializer* is a *braced-init-list* and the number of array elements for which initializers are provided (including the terminating `'\0'` in a string literal (2.13.5)) exceeds the number of elements to initialize.

If the *expression*, after converting to `std::size_t`, is a core constant expression and the expression is erroneous, the program is ill-formed. Otherwise, a *new-expression* with an erroneous expression does not call an allocation function and terminates by throwing an exception of a type that would match a handler (15.3) of type `std::bad_array_new_length` (18.6.2.2). When the value of the *expression* is zero, the allocation function is called to allocate an array with no elements.

8 A *new-expression* may obtain storage for the object by calling an *allocation function* (3.7.4.1). If the *new-expression* terminates by throwing an exception, it may release storage by calling a deallocation function (3.7.4.2). If the allocated type is a non-array type, the allocation function's name is `operator new` and the deallocation function's name is `operator delete`. If the allocated type is an array type, the allocation function's name is `operator new[]` and the deallocation function's name is `operator delete[]`. [*Note:* an implementation shall provide default definitions for the global allocation functions (3.7.4, 18.6.1.1, 18.6.1.2). A C++ program can provide alternative definitions of these functions (17.6.4.6) and/or class-specific versions (12.5). — *end note*]

9 If the *new-expression* begins with a unary `::` operator, the allocation function's name is looked up in the global scope. Otherwise, if the allocated type is a class type `T` or array thereof, the allocation function's name is looked up in the scope of `T`. If this lookup fails to find the name, or if the allocated type is not a class type, the allocation function's name is looked up in the global scope.

10 An implementation is allowed to omit a call to a replaceable global allocation function (18.6.1.1, 18.6.1.2). When it does so, the storage is instead provided by the implementation or provided by extending the

77) If the conversion function returns a signed integer type, the second standard conversion converts to the unsigned type `std::size_t` and thus thwarts any attempt to detect a negative value afterwards.

allocation of another *new-expression*. The implementation may extend the allocation of a *new-expression* `e1` to provide storage for a *new-expression* `e2` if the following would be true were the allocation not extended:

(10.1) — the evaluation of `e1` is sequenced before the evaluation of `e2`, and

(10.2) — `e2` is evaluated whenever `e1` obtains storage, and

(10.3) — both `e1` and `e2` invoke the same replaceable global allocation function, and

(10.4) — if the allocation function invoked by `e1` and `e2` is throwing, any exceptions thrown in the evaluation of either `e1` or `e2` would be first caught in the same handler, and

(10.5) — the pointer values produced by `e1` and `e2` are operands to evaluated *delete-expression*s, and

(10.6) — the evaluation of `e2` is sequenced before the evaluation of the *delete-expression* whose operand is the pointer value produced by `e1`.

[*Example:*

```
void mergeable(int x) {
  // These allocations are safe for merging:
  std::unique_ptr<char[]> a{new (std::nothrow) char[8]};
  std::unique_ptr<char[]> b{new (std::nothrow) char[8]};
  std::unique_ptr<char[]> c{new (std::nothrow) char[x]};

  g(a.get(), b.get(), c.get());
}

void unmergeable(int x) {
  std::unique_ptr<char[]> a{new char[8]};
  try {
    // Merging this allocation would change its catch handler.
    std::unique_ptr<char[]> b{new char[x]};
  } catch (const std::bad_alloc& e) {
    std::cerr << "Allocation failed: " << e.what() << std::endl;
    throw;
  }
}
```

— *end example*]

11 When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type `std::size_t`. That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array. For arrays of `char` and `unsigned char`, the difference between the result of the *new-expression* and the address returned by the allocation function shall be an integral multiple of the strictest fundamental alignment requirement (3.11) of any object type whose size is no greater than the size of the array being created. [*Note:* Because allocation functions are assumed to return pointers to storage that is appropriately aligned for objects of any type with fundamental alignment, this constraint on array allocation overhead permits the common idiom of allocating character arrays into which objects of other types will later be placed. — *end note*]

12 When a *new-expression* calls an allocation function and that allocation has been extended, the size argument to the allocation call shall be no greater than the sum of the sizes for the omitted calls as specified above, plus the size for the extended call had it not been extended, plus any padding necessary to align the allocated objects within the allocated memory.

13 The *new-placement* syntax is used to supply additional arguments to an allocation function. If used, overload resolution is performed on a function call created by assembling an argument list consisting of the amount of space requested (the first argument) and the expressions in the *new-placement* part of the *new-expression* (the second and succeeding arguments). The first of these arguments has type `std::size_t` and the remaining arguments have the corresponding types of the expressions in the *new-placement*; such an expression is called a *placement new-expression*.

14 [*Example:*

(14.1) — `new T` results in a call of `operator new(sizeof(T))`,

(14.2) — `new(2,f) T` results in a call of `operator new(sizeof(T),2,f)`,

(14.3) — `new T[5]` results in a call of `operator new[](sizeof(T)*5+x)`, and

(14.4) — `new(2,f) T[5]` results in a call of `operator new[](sizeof(T)*5+y,2,f)`.

Here, `x` and `y` are non-negative unspecified values representing array allocation overhead; the result of the *new-expression* will be offset by this amount from the value returned by `operator new[]`. This overhead may be applied in all array *new-expression*s, including those referencing the library function `operator new[](std::size_t, void*)` and other placement allocation functions. The amount of overhead may vary from one invocation of `new` to another. — *end example*]

15 [*Note:* unless an allocation function has a non-throwing exception specification (15.4), it indicates failure to allocate storage by throwing a `std::bad_alloc` exception (3.7.4.1, Clause 15, 18.6.2.1); it returns a non-null pointer otherwise. If the allocation function has a non-throwing exception specification, it returns null to indicate failure to allocate storage and a non-null pointer otherwise. — *end note*] If the allocation function is a reserved placement allocation function (18.6.1.3) that returns null, the behavior is undefined. Otherwise, if the allocation function returns null, initialization shall not be done, the deallocation function shall not be called, and the value of the *new-expression* shall be null.

16 [*Note:* when the allocation function returns a value other than null, it must be a pointer to a block of storage in which space for the object has been reserved. The block of storage is assumed to be appropriately aligned and of the requested size. The address of the created object will not necessarily be the same as that of the block if the object is an array. — *end note*]

17 A *new-expression* that creates an object of type `T` initializes that object as follows:

(17.1) — If the *new-initializer* is omitted, the object is default-initialized (8.5). [*Note:* If no initialization is performed, the object has an indeterminate value. — *end note*]

(17.2) — Otherwise, the *new-initializer* is interpreted according to the initialization rules of 8.5 for direct-initialization.

18 The invocation of the allocation function is indeterminately sequenced with respect to the evaluations of expressions in the *new-initializer*. Initialization of the allocated object is sequenced before the value computation of the *new-expression*. It is unspecified whether expressions in the *new-initializer* are evaluated if the allocation function returns the null pointer or exits using an exception.

19 If the *new-expression* creates an object or an array of objects of class type, access and ambiguity control are done for the allocation function, the deallocation function (12.5), and the constructor (12.1). If the *new-expression* creates an array of objects of class type, the destructor is potentially invoked (12.4).

20 If any part of the object initialization described above[78] terminates by throwing an exception, storage has been obtained for the object, and a suitable deallocation function can be found, the deallocation function is called to free the memory in which the object was being constructed, after which the exception continues to

78) This may include evaluating a *new-initializer* and/or calling a constructor.

propagate in the context of the *new-expression*. If no unambiguous matching deallocation function can be found, propagating the exception does not cause the object's memory to be freed. [*Note:* This is appropriate when the called allocation function does not allocate memory; otherwise, it is likely to result in a memory leak. — *end note*]

21 If the *new-expression* begins with a unary `::` operator, the deallocation function's name is looked up in the global scope. Otherwise, if the allocated type is a class type `T` or an array thereof, the deallocation function's name is looked up in the scope of `T`. If this lookup fails to find the name, or if the allocated type is not a class type or array thereof, the deallocation function's name is looked up in the global scope.

22 A declaration of a placement deallocation function matches the declaration of a placement allocation function if it has the same number of parameters and, after parameter transformations (8.3.5), all parameter types except the first are identical. If the lookup finds a single matching deallocation function, that function will be called; otherwise, no deallocation function will be called. If the lookup finds the two-parameter form of a usual deallocation function (3.7.4.2) and that function, considered as a placement deallocation function, would have been selected as a match for the allocation function, the program is ill-formed. For a non-placement allocation function, the normal deallocation function lookup is used to find the matching deallocation function (5.3.5) [*Example:*

```
struct S {
  // Placement allocation function:
  static void* operator new(std::size_t, std::size_t);

  // Usual (non-placement) deallocation function:
  static void operator delete(void*, std::size_t);
};

S* p = new (0) S;    // ill-formed: non-placement deallocation function matches
                     // placement allocation function
```

— *end example*]

23 If a *new-expression* calls a deallocation function, it passes the value returned from the allocation function call as the first argument of type `void*`. If a placement deallocation function is called, it is passed the same additional arguments as were passed to the placement allocation function, that is, the same arguments as those specified with the *new-placement* syntax. If the implementation is allowed to make a copy of any argument as part of the call to the allocation function, it is allowed to make a copy (of the same original value) as part of the call to the deallocation function or to reuse the copy made as part of the call to the allocation function. If the copy is elided in one place, it need not be elided in the other.

5.3.5 Delete [expr.delete]

1 The *delete-expression* operator destroys a most derived object (1.8) or array created by a *new-expression*.

delete-expression:
`::`$_{opt}$`delete` *cast-expression*
`::`$_{opt}$`delete [ ]` *cast-expression*

The first alternative is for non-array objects, and the second is for arrays. Whenever the `delete` keyword is immediately followed by empty square brackets, it shall be interpreted as the second alternative.[79] The operand shall be of pointer to object type or of class type. If of class type, the operand is contextually implicitly converted (Clause 4) to a pointer to object type.[80] The *delete-expression*'s result has type `void`.

2 If the operand has a class type, the operand is converted to a pointer type by calling the above-mentioned conversion function, and the converted operand is used in place of the original operand for the remainder of

79) A lambda expression with a *lambda-introducer* that consists of empty square brackets can follow the `delete` keyword if the lambda expression is enclosed in parentheses.

80) This implies that an object cannot be deleted using a pointer of type `void*` because `void` is not an object type.

this section. In the first alternative (*delete object*), the value of the operand of `delete` may be a null pointer value, a pointer to a non-array object created by a previous *new-expression*, or a pointer to a subobject (1.8) representing a base class of such an object (Clause 10). If not, the behavior is undefined. In the second alternative (*delete array*), the value of the operand of `delete` may be a null pointer value or a pointer value that resulted from a previous array *new-expression*.[81] If not, the behavior is undefined. [*Note:* this means that the syntax of the *delete-expression* must match the type of the object allocated by `new`, not the syntax of the *new-expression*. — *end note*] [*Note:* a pointer to a `const` type can be the operand of a *delete-expression*; it is not necessary to cast away the constness (5.2.11) of the pointer expression before it is used as the operand of the *delete-expression*. — *end note*]

3 In the first alternative (*delete object*), if the static type of the object to be deleted is different from its dynamic type, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined. In the second alternative (*delete array*) if the dynamic type of the object to be deleted differs from its static type, the behavior is undefined.

4 The *cast-expression* in a *delete-expression* shall be evaluated exactly once.

5 If the object being deleted has incomplete class type at the point of deletion and the complete class has a non-trivial destructor or a deallocation function, the behavior is undefined.

6 If the value of the operand of the *delete-expression* is not a null pointer value, the *delete-expression* will invoke the destructor (if any) for the object or the elements of the array being deleted. In the case of an array, the elements will be destroyed in order of decreasing address (that is, in reverse order of the completion of their constructor; see 12.6.2).

7 If the value of the operand of the *delete-expression* is not a null pointer value, then:

(7.1) — If the allocation call for the *new-expression* for the object to be deleted was not omitted and the allocation was not extended (5.3.4), the *delete-expression* shall call a deallocation function (3.7.4.2). The value returned from the allocation call of the *new-expression* shall be passed as the first argument to the deallocation function.

(7.2) — Otherwise, if the allocation was extended or was provided by extending the allocation of another *new-expression*, and the *delete-expression* for every other pointer value produced by a *new-expression* that had storage provided by the extended *new-expression* has been evaluated, the *delete-expression* shall call a deallocation function. The value returned from the allocation call of the extended *new-expression* shall be passed as the first argument to the deallocation function.

(7.3) — Otherwise, the *delete-expression* will not call a *deallocation function* (3.7.4.2).

Otherwise, it is unspecified whether the deallocation function will be called. [*Note:* The deallocation function is called regardless of whether the destructor for the object or some element of the array throws an exception. — *end note*]

8 [*Note:* An implementation provides default definitions of the global deallocation functions `operator delete()` for non-arrays (18.6.1.1) and `operator delete[]()` for arrays (18.6.1.2). A C++ program can provide alternative definitions of these functions (17.6.4.6), and/or class-specific versions (12.5). — *end note*]

9 When the keyword `delete` in a *delete-expression* is preceded by the unary `::` operator, the deallocation function's name is looked up in global scope. Otherwise, the lookup considers class-specific deallocation functions (12.5). If no class-specific deallocation function is found, the deallocation function's name is looked up in global scope.

10 If deallocation function lookup finds both a usual deallocation function with only a pointer parameter and a usual deallocation function with both a pointer parameter and a size parameter, the function to be called is selected as follows:

81) For non-zero-length arrays, this is the same as a pointer to the first element of the array created by that *new-expression*. Zero-length arrays do not have a first element.

(10.1) — If the type is complete and if, for the second alternative (delete array) only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multi-dimensional) array thereof, the function with two parameters is selected.

(10.2) — Otherwise, it is unspecified which of the two deallocation functions is selected.

11 When a *delete-expression* is executed, the selected deallocation function shall be called with the address of the block of storage to be reclaimed as its first argument and (if the two-parameter deallocation function is used) the size of the block as its second argument.[82]

12 Access and ambiguity control are done for both the deallocation function and the destructor (12.4, 12.5).

5.3.6 Alignof [expr.alignof]

1 An `alignof` expression yields the alignment requirement of its operand type. The operand shall be a *type-id* representing a complete object type, or an array thereof, or a reference to one of those types.

2 The result is an integral constant of type `std::size_t`.

3 When `alignof` is applied to a reference type, the result is the alignment of the referenced type. When `alignof` is applied to an array type, the result is the alignment of the element type.

5.3.7 `noexcept` operator [expr.unary.noexcept]

1 The `noexcept` operator determines whether the evaluation of its operand, which is an unevaluated operand (Clause 5), can throw an exception (15.1).

noexcept-expression:
`noexcept (` *expression* `)`

2 The result of the `noexcept` operator is a constant of type `bool` and is a prvalue.

3 The result of the `noexcept` operator is `true` if the set of potential exceptions of the expression (15.4) is empty, and `false` otherwise.

5.4 Explicit type conversion (cast notation) [expr.cast]

1 The result of the expression `(T)` *cast-expression* is of type `T`. The result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type and an xvalue if `T` is an rvalue reference to object type; otherwise the result is a prvalue. [*Note:* if `T` is a non-class type that is cv-qualified, the *cv-qualifiers* are discarded when determining the type of the resulting prvalue; see Clause 5. — *end note*]

2 An explicit type conversion can be expressed using functional notation (5.2.3), a type conversion operator (`dynamic_cast`, `static_cast`, `reinterpret_cast`, `const_cast`), or the *cast* notation.

cast-expression:
unary-expression
`(` *type-id* `)` *cast-expression*

3 Any type conversion not mentioned below and not explicitly defined by the user (12.3) is ill-formed.

4 The conversions performed by

(4.1) — a `const_cast` (5.2.11),

(4.2) — a `static_cast` (5.2.9),

(4.3) — a `static_cast` followed by a `const_cast`,

(4.4) — a `reinterpret_cast` (5.2.10), or

82) If the static type of the object to be deleted is complete and is different from the dynamic type, and the destructor is not virtual, the size might be incorrect, but that case is already undefined, as stated above.

(4.5) — a `reinterpret_cast` followed by a `const_cast`,

can be performed using the cast notation of explicit type conversion. The same semantic restrictions and behaviors apply, with the exception that in performing a `static_cast` in the following situations the conversion is valid even if the base class is inaccessible:

(4.6) — a pointer to an object of derived class type or an lvalue or rvalue of derived class type may be explicitly converted to a pointer or reference to an unambiguous base class type, respectively;

(4.7) — a pointer to member of derived class type may be explicitly converted to a pointer to member of an unambiguous non-virtual base class type;

(4.8) — a pointer to an object of an unambiguous non-virtual base class type, a glvalue of an unambiguous non-virtual base class type, or a pointer to member of an unambiguous non-virtual base class type may be explicitly converted to a pointer, a reference, or a pointer to member of a derived class type, respectively.

If a conversion can be interpreted in more than one of the ways listed above, the interpretation that appears first in the list is used, even if a cast resulting from that interpretation is ill-formed. If a conversion can be interpreted in more than one way as a `static_cast` followed by a `const_cast`, the conversion is ill-formed. [*Example:*

```
struct A { };
struct I1 : A { };
struct I2 : A { };
struct D : I1, I2 { };
A* foo( D* p ) {
  return (A*)( p ); // ill-formed static_cast interpretation
}
```

— *end example*]

5 The operand of a cast using the cast notation can be a prvalue of type "pointer to incomplete class type". The destination type of a cast using the cast notation can be "pointer to incomplete class type". If both the operand and destination types are class types and one or both are incomplete, it is unspecified whether the `static_cast` or the `reinterpret_cast` interpretation is used, even if there is an inheritance relationship between the two classes. [*Note:* For example, if the classes were defined later in the translation unit, a multi-pass compiler would be permitted to interpret a cast between pointers to the classes as if the class types were complete at the point of the cast. — *end note*]

5.5 Pointer-to-member operators [expr.mptr.oper]

1 The pointer-to-member operators `->*` and `.*` group left-to-right.

pm-expression:
cast-expression
pm-expression `.*` *cast-expression*
pm-expression `->*` *cast-expression*

2 The binary operator `.*` binds its second operand, which shall be of type "pointer to member of `T`" to its first operand, which shall be of class `T` or of a class of which `T` is an unambiguous and accessible base class. The result is an object or a function of the type specified by the second operand.

3 The binary operator `->*` binds its second operand, which shall be of type "pointer to member of `T`" to its first operand, which shall be of type "pointer to `U`" where `U` is either `T` or a class of which `T` is an unambiguous and accessible base class. The expression `E1->*E2` is converted into the equivalent form `(*(E1)).*E2`.

4 Abbreviating *pm-expression*`.*`*cast-expression* as `E1.*E2`, `E1` is called the *object expression*. If the dynamic type of `E1` does not contain the member to which `E2` refers, the behavior is undefined.

5 The restrictions on *cv*-qualification, and the manner in which the *cv*-qualifiers of the operands are combined to produce the *cv*-qualifiers of the result, are the same as the rules for `E1.E2` given in 5.2.5. [*Note:* it is not possible to use a pointer to member that refers to a `mutable` member to modify a `const` class object. For example,

```
struct S {
  S() : i(0) { }
  mutable int i;
};
void f()
{
const S cs;
int S::* pm = &S::i;              // pm refers to mutable member S::i
cs.*pm = 88;                      // ill-formed: cs is a const object
}
```

— *end note*]

6 If the result of `.*` or `->*` is a function, then that result can be used only as the operand for the function call operator `()`. [*Example:*

```
(ptr_to_obj->*ptr_to_mfct)(10);
```

calls the member function denoted by `ptr_to_mfct` for the object pointed to by `ptr_to_obj`. — *end example*] In a `.*` expression whose object expression is an rvalue, the program is ill-formed if the second operand is a pointer to member function with *ref-qualifier* `&`. In a `.*` expression whose object expression is an lvalue, the program is ill-formed if the second operand is a pointer to member function with *ref-qualifier* `&&`. The result of a `.*` expression whose second operand is a pointer to a data member is an lvalue if the first operand is an lvalue and an xvalue otherwise. The result of a `.*` expression whose second operand is a pointer to a member function is a prvalue. If the second operand is the null pointer to member value (4.11), the behavior is undefined.

5.6 Multiplicative operators [expr.mul]

1 The multiplicative operators `*`, `/`, and `%` group left-to-right.

multiplicative-expression:
pm-expression
multiplicative-expression `*` *pm-expression*
multiplicative-expression `/` *pm-expression*
multiplicative-expression `%` *pm-expression*

2 The operands of `*` and `/` shall have arithmetic or unscoped enumeration type; the operands of `%` shall have integral or unscoped enumeration type. The usual arithmetic conversions are performed on the operands and determine the type of the result.

3 The binary `*` operator indicates multiplication.

4 The binary `/` operator yields the quotient, and the binary `%` operator yields the remainder from the division of the first expression by the second. If the second operand of `/` or `%` is zero the behavior is undefined. For integral operands the `/` operator yields the algebraic quotient with any fractional part discarded;[83] if the quotient `a/b` is representable in the type of the result, `(a/b)*b + a%b` is equal to `a`; otherwise, the behavior of both `a/b` and `a%b` is undefined.

5.7 Additive operators [expr.add]

1 The additive operators `+` and `-` group left-to-right. The usual arithmetic conversions are performed for operands of arithmetic or enumeration type.

83) This is often called truncation towards zero.

additive-expression:
multiplicative-expression
additive-expression `+` *multiplicative-expression*
additive-expression `-` *multiplicative-expression*

For addition, either both operands shall have arithmetic or unscoped enumeration type, or one operand shall be a pointer to a completely-defined object type and the other shall have integral or unscoped enumeration type.

2 For subtraction, one of the following shall hold:

(2.1) — both operands have arithmetic or unscoped enumeration type; or

(2.2) — both operands are pointers to cv-qualified or cv-unqualified versions of the same completely-defined object type; or

(2.3) — the left operand is a pointer to a completely-defined object type and the right operand has integral or unscoped enumeration type.

3 The result of the binary `+` operator is the sum of the operands. The result of the binary `-` operator is the difference resulting from the subtraction of the second operand from the first.

4 When an expression that has integral type is added to or subtracted from a pointer, the result has the type of the pointer operand. If the pointer operand points to an element of an array object[84], and the array is large enough, the result points to an element offset from the original element such that the difference of the subscripts of the resulting and original array elements equals the integral expression. In other words, if the expression `P` points to the i-th element of an array object, the expressions `(P)+N` (equivalently, `N+(P)`) and `(P)-N` (where `N` has the value n) point to, respectively, the $i + n$-th and $i - n$-th elements of the array object, provided they exist. Moreover, if the expression `P` points to the last element of an array object, the expression `(P)+1` points one past the last element of the array object, and if the expression `Q` points one past the last element of an array object, the expression `(Q)-1` points to the last element of the array object. If both the pointer operand and the result point to elements of the same array object, or one past the last element of the array object, the evaluation shall not produce an overflow; otherwise, the behavior is undefined.

5 When two pointers to elements of the same array object are subtracted, the result is the difference of the subscripts of the two array elements. The type of the result is an implementation-defined signed integral type; this type shall be the same type that is defined as `std::ptrdiff_t` in the `<cstddef>` header (18.2). As with any other arithmetic overflow, if the result does not fit in the space provided, the behavior is undefined. In other words, if the expressions `P` and `Q` point to, respectively, the i-th and j-th elements of an array object, the expression `(P)-(Q)` has the value $i - j$ provided the value fits in an object of type `std::ptrdiff_t`. Moreover, if the expression `P` points either to an element of an array object or one past the last element of an array object, and the expression `Q` points to the last element of the same array object, the expression `((Q)+1)-(P)` has the same value as `((Q)-(P))+1` and as `-((P)-((Q)+1))`, and has the value zero if the expression `P` points one past the last element of the array object, even though the expression `(Q)+1` does not point to an element of the array object. Unless both pointers point to elements of the same array object, or one past the last element of the array object, the behavior is undefined.[85]

6 For addition or subtraction, if the expressions `P` or `Q` have type "pointer to *cv* `T`", where `T` and the array element type are not similar (4.4), the behavior is undefined. [*Note:* In particular, a pointer to a base class cannot be used for pointer arithmetic when the array contains objects of a derived class type. — *end note*]

84) An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1.

85) Another way to approach pointer arithmetic is first to convert the pointer(s) to character pointer(s): In this scheme the integral value of the expression added to or subtracted from the converted pointer is first multiplied by the size of the object originally pointed to, and the resulting pointer is converted back to the original type. For pointer subtraction, the result of the difference between the character pointers is similarly divided by the size of the object originally pointed to.

When viewed in this way, an implementation need only provide one extra byte (which might overlap another object in the program) just after the end of the object in order to satisfy the "one past the last element" requirements.

7 If the value 0 is added to or subtracted from a pointer value, the result compares equal to the original pointer value. If two pointers point to the same object or both point one past the end of the same array or both are null, and the two pointers are subtracted, the result compares equal to the value 0 converted to the type `std::ptrdiff_t`.

5.8 Shift operators [expr.shift]

1 The shift operators `<<` and `>>` group left-to-right.

shift-expression:
additive-expression
shift-expression `<<` *additive-expression*
shift-expression `>>` *additive-expression*

The operands shall be of integral or unscoped enumeration type and integral promotions are performed. The type of the result is that of the promoted left operand. The behavior is undefined if the right operand is negative, or greater than or equal to the length in bits of the promoted left operand.

2 The value of `E1 << E2` is `E1` left-shifted `E2` bit positions; vacated bits are zero-filled. If `E1` has an unsigned type, the value of the result is $E1 \times 2^{E2}$, reduced modulo one more than the maximum value representable in the result type. Otherwise, if `E1` has a signed type and non-negative value, and $E1 \times 2^{E2}$ is representable in the corresponding unsigned type of the result type, then that value, converted to the result type, is the resulting value; otherwise, the behavior is undefined.

3 The value of `E1 >> E2` is `E1` right-shifted `E2` bit positions. If `E1` has an unsigned type or if `E1` has a signed type and a non-negative value, the value of the result is the integral part of the quotient of $E1/2^{E2}$. If `E1` has a signed type and a negative value, the resulting value is implementation-defined.

5.9 Relational operators [expr.rel]

1 The relational operators group left-to-right. [*Example:* `a<b<c` means `(a<b)<c` and *not* `(a<b)&&(b<c)`. — *end example*]

relational-expression:
shift-expression
relational-expression `<` *shift-expression*
relational-expression `>` *shift-expression*
relational-expression `<=` *shift-expression*
relational-expression `>=` *shift-expression*

The operands shall have arithmetic, enumeration, or pointer type. The operators `<` (less than), `>` (greater than), `<=` (less than or equal to), and `>=` (greater than or equal to) all yield `false` or `true`. The type of the result is `bool`.

2 The usual arithmetic conversions are performed on operands of arithmetic or enumeration type. If both operands are pointers, pointer conversions (4.10) and qualification conversions (4.4) are performed to bring them to their composite pointer type (Clause 5). After conversions, the operands shall have the same type.

3 Comparing pointers to objects[86] is defined as follows:

(3.1) — If two pointers point to different elements of the same array, or to subobjects thereof, the pointer to the element with the higher subscript compares greater.

(3.2) — If one pointer points to an element of an array, or to a subobject thereof, and another pointer points one past the last element of the array, the latter pointer compares greater.

(3.3) — If two pointers point to different non-static data members of the same object, or to subobjects of such members, recursively, the pointer to the later declared member compares greater provided the two members have the same access control (Clause 11) and provided their class is not a union.

86) An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1.

4 If two operands `p` and `q` compare equal (5.10), `p<=q` and `p>=q` both yield `true` and `p<q` and `p>q` both yield `false`. Otherwise, if a pointer `p` compares greater than a pointer `q`, `p>=q`, `p>q`, `q<=p`, and `q<p` all yield `true` and `p<=q`, `p<q`, `q>=p`, and `q>p` all yield `false`. Otherwise, the result of each of the operators is unspecified.

5 If both operands (after conversions) are of arithmetic or enumeration type, each of the operators shall yield `true` if the specified relationship is true and `false` if it is false.

5.10 Equality operators [expr.eq]

equality-expression:
relational-expression
equality-expression `==` *relational-expression*
equality-expression `!=` *relational-expression*

1 The `==` (equal to) and the `!=` (not equal to) operators group left-to-right. The operands shall have arithmetic, enumeration, pointer, or pointer to member type, or type `std::nullptr_t`. The operators `==` and `!=` both yield `true` or `false`, i.e., a result of type `bool`. In each case below, the operands shall have the same type after the specified conversions have been applied.

2 If at least one of the operands is a pointer, pointer conversions (4.10) and qualification conversions (4.4) are performed on both operands to bring them to their composite pointer type (Clause 5). Comparing pointers is defined as follows:

(2.1) — If one pointer represents the address of a complete object, and another pointer represents the address one past the last element of a different complete object [87], the result of the comparison is unspecified.

(2.2) — Otherwise, if the pointers are both null, both point to the same function, or both represent the same address (3.9.2), they compare equal.

(2.3) — Otherwise, the pointers compare unequal.

3 If at least one of the operands is a pointer to member, pointer to member conversions (4.11) and qualification conversions (4.4) are performed on both operands to bring them to their composite pointer type (Clause 5). Comparing pointers to members is defined as follows:

(3.1) — If two pointers to members are both the null member pointer value, they compare equal.

(3.2) — If only one of two pointers to members is the null member pointer value, they compare unequal.

(3.3) — If either is a pointer to a virtual member function, the result is unspecified.

(3.4) — If one refers to a member of class `C1` and the other refers to a member of a different class `C2`, where neither is a base class of the other, the result is unspecified. [*Example:*

```
struct A {};
struct B : A { int x; };
struct C : A { int x; };

int A::*bx = (int(A::*))&B::x;
int A::*cx = (int(A::*))&C::x;

bool b1 = (bx == cx);    // unspecified
```

— *end example*]

(3.5) — If both refer to (possibly different) members of the same union (9.5), they compare equal.

87) An object that is not an array element is considered to belong to a single-element array for this purpose; see 5.3.1.

(3.6) — Otherwise, two pointers to members compare equal if they would refer to the same member of the same most derived object (1.8) or the same subobject if indirection with a hypothetical object of the associated class type were performed, otherwise they compare unequal. [*Example:*

```
struct B {
  int f();
};
struct L : B { };
struct R : B { };
struct D : L, R { };

int (B::*pb)() = &B::f;
int (L::*pl)() = pb;
int (R::*pr)() = pb;
int (D::*pdl)() = pl;
int (D::*pdr)() = pr;
bool x = (pdl == pdr);          // false
bool y = (pb == pl);            // true
```

— *end example*]

4 Two operands of type `std::nullptr_t` or one operand of type `std::nullptr_t` and the other a null pointer constant compare equal.

5 If two operands compare equal, the result is `true` for the `==` operator and `false` for the `!=` operator. If two operands compare unequal, the result is `false` for the `==` operator and `true` for the `!=` operator. Otherwise, the result of each of the operators is unspecified.

6 If both operands are of arithmetic or enumeration type, the usual arithmetic conversions are performed on both operands; each of the operators shall yield `true` if the specified relationship is true and `false` if it is false.

5.11 Bitwise AND operator [expr.bit.and]

and-expression:
equality-expression
and-expression `&` *equality-expression*

1 The usual arithmetic conversions are performed; the result is the bitwise AND function of the operands. The operator applies only to integral or unscoped enumeration operands.

5.12 Bitwise exclusive OR operator [expr.xor]

exclusive-or-expression:
and-expression
exclusive-or-expression `^` *and-expression*

1 The usual arithmetic conversions are performed; the result is the bitwise exclusive OR function of the operands. The operator applies only to integral or unscoped enumeration operands.

5.13 Bitwise inclusive OR operator [expr.or]

inclusive-or-expression:
exclusive-or-expression
inclusive-or-expression `|` *exclusive-or-expression*

1 The usual arithmetic conversions are performed; the result is the bitwise inclusive OR function of its operands. The operator applies only to integral or unscoped enumeration operands.

5.14 Logical AND operator [expr.log.and]

logical-and-expression:
inclusive-or-expression
logical-and-expression `&&` *inclusive-or-expression*

1 The `&&` operator groups left-to-right. The operands are both contextually converted to `bool` (Clause 4). The result is `true` if both operands are `true` and `false` otherwise. Unlike `&`, `&&` guarantees left-to-right evaluation: the second operand is not evaluated if the first operand is `false`.

2 The result is a `bool`. If the second expression is evaluated, every value computation and side effect associated with the first expression is sequenced before every value computation and side effect associated with the second expression.

5.15 Logical OR operator [expr.log.or]

logical-or-expression:
logical-and-expression
logical-or-expression || *logical-and-expression*

1 The || operator groups left-to-right. The operands are both contextually converted to `bool` (Clause 4). It returns `true` if either of its operands is `true`, and `false` otherwise. Unlike |, || guarantees left-to-right evaluation; moreover, the second operand is not evaluated if the first operand evaluates to `true`.

2 The result is a `bool`. If the second expression is evaluated, every value computation and side effect associated with the first expression is sequenced before every value computation and side effect associated with the second expression.

5.16 Conditional operator [expr.cond]

conditional-expression:
logical-or-expression
logical-or-expression `?` *expression* `:` *assignment-expression*

1 Conditional expressions group right-to-left. The first expression is contextually converted to `bool` (Clause 4). It is evaluated and if it is `true`, the result of the conditional expression is the value of the second expression, otherwise that of the third expression. Only one of the second and third expressions is evaluated. Every value computation and side effect associated with the first expression is sequenced before every value computation and side effect associated with the second or third expression.

2 If either the second or the third operand has type `void`, one of the following shall hold:

(2.1) — The second or the third operand (but not both) is a (possibly parenthesized) *throw-expression* (5.17); the result is of the type and value category of the other. The *conditional-expression* is a bit-field if that operand is a bit-field.

(2.2) — Both the second and the third operands have type `void`; the result is of type `void` and is a prvalue. [*Note:* This includes the case where both operands are *throw-expression*s. — *end note*]

3 Otherwise, if the second and third operand have different types and either has (possibly cv-qualified) class type, or if both are glvalues of the same value category and the same type except for cv-qualification, an attempt is made to convert each of those operands to the type of the other. The process for determining whether an operand expression `E1` of type `T1` can be converted to match an operand expression `E2` of type `T2` is defined as follows:

(3.1) — If `E2` is an lvalue: `E1` can be converted to match `E2` if `E1` can be implicitly converted (Clause 4) to the type "lvalue reference to `T2`", subject to the constraint that in the conversion the reference must bind directly (8.5.3) to an lvalue.

(3.2) — If `E2` is an xvalue: `E1` can be converted to match `E2` if `E1` can be implicitly converted to the type "rvalue reference to `T2`", subject to the constraint that the reference must bind directly.

(3.3) — If `E2` is a prvalue or if neither of the conversions above can be done and at least one of the operands has (possibly cv-qualified) class type:

(3.3.1) — if `E1` and `E2` have class type, and the underlying class types are the same or one is a base class of the other: `E1` can be converted to match `E2` if the class of `T2` is the same type as, or a base class of, the class of `T1`, and the cv-qualification of `T2` is the same cv-qualification as, or a greater cv-qualification than, the cv-qualification of `T1`. If the conversion is applied, `E1` is changed to a prvalue of type `T2` by copy-initializing a temporary of type `T2` from `E1` and using that temporary as the converted operand.

(3.3.2) — Otherwise (if `E1` or `E2` has a non-class type, or if they both have class types but the underlying classes are not the same and neither is a base class of the other): `E1` can be converted to match `E2` if `E1` can be implicitly converted to the type that `E2` would have after applying the lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions.

Using this process, it is determined whether the second operand can be converted to match the third operand, and whether the third operand can be converted to match the second operand. If both can be converted, or one can be converted but the conversion is ambiguous, the program is ill-formed. If neither can be converted, the operands are left unchanged and further checking is performed as described below. If exactly one conversion is possible, that conversion is applied to the chosen operand and the converted operand is used in place of the original operand for the remainder of this section.

4 If the second and third operands are glvalues of the same value category and have the same type, the result is of that type and value category and it is a bit-field if the second or the third operand is a bit-field, or if both are bit-fields.

5 Otherwise, the result is a prvalue. If the second and third operands do not have the same type, and either has (possibly cv-qualified) class type, overload resolution is used to determine the conversions (if any) to be applied to the operands (13.3.1.2, 13.6). If the overload resolution fails, the program is ill-formed. Otherwise, the conversions thus determined are applied, and the converted operands are used in place of the original operands for the remainder of this section.

6 Lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) standard conversions are performed on the second and third operands. After those conversions, one of the following shall hold:

(6.1) — The second and third operands have the same type; the result is of that type. If the operands have class type, the result is a prvalue temporary of the result type, which is copy-initialized from either the second operand or the third operand depending on the value of the first operand.

(6.2) — The second and third operands have arithmetic or enumeration type; the usual arithmetic conversions are performed to bring them to a common type, and the result is of that type.

(6.3) — One or both of the second and third operands have pointer type; pointer conversions (4.10) and qualification conversions (4.4) are performed to bring them to their composite pointer type (Clause 5). The result is of the composite pointer type.

(6.4) — One or both of the second and third operands have pointer to member type; pointer to member conversions (4.11) and qualification conversions (4.4) are performed to bring them to their composite pointer type (Clause 5). The result is of the composite pointer type.

(6.5) — Both the second and third operands have type `std::nullptr_t` or one has that type and the other is a null pointer constant. The result is of type `std::nullptr_t`.

5.17 Throwing an exception [expr.throw]

throw-expression:
`throw` *assignment-expression*$_{opt}$

1 A *throw-expression* is of type `void`.

2 Evaluating a *throw-expression* with an operand throws an exception (15.1); the type of the exception object is determined by removing any top-level *cv-qualifiers* from the static type of the operand and adjusting the type from "array of `T`" or "function returning `T`" to "pointer to `T`" or "pointer to function returning `T`", respectively.

3 A *throw-expression* with no operand rethrows the currently handled exception (15.3). The exception is reactivated with the existing exception object; no new exception object is created. The exception is no longer considered to be caught. [*Example:* Code that must be executed because of an exception, but cannot completely handle the exception itself, can be written like this:

```
try {
    // ...
} catch (...) {      // catch all exceptions
  // respond (partially) to exception
  throw;             // pass the exception to some
                     // other handler
}
```

— *end example*]

4 If no exception is presently being handled, evaluating a *throw-expression* with no operand calls `std::terminate()` (15.5.1).

5.18 Assignment and compound assignment operators [expr.ass]

1 The assignment operator (`=`) and the compound assignment operators all group right-to-left. All require a modifiable lvalue as their left operand and return an lvalue referring to the left operand. The result in all cases is a bit-field if the left operand is a bit-field. In all cases, the assignment is sequenced after the value computation of the right and left operands, and before the value computation of the assignment expression. With respect to an indeterminately-sequenced function call, the operation of a compound assignment is a single evaluation. [*Note:* Therefore, a function call shall not intervene between the lvalue-to-rvalue conversion and the side effect associated with any single compound assignment operator. — *end note*]

assignment-expression:
    conditional-expression
    logical-or-expression assignment-operator initializer-clause
    throw-expression

assignment-operator: one of
    `= *= /= %= += -= >>= <<= &= ^= |=`

2 In simple assignment (`=`), the value of the expression replaces that of the object referred to by the left operand.

3 If the left operand is not of class type, the expression is implicitly converted (Clause 4) to the cv-unqualified type of the left operand.

4 If the left operand is of class type, the class shall be complete. Assignment to objects of a class is defined by the copy/move assignment operator (12.8, 13.5.3).

5 [*Note:* For class objects, assignment is not in general the same as initialization (8.5, 12.1, 12.6, 12.8). — *end note*]

6 When the left operand of an assignment operator is a bit-field that cannot represent the value of the expression, the resulting value of the bit-field is implementation-defined.

7 The behavior of an expression of the form `E1` *op* `= E2` is equivalent to `E1 = E1` *op* `E2` except that `E1` is evaluated only once. In `+=` and `-=`, `E1` shall either have arithmetic type or be a pointer to a possibly cv-qualified completely-defined object type. In all other cases, `E1` shall have arithmetic type.

8 If the value being stored in an object is read via another object that overlaps in any way the storage of

the first object, then the overlap shall be exact and the two objects shall have the same type, otherwise the behavior is undefined. [*Note:* This restriction applies to the relationship between the left and right sides of the assignment operation; it is not a statement about how the target of the assignment may be aliased in general. See 3.10. — *end note*]

9 A *braced-init-list* may appear on the right-hand side of

(9.1) — an assignment to a scalar, in which case the initializer list shall have at most a single element. The meaning of `x={v}`, where `T` is the scalar type of the expression `x`, is that of `x=T{v}`. The meaning of `x={}` is `x=T{}`.

(9.2) — an assignment to an object of class type, in which case the initializer list is passed as the argument to the assignment operator function selected by overload resolution (13.5.3, 13.3).

[*Example:*

```
complex<double> z;
z = { 1,2 };                // meaning z.operator=({1,2})
z += { 1, 2 };              // meaning z.operator+=({1,2})
int a, b;
a = b = { 1 };              // meaning a=b=1;
a = { 1 } = b;              // syntax error
```

— *end example*]

5.19 Comma operator [expr.comma]

1 The comma operator groups left-to-right.

expression:
assignment-expression
expression , *assignment-expression*

A pair of expressions separated by a comma is evaluated left-to-right; the left expression is a discarded-value expression (Clause 5).[88] Every value computation and side effect associated with the left expression is sequenced before every value computation and side effect associated with the right expression. The type and value of the result are the type and value of the right operand; the result is of the same value category as its right operand, and is a bit-field if its right operand is a bit-field. If the value of the right operand is a temporary (12.2), the result is that temporary.

2 In contexts where comma is given a special meaning, [*Example:* in lists of arguments to functions (5.2.2) and lists of initializers (8.5) — *end example*] the comma operator as described in Clause 5 can appear only in parentheses. [*Example:*

```
f(a, (t=3, t+2), c);
```

has three arguments, the second of which has the value `5`. — *end example*]

5.20 Constant expressions [expr.const]

1 Certain contexts require expressions that satisfy additional requirements as detailed in this sub-clause; other contexts have different semantics depending on whether or not an expression satisfies these requirements. Expressions that satisfy these requirements are called *constant expressions*. [*Note:* Constant expressions can be evaluated during translation. — *end note*]

constant-expression:
conditional-expression

88) However, an invocation of an overloaded comma operator is an ordinary function call; hence, the evaluations of its argument expressions are unsequenced relative to one another (see 1.9).

2 A *conditional-expression* `e` is a *core constant expression* unless the evaluation of `e`, following the rules of the abstract machine (1.9), would evaluate one of the following expressions:

(2.1) — `this` (5.1.1), except in a `constexpr` function or a `constexpr` constructor that is being evaluated as part of `e`;

(2.2) — an invocation of a function other than a `constexpr` constructor for a literal class, a `constexpr` function, or an implicit invocation of a trivial destructor (12.4) [*Note:* Overload resolution (13.3) is applied as usual — *end note*];

(2.3) — an invocation of an undefined `constexpr` function or an undefined `constexpr` constructor;

(2.4) — an expression that would exceed the implementation-defined limits (see Annex B);

(2.5) — an operation that would have undefined behavior as specified in Clauses 1 through 16 of this International Standard [*Note:* including, for example, signed integer overflow (Clause 5), certain pointer arithmetic (5.7), division by zero (5.6), or certain shift operations (5.8) — *end note*];

(2.6) — a *lambda-expression* (5.1.2);

(2.7) — an lvalue-to-rvalue conversion (4.1) unless it is applied to

(2.7.1) — a non-volatile glvalue of integral or enumeration type that refers to a complete non-volatile const object with a preceding initialization, initialized with a constant expression, or

(2.7.2) — a non-volatile glvalue that refers to a subobject of a string literal (2.13.5), or

(2.7.3) — a non-volatile glvalue that refers to a non-volatile object defined with `constexpr`, or that refers to a non-mutable sub-object of such an object, or

(2.7.4) — a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of `e`;

(2.8) — an lvalue-to-rvalue conversion (4.1) or modification (5.18, 5.2.6, 5.3.2) that is applied to a glvalue that refers to a non-active member of a union or a subobject thereof;

(2.9) — an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either

(2.9.1) — it is initialized with a constant expression or

(2.9.2) — it is a non-static data member of an object whose lifetime began within the evaluation of `e`;

(2.10) — in a *lambda-expression*, a reference to `this` or to a variable with automatic storage duration defined outside that *lambda-expression*, where the reference would be an odr-use (3.2, 5.1.2);

(2.11) — a conversion from type *cv* `void *` to a pointer-to-object type;

(2.12) — a dynamic cast (5.2.7);

(2.13) — a `reinterpret_cast` (5.2.10);

(2.14) — a pseudo-destructor call (5.2.4);

(2.15) — modification of an object (5.18, 5.2.6, 5.3.2) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of `e`;

(2.16) — a typeid expression (5.2.8) whose operand is a glvalue of a polymorphic class type;

(2.17) — a *new-expression* (5.3.4);

(2.18) — a *delete-expression* (5.3.5);

(2.19) — a relational (5.9) or equality (5.10) operator where the result is unspecified; or

(2.20) — a *throw-expression* (5.17).

If `e` satisfies the constraints of a core constant expression, but evaluation of `e` would evaluate an operation that has undefined behavior as specified in Clauses 17 through 30 of this International Standard, it is unspecified whether `e` is a core constant expression.

[*Example:*

```
int x;                                  // not constant
struct A {
  constexpr A(bool b) : m(b?42:x) { }
  int m;
};
constexpr int v = A(true).m;            // OK: constructor call initializes
                                        // m with the value 42
constexpr int w = A(false).m;           // error: initializer for m is
                                        // x, which is non-constant

constexpr int f1(int k) {
  constexpr int x = k;                  // error: x is not initialized by a
                                        // constant expression because lifetime of k
                                        // began outside the initializer of x
  return x;
}
constexpr int f2(int k) {
  int x = k;                            // OK: not required to be a constant expression
                                        // because x is not constexpr
  return x;
}

constexpr int incr(int &n) {
  return ++n;
}
constexpr int g(int k) {
  constexpr int x = incr(k);            // error: incr(k) is not a core constant
                                        // expression because lifetime of k
                                        // began outside the expression incr(k)
  return x;
}
constexpr int h(int k) {
  int x = incr(k);                      // OK: incr(k) is not required to be a core
                                        // constant expression
  return x;
}
constexpr int y = h(1);                 // OK: initializes y with the value 2
                                        // h(1) is a core constant expression because
                                        // the lifetime of k begins inside h(1)
```

— *end example*]

3 An *integral constant expression* is an expression of integral or unscoped enumeration type, implicitly converted to a prvalue, where the converted expression is a core constant expression. [*Note:* Such expressions may be used as array bounds (8.3.4, 5.3.4), as bit-field lengths (9.6), as enumerator initializers if the underlying type is not fixed (7.2), and as alignments (7.6.2). — *end note*]

4 A *converted constant expression* of type `T` is an expression, implicitly converted to type `T`, where the converted expression is a constant expression and the implicit conversion sequence contains only

(4.1) — user-defined conversions,

(4.2) — lvalue-to-rvalue conversions (4.1),

(4.3) — array-to-pointer conversions (4.2),

(4.4) — function-to-pointer conversions (4.3),

(4.5) — qualification conversions (4.4),

(4.6) — integral promotions (4.5),

(4.7) — integral conversions (4.7) other than narrowing conversions (8.5.4),

(4.8) — null pointer conversions (4.10) from `std::nullptr_t`, and

(4.9) — null member pointer conversions (4.11) from `std::nullptr_t`,

and where the reference binding (if any) binds directly. [*Note:* such expressions may be used in `new` expressions (5.3.4), as case expressions (6.4.2), as enumerator initializers if the underlying type is fixed (7.2), as array bounds (8.3.4), and as non-type template arguments (14.3). — *end note*]

5 A *constant expression* is either a glvalue core constant expression whose value refers to an entity that is a permitted result of a constant expression (as defined below), or a prvalue core constant expression whose value is an object where, for that object and its subobjects:

(5.1) — each non-static data member of reference type refers to an entity that is a permitted result of a constant expression, and

(5.2) — if the object or subobject is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object (5.7), the address of a function, or a null pointer value.

An entity is a *permitted result of a constant expression* if it is an object with static storage duration that is either not a temporary object or is a temporary object whose value satisfies the above constraints, or it is a function.

6 [*Note:* Since this International Standard imposes no restrictions on the accuracy of floating-point operations, it is unspecified whether the evaluation of a floating-point expression during translation yields the same result as the evaluation of the same expression (or the same operations on the same values) during program execution.[89] [*Example:*

```
bool f() {
    char array[1 + int(1 + 0.2 - 0.1 - 0.1)];   // Must be evaluated during translation
    int size = 1 + int(1 + 0.2 - 0.1 - 0.1);    // May be evaluated at runtime
    return sizeof(array) == size;
}
```

It is unspecified whether the value of `f()` will be `true` or `false`. — *end example*] — *end note*]

7 If an expression of literal class type is used in a context where an integral constant expression is required, then that expression is contextually implicitly converted (Clause 4) to an integral or unscoped enumeration type and the selected conversion function shall be `constexpr`. [*Example:*

89) Nonetheless, implementations are encouraged to provide consistent results, irrespective of whether the evaluation was performed during translation and/or during program execution.

```
struct A {
  constexpr A(int i) : val(i) { }
  constexpr operator int() const { return val; }
  constexpr operator long() const { return 43; }
private:
  int val;
};
template<int> struct X { };
constexpr A a = 42;
X<a> x;              // OK: unique conversion to int
int ary[a];          // error: ambiguous conversion
```

— *end example*]

6 Statements [stmt.stmt]

1 Except as indicated, statements are executed in sequence.

statement:
labeled-statement
attribute-specifier-seq$_{opt}$ *expression-statement*
attribute-specifier-seq$_{opt}$ *compound-statement*
attribute-specifier-seq$_{opt}$ *selection-statement*
attribute-specifier-seq$_{opt}$ *iteration-statement*
attribute-specifier-seq$_{opt}$ *jump-statement*
declaration-statement
attribute-specifier-seq$_{opt}$ *try-block*

The optional *attribute-specifier-seq* appertains to the respective statement.

6.1 Labeled statement [stmt.label]

1 A statement can be labeled.

labeled-statement:
attribute-specifier-seq$_{opt}$ *identifier* `:` *statement*
attribute-specifier-seq$_{opt}$`case` *constant-expression* `:` *statement*
attribute-specifier-seq$_{opt}$`default :` *statement*

The optional *attribute-specifier-seq* appertains to the label. An identifier label declares the identifier. The only use of an identifier label is as the target of a `goto`. The scope of a label is the function in which it appears. Labels shall not be redeclared within a function. A label can be used in a `goto` statement before its definition. Labels have their own name space and do not interfere with other identifiers.

2 Case labels and default labels shall occur only in switch statements.

6.2 Expression statement [stmt.expr]

1 Expression statements have the form

expression-statement:
expression$_{opt}$`;`

The expression is a discarded-value expression (Clause 5). All side effects from an expression statement are completed before the next statement is executed. An expression statement with the expression missing is called a null statement. [*Note:* Most statements are expression statements — usually assignments or function calls. A null statement is useful to carry a label just before the `}` of a compound statement and to supply a null body to an iteration statement such as a `while` statement (6.5.1). — *end note*]

6.3 Compound statement or block [stmt.block]

1 So that several statements can be used where one is expected, the compound statement (also, and equivalently, called "block") is provided.

compound-statement:
`{` *statement-seq*$_{opt}$`}`

statement-seq:
statement
statement-seq statement

A compound statement defines a block scope (3.3). [*Note:* A declaration is a statement (6.7). — *end note*]

6.4 Selection statements [stmt.select]

1 Selection statements choose one of several flows of control.

selection-statement:
`if` `(` *condition* `)` *statement*
`if` `(` *condition* `)` *statement* `else` *statement*
`switch` `(` *condition* `)` *statement*

condition:
expression
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator* `=` *initializer-clause*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator braced-init-list*

See 8.3 for the optional *attribute-specifier-seq* in a condition. In Clause 6, the term *substatement* refers to the contained statement or statements that appear in the syntax notation. The substatement in a *selection-statement* (each substatement, in the `else` form of the `if` statement) implicitly defines a block scope (3.3). If the substatement in a selection-statement is a single statement and not a *compound-statement,* it is as if it was rewritten to be a compound-statement containing the original substatement. [*Example:*

```
if (x)
  int i;
```

can be equivalently rewritten as

```
if (x) {
  int i;
}
```

Thus after the `if` statement, `i` is no longer in scope. — *end example*]

2 The rules for conditions apply both to *selection-statement*s and to the `for` and `while` statements (6.5). The declarator shall not specify a function or an array. The *decl-specifier-seq* shall not define a class or enumeration. If the `auto` *type-specifier* appears in the *decl-specifier-seq*, the type of the identifier being declared is deduced from the initializer as described in 7.1.6.4.

3 A name introduced by a declaration in a condition (either introduced by the *decl-specifier-seq* or the declarator of the condition) is in scope from its point of declaration until the end of the substatements controlled by the condition. If the name is re-declared in the outermost block of a substatement controlled by the condition, the declaration that re-declares the name is ill-formed. [*Example:*

```
if (int x = f()) {
  int x;            // ill-formed, redeclaration of x
}
else {
  int x;            // ill-formed, redeclaration of x
}
```

— *end example*]

4 The value of a condition that is an initialized declaration in a statement other than a `switch` statement is the value of the declared variable contextually converted to `bool` (Clause 4). If that conversion is ill-formed, the program is ill-formed. The value of a condition that is an initialized declaration in a `switch` statement is the value of the declared variable if it has integral or enumeration type, or of that variable implicitly converted to integral or enumeration type otherwise. The value of a condition that is an expression is the value of the expression, contextually converted to `bool` for statements other than `switch`; if that conversion is ill-formed, the program is ill-formed. The value of the condition will be referred to as simply "the condition" where the usage is unambiguous.

5 If a condition can be syntactically resolved as either an expression or the declaration of a block-scope name, it is interpreted as a declaration.

6 In the *decl-specifier-seq* of a *condition*, each *decl-specifier* shall be either a *type-specifier* or `constexpr`.

6.4.1 The `if` statement [stmt.if]

1 If the condition (6.4) yields `true` the first substatement is executed. If the `else` part of the selection statement is present and the condition yields `false`, the second substatement is executed. If the first substatement is reached via a label, the condition is not evaluated and the second substatement is not executed. In the second form of `if` statement (the one including `else`), if the first substatement is also an `if` statement then that inner `if` statement shall contain an `else` part.[90]

6.4.2 The `switch` statement [stmt.switch]

1 The `switch` statement causes control to be transferred to one of several statements depending on the value of a condition.

2 The condition shall be of integral type, enumeration type, or class type. If of class type, the condition is contextually implicitly converted (Clause 4) to an integral or enumeration type. If the (possibly converted) type is subject to integral promotions (4.5), the condition is converted to the promoted type. Any statement within the `switch` statement can be labeled with one or more case labels as follows:

`case` *constant-expression* `:`

where the *constant-expression* shall be a converted constant expression (5.20) of the adjusted type of the switch condition. No two of the case constants in the same switch shall have the same value after conversion.

3 There shall be at most one label of the form

`default :`

within a `switch` statement.

4 Switch statements can be nested; a `case` or `default` label is associated with the smallest switch enclosing it.

5 When the `switch` statement is executed, its condition is evaluated and compared with each case constant. If one of the case constants is equal to the value of the condition, control is passed to the statement following the matched case label. If no case constant matches the condition, and if there is a `default` label, control passes to the statement labeled by the default label. If no case matches and if there is no `default` then none of the statements in the switch is executed.

6 `case` and `default` labels in themselves do not alter the flow of control, which continues unimpeded across such labels. To exit from a switch, see `break`, 6.6.1. [*Note:* Usually, the substatement that is the subject of a switch is compound and `case` and `default` labels appear on the top-level statements contained within the (compound) substatement, but this is not required. Declarations can appear in the substatement of a *switch-statement*. — *end note*]

6.5 Iteration statements [stmt.iter]

1 Iteration statements specify looping.

iteration-statement:
`while (` *condition* `)` *statement*
`do` *statement* `while (` *expression* `) ;`
`for (` *for-init-statement* *condition*$_{opt}$`;` *expression*$_{opt}$`)` *statement*
`for (` *for-range-declaration* `:` *for-range-initializer* `)` *statement*

for-init-statement:
expression-statement
simple-declaration

90) In other words, the `else` is associated with the nearest un-elsed `if`.

for-range-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator*
for-range-initializer:
expression
braced-init-list

See 8.3 for the optional *attribute-specifier-seq* in a *for-range-declaration*. [*Note:* A *for-init-statement* ends with a semicolon. — *end note*]

2 The substatement in an *iteration-statement* implicitly defines a block scope (3.3) which is entered and exited each time through the loop.

If the substatement in an iteration-statement is a single statement and not a *compound-statement*, it is as if it was rewritten to be a compound-statement containing the original statement. [*Example:*

```
while (--x >= 0)
  int i;
```

can be equivalently rewritten as

```
while (--x >= 0) {
  int i;
}
```

3 Thus after the `while` statement, `i` is no longer in scope. — *end example*]

4 [*Note:* The requirements on conditions in iteration statements are described in 6.4. — *end note*]

6.5.1 The `while` statement [stmt.while]

1 In the `while` statement the substatement is executed repeatedly until the value of the condition (6.4) becomes `false`. The test takes place before each execution of the substatement.

2 When the condition of a `while` statement is a declaration, the scope of the variable that is declared extends from its point of declaration (3.3.2) to the end of the `while` statement. A `while` statement of the form

```
while (T t = x) statement
```

is equivalent to

```
label:
{                       // start of condition scope
  T t = x;
  if (t) {
    statement
    goto label;
  }
}                       // end of condition scope
```

The variable created in a condition is destroyed and created with each iteration of the loop. [*Example:*

```
struct A {
  int val;
  A(int i) : val(i) { }
  ~A() { }
  operator bool() { return val != 0; }
};
int i = 1;
while (A a = i) {
  // ...
  i = 0;
}
```

In the while-loop, the constructor and destructor are each called twice, once for the condition that succeeds and once for the condition that fails. — *end example*]

6.5.2 The `do` statement [stmt.do]

1 The expression is contextually converted to `bool` (Clause 4); if that conversion is ill-formed, the program is ill-formed.

2 In the `do` statement the substatement is executed repeatedly until the value of the expression becomes `false`. The test takes place after each execution of the statement.

6.5.3 The `for` statement [stmt.for]

1 The `for` statement

`for (` *for-init-statement* *condition*$_{opt}$`;` *expression*$_{opt}$`)` *statement*

is equivalent to

```
{
    for-init-statement
    while ( condition ) {
        statement
        expression ;
    }
}
```

except that names declared in the *for-init-statement* are in the same declarative region as those declared in the condition, and except that a `continue` in statement (not enclosed in another iteration statement) will execute expression before re-evaluating condition. [*Note:* Thus the first statement specifies initialization for the loop; the condition (6.4) specifies a test, made before each iteration, such that the loop is exited when the condition becomes `false`; the expression often specifies incrementing that is done after each iteration. — *end note*]

2 Either or both of the condition and the expression can be omitted. A missing condition makes the implied `while` clause equivalent to `while(true)`.

3 If the *for-init-statement* is a declaration, the scope of the name(s) declared extends to the end of the `for` statement. [*Example:*

```
int i = 42;
int a[10];

for (int i = 0; i < 10; i++)
  a[i] = i;

int j = i;          // j = 42
```

— *end example*]

6.5.4 The range-based `for` statement [stmt.ranged]

1 For a range-based `for` statement of the form

`for (` *for-range-declaration* `:` *expression* `)` *statement*

let *range-init* be equivalent to the *expression* surrounded by parentheses[91]

`(` *expression* `)`

91) this ensures that a top-level comma operator cannot be reinterpreted as a delimiter between *init-declarator*s in the declaration of `__range`.

and for a range-based `for` statement of the form

`for (` *for-range-declaration* `:` *braced-init-list* `)` *statement*

let *range-init* be equivalent to the *braced-init-list*. In each case, a range-based `for` statement is equivalent to

```
{
  auto && __range = range-init;
  for ( auto __begin = begin-expr,
             __end = end-expr;
        __begin != __end;
        ++__begin ) {
    for-range-declaration = *__begin;
    statement
  }
}
```

where `__range`, `__begin`, and `__end` are variables defined for exposition only, and `_RangeT` is the type of the expression, and *begin-expr* and *end-expr* are determined as follows:

(1.1) — if `_RangeT` is an array type, *begin-expr* and *end-expr* are `__range` and `__range + __bound`, respectively, where `__bound` is the array bound. If `_RangeT` is an array of unknown size or an array of incomplete type, the program is ill-formed;

(1.2) — if `_RangeT` is a class type, the *unqualified-id*s `begin` and `end` are looked up in the scope of class `_RangeT` as if by class member access lookup (3.4.5), and if either (or both) finds at least one declaration, *begin-expr* and *end-expr* are `__range.begin()` and `__range.end()`, respectively;

(1.3) — otherwise, *begin-expr* and *end-expr* are `begin(__range)` and `end(__range)`, respectively, where `begin` and `end` are looked up in the associated namespaces (3.4.2). [*Note:* Ordinary unqualified lookup (3.4.1) is not performed. — *end note*]

[*Example:*

```
int array[5] = { 1, 2, 3, 4, 5 };
for (int& x : array)
  x *= 2;
```

— *end example*]

2 In the *decl-specifier-seq* of a *for-range-declaration*, each *decl-specifier* shall be either a *type-specifier* or `constexpr`. The *decl-specifier-seq* shall not define a class or enumeration.

6.6 Jump statements [stmt.jump]

1 Jump statements unconditionally transfer control.

jump-statement:
`break ;`
`continue ;`
`return` *expression*$_{opt}$`;`
`return` *braced-init-list* `;`
`goto` *identifier* `;`

2 On exit from a scope (however accomplished), objects with automatic storage duration (3.7.3) that have been constructed in that scope are destroyed in the reverse order of their construction. [*Note:* For temporaries, see 12.2. — *end note*] Transfer out of a loop, out of a block, or back past an initialized variable with automatic storage duration involves the destruction of objects with automatic storage duration that are in scope at the point transferred from but not at the point transferred to. (See 6.7 for transfers into blocks).

[*Note:* However, the program can be terminated (by calling `std::exit()` or `std::abort()` (18.5), for example) without destroying class objects with automatic storage duration. — *end note*]

6.6.1 The `break` statement [stmt.break]

1 The `break` statement shall occur only in an *iteration-statement* or a `switch` statement and causes termination of the smallest enclosing *iteration-statement* or `switch` statement; control passes to the statement following the terminated statement, if any.

6.6.2 The `continue` statement [stmt.cont]

1 The `continue` statement shall occur only in an *iteration-statement* and causes control to pass to the loop-continuation portion of the smallest enclosing *iteration-statement*, that is, to the end of the loop. More precisely, in each of the statements

```
while (foo) {
  {
    // ...
  }
contin: ;
}
```

```
do {
  {
    // ...
  }
contin: ;
} while (foo);
```

```
for (;;) {
  {
    // ...
  }
contin: ;
}
```

a `continue` not contained in an enclosed iteration statement is equivalent to `goto contin`.

6.6.3 The `return` statement [stmt.return]

1 A function returns to its caller by the `return` statement.

2 The *expression* or *braced-init-list* of a return statement is called its operand. A return statement with no operand shall be used only in a function whose return type is *cv* `void`, a constructor (12.1), or a destructor (12.4). A return statement with an operand of type `void` shall be used only in a function whose return type is *cv* `void`. A return statement with any other operand shall be used only in a function whose return type is not *cv* `void`; the return statement initializes the object or reference to be returned by copy-initialization (8.5) from the operand. [*Note:* A return statement can involve the construction and copy or move of a temporary object (12.2). A copy or move operation associated with a return statement may be elided or considered as an rvalue for the purpose of overload resolution in selecting a constructor (12.8). — *end note*] [*Example:*

```
std::pair<std::string,int> f(const char* p, int x) {
  return {p,x};
}
```

— *end example*] Flowing off the end of a function is equivalent to a `return` with no value; this results in undefined behavior in a value-returning function.

3 The copy-initialization of the returned entity is sequenced before the destruction of temporaries at the end of the full-expression established by the operand of the return statement, which, in turn, is sequenced before the destruction of local variables (6.6) of the block enclosing the return statement.

6.6.4 The `goto` statement [stmt.goto]

1 The `goto` statement unconditionally transfers control to the statement labeled by the identifier. The identifier shall be a label (6.1) located in the current function.

6.7 Declaration statement [stmt.dcl]

1 A declaration statement introduces one or more new identifiers into a block; it has the form

declaration-statement:
block-declaration

If an identifier introduced by a declaration was previously declared in an outer block, the outer declaration is hidden for the remainder of the block, after which it resumes its force.

2 Variables with automatic storage duration (3.7.3) are initialized each time their *declaration-statement* is executed. Variables with automatic storage duration declared in the block are destroyed on exit from the block (6.6).

3 It is possible to transfer into a block, but not in a way that bypasses declarations with initialization. A program that jumps[92] from a point where a variable with automatic storage duration is not in scope to a point where it is in scope is ill-formed unless the variable has scalar type, class type with a trivial default constructor and a trivial destructor, a cv-qualified version of one of these types, or an array of one of the preceding types and is declared without an initializer (8.5). [*Example:*

```
void f() {
  // ...
  goto lx;          // ill-formed: jump into scope of a
  // ...
ly:
  X a = 1;
  // ...
lx:
  goto ly;          // OK, jump implies destructor
                    // call for a followed by construction
                    // again immediately following label ly
}
```

— *end example*]

4 The zero-initialization (8.5) of all block-scope variables with static storage duration (3.7.1) or thread storage duration (3.7.2) is performed before any other initialization takes place. Constant initialization (3.6.2) of a block-scope entity with static storage duration, if applicable, is performed before its block is first entered. An implementation is permitted to perform early initialization of other block-scope variables with static or thread storage duration under the same conditions that an implementation is permitted to statically initialize a variable with static or thread storage duration in namespace scope (3.6.2). Otherwise such a variable is initialized the first time control passes through its declaration; such a variable is considered initialized upon the completion of its initialization. If the initialization exits by throwing an exception, the initialization is not complete, so it will be tried again the next time control enters the declaration. If control enters the declaration concurrently while the variable is being initialized, the concurrent execution shall wait for completion of the initialization.[93] If control re-enters the declaration recursively while the variable is being initialized, the behavior is undefined. [*Example:*

```
int foo(int i) {
  static int s = foo(2*i);      // recursive call - undefined
  return i+1;
}
```

— *end example*]

5 The destructor for a block-scope object with static or thread storage duration will be executed if and only if it was constructed. [*Note:* 3.6.3 describes the order in which block-scope objects with static and thread storage duration are destroyed. — *end note*]

92) The transfer from the condition of a `switch` statement to a `case` label is considered a jump in this respect.
93) The implementation must not introduce any deadlock around execution of the initializer.

6.8 Ambiguity resolution [stmt.ambig]

1 There is an ambiguity in the grammar involving *expression-statement*s and *declaration*s: An *expression-statement* with a function-style explicit type conversion (5.2.3) as its leftmost subexpression can be indistinguishable from a *declaration* where the first *declarator* starts with a `(`. In those cases the *statement* is a *declaration*.

2 [*Note:* If the *statement* cannot syntactically be a *declaration*, there is no ambiguity, so this rule does not apply. The whole *statement* might need to be examined to determine whether this is the case. This resolves the meaning of many examples. [*Example:* Assuming `T` is a *simple-type-specifier* (7.1.6),

```
T(a)->m = 7;          // expression-statement
T(a)++;               // expression-statement
T(a,5)<<c;            // expression-statement

T(*d)(int);           // declaration
T(e)[5];              // declaration
T(f) = { 1, 2 };      // declaration
T(*g)(double(3));     // declaration
```

In the last example above, `g`, which is a pointer to `T`, is initialized to `double(3)`. This is of course ill-formed for semantic reasons, but that does not affect the syntactic analysis. — *end example*]

The remaining cases are *declaration*s. [*Example:*

```
class T {
  // ...
public:
  T();
  T(int);
  T(int, int);
};
T(a);                 // declaration
T(*b)();              // declaration
T(c)=7;               // declaration
T(d),e,f=3;           // declaration
extern int h;
T(g)(h,2);            // declaration
```

— *end example*] — *end note*]

3 The disambiguation is purely syntactic; that is, the meaning of the names occurring in such a statement, beyond whether they are *type-name*s or not, is not generally used in or changed by the disambiguation. Class templates are instantiated as necessary to determine if a qualified name is a *type-name*. Disambiguation precedes parsing, and a statement disambiguated as a declaration may be an ill-formed declaration. If, during parsing, a name in a template parameter is bound differently than it would be bound during a trial parse, the program is ill-formed. No diagnostic is required. [*Note:* This can occur only when the name is declared earlier in the declaration. — *end note*] [*Example:*

```
struct T1 {
  T1 operator()(int x) { return T1(x); }
  int operator=(int x) { return x; }
  T1(int) { }
};
struct T2 { T2(int){ } };
int a, (*(*b)(T2))(int), c, d;

void f() {
```

```
    // disambiguation requires this to be parsed as a declaration:
    T1(a) = 3,
    T2(4),                          // T2 will be declared as
    (*(*b)(T2(c)))(int(d));         // a variable of type T1
                                    // but this will not allow
                                    // the last part of the
                                    // declaration to parse
                                    // properly since it depends
                                    // on T2 being a type-name
}
```

— *end example*]

7 Declarations [dcl.dcl]

1 Declarations generally specify how names are to be interpreted. Declarations have the form

declaration-seq:
declaration
declaration-seq declaration

declaration:
block-declaration
function-definition
template-declaration
explicit-instantiation
explicit-specialization
linkage-specification
namespace-definition
empty-declaration
attribute-declaration

block-declaration:
simple-declaration
asm-definition
namespace-alias-definition
using-declaration
using-directive
static_assert-declaration
alias-declaration
opaque-enum-declaration

alias-declaration:
`using` *identifier attribute-specifier-seq*$_{opt}$ `=` *type-id* `;`

simple-declaration:
decl-specifier-seq$_{opt}$ *init-declarator-list*$_{opt}$ `;`
attribute-specifier-seq decl-specifier-seq$_{opt}$ *init-declarator-list* `;`

static_assert-declaration:
`static_assert (` *constant-expression* `) ;`
`static_assert (` *constant-expression* `,` *string-literal* `) ;`

empty-declaration:
`;`

attribute-declaration:
attribute-specifier-seq `;`

[*Note:* *asm-definition*s are described in 7.4, and *linkage-specification*s are described in 7.5. *Function-definition*s are described in 8.4 and *template-declaration*s are described in Clause 14. *Namespace-definition*s are described in 7.3.1, *using-declaration*s are described in 7.3.3 and *using-directive*s are described in 7.3.4. — *end note*]

2 The *simple-declaration*

attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *init-declarator-list*$_{opt}$ `;`

is divided into three parts. Attributes are described in 7.6. *decl-specifier*s, the principal components of a *decl-specifier-seq*, are described in 7.1. *declarator*s, the components of an *init-declarator-list*, are described in Clause 8. The *attribute-specifier-seq* in a *simple-declaration* appertains to each of the entities declared by the *declarator*s of the *init-declarator-list*. [*Note:* In the declaration for an entity, attributes appertaining

to that entity may appear at the start of the declaration and after the *declarator-id* for that declaration. — *end note*] [*Example:*

```
[[noreturn]] void f [[noreturn]] (); // OK
```

— *end example*]

3 Except where otherwise specified, the meaning of an *attribute-declaration* is implementation-defined.

4 A declaration occurs in a scope (3.3); the scope rules are summarized in 3.4. A declaration that declares a function or defines a class, namespace, template, or function also has one or more scopes nested within it. These nested scopes, in turn, can have declarations nested within them. Unless otherwise stated, utterances in Clause 7 about components in, of, or contained by a declaration or subcomponent thereof refer only to those components of the declaration that are *not* nested within scopes nested within the declaration.

5 In a *simple-declaration*, the optional *init-declarator-list* can be omitted only when declaring a class (Clause 9) or enumeration (7.2), that is, when the *decl-specifier-seq* contains either a *class-specifier*, an *elaborated-type-specifier* with a *class-key* (9.1), or an *enum-specifier*. In these cases and whenever a *class-specifier* or *enum-specifier* is present in the *decl-specifier-seq*, the identifiers in these specifiers are among the names being declared by the declaration (as *class-name*s, *enum-name*s, or *enumerator*s, depending on the syntax). In such cases, the *decl-specifier-seq* shall introduce one or more names into the program, or shall redeclare a name introduced by a previous declaration. [*Example:*

```
enum { };           // ill-formed
typedef class { };  // ill-formed
```

— *end example*]

6 In a *static_assert-declaration* the *constant-expression* shall be a constant expression (5.20) that can be contextually converted to `bool` (Clause 4). If the value of the expression when so converted is `true`, the declaration has no effect. Otherwise, the program is ill-formed, and the resulting diagnostic message (1.4) shall include the text of the *string-literal*, if one is supplied, except that characters not in the basic source character set (2.3) are not required to appear in the diagnostic message. [*Example:*

```
static_assert(char(-1) < 0, "this library requires plain 'char' to be signed");
```

— *end example*]

7 An *empty-declaration* has no effect.

8 Each *init-declarator* in the *init-declarator-list* contains exactly one *declarator-id*, which is the name declared by that *init-declarator* and hence one of the names declared by the declaration. The *type-specifiers* (7.1.6) in the *decl-specifier-seq* and the recursive *declarator* structure of the *init-declarator* describe a type (8.3), which is then associated with the name being declared by the *init-declarator*.

9 If the *decl-specifier-seq* contains the `typedef` specifier, the declaration is called a *typedef declaration* and the name of each *init-declarator* is declared to be a *typedef-name*, synonymous with its associated type (7.1.3). If the *decl-specifier-seq* contains no `typedef` specifier, the declaration is called a *function declaration* if the type associated with the name is a function type (8.3.5) and an *object declaration* otherwise.

10 Syntactic components beyond those found in the general form of declaration are added to a function declaration to make a *function-definition*. An object declaration, however, is also a definition unless it contains the `extern` specifier and has no initializer (3.1). A definition causes the appropriate amount of storage to be reserved and any appropriate initialization (8.5) to be done.

11 Only in function declarations for constructors, destructors, and type conversions can the *decl-specifier-seq* be omitted.[94]

94) The "implicit int" rule of C is no longer supported.

7.1 Specifiers [dcl.spec]

1 The specifiers that can be used in a declaration are

decl-specifier:
storage-class-specifier
type-specifier
function-specifier
`friend`
`typedef`
`constexpr`

decl-specifier-seq:
decl-specifier attribute-specifier-seq$_{opt}$
decl-specifier decl-specifier-seq

The optional *attribute-specifier-seq* in a *decl-specifier-seq* appertains to the type determined by the preceding *decl-specifier*s (8.3). The *attribute-specifier-seq* affects the type only for the declaration it appears in, not other declarations involving the same type.

2 Each *decl-specifier* shall appear at most once in the complete *decl-specifier-seq* of a declaration, except that `long` may appear twice.

3 If a *type-name* is encountered while parsing a *decl-specifier-seq*, it is interpreted as part of the *decl-specifier-seq* if and only if there is no previous *type-specifier* other than a *cv-qualifier* in the *decl-specifier-seq*. The sequence shall be self-consistent as described below. [*Example:*

```
typedef char* Pc;
static Pc;                          // error: name missing
```

Here, the declaration `static Pc` is ill-formed because no name was specified for the static variable of type `Pc`. To get a variable called `Pc`, a *type-specifier* (other than `const` or `volatile`) has to be present to indicate that the *typedef-name* `Pc` is the name being (re)declared, rather than being part of the *decl-specifier* sequence. For another example,

```
void f(const Pc);                   // void f(char* const) (not const char*)
void g(const int Pc);               // void g(const int)
```

— *end example*]

4 [*Note:* Since `signed`, `unsigned`, `long`, and `short` by default imply `int`, a *type-name* appearing after one of those specifiers is treated as the name being (re)declared. [*Example:*

```
void h(unsigned Pc);                // void h(unsigned int)
void k(unsigned int Pc);            // void k(unsigned int)
```

— *end example*] — *end note*]

7.1.1 Storage class specifiers [dcl.stc]

1 The storage class specifiers are

storage-class-specifier:
`register`
`static`
`thread_local`
`extern`
`mutable`

At most one *storage-class-specifier* shall appear in a given *decl-specifier-seq*, except that `thread_local` may appear with `static` or `extern`. If `thread_local` appears in any declaration of a variable it shall be present in all declarations of that entity. If a *storage-class-specifier* appears in a *decl-specifier-seq*, there can be

no `typedef` specifier in the same *decl-specifier-seq* and the *init-declarator-list* of the declaration shall not be empty (except for an anonymous union declared in a named namespace or in the global namespace, which shall be declared `static` (9.5)). The *storage-class-specifier* applies to the name declared by each *init-declarator* in the list and not to any names declared by other specifiers. A *storage-class-specifier* other than `thread_local` shall not be specified in an explicit specialization (14.7.3) or an explicit instantiation (14.7.2) directive.

2 The `register` specifier shall be applied only to names of variables declared in a block (6.3) or to function parameters (8.4). It specifies that the named variable has automatic storage duration (3.7.3). A variable declared without a *storage-class-specifier* at block scope or declared as a function parameter has automatic storage duration by default.

3 A `register` specifier is a hint to the implementation that the variable so declared will be heavily used. [*Note:* The hint can be ignored and in most implementations it will be ignored if the address of the variable is taken. This use is deprecated (see D.2). — *end note*]

4 The `thread_local` specifier indicates that the named entity has thread storage duration (3.7.2). It shall be applied only to the names of variables of namespace or block scope and to the names of static data members. When `thread_local` is applied to a variable of block scope the *storage-class-specifier* `static` is implied if no other *storage-class-specifier* appears in the *decl-specifier-seq*.

5 The `static` specifier can be applied only to names of variables and functions and to anonymous unions (9.5). There can be no `static` function declarations within a block, nor any `static` function parameters. A `static` specifier used in the declaration of a variable declares the variable to have static storage duration (3.7.1), unless accompanied by the `thread_local` specifier, which declares the variable to have thread storage duration (3.7.2). A `static` specifier can be used in declarations of class members; 9.4 describes its effect. For the linkage of a name declared with a `static` specifier, see 3.5.

6 The `extern` specifier can be applied only to the names of variables and functions. The `extern` specifier cannot be used in the declaration of class members or function parameters. For the linkage of a name declared with an `extern` specifier, see 3.5. [*Note:* The `extern` keyword can also be used in *explicit-instantiation*s and *linkage-specification*s, but it is not a *storage-class-specifier* in such contexts. — *end note*]

7 The linkages implied by successive declarations for a given entity shall agree. That is, within a given scope, each declaration declaring the same variable name or the same overloading of a function name shall imply the same linkage. Each function in a given set of overloaded functions can have a different linkage, however. [*Example:*

```
static char* f();               // f() has internal linkage
char* f()                       // f() still has internal linkage
  { /* ... */ }

char* g();                      // g() has external linkage
static char* g()                // error: inconsistent linkage
  { /* ... */ }

void h();
inline void h();                // external linkage

inline void l();
void l();                       // external linkage

inline void m();
extern void m();                // external linkage

static void n();
inline void n();                // internal linkage
```

```
static int a;                   // a has internal linkage
int a;                          // error: two definitions

static int b;                   // b has internal linkage
extern int b;                   // b still has internal linkage

int c;                          // c has external linkage
static int c;                   // error: inconsistent linkage

extern int d;                   // d has external linkage
static int d;                   // error: inconsistent linkage
```

— *end example*]

8 The name of a declared but undefined class can be used in an `extern` declaration. Such a declaration can only be used in ways that do not require a complete class type. [*Example:*

```
struct S;
extern S a;
extern S f();
extern void g(S);

void h() {
  g(a);                         // error: S is incomplete
  f();                          // error: S is incomplete
}
```

— *end example*]

9 The `mutable` specifier shall appear only in the declaration of a non-static data member (9.2) whose type is neither const-qualified nor a reference type. [*Example:*

```
class X {
  mutable const int* p;         // OK
  mutable int* const q;         // ill-formed
};
```

— *end example*]

10 The `mutable` specifier on a class data member nullifies a `const` specifier applied to the containing class object and permits modification of the mutable class member even though the rest of the object is `const` (7.1.6.1).

7.1.2 Function specifiers [dcl.fct.spec]

1 *Function-specifiers* can be used only in function declarations.

function-specifier:
`inline`
`virtual`
`explicit`

2 A function declaration (8.3.5, 9.3, 11.3) with an `inline` specifier declares an *inline function*. The inline specifier indicates to the implementation that inline substitution of the function body at the point of call is to be preferred to the usual function call mechanism. An implementation is not required to perform this inline substitution at the point of call; however, even if this inline substitution is omitted, the other rules for inline functions defined by 7.1.2 shall still be respected.

3 A function defined within a class definition is an inline function. The `inline` specifier shall not appear on a block scope function declaration.[95] If the `inline` specifier is used in a friend declaration, that declaration shall be a definition or the function shall have previously been declared inline.

4 An inline function shall be defined in every translation unit in which it is odr-used and shall have exactly the same definition in every case (3.2). [*Note:* A call to the inline function may be encountered before its definition appears in the translation unit. — *end note*] If the definition of a function appears in a translation unit before its first declaration as inline, the program is ill-formed. If a function with external linkage is declared inline in one translation unit, it shall be declared inline in all translation units in which it appears; no diagnostic is required. An `inline` function with external linkage shall have the same address in all translation units. A `static` local variable in an `extern inline` function always refers to the same object. A type defined within the body of an `extern inline` function is the same type in every translation unit.

5 The `virtual` specifier shall be used only in the initial declaration of a non-static class member function; see 10.3.

6 The `explicit` specifier shall be used only in the declaration of a constructor or conversion function within its class definition; see 12.3.1 and 12.3.2.

7.1.3 The `typedef` specifier [dcl.typedef]

1 Declarations containing the *decl-specifier* `typedef` declare identifiers that can be used later for naming fundamental (3.9.1) or compound (3.9.2) types. The `typedef` specifier shall not be combined in a *decl-specifier-seq* with any other kind of specifier except a *type-specifier,* and it shall not be used in the *decl-specifier-seq* of a *parameter-declaration* (8.3.5) nor in the *decl-specifier-seq* of a *function-definition* (8.4).

typedef-name:
identifier

A name declared with the `typedef` specifier becomes a *typedef-name*. Within the scope of its declaration, a *typedef-name* is syntactically equivalent to a keyword and names the type associated with the identifier in the way described in Clause 8. A *typedef-name* is thus a synonym for another type. A *typedef-name* does not introduce a new type the way a class declaration (9.1) or enum declaration does. [*Example:* after

```
typedef int MILES, *KLICKSP;
```

the constructions

```
MILES distance;
extern KLICKSP metricp;
```

are all correct declarations; the type of `distance` is `int` and that of `metricp` is "pointer to `int`". — *end example*]

2 A *typedef-name* can also be introduced by an *alias-declaration*. The *identifier* following the `using` keyword becomes a *typedef-name* and the optional *attribute-specifier-seq* following the *identifier* appertains to that *typedef-name*. It has the same semantics as if it were introduced by the `typedef` specifier. In particular, it does not define a new type. [*Example:*

```
using handler_t = void (*)(int);
extern handler_t ignore;
extern void (*ignore)(int);          // redeclare ignore
using cell = pair<void*, cell*>;     // ill-formed
```

— *end example*]

3 In a given non-class scope, a `typedef` specifier can be used to redefine the name of any type declared in that scope to refer to the type to which it already refers. [*Example:*

95) The inline keyword has no effect on the linkage of a function.

```
typedef struct s { /* ... */ } s;
typedef int I;
typedef int I;
typedef I I;
```

— *end example*]

4 In a given class scope, a `typedef` specifier can be used to redefine any *class-name* declared in that scope that is not also a *typedef-name* to refer to the type to which it already refers. [*Example:*

```
struct S {
  typedef struct A { } A;       // OK
  typedef struct B B;           // OK
  typedef A A;                  // error
};
```

— *end example*]

5 If a `typedef` specifier is used to redefine in a given scope an entity that can be referenced using an *elaborated-type-specifier*, the entity can continue to be referenced by an *elaborated-type-specifier* or as an enumeration or class name in an enumeration or class definition respectively. [*Example:*

```
struct S;
typedef struct S S;
int main() {
  struct S* p;                  // OK
}
struct S { };                   // OK
```

— *end example*]

6 In a given scope, a `typedef` specifier shall not be used to redefine the name of any type declared in that scope to refer to a different type. [*Example:*

```
class complex { /* ... */ };
typedef int complex;            // error: redefinition
```

— *end example*]

7 Similarly, in a given scope, a class or enumeration shall not be declared with the same name as a *typedef-name* that is declared in that scope and refers to a type other than the class or enumeration itself. [*Example:*

```
typedef int complex;
class complex { /* ...  */ };   // error: redefinition
```

— *end example*]

8 [*Note:* A *typedef-name* that names a class type, or a cv-qualified version thereof, is also a *class-name* (9.1). If a *typedef-name* is used to identify the subject of an *elaborated-type-specifier* (7.1.6.3), a class definition (Clause 9), a constructor declaration (12.1), or a destructor declaration (12.4), the program is ill-formed. — *end note*] [*Example:*

```
struct S {
  S();
  ~S();
};

typedef struct S T;

S a = T();                      // OK
struct T * p;                   // error
```

— *end example*]

9 If the typedef declaration defines an unnamed class (or enum), the first *typedef-name* declared by the declaration to be that class type (or enum type) is used to denote the class type (or enum type) for linkage purposes only (3.5). [*Example:*

```
typedef struct { } *ps, S;      // S is the class name for linkage purposes
```

— *end example*]

7.1.4 The `friend` specifier [dcl.friend]

1 The `friend` specifier is used to specify access to class members; see 11.3.

7.1.5 The `constexpr` specifier [dcl.constexpr]

1 The `constexpr` specifier shall be applied only to the definition of a variable or variable template, the declaration of a function or function template, or the declaration of a static data member of a literal type (3.9). If any declaration of a function or function template has a `constexpr` specifier, then all its declarations shall contain the `constexpr` specifier. [*Note:* An explicit specialization can differ from the template declaration with respect to the `constexpr` specifier. — *end note*] [*Note:* Function parameters cannot be declared `constexpr`. — *end note*] [*Example:*

```
constexpr void square(int &x);  // OK: declaration
constexpr int bufsz = 1024;     // OK: definition
constexpr struct pixel {        // error: pixel is a type
  int x;
  int y;
  constexpr pixel(int);         // OK: declaration
};
constexpr pixel::pixel(int a)
  : x(a), y(x)                  // OK: definition
  { square(x); }
constexpr pixel small(2);       // error: square not defined, so small(2)
                                // not constant (5.20) so constexpr not satisfied

constexpr void square(int &x) { // OK: definition
  x *= x;
}
constexpr pixel large(4);       // OK: square defined
int next(constexpr int x) {     // error: not for parameters
     return x + 1;
}
extern constexpr int memsz;     // error: not a definition
```

— *end example*]

2 A `constexpr` specifier used in the declaration of a function that is not a constructor declares that function to be a *constexpr function*. Similarly, a `constexpr` specifier used in a constructor declaration declares that constructor to be a *constexpr constructor*. `constexpr` functions and `constexpr` constructors are implicitly inline functions (7.1.2).

3 The definition of a `constexpr` function shall satisfy the following constraints:

(3.1) — it shall not be virtual (10.3);

(3.2) — its return type shall be a literal type;

(3.3) — each of its parameter types shall be a literal type;

(3.4) — its *function-body* shall be `= delete`, `= default`, or a *compound-statement* that does not contain

(3.4.1) — an *asm-definition*,

(3.4.2) — a `goto` statement,

(3.4.3) — a *try-block*, or

(3.4.4) — a definition of a variable of non-literal type or of static or thread storage duration or for which no initialization is performed.

[*Example:*

```
constexpr int square(int x)
  { return x * x; }             // OK
constexpr long long_max()
  { return 2147483647; }        // OK
constexpr int abs(int x) {
  if (x < 0)
    x = -x;
  return x;                     // OK
}
constexpr int first(int n) {
  static int value = n;         // error: variable has static storage duration
  return value;
}
constexpr int uninit() {
  int a;                        // error: variable is uninitialized
  return a;
}
constexpr int prev(int x)
  { return --x; }               // OK
constexpr int g(int x, int n) { // OK
  int r = 1;
  while (--n > 0) r *= x;
  return r;
}
```

— *end example*]

4 The definition of a `constexpr` constructor shall satisfy the following constraints:

(4.1) — the class shall not have any virtual base classes;

(4.2) — each of the parameter types shall be a literal type;

(4.3) — its *function-body* shall not be a *function-try-block*;

In addition, either its *function-body* shall be `= delete`, or it shall satisfy the following constraints:

(4.4) — either its *function-body* shall be `= default`, or the *compound-statement* of its *function-body* shall satisfy the constraints for a *function-body* of a `constexpr` function;

(4.5) — every non-variant non-static data member and base class sub-object shall be initialized (12.6.2);

(4.6) — if the class is a union having variant members (9.5), exactly one of them shall be initialized;

(4.7) — if the class is a union-like class, but is not a union, for each of its anonymous union members having variant members, exactly one of them shall be initialized;

(4.8) — for a non-delegating constructor, every constructor selected to initialize non-static data members and base class sub-objects shall be a `constexpr` constructor;

(4.9) — for a delegating constructor, the target constructor shall be a `constexpr` constructor.

[*Example:*

```
struct Length {
  constexpr explicit Length(int i = 0) : val(i) { }
private:
  int val;
};
```

— *end example*]

5 For a non-template, non-defaulted `constexpr` function or a non-template, non-defaulted, non-inheriting `constexpr` constructor, if no argument values exist such that an invocation of the function or constructor could be an evaluated subexpression of a core constant expression (5.20), or, for a constructor, a constant initializer for some object (3.6.2), the program is ill-formed; no diagnostic required. [*Example:*

```
constexpr int f(bool b)
  { return b ? throw 0 : 0; }                // OK
constexpr int f() { return f(true); }        // ill-formed, no diagnostic required

struct B {
  constexpr B(int x) : i(0) { }              // x is unused
  int i;
};

int global;

struct D : B {
  constexpr D() : B(global) { }              // ill-formed, no diagnostic required
                                             // lvalue-to-rvalue conversion on non-constant global
};
```

— *end example*]

6 If the instantiated template specialization of a `constexpr` function template or member function of a class template would fail to satisfy the requirements for a `constexpr` function or `constexpr` constructor, that specialization is still a `constexpr` function or `constexpr` constructor, even though a call to such a function cannot appear in a constant expression. If no specialization of the template would satisfy the requirements for a `constexpr` function or `constexpr` constructor when considered as a non-template function or constructor, the template is ill-formed; no diagnostic required.

7 A call to a `constexpr` function produces the same result as a call to an equivalent non-`constexpr` function in all respects except that a call to a `constexpr` function can appear in a constant expression.

8 The `constexpr` specifier has no effect on the type of a `constexpr` function or a `constexpr` constructor. [*Example:*

```
constexpr int bar(int x, int y) // OK
    { return x + y + x*y; }
// ...
int bar(int x, int y)           // error: redefinition of bar
    { return x * 2 + 3 * y; }
```

— *end example*]

9 A `constexpr` specifier used in an object declaration declares the object as `const`. Such an object shall have literal type and shall be initialized. If it is initialized by a constructor call, that call shall be a constant

expression (5.20). Otherwise, or if a `constexpr` specifier is used in a reference declaration, every full-expression that appears in its initializer shall be a constant expression. [*Note:* Each implicit conversion used in converting the initializer expressions and each constructor call used for the initialization is part of such a full-expression. — *end note*] [*Example:*

```
struct pixel {
  int x, y;
};
constexpr pixel ur = { 1294, 1024 };// OK
constexpr pixel origin;             // error: initializer missing
```

— *end example*]

7.1.6 Type specifiers [dcl.type]

1 The type-specifiers are

type-specifier:
trailing-type-specifier
class-specifier
enum-specifier

trailing-type-specifier:
simple-type-specifier
elaborated-type-specifier
typename-specifier
cv-qualifier

type-specifier-seq:
type-specifier attribute-specifier-seq$_{opt}$
type-specifier type-specifier-seq

trailing-type-specifier-seq:
trailing-type-specifier attribute-specifier-seq$_{opt}$
trailing-type-specifier trailing-type-specifier-seq

The optional *attribute-specifier-seq* in a *type-specifier-seq* or a *trailing-type-specifier-seq* appertains to the type denoted by the preceding *type-specifier*s (8.3). The *attribute-specifier-seq* affects the type only for the declaration it appears in, not other declarations involving the same type.

2 As a general rule, at most one *type-specifier* is allowed in the complete *decl-specifier-seq* of a *declaration* or in a *type-specifier-seq* or *trailing-type-specifier-seq*. The only exceptions to this rule are the following:

(2.1) — `const` can be combined with any type specifier except itself.

(2.2) — `volatile` can be combined with any type specifier except itself.

(2.3) — `signed` or `unsigned` can be combined with `char`, `long`, `short`, or `int`.

(2.4) — `short` or `long` can be combined with `int`.

(2.5) — `long` can be combined with `double`.

(2.6) — `long` can be combined with `long`.

3 Except in a declaration of a constructor, destructor, or conversion function, at least one *type-specifier* that is not a *cv-qualifier* shall appear in a complete *type-specifier-seq* or a complete *decl-specifier-seq*.[96] A *type-specifier-seq* shall not define a class or enumeration unless it appears in the *type-id* of an *alias-declaration* (7.1.3) that is not the *declaration* of a *template-declaration*.

96) There is no special provision for a *decl-specifier-seq* that lacks a *type-specifier* or that has a *type-specifier* that only specifies *cv-qualifier*s. The "implicit int" rule of C is no longer supported.

4 [*Note:* *enum-specifier*s, *class-specifier*s, and *typename-specifier*s are discussed in 7.2, Clause 9, and 14.6, respectively. The remaining *type-specifier*s are discussed in the rest of this section. — *end note*]

7.1.6.1 The *cv-qualifiers* [dcl.type.cv]

1 There are two *cv-qualifiers*, `const` and `volatile`. Each *cv-qualifier* shall appear at most once in a *cv-qualifier-seq*. If a *cv-qualifier* appears in a *decl-specifier-seq*, the *init-declarator-list* of the declaration shall not be empty. [*Note:* 3.9.3 and 8.3.5 describe how cv-qualifiers affect object and function types. — *end note*] Redundant cv-qualifications are ignored. [*Note:* For example, these could be introduced by typedefs. — *end note*]

2 [*Note:* Declaring a variable `const` can affect its linkage (7.1.1) and its usability in constant expressions (5.20). As described in 8.5, the definition of an object or subobject of const-qualified type must specify an initializer or be subject to default-initialization. — *end note*]

3 A pointer or reference to a cv-qualified type need not actually point or refer to a cv-qualified object, but it is treated as if it does; a const-qualified access path cannot be used to modify an object even if the object referenced is a non-const object and can be modified through some other access path. [*Note:* Cv-qualifiers are supported by the type system so that they cannot be subverted without casting (5.2.11). — *end note*]

4 Except that any class member declared `mutable` (7.1.1) can be modified, any attempt to modify a `const` object during its lifetime (3.8) results in undefined behavior. [*Example:*

```
const int ci = 3;               // cv-qualified (initialized as required)
ci = 4;                         // ill-formed: attempt to modify const

int i = 2;                      // not cv-qualified
const int* cip;                 // pointer to const int
cip = &i;                       // OK: cv-qualified access path to unqualified
*cip = 4;                       // ill-formed: attempt to modify through ptr to const

int* ip;
ip = const_cast<int*>(cip);     // cast needed to convert const int* to int*
*ip = 4;                        // defined: *ip points to i, a non-const object

const int* ciq = new const int (3);     // initialized as required
int* iq = const_cast<int*>(ciq);        // cast required
*iq = 4;                                // undefined: modifies a const object
```

5 For another example

```
struct X {
  mutable int i;
  int j;
};
struct Y {
  X x;
  Y();
};

const Y y;
y.x.i++;                        // well-formed: mutable member can be modified
y.x.j++;                        // ill-formed: const-qualified member modified
Y* p = const_cast<Y*>(&y);      // cast away const-ness of y
p->x.i = 99;                    // well-formed: mutable member can be modified
p->x.j = 99;                    // undefined: modifies a const member
```

— *end example*]

6 What constitutes an access to an object that has volatile-qualified type is implementation-defined. If an attempt is made to refer to an object defined with a volatile-qualified type through the use of a glvalue with a non-volatile-qualified type, the program behavior is undefined.

7 [*Note:* `volatile` is a hint to the implementation to avoid aggressive optimization involving the object because the value of the object might be changed by means undetectable by an implementation. Furthermore, for some implementations, `volatile` might indicate that special hardware instructions are required to access the object. See 1.9 for detailed semantics. In general, the semantics of `volatile` are intended to be the same in C++ as they are in C. — *end note*]

7.1.6.2 Simple type specifiers [dcl.type.simple]

1 The simple type specifiers are

simple-type-specifier:
nested-name-specifier$_{opt}$ *type-name*
nested-name-specifier `template` *simple-template-id*
`char`
`char16_t`
`char32_t`
`wchar_t`
`bool`
`short`
`int`
`long`
`signed`
`unsigned`
`float`
`double`
`void`
`auto`
decltype-specifier

type-name:
class-name
enum-name
typedef-name
simple-template-id

decltype-specifier:
`decltype (` *expression* `)`
`decltype ( auto )`

2 The *simple-type-specifier* `auto` is a placeholder for a type to be deduced (7.1.6.4). The other *simple-type-specifier*s specify either a previously-declared type, a type determined from an expression, or one of the fundamental types (3.9.1). Table 9 summarizes the valid combinations of *simple-type-specifier*s and the types they specify.

3 When multiple *simple-type-specifiers* are allowed, they can be freely intermixed with other *decl-specifiers* in any order. [*Note:* It is implementation-defined whether objects of `char` type are represented as signed or unsigned quantities. The `signed` specifier forces `char` objects to be signed; it is redundant in other contexts. — *end note*]

Table 9 — *simple-type-specifier*s and the types they specify

Specifier(s)	Type
type-name	the type named
simple-template-id	the type as defined in 14.2
char	“char”
unsigned char	“unsigned char”
signed char	“signed char”
char16_t	“char16_t”
char32_t	“char32_t”
bool	“bool”
unsigned	“unsigned int”
unsigned int	“unsigned int”
signed	“int”
signed int	“int”
int	“int”
unsigned short int	“unsigned short int”
unsigned short	“unsigned short int”
unsigned long int	“unsigned long int”
unsigned long	“unsigned long int”
unsigned long long int	“unsigned long long int”
unsigned long long	“unsigned long long int”
signed long int	“long int”
signed long	“long int”
signed long long int	“long long int”
signed long long	“long long int”
long long int	“long long int”
long long	“long long int”
long int	“long int”
long	“long int”
signed short int	“short int”
signed short	“short int”
short int	“short int”
short	“short int”
wchar_t	“wchar_t”
float	“float”
double	“double”
long double	“long double”
void	“void”
auto	placeholder for a type to be deduced
decltype(*expression*)	the type as defined below

4 For an expression `e`, the type denoted by `decltype(e)` is defined as follows:

(4.1) — if `e` is an unparenthesized *id-expression* or an unparenthesized class member access (5.2.5), `decltype(e)` is the type of the entity named by `e`. If there is no such entity, or if `e` names a set of overloaded functions, the program is ill-formed;

(4.2) — otherwise, if `e` is an xvalue, `decltype(e)` is `T&&`, where `T` is the type of `e`;

(4.3) — otherwise, if `e` is an lvalue, `decltype(e)` is `T&`, where `T` is the type of `e`;

(4.4) — otherwise, `decltype(e)` is the type of `e`.

The operand of the `decltype` specifier is an unevaluated operand (Clause 5).

[*Example:*

```
const int&& foo();
int i;
struct A { double x; };
const A* a = new A();
decltype(foo()) x1 = 17;        // type is const int&&
decltype(i) x2;                 // type is int
decltype(a->x) x3;              // type is double
decltype((a->x)) x4 = x3;       // type is const double&
```

— *end example*] [*Note:* The rules for determining types involving `decltype(auto)` are specified in 7.1.6.4. — *end note*]

5 [*Note:* in the case where the operand of a *decltype-specifier* is a function call and the return type of the function is a class type, a special rule (5.2.2) ensures that the return type is not required to be complete (as it would be if the call appeared in a sub-expression or outside of a *decltype-specifier*). In this context, the common purpose of writing the expression is merely to refer to its type. In that sense, a *decltype-specifier* is analogous to a use of a *typedef-name*, so the usual reasons for requiring a complete type do not apply. In particular, it is not necessary to allocate storage for a temporary object or to enforce the semantic constraints associated with invoking the type's destructor. [*Example:*

```
template<class T> struct A { ~A() = delete; };
template<class T> auto h()
  -> A<T>;
template<class T> auto i(T)     // identity
  -> T;
template<class T> auto f(T)     // #1
  -> decltype(i(h<T>()));       // forces completion of A<T> and implicitly uses
                                // A<T>::~A() for the temporary introduced by the
                                // use of h(). (A temporary is not introduced
                                // as a result of the use of i().)
template<class T> auto f(T)     // #2
  -> void;
auto g() -> void {
  f(42);                        // OK: calls #2. (#1 is not a viable candidate: type
                                // deduction fails (14.8.2) because A<int>::~A()
                                // is implicitly used in its decltype-specifier)
}
template<class T> auto q(T)
  -> decltype((h<T>()));        // does not force completion of A<T>; A<T>::~A() is
                                // not implicitly used within the context of this decltype-specifier
void r() {
```

```
  q(42);                          // Error: deduction against q succeeds, so overload resolution
                                  // selects the specialization "q(T) -> decltype((h<T>())) [with T=int]".
                                  // The return type is A<int>, so a temporary is introduced and its
                                  // destructor is used, so the program is ill-formed.
}
```

— *end example*] — *end note*]

7.1.6.3 Elaborated type specifiers [dcl.type.elab]

elaborated-type-specifier:
class-key attribute-specifier-seq$_{opt}$ *nested-name-specifier*$_{opt}$ *identifier*
class-key simple-template-id
class-key nested-name-specifier `template`$_{opt}$ *simple-template-id*
`enum` *nested-name-specifier*$_{opt}$ *identifier*

1 An *attribute-specifier-seq* shall not appear in an *elaborated-type-specifier* unless the latter is the sole constituent of a declaration. If an *elaborated-type-specifier* is the sole constituent of a declaration, the declaration is ill-formed unless it is an explicit specialization (14.7.3), an explicit instantiation (14.7.2) or it has one of the following forms:

class-key attribute-specifier-seq$_{opt}$ *identifier* `;`
`friend` *class-key* `::`$_{opt}$ *identifier* `;`
`friend` *class-key* `::`$_{opt}$ *simple-template-id* `;`
`friend` *class-key nested-name-specifier identifier* `;`
`friend` *class-key nested-name-specifier* `template`$_{opt}$ *simple-template-id* `;`

In the first case, the *attribute-specifier-seq*, if any, appertains to the class being declared; the attributes in the *attribute-specifier-seq* are thereafter considered attributes of the class whenever it is named.

2 3.4.4 describes how name lookup proceeds for the *identifier* in an *elaborated-type-specifier*. If the *identifier* resolves to a *class-name* or *enum-name*, the *elaborated-type-specifier* introduces it into the declaration the same way a *simple-type-specifier* introduces its *type-name*. If the *identifier* resolves to a *typedef-name* or the *simple-template-id* resolves to an alias template specialization, the *elaborated-type-specifier* is ill-formed. [*Note:* This implies that, within a class template with a template *type-parameter* `T`, the declaration

```
friend class T;
```

is ill-formed. However, the similar declaration `friend T;` is allowed (11.3). — *end note*]

3 The *class-key* or `enum` keyword present in the *elaborated-type-specifier* shall agree in kind with the declaration to which the name in the *elaborated-type-specifier* refers. This rule also applies to the form of *elaborated-type-specifier* that declares a *class-name* or `friend` class since it can be construed as referring to the definition of the class. Thus, in any *elaborated-type-specifier*, the `enum` keyword shall be used to refer to an enumeration (7.2), the `union` *class-key* shall be used to refer to a union (Clause 9), and either the `class` or `struct` *class-key* shall be used to refer to a class (Clause 9) declared using the `class` or `struct` *class-key*. [*Example:*

```
enum class E { a, b };
enum E x = E::a;                  // OK
```

— *end example*]

7.1.6.4 `auto` specifier [dcl.spec.auto]

1 The `auto` and `decltype(auto)` *type-specifier*s are used to designate a placeholder type that will be replaced later by deduction from an initializer. The `auto` *type-specifier* is also used to introduce a function type having a *trailing-return-type* or to signify that a lambda is a generic lambda.

2 The placeholder type can appear with a function declarator in the *decl-specifier-seq*, *type-specifier-seq*, *conversion-function-id*, or *trailing-return-type*, in any context where such a declarator is valid. If the function

declarator includes a *trailing-return-type* (8.3.5), that *trailing-return-type* specifies the declared return type of the function. Otherwise, the function declarator shall declare a function. If the declared return type of the function contains a placeholder type, the return type of the function is deduced from `return` statements in the body of the function, if any.

3 If the `auto` *type-specifier* appears as one of the *decl-specifier*s in the *decl-specifier-seq* of a *parameter-declaration* of a *lambda-expression*, the lambda is a *generic lambda* (5.1.2). [*Example:*

```
auto glambda = [](int i, auto a) { return i; }; // OK: a generic lambda
```

— *end example*]

4 The type of a variable declared using `auto` or `decltype(auto)` is deduced from its initializer. This use is allowed when declaring variables in a block (6.3), in namespace scope (3.3.6), and in a *for-init-statement* (6.5.3). `auto` or `decltype(auto)` shall appear as one of the *decl-specifier*s in the *decl-specifier-seq* and the *decl-specifier-seq* shall be followed by one or more *init-declarator*s, each of which shall have a non-empty *initializer*. In an *initializer* of the form

```
( expression-list )
```

the *expression-list* shall be a single *assignment-expression*.

[*Example:*

```
auto x = 5;                 // OK: x has type int
const auto *v = &x, u = 6;  // OK: v has type const int*, u has type const int
static auto y = 0.0;        // OK: y has type double
auto int r;                 // error: auto is not a storage-class-specifier
auto f() -> int;            // OK: f returns int
auto g() { return 0.0; }    // OK: g returns double
auto h();                   // OK: h's return type will be deduced when it is defined
```

— *end example*]

5 A placeholder type can also be used in declaring a variable in the *condition* of a selection statement (6.4) or an iteration statement (6.5), in the *type-specifier-seq* in the *new-type-id* or *type-id* of a *new-expression* (5.3.4), in a *for-range-declaration*, and in declaring a static data member with a *brace-or-equal-initializer* that appears within the *member-specification* of a class definition (9.4.2).

6 A program that uses `auto` or `decltype(auto)` in a context not explicitly allowed in this section is ill-formed.

7 When a variable declared using a placeholder type is initialized, or a `return` statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type or variable type is determined from the type of its initializer. In the case of a `return` with no operand or with an operand of type `void`, the declared return type shall be `auto` and the deduced return type is `void`. Otherwise, let `T` be the declared type of the variable or return type of the function. If the placeholder is the `auto` *type-specifier*, the deduced type is determined using the rules for template argument deduction. If the initialization is direct-list-initialization then the *braced-init-list* shall contain only a single *assignment-expression* `L`. If the deduction is for a `return` statement and the initializer is a *braced-init-list* (8.5.4), the program is ill-formed. Otherwise, obtain `P` from `T` by replacing the occurrences of `auto` with either a new invented type template parameter `U` or, if the initialization is copy-list-initialization, with `std::initializer_list<U>`. Deduce a value for `U` using the rules of template argument deduction from a function call (14.8.2.1), where `P` is a function template parameter type and the corresponding argument is the initializer, or `L` in the case of direct-list-initialization. If the deduction fails, the declaration is ill-formed. Otherwise, the type deduced for the variable or return type is obtained by substituting the deduced `U` into `P`. [*Example:*

```
auto x1 = { 1, 2 };         // decltype(x1) is std::initializer_list<int>
auto x2 = { 1, 2.0 };       // error: cannot deduce element type
```

```
auto x3{ 1, 2 };            // error: not a single element
auto x4 = { 3 };            // decltype(x4) is std::initializer_list<int>
auto x5{ 3 };               // decltype(x5) is int
```

— *end example*]

[*Example:*

```
const auto &i = expr;
```

The type of `i` is the deduced type of the parameter `u` in the call `f(expr)` of the following invented function template:

```
template <class U> void f(const U& u);
```

— *end example*]

If the placeholder is the `decltype(auto)` *type-specifier*, the declared type of the variable or return type of the function shall be the placeholder alone. The type deduced for the variable or return type is determined as described in 7.1.6.2, as though the *initializer-clause* or *expression-list* of the *initializer* or the *expression* of the `return` statement had been the operand of the `decltype`. [*Example:*

```
int i;
int&& f();
auto           x2a(i);         // decltype(x2a) is int
decltype(auto) x2d(i);         // decltype(x2d) is int
auto           x3a = i;        // decltype(x3a) is int
decltype(auto) x3d = i;        // decltype(x3d) is int
auto           x4a = (i);      // decltype(x4a) is int
decltype(auto) x4d = (i);      // decltype(x4d) is int&
auto           x5a = f();      // decltype(x5a) is int
decltype(auto) x5d = f();      // decltype(x5d) is int&&
auto           x6a = { 1, 2 }; // decltype(x6a) is std::initializer_list<int>
decltype(auto) x6d = { 1, 2 }; // error, { 1, 2 } is not an expression
auto          *x7a = &i;       // decltype(x7a) is int*
decltype(auto)*x7d = &i;       // error, declared type is not plain decltype(auto)
```

— *end example*]

8 If the *init-declarator-list* contains more than one *init-declarator*, they shall all form declarations of variables. The type of each declared variable is determined as described above, and if the type that replaces the placeholder type is not the same in each deduction, the program is ill-formed.

[*Example:*

```
auto x = 5, *y = &x;        // OK: auto is int
auto a = 5, b = { 1, 2 };   // error: different types for auto
```

— *end example*]

9 If a function with a declared return type that contains a placeholder type has multiple `return` statements, the return type is deduced for each `return` statement. If the type deduced is not the same in each deduction, the program is ill-formed.

10 If a function with a declared return type that uses a placeholder type has no `return` statements, the return type is deduced as though from a `return` statement with no operand at the closing brace of the function body. [*Example:*

```
auto  f() { } // OK, return type is void
auto* g() { } // error, cannot deduce auto* from void()
```

— *end example*]

11 If the type of an entity with an undeduced placeholder type is needed to determine the type of an expression, the program is ill-formed. Once a `return` statement has been seen in a function, however, the return type deduced from that statement can be used in the rest of the function, including in other `return` statements. [*Example:*

```
auto n = n;             // error, n's type is unknown
auto f();
void g() { &f; }        // error, f's return type is unknown
auto sum(int i) {
  if (i == 1)
    return i;           // sum's return type is int
  else
    return sum(i-1)+i; // OK, sum's return type has been deduced
}
```

— *end example*]

12 Return type deduction for a function template with a placeholder in its declared type occurs when the definition is instantiated even if the function body contains a `return` statement with a non-type-dependent operand. [*Note:* Therefore, any use of a specialization of the function template will cause an implicit instantiation. Any errors that arise from this instantiation are not in the immediate context of the function type and can result in the program being ill-formed. — *end note*] [*Example:*

```
template <class T> auto f(T t) { return t; }  // return type deduced at instantiation time
typedef decltype(f(1)) fint_t;                // instantiates f<int> to deduce return type
template<class T> auto f(T* t) { return *t; }
void g() { int (*p)(int*) = &f; }             // instantiates both fs to determine return types,
                                              // chooses second
```

— *end example*]

13 Redeclarations or specializations of a function or function template with a declared return type that uses a placeholder type shall also use that placeholder, not a deduced type. [*Example:*

```
auto f();
auto f() { return 42; } // return type is int
auto f();               // OK
int f();                // error, cannot be overloaded with auto f()
decltype(auto) f();     // error, auto and decltype(auto) don't match

template <typename T> auto g(T t) { return t; } // #1
template auto g(int);                           // OK, return type is int
template char g(char);                          // error, no matching template
template<> auto g(double);                      // OK, forward declaration with unknown return type

template <class T> T g(T t) { return t; } // OK, not functionally equivalent to #1
template char g(char);                    // OK, now there is a matching template
template auto g(float);                   // still matches #1

void h() { return g(42); } // error, ambiguous

template <typename T> struct A {
  friend T frf(T);
};
auto frf(int i) { return i; } // not a friend of A<int>
```

— *end example*]

14 A function declared with a return type that uses a placeholder type shall not be `virtual` (10.3).

15 An explicit instantiation declaration (14.7.2) does not cause the instantiation of an entity declared using a placeholder type, but it also does not prevent that entity from being instantiated as needed to determine its type. [*Example:*

```
template <typename T> auto f(T t) { return t; }
extern template auto f(int); // does not instantiate f<int>
int (*p)(int) = f;           // instantiates f<int> to determine its return type, but an explicit
                             // instantiation definition is still required somewhere in the program
```

— *end example*]

7.2 Enumeration declarations [dcl.enum]

1 An enumeration is a distinct type (3.9.2) with named constants. Its name becomes an *enum-name*, within its scope.

enum-name:
identifier

enum-specifier:
enum-head `{` *enumerator-list*$_{opt}$ `}`
enum-head `{` *enumerator-list* `,` `}`

enum-head:
enum-key attribute-specifier-seq$_{opt}$ *identifier*$_{opt}$ *enum-base*$_{opt}$
enum-key attribute-specifier-seq$_{opt}$ *nested-name-specifier identifier*
enum-base$_{opt}$

opaque-enum-declaration:
enum-key attribute-specifier-seq$_{opt}$ *identifier enum-base*$_{opt}$ `;`

enum-key:
`enum`
`enum class`
`enum struct`

enum-base:
`:` *type-specifier-seq*

enumerator-list:
enumerator-definition
enumerator-list `,` *enumerator-definition*

enumerator-definition:
enumerator
enumerator `=` *constant-expression*

enumerator:
identifier attribute-specifier-seq$_{opt}$

The optional *attribute-specifier-seq* in the *enum-head* and the *opaque-enum-declaration* appertains to the enumeration; the attributes in that *attribute-specifier-seq* are thereafter considered attributes of the enumeration whenever it is named. A `:` following "`enum` *identifier*" within the *decl-specifier-seq* of a *member-declaration* is parsed as part of an *enum-base*. [*Note:* This resolves a potential ambiguity between the declaration of an enumeration with an *enum-base* and the declaration of an unnamed bit-field of enumeration type. [*Example:*

```
struct S {
  enum E : int {};
  enum E : int {};  // error: redeclaration of enumeration
};
```

— *end example*] — *end note*]

2 The enumeration type declared with an *enum-key* of only `enum` is an unscoped enumeration, and its *enumerator*s are *unscoped enumerators*. The *enum-key*s `enum class` and `enum struct` are semantically equivalent; an enumeration type declared with one of these is a *scoped enumeration*, and its *enumerator*s are *scoped enumerators*. The optional *identifier* shall not be omitted in the declaration of a scoped enumeration. The *type-specifier-seq* of an *enum-base* shall name an integral type; any cv-qualification is ignored. An *opaque-enum-declaration* declaring an unscoped enumeration shall not omit the *enum-base*. The identifiers in an *enumerator-list* are declared as constants, and can appear wherever constants are required. An *enumerator-definition* with `=` gives the associated *enumerator* the value indicated by the *constant-expression*. If the first *enumerator* has no *initializer*, the value of the corresponding constant is zero. An *enumerator-definition* without an *initializer* gives the *enumerator* the value obtained by increasing the value of the previous *enumerator* by one. [*Example:*

```
enum { a, b, c=0 };
enum { d, e, f=e+2 };
```

defines `a`, `c`, and `d` to be zero, `b` and `e` to be `1`, and `f` to be `3`. — *end example*] The optional *attribute-specifier-seq* in an *enumerator* appertains to that enumerator.

3 An *opaque-enum-declaration* is either a redeclaration of an enumeration in the current scope or a declaration of a new enumeration. [*Note:* An enumeration declared by an *opaque-enum-declaration* has fixed underlying type and is a complete type. The list of enumerators can be provided in a later redeclaration with an *enum-specifier*. — *end note*] A scoped enumeration shall not be later redeclared as unscoped or with a different underlying type. An unscoped enumeration shall not be later redeclared as scoped and each redeclaration shall include an *enum-base* specifying the same underlying type as in the original declaration.

4 If the *enum-key* is followed by a *nested-name-specifier*, the *enum-specifier* shall refer to an enumeration that was previously declared directly in the class or namespace to which the *nested-name-specifier* refers (i.e., neither inherited nor introduced by a *using-declaration*), and the *enum-specifier* shall appear in a namespace enclosing the previous declaration.

5 Each enumeration defines a type that is different from all other types. Each enumeration also has an underlying type. The underlying type can be explicitly specified using an *enum-base*. For a scoped enumeration type, the underlying type is `int` if it is not explicitly specified. In both of these cases, the underlying type is said to be *fixed*. Following the closing brace of an *enum-specifier*, each enumerator has the type of its enumeration. If the underlying type is fixed, the type of each enumerator prior to the closing brace is the underlying type and the *constant-expression* in the *enumerator-definition* shall be a converted constant expression of the underlying type (5.20). If the underlying type is not fixed, the type of each enumerator prior to the closing brace is determined as follows:

(5.1) — If an initializer is specified for an enumerator, the *constant-expression* shall be an integral constant expression (5.20). If the expression has unscoped enumeration type, the enumerator has the underlying type of that enumeration type, otherwise it has the same type as the expression.

(5.2) — If no initializer is specified for the first enumerator, its type is an unspecified signed integral type.

(5.3) — Otherwise the type of the enumerator is the same as that of the preceding enumerator unless the incremented value is not representable in that type, in which case the type is an unspecified integral type sufficient to contain the incremented value. If no such type exists, the program is ill-formed.

6 An enumeration whose underlying type is fixed is an incomplete type from its point of declaration (3.3.2) to immediately after its *enum-base* (if any), at which point it becomes a complete type. An enumeration whose underlying type is not fixed is an incomplete type from its point of declaration to immediately after the closing `}` of its *enum-specifier*, at which point it becomes a complete type.

7 For an enumeration whose underlying type is not fixed, the underlying type is an integral type that can represent all the enumerator values defined in the enumeration. If no integral type can represent all the

enumerator values, the enumeration is ill-formed. It is implementation-defined which integral type is used as the underlying type except that the underlying type shall not be larger than `int` unless the value of an enumerator cannot fit in an `int` or `unsigned int`. If the *enumerator-list* is empty, the underlying type is as if the enumeration had a single enumerator with value 0.

8 For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, for an enumeration where e_{min} is the smallest enumerator and e_{max} is the largest, the values of the enumeration are the values in the range b_{min} to b_{max}, defined as follows: Let K be 1 for a two's complement representation and 0 for a one's complement or sign-magnitude representation. b_{max} is the smallest value greater than or equal to $max(|e_{min}| - K, |e_{max}|)$ and equal to $2^M - 1$, where M is a non-negative integer. b_{min} is zero if e_{min} is non-negative and $-(b_{max} + K)$ otherwise. The size of the smallest bit-field large enough to hold all the values of the enumeration type is $max(M, 1)$ if b_{min} is zero and $M + 1$ otherwise. It is possible to define an enumeration that has values not defined by any of its enumerators. If the *enumerator-list* is empty, the values of the enumeration are as if the enumeration had a single enumerator with value 0.[97]

9 Two enumeration types are *layout-compatible* if they have the same underlying type.

10 The value of an enumerator or an object of an unscoped enumeration type is converted to an integer by integral promotion (4.5). [*Example:*

```
enum color { red, yellow, green=20, blue };
color col = red;
color* cp = &col;
if (*cp == blue)              // ...
```

makes `color` a type describing various colors, and then declares `col` as an object of that type, and `cp` as a pointer to an object of that type. The possible values of an object of type `color` are `red`, `yellow`, `green`, `blue`; these values can be converted to the integral values `0`, `1`, `20`, and `21`. Since enumerations are distinct types, objects of type `color` can be assigned only values of type `color`.

```
color c = 1;                  // error: type mismatch,
                              // no conversion from int to color

int i = yellow;               // OK: yellow converted to integral value 1
                              // integral promotion
```

Note that this implicit `enum` to `int` conversion is not provided for a scoped enumeration:

```
enum class Col { red, yellow, green };
int x = Col::red;             // error: no Col to int conversion
Col y = Col::red;
if (y) { }                    // error: no Col to bool conversion
```

— *end example*]

11 Each *enum-name* and each unscoped *enumerator* is declared in the scope that immediately contains the *enum-specifier*. Each scoped *enumerator* is declared in the scope of the enumeration. These names obey the scope rules defined for all names in (3.3) and (3.4).[*Example:*

```
enum direction { left='l', right='r' };

void g()  {
  direction d;                // OK
  d = left;                   // OK
```

97) This set of values is used to define promotion and conversion semantics for the enumeration type. It does not preclude an expression of enumeration type from having a value that falls outside this range.

```
  d = direction::right;          // OK
}

enum class altitude { high='h', low='l' };

void h()  {
  altitude a;                    // OK
  a = high;                      // error: high not in scope
  a = altitude::low;             // OK
}
```

— *end example*] An enumerator declared in class scope can be referred to using the class member access operators (`::`, `.` (dot) and `->` (arrow)), see 5.2.5. [*Example:*

```
struct X {
  enum direction { left='l', right='r' };
  int f(int i) { return i==left ? 0 : i==right ? 1 : 2; }
};

void g(X* p) {
  direction d;                   // error: direction not in scope
  int i;
  i = p->f(left);                // error: left not in scope
  i = p->f(X::right);            // OK
  i = p->f(p->left);             // OK
  // ...
}
```

— *end example*]

7.3 Namespaces [basic.namespace]

1 A namespace is an optionally-named declarative region. The name of a namespace can be used to access entities declared in that namespace; that is, the members of the namespace. Unlike other declarative regions, the definition of a namespace can be split over several parts of one or more translation units.

2 The outermost declarative region of a translation unit is a namespace; see 3.3.6.

7.3.1 Namespace definition [namespace.def]

1 The grammar for a *namespace-definition* is

namespace-name:
identifier
namespace-alias

namespace-definition:
named-namespace-definition
unnamed-namespace-definition
nested-namespace-definition

named-namespace-definition:
`inline`$_{opt}$ `namespace` *attribute-specifier-seq*$_{opt}$ *identifier* `{` *namespace-body* `}`

unnamed-namespace-definition:
`inline`$_{opt}$ `namespace` *attribute-specifier-seq*$_{opt}$ `{` *namespace-body* `}`

nested-namespace-definition:
`namespace` *enclosing-namespace-specifier* `::` *identifier* `{` *namespace-body* `}`

enclosing-namespace-specifier:
identifier
enclosing-namespace-specifier `::` *identifier*

namespace-body:
declaration-seq$_{opt}$

2 Every *namespace-definition* shall appear in the global scope or in a namespace scope (3.3.6).

3 In a *named-namespace-definition*, the *identifier* is the name of the namespace. If the *identifier*, when looked up (3.4.1), refers to a *namespace-name* (but not a *namespace-alias*) introduced in the declarative region in which the *named-namespace-definition* appears, the *namespace-definition extends* the previously-declared namespace. Otherwise, the *identifier* is introduced as a *namespace-name* into the declarative region in which the *named-namespace-definition* appears.

4 Because a *namespace-definition* contains *declaration*s in its *namespace-body* and a *namespace-definition* is itself a *declaration*, it follows that *namespace-definitions* can be nested. [*Example:*

```
namespace Outer {
  int i;
  namespace Inner {
    void f() { i++; }           // Outer::i
    int i;
    void g() { i++; }           // Inner::i
  }
}
```

— *end example*]

5 The *enclosing namespaces* of a declaration are those namespaces in which the declaration lexically appears, except for a redeclaration of a namespace member outside its original namespace (e.g., a definition as specified in 7.3.1.2). Such a redeclaration has the same enclosing namespaces as the original declaration. [*Example:*

```
namespace Q {
  namespace V {
    void f();   // enclosing namespaces are the global namespace, Q, and Q::V
    class C { void m(); };
  }
  void V::f() { // enclosing namespaces are the global namespace, Q, and Q::V
    extern void h();  // ... so this declares Q::V::h
  }
  void V::C::m() { // enclosing namespaces are the global namespace, Q, and Q::V
  }
}
```

— *end example*]

6 If the optional initial `inline` keyword appears in a *namespace-definition* for a particular namespace, that namespace is declared to be an *inline namespace*. The `inline` keyword may be used on a *namespace-definition* that extends a namespace only if it was previously used on the *namespace-definition* that initially declared the *namespace-name* for that namespace.

7 The optional *attribute-specifier-seq* in a *named-namespace-definition* appertains to the namespace being defined or extended.

8 Members of an inline namespace can be used in most respects as though they were members of the enclosing namespace. Specifically, the inline namespace and its enclosing namespace are both added to the set of associated namespaces used in argument-dependent lookup (3.4.2) whenever one of them is, and a *using-directive* (7.3.4) that names the inline namespace is implicitly inserted into the enclosing namespace as for an unnamed namespace (7.3.1.1). Furthermore, each member of the inline namespace can subsequently be partially specialized (14.5.5), explicitly instantiated (14.7.2), or explicitly specialized (14.7.3) as though it were a member of the enclosing namespace. Finally, looking up a name in the enclosing namespace via

explicit qualification (3.4.3.2) will include members of the inline namespace brought in by the *using-directive* even if there are declarations of that name in the enclosing namespace.

9 These properties are transitive: if a namespace `N` contains an inline namespace `M`, which in turn contains an inline namespace `O`, then the members of `O` can be used as though they were members of `M` or `N`. The *inline namespace set* of `N` is the transitive closure of all inline namespaces in `N`. The *enclosing namespace set* of `O` is the set of namespaces consisting of the innermost non-inline namespace enclosing an inline namespace `O`, together with any intervening inline namespaces.

10 A *nested-namespace-definition* with an *enclosing-namespace-specifier* `E`, *identifier* `I` and *namespace-body* `B` is equivalent to

```
namespace E { namespace I { B } }
```

[*Example:*

```
namespace A::B::C {
  int i;
}
```

The above has the same effect as:

```
namespace A {
  namespace B {
    namespace C {
      int i;
    }
  }
}
```

— *end example*]

7.3.1.1 Unnamed namespaces [namespace.unnamed]

1 An *unnamed-namespace-definition* behaves as if it were replaced by

```
inline_opt namespace unique { /* empty body */ }
using namespace unique ;
namespace unique { namespace-body }
```

where `inline` appears if and only if it appears in the *unnamed-namespace-definition* and all occurrences of ***unique*** in a translation unit are replaced by the same identifier, and this identifier differs from all other identifiers in the translation unit. The optional *attribute-specifier-seq* in the *unnamed-namespace-definition* appertains to ***unique***. [*Example:*

```
namespace { int i; }            // unique::i
void f() { i++; }               // unique::i++

namespace A {
  namespace {
    int i;                      // A::unique::i
    int j;                      // A::unique::j
  }
  void g() { i++; }             // A::unique::i++
}

using namespace A;
void h() {
  i++;                          // error: unique::i or A::unique::i
  A::i++;                       // A::unique::i
```

```
  j++;                          // A::unique::j
}
```

— *end example*]

7.3.1.2 Namespace member definitions [namespace.memdef]

1 A declaration in a namespace `N` (excluding declarations in nested scopes) whose *declarator-id* is an *unqualified-id* declares (or redeclares) a member of `N`, and may be a definition. [*Note:* An explicit instantiation (14.7.2) or explicit specialization (14.7.3) of a template does not introduce a name and thus may be declared using an *unqualified-id* in a member of the enclosing namespace set, if the primary template is declared in an inline namespace. — *end note*] [*Example:*

```
namespace X {
  void f() { /* ... */ }  // OK: introduces X::f()

  namespace M {
    void g();             // OK: introduces X::M::g()
  }
  using M::g;
  void g();               // error: conflicts with X::M::g()
}
```

— *end example*]

2 Members of a named namespace can also be defined outside that namespace by explicit qualification (3.4.3.2) of the name being defined, provided that the entity being defined was already declared in the namespace and the definition appears after the point of declaration in a namespace that encloses the declaration's namespace. [*Example:*

```
namespace Q {
  namespace V {
    void f();
  }
  void V::f() { /* ... */ }     // OK
  void V::g() { /* ... */ }     // error: g() is not yet a member of V
  namespace V {
    void g();
  }
}

namespace R {
  void Q::V::g() { /* ... */ }  // error: R doesn't enclose Q
}
```

— *end example*]

3 If a `friend` declaration in a non-local class first declares a class, function, class template or function template[98] the friend is a member of the innermost enclosing namespace. The `friend` declaration does not by itself make the name visible to unqualified lookup (3.4.1) or qualified lookup (3.4.3). [*Note:* The name of the friend will be visible in its namespace if a matching declaration is provided at namespace scope (either before or after the class definition granting friendship). — *end note*] If a friend function or function template is called, its name may be found by the name lookup that considers functions from namespaces and classes associated with the types of the function arguments (3.4.2). If the name in a `friend` declaration is neither qualified nor a *template-id* and the declaration is a function or an *elaborated-type-specifier*, the lookup to

98) this implies that the name of the class or function is unqualified.

determine whether the entity has been previously declared shall not consider any scopes outside the innermost enclosing namespace. [*Note:* The other forms of `friend` declarations cannot declare a new member of the innermost enclosing namespace and thus follow the usual lookup rules. — *end note*] [*Example:*

```
// Assume f and g have not yet been declared.
void h(int);
template <class T> void f2(T);
namespace A {
  class X {
    friend void f(X);           // A::f(X) is a friend
    class Y {
      friend void g();          // A::g is a friend
      friend void h(int);       // A::h is a friend
                                // ::h not considered
      friend void f2<>(int);    // ::f2<>(int) is a friend
    };
  };

  // A::f, A::g and A::h are not visible here
  X x;
  void g() { f(x); }            // definition of A::g
  void f(X) { /* ...  */}       // definition of A::f
  void h(int) { /* ...  */ }    // definition of A::h
  // A::f, A::g and A::h are visible here and known to be friends
}

using A::x;

void h() {
  A::f(x);
  A::X::f(x);                   // error: f is not a member of A::X
  A::X::Y::g();                 // error: g is not a member of A::X::Y
}
```

— *end example*]

7.3.2 Namespace alias [namespace.alias]

1 A *namespace-alias-definition* declares an alternate name for a namespace according to the following grammar:

namespace-alias:
identifier

namespace-alias-definition:
`namespace` *identifier* `=` *qualified-namespace-specifier* `;`

qualified-namespace-specifier:
nested-name-specifier$_{opt}$ *namespace-name*

2 The *identifier* in a *namespace-alias-definition* is a synonym for the name of the namespace denoted by the *qualified-namespace-specifier* and becomes a *namespace-alias*. [*Note:* When looking up a *namespace-name* in a *namespace-alias-definition*, only namespace names are considered, see 3.4.6. — *end note*]

3 In a declarative region, a *namespace-alias-definition* can be used to redefine a *namespace-alias* declared in that declarative region to refer only to the namespace to which it already refers. [*Example:* the following declarations are well-formed:

```
namespace Company_with_very_long_name { /* ... */ }
namespace CWVLN = Company_with_very_long_name;
namespace CWVLN = Company_with_very_long_name;          // OK: duplicate
```

```
namespace CWVLN = CWVLN;
```

— *end example*]

7.3.3 The `using` declaration [namespace.udecl]

1 A *using-declaration* introduces a name into the declarative region in which the *using-declaration* appears.

using-declaration:
`using typename`$_{opt}$ *nested-name-specifier unqualified-id* `;`

The member name specified in a *using-declaration* is declared in the declarative region in which the *using-declaration* appears. [*Note:* Only the specified name is so declared; specifying an enumeration name in a *using-declaration* does not declare its enumerators in the *using-declaration*'s declarative region. — *end note*] If a *using-declaration* names a constructor (3.4.3.1), it implicitly declares a set of constructors in the class in which the *using-declaration* appears (12.9); otherwise the name specified in a *using-declaration* is a synonym for a set of declarations in another namespace or class.

2 Every *using-declaration* is a *declaration* and a *member-declaration* and so can be used in a class definition. [*Example:*

```
struct B {
  void f(char);
  void g(char);
  enum E { e };
  union { int x; };
};

struct D : B {
  using B::f;
  void f(int) { f('c'); }       // calls B::f(char)
  void g(int) { g('c'); }       // recursively calls D::g(int)
};
```

— *end example*]

3 In a *using-declaration* used as a *member-declaration*, the *nested-name-specifier* shall name a base class of the class being defined. If such a *using-declaration* names a constructor, the *nested-name-specifier* shall name a direct base class of the class being defined; otherwise it introduces the set of declarations found by member name lookup (10.2, 3.4.3.1). [*Example:*

```
class C {
  int g();
};

class D2 : public B {
  using B::f;                   // OK: B is a base of D2
  using B::e;                   // OK: e is an enumerator of base B
  using B::x;                   // OK: x is a union member of base B
  using C::g;                   // error: C isn't a base of D2
};
```

— *end example*]

4 [*Note:* Since destructors do not have names, a *using-declaration* cannot refer to a destructor for a base class. Since specializations of member templates for conversion functions are not found by name lookup, they are not considered when a *using-declaration* specifies a conversion function (14.5.2). — *end note*] If an assignment operator brought from a base class into a derived class scope has the signature of a copy/move

assignment operator for the derived class (12.8), the *using-declaration* does not by itself suppress the implicit declaration of the derived class assignment operator; the copy/move assignment operator from the base class is hidden or overridden by the implicitly-declared copy/move assignment operator of the derived class, as described below.

5 A *using-declaration* shall not name a *template-id*. [*Example:*

```
struct A {
  template <class T> void f(T);
  template <class T> struct X { };
};
struct B : A {
  using A::f<double>;           // ill-formed
  using A::X<int>;              // ill-formed
};
```

— *end example*]

6 A *using-declaration* shall not name a namespace.

7 A *using-declaration* shall not name a scoped enumerator.

8 A *using-declaration* for a class member shall be a *member-declaration*. [*Example:*

```
struct X {
  int i;
  static int s;
};

void f() {
  using X::i;       // error: X::i is a class member
                    // and this is not a member declaration.
  using X::s;       // error: X::s is a class member
                    // and this is not a member declaration.
}
```

— *end example*]

9 Members declared by a *using-declaration* can be referred to by explicit qualification just like other member names (3.4.3.2). [*Example:*

```
void f();

namespace A {
  void g();
}

namespace X {
  using ::f;        // global f
  using A::g;       // A's g
}

void h()
{
  X::f();           // calls ::f
  X::g();           // calls A::g
}
```

— *end example*]

10 A *using-declaration* is a *declaration* and can therefore be used repeatedly where (and only where) multiple declarations are allowed. [*Example:*

```
namespace A {
  int i;
}

namespace A1 {
  using A::i;
  using A::i;       // OK: double declaration
}

struct B {
  int i;
};

struct X : B {
  using B::i;
  using B::i;       // error: double member declaration
};
```

— *end example*]

11 Members added to the namespace after the *using-declaration* are not considered when a use of the name is made. [*Note:* Thus, additional overloads added after the *using-declaration* are ignored, but default function arguments (8.3.6), default template arguments (14.1), and template specializations (14.5.5, 14.7.3) are considered. — *end note*] [*Example:*

```
namespace A {
  void f(int);
}

using A::f;         // f is a synonym for A::f;
                    // that is, for A::f(int).
namespace A {
  void f(char);
}

void foo() {
  f('a');           // calls f(int),
}                   // even though f(char) exists.

void bar() {
  using A::f;       // f is a synonym for A::f;
                    // that is, for A::f(int) and A::f(char).
  f('a');           // calls f(char)
}
```

— *end example*]

12 [*Note:* Partial specializations of class templates are found by looking up the primary class template and then considering all partial specializations of that template. If a *using-declaration* names a class template, partial specializations introduced after the *using-declaration* are effectively visible because the primary template is visible (14.5.5). — *end note*]

13 Since a *using-declaration* is a declaration, the restrictions on declarations of the same name in the same declarative region (3.3) also apply to *using-declarations*. [*Example:*

```
namespace A {
  int x;
}

namespace B {
  int i;
  struct g { };
  struct x { };
  void f(int);
  void f(double);
  void g(char);      // OK: hides struct g
}

void func() {
  int i;
  using B::i;        // error: i declared twice
  void f(char);
  using B::f;        // OK: each f is a function
  f(3.5);            // calls B::f(double)
  using B::g;
  g('a');            // calls B::g(char)
  struct g g1;       // g1 has class type B::g
  using B::x;
  using A::x;        // OK: hides struct B::x
  x = 99;            // assigns to A::x
  struct x x1;       // x1 has class type B::x
}
```

— *end example*]

14 If a function declaration in namespace scope or block scope has the same name and the same parameter-type-list (8.3.5) as a function introduced by a *using-declaration*, and the declarations do not declare the same function, the program is ill-formed. If a function template declaration in namespace scope has the same name, parameter-type-list, return type, and template parameter list as a function template introduced by a *using-declaration*, the program is ill-formed. [*Note:* Two *using-declaration*s may introduce functions with the same name and the same parameter-type-list. If, for a call to an unqualified function name, function overload resolution selects the functions introduced by such *using-declaration*s, the function call is ill-formed. [*Example:*

```
namespace B {
  void f(int);
  void f(double);
}
namespace C {
  void f(int);
  void f(double);
  void f(char);
}

void h() {
  using B::f;        // B::f(int) and B::f(double)
  using C::f;        // C::f(int), C::f(double), and C::f(char)
  f('h');            // calls C::f(char)
  f(1);              // error: ambiguous: B::f(int) or C::f(int)?
  void f(int);       // error: f(int) conflicts with C::f(int) and B::f(int)
}
```

— *end example*] — *end note*]

15 When a *using-declaration* brings names from a base class into a derived class scope, member functions and member function templates in the derived class override and/or hide member functions and member function templates with the same name, parameter-type-list (8.3.5), cv-qualification, and *ref-qualifier* (if any) in a base class (rather than conflicting). [*Note:* For *using-declaration*s that name a constructor, see 12.9. — *end note*] [*Example:*

```
struct B {
  virtual void f(int);
  virtual void f(char);
  void g(int);
  void h(int);
};

struct D : B {
  using B::f;
  void f(int);       // OK: D::f(int) overrides B::f(int);

  using B::g;
  void g(char);      // OK

  using B::h;
  void h(int);       // OK: D::h(int) hides B::h(int)
};

void k(D* p)
{
  p->f(1);           // calls D::f(int)
  p->f('a');         // calls B::f(char)
  p->g(1);           // calls B::g(int)
  p->g('a');         // calls D::g(char)
}
```

— *end example*]

16 For the purpose of overload resolution, the functions which are introduced by a *using-declaration* into a derived class will be treated as though they were members of the derived class. In particular, the implicit `this` parameter shall be treated as if it were a pointer to the derived class rather than to the base class. This has no effect on the type of the function, and in all other respects the function remains a member of the base class.

17 The access rules for inheriting constructors are specified in 12.9; otherwise all instances of the name mentioned in a *using-declaration* shall be accessible. In particular, if a derived class uses a *using-declaration* to access a member of a base class, the member name shall be accessible. If the name is that of an overloaded member function, then all functions named shall be accessible. The base class members mentioned by a *using-declaration* shall be visible in the scope of at least one of the direct base classes of the class where the *using-declaration* is specified. [*Note:* Because a *using-declaration* designates a base class member (and not a member subobject or a member function of a base class subobject), a *using-declaration* cannot be used to resolve inherited member ambiguities. For example,

```
struct A { int x(); };
struct B : A { };
struct C : A {
  using A::x;
  int x(int);
};
```

```
struct D : B, C {
  using C::x;
  int x(double);
};
int f(D* d) {
  return d->x();     // ambiguous: B::x or C::x
}
```

— *end note*]

18 The alias created by the *using-declaration* has the usual accessibility for a *member-declaration*. [*Note:* A *using-declaration* that names a constructor does not create aliases; see 12.9 for the pertinent accessibility rules. — *end note*] [*Example:*

```
class A {
private:
    void f(char);
public:
    void f(int);
protected:
    void g();
};

class B : public A {
  using A::f;        // error: A::f(char) is inaccessible
public:
  using A::g;        // B::g is a public synonym for A::g
};
```

— *end example*]

19 If a *using-declaration* uses the keyword **typename** and specifies a dependent name (14.6.2), the name introduced by the *using-declaration* is treated as a *typedef-name* (7.1.3).

7.3.4 Using directive [namespace.udir]

using-directive:
attribute-specifier-seq$_{opt}$ **using namespace** *nested-name-specifier*$_{opt}$ *namespace-name* **;**

1 A *using-directive* shall not appear in class scope, but may appear in namespace scope or in block scope. [*Note:* When looking up a *namespace-name* in a *using-directive*, only namespace names are considered, see 3.4.6. — *end note*] The optional *attribute-specifier-seq* appertains to the *using-directive*.

2 A *using-directive* specifies that the names in the nominated namespace can be used in the scope in which the *using-directive* appears after the *using-directive*. During unqualified name lookup (3.4.1), the names appear as if they were declared in the nearest enclosing namespace which contains both the *using-directive* and the nominated namespace. [*Note:* In this context, "contains" means "contains directly or indirectly". — *end note*]

3 A *using-directive* does not add any members to the declarative region in which it appears. [*Example:*

```
namespace A {
  int i;
  namespace B {
    namespace C {
      int i;
    }
    using namespace A::B::C;
```

```
    void f1() {
      i = 5;          // OK, C::i visible in B and hides A::i
    }
  }
  namespace D {
    using namespace B;
    using namespace C;
    void f2() {
      i = 5;          // ambiguous, B::C::i or A::i?
    }
  }
  void f3() {
    i = 5;            // uses A::i
  }
}
void f4() {
  i = 5;              // ill-formed; neither i is visible
}
```

— *end example*]

4 For unqualified lookup (3.4.1), the *using-directive* is transitive: if a scope contains a *using-directive* that nominates a second namespace that itself contains *using-directive*s, the effect is as if the *using-directive*s from the second namespace also appeared in the first. [*Note:* For qualified lookup, see 3.4.3.2. — *end note*] [*Example:*

```
namespace M {
  int i;
}

namespace N {
  int i;
  using namespace M;
}

void f() {
  using namespace N;
  i = 7;              // error: both M::i and N::i are visible
}
```

For another example,

```
namespace A {
  int i;
}
namespace B {
  int i;
  int j;
  namespace C {
    namespace D {
      using namespace A;
      int j;
      int k;
      int a = i;      // B::i hides A::i
    }
    using namespace D;
    int k = 89;       // no problem yet
```

```
    int l = k;      // ambiguous: C::k or D::k
    int m = i;      // B::i hides A::i
    int n = j;      // D::j hides B::j
  }
}
```

— *end example*]

5 If a namespace is extended (7.3.1) after a *using-directive* for that namespace is given, the additional members of the extended namespace and the members of namespaces nominated by *using-directive*s in the extending *namespace-definition* can be used after the extending *namespace-definition*.

6 If name lookup finds a declaration for a name in two different namespaces, and the declarations do not declare the same entity and do not declare functions, the use of the name is ill-formed. [*Note:* In particular, the name of a variable, function or enumerator does not hide the name of a class or enumeration declared in a different namespace. For example,

```
namespace A {
  class X { };
  extern "C"   int g();
  extern "C++" int h();
}
namespace B {
  void X(int);
  extern "C"   int g();
  extern "C++" int h(int);
}
using namespace A;
using namespace B;

void f() {
  X(1);             // error: name X found in two namespaces
  g();              // OK: name g refers to the same entity
  h();              // OK: overload resolution selects A::h
}
```

— *end note*]

7 During overload resolution, all functions from the transitive search are considered for argument matching. The set of declarations found by the transitive search is unordered. [*Note:* In particular, the order in which namespaces were considered and the relationships among the namespaces implied by the *using-directive*s do not cause preference to be given to any of the declarations found by the search. — *end note*] An ambiguity exists if the best match finds two functions with the same signature, even if one is in a namespace reachable through *using-directive*s in the namespace of the other.[99] [*Example:*

```
namespace D {
  int d1;
  void f(char);
}
using namespace D;

int d1;             // OK: no conflict with D::d1
```

99) During name lookup in a class hierarchy, some ambiguities may be resolved by considering whether one member hides the other along some paths (10.2). There is no such disambiguation when considering the set of names found as a result of following *using-directive*s.

```
namespace E {
  int e;
  void f(int);
}

namespace D {       // namespace extension
  int d2;
  using namespace E;
  void f(int);
}

void f() {
  d1++;             // error: ambiguous ::d1 or D::d1?
  ::d1++;           // OK
  D::d1++;          // OK
  d2++;             // OK: D::d2
  e++;              // OK: E::e
  f(1);             // error: ambiguous: D::f(int) or E::f(int)?
  f('a');           // OK: D::f(char)
}
```

— *end example*]

7.4 The `asm` declaration [dcl.asm]

1 An `asm` declaration has the form

asm-definition:
`asm` `(` *string-literal* `)` `;`

The `asm` declaration is conditionally-supported; its meaning is implementation-defined. [*Note:* Typically it is used to pass information through the implementation to an assembler. — *end note*]

7.5 Linkage specifications [dcl.link]

1 All function types, function names with external linkage, and variable names with external linkage have a *language linkage*. [*Note:* Some of the properties associated with an entity with language linkage are specific to each implementation and are not described here. For example, a particular language linkage may be associated with a particular form of representing names of objects and functions with external linkage, or with a particular calling convention, etc. — *end note*] The default language linkage of all function types, function names, and variable names is C++ language linkage. Two function types with different language linkages are distinct types even if they are otherwise identical.

2 Linkage (3.5) between C++ and non-C++ code fragments can be achieved using a *linkage-specification*:

linkage-specification:
`extern` *string-literal* `{` *declaration-seq*$_{opt}$`}`
`extern` *string-literal declaration*

The *string-literal* indicates the required language linkage. This International Standard specifies the semantics for the *string-literal*s `"C"` and `"C++"`. Use of a *string-literal* other than `"C"` or `"C++"` is conditionally-supported, with implementation-defined semantics. [*Note:* Therefore, a linkage-specification with a *string-literal* that is unknown to the implementation requires a diagnostic. — *end note*] [*Note:* It is recommended that the spelling of the *string-literal* be taken from the document defining that language. For example, `Ada` (not `ADA`) and `Fortran` or `FORTRAN`, depending on the vintage. — *end note*]

3 Every implementation shall provide for linkage to functions written in the C programming language, `"C"`, and linkage to C++ functions, `"C++"`. [*Example:*

```
complex sqrt(complex);          // C++ linkage by default
```

```
extern "C" {
  double sqrt(double);          // C linkage
}
```

— *end example*]

4 Linkage specifications nest. When linkage specifications nest, the innermost one determines the language linkage. A linkage specification does not establish a scope. A *linkage-specification* shall occur only in namespace scope (3.3). In a *linkage-specification*, the specified language linkage applies to the function types of all function declarators, function names with external linkage, and variable names with external linkage declared within the *linkage-specification*. [*Example:*

```
extern "C" void f1(void(*pf)(int));
                                // the name f1 and its function type have C language
                                // linkage; pf is a pointer to a C function
extern "C" typedef void FUNC();
FUNC f2;                        // the name f2 has C++ language linkage and the
                                // function's type has C language linkage
extern "C" FUNC f3;             // the name of function f3 and the function's type
                                // have C language linkage
void (*pf2)(FUNC*);             // the name of the variable pf2 has C++ linkage and
                                // the type of pf2 is pointer to C++ function that
                                // takes one parameter of type pointer to C function

extern "C" {
  static void f4();             // the name of the function f4 has
                                // internal linkage (not C language
                                // linkage) and the function's type
                                // has C language linkage.
}

extern "C" void f5() {
  extern void f4();             // OK: Name linkage (internal)
                                // and function type linkage (C
                                // language linkage) obtained from
                                // previous declaration.
}

extern void f4();               // OK: Name linkage (internal)
                                // and function type linkage (C
                                // language linkage) obtained from
                                // previous declaration.

void f6() {
  extern void f4();             // OK: Name linkage (internal)
                                // and function type linkage (C
                                // language linkage) obtained from
                                // previous declaration.
}
```

— *end example*] A C language linkage is ignored in determining the language linkage of the names of class members and the function type of class member functions. [*Example:*

```
extern "C" typedef void FUNC_c();
class C {
  void mf1(FUNC_c*);            // the name of the function mf1 and the member
                                // function's type have C++ language linkage; the
```

```
                                // parameter has type pointer to C function
  FUNC_c mf2;                   // the name of the function mf2 and the member
                                // function's type have C++ language linkage
  static FUNC_c* q;             // the name of the data member q has C++ language
                                // linkage and the data member's type is pointer to
                                // C function
};

extern "C" {
  class X {
    void mf();                  // the name of the function mf and the member
                                // function's type have C++ language linkage
    void mf2(void(*)());        // the name of the function mf2 has C++ language
                                // linkage; the parameter has type pointer to
                                // C function
  };
}
```

— *end example*]

5 If two declarations declare functions with the same name and *parameter-type-list* (8.3.5) to be members of the same namespace or declare objects with the same name to be members of the same namespace and the declarations give the names different language linkages, the program is ill-formed; no diagnostic is required if the declarations appear in different translation units. Except for functions with C++ linkage, a function declaration without a linkage specification shall not precede the first linkage specification for that function. A function can be declared without a linkage specification after an explicit linkage specification has been seen; the linkage explicitly specified in the earlier declaration is not affected by such a function declaration.

6 At most one function with a particular name can have C language linkage. Two declarations for a function with C language linkage with the same function name (ignoring the namespace names that qualify it) that appear in different namespace scopes refer to the same function. Two declarations for a variable with C language linkage with the same name (ignoring the namespace names that qualify it) that appear in different namespace scopes refer to the same variable. An entity with C language linkage shall not be declared with the same name as a variable in global scope, unless both declarations denote the same entity; no diagnostic is required if the declarations appear in different translation units. A variable with C language linkage shall not be declared with the same name as a function with C language linkage (ignoring the namespace names that qualify the respective names); no diagnostic is required if the declarations appear in different translation units. [*Note:* Only one definition for an entity with a given name with C language linkage may appear in the program (see 3.2); this implies that such an entity must not be defined in more than one namespace scope. — *end note*] [*Example:*

```
int x;
namespace A {
  extern "C" int f();
  extern "C" int g() { return 1; }
  extern "C" int h();
  extern "C" int x();               // ill-formed: same name as global-space object x
}

namespace B {
  extern "C" int f();               // A::f and B::f refer to the same function
  extern "C" int g() { return 1; }  // ill-formed, the function g
                                    // with C language linkage has two definitions
}
```

```
int A::f() { return 98; }           //definition for the function f with C language linkage
extern "C" int h() { return 97; }   // definition for the function h with C language linkage
                                    // A::h and ::h refer to the same function
```

— *end example*]

7 A declaration directly contained in a *linkage-specification* is treated as if it contains the `extern` specifier (7.1.1) for the purpose of determining the linkage of the declared name and whether it is a definition. Such a declaration shall not specify a storage class. [*Example:*

```
extern "C" double f();
static double f();                  // error
extern "C" int i;                   // declaration
extern "C" {
  int i;                            // definition
}
extern "C" static void g();         // error
```

— *end example*]

8 [*Note:* Because the language linkage is part of a function type, when indirecting through a pointer to C function, the function to which the resulting lvalue refers is considered a C function. — *end note*]

9 Linkage from C++ to objects defined in other languages and to objects defined in C++ from other languages is implementation-defined and language-dependent. Only where the object layout strategies of two language implementations are similar enough can such linkage be achieved.

7.6 Attributes [dcl.attr]

7.6.1 Attribute syntax and semantics [dcl.attr.grammar]

1 Attributes specify additional information for various source constructs such as types, variables, names, blocks, or translation units.

attribute-specifier-seq:
attribute-specifier-seq$_{opt}$ *attribute-specifier*

attribute-specifier:
`[ [` *attribute-list* `] ]`
alignment-specifier

alignment-specifier:
`alignas (` *type-id* `...`$_{opt}$`)`
`alignas (` *constant-expression* `...`$_{opt}$`)`

attribute-list:
attribute$_{opt}$
attribute-list `,` *attribute*$_{opt}$
attribute `...`
attribute-list `,` *attribute* `...`

attribute:
attribute-token attribute-argument-clause$_{opt}$

attribute-token:
identifier
attribute-scoped-token

attribute-scoped-token:
attribute-namespace `::` *identifier*

attribute-namespace:
identifier

attribute-argument-clause:
`(` *balanced-token-seq* `)`

balanced-token-seq:
balanced-token$_{opt}$
balanced-token-seq balanced-token
balanced-token:
`(` *balanced-token-seq* `)`
`[` *balanced-token-seq* `]`
`{` *balanced-token-seq* `}`
any *token* other than a parenthesis, a bracket, or a brace

2 [*Note:* For each individual attribute, the form of the *balanced-token-seq* will be specified. — *end note*]

3 In an *attribute-list*, an ellipsis may appear only if that *attribute*'s specification permits it. An *attribute* followed by an ellipsis is a pack expansion (14.5.3). An *attribute-specifier* that contains no *attribute*s has no effect. The order in which the *attribute-tokens* appear in an *attribute-list* is not significant. If a keyword (2.11) or an alternative token (2.5) that satisfies the syntactic requirements of an *identifier* (2.10) is contained in an *attribute-token*, it is considered an identifier. No name lookup (3.4) is performed on any of the identifiers contained in an *attribute-token*. The *attribute-token* determines additional requirements on the *attribute-argument-clause* (if any). The use of an *attribute-scoped-token* is conditionally-supported, with implementation-defined behavior. [*Note:* Each implementation should choose a distinctive name for the *attribute-namespace* in an *attribute-scoped-token*. — *end note*]

4 Each *attribute-specifier-seq* is said to *appertain* to some entity or statement, identified by the syntactic context where it appears (Clause 6, Clause 7, Clause 8). If an *attribute-specifier-seq* that appertains to some entity or statement contains an *attribute* that is not allowed to apply to that entity or statement, the program is ill-formed. If an *attribute-specifier-seq* appertains to a friend declaration (11.3), that declaration shall be a definition. No *attribute-specifier-seq* shall appertain to an explicit instantiation (14.7.2).

5 For an *attribute-token* not specified in this International Standard, the behavior is implementation-defined.

6 Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier*. [*Note:* If two consecutive left square brackets appear where an *attribute-specifier* is not allowed, the program is ill-formed even if the brackets match an alternative grammar production. — *end note*] [*Example:*

```
int p[10];
void f() {
  int x = 42, y[5];
  int(p[[x] { return x; }()]);  // error: invalid attribute on a nested
                                // declarator-id and not a function-style cast of
                                // an element of p.
  y[[] { return 2; }()] = 2;    // error even though attributes are not allowed
                                // in this context.
}
```

— *end example*]

7.6.2 Alignment specifier [dcl.align]

1 An *alignment-specifier* may be applied to a variable or to a class data member, but it shall not be applied to a bit-field, a function parameter, an *exception-declaration* (15.3), or a variable declared with the `register` storage class specifier. An *alignment-specifier* may also be applied to the declaration or definition of a class (in an *elaborated-type-specifier* (7.1.6.3) or *class-head* (Clause 9), respectively) and to the declaration or definition of an enumeration (in an *opaque-enum-declaration* or *enum-head*, respectively (7.2)). An *alignment-specifier* with an ellipsis is a pack expansion (14.5.3).

2 When the *alignment-specifier* is of the form `alignas(` *constant-expression* `)`:

(2.1) — the *constant-expression* shall be an integral constant expression

(2.2) — if the constant expression evaluates to a fundamental alignment, the alignment requirement of the declared entity shall be the specified fundamental alignment

(2.3) — if the constant expression evaluates to an extended alignment and the implementation supports that alignment in the context of the declaration, the alignment of the declared entity shall be that alignment

(2.4) — if the constant expression evaluates to an extended alignment and the implementation does not support that alignment in the context of the declaration, the program is ill-formed

(2.5) — if the constant expression evaluates to zero, the alignment specifier shall have no effect

(2.6) — otherwise, the program is ill-formed.

3 When the *alignment-specifier* is of the form `alignas(` *type-id* `)`, it shall have the same effect as `alignas(alignof(`*type-id*`))` (5.3.6).

4 When multiple *alignment-specifier*s are specified for an entity, the alignment requirement shall be set to the strictest specified alignment.

5 The combined effect of all *alignment-specifier*s in a declaration shall not specify an alignment that is less strict than the alignment that would be required for the entity being declared if all *alignment-specifier*s appertaining to that entity were omitted. [*Example:*

```
struct alignas(8) S {};
struct alignas(1) U {
  S s;
};    // Error: U specifies an alignment that is less strict than
      // if the alignas(1) were omitted.
```

— *end example*]

6 If the defining declaration of an entity has an *alignment-specifier*, any non-defining declaration of that entity shall either specify equivalent alignment or have no *alignment-specifier*. Conversely, if any declaration of an entity has an *alignment-specifier*, every defining declaration of that entity shall specify an equivalent alignment. No diagnostic is required if declarations of an entity have different *alignment-specifier*s in different translation units.

[*Example:*

```
// Translation unit #1:
struct S { int x; } s, p = &s;

// Translation unit #2:
struct alignas(16) S;           // error: definition of S lacks alignment; no
extern S* p;                    // diagnostic required
```

— *end example*]

7 [*Example:* An aligned buffer with an alignment requirement of `A` and holding `N` elements of type `T` other than `char`, `signed char`, or `unsigned char` can be declared as:

```
alignas(T) alignas(A) T buffer[N];
```

Specifying `alignas(T)` ensures that the final requested alignment will not be weaker than `alignof(T)`, and therefore the program will not be ill-formed. — *end example*]

8 [*Example:*

```
alignas(double) void f();                          // error: alignment applied to function
alignas(double) unsigned char c[sizeof(double)];   // array of characters, suitably aligned for a double
```

```
extern unsigned char c[sizeof(double)];          // no alignas necessary
alignas(float)
  extern unsigned char c[sizeof(double)];        // error: different alignment in declaration
```

— *end example*]

7.6.3 Noreturn attribute [dcl.attr.noreturn]

1 The *attribute-token* `noreturn` specifies that a function does not return. It shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. The attribute may be applied to the *declarator-id* in a function declaration. The first declaration of a function shall specify the `noreturn` attribute if any declaration of that function specifies the `noreturn` attribute. If a function is declared with the `noreturn` attribute in one translation unit and the same function is declared without the `noreturn` attribute in another translation unit, the program is ill-formed; no diagnostic required.

2 If a function `f` is called where `f` was previously declared with the `noreturn` attribute and `f` eventually returns, the behavior is undefined. [*Note:* The function may terminate by throwing an exception. — *end note*] [*Note:* Implementations are encouraged to issue a warning if a function marked `[[noreturn]]` might return. — *end note*]

3 [*Example:*

```
[[ noreturn ]] void f() {
  throw "error";        // OK
}

[[ noreturn ]] void q(int i) { // behavior is undefined if called with an argument <= 0
  if (i > 0)
    throw "positive";
}
```

— *end example*]

7.6.4 Carries dependency attribute [dcl.attr.depend]

1 The *attribute-token* `carries_dependency` specifies dependency propagation into and out of functions. It shall appear at most once in each *attribute-list* and no *attribute-argument-clause* shall be present. The attribute may be applied to the *declarator-id* of a *parameter-declaration* in a function declaration or lambda, in which case it specifies that the initialization of the parameter carries a dependency to (1.10) each lvalue-to-rvalue conversion (4.1) of that object. The attribute may also be applied to the *declarator-id* of a function declaration, in which case it specifies that the return value, if any, carries a dependency to the evaluation of the function call expression.

2 The first declaration of a function shall specify the `carries_dependency` attribute for its *declarator-id* if any declaration of the function specifies the `carries_dependency` attribute. Furthermore, the first declaration of a function shall specify the `carries_dependency` attribute for a parameter if any declaration of that function specifies the `carries_dependency` attribute for that parameter. If a function or one of its parameters is declared with the `carries_dependency` attribute in its first declaration in one translation unit and the same function or one of its parameters is declared without the `carries_dependency` attribute in its first declaration in another translation unit, the program is ill-formed; no diagnostic required.

3 [*Note:* The `carries_dependency` attribute does not change the meaning of the program, but may result in generation of more efficient code. — *end note*]

4 [*Example:*

```
/* Translation unit A. */
```

```
struct foo { int* a; int* b; };
std::atomic<struct foo *> foo_head[10];
int foo_array[10][10];

[[carries_dependency]] struct foo* f(int i) {
  return foo_head[i].load(memory_order_consume);
}

int g(int* x, int* y [[carries_dependency]]) {
  return kill_dependency(foo_array[*x][*y]);
}

/* Translation unit B. */

[[carries_dependency]] struct foo* f(int i);
int g(int* x, int* y [[carries_dependency]]);

int c = 3;

void h(int i) {
  struct foo* p;

  p = f(i);
  do_something_with(g(&c, p->a));
  do_something_with(g(p->a, &c));
}
```

5 The `carries_dependency` attribute on function `f` means that the return value carries a dependency out of `f`, so that the implementation need not constrain ordering upon return from `f`. Implementations of `f` and its caller may choose to preserve dependencies instead of emitting hardware memory ordering instructions (a.k.a. fences).

6 Function `g`'s second parameter has a `carries_dependency` attribute, but its first parameter does not. Therefore, function `h`'s first call to `g` carries a dependency into `g`, but its second call does not. The implementation might need to insert a fence prior to the second call to `g`.

— *end example*]

7.6.5 Deprecated attribute [dcl.attr.deprecated]

1 The *attribute-token* `deprecated` can be used to mark names and entities whose use is still allowed, but is discouraged for some reason. [*Note:* in particular, `deprecated` is appropriate for names and entities that are deemed obsolescent or unsafe. — *end note*] It shall appear at most once in each *attribute-list*. An *attribute-argument-clause* may be present and, if present, it shall have the form:

```
( string-literal )
```

[*Note:* the *string-literal* in the *attribute-argument-clause* could be used to explain the rationale for deprecation and/or to suggest a replacing entity. — *end note*]

2 The attribute may be applied to the declaration of a class, a *typedef-name*, a variable, a non-static data member, a function, a namespace, an enumeration, an enumerator, or a template specialization.

3 A name or entity declared without the `deprecated` attribute can later be re-declared with the attribute and vice-versa. [*Note:* Thus, an entity initially declared without the attribute can be marked as deprecated by a subsequent redeclaration. However, after an entity is marked as deprecated, later redeclarations do not

un-deprecate the entity. — *end note*] Redeclarations using different forms of the attribute (with or without the *attribute-argument-clause* or with different *attribute-argument-clause*s) are allowed.

4 [*Note:* Implementations may use the `deprecated` attribute to produce a diagnostic message in case the program refers to a name or entity other than to declare it, after a declaration that specifies the attribute. The diagnostic message may include the text provided within the *attribute-argument-clause* of any `deprecated` attribute applied to the name or entity. — *end note*]

8 Declarators [dcl.decl]

1 A declarator declares a single variable, function, or type, within a declaration. The *init-declarator-list* appearing in a declaration is a comma-separated sequence of declarators, each of which can have an initializer.

init-declarator-list:
init-declarator
init-declarator-list **,** *init-declarator*

init-declarator:
declarator initializer$_{opt}$

2 The three components of a *simple-declaration* are the attributes (7.6), the specifiers (*decl-specifier-seq*; 7.1) and the declarators (*init-declarator-list*). The specifiers indicate the type, storage class or other properties of the entities being declared. The declarators specify the names of these entities and (optionally) modify the type of the specifiers with operators such as `*` (pointer to) and `()` (function returning). Initial values can also be specified in a declarator; initializers are discussed in 8.5 and 12.6.

3 Each *init-declarator* in a declaration is analyzed separately as if it was in a declaration by itself.[100]

4 Declarators have the syntax

declarator:
ptr-declarator
noptr-declarator parameters-and-qualifiers trailing-return-type

ptr-declarator:
noptr-declarator
ptr-operator ptr-declarator

noptr-declarator:
declarator-id attribute-specifier-seq$_{opt}$
noptr-declarator parameters-and-qualifiers
noptr-declarator **[** *constant-expression*$_{opt}$**]** *attribute-specifier-seq*$_{opt}$
(*ptr-declarator* **)**

parameters-and-qualifiers:
(*parameter-declaration-clause* **)** *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$

100) A declaration with several declarators is usually equivalent to the corresponding sequence of declarations each with a single declarator. That is
```
T D1, D2, ... Dn;
```
is usually equivalent to
```
T D1; T D2; ... T Dn;
```
where `T` is a *decl-specifier-seq* and each `Di` is an *init-declarator*. An exception occurs when a name introduced by one of the *declarator*s hides a type name used by the *decl-specifiers*, so that when the same *decl-specifiers* are used in a subsequent declaration, they do not have the same meaning, as in
```
struct S ... ;
S S, T; // declare two instances of struct S
```
which is not equivalent to
```
struct S ... ;
S S;
S T; // error
```
Another exception occurs when `T` is `auto` (7.1.6.4), for example:
```
auto i = 1, j = 2.0; // error: deduced types for i and j do not match
```
as opposed to
```
auto i = 1; // OK: i deduced to have type int
auto j = 2.0; // OK: j deduced to have type double
```

trailing-return-type:
`->` *trailing-type-specifier-seq abstract-declarator*$_{opt}$

ptr-operator:
`*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$
`&` *attribute-specifier-seq*$_{opt}$
`&&` *attribute-specifier-seq*$_{opt}$
nested-name-specifier `*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$

cv-qualifier-seq:
cv-qualifier cv-qualifier-seq$_{opt}$

cv-qualifier:
`const`
`volatile`

ref-qualifier:
`&`
`&&`

declarator-id:
`...`$_{opt}$ *id-expression*

5 The optional *attribute-specifier-seq* in a *trailing-return-type* appertains to the indicated return type. The *type-id* in a *trailing-return-type* includes the longest possible sequence of *abstract-declarator*s. [*Note:* This resolves the ambiguous binding of array and function declarators. [*Example:*

```
auto f()->int(*)[4];   // function returning a pointer to array[4] of int
                       // not function returning array[4] of pointer to int
```

— *end example*] — *end note*]

8.1 Type names [dcl.name]

1 To specify type conversions explicitly, and as an argument of `sizeof`, `alignof`, `new`, or `typeid`, the name of a type shall be specified. This can be done with a *type-id*, which is syntactically a declaration for a variable or function of that type that omits the name of the entity.

type-id:
type-specifier-seq abstract-declarator$_{opt}$

abstract-declarator:
ptr-abstract-declarator
noptr-abstract-declarator$_{opt}$ *parameters-and-qualifiers trailing-return-type*
abstract-pack-declarator

ptr-abstract-declarator:
noptr-abstract-declarator
ptr-operator ptr-abstract-declarator$_{opt}$

noptr-abstract-declarator:
noptr-abstract-declarator$_{opt}$ *parameters-and-qualifiers*
noptr-abstract-declarator$_{opt}$ `[` *constant-expression*$_{opt}$ `]` *attribute-specifier-seq*$_{opt}$
`(` *ptr-abstract-declarator* `)`

abstract-pack-declarator:
noptr-abstract-pack-declarator
ptr-operator abstract-pack-declarator

noptr-abstract-pack-declarator:
noptr-abstract-pack-declarator parameters-and-qualifiers
noptr-abstract-pack-declarator `[` *constant-expression*$_{opt}$ `]` *attribute-specifier-seq*$_{opt}$
`...`

It is possible to identify uniquely the location in the *abstract-declarator* where the identifier would appear if the construction were a declarator in a declaration. The named type is then the same as the type of the hypothetical identifier. [*Example:*

```
int                 // int i
int *               // int *pi
int *[3]            // int *p[3]
int (*)[3]          // int (*p3i)[3]
int *()             // int *f()
int (*)(double)     // int (*pf)(double)
```

name respectively the types "`int`", "pointer to `int`", "array of 3 pointers to `int`", "pointer to array of 3 `int`", "function of (no parameters) returning pointer to `int`", and "pointer to a function of (`double`) returning `int`". — *end example*]

2 A type can also be named (often more easily) by using a *typedef* (7.1.3).

8.2 Ambiguity resolution [dcl.ambig.res]

1 The ambiguity arising from the similarity between a function-style cast and a declaration mentioned in 6.8 can also occur in the context of a declaration. In that context, the choice is between a function declaration with a redundant set of parentheses around a parameter name and an object declaration with a function-style cast as the initializer. Just as for the ambiguities mentioned in 6.8, the resolution is to consider any construct that could possibly be a declaration a declaration. [*Note:* A declaration can be explicitly disambiguated by a nonfunction-style cast, by an `=` to indicate initialization or by removing the redundant parentheses around the parameter name. — *end note*] [*Example:*

```
struct S {
  S(int);
};

void foo(double a) {
  S w(int(a));       // function declaration
  S x(int());        // function declaration
  S y((int)a);       // object declaration
  S z = int(a);      // object declaration
}
```

— *end example*]

2 The ambiguity arising from the similarity between a function-style cast and a *type-id* can occur in different contexts. The ambiguity appears as a choice between a function-style cast expression and a declaration of a type. The resolution is that any construct that could possibly be a *type-id* in its syntactic context shall be considered a *type-id*.

3 [*Example:*

```
#include <cstddef>
char* p;
void* operator new(std::size_t, int);
void foo()  {
  const int x = 63;
  new (int(*p)) int;            // new-placement
  new (int(*[x]));              // parenthesized type-id
}
```

4 For another example,

```
template <class T>
struct S {
  T* p;
};
```

```
S<int()> x;                    // type-id
S<int(1)> y;                   // expression (ill-formed)
```

5 For another example,

```
void foo() {
  sizeof(int(1));              // expression
  sizeof(int());               // type-id (ill-formed)
}
```

6 For another example,

```
void foo() {
  (int(1));                    // expression
  (int())1;                    // type-id (ill-formed)
}
```

— *end example*]

7 Another ambiguity arises in a *parameter-declaration-clause* of a function declaration, or in a *type-id* that is the operand of a `sizeof` or `typeid` operator, when a *type-name* is nested in parentheses. In this case, the choice is between the declaration of a parameter of type pointer to function and the declaration of a parameter with redundant parentheses around the *declarator-id*. The resolution is to consider the *type-name* as a *simple-type-specifier* rather than a *declarator-id*. [*Example:*

```
class C { };
void f(int(C)) { }             // void f(int(*fp)(C c)) { }
                               // not: void f(int C);

int g(C);

void foo() {
  f(1);                        // error: cannot convert 1 to function pointer
  f(g);                        // OK
}
```

For another example,

```
class C { };
void h(int *(C[10]));          // void h(int *(*_fp)(C _parm[10]));
                               // not: void h(int *C[10]);
```

— *end example*]

8.3 Meaning of declarators [dcl.meaning]

1 A list of declarators appears after an optional (Clause 7) *decl-specifier-seq* (7.1). Each declarator contains exactly one *declarator-id*; it names the identifier that is declared. An *unqualified-id* occurring in a *declarator-id* shall be a simple *identifier* except for the declaration of some special functions (12.1, 12.3, 12.4, 13.5) and for the declaration of template specializations or partial specializations (14.7). When the *declarator-id* is qualified, the declaration shall refer to a previously declared member of the class or namespace to which the qualifier refers (or, in the case of a namespace, of an element of the inline namespace set of that namespace (7.3.1)) or to a specialization thereof; the member shall not merely have been introduced by a *using-declaration* in the scope of the class or namespace nominated by the *nested-name-specifier* of the *declarator-id*. The *nested-name-specifier* of a qualified *declarator-id* shall not begin with a *decltype-specifier*. [*Note:* If the qualifier is the global `::` scope resolution operator, the *declarator-id* refers to a name declared

in the global namespace scope. — *end note*] The optional *attribute-specifier-seq* following a *declarator-id* appertains to the entity that is declared.

2 A `static`, `thread_local`, `extern`, `register`, `mutable`, `friend`, `inline`, `virtual`, or `typedef` specifier applies directly to each *declarator-id* in an *init-declarator-list*; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

3 Thus, a declaration of a particular identifier has the form

```
T D
```

where `T` is of the form *attribute-specifier-seq*$_{opt}$ *decl-specifier-seq* and `D` is a declarator. Following is a recursive procedure for determining the type specified for the contained *declarator-id* by such a declaration.

4 First, the *decl-specifier-seq* determines a type. In a declaration

```
T D
```

the *decl-specifier-seq* `T` determines the type `T`. [*Example:* in the declaration

```
int unsigned i;
```

the type specifiers `int unsigned` determine the type "`unsigned int`" (7.1.6.2). — *end example*]

5 In a declaration *attribute-specifier-seq*$_{opt}$ `T D` where `D` is an unadorned identifier the type of this identifier is "`T`".

6 In a declaration `T D` where `D` has the form

(D1)

the type of the contained *declarator-id* is the same as that of the contained *declarator-id* in the declaration

```
T D1
```

Parentheses do not alter the type of the embedded *declarator-id*, but they can alter the binding of complex declarators.

8.3.1 Pointers [dcl.ptr]

1 In a declaration `T D` where `D` has the form

`*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$`D1`

and the type of the identifier in the declaration `T D1` is "*derived-declarator-type-list* `T`", then the type of the identifier of `D` is "*derived-declarator-type-list cv-qualifier-seq* pointer to `T`". The *cv-qualifier*s apply to the pointer and not to the object pointed to. Similarly, the optional *attribute-specifier-seq* (7.6.1) appertains to the pointer and not to the object pointed to.

2 [*Example:* the declarations

```
const int ci = 10, *pc = &ci, *const cpc = pc, **ppc;
int i, *p, *const cp = &i;
```

declare `ci`, a constant integer; `pc`, a pointer to a constant integer; `cpc`, a constant pointer to a constant integer; `ppc`, a pointer to a pointer to a constant integer; `i`, an integer; `p`, a pointer to integer; and `cp`, a constant pointer to integer. The value of `ci`, `cpc`, and `cp` cannot be changed after initialization. The value of `pc` can be changed, and so can the object pointed to by `cp`. Examples of some correct operations are

```
i = ci;
*cp = ci;
pc++;
pc = cpc;
```

```
pc = p;
ppc = &pc;
```

Examples of ill-formed operations are

```
ci = 1;             // error
ci++;               // error
*pc = 2;            // error
cp = &ci;           // error
cpc++;              // error
p = pc;             // error
ppc = &p;           // error
```

Each is unacceptable because it would either change the value of an object declared `const` or allow it to be changed through a cv-unqualified pointer later, for example:

```
*ppc = &ci;         // OK, but would make p point to ci ...
                    // ... because of previous error
*p = 5;             // clobber ci
```

— *end example*]

3 See also 5.18 and 8.5.

4 [*Note:* Forming a pointer to reference type is ill-formed; see 8.3.2. Forming a pointer to function type is ill-formed if the function type has *cv-qualifier*s or a *ref-qualifier*; see 8.3.5. Since the address of a bit-field (9.6) cannot be taken, a pointer can never point to a bit-field. — *end note*]

8.3.2 References [dcl.ref]

1 In a declaration `T D` where `D` has either of the forms

`&` *attribute-specifier-seq*$_{opt}$`D1`
`&&` *attribute-specifier-seq*$_{opt}$`D1`

and the type of the identifier in the declaration `T D1` is "*derived-declarator-type-list* `T`", then the type of the identifier of `D` is "*derived-declarator-type-list* reference to `T`". The optional *attribute-specifier-seq* appertains to the reference type. Cv-qualified references are ill-formed except when the cv-qualifiers are introduced through the use of a *typedef-name* (7.1.3, 14.1) or *decltype-specifier* (7.1.6.2), in which case the cv-qualifiers are ignored. [*Example:*

```
typedef int& A;
const A aref = 3;   // ill-formed; lvalue reference to non-const initialized with rvalue
```

The type of `aref` is "lvalue reference to `int`", not "lvalue reference to `const int`". — *end example*] [*Note:* A reference can be thought of as a name of an object. — *end note*] A declarator that specifies the type "reference to *cv* `void`" is ill-formed.

2 A reference type that is declared using `&` is called an *lvalue reference*, and a reference type that is declared using `&&` is called an *rvalue reference*. Lvalue references and rvalue references are distinct types. Except where explicitly noted, they are semantically equivalent and commonly referred to as references.

3 [*Example:*

```
void f(double& a) { a += 3.14; }
// ...
double d = 0;
f(d);
```

declares `a` to be a reference parameter of `f` so the call `f(d)` will add `3.14` to `d`.

```
int v[20];
// ...
int& g(int i) { return v[i]; }
// ...
g(3) = 7;
```

declares the function `g()` to return a reference to an integer so `g(3)=7` will assign `7` to the fourth element of the array `v`. For another example,

```
struct link {
  link* next;
};

link* first;

void h(link*& p) {  // p is a reference to pointer
  p->next = first;
  first = p;
  p = 0;
}

void k() {
   link* q = new link;
   h(q);
}
```

declares `p` to be a reference to a pointer to `link` so `h(q)` will leave `q` with the value zero. See also 8.5.3. — *end example*]

4 It is unspecified whether or not a reference requires storage (3.7).

5 There shall be no references to references, no arrays of references, and no pointers to references. The declaration of a reference shall contain an *initializer* (8.5.3) except when the declaration contains an explicit `extern` specifier (7.1.1), is a class member (9.2) declaration within a class definition, or is the declaration of a parameter or a return type (8.3.5); see 3.1. A reference shall be initialized to refer to a valid object or function. [*Note:* in particular, a null reference cannot exist in a well-defined program, because the only way to create such a reference would be to bind it to the "object" obtained by indirection through a null pointer, which causes undefined behavior. As described in 9.6, a reference cannot be bound directly to a bit-field. — *end note*]

6 If a *typedef-name* (7.1.3, 14.1) or a *decltype-specifier* (7.1.6.2) denotes a type `TR` that is a reference to a type `T`, an attempt to create the type "lvalue reference to *cv* `TR`" creates the type "lvalue reference to `T`", while an attempt to create the type "rvalue reference to *cv* `TR`" creates the type `TR`. [*Example:*

```
int i;
typedef int& LRI;
typedef int&& RRI;

LRI& r1 = i;                    // r1 has the type int&
const LRI& r2 = i;              // r2 has the type int&
const LRI&& r3 = i;             // r3 has the type int&

RRI& r4 = i;                    // r4 has the type int&
RRI&& r5 = 5;                   // r5 has the type int&&

decltype(r2)& r6 = i;           // r6 has the type int&
decltype(r2)&& r7 = i;          // r7 has the type int&
```

— *end example*]

7 [*Note:* Forming a reference to function type is ill-formed if the function type has *cv-qualifier*s or a *ref-qualifier*; see 8.3.5. — *end note*]

8.3.3 Pointers to members [dcl.mptr]

1 In a declaration `T D` where `D` has the form

nested-name-specifier `*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$ *D1*

and the *nested-name-specifier* denotes a class, and the type of the identifier in the declaration `T D1` is "*derived-declarator-type-list* `T`", then the type of the identifier of `D` is "*derived-declarator-type-list cv-qualifier-seq* pointer to member of class *nested-name-specifier* of type `T`". The optional *attribute-specifier-seq* (7.6.1) appertains to the pointer-to-member.

2 [*Example:*

```
struct X {
  void f(int);
  int a;
};
struct Y;

int X::* pmi = &X::a;
void (X::* pmf)(int) = &X::f;
double X::* pmd;
char Y::* pmc;
```

declares `pmi`, `pmf`, `pmd` and `pmc` to be a pointer to a member of `X` of type `int`, a pointer to a member of `X` of type `void(int)`, a pointer to a member of `X` of type `double` and a pointer to a member of `Y` of type `char` respectively. The declaration of `pmd` is well-formed even though `X` has no members of type `double`. Similarly, the declaration of `pmc` is well-formed even though `Y` is an incomplete type. `pmi` and `pmf` can be used like this:

```
X obj;
// ...
obj.*pmi = 7;        // assign 7 to an integer
                     // member of obj
(obj.*pmf)(7);       // call a function member of obj
                     // with the argument 7
```

— *end example*]

3 A pointer to member shall not point to a static member of a class (9.4), a member with reference type, or "*cv* `void`".

[*Note:* See also 5.3 and 5.5. The type "pointer to member" is distinct from the type "pointer", that is, a pointer to member is declared only by the pointer to member declarator syntax, and never by the pointer declarator syntax. There is no "reference-to-member" type in C++. — *end note*]

8.3.4 Arrays [dcl.array]

1 In a declaration `T D` where `D` has the form

`D1 [` *constant-expression*$_{opt}$`]` *attribute-specifier-seq*$_{opt}$

and the type of the identifier in the declaration `T D1` is "*derived-declarator-type-list* `T`", then the type of the identifier of `D` is an array type; if the type of the identifier of `D` contains the `auto` *type-specifier*, the program is ill-formed. `T` is called the array *element type*; this type shall not be a reference type, the (possibly cv-qualified) type `void`, a function type or an abstract class type. If the *constant-expression* (5.20) is present, it

shall be a converted constant expression of type `std::size_t` and its value shall be greater than zero. The constant expression specifies the *bound* of (number of elements in) the array. If the value of the constant expression is `N`, the array has `N` elements numbered `0` to `N-1`, and the type of the identifier of `D` is "*derived-declarator-type-list* array of `N T`". An object of array type contains a contiguously allocated non-empty set of `N` subobjects of type `T`. Except as noted below, if the constant expression is omitted, the type of the identifier of `D` is "*derived-declarator-type-list* array of unknown bound of `T`", an incomplete object type. The type "*derived-declarator-type-list* array of `N T`" is a different type from the type "*derived-declarator-type-list* array of unknown bound of `T`", see 3.9. Any type of the form "*cv-qualifier-seq* array of `N T`" is adjusted to "array of `N` *cv-qualifier-seq* `T`", and similarly for "array of unknown bound of `T`". The optional *attribute-specifier-seq* appertains to the array. [*Example:*

```
typedef int A[5], AA[2][3];
typedef const A CA;             // type is "array of 5 const int"
typedef const AA CAA;           // type is "array of 2 array of 3 const int"
```

— *end example*] [*Note:* An "array of `N` *cv-qualifier-seq* `T`" has cv-qualified type; see 3.9.3. — *end note*]

2 An array can be constructed from one of the fundamental types (except `void`), from a pointer, from a pointer to member, from a class, from an enumeration type, or from another array.

3 When several "array of" specifications are adjacent, a multidimensional array is created; only the first of the constant expressions that specify the bounds of the arrays may be omitted. In addition to declarations in which an incomplete object type is allowed, an array bound may be omitted in some cases in the declaration of a function parameter (8.3.5). An array bound may also be omitted when the declarator is followed by an *initializer* (8.5). In this case the bound is calculated from the number of initial elements (say, `N`) supplied (8.5.1), and the type of the identifier of `D` is "array of `N T`". Furthermore, if there is a preceding declaration of the entity in the same scope in which the bound was specified, an omitted array bound is taken to be the same as in that earlier declaration, and similarly for the definition of a static data member of a class.

4 [*Example:*

```
float fa[17], *afp[17];
```

declares an array of `float` numbers and an array of pointers to `float` numbers. For another example,

```
static int x3d[3][5][7];
```

declares a static three-dimensional array of integers, with rank $3 \times 5 \times 7$. In complete detail, `x3d` is an array of three items; each item is an array of five arrays; each of the latter arrays is an array of seven integers. Any of the expressions `x3d`, `x3d[i]`, `x3d[i][j]`, `x3d[i][j][k]` can reasonably appear in an expression. Finally,

```
extern int x[10];
struct S {
  static int y[10];
};

int x[];                        // OK: bound is 10
int S::y[];                     // OK: bound is 10

void f() {
  extern int x[];
  int i = sizeof(x);            // error: incomplete object type
}
```

— *end example*]

5 [*Note:* conversions affecting expressions of array type are described in 4.2. Objects of array types cannot be modified, see 3.10. — *end note*]

6 [*Note:* Except where it has been declared for a class (13.5.5), the subscript operator `[]` is interpreted in such a way that `E1[E2]` is identical to `*((E1)+(E2))`. Because of the conversion rules that apply to `+`, if `E1` is an array and `E2` an integer, then `E1[E2]` refers to the `E2`-th member of `E1`. Therefore, despite its asymmetric appearance, subscripting is a commutative operation.

7 A consistent rule is followed for multidimensional arrays. If `E` is an n-dimensional array of rank $i \times j \times \ldots \times k$, then `E` appearing in an expression that is subject to the array-to-pointer conversion (4.2) is converted to a pointer to an $(n-1)$-dimensional array with rank $j \times \ldots \times k$. If the `*` operator, either explicitly or implicitly as a result of subscripting, is applied to this pointer, the result is the pointed-to $(n-1)$-dimensional array, which itself is immediately converted into a pointer.

8 [*Example:* consider

```
int x[3][5];
```

Here `x` is a 3×5 array of integers. When `x` appears in an expression, it is converted to a pointer to (the first of three) five-membered arrays of integers. In the expression `x[i]` which is equivalent to `*(x+i)`, `x` is first converted to a pointer as described; then `x+i` is converted to the type of `x`, which involves multiplying `i` by the length of the object to which the pointer points, namely five integer objects. The results are added and indirection applied to yield an array (of five integers), which in turn is converted to a pointer to the first of the integers. If there is another subscript the same argument applies again; this time the result is an integer. — *end example*] — *end note*]

9 [*Note:* It follows from all this that arrays in C++ are stored row-wise (last subscript varies fastest) and that the first subscript in the declaration helps determine the amount of storage consumed by an array but plays no other part in subscript calculations. — *end note*]

8.3.5 Functions [dcl.fct]

1 In a declaration `T D` where `D` has the form

`D1 (` *parameter-declaration-clause* `)` *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$

and the type of the contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", the type of the *declarator-id* in `D` is "*derived-declarator-type-list* function of (*parameter-declaration-clause*) *cv-qualifier-seq*$_{opt}$ *ref-qualifier*$_{opt}$ returning `T`". The optional *attribute-specifier-seq* appertains to the function type.

2 In a declaration `T D` where `D` has the form

`D1 (` *parameter-declaration-clause* `)` *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$ *trailing-return-type*

and the type of the contained *declarator-id* in the declaration `T D1` is "*derived-declarator-type-list* `T`", `T` shall be the single *type-specifier* `auto`. The type of the *declarator-id* in `D` is "*derived-declarator-type-list* function of (*parameter-declaration-clause*) *cv-qualifier-seq*$_{opt}$ *ref-qualifier*$_{opt}$ returning `U`", where `U` is the type specified by the *trailing-return-type*. The optional *attribute-specifier-seq* appertains to the function type.

3 A type of either form is a *function type*.[101]

parameter-declaration-clause:
parameter-declaration-list$_{opt}$ `...`$_{opt}$
parameter-declaration-list `,` `...`

parameter-declaration-list:
parameter-declaration
parameter-declaration-list `,` *parameter-declaration*

101) As indicated by syntax, cv-qualifiers are a significant component in function return types.

parameter-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator* `=` *initializer-clause*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$
attribute-specifier-seq$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$`=` *initializer-clause*

The optional *attribute-specifier-seq* in a *parameter-declaration* appertains to the parameter.

4 The *parameter-declaration-clause* determines the arguments that can be specified, and their processing, when the function is called. [*Note:* the *parameter-declaration-clause* is used to convert the arguments specified on the function call; see 5.2.2. — *end note*] If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type *cv* `void`. If the *parameter-declaration-clause* terminates with an ellipsis or a function parameter pack (14.5.3), the number of arguments shall be equal to or greater than the number of parameters that do not have a default argument and are not function parameter packs. Where syntactically correct and where "`...`" is not part of an *abstract-declarator*, "`, ...`" is synonymous with "`...`". [*Example:* the declaration

```
int printf(const char*, ...);
```

declares a function that can be called with varying numbers and types of arguments.

```
printf("hello world");
printf("a=%d b=%d", a, b);
```

However, the first argument must be of a type that can be converted to a `const char*` — *end example*] [*Note:* The standard header `<cstdarg>` contains a mechanism for accessing arguments passed using the ellipsis (see 5.2.2 and 18.10). — *end note*]

5 A single name can be used for several different functions in a single scope; this is function overloading (Clause 13). All declarations for a function shall agree exactly in both the return type and the parameter-type-list. The type of a function is determined using the following rules. The type of each parameter (including function parameter packs) is determined from its own *decl-specifier-seq* and *declarator*. After determining the type of each parameter, any parameter of type "array of `T`" or "function returning `T`" is adjusted to be "pointer to `T`" or "pointer to function returning `T`", respectively. After producing the list of parameter types, any top-level *cv-qualifier*s modifying a parameter type are deleted when forming the function type. The resulting list of transformed parameter types and the presence or absence of the ellipsis or a function parameter pack is the function's *parameter-type-list*. [*Note:* This transformation does not affect the types of the parameters. For example, `int(*)(const int p, decltype(p)*)` and `int(*)(int, const int*)` are identical types. — *end note*]

6 A function type with a *cv-qualifier-seq* or a *ref-qualifier* (including a type named by *typedef-name* (7.1.3, 14.1)) shall appear only as:

(6.1) — the function type for a non-static member function,

(6.2) — the function type to which a pointer to member refers,

(6.3) — the top-level function type of a function typedef declaration or *alias-declaration*,

(6.4) — the *type-id* in the default argument of a *type-parameter* (14.1), or

(6.5) — the *type-id* of a *template-argument* for a *type-parameter* (14.3.1).

[*Example:*

```
typedef int FIC(int) const;
FIC f;              // ill-formed: does not declare a member function
```

```
struct S {
  FIC f;            // OK
};
FIC S::*pm = &S::f; // OK
```

— *end example*]

7 The effect of a *cv-qualifier-seq* in a function declarator is not the same as adding cv-qualification on top of the function type. In the latter case, the cv-qualifiers are ignored. [*Note:* a function type that has a *cv-qualifier-seq* is not a cv-qualified type; there are no cv-qualified function types. — *end note*] [*Example:*

```
typedef void F();
struct S {
  const F f;        // OK: equivalent to: void f();
};
```

— *end example*]

8 The return type, the parameter-type-list, the *ref-qualifier*, and the *cv-qualifier-seq*, but not the default arguments (8.3.6) or the exception specification (15.4), are part of the function type. [*Note:* Function types are checked during the assignments and initializations of pointers to functions, references to functions, and pointers to member functions. — *end note*]

9 [*Example:* the declaration

```
int fseek(FILE*, long, int);
```

declares a function taking three arguments of the specified types, and returning `int` (7.1.6). — *end example*]

10 Functions shall not have a return type of type array or function, although they may have a return type of type pointer or reference to such things. There shall be no arrays of functions, although there can be arrays of pointers to functions.

11 Types shall not be defined in return or parameter types. The type of a parameter or the return type for a function definition shall not be an incomplete (possibly cv-qualified) class type in the context of the function definition unless the function is deleted (8.4.3).

12 A typedef of function type may be used to declare a function but shall not be used to define a function (8.4). [*Example:*

```
typedef void F();
F  fv;              // OK: equivalent to void fv();
F  fv { }           // ill-formed
void fv() { }       // OK: definition of fv
```

— *end example*]

13 An identifier can optionally be provided as a parameter name; if present in a function definition (8.4), it names a parameter. [*Note:* In particular, parameter names are also optional in function definitions and names used for a parameter in different declarations and the definition of a function need not be the same. If a parameter name is present in a function declaration that is not a definition, it cannot be used outside of its function declarator because that is the extent of its potential scope (3.3.4). — *end note*]

14 [*Example:* the declaration

```
int i,
    *pi,
    f(),
    *fpi(int),
    (*pif)(const char*, const char*),
    (*fpif(int))(int);
```

declares an integer `i`, a pointer `pi` to an integer, a function `f` taking no arguments and returning an integer, a function `fpi` taking an integer argument and returning a pointer to an integer, a pointer `pif` to a function which takes two pointers to constant characters and returns an integer, a function `fpif` taking an integer argument and returning a pointer to a function that takes an integer argument and returns an integer. It is especially useful to compare `fpi` and `pif`. The binding of `*fpi(int)` is `*(fpi(int))`, so the declaration suggests, and the same construction in an expression requires, the calling of a function `fpi`, and then using indirection through the (pointer) result to yield an integer. In the declarator `(*pif)(const char*, const char*)`, the extra parentheses are necessary to indicate that indirection through a pointer to a function yields a function, which is then called. — *end example*] [*Note:* Typedefs and *trailing-return-type*s are sometimes convenient when the return type of a function is complex. For example, the function `fpif` above could have been declared

```
typedef int  IFUNC(int);
IFUNC*  fpif(int);
```

or

```
auto fpif(int)->int(*)(int);
```

A *trailing-return-type* is most useful for a type that would be more complicated to specify before the *declarator-id*:

```
template <class T, class U> auto add(T t, U u) -> decltype(t + u);
```

rather than

```
template <class T, class U> decltype((*(T*)0) + (*(U*)0)) add(T t, U u);
```

— *end note*]

15 A *non-template function* is a function that is not a function template specialization. [*Note:* A function template is not a function. — *end note*]

16 A *declarator-id* or *abstract-declarator* containing an ellipsis shall only be used in a *parameter-declaration*. Such a *parameter-declaration* is a parameter pack (14.5.3). When it is part of a *parameter-declaration-clause*, the parameter pack is a function parameter pack (14.5.3). [*Note:* Otherwise, the *parameter-declaration* is part of a *template-parameter-list* and the parameter pack is a template parameter pack; see 14.1. — *end note*] A function parameter pack is a pack expansion (14.5.3). [*Example:*

```
template<typename... T> void f(T (* ...t)(int, int));

int add(int, int);
float subtract(int, int);

void g() {
  f(add, subtract);
}
```

— *end example*]

17 There is a syntactic ambiguity when an ellipsis occurs at the end of a *parameter-declaration-clause* without a preceding comma. In this case, the ellipsis is parsed as part of the *abstract-declarator* if the type of the parameter either names a template parameter pack that has not been expanded or contains `auto`; otherwise, it is parsed as part of the *parameter-declaration-clause*.[102]

102) One can explicitly disambiguate the parse either by introducing a comma (so the ellipsis will be parsed as part of the *parameter-declaration-clause*) or by introducing a name for the parameter (so the ellipsis will be parsed as part of the *declarator-id*).

8.3.6 Default arguments [dcl.fct.default]

1 If an *initializer-clause* is specified in a *parameter-declaration* this *initializer-clause* is used as a default argument. Default arguments will be used in calls where trailing arguments are missing.

2 [*Example:* the declaration

```
void point(int = 3, int = 4);
```

declares a function that can be called with zero, one, or two arguments of type `int`. It can be called in any of these ways:

```
point(1,2);  point(1);  point();
```

The last two calls are equivalent to `point(1,4)` and `point(3,4)`, respectively. — *end example*]

3 A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or *lambda-declarator* or in a *template-parameter* (14.1); in the latter case, the *initializer-clause* shall be an *assignment-expression*. A default argument shall not be specified for a parameter pack. If it is specified in a *parameter-declaration-clause*, it shall not occur within a *declarator* or *abstract-declarator* of a *parameter-declaration*.[103]

4 For non-template functions, default arguments can be added in later declarations of a function in the same scope. Declarations in different scopes have completely distinct sets of default arguments. That is, declarations in inner scopes do not acquire default arguments from declarations in outer scopes, and vice versa. In a given function declaration, each parameter subsequent to a parameter with a default argument shall have a default argument supplied in this or a previous declaration or shall be a function parameter pack. A default argument shall not be redefined by a later declaration (not even to the same value). [*Example:*

```
void g(int = 0, ...);           // OK, ellipsis is not a parameter so it can follow
                                // a parameter with a default argument
void f(int, int);
void f(int, int = 7);
void h() {
  f(3);                         // OK, calls f(3, 7)
  void f(int = 1, int);         // error: does not use default
                                // from surrounding scope
}
void m() {
  void f(int, int);             // has no defaults
  f(4);                         // error: wrong number of arguments
  void f(int, int = 5);         // OK
  f(4);                         // OK, calls f(4, 5);
  void f(int, int = 5);         // error: cannot redefine, even to
                                // same value
}
void n() {
  f(6);                         // OK, calls f(6, 7)
}
```

— *end example*] For a given inline function defined in different translation units, the accumulated sets of default arguments at the end of the translation units shall be the same; see 3.2. If a friend declaration specifies a default argument expression, that declaration shall be a definition and shall be the only declaration of the function or function template in the translation unit.

103) This means that default arguments cannot appear, for example, in declarations of pointers to functions, references to functions, or `typedef` declarations.

5 The default argument has the same semantic constraints as the initializer in a declaration of a variable of the parameter type, using the copy-initialization semantics (8.5). The names in the default argument are bound, and the semantic constraints are checked, at the point where the default argument appears. Name lookup and checking of semantic constraints for default arguments in function templates and in member functions of class templates are performed as described in 14.7.1. [*Example:* in the following code, `g` will be called with the value `f(2)`:

```
int a = 1;
int f(int);
int g(int x = f(a));            // default argument: f(::a)

void h() {
  a = 2;
  {
  int a = 3;
  g();                          // g(f(::a))
  }
}
```

— *end example*] [*Note:* In member function declarations, names in default arguments are looked up as described in 3.4.1. Access checking applies to names in default arguments as described in Clause 11. — *end note*]

6 Except for member functions of class templates, the default arguments in a member function definition that appears outside of the class definition are added to the set of default arguments provided by the member function declaration in the class definition; the program is ill-formed if a default constructor (12.1), copy or move constructor, or copy or move assignment operator (12.8) is so declared. Default arguments for a member function of a class template shall be specified on the initial declaration of the member function within the class template. [*Example:*

```
class C {
  void f(int i = 3);
  void g(int i, int j = 99);
};

void C::f(int i = 3) {          // error: default argument already
}                               // specified in class scope
void C::g(int i = 88, int j) {  // in this translation unit,
}                               // C::g can be called with no argument
```

— *end example*]

7 Local variables shall not be used in a default argument. [*Example:*

```
void f() {
  int i;
  extern void g(int x = i);     //error
  // ...
}
```

— *end example*]

8 The keyword `this` shall not be used in a default argument of a member function. [*Example:*

```
class A {
  void f(A* p = this) { }       // error
};
```

— *end example*]

9 A default argument is evaluated each time the function is called with no argument for the corresponding parameter. The order of evaluation of function arguments is unspecified. Consequently, parameters of a function shall not be used in a default argument, even if they are not evaluated. Parameters of a function declared before a default argument are in scope and can hide namespace and class member names. [*Example:*

```
int a;
int f(int a, int b = a);            // error: parameter a
                                    // used as default argument
typedef int I;
int g(float I, int b = I(2));       // error: parameter I found
int h(int a, int b = sizeof(a));    // error, parameter a used
                                    // in default argument
```

— *end example*] Similarly, a non-static member shall not be used in a default argument, even if it is not evaluated, unless it appears as the *id-expression* of a class member access expression (5.2.5) or unless it is used to form a pointer to member (5.3.1). [*Example:* the declaration of `X::mem1()` in the following example is ill-formed because no object is supplied for the non-static member `X::a` used as an initializer.

```
int b;
class X {
  int a;
  int mem1(int i = a);          // error: non-static member a
                                // used as default argument
  int mem2(int i = b);          // OK; use X::b
  static int b;
};
```

The declaration of `X::mem2()` is meaningful, however, since no object is needed to access the static member `X::b`. Classes, objects, and members are described in Clause 9. — *end example*] A default argument is not part of the type of a function. [*Example:*

```
int f(int = 0);

void h() {
  int j = f(1);
  int k = f();                  // OK, means f(0)
}

int (*p1)(int) = &f;
int (*p2)() = &f;               // error: type mismatch
```

— *end example*] When a declaration of a function is introduced by way of a *using-declaration* (7.3.3), any default argument information associated with the declaration is made known as well. If the function is redeclared thereafter in the namespace with additional default arguments, the additional arguments are also known at any point following the redeclaration where the *using-declaration* is in scope.

10 A virtual function call (10.3) uses the default arguments in the declaration of the virtual function determined by the static type of the pointer or reference denoting the object. An overriding function in a derived class does not acquire default arguments from the function it overrides. [*Example:*

```
struct A {
  virtual void f(int a = 7);
};
struct B : public A {
  void f(int a);
```

```
};
void m() {
  B* pb = new B;
  A* pa = pb;
  pa->f();          // OK, calls pa->B::f(7)
  pb->f();          // error: wrong number of arguments for B::f()
}
```

— *end example*]

8.4 Function definitions [dcl.fct.def]

8.4.1 In general [dcl.fct.def.general]

1 Function definitions have the form

function-definition:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *declarator virt-specifier-seq*$_{opt}$ *function-body*

function-body:
ctor-initializer$_{opt}$ *compound-statement*
function-try-block
`= default ;`
`= delete ;`

Any informal reference to the body of a function should be interpreted as a reference to the non-terminal *function-body*. The optional *attribute-specifier-seq* in a *function-definition* appertains to the function. A *virt-specifier-seq* can be part of a *function-definition* only if it is a *member-declaration* (9.2).

2 The *declarator* in a *function-definition* shall have the form

`D1 (` *parameter-declaration-clause* `)` *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$ *trailing-return-type*$_{opt}$

as described in 8.3.5. A function shall be defined only in namespace or class scope.

3 [*Example:* a simple example of a complete function definition is

```
int max(int a, int b, int c) {
  int m = (a > b) ? a : b;
  return (m > c) ? m : c;
}
```

Here `int` is the *decl-specifier-seq*; `max(int a, int b, int c)` is the *declarator*; `{ /* ... */ }` is the *function-body*. — *end example*]

4 A *ctor-initializer* is used only in a constructor; see 12.1 and 12.6.

5 [*Note:* A *cv-qualifier-seq* affects the type of `this` in the body of a member function; see 8.3.2. — *end note*]

6 [*Note:* Unused parameters need not be named. For example,

```
void print(int a, int) {
  std::printf("a = %d\n",a);
}
```

— *end note*]

7 In the *function-body*, a *function-local predefined variable* denotes a block-scope object of static storage duration that is implicitly defined (see 3.3.3).

8 The function-local predefined variable `__func__` is defined as if a definition of the form

```
static const char __func__[] = "function-name";
```

had been provided, where *function-name* is an implementation-defined string. It is unspecified whether such a variable has an address distinct from that of any other object in the program.[104]

[*Example:*

```
struct S {
  S() : s(__func__) { }             // OK
  const char* s;
};
void f(const char* s = __func__);   // error: __func__ is undeclared
```

— *end example*]

8.4.2 Explicitly-defaulted functions [dcl.fct.def.default]

1 A function definition of the form:

attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *declarator virt-specifier-seq*$_{opt}$ `= default ;`

is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

(1.1) — be a special member function,

(1.2) — have the same declared function type (except for possibly differing *ref-qualifier*s and except that in the case of a copy constructor or copy assignment operator, the parameter type may be "reference to non-const `T`", where `T` is the name of the member function's class) as if it had been implicitly declared, and

(1.3) — not have default arguments.

2 An explicitly-defaulted function that is not defined as deleted may be declared `constexpr` only if it would have been implicitly declared as `constexpr`. If a function is explicitly defaulted on its first declaration,

(2.1) — it is implicitly considered to be `constexpr` if the implicit declaration would be, and,

(2.2) — it has the same exception specification as if it had been implicitly declared (15.4).

3 If a function that is explicitly defaulted is declared with an *exception-specification* that is not compatible (15.4) with the exception specification of the implicit declaration, then

(3.1) — if the function is explicitly defaulted on its first declaration, it is defined as deleted;

(3.2) — otherwise, the program is ill-formed.

4 [*Example:*

```
struct S {
  constexpr S() = default;                  // ill-formed: implicit S() is not constexpr
  S(int a = 0) = default;                   // ill-formed: default argument
  void operator=(const S&) = default;       // ill-formed: non-matching return type
  ~S() throw(int) = default;                // deleted: exception specification does not match
private:
  int i;
  S(S&);                                    // OK: private copy constructor
};
S::S(S&) = default;                         // OK: defines copy constructor
```

104) Implementations are permitted to provide additional predefined variables with names that are reserved to the implementation (2.10). If a predefined variable is not odr-used (3.2), its string value need not be present in the program image.

— *end example*]

5 Explicitly-defaulted functions and implicitly-declared functions are collectively called *defaulted* functions, and the implementation shall provide implicit definitions for them (12.1 12.4, 12.8), which might mean defining them as deleted. A function is *user-provided* if it is user-declared and not explicitly defaulted or deleted on its first declaration. A user-provided explicitly-defaulted function (i.e., explicitly defaulted after its first declaration) is defined at the point where it is explicitly defaulted; if such a function is implicitly defined as deleted, the program is ill-formed. [*Note:* Declaring a function as defaulted after its first declaration can provide efficient execution and concise definition while enabling a stable binary interface to an evolving code base. — *end note*]

6 [*Example:*

```
struct trivial {
  trivial() = default;
  trivial(const trivial&) = default;
  trivial(trivial&&) = default;
  trivial& operator=(const trivial&) = default;
  trivial& operator=(trivial&&) = default;
  ~trivial() = default;
};

struct nontrivial1 {
  nontrivial1();
};
nontrivial1::nontrivial1() = default;            // not first declaration
```

— *end example*]

8.4.3 Deleted definitions [dcl.fct.def.delete]

1 A function definition of the form:

attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *declarator virt-specifier-seq*$_{opt}$ `= delete ;`

is called a *deleted definition*. A function with a deleted definition is also called a *deleted function*.

2 A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed. [*Note:* This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. If a function is overloaded, it is referenced only if the function is selected by overload resolution. — *end note*]

3 [*Example:* One can enforce non-default initialization and non-integral initialization with

```
struct onlydouble {
  onlydouble() = delete;                 // OK, but redundant
  onlydouble(std::intmax_t) = delete;
  onlydouble(double);
};
```

— *end example*]

[*Example:* One can prevent use of a class in certain *new-expression*s by using deleted definitions of a user-declared `operator new` for that class.

```
struct sometype {
  void* operator new(std::size_t) = delete;
  void* operator new[](std::size_t) = delete;
};
sometype* p = new sometype;      // error, deleted class operator new
```

```
sometype* q = new sometype[3];  // error, deleted class operator new[]
```

— *end example*]

[*Example:* One can make a class uncopyable, i.e. move-only, by using deleted definitions of the copy constructor and copy assignment operator, and then providing defaulted definitions of the move constructor and move assignment operator.

```
struct moveonly {
  moveonly() = default;
  moveonly(const moveonly&) = delete;
  moveonly(moveonly&&) = default;
  moveonly& operator=(const moveonly&) = delete;
  moveonly& operator=(moveonly&&) = default;
  ~moveonly() = default;
};
moveonly* p;
moveonly q(*p); // error, deleted copy constructor
```

— *end example*]

4 A deleted function is implicitly an inline function (7.1.2). [*Note:* The one-definition rule (3.2) applies to deleted definitions. — *end note*] A deleted definition of a function shall be the first declaration of the function or, for an explicit specialization of a function template, the first declaration of that specialization. [*Example:*

```
struct sometype {
  sometype();
};
sometype::sometype() = delete;      // ill-formed; not first declaration
```

— *end example*]

8.5 Initializers [dcl.init]

1 A declarator can specify an initial value for the identifier being declared. The identifier designates a variable being initialized. The process of initialization described in the remainder of 8.5 applies also to initializations specified by other syntactic contexts, such as the initialization of function parameters with argument expressions (5.2.2) or the initialization of return values (6.6.3).

initializer:
    brace-or-equal-initializer
    `(` *expression-list* `)`

brace-or-equal-initializer:
    `=` *initializer-clause*
    braced-init-list

initializer-clause:
    assignment-expression
    braced-init-list

initializer-list:
    initializer-clause `...`$_{opt}$
    initializer-list `,` *initializer-clause* `...`$_{opt}$

braced-init-list:
    `{` *initializer-list* `,`$_{opt}$ `}`
    `{ }`

2 Except for objects declared with the `constexpr` specifier, for which see 7.1.5, an *initializer* in the definition of a variable can consist of arbitrary expressions involving literals and previously declared variables and functions, regardless of the variable's storage duration. [*Example:*

```
int f(int);
int a = 2;
int b = f(a);
int c(b);
```

— *end example*]

3 [*Note:* Default arguments are more restricted; see 8.3.6.

4 The order of initialization of variables with static storage duration is described in 3.6 and 6.7. — *end note*]

5 A declaration of a block-scope variable with external or internal linkage that has an *initializer* is ill-formed.

6 To *zero-initialize* an object or reference of type `T` means:

(6.1) — if `T` is a scalar type (3.9), the object is initialized to the value obtained by converting the integer literal `0` (zero) to `T`;[105]

(6.2) — if `T` is a (possibly cv-qualified) non-union class type, each non-static data member and each base-class subobject is zero-initialized and padding is initialized to zero bits;

(6.3) — if `T` is a (possibly cv-qualified) union type, the object's first non-static named data member is zero-initialized and padding is initialized to zero bits;

(6.4) — if `T` is an array type, each element is zero-initialized;

(6.5) — if `T` is a reference type, no initialization is performed.

7 To *default-initialize* an object of type `T` means:

(7.1) — If `T` is a (possibly cv-qualified) class type (Clause 9), constructors are considered. The applicable constructors are enumerated (13.3.1.3), and the best one for the *initializer* `()` is chosen through overload resolution (13.3). The constructor thus selected is called, with an empty argument list, to initialize the object.

(7.2) — If `T` is an array type, each element is default-initialized.

(7.3) — Otherwise, no initialization is performed.

If a program calls for the default initialization of an object of a const-qualified type `T`, `T` shall be a class type with a user-provided default constructor.

8 To *value-initialize* an object of type `T` means:

(8.1) — if `T` is a (possibly cv-qualified) class type (Clause 9) with either no default constructor (12.1) or a default constructor that is user-provided or deleted, then the object is default-initialized;

(8.2) — if `T` is a (possibly cv-qualified) class type without a user-provided or deleted default constructor, then the object is zero-initialized and the semantic constraints for default-initialization are checked, and if `T` has a non-trivial default constructor, the object is default-initialized;

(8.3) — if `T` is an array type, then each element is value-initialized;

(8.4) — otherwise, the object is zero-initialized.

An object that is value-initialized is deemed to be constructed and thus subject to provisions of this International Standard applying to "constructed" objects, objects "for which the constructor has completed," etc., even if no constructor is invoked for the object's initialization.

9 A program that calls for default-initialization or value-initialization of an entity of reference type is ill-formed.

105) As specified in 4.10, converting an integer literal whose value is `0` to a pointer type results in a null pointer value.

10 [*Note:* Every object of static storage duration is zero-initialized at program startup before any other initialization takes place. In some cases, additional initialization is done later. — *end note*]

11 An object whose initializer is an empty set of parentheses, i.e., `()`, shall be value-initialized.

[*Note:* Since `()` is not permitted by the syntax for *initializer*,

```
X a();
```

is not the declaration of an object of class `X`, but the declaration of a function taking no argument and returning an `X`. The form `()` is permitted in certain other initialization contexts (5.3.4, 5.2.3, 12.6.2). — *end note*]

12 If no initializer is specified for an object, the object is default-initialized. When storage for an object with automatic or dynamic storage duration is obtained, the object has an *indeterminate value*, and if no initialization is performed for the object, that object retains an indeterminate value until that value is replaced (5.18). [*Note:* Objects with static or thread storage duration are zero-initialized, see 3.6.2. — *end note*] If an indeterminate value is produced by an evaluation, the behavior is undefined except in the following cases:

(12.1) — If an indeterminate value of unsigned narrow character type (3.9.1) is produced by the evaluation of:

(12.1.1) — the second or third operand of a conditional expression (5.16),

(12.1.2) — the right operand of a comma expression (5.19),

(12.1.3) — the operand of a cast or conversion to an unsigned narrow character type (4.7, 5.2.3, 5.2.9, 5.4), or

(12.1.4) — a discarded-value expression (Clause 5),

then the result of the operation is an indeterminate value.

(12.2) — If an indeterminate value of unsigned narrow character type is produced by the evaluation of the right operand of a simple assignment operator (5.18) whose first operand is an lvalue of unsigned narrow character type, an indeterminate value replaces the value of the object referred to by the left operand.

(12.3) — If an indeterminate value of unsigned narrow character type is produced by the evaluation of the initialization expression when initializing an object of unsigned narrow character type, that object is initialized to an indeterminate value.

[*Example:*

```
int f(bool b) {
  unsigned char c;
  unsigned char d = c; // OK, d has an indeterminate value
  int e = d;           // undefined behavior
  return b ? d : 0;    // undefined behavior if b is true
}
```

— *end example*]

13 An initializer for a static member is in the scope of the member's class. [*Example:*

```
int a;

struct X {
  static int a;
  static int b;
};
```

```
int X::a = 1;
int X::b = a;       // X::b = X::a
```

— *end example*]

14 If the entity being initialized does not have class type, the *expression-list* in a parenthesized initializer shall be a single expression.

15 The initialization that occurs in the `=` form of a *brace-or-equal-initializer* or *condition* (6.4), as well as in argument passing, function return, throwing an exception (15.1), handling an exception (15.3), and aggregate member initialization (8.5.1), is called *copy-initialization*. [*Note:* Copy-initialization may invoke a move (12.8). — *end note*]

16 The initialization that occurs in the forms

```
T x(a);
T x{a};
```

as well as in `new` expressions (5.3.4), `static_cast` expressions (5.2.9), functional notation type conversions (5.2.3), *mem-initializer*s (12.6.2), and the *braced-init-list* form of a *condition* is called *direct-initialization*.

17 The semantics of initializers are as follows. The *destination type* is the type of the object or reference being initialized and the *source type* is the type of the initializer expression. If the initializer is not a single (possibly parenthesized) expression, the source type is not defined.

(17.1) — If the initializer is a (non-parenthesized) *braced-init-list*, the object or reference is list-initialized (8.5.4).

(17.2) — If the destination type is a reference type, see 8.5.3.

(17.3) — If the destination type is an array of characters, an array of `char16_t`, an array of `char32_t`, or an array of `wchar_t`, and the initializer is a string literal, see 8.5.2.

(17.4) — If the initializer is `()`, the object is value-initialized.

(17.5) — Otherwise, if the destination type is an array, the program is ill-formed.

(17.6) — If the destination type is a (possibly cv-qualified) class type:

(17.6.1) — If the initialization is direct-initialization, or if it is copy-initialization where the cv-unqualified version of the source type is the same class as, or a derived class of, the class of the destination, constructors are considered. The applicable constructors are enumerated (13.3.1.3), and the best one is chosen through overload resolution (13.3). The constructor so selected is called to initialize the object, with the initializer expression or *expression-list* as its argument(s). If no constructor applies, or the overload resolution is ambiguous, the initialization is ill-formed.

(17.6.2) — Otherwise (i.e., for the remaining copy-initialization cases), user-defined conversion sequences that can convert from the source type to the destination type or (when a conversion function is used) to a derived class thereof are enumerated as described in 13.3.1.4, and the best one is chosen through overload resolution (13.3). If the conversion cannot be done or is ambiguous, the initialization is ill-formed. The function selected is called with the initializer expression as its argument; if the function is a constructor, the call initializes a temporary of the cv-unqualified version of the destination type. The temporary is a prvalue. The result of the call (which is the temporary for the constructor case) is then used to direct-initialize, according to the rules above, the object that is the destination of the copy-initialization. In certain cases, an implementation is permitted to eliminate the copying inherent in this direct-initialization by constructing the intermediate result directly into the object being initialized; see 12.2, 12.8.

(17.7) — Otherwise, if the source type is a (possibly cv-qualified) class type, conversion functions are considered. The applicable conversion functions are enumerated (13.3.1.5), and the best one is chosen through overload resolution (13.3). The user-defined conversion so selected is called to convert the initializer expression into the object being initialized. If the conversion cannot be done or is ambiguous, the initialization is ill-formed.

(17.8) — Otherwise, the initial value of the object being initialized is the (possibly converted) value of the initializer expression. Standard conversions (Clause 4) will be used, if necessary, to convert the initializer expression to the cv-unqualified version of the destination type; no user-defined conversions are considered. If the conversion cannot be done, the initialization is ill-formed. When initializing a bit-field with a value that it cannot represent, the resulting value of the bit-field is implementation-defined. [*Note:* An expression of type "*cv1* `T`" can initialize an object of type "*cv2* `T`" independently of the cv-qualifiers *cv1* and *cv2*.

```
int a;
const int b = a;
int c = b;
```

— *end note*]

18 An *initializer-clause* followed by an ellipsis is a pack expansion (14.5.3).

8.5.1 Aggregates [dcl.init.aggr]

1 An *aggregate* is an array or a class (Clause 9) with no user-provided constructors (12.1), no private or protected non-static data members (Clause 11), no base classes (Clause 10), and no virtual functions (10.3).

2 When an aggregate is initialized by an initializer list, as specified in 8.5.4, the elements of the initializer list are taken as initializers for the members of the aggregate, in increasing subscript or member order. Each member is copy-initialized from the corresponding *initializer-clause*. If the *initializer-clause* is an expression and a narrowing conversion (8.5.4) is required to convert the expression, the program is ill-formed. [*Note:* If an *initializer-clause* is itself an initializer list, the member is list-initialized, which will result in a recursive application of the rules in this section if the member is an aggregate. — *end note*] [*Example:*

```
struct A {
  int x;
  struct B {
    int i;
    int j;
  } b;
} a = { 1, { 2, 3 } };
```

initializes `a.x` with 1, `a.b.i` with 2, `a.b.j` with 3. — *end example*]

3 An aggregate that is a class can also be initialized with a single expression not enclosed in braces, as described in 8.5.

4 An array of unknown size initialized with a brace-enclosed *initializer-list* containing `n` *initializer-clause*s, where `n` shall be greater than zero, is defined as having `n` elements (8.3.4). [*Example:*

```
int x[] = { 1, 3, 5 };
```

declares and initializes `x` as a one-dimensional array that has three elements since no size was specified and there are three initializers. — *end example*] An empty initializer list `{}` shall not be used as the *initializer-clause* for an array of unknown bound.[106]

106) The syntax provides for empty *initializer-list*s, but nonetheless C++ does not have zero length arrays.

5 Static data members and anonymous bit-fields are not considered members of the class for purposes of aggregate initialization. [*Example:*

```
struct A {
  int i;
  static int s;
  int j;
  int :17;
  int k;
} a = { 1, 2, 3 };
```

Here, the second initializer 2 initializes `a.j` and not the static data member `A::s`, and the third initializer 3 initializes `a.k` and not the anonymous bit-field before it. — *end example*]

6 An *initializer-list* is ill-formed if the number of *initializer-clause*s exceeds the number of members or elements to initialize. [*Example:*

```
char cv[4] = { 'a', 's', 'd', 'f', 0 };      // error
```

is ill-formed. — *end example*]

7 If there are fewer *initializer-clause*s in the list than there are members in the aggregate, then each member not explicitly initialized shall be initialized from its *brace-or-equal-initializer* or, if there is no *brace-or-equal-initializer*, from an empty initializer list (8.5.4). [*Example:*

```
struct S { int a; const char* b; int c; int d = b[a]; };
S ss = { 1, "asdf" };
```

initializes `ss.a` with 1, `ss.b` with `"asdf"`, `ss.c` with the value of an expression of the form `int{}` (that is, `0`), and `ss.d` with the value of `ss.b[ss.a]` (that is, `'s'`), and in

```
struct X { int i, j, k = 42; };
X a[] = { 1, 2, 3, 4, 5, 6 };
X b[2] = { { 1, 2, 3 }, { 4, 5, 6 } };
```

`a` and `b` have the same value — *end example*]

8 If a reference member is initialized from its *brace-or-equal-initializer* and a potentially-evaluated subexpression thereof is an aggregate initialization that would use that *brace-or-equal-initializer*, the program is ill-formed. [*Example:*

```
struct A;
extern A a;
struct A {
  const A& a1 { A{a,a} };    // OK
  const A& a2 { A{} };       // error
};
A a{a,a};                    // OK
```

— *end example*]

9 If an aggregate class `C` contains a subaggregate member `m` that has no members for purposes of aggregate initialization, the *initializer-clause* for `m` shall not be omitted from an *initializer-list* for an object of type `C` unless the *initializer-clause*s for all members of `C` following `m` are also omitted. [*Example:*

```
struct S { } s;
struct A {
  S s1;
  int i1;
```

```
  S s2;
  int i2;
  S s3;
  int i3;
} a = {
  { },      // Required initialization
  0,
  s,        // Required initialization
  0
};          // Initialization not required for A::s3 because A::i3 is also not initialized
```

— *end example*]

10 If an incomplete or empty *initializer-list* leaves a member of reference type uninitialized, the program is ill-formed.

11 When initializing a multi-dimensional array, the *initializer-clause*s initialize the elements with the last (rightmost) index of the array varying the fastest (8.3.4). [*Example:*

```
int x[2][2] = { 3, 1, 4, 2 };
```

initializes `x[0][0]` to `3`, `x[0][1]` to `1`, `x[1][0]` to `4`, and `x[1][1]` to `2`. On the other hand,

```
float y[4][3] = {
  { 1 }, { 2 }, { 3 }, { 4 }
};
```

initializes the first column of `y` (regarded as a two-dimensional array) and leaves the rest zero. — *end example*]

12 Braces can be elided in an *initializer-list* as follows. If the *initializer-list* begins with a left brace, then the succeeding comma-separated list of *initializer-clause*s initializes the members of a subaggregate; it is erroneous for there to be more *initializer-clause*s than members. If, however, the *initializer-list* for a subaggregate does not begin with a left brace, then only enough *initializer-clause*s from the list are taken to initialize the members of the subaggregate; any remaining *initializer-clause*s are left to initialize the next member of the aggregate of which the current subaggregate is a member. [*Example:*

```
float y[4][3] = {
  { 1, 3, 5 },
  { 2, 4, 6 },
  { 3, 5, 7 },
};
```

is a completely-braced initialization: 1, 3, and 5 initialize the first row of the array `y[0]`, namely `y[0][0]`, `y[0][1]`, and `y[0][2]`. Likewise the next two lines initialize `y[1]` and `y[2]`. The initializer ends early and therefore `y[3]`s elements are initialized as if explicitly initialized with an expression of the form `float()`, that is, are initialized with `0.0`. In the following example, braces in the *initializer-list* are elided; however the *initializer-list* has the same effect as the completely-braced *initializer-list* of the above example,

```
float y[4][3] = {
  1, 3, 5, 2, 4, 6, 3, 5, 7
};
```

The initializer for `y` begins with a left brace, but the one for `y[0]` does not, therefore three elements from the list are used. Likewise the next three are taken successively for `y[1]` and `y[2]`. — *end example*]

13 All implicit type conversions (Clause 4) are considered when initializing the aggregate member with an *assignment-expression*. If the *assignment-expression* can initialize a member, the member is initialized.

Otherwise, if the member is itself a subaggregate, brace elision is assumed and the *assignment-expression* is considered for the initialization of the first member of the subaggregate. [*Note:* As specified above, brace elision cannot apply to subaggregates with no members for purposes of aggregate initialization; an *initializer-clause* for the entire subobject is required. — *end note*]

[*Example:*

```
struct A {
  int i;
  operator int();
};
struct B {
  A a1, a2;
  int z;
};
A a;
B b = { 4, a, a };
```

Braces are elided around the *initializer-clause* for `b.a1.i`. `b.a1.i` is initialized with 4, `b.a2` is initialized with `a`, `b.z` is initialized with whatever `a.operator int()` returns. — *end example*]

14 [*Note:* An aggregate array or an aggregate class may contain members of a class type with a user-provided constructor (12.1). Initialization of these aggregate objects is described in 12.6.1. — *end note*]

15 [*Note:* Whether the initialization of aggregates with static storage duration is static or dynamic is specified in 3.6.2 and 6.7. — *end note*]

16 When a union is initialized with a brace-enclosed initializer, the braces shall only contain an *initializer-clause* for the first non-static data member of the union. [*Example:*

```
union u { int a; const char* b; };
u a = { 1 };
u b = a;
u c = 1;                        // error
u d = { 0, "asdf" };            // error
u e = { "asdf" };               // error
```

— *end example*]

17 [*Note:* As described above, the braces around the *initializer-clause* for a union member can be omitted if the union is a member of another aggregate. — *end note*]

8.5.2 Character arrays [dcl.init.string]

1 An array of narrow character type (3.9.1), `char16_t` array, `char32_t` array, or `wchar_t` array can be initialized by a narrow string literal, `char16_t` string literal, `char32_t` string literal, or wide string literal, respectively, or by an appropriately-typed string literal enclosed in braces (2.13.5). Successive characters of the value of the string literal initialize the elements of the array. [*Example:*

```
char msg[] = "Syntax error on line %s\n";
```

shows a character array whose members are initialized with a *string-literal*. Note that because `'\n'` is a single character and because a trailing `'\0'` is appended, `sizeof(msg)` is `25`. — *end example*]

2 There shall not be more initializers than there are array elements. [*Example:*

```
char cv[4] = "asdf";            // error
```

is ill-formed since there is no space for the implied trailing `'\0'`. — *end example*]

3 If there are fewer initializers than there are array elements, each element not explicitly initialized shall be zero-initialized (8.5).

8.5.3 References [dcl.init.ref]

1 A variable declared to be a `T&` or `T&&`, that is, "reference to type `T`" (8.3.2), shall be initialized by an object, or function, of type `T` or by an object that can be converted into a `T`. [*Example:*

```
int g(int);
void f() {
  int i;
  int& r = i;                    // r refers to i
  r = 1;                         // the value of i becomes 1
  int* p = &r;                   // p points to i
  int& rr = r;                   // rr refers to what r refers to, that is, to i
  int (&rg)(int) = g;            // rg refers to the function g
  rg(i);                         // calls function g
  int a[3];
  int (&ra)[3] = a;              // ra refers to the array a
  ra[1] = i;                     // modifies a[1]
}
```

— *end example*]

2 A reference cannot be changed to refer to another object after initialization. Note that initialization of a reference is treated very differently from assignment to it. Argument passing (5.2.2) and function value return (6.6.3) are initializations.

3 The initializer can be omitted for a reference only in a parameter declaration (8.3.5), in the declaration of a function return type, in the declaration of a class member within its class definition (9.2), and where the `extern` specifier is explicitly used. [*Example:*

```
int& r1;                         // error: initializer missing
extern int& r2;                  // OK
```

— *end example*]

4 Given types "*cv1* `T1`" and "*cv2* `T2`", "*cv1* `T1`" is *reference-related* to "*cv2* `T2`" if `T1` is the same type as `T2`, or `T1` is a base class of `T2`. "*cv1* `T1`" is *reference-compatible* with "*cv2* `T2`" if `T1` is reference-related to `T2` and *cv1* is the same cv-qualification as, or greater cv-qualification than, *cv2*. In all cases where the reference-related or reference-compatible relationship of two types is used to establish the validity of a reference binding, and `T1` is a base class of `T2`, a program that necessitates such a binding is ill-formed if `T1` is an inaccessible (Clause 11) or ambiguous (10.2) base class of `T2`.

5 A reference to type "*cv1* `T1`" is initialized by an expression of type "*cv2* `T2`" as follows:

(5.1) — If the reference is an lvalue reference and the initializer expression

(5.1.1) — is an lvalue (but is not a bit-field), and "*cv1* `T1`" is reference-compatible with "*cv2* `T2`", or

(5.1.2) — has a class type (i.e., `T2` is a class type), where `T1` is not reference-related to `T2`, and can be converted to an lvalue of type "*cv3* `T3`", where "*cv1* `T1`" is reference-compatible with "*cv3* `T3`"[107] (this conversion is selected by enumerating the applicable conversion functions (13.3.1.6) and choosing the best one through overload resolution (13.3)),

then the reference is bound to the initializer expression lvalue in the first case and to the lvalue result of the conversion in the second case (or, in either case, to the appropriate base class subobject of the object). [*Note:* The usual lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer

107) This requires a conversion function (12.3.2) returning a reference type.

(4.3) standard conversions are not needed, and therefore are suppressed, when such direct bindings to lvalues are done. — *end note*]

[*Example:*

```
double d = 2.0;
double& rd = d;                 // rd refers to d
const double& rcd = d;          // rcd refers to d

struct A { };
struct B : A { operator int&(); } b;
A& ra = b;                      // ra refers to A subobject in b
const A& rca = b;               // rca refers to A subobject in b
int& ir = B();                  // ir refers to the result of B::operator int&
```

— *end example*]

(5.2) — Otherwise, the reference shall be an lvalue reference to a non-volatile const type (i.e., *cv1* shall be `const`), or the reference shall be an rvalue reference. [*Example:*

```
double& rd2 = 2.0;              // error: not an lvalue and reference not const
int  i = 2;
double& rd3 = i;                // error: type mismatch and reference not const
```

— *end example*]

(5.2.1) — If the initializer expression

(5.2.1.1) — is an xvalue (but not a bit-field), class prvalue, array prvalue or function lvalue and "*cv1* `T1`" is reference-compatible with "*cv2* `T2`", or

(5.2.1.2) — has a class type (i.e., `T2` is a class type), where `T1` is not reference-related to `T2`, and can be converted to an xvalue, class prvalue, or function lvalue of type "*cv3* `T3`", where "*cv1* `T1`" is reference-compatible with "*cv3* `T3`" (see 13.3.1.6),

then the reference is bound to the value of the initializer expression in the first case and to the result of the conversion in the second case (or, in either case, to an appropriate base class subobject).

[*Example:*

```
struct A { };
struct B : A { } b;
extern B f();
const A& rca2 = f();                // bound to the A subobject of the B rvalue.
A&& rra = f();                      // same as above
struct X {
  operator B();
  operator int&();
} x;
const A& r = x;                     // bound to the A subobject of the result of the conversion
int i2 = 42;
int&& rri = static_cast<int&&>(i2); // bound directly to i2
B&& rrb = x;                        // bound directly to the result of operator B
```

— *end example*]

(5.2.2) — Otherwise:

(5.2.2.1) — If `T1` or `T2` is a class type and `T1` is not reference-related to `T2`, user-defined conversions are considered using the rules for copy-initialization of an object of type "*cv1* `T1`" by user-defined conversion (8.5, 13.3.1.4, 13.3.1.5); the program is ill-formed if the corresponding

non-reference copy-initialization would be ill-formed. The result of the call to the conversion function, as described for the non-reference copy-initialization, is then used to direct-initialize the reference. For this direct-initialization, user-defined conversions are not considered.

(5.2.2.2) — Otherwise, a temporary of type "*cv1* `T1`" is created and copy-initialized (8.5) from the initializer expression. The reference is then bound to the temporary.

If `T1` is reference-related to `T2`:

(5.2.2.3) — *cv1* shall be the same cv-qualification as, or greater cv-qualification than, *cv2*; and

(5.2.2.4) — if the reference is an rvalue reference, the initializer expression shall not be an lvalue.

[*Example:*

```
struct Banana { };
struct Enigma { operator const Banana(); };
struct Alaska { operator Banana&(); };
void enigmatic() {
  typedef const Banana ConstBanana;
  Banana &&banana1 = ConstBanana(); // ill-formed
  Banana &&banana2 = Enigma();      // ill-formed
  Banana &&banana3 = Alaska();      // ill-formed
}

const double& rcd2 = 2;          // rcd2 refers to temporary with value 2.0
double&& rrd = 2;                // rrd refers to temporary with value 2.0
const volatile int cvi = 1;
const int& r2 = cvi;             // error: type qualifiers dropped
struct A { operator volatile int&(); } a;
const int& r3 = a;               // error: type qualifiers dropped
                                 // from result of conversion function
double d2 = 1.0;
double&& rrd2 = d2;              // error: initializer is lvalue of related type
struct X { operator int&(); };
int&& rri2 = X();                // error: result of conversion function is lvalue of related type
int i3 = 2;
double&& rrd3 = i3;              // rrd3 refers to temporary with value 2.0
```

— *end example*]

In all cases except the last (i.e., creating and initializing a temporary from the initializer expression), the reference is said to *bind directly* to the initializer expression.

6 [*Note:* 12.2 describes the lifetime of temporaries bound to references. — *end note*]

8.5.4 List-initialization [dcl.init.list]

1 *List-initialization* is initialization of an object or reference from a *braced-init-list*. Such an initializer is called an *initializer list*, and the comma-separated *initializer-clause*s of the list are called the *elements* of the initializer list. An initializer list may be empty. List-initialization can occur in direct-initialization or copy-initialization contexts; list-initialization in a direct-initialization context is called *direct-list-initialization* and list-initialization in a copy-initialization context is called *copy-list-initialization*. [*Note:* List-initialization can be used

(1.1) — as the initializer in a variable definition (8.5)

(1.2) — as the initializer in a *new-expression* (5.3.4)

(1.3) — in a return statement (6.6.3)

(1.4) — as a *for-range-initializer* (6.5)

(1.5) — as a function argument (5.2.2)

(1.6) — as a subscript (5.2.1)

(1.7) — as an argument to a constructor invocation (8.5, 5.2.3)

(1.8) — as an initializer for a non-static data member (9.2)

(1.9) — in a *mem-initializer* (12.6.2)

(1.10) — on the right-hand side of an assignment (5.18)

[*Example:*

```
int a = {1};
std::complex<double> z{1,2};
new std::vector<std::string>{"once", "upon", "a", "time"};  // 4 string elements
f( {"Nicholas","Annemarie"} );  // pass list of two elements
return { "Norah" };             // return list of one element
int* e {};                      // initialization to zero / null pointer
x = double{1};                  // explicitly construct a double
std::map<std::string,int> anim = { {"bear",4}, {"cassowary",2}, {"tiger",7} };
```

— *end example*] — *end note*]

2 A constructor is an *initializer-list constructor* if its first parameter is of type `std::initializer_list<E>` or reference to possibly cv-qualified `std::initializer_list<E>` for some type `E`, and either there are no other parameters or else all other parameters have default arguments (8.3.6). [*Note:* Initializer-list constructors are favored over other constructors in list-initialization (13.3.1.7). Passing an initializer list as the argument to the constructor template `template<class T> C(T)` of a class `C` does not create an initializer-list constructor, because an initializer list argument causes the corresponding parameter to be a non-deduced context (14.8.2.1). — *end note*] The template `std::initializer_list` is not predefined; if the header `<initializer_list>` is not included prior to a use of `std::initializer_list` — even an implicit use in which the type is not named (7.1.6.4) — the program is ill-formed.

3 List-initialization of an object or reference of type `T` is defined as follows:

(3.1) — If `T` is a class type and the initializer list has a single element of type *cv* `U`, where `U` is `T` or a class derived from `T`, the object is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization).

(3.2) — Otherwise, if `T` is a character array and the initializer list has a single element that is an appropriately-typed string literal (8.5.2), initialization is performed as described in that section.

(3.3) — Otherwise, if `T` is an aggregate, aggregate initialization is performed (8.5.1).

[*Example:*

```
double ad[] = { 1, 2.0 };          // OK
int ai[] = { 1, 2.0 };             // error: narrowing

struct S2 {
  int m1;
  double m2, m3;
};
S2 s21 = { 1, 2, 3.0 };            // OK
S2 s22 { 1.0, 2, 3 };              // error: narrowing
S2 s23 { };                        // OK: default to 0,0,0
```

— *end example*]

(3.4) — Otherwise, if the initializer list has no elements and `T` is a class type with a default constructor, the object is value-initialized.

(3.5) — Otherwise, if `T` is a specialization of `std::initializer_list<E>`, a prvalue `initializer_list` object is constructed as described below and used to initialize the object according to the rules for initialization of an object from a class of the same type (8.5).

(3.6) — Otherwise, if `T` is a class type, constructors are considered. The applicable constructors are enumerated and the best one is chosen through overload resolution (13.3, 13.3.1.7). If a narrowing conversion (see below) is required to convert any of the arguments, the program is ill-formed.

[*Example:*

```
struct S {
  S(std::initializer_list<double>); // #1
  S(std::initializer_list<int>);    // #2
  S();                              // #3
  // ...
};
S s1 = { 1.0, 2.0, 3.0 };           // invoke #1
S s2 = { 1, 2, 3 };                 // invoke #2
S s3 = { };                         // invoke #3
```

— *end example*]

[*Example:*

```
struct Map {
  Map(std::initializer_list<std::pair<std::string,int>>);
};
Map ship = {{"Sophie",14}, {"Surprise",28}};
```

— *end example*]

[*Example:*

```
struct S {
  // no initializer-list constructors
  S(int, double, double);           // #1
  S();                              // #2
  // ...
};
S s1 = { 1, 2, 3.0 };               // OK: invoke #1
S s2 { 1.0, 2, 3 };                 // error: narrowing
S s3 { };                           // OK: invoke #2
```

— *end example*]

(3.7) — Otherwise, if the initializer list has a single element of type `E` and either `T` is not a reference type or its referenced type is reference-related to `E`, the object or reference is initialized from that element (by copy-initialization for copy-list-initialization, or by direct-initialization for direct-list-initialization); if a narrowing conversion (see below) is required to convert the element to `T`, the program is ill-formed.

[*Example:*

```
int x1 {2};                         // OK
int x2 {2.0};                       // error: narrowing
```

— *end example*]

(3.8) — Otherwise, if `T` is a reference type, a prvalue temporary of the type referenced by `T` is copy-list-initialized or direct-list-initialized, depending on the kind of initialization for the reference, and the reference is bound to that temporary. [*Note:* As usual, the binding will fail and the program is ill-formed if the reference type is an lvalue reference to a non-const type. — *end note*]

[*Example:*

```
struct S {
  S(std::initializer_list<double>); // #1
  S(const std::string&);            // #2
  // ...
};
const S& r1 = { 1, 2, 3.0 };        // OK: invoke #1
const S& r2 { "Spinach" };          // OK: invoke #2
S& r3 = { 1, 2, 3 };                // error: initializer is not an lvalue
const int& i1 = { 1 };              // OK
const int& i2 = { 1.1 };            // error: narrowing
const int (&iar)[2] = { 1, 2 };     // OK: iar is bound to temporary array
```

— *end example*]

(3.9) — Otherwise, if the initializer list has no elements, the object is value-initialized.

[*Example:*

```
int** pp {};                        // initialized to null pointer
```

— *end example*]

(3.10) — Otherwise, the program is ill-formed.

[*Example:*

```
struct A { int i; int j; };
A a1 { 1, 2 };                      // aggregate initialization
A a2 { 1.2 };                       // error: narrowing
struct B {
  B(std::initializer_list<int>);
};
B b1 { 1, 2 };                      // creates initializer_list<int> and calls constructor
B b2 { 1, 2.0 };                    // error: narrowing
struct C {
  C(int i, double j);
};
C c1 = { 1, 2.2 };                  // calls constructor with arguments (1, 2.2)
C c2 = { 1.1, 2 };                  // error: narrowing

int j { 1 };                        // initialize to 1
int k { };                          // initialize to 0
```

— *end example*]

4 Within the *initializer-list* of a *braced-init-list*, the *initializer-clause*s, including any that result from pack expansions (14.5.3), are evaluated in the order in which they appear. That is, every value computation and side effect associated with a given *initializer-clause* is sequenced before every value computation and side effect associated with any *initializer-clause* that follows it in the comma-separated list of the *initializer-list*.

[*Note:* This evaluation ordering holds regardless of the semantics of the initialization; for example, it applies when the elements of the *initializer-list* are interpreted as arguments of a constructor call, even though ordinarily there are no sequencing constraints on the arguments of a call. — *end note*]

5 An object of type `std::initializer_list<E>` is constructed from an initializer list as if the implementation allocated a temporary array of N elements of type `const E`, where N is the number of elements in the initializer list. Each element of that array is copy-initialized with the corresponding element of the initializer list, and the `std::initializer_list<E>` object is constructed to refer to that array. [*Note:* A constructor or conversion function selected for the copy shall be accessible (Clause 11) in the context of the initializer list. — *end note*] If a narrowing conversion is required to initialize any of the elements, the program is ill-formed.[*Example:*

```
struct X {
  X(std::initializer_list<double> v);
};
X x{ 1,2,3 };
```

The initialization will be implemented in a way roughly equivalent to this:

```
const double __a[3] = {double{1}, double{2}, double{3}};
X x(std::initializer_list<double>(__a, __a+3));
```

assuming that the implementation can construct an `initializer_list` object with a pair of pointers. — *end example*]

6 The array has the same lifetime as any other temporary object (12.2), except that initializing an `initializer_list` object from the array extends the lifetime of the array exactly like binding a reference to a temporary. [*Example:*

```
typedef std::complex<double> cmplx;
std::vector<cmplx> v1 = { 1, 2, 3 };

void f() {
  std::vector<cmplx> v2{ 1, 2, 3 };
  std::initializer_list<int> i3 = { 1, 2, 3 };
}

struct A {
  std::initializer_list<int> i4;
  A() : i4{ 1, 2, 3 } {}  // creates an A with a dangling reference
};
```

For `v1` and `v2`, the `initializer_list` object is a parameter in a function call, so the array created for `{ 1, 2, 3 }` has full-expression lifetime. For `i3`, the `initializer_list` object is a variable, so the array persists for the lifetime of the variable. For `i4`, the `initializer_list` object is initialized in a constructor's *ctor-initializer*, so the array persists only until the constructor exits, and so any use of the elements of `i4` after the constructor exits produces undefined behavior. — *end example*] [*Note:* The implementation is free to allocate the array in read-only memory if an explicit array with the same initializer could be so allocated. — *end note*]

7 A *narrowing conversion* is an implicit conversion

(7.1) — from a floating-point type to an integer type, or

(7.2) — from `long double` to `double` or `float`, or from `double` to `float`, except where the source is a constant expression and the actual value after conversion is within the range of values that can be represented (even if it cannot be represented exactly), or

(7.3) — from an integer type or unscoped enumeration type to a floating-point type, except where the source is a constant expression and the actual value after conversion will fit into the target type and will produce the original value when converted back to the original type, or

(7.4) — from an integer type or unscoped enumeration type to an integer type that cannot represent all the values of the original type, except where the source is a constant expression whose value after integral promotions will fit into the target type.

[*Note:* As indicated above, such conversions are not allowed at the top level in list-initializations. — *end note*] [*Example:*

```
int x = 999;                // x is not a constant expression
const int y = 999;
const int z = 99;
char c1 = x;                // OK, though it might narrow (in this case, it does narrow)
char c2{x};                 // error: might narrow
char c3{y};                 // error: narrows (assuming char is 8 bits)
char c4{z};                 // OK: no narrowing needed
unsigned char uc1 = {5};    // OK: no narrowing needed
unsigned char uc2 = {-1}; // error: narrows
unsigned int ui1 = {-1};  // error: narrows
signed int si1 =
  { (unsigned int)-1 };     // error: narrows
int ii = {2.0};             // error: narrows
float f1 { x };             // error: might narrow
float f2 { 7 };             // OK: 7 can be exactly represented as a float
int f(int);
int a[] =
  { 2, f(2), f(2.0) };      // OK: the double-to-int conversion is not at the top level
```

— *end example*]

9 Classes [class]

1 A class is a type. Its name becomes a *class-name* (9.1) within its scope.

class-name:
identifier
simple-template-id

*Class-specifier*s and *elaborated-type-specifier*s (7.1.6.3) are used to make *class-name*s. An object of a class consists of a (possibly empty) sequence of members and base class objects.

class-specifier:
class-head `{` *member-specification*$_{opt}$`}`

class-head:
class-key attribute-specifier-seq$_{opt}$ *class-head-name class-virt-specifier*$_{opt}$ *base-clause*$_{opt}$
class-key attribute-specifier-seq$_{opt}$ *base-clause*$_{opt}$

class-head-name:
nested-name-specifier$_{opt}$ *class-name*

class-virt-specifier:
`final`

class-key:
`class`
`struct`
`union`

A *class-specifier* whose *class-head* omits the *class-head-name* defines an unnamed class. [*Note:* An unnamed class thus can't be `final`. — *end note*]

2 A *class-name* is inserted into the scope in which it is declared immediately after the *class-name* is seen. The *class-name* is also inserted into the scope of the class itself; this is known as the *injected-class-name*. For purposes of access checking, the injected-class-name is treated as if it were a public member name. A *class-specifier* is commonly referred to as a class definition. A class is considered defined after the closing brace of its *class-specifier* has been seen even though its member functions are in general not yet defined. The optional *attribute-specifier-seq* appertains to the class; the attributes in the *attribute-specifier-seq* are thereafter considered attributes of the class whenever it is named.

3 If a class is marked with the *class-virt-specifier* `final` and it appears as a *base-type-specifier* in a *base-clause* (Clause 10), the program is ill-formed. Whenever a *class-key* is followed by a *class-head-name*, the *identifier* `final`, and a colon or left brace, `final` is interpreted as a *class-virt-specifier*. [*Example:*

```
struct A;
struct A final {};      // OK: definition of struct A,
                        // not value-initialization of variable final

struct X {
 struct C { constexpr operator int() { return 5; } };
 struct B final : C{};  // OK: definition of nested class B,
                        // not declaration of a bit-field member final
};
```

— *end example*]

4 Complete objects and member subobjects of class type shall have nonzero size.[108] [*Note:* Class objects can

108) Base class subobjects are not so constrained.

be assigned, passed as arguments to functions, and returned by functions (except objects of classes for which copying or moving has been restricted; see 12.8). Other plausible operators, such as equality comparison, can be defined by the user; see 13.5. — *end note*]

5 A *union* is a class defined with the *class-key* `union`; it holds at most one data member at a time (9.5). [*Note:* Aggregates of class type are described in 8.5.1. — *end note*]

6 A *trivially copyable class* is a class that:

(6.1) — has no non-trivial copy constructors (12.8),

(6.2) — has no non-trivial move constructors (12.8),

(6.3) — has no non-trivial copy assignment operators (13.5.3, 12.8),

(6.4) — has no non-trivial move assignment operators (13.5.3, 12.8), and

(6.5) — has a trivial destructor (12.4).

A *trivial class* is a class that has a default constructor (12.1), has no non-trivial default constructors, and is trivially copyable. [*Note:* In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. — *end note*]

7 A class `S` is a *standard-layout class* if it:

(7.1) — has no non-static data members of type non-standard-layout class (or array of such types) or reference,

(7.2) — has no virtual functions (10.3) and no virtual base classes (10.1),

(7.3) — has the same access control (Clause 11) for all non-static data members,

(7.4) — has no non-standard-layout base classes,

(7.5) — has at most one base class subobject of any given type,

(7.6) — has all non-static data members and bit-fields in the class and its base classes first declared in the same class, and

(7.7) — has no element of the set $M(\texttt{S})$ of types (defined below) as a base class.[109]

$M(\texttt{X})$ is defined as follows:

(7.8) — If `X` is a non-union class type, the set $M(\texttt{X})$ is empty if `X` has no (possibly inherited (Clause 10)) non-static data members; otherwise, it consists of the type of the first non-static data member of `X` (where said member may be an anonymous union), $\texttt{X}_0$, and the elements of $M(\texttt{X}_0)$.

(7.9) — If `X` is a union type, the set $M(\texttt{X})$ is the union of all $M(\texttt{U}_i)$ and the set containing all $\texttt{U}_i$, where each $\texttt{U}_i$ is the type of the ith non-static data member of `X`.

(7.10) — If `X` is a non-class type, the set $M(\texttt{X})$ is empty.

[*Note:* $M(\texttt{X})$ is the set of the types of all non-base-class subobjects that are guaranteed in a standard-layout class to be at a zero offset in `X`. — *end note*]

[*Example:*

109) This ensures that two subobjects that have the same class type and that belong to the same most derived object are not allocated at the same address (5.10).

```
struct B { int i; };          // standard-layout class
struct C : B { };             // standard-layout class
struct D : C { };             // standard-layout class
struct E : D { char : 4; };   // not a standard-layout class

struct Q {};
struct S : Q { };
struct T : Q { };
struct U : S, T { };          // not a standard-layout class
```

— *end example*]

8 A *standard-layout struct* is a standard-layout class defined with the *class-key* `struct` or the *class-key* `class`. A *standard-layout union* is a standard-layout class defined with the *class-key* `union`.

9 [*Note:* Standard-layout classes are useful for communicating with code written in other programming languages. Their layout is specified in 9.2. — *end note*]

10 A *POD struct*[110] is a non-union class that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD struct, non-POD union (or array of such types). Similarly, a *POD union* is a union that is both a trivial class and a standard-layout class, and has no non-static data members of type non-POD struct, non-POD union (or array of such types). A *POD class* is a class that is either a POD struct or a POD union.

[*Example:*

```
struct N {          // neither trivial nor standard-layout
  int i;
  int j;
  virtual ~N();
};

struct T {          // trivial but not standard-layout
  int i;
private:
  int j;
};

struct SL {         // standard-layout but not trivial
  int i;
  int j;
  ~SL();
};

struct POD {        // both trivial and standard-layout
  int i;
  int j;
};
```

— *end example*]

11 If a *class-head-name* contains a *nested-name-specifier*, the *class-specifier* shall refer to a class that was previously declared directly in the class or namespace to which the *nested-name-specifier* refers, or in an element of the inline namespace set (7.3.1) of that namespace (i.e., not merely inherited or introduced by a *using-declaration*), and the *class-specifier* shall appear in a namespace enclosing the previous declaration.

110) The acronym POD stands for "plain old data".

In such cases, the *nested-name-specifier* of the *class-head-name* of the definition shall not begin with a *decltype-specifier*.

9.1 Class names [class.name]

1 A class definition introduces a new type. [*Example:*

```
struct X { int a; };
struct Y { int a; };
X a1;
Y a2;
int a3;
```

declares three variables of three different types. This implies that

```
a1 = a2;                    // error: Y assigned to X
a1 = a3;                    // error: int assigned to X
```

are type mismatches, and that

```
int f(X);
int f(Y);
```

declare an overloaded (Clause 13) function `f()` and not simply a single function `f()` twice. For the same reason,

```
struct S { int a; };
struct S { int a; };        // error, double definition
```

is ill-formed because it defines `S` twice. — *end example*]

2 A class declaration introduces the class name into the scope where it is declared and hides any class, variable, function, or other declaration of that name in an enclosing scope (3.3). If a class name is declared in a scope where a variable, function, or enumerator of the same name is also declared, then when both declarations are in scope, the class can be referred to only using an *elaborated-type-specifier* (3.4.4). [*Example:*

```
struct stat {
  // ...
};

stat gstat;                 // use plain stat to
                            // define variable

int stat(struct stat*);     // redeclare stat as function

void f() {
  struct stat* ps;          // struct prefix needed
                            // to name struct stat
  stat(ps);                 // call stat()
}
```

— *end example*] A *declaration* consisting solely of *class-key identifier;* is either a redeclaration of the name in the current scope or a forward declaration of the identifier as a class name. It introduces the class name into the current scope. [*Example:*

```
struct s { int a; };

void g() {
  struct s;                 // hide global struct s
```

```
                                // with a block-scope declaration
  s* p;                         // refer to local struct s
  struct s { char* p; };        // define local struct s
  struct s;                     // redeclaration, has no effect
}
```

— *end example*] [*Note:* Such declarations allow definition of classes that refer to each other. [*Example:*

```
class Vector;

class Matrix {
  // ...
  friend Vector operator*(const Matrix&, const Vector&);
};

class Vector {
  // ...
  friend Vector operator*(const Matrix&, const Vector&);
};
```

Declaration of `friend`s is described in 11.3, operator functions in 13.5. — *end example*] — *end note*]

3 [*Note:* An *elaborated-type-specifier* (7.1.6.3) can also be used as a *type-specifier* as part of a declaration. It differs from a class declaration in that if a class of the elaborated name is in scope the elaborated name will refer to it. — *end note*] [*Example:*

```
struct s { int a; };

void g(int s) {
  struct s* p = new struct s;   // global s
  p->a = s;                     // parameter s
}
```

— *end example*]

4 [*Note:* The declaration of a class name takes effect immediately after the *identifier* is seen in the class definition or *elaborated-type-specifier*. For example,

```
class A * A;
```

first specifies `A` to be the name of a class and then redefines it as the name of a pointer to an object of that class. This means that the elaborated form `class A` must be used to refer to the class. Such artistry with names can be confusing and is best avoided. — *end note*]

5 A *typedef-name* (7.1.3) that names a class type, or a cv-qualified version thereof, is also a *class-name*. If a *typedef-name* that names a cv-qualified class type is used where a *class-name* is required, the cv-qualifiers are ignored. A *typedef-name* shall not be used as the *identifier* in a *class-head*.

9.2 Class members [class.mem]

member-specification:
member-declaration *member-specification*$_{opt}$
access-specifier : *member-specification*$_{opt}$

member-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *member-declarator-list*$_{opt}$ `;`
function-definition
using-declaration
static_assert-declaration
template-declaration
alias-declaration
empty-declaration

member-declarator-list:
member-declarator
member-declarator-list `,` *member-declarator*

member-declarator:
declarator virt-specifier-seq$_{opt}$ *pure-specifier*$_{opt}$
declarator brace-or-equal-initializer$_{opt}$
identifier$_{opt}$ *attribute-specifier-seq*$_{opt}$ `:` *constant-expression*

virt-specifier-seq:
virt-specifier
virt-specifier-seq virt-specifier

virt-specifier:
`override`
`final`

pure-specifier:
`= 0`

1 The *member-specification* in a class definition declares the full set of members of the class; no member can be added elsewhere. Members of a class are data members, member functions (9.3), nested types, and enumerators. Data members and member functions are static or non-static; see 9.4. Nested types are classes (9.1, 9.7) and enumerations (7.2) defined in the class, and arbitrary types declared as members by use of a typedef declaration (7.1.3). The enumerators of an unscoped enumeration (7.2) defined in the class are members of the class. Except when used to declare friends (11.3), to declare an unnamed bit-field (9.6), or to introduce the name of a member of a base class into a derived class (7.3.3), or when the declaration is an *empty-declaration*, *member-declaration*s declare members of the class, and each such *member-declaration* shall declare at least one member name of the class. A member shall not be declared twice in the *member-specification*, except that a nested class or member class template can be declared and then later defined, and except that an enumeration can be introduced with an *opaque-enum-declaration* and later redeclared with an *enum-specifier*.

2 A class is considered a completely-defined object type (3.9) (or complete type) at the closing `}` of the *class-specifier*. Within the class *member-specification*, the class is regarded as complete within function bodies, default arguments, *using-declaration*s introducing inheriting constructors (12.9), *exception-specification*s, and *brace-or-equal-initializer*s for non-static data members (including such things in nested classes). Otherwise it is regarded as incomplete within its own class *member-specification*.

3 [*Note:* A single name can denote several function members provided their types are sufficiently different (Clause 13). — *end note*]

4 A *brace-or-equal-initializer* shall appear only in the declaration of a data member. (For static data members, see 9.4.2; for non-static data members, see 12.6.2). A *brace-or-equal-initializer* for a non-static data member shall not directly or indirectly cause the implicit definition of a defaulted default constructor for the enclosing class or the exception specification of that constructor.

5 A member shall not be declared with the `extern` or `register` *storage-class-specifier*. Within a class definition, a member shall not be declared with the `thread_local` *storage-class-specifier* unless also declared `static`.

6 The *decl-specifier-seq* may be omitted in constructor, destructor, and conversion function declarations only; when declaring another kind of member the *decl-specifier-seq* shall contain a *type-specifier* that is not a *cv-*

qualifier. The *member-declarator-list* can be omitted only after a *class-specifier* or an *enum-specifier* or in a `friend` declaration (11.3). A *pure-specifier* shall be used only in the declaration of a virtual function (10.3).

7 The optional *attribute-specifier-seq* in a *member-declaration* appertains to each of the entities declared by the *member-declarator*s; it shall not appear if the optional *member-declarator-list* is omitted.

8 A *virt-specifier-seq* shall contain at most one of each *virt-specifier*. A *virt-specifier-seq* shall appear only in the declaration of a virtual member function (10.3).

9 Non-`static` (9.4) data members shall not have incomplete types. In particular, a class `C` shall not contain a non-static member of class `C`, but it can contain a pointer or reference to an object of class `C`.

10 [*Note:* See 5.1 for restrictions on the use of non-static data members and non-static member functions. — *end note*]

11 [*Note:* The type of a non-static member function is an ordinary function type, and the type of a non-static data member is an ordinary object type. There are no special member function types or data member types. — *end note*]

12 [*Example:* A simple example of a class definition is

```
struct tnode {
  char tword[20];
  int count;
  tnode* left;
  tnode* right;
};
```

which contains an array of twenty characters, an integer, and two pointers to objects of the same type. Once this definition has been given, the declaration

```
tnode s, *sp;
```

declares `s` to be a `tnode` and `sp` to be a pointer to a `tnode`. With these declarations, `sp->count` refers to the `count` member of the object to which `sp` points; `s.left` refers to the `left` subtree pointer of the object `s`; and `s.right->tword[0]` refers to the initial character of the `tword` member of the `right` subtree of `s`. — *end example*]

13 Nonstatic data members of a (non-union) class with the same access control (Clause 11) are allocated so that later members have higher addresses within a class object. The order of allocation of non-static data members with different access control is unspecified (Clause 11). Implementation alignment requirements might cause two adjacent members not to be allocated immediately after each other; so might requirements for space for managing virtual functions (10.3) and virtual base classes (10.1).

14 If `T` is the name of a class, then each of the following shall have a name different from `T`:

(14.1) — every static data member of class `T`;

(14.2) — every member function of class `T` [*Note:* This restriction does not apply to constructors, which do not have names (12.1) — *end note*];

(14.3) — every member of class `T` that is itself a type;

(14.4) — every member template of class `T`;

(14.5) — every enumerator of every member of class `T` that is an unscoped enumerated type; and

(14.6) — every member of every anonymous union that is a member of class `T`.

15 In addition, if class `T` has a user-declared constructor (12.1), every non-static data member of class `T` shall have a name different from `T`.

16 The *common initial sequence* of two standard-layout struct (Clause 9) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that corresponding entities have layout-compatible types and either neither entity is a bit-field or both are bit-fields with the same width. [*Example:*

```
struct A { int a; char b; };
struct B { const int b1; volatile char b2; };
struct C { int c; unsigned : 0; char b; };
struct D { int d; char b : 4; };
struct E { unsigned int e; char b; };
```

The common initial sequence of `A` and `B` comprises all members of either class. The common initial sequence of `A` and `C` and of `A` and `D` comprises the first member in each case. The common initial sequence of `A` and `E` is empty. — *end example*]

17 Two standard-layout struct (Clause 9) types are layout-compatible if their common initial sequence comprises all members and bit-fields of both classes (3.9).

18 Two standard-layout unions are layout-compatible if they have the same number of non-static data members and corresponding non-static data members (in any order) have layout-compatible types (3.9).

19 In a standard-layout union with an active member (9.5) of struct type `T1`, it is permitted to read a non-static data member `m` of another union member of struct type `T2` provided `m` is part of the common initial sequence of `T1` and `T2`. [*Note:* Reading a volatile object through a non-volatile glvalue has undefined behavior (7.1.6.1). — *end note*]

20 If a standard-layout class object has any non-static data members, its address is the same as the address of its first non-static data member. Otherwise, its address is the same as the address of its first base class subobject (if any). [*Note:* There might therefore be unnamed padding within a standard-layout struct object, but not at its beginning, as necessary to achieve appropriate alignment. — *end note*]

9.3 Member functions [class.mfct]

1 Functions declared in the definition of a class, excluding those declared with a `friend` specifier (11.3), are called member functions of that class. A member function may be declared `static` in which case it is a *static* member function of its class (9.4); otherwise it is a *non-static* member function of its class (9.3.1, 9.3.2).

2 A member function may be defined (8.4) in its class definition, in which case it is an *inline* member function (7.1.2), or it may be defined outside of its class definition if it has already been declared but not defined in its class definition. A member function definition that appears outside of the class definition shall appear in a namespace scope enclosing the class definition. Except for member function definitions that appear outside of a class definition, and except for explicit specializations of member functions of class templates and member function templates (14.7) appearing outside of the class definition, a member function shall not be redeclared.

3 An `inline` member function (whether static or non-static) may also be defined outside of its class definition provided either its declaration in the class definition or its definition outside of the class definition declares the function as `inline`. [*Note:* Member functions of a class in namespace scope have the linkage of that class. Member functions of a local class (9.8) have no linkage. See 3.5. — *end note*]

4 There shall be at most one definition of a non-inline member function in a program; no diagnostic is required. There may be more than one `inline` member function definition in a program. See 3.2 and 7.1.2.

5 If the definition of a member function is lexically outside its class definition, the member function name shall be qualified by its class name using the `::` operator. [*Note:* A name used in a member function definition (that is, in the *parameter-declaration-clause* including the default arguments (8.3.6) or in the member function body) is looked up as described in 3.4. — *end note*] [*Example:*

```
struct X {
```

```
  typedef int T;
  static T count;
  void f(T);
};
void X::f(T t = count) { }
```

The member function `f` of class `X` is defined in global scope; the notation `X::f` specifies that the function `f` is a member of class `X` and in the scope of class `X`. In the function definition, the parameter type `T` refers to the typedef member `T` declared in class `X` and the default argument `count` refers to the static data member `count` declared in class `X`. — *end example*]

6 A `static` local variable in a member function always refers to the same object, whether or not the member function is `inline`.

7 Previously declared member functions may be mentioned in `friend` declarations.

8 Member functions of a local class shall be defined inline in their class definition, if they are defined at all.

9 [*Note:* A member function can be declared (but not defined) using a typedef for a function type. The resulting member function has exactly the same type as it would have if the function declarator were provided explicitly, see 8.3.5. For example,

```
typedef void fv(void);
typedef void fvc(void) const;
struct S {
  fv memfunc1;      // equivalent to: void memfunc1(void);
  void memfunc2();
  fvc memfunc3;     // equivalent to: void memfunc3(void) const;
};
fv  S::* pmfv1 = &S::memfunc1;
fv  S::* pmfv2 = &S::memfunc2;
fvc S::* pmfv3 = &S::memfunc3;
```

Also see 14.3. — *end note*]

9.3.1 Nonstatic member functions [class.mfct.non-static]

1 A *non-static* member function may be called for an object of its class type, or for an object of a class derived (Clause 10) from its class type, using the class member access syntax (5.2.5, 13.3.1.1). A non-static member function may also be called directly using the function call syntax (5.2.2, 13.3.1.1) from within the body of a member function of its class or of a class derived from its class.

2 If a non-static member function of a class `X` is called for an object that is not of type `X`, or of a type derived from `X`, the behavior is undefined.

3 When an *id-expression* (5.1) that is not part of a class member access syntax (5.2.5) and not used to form a pointer to member (5.3.1) is used in a member of class `X` in a context where `this` can be used (5.1.1), if name lookup (3.4) resolves the name in the *id-expression* to a non-static non-type member of some class `C`, and if either the *id-expression* is potentially evaluated or `C` is `X` or a base class of `X`, the *id-expression* is transformed into a class member access expression (5.2.5) using `(*this)` (9.3.2) as the *postfix-expression* to the left of the `.` operator. [*Note:* If `C` is not `X` or a base class of `X`, the class member access expression is ill-formed. — *end note*] Similarly during name lookup, when an *unqualified-id* (5.1) used in the definition of a member function for class `X` resolves to a `static` member, an enumerator or a nested type of class `X` or of a base class of `X`, the *unqualified-id* is transformed into a *qualified-id* (5.1) in which the *nested-name-specifier* names the class of the member function. These transformations do not apply in the template definition context (14.6.2.1). [*Example:*

```
struct tnode {
```

```
  char tword[20];
  int count;
  tnode* left;
  tnode* right;
  void set(const char*, tnode* l, tnode* r);
};

void tnode::set(const char* w, tnode* l, tnode* r) {
  count = strlen(w)+1;
  if (sizeof(tword)<=count)
      perror("tnode string too long");
  strcpy(tword,w);
  left = l;
  right = r;
}

void f(tnode n1, tnode n2) {
  n1.set("abc",&n2,0);
  n2.set("def",0,0);
}
```

In the body of the member function `tnode::set`, the member names `tword`, `count`, `left`, and `right` refer to members of the object for which the function is called. Thus, in the call `n1.set("abc",&n2,0)`, `tword` refers to `n1.tword`, and in the call `n2.set("def",0,0)`, it refers to `n2.tword`. The functions `strlen`, `perror`, and `strcpy` are not members of the class `tnode` and should be declared elsewhere.[111] — *end example*]

4 A non-static member function may be declared `const`, `volatile`, or `const volatile`. These *cv-qualifiers* affect the type of the `this` pointer (9.3.2). They also affect the function type (8.3.5) of the member function; a member function declared `const` is a *const* member function, a member function declared `volatile` is a *volatile* member function and a member function declared `const volatile` is a *const volatile* member function. [*Example:*

```
struct X {
  void g() const;
  void h() const volatile;
};
```

`X::g` is a `const` member function and `X::h` is a `const volatile` member function. — *end example*]

5 A non-static member function may be declared with a *ref-qualifier* (8.3.5); see 13.3.1.

6 A non-static member function may be declared *virtual* (10.3) or *pure virtual* (10.4).

9.3.2 The `this` pointer [class.this]

1 In the body of a non-static (9.3) member function, the keyword `this` is a prvalue expression whose value is the address of the object for which the function is called. The type of `this` in a member function of a class `X` is `X*`. If the member function is declared `const`, the type of `this` is `const X*`, if the member function is declared `volatile`, the type of `this` is `volatile X*`, and if the member function is declared `const volatile`, the type of `this` is `const volatile X*`. [*Note:* thus in a `const` member function, the object for which the function is called is accessed through a `const` access path. — *end note*] [*Example:*

```
struct s {
  int a;
  int f() const;
```

111) See, for example, `<cstring>` (21.8).

```
  int g() { return a++; }
  int h() const { return a++; } // error
};

int s::f() const { return a; }
```

The `a++` in the body of `s::h` is ill-formed because it tries to modify (a part of) the object for which `s::h()` is called. This is not allowed in a `const` member function because `this` is a pointer to `const`; that is, `*this` has `const` type. — *end example*]

2 Similarly, `volatile` semantics (7.1.6.1) apply in `volatile` member functions when accessing the object and its non-static data members.

3 A *cv-qualified* member function can be called on an object-expression (5.2.5) only if the object-expression is as cv-qualified or less-cv-qualified than the member function. [*Example:*

```
void k(s& x, const s& y) {
  x.f();
  x.g();
  y.f();
  y.g();                          // error
}
```

The call `y.g()` is ill-formed because `y` is `const` and `s::g()` is a non-`const` member function, that is, `s::g()` is less-qualified than the object-expression `y`. — *end example*]

4 Constructors (12.1) and destructors (12.4) shall not be declared `const`, `volatile` or `const volatile`. [*Note:* However, these functions can be invoked to create and destroy objects with cv-qualified types, see (12.1) and (12.4). — *end note*]

9.4 Static members [class.static]

1 A data or function member of a class may be declared `static` in a class definition, in which case it is a *static member* of the class.

2 A `static` member `s` of class `X` may be referred to using the *qualified-id* expression `X::s`; it is not necessary to use the class member access syntax (5.2.5) to refer to a `static` member. A `static` member may be referred to using the class member access syntax, in which case the object expression is evaluated. [*Example:*

```
struct process {
  static void reschedule();
};
process& g();

void f() {
  process::reschedule();          // OK: no object necessary
  g().reschedule();               // g() is called
}
```

— *end example*]

3 A `static` member may be referred to directly in the scope of its class or in the scope of a class derived (Clause 10) from its class; in this case, the `static` member is referred to as if a *qualified-id* expression was used, with the *nested-name-specifier* of the *qualified-id* naming the class scope from which the static member is referenced. [*Example:*

```
int g();
struct X {
  static int g();
```

```
};
struct Y : X {
  static int i;
};
int Y::i = g();                 // equivalent to Y::g();
```

— *end example*]

4 If an *unqualified-id* (5.1) is used in the definition of a `static` member following the member's *declarator-id*, and name lookup (3.4.1) finds that the *unqualified-id* refers to a `static` member, enumerator, or nested type of the member's class (or of a base class of the member's class), the *unqualified-id* is transformed into a *qualified-id* expression in which the *nested-name-specifier* names the class scope from which the member is referenced. [*Note:* See 5.1 for restrictions on the use of non-static data members and non-static member functions. — *end note*]

5 Static members obey the usual class member access rules (Clause 11). When used in the declaration of a class member, the `static` specifier shall only be used in the member declarations that appear within the *member-specification* of the class definition. [*Note:* It cannot be specified in member declarations that appear in namespace scope. — *end note*]

9.4.1 Static member functions [class.static.mfct]

1 [*Note:* The rules described in 9.3 apply to `static` member functions. — *end note*]

2 [*Note:* A `static` member function does not have a `this` pointer (9.3.2). — *end note*] A `static` member function shall not be `virtual`. There shall not be a `static` and a non-static member function with the same name and the same parameter types (13.1). A `static` member function shall not be declared `const`, `volatile`, or `const volatile`.

9.4.2 Static data members [class.static.data]

1 A `static` data member is not part of the subobjects of a class. If a `static` data member is declared `thread_local` there is one copy of the member per thread. If a `static` data member is not declared `thread_local` there is one copy of the data member that is shared by all the objects of the class.

2 The declaration of a `static` data member in its class definition is not a definition and may be of an incomplete type other than cv-qualified `void`. The definition for a `static` data member shall appear in a namespace scope enclosing the member's class definition. In the definition at namespace scope, the name of the `static` data member shall be qualified by its class name using the `::` operator. The *initializer* expression in the definition of a `static` data member is in the scope of its class (3.3.7). [*Example:*

```
class process {
  static process* run_chain;
  static process* running;
};

process* process::running = get_main();
process* process::run_chain = running;
```

The `static` data member `run_chain` of class `process` is defined in global scope; the notation `process::run_chain` specifies that the member `run_chain` is a member of class `process` and in the scope of class `process`. In the `static` data member definition, the *initializer* expression refers to the `static` data member `running` of class `process`. — *end example*]

[*Note:* Once the `static` data member has been defined, it exists even if no objects of its class have been created. [*Example:* in the example above, `run_chain` and `running` exist even if no objects of class `process` are created by the program. — *end example*] — *end note*]

3 If a non-volatile `const static` data member is of integral or enumeration type, its declaration in the class definition can specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression (5.20). A `static` data member of literal type can be declared in the class definition with the `constexpr` specifier; if so, its declaration shall specify a *brace-or-equal-initializer* in which every *initializer-clause* that is an *assignment-expression* is a constant expression. [*Note:* In both these cases, the member may appear in constant expressions. — *end note*] The member shall still be defined in a namespace scope if it is odr-used (3.2) in the program and the namespace scope definition shall not contain an *initializer*.

4 [*Note:* There shall be exactly one definition of a `static` data member that is odr-used (3.2) in a program; no diagnostic is required. — *end note*] Unnamed classes and classes contained directly or indirectly within unnamed classes shall not contain `static` data members.

5 `Static` data members of a class in namespace scope have the linkage of that class (3.5). A local class shall not have `static` data members.

6 `Static` data members are initialized and destroyed exactly like non-local variables (3.6.2, 3.6.3).

7 A `static` data member shall not be `mutable` (7.1.1).

9.5 Unions [class.union]

1 In a union, at most one of the non-static data members can be active at any time, that is, the value of at most one of the non-static data members can be stored in a union at any time. [*Note:* One special guarantee is made in order to simplify the use of unions: If a standard-layout union contains several standard-layout structs that share a common initial sequence (9.2), and if an object of this standard-layout union type contains one of the standard-layout structs, it is permitted to inspect the common initial sequence of any of standard-layout struct members; see 9.2. — *end note*] The size of a union is sufficient to contain the largest of its non-static data members. Each non-static data member is allocated as if it were the sole member of a struct. All non-static data members of a union object have the same address.

2 A union can have member functions (including constructors and destructors), but it shall not have virtual (10.3) functions. A union shall not have base classes. A union shall not be used as a base class. If a union contains a non-static data member of reference type the program is ill-formed. [*Note:* If any non-static data member of a union has a non-trivial default constructor (12.1), copy constructor (12.8), move constructor (12.8), copy assignment operator (12.8), move assignment operator (12.8), or destructor (12.4), the corresponding member function of the union must be user-provided or it will be implicitly deleted (8.4.3) for the union. — *end note*]

3 [*Example:* Consider the following union:

```
union U {
  int i;
  float f;
  std::string s;
};
```

Since `std::string` (21.3) declares non-trivial versions of all of the special member functions, `U` will have an implicitly deleted default constructor, copy/move constructor, copy/move assignment operator, and destructor. To use `U`, some or all of these member functions must be user-provided. — *end example*]

4 [*Note:* In general, one must use explicit destructor calls and placement new operators to change the active member of a union. — *end note*] [*Example:* Consider an object `u` of a `union` type `U` having non-static data members `m` of type `M` and `n` of type `N`. If `M` has a non-trivial destructor and `N` has a non-trivial constructor (for instance, if they declare or inherit virtual functions), the active member of `u` can be safely switched from `m` to `n` using the destructor and placement new operator as follows:

```
u.m.~M();
```

```
new (&u.n) N;
```

— *end example*]

5 A union of the form

`union {` *member-specification* `} ;`

is called an *anonymous union*; it defines an unnamed object of unnamed type. Each *member-declaration* in the *member-specification* of an anonymous union shall either define a non-static data member or be a *static_assert-declaration*. [*Note:* Nested types, anonymous unions, and functions cannot be declared within an anonymous union. — *end note*] The names of the members of an anonymous union shall be distinct from the names of any other entity in the scope in which the anonymous union is declared. For the purpose of name lookup, after the anonymous union definition, the members of the anonymous union are considered to have been defined in the scope in which the anonymous union is declared. [*Example:*

```
void f() {
  union { int a; const char* p; };
  a = 1;
  p = "Jennifer";
}
```

Here `a` and `p` are used like ordinary (nonmember) variables, but since they are union members they have the same address. — *end example*]

6 Anonymous unions declared in a named namespace or in the global namespace shall be declared `static`. Anonymous unions declared at block scope shall be declared with any storage class allowed for a block-scope variable, or with no storage class. A storage class is not allowed in a declaration of an anonymous union in a class scope. An anonymous union shall not have `private` or `protected` members (Clause 11). An anonymous union shall not have function members.

7 A union for which objects, pointers, or references are declared is not an anonymous union. [*Example:*

```
void f() {
  union { int aa; char* p; } obj, *ptr = &obj;
  aa = 1;                          // error
  ptr->aa = 1;                     // OK
}
```

The assignment to plain `aa` is ill-formed since the member name is not visible outside the union, and even if it were visible, it is not associated with any particular object. — *end example*] [*Note:* Initialization of unions with no user-declared constructors is described in (8.5.1). — *end note*]

8 A *union-like class* is a union or a class that has an anonymous union as a direct member. A union-like class `X` has a set of *variant members*. If `X` is a union, a non-static data member of `X` that is not an anonymous union is a variant member of `X`. In addition, a non-static data member of an anonymous union that is a member of `X` is also a variant member of `X`. At most one variant member of a union may have a *brace-or-equal-initializer*. [*Example:*

```
union U {
  int x = 0;
  union {
    int k;
  };
  union {
    int z;
    int y = 1; // error: initialization for second variant member of U
  };
};
```

— *end example*]

9.6 Bit-fields [class.bit]

1 A *member-declarator* of the form

identifier$_{opt}$ *attribute-specifier-seq*$_{opt}$: *constant-expression*

specifies a bit-field; its length is set off from the bit-field name by a colon. The optional *attribute-specifier-seq* appertains to the entity being declared. The bit-field attribute is not part of the type of the class member. The *constant-expression* shall be an integral constant expression with a value greater than or equal to zero. The value of the integral constant expression may be larger than the number of bits in the object representation (3.9) of the bit-field's type; in such cases the extra bits are used as padding bits and do not participate in the value representation (3.9) of the bit-field. Allocation of bit-fields within a class object is implementation-defined. Alignment of bit-fields is implementation-defined. Bit-fields are packed into some addressable allocation unit. [*Note:* Bit-fields straddle allocation units on some machines and not on others. Bit-fields are assigned right-to-left on some machines, left-to-right on others. — *end note*]

2 A declaration for a bit-field that omits the *identifier* declares an *unnamed* bit-field. Unnamed bit-fields are not members and cannot be initialized. [*Note:* An unnamed bit-field is useful for padding to conform to externally-imposed layouts. — *end note*] As a special case, an unnamed bit-field with a width of zero specifies alignment of the next bit-field at an allocation unit boundary. Only when declaring an unnamed bit-field may the value of the *constant-expression* be equal to zero.

3 A bit-field shall not be a static member. A bit-field shall have integral or enumeration type (3.9.1). A `bool` value can successfully be stored in a bit-field of any nonzero size. The address-of operator `&` shall not be applied to a bit-field, so there are no pointers to bit-fields. A non-const reference shall not be bound to a bit-field (8.5.3). [*Note:* If the initializer for a reference of type `const T&` is an lvalue that refers to a bit-field, the reference is bound to a temporary initialized to hold the value of the bit-field; the reference is not bound to the bit-field directly. See 8.5.3. — *end note*]

4 If the value `true` or `false` is stored into a bit-field of type `bool` of any size (including a one bit bit-field), the original `bool` value and the value of the bit-field shall compare equal. If the value of an enumerator is stored into a bit-field of the same enumeration type and the number of bits in the bit-field is large enough to hold all the values of that enumeration type (7.2), the original enumerator value and the value of the bit-field shall compare equal. [*Example:*

```
enum BOOL { FALSE=0, TRUE=1 };
struct A {
  BOOL b:1;
};
A a;
void f() {
  a.b = TRUE;
  if (a.b == TRUE)              // yields true
    { /* ... */ }
}
```

— *end example*]

9.7 Nested class declarations [class.nest]

1 A class can be declared within another class. A class declared within another is called a *nested* class. The name of a nested class is local to its enclosing class. The nested class is in the scope of its enclosing class. [*Note:* See 5.1 for restrictions on the use of non-static data members and non-static member functions. — *end note*]

[*Example:*

```
int x;
int y;

struct enclose {
  int x;
  static int s;

  struct inner {
    void f(int i) {
      int a = sizeof(x);        // OK: operand of sizeof is an unevaluated operand
      x = i;                    // error: assign to enclose::x
      s = i;                    // OK: assign to enclose::s
      ::x = i;                  // OK: assign to global x
      y = i;                    // OK: assign to global y
    }
    void g(enclose* p, int i) {
      p->x = i;                 // OK: assign to enclose::x
    }
  };
};

inner* p = 0;                   // error: inner not in scope
```

— *end example*]

2 Member functions and static data members of a nested class can be defined in a namespace scope enclosing the definition of their class. [*Example:*

```
struct enclose {
  struct inner {
    static int x;
    void f(int i);
  };
};

int enclose::inner::x = 1;

void enclose::inner::f(int i) { /* ... */ }
```

— *end example*]

3 If class `X` is defined in a namespace scope, a nested class `Y` may be declared in class `X` and later defined in the definition of class `X` or be later defined in a namespace scope enclosing the definition of class `X`. [*Example:*

```
class E {
  class I1;                     // forward declaration of nested class
  class I2;
  class I1 { };                 // definition of nested class
};
class E::I2 { };                // definition of nested class
```

— *end example*]

4 Like a member function, a friend function (11.3) defined within a nested class is in the lexical scope of that class; it obeys the same rules for name binding as a static member function of that class (9.4), but it has no special access rights to members of an enclosing class.

9.8 Local class declarations [class.local]

1 A class can be declared within a function definition; such a class is called a *local* class. The name of a local class is local to its enclosing scope. The local class is in the scope of the enclosing scope, and has the same access to names outside the function as does the enclosing function. Declarations in a local class shall not odr-use (3.2) a variable with automatic storage duration from an enclosing scope. [*Example:*

```
int x;
void f() {
  static int s ;
  int x;
  const int N = 5;
  extern int q();

  struct local {
    int g() { return x; }       // error: odr-use of automatic variable x
    int h() { return s; }       // OK
    int k() { return ::x; }     // OK
    int l() { return q(); }     // OK
    int m() { return N; }       // OK: not an odr-use
    int* n() { return &N; }     // error: odr-use of automatic variable N
  };
}

local* p = 0;                   // error: local not in scope
```

— *end example*]

2 An enclosing function has no special access to members of the local class; it obeys the usual access rules (Clause 11). Member functions of a local class shall be defined within their class definition, if they are defined at all.

3 If class `X` is a local class a nested class `Y` may be declared in class `X` and later defined in the definition of class `X` or be later defined in the same scope as the definition of class `X`. A class nested within a local class is a local class.

4 A local class shall not have static data members.

9.9 Nested type names [class.nested.type]

1 Type names obey exactly the same scope rules as other names. In particular, type names defined within a class definition cannot be used outside their class without qualification. [*Example:*

```
struct X {
  typedef int I;
  class Y { /* ... */ };
  I a;
};

I b;                            // error
Y c;                            // error
X::Y d;                         // OK
X::I e;                         // OK
```

— *end example*]

10 Derived classes [class.derived]

1 A list of base classes can be specified in a class definition using the notation:

base-clause:
: *base-specifier-list*

base-specifier-list:
base-specifier ...$_{opt}$
base-specifier-list , *base-specifier* ...$_{opt}$

base-specifier:
attribute-specifier-seq$_{opt}$ *base-type-specifier*
attribute-specifier-seq$_{opt}$ `virtual` *access-specifier*$_{opt}$ *base-type-specifier*
attribute-specifier-seq$_{opt}$ *access-specifier* `virtual`$_{opt}$ *base-type-specifier*

class-or-decltype:
nested-name-specifier$_{opt}$ *class-name*
decltype-specifier

base-type-specifier:
class-or-decltype

access-specifier:
`private`
`protected`
`public`

The optional *attribute-specifier-seq* appertains to the *base-specifier*.

2 The type denoted by a *base-type-specifier* shall be a class type that is not an incompletely defined class (Clause 9); this class is called a *direct base class* for the class being defined. During the lookup for a base class name, non-type names are ignored (3.3.10). If the name found is not a *class-name*, the program is ill-formed. A class `B` is a base class of a class `D` if it is a direct base class of `D` or a direct base class of one of `D`'s base classes. A class is an *indirect* base class of another if it is a base class but not a direct base class. A class is said to be (directly or indirectly) *derived* from its (direct or indirect) base classes. [*Note:* See Clause 11 for the meaning of *access-specifier*. — *end note*] Unless redeclared in the derived class, members of a base class are also considered to be members of the derived class. The base class members are said to be *inherited* by the derived class. Inherited members can be referred to in expressions in the same manner as other members of the derived class, unless their names are hidden or ambiguous (10.2). [*Note:* The scope resolution operator `::` (5.1) can be used to refer to a direct or indirect base member explicitly. This allows access to a name that has been redeclared in the derived class. A derived class can itself serve as a base class subject to access control; see 11.2. A pointer to a derived class can be implicitly converted to a pointer to an accessible unambiguous base class (4.10). An lvalue of a derived class type can be bound to a reference to an accessible unambiguous base class (8.5.3). — *end note*]

3 The *base-specifier-list* specifies the type of the *base class subobjects* contained in an object of the derived class type. [*Example:*

```
struct Base {
  int a, b, c;
};

struct Derived : Base {
  int b;
};
```

```
struct Derived2 : Derived {
  int c;
};
```

Here, an object of class `Derived2` will have a subobject of class `Derived` which in turn will have a subobject of class `Base`. — *end example*]

4 A *base-specifier* followed by an ellipsis is a pack expansion (14.5.3).

5 The order in which the base class subobjects are allocated in the most derived object (1.8) is unspecified. [*Note:* a derived class and its base class subobjects can be represented by a directed acyclic graph (DAG) where an arrow means "directly derived from." A DAG of subobjects is often referred to as a "subobject lattice."

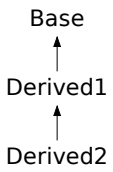


Figure 2 — Directed acyclic graph

6 The arrows need not have a physical representation in memory. — *end note*]

7 [*Note:* Initialization of objects representing base classes can be specified in constructors; see 12.6.2. — *end note*]

8 [*Note:* A base class subobject might have a layout (3.7) different from the layout of a most derived object of the same type. A base class subobject might have a polymorphic behavior (12.7) different from the polymorphic behavior of a most derived object of the same type. A base class subobject may be of zero size (Clause 9); however, two subobjects that have the same class type and that belong to the same most derived object must not be allocated at the same address (5.10). — *end note*]

10.1 Multiple base classes [class.mi]

1 A class can be derived from any number of base classes. [*Note:* The use of more than one direct base class is often called multiple inheritance. — *end note*] [*Example:*

```
class A { /* ... */ };
class B { /* ... */ };
class C { /* ... */ };
class D : public A, public B, public C { /* ... */ };
```

— *end example*]

2 [*Note:* The order of derivation is not significant except as specified by the semantics of initialization by constructor (12.6.2), cleanup (12.4), and storage layout (9.2, 11.1). — *end note*]

3 A class shall not be specified as a direct base class of a derived class more than once. [*Note:* A class can be an indirect base class more than once and can be a direct and an indirect base class. There are limited things that can be done with such a class. The non-static data members and member functions of the direct base class cannot be referred to in the scope of the derived class. However, the static members, enumerations and types can be unambiguously referred to. — *end note*] [*Example:*

```
class X { /* ... */ };
```

```
class Y : public X, public X { /* ... */ };              // ill-formed

class L { public: int next;  /* ... */ };
class A : public L { /* ... */ };
class B : public L { /* ... */ };
class C : public A, public B { void f(); /* ... */ };    // well-formed
class D : public A, public L { void f(); /* ... */ };    // well-formed
```

— *end example*]

4 A base class specifier that does not contain the keyword `virtual`, specifies a *non-virtual* base class. A base class specifier that contains the keyword `virtual`, specifies a *virtual* base class. For each distinct occurrence of a non-virtual base class in the class lattice of the most derived class, the most derived object (1.8) shall contain a corresponding distinct base class subobject of that type. For each distinct base class that is specified virtual, the most derived object shall contain a single base class subobject of that type. [*Example:* for an object of class type `C`, each distinct occurrence of a (non-virtual) base class `L` in the class lattice of `C` corresponds one-to-one with a distinct `L` subobject within the object of type `C`. Given the class `C` defined above, an object of class `C` will have two subobjects of class `L` as shown below.

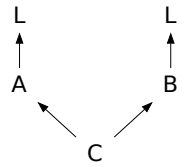


Figure 3 — Non-virtual base

5 In such lattices, explicit qualification can be used to specify which subobject is meant. The body of function `C::f` could refer to the member `next` of each `L` subobject:

```
void C::f() { A::next = B::next; }       // well-formed
```

Without the `A::` or `B::` qualifiers, the definition of `C::f` above would be ill-formed because of ambiguity (10.2).

6 For another example,

```
class V { /* ... */ };
class A : virtual public V { /* ... */ };
class B : virtual public V { /* ... */ };
class C : public A, public B { /* ... */ };
```

for an object `c` of class type `C`, a single subobject of type `V` is shared by every base subobject of `c` that has a `virtual` base class of type `V`. Given the class `C` defined above, an object of class `C` will have one subobject of class `V`, as shown below.

7 A class can have both virtual and non-virtual base classes of a given type.

```
class B { /* ... */ };
class X : virtual public B { /* ... */ };
class Y : virtual public B { /* ... */ };
class Z : public B { /* ... */ };
class AA : public X, public Y, public Z { /* ... */ };
```

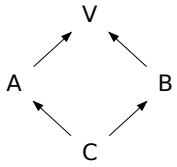


Figure 4 — Virtual base

For an object of class `AA`, all `virtual` occurrences of base class `B` in the class lattice of `AA` correspond to a single `B` subobject within the object of type `AA`, and every other occurrence of a (non-virtual) base class `B` in the class lattice of `AA` corresponds one-to-one with a distinct `B` subobject within the object of type `AA`. Given the class `AA` defined above, class `AA` has two subobjects of class `B`: `Z`'s `B` and the virtual `B` shared by `X` and `Y`, as shown below.

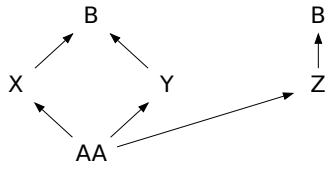


Figure 5 — Virtual and non-virtual base

— *end example*]

10.2 Member name lookup [class.member.lookup]

1 Member name lookup determines the meaning of a name (*id-expression*) in a class scope (3.3.7). Name lookup can result in an *ambiguity*, in which case the program is ill-formed. For an *id-expression*, name lookup begins in the class scope of `this`; for a *qualified-id*, name lookup begins in the scope of the *nested-name-specifier*. Name lookup takes place before access control (3.4, Clause 11).

2 The following steps define the result of name lookup for a member name `f` in a class scope `C`.

3 The *lookup set* for `f` in `C`, called $S(f, C)$, consists of two component sets: the *declaration set*, a set of members named `f`; and the *subobject set*, a set of subobjects where declarations of these members (possibly including *using-declaration*s) were found. In the declaration set, *using-declaration*s are replaced by the set of designated members that are not hidden or overridden by members of the derived class (7.3.3), and type declarations (including injected-class-names) are replaced by the types they designate. $S(f, C)$ is calculated as follows:

4 If `C` contains a declaration of the name `f`, the declaration set contains every declaration of `f` declared in `C` that satisfies the requirements of the language construct in which the lookup occurs. [*Note:* Looking up a name in an *elaborated-type-specifier* (3.4.4) or *base-specifier* (Clause 10), for instance, ignores all non-type declarations, while looking up a name in a *nested-name-specifier* (3.4.3) ignores function, variable, and enumerator declarations. As another example, looking up a name in a *using-declaration* (7.3.3) includes the declaration of a class or enumeration that would ordinarily be hidden by another declaration of that name

in the same scope. — *end note*] If the resulting declaration set is not empty, the subobject set contains `C` itself, and calculation is complete.

5 Otherwise (i.e., `C` does not contain a declaration of `f` or the resulting declaration set is empty), $S(f, C)$ is initially empty. If `C` has base classes, calculate the lookup set for `f` in each direct base class subobject B_i, and merge each such lookup set $S(f, B_i)$ in turn into $S(f, C)$.

6 The following steps define the result of merging lookup set $S(f, B_i)$ into the intermediate $S(f, C)$:

(6.1) — If each of the subobject members of $S(f, B_i)$ is a base class subobject of at least one of the subobject members of $S(f, C)$, or if $S(f, B_i)$ is empty, $S(f, C)$ is unchanged and the merge is complete. Conversely, if each of the subobject members of $S(f, C)$ is a base class subobject of at least one of the subobject members of $S(f, B_i)$, or if $S(f, C)$ is empty, the new $S(f, C)$ is a copy of $S(f, B_i)$.

(6.2) — Otherwise, if the declaration sets of $S(f, B_i)$ and $S(f, C)$ differ, the merge is ambiguous: the new $S(f, C)$ is a lookup set with an invalid declaration set and the union of the subobject sets. In subsequent merges, an invalid declaration set is considered different from any other.

(6.3) — Otherwise, the new $S(f, C)$ is a lookup set with the shared set of declarations and the union of the subobject sets.

7 The result of name lookup for `f` in `C` is the declaration set of $S(f, C)$. If it is an invalid set, the program is ill-formed. [*Example:*

```
struct A { int x; };                    // S(x,A) = { { A::x }, { A } }
struct B { float x; };                  // S(x,B) = { { B::x }, { B } }
struct C: public A, public B { };       // S(x,C) = { invalid, { A in C, B in C } }
struct D: public virtual C { };         // S(x,D) = S(x,C)
struct E: public virtual C { char x; }; // S(x,E) = { { E::x }, { E } }
struct F: public D, public E { };       // S(x,F) = S(x,E)
int main() {
  F f;
  f.x = 0;                              // OK, lookup finds E::x
}
```

$S(x, F)$ is unambiguous because the `A` and `B` base subobjects of `D` are also base subobjects of `E`, so $S(x, D)$ is discarded in the first merge step. — *end example*]

8 If the name of an overloaded function is unambiguously found, overloading resolution (13.3) also takes place before access control. Ambiguities can often be resolved by qualifying a name with its class name. [*Example:*

```
struct A {
  int f();
};

struct B {
  int f();
};

struct C : A, B {
  int f() { return A::f() + B::f(); }
};
```

— *end example*]

9 [*Note:* A static member, a nested type or an enumerator defined in a base class `T` can unambiguously be found even if an object has more than one base class subobject of type `T`. Two base class subobjects share the non-static member subobjects of their common virtual base classes. — *end note*] [*Example:*

```
struct V {
  int v;
};
struct A {
  int a;
  static int   s;
  enum { e };
};
struct B : A, virtual V { };
struct C : A, virtual V { };
struct D : B, C { };

void f(D* pd) {
  pd->v++;          // OK: only one v (virtual)
  pd->s++;          // OK: only one s (static)
  int i = pd->e;    // OK: only one e (enumerator)
  pd->a++;          // error, ambiguous: two as in D
}
```

— *end example*]

10 [*Note:* When virtual base classes are used, a hidden declaration can be reached along a path through the subobject lattice that does not pass through the hiding declaration. This is not an ambiguity. The identical use with non-virtual base classes is an ambiguity; in that case there is no unique instance of the name that hides all the others. — *end note*] [*Example:*

```
struct V { int f();  int x; };
struct W { int g();  int y; };
struct B : virtual V, W {
  int f();  int x;
  int g();  int y;
};
struct C : virtual V, W { };

struct D : B, C { void glorp(); };
```

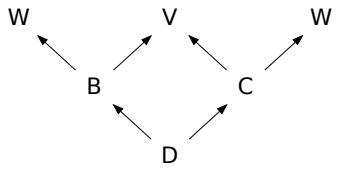


Figure 6 — Name lookup

11 [*Note:* The names declared in V and the left-hand instance of W are hidden by those in B, but the names declared in the right-hand instance of W are not hidden at all. — *end note*]

```
void D::glorp() {
  x++;              // OK: B::x hides V::x
  f();              // OK: B::f() hides V::f()
  y++;              // error: B::y and C's W::y
  g();              // error: B::g() and C's W::g()
```

```
}
```

— *end example*]

12 An explicit or implicit conversion from a pointer to or an expression designating an object of a derived class to a pointer or reference to one of its base classes shall unambiguously refer to a unique object representing the base class. [*Example:*

```
struct V { };
struct A { };
struct B : A, virtual V { };
struct C : A, virtual V { };
struct D : B, C { };

void g() {
  D d;
  B* pb = &d;
  A* pa = &d;        // error, ambiguous: C's A or B's A?
  V* pv = &d;        // OK: only one V subobject
}
```

— *end example*]

13 [*Note:* Even if the result of name lookup is unambiguous, use of a name found in multiple subobjects might still be ambiguous (4.11, 5.2.5, 11.2). — *end note*] [*Example:*

```
struct B1 {
  void f();
  static void f(int);
  int i;
};
struct B2 {
  void f(double);
};
struct I1: B1 { };
struct I2: B1 { };

struct D: I1, I2, B2 {
  using B1::f;
  using B2::f;
  void g() {
    f();                        // Ambiguous conversion of this
    f(0);                       // Unambiguous (static)
    f(0.0);                     // Unambiguous (only one B2)
    int B1::* mpB1 = &D::i;     // Unambiguous
    int D::* mpD = &D::i;       // Ambiguous conversion
  }
};
```

— *end example*]

10.3 Virtual functions [class.virtual]

1 Virtual functions support dynamic binding and object-oriented programming. A class that declares or inherits a virtual function is called a *polymorphic class*.

2 If a virtual member function `vf` is declared in a class `Base` and in a class `Derived`, derived directly or indirectly from `Base`, a member function `vf` with the same name, parameter-type-list (8.3.5), cv-qualification, and ref-

qualifier (or absence of same) as `Base::vf` is declared, then `Derived::vf` is also virtual (whether or not it is so declared) and it *overrides*[112] `Base::vf`. For convenience we say that any virtual function overrides itself. A virtual member function `C::vf` of a class object `S` is a *final overrider* unless the most derived class (1.8) of which `S` is a base class subobject (if any) declares or inherits another member function that overrides `vf`. In a derived class, if a virtual member function of a base class subobject has more than one final overrider the program is ill-formed. [*Example:*

```
struct A {
  virtual void f();
};
struct B : virtual A {
  virtual void f();
};
struct C : B , virtual A {
  using A::f;
};

void foo() {
  C c;
  c.f();              // calls B::f, the final overrider
  c.C::f();           // calls A::f because of the using-declaration
}
```

— *end example*]

[*Example:*

```
struct A { virtual void f(); };
struct B : A { };
struct C : A { void f(); };
struct D : B, C { };  // OK: A::f and C::f are the final overriders
                      // for the B and C subobjects, respectively
```

— *end example*]

3 [*Note:* A virtual member function does not have to be visible to be overridden, for example,

```
struct B {
  virtual void f();
};
struct D : B {
  void f(int);
};
struct D2 : D {
  void f();
};
```

the function `f(int)` in class `D` hides the virtual function `f()` in its base class `B`; `D::f(int)` is not a virtual function. However, `f()` declared in class `D2` has the same name and the same parameter list as `B::f()`, and therefore is a virtual function that overrides the function `B::f()` even though `B::f()` is not visible in class `D2`. — *end note*]

4 If a virtual function `f` in some class `B` is marked with the *virt-specifier* `final` and in a class `D` derived from `B` a function `D::f` overrides `B::f`, the program is ill-formed. [*Example:*

112) A function with the same name but a different parameter list (Clause 13) as a virtual function is not necessarily virtual and does not override. The use of the `virtual` specifier in the declaration of an overriding function is legal but redundant (has empty semantics). Access control (Clause 11) is not considered in determining overriding.

```
struct B {
  virtual void f() const final;
};

struct D : B {
  void f() const;      // error: D::f attempts to override final B::f
};
```

— *end example*]

5 If a virtual function is marked with the *virt-specifier* `override` and does not override a member function of a base class, the program is ill-formed. [*Example:*

```
struct B {
  virtual void f(int);
};

struct D : B {
  virtual void f(long) override;  // error: wrong signature overriding B::f
  virtual void f(int) override;   // OK
};
```

— *end example*]

6 Even though destructors are not inherited, a destructor in a derived class overrides a base class destructor declared virtual; see 12.4 and 12.5.

7 The return type of an overriding function shall be either identical to the return type of the overridden function or *covariant* with the classes of the functions. If a function `D::f` overrides a function `B::f`, the return types of the functions are covariant if they satisfy the following criteria:

(7.1) — both are pointers to classes, both are lvalue references to classes, or both are rvalue references to classes[113]

(7.2) — the class in the return type of `B::f` is the same class as the class in the return type of `D::f`, or is an unambiguous and accessible direct or indirect base class of the class in the return type of `D::f`

(7.3) — both pointers or references have the same cv-qualification and the class type in the return type of `D::f` has the same cv-qualification as or less cv-qualification than the class type in the return type of `B::f`.

8 If the class type in the covariant return type of `D::f` differs from that of `B::f`, the class type in the return type of `D::f` shall be complete at the point of declaration of `D::f` or shall be the class type `D`. When the overriding function is called as the final overrider of the overridden function, its result is converted to the type returned by the (statically chosen) overridden function (5.2.2). [*Example:*

```
class B { };
class D : private B { friend class Derived; };
struct Base {
  virtual void vf1();
  virtual void vf2();
  virtual void vf3();
  virtual B*   vf4();
  virtual B*   vf5();
  void f();
};
```

113) Multi-level pointers to classes or references to multi-level pointers to classes are not allowed.

```
struct No_good : public Base {
  D*  vf4();         // error: B (base class of D) inaccessible
};

class A;
struct Derived : public Base {
    void vf1();      // virtual and overrides Base::vf1()
    void vf2(int);   // not virtual, hides Base::vf2()
    char vf3();      // error: invalid difference in return type only
    D*  vf4();       // OK: returns pointer to derived class
    A*  vf5();       // error: returns pointer to incomplete class
    void f();
};

void g() {
  Derived d;
  Base* bp = &d;                // standard conversion:
                                // Derived* to Base*
  bp->vf1();                    // calls Derived::vf1()
  bp->vf2();                    // calls Base::vf2()
  bp->f();                      // calls Base::f() (not virtual)
  B*  p = bp->vf4();            // calls Derived::pf() and converts the
                                // result to B*
  Derived*  dp = &d;
  D*  q = dp->vf4();            // calls Derived::pf() and does not
                                // convert the result to B*
  dp->vf2();                    // ill-formed: argument mismatch
}
```

— *end example*]

9 [*Note:* The interpretation of the call of a virtual function depends on the type of the object for which it is called (the dynamic type), whereas the interpretation of a call of a non-virtual member function depends only on the type of the pointer or reference denoting that object (the static type) (5.2.2). — *end note*]

10 [*Note:* The `virtual` specifier implies membership, so a virtual function cannot be a nonmember (7.1.2) function. Nor can a virtual function be a static member, since a virtual function call relies on a specific object for determining which function to invoke. A virtual function declared in one class can be declared a `friend` in another class. — *end note*]

11 A virtual function declared in a class shall be defined, or declared pure (10.4) in that class, or both; but no diagnostic is required (3.2).

12 [*Example:* here are some uses of virtual functions with multiple base classes:

```
struct A {
  virtual void f();
};

struct B1 : A {                 // note non-virtual derivation
  void f();
};

struct B2 : A {
  void f();
};
```

```
struct D : B1, B2 {              // D has two separate A subobjects
};

void foo() {
  D   d;
// A* ap = &d; // would be ill-formed: ambiguous
  B1*  b1p = &d;
  A*   ap = b1p;
  D*   dp = &d;
  ap->f();                       // calls D::B1::f
  dp->f();                       // ill-formed: ambiguous
}
```

In class `D` above there are two occurrences of class `A` and hence two occurrences of the virtual member function `A::f`. The final overrider of `B1::A::f` is `B1::f` and the final overrider of `B2::A::f` is `B2::f`.

13 The following example shows a function that does not have a unique final overrider:

```
struct A {
  virtual void f();
};

struct VB1 : virtual A {         // note virtual derivation
  void f();
};

struct VB2 : virtual A {
  void f();
};

struct Error : VB1, VB2 {        // ill-formed
};

struct Okay : VB1, VB2 {
  void f();
};
```

Both `VB1::f` and `VB2::f` override `A::f` but there is no overrider of both of them in class `Error`. This example is therefore ill-formed. Class `Okay` is well formed, however, because `Okay::f` is a final overrider.

14 The following example uses the well-formed classes from above.

```
struct VB1a : virtual A {        // does not declare f
};

struct Da : VB1a, VB2 {
};

void foe() {
  VB1a*  vb1ap = new Da;
  vb1ap->f();                    // calls VB2::f
}
```

— *end example*]

15 Explicit qualification with the scope operator (5.1) suppresses the virtual call mechanism. [*Example:*

```
class B { public: virtual void f(); };
class D : public B { public: void f(); };

void D::f() { /* ... */ B::f(); }
```

Here, the function call in `D::f` really does call `B::f` and not `D::f`. — *end example*]

16 A function with a deleted definition (8.4) shall not override a function that does not have a deleted definition. Likewise, a function that does not have a deleted definition shall not override a function with a deleted definition.

10.4 Abstract classes [class.abstract]

1 The abstract class mechanism supports the notion of a general concept, such as a `shape`, of which only more concrete variants, such as `circle` and `square`, can actually be used. An abstract class can also be used to define an interface for which derived classes provide a variety of implementations.

2 An *abstract class* is a class that can be used only as a base class of some other class; no objects of an abstract class can be created except as subobjects of a class derived from it. A class is abstract if it has at least one *pure virtual function*. [*Note:* Such a function might be inherited: see below. — *end note*] A virtual function is specified *pure* by using a *pure-specifier* (9.2) in the function declaration in the class definition. A pure virtual function need be defined only if called with, or as if with (12.4), the *qualified-id* syntax (5.1). [*Example:*

```
class point { /* ... */ };
class shape {                    // abstract class
  point center;
public:
  point where() { return center; }
  void move(point p) { center=p; draw(); }
  virtual void rotate(int) = 0; // pure virtual
  virtual void draw() = 0;      // pure virtual
};
```

— *end example*] [*Note:* A function declaration cannot provide both a *pure-specifier* and a definition — *end note*] [*Example:*

```
struct C {
  virtual void f() = 0 { };      // ill-formed
};
```

— *end example*]

3 An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared. [*Example:*

```
shape x;                         // error: object of abstract class
shape* p;                        // OK
shape f();                       // error
void g(shape);                   // error
shape& h(shape&);                // OK
```

— *end example*]

4 A class is abstract if it contains or inherits at least one pure virtual function for which the final overrider is pure virtual. [*Example:*

```
class ab_circle : public shape {
  int radius;
```

```
public:
  void rotate(int) { }
  // ab_circle::draw() is a pure virtual
};
```

Since `shape::draw()` is a pure virtual function `ab_circle::draw()` is a pure virtual by default. The alternative declaration,

```
class circle : public shape {
  int radius;
public:
  void rotate(int) { }
  void draw();                    // a definition is required somewhere
};
```

would make class `circle` nonabstract and a definition of `circle::draw()` must be provided. — *end example*]

5 [*Note:* An abstract class can be derived from a class that is not abstract, and a pure virtual function may override a virtual function which is not pure. — *end note*]

6 Member functions can be called from a constructor (or destructor) of an abstract class; the effect of making a virtual call (10.3) to a pure virtual function directly or indirectly for the object being created (or destroyed) from such a constructor (or destructor) is undefined.

11 Member access control [class.access]

1 A member of a class can be

(1.1) — `private`; that is, its name can be used only by members and friends of the class in which it is declared.

(1.2) — `protected`; that is, its name can be used only by members and friends of the class in which it is declared, by classes derived from that class, and by their friends (see 11.4).

(1.3) — `public`; that is, its name can be used anywhere without access restriction.

2 A member of a class can also access all the names to which the class has access. A local class of a member function may access the same names that the member function itself may access.[114]

3 Members of a class defined with the keyword `class` are `private` by default. Members of a class defined with the keywords `struct` or `union` are `public` by default. [*Example:*

```
class X {
  int a;            // X::a is private by default
};

struct S {
  int a;            // S::a is public by default
};
```

— *end example*]

4 Access control is applied uniformly to all names, whether the names are referred to from declarations or expressions. [*Note:* Access control applies to names nominated by `friend` declarations (11.3) and *using-declarations* (7.3.3). — *end note*] In the case of overloaded function names, access control is applied to the function selected by overload resolution. [*Note:* Because access control applies to names, if access control is applied to a typedef name, only the accessibility of the typedef name itself is considered. The accessibility of the entity referred to by the typedef is not considered. For example,

```
class A {
  class B { };
public:
  typedef B BB;
};

void f() {
  A::BB x;          // OK, typedef name A::BB is public
  A::B y;           // access error, A::B is private
}
```

— *end note*]

5 It should be noted that it is *access* to members and base classes that is controlled, not their *visibility*. Names of members are still visible, and implicit conversions to base classes are still considered, when those members and base classes are inaccessible. The interpretation of a given construct is established without regard to

114) Access permissions are thus transitive and cumulative to nested and local classes.

access control. If the interpretation established makes use of inaccessible member names or base classes, the construct is ill-formed.

6 All access controls in Clause 11 affect the ability to access a class member name from the declaration of a particular entity, including parts of the declaration preceding the name of the entity being declared and, if the entity is a class, the definitions of members of the class appearing outside the class's *member-specification*. [*Note:* this access also applies to implicit references to constructors, conversion functions, and destructors. — *end note*] [*Example:*

```
class A {
  typedef int I;     // private member
  I f();
  friend I g(I);
  static I x;
  template<int> struct Q;
  template<int> friend struct R;
protected:
    struct B { };
};

A::I A::f() { return 0; }
A::I g(A::I p = A::x);
A::I g(A::I p) { return 0; }
A::I A::x = 0;
template<A::I> struct A::Q { };
template<A::I> struct R { };

struct D: A::B, A { };
```

7 Here, all the uses of `A::I` are well-formed because `A::f`, `A::x`, and `A::Q` are members of class `A` and `g` and `R` are friends of class `A`. This implies, for example, that access checking on the first use of `A::I` must be deferred until it is determined that this use of `A::I` is as the return type of a member of class `A`. Similarly, the use of `A::B` as a *base-specifier* is well-formed because `D` is derived from `A`, so checking of *base-specifier*s must be deferred until the entire *base-specifier-list* has been seen. — *end example*]

8 The names in a default argument (8.3.6) are bound at the point of declaration, and access is checked at that point rather than at any points of use of the default argument. Access checking for default arguments in function templates and in member functions of class templates is performed as described in 14.7.1.

9 The names in a default *template-argument* (14.1) have their access checked in the context in which they appear rather than at any points of use of the default *template-argument*. [*Example:*

```
class B { };
template <class T> class C {
protected:
  typedef T TT;
};

template <class U, class V = typename U::TT>
class D : public U { };

D <C<B> >* d;        // access error, C::TT is protected
```

— *end example*]

11.1 Access specifiers [class.access.spec]

1 Member declarations can be labeled by an *access-specifier* (Clause 10):

access-specifier : *member-specification*$_{opt}$

An *access-specifier* specifies the access rules for members following it until the end of the class or until another *access-specifier* is encountered. [*Example:*

```
class X {
  int a;            // X::a is private by default: class used
public:
  int b;            // X::b is public
  int c;            // X::c is public
};
```

— *end example*]

2 Any number of access specifiers is allowed and no particular order is required. [*Example:*

```
struct S {
  int a;            // S::a is public by default: struct used
protected:
  int b;            // S::b is protected
private:
  int c;            // S::c is private
public:
  int d;            // S::d is public
};
```

— *end example*]

3 [*Note:* The effect of access control on the order of allocation of data members is described in 9.2. — *end note*]

4 When a member is redeclared within its class definition, the access specified at its redeclaration shall be the same as at its initial declaration. [*Example:*

```
struct S {
  class A;
  enum E : int;
private:
  class A { };         // error: cannot change access
  enum E: int { e0 }; // error: cannot change access
};
```

— *end example*]

5 [*Note:* In a derived class, the lookup of a base class name will find the injected-class-name instead of the name of the base class in the scope in which it was declared. The injected-class-name might be less accessible than the name of the base class in the scope in which it was declared. — *end note*]

[*Example:*

```
class A { };
class B : private A { };
class C : public B {
  A* p;             // error: injected-class-name A is inaccessible
  ::A* q;           // OK
};
```

— *end example*]

11.2 Accessibility of base classes and base class members [class.access.base]

1 If a class is declared to be a base class (Clause 10) for another class using the `public` access specifier, the `public` members of the base class are accessible as `public` members of the derived class and `protected` members of the base class are accessible as `protected` members of the derived class. If a class is declared to be a base class for another class using the `protected` access specifier, the `public` and `protected` members of the base class are accessible as `protected` members of the derived class. If a class is declared to be a base class for another class using the `private` access specifier, the `public` and `protected` members of the base class are accessible as `private` members of the derived class[115].

2 In the absence of an *access-specifier* for a base class, `public` is assumed when the derived class is defined with the *class-key* `struct` and `private` is assumed when the class is defined with the *class-key* `class`. [*Example:*

```
class B { /* ... */ };
class D1 : private B { /* ... */ };
class D2 : public B { /* ... */ };
class D3 : B { /* ... */ };      // B private by default
struct D4 : public B { /* ... */ };
struct D5 : private B { /* ... */ };
struct D6 : B { /* ... */ };     // B public by default
class D7 : protected B { /* ... */ };
struct D8 : protected B { /* ... */ };
```

Here `B` is a public base of `D2`, `D4`, and `D6`, a private base of `D1`, `D3`, and `D5`, and a protected base of `D7` and `D8`. — *end example*]

3 [*Note:* A member of a private base class might be inaccessible as an inherited member name, but accessible directly. Because of the rules on pointer conversions (4.10) and explicit casts (5.4), a conversion from a pointer to a derived class to a pointer to an inaccessible base class might be ill-formed if an implicit conversion is used, but well-formed if an explicit cast is used. For example,

```
class B {
public:
  int mi;                       // non-static member
  static int si;                // static member
};
class D : private B {
};
class DD : public D {
  void f();
};

void DD::f() {
  mi = 3;                       // error: mi is private in D
  si = 3;                       // error: si is private in D
  ::B  b;
  b.mi = 3;                     // OK (b.mi is different from this->mi)
  b.si = 3;                     // OK (b.si is different from this->si)
  ::B::si = 3;                  // OK
  ::B* bp1 = this;              // error: B is a private base class
  ::B* bp2 = (::B*)this;        // OK with cast
  bp2->mi = 3;                  // OK: access through a pointer to B.
}
```

115) As specified previously in Clause 11, private members of a base class remain inaccessible even to derived classes unless `friend` declarations within the base class definition are used to grant access explicitly.

— *end note*]

4 A base class `B` of `N` is *accessible* at R, if

(4.1) — an invented public member of `B` would be a public member of `N`, or

(4.2) — R occurs in a member or friend of class `N`, and an invented public member of `B` would be a private or protected member of `N`, or

(4.3) — R occurs in a member or friend of a class `P` derived from `N`, and an invented public member of `B` would be a private or protected member of `P`, or

(4.4) — there exists a class `S` such that `B` is a base class of `S` accessible at R and `S` is a base class of `N` accessible at R.

[*Example:*

```
class B {
public:
  int m;
};

class S: private B {
  friend class N;
};

class N: private S {
  void f() {
    B* p = this;     // OK because class S satisfies the fourth condition
                     // above: B is a base class of N accessible in f() because
                     // B is an accessible base class of S and S is an accessible
                     // base class of N.
  }
};
```

— *end example*]

5 If a base class is accessible, one can implicitly convert a pointer to a derived class to a pointer to that base class (4.10, 4.11). [*Note:* It follows that members and friends of a class `X` can implicitly convert an `X*` to a pointer to a private or protected immediate base class of `X`. — *end note*] The access to a member is affected by the class in which the member is named. This naming class is the class in which the member name was looked up and found. [*Note:* This class can be explicit, e.g., when a *qualified-id* is used, or implicit, e.g., when a class member access operator (5.2.5) is used (including cases where an implicit "`this->`" is added). If both a class member access operator and a *qualified-id* are used to name the member (as in `p->T::m`), the class naming the member is the class denoted by the *nested-name-specifier* of the *qualified-id* (that is, `T`). — *end note*] A member `m` is accessible at the point R when named in class `N` if

(5.1) — `m` as a member of `N` is public, or

(5.2) — `m` as a member of `N` is private, and R occurs in a member or friend of class `N`, or

(5.3) — `m` as a member of `N` is protected, and R occurs in a member or friend of class `N`, or in a member of a class `P` derived from `N`, where `m` as a member of `P` is public, private, or protected, or

(5.4) — there exists a base class `B` of `N` that is accessible at R, and `m` is accessible at R when named in class `B`. [*Example:*

```
class B;
class A {
private:
  int i;
  friend void f(B*);
};
class B : public A { };
void f(B* p) {
  p->i = 1;          // OK: B* can be implicitly converted to A*,
                     // and f has access to i in A
}
```

— *end example*]

6 If a class member access operator, including an implicit "`this->`", is used to access a non-static data member or non-static member function, the reference is ill-formed if the left operand (considered as a pointer in the "`.`" operator case) cannot be implicitly converted to a pointer to the naming class of the right operand. [*Note:* This requirement is in addition to the requirement that the member be accessible as named. — *end note*]

11.3 Friends [class.friend]

1 A friend of a class is a function or class that is given permission to use the private and protected member names from the class. A class specifies its friends, if any, by way of friend declarations. Such declarations give special access rights to the friends, but they do not make the nominated friends members of the befriending class. [*Example:* the following example illustrates the differences between members and friends:

```
class X {
  int a;
  friend void friend_set(X*, int);
public:
  void member_set(int);
};

void friend_set(X* p, int i) { p->a = i; }
void X::member_set(int i) { a = i; }

void f() {
  X obj;
  friend_set(&obj,10);
  obj.member_set(10);
}
```

— *end example*]

2 Declaring a class to be a friend implies that the names of private and protected members from the class granting friendship can be accessed in the *base-specifier*s and member declarations of the befriended class. [*Example:*

```
class A {
  class B { };
  friend class X;
};

struct X : A::B {    // OK: A::B accessible to friend
  A::B mx;           // OK: A::B accessible to member of friend
```

```
  class Y {
    A::B my;        // OK: A::B accessible to nested member of friend
  };
};
```

— *end example*] [*Example:*

```
class X {
  enum { a=100 };
  friend class Y;
};

class Y {
  int v[X::a];      // OK, Y is a friend of X
};

class Z {
  int v[X::a];      // error: X::a is private
};
```

— *end example*]

A class shall not be defined in a friend declaration. [*Example:*

```
class A {
  friend class B { }; // error: cannot define class in friend declaration
};
```

— *end example*]

3 A `friend` declaration that does not declare a function shall have one of the following forms:

`friend` *elaborated-type-specifier* ;
`friend` *simple-type-specifier* ;
`friend` *typename-specifier* ;

[*Note:* A `friend` declaration may be the *declaration* in a *template-declaration* (Clause 14, 14.5.4). — *end note*] If the type specifier in a `friend` declaration designates a (possibly cv-qualified) class type, that class is declared as a friend; otherwise, the `friend` declaration is ignored. [*Example:*

```
class C;
typedef C Ct;

class X1 {
  friend C;         // OK: class C is a friend
};

class X2 {
  friend Ct;        // OK: class C is a friend
  friend D;         // error: no type-name D in scope
  friend class D;   // OK: elaborated-type-specifier declares new class
};

template <typename T> class R {
  friend T;
};

R<C> rc;            // class C is a friend of R<C>
R<int> Ri;          // OK: "friend int;" is ignored
```

— *end example*]

4 A function first declared in a friend declaration has the linkage of the namespace of which it is a member (3.5). Otherwise, the function retains its previous linkage (7.1.1).

5 When a `friend` declaration refers to an overloaded name or operator, only the function specified by the parameter types becomes a friend. A member function of a class `X` can be a friend of a class `Y`. [*Example:*

```
class Y {
  friend char* X::foo(int);
  friend X::X(char);            // constructors can be friends
  friend X::~X();               // destructors can be friends
};
```

— *end example*]

6 A function can be defined in a friend declaration of a class if and only if the class is a non-local class (9.8), the function name is unqualified, and the function has namespace scope. [*Example:*

```
class M {
  friend void f() { }           // definition of global f, a friend of M,
                                // not the definition of a member function
};
```

— *end example*]

7 Such a function is implicitly an inline function (7.1.2). A `friend` function defined in a class is in the (lexical) scope of the class in which it is defined. A friend function defined outside the class is not (3.4.1).

8 No *storage-class-specifier* shall appear in the *decl-specifier-seq* of a friend declaration.

9 A name nominated by a friend declaration shall be accessible in the scope of the class containing the friend declaration. The meaning of the friend declaration is the same whether the friend declaration appears in the `private`, `protected` or `public` (9.2) portion of the class *member-specification*.

10 Friendship is neither inherited nor transitive. [*Example:*

```
class A {
  friend class B;
  int a;
};

class B {
  friend class C;
};

class C  {
  void f(A* p) {
    p->a++;             // error: C is not a friend of A
                        // despite being a friend of a friend
  }
};

class D : public B  {
  void f(A* p) {
    p->a++;             // error: D is not a friend of A
                        // despite being derived from a friend
  }
};
```

— *end example*]

11 If a friend declaration appears in a local class (9.8) and the name specified is an unqualified name, a prior declaration is looked up without considering scopes that are outside the innermost enclosing non-class scope. For a friend function declaration, if there is no prior declaration, the program is ill-formed. For a friend class declaration, if there is no prior declaration, the class that is specified belongs to the innermost enclosing non-class scope, but if it is subsequently referenced, its name is not found by name lookup until a matching declaration is provided in the innermost enclosing non-class scope. [*Example:*

```
class X;
void a();
void f() {
  class Y;
  extern void b();
  class A {
  friend class X;   // OK, but X is a local class, not ::X
  friend class Y;   // OK
  friend class Z;   // OK, introduces local class Z
  friend void a();  // error, ::a is not considered
  friend void b();  // OK
  friend void c();  // error
  };
  X* px;            // OK, but ::X is found
  Z* pz;            // error, no Z is found
}
```

— *end example*]

11.4 Protected member access [class.protected]

1 An additional access check beyond those described earlier in Clause 11 is applied when a non-static data member or non-static member function is a protected member of its naming class (11.2)[116] As described earlier, access to a protected member is granted because the reference occurs in a friend or member of some class `C`. If the access is to form a pointer to member (5.3.1), the *nested-name-specifier* shall denote `C` or a class derived from `C`. All other accesses involve a (possibly implicit) object expression (5.2.5). In this case, the class of the object expression shall be `C` or a class derived from `C`. [*Example:*

```
class B {
protected:
  int i;
  static int j;
};

class D1 : public B {
};

class D2 : public B {
  friend void fr(B*,D1*,D2*);
  void mem(B*,D1*);
};

void fr(B* pb, D1* p1, D2* p2) {
  pb->i = 1;                    // ill-formed
  p1->i = 2;                    // ill-formed
  p2->i = 3;                    // OK (access through a D2)
```

116) This additional check does not apply to other members, e.g., static data members or enumerator member constants.

```
  p2->B::i = 4;                 // OK (access through a D2, even though
                                // naming class is B)
  int B::* pmi_B = &B::i;       // ill-formed
  int B::* pmi_B2 = &D2::i;     // OK (type of &D2::i is int B::*)
  B::j = 5;                     // ill-formed (not a friend of naming class B)
  D2::j = 6;                    // OK (because refers to static member)
}

void D2::mem(B* pb, D1* p1) {
  pb->i = 1;                    // ill-formed
  p1->i = 2;                    // ill-formed
  i = 3;                        // OK (access through this)
  B::i = 4;                     // OK (access through this, qualification ignored)
  int B::* pmi_B = &B::i;       // ill-formed
  int B::* pmi_B2 = &D2::i;     // OK
  j = 5;                        // OK (because j refers to static member)
  B::j = 6;                     // OK (because B::j refers to static member)
}

void g(B* pb, D1* p1, D2* p2) {
  pb->i = 1;                    // ill-formed
  p1->i = 2;                    // ill-formed
  p2->i = 3;                    // ill-formed
}
```

— *end example*]

11.5 Access to virtual functions [class.access.virt]

1 The access rules (Clause 11) for a virtual function are determined by its declaration and are not affected by the rules for a function that later overrides it. [*Example:*

```
class B {
public:
  virtual int f();
};

class D : public B {
private:
  int f();
};

void f() {
  D d;
  B* pb = &d;
  D* pd = &d;

  pb->f();                      // OK: B::f() is public,
                                // D::f() is invoked
  pd->f();                      // error: D::f() is private
}
```

— *end example*]

2 Access is checked at the call point using the type of the expression used to denote the object for which the member function is called (`B*` in the example above). The access of the member function in the class in which it was defined (`D` in the example above) is in general not known.

11.6 Multiple access [class.paths]

1 If a name can be reached by several paths through a multiple inheritance graph, the access is that of the path that gives most access. [*Example:*

```
class W { public: void f(); };
class A : private virtual W { };
class B : public virtual W { };
class C : public A, public B {
  void f() { W::f(); }          // OK
};
```

2 Since `W::f()` is available to `C::f()` along the public path through `B`, access is allowed. — *end example*]

11.7 Nested classes [class.access.nest]

1 A nested class is a member and as such has the same access rights as any other member. The members of an enclosing class have no special access to members of a nested class; the usual access rules (Clause 11) shall be obeyed. [*Example:*

```
class E {
  int x;
  class B { };

  class I {
    B b;                        // OK: E::I can access E::B
    int y;
    void f(E* p, int i) {
      p->x = i;                 // OK: E::I can access E::x
    }
  };

  int g(I* p) {
    return p->y;                // error: I::y is private
  }
};
```

— *end example*]

12 Special member functions [special]

1 The default constructor (12.1), copy constructor and copy assignment operator (12.8), move constructor and move assignment operator (12.8), and destructor (12.4) are *special member functions*. [*Note:* The implementation will implicitly declare these member functions for some class types when the program does not explicitly declare them. The implementation will implicitly define them if they are odr-used (3.2). See 12.1, 12.4 and 12.8. — *end note*] An implicitly-declared special member function is declared at the closing `}` of the *class-specifier*. Programs shall not define implicitly-declared special member functions.

2 Programs may explicitly refer to implicitly-declared special member functions. [*Example:* a program may explicitly call, take the address of or form a pointer to member to an implicitly-declared special member function.

```
struct A { };                   // implicitly declared A::operator=
struct B : A {
  B& operator=(const B &);
};
B& B::operator=(const B& s) {
  this->A::operator=(s);        // well formed
  return *this;
}
```

— *end example*]

3 [*Note:* The special member functions affect the way objects of class type are created, copied, moved, and destroyed, and how values can be converted to values of other types. Often such special member functions are called implicitly. — *end note*]

4 Special member functions obey the usual access rules (Clause 11). [*Example:* declaring a constructor `protected` ensures that only derived classes and friends can create objects using it. — *end example*]

5 For a class, its non-static data members, its non-virtual direct base classes, and, if the class is not abstract (10.4), its virtual base classes are called its *potentially constructed subobjects*.

12.1 Constructors [class.ctor]

1 Constructors do not have names. In a declaration of a constructor, the *declarator* is a function declarator (8.3.5) of the form

ptr-declarator `(` *parameter-declaration-clause* `)` *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$

where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

(1.1) — in a *member-declaration* that belongs to the *member-specification* of a class but is not a friend declaration (11.3), the *id-expression* is the injected-class-name (Clause 9) of the immediately-enclosing class;

(1.2) — in a *member-declaration* that belongs to the *member-specification* of a class template but is not a friend declaration, the *id-expression* is a *class-name* that names the current instantiation (14.6.2.1) of the immediately-enclosing class template; or

(1.3) — in a declaration at namespace scope or in a friend declaration, the *id-expression* is a *qualified-id* that names a constructor (3.4.3.1).

The *class-name* shall not be a *typedef-name*. In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, `explicit`, or `constexpr`. [*Example:*

```
struct S {
  S();              // declares the constructor
};

S::S() { }          // defines the constructor
```

— *end example*]

2 A constructor is used to initialize objects of its class type. Because constructors do not have names, they are never found during name lookup; however an explicit type conversion using the functional notation (5.2.3) will cause a constructor to be called to initialize an object. [*Note:* For initialization of objects of class type see 12.6. — *end note*]

3 A constructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics (7.1.6.1) are not applied on an object under construction. They come into effect when the constructor for the most derived object (1.8) ends.

4 A *default* constructor for a class `X` is a constructor of class `X` that either has no parameters or else each parameter that is not a function parameter pack has a default argument. If there is no user-declared constructor for class `X`, a non-explicit constructor having no parameters is implicitly declared as defaulted (8.4). An implicitly-declared default constructor is an `inline public` member of its class. A defaulted default constructor for class `X` is defined as deleted if:

(4.1) — `X` is a union-like class that has a variant member with a non-trivial default constructor,

(4.2) — any non-static data member with no *brace-or-equal-initializer* is of reference type,

(4.3) — any non-variant non-static data member of const-qualified type (or array thereof) with no *brace-or-equal-initializer* does not have a user-provided default constructor,

(4.4) — `X` is a union and all of its variant members are of const-qualified type (or array thereof),

(4.5) — `X` is a non-union class and all members of any anonymous union member are of const-qualified type (or array thereof),

(4.6) — any potentially constructed subobject, except for a non-static data member with a *brace-or-equal-initializer*, has class type `M` (or array thereof) and either `M` has no default constructor or overload resolution (13.3) as applied to `M`'s default constructor results in an ambiguity or in a function that is deleted or inaccessible from the defaulted default constructor, or

(4.7) — any potentially constructed subobject has a type with a destructor that is deleted or inaccessible from the defaulted default constructor.

A default constructor is trivial if it is not user-provided and if:

(4.8) — its class has no virtual functions (10.3) and no virtual base classes (10.1), and

(4.9) — no non-static data member of its class has a *brace-or-equal-initializer*, and

(4.10) — all the direct base classes of its class have trivial default constructors, and

(4.11) — for all the non-static data members of its class that are of class type (or array thereof), each such class has a trivial default constructor.

Otherwise, the default constructor is *non-trivial*.

5 A default constructor that is defaulted and not defined as deleted is *implicitly defined* when it is odr-used (3.2) to create an object of its class type (1.8) or when it is explicitly defaulted after its first declaration. The implicitly-defined default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no *ctor-initializer* (12.6.2) and an empty *compound-statement*. If that user-written default constructor would be ill-formed, the program is ill-formed. If that user-written default constructor would satisfy the requirements of a `constexpr` constructor (7.1.5), the implicitly-defined default constructor is `constexpr`. Before the defaulted default constructor for a class is implicitly defined, all the non-user-provided default constructors for its base classes and its non-static data members shall have been implicitly defined. [*Note:* An implicitly-declared default constructor has an exception specification (15.4). An explicitly-defaulted definition might have an implicit exception specification, see 8.4. — *end note*]

6 Default constructors are called implicitly to create class objects of static, thread, or automatic storage duration (3.7.1, 3.7.2, 3.7.3) defined without an initializer (8.5), are called to create class objects of dynamic storage duration (3.7.4) created by a *new-expression* in which the *new-initializer* is omitted (5.3.4), or are called when the explicit type conversion syntax (5.2.3) is used. A program is ill-formed if the default constructor for an object is implicitly used and the constructor is not accessible (Clause 11).

7 [*Note:* 12.6.2 describes the order in which constructors for base classes and non-static data members are called and describes how arguments can be specified for the calls to these constructors. — *end note*]

8 A `return` statement in the body of a constructor shall not specify a return value. The address of a constructor shall not be taken.

9 A functional notation type conversion (5.2.3) can be used to create new objects of its type. [*Note:* The syntax looks like an explicit call of the constructor. — *end note*] [*Example:*

```
complex zz = complex(1,2.3);
cprint( complex(7.8,1.2) );
```

— *end example*]

10 An object created in this way is unnamed. [*Note:* 12.2 describes the lifetime of temporary objects. — *end note*] [*Note:* Explicit constructor calls do not yield lvalues, see 3.10. — *end note*]

11 [*Note:* some language constructs have special semantics when used during construction; see 12.6.2 and 12.7. — *end note*]

12 During the construction of a `const` object, if the value of the object or any of its subobjects is accessed through a glvalue that is not obtained, directly or indirectly, from the constructor's `this` pointer, the value of the object or subobject thus obtained is unspecified. [*Example:*

```
struct C;
void no_opt(C*);

struct C {
  int c;
  C() : c(0) { no_opt(this); }
};

const C cobj;

void no_opt(C* cptr) {
  int i = cobj.c * 100;          // value of cobj.c is unspecified
  cptr->c = 1;
  cout << cobj.c * 100           // value of cobj.c is unspecified
       << '\n';
```

```
}
```

— *end example*]

12.2 Temporary objects [class.temporary]

1 Temporaries of class type are created in various contexts: binding a reference to a prvalue (8.5.3), returning a prvalue (6.6.3), a conversion that creates a prvalue (4.1, 5.2.9, 5.2.11, 5.4), throwing an exception (15.1), and in some initializations (8.5). [*Note:* The lifetime of exception objects is described in 15.1. — *end note*] Even when the creation of the temporary object is unevaluated (Clause 5) or otherwise avoided (12.8), all the semantic restrictions shall be respected as if the temporary object had been created and later destroyed. [*Note:* This includes accessibility (11) and whether it is deleted, for the constructor selected and for the destructor. However, in the special case of a function call used as the operand of a *decltype-specifier* (5.2.2), no temporary is introduced, so the foregoing does not apply to the prvalue of any such function call. — *end note*]

2 [*Example:* Consider the following code:

```
class X {
public:
  X(int);
  X(const X&);
  X& operator=(const X&);
  ~X();
};

class Y {
public:
  Y(int);
  Y(Y&&);
  ~Y();
};

X f(X);
Y g(Y);

void h() {
  X a(1);
  X b = f(X(2));
  Y c = g(Y(3));
  a = f(a);
}
```

An implementation might use a temporary in which to construct `X(2)` before passing it to `f()` using `X`'s copy constructor; alternatively, `X(2)` might be constructed in the space used to hold the argument. Likewise, an implementation might use a temporary in which to construct `Y(3)` before passing it to `g()` using `Y`'s move constructor; alternatively, `Y(3)` might be constructed in the space used to hold the argument. Also, a temporary might be used to hold the result of `f(X(2))` before copying it to `b` using `X`'s copy constructor; alternatively, `f()`'s result might be constructed in `b`. Likewise, a temporary might be used to hold the result of `g(Y(3))` before moving it to `c` using `Y`'s move constructor; alternatively, `g()`'s result might be constructed in `c`. On the other hand, the expression `a=f(a)` requires a temporary for the result of `f(a)`, which is then assigned to `a`. — *end example*]

3 When an implementation introduces a temporary object of a class that has a non-trivial constructor (12.1, 12.8), it shall ensure that a constructor is called for the temporary object. Similarly, the destructor shall be called for a temporary with a non-trivial destructor (12.4). Temporary objects are destroyed as the last step

in evaluating the full-expression (1.9) that (lexically) contains the point where they were created. This is true even if that evaluation ends in throwing an exception. The value computations and side effects of destroying a temporary object are associated only with the full-expression, not with any specific subexpression.

4 There are two contexts in which temporaries are destroyed at a different point than the end of the full-expression. The first context is when a default constructor is called to initialize an element of an array. If the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.

5 The second context is when a reference is bound to a temporary.[117] The temporary to which the reference is bound or the temporary that is the complete object of a subobject to which the reference is bound persists for the lifetime of the reference except:

(5.1) — A temporary object bound to a reference parameter in a function call (5.2.2) persists until the completion of the full-expression containing the call.

(5.2) — The lifetime of a temporary bound to the returned value in a function return statement (6.6.3) is not extended; the temporary is destroyed at the end of the full-expression in the return statement.

(5.3) — A temporary bound to a reference in a *new-initializer* (5.3.4) persists until the completion of the full-expression containing the *new-initializer*. [*Example:*

```
struct S { int mi; const std::pair<int,int>& mp; };
S a { 1, {2,3} };
S* p = new S{ 1, {2,3} };    // Creates dangling reference
```

— *end example*] [*Note:* This may introduce a dangling reference, and implementations are encouraged to issue a warning in such a case. — *end note*]

The destruction of a temporary whose lifetime is not extended by being bound to a reference is sequenced before the destruction of every temporary which is constructed earlier in the same full-expression. If the lifetime of two or more temporaries to which references are bound ends at the same point, these temporaries are destroyed at that point in the reverse order of the completion of their construction. In addition, the destruction of temporaries bound to references shall take into account the ordering of destruction of objects with static, thread, or automatic storage duration (3.7.1, 3.7.2, 3.7.3); that is, if `obj1` is an object with the same storage duration as the temporary and created before the temporary is created the temporary shall be destroyed before `obj1` is destroyed; if `obj2` is an object with the same storage duration as the temporary and created after the temporary is created the temporary shall be destroyed after `obj2` is destroyed. [*Example:*

```
struct S {
  S();
  S(int);
  friend S operator+(const S&, const S&);
  ~S();
};
S obj1;
const S& cr = S(16)+S(23);
S obj2;
```

the expression `S(16) + S(23)` creates three temporaries: a first temporary `T1` to hold the result of the expression `S(16)`, a second temporary `T2` to hold the result of the expression `S(23)`, and a third temporary `T3` to hold the result of the addition of these two expressions. The temporary `T3` is then bound to the reference `cr`. It is unspecified whether `T1` or `T2` is created first. On an implementation where `T1` is created before `T2`, `T2` shall be destroyed before `T1`. The temporaries `T1` and `T2` are bound to the reference parameters of

117) The same rules apply to initialization of an `initializer_list` object (8.5.4) with its underlying temporary array

`operator+`; these temporaries are destroyed at the end of the full-expression containing the call to `operator+`. The temporary `T3` bound to the reference `cr` is destroyed at the end of `cr`'s lifetime, that is, at the end of the program. In addition, the order in which `T3` is destroyed takes into account the destruction order of other objects with static storage duration. That is, because `obj1` is constructed before `T3`, and `T3` is constructed before `obj2`, `obj2` shall be destroyed before `T3`, and `T3` shall be destroyed before `obj1`. — *end example*]

12.3 Conversions [class.conv]

1 Type conversions of class objects can be specified by constructors and by conversion functions. These conversions are called *user-defined conversions* and are used for implicit type conversions (Clause 4), for initialization (8.5), and for explicit type conversions (5.4, 5.2.9).

2 User-defined conversions are applied only where they are unambiguous (10.2, 12.3.2). Conversions obey the access control rules (Clause 11). Access control is applied after ambiguity resolution (3.4).

3 [*Note:* See 13.3 for a discussion of the use of conversions in function calls as well as examples below. — *end note*]

4 At most one user-defined conversion (constructor or conversion function) is implicitly applied to a single value.

[*Example:*

```
struct X {
  operator int();
};

struct Y {
  operator X();
};

Y a;
int b = a;          // error
                    // a.operator X().operator int() not tried
int c = X(a);       // OK: a.operator X().operator int()
```

— *end example*]

5 User-defined conversions are used implicitly only if they are unambiguous. A conversion function in a derived class does not hide a conversion function in a base class unless the two functions convert to the same type. Function overload resolution (13.3.3) selects the best conversion function to perform the conversion. [*Example:*

```
struct X {
  operator int();
};

struct Y : X {
    operator char();
};

void f(Y& a) {
  if (a) {          // ill-formed:
                    // X::operator int() or Y::operator char()
  }
}
```

— *end example*]

12.3.1 Conversion by constructor [class.conv.ctor]

1 A constructor declared without the *function-specifier* `explicit` specifies a conversion from the types of its parameters to the type of its class. Such a constructor is called a *converting constructor*. [*Example:*

```
struct X {
    X(int);
    X(const char*, int =0);
    X(int, int);
};

void f(X arg) {
  X a = 1;          // a = X(1)
  X b = "Jessie";   // b = X("Jessie",0)
  a = 2;            // a = X(2)
  f(3);             // f(X(3))
  f({1, 2});        // f(X(1,2))
}
```

— *end example*]

2 [*Note:* An explicit constructor constructs objects just like non-explicit constructors, but does so only where the direct-initialization syntax (8.5) or where casts (5.2.9, 5.4) are explicitly used; see also 13.3.1.4. A default constructor may be an explicit constructor; such a constructor will be used to perform default-initialization or value-initialization (8.5). [*Example:*

```
struct Z {
  explicit Z();
  explicit Z(int);
  explicit Z(int, int);
};

Z a;                            // OK: default-initialization performed
Z a1 = 1;                       // error: no implicit conversion
Z a3 = Z(1);                    // OK: direct initialization syntax used
Z a2(1);                        // OK: direct initialization syntax used
Z* p = new Z(1);                // OK: direct initialization syntax used
Z a4 = (Z)1;                    // OK: explicit cast used
Z a5 = static_cast<Z>(1);       // OK: explicit cast used
Z a6 = { 3, 4 };                // error: no implicit conversion
```

— *end example*] — *end note*]

3 A non-explicit copy/move constructor (12.8) is a converting constructor. [*Note:* An implicitly-declared copy/move constructor is not an explicit constructor; it may be called for implicit type conversions. — *end note*]

12.3.2 Conversion functions [class.conv.fct]

1 A member function of a class `X` having no parameters with a name of the form

conversion-function-id:
`operator` *conversion-type-id*

conversion-type-id:
type-specifier-seq conversion-declarator$_{opt}$

conversion-declarator:
ptr-operator conversion-declarator$_{opt}$

specifies a conversion from `X` to the type specified by the *conversion-type-id*. Such functions are called conversion functions. No return type can be specified. If a conversion function is a member function, the type of the conversion function (8.3.5) is "function taking no parameter returning *conversion-type-id*". A conversion function is never used to convert a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to (possibly cv-qualified) void.[118]

[*Example:*

```
struct X {
  operator int();
};

void f(X a) {
  int i = int(a);
  i = (int)a;
  i = a;
}
```

In all three cases the value assigned will be converted by `X::operator int()`. — *end example*]

2 A conversion function may be explicit (7.1.2), in which case it is only considered as a user-defined conversion for direct-initialization (8.5). Otherwise, user-defined conversions are not restricted to use in assignments and initializations. [*Example:*

```
class Y { };
struct Z {
  explicit operator Y() const;
};

void h(Z z) {
  Y y1(z);          // OK: direct-initialization
  Y y2 = z;         // ill-formed: copy-initialization
  Y y3 = (Y)z;      // OK: cast notation
}

void g(X a, X b) {
  int i = (a) ? 1+a : 0;
  int j = (a&&b) ? a+b : i;
  if (a) {
  }
}
```

— *end example*]

3 The *conversion-type-id* shall not represent a function type nor an array type. The *conversion-type-id* in a *conversion-function-id* is the longest possible sequence of *conversion-declarator*s. [*Note:* This prevents ambiguities between the declarator operator * and its expression counterparts. [*Example:*

```
&ac.operator int*i; // syntax error:
                    // parsed: &(ac.operator int *)i
                    // not as: &(ac.operator int)*i
```

118) These conversions are considered as standard conversions for the purposes of overload resolution (13.3.3.1, 13.3.3.1.4) and therefore initialization (8.5) and explicit casts (5.2.9). A conversion to `void` does not invoke any conversion function (5.2.9). Even though never directly called to perform a conversion, such conversion functions can be declared and can potentially be reached through a call to a virtual conversion function in a base class.

The `*` is the pointer declarator and not the multiplication operator. — *end example*] — *end note*]

4 Conversion functions are inherited.

5 Conversion functions can be virtual.

6 Conversion functions cannot be declared `static`.

7 A conversion function template shall not have a deduced return type (7.1.6.4).

12.4 Destructors [class.dtor]

1 In a declaration of a destructor, the *declarator* is a function declarator (8.3.5) of the form

ptr-declarator `(` *parameter-declaration-clause* `)` *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$

where the *ptr-declarator* consists solely of an *id-expression*, an optional *attribute-specifier-seq*, and optional surrounding parentheses, and the *id-expression* has one of the following forms:

(1.1) — in a *member-declaration* that belongs to the *member-specification* of a class but is not a friend declaration (11.3), the *id-expression* is `~`*class-name* and the *class-name* is the injected-class-name (Clause 9) of the immediately-enclosing class;

(1.2) — in a *member-declaration* that belongs to the *member-specification* of a class template but is not a friend declaration, the *id-expression* is `~`*class-name* and the *class-name* names the current instantiation (14.6.2.1) of the immediately-enclosing class template; or

(1.3) — in a declaration at namespace scope or in a friend declaration, the *id-expression* is *nested-name-specifier* `~`*class-name* and the *class-name* names the same class as the *nested-name-specifier*.

The *class-name* shall not be a *typedef-name*. A destructor shall take no arguments (8.3.5). In a destructor declaration, each *decl-specifier* of the optional *decl-specifier-seq* shall be `friend`, `inline`, or `virtual`.

2 A destructor is used to destroy objects of its class type. The address of a destructor shall not be taken. A destructor can be invoked for a `const`, `volatile` or `const volatile` object. `const` and `volatile` semantics (7.1.6.1) are not applied on an object under destruction. They stop being in effect when the destructor for the most derived object (1.8) starts.

3 A declaration of a destructor that does not have an *exception-specification* has the same exception specification as if had been implicitly declared (15.4).

4 If a class has no user-declared destructor, a destructor is implicitly declared as defaulted (8.4). An implicitly-declared destructor is an `inline public` member of its class.

5 A defaulted destructor for a class `X` is defined as deleted if:

(5.1) — `X` is a union-like class that has a variant member with a non-trivial destructor,

(5.2) — any potentially constructed subobject has class type `M` (or array thereof) and `M` has a deleted destructor or a destructor that is inaccessible from the defaulted destructor,

(5.3) — or, for a virtual destructor, lookup of the non-array deallocation function results in an ambiguity or in a function that is deleted or inaccessible from the defaulted destructor.

A destructor is trivial if it is not user-provided and if:

(5.4) — the destructor is not `virtual`,

(5.5) — all of the direct base classes of its class have trivial destructors, and

(5.6) — for all of the non-static data members of its class that are of class type (or array thereof), each such class has a trivial destructor.

Otherwise, the destructor is *non-trivial*.

6 A destructor that is defaulted and not defined as deleted is *implicitly defined* when it is odr-used (3.2) to destroy an object of its class type (3.7) or when it is explicitly defaulted after its first declaration.

7 Before the defaulted destructor for a class is implicitly defined, all the non-user-provided destructors for its base classes and its non-static data members shall have been implicitly defined.

8 After executing the body of the destructor and destroying any automatic objects allocated within the body, a destructor for class `X` calls the destructors for `X`'s direct non-variant non-static data members, the destructors for `X`'s direct base classes and, if `X` is the type of the most derived class (12.6.2), its destructor calls the destructors for `X`'s virtual base classes. All destructors are called as if they were referenced with a qualified name, that is, ignoring any possible virtual overriding destructors in more derived classes. Bases and members are destroyed in the reverse order of the completion of their constructor (see 12.6.2). A `return` statement (6.6.3) in a destructor might not directly return to the caller; before transferring control to the caller, the destructors for the members and bases are called. Destructors for elements of an array are called in reverse order of their construction (see 12.6).

9 A destructor can be declared `virtual` (10.3) or pure `virtual` (10.4); if any objects of that class or any derived class are created in the program, the destructor shall be defined. If a class has a base class with a virtual destructor, its destructor (whether user- or implicitly-declared) is virtual.

10 [*Note:* some language constructs have special semantics when used during destruction; see 12.7. — *end note*]

11 A destructor is invoked implicitly

(11.1) — for a constructed object with static storage duration (3.7.1) at program termination (3.6.3),

(11.2) — for a constructed object with thread storage duration (3.7.2) at thread exit,

(11.3) — for a constructed object with automatic storage duration (3.7.3) when the block in which an object is created exits (6.7),

(11.4) — for a constructed temporary object when its lifetime ends (12.2).

In each case, the context of the invocation is the context of the construction of the object. A destructor is also invoked implicitly through use of a *delete-expression* (5.3.5) for a constructed object allocated by a *new-expression* (5.3.4); the context of the invocation is the *delete-expression*. [*Note:* An array of class type contains several subobjects for each of which the destructor is invoked. — *end note*] A destructor can also be invoked explicitly. A destructor is *potentially invoked* if it is invoked or as specified in 5.3.4 and 12.6.2. A program is ill-formed if a destructor that is potentially invoked is deleted or not accessible from the context of the invocation.

12 At the point of definition of a virtual destructor (including an implicit definition (12.8)), the non-array deallocation function is determined as if for the expression `delete this` appearing in a non-virtual destructor of the destructor's class (see 5.3.5). If the lookup fails or if the deallocation function has a deleted definition (8.4), the program is ill-formed. [*Note:* This assures that a deallocation function corresponding to the dynamic type of an object is available for the *delete-expression* (12.5). — *end note*]

13 In an explicit destructor call, the destructor name appears as a `~` followed by a *type-name* or *decltype-specifier* that denotes the destructor's class type. The invocation of a destructor is subject to the usual rules for member functions (9.3); that is, if the object is not of the destructor's class type and not of a class derived from the destructor's class type (including when the destructor is invoked via a null pointer value), the program has undefined behavior. [*Note:* invoking `delete` on a null pointer does not call the destructor; see 5.3.5. — *end note*] [*Example:*

```
struct B {
  virtual ~B() { }
```

```
};
struct D : B {
  ~D() { }
};

D D_object;
typedef B B_alias;
B* B_ptr = &D_object;

void f() {
  D_object.B::~B();              // calls B's destructor
  B_ptr->~B();                   // calls D's destructor
  B_ptr->~B_alias();             // calls D's destructor
  B_ptr->B_alias::~B();          // calls B's destructor
  B_ptr->B_alias::~B_alias();    // calls B's destructor
}
```

— *end example*] [*Note:* An explicit destructor call must always be written using a member access operator (5.2.5) or a qualified-id (5.1); in particular, the *unary-expression* `~X()` in a member function is not an explicit destructor call (5.3.1). — *end note*]

14 [*Note:* explicit calls of destructors are rarely needed. One use of such calls is for objects placed at specific addresses using a placement *new-expression*. Such use of explicit placement and destruction of objects can be necessary to cope with dedicated hardware resources and for writing memory management facilities. For example,

```
void* operator new(std::size_t, void* p) { return p; }
struct X {
  X(int);
  ~X();
};
void f(X* p);

void g() {                       // rare, specialized use:
  char* buf = new char[sizeof(X)];
  X* p = new(buf) X(222);        // use buf[] and initialize
  f(p);
  p->X::~X();                    // cleanup
}
```

— *end note*]

15 Once a destructor is invoked for an object, the object no longer exists; the behavior is undefined if the destructor is invoked for an object whose lifetime has ended (3.8). [*Example:* if the destructor for an automatic object is explicitly invoked, and the block is subsequently left in a manner that would ordinarily invoke implicit destruction of the object, the behavior is undefined. — *end example*]

16 [*Note:* the notation for explicit call of a destructor can be used for any scalar type name (5.2.4). Allowing this makes it possible to write code without having to know if a destructor exists for a given type. For example,

```
typedef int I;
I* p;
p->I::~I();
```

— *end note*]

12.5 Free store [class.free]

1 Any allocation function for a class `T` is a static member (even if not explicitly declared `static`).

2 [*Example:*

```
class Arena;
struct B {
  void* operator new(std::size_t, Arena*);
};
struct D1 : B {
};

Arena*  ap;
void foo(int i) {
  new (ap) D1;       // calls B::operator new(std::size_t, Arena*)
  new D1[i];         // calls ::operator new[](std::size_t)
  new D1;            // ill-formed: ::operator new(std::size_t) hidden
}
```

— *end example*]

3 When an object is deleted with a *delete-expression* (5.3.5), a *deallocation function* (`operator delete()` for non-array objects or `operator delete[]()` for arrays) is (implicitly) called to reclaim the storage occupied by the object (3.7.4.2).

4 Class-specific deallocation function lookup is a part of general deallocation function lookup (5.3.5) and occurs as follows. If the *delete-expression* is used to deallocate a class object whose static type has a virtual destructor, the deallocation function is the one selected at the point of definition of the dynamic type's virtual destructor (12.4).[119] Otherwise, if the *delete-expression* is used to deallocate an object of class `T` or array thereof, the static and dynamic types of the object shall be identical and the deallocation function's name is looked up in the scope of `T`. If this lookup fails to find the name, general deallocation function lookup (5.3.5) continues. If the result of the lookup is ambiguous or inaccessible, or if the lookup selects a placement deallocation function, the program is ill-formed.

5 Any deallocation function for a class `X` is a static member (even if not explicitly declared `static`). [*Example:*

```
class X {
  void operator delete(void*);
  void operator delete[](void*, std::size_t);
};

class Y {
  void operator delete(void*, std::size_t);
  void operator delete[](void*);
};
```

— *end example*]

6 Since member allocation and deallocation functions are `static` they cannot be virtual. [*Note:* however, when the *cast-expression* of a *delete-expression* refers to an object of class type, because the deallocation function actually called is looked up in the scope of the class that is the dynamic type of the object, if the destructor is virtual, the effect is the same. For example,

```
struct B {
  virtual ~B();
```

119) A similar provision is not needed for the array version of `operator delete` because 5.3.5 requires that in this situation, the static type of the object to be deleted be the same as its dynamic type.

```
  void operator delete(void*, std::size_t);
};

struct D : B {
  void operator delete(void*);
};

void f() {
  B* bp = new D;
  delete bp;        //1: uses D::operator delete(void*)
}
```

Here, storage for the non-array object of class `D` is deallocated by `D::operator delete()`, due to the virtual destructor. — *end note*] [*Note:* Virtual destructors have no effect on the deallocation function actually called when the *cast-expression* of a *delete-expression* refers to an array of objects of class type. For example,

```
struct B {
  virtual ~B();
  void operator delete[](void*, std::size_t);
};

struct D : B {
  void operator delete[](void*, std::size_t);
};

void f(int i) {
  D* dp = new D[i];
  delete [] dp;     // uses D::operator delete[](void*, std::size_t)
  B* bp = new D[i];
  delete[] bp;      // undefined behavior
}
```

— *end note*]

7 Access to the deallocation function is checked statically. Hence, even though a different one might actually be executed, the statically visible deallocation function is required to be accessible. [*Example:* for the call on line //1 above, if `B::operator delete()` had been `private`, the delete expression would have been ill-formed. — *end example*]

8 [*Note:* If a deallocation function has no explicit *exception-specification*, it has a non-throwing exception specification (15.4). — *end note*]

12.6 Initialization [class.init]

1 When no initializer is specified for an object of (possibly cv-qualified) class type (or array thereof), or the initializer has the form `()`, the object is initialized as specified in 8.5.

2 An object of class type (or array thereof) can be explicitly initialized; see 12.6.1 and 12.6.2.

3 When an array of class objects is initialized (either explicitly or implicitly) and the elements are initialized by constructor, the constructor shall be called for each element of the array, following the subscript order; see 8.3.4. [*Note:* Destructors for the array elements are called in reverse order of their construction. — *end note*]

12.6.1 Explicit initialization [class.expl.init]

1 An object of class type can be initialized with a parenthesized *expression-list*, where the *expression-list* is construed as an argument list for a constructor that is called to initialize the object. Alternatively, a single *assignment-expression* can be specified as an *initializer* using the `=` form of initialization. Either direct-initialization semantics or copy-initialization semantics apply; see 8.5. [*Example:*

```
struct complex {
  complex();
  complex(double);
  complex(double,double);
};

complex sqrt(complex,complex);

complex a(1);                   // initialize by a call of
                                // complex(double)
complex b = a;                  // initialize by a copy of a
complex c = complex(1,2);       // construct complex(1,2)
                                // using complex(double,double)
                                // copy/move it into c
complex d = sqrt(b,c);          // call sqrt(complex,complex)
                                // and copy/move the result into d
complex e;                      // initialize by a call of
                                // complex()
complex f = 3;                  // construct complex(3) using
                                // complex(double)
                                // copy/move it into f
complex g = { 1, 2 };           // initialize by a call of
                                // complex(double, double)
```

— *end example*] [*Note:* overloading of the assignment operator (13.5.3) has no effect on initialization. — *end note*]

2 An object of class type can also be initialized by a *braced-init-list*. List-initialization semantics apply; see 8.5 and 8.5.4. [*Example:*

```
complex v[6] = { 1, complex(1,2), complex(), 2 };
```

Here, `complex::complex(double)` is called for the initialization of `v[0]` and `v[3]`, `complex::complex(double, double)` is called for the initialization of `v[1]`, `complex::complex()` is called for the initialization `v[2]`, `v[4]`, and `v[5]`. For another example,

```
struct X {
  int i;
  float f;
  complex c;
} x = { 99, 88.8, 77.7 };
```

Here, `x.i` is initialized with 99, `x.f` is initialized with 88.8, and `complex::complex(double)` is called for the initialization of `x.c`. — *end example*] [*Note:* Braces can be elided in the *initializer-list* for any aggregate, even if the aggregate has members of a class type with user-defined type conversions; see 8.5.1. — *end note*]

3 [*Note:* If `T` is a class type with no default constructor, any declaration of an object of type `T` (or array thereof) is ill-formed if no *initializer* is explicitly specified (see 12.6 and 8.5). — *end note*]

4 [*Note:* the order in which objects with static or thread storage duration are initialized is described in 3.6.2 and 6.7. — *end note*]

12.6.2 Initializing bases and members [class.base.init]

1 In the definition of a constructor for a class, initializers for direct and virtual base subobjects and non-static data members can be specified by a *ctor-initializer*, which has the form

ctor-initializer:
: *mem-initializer-list*
mem-initializer-list:
mem-initializer `...`$_{opt}$
mem-initializer-list `,` *mem-initializer* `...`$_{opt}$
mem-initializer:
mem-initializer-id `(` *expression-list*$_{opt}$`)`
mem-initializer-id braced-init-list
mem-initializer-id:
class-or-decltype
identifier

2 In a *mem-initializer-id* an initial unqualified *identifier* is looked up in the scope of the constructor's class and, if not found in that scope, it is looked up in the scope containing the constructor's definition. [*Note:* If the constructor's class contains a member with the same name as a direct or virtual base class of the class, a *mem-initializer-id* naming the member or base class and composed of a single identifier refers to the class member. A *mem-initializer-id* for the hidden base class may be specified using a qualified name. — *end note*] Unless the *mem-initializer-id* names the constructor's class, a non-static data member of the constructor's class, or a direct or virtual base of that class, the *mem-initializer* is ill-formed.

3 A *mem-initializer-list* can initialize a base class using any *class-or-decltype* that denotes that base class type. [*Example:*

```
struct A { A(); };
  typedef A global_A;
  struct B { };
  struct C: public A, public B { C(); };
  C::C(): global_A() { }        // mem-initializer for base A
```

— *end example*]

4 If a *mem-initializer-id* is ambiguous because it designates both a direct non-virtual base class and an inherited virtual base class, the *mem-initializer* is ill-formed. [*Example:*

```
struct A { A(); };
struct B: public virtual A { };
struct C: public A, public B { C(); };
C::C(): A() { }                 // ill-formed: which A?
```

— *end example*]

5 A *ctor-initializer* may initialize a variant member of the constructor's class. If a *ctor-initializer* specifies more than one *mem-initializer* for the same member or for the same base class, the *ctor-initializer* is ill-formed.

6 A *mem-initializer-list* can delegate to another constructor of the constructor's class using any *class-or-decltype* that denotes the constructor's class itself. If a *mem-initializer-id* designates the constructor's class, it shall be the only *mem-initializer*; the constructor is a *delegating constructor*, and the constructor selected by the *mem-initializer* is the *target constructor*. The *principal constructor* is the first constructor invoked in the construction of an object (that is, not a target constructor for that object's construction). The target constructor is selected by overload resolution. Once the target constructor returns, the body of the delegating constructor is executed. If a constructor delegates to itself directly or indirectly, the program is ill-formed; no diagnostic is required. [*Example:*

```
struct C {
```

```
  C( int ) { }                  // #1: non-delegating constructor
  C(): C(42) { }                // #2: delegates to #1
  C( char c ) : C(42.0) { }     // #3: ill-formed due to recursion with #4
  C( double d ) : C('a') { }    // #4: ill-formed due to recursion with #3
};
```

— *end example*]

7 The *expression-list* or *braced-init-list* in a *mem-initializer* is used to initialize the designated subobject (or, in the case of a delegating constructor, the complete class object) according to the initialization rules of 8.5 for direct-initialization.

[*Example:*

```
struct B1 { B1(int); /* ... */ };
struct B2 { B2(int); /* ... */ };
struct D : B1, B2 {
  D(int);
  B1 b;
  const int c;
};

D::D(int a) : B2(a+1), B1(a+2), c(a+3), b(a+4)
  { /* ... */ }
D d(10);
```

— *end example*] The initialization performed by each *mem-initializer* constitutes a full-expression. Any expression in a *mem-initializer* is evaluated as part of the full-expression that performs the initialization. A *mem-initializer* where the *mem-initializer-id* denotes a virtual base class is ignored during execution of a constructor of any class that is not the most derived class.

8 A temporary expression bound to a reference member in a *mem-initializer* is ill-formed. [*Example:*

```
struct A {
  A() : v(42) { }  // error
  const int& v;
};
```

— *end example*]

9 In a non-delegating constructor, if a given potentially constructed subobject is not designated by a *mem-initializer-id* (including the case where there is no *mem-initializer-list* because the constructor has no *ctor-initializer*), then

(9.1) — if the entity is a non-static data member that has a *brace-or-equal-initializer* and either

(9.1.1) — the constructor's class is a union (9.5), and no other variant member of that union is designated by a *mem-initializer-id* or

(9.1.2) — the constructor's class is not a union, and, if the entity is a member of an anonymous union, no other member of that union is designated by a *mem-initializer-id*,

the entity is initialized as specified in 8.5;

(9.2) — otherwise, if the entity is an anonymous union or a variant member (9.5), no initialization is performed;

(9.3) — otherwise, the entity is default-initialized (8.5).

[*Note:* An abstract class (10.4) is never a most derived class, thus its constructors never initialize virtual base classes, therefore the corresponding *mem-initializer*s may be omitted. — *end note*] An attempt to initialize more than one non-static data member of a union renders the program ill-formed. [*Note:* After the

call to a constructor for class `X` for an object with automatic or dynamic storage duration has completed, if the constructor was not invoked as part of value-initialization and a member of `X` is neither initialized nor given a value during execution of the *compound-statement* of the body of the constructor, the member has an indeterminate value. — *end note*] [*Example:*

```
struct A {
  A();
};

struct B {
  B(int);
};

struct C {
  C() { }                 // initializes members as follows:
  A a;                      // OK: calls A::A()
  const B b;                // error: B has no default constructor
  int i;                    // OK: i has indeterminate value
  int j = 5;                // OK: j has the value 5
};
```

— *end example*]

10 If a given non-static data member has both a *brace-or-equal-initializer* and a *mem-initializer*, the initialization specified by the *mem-initializer* is performed, and the non-static data member's *brace-or-equal-initializer* is ignored. [*Example:* Given

```
struct A {
  int i = /* some integer expression with side  effects  */ ;
  A(int arg) : i(arg) { }
  // ...
};
```

the `A(int)` constructor will simply initialize `i` to the value of `arg`, and the side effects in `i`'s *brace-or-equal-initializer* will not take place. — *end example*]

11 A temporary expression bound to a reference member from a *brace-or-equal-initializer* is ill-formed. [*Example:*

```
struct A {
  A() = default;          // OK
  A(int v) : v(v) { }     // OK
  const int& v = 42;      // OK
};
A a1;                     // error: ill-formed binding of temporary to reference
A a2(1);                  // OK, unfortunately
```

— *end example*]

12 In a non-delegating constructor, the destructor for each potentially constructed subobject of class type is potentially invoked (12.4). [*Note:* This provision ensures that destructors can be called for fully-constructed sub-objects in case an exception is thrown (15.2). — *end note*]

13 In a non-delegating constructor, initialization proceeds in the following order:

(13.1) — First, and only for the constructor of the most derived class (1.8), virtual base classes are initialized in the order they appear on a depth-first left-to-right traversal of the directed acyclic graph of base classes, where "left-to-right" is the order of appearance of the base classes in the derived class *base-specifier-list*.

(13.2) — Then, direct base classes are initialized in declaration order as they appear in the *base-specifier-list* (regardless of the order of the *mem-initializers*).

(13.3) — Then, non-static data members are initialized in the order they were declared in the class definition (again regardless of the order of the *mem-initializers*).

(13.4) — Finally, the *compound-statement* of the constructor body is executed.

[*Note:* The declaration order is mandated to ensure that base and member subobjects are destroyed in the reverse order of initialization. — *end note*]

14 [*Example:*

```
struct V {
  V();
  V(int);
};

struct A : virtual V {
  A();
  A(int);
};

struct B : virtual V {
  B();
  B(int);
};

struct C : A, B, virtual V {
  C();
  C(int);
};

A::A(int i) : V(i) { /* ... */ }
B::B(int i) { /* ... */ }
C::C(int i) { /* ... */ }

V v(1);             // use V(int)
A a(2);             // use V(int)
B b(3);             // use V()
C c(4);             // use V()
```

— *end example*]

15 Names in the *expression-list* or *braced-init-list* of a *mem-initializer* are evaluated in the scope of the constructor for which the *mem-initializer* is specified. [*Example:*

```
class X {
  int a;
  int b;
  int i;
  int j;
public:
  const int& r;
  X(int i): r(a), b(i), i(i), j(this->i) { }
};
```

initializes `X::r` to refer to `X::a`, initializes `X::b` with the value of the constructor parameter `i`, initializes `X::i` with the value of the constructor parameter `i`, and initializes `X::j` with the value of `X::i`; this takes place each time an object of class `X` is created. — *end example*] [*Note:* Because the *mem-initializer* are evaluated in the scope of the constructor, the `this` pointer can be used in the *expression-list* of a *mem-initializer* to refer to the object being initialized. — *end note*]

16 Member functions (including virtual member functions, 10.3) can be called for an object under construction. Similarly, an object under construction can be the operand of the `typeid` operator (5.2.8) or of a `dynamic_cast` (5.2.7). However, if these operations are performed in a *ctor-initializer* (or in a function called directly or indirectly from a *ctor-initializer*) before all the *mem-initializer*s for base classes have completed, the result of the operation is undefined. [*Example:*

```
class A {
public:
  A(int);
};

class B : public A {
  int j;
public:
  int f();
  B() : A(f()),      // undefined: calls member function
                     // but base A not yet initialized
  j(f()) { }         // well-defined: bases are all initialized
};

class C {
public:
  C(int);
};

class D : public B, C {
  int i;
public:
  D() : C(f()),      // undefined: calls member function
                     // but base C not yet initialized
  i(f()) { }         // well-defined: bases are all initialized
};
```

— *end example*]

17 [*Note:* 12.7 describes the result of virtual function calls, `typeid` and `dynamic_cast`s during construction for the well-defined cases; that is, describes the *polymorphic behavior* of an object under construction. — *end note*]

18 A *mem-initializer* followed by an ellipsis is a pack expansion (14.5.3) that initializes the base classes specified by a pack expansion in the *base-specifier-list* for the class. [*Example:*

```
template<class... Mixins>
class X : public Mixins... {
public:
  X(const Mixins&... mixins) : Mixins(mixins)... { }
};
```

— *end example*]

12.7 Construction and destruction [class.cdtor]

1 For an object with a non-trivial constructor, referring to any non-static member or base class of the object before the constructor begins execution results in undefined behavior. For an object with a non-trivial destructor, referring to any non-static member or base class of the object after the destructor finishes execution results in undefined behavior. [*Example:*

```
struct X { int i; };
struct Y : X { Y(); };                  // non-trivial
struct A { int a; };
struct B : public A { int j; Y y; };    // non-trivial

extern B bobj;
B* pb = &bobj;                          // OK
int* p1 = &bobj.a;                      // undefined, refers to base class member
int* p2 = &bobj.y.i;                    // undefined, refers to member's member

A* pa = &bobj;                          // undefined, upcast to a base class type
B bobj;                                 // definition of bobj

extern X xobj;
int* p3 = &xobj.i;                      //OK, X is a trivial class
X xobj;
```

2 For another example,

```
struct W { int j; };
struct X : public virtual W { };
struct Y {
  int* p;
  X x;
  Y() : p(&x.j) {   // undefined, x is not yet constructed
    }
};
```

— *end example*]

3 To explicitly or implicitly convert a pointer (a glvalue) referring to an object of class `X` to a pointer (reference) to a direct or indirect base class `B` of `X`, the construction of `X` and the construction of all of its direct or indirect bases that directly or indirectly derive from `B` shall have started and the destruction of these classes shall not have completed, otherwise the conversion results in undefined behavior. To form a pointer to (or access the value of) a direct non-static member of an object `obj`, the construction of `obj` shall have started and its destruction shall not have completed, otherwise the computation of the pointer value (or accessing the member value) results in undefined behavior. [*Example:*

```
struct A { };
struct B : virtual A { };
struct C : B { };
struct D : virtual A { D(A*); };
struct X { X(A*); };

struct E : C, D, X {
  E() : D(this),    // undefined: upcast from E* to A*
                    // might use path E* → D* → A*
                    // but D is not constructed
                    // D((C*)this), // defined:
                    // E* → C* defined because E() has started
```

```
                    // and C* → A* defined because
                    // C fully constructed
  X(this) {         // defined: upon construction of X,
                    // C/B/D/A sublattice is fully constructed
  }
};
```

— *end example*]

4 Member functions, including virtual functions (10.3), can be called during construction or destruction (12.6.2). When a virtual function is called directly or indirectly from a constructor or from a destructor, including during the construction or destruction of the class's non-static data members, and the object to which the call applies is the object (call it `x`) under construction or destruction, the function called is the final overrider in the constructor's or destructor's class and not one overriding it in a more-derived class. If the virtual function call uses an explicit class member access (5.2.5) and the object expression refers to the complete object of `x` or one of that object's base class subobjects but not `x` or one of its base class subobjects, the behavior is undefined. [*Example:*

```
struct V {
  virtual void f();
  virtual void g();
};

struct A : virtual V {
  virtual void f();
};

struct B : virtual V {
  virtual void g();
  B(V*, A*);
};

struct D : A, B {
  virtual void f();
  virtual void g();
  D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
  f();              // calls V::f, not A::f
  g();              // calls B::g, not D::g
  v->g();           // v is base of B, the call is well-defined, calls B::g
  a->f();           // undefined behavior, a's type not a base of B
}
```

— *end example*]

5 The `typeid` operator (5.2.8) can be used during construction or destruction (12.6.2). When `typeid` is used in a constructor (including the *mem-initializer* or *brace-or-equal-initializer* for a non-static data member) or in a destructor, or used in a function called (directly or indirectly) from a constructor or destructor, if the operand of `typeid` refers to the object under construction or destruction, `typeid` yields the `std::type_info` object representing the constructor or destructor's class. If the operand of `typeid` refers to the object under construction or destruction and the static type of the operand is neither the constructor or destructor's class nor one of its bases, the result of `typeid` is undefined.

6 `dynamic_casts` (5.2.7) can be used during construction or destruction (12.6.2). When a `dynamic_cast` is used in a constructor (including the *mem-initializer* or *brace-or-equal-initializer* for a non-static data member) or in a destructor, or used in a function called (directly or indirectly) from a constructor or destructor, if the operand of the `dynamic_cast` refers to the object under construction or destruction, this object is considered to be a most derived object that has the type of the constructor or destructor's class. If the operand of the `dynamic_cast` refers to the object under construction or destruction and the static type of the operand is not a pointer to or object of the constructor or destructor's own class or one of its bases, the `dynamic_cast` results in undefined behavior.

[*Example:*

```
struct V {
  virtual void f();
};

struct A : virtual V { };

struct B : virtual V {
  B(V*, A*);
};

struct D : A, B {
  D() : B((A*)this, this) { }
};

B::B(V* v, A* a) {
  typeid(*this);                // type_info for B
  typeid(*v);                   // well-defined: *v has type V, a base of B
                                // yields type_info for B
  typeid(*a);                   // undefined behavior: type A not a base of B
  dynamic_cast<B*>(v);          // well-defined: v of type V*, V base of B
                                // results in B*
  dynamic_cast<B*>(a);          // undefined behavior,
                                // a has type A*, A not a base of B
}
```

— *end example*]

12.8 Copying and moving class objects [class.copy]

1 A class object can be copied or moved in two ways: by initialization (12.1, 8.5), including for function argument passing (5.2.2) and for function value return (6.6.3); and by assignment (5.18). Conceptually, these two operations are implemented by a copy/move constructor (12.1) and copy/move assignment operator (13.5.3).

2 A non-template constructor for class `X` is a copy constructor if its first parameter is of type `X&`, `const X&`, `volatile X&` or `const volatile X&`, and either there are no other parameters or else all other parameters have default arguments (8.3.6). [*Example:* `X::X(const X&)` and `X::X(X&,int=1)` are copy constructors.

```
struct X {
  X(int);
  X(const X&, int = 1);
};
X a(1);             // calls X(int);
X b(a, 0);          // calls X(const X&, int);
X c = b;            // calls X(const X&, int);
```

— *end example*]

3 A non-template constructor for class `X` is a move constructor if its first parameter is of type `X&&`, `const X&&`, `volatile X&&`, or `const volatile X&&`, and either there are no other parameters or else all other parameters have default arguments (8.3.6). [*Example:* `Y::Y(Y&&)` is a move constructor.

```
struct Y {
  Y(const Y&);
  Y(Y&&);
};
extern Y f(int);
Y d(f(1));          // calls Y(Y&&)
Y e = d;            // calls Y(const Y&)
```

— *end example*]

4 [*Note:* All forms of copy/move constructor may be declared for a class. [*Example:*

```
struct X {
  X(const X&);
  X(X&);            // OK
  X(X&&);
  X(const X&&);     // OK, but possibly not sensible
};
```

— *end example*] — *end note*]

5 [*Note:* If a class `X` only has a copy constructor with a parameter of type `X&`, an initializer of type `const X` or `volatile X` cannot initialize an object of type (possibly cv-qualified) `X`. [*Example:*

```
struct X {
  X();              // default constructor
  X(X&);            // copy constructor with a nonconst parameter
};
const X cx;
X x = cx;           // error: X::X(X&) cannot copy cx into x
```

— *end example*] — *end note*]

6 A declaration of a constructor for a class `X` is ill-formed if its first parameter is of type (optionally cv-qualified) `X` and either there are no other parameters or else all other parameters have default arguments. A member function template is never instantiated to produce such a constructor signature. [*Example:*

```
struct S {
  template<typename T> S(T);
  S();
};

S g;

void h() {
  S a(g);           // does not instantiate the member template to produce S::S<S>(S);
                    // uses the implicitly declared copy constructor
}
```

— *end example*]

7 If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defined as defaulted (8.4). The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor.

8 The implicitly-declared copy constructor for a class `X` will have the form

```
X::X(const X&)
```

if each potentially constructed subobject of a class type `M` (or array thereof) has a copy constructor whose first parameter is of type `const M&` or `const volatile M&`.[120] Otherwise, the implicitly-declared copy constructor will have the form

```
X::X(X&)
```

9 If the definition of a class `X` does not explicitly declare a move constructor, a non-explicit one will be implicitly declared as defaulted if and only if

(9.1) — `X` does not have a user-declared copy constructor,

(9.2) — `X` does not have a user-declared copy assignment operator,

(9.3) — `X` does not have a user-declared move assignment operator, and

(9.4) — `X` does not have a user-declared destructor.

[*Note:* When the move constructor is not implicitly declared or explicitly supplied, expressions that otherwise would have invoked the move constructor may instead invoke a copy constructor. — *end note*]

10 The implicitly-declared move constructor for class `X` will have the form

```
X::X(X&&)
```

11 An implicitly-declared copy/move constructor is an `inline public` member of its class. A defaulted copy/move constructor for a class `X` is defined as deleted (8.4.3) if `X` has:

(11.1) — a variant member with a non-trivial corresponding constructor and `X` is a union-like class,

(11.2) — a potentially constructed subobject type `M` (or array thereof) that cannot be copied/moved because overload resolution (13.3), as applied to `M`'s corresponding constructor, results in an ambiguity or a function that is deleted or inaccessible from the defaulted constructor,

(11.3) — any potentially constructed subobject of a type with a destructor that is deleted or inaccessible from the defaulted constructor, or,

(11.4) — for the copy constructor, a non-static data member of rvalue reference type.

A defaulted move constructor that is defined as deleted is ignored by overload resolution (13.3, 13.4). [*Note:* A deleted move constructor would otherwise interfere with initialization from an rvalue which can use the copy constructor instead. — *end note*]

12 A copy/move constructor for class `X` is trivial if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if

(12.1) — class `X` has no virtual functions (10.3) and no virtual base classes (10.1), and

(12.2) — class `X` has no non-static data members of volatile-qualified type, and

(12.3) — the constructor selected to copy/move each direct base class subobject is trivial, and

(12.4) — for each non-static data member of `X` that is of class type (or array thereof), the constructor selected to copy/move that member is trivial;

120) This implies that the reference parameter of the implicitly-declared copy constructor cannot bind to a `volatile` lvalue; see C.1.9.

otherwise the copy/move constructor is *non-trivial*.

13 A copy/move constructor that is defaulted and not defined as deleted is *implicitly defined* if it is odr-used (3.2) or when it is explicitly defaulted after its first declaration. [*Note:* The copy/move constructor is implicitly defined even if the implementation elided its odr-use (3.2, 12.2). — *end note*] If the implicitly-defined constructor would satisfy the requirements of a `constexpr` constructor (7.1.5), the implicitly-defined constructor is `constexpr`.

14 Before the defaulted copy/move constructor for a class is implicitly defined, all non-user-provided copy/-move constructors for its potentially constructed subobjects shall have been implicitly defined. [*Note:* An implicitly-declared copy/move constructor has an implied exception specification (15.4). — *end note*]

15 The implicitly-defined copy/move constructor for a non-union class `X` performs a memberwise copy/move of its bases and members. [*Note:* *brace-or-equal-initializer*s of non-static data members are ignored. See also the example in 12.6.2. — *end note*] The order of initialization is the same as the order of initialization of bases and members in a user-defined constructor (see 12.6.2). Let `x` be either the parameter of the constructor or, for the move constructor, an xvalue referring to the parameter. Each base or non-static data member is copied/moved in the manner appropriate to its type:

(15.1) — if the member is an array, each element is direct-initialized with the corresponding subobject of `x`;

(15.2) — if a member `m` has rvalue reference type `T&&`, it is direct-initialized with `static_cast<T&&>(x.m)`;

(15.3) — otherwise, the base or member is direct-initialized with the corresponding base or member of `x`.

Virtual base class subobjects shall be initialized only once by the implicitly-defined copy/move constructor (see 12.6.2).

16 The implicitly-defined copy/move constructor for a union `X` copies the object representation (3.9) of `X`.

17 A user-declared *copy* assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one parameter of type `X`, `X&`, `const X&`, `volatile X&` or `const volatile X&`.[121] [*Note:* An overloaded assignment operator must be declared to have only one parameter; see 13.5.3. — *end note*] [*Note:* More than one form of copy assignment operator may be declared for a class. — *end note*] [*Note:* If a class `X` only has a copy assignment operator with a parameter of type `X&`, an expression of type const `X` cannot be assigned to an object of type `X`. [*Example:*

```
struct X {
  X();
  X& operator=(X&);
};
const X cx;
X x;
void f() {
  x = cx;             // error: X::operator=(X&) cannot assign cx into x
}
```

— *end example*] — *end note*]

18 If the class definition does not explicitly declare a copy assignment operator, one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is defined as defaulted (8.4). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor. The implicitly-declared copy assignment operator for a class `X` will have the form

121) Because a template assignment operator or an assignment operator taking an rvalue reference parameter is never a copy assignment operator, the presence of such an assignment operator does not suppress the implicit declaration of a copy assignment operator. Such assignment operators participate in overload resolution with other assignment operators, including copy assignment operators, and, if selected, will be used to assign an object.

```
X& X::operator=(const X&)
```

if

(18.1) — each direct base class `B` of `X` has a copy assignment operator whose parameter is of type `const B&`, `const volatile B&` or `B`, and

(18.2) — for all the non-static data members of `X` that are of a class type `M` (or array thereof), each such class type has a copy assignment operator whose parameter is of type `const M&`, `const volatile M&` or `M`.[122]

Otherwise, the implicitly-declared copy assignment operator will have the form

```
X& X::operator=(X&)
```

19 A user-declared move assignment operator `X::operator=` is a non-static non-template member function of class `X` with exactly one parameter of type `X&&`, `const X&&`, `volatile X&&`, or `const volatile X&&`. [*Note:* An overloaded assignment operator must be declared to have only one parameter; see 13.5.3. — *end note*] [*Note:* More than one form of move assignment operator may be declared for a class. — *end note*]

20 If the definition of a class `X` does not explicitly declare a move assignment operator, one will be implicitly declared as defaulted if and only if

(20.1) — `X` does not have a user-declared copy constructor,

(20.2) — `X` does not have a user-declared move constructor,

(20.3) — `X` does not have a user-declared copy assignment operator, and

(20.4) — `X` does not have a user-declared destructor.

[*Example:* The class definition

```
struct S {
  int a;
  S& operator=(const S&) = default;
};
```

will not have a default move assignment operator implicitly declared because the copy assignment operator has been user-declared. The move assignment operator may be explicitly defaulted.

```
struct S {
  int a;
  S& operator=(const S&) = default;
  S& operator=(S&&) = default;
};
```

— *end example*]

21 The implicitly-declared move assignment operator for a class `X` will have the form

```
X& X::operator=(X&&);
```

22 The implicitly-declared copy/move assignment operator for class `X` has the return type `X&`; it returns the object for which the assignment operator is invoked, that is, the object assigned to. An implicitly-declared copy/move assignment operator is an `inline public` member of its class.

23 A defaulted copy/move assignment operator for class `X` is defined as deleted if `X` has:

122) This implies that the reference parameter of the implicitly-declared copy assignment operator cannot bind to a `volatile` lvalue; see C.1.9.

(23.1) — a variant member with a non-trivial corresponding assignment operator and `X` is a union-like class, or

(23.2) — a non-static data member of `const` non-class type (or array thereof), or

(23.3) — a non-static data member of reference type, or

(23.4) — a potentially constructed subobject of class type `M` (or array thereof) that cannot be copied/moved because overload resolution (13.3), as applied to `M`'s corresponding assignment operator, results in an ambiguity or a function that is deleted or inaccessible from the defaulted assignment operator.

A defaulted move assignment operator that is defined as deleted is ignored by overload resolution (13.3, 13.4).

24 Because a copy/move assignment operator is implicitly declared for a class if not declared by the user, a base class copy/move assignment operator is always hidden by the corresponding assignment operator of a derived class (13.5.3). A *using-declaration* (7.3.3) that brings in from a base class an assignment operator with a parameter type that could be that of a copy/move assignment operator for the derived class is not considered an explicit declaration of such an operator and does not suppress the implicit declaration of the derived class operator; the operator introduced by the *using-declaration* is hidden by the implicitly-declared operator in the derived class.

25 A copy/move assignment operator for class `X` is trivial if it is not user-provided, its parameter-type-list is equivalent to the parameter-type-list of an implicit declaration, and if

(25.1) — class `X` has no virtual functions (10.3) and no virtual base classes (10.1), and

(25.2) — class `X` has no non-static data members of volatile-qualified type, and

(25.3) — the assignment operator selected to copy/move each direct base class subobject is trivial, and

(25.4) — for each non-static data member of `X` that is of class type (or array thereof), the assignment operator selected to copy/move that member is trivial;

otherwise the copy/move assignment operator is *non-trivial*.

26 A copy/move assignment operator for a class `X` that is defaulted and not defined as deleted is *implicitly defined* when it is odr-used (3.2) (e.g., when it is selected by overload resolution to assign to an object of its class type) or when it is explicitly defaulted after its first declaration. The implicitly-defined copy/move assignment operator is `constexpr` if

(26.1) — `X` is a literal type, and

(26.2) — the assignment operator selected to copy/move each direct base class subobject is a `constexpr` function, and

(26.3) — for each non-static data member of `X` that is of class type (or array thereof), the assignment operator selected to copy/move that member is a `constexpr` function.

27 Before the defaulted copy/move assignment operator for a class is implicitly defined, all non-user-provided copy/move assignment operators for its direct base classes and its non-static data members shall have been implicitly defined. [*Note:* An implicitly-declared copy/move assignment operator has an implied exception specification (15.4). — *end note*]

28 The implicitly-defined copy/move assignment operator for a non-union class `X` performs memberwise copy-/move assignment of its subobjects. The direct base classes of `X` are assigned first, in the order of their declaration in the *base-specifier-list*, and then the immediate non-static data members of `X` are assigned, in the order in which they were declared in the class definition. Let `x` be either the parameter of the function or, for the move operator, an xvalue referring to the parameter. Each subobject is assigned in the manner appropriate to its type:

(28.1) — if the subobject is of class type, as if by a call to `operator=` with the subobject as the object expression and the corresponding subobject of `x` as a single function argument (as if by explicit qualification; that is, ignoring any possible virtual overriding functions in more derived classes);

(28.2) — if the subobject is an array, each element is assigned, in the manner appropriate to the element type;

(28.3) — if the subobject is of scalar type, the built-in assignment operator is used.

It is unspecified whether subobjects representing virtual base classes are assigned more than once by the implicitly-defined copy/move assignment operator. [*Example:*

```
struct V { };
struct A : virtual V { };
struct B : virtual V { };
struct C : B, A { };
```

It is unspecified whether the virtual base class subobject `V` is assigned twice by the implicitly-defined copy-/move assignment operator for `C`. — *end example*]

29 The implicitly-defined copy assignment operator for a union `X` copies the object representation (3.9) of `X`.

30 A program is ill-formed if the copy/move constructor or the copy/move assignment operator for an object is implicitly odr-used and the special member function is not accessible (Clause 11). [*Note:* Copying/moving one object into another using the copy/move constructor or the copy/move assignment operator does not change the layout or size of either object. — *end note*]

31 When certain criteria are met, an implementation is allowed to omit the copy/move construction of a class object, even if the constructor selected for the copy/move operation and/or the destructor for the object have side effects. In such cases, the implementation treats the source and target of the omitted copy/move operation as simply two different ways of referring to the same object. If the first parameter of the selected constructor is an rvalue reference to the object's type, the destruction of that object occurs when the target would have been destroyed; otherwise, the destruction occurs at the later of the times when the two objects would have been destroyed without the optimization.[123] This elision of copy/move operations, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):

(31.1) — in a `return` statement in a function with a class return type, when the *expression* is the name of a non-volatile automatic object (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler* (15.3)) with the same type (ignoring cv-qualification) as the function return type, the copy/move operation can be omitted by constructing the automatic object directly into the function's return value

(31.2) — in a *throw-expression* (5.17), when the operand is the name of a non-volatile automatic object (other than a function or catch-clause parameter) whose scope does not extend beyond the end of the innermost enclosing *try-block* (if there is one), the copy/move operation from the operand to the exception object (15.1) can be omitted by constructing the automatic object directly into the exception object

(31.3) — when a temporary class object that has not been bound to a reference (12.2) would be copied/moved to a class object with the same type (ignoring cv-qualification), the copy/move operation can be omitted by constructing the temporary object directly into the target of the omitted copy/move

(31.4) — when the *exception-declaration* of an exception handler (Clause 15) declares an object of the same type (except for cv-qualification) as the exception object (15.1), the copy operation can be omitted by treating the *exception-declaration* as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the

123) Because only one object is destroyed instead of two, and one copy/move constructor is not executed, there is still one object destroyed for each one constructed.

exception-declaration. [*Note:* There cannot be a move from the exception object because it is always an lvalue. — *end note*]

[*Example:*

```
class Thing {
public:
  Thing();
  ~Thing();
  Thing(const Thing&);
};

Thing f() {
  Thing t;
  return t;
}

Thing t2 = f();
```

Here the criteria for elision can be combined to eliminate two calls to the copy constructor of class `Thing`: the copying of the local automatic object `t` into the temporary object for the return value of function `f()` and the copying of that temporary object into object `t2`. Effectively, the construction of the local object `t` can be viewed as directly initializing the global object `t2`, and that object's destruction will occur at program exit. Adding a move constructor to `Thing` has the same effect, but it is the move construction from the temporary object to `t2` that is elided. — *end example*]

32 When the criteria for elision of a copy/move operation are met, but not for an *exception-declaration*, and the object to be copied is designated by an lvalue, or when the *expression* in a `return` statement is a (possibly parenthesized) *id-expression* that names an object with automatic storage duration declared in the body or *parameter-declaration-clause* of the innermost enclosing function or *lambda-expression*, overload resolution to select the constructor for the copy is first performed as if the object were designated by an rvalue. If the first overload resolution fails or was not performed, or if the type of the first parameter of the selected constructor is not an rvalue reference to the object's type (possibly cv-qualified), overload resolution is performed again, considering the object as an lvalue. [*Note:* This two-stage overload resolution must be performed regardless of whether copy elision will occur. It determines the constructor to be called if elision is not performed, and the selected constructor must be accessible even if the call is elided. — *end note*]

[*Example:*

```
class Thing {
public:
  Thing();
  ~Thing();
  Thing(Thing&&);
private:
  Thing(const Thing&);
};

Thing f(bool b) {
  Thing t;
  if (b)
    throw t;                    // OK: Thing(Thing&&) used (or elided) to throw t
  return t;                     // OK: Thing(Thing&&) used (or elided) to return t
}

Thing t2 = f(false);            // OK: Thing(Thing&&) used (or elided) to construct t2
```

— *end example*]

12.9 Inheriting constructors [class.inhctor]

1 A *using-declaration* (7.3.3) that names a constructor implicitly declares a set of *inheriting constructors*. The *candidate set of inherited constructors* from the class `X` named in the *using-declaration* consists of actual constructors and notional constructors that result from the transformation of defaulted parameters and ellipsis parameter specifications as follows:

(1.1) — for each non-template constructor of `X`, the constructor that results from omitting any ellipsis parameter specification, and

(1.2) — for each non-template constructor of `X` that has at least one parameter with a default argument, the set of constructors that results from omitting any ellipsis parameter specification and successively omitting parameters with a default argument from the end of the parameter-type-list, and

(1.3) — for each constructor template of `X`, the constructor template that results from omitting any ellipsis parameter specification, and

(1.4) — for each constructor template of `X` that has at least one parameter with a default argument, the set of constructor templates that results from omitting any ellipsis parameter specification and successively omitting parameters with a default argument from the end of the parameter-type-list.

2 The *constructor characteristics* of a constructor or constructor template are

(2.1) — the template parameter list (14.1), if any,

(2.2) — the parameter-type-list (8.3.5), and

(2.3) — absence or presence of `explicit` (12.3.1).

3 For each non-template constructor in the candidate set of inherited constructors other than a constructor having no parameters or a copy/move constructor having a single parameter, a constructor is implicitly declared with the same constructor characteristics unless there is a user-declared constructor with the same signature in the complete class where the *using-declaration* appears or the constructor would be a default, copy, or move constructor for that class. Similarly, for each constructor template in the candidate set of inherited constructors, a constructor template is implicitly declared with the same constructor characteristics unless there is an equivalent user-declared constructor template (14.5.6.1) in the complete class where the *using-declaration* appears. [*Note:* Default arguments are not inherited. An exception specification is implied as specified in 15.4. — *end note*]

4 A constructor so declared has the same access as the corresponding constructor in `X`. It is `constexpr` if the user-written constructor (see below) would satisfy the requirements of a `constexpr` constructor (7.1.5). It is deleted if the corresponding constructor in `X` is deleted (8.4.3) or if a defaulted default constructor (12.1) would be deleted, except that the construction of the direct base class `X` is not considered in the determination. An inheriting constructor shall not be explicitly instantiated (14.7.2) or explicitly specialized (14.7.3).

5 [*Note:* Default and copy/move constructors may be implicitly declared as specified in 12.1 and 12.8. — *end note*]

6 [*Example:*

```
struct B1 {
  B1(int);
};

struct B2 {
  B2(int = 13, int = 42);
```

```
};

struct D1 : B1 {
  using B1::B1;
};

struct D2 : B2 {
  using B2::B2;
};
```

The candidate set of inherited constructors in `D1` for `B1` is

(6.1) — `B1(const B1&)`

(6.2) — `B1(B1&&)`

(6.3) — `B1(int)`

The set of constructors present in `D1` is

(6.4) — `D1()`, implicitly-declared default constructor, ill-formed if odr-used

(6.5) — `D1(const D1&)`, implicitly-declared copy constructor, not inherited

(6.6) — `D1(D1&&)`, implicitly-declared move constructor, not inherited

(6.7) — `D1(int)`, implicitly-declared inheriting constructor

The candidate set of inherited constructors in `D2` for `B2` is

(6.8) — `B2(const B2&)`

(6.9) — `B2(B2&&)`

(6.10) — `B2(int = 13, int = 42)`

(6.11) — `B2(int = 13)`

(6.12) — `B2()`

The set of constructors present in `D2` is

(6.13) — `D2()`, implicitly-declared default constructor, not inherited

(6.14) — `D2(const D2&)`, implicitly-declared copy constructor, not inherited

(6.15) — `D2(D2&&)`, implicitly-declared move constructor, not inherited

(6.16) — `D2(int, int)`, implicitly-declared inheriting constructor

(6.17) — `D2(int)`, implicitly-declared inheriting constructor

— *end example*]

7 [*Note:* If two *using-declaration*s declare inheriting constructors with the same signatures, the program is ill-formed (9.2, 13.1), because an implicitly-declared constructor introduced by the first *using-declaration* is not a user-declared constructor and thus does not preclude another declaration of a constructor with the same signature by a subsequent *using-declaration*. [*Example:*

```
struct B1 {
  B1(int);
};

struct B2 {
  B2(int);
};

struct D1 : B1, B2 {
  using B1::B1;
  using B2::B2;
};                  // ill-formed: attempts to declare D1(int) twice

struct D2 : B1, B2 {
  using B1::B1;
  using B2::B2;
  D2(int);          // OK: user declaration supersedes both implicit declarations
};
```

— end example] *— end note*]

8 An inheriting constructor for a class is implicitly defined when it is odr-used (3.2) to create an object of its class type (1.8). An implicitly-defined inheriting constructor performs the set of initializations of the class that would be performed by a user-written `inline` constructor for that class with a *mem-initializer-list* whose only *mem-initializer* has a *mem-initializer-id* that names the base class denoted in the *nested-name-specifier* of the *using-declaration* and an *expression-list* as specified below, and where the *compound-statement* in its function body is empty (12.6.2). If that user-written constructor would be ill-formed, the program is ill-formed. Each *expression* in the *expression-list* is of the form `static_cast<T&&>(p)`, where `p` is the name of the corresponding constructor parameter and `T` is the declared type of `p`.

9 [*Example:*

```
struct B1 {
  B1(int) { }
};

struct B2 {
  B2(double) { }
};

struct D1 : B1 {
  using B1::B1;     // implicitly declares D1(int)
  int x;
};

void test() {
  D1 d(6);          // OK: d.x is not initialized
  D1 e;             // error: D1 has no default constructor
}

struct D2 : B2 {
  using B2::B2;     // OK: implicitly declares D2(double)
  B1 b;
};

D2 f(1.0);          // error: B1 has no default constructor
```

```
template< class T >
struct D : T {
  using T::T;        // declares all constructors from class T
  ~D() { std::clog << "Destroying wrapper" << std::endl; }
};
```

Class template `D` wraps any class and forwards all of its constructors, while writing a message to the standard log whenever an object of class `D` is destroyed. — *end example*]

13 Overloading [over]

1 When two or more different declarations are specified for a single name in the same scope, that name is said to be *overloaded*. By extension, two declarations in the same scope that declare the same name but with different types are called *overloaded declarations*. Only function and function template declarations can be overloaded; variable and type declarations cannot be overloaded.

2 When an overloaded function name is used in a call, which overloaded function declaration is being referenced is determined by comparing the types of the arguments at the point of use with the types of the parameters in the overloaded declarations that are visible at the point of use. This function selection process is called *overload resolution* and is defined in 13.3. [*Example:*

```
double abs(double);
int abs(int);

abs(1);             // calls abs(int);
abs(1.0);           // calls abs(double);
```

— *end example*]

13.1 Overloadable declarations [over.load]

1 Not all function declarations can be overloaded. Those that cannot be overloaded are specified here. A program is ill-formed if it contains two such non-overloadable declarations in the same scope. [*Note:* This restriction applies to explicit declarations in a scope, and between such declarations and declarations made through a *using-declaration* (7.3.3). It does not apply to sets of functions fabricated as a result of name lookup (e.g., because of *using-directives*) or overload resolution (e.g., for operator functions). — *end note*]

2 Certain function declarations cannot be overloaded:

(2.1) — Function declarations that differ only in the return type cannot be overloaded.

(2.2) — Member function declarations with the same name and the same *parameter-type-list* cannot be overloaded if any of them is a `static` member function declaration (9.4). Likewise, member function template declarations with the same name, the same *parameter-type-list*, and the same template parameter lists cannot be overloaded if any of them is a `static` member function template declaration. The types of the implicit object parameters constructed for the member functions for the purpose of overload resolution (13.3.1) are not considered when comparing parameter-type-lists for enforcement of this rule. In contrast, if there is no `static` member function declaration among a set of member function declarations with the same name and the same parameter-type-list, then these member function declarations can be overloaded if they differ in the type of their implicit object parameter. [*Example:* the following illustrates this distinction:

```
class X {
  static void f();
  void f();                     // ill-formed
  void f() const;               // ill-formed
  void f() const volatile;      // ill-formed
  void g();
  void g() const;               // OK: no static g
  void g() const volatile;      // OK: no static g
};
```

— *end example*]

(2.3) — Member function declarations with the same name and the same *parameter-type-list* as well as member function template declarations with the same name, the same *parameter-type-list*, and the same template parameter lists cannot be overloaded if any of them, but not all, have a *ref-qualifier* (8.3.5). [*Example:*

```
class Y {
  void h() &;
  void h() const &;              // OK
  void h() &&;                   // OK, all declarations have a ref-qualifier
  void i() &;
  void i() const;                // ill-formed, prior declaration of i
                                 // has a ref-qualifier
};
```

— *end example*]

3 [*Note:* As specified in 8.3.5, function declarations that have equivalent parameter declarations declare the same function and therefore cannot be overloaded:

(3.1) — Parameter declarations that differ only in the use of equivalent typedef "types" are equivalent. A `typedef` is not a separate type, but only a synonym for another type (7.1.3). [*Example:*

```
typedef int Int;

void f(int i);
void f(Int i);                   // OK: redeclaration of f(int)
void f(int i) { /* ...  */ }
void f(Int i) { /* ...  */ }     // error: redefinition of f(int)
```

— *end example*]

Enumerations, on the other hand, are distinct types and can be used to distinguish overloaded function declarations. [*Example:*

```
enum E { a };

void f(int i) { /* ...  */ }
void f(E i)   { /* ...  */ }
```

— *end example*]

(3.2) — Parameter declarations that differ only in a pointer `*` versus an array `[]` are equivalent. That is, the array declaration is adjusted to become a pointer declaration (8.3.5). Only the second and subsequent array dimensions are significant in parameter types (8.3.4). [*Example:*

```
int f(char*);
int f(char[]);                   // same as f(char*);
int f(char[7]);                  // same as f(char*);
int f(char[9]);                  // same as f(char*);

int g(char(*)[10]);
int g(char[5][10]);              // same as g(char(*)[10]);
int g(char[7][10]);              // same as g(char(*)[10]);
int g(char(*)[20]);              // different from g(char(*)[10]);
```

— *end example*]

(3.3) — Parameter declarations that differ only in that one is a function type and the other is a pointer to the same function type are equivalent. That is, the function type is adjusted to become a pointer to function type (8.3.5). [*Example:*

```
void h(int());
void h(int (*)());              // redeclaration of h(int())
void h(int x()) { }             // definition of h(int())
void h(int (*x)()) { }          // ill-formed: redefinition of h(int())
```

— *end example*]

(3.4) — Parameter declarations that differ only in the presence or absence of `const` and/or `volatile` are equivalent. That is, the `const` and `volatile` type-specifiers for each parameter type are ignored when determining which function is being declared, defined, or called. [*Example:*

```
typedef const int cInt;

int f (int);
int f (const int);              // redeclaration of f(int)
int f (int) { /* ... */ }       // definition of f(int)
int f (cInt) { /* ... */ }      // error: redefinition of f(int)
```

— *end example*]

Only the `const` and `volatile` type-specifiers at the outermost level of the parameter type specification are ignored in this fashion; `const` and `volatile` type-specifiers buried within a parameter type specification are significant and can be used to distinguish overloaded function declarations.[124] In particular, for any type `T`, "pointer to `T`", "pointer to `const T`", and "pointer to `volatile T`" are considered distinct parameter types, as are "reference to `T`", "reference to `const T`", and "reference to `volatile T`".

(3.5) — Two parameter declarations that differ only in their default arguments are equivalent. [*Example:* consider the following:

```
void f (int i, int j);
void f (int i, int j = 99);     // OK: redeclaration of f(int, int)
void f (int i = 88, int j);     // OK: redeclaration of f(int, int)
void f ();                      // OK: overloaded declaration of f

void prog () {
    f (1, 2);                   // OK: call f(int, int)
    f (1);                      // OK: call f(int, int)
    f ();                       // Error: f(int, int) or f()?
}
```

— *end example*] — *end note*]

13.2 Declaration matching [over.dcl]

1 Two function declarations of the same name refer to the same function if they are in the same scope and have equivalent parameter declarations (13.1). A function member of a derived class is *not* in the same scope as a function member of the same name in a base class. [*Example:*

124) When a parameter type includes a function type, such as in the case of a parameter type that is a pointer to function, the `const` and `volatile` type-specifiers at the outermost level of the parameter type specifications for the inner function type are also ignored.

```
struct B {
  int f(int);
};

struct D : B {
  int f(const char*);
};
```

Here `D::f(const char*)` hides `B::f(int)` rather than overloading it.

```
void h(D* pd) {
  pd->f(1);                     // error:
                                // D::f(const char*) hides B::f(int)
  pd->B::f(1);                  // OK
  pd->f("Ben");                 // OK, calls D::f
}
```

— *end example*]

2 A locally declared function is not in the same scope as a function in a containing scope. [*Example:*

```
void f(const char*);
void g() {
  extern void f(int);
  f("asdf");                    // error: f(int) hides f(const char*)
                                // so there is no f(const char*) in this scope
}

void caller () {
  extern void callee(int, int);
  {
    extern void callee(int);    // hides callee(int, int)
    callee(88, 99);             // error: only callee(int) in scope
  }
}
```

— *end example*]

3 Different versions of an overloaded member function can be given different access rules. [*Example:*

```
class buffer {
private:
    char* p;
    int size;
protected:
    buffer(int s, char* store) { size = s; p = store; }
public:
    buffer(int s) { p = new char[size = s]; }
};
```

— *end example*]

13.3 Overload resolution [over.match]

1 Overload resolution is a mechanism for selecting the best function to call given a list of expressions that are to be the arguments of the call and a set of *candidate functions* that can be called based on the context of the call. The selection criteria for the best function are the number of arguments, how well the arguments match the parameter-type-list of the candidate function, how well (for non-static member functions) the

object matches the implicit object parameter, and certain other properties of the candidate function. [*Note:* The function selected by overload resolution is not guaranteed to be appropriate for the context. Other restrictions, such as the accessibility of the function, can make its use in the calling context ill-formed. — *end note*]

2 Overload resolution selects the function to call in seven distinct contexts within the language:

(2.1) — invocation of a function named in the function call syntax (13.3.1.1.1);

(2.2) — invocation of a function call operator, a pointer-to-function conversion function, a reference-to-pointer-to-function conversion function, or a reference-to-function conversion function on a class object named in the function call syntax (13.3.1.1.2);

(2.3) — invocation of the operator referenced in an expression (13.3.1.2);

(2.4) — invocation of a constructor for default- or direct-initialization (8.5) of a class object (13.3.1.3);

(2.5) — invocation of a user-defined conversion for copy-initialization (8.5) of a class object (13.3.1.4);

(2.6) — invocation of a conversion function for initialization of an object of a nonclass type from an expression of class type (13.3.1.5); and

(2.7) — invocation of a conversion function for conversion to a glvalue or class prvalue to which a reference (8.5.3) will be directly bound (13.3.1.6).

Each of these contexts defines the set of candidate functions and the list of arguments in its own unique way. But, once the candidate functions and argument lists have been identified, the selection of the best function is the same in all cases:

(2.8) — First, a subset of the candidate functions (those that have the proper number of arguments and meet certain other conditions) is selected to form a set of viable functions (13.3.2).

(2.9) — Then the best viable function is selected based on the implicit conversion sequences (13.3.3.1) needed to match each argument to the corresponding parameter of each viable function.

3 If a best viable function exists and is unique, overload resolution succeeds and produces it as the result. Otherwise overload resolution fails and the invocation is ill-formed. When overload resolution succeeds, and the best viable function is not accessible (Clause 11) in the context in which it is used, the program is ill-formed.

13.3.1 Candidate functions and argument lists [over.match.funcs]

1 The subclauses of 13.3.1 describe the set of candidate functions and the argument list submitted to overload resolution in each of the seven contexts in which overload resolution is used. The source transformations and constructions defined in these subclauses are only for the purpose of describing the overload resolution process. An implementation is not required to use such transformations and constructions.

2 The set of candidate functions can contain both member and non-member functions to be resolved against the same argument list. So that argument and parameter lists are comparable within this heterogeneous set, a member function is considered to have an extra parameter, called the *implicit object parameter*, which represents the object for which the member function has been called. For the purposes of overload resolution, both static and non-static member functions have an implicit object parameter, but constructors do not.

3 Similarly, when appropriate, the context can construct an argument list that contains an *implied object argument* to denote the object to be operated on. Since arguments and parameters are associated by position within their respective lists, the convention is that the implicit object parameter, if present, is always the first parameter and the implied object argument, if present, is always the first argument.

4 For non-static member functions, the type of the implicit object parameter is

(4.1) — "lvalue reference to *cv* `X`" for functions declared without a *ref-qualifier* or with the `&` *ref-qualifier*

(4.2) — "rvalue reference to *cv* `X`" for functions declared with the `&&` *ref-qualifier*

where `X` is the class of which the function is a member and *cv* is the cv-qualification on the member function declaration. [*Example:* for a `const` member function of class `X`, the extra parameter is assumed to have type "reference to `const X`". — *end example*] For conversion functions, the function is considered to be a member of the class of the implied object argument for the purpose of defining the type of the implicit object parameter. For non-conversion functions introduced by a *using-declaration* into a derived class, the function is considered to be a member of the derived class for the purpose of defining the type of the implicit object parameter. For static member functions, the implicit object parameter is considered to match any object (since if the function is selected, the object is discarded). [*Note:* No actual type is established for the implicit object parameter of a static member function, and no attempt will be made to determine a conversion sequence for that parameter (13.3.3). — *end note*]

5 During overload resolution, the implied object argument is indistinguishable from other arguments. The implicit object parameter, however, retains its identity since conversions on the corresponding argument shall obey these additional rules:

(5.1) — no temporary object can be introduced to hold the argument for the implicit object parameter; and

(5.2) — no user-defined conversions can be applied to achieve a type match with it.

For non-static member functions declared without a *ref-qualifier*, an additional rule applies:

(5.3) — even if the implicit object parameter is not `const`-qualified, an rvalue can be bound to the parameter as long as in all other respects the argument can be converted to the type of the implicit object parameter. [*Note:* The fact that such an argument is an rvalue does not affect the ranking of implicit conversion sequences (13.3.3.2). — *end note*]

6 Because other than in list-initialization only one user-defined conversion is allowed in an implicit conversion sequence, special rules apply when selecting the best user-defined conversion (13.3.3, 13.3.3.1). [*Example:*

```
class T {
public:
  T();
};

class C : T {
public:
  C(int);
};
T a = 1;            // ill-formed: T(C(1)) not tried
```

— *end example*]

7 In each case where a candidate is a function template, candidate function template specializations are generated using template argument deduction (14.8.3, 14.8.2). Those candidates are then handled as candidate functions in the usual way.[125] A given name can refer to one or more function templates and also to a set of overloaded non-template functions. In such a case, the candidate functions generated from each function template are combined with the set of non-template candidate functions.

8 A defaulted move constructor or assignment operator (12.8) that is defined as deleted is excluded from the set of candidate functions in all contexts.

125) The process of argument deduction fully determines the parameter types of the function template specializations, i.e., the parameters of function template specializations contain no template parameter types. Therefore, except where specified otherwise, function template specializations and non-template functions (8.3.5) are treated equivalently for the remainder of overload resolution.

13.3.1.1 Function call syntax [over.match.call]

1 In a function call (5.2.2)

postfix-expression `(` *expression-list*$_{opt}$`)`

if the *postfix-expression* denotes a set of overloaded functions and/or function templates, overload resolution is applied as specified in 13.3.1.1.1. If the *postfix-expression* denotes an object of class type, overload resolution is applied as specified in 13.3.1.1.2.

2 If the *postfix-expression* denotes the address of a set of overloaded functions and/or function templates, overload resolution is applied using that set as described above. If the function selected by overload resolution is a non-static member function, the program is ill-formed. [*Note:* The resolution of the address of an overload set in other contexts is described in 13.4. — *end note*]

13.3.1.1.1 Call to named function [over.call.func]

1 Of interest in 13.3.1.1.1 are only those function calls in which the *postfix-expression* ultimately contains a name that denotes one or more functions that might be called. Such a *postfix-expression*, perhaps nested arbitrarily deep in parentheses, has one of the following forms:

postfix-expression:
postfix-expression `.` *id-expression*
postfix-expression `->` *id-expression*
primary-expression

These represent two syntactic subcategories of function calls: qualified function calls and unqualified function calls.

2 In qualified function calls, the name to be resolved is an *id-expression* and is preceded by an `->` or `.` operator. Since the construct `A->B` is generally equivalent to `(*A).B`, the rest of Clause 13 assumes, without loss of generality, that all member function calls have been normalized to the form that uses an object and the `.` operator. Furthermore, Clause 13 assumes that the *postfix-expression* that is the left operand of the `.` operator has type "*cv* `T`" where `T` denotes a class[126]. Under this assumption, the *id-expression* in the call is looked up as a member function of `T` following the rules for looking up names in classes (10.2). The function declarations found by that lookup constitute the set of candidate functions. The argument list is the *expression-list* in the call augmented by the addition of the left operand of the `.` operator in the normalized member function call as the implied object argument (13.3.1).

3 In unqualified function calls, the name is not qualified by an `->` or `.` operator and has the more general form of a *primary-expression*. The name is looked up in the context of the function call following the normal rules for name lookup in function calls (3.4). The function declarations found by that lookup constitute the set of candidate functions. Because of the rules for name lookup, the set of candidate functions consists (1) entirely of non-member functions or (2) entirely of member functions of some class `T`. In case (1), the argument list is the same as the *expression-list* in the call. In case (2), the argument list is the *expression-list* in the call augmented by the addition of an implied object argument as in a qualified function call. If the keyword `this` (9.3.2) is in scope and refers to class `T`, or a derived class of `T`, then the implied object argument is `(*this)`. If the keyword `this` is not in scope or refers to another class, then a contrived object of type `T` becomes the implied object argument[127]. If the argument list is augmented by a contrived object and overload resolution selects one of the non-static member functions of `T`, the call is ill-formed.

126) Note that cv-qualifiers on the type of objects are significant in overload resolution for both glvalue and class prvalue objects.

127) An implied object argument must be contrived to correspond to the implicit object parameter attributed to member functions during overload resolution. It is not used in the call to the selected function. Since the member functions all have the same implicit object parameter, the contrived object will not be the cause to select or reject a function.

13.3.1.1.2 Call to object of class type [over.call.object]

1 If the *primary-expression* `E` in the function call syntax evaluates to a class object of type "*cv* `T`", then the set of candidate functions includes at least the function call operators of `T`. The function call operators of `T` are obtained by ordinary lookup of the name `operator()` in the context of `(E).operator()`.

2 In addition, for each non-explicit conversion function declared in `T` of the form

> `operator` *conversion-type-id* `()` *cv-qualifier ref-qualifier*$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$`;`

where *cv-qualifier* is the same cv-qualification as, or a greater cv-qualification than, *cv*, and where *conversion-type-id* denotes the type "pointer to function of `(P1,...,Pn)` returning `R`", or the type "reference to pointer to function of `(P1,...,Pn)` returning `R`", or the type "reference to function of `(P1,...,Pn)` returning `R`", a *surrogate call function* with the unique name *call-function* and having the form

> `R` *call-function* `(` *conversion-type-id* `F, P1 a1, ... ,Pn an) { return F (a1,... ,an); }`

is also considered as a candidate function. Similarly, surrogate call functions are added to the set of candidate functions for each non-explicit conversion function declared in a base class of `T` provided the function is not hidden within `T` by another intervening declaration[128].

3 If such a surrogate call function is selected by overload resolution, the corresponding conversion function will be called to convert `E` to the appropriate function pointer or reference, and the function will then be invoked with the arguments of the call. If the conversion function cannot be called (e.g., because of an ambiguity), the program is ill-formed.

4 The argument list submitted to overload resolution consists of the argument expressions present in the function call syntax preceded by the implied object argument `(E)`. [*Note:* When comparing the call against the function call operators, the implied object argument is compared against the implicit object parameter of the function call operator. When comparing the call against a surrogate call function, the implied object argument is compared against the first parameter of the surrogate call function. The conversion function from which the surrogate call function was derived will be used in the conversion sequence for that parameter since it converts the implied object argument to the appropriate function pointer or reference required by that first parameter. — *end note*] [*Example:*

```
int f1(int);
int f2(float);
typedef int (*fp1)(int);
typedef int (*fp2)(float);
struct A {
  operator fp1() { return f1; }
  operator fp2() { return f2; }
} a;
int i = a(1);        // calls f1 via pointer returned from
                     // conversion function
```

— *end example*]

13.3.1.2 Operators in expressions [over.match.oper]

1 If no operand of an operator in an expression has a type that is a class or an enumeration, the operator is assumed to be a built-in operator and interpreted according to Clause 5. [*Note:* Because `.`, `.*`, and `::` cannot be overloaded, these operators are always built-in operators interpreted according to Clause 5. `?:` cannot be overloaded, but the rules in this subclause are used to determine the conversions to be applied to the second and third operands when they have class or enumeration type (5.16). — *end note*] [*Example:*

128) Note that this construction can yield candidate call functions that cannot be differentiated one from the other by overload resolution because they have identical declarations or differ only in their return type. The call will be ambiguous if overload resolution cannot select a match to the call that is uniquely better than such undifferentiable functions.

```
struct String {
  String (const String&);
  String (const char*);
  operator const char* ();
};
String operator + (const String&, const String&);

void f(void) {
 const char* p= "one" + "two";   // ill-formed because neither
                                 // operand has class or enumeration type
 int I = 1 + 1;                  // Always evaluates to 2 even if
                                 // class or enumeration types exist that
                                 // would perform the operation.
}
```

— *end example*]

2 If either operand has a type that is a class or an enumeration, a user-defined operator function might be declared that implements this operator or a user-defined conversion can be necessary to convert the operand to a type that is appropriate for a built-in operator. In this case, overload resolution is used to determine which operator function or built-in operator is to be invoked to implement the operator. Therefore, the operator notation is first transformed to the equivalent function-call notation as summarized in Table 10 (where `@` denotes one of the operators covered in the specified subclause).

Table 10 — Relationship between operator and function call notation

Subclause	Expression	As member function	As non-member function
13.5.1	@a	(a).operator@ ( )	operator@ (a)
13.5.2	a@b	(a).operator@ (b)	operator@ (a, b)
13.5.3	a=b	(a).operator= (b)	
13.5.5	a[b]	(a).operator[](b)	
13.5.6	a->	(a).operator-> ( )	
13.5.7	a@	(a).operator@ (0)	operator@ (a, 0)

3 For a unary operator `@` with an operand of a type whose cv-unqualified version is `T1`, and for a binary operator `@` with a left operand of a type whose cv-unqualified version is `T1` and a right operand of a type whose cv-unqualified version is `T2`, three sets of candidate functions, designated *member candidates*, *non-member candidates* and *built-in candidates*, are constructed as follows:

(3.1) — If `T1` is a complete class type or a class currently being defined, the set of member candidates is the result of the qualified lookup of `T1::operator@` (13.3.1.1.1); otherwise, the set of member candidates is empty.

(3.2) — The set of non-member candidates is the result of the unqualified lookup of `operator@` in the context of the expression according to the usual rules for name lookup in unqualified function calls (3.4.2) except that all member functions are ignored. However, if no operand has a class type, only those non-member functions in the lookup set that have a first parameter of type `T1` or "reference to (possibly cv-qualified) `T1`", when `T1` is an enumeration type, or (if there is a right operand) a second parameter of type `T2` or "reference to (possibly cv-qualified) `T2`", when `T2` is an enumeration type, are candidate functions.

(3.3) — For the operator `,`, the unary operator `&`, or the operator `->`, the built-in candidates set is empty. For all other operators, the built-in candidates include all of the candidate operator functions defined in 13.6 that, compared to the given operator,

(3.3.1) — have the same operator name, and

(3.3.2) — accept the same number of operands, and

(3.3.3) — accept operand types to which the given operand or operands can be converted according to 13.3.3.1, and

(3.3.4) — do not have the same parameter-type-list as any non-member candidate that is not a function template specialization.

4 For the built-in assignment operators, conversions of the left operand are restricted as follows:

(4.1) — no temporaries are introduced to hold the left operand, and

(4.2) — no user-defined conversions are applied to the left operand to achieve a type match with the left-most parameter of a built-in candidate.

5 For all other operators, no such restrictions apply.

6 The set of candidate functions for overload resolution is the union of the member candidates, the non-member candidates, and the built-in candidates. The argument list contains all of the operands of the operator. The best function from the set of candidate functions is selected according to 13.3.2 and 13.3.3.[129] [*Example:*

```
struct A {
  operator int();
};
A operator+(const A&, const A&);
void m() {
  A a, b;
  a + b;            // operator+(a,b) chosen over int(a) + int(b)
}
```

— *end example*]

7 If a built-in candidate is selected by overload resolution, the operands of class type are converted to the types of the corresponding parameters of the selected operation function, except that the second standard conversion sequence of a user-defined conversion sequence (13.3.3.1.2) is not applied. Then the operator is treated as the corresponding built-in operator and interpreted according to Clause 5. [*Example:*

```
struct X {
  operator double();
};

struct Y {
  operator int*();
};

int *a = Y() + 100.0;       // error: pointer arithmetic requires integral operand
int *b = Y() + X();         // error: pointer arithmetic requires integral operand
```

— *end example*]

8 The second operand of operator `->` is ignored in selecting an `operator->` function, and is not an argument when the `operator->` function is called. When `operator->` returns, the operator `->` is applied to the value returned, with the original second operand.[130]

129) If the set of candidate functions is empty, overload resolution is unsuccessful.

130) If the value returned by the `operator->` function has class type, this may result in selecting and calling another `operator->` function. The process repeats until an `operator->` function returns a value of non-class type.

9 If the operator is the operator `,`, the unary operator `&`, or the operator `->`, and there are no viable functions, then the operator is assumed to be the built-in operator and interpreted according to Clause 5.

10 [*Note:* The lookup rules for operators in expressions are different than the lookup rules for operator function names in a function call, as shown in the following example:

```
struct A { };
void operator + (A, A);

struct B {
  void operator + (B);
  void f ();
};

A a;

void B::f() {
  operator+ (a,a);              // error: global operator hidden by member
  a + a;                        // OK: calls global operator+
}
```

— *end note*]

13.3.1.3 Initialization by constructor [over.match.ctor]

1 When objects of class type are direct-initialized (8.5), copy-initialized from an expression of the same or a derived class type (8.5), or default-initialized (8.5), overload resolution selects the constructor. For direct-initialization or default-initialization, the candidate functions are all the constructors of the class of the object being initialized. For copy-initialization, the candidate functions are all the converting constructors (12.3.1) of that class. The argument list is the *expression-list* or *assignment-expression* of the *initializer*.

13.3.1.4 Copy-initialization of class by user-defined conversion [over.match.copy]

1 Under the conditions specified in 8.5, as part of a copy-initialization of an object of class type, a user-defined conversion can be invoked to convert an initializer expression to the type of the object being initialized. Overload resolution is used to select the user-defined conversion to be invoked. [*Note:* The conversion performed for indirect binding to a reference to a possibly cv-qualified class type is determined in terms of a corresponding non-reference copy-initialization. — *end note*] Assuming that "*cv1* `T`" is the type of the object being initialized, with `T` a class type, the candidate functions are selected as follows:

(1.1) — The converting constructors (12.3.1) of `T` are candidate functions.

(1.2) — When the type of the initializer expression is a class type "*cv* `S`", the non-explicit conversion functions of `S` and its base classes are considered. When initializing a temporary to be bound to the first parameter of a constructor where the parameter is of type "reference to possibly *cv*-qualified `T`" and the constructor is called with a single argument in the context of direct-initialization of an object of type "*cv2* `T`", explicit conversion functions are also considered. Those that are not hidden within `S` and yield a type whose cv-unqualified version is the same type as `T` or is a derived class thereof are candidate functions. Conversion functions that return "reference to `X`" return lvalues or xvalues, depending on the type of reference, of type `X` and are therefore considered to yield `X` for this process of selecting candidate functions.

2 In both cases, the argument list has one argument, which is the initializer expression. [*Note:* This argument will be compared against the first parameter of the constructors and against the implicit object parameter of the conversion functions. — *end note*]

13.3.1.5 Initialization by conversion function [over.match.conv]

1 Under the conditions specified in 8.5, as part of an initialization of an object of nonclass type, a conversion function can be invoked to convert an initializer expression of class type to the type of the object being initialized. Overload resolution is used to select the conversion function to be invoked. Assuming that "*cv1* `T`" is the type of the object being initialized, and "*cv* `S`" is the type of the initializer expression, with `S` a class type, the candidate functions are selected as follows:

(1.1) — The conversion functions of `S` and its base classes are considered. Those non-explicit conversion functions that are not hidden within `S` and yield type `T` or a type that can be converted to type `T` via a standard conversion sequence (13.3.3.1.1) are candidate functions. For direct-initialization, those explicit conversion functions that are not hidden within `S` and yield type `T` or a type that can be converted to type `T` with a qualification conversion (4.4) are also candidate functions. Conversion functions that return a cv-qualified type are considered to yield the cv-unqualified version of that type for this process of selecting candidate functions. Conversion functions that return "reference to *cv2* `X`" return lvalues or xvalues, depending on the type of reference, of type "*cv2* `X`" and are therefore considered to yield `X` for this process of selecting candidate functions.

2 The argument list has one argument, which is the initializer expression. [*Note:* This argument will be compared against the implicit object parameter of the conversion functions. — *end note*]

13.3.1.6 Initialization by conversion function for direct reference binding [over.match.ref]

1 Under the conditions specified in 8.5.3, a reference can be bound directly to a glvalue or class prvalue that is the result of applying a conversion function to an initializer expression. Overload resolution is used to select the conversion function to be invoked. Assuming that "*cv1* `T`" is the underlying type of the reference being initialized, and "*cv* `S`" is the type of the initializer expression, with `S` a class type, the candidate functions are selected as follows:

(1.1) — The conversion functions of `S` and its base classes are considered. Those non-explicit conversion functions that are not hidden within `S` and yield type "lvalue reference to *cv2* `T2`" (when initializing an lvalue reference or an rvalue reference to function) or "*cv2* `T2`" or "rvalue reference to *cv2* `T2`" (when initializing an rvalue reference or an lvalue reference to function), where "*cv1* `T`" is reference-compatible (8.5.3) with "*cv2* `T2`", are candidate functions. For direct-initialization, those explicit conversion functions that are not hidden within `S` and yield type "lvalue reference to *cv2* `T2`" or "*cv2* `T2`" or "rvalue reference to *cv2* `T2`", respectively, where `T2` is the same type as `T` or can be converted to type `T` with a qualification conversion (4.4), are also candidate functions.

2 The argument list has one argument, which is the initializer expression. [*Note:* This argument will be compared against the implicit object parameter of the conversion functions. — *end note*]

13.3.1.7 Initialization by list-initialization [over.match.list]

1 When objects of non-aggregate class type `T` are list-initialized such that 8.5.4 specifies that overload resolution is performed according to the rules in this section, overload resolution selects the constructor in two phases:

(1.1) — Initially, the candidate functions are the initializer-list constructors (8.5.4) of the class `T` and the argument list consists of the initializer list as a single argument.

(1.2) — If no viable initializer-list constructor is found, overload resolution is performed again, where the candidate functions are all the constructors of the class `T` and the argument list consists of the elements of the initializer list.

If the initializer list has no elements and `T` has a default constructor, the first phase is omitted. In copy-list-initialization, if an `explicit` constructor is chosen, the initialization is ill-formed. [*Note:* This differs from other situations (13.3.1.3, 13.3.1.4), where only converting constructors are considered for copy-initialization. This restriction only applies if this initialization is part of the final result of overload resolution. — *end note*]

13.3.2 Viable functions [over.match.viable]

1 From the set of candidate functions constructed for a given context (13.3.1), a set of viable functions is chosen, from which the best function will be selected by comparing argument conversion sequences for the best fit (13.3.3). The selection of viable functions considers relationships between arguments and function parameters other than the ranking of conversion sequences.

2 First, to be a viable function, a candidate function shall have enough parameters to agree in number with the arguments in the list.

(2.1) — If there are m arguments in the list, all candidate functions having exactly m parameters are viable.

(2.2) — A candidate function having fewer than m parameters is viable only if it has an ellipsis in its parameter list (8.3.5). For the purposes of overload resolution, any argument for which there is no corresponding parameter is considered to "match the ellipsis" (13.3.3.1.3) .

(2.3) — A candidate function having more than m parameters is viable only if the *(m+1)*-st parameter has a default argument (8.3.6).[131] For the purposes of overload resolution, the parameter list is truncated on the right, so that there are exactly m parameters.

3 Second, for `F` to be a viable function, there shall exist for each argument an *implicit conversion sequence* (13.3.3.1) that converts that argument to the corresponding parameter of `F`. If the parameter has reference type, the implicit conversion sequence includes the operation of binding the reference, and the fact that an lvalue reference to non-`const` cannot be bound to an rvalue and that an rvalue reference cannot be bound to an lvalue can affect the viability of the function (see 13.3.3.1.4).

13.3.3 Best viable function [over.match.best]

1 Define ICS*i*(`F`) as follows:

(1.1) — if `F` is a static member function, ICS*1*(`F`) is defined such that ICS*1*(`F`) is neither better nor worse than ICS*1*(`G`) for any function `G`, and, symmetrically, ICS*1*(`G`) is neither better nor worse than ICS*1*(`F`)[132]; otherwise,

(1.2) — let ICS*i*(`F`) denote the implicit conversion sequence that converts the i-th argument in the list to the type of the i-th parameter of viable function `F`. 13.3.3.1 defines the implicit conversion sequences and 13.3.3.2 defines what it means for one implicit conversion sequence to be a better conversion sequence or worse conversion sequence than another.

Given these definitions, a viable function `F1` is defined to be a *better* function than another viable function `F2` if for all arguments i, ICS*i*(`F1`) is not a worse conversion sequence than ICS*i*(`F2`), and then

(1.3) — for some argument j, ICS*j*(`F1`) is a better conversion sequence than ICS*j*(`F2`), or, if not that,

(1.4) — the context is an initialization by user-defined conversion (see 8.5, 13.3.1.5, and 13.3.1.6) and the standard conversion sequence from the return type of `F1` to the destination type (i.e., the type of the entity being initialized) is a better conversion sequence than the standard conversion sequence from the return type of `F2` to the destination type. [*Example:*

```
struct A {
  A();
  operator int();
  operator double();
} a;
```

131) According to 8.3.6, parameters following the *(m+1)*-st parameter must also have default arguments.
132) If a function is a static member function, this definition means that the first argument, the implied object argument, has no effect in the determination of whether the function is better or worse than any other function.

```
int i = a;                      // a.operator int() followed by no conversion
                                // is better than a.operator double() followed by
                                // a conversion to int
float x = a;                    // ambiguous: both possibilities require conversions,
                                // and neither is better than the other
```

— *end example*] or, if not that,

(1.5) — the context is an initialization by conversion function for direct reference binding (13.3.1.6) of a reference to function type, the return type of `F1` is the same kind of reference (i.e. lvalue or rvalue) as the reference being initialized, and the return type of `F2` is not [*Example:*

```
template <class T> struct A {
  operator T&();        // #1
  operator T&&();       // #2
};
typedef int Fn();
A<Fn> a;
Fn& lf = a;             // calls #1
Fn&& rf = a;            // calls #2
```

— *end example*] or, if not that,

(1.6) — `F1` is not a function template specialization and `F2` is a function template specialization, or, if not that,

(1.7) — `F1` and `F2` are function template specializations, and the function template for `F1` is more specialized than the template for `F2` according to the partial ordering rules described in 14.5.6.2.

2 If there is exactly one viable function that is a better function than all other viable functions, then it is the one selected by overload resolution; otherwise the call is ill-formed[133].

[*Example:*

```
void Fcn(const int*,  short);
void Fcn(int*, int);

int i;
short s = 0;

void f() {
  Fcn(&i, s);                   // is ambiguous because
                                // &i → int* is better than &i → const int*
                                // but s → short is also better than s → int

  Fcn(&i, 1L);                  // calls Fcn(int*, int), because
                                // &i → int* is better than &i → const int*
                                // and 1L → short and 1L → int are indistinguishable

  Fcn(&i,'c');                  // calls Fcn(int*, int), because
                                // &i → int* is better than &i → const int*
                                // and c → int is better than c → short
}
```

133) The algorithm for selecting the best viable function is linear in the number of viable functions. Run a simple tournament to find a function `W` that is not worse than any opponent it faced. Although another function `F` that `W` did not face might be at least as good as `W`, `F` cannot be the best function because at some point in the tournament `F` encountered another function `G` such that `F` was not better than `G`. Hence, `W` is either the best function or there is no best function. So, make a second pass over the viable functions to verify that `W` is better than all other functions.

— *end example*]

3 If the best viable function resolves to a function for which multiple declarations were found, and if at least two of these declarations — or the declarations they refer to in the case of *using-declaration*s — specify a default argument that made the function viable, the program is ill-formed. [*Example:*

```
namespace A {
  extern "C" void f(int = 5);
}
namespace B {
  extern "C" void f(int = 5);
}

using A::f;
using B::f;

void use() {
  f(3);                         // OK, default argument was not used for viability
  f();                          // Error: found default argument twice
}
```

— *end example*]

13.3.3.1 Implicit conversion sequences [over.best.ics]

1 An *implicit conversion sequence* is a sequence of conversions used to convert an argument in a function call to the type of the corresponding parameter of the function being called. The sequence of conversions is an implicit conversion as defined in Clause 4, which means it is governed by the rules for initialization of an object or reference by a single expression (8.5, 8.5.3).

2 Implicit conversion sequences are concerned only with the type, cv-qualification, and value category of the argument and how these are converted to match the corresponding properties of the parameter. Other properties, such as the lifetime, storage class, alignment, accessibility of the argument, whether the argument is a bit-field, and whether a function is deleted (8.4.3), are ignored. So, although an implicit conversion sequence can be defined for a given argument-parameter pair, the conversion from the argument to the parameter might still be ill-formed in the final analysis.

3 A well-formed implicit conversion sequence is one of the following forms:

(3.1) — a *standard conversion sequence* (13.3.3.1.1),

(3.2) — a *user-defined conversion sequence* (13.3.3.1.2), or

(3.3) — an *ellipsis conversion sequence* (13.3.3.1.3).

4 However, if the target is

(4.1) — the first parameter of a constructor or

(4.2) — the implicit object parameter of a user-defined conversion function

and the constructor or user-defined conversion function is a candidate by

(4.3) — 13.3.1.3, when the argument is the temporary in the second step of a class copy-initialization, or

(4.4) — 13.3.1.4, 13.3.1.5, or 13.3.1.6 (in all cases),

user-defined conversion sequences are not considered. [*Note:* These rules prevent more than one user-defined conversion from being applied during overload resolution, thereby avoiding infinite recursion. — *end note*] [*Example:*

```
struct Y { Y(int); };
struct A { operator int(); };
Y y1 = A();  // error: A::operator int() is not a candidate

struct X { };
struct B { operator X(); };
B b;
X x({b});    // error: B::operator X() is not a candidate
```

— *end example*]

5 For the case where the parameter type is a reference, see 13.3.3.1.4.

6 When the parameter type is not a reference, the implicit conversion sequence models a copy-initialization of the parameter from the argument expression. The implicit conversion sequence is the one required to convert the argument expression to a prvalue of the type of the parameter. [*Note:* When the parameter has a class type, this is a conceptual conversion defined for the purposes of Clause 13; the actual initialization is defined in terms of constructors and is not a conversion. — *end note*] Any difference in top-level cv-qualification is subsumed by the initialization itself and does not constitute a conversion. [*Example:* a parameter of type `A` can be initialized from an argument of type `const A`. The implicit conversion sequence for that case is the identity sequence; it contains no "conversion" from `const A` to `A`. — *end example*] When the parameter has a class type and the argument expression has the same type, the implicit conversion sequence is an identity conversion. When the parameter has a class type and the argument expression has a derived class type, the implicit conversion sequence is a derived-to-base Conversion from the derived class to the base class. [*Note:* There is no such standard conversion; this derived-to-base Conversion exists only in the description of implicit conversion sequences. — *end note*] A derived-to-base Conversion has Conversion rank (13.3.3.1.1).

7 In all contexts, when converting to the implicit object parameter or when converting to the left operand of an assignment operation only standard conversion sequences that create no temporary object for the result are allowed.

8 If no conversions are required to match an argument to a parameter type, the implicit conversion sequence is the standard conversion sequence consisting of the identity conversion (13.3.3.1.1).

9 If no sequence of conversions can be found to convert an argument to a parameter type, an implicit conversion sequence cannot be formed.

10 If several different sequences of conversions exist that each convert the argument to the parameter type, the implicit conversion sequence associated with the parameter is defined to be the unique conversion sequence designated the *ambiguous conversion sequence*. For the purpose of ranking implicit conversion sequences as described in 13.3.3.2, the ambiguous conversion sequence is treated as a user-defined sequence that is indistinguishable from any other user-defined conversion sequence[134]. If a function that uses the ambiguous

134) The ambiguous conversion sequence is ranked with user-defined conversion sequences because multiple conversion sequences for an argument can exist only if they involve different user-defined conversions. The ambiguous conversion sequence is indistinguishable from any other user-defined conversion sequence because it represents at least two user-defined conversion sequences, each with a different user-defined conversion, and any other user-defined conversion sequence must be indistinguishable from at least one of them.

This rule prevents a function from becoming non-viable because of an ambiguous conversion sequence for one of its parameters. Consider this example,

```
class B;
class A { A (B&);};
class B { operator A (); };
class C { C (B&); };
```

conversion sequence is selected as the best viable function, the call will be ill-formed because the conversion of one of the arguments in the call is ambiguous.

11 The three forms of implicit conversion sequences mentioned above are defined in the following subclauses.

13.3.3.1.1 Standard conversion sequences [over.ics.scs]

1 Table 11 summarizes the conversions defined in Clause 4 and partitions them into four disjoint categories: Lvalue Transformation, Qualification Adjustment, Promotion, and Conversion. [*Note:* These categories are orthogonal with respect to value category, cv-qualification, and data representation: the Lvalue Transformations do not change the cv-qualification or data representation of the type; the Qualification Adjustments do not change the value category or data representation of the type; and the Promotions and Conversions do not change the value category or cv-qualification of the type. — *end note*]

2 [*Note:* As described in Clause 4, a standard conversion sequence is either the Identity conversion by itself (that is, no conversion) or consists of one to three conversions from the other four categories. At most one conversion from each category is allowed in a single standard conversion sequence. If there are two or more conversions in the sequence, the conversions are applied in the canonical order: **Lvalue Transformation**, **Promotion** or **Conversion**, **Qualification Adjustment**. — *end note*]

3 Each conversion in Table 11 also has an associated rank (Exact Match, Promotion, or Conversion). These are used to rank standard conversion sequences (13.3.3.2). The rank of a conversion sequence is determined by considering the rank of each conversion in the sequence and the rank of any reference binding (13.3.3.1.4). If any of those has Conversion rank, the sequence has Conversion rank; otherwise, if any of those has Promotion rank, the sequence has Promotion rank; otherwise, the sequence has Exact Match rank.

Table 11 — Conversions

Conversion	Category	Rank	Subclause
No conversions required	Identity	Exact Match	
Lvalue-to-rvalue conversion	Lvalue Transformation		4.1
Array-to-pointer conversion			4.2
Function-to-pointer conversion			4.3
Qualification conversions	Qualification Adjustment		4.4
Integral promotions	Promotion	Promotion	4.5
Floating point promotion			4.6
Integral conversions	Conversion	Conversion	4.7
Floating point conversions			4.8
Floating-integral conversions			4.9
Pointer conversions			4.10
Pointer to member conversions			4.11
Boolean conversions			4.12

```
void f(A) { }
void f(C) { }
B b;
f(b);                           // ambiguous because b → C via constructor and
                                // b → A via constructor or conversion function.
```

If it were not for this rule, `f(A)` would be eliminated as a viable function for the call `f(b)` causing overload resolution to select `f(C)` as the function to call even though it is not clearly the best choice. On the other hand, if an `f(B)` were to be declared then `f(b)` would resolve to that `f(B)` because the exact match with `f(B)` is better than any of the sequences required to match `f(A)`.

13.3.3.1.2 User-defined conversion sequences [over.ics.user]

1 A user-defined conversion sequence consists of an initial standard conversion sequence followed by a user-defined conversion (12.3) followed by a second standard conversion sequence. If the user-defined conversion is specified by a constructor (12.3.1), the initial standard conversion sequence converts the source type to the type required by the argument of the constructor. If the user-defined conversion is specified by a conversion function (12.3.2), the initial standard conversion sequence converts the source type to the implicit object parameter of the conversion function.

2 The second standard conversion sequence converts the result of the user-defined conversion to the target type for the sequence. Since an implicit conversion sequence is an initialization, the special rules for initialization by user-defined conversion apply when selecting the best user-defined conversion for a user-defined conversion sequence (see 13.3.3 and 13.3.3.1).

3 If the user-defined conversion is specified by a specialization of a conversion function template, the second standard conversion sequence shall have exact match rank.

4 A conversion of an expression of class type to the same class type is given Exact Match rank, and a conversion of an expression of class type to a base class of that type is given Conversion rank, in spite of the fact that a constructor (i.e., a user-defined conversion function) is called for those cases.

13.3.3.1.3 Ellipsis conversion sequences [over.ics.ellipsis]

1 An ellipsis conversion sequence occurs when an argument in a function call is matched with the ellipsis parameter specification of the function called (see 5.2.2).

13.3.3.1.4 Reference binding [over.ics.ref]

1 When a parameter of reference type binds directly (8.5.3) to an argument expression, the implicit conversion sequence is the identity conversion, unless the argument expression has a type that is a derived class of the parameter type, in which case the implicit conversion sequence is a derived-to-base Conversion (13.3.3.1). [*Example:*

```
struct A {};
struct B : public A {} b;
int f(A&);
int f(B&);
int i = f(b);                   // calls f(B&), an exact match, rather than
                                // f(A&), a conversion
```

— *end example*] If the parameter binds directly to the result of applying a conversion function to the argument expression, the implicit conversion sequence is a user-defined conversion sequence (13.3.3.1.2), with the second standard conversion sequence either an identity conversion or, if the conversion function returns an entity of a type that is a derived class of the parameter type, a derived-to-base Conversion.

2 When a parameter of reference type is not bound directly to an argument expression, the conversion sequence is the one required to convert the argument expression to the underlying type of the reference according to 13.3.3.1. Conceptually, this conversion sequence corresponds to copy-initializing a temporary of the underlying type with the argument expression. Any difference in top-level cv-qualification is subsumed by the initialization itself and does not constitute a conversion.

3 Except for an implicit object parameter, for which see 13.3.1, a standard conversion sequence cannot be formed if it requires binding an lvalue reference other than a reference to a non-volatile `const` type to an rvalue or binding an rvalue reference to an lvalue other than a function lvalue. [*Note:* This means, for example, that a candidate function cannot be a viable function if it has a non-`const` lvalue reference parameter (other than the implicit object parameter) and the corresponding argument is a temporary or would require one to be created to initialize the lvalue reference (see 8.5.3). — *end note*]

4 Other restrictions on binding a reference to a particular argument that are not based on the types of the reference and the argument do not affect the formation of a standard conversion sequence, however. [*Example:* a function with an "lvalue reference to `int`" parameter can be a viable candidate even if the corresponding argument is an `int` bit-field. The formation of implicit conversion sequences treats the `int` bit-field as an `int` lvalue and finds an exact match with the parameter. If the function is selected by overload resolution, the call will nonetheless be ill-formed because of the prohibition on binding a non-`const` lvalue reference to a bit-field (8.5.3). — *end example*]

13.3.3.1.5 List-initialization sequence [over.ics.list]

1 When an argument is an initializer list (8.5.4), it is not an expression and special rules apply for converting it to a parameter type.

2 If the parameter type is a class `X` and the initializer list has a single element of type *cv* `U`, where `U` is `X` or a class derived from `X`, the implicit conversion sequence is the one required to convert the element to the parameter type.

3 Otherwise, if the parameter type is a character array[135] and the initializer list has a single element that is an appropriately-typed string literal (8.5.2), the implicit conversion sequence is the identity conversion.

4 Otherwise, if the parameter type is `std::initializer_list<X>` and all the elements of the initializer list can be implicitly converted to `X`, the implicit conversion sequence is the worst conversion necessary to convert an element of the list to `X`, or if the initializer list has no elements, the identity conversion. This conversion can be a user-defined conversion even in the context of a call to an initializer-list constructor. [*Example:*

```
void f(std::initializer_list<int>);
f( {} );                    // OK: f(initializer_list<int>) identity conversion
f( {1,2,3} );               // OK: f(initializer_list<int>) identity conversion
f( {'a','b'} );             // OK: f(initializer_list<int>) integral promotion
f( {1.0} );                 // error: narrowing

struct A {
  A(std::initializer_list<double>);           // #1
  A(std::initializer_list<complex<double>>);  // #2
  A(std::initializer_list<std::string>);      // #3
};
A a{ 1.0,2.0 };             // OK, uses #1

void g(A);
g({ "foo", "bar" });        // OK, uses #3

typedef int IA[3];
void h(const IA&);
h({ 1, 2, 3 });             // OK: identity conversion
```

— *end example*]

5 Otherwise, if the parameter type is "array of `N X`", if the initializer list has exactly `N` elements or if it has fewer than `N` elements and `X` is default-constructible, and if all the elements of the initializer list can be implicitly converted to `X`, the implicit conversion sequence is the worst conversion necessary to convert an element of the list to `X`.

6 Otherwise, if the parameter is a non-aggregate class `X` and overload resolution per 13.3.1.7 chooses a single best constructor of `X` to perform the initialization of an object of type `X` from the argument initializer list, the implicit conversion sequence is a user-defined conversion sequence with the second standard conversion sequence an identity conversion. If multiple constructors are viable but none is better than the others, the

135) Since there are no parameters of array type, this will only occur as the underlying type of a reference parameter.

implicit conversion sequence is the ambiguous conversion sequence. User-defined conversions are allowed for conversion of the initializer list elements to the constructor parameter types except as noted in 13.3.3.1. [*Example:*

```
struct A {
  A(std::initializer_list<int>);
};
void f(A);
f( {'a', 'b'} );            // OK: f(A(std::initializer_list<int>)) user-defined conversion

struct B {
  B(int, double);
};
void g(B);
g( {'a', 'b'} );            // OK: g(B(int,double)) user-defined conversion
g( {1.0, 1.0} );            // error: narrowing

void f(B);
f( {'a', 'b'} );            // error: ambiguous f(A) or f(B)

struct C {
  C(std::string);
};
void h(C);
h({"foo"});                 // OK: h(C(std::string("foo")))

struct D {
  D(A, C);
};
void i(D);
i({ {1,2}, {"bar"} });      // OK: i(D(A(std::initializer_list<int>{1,2}),C(std::string("bar"))))
```

— *end example*]

7 Otherwise, if the parameter has an aggregate type which can be initialized from the initializer list according to the rules for aggregate initialization (8.5.1), the implicit conversion sequence is a user-defined conversion sequence with the second standard conversion sequence an identity conversion. [*Example:*

```
struct A {
  int m1;
  double m2;
};

void f(A);
f( {'a', 'b'} );            // OK: f(A(int,double)) user-defined conversion
f( {1.0} );                 // error: narrowing
```

— *end example*]

8 Otherwise, if the parameter is a reference, see 13.3.3.1.4. [*Note:* The rules in this section will apply for initializing the underlying temporary for the reference. — *end note*] [*Example:*

```
struct A {
  int m1;
  double m2;
};
```

```
void f(const A&);
f( {'a', 'b'} );            // OK: f(A(int,double)) user-defined conversion
f( {1.0} );                 // error: narrowing

void g(const double &);
g({1});                     // same conversion as int to double
```

— *end example*]

9 Otherwise, if the parameter type is not a class:

(9.1) — if the initializer list has one element that is not itself an initializer list, the implicit conversion sequence is the one required to convert the element to the parameter type; [*Example:*

```
void f(int);
f( {'a'} );                 // OK: same conversion as char to int
f( {1.0} );                 // error: narrowing
```

— *end example*]

(9.2) — if the initializer list has no elements, the implicit conversion sequence is the identity conversion. [*Example:*

```
void f(int);
f( { } );                   // OK: identity conversion
```

— *end example*]

10 In all cases other than those enumerated above, no conversion is possible.

13.3.3.2 Ranking implicit conversion sequences [over.ics.rank]

1 13.3.3.2 defines a partial ordering of implicit conversion sequences based on the relationships *better conversion sequence* and *better conversion*. If an implicit conversion sequence S1 is defined by these rules to be a better conversion sequence than S2, then it is also the case that S2 is a *worse conversion sequence* than S1. If conversion sequence S1 is neither better than nor worse than conversion sequence S2, S1 and S2 are said to be *indistinguishable conversion sequences*.

2 When comparing the basic forms of implicit conversion sequences (as defined in 13.3.3.1)

(2.1) — a standard conversion sequence (13.3.3.1.1) is a better conversion sequence than a user-defined conversion sequence or an ellipsis conversion sequence, and

(2.2) — a user-defined conversion sequence (13.3.3.1.2) is a better conversion sequence than an ellipsis conversion sequence (13.3.3.1.3).

3 Two implicit conversion sequences of the same form are indistinguishable conversion sequences unless one of the following rules applies:

(3.1) — List-initialization sequence `L1` is a better conversion sequence than list-initialization sequence `L2` if

(3.1.1) — `L1` converts to `std::initializer_list<X>` for some `X` and `L2` does not, or, if not that,

(3.1.2) — `L1` converts to type "array of `N1 T`", `L2` converts to type "array of `N2 T`", and `N1` is smaller than `N2`,

even if one of the other rules in this paragraph would otherwise apply. [*Example:*

```
void f1(int);                                   // #1
void f1(std::initializer_list<long>);           // #2
void g1() { f1({42}); }                         // chooses #2
```

```
void f2(std::pair<const char*, const char*>); // #3
void f2(std::initializer_list<std::string>);  // #4
void g2() { f2({"foo","bar"}); }              // chooses #4
```

— *end example*]

(3.2) — Standard conversion sequence `S1` is a better conversion sequence than standard conversion sequence `S2` if

(3.2.1) — `S1` is a proper subsequence of `S2` (comparing the conversion sequences in the canonical form defined by 13.3.3.1.1, excluding any Lvalue Transformation; the identity conversion sequence is considered to be a subsequence of any non-identity conversion sequence) or, if not that,

(3.2.2) — the rank of `S1` is better than the rank of `S2`, or `S1` and `S2` have the same rank and are distinguishable by the rules in the paragraph below, or, if not that,

(3.2.3) — `S1` and `S2` are reference bindings (8.5.3) and neither refers to an implicit object parameter of a non-static member function declared without a *ref-qualifier*, and `S1` binds an rvalue reference to an rvalue and `S2` binds an lvalue reference.

[*Example:*

```
int i;
int f1();
int&& f2();
int g(const int&);
int g(const int&&);
int j = g(i);                   // calls g(const int&)
int k = g(f1());                // calls g(const int&&)
int l = g(f2());                // calls g(const int&&)

struct A {
  A& operator<<(int);
  void p() &;
  void p() &&;
};
A& operator<<(A&&, char);
A() << 1;                       // calls A::operator<<(int)
A() << 'c';                     // calls operator<<(A&&, char)
A a;
a << 1;                         // calls A::operator<<(int)
a << 'c';                       // calls A::operator<<(int)
A().p();                        // calls A::p()&&
a.p();                          // calls A::p()&
```

— *end example*] or, if not that,

(3.2.4) — `S1` and `S2` are reference bindings (8.5.3) and `S1` binds an lvalue reference to a function lvalue and `S2` binds an rvalue reference to a function lvalue. [*Example:*

```
int f(void(&)());               // #1
int f(void(&&)());              // #2
void g();
int i1 = f(g);                  // calls #1
```

— *end example*] or, if not that,

(3.2.5) — `S1` and `S2` differ only in their qualification conversion and yield similar types `T1` and `T2` (4.4), respectively, and the cv-qualification signature of type `T1` is a proper subset of the cv-qualification signature of type `T2`. [*Example:*

```
int f(const volatile int *);
int f(const int *);
int i;
int j = f(&i);                  // calls f(const int*)
```

— *end example*] or, if not that,

(3.2.6) — `S1` and `S2` are reference bindings (8.5.3), and the types to which the references refer are the same type except for top-level cv-qualifiers, and the type to which the reference initialized by `S2` refers is more cv-qualified than the type to which the reference initialized by `S1` refers. [*Example:*

```
int f(const int &);
int f(int &);
int g(const int &);
int g(int);

int i;
int j = f(i);                   // calls f(int &)
int k = g(i);                   // ambiguous

struct X {
  void f() const;
  void f();
};
void g(const X& a, X b) {
  a.f();                        // calls X::f() const
  b.f();                        // calls X::f()
}
```

— *end example*]

(3.3) — User-defined conversion sequence `U1` is a better conversion sequence than another user-defined conversion sequence `U2` if they contain the same user-defined conversion function or constructor or they initialize the same class in an aggregate initialization and in either case the second standard conversion sequence of `U1` is better than the second standard conversion sequence of `U2`. [*Example:*

```
struct A {
  operator short();
} a;
int f(int);
int f(float);
int i = f(a);                   // calls f(int), because short → int is
                                // better than short → float.
```

— *end example*]

4 Standard conversion sequences are ordered by their ranks: an Exact Match is a better conversion than a Promotion, which is a better conversion than a Conversion. Two conversion sequences with the same rank are indistinguishable unless one of the following rules applies:

(4.1) — A conversion that does not convert a pointer, a pointer to member, or `std::nullptr_t` to `bool` is better than one that does.

(4.2) — A conversion that promotes an enumeration whose underlying type is fixed to its underlying type is better than one that promotes to the promoted underlying type, if the two are different.

(4.3) — If class `B` is derived directly or indirectly from class `A`, conversion of `B*` to `A*` is better than conversion of `B*` to `void*`, and conversion of `A*` to `void*` is better than conversion of `B*` to `void*`.

(4.4) — If class `B` is derived directly or indirectly from class `A` and class `C` is derived directly or indirectly from `B`,

(4.4.1) — conversion of `C*` to `B*` is better than conversion of `C*` to `A*`, [*Example:*

```
struct A {};
struct B : public A {};
struct C : public B {};
C* pc;
int f(A*);
int f(B*);
int i = f(pc);                  // calls f(B*)
```

— *end example*]

(4.4.2) — binding of an expression of type `C` to a reference to type `B` is better than binding an expression of type `C` to a reference to type `A`,

(4.4.3) — conversion of `A::*` to `B::*` is better than conversion of `A::*` to `C::*`,

(4.4.4) — conversion of `C` to `B` is better than conversion of `C` to `A`,

(4.4.5) — conversion of `B*` to `A*` is better than conversion of `C*` to `A*`,

(4.4.6) — binding of an expression of type `B` to a reference to type `A` is better than binding an expression of type `C` to a reference to type `A`,

(4.4.7) — conversion of `B::*` to `C::*` is better than conversion of `A::*` to `C::*`, and

(4.4.8) — conversion of `B` to `A` is better than conversion of `C` to `A`.

[*Note:* Compared conversion sequences will have different source types only in the context of comparing the second standard conversion sequence of an initialization by user-defined conversion (see 13.3.3); in all other contexts, the source types will be the same and the target types will be different. — *end note*]

13.4 Address of overloaded function [over.over]

1 A use of an overloaded function name without arguments is resolved in certain contexts to a function, a pointer to function or a pointer to member function for a specific function from the overload set. A function template name is considered to name a set of overloaded functions in such contexts. The function selected is the one whose type is identical to the function type of the target type required in the context. [*Note:* That is, the class of which the function is a member is ignored when matching a pointer-to-member-function type. — *end note*] The target can be

(1.1) — an object or reference being initialized (8.5, 8.5.3, 8.5.4),

(1.2) — the left side of an assignment (5.18),

(1.3) — a parameter of a function (5.2.2),

(1.4) — a parameter of a user-defined operator (13.5),

(1.5) — the return value of a function, operator function, or conversion (6.6.3),

(1.6) — an explicit type conversion (5.2.3, 5.2.9, 5.4), or

(1.7) — a non-type *template-parameter* (14.3.2).

The overloaded function name can be preceded by the `&` operator. An overloaded function name shall not be used without arguments in contexts other than those listed. [*Note:* Any redundant set of parentheses surrounding the overloaded function name is ignored (5.1). — *end note*]

2 If the name is a function template, template argument deduction is done (14.8.2.2), and if the argument deduction succeeds, the resulting template argument list is used to generate a single function template specialization, which is added to the set of overloaded functions considered. [*Note:* As described in 14.8.1, if deduction fails and the function template name is followed by an explicit template argument list, the *template-id* is then examined to see whether it identifies a single function template specialization. If it does, the *template-id* is considered to be an lvalue for that function template specialization. The target type is not used in that determination. — *end note*]

3 Non-member functions and static member functions match targets of pointer to function type or reference to function type. Non-static member functions match targets of pointer to member function type. If a non-static member function is selected, the reference to the overloaded function name is required to have the form of a pointer to member as described in 5.3.1.

4 If more than one function is selected, any function template specializations in the set are eliminated if the set also contains a function that is not a function template specialization, and any given function template specialization `F1` is eliminated if the set contains a second function template specialization whose function template is more specialized than the function template of `F1` according to the partial ordering rules of 14.5.6.2. After such eliminations, if any, there shall remain exactly one selected function.

5 [*Example:*

```
int f(double);
int f(int);
int (*pfd)(double) = &f;         // selects f(double)
int (*pfi)(int) = &f;            // selects f(int)
int (*pfe)(...) = &f;            // error: type mismatch
int (&rfi)(int) = f;             // selects f(int)
int (&rfd)(double) = f;          // selects f(double)
void g() {
  (int (*)(int))&f;              // cast expression as selector
}
```

The initialization of `pfe` is ill-formed because no `f()` with type `int(...)` has been declared, and not because of any ambiguity. For another example,

```
struct X {
  int f(int);
  static int f(long);
};

int (X::*p1)(int)  = &X::f;      // OK
int    (*p2)(int)  = &X::f;      // error: mismatch
int    (*p3)(long) = &X::f;      // OK
int (X::*p4)(long) = &X::f;      // error: mismatch
int (X::*p5)(int)  = &(X::f);    // error: wrong syntax for
                                 // pointer to member
int    (*p6)(long) = &(X::f);    // OK
```

— *end example*]

6 [*Note:* If `f()` and `g()` are both overloaded functions, the cross product of possibilities must be considered to resolve `f(&g)`, or the equivalent expression `f(g)`. — *end note*]

7 [*Note:* There are no standard conversions (Clause 4) of one pointer-to-function type into another. In particular, even if `B` is a public base of `D`, we have

```
D* f();
B* (*p1)() = &f;                // error

void g(D*);
void (*p2)(B*) = &g;            // error
```

— *end note*]

13.5 Overloaded operators [over.oper]

1 A function declaration having one of the following *operator-function-id*s as its name declares an *operator function*. A function template declaration having one of the following *operator-function-id*s as its name declares an *operator function template*. A specialization of an operator function template is also an operator function. An operator function is said to *implement* the operator named in its *operator-function-id*.

operator-function-id:
`operator` *operator*

operator: one of

```
new   delete   new[]    delete[]
+     -        *        /        %        ^        &        |        ~
!     =        <        >        +=       -=       *=       /=       %=
^=    &=       |=       <<       >>       >>=      <<=      ==       !=
<=    >=       &&       ||       ++       --       ,        ->*      ->
()    []
```

[*Note:* The last two operators are function call (5.2.2) and subscripting (5.2.1). The operators `new[]`, `delete[]`, `()`, and `[]` are formed from more than one token. — *end note*]

2 Both the unary and binary forms of

```
+    -    *    &
```

can be overloaded.

3 The following operators cannot be overloaded:

```
.    .*   ::   ?:
```

nor can the preprocessing symbols `#` and `##` (Clause 16).

4 Operator functions are usually not called directly; instead they are invoked to evaluate the operators they implement (13.5.1 – 13.5.7). They can be explicitly called, however, using the *operator-function-id* as the name of the function in the function call syntax (5.2.2). [*Example:*

```
complex z = a.operator+(b);     // complex z = a+b;
void* p = operator new(sizeof(int)*n);
```

— *end example*]

5 The allocation and deallocation functions, `operator new`, `operator new[]`, `operator delete` and `operator delete[]`, are described completely in 3.7.4. The attributes and restrictions found in the rest of this subclause do not apply to them unless explicitly stated in 3.7.4.

6 An operator function shall either be a non-static member function or be a non-member function that has at least one parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration. It is not possible to change the precedence, grouping, or number of operands of operators. The meaning of the operators `=`, (unary) `&`, and `,` (comma), predefined for each type, can be changed for

specific class and enumeration types by defining operator functions that implement these operators. Operator functions are inherited in the same manner as other base class functions.

7 The identities among certain predefined operators applied to basic types (for example, `++a` ≡ `a+=1`) need not hold for operator functions. Some predefined operators, such as `+=`, require an operand to be an lvalue when applied to basic types; this is not required by operator functions.

8 An operator function cannot have default arguments (8.3.6), except where explicitly stated below. Operator functions cannot have more or fewer parameters than the number required for the corresponding operator, as described in the rest of this subclause.

9 Operators not mentioned explicitly in subclauses 13.5.3 through 13.5.7 act as ordinary unary and binary operators obeying the rules of 13.5.1 or 13.5.2.

13.5.1 Unary operators [over.unary]

1 A prefix unary operator shall be implemented by a non-static member function (9.3) with no parameters or a non-member function with one parameter. Thus, for any prefix unary operator `@`, `@x` can be interpreted as either `x.operator@()` or `operator@(x)`. If both forms of the operator function have been declared, the rules in 13.3.1.2 determine which, if any, interpretation is used. See 13.5.7 for an explanation of the postfix unary operators `++` and `--`.

2 The unary and binary forms of the same operator are considered to have the same name. [*Note:* Consequently, a unary operator can hide a binary operator from an enclosing scope, and vice versa. — *end note*]

13.5.2 Binary operators [over.binary]

1 A binary operator shall be implemented either by a non-static member function (9.3) with one parameter or by a non-member function with two parameters. Thus, for any binary operator `@`, `x@y` can be interpreted as either `x.operator@(y)` or `operator@(x,y)`. If both forms of the operator function have been declared, the rules in 13.3.1.2 determine which, if any, interpretation is used.

13.5.3 Assignment [over.ass]

1 An assignment operator shall be implemented by a non-static member function with exactly one parameter. Because a copy assignment operator `operator=` is implicitly declared for a class if not declared by the user (12.8), a base class assignment operator is always hidden by the copy assignment operator of the derived class.

2 Any assignment operator, even the copy and move assignment operators, can be virtual. [*Note:* For a derived class `D` with a base class `B` for which a virtual copy/move assignment has been declared, the copy/move assignment operator in `D` does not override `B`'s virtual copy/move assignment operator. [*Example:*

```
struct B {
  virtual int operator= (int);
  virtual B& operator= (const B&);
};
struct D : B {
  virtual int operator= (int);
  virtual D& operator= (const B&);
};

D dobj1;
D dobj2;
B* bptr = &dobj1;
void f() {
  bptr->operator=(99);          // calls D::operator=(int)
```

```
  *bptr = 99;                   // ditto
  bptr->operator=(dobj2);       // calls D::operator=(const B&)
  *bptr = dobj2;                // ditto
  dobj1 = dobj2;                // calls implicitly-declared
                                // D::operator=(const D&)
}
```

— *end example*] — *end note*]

13.5.4 Function call [over.call]

1 `operator()` shall be a non-static member function with an arbitrary number of parameters. It can have default arguments. It implements the function call syntax

postfix-expression `(` *expression-list*$_{opt}$`)`

where the *postfix-expression* evaluates to a class object and the possibly empty *expression-list* matches the parameter list of an `operator()` member function of the class. Thus, a call `x(arg1,...)` is interpreted as `x.operator()(arg1, ...)` for a class object `x` of type `T` if `T::operator()(T1, T2, T3)` exists and if the operator is selected as the best match function by the overload resolution mechanism (13.3.3).

13.5.5 Subscripting [over.sub]

1 `operator[]` shall be a non-static member function with exactly one parameter. It implements the subscripting syntax

postfix-expression `[` *expression* `]`

or

postfix-expression `[` *braced-init-list* `]`

Thus, a subscripting expression `x[y]` is interpreted as `x.operator[](y)` for a class object `x` of type `T` if `T::operator[](T1)` exists and if the operator is selected as the best match function by the overload resolution mechanism (13.3.3). [*Example:*

```
struct X {
  Z operator[](std::initializer_list<int>);
};
X x;
x[{1,2,3}] = 7;           // OK: meaning x.operator[]({1,2,3})
int a[10];
a[{1,2,3}] = 7;           // error: built-in subscript operator
```

— *end example*]

13.5.6 Class member access [over.ref]

1 `operator->` shall be a non-static member function taking no parameters. It implements the class member access syntax that uses `->`.

postfix-expression `->` `template`$_{opt}$ *id-expression*
postfix-expression `->` *pseudo-destructor-name*

An expression `x->m` is interpreted as `(x.operator->())->m` for a class object `x` of type `T` if `T::operator->()` exists and if the operator is selected as the best match function by the overload resolution mechanism (13.3).

13.5.7 Increment and decrement [over.inc]

1 The user-defined function called `operator++` implements the prefix and postfix `++` operator. If this function is a non-static member function with no parameters, or a non-member function with one parameter, it defines the prefix increment operator `++` for objects of that type. If the function is a non-static member

function with one parameter (which shall be of type `int`) or a non-member function with two parameters (the second of which shall be of type `int`), it defines the postfix increment operator `++` for objects of that type. When the postfix increment is called as a result of using the `++` operator, the `int` argument will have value zero.[136] [*Example:*

```
struct X {
  X&   operator++();              // prefix ++a
  X    operator++(int);           // postfix a++
};

struct Y { };
Y&   operator++(Y&);              // prefix ++b
Y    operator++(Y&, int);         // postfix b++

void f(X a, Y b) {
  ++a;                            // a.operator++();
  a++;                            // a.operator++(0);
  ++b;                            // operator++(b);
  b++;                            // operator++(b, 0);

  a.operator++();                 // explicit call: like ++a;
  a.operator++(0);                // explicit call: like a++;
  operator++(b);                  // explicit call: like ++b;
  operator++(b, 0);               // explicit call: like b++;
}
```

— *end example*]

2 The prefix and postfix decrement operators `--` are handled analogously.

13.5.8 User-defined literals [over.literal]

literal-operator-id:
`operator` *string-literal identifier*
`operator` *user-defined-string-literal*

1 The *string-literal* or *user-defined-string-literal* in a *literal-operator-id* shall have no *encoding-prefix* and shall contain no characters other than the implicit terminating `'\0'`. The *ud-suffix* of the *user-defined-string-literal* or the *identifier* in a *literal-operator-id* is called a *literal suffix identifier*. Some literal suffix identifiers are reserved for future standardization; see 17.6.4.3.4. A declaration whose *literal-operator-id* uses such a literal suffix identifier is ill-formed; no diagnostic required.

2 A declaration whose *declarator-id* is a *literal-operator-id* shall be a declaration of a namespace-scope function or function template (it could be a friend function (11.3)), an explicit instantiation or specialization of a function template, or a *using-declaration* (7.3.3). A function declared with a *literal-operator-id* is a *literal operator*. A function template declared with a *literal-operator-id* is a *literal operator template*.

3 The declaration of a literal operator shall have a *parameter-declaration-clause* equivalent to one of the following:

```
const char*
unsigned long long int
long double
char
wchar_t
char16_t
```

136) Calling `operator++` explicitly, as in expressions like `a.operator++(2)`, has no special properties: The argument to `operator++` is 2.

```
char32_t
const char*, std::size_t
const wchar_t*, std::size_t
const char16_t*, std::size_t
const char32_t*, std::size_t
```

If a parameter has a default argument (8.3.6), the program is ill-formed.

4 A *raw literal operator* is a literal operator with a single parameter whose type is `const char*`.

5 The declaration of a literal operator template shall have an empty *parameter-declaration-clause* and its *template-parameter-list* shall have a single *template-parameter* that is a non-type template parameter pack (14.5.3) with element type `char`.

6 Literal operators and literal operator templates shall not have C language linkage.

7 [*Note:* Literal operators and literal operator templates are usually invoked implicitly through user-defined literals (2.13.8). However, except for the constraints described above, they are ordinary namespace-scope functions and function templates. In particular, they are looked up like ordinary functions and function templates and they follow the same overload resolution rules. Also, they can be declared `inline` or `constexpr`, they may have internal or external linkage, they can be called explicitly, their addresses can be taken, etc. — *end note*]

8 [*Example:*

```
void operator "" _km(long double);                  // OK
string operator "" _i18n(const char*, std::size_t); // OK
template <char...> double operator "" _\u03C0();    // OK: UCN for lowercase pi
float operator ""_e(const char*);                   // OK
float operator ""E(const char*);                    // error: reserved literal suffix (17.6.4.3.4, 2.13.8)
double operator""_Bq(long double);                  // OK: does not use the reserved identifier _Bq (2.10)
double operator"" _Bq(long double);                 // uses the reserved identifier _Bq (2.10)
float operator " " B(const char*);                  // error: non-empty string-literal
string operator "" 5X(const char*, std::size_t);    // error: invalid literal suffix identifier
double operator "" _miles(double);                  // error: invalid parameter-declaration-clause
template <char...> int operator "" _j(const char*); // error: invalid parameter-declaration-clause
extern "C" void operator "" _m(long double);        // error: C language linkage
```

— *end example*]

13.6 Built-in operators [over.built]

1 The candidate operator functions that represent the built-in operators defined in Clause 5 are specified in this subclause. These candidate functions participate in the operator overload resolution process as described in 13.3.1.2 and are used for no other purpose. [*Note:* Because built-in operators take only operands with non-class type, and operator overload resolution occurs only when an operand expression originally has class or enumeration type, operator overload resolution can resolve to a built-in operator only when an operand has a class type that has a user-defined conversion to a non-class type appropriate for the operator, or when an operand has an enumeration type that can be converted to a type appropriate for the operator. Also note that some of the candidate operator functions given in this subclause are more permissive than the built-in operators themselves. As described in 13.3.1.2, after a built-in operator is selected by overload resolution the expression is subject to the requirements for the built-in operator given in Clause 5, and therefore to any additional semantic constraints given there. If there is a user-written candidate with the same name and parameter types as a built-in candidate operator function, the built-in operator function is hidden and is not included in the set of candidate functions. — *end note*]

2 In this subclause, the term *promoted integral type* is used to refer to those integral types which are preserved by integral promotion (including e.g. `int` and `long` but excluding e.g. `char`). Similarly, the term *promoted*

arithmetic type refers to floating types plus promoted integral types. [*Note:* In all cases where a promoted integral type or promoted arithmetic type is required, an operand of enumeration type will be acceptable by way of the integral promotions. — *end note*]

3 For every pair (*T*, *VQ*), where *T* is an arithmetic type, and *VQ* is either `volatile` or empty, there exist candidate operator functions of the form

```
VQ T& operator++(VQ T&);
T operator++(VQ T&, int);
```

4 For every pair (*T*, *VQ*), where *T* is an arithmetic type other than *bool*, and *VQ* is either `volatile` or empty, there exist candidate operator functions of the form

```
VQ T& operator--(VQ T&);
T operator--(VQ T&, int);
```

5 For every pair (*T*, *VQ*), where *T* is a cv-qualified or cv-unqualified object type, and *VQ* is either `volatile` or empty, there exist candidate operator functions of the form

```
T*VQ& operator++(T*VQ&);
T*VQ& operator--(T*VQ&);
T*    operator++(T*VQ&, int);
T*    operator--(T*VQ&, int);
```

6 For every cv-qualified or cv-unqualified object type *T*, there exist candidate operator functions of the form

```
T&    operator*(T*);
```

7 For every function type *T* that does not have cv-qualifiers or a *ref-qualifier*, there exist candidate operator functions of the form

```
T&    operator*(T*);
```

8 For every type *T* there exist candidate operator functions of the form

```
T*    operator+(T*);
```

9 For every promoted arithmetic type *T*, there exist candidate operator functions of the form

```
T operator+(T);
T operator-(T);
```

10 For every promoted integral type *T*, there exist candidate operator functions of the form

```
T operator~(T);
```

11 For every quintuple (*C1*, *C2*, *T*, *CV1*, *CV2*), where *C2* is a class type, *C1* is the same type as C2 or is a derived class of C2, *T* is an object type or a function type, and *CV1* and *CV2* are *cv-qualifier-seq*s, there exist candidate operator functions of the form

```
CV12 T& operator->*(CV1 C1*, CV2 T C2::*);
```

where *CV12* is the union of *CV1* and *CV2*. The return type is shown for exposition only; see 5.5 for the determination of the operator's result type.

12 For every pair of promoted arithmetic types *L* and *R*, there exist candidate operator functions of the form

```
LR      operator*(L, R);
LR      operator/(L, R);
LR      operator+(L, R);
```

```
LR      operator-(L, R);
bool    operator<(L, R);
bool    operator>(L, R);
bool    operator<=(L, R);
bool    operator>=(L, R);
bool    operator==(L, R);
bool    operator!=(L, R);
```

where *LR* is the result of the usual arithmetic conversions between types *L* and *R*.

13 For every cv-qualified or cv-unqualified object type *T* there exist candidate operator functions of the form

```
T*      operator+(T*, std::ptrdiff_t);
T&      operator[](T*, std::ptrdiff_t);
T*      operator-(T*, std::ptrdiff_t);
T*      operator+(std::ptrdiff_t, T*);
T&      operator[](std::ptrdiff_t, T*);
```

14 For every *T*, where *T* is a pointer to object type, there exist candidate operator functions of the form

```
std::ptrdiff_t   operator-(T, T);
```

15 For every *T*, where *T* is an enumeration type or a pointer type, there exist candidate operator functions of the form

```
bool    operator<(T, T);
bool    operator>(T, T);
bool    operator<=(T, T);
bool    operator>=(T, T);
bool    operator==(T, T);
bool    operator!=(T, T);
```

16 For every pointer to member type *T* or type `std::nullptr_t` there exist candidate operator functions of the form

```
bool    operator==(T, T);
bool    operator!=(T, T);
```

17 For every pair of promoted integral types *L* and *R*, there exist candidate operator functions of the form

```
LR      operator%(L, R);
LR      operator&(L, R);
LR      operator^(L, R);
LR      operator|(L, R);
L       operator<<(L, R);
L       operator>>(L, R);
```

where *LR* is the result of the usual arithmetic conversions between types *L* and *R*.

18 For every triple (*L*, *VQ*, *R*), where *L* is an arithmetic type, *VQ* is either `volatile` or empty, and *R* is a promoted arithmetic type, there exist candidate operator functions of the form

```
VQ L&   operator=(VQ L&, R);
VQ L&   operator*=(VQ L&, R);
VQ L&   operator/=(VQ L&, R);
VQ L&   operator+=(VQ L&, R);
VQ L&   operator-=(VQ L&, R);
```

19 For every pair (*T*, *VQ*), where *T* is any type and *VQ* is either `volatile` or empty, there exist candidate operator functions of the form

```
T*VQ&   operator=(T*VQ&, T*);
```

20 For every pair (T, VQ), where T is an enumeration or pointer to member type and VQ is either `volatile` or empty, there exist candidate operator functions of the form

```
VQ T&   operator=(VQ T&, T);
```

21 For every pair (T, VQ), where T is a cv-qualified or cv-unqualified object type and VQ is either `volatile` or empty, there exist candidate operator functions of the form

```
T*VQ&   operator+=(T*VQ&, std::ptrdiff_t);
T*VQ&   operator-=(T*VQ&, std::ptrdiff_t);
```

22 For every triple (L, VQ, R), where L is an integral type, VQ is either `volatile` or empty, and R is a promoted integral type, there exist candidate operator functions of the form

```
VQ L&   operator%=(VQ L&, R);
VQ L&   operator<<=(VQ L&, R);
VQ L&   operator>>=(VQ L&, R);
VQ L&   operator&=(VQ L&, R);
VQ L&   operator^=(VQ L&, R);
VQ L&   operator|=(VQ L&, R);
```

23 There also exist candidate operator functions of the form

```
bool    operator!(bool);
bool    operator&&(bool, bool);
bool    operator||(bool, bool);
```

24 For every pair of promoted arithmetic types L and R, there exist candidate operator functions of the form

```
LR      operator?:(bool, L , R);
```

where LR is the result of the usual arithmetic conversions between types L and R. [*Note:* As with all these descriptions of candidate functions, this declaration serves only to describe the built-in operator for purposes of overload resolution. The operator "`?:`" cannot be overloaded. — *end note*]

25 For every type T, where T is a pointer, pointer-to-member, or scoped enumeration type, there exist candidate operator functions of the form

```
T       operator?:(bool, T, T);
```

14 Templates [temp]

1 A *template* defines a family of classes or functions or an alias for a family of types.

template-declaration:
`template` `<` *template-parameter-list* `>` *declaration*

template-parameter-list:
template-parameter
template-parameter-list `,` *template-parameter*

[*Note:* The `>` token following the *template-parameter-list* of a *template-declaration* may be the product of replacing a `>>` token by two consecutive `>` tokens (14.2). — *end note*]

The *declaration* in a *template-declaration* shall

(1.1) — declare or define a function, a class, or a variable, or

(1.2) — define a member function, a member class, a member enumeration, or a static data member of a class template or of a class nested within a class template, or

(1.3) — define a member template of a class or class template, or

(1.4) — be an *alias-declaration*.

A *template-declaration* is a *declaration*. A *template-declaration* is also a definition if its *declaration* defines a function, a class, a variable, or a static data member. A declaration introduced by a template declaration of a variable is a *variable template*. A variable template at class scope is a *static data member template*.

[*Example:*

```
template<class T>
  constexpr T pi = T(3.1415926535897932385L);
template<class T>
T circular_area(T r) {
  return pi<T> * r * r;
}
struct matrix_constants {
  template<class T>
   using pauli = hermitian_matrix<T, 2>;
  template<class T>
   constexpr pauli<T> sigma1 = { { 0, 1 }, { 1, 0 } };
  template<class T>
   constexpr pauli<T> sigma2 = { { 0, -1i }, { 1i, 0 } };
  template<class T>
   constexpr pauli<T> sigma3 = { { 1, 0 }, { 0, -1 } };
};
```

— *end example*]

2 A *template-declaration* can appear only as a namespace scope or class scope declaration. In a function template declaration, the last component of the *declarator-id* shall not be a *template-id*. [*Note:* That last component may be an *identifier*, an *operator-function-id*, a *conversion-function-id*, or a *literal-operator-id*. In a class template declaration, if the class name is a *simple-template-id*, the declaration declares a class template partial specialization (14.5.5). — *end note*]

3 In a *template-declaration*, explicit specialization, or explicit instantiation the *init-declarator-list* in the declaration shall contain at most one declarator. When such a declaration is used to declare a class template, no declarator is permitted.

4 A template name has linkage (3.5). Specializations (explicit or implicit) of a template that has internal linkage are distinct from all specializations in other translation units. A template, a template explicit specialization (14.7.3), and a class template partial specialization shall not have C linkage. Use of a linkage specification other than C or C++ with any of these constructs is conditionally-supported, with implementation-defined semantics. Template definitions shall obey the one-definition rule (3.2). [*Note:* Default arguments for function templates and for member functions of class templates are considered definitions for the purpose of template instantiation (14.5) and must also obey the one-definition rule. — *end note*]

5 A class template shall not have the same name as any other template, class, function, variable, enumeration, enumerator, namespace, or type in the same scope (3.3), except as specified in (14.5.5). Except that a function template can be overloaded either by non-template functions (8.3.5) with the same name or by other function templates with the same name (14.8.3), a template name declared in namespace scope or in class scope shall be unique in that scope.

6 A function template, member function of a class template, variable template, or static data member of a class template shall be defined in every translation unit in which it is implicitly instantiated (14.7.1) unless the corresponding specialization is explicitly instantiated (14.7.2) in some translation unit; no diagnostic is required.

14.1 Template parameters [temp.param]

1 The syntax for *template-parameter*s is:

template-parameter:
type-parameter
parameter-declaration

type-parameter:
type-parameter-key `...`$_{opt}$ *identifier*$_{opt}$
type-parameter-key *identifier*$_{opt}$ `=` *type-id*
`template` `<` *template-parameter-list* `>` *type-parameter-key* `...`$_{opt}$ *identifier*$_{opt}$
`template` `<` *template-parameter-list* `>` *type-parameter-key* *identifier*$_{opt}$ `=` *id-expression*

type-parameter-key:
`class`
`typename`

[*Note:* The `>` token following the *template-parameter-list* of a *type-parameter* may be the product of replacing a `>>` token by two consecutive `>` tokens (14.2). — *end note*]

2 There is no semantic difference between `class` and `typename` in a *type-parameter-key*. `typename` followed by an *unqualified-id* names a template type parameter. `typename` followed by a *qualified-id* denotes the type in a non-type[137] *parameter-declaration*. A *template-parameter* of the form `class` *identifier* is a *type-parameter*. [*Example:*

```
class T { /* ... */ };
int i;

template<class T, T i> void f(T t) {
  T t1 = i;       // template-parameters T and i
  ::T t2 = ::i;   // global namespace members T and i
}
```

137) Since template *template-parameter*s and template *template-argument*s are treated as types for descriptive purposes, the terms *non-type parameter* and *non-type argument* are used to refer to non-type, non-template parameters and arguments.

Here, the template `f` has a *type-parameter* called `T`, rather than an unnamed non-type *template-parameter* of class `T`. — *end example*] A storage class shall not be specified in a *template-parameter* declaration. Types shall not be defined in a *template-parameter* declaration.

3 A *type-parameter* whose identifier does not follow an ellipsis defines its *identifier* to be a *typedef-name* (if declared without `template`) or *template-name* (if declared with `template`) in the scope of the template declaration. [*Note:* A template argument may be a class template or alias template. For example,

```
template<class T> class myarray { /* ... */ };

template<class K, class V, template<class T> class C = myarray>
class Map {
  C<K> key;
  C<V> value;
};
```

— *end note*]

4 A non-type *template-parameter* shall have one of the following (optionally *cv-qualified*) types:

(4.1) — integral or enumeration type,

(4.2) — pointer to object or pointer to function,

(4.3) — lvalue reference to object or lvalue reference to function,

(4.4) — pointer to member,

(4.5) — `std::nullptr_t`.

5 [*Note:* Other types are disallowed either explicitly below or implicitly by the rules governing the form of *template-argument*s (14.3). — *end note*] The top-level *cv-qualifiers* on the *template-parameter* are ignored when determining its type.

6 A non-type non-reference *template-parameter* is a prvalue. It shall not be assigned to or in any other way have its value changed. A non-type non-reference *template-parameter* cannot have its address taken. When a non-type non-reference *template-parameter* is used as an initializer for a reference, a temporary is always used. [*Example:*

```
template<const X& x, int i> void f() {
  i++;                          // error: change of template-parameter value

  &x;                           // OK
  &i;                           // error: address of non-reference template-parameter

  int& ri = i;                  // error: non-const reference bound to temporary
  const int& cri = i;           // OK: const reference bound to temporary
}
```

— *end example*]

7 A non-type *template-parameter* shall not be declared to have floating point, class, or void type. [*Example:*

```
template<double d> class X;     // error
template<double* pd> class Y;   // OK
template<double& rd> class Z;   // OK
```

— *end example*]

8 A non-type *template-parameter* of type "array of `T`" or "function returning `T`" is adjusted to be of type "pointer to `T`" or "pointer to function returning `T`", respectively. [*Example:*

```
template<int* a>   struct R { /* ... */ };
template<int b[5]> struct S { /* ... */ };
int p;
R<&p> w;                        // OK
S<&p> x;                        // OK due to parameter adjustment
int v[5];
R<v> y;                         // OK due to implicit argument conversion
S<v> z;                         // OK due to both adjustment and conversion
```

— *end example*]

9 A *default template-argument* is a *template-argument* (14.3) specified after = in a *template-parameter*. A default *template-argument* may be specified for any kind of *template-parameter* (type, non-type, template) that is not a template parameter pack (14.5.3). A default *template-argument* may be specified in a template declaration. A default *template-argument* shall not be specified in the *template-parameter-list*s of the definition of a member of a class template that appears outside of the member's class. A default *template-argument* shall not be specified in a friend class template declaration. If a friend function template declaration specifies a default *template-argument*, that declaration shall be a definition and shall be the only declaration of the function template in the translation unit.

10 The set of default *template-argument*s available for use is obtained by merging the default arguments from all prior declarations of the template in the same way default function arguments are (8.3.6). [*Example:*

```
template<class T1, class T2 = int> class A;
template<class T1 = int, class T2> class A;
```

is equivalent to

```
template<class T1 = int, class T2 = int> class A;
```

— *end example*]

11 If a *template-parameter* of a class template or alias template has a default *template-argument*, each subsequent *template-parameter* shall either have a default *template-argument* supplied or be a template parameter pack. If a *template-parameter* of a primary class template or alias template is a template parameter pack, it shall be the last *template-parameter*. A template parameter pack of a function template shall not be followed by another template parameter unless that template parameter can be deduced from the *parameter-type-list* of the function template or has a default argument (14.8.2). [*Example:*

```
template<class T1 = int, class T2> class B;   // error

// U can be neither deduced from the parameter-type-list nor specified
template<class... T, class... U> void f() { } // error
template<class... T, class U> void g() { }    // error
```

— *end example*]

12 A *template-parameter* shall not be given default arguments by two different declarations in the same scope. [*Example:*

```
template<class T = int> class X;
template<class T = int> class X { /*... */ }; // error
```

— *end example*]

13 When parsing a default *template-argument* for a non-type *template-parameter*, the first non-nested > is taken as the end of the *template-parameter-list* rather than a greater-than operator. [*Example:*

```
template<int i = 3 > 4 >        // syntax error
```

```
class X { /* ... */ };

template<int i = (3 > 4) >      // OK
class Y { /* ... */ };
```

— *end example*]

14 A *template-parameter* of a template *template-parameter* is permitted to have a default *template-argument*. When such default arguments are specified, they apply to the template *template-parameter* in the scope of the template *template-parameter*. [*Example:*

```
template <class T = float> struct B {};
template <template <class TT = float> class T> struct A {
  inline void f();
  inline void g();
};
template <template <class TT> class T> void A<T>::f() {
  T<> t;            // error - TT has no default template argument
}
template <template <class TT = char> class T> void A<T>::g() {
    T<> t;          // OK - T<char>
}
```

— *end example*]

15 If a *template-parameter* is a *type-parameter* with an ellipsis prior to its optional *identifier* or is a *parameter-declaration* that declares a parameter pack (8.3.5), then the *template-parameter* is a template parameter pack (14.5.3). A template parameter pack that is a *parameter-declaration* whose type contains one or more unexpanded parameter packs is a pack expansion. Similarly, a template parameter pack that is a *type-parameter* with a *template-parameter-list* containing one or more unexpanded parameter packs is a pack expansion. A template parameter pack that is a pack expansion shall not expand a parameter pack declared in the same *template-parameter-list*. [*Example:*

```
template <class... Types> class Tuple;                  // Types is a template type parameter pack
                                                        // but not a pack expansion
template <class T, int... Dims> struct multi_array;     // Dims is a non-type template parameter pack
                                                        // but not a pack expansion
template<class... T> struct value_holder {
  template<T... Values> struct apply { };               // Values is a non-type template parameter pack
                                                        // and a pack expansion
};
template<class... T, T... Values> struct static_array;// error: Values expands template type parameter
                                                        // pack T within the same template parameter list
```

— *end example*]

14.2 Names of template specializations [temp.names]

1 A template specialization (14.7) can be referred to by a *template-id*:

simple-template-id:
template-name < *template-argument-list*$_{opt}$ >

template-id:
simple-template-id
operator-function-id < *template-argument-list*$_{opt}$ >
literal-operator-id < *template-argument-list*$_{opt}$ >

template-name:
identifier

template-argument-list:
template-argument ...$_{opt}$
template-argument-list , *template-argument* ...$_{opt}$
template-argument:
constant-expression
type-id
id-expression

[*Note:* The name lookup rules (3.4) are used to associate the use of a name with a template declaration; that is, to identify a name as a *template-name*. — *end note*]

2 For a *template-name* to be explicitly qualified by the template arguments, the name must be known to refer to a template.

3 After name lookup (3.4) finds that a name is a *template-name* or that an *operator-function-id* or a *literal-operator-id* refers to a set of overloaded functions any member of which is a function template, if this is followed by a `<`, the `<` is always taken as the delimiter of a *template-argument-list* and never as the less-than operator. When parsing a *template-argument-list*, the first non-nested `>`[138] is taken as the ending delimiter rather than a greater-than operator. Similarly, the first non-nested `>>` is treated as two consecutive but distinct `>` tokens, the first of which is taken as the end of the *template-argument-list* and completes the *template-id*. [*Note:* The second `>` token produced by this replacement rule may terminate an enclosing *template-id* construct or it may be part of a different construct (e.g. a cast). — *end note*] [*Example:*

```
template<int i> class X { /* ...  */ };

X< 1>2 > x1;                        // syntax error
X<(1>2)> x2;                        // OK

template<class T> class Y { /* ...  */ };
Y<X<1>> x3;                         // OK, same as Y<X<1> > x3;
Y<X<6>>1>> x4;                      // syntax error
Y<X<(6>>1)>> x5;                    // OK
```

— *end example*]

4 When the name of a member template specialization appears after `.` or `->` in a *postfix-expression* or after a *nested-name-specifier* in a *qualified-id*, and the object expression of the *postfix-expression* is type-dependent or the *nested-name-specifier* in the *qualified-id* refers to a dependent type, but the name is not a member of the current instantiation (14.6.2.1), the member template name must be prefixed by the keyword `template`. Otherwise the name is assumed to name a non-template. [*Example:*

```
struct X {
  template<std::size_t> X* alloc();
  template<std::size_t> static X* adjust();
};
template<class T> void f(T* p) {
  T* p1 = p->alloc<200>();          // ill-formed: < means less than
  T* p2 = p->template alloc<200>(); // OK: < starts template argument list
  T::adjust<100>();                 // ill-formed: < means less than
  T::template adjust<100>();        // OK: < starts template argument list
}
```

— *end example*]

5 A name prefixed by the keyword `template` shall be a *template-id* or the name shall refer to a class template. [*Note:* The keyword `template` may not be applied to non-template members of class templates. — *end*

138) A `>` that encloses the *type-id* of a `dynamic_cast`, `static_cast`, `reinterpret_cast` or `const_cast`, or which encloses the *template-argument*s of a subsequent *template-id*, is considered nested for the purpose of this description.

note] [*Note:* As is the case with the `typename` prefix, the `template` prefix is allowed in cases where it is not strictly necessary; i.e., when the *nested-name-specifier* or the expression on the left of the `->` or `.` is not dependent on a *template-parameter*, or the use does not appear in the scope of a template. — *end note*] [*Example:*

```
template <class T> struct A {
  void f(int);
  template <class U> void f(U);
};

template <class T> void f(T t) {
  A<T> a;
  a.template f<>(t);                // OK: calls template
  a.template f(t);                  // error: not a template-id
}

template <class T> struct B {
  template <class T2> struct C { };
};

// OK: T::template C names a class template:
template <class T, template <class X> class TT = T::template C> struct D { };
D<B<int> > db;
```

— *end example*]

6 A *simple-template-id* that names a class template specialization is a *class-name* (Clause 9).

7 A *template-id* that names an alias template specialization is a *type-name*.

14.3 Template arguments [temp.arg]

1 There are three forms of *template-argument*, corresponding to the three forms of *template-parameter*: type, non-type and template. The type and form of each *template-argument* specified in a *template-id* shall match the type and form specified for the corresponding parameter declared by the template in its *template-parameter-list*. When the parameter declared by the template is a template parameter pack (14.5.3), it will correspond to zero or more *template-argument*s. [*Example:*

```
template<class T> class Array {
  T* v;
  int sz;
public:
  explicit Array(int);
  T& operator[](int);
  T& elem(int i) { return v[i]; }
};

Array<int> v1(20);
typedef std::complex<double> dcomplex;  // std::complex is a standard
                                        // library template
Array<dcomplex> v2(30);
Array<dcomplex> v3(40);

void bar() {
  v1[3] = 7;
  v2[3] = v3.elem(4) = dcomplex(7,8);
}
```

— *end example*]

2 In a *template-argument*, an ambiguity between a *type-id* and an expression is resolved to a *type-id*, regardless of the form of the corresponding *template-parameter*.[139] [*Example:*

```
template<class T> void f();
template<int I> void f();

void g() {
  f<int()>();       // int() is a type-id: call the first f()
}
```

— *end example*]

3 The name of a *template-argument* shall be accessible at the point where it is used as a *template-argument*. [*Note:* If the name of the *template-argument* is accessible at the point where it is used as a *template-argument*, there is no further access restriction in the resulting instantiation where the corresponding *template-parameter* name is used. — *end note*] [*Example:*

```
template<class T> class X {
  static T t;
};

class Y {
private:
  struct S { /* ... */ };
  X<S> x;           // OK: S is accessible
                    // X<Y::S> has a static member of type Y::S
                    // OK: even though Y::S is private
};

X<Y::S> y;          // error: S not accessible
```

— *end example*] For a *template-argument* that is a class type or a class template, the template definition has no special access rights to the members of the *template-argument*. [*Example:*

```
template <template <class TT> class T> class A {
  typename T<int>::S s;
};

template <class U> class B {
private:
  struct S { /* ... */ };
};

A<B> b;             // ill-formed: A has no access to B::S
```

— *end example*]

4 When template argument packs or default *template-argument*s are used, a *template-argument* list can be empty. In that case the empty <> brackets shall still be used as the *template-argument-list*. [*Example:*

```
template<class T = char> class String;
String<>* p;                    // OK: String<char>
String* q;                      // syntax error
```

139) There is no such ambiguity in a default *template-argument* because the form of the *template-parameter* determines the allowable forms of the *template-argument*.

```
template<class ... Elements> class Tuple;
Tuple<>* t;                     // OK: Elements is empty
Tuple* u;                       // syntax error
```

— *end example*]

5 An explicit destructor call (12.4) for an object that has a type that is a class template specialization may explicitly specify the *template-argument*s. [*Example:*

```
template<class T> struct A {
  ~A();
};
void f(A<int>* p, A<int>* q) {
  p->A<int>::~A();              // OK: destructor call
  q->A<int>::~A<int>();         // OK: destructor call
}
```

— *end example*]

6 If the use of a *template-argument* gives rise to an ill-formed construct in the instantiation of a template specialization, the program is ill-formed.

7 When the template in a *template-id* is an overloaded function template, both non-template functions in the overload set and function templates in the overload set for which the *template-argument*s do not match the *template-parameter*s are ignored. If none of the function templates have matching *template-parameter*s, the program is ill-formed.

8 A *template-argument* followed by an ellipsis is a pack expansion (14.5.3).

14.3.1 Template type arguments [temp.arg.type]

1 A *template-argument* for a *template-parameter* which is a type shall be a *type-id*.

2 [*Example:*

```
template <class T> class X { };
template <class T> void f(T t) { }
struct { } unnamed_obj;

void f() {
  struct A { };
  enum { e1 };
  typedef struct { } B;
  B b;
  X<A> x1;          // OK
  X<A*> x2;         // OK
  X<B> x3;          // OK
  f(e1);            // OK
  f(unnamed_obj);   // OK
  f(b);             // OK
}
```

— *end example*] [*Note:* A template type argument may be an incomplete type (3.9). — *end note*]

3 If a declaration acquires a function type through a type dependent on a *template-parameter* and this causes a declaration that does not use the syntactic form of a function declarator to have function type, the program is ill-formed. [*Example:*

```
template<class T> struct A {
  static T t;
```

```
};
typedef int function();
A<function> a;                  // ill-formed: would declare A<function>::t
                                // as a static member function
```

— *end example*]

14.3.2 Template non-type arguments [temp.arg.nontype]

1 A *template-argument* for a non-type *template-parameter* shall be a converted constant expression (5.20) of the type of the *template-parameter*. For a non-type *template-parameter* of reference or pointer type, the value of the constant expression shall not refer to (or for a pointer type, shall not be the address of):

(1.1) — a subobject (1.8),

(1.2) — a temporary object (12.2),

(1.3) — a string literal (2.13.5),

(1.4) — the result of a `typeid` expression (5.2.8), or

(1.5) — a predefined `__func__` variable (8.4.1).

[*Note:* If the *template-argument* represents a set of overloaded functions (or a pointer or member pointer to such), the matching function is selected from the set (13.4). — *end note*]

2 [*Example:*

```
template<const int* pci> struct X { /* ... */ };
int ai[10];
X<ai> xi;                       // array to pointer and qualification conversions

struct Y { /* ... */ };
template<const Y& b> struct Z { /* ... */ };
Y y;
Z<y> z;                         // no conversion, but note extra cv-qualification

template<int (&pa)[5]> struct W { /* ... */ };
int b[5];
W<b> w;                         // no conversion

void f(char);
void f(int);

template<void (*pf)(int)> struct A { /* ... */ };

A<&f> a;                        // selects f(int)
```

— *end example*]

3 [*Note:* A string literal (2.13.5) is not an acceptable *template-argument*. [*Example:*

```
template<class T, const char* p> class X {
  /* ... */
};

X<int, "Studebaker"> x1;        // error: string literal as template-argument

const char p[] = "Vivisectionist";
X<int,p> x2;                    // OK
```

— *end example*] — *end note*]

4 [*Note:* The address of an array element or non-static data member is not an acceptable *template-argument*. [*Example:*

```
template<int* p> class X { };

int a[10];
struct S { int m; static int s; } s;

X<&a[2]> x3;                    // error: address of array element
X<&s.m> x4;                     // error: address of non-static member
X<&s.s> x5;                     // OK: address of static member
X<&S::s> x6;                    // OK: address of static member
```

— *end example*] — *end note*]

5 [*Note:* A temporary object is not an acceptable *template-argument* when the corresponding *template-parameter* has reference type. [*Example:*

```
template<const int& CRI> struct B { /* ... */ };

B<1> b2;                        // error: temporary would be required for template argument

int c = 1;
B<c> b1;                        // OK
```

— *end example*] — *end note*]

14.3.3 Template template arguments [temp.arg.template]

1 A *template-argument* for a template *template-parameter* shall be the name of a class template or an alias template, expressed as *id-expression*. When the *template-argument* names a class template, only primary class templates are considered when matching the template template argument with the corresponding parameter; partial specializations are not considered even if their parameter lists match that of the template template parameter.

2 Any partial specializations (14.5.5) associated with the primary class template or primary variable template are considered when a specialization based on the template *template-parameter* is instantiated. If a specialization is not visible at the point of instantiation, and it would have been selected had it been visible, the program is ill-formed; no diagnostic is required. [*Example:*

```
template<class T> class A {       // primary template
  int x;
};
template<class T> class A<T*> { // partial specialization
  long x;
};
template<template<class U> class V> class C {
  V<int>  y;
  V<int*> z;
};
C<A> c;                           // V<int> within C<A> uses the primary template,
                                  // so c.y.x has type int
                                  // V<int*> within C<A> uses the partial specialization,
                                  // so c.z.x has type long
```

— *end example*]

3 A *template-argument* matches a template *template-parameter* `P` when each of the template parameters in the *template-parameter-list* of the *template-argument*'s corresponding class template or alias template `A` matches the corresponding template parameter in the *template-parameter-list* of `P`. Two template parameters match if they are of the same kind (type, non-type, template), for non-type *template-parameter*s, their types are equivalent (14.5.6.1), and for template *template-parameter*s, each of their corresponding *template-parameter*s matches, recursively. When `P`'s *template-parameter-list* contains a template parameter pack (14.5.3), the template parameter pack will match zero or more template parameters or template parameter packs in the *template-parameter-list* of `A` with the same type and form as the template parameter pack in `P` (ignoring whether those template parameters are template parameter packs).

[*Example:*

```
template<class T> class A { /* ... */ };
template<class T, class U = T> class B { /* ... */ };
template <class ... Types> class C { /* ... */ };

template<template<class> class P> class X { /* ... */ };
template<template<class ...> class Q> class Y { /* ... */ };

X<A> xa;            // OK
X<B> xb;            // ill-formed: default arguments for the parameters of a template argument are ignored
X<C> xc;            // ill-formed: a template parameter pack does not match a template parameter

Y<A> ya;            // OK
Y<B> yb;            // OK
Y<C> yc;            // OK
```

— *end example*]

[*Example:*

```
template <class T> struct eval;

template <template <class, class...> class TT, class T1, class... Rest>
struct eval<TT<T1, Rest...>> { };

template <class T1> struct A;
template <class T1, class T2> struct B;
template <int N> struct C;
template <class T1, int N> struct D;
template <class T1, class T2, int N = 17> struct E;

eval<A<int>> eA;                // OK: matches partial specialization of eval
eval<B<int, float>> eB;         // OK: matches partial specialization of eval
eval<C<17>> eC;                 // error: C does not match TT in partial specialization
eval<D<int, 17>> eD;            // error: D does not match TT in partial specialization
eval<E<int, float>> eE;         // error: E does not match TT in partial specialization
```

— *end example*]

14.4 Type equivalence [temp.type]

1 Two *template-id*s refer to the same class, function, or variable if

(1.1) — their *template-name*s, *operator-function-id*s, or *literal-operator-id*s refer to the same template and

(1.2) — their corresponding type *template-argument*s are the same type and

(1.3) — their corresponding non-type template arguments of integral or enumeration type have identical values and

(1.4) — their corresponding non-type *template-argument*s of pointer type refer to the same object or function or are both the null pointer value and

(1.5) — their corresponding non-type *template-argument*s of pointer-to-member type refer to the same class member or are both the null member pointer value and

(1.6) — their corresponding non-type *template-argument*s of reference type refer to the same object or function and

(1.7) — their corresponding template *template-argument*s refer to the same template.

[*Example:*

```
template<class E, int size> class buffer { /* ... */ };
buffer<char,2*512> x;
buffer<char,1024> y;
```

declares `x` and `y` to be of the same type, and

```
template<class T, void(*err_fct)()> class list { /* ... */ };
list<int,&error_handler1> x1;
list<int,&error_handler2> x2;
list<int,&error_handler2> x3;
list<char,&error_handler2> x4;
```

declares `x2` and `x3` to be of the same type. Their type differs from the types of `x1` and `x4`.

```
template<class T> struct X { };
template<class> struct Y { };
template<class T> using Z = Y<T>;
X<Y<int> > y;
X<Z<int> > z;
```

declares `y` and `z` to be of the same type. — *end example*]

2 If an expression *e* involves a template parameter, `decltype(`*e*`)` denotes a unique dependent type. Two such *decltype-specifier*s refer to the same type only if their *expression*s are equivalent (14.5.6.1). [*Note:* however, it may be aliased, e.g., by a *typedef-name*. — *end note*]

14.5 Template declarations [temp.decls]

1 A *template-id*, that is, the *template-name* followed by a *template-argument-list* shall not be specified in the declaration of a primary template declaration. [*Example:*

```
template<class T1, class T2, int I> class A<T1, T2, I> { };      // error
template<class T1, int I> void sort<T1, I>(T1 data[I]);          // error
```

— *end example*] [*Note:* However, this syntax is allowed in class template partial specializations (14.5.5). — *end note*]

2 For purposes of name lookup and instantiation, default arguments and *exception-specification*s of function templates and default arguments and *exception-specification*s of member functions of class templates are considered definitions; each default argument or *exception-specification* is a separate definition which is unrelated to the function template definition or to any other default arguments or *exception-specification*s.

3 Because an *alias-declaration* cannot declare a *template-id*, it is not possible to partially or explicitly specialize an alias template.

14.5.1 Class templates [temp.class]

1 A class *template* defines the layout and operations for an unbounded set of related types. [*Example:* a single class template `List` might provide a common definition for list of `int`, list of `float`, and list of pointers to `Shapes`. — *end example*]

[*Example:* An array class template might be declared like this:

```
template<class T> class Array {
  T* v;
  int sz;
public:
  explicit Array(int);
  T& operator[](int);
  T& elem(int i) { return v[i]; }
};
```

2 The prefix `template <class T>` specifies that a template is being declared and that a *type-name* `T` will be used in the declaration. In other words, `Array` is a parameterized type with `T` as its parameter. — *end example*]

3 When a member function, a member class, a member enumeration, a static data member or a member template of a class template is defined outside of the class template definition, the member definition is defined as a template definition in which the *template-parameter*s are those of the class template. The names of the template parameters used in the definition of the member may be different from the template parameter names used in the class template definition. The template argument list following the class template name in the member definition shall name the parameters in the same order as the one used in the template parameter list of the member. Each template parameter pack shall be expanded with an ellipsis in the template argument list. [*Example:*

```
template<class T1, class T2> struct A {
  void f1();
  void f2();
};

template<class T2, class T1> void A<T2,T1>::f1() { }    // OK
template<class T2, class T1> void A<T1,T2>::f2() { }    // error

template<class ... Types> struct B {
  void f3();
  void f4();
};

template<class ... Types> void B<Types ...>::f3() { }    // OK
template<class ... Types> void B<Types>::f4() { }        // error
```

— *end example*]

4 In a redeclaration, partial specialization, explicit specialization or explicit instantiation of a class template, the *class-key* shall agree in kind with the original class template declaration (7.1.6.3).

14.5.1.1 Member functions of class templates [temp.mem.func]

1 A member function of a class template may be defined outside of the class template definition in which it is declared. [*Example:*

```
template<class T> class Array {
  T* v;
```

```
  int sz;
public:
  explicit Array(int);
  T& operator[](int);
  T& elem(int i) { return v[i]; }
};
```

declares three function templates. The subscript function might be defined like this:

```
template<class T> T& Array<T>::operator[](int i) {
  if (i<0 || sz<=i) error("Array: range error");
  return v[i];
}
```

— *end example*]

2 The *template-argument*s for a member function of a class template are determined by the *template-argument*s of the type of the object for which the member function is called. [*Example:* the *template-argument* for `Array<T>::operator[]()` will be determined by the `Array` to which the subscripting operation is applied.

```
Array<int> v1(20);
Array<dcomplex> v2(30);

v1[3] = 7;                      // Array<int>::operator[]()
v2[3] = dcomplex(7,8);          // Array<dcomplex>::operator[]()
```

— *end example*]

14.5.1.2 Member classes of class templates [temp.mem.class]

1 A member class of a class template may be defined outside the class template definition in which it is declared. [*Note:* The member class must be defined before its first use that requires an instantiation (14.7.1). For example,

```
template<class T> struct A {
  class B;
};
A<int>::B* b1;                  // OK: requires A to be defined but not A::B
template<class T> class A<T>::B { };
A<int>::B  b2;                  // OK: requires A::B to be defined
```

— *end note*]

14.5.1.3 Static data members of class templates [temp.static]

1 A definition for a static data member or static data member template may be provided in a namespace scope enclosing the definition of the static member's class template. [*Example:*

```
template<class T> class X {
  static T s;
};
template<class T> T X<T>::s = 0;

struct limits {
  template<class T>
    static const T min;           // declaration
};

template<class T>
  const T limits::min = { };      // definition
```

— *end example*]

2 An explicit specialization of a static data member declared as an array of unknown bound can have a different bound from its definition, if any. [*Example:*

```
template <class T> struct A {
  static int i[];
};
template <class T> int A<T>::i[4];     // 4 elements
template <> int A<int>::i[] = { 1 };   // OK: 1 element
```

— *end example*]

14.5.1.4 Enumeration members of class templates [temp.mem.enum]

1 An enumeration member of a class template may be defined outside the class template definition. [*Example:*

```
template<class T> struct A {
  enum E : T;
};
A<int> a;
template<class T> enum A<T>::E : T { e1, e2 };
A<int>::E e = A<int>::e1;
```

— *end example*]

14.5.2 Member templates [temp.mem]

1 A template can be declared within a class or class template; such a template is called a member template. A member template can be defined within or outside its class definition or class template definition. A member template of a class template that is defined outside of its class template definition shall be specified with the *template-parameter*s of the class template followed by the *template-parameter*s of the member template. [*Example:*

```
template<class T> struct string {
  template<class T2> int compare(const T2&);
  template<class T2> string(const string<T2>& s) { /* ... */ }
};

template<class T> template<class T2> int string<T>::compare(const T2& s) {
}
```

— *end example*]

2 A local class of non-closure type shall not have member templates. Access control rules (Clause 11) apply to member template names. A destructor shall not be a member template. A non-template member function (8.3.5) with a given name and type and a member function template of the same name, which could be used to generate a specialization of the same type, can both be declared in a class. When both exist, a use of that name and type refers to the non-template member unless an explicit template argument list is supplied. [*Example:*

```
template <class T> struct A {
  void f(int);
  template <class T2> void f(T2);
};

template <> void A<int>::f(int) { }                    // non-template member function
template <> template <> void A<int>::f<>(int) { }      // member function template specialization
```

```
int main() {
  A<char> ac;
  ac.f(1);          // non-template
  ac.f('c');        // template
  ac.f<>(1);        // template
}
```

— *end example*]

3 A member function template shall not be virtual. [*Example:*

```
template <class T> struct AA {
  template <class C> virtual void g(C);   // error
  virtual void f();                       // OK
};
```

— *end example*]

4 A specialization of a member function template does not override a virtual function from a base class. [*Example:*

```
class B {
  virtual void f(int);
};

class D : public B {
  template <class T> void f(T); // does not override B::f(int)
  void f(int i) { f<>(i); }     // overriding function that calls
                                // the template instantiation
};
```

— *end example*]

5 A specialization of a conversion function template is referenced in the same way as a non-template conversion function that converts to the same type. [*Example:*

```
struct A {
  template <class T> operator T*();
};
template <class T> A::operator T*(){ return 0; }
template <> A::operator char*(){ return 0; }    // specialization
template A::operator void*();                   // explicit instantiation

int main() {
  A a;
  int* ip;
  ip = a.operator int*();       // explicit call to template operator
                                // A::operator int*()
}
```

— *end example*] [*Note:* Because the explicit template argument list follows the function template name, and because conversion member function templates and constructor member function templates are called without using a function name, there is no way to provide an explicit template argument list for these function templates. — *end note*]

6 A specialization of a conversion function template is not found by name lookup. Instead, any conversion function templates visible in the context of the use are considered. For each such operator, if argument deduction succeeds (14.8.2.3), the resulting specialization is used as if found by name lookup.

7 A *using-declaration* in a derived class cannot refer to a specialization of a conversion function template in a base class.

8 Overload resolution (13.3.3.2) and partial ordering (14.5.6.2) are used to select the best conversion function among multiple specializations of conversion function templates and/or non-template conversion functions.

14.5.3 Variadic templates [temp.variadic]

1 A *template parameter pack* is a template parameter that accepts zero or more template arguments. [*Example:*

```
template<class ... Types> struct Tuple { };

Tuple<> t0;                     // Types contains no arguments
Tuple<int> t1;                  // Types contains one argument: int
Tuple<int, float> t2;           // Types contains two arguments: int and float
Tuple<0> error;                 // error: 0 is not a type
```

— *end example*]

2 A *function parameter pack* is a function parameter that accepts zero or more function arguments. [*Example:*

```
template<class ... Types> void f(Types ... args);

f();                // OK: args contains no arguments
f(1);               // OK: args contains one argument: int
f(2, 1.0);          // OK: args contains two arguments: int and double
```

— *end example*]

3 A *parameter pack* is either a template parameter pack or a function parameter pack.

4 A *pack expansion* consists of a *pattern* and an ellipsis, the instantiation of which produces zero or more instantiations of the pattern in a list (described below). The form of the pattern depends on the context in which the expansion occurs. Pack expansions can occur in the following contexts:

(4.1) — In a function parameter pack (8.3.5); the pattern is the *parameter-declaration* without the ellipsis.

(4.2) — In a template parameter pack that is a pack expansion (14.1):

(4.2.1) — if the template parameter pack is a *parameter-declaration*; the pattern is the *parameter-declaration* without the ellipsis;

(4.2.2) — if the template parameter pack is a *type-parameter* with a *template-parameter-list*; the pattern is the corresponding *type-parameter* without the ellipsis.

(4.3) — In an *initializer-list* (8.5); the pattern is an *initializer-clause*.

(4.4) — In a *base-specifier-list* (Clause 10); the pattern is a *base-specifier*.

(4.5) — In a *mem-initializer-list* (12.6.2) for a *mem-initializer* whose *mem-initializer-id* denotes a base class; the pattern is the *mem-initializer*.

(4.6) — In a *template-argument-list* (14.3); the pattern is a *template-argument*.

(4.7) — In a *dynamic-exception-specification* (15.4); the pattern is a *type-id*.

(4.8) — In an *attribute-list* (7.6.1); the pattern is an *attribute*.

(4.9) — In an *alignment-specifier* (7.6.2); the pattern is the *alignment-specifier* without the ellipsis.

(4.10) — In a *capture-list* (5.1.2); the pattern is a *capture*.

(4.11) — In a `sizeof...` expression (5.3.3); the pattern is an *identifier*.

(4.12) — In a *fold-expression* (5.1.3); the pattern is the *cast-expression* that contains an unexpanded parameter pack.

5 For the purpose of determining whether a parameter pack satisfies a rule regarding entities other than parameter packs, the parameter pack is considered to be the entity that would result from an instantiation of the pattern in which it appears.

[*Example:*

```
template<class ... Types> void f(Types ... rest);
template<class ... Types> void g(Types ... rest) {
  f(&rest ...);      // "&rest ..." is a pack expansion; "&rest" is its pattern
}
```

— *end example*]

6 A parameter pack whose name appears within the pattern of a pack expansion is expanded by that pack expansion. An appearance of the name of a parameter pack is only expanded by the innermost enclosing pack expansion. The pattern of a pack expansion shall name one or more parameter packs that are not expanded by a nested pack expansion; such parameter packs are called *unexpanded* parameter packs in the pattern. All of the parameter packs expanded by a pack expansion shall have the same number of arguments specified. An appearance of a name of a parameter pack that is not expanded is ill-formed. [*Example:*

```
template<typename...> struct Tuple {};
template<typename T1, typename T2> struct Pair {};

template<class ... Args1> struct zip {
  template<class ... Args2> struct with {
    typedef Tuple<Pair<Args1, Args2> ... > type;
  };
};

typedef zip<short, int>::with<unsigned short, unsigned>::type T1;
    // T1 is Tuple<Pair<short, unsigned short>, Pair<int, unsigned>>
typedef zip<short>::with<unsigned short, unsigned>::type T2;
    // error: different number of arguments specified for Args1 and Args2

template<class ... Args>
  void g(Args ... args) {                // OK: Args is expanded by the function parameter pack args
    f(const_cast<const Args*>(&args)...); // OK: "Args" and "args" are expanded
    f(5 ...);                             // error: pattern does not contain any parameter packs
    f(args);                              // error: parameter pack "args" is not expanded
    f(h(args ...) + args ...);            // OK: first "args" expanded within h, second
                                          // "args" expanded within f
  }
```

— *end example*]

7 The instantiation of a pack expansion that is neither a `sizeof...` expression nor a *fold-expression* produces a list $\mathtt{E}_1, \mathtt{E}_2, ..., \mathtt{E}_N$, where N is the number of elements in the pack expansion parameters. Each $\mathtt{E}_i$ is generated by instantiating the pattern and replacing each pack expansion parameter with its ith element. Such an element, in the context of the instantiation, is interpreted as follows:

(7.1) — if the pack is a template parameter pack, the element is a template parameter (14.1) of the corresponding kind (type or non-type) designating the type or value from the template argument; otherwise,

(7.2) — if the pack is a function parameter pack, the element is an *id-expression* designating the function parameter that resulted from the instantiation of the pattern where the pack is declared.

All of the $\texttt{E}_i$ become elements in the enclosing list. [*Note:* The variety of list varies with the context: *expression-list*, *base-specifier-list*, *template-argument-list*, etc. — *end note*] When N is zero, the instantiation of the expansion produces an empty list. Such an instantiation does not alter the syntactic interpretation of the enclosing construct, even in cases where omitting the list entirely would otherwise be ill-formed or would result in an ambiguity in the grammar. [*Example:*

```
template<class... T> struct X : T... { };
template<class... T> void f(T... values) {
  X<T...> x(values...);
}

template void f<>();  // OK: X<> has no base classes
                      // x is a variable of type X<> that is value-initialized
```

— *end example*]

8 The instantiation of a `sizeof...` expression (5.3.3) produces an integral constant containing the number of elements in the parameter pack it expands.

9 The instantiation of a *fold-expression* produces:

(9.1) — $(\texttt{E}_1\ \mathit{op}\ \texttt{E}_2)\ \mathit{op}\ \cdots)\ \mathit{op}\ \texttt{E}_N$ for a unary left fold,

(9.2) — $\texttt{E}_1\ \mathit{op}\ (\cdots\ \mathit{op}\ (\texttt{E}_{N-1}\ \mathit{op}\ \texttt{E}_N))$ for a unary right fold,

(9.3) — $(((\texttt{E}\ \mathit{op}\ \texttt{E}_1)\ \mathit{op}\ \texttt{E}_2)\ \mathit{op}\ \cdots)\ \mathit{op}\ \texttt{E}_N$ for a binary left fold, and

(9.4) — $\texttt{E}_1\ \mathit{op}\ (\cdots\ \mathit{op}\ (\texttt{E}_{N-1}\ \mathit{op}\ (\texttt{E}_N\ \mathit{op}\ \texttt{E})))$ for a binary right fold.

In each case, *op* is the *fold-operator*, N is the number of elements in the pack expansion parameters, and each $\texttt{E}_i$ is generated by instantiating the pattern and replacing each pack expansion parameter with its ith element. For a binary fold-expression, `E` is generated by instantiating the *cast-expression* that did not contain an unexpanded parameter pack. [*Example:*

```
template<typename ...Args>
  bool all(Args ...args) { return (... && args); }

bool b = all(true, true, true, false);
```

Within the instantiation of `all`, the returned expression expands to `((true && true) && true) && false`, which evalutes to `false`. — *end example*] If N is zero for a unary fold-expression, the value of the expression is shown in Table 12; if the operator is not listed in Table 12, the instantiation is ill-formed.

Table 12 — Value of folding empty sequences

Operator	Value when parameter pack is empty
*	1
+	int()
&	-1
|	int()
&&	true
||	false
,	void()

14.5.4 Friends [temp.friend]

1 A friend of a class or class template can be a function template or class template, a specialization of a function template or class template, or a non-template function or class. For a friend function declaration that is not a template declaration:

(1.1) — if the name of the friend is a qualified or unqualified *template-id*, the friend declaration refers to a specialization of a function template, otherwise,

(1.2) — if the name of the friend is a *qualified-id* and a matching non-template function is found in the specified class or namespace, the friend declaration refers to that function, otherwise,

(1.3) — if the name of the friend is a *qualified-id* and a matching function template is found in the specified class or namespace, the friend declaration refers to the deduced specialization of that function template (14.8.2.6), otherwise,

(1.4) — the name shall be an *unqualified-id* that declares (or redeclares) a non-template function.

[*Example:*

```
template<class T> class task;
template<class T> task<T>* preempt(task<T>*);

template<class T> class task {
  friend void next_time();
  friend void process(task<T>*);
  friend task<T>* preempt<T>(task<T>*);
  template<class C> friend int func(C);

  friend class task<int>;
  template<class P> friend class frd;
};
```

Here, each specialization of the `task` class template has the function `next_time` as a friend; because `process` does not have explicit *template-argument*s, each specialization of the `task` class template has an appropriately typed function `process` as a friend, and this friend is not a function template specialization; because the friend `preempt` has an explicit *template-argument* `T`, each specialization of the `task` class template has the appropriate specialization of the function template `preempt` as a friend; and each specialization of the `task` class template has all specializations of the function template `func` as friends. Similarly, each specialization of the `task` class template has the class template specialization `task<int>` as a friend, and has all specializations of the class template `frd` as friends. — *end example*]

2 A friend template may be declared within a class or class template. A friend function template may be defined within a class or class template, but a friend class template may not be defined in a class or class template. In these cases, all specializations of the friend class or friend function template are friends of the class or class template granting friendship. [*Example:*

```
class A {
  template<class T> friend class B;                 // OK
  template<class T> friend void f(T){ /* ...  */ }  // OK
};
```

— *end example*]

3 A template friend declaration specifies that all specializations of that template, whether they are implicitly instantiated (14.7.1), partially specialized (14.5.5) or explicitly specialized (14.7.3), are friends of the class containing the template friend declaration. [*Example:*

```
class X {
  template<class T> friend struct A;
  class Y { };
};

template<class T> struct A { X::Y ab; };            // OK
template<class T> struct A<T*> { X::Y ab; };        // OK
```

— *end example*]

4 When a function is defined in a friend function declaration in a class template, the function is instantiated when the function is odr-used (3.2). The same restrictions on multiple declarations and definitions that apply to non-template function declarations and definitions also apply to these implicit definitions.

5 A member of a class template may be declared to be a friend of a non-template class. In this case, the corresponding member of every specialization of the primary class template and class template partial specializations thereof is a friend of the class granting friendship. For explicit specializations and specializations of partial specializations, the corresponding member is the member (if any) that has the same name, kind (type, function, class template, or function template), template parameters, and signature as the member of the class template instantiation that would otherwise have been generated. [*Example:*

```
template<class T> struct A {
  struct B { };
  void f();
  struct D {
    void g();
  };
};
template<> struct A<int> {
  struct B { };
  int f();
  struct D {
    void g();
  };
};

class C {
  template<class T> friend struct A<T>::B;     // grants friendship to A<int>::B even though
                                               // it is not a specialization of A<T>::B
  template<class T> friend void A<T>::f();     // does not grant friendship to A<int>::f()
                                               // because its return type does not match
  template<class T> friend void A<T>::D::g(); // does not grant friendship to A<int>::D::g()
                                               // because A<int>::D is not a specialization of A<T>::D
};
```

— *end example*]

6 [*Note:* A friend declaration may first declare a member of an enclosing namespace scope (14.6.5). — *end note*]

7 A friend template shall not be declared in a local class.

8 Friend declarations shall not declare partial specializations. [*Example:*

```
template<class T> class A { };
class X {
  template<class T> friend class A<T*>; // error
};
```

— *end example*]

9 When a friend declaration refers to a specialization of a function template, the function parameter declarations shall not include default arguments, nor shall the inline specifier be used in such a declaration.

14.5.5 Class template partial specializations [temp.class.spec]

1 A *primary* class template declaration is one in which the class template name is an identifier. A template declaration in which the class template name is a *simple-template-id* is a *partial specialization* of the class template named in the *simple-template-id*. A partial specialization of a class template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization (14.5.5.1). The primary template shall be declared before any specializations of that template. A partial specialization shall be declared before the first use of a class template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation in every translation unit in which such a use occurs; no diagnostic is required.

2 Each class template partial specialization is a distinct template and definitions shall be provided for the members of a template partial specialization (14.5.5.3).

3 [*Example:*

```
template<class T1, class T2, int I> class A             { };    // #1
template<class T, int I>            class A<T, T*, I>   { };    // #2
template<class T1, class T2, int I> class A<T1*, T2, I> { };    // #3
template<class T>                   class A<int, T*, 5> { };    // #4
template<class T1, class T2, int I> class A<T1, T2*, I> { };    // #5
```

The first declaration declares the primary (unspecialized) class template. The second and subsequent declarations declare partial specializations of the primary template. — *end example*]

4 The template parameters are specified in the angle bracket enclosed list that immediately follows the keyword `template`. For partial specializations, the template argument list is explicitly written immediately following the class template name. For primary templates, this list is implicitly described by the template parameter list. Specifically, the order of the template arguments is the sequence in which they appear in the template parameter list. [*Example:* the template argument list for the primary template in the example above is `<T1, T2, I>`. — *end example*] [*Note:* The template argument list shall not be specified in the primary template declaration. For example,

```
template<class T1, class T2, int I> class A<T1, T2, I>  { };    // error
```

— *end note*]

5 A class template partial specialization may be declared or redeclared in any namespace scope in which the corresponding primary template may be defined (7.3.1.2 and 14.5.2). [*Example:*

```
template<class T> struct A {
  struct C {
    template<class T2> struct B { };
  };
};

// partial specialization of A<T>::C::B<T2>
template<class T> template<class T2>
  struct A<T>::C::B<T2*> { };

A<short>::C::B<int*> absip;      // uses partial specialization
```

— *end example*]

6 Partial specialization declarations themselves are not found by name lookup. Rather, when the primary template name is used, any previously-declared partial specializations of the primary template are also considered. One consequence is that a *using-declaration* which refers to a class template does not restrict the set of partial specializations which may be found through the *using-declaration*. [*Example:*

```
namespace N {
  template<class T1, class T2> class A { };          // primary template
}

using N::A;                                  // refers to the primary template

namespace N {
  template<class T> class A<T, T*> { }; // partial specialization
}

A<int,int*> a;                     // uses the partial specialization, which is found through
                                   // the using declaration which refers to the primary template
```

— *end example*]

7 A non-type argument is non-specialized if it is the name of a non-type parameter. All other non-type arguments are specialized.

8 Within the argument list of a class template partial specialization, the following restrictions apply:

(8.1) — A partially specialized non-type argument expression shall not involve a template parameter of the partial specialization except when the argument expression is a simple *identifier*. [*Example:*

```
template <int I, int J> struct A {};
template <int I> struct A<I+5, I*2> {}; // error

template <int I, int J> struct B {};
template <int I> struct B<I, I> {};     // OK
```

— *end example*]

(8.2) — The type of a template parameter corresponding to a specialized non-type argument shall not be dependent on a parameter of the specialization. [*Example:*

```
template <class T, T t> struct C {};
template <class T> struct C<T, 1>;                  // error

template< int X, int (*array_ptr)[X] > class A {};
int array[5];
template< int X > class A<X,&array> { };            // error
```

— *end example*]

(8.3) — The argument list of the specialization shall not be identical to the implicit argument list of the primary template.

(8.4) — The specialization shall be more specialized than the primary template (14.5.5.2).

(8.5) — The template parameter list of a specialization shall not contain default template argument values.[140]

(8.6) — An argument shall not contain an unexpanded parameter pack. If an argument is a pack expansion (14.5.3), it shall be the last argument in the template argument list.

140) There is no way in which they could be used.

14.5.5.1 Matching of class template partial specializations [temp.class.spec.match]

1 When a class template is used in a context that requires an instantiation of the class, it is necessary to determine whether the instantiation is to be generated using the primary template or one of the partial specializations. This is done by matching the template arguments of the class template specialization with the template argument lists of the partial specializations.

(1.1) — If exactly one matching specialization is found, the instantiation is generated from that specialization.

(1.2) — If more than one matching specialization is found, the partial order rules (14.5.5.2) are used to determine whether one of the specializations is more specialized than the others. If none of the specializations is more specialized than all of the other matching specializations, then the use of the class template is ambiguous and the program is ill-formed.

(1.3) — If no matches are found, the instantiation is generated from the primary template.

2 A partial specialization matches a given actual template argument list if the template arguments of the partial specialization can be deduced from the actual template argument list (14.8.2). [*Example:*

```
A<int, int, 1>   a1;          // uses #1
A<int, int*, 1>  a2;          // uses #2, T is int, I is 1
A<int, char*, 5> a3;          // uses #4, T is char
A<int, char*, 1> a4;          // uses #5, T1 is int, T2 is char, I is 1
A<int*, int*, 2> a5;          // ambiguous: matches #3 and #5
```

— *end example*]

3 A non-type template argument can also be deduced from the value of an actual template argument of a non-type parameter of the primary template. [*Example:* the declaration of `a2` above. — *end example*]

4 In a type name that refers to a class template specialization, (e.g., `A<int, int, 1>`) the argument list shall match the template parameter list of the primary template. The template arguments of a specialization are deduced from the arguments of the primary template.

14.5.5.2 Partial ordering of class template specializations [temp.class.order]

1 For two class template partial specializations, the first is *more specialized* than the second if, given the following rewrite to two function templates, the first function template is more specialized than the second according to the ordering rules for function templates (14.5.6.2):

(1.1) — the first function template has the same template parameters as the first partial specialization and has a single function parameter whose type is a class template specialization with the template arguments of the first partial specialization, and

(1.2) — the second function template has the same template parameters as the second partial specialization and has a single function parameter whose type is a class template specialization with the template arguments of the second partial specialization.

2 [*Example:*

```
template<int I, int J, class T> class X { };
template<int I, int J>          class X<I, J, int> { }; // #1
template<int I>                 class X<I, I, int> { }; // #2

template<int I, int J> void f(X<I, J, int>);            // A
template<int I>        void f(X<I, I, int>);            // B
```

The partial specialization `#2` is more specialized than the partial specialization `#1` because the function template `B` is more specialized than the function template `A` according to the ordering rules for function templates. — *end example*]

14.5.5.3 Members of class template specializations [temp.class.spec.mfunc]

1 The template parameter list of a member of a class template partial specialization shall match the template parameter list of the class template partial specialization. The template argument list of a member of a class template partial specialization shall match the template argument list of the class template partial specialization. A class template specialization is a distinct template. The members of the class template partial specialization are unrelated to the members of the primary template. Class template partial specialization members that are used in a way that requires a definition shall be defined; the definitions of members of the primary template are never used as definitions for members of a class template partial specialization. An explicit specialization of a member of a class template partial specialization is declared in the same way as an explicit specialization of the primary template. [*Example:*

```
// primary template
template<class T, int I> struct A {
  void f();
};

template<class T, int I> void A<T,I>::f() { }

// class template partial specialization
template<class T> struct A<T,2> {
  void f();
  void g();
  void h();
};

// member of class template partial specialization
template<class T> void A<T,2>::g() { }

// explicit specialization
template<> void A<char,2>::h() { }

int main() {
  A<char,0> a0;
  A<char,2> a2;
  a0.f();                       // OK, uses definition of primary template's member
  a2.g();                       // OK, uses definition of
                                // partial specialization's member
  a2.h();                       // OK, uses definition of
                                // explicit specialization's member
  a2.f();                       // ill-formed, no definition of f for A<T,2>
                                // the primary template is not used here
}
```

— *end example*]

2 If a member template of a class template is partially specialized, the member template partial specializations are member templates of the enclosing class template; if the enclosing class template is instantiated (14.7.1, 14.7.2), a declaration for every member template partial specialization is also instantiated as part of creating the members of the class template specialization. If the primary member template is explicitly specialized for a given (implicit) specialization of the enclosing class template, the partial specializations of the member template are ignored for this specialization of the enclosing class template. If a partial specialization of the member template is explicitly specialized for a given (implicit) specialization of the enclosing class template, the primary member template and its other partial specializations are still considered for this specialization of the enclosing class template. [*Example:*

```
template<class T> struct A {
  template<class T2> struct B {};                    // #1
  template<class T2> struct B<T2*> {};               // #2
};

template<> template<class T2> struct A<short>::B {};  // #3

A<char>::B<int*>  abcip;  // uses #2
A<short>::B<int*> absip;  // uses #3
A<char>::B<int>   abci;   // uses #1
```

— *end example*]

14.5.6 Function templates [temp.fct]

1 A function template defines an unbounded set of related functions. [*Example:* a family of sort functions might be declared like this:

```
template<class T> class Array { };
template<class T> void sort(Array<T>&);
```

— *end example*]

2 A function template can be overloaded with other function templates and with non-template functions (8.3.5). A non-template function is not related to a function template (i.e., it is never considered to be a specialization), even if it has the same name and type as a potentially generated function template specialization.[141]

14.5.6.1 Function template overloading [temp.over.link]

1 It is possible to overload function templates so that two different function template specializations have the same type. [*Example:*

```
// file1.c
template<class T>
  void f(T*);
void g(int* p) {
  f(p); // calls f<int>(int*)
}
```

```
// file2.c
template<class T>
  void f(T);
void h(int* p) {
  f(p); // calls f<int*>(int*)
}
```

— *end example*]

2 Such specializations are distinct functions and do not violate the one-definition rule (3.2).

3 The signature of a function template is defined in 1.3. The names of the template parameters are significant only for establishing the relationship between the template parameters and the rest of the signature. [*Note:* Two distinct function templates may have identical function return types and function parameter lists, even if overload resolution alone cannot distinguish them.

```
template<class T> void f();
template<int I> void f();        // OK: overloads the first template
                                 // distinguishable with an explicit template argument list
```

— *end note*]

4 When an expression that references a template parameter is used in the function parameter list or the return type in the declaration of a function template, the expression that references the template parameter is part

141) That is, declarations of non-template functions do not merely guide overload resolution of function template specializations with the same name. If such a non-template function is odr-used (3.2) in a program, it must be defined; it will not be implicitly instantiated using the function template definition.

of the signature of the function template. This is necessary to permit a declaration of a function template in one translation unit to be linked with another declaration of the function template in another translation unit and, conversely, to ensure that function templates that are intended to be distinct are not linked with one another. [*Example:*

```
template <int I, int J> A<I+J> f(A<I>, A<J>);   // #1
template <int K, int L> A<K+L> f(A<K>, A<L>);   // same as #1
template <int I, int J> A<I-J> f(A<I>, A<J>);   // different from #1
```

— *end example*] [*Note:* Most expressions that use template parameters use non-type template parameters, but it is possible for an expression to reference a type parameter. For example, a template type parameter can be used in the `sizeof` operator. — *end note*]

5 Two expressions involving template parameters are considered *equivalent* if two function definitions containing the expressions would satisfy the one-definition rule (3.2), except that the tokens used to name the template parameters may differ as long as a token used to name a template parameter in one expression is replaced by another token that names the same template parameter in the other expression. For determining whether two dependent names (14.6.2) are equivalent, only the name itself is considered, not the result of name lookup in the context of the template. If multiple declarations of the same function template differ in the result of this name lookup, the result for the first declaration is used. [*Example:*

```
template <int I, int J> void f(A<I+J>);          // #1
template <int K, int L> void f(A<K+L>);          // same as #1

template <class T> decltype(g(T())) h();
int g(int);
template <class T> decltype(g(T())) h()          // redeclaration of h() uses the earlier lookup
  { return g(T()); }                             // ...although the lookup here does find g(int)
int i = h<int>();                                // template argument substitution fails; g(int)
                                                 // was not in scope at the first declaration of h()
```

— *end example*] Two expressions involving template parameters that are not equivalent are *functionally equivalent* if, for any given set of template arguments, the evaluation of the expression results in the same value.

6 Two function templates are *equivalent* if they are declared in the same scope, have the same name, have identical template parameter lists, and have return types and parameter lists that are equivalent using the rules described above to compare expressions involving template parameters. Two function templates are *functionally equivalent* if they are equivalent except that one or more expressions that involve template parameters in the return types and parameter lists are functionally equivalent using the rules described above to compare expressions involving template parameters. If a program contains declarations of function templates that are functionally equivalent but not equivalent, the program is ill-formed; no diagnostic is required.

7 [*Note:* This rule guarantees that equivalent declarations will be linked with one another, while not requiring implementations to use heroic efforts to guarantee that functionally equivalent declarations will be treated as distinct. For example, the last two declarations are functionally equivalent and would cause a program to be ill-formed:

```
// Guaranteed to be the same
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+10>);

// Guaranteed to be different
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+11>);
```

```
// Ill-formed, no diagnostic required
template <int I> void f(A<I>, A<I+10>);
template <int I> void f(A<I>, A<I+1+2+3+4>);
```

— *end note*]

14.5.6.2 Partial ordering of function templates [temp.func.order]

1 If a function template is overloaded, the use of a function template specialization might be ambiguous because template argument deduction (14.8.2) may associate the function template specialization with more than one function template declaration. *Partial ordering* of overloaded function template declarations is used in the following contexts to select the function template to which a function template specialization refers:

(1.1) — during overload resolution for a call to a function template specialization (13.3.3);

(1.2) — when the address of a function template specialization is taken;

(1.3) — when a placement operator delete that is a function template specialization is selected to match a placement operator new (3.7.4.2, 5.3.4);

(1.4) — when a friend function declaration (14.5.4), an explicit instantiation (14.7.2) or an explicit specialization (14.7.3) refers to a function template specialization.

2 Partial ordering selects which of two function templates is more specialized than the other by transforming each template in turn (see next paragraph) and performing template argument deduction using the function type. The deduction process determines whether one of the templates is more specialized than the other. If so, the more specialized template is the one chosen by the partial ordering process.

3 To produce the transformed template, for each type, non-type, or template template parameter (including template parameter packs (14.5.3) thereof) synthesize a unique type, value, or class template respectively and substitute it for each occurrence of that parameter in the function type of the template. If only one of the function templates M is a non-static member of some class A, M is considered to have a new first parameter inserted in its function parameter list. Given *cv* as the cv-qualifiers of M (if any), the new parameter is of type "rvalue reference to *cv* A" if the optional *ref-qualifier* of M is `&&` or if M has no *ref-qualifier* and the first parameter of the other template has rvalue reference type. Otherwise, the new parameter is of type "lvalue reference to *cv* A". [*Note:* This allows a non-static member to be ordered with respect to a nonmember function and for the results to be equivalent to the ordering of two equivalent nonmembers. — *end note*] [*Example:*

```
struct A { };
template<class T> struct B {
  template<class R> int operator*(R&);              // #1
};

template<class T, class R> int operator*(T&, R&);   // #2

// The declaration of B::operator* is transformed into the equivalent of
// template<class R> int operator*(B<A>&, R&);      // #1a

int main() {
  A a;
  B<A> b;
  b * a;                                            // calls #1a
}
```

— *end example*]

4 Using the transformed function template's function type, perform type deduction against the other template as described in 14.8.2.4.

[*Example:*

```
template<class T> struct A { A(); };

template<class T> void f(T);
template<class T> void f(T*);
template<class T> void f(const T*);

template<class T> void g(T);
template<class T> void g(T&);

template<class T> void h(const T&);
template<class T> void h(A<T>&);

void m() {
  const int* p;
  f(p);             // f(const T*) is more specialized than f(T) or f(T*)
  float x;
  g(x);             // Ambiguous: g(T) or g(T&)
  A<int> z;
  h(z);             // overload resolution selects h(A<T>&)
  const A<int> z2;
  h(z2);            // h(const T&) is called because h(A<T>&) is not callable
}
```

— *end example*]

5 [*Note:* Since partial ordering in a call context considers only parameters for which there are explicit call arguments, some parameters are ignored (namely, function parameter packs, parameters with default arguments, and ellipsis parameters). [*Example:*

```
template<class T> void f(T);            // #1
template<class T> void f(T*, int=1);    // #2
template<class T> void g(T);            // #3
template<class T> void g(T*, ...);      // #4

int main() {
  int* ip;
  f(ip);            // calls #2
  g(ip);            // calls #4
}
```

— *end example*] [*Example:*

```
template<class T, class U> struct A { };

template<class T, class U> void f(U, A<U, T>* p = 0); // #1
template<         class U> void f(U, A<U, U>* p = 0); // #2
template<class T         > void g(T, T = T());        // #3
template<class T, class... U> void g(T, U ...);       // #4

void h() {
  f<int>(42, (A<int, int>*)0);                        // calls #2
```

```
  f<int>(42);                                   // error: ambiguous
  g(42);                                        // error: ambiguous
}
```

— *end example*] [*Example:*

```
template<class T, class... U> void f(T, U...);      // #1
template<class T           > void f(T);             // #2
template<class T, class... U> void g(T*, U...);     // #3
template<class T           > void g(T);             // #4

void h(int i) {
  f(&i);                                        // error: ambiguous
  g(&i);                                        // OK: calls #3
}
```

— *end example*] — *end note*]

14.5.7 Alias templates [temp.alias]

1 A *template-declaration* in which the *declaration* is an *alias-declaration* (Clause 7) declares the *identifier* to be a *alias template*. An alias template is a name for a family of types. The name of the alias template is a *template-name*.

2 When a *template-id* refers to the specialization of an alias template, it is equivalent to the associated type obtained by substitution of its *template-argument*s for the *template-parameter*s in the *type-id* of the alias template. [*Note:* An alias template name is never deduced. — *end note*] [*Example:*

```
template<class T> struct Alloc { /* ... */ };
template<class T> using Vec = vector<T, Alloc<T>>;
Vec<int> v;         // same as vector<int, Alloc<int>> v;

template<class T>
  void process(Vec<T>& v)
  { /* ... */ }

template<class T>
  void process(vector<T, Alloc<T>>& w)
  { /* ... */ }     // error: redefinition

template<template<class> class TT>
  void f(TT<int>);

f(v);               // error: Vec not deduced

template<template<class,class> class TT>
  void g(TT<int, Alloc<int>>);
g(v);               // OK: TT = vector
```

— *end example*]

3 However, if the *template-id* is dependent, subsequent template argument substitution still applies to the *template-id*. [*Example:*

```
template<typename...> using void_t = void;
template<typename T> void_t<typename T::foo> f();
f<int>(); // error, int does not have a nested type foo
```

— *end example*]

4 The *type-id* in an alias template declaration shall not refer to the alias template being declared. The type produced by an alias template specialization shall not directly or indirectly make use of that specialization. [*Example:*

```
template <class T> struct A;
template <class T> using B = typename A<T>::U;
template <class T> struct A {
  typedef B<T> U;
};
B<short> b;          // error: instantiation of B<short> uses own type via A<short>::U
```

— *end example*]

14.6 Name resolution [temp.res]

1 Three kinds of names can be used within a template definition:

(1.1) — The name of the template itself, and names declared within the template itself.

(1.2) — Names dependent on a *template-parameter* (14.6.2).

(1.3) — Names from scopes which are visible within the template definition.

2 A name used in a template declaration or definition and that is dependent on a *template-parameter* is assumed not to name a type unless the applicable name lookup finds a type name or the name is qualified by the keyword `typename`. [*Example:*

```
// no B declared here

class X;

template<class T> class Y {
  class Z;                      // forward declaration of member class

  void f() {
    X* a1;                      // declare pointer to X
    T* a2;                      // declare pointer to T
    Y* a3;                      // declare pointer to Y<T>
    Z* a4;                      // declare pointer to Z
    typedef typename T::A TA;
    TA* a5;                     // declare pointer to T's A
    typename T::A* a6;          // declare pointer to T's A
    T::A* a7;                   // T::A is not a type name:
                                // multiply T::A by a7; ill-formed,
                                // no visible declaration of a7
    B* a8;                      // B is not a type name:
                                // multiply B by a8; ill-formed,
                                // no visible declarations of B and a8
  }
};
```

— *end example*]

3 When a *qualified-id* is intended to refer to a type that is not a member of the current instantiation (14.6.2.1) and its *nested-name-specifier* refers to a dependent type, it shall be prefixed by the keyword `typename`, forming a *typename-specifier*. If the *qualified-id* in a *typename-specifier* does not denote a type, the program is ill-formed.

typename-specifier:
`typename` *nested-name-specifier identifier*
`typename` *nested-name-specifier* `template`$_{opt}$ *simple-template-id*

4 If a specialization of a template is instantiated for a set of *template-argument*s such that the *qualified-id* prefixed by `typename` does not denote a type, the specialization is ill-formed. The usual qualified name lookup (3.4.3) is used to find the *qualified-id* even in the presence of `typename`. [*Example:*

```
struct A {
  struct X { };
  int X;
};
struct B {
  struct X { };
};
template<class T> void f(T t) {
  typename T::X x;
}
void foo() {
  A a;
  B b;
  f(b);             // OK: T::X refers to B::X
  f(a);             // error: T::X refers to the data member A::X not the struct A::X
}
```

— *end example*]

5 A qualified name used as the name in a *mem-initializer-id*, a *base-specifier*, or an *elaborated-type-specifier* is implicitly assumed to name a type, without the use of the `typename` keyword. In a *nested-name-specifier* that immediately contains a *nested-name-specifier* that depends on a template parameter, the *identifier* or *simple-template-id* is implicitly assumed to name a type, without the use of the `typename` keyword. [*Note:* The `typename` keyword is not permitted by the syntax of these constructs. — *end note*]

6 If, for a given set of template arguments, a specialization of a template is instantiated that refers to a *qualified-id* that denotes a type, and the *qualified-id* refers to a member of an unknown specialization, the *qualified-id* shall either be prefixed by `typename` or shall be used in a context in which it implicitly names a type as described above. [*Example:*

```
template <class T> void f(int i) {
  T::x * i;         // T::x must not be a type
}

struct Foo {
  typedef int x;
};

struct Bar {
  static int const x = 5;
};

int main() {
  f<Bar>(1);        // OK
  f<Foo>(1);        // error: Foo::x is a type
}
```

— *end example*]

7 Within the definition of a class template or within the definition of a member of a class template following the *declarator-id*, the keyword `typename` is not required when referring to the name of a previously declared member of the class template that declares a type. [*Note:* such names can be found using unqualified name lookup (3.4.1), class member lookup (3.4.3.1) into the current instantiation (14.6.2.1), or class member access expression lookup (3.4.5) when the type of the object expression is the current instantiation (14.6.2.2). — *end note*] [*Example:*

```
template<class T> struct A {
  typedef int B;
  B b;              // OK, no typename required
};
```

— *end example*]

8 Knowing which names are type names allows the syntax of every template to be checked. No diagnostic shall be issued for a template for which a valid specialization can be generated. If no valid specialization can be generated for a template, and that template is not instantiated, the template is ill-formed, no diagnostic required. If every valid specialization of a variadic template requires an empty template parameter pack, the template is ill-formed, no diagnostic required. If a hypothetical instantiation of a template immediately following its definition would be ill-formed due to a construct that does not depend on a template parameter, the program is ill-formed; no diagnostic is required. If the interpretation of such a construct in the hypothetical instantiation is different from the interpretation of the corresponding construct in any actual instantiation of the template, the program is ill-formed; no diagnostic is required. [*Note:* This can happen in situations including the following:

(8.1) — a type used in a non-dependent name is incomplete at the point at which a template is defined but is complete at the point at which an instantiation is performed, or

(8.2) — an instantiation uses a default argument or default template argument that had not been defined at the point at which the template was defined, or

(8.3) — constant expression evaluation (5.20) within the template instantiation uses

(8.3.1) — the value of a `const` object of integral or unscoped enumeration type or

(8.3.2) — the value of a `constexpr` object or

(8.3.3) — the value of a reference or

(8.3.4) — the definition of a `constexpr` function,

and that entity was not defined when the template was defined, or

(8.4) — a class template specialization or variable template specialization that is specified by a non-dependent *simple-template-id* is used by the template, and either it is instantiated from a partial specialization that was not defined when the template was defined or it names an explicit specialization that was not declared when the template was defined.

— *end note*] [*Note:* If a template is instantiated, errors will be diagnosed according to the other rules in this Standard. Exactly when these errors are diagnosed is a quality of implementation issue. — *end note*] [*Example:*

```
int j;
template<class T> class X {
  void f(T t, int i, char* p) {
    t = i;          // diagnosed if X::f is instantiated
                    // and the assignment to t is an error
```

```
    p = i;          // may be diagnosed even if X::f is
                    // not instantiated
    p = j;          // may be diagnosed even if X::f is
                    // not instantiated
  }
  void g(T t) {
    +;              // may be diagnosed even if X::g is
                    // not instantiated
  }
};

template<class... T> struct A {
  void operator++(int, T... t);                 // error: too many parameters
};
template<class... T> union X : T... { };        // error: union with base class
template<class... T> struct A : T...,  T... { };// error: duplicate base class
```

— *end example*]

9 When looking for the declaration of a name used in a template definition, the usual lookup rules (3.4.1, 3.4.2) are used for non-dependent names. The lookup of names dependent on the template parameters is postponed until the actual template argument is known (14.6.2). [*Example:*

```
#include <iostream>
using namespace std;

template<class T> class Set {
  T* p;
  int cnt;
public:
  Set();
  Set<T>(const Set<T>&);
  void printall() {
    for (int i = 0; i<cnt; i++)
      cout << p[i] << '\n';
  }
};
```

in the example, `i` is the local variable `i` declared in `printall`, `cnt` is the member `cnt` declared in `Set`, and `cout` is the standard output stream declared in `iostream`. However, not every declaration can be found this way; the resolution of some names must be postponed until the actual *template-argument*s are known. For example, even though the name `operator<<` is known within the definition of `printall()` and a declaration of it can be found in `<iostream>`, the actual declaration of `operator<<` needed to print `p[i]` cannot be known until it is known what type `T` is (14.6.2). — *end example*]

10 If a name does not depend on a *template-parameter* (as defined in 14.6.2), a declaration (or set of declarations) for that name shall be in scope at the point where the name appears in the template definition; the name is bound to the declaration (or declarations) found at that point and this binding is not affected by declarations that are visible at the point of instantiation. [*Example:*

```
void f(char);

template<class T> void g(T t) {
  f(1);             // f(char)
  f(T(1));          // dependent
  f(t);             // dependent
  dd++;             // not dependent
```

```
                    // error: declaration for dd not found
}

enum E { e };
void f(E);

double dd;
void h() {
  g(e);             // will cause one call of f(char) followed
                    // by two calls of f(E)
  g('a');           // will cause three calls of f(char)
}
```

— *end example*]

11 [*Note:* For purposes of name lookup, default arguments and *exception-specification*s of function templates and default arguments and *exception-specification*s of member functions of class templates are considered definitions (14.5). — *end note*]

14.6.1 Locally declared names [temp.local]

1 Like normal (non-template) classes, class templates have an injected-class-name (Clause 9). The injected-class-name can be used as a *template-name* or a *type-name*. When it is used with a *template-argument-list*, as a *template-argument* for a template *template-parameter*, or as the final identifier in the *elaborated-type-specifier* of a friend class template declaration, it refers to the class template itself. Otherwise, it is equivalent to the *template-name* followed by the *template-parameter*s of the class template enclosed in <>.

2 Within the scope of a class template specialization or partial specialization, when the injected-class-name is used as a *type-name*, it is equivalent to the *template-name* followed by the *template-argument*s of the class template specialization or partial specialization enclosed in <>. [*Example:*

```
template<template<class> class T> class A { };
template<class T> class Y;
template<> class Y<int> {
  Y* p;                             // meaning Y<int>
  Y<char>* q;                       // meaning Y<char>
  A<Y>* a;                          // meaning A<::Y>
  class B {
    template<class> friend class Y; // meaning ::Y
  };
};
```

— *end example*]

3 The injected-class-name of a class template or class template specialization can be used either as a *template-name* or a *type-name* wherever it is in scope. [*Example:*

```
template <class T> struct Base {
  Base* p;
};

template <class T> struct Derived: public Base<T> {
  typename Derived::Base* p;    // meaning Derived::Base<T>
};

template<class T, template<class> class U = T::template Base> struct Third { };
Third<Base<int> > t;            // OK: default argument uses injected-class-name as a template
```

— *end example*]

4 A lookup that finds an injected-class-name (10.2) can result in an ambiguity in certain cases (for example, if it is found in more than one base class). If all of the injected-class-names that are found refer to specializations of the same class template, and if the name is used as a *template-name*, the reference refers to the class template itself and not a specialization thereof, and is not ambiguous. [*Example:*

```
template <class T> struct Base { };
template <class T> struct Derived: Base<int>, Base<char> {
  typename Derived::Base b;              // error: ambiguous
  typename Derived::Base<double> d;      // OK
};
```

— *end example*]

5 When the normal name of the template (i.e., the name from the enclosing scope, not the injected-class-name) is used, it always refers to the class template itself and not a specialization of the template. [*Example:*

```
template<class T> class X {
  X* p;              // meaning X<T>
  X<T>* p2;
  X<int>* p3;
  ::X* p4;           // error: missing template argument list
                     // ::X does not refer to the injected-class-name
};
```

— *end example*]

6 A *template-parameter* shall not be redeclared within its scope (including nested scopes). A *template-parameter* shall not have the same name as the template name. [*Example:*

```
template<class T, int i> class Y {
  int T;             // error: template-parameter redeclared
  void f() {
    char T;          // error: template-parameter redeclared
  }
};

template<class X> class X;       // error: template-parameter redeclared
```

— *end example*]

7 In the definition of a member of a class template that appears outside of the class template definition, the name of a member of the class template hides the name of a *template-parameter* of any enclosing class templates (but not a *template-parameter* of the member if the member is a class or function template). [*Example:*

```
template<class T> struct A {
  struct B { /* ...  */ };
  typedef void C;
  void f();
  template<class U> void g(U);
};

template<class B> void A<B>::f() {
  B b;               // A's B, not the template parameter
}

template<class B> template<class C> void A<B>::g(C) {
```

```
  B b;              // A's B, not the template parameter
  C c;              // the template parameter C, not A's C
}
```

— *end example*]

8 In the definition of a member of a class template that appears outside of the namespace containing the class template definition, the name of a *template-parameter* hides the name of a member of this namespace. [*Example:*

```
namespace N {
  class C { };
  template<class T> class B {
    void f(T);
  };
}
template<class C> void N::B<C>::f(C) {
  C b;              // C is the template parameter, not N::C
}
```

— *end example*]

9 In the definition of a class template or in the definition of a member of such a template that appears outside of the template definition, for each non-dependent base class (14.6.2.1), if the name of the base class or the name of a member of the base class is the same as the name of a *template-parameter*, the base class name or member name hides the *template-parameter* name (3.3.10). [*Example:*

```
struct A {
  struct B { /* ... */ };
  int a;
  int Y;
};

template<class B, class a> struct X : A {
  B b;              // A's B
  a b;              // error: A's a isn't a type name
};
```

— *end example*]

14.6.2 Dependent names [temp.dep]

1 Inside a template, some constructs have semantics which may differ from one instantiation to another. Such a construct *depends* on the template parameters. In particular, types and expressions may depend on the type and/or value of template parameters (as determined by the template arguments) and this determines the context for name lookup for certain names. Expressions may be *type-dependent* (on the type of a template parameter) or *value-dependent* (on the value of a non-type template parameter). In an expression of the form:

postfix-expression `(` *expression-list*$_{opt}$`)`

where the *postfix-expression* is an *unqualified-id*, the *unqualified-id* denotes a *dependent name* if

(1.1) — any of the expressions in the *expression-list* is a pack expansion (14.5.3),

(1.2) — any of the expressions or *braced-init-lists* in the *expression-list* is type-dependent (14.6.2.2), or

(1.3) — if the *unqualified-id* is a *template-id* in which any of the template arguments depends on a template parameter.

If an operand of an operator is a type-dependent expression, the operator also denotes a dependent name. Such names are unbound and are looked up at the point of the template instantiation (14.6.4.1) in both the context of the template definition and the context of the point of instantiation.

2 [*Example:*

```
template<class T> struct X : B<T> {
  typename T::A* pa;
  void f(B<T>* pb) {
    static int i = B<T>::i;
    pb->j++;
  }
};
```

the base class name `B<T>`, the type name `T::A`, the names `B<T>::i` and `pb->j` explicitly depend on the *template-parameter*. — *end example*]

3 In the definition of a class or class template, the scope of a dependent base class (14.6.2.1) is not examined during unqualified name lookup either at the point of definition of the class template or member or during an instantiation of the class template or member. [*Example:*

```
typedef double A;
template<class T> class B {
  typedef int A;
};
template<class T> struct X : B<T> {
  A a;              // a has type double
};
```

The type name `A` in the definition of `X<T>` binds to the typedef name defined in the global namespace scope, not to the typedef name defined in the base class `B<T>`. — *end example*] [*Example:*

```
struct A {
  struct B { /* ... */ };
  int a;
  int Y;
};

int a;

template<class T> struct Y : T {
  struct B { /* ... */ };
  B b;                          // The B defined in Y
  void f(int i) { a = i; }      // ::a
  Y* p;                         // Y<T>
};

Y<A> ya;
```

The members `A::B`, `A::a`, and `A::Y` of the template argument `A` do not affect the binding of names in `Y<A>`. — *end example*]

14.6.2.1 Dependent types [temp.dep.type]

1 A name refers to the *current instantiation* if it is

(1.1) — in the definition of a class template, a nested class of a class template, a member of a class template, or a member of a nested class of a class template, the injected-class-name (Clause 9) of the class template or nested class,

(1.2) — in the definition of a primary class template or a member of a primary class template, the name of the class template followed by the template argument list of the primary template (as described below) enclosed in <> (or an equivalent template alias specialization),

(1.3) — in the definition of a nested class of a class template, the name of the nested class referenced as a member of the current instantiation, or

(1.4) — in the definition of a partial specialization or a member of a partial specialization, the name of the class template followed by the template argument list of the partial specialization enclosed in <> (or an equivalent template alias specialization). If the *n*th template parameter is a parameter pack, the *n*th template argument is a pack expansion (14.5.3) whose pattern is the name of the parameter pack.

2 The template argument list of a primary template is a template argument list in which the *n*th template argument has the value of the *n*th template parameter of the class template. If the *n*th template parameter is a template parameter pack (14.5.3), the *n*th template argument is a pack expansion (14.5.3) whose pattern is the name of the template parameter pack.

3 A template argument that is equivalent to a template parameter (i.e., has the same constant value or the same type as the template parameter) can be used in place of that template parameter in a reference to the current instantiation. In the case of a non-type template argument, the argument must have been given the value of the template parameter and not an expression in which the template parameter appears as a subexpression. [*Example:*

```
template <class T> class A {
  A* p1;                        // A is the current instantiation
  A<T>* p2;                     // A<T> is the current instantiation
  A<T*> p3;                     // A<T*> is not the current instantiation
  ::A<T>* p4;                   // ::A<T> is the current instantiation
  class B {
    B* p1;                      // B is the current instantiation
    A<T>::B* p2;                // A<T>::B is the current instantiation
    typename A<T*>::B* p3;      // A<T*>::B is not the
                                // current instantiation
  };
};

template <class T> class A<T*> {
  A<T*>* p1;                    // A<T*> is the current instantiation
  A<T>* p2;                     // A<T> is not the current instantiation
};

template <class T1, class T2, int I> struct B {
  B<T1, T2, I>* b1;             // refers to the current instantiation
  B<T2, T1, I>* b2;             // not the current instantiation
  typedef T1 my_T1;
  static const int my_I = I;
  static const int my_I2 = I+0;
  static const int my_I3 = my_I;
  B<my_T1, T2, my_I>* b3;       // refers to the current instantiation
  B<my_T1, T2, my_I2>* b4;      // not the current instantiation
  B<my_T1, T2, my_I3>* b5;      // refers to the current instantiation
};
```

— *end example*]

4 A *dependent base class* is a base class that is a dependent type and is not the current instantiation. [*Note:* a base class can be the current instantiation in the case of a nested class naming an enclosing class as a base. [*Example:*

```
template<class T> struct A {
  typedef int M;
  struct B {
    typedef void M;
    struct C;
  };
};

template<class T> struct A<T>::B::C : A<T> {
  M m; // OK, A<T>::M
};
```

— *end example*] — *end note*]

5 A name is a *member of the current instantiation* if it is

(5.1) — An unqualified name that, when looked up, refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [*Note:* This can only occur when looking up a name in a scope enclosed by the definition of a class template. — *end note*]

(5.2) — A *qualified-id* in which the *nested-name-specifier* refers to the current instantiation and that, when looked up, refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [*Note:* if no such member is found, and the current instantiation has any dependent base classes, then the *qualified-id* is a member of an unknown specialization; see below. — *end note*]

(5.3) — An *id-expression* denoting the member in a class member access expression (5.2.5) for which the type of the object expression is the current instantiation, and the *id-expression*, when looked up (3.4.5), refers to at least one member of a class that is the current instantiation or a non-dependent base class thereof. [*Note:* if no such member is found, and the current instantiation has any dependent base classes, then the *id-expression* is a member of an unknown specialization; see below. — *end note*]

[*Example:*

```
template <class T> class A {
  static const int i = 5;
  int n1[i];         // i refers to a member of the current instantiation
  int n2[A::i];      // A::i refers to a member of the current instantiation
  int n3[A<T>::i];   // A<T>::i refers to a member of the current instantiation
  int f();
};

template <class T> int A<T>::f() {
  return i;          // i refers to a member of the current instantiation
}
```

— *end example*]

A name is a *dependent member of the current instantiation* if it is a member of the current instantiation that, when looked up, refers to at least one member of a class that is the current instantiation.

6 A name is a *member of an unknown specialization* if it is

(6.1) — A *qualified-id* in which the *nested-name-specifier* names a dependent type that is not the current instantiation.

(6.2) — A *qualified-id* in which the *nested-name-specifier* refers to the current instantiation, the current instantiation has at least one dependent base class, and name lookup of the *qualified-id* does not find any member of a class that is the current instantiation or a non-dependent base class thereof.

(6.3) — An *id-expression* denoting the member in a class member access expression (5.2.5) in which either

(6.3.1) — the type of the object expression is the current instantiation, the current instantiation has at least one dependent base class, and name lookup of the *id-expression* does not find a member of a class that is the current instantiation or a non-dependent base class thereof; or

(6.3.2) — the type of the object expression is dependent and is not the current instantiation.

7 If a *qualified-id* in which the *nested-name-specifier* refers to the current instantiation is not a member of the current instantiation or a member of an unknown specialization, the program is ill-formed even if the template containing the *qualified-id* is not instantiated; no diagnostic required. Similarly, if the *id-expression* in a class member access expression for which the type of the object expression is the current instantiation does not refer to a member of the current instantiation or a member of an unknown specialization, the program is ill-formed even if the template containing the member access expression is not instantiated; no diagnostic required. [*Example:*

```
template<class T> class A {
  typedef int type;
  void f() {
    A<T>::type i;           // OK: refers to a member of the current instantiation
    typename A<T>::other j; // error: neither a member of the current instantiation nor
                            // a member of an unknown specialization
  }
};
```

— *end example*]

8 If, for a given set of template arguments, a specialization of a template is instantiated that refers to a member of the current instantiation with a *qualified-id* or class member access expression, the name in the *qualified-id* or class member access expression is looked up in the template instantiation context. If the result of this lookup differs from the result of name lookup in the template definition context, name lookup is ambiguous. [*Example:*

```
struct A {
  int m;
};

struct B {
  int m;
};

template<typename T>
struct C : A, T {
  int f() { return this->m; }  // finds A::m in the template definition context
  int g() { return m; }        // finds A::m in the template definition context
};

template int C<B>::f();        // error: finds both A::m and B::m
template int C<B>::g();        // OK: transformation to class member access syntax
                               // does not occur in the template definition context; see 9.3.1
```

— *end example*]

9 A type is dependent if it is

(9.1) — a template parameter,

(9.2) — a member of an unknown specialization,

(9.3) — a nested class or enumeration that is a dependent member of the current instantiation,

(9.4) — a cv-qualified type where the cv-unqualified type is dependent,

(9.5) — a compound type constructed from any dependent type,

(9.6) — an array type whose element type is dependent or whose bound (if any) is value-dependent,

(9.7) — a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent, or

(9.8) — denoted by `decltype(`*expression*`)`, where *expression* is type-dependent (14.6.2.2).

10 [*Note:* Because typedefs do not introduce new types, but instead simply refer to other types, a name that refers to a typedef that is a member of the current instantiation is dependent only if the type referred to is dependent. — *end note*]

14.6.2.2 Type-dependent expressions [temp.dep.expr]

1 Except as described below, an expression is type-dependent if any subexpression is type-dependent.

2 `this` is type-dependent if the class type of the enclosing member function is dependent (14.6.2.1).

3 An *id-expression* is type-dependent if it contains

(3.1) — an *identifier* associated by name lookup with one or more declarations declared with a dependent type,

(3.2) — an *identifier* associated by name lookup with one or more declarations of member functions of the current instantiation declared with a return type that contains a placeholder type (7.1.6.4),

(3.3) — the *identifier* `__func__` (8.4.1), where any enclosing function is a template, a member of a class template, or a generic lambda,

(3.4) — a *template-id* that is dependent,

(3.5) — a *conversion-function-id* that specifies a dependent type, or

(3.6) — a *nested-name-specifier* or a *qualified-id* that names a member of an unknown specialization;

or if it names a dependent member of the current instantiation that is a static data member of type "array of unknown bound of `T`" for some `T` (14.5.1.3). Expressions of the following forms are type-dependent only if the type specified by the *type-id*, *simple-type-specifier* or *new-type-id* is dependent, even if any subexpression is type-dependent:

simple-type-specifier `(` *expression-list*$_{opt}$ `)`
`::`$_{opt}$ `new` *new-placement*$_{opt}$ *new-type-id* *new-initializer*$_{opt}$
`::`$_{opt}$ `new` *new-placement*$_{opt}$ `(` *type-id* `)` *new-initializer*$_{opt}$
`dynamic_cast <` *type-id* `> (` *expression* `)`
`static_cast <` *type-id* `> (` *expression* `)`
`const_cast <` *type-id* `> (` *expression* `)`
`reinterpret_cast <` *type-id* `> (` *expression* `)`
`(` *type-id* `)` *cast-expression*

4 Expressions of the following forms are never type-dependent (because the type of the expression cannot be dependent):

literal
postfix-expression `.` *pseudo-destructor-name*
postfix-expression `->` *pseudo-destructor-name*
`sizeof` *unary-expression*
`sizeof (` *type-id* `)`
`sizeof ... (` *identifier* `)`
`alignof (` *type-id* `)`
`typeid (` *expression* `)`
`typeid (` *type-id* `)`
`::`$_{opt}$ `delete` *cast-expression*
`::`$_{opt}$ `delete [ ]` *cast-expression*
`throw` *assignment-expression*$_{opt}$
`noexcept (` *expression* `)`

[*Note:* For the standard library macro `offsetof`, see 18.2. — *end note*]

5 A class member access expression (5.2.5) is type-dependent if the expression refers to a member of the current instantiation and the type of the referenced member is dependent, or the class member access expression refers to a member of an unknown specialization. [*Note:* In an expression of the form `x.y` or `xp->y` the type of the expression is usually the type of the member `y` of the class of `x` (or the class pointed to by `xp`). However, if `x` or `xp` refers to a dependent type that is not the current instantiation, the type of `y` is always dependent. If `x` or `xp` refers to a non-dependent type or refers to the current instantiation, the type of `y` is the type of the class member access expression. — *end note*]

6 A *braced-init-list* is type-dependent if any element is type-dependent or is a pack expansion.

7 A *fold-expression* is type-dependent.

14.6.2.3 Value-dependent expressions [temp.dep.constexpr]

1 Except as described below, an expression used in a context where a constant expression is required is value-dependent if any subexpression is value-dependent.

2 An *id-expression* is value-dependent if:

(2.1) — it is a name declared with a dependent type,

(2.2) — it is the name of a non-type template parameter,

(2.3) — it names a member of an unknown specialization,

(2.4) — it names a static data member that is a dependent member of the current instantiation and is not initialized in a *member-declarator*,

(2.5) — it names a static member function that is a dependent member of the current instantiation, or

(2.6) — it is a constant with literal type and is initialized with an expression that is value-dependent.

Expressions of the following form are value-dependent if the *unary-expression* or *expression* is type-dependent or the *type-id* is dependent:

`sizeof` *unary-expression*
`sizeof (` *type-id* `)`
`typeid (` *expression* `)`
`typeid (` *type-id* `)`
`alignof (` *type-id* `)`
`noexcept (` *expression* `)`

[*Note:* For the standard library macro `offsetof`, see 18.2. — *end note*]

3 Expressions of the following form are value-dependent if either the *type-id* or *simple-type-specifier* is dependent or the *expression* or *cast-expression* is value-dependent:

simple-type-specifier `(` *expression-list*$_{opt}$`)`
`static_cast` `<` *type-id* `>` `(` *expression* `)`
`const_cast` `<` *type-id* `>` `(` *expression* `)`
`reinterpret_cast` `<` *type-id* `>` `(` *expression* `)`
`(` *type-id* `)` *cast-expression*

4 Expressions of the following form are value-dependent:

`sizeof ...` `(` *identifier* `)`
fold-expression

5 An expression of the form `&`*qualified-id* where the *qualified-id* names a dependent member of the current instantiation is value-dependent.

14.6.2.4 Dependent template arguments [temp.dep.temp]

1 A type *template-argument* is dependent if the type it specifies is dependent.

2 A non-type *template-argument* is dependent if its type is dependent or the constant expression it specifies is value-dependent.

3 Furthermore, a non-type *template-argument* is dependent if the corresponding non-type *template-parameter* is of reference or pointer type and the *template-argument* designates or points to a member of the current instantiation or a member of a dependent type.

4 A template *template-argument* is dependent if it names a *template-parameter* or is a *qualified-id* that refers to a member of an unknown specialization.

14.6.3 Non-dependent names [temp.nondep]

1 Non-dependent names used in a template definition are found using the usual name lookup and bound at the point they are used. [*Example:*

```
void g(double);
void h();

template<class T> class Z {
public:
  void f() {
    g(1);           // calls g(double)
    h++;            // ill-formed: cannot increment function;
                    // this could be diagnosed either here or
                    // at the point of instantiation
  }
};

void g(int);        // not in scope at the point of the template
                    // definition, not considered for the call g(1)
```

— *end example*]

14.6.4 Dependent name resolution [temp.dep.res]

1 In resolving dependent names, names from the following sources are considered:

(1.1) — Declarations that are visible at the point of definition of the template.

(1.2) — Declarations from namespaces associated with the types of the function arguments both from the instantiation context (14.6.4.1) and from the definition context.

14.6.4.1 Point of instantiation [temp.point]

1 For a function template specialization, a member function template specialization, or a specialization for a member function or static data member of a class template, if the specialization is implicitly instantiated because it is referenced from within another template specialization and the context from which it is referenced depends on a template parameter, the point of instantiation of the specialization is the point of instantiation of the enclosing specialization. Otherwise, the point of instantiation for such a specialization immediately follows the namespace scope declaration or definition that refers to the specialization.

2 If a function template or member function of a class template is called in a way which uses the definition of a default argument of that function template or member function, the point of instantiation of the default argument is the point of instantiation of the function template or member function specialization.

3 For an *exception-specification* of a function template specialization or specialization of a member function of a class template, if the *exception-specification* is implicitly instantiated because it is needed by another template specialization and the context that requires it depends on a template parameter, the point of instantiation of the *exception-specification* is the point of instantiation of the specialization that requires it. Otherwise, the point of instantiation for such an *exception-specification* immediately follows the namespace scope declaration or definition that requires the *exception-specification*.

4 For a class template specialization, a class member template specialization, or a specialization for a class member of a class template, if the specialization is implicitly instantiated because it is referenced from within another template specialization, if the context from which the specialization is referenced depends on a template parameter, and if the specialization is not instantiated previous to the instantiation of the enclosing template, the point of instantiation is immediately before the point of instantiation of the enclosing template. Otherwise, the point of instantiation for such a specialization immediately precedes the namespace scope declaration or definition that refers to the specialization.

5 If a virtual function is implicitly instantiated, its point of instantiation is immediately following the point of instantiation of its enclosing class template specialization.

6 An explicit instantiation definition is an instantiation point for the specialization or specializations specified by the explicit instantiation.

7 The instantiation context of an expression that depends on the template arguments is the set of declarations with external linkage declared prior to the point of instantiation of the template specialization in the same translation unit.

8 A specialization for a function template, a member function template, or of a member function or static data member of a class template may have multiple points of instantiations within a translation unit, and in addition to the points of instantiation described above, for any such specialization that has a point of instantiation within the translation unit, the end of the translation unit is also considered a point of instantiation. A specialization for a class template has at most one point of instantiation within a translation unit. A specialization for any template may have points of instantiation in multiple translation units. If two different points of instantiation give a template specialization different meanings according to the one-definition rule (3.2), the program is ill-formed, no diagnostic required.

14.6.4.2 Candidate functions [temp.dep.candidate]

1 For a function call where the *postfix-expression* is a dependent name, the candidate functions are found using the usual lookup rules (3.4.1, 3.4.2) except that:

(1.1) — For the part of the lookup using unqualified name lookup (3.4.1), only function declarations from the template definition context are found.

(1.2) — For the part of the lookup using associated namespaces (3.4.2), only function declarations found in either the template definition context or the template instantiation context are found.

If the call would be ill-formed or would find a better match had the lookup within the associated namespaces considered all the function declarations with external linkage introduced in those namespaces in all translation units, not just considering those declarations found in the template definition and template instantiation contexts, then the program has undefined behavior.

14.6.5 Friend names declared within a class template [temp.inject]

1 Friend classes or functions can be declared within a class template. When a template is instantiated, the names of its friends are treated as if the specialization had been explicitly declared at its point of instantiation.

2 As with non-template classes, the names of namespace-scope friend functions of a class template specialization are not visible during an ordinary lookup unless explicitly declared at namespace scope (11.3). Such names may be found under the rules for associated classes (3.4.2).[142] [*Example:*

```
template<typename T> struct number {
  number(int);
  friend number gcd(number x, number y) { return 0; };
};

void g() {
  number<double> a(3), b(4);
  a = gcd(a,b);      // finds gcd because number<double> is an
                     // associated class, making gcd visible
                     // in its namespace (global scope)
  b = gcd(3,4);      // ill-formed; gcd is not visible
}
```

— *end example*]

14.7 Template instantiation and specialization [temp.spec]

1 The act of instantiating a function, a class, a member of a class template or a member template is referred to as *template instantiation*.

2 A function instantiated from a function template is called an instantiated function. A class instantiated from a class template is called an instantiated class. A member function, a member class, a member enumeration, or a static data member of a class template instantiated from the member definition of the class template is called, respectively, an instantiated member function, member class, member enumeration, or static data member. A member function instantiated from a member function template is called an instantiated member function. A member class instantiated from a member class template is called an instantiated member class.

3 An explicit specialization may be declared for a function template, a class template, a member of a class template or a member template. An explicit specialization declaration is introduced by `template<>`. In an explicit specialization declaration for a class template, a member of a class template or a class member template, the name of the class that is explicitly specialized shall be a *simple-template-id*. In the explicit specialization declaration for a function template or a member function template, the name of the function or member function explicitly specialized may be a *template-id*. [*Example:*

```
template<class T = int> struct A {
  static int x;
};
template<class U> void g(U) { }

template<> struct A<double> { };        // specialize for T == double
template<> struct A<> { };              // specialize for T == int
```

142) Friend declarations do not introduce new names into any scope, either when the template is declared or when it is instantiated.

```
template<> void g(char) { }             // specialize for U == char
                                        // U is deduced from the parameter type
template<> void g<int>(int) { }         // specialize for U == int
template<> int A<char>::x = 0;          // specialize for T == char

template<class T = int> struct B {
  static int x;
};
template<> int B<>::x = 1;              // specialize for T == int
```

— *end example*]

4 An instantiated template specialization can be either implicitly instantiated (14.7.1) for a given argument list or be explicitly instantiated (14.7.2). A specialization is a class, function, or class member that is either instantiated or explicitly specialized (14.7.3).

5 For a given template and a given set of *template-argument*s,

(5.1) — an explicit instantiation definition shall appear at most once in a program,

(5.2) — an explicit specialization shall be defined at most once in a program (according to 3.2), and

(5.3) — both an explicit instantiation and a declaration of an explicit specialization shall not appear in a program unless the explicit instantiation follows a declaration of the explicit specialization.

An implementation is not required to diagnose a violation of this rule.

6 Each class template specialization instantiated from a template has its own copy of any static members. [*Example:*

```
template<class T> class X {
  static T s;
};
template<class T> T X<T>::s = 0;
X<int> aa;
X<char*> bb;
```

`X<int>` has a static member `s` of type `int` and `X<char*>` has a static member `s` of type `char*`. — *end example*]

14.7.1 Implicit instantiation [temp.inst]

1 Unless a class template specialization has been explicitly instantiated (14.7.2) or explicitly specialized (14.7.3), the class template specialization is implicitly instantiated when the specialization is referenced in a context that requires a completely-defined object type or when the completeness of the class type affects the semantics of the program. [*Note:* Within a template declaration, a local class or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *exception-specification*s, and non-static data member initializers, if any). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. — *end note*] The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not of the definitions, default arguments, or *exception-specification*s of the class member functions, member classes, scoped member enumerations, static data members and member templates; and it causes the implicit instantiation of the definitions of unscoped member enumerations and member anonymous unions. However, for the purpose of determining whether an instantiated redeclaration of a member is valid according to 9.2, a declaration that corresponds to a definition in the template is considered to be a definition. [*Example:*

```
template<class T, class U>
struct Outer {
  template<class X, class Y> struct Inner;
  template<class Y> struct Inner<T, Y>;         // #1a
  template<class Y> struct Inner<T, Y> { };     // #1b; OK: valid redeclaration of #1a
  template<class Y> struct Inner<U, Y> { };     // #2
};

Outer<int, int> outer;                          // error at #2
```

`Outer<int, int>::Inner<int, Y>` is redeclared at #1b. (It is not defined but noted as being associated with a definition in `Outer<T, U>`.) #2 is also a redeclaration of #1a. It is noted as associated with a definition, so it is an invalid redeclaration of the same partial specialization. — *end example*]

2 Unless a member of a class template or a member template has been explicitly instantiated or explicitly specialized, the specialization of the member is implicitly instantiated when the specialization is referenced in a context that requires the member definition to exist; in particular, the initialization (and any associated side-effects) of a static data member does not occur unless the static data member is itself used in a way that requires the definition of the static data member to exist.

3 Unless a function template specialization has been explicitly instantiated or explicitly specialized, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist. Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.

4 [*Example:*

```
template<class T> struct Z {
  void f();
  void g();
};

void h() {
  Z<int> a;          // instantiation of class Z<int> required
  Z<char>* p;        // instantiation of class Z<char> not required
  Z<double>* q;      // instantiation of class Z<double> not required

  a.f();             // instantiation of Z<int>::f() required
  p->g();            // instantiation of class Z<char> required, and
                     // instantiation of Z<char>::g() required
}
```

Nothing in this example requires `class Z<double>`, `Z<int>::g()`, or `Z<char>::f()` to be implicitly instantiated. — *end example*]

5 Unless a variable template specialization has been explicitly instantiated or explicitly specialized, the variable template specialization is implicitly instantiated when the specialization is used. A default template argument for a variable template is implicitly instantiated when the variable template is referenced in a context that requires the value of the default argument.

6 A class template specialization is implicitly instantiated if the class type is used in a context that requires a completely-defined object type or if the completeness of the class type might affect the semantics of the program. [*Note:* In particular, if the semantics of an expression depend on the member or base class lists of a class template specialization, the class template specialization is implicitly generated. For instance, deleting a pointer to class type depends on whether or not the class declares a destructor, and conversion

between pointer to class types depends on the inheritance relationship between the two classes involved. — *end note*] [*Example:*

```
template<class T> class B { /* ... */ };
template<class T> class D : public B<T> { /* ... */ };

void f(void*);
void f(B<int>*);

void g(D<int>* p, D<char>* pp, D<double>* ppp) {
  f(p);             // instantiation of D<int> required: call f(B<int>*)
  B<char>* q = pp;  // instantiation of D<char> required:
                    // convert D<char>* to B<char>*
  delete ppp;       // instantiation of D<double> required
}
```

— *end example*]

7 If the overload resolution process can determine the correct function to call without instantiating a class template definition, it is unspecified whether that instantiation actually takes place. [*Example:*

```
template <class T> struct S {
  operator int();
};

void f(int);
void f(S<int>&);
void f(S<float>);

void g(S<int>& sr) {
  f(sr);            // instantiation of S<int> allowed but not required
                    // instantiation of S<float> allowed but not required
};
```

— *end example*]

8 If an implicit instantiation of a class template specialization is required and the template is declared but not defined, the program is ill-formed. [*Example:*

```
template<class T> class X;

X<char> ch;         // error: definition of X required
```

— *end example*]

9 The implicit instantiation of a class template does not cause any static data members of that class to be implicitly instantiated.

10 If a function template or a member function template specialization is used in a way that involves overload resolution, a declaration of the specialization is implicitly instantiated (14.8.3).

11 An implementation shall not implicitly instantiate a function template, a variable template, a member template, a non-virtual member function, a member class, or a static data member of a class template that does not require instantiation. It is unspecified whether or not an implementation implicitly instantiates a virtual member function of a class template if the virtual member function would not otherwise be instantiated. The use of a template specialization in a default argument shall not cause the template to be implicitly instantiated except that a class template may be instantiated where its complete type is needed to determine the correctness of the default argument. The use of a default argument in a function call causes specializations in the default argument to be implicitly instantiated.

12 Implicitly instantiated class, function, and variable template specializations are placed in the namespace where the template is defined. Implicitly instantiated specializations for members of a class template are placed in the namespace where the enclosing class template is defined. Implicitly instantiated member templates are placed in the namespace where the enclosing class or class template is defined. [*Example:*

```
namespace N {
  template<class T> class List {
  public:
    T* get();
  };
}

template<class K, class V> class Map {
public:
  N::List<V> lt;
  V get(K);
};

void g(Map<const char*,int>& m) {
  int i = m.get("Nicholas");
}
```

a call of `lt.get()` from `Map<const char*,int>::get()` would place `List<int>::get()` in the namespace `N` rather than in the global namespace. — *end example*]

13 If a function template `f` is called in a way that requires a default argument to be used, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the default argument is done as if the default argument had been an initializer used in a function template specialization with the same scope, the same template parameters and the same access as that of the function template `f` used at that point, except that the scope in which a closure type is declared (5.1.2) – and therefore its associated namespaces – remain as determined from the context of the definition for the default argument. This analysis is called *default argument instantiation*. The instantiated default argument is then used as the argument of `f`.

14 Each default argument is instantiated independently. [*Example:*

```
template<class T> void f(T x, T y = ydef(T()), T z = zdef(T()));

class  A { };

A zdef(A);

void g(A a, A b, A c) {
  f(a, b, c);       // no default argument instantiation
  f(a, b);          // default argument z = zdef(T()) instantiated
  f(a);             // ill-formed; ydef is not declared
}
```

— *end example*]

15 The *exception-specification* of a function template specialization is not instantiated along with the function declaration; it is instantiated when needed (15.4). If such an *exception-specification* is needed but has not yet been instantiated, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the *exception-specification* is done as if it were being done as part of instantiating the declaration of the specialization at that point.

16 [*Note:* 14.6.4.1 defines the point of instantiation of a template specialization. — *end note*]

17 There is an implementation-defined quantity that specifies the limit on the total depth of recursive instantiations, which could involve more than one template. The result of an infinite recursion in instantiation is undefined. [*Example:*

```
template<class T> class X {
  X<T>* p;          // OK
  X<T*> a;          // implicit generation of X<T> requires
                    // the implicit instantiation of X<T*> which requires
                    // the implicit instantiation of X<T**> which ...
};
```

— *end example*]

14.7.2 Explicit instantiation [temp.explicit]

1 A class, function, variable, or member template specialization can be explicitly instantiated from its template. A member function, member class or static data member of a class template can be explicitly instantiated from the member definition associated with its class template. An explicit instantiation of a function template or member function of a class template shall not use the `inline` or `constexpr` specifiers.

2 The syntax for explicit instantiation is:

explicit-instantiation:
`extern`$_{opt}$ `template` *declaration*

There are two forms of explicit instantiation: an explicit instantiation definition and an explicit instantiation declaration. An explicit instantiation declaration begins with the `extern` keyword.

3 If the explicit instantiation is for a class or member class, the *elaborated-type-specifier* in the *declaration* shall include a *simple-template-id*. If the explicit instantiation is for a function or member function, the *unqualified-id* in the *declaration* shall be either a *template-id* or, where all template arguments can be deduced, a *template-name* or *operator-function-id*. [*Note:* The declaration may declare a *qualified-id*, in which case the *unqualified-id* of the *qualified-id* must be a *template-id*. — *end note*] If the explicit instantiation is for a member function, a member class or a static data member of a class template specialization, the name of the class template specialization in the *qualified-id* for the member name shall be a *simple-template-id*. If the explicit instantiation is for a variable, the *unqualified-id* in the declaration shall be a *template-id*. An explicit instantiation shall appear in an enclosing namespace of its template. If the name declared in the explicit instantiation is an unqualified name, the explicit instantiation shall appear in the namespace where its template is declared or, if that namespace is inline (7.3.1), any namespace from its enclosing namespace set. [*Note:* Regarding qualified names in declarators, see 8.3. — *end note*] [*Example:*

```
template<class T> class Array { void mf(); };
template class Array<char>;
template void Array<int>::mf();

template<class T> void sort(Array<T>& v) { /* ... */ }
template void sort(Array<char>&);       // argument is deduced here

namespace N {
  template<class T> void f(T&) { }
}
template void N::f<int>(int&);
```

— *end example*]

4 A declaration of a function template, a variable template, a member function or static data member of a class template, or a member function template of a class or class template shall precede an explicit instantiation of that entity. A definition of a class template, a member class of a class template, or a member class

template of a class or class template shall precede an explicit instantiation of that entity unless the explicit instantiation is preceded by an explicit specialization of the entity with the same template arguments. If the *declaration* of the explicit instantiation names an implicitly-declared special member function (Clause 12), the program is ill-formed.

5 For a given set of template arguments, if an explicit instantiation of a template appears after a declaration of an explicit specialization for that template, the explicit instantiation has no effect. Otherwise, for an explicit instantiation definition the definition of a function template, a variable template, a member function template, or a member function or static data member of a class template shall be present in every translation unit in which it is explicitly instantiated.

6 An explicit instantiation of a class, function template, or variable template specialization is placed in the namespace in which the template is defined. An explicit instantiation for a member of a class template is placed in the namespace where the enclosing class template is defined. An explicit instantiation for a member template is placed in the namespace where the enclosing class or class template is defined. [*Example:*

```
namespace N {
  template<class T> class Y { void mf() { } };
}

template class Y<int>;                  // error: class template Y not visible
                                        // in the global namespace

using N::Y;
template class Y<int>;                  // error: explicit instantiation outside of the
                                        // namespace of the template

template class N::Y<char*>;             // OK: explicit instantiation in namespace N
template void N::Y<double>::mf();       // OK: explicit instantiation
                                        // in namespace N
```

— *end example*]

7 A trailing *template-argument* can be left unspecified in an explicit instantiation of a function template specialization or of a member function template specialization provided it can be deduced from the type of a function parameter (14.8.2). [*Example:*

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

// instantiate sort(Array<int>&) - template-argument deduced
template void sort<>(Array<int>&);
```

— *end example*]

8 An explicit instantiation that names a class template specialization is also an explicit instantiation of the same kind (declaration or definition) of each of its members (not including members inherited from base classes and members that are templates) that has not been previously explicitly specialized in the translation unit containing the explicit instantiation, except as described below. [*Note:* In addition, it will typically be an explicit instantiation of certain implementation-dependent data about the class. — *end note*]

9 An explicit instantiation definition that names a class template specialization explicitly instantiates the class template specialization and is an explicit instantiation definition of only those members that have been defined at the point of instantiation.

10 Except for inline functions, declarations with types deduced from their initializer or return value (7.1.6.4), `const` variables of literal types, variables of reference types, and class template specializations, explicit instantiation declarations have the effect of suppressing the implicit instantiation of the entity to which they

refer. [*Note:* The intent is that an inline function that is the subject of an explicit instantiation declaration will still be implicitly instantiated when odr-used (3.2) so that the body can be considered for inlining, but that no out-of-line copy of the inline function would be generated in the translation unit. — *end note*]

11 If an entity is the subject of both an explicit instantiation declaration and an explicit instantiation definition in the same translation unit, the definition shall follow the declaration. An entity that is the subject of an explicit instantiation declaration and that is also used in a way that would otherwise cause an implicit instantiation (14.7.1) in the translation unit shall be the subject of an explicit instantiation definition somewhere in the program; otherwise the program is ill-formed, no diagnostic required. [*Note:* This rule does apply to inline functions even though an explicit instantiation declaration of such an entity has no other normative effect. This is needed to ensure that if the address of an inline function is taken in a translation unit in which the implementation chose to suppress the out-of-line body, another translation unit will supply the body. — *end note*] An explicit instantiation declaration shall not name a specialization of a template with internal linkage.

12 The usual access checking rules do not apply to names used to specify explicit instantiations. [*Note:* In particular, the template arguments and names used in the function declarator (including parameter types, return types and exception specifications) may be private types or objects which would normally not be accessible and the template may be a member template or member function which would not normally be accessible. — *end note*]

13 An explicit instantiation does not constitute a use of a default argument, so default argument instantiation is not done. [*Example:*

```
char* p = 0;
template<class T> T g(T x = &p) { return x; }
template int g<int>(int);       // OK even though &p isn't an int.
```

— *end example*]

14.7.3 Explicit specialization [temp.expl.spec]

1 An explicit specialization of any of the following:

(1.1) — function template

(1.2) — class template

(1.3) — variable template

(1.4) — member function of a class template

(1.5) — static data member of a class template

(1.6) — member class of a class template

(1.7) — member enumeration of a class template

(1.8) — member class template of a class or class template

(1.9) — member function template of a class or class template

can be declared by a declaration introduced by `template<>`; that is:

explicit-specialization:
`template < >` *declaration*

[*Example:*

```
template<class T> class stream;

template<> class stream<char> { /* ... */ };

template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

template<> void sort<char*>(Array<char*>&) ;
```

Given these declarations, `stream<char>` will be used as the definition of streams of `char`s; other streams will be handled by class template specializations instantiated from the class template. Similarly, `sort<char*>` will be used as the sort function for arguments of type `Array<char*>`; other `Array` types will be sorted by functions generated from the template. — *end example*]

2 An explicit specialization shall be declared in a namespace enclosing the specialized template. An explicit specialization whose *declarator-id* is not qualified shall be declared in the nearest enclosing namespace of the template, or, if the namespace is inline (7.3.1), any namespace from its enclosing namespace set. Such a declaration may also be a definition. If the declaration is not a definition, the specialization may be defined later (7.3.1.2).

3 A declaration of a function template, class template, or variable template being explicitly specialized shall precede the declaration of the explicit specialization. [*Note:* A declaration, but not a definition of the template is required. — *end note*] The definition of a class or class template shall precede the declaration of an explicit specialization for a member template of the class or class template. [*Example:*

```
template<> class X<int> { /* ... */ };          // error: X not a template

template<class T> class X;

template<> class X<char*> { /* ... */ };        // OK: X is a template
```

— *end example*]

4 A member function, a member function template, a member class, a member enumeration, a member class template, a static data member, or a static data member template of a class template may be explicitly specialized for a class specialization that is implicitly instantiated; in this case, the definition of the class template shall precede the explicit specialization for the member of the class template. If such an explicit specialization for the member of a class template names an implicitly-declared special member function (Clause 12), the program is ill-formed.

5 A member of an explicitly specialized class is not implicitly instantiated from the member declaration of the class template; instead, the member of the class template specialization shall itself be explicitly defined if its definition is required. In this case, the definition of the class template explicit specialization shall be in scope at the point at which the member is defined. The definition of an explicitly specialized class is unrelated to the definition of a generated specialization. That is, its members need not have the same names, types, etc. as the members of a generated specialization. Members of an explicitly specialized class template are defined in the same manner as members of normal classes, and not using the `template<>` syntax. The same is true when defining a member of an explicitly specialized member class. However, `template<>` is used in defining a member of an explicitly specialized member class template that is specialized as a class template. [*Example:*

```
template<class T> struct A {
  struct B { };
  template<class U> struct C { };
};
```

```
template<> struct A<int> {
  void f(int);
};

void h() {
  A<int> a;
  a.f(16);          // A<int>::f must be defined somewhere
}

// template<> not used for a member of an
// explicitly specialized class template
void A<int>::f(int) { /* ... */ }

template<> struct A<char>::B {
  void f();
};
// template<> also not used when defining a member of
// an explicitly specialized member class
void A<char>::B::f() { /* ... */ }

template<> template<class U> struct A<char>::C {
  void f();
};
// template<> is used when defining a member of an explicitly
// specialized member class template specialized as a class template
template<>
template<class U> void A<char>::C<U>::f() { /* ... */ }

template<> struct A<short>::B {
  void f();
};
template<> void A<short>::B::f() { /* ... */ }  // error: template<> not permitted

template<> template<class U> struct A<short>::C {
  void f();
};
template<class U> void A<short>::C<U>::f() { /* ... */ }  // error: template<> required
```

— *end example*]

6 If a template, a member template or a member of a class template is explicitly specialized then that specialization shall be declared before the first use of that specialization that would cause an implicit instantiation to take place, in every translation unit in which such a use occurs; no diagnostic is required. If the program does not provide a definition for an explicit specialization and either the specialization is used in a way that would cause an implicit instantiation to take place or the member is a virtual member function, the program is ill-formed, no diagnostic required. An implicit instantiation is never generated for an explicit specialization that is declared but not defined. [*Example:*

```
class String { };
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v) { /* ... */ }

void f(Array<String>& v) {
  sort(v);          // use primary template
                    // sort(Array<T>&), T is String
}
```

```
template<> void sort<String>(Array<String>& v); // error: specialization
                                                // after use of primary template
template<> void sort<>(Array<char*>& v);        // OK: sort<char*> not yet used
template<class T> struct A {
  enum E : T;
  enum class S : T;
};
template<> enum A<int>::E : int { eint };         // OK
template<> enum class A<int>::S : int { sint };   // OK
template<class T> enum A<T>::E : T { eT };
template<class T> enum class A<T>::S : T { sT };
template<> enum A<char>::E : char { echar };       // ill-formed, A<char>::E was instantiated
                                                   // when A<char> was instantiated
template<> enum class A<char>::S : char { schar }; // OK
```

— *end example*]

7 The placement of explicit specialization declarations for function templates, class templates, variable templates, member functions of class templates, static data members of class templates, member classes of class templates, member enumerations of class templates, member class templates of class templates, member function templates of class templates, static data member templates of class templates, member functions of member templates of class templates, member functions of member templates of non-template classes, static data member templates of non-template classes, member function templates of member classes of class templates, etc., and the placement of partial specialization declarations of class templates, variable templates, member class templates of non-template classes, static data member templates of non-template classes, member class templates of class templates, etc., can affect whether a program is well-formed according to the relative positioning of the explicit specialization declarations and their points of instantiation in the translation unit as specified above and below. When writing a specialization, be careful about its location; or to make it compile will be such a trial as to kindle its self-immolation.

8 A template explicit specialization is in the scope of the namespace in which the template was defined. [*Example:*

```
namespace N {
  template<class T> class X { /* ... */ };
  template<class T> class Y { /* ... */ };

  template<> class X<int> { /* ... */ };          // OK: specialization
                                                  // in same namespace
  template<> class Y<double>;                     // forward declare intent to
                                                  // specialize for double
}

template<> class N::Y<double> { /* ... */ };      // OK: specialization
                                                  // in enclosing namespace
template<> class N::Y<short> { /* ... */ };       // OK: specialization
                                                  // in enclosing namespace
```

— *end example*]

9 A *simple-template-id* that names a class template explicit specialization that has been declared but not defined can be used exactly like the names of other incompletely-defined classes (3.9). [*Example:*

```
template<class T> class X;      // X is a class template
template<> class X<int>;
```

```
X<int>* p;                      // OK: pointer to declared class X<int>
X<int> x;                       // error: object of incomplete class X<int>
```

— *end example*]

10 A trailing *template-argument* can be left unspecified in the *template-id* naming an explicit function template specialization provided it can be deduced from the function argument type. [*Example:*

```
template<class T> class Array { /* ... */ };
template<class T> void sort(Array<T>& v);

// explicit specialization for sort(Array<int>&)
// with deduced template-argument of type int
template<> void sort(Array<int>&);
```

— *end example*]

11 A function with the same name as a template and a type that exactly matches that of a template specialization is not an explicit specialization (14.5.6).

12 An explicit specialization of a function template is inline only if it is declared with the `inline` specifier or defined as deleted, and independently of whether its function template is inline. [*Example:*

```
template<class T> void f(T) { /* ... */ }
template<class T> inline T g(T) { /* ... */ }

template<> inline void f<>(int) { /* ... */ }   // OK: inline
template<> int g<>(int) { /* ... */ }           // OK: not inline
```

— *end example*]

13 An explicit specialization of a static data member of a template or an explicit specialization of a static data member template is a definition if the declaration includes an initializer; otherwise, it is a declaration. [*Note:* The definition of a static data member of a template that requires default initialization must use a *braced-init-list*:

```
template<> X Q<int>::x;         // declaration
template<> X Q<int>::x ();      // error: declares a function
template<> X Q<int>::x { };     // definition
```

— *end note*]

14 A member or a member template of a class template may be explicitly specialized for a given implicit instantiation of the class template, even if the member or member template is defined in the class template definition. An explicit specialization of a member or member template is specified using the syntax for explicit specialization. [*Example:*

```
template<class T> struct A {
  void f(T);
  template<class X1> void g1(T, X1);
  template<class X2> void g2(T, X2);
  void h(T) { }
};

// specialization
template<> void A<int>::f(int);

// out of class member template definition
template<class T> template<class X1> void A<T>::g1(T, X1) { }
```

```
// member template specialization
template<> template<class X1> void A<int>::g1(int, X1);

//member template specialization
template<> template<>
  void A<int>::g1(int, char);           // X1 deduced as char
template<> template<>
  void A<int>::g2<char>(int, char);     // X2 specified as char

// member specialization even if defined in class definition
template<> void A<int>::h(int) { }
```

— *end example*]

15 A member or a member template may be nested within many enclosing class templates. In an explicit specialization for such a member, the member declaration shall be preceded by a `template<>` for each enclosing class template that is explicitly specialized. [*Example:*

```
template<class T1> class A {
  template<class T2> class B {
    void mf();
  };
};
template<> template<> class A<int>::B<double>;
template<> template<> void A<char>::B<char>::mf();
```

— *end example*]

16 In an explicit specialization declaration for a member of a class template or a member template that appears in namespace scope, the member template and some of its enclosing class templates may remain unspecialized, except that the declaration shall not explicitly specialize a class member template if its enclosing class templates are not explicitly specialized as well. In such explicit specialization declaration, the keyword `template` followed by a *template-parameter-list* shall be provided instead of the `template<>` preceding the explicit specialization declaration of the member. The types of the *template-parameters* in the *template-parameter-list* shall be the same as those specified in the primary template definition. [*Example:*

```
template <class T1> class A {
  template<class T2> class B {
    template<class T3> void mf1(T3);
    void mf2();
  };
};
template <> template <class X>
  class A<int>::B {
      template <class T> void mf1(T);
  };
template <> template <> template<class T>
  void A<int>::B<double>::mf1(T t) { }
template <class Y> template <>
  void A<Y>::B<double>::mf2() { }       // ill-formed; B<double> is specialized but
                                        // its enclosing class template A is not
```

— *end example*]

17 A specialization of a member function template, member class template, or static data member template of a non-specialized class template is itself a template.

18 An explicit specialization declaration shall not be a friend declaration.

19 Default function arguments shall not be specified in a declaration or a definition for one of the following explicit specializations:

(19.1) — the explicit specialization of a function template;

(19.2) — the explicit specialization of a member function template;

(19.3) — the explicit specialization of a member function of a class template where the class template specialization to which the member function specialization belongs is implicitly instantiated. [*Note:* Default function arguments may be specified in the declaration or definition of a member function of a class template specialization that is explicitly specialized. — *end note*]

14.8 Function template specializations [temp.fct.spec]

1 A function instantiated from a function template is called a function template specialization; so is an explicit specialization of a function template. Template arguments can be explicitly specified when naming the function template specialization, deduced from the context (e.g., deduced from the function arguments in a call to the function template specialization, see 14.8.2), or obtained from default template arguments.

2 Each function template specialization instantiated from a template has its own copy of any static variable. [*Example:*

```
template<class T> void f(T* p) {
  static T s;
};

void g(int a, char* b) {
  f(&a);            // calls f<int>(int*)
  f(&b);            // calls f<char*>(char**)
}
```

Here `f<int>(int*)` has a static variable `s` of type `int` and `f<char*>(char**)` has a static variable `s` of type `char*`. — *end example*]

14.8.1 Explicit template argument specification [temp.arg.explicit]

1 Template arguments can be specified when referring to a function template specialization by qualifying the function template name with the list of *template-argument*s in the same way as *template-argument*s are specified in uses of a class template specialization. [*Example:*

```
template<class T> void sort(Array<T>& v);
void f(Array<dcomplex>& cv, Array<int>& ci) {
  sort<dcomplex>(cv);           // sort(Array<dcomplex>&)
  sort<int>(ci);                // sort(Array<int>&)
}
```

and

```
template<class U, class V> U convert(V v);

void g(double d) {
  int i = convert<int,double>(d);       // int convert(double)
  char c = convert<char,double>(d);     // char convert(double)
}
```

— *end example*]

2 A template argument list may be specified when referring to a specialization of a function template

(2.1) — when a function is called,

(2.2) — when the address of a function is taken, when a function initializes a reference to function, or when a pointer to member function is formed,

(2.3) — in an explicit specialization,

(2.4) — in an explicit instantiation, or

(2.5) — in a friend declaration.

3 Trailing template arguments that can be deduced (14.8.2) or obtained from default *template-argument*s may be omitted from the list of explicit *template-argument*s. A trailing template parameter pack (14.5.3) not otherwise deduced will be deduced to an empty sequence of template arguments. If all of the template arguments can be deduced, they may all be omitted; in this case, the empty template argument list `<>` itself may also be omitted. In contexts where deduction is done and fails, or in contexts where deduction is not done, if a template argument list is specified and it, along with any default template arguments, identifies a single function template specialization, then the *template-id* is an lvalue for the function template specialization. [*Example:*

```
template<class X, class Y> X f(Y);
template<class X, class Y, class ... Z> X g(Y);
void h() {
  int i = f<int>(5.6);          // Y is deduced to be double
  int j = f(5.6);               // ill-formed: X cannot be deduced
  f<void>(f<int, bool>);        // Y for outer f deduced to be
                                // int (*)(bool)
  f<void>(f<int>);              // ill-formed: f<int> does not denote a
                                // single function template specialization
  int k = g<int>(5.6);          // Y is deduced to be double, Z is deduced to an empty sequence
  f<void>(g<int, bool>);        // Y for outer f is deduced to be
                                // int (*)(bool), Z is deduced to an empty sequence
}
```

— *end example*]

4 [*Note:* An empty template argument list can be used to indicate that a given use refers to a specialization of a function template even when a non-template function (8.3.5) is visible that would otherwise be used. For example:

```
template <class T> int f(T);    // #1
int f(int);                     // #2
int k = f(1);                   // uses #2
int l = f<>(1);                 // uses #1
```

— *end note*]

5 Template arguments that are present shall be specified in the declaration order of their corresponding *template-parameter*s. The template argument list shall not specify more *template-argument*s than there are corresponding *template-parameter*s unless one of the *template-parameter*s is a template parameter pack. [*Example:*

```
template<class X, class Y, class Z> X f(Y,Z);
template<class ... Args> void f2();
void g() {
  f<int,const char*,double>("aa",3.0);
  f<int,const char*>("aa",3.0);       // Z is deduced to be double
  f<int>("aa",3.0);             // Y is deduced to be const char*, and
```

```
                                  // Z is deduced to be double
  f("aa",3.0);                    // error: X cannot be deduced
  f2<char, short, int, long>(); // OK
}
```

— *end example*]

6 Implicit conversions (Clause 4) will be performed on a function argument to convert it to the type of the corresponding function parameter if the parameter type contains no *template-parameter*s that participate in template argument deduction. [*Note:* Template parameters do not participate in template argument deduction if they are explicitly specified. For example,

```
template<class T> void f(T);

class Complex {
  Complex(double);
};

void g() {
  f<Complex>(1);                  // OK, means f<Complex>(Complex(1))
}
```

— *end note*]

7 [*Note:* Because the explicit template argument list follows the function template name, and because conversion member function templates and constructor member function templates are called without using a function name, there is no way to provide an explicit template argument list for these function templates. — *end note*]

8 [*Note:* For simple function names, argument dependent lookup (3.4.2) applies even when the function name is not visible within the scope of the call. This is because the call still has the syntactic form of a function call (3.4.1). But when a function template with explicit template arguments is used, the call does not have the correct syntactic form unless there is a function template with that name visible at the point of the call. If no such name is visible, the call is not syntactically well-formed and argument-dependent lookup does not apply. If some such name is visible, argument dependent lookup applies and additional function templates may be found in other namespaces. [*Example:*

```
namespace A {
  struct B { };
  template<int X> void f(B);
}
namespace C {
  template<class T> void f(T t);
}
void g(A::B b) {
  f<3>(b);                        // ill-formed: not a function call
  A::f<3>(b);                     // well-formed
  C::f<3>(b);                     // ill-formed; argument dependent lookup
                                  // applies only to unqualified names
  using C::f;
  f<3>(b);                        // well-formed because C::f is visible; then
                                  // A::f is found by argument dependent lookup
}
```

— *end example*] — *end note*]

9 Template argument deduction can extend the sequence of template arguments corresponding to a template parameter pack, even when the sequence contains explicitly specified template arguments. [*Example:*

```
template<class ... Types> void f(Types ... values);

void g() {
  f<int*, float*>(0, 0, 0);      // Types is deduced to the sequence int*, float*, int
}
```

— *end example*]

14.8.2 Template argument deduction [temp.deduct]

1 When a function template specialization is referenced, all of the template arguments shall have values. The values can be explicitly specified or, in some cases, be deduced from the use or obtained from default *template-argument*s. [*Example:*

```
void f(Array<dcomplex>& cv, Array<int>& ci) {
  sort(cv);                      // calls sort(Array<dcomplex>&)
  sort(ci);                      // calls sort(Array<int>&)
}
```

and

```
void g(double d) {
  int i = convert<int>(d);       // calls convert<int,double>(double)
  int c = convert<char>(d);      // calls convert<char,double>(double)
}
```

— *end example*]

2 When an explicit template argument list is specified, the template arguments must be compatible with the template parameter list and must result in a valid function type as described below; otherwise type deduction fails. Specifically, the following steps are performed when evaluating an explicitly specified template argument list with respect to a given function template:

(2.1) — The specified template arguments must match the template parameters in kind (i.e., type, non-type, template). There must not be more arguments than there are parameters unless at least one parameter is a template parameter pack, and there shall be an argument for each non-pack parameter. Otherwise, type deduction fails.

(2.2) — Non-type arguments must match the types of the corresponding non-type template parameters, or must be convertible to the types of the corresponding non-type parameters as specified in 14.3.2, otherwise type deduction fails.

(2.3) — The specified template argument values are substituted for the corresponding template parameters as specified below.

3 After this substitution is performed, the function parameter type adjustments described in 8.3.5 are performed. [*Example:* A parameter type of "`void (const int, int[5])`" becomes "`void(*)(int,int*)`". — *end example*] [*Note:* A top-level qualifier in a function parameter declaration does not affect the function type but still affects the type of the function parameter variable within the function. — *end note*] [*Example:*

```
template <class T> void f(T t);
template <class X> void g(const X x);
template <class Z> void h(Z, Z*);

int main() {
  // #1: function type is f(int), t is non const
  f<int>(1);
```

```
  // #2: function type is f(int), t is const
  f<const int>(1);

  // #3: function type is g(int), x is const
  g<int>(1);

  // #4: function type is g(int), x is const
  g<const int>(1);

  // #5: function type is h(int, const int*)
  h<const int>(1,0);
}
```

— *end example*]

4 [*Note:* `f<int>(1)` and `f<const int>(1)` call distinct functions even though both of the functions called have the same function type. — *end note*]

5 The resulting substituted and adjusted function type is used as the type of the function template for template argument deduction. If a template argument has not been deduced and its corresponding template parameter has a default argument, the template argument is determined by substituting the template arguments determined for preceding template parameters into the default argument. If the substitution results in an invalid type, as described above, type deduction fails. [*Example:*

```
template <class T, class U = double>
void f(T t = 0, U u = 0);

void g() {
  f(1, 'c');        // f<int,char>(1,'c')
  f(1);             // f<int,double>(1,0)
  f();              // error: T cannot be deduced
  f<int>();         // f<int,double>(0,0)
  f<int,char>();    // f<int,char>(0,0)
}
```

— *end example*]

When all template arguments have been deduced or obtained from default template arguments, all uses of template parameters in the template parameter list of the template and the function type are replaced with the corresponding deduced or default argument values. If the substitution results in an invalid type, as described above, type deduction fails.

6 At certain points in the template argument deduction process it is necessary to take a function type that makes use of template parameters and replace those template parameters with the corresponding template arguments. This is done at the beginning of template argument deduction when any explicitly specified template arguments are substituted into the function type, and again at the end of template argument deduction when any template arguments that were deduced or obtained from default arguments are substituted.

7 The substitution occurs in all types and expressions that are used in the function type and in template parameter declarations. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside `sizeof`, `decltype`, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. [*Note:* The equivalent substitution in exception specifications is done only when the *exception-specification* is instantiated, at which point a program is ill-formed if the substitution results in an invalid type or expression. — *end note*] [*Example:*

```
template <class T> struct A { using X = typename T::X; };
template <class T> typename T::X f(typename A<T>::X);
template <class T> void f(...) { }
template <class T> auto g(typename A<T>::X) -> typename T::X;
template <class T> void g(...) { }

void h() {
  f<int>(0); // OK, substituting return type causes deduction to fail
  g<int>(0); // error, substituting parameter type instantiates A<int>
}
```

— *end example*]

8 If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written using the substituted arguments. [*Note:* If no diagnostic is required, the program is still ill-formed. Access checking is done as part of the substitution process. — *end note*] Only invalid types and expressions in the immediate context of the function type and its template parameter types can result in a deduction failure. [*Note:* The evaluation of the substituted types and expressions can result in side effects such as the instantiation of class template specializations and/or function template specializations, the generation of implicitly-defined functions, etc. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

[*Example:*

```
struct X { };
struct Y {
  Y(X){}
};

template <class T> auto f(T t1, T t2) -> decltype(t1 + t2); // #1
X f(Y, Y);  // #2

X x1, x2;
X x3 = f(x1, x2);  // deduction fails on #1 (cannot add X+X), calls #2
```

— *end example*]

[*Note:* Type deduction may fail for the following reasons:

(8.1) — Attempting to instantiate a pack expansion containing multiple parameter packs of differing lengths.

(8.2) — Attempting to create an array with an element type that is `void`, a function type, a reference type, or an abstract class type, or attempting to create an array with a size that is zero or negative. [*Example:*

```
template <class T> int f(T[5]);
int I = f<int>(0);
int j = f<void>(0);             // invalid array
```

— *end example*]

(8.3) — Attempting to use a type that is not a class or enumeration type in a qualified name. [*Example:*

```
template <class T> int f(typename T::B*);
int i = f<int>(0);
```

— *end example*]

(8.4) — Attempting to use a type in a *nested-name-specifier* of a *qualified-id* when that type does not contain the specified member, or

(8.4.1) — the specified member is not a type where a type is required, or

(8.4.2) — the specified member is not a template where a template is required, or

(8.4.3) — the specified member is not a non-type where a non-type is required.

[*Example:*

```
template <int I> struct X { };
template <template <class T> class> struct Z { };
template <class T> void f(typename T::Y*){}
template <class T> void g(X<T::N>*){}
template <class T> void h(Z<T::template TT>*){}
struct A {};
struct B { int Y; };
struct C {
  typedef int N;
};
struct D {
  typedef int TT;
};

int main() {
  // Deduction fails in each of these cases:
  f<A>(0);  // A does not contain a member Y
  f<B>(0);  // The Y member of B is not a type
  g<C>(0);  // The N member of C is not a non-type
  h<D>(0);  // The TT member of D is not a template
}
```

— *end example*]

(8.5) — Attempting to create a pointer to reference type.

(8.6) — Attempting to create a reference to `void`.

(8.7) — Attempting to create "pointer to member of `T`" when `T` is not a class type. [*Example:*

```
template <class T> int f(int T::*);
int i = f<int>(0);
```

— *end example*]

(8.8) — Attempting to give an invalid type to a non-type template parameter. [*Example:*

```
template <class T, T> struct S {};
template <class T> int f(S<T, T()>*);
struct X {};
int i0 = f<X>(0);
```

— *end example*]

(8.9) — Attempting to perform an invalid conversion in either a template argument expression, or an expression used in the function declaration. [*Example:*

```
template <class T, T*> int f(int);
int i2 = f<int,1>(0);           // can't conv 1 to int*
```

— *end example*]

(8.10) — Attempting to create a function type in which a parameter has a type of `void`, or in which the return type is a function type or array type.

(8.11) — Attempting to create a function type in which a parameter type or the return type is an abstract class type (10.4).

— *end note*]

9 [*Example:* In the following example, assuming a `signed char` cannot represent the value 1000, a narrowing conversion (8.5.4) would be required to convert the *template-argument* of type `int` to `signed char`, therefore substitution fails for the second template (14.3.2).

```
template <int> int f(int);
template <signed char> int f(int);
int i1 = f<1000>(0);          // OK
int i2 = f<1>(0);             // ambiguous; not narrowing
```

— *end example*]

14.8.2.1 Deducing template arguments from a function call [temp.deduct.call]

1 Template argument deduction is done by comparing each function template parameter type (call it `P`) with the type of the corresponding argument of the call (call it `A`) as described below. If `P` is a dependent type, removing references and cv-qualifiers from `P` gives `std::initializer_list<P′>` or `P′[N]` for some `P′` and `N` and the argument is a non-empty initializer list (8.5.4), then deduction is performed instead for each element of the initializer list, taking `P′` as a function template parameter type and the initializer element as its argument, and in the `P′[N]` case, if `N` is a non-type template parameter, `N` is deduced from the length of the initializer list. Otherwise, an initializer list argument causes the parameter to be considered a non-deduced context (14.8.2.5). [*Example:*

```
template<class T> void f(std::initializer_list<T>);
f({1,2,3});                   // T deduced to int
f({1,"asdf"});                // error: T deduced to both int and const char*

template<class T> void g(T);
g({1,2,3});                   // error: no argument deduced for T

template<class T, int N> void h(T const(&)[N]);
h({1,2,3});                   // T deduced to int, N deduced to 3

template<class T> void j(T const(&)[3]);
j({42});                      // T deduced to int, array bound not considered

struct Aggr { int i; int j; };
template<int N> void k(Aggr const(&)[N]);
k({1,2,3});                   // error: deduction fails, no conversion from int to Aggr
k({{1},{2},{3}});             // OK, N deduced to 3

template<int M, int N> void m(int const(&)[M][N]);
m({{1,2},{3,4}});             // M and N both deduced to 2

template<class T, int N> void n(T const(&)[N], T);
n({{1},{2},{3}},Aggr());      // OK, T is Aggr, N is 3
```

— *end example*] For a function parameter pack that occurs at the end of the *parameter-declaration-list*, the type `A` of each remaining argument of the call is compared with the type `P` of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the

template parameter packs expanded by the function parameter pack. When a function parameter pack appears in a non-deduced context (14.8.2.5), the type of that parameter pack is never deduced. [*Example:*

```
template<class ... Types> void f(Types& ...);
template<class T1, class ... Types> void g(T1, Types ...);
template<class T1, class ... Types> void g1(Types ..., T1);

void h(int x, float& y) {
  const int z = x;
  f(x, y, z);                  // Types is deduced to int, float, const int
  g(x, y, z);                  // T1 is deduced to int; Types is deduced to float, int
  g1(x, y, z);                 // error: Types is not deduced
  g1<int, int, int>(x, y, z);  // OK, no deduction occurs

}
```

— *end example*]

2 If P is not a reference type:

(2.1) — If A is an array type, the pointer type produced by the array-to-pointer standard conversion (4.2) is used in place of A for type deduction; otherwise,

(2.2) — If A is a function type, the pointer type produced by the function-to-pointer standard conversion (4.3) is used in place of A for type deduction; otherwise,

(2.3) — If A is a cv-qualified type, the top level cv-qualifiers of A's type are ignored for type deduction.

3 If P is a cv-qualified type, the top level cv-qualifiers of P's type are ignored for type deduction. If P is a reference type, the type referred to by P is used for type deduction. A *forwarding reference* is an rvalue reference to a cv-unqualified template parameter. If P is a forwarding reference and the argument is an lvalue, the type "lvalue reference to A" is used in place of A for type deduction. [*Example:*

```
template <class T> int f(T&& heisenreference);
template <class T> int g(const T&&);
int i;
int n1 = f(i);                 // calls f<int&>(int&)
int n2 = f(0);                 // calls f<int>(int&&)
int n3 = g(i);                 // error: would call g<int>(const int&&), which
                               // would bind an rvalue reference to an lvalue
```

— *end example*]

4 In general, the deduction process attempts to find template argument values that will make the deduced A identical to A (after the type A is transformed as described above). However, there are three cases that allow a difference:

(4.1) — If the original P is a reference type, the deduced A (i.e., the type referred to by the reference) can be more cv-qualified than the transformed A.

(4.2) — The transformed A can be another pointer or pointer to member type that can be converted to the deduced A via a qualification conversion (4.4).

(4.3) — If P is a class and P has the form *simple-template-id*, then the transformed A can be a derived class of the deduced A. Likewise, if P is a pointer to a class of the form *simple-template-id*, the transformed A can be a pointer to a derived class pointed to by the deduced A.

[*Note:* as specified in 14.8.1, implicit conversions will be performed on a function argument to convert it to the type of the corresponding function parameter if the parameter contains no *template-parameter*s that participate in template argument deduction. Such conversions are also allowed, in addition to the ones described in the preceding list. — *end note*]

5 These alternatives are considered only if type deduction would otherwise fail. If they yield more than one possible deduced `A`, the type deduction fails. [*Note:* If a *template-parameter* is not used in any of the function parameters of a function template, or is used only in a non-deduced context, its corresponding *template-argument* cannot be deduced from a function call and the *template-argument* must be explicitly specified. — *end note*]

6 When `P` is a function type, pointer to function type, or pointer to member function type:

(6.1) — If the argument is an overload set containing one or more function templates, the parameter is treated as a non-deduced context.

(6.2) — If the argument is an overload set (not containing function templates), trial argument deduction is attempted using each of the members of the set. If deduction succeeds for only one of the overload set members, that member is used as the argument value for the deduction. If deduction succeeds for more than one member of the overload set the parameter is treated as a non-deduced context.

7 [*Example:*

```
// Only one function of an overload set matches the call so the function
// parameter is a deduced context.
template <class T> int f(T (*p)(T));
int g(int);
int g(char);
int i = f(g);        // calls f(int (*)(int))
```

— *end example*]

8 [*Example:*

```
// Ambiguous deduction causes the second function parameter to be a
// non-deduced context.
template <class T> int f(T, T (*p)(T));
int g(int);
char g(char);
int i = f(1, g);     // calls f(int, int (*)(int))
```

— *end example*]

9 [*Example:*

```
// The overload set contains a template, causing the second function
// parameter to be a non-deduced context.
template <class T> int f(T, T (*p)(T));
char g(char);
template <class T> T g(T);
int i = f(1, g);     // calls f(int, int (*)(int))
```

— *end example*]

14.8.2.2 Deducing template arguments taking the address of a function template [temp.deduct.funcaddr]

1 Template arguments can be deduced from the type specified when taking the address of an overloaded function (13.4). The function template's function type and the specified type are used as the types of `P` and `A`, and the deduction is done as described in 14.8.2.5.

2 A placeholder type (7.1.6.4) in the return type of a function template is a non-deduced context. If template argument deduction succeeds for such a function, the return type is determined from instantiation of the function body.

14.8.2.3 Deducing conversion function template arguments [temp.deduct.conv]

1 Template argument deduction is done by comparing the return type of the conversion function template (call it `P`) with the type that is required as the result of the conversion (call it `A`; see 8.5, 13.3.1.5, and 13.3.1.6 for the determination of that type) as described in 14.8.2.5.

2 If `P` is a reference type, the type referred to by `P` is used in place of `P` for type deduction and for any further references to or transformations of `P` in the remainder of this section.

3 If `A` is not a reference type:

(3.1) — If `P` is an array type, the pointer type produced by the array-to-pointer standard conversion (4.2) is used in place of `P` for type deduction; otherwise,

(3.2) — If `P` is a function type, the pointer type produced by the function-to-pointer standard conversion (4.3) is used in place of `P` for type deduction; otherwise,

(3.3) — If `P` is a cv-qualified type, the top level cv-qualifiers of `P`'s type are ignored for type deduction.

4 If `A` is a cv-qualified type, the top level cv-qualifiers of `A`'s type are ignored for type deduction. If `A` is a reference type, the type referred to by `A` is used for type deduction.

5 In general, the deduction process attempts to find template argument values that will make the deduced `A` identical to `A`. However, there are two cases that allow a difference:

(5.1) — If the original `A` is a reference type, `A` can be more cv-qualified than the deduced `A` (i.e., the type referred to by the reference)

(5.2) — The deduced `A` can be another pointer or pointer to member type that can be converted to `A` via a qualification conversion.

6 These alternatives are considered only if type deduction would otherwise fail. If they yield more than one possible deduced `A`, the type deduction fails.

7 When the deduction process requires a qualification conversion for a pointer or pointer to member type as described above, the following process is used to determine the deduced template argument values:

If `A` is a type

$$cv_{1,0} \text{ “pointer to ...” } cv_{1,n-1} \text{ “pointer to” } cv_{1,n}\, T1$$

and `P` is a type

$$cv_{2,0} \text{ “pointer to ...” } cv_{2,n-1} \text{ “pointer to” } cv_{2,n}\, T2$$

The cv-unqualified `T1` and `T2` are used as the types of `A` and `P` respectively for type deduction. [*Example:*

```
struct A {
  template <class T> operator T***();
};
A a;
const int * const * const * p1 = a;      // T is deduced as int, not const int
```

— *end example*]

14.8.2.4 Deducing template arguments during partial ordering [temp.deduct.partial]

1 Template argument deduction is done by comparing certain types associated with the two function templates being compared.

2 Two sets of types are used to determine the partial ordering. For each of the templates involved there is the original function type and the transformed function type. [*Note:* The creation of the transformed type is described in 14.5.6.2. — *end note*] The deduction process uses the transformed type as the argument template and the original type of the other template as the parameter template. This process is done twice for each type involved in the partial ordering comparison: once using the transformed template-1 as the argument template and template-2 as the parameter template and again using the transformed template-2 as the argument template and template-1 as the parameter template.

3 The types used to determine the ordering depend on the context in which the partial ordering is done:

(3.1) — In the context of a function call, the types used are those function parameter types for which the function call has arguments.[143]

(3.2) — In the context of a call to a conversion function, the return types of the conversion function templates are used.

(3.3) — In other contexts (14.5.6.2) the function template's function type is used.

4 Each type nominated above from the parameter template and the corresponding type from the argument template are used as the types of `P` and `A`.

5 Before the partial ordering is done, certain transformations are performed on the types used for partial ordering:

(5.1) — If `P` is a reference type, `P` is replaced by the type referred to.

(5.2) — If `A` is a reference type, `A` is replaced by the type referred to.

6 If both `P` and `A` were reference types (before being replaced with the type referred to above), determine which of the two types (if any) is more cv-qualified than the other; otherwise the types are considered to be equally cv-qualified for partial ordering purposes. The result of this determination will be used below.

7 Remove any top-level cv-qualifiers:

(7.1) — If `P` is a cv-qualified type, `P` is replaced by the cv-unqualified version of `P`.

(7.2) — If `A` is a cv-qualified type, `A` is replaced by the cv-unqualified version of `A`.

8 If `A` was transformed from a function parameter pack and `P` is not a parameter pack, type deduction fails. Otherwise, using the resulting types `P` and `A`, the deduction is then done as described in 14.8.2.5. If `P` is a function parameter pack, the type `A` of each remaining parameter type of the argument template is compared with the type `P` of the *declarator-id* of the function parameter pack. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. If deduction succeeds for a given type, the type from the argument template is considered to be at least as specialized as the type from the parameter template. [*Example:*

```
template<class... Args>           void f(Args... args);          // #1
template<class T1, class... Args> void f(T1 a1, Args... args);   // #2
template<class T1, class T2>      void f(T1 a1, T2 a2);          // #3

f();                    // calls #1
```

143) Default arguments are not considered to be arguments in this context; they only become arguments after a function has been selected.

```
f(1, 2, 3);             // calls #2
f(1, 2);                // calls #3; non-variadic template #3 is more
                        // specialized than the variadic templates #1 and #2
```

— *end example*]

9 If, for a given type, deduction succeeds in both directions (i.e., the types are identical after the transformations above) and both `P` and `A` were reference types (before being replaced with the type referred to above):

(9.1) — if the type from the argument template was an lvalue reference and the type from the parameter template was not, the parameter type is not considered to be at least as specialized as the argument type; otherwise,

(9.2) — if the type from the argument template is more cv-qualified than the type from the parameter template (as described above), the parameter type is not considered to be at least as specialized as the argument type.

10 Function template `F` is at least as specialized as function template `G` if, for each pair of types used to determine the ordering, the type from `F` is at least as specialized as the type from `G`. `F` is *more specialized than* `G` if `F` is at least as specialized as `G` and `G` is not at least as specialized as `F`.

11 In most cases, all template parameters must have values in order for deduction to succeed, but for partial ordering purposes a template parameter may remain without a value provided it is not used in the types being used for partial ordering. [*Note:* A template parameter used in a non-deduced context is considered used. — *end note*] [*Example:*

```
template <class T> T f(int);        // #1
template <class T, class U> T f(U); // #2
void g() {
  f<int>(1);        // calls #1
}
```

— *end example*]

12 [*Note:* Partial ordering of function templates containing template parameter packs is independent of the number of deduced arguments for those template parameter packs. — *end note*] [*Example:*

```
template<class ...> struct Tuple { };
template<class ... Types> void g(Tuple<Types ...>);                 // #1
template<class T1, class ... Types> void g(Tuple<T1, Types ...>);   // #2
template<class T1, class ... Types> void g(Tuple<T1, Types& ...>);  // #3

g(Tuple<>());                   // calls #1
g(Tuple<int, float>());         // calls #2
g(Tuple<int, float&>());        // calls #3
g(Tuple<int>());                // calls #3
```

— *end example*]

14.8.2.5 Deducing template arguments from a type [temp.deduct.type]

1 Template arguments can be deduced in several different contexts, but in each case a type that is specified in terms of template parameters (call it `P`) is compared with an actual type (call it `A`), and an attempt is made to find template argument values (a type for a type parameter, a value for a non-type parameter, or a template for a template parameter) that will make `P`, after substitution of the deduced values (call it the deduced `A`), compatible with `A`.

2 In some cases, the deduction is done using a single set of types `P` and `A`, in other cases, there will be a set of corresponding types `P` and `A`. Type deduction is done independently for each `P/A` pair, and the deduced template argument values are then combined. If type deduction cannot be done for any `P/A` pair, or if for any pair the deduction leads to more than one possible set of deduced values, or if different pairs yield different deduced values, or if any template argument remains neither deduced nor explicitly specified, template argument deduction fails.

3 A given type `P` can be composed from a number of other types, templates, and non-type values:

(3.1) — A function type includes the types of each of the function parameters and the return type.

(3.2) — A pointer to member type includes the type of the class object pointed to and the type of the member pointed to.

(3.3) — A type that is a specialization of a class template (e.g., `A<int>`) includes the types, templates, and non-type values referenced by the template argument list of the specialization.

(3.4) — An array type includes the array element type and the value of the array bound.

4 In most cases, the types, templates, and non-type values that are used to compose `P` participate in template argument deduction. That is, they may be used to determine the value of a template argument, and the value so determined must be consistent with the values determined elsewhere. In certain contexts, however, the value does not participate in type deduction, but instead uses the values of template arguments that were either deduced elsewhere or explicitly specified. If a template parameter is used only in non-deduced contexts and is not explicitly specified, template argument deduction fails.

5 The non-deduced contexts are:

(5.1) — The *nested-name-specifier* of a type that was specified using a *qualified-id*.

(5.2) — The *expression* of a *decltype-specifier*.

(5.3) — A non-type template argument or an array bound in which a subexpression references a template parameter.

(5.4) — A template parameter used in the parameter type of a function parameter that has a default argument that is being used in the call for which argument deduction is being done.

(5.5) — A function parameter for which argument deduction cannot be done because the associated function argument is a function, or a set of overloaded functions (13.4), and one or more of the following apply:

(5.5.1) — more than one function matches the function parameter type (resulting in an ambiguous deduction), or

(5.5.2) — no function matches the function parameter type, or

(5.5.3) — the set of functions supplied as an argument contains one or more function templates.

(5.6) — A function parameter for which the associated argument is an initializer list (8.5.4) but the parameter does not have a type for which deduction from an initializer list is specified (14.8.2.1). [*Example:*

```
template<class T> void g(T);
g({1,2,3});                 // error: no argument deduced for T
```

— *end example*]

(5.7) — A function parameter pack that does not occur at the end of the *parameter-declaration-list*.

6 When a type name is specified in a way that includes a non-deduced context, all of the types that comprise that type name are also non-deduced. However, a compound type can include both deduced and non-deduced types. [*Example:* If a type is specified as `A<T>::B<T2>`, both `T` and `T2` are non-deduced. Likewise, if a type is specified as `A<I+J>::X<T>`, `I`, `J`, and `T` are non-deduced. If a type is specified as `void f(typename A<T>::B, A<T>)`, the `T` in `A<T>::B` is non-deduced but the `T` in `A<T>` is deduced. — *end example*]

7 [*Example:* Here is an example in which different parameter/argument pairs produce inconsistent template argument deductions:

```
template<class T> void f(T x, T y) { /* ... */ }
struct A { /* ... */ };
struct B : A { /* ... */ };
void g(A a, B b) {
  f(a,b);           // error: T could be A or B
  f(b,a);           // error: T could be A or B
  f(a,a);           // OK: T is A
  f(b,b);           // OK: T is B
}
```

Here is an example where two template arguments are deduced from a single function parameter/argument pair. This can lead to conflicts that cause type deduction to fail:

```
template <class T, class U> void f(  T (*)( T, U, U )  );

int g1( int, float, float);
char g2( int, float, float);
int g3( int, char, float);

void r() {
  f(g1);            // OK: T is int and U is float
  f(g2);            // error: T could be char or int
  f(g3);            // error: U could be char or float
}
```

Here is an example where a qualification conversion applies between the argument type on the function call and the deduced template argument type:

```
template<class T> void f(const T*) { }
int* p;
void s() {
  f(p);             // f(const int*)
}
```

Here is an example where the template argument is used to instantiate a derived class type of the corresponding function parameter type:

```
template <class T> struct B { };
template <class T> struct D : public B<T> {};
struct D2 : public B<int> {};
template <class T> void f(B<T>&){}
void t() {
  D<int> d;
  D2     d2;
  f(d);             // calls f(B<int>&)
  f(d2);            // calls f(B<int>&)
}
```

— *end example*]

8 A template type argument `T`, a template template argument `TT` or a template non-type argument `i` can be deduced if `P` and `A` have one of the following forms:

```
T
cv-list T
T*
T&
T&&
T[integer-constant]
template-name<T>  (where template-name refers to a class template)
type(T)
T()
T(T)
T type::*
type T::*
T T::*
T (type::*)()
type (T::*)()
type (type::*)(T)
type (T::*)(T)
T (type::*)(T)
T (T::*)()
T (T::*)(T)
type[i]
template-name<i>  (where template-name refers to a class template)
TT<T>
TT<i>
TT<>
```

where `(T)` represents a *parameter-type-list* where at least one parameter type contains a `T`, and `()` represents a *parameter-type-list* where no parameter type contains a `T`. Similarly, `<T>` represents template argument lists where at least one argument contains a `T`, `<i>` represents template argument lists where at least one argument contains an `i` and `<>` represents template argument lists where no argument contains a `T` or an `i`.

9 If `P` has a form that contains `<T>` or `<i>`, then each argument $\texttt{P}_i$ of the respective template argument list `P` is compared with the corresponding argument $\texttt{A}_i$ of the corresponding template argument list of `A`. If the template argument list of `P` contains a pack expansion that is not the last template argument, the entire template argument list is a non-deduced context. If $\texttt{P}_i$ is a pack expansion, then the pattern of $\texttt{P}_i$ is compared with each remaining argument in the template argument list of `A`. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by $\texttt{P}_i$. During partial ordering (14.8.2.4), if $\texttt{A}_i$ was originally a pack expansion:

(9.1) — if `P` does not contain a template argument corresponding to $\texttt{A}_i$ then $\texttt{A}_i$ is ignored;

(9.2) — otherwise, if $\texttt{P}_i$ is not a pack expansion, template argument deduction fails.

[*Example:*

```
template<class T1, class... Z> class S;                              // #1
template<class T1, class... Z> class S<T1, const Z&...> { };         // #2
template<class T1, class T2>   class S<T1, const T2&> { };           // #3
S<int, const int&> s;         // both #2 and #3 match; #3 is more specialized

template<class T, class... U>            struct A { };               // #1
template<class T1, class T2, class... U> struct A<T1, T2*, U...> { }; // #2
```

```
template<class T1, class T2>              struct A<T1, T2> { };        // #3
template struct A<int, int*>; // selects #2
```

— *end example*]

10 Similarly, if `P` has a form that contains `(T)`, then each parameter type $\texttt{P}_i$ of the respective *parameter-type-list* of `P` is compared with the corresponding parameter type $\texttt{A}_i$ of the corresponding *parameter-type-list* of `A`. If `P` and `A` are function types that originated from deduction when taking the address of a function template (14.8.2.2) or when deducing template arguments from a function declaration (14.8.2.6) and $\texttt{P}_i$ and $\texttt{A}_i$ are parameters of the top-level *parameter-type-list* of `P` and `A`, respectively, $\texttt{P}_i$ is adjusted if it is a forwarding reference (14.8.2.1) and $\texttt{A}_i$ is an lvalue reference, in which case the type of $\texttt{P}_i$ is changed to be the template parameter type (i.e., `T&&` is changed to simply `T`). [*Note:* As a result, when $\texttt{P}_i$ is `T&&` and $\texttt{A}_i$ is `X&`, the adjusted $\texttt{P}_i$ will be `T`, causing `T` to be deduced as `X&`. — *end note*] [*Example:*

```
template <class T> void f(T&&);
template <> void f(int&) { }  // #1
template <> void f(int&&) { } // #2
void g(int i) {
  f(i);                       // calls f<int&>(int&), i.e., #1
  f(0);                       // calls f<int>(int&&), i.e., #2
}
```

— *end example*]

If the *parameter-declaration* corresponding to $\texttt{P}_i$ is a function parameter pack, then the type of its *declarator-id* is compared with each remaining parameter type in the *parameter-type-list* of `A`. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. During partial ordering (14.8.2.4), if $\texttt{A}_i$ was originally a function parameter pack:

(10.1) — if `P` does not contain a function parameter type corresponding to $\texttt{A}_i$ then $\texttt{A}_i$ is ignored;

(10.2) — otherwise, if $\texttt{P}_i$ is not a function parameter pack, template argument deduction fails.

[*Example:*

```
template<class T, class... U> void f(T*, U...) { }    // #1
template<class T>             void f(T) { }           // #2
template void f(int*);      // selects #1
```

— *end example*]

11 These forms can be used in the same way as `T` is for further composition of types. [*Example:*

```
X<int> (*)(char[6])
```

is of the form

```
template-name<T> (*)(type[i])
```

which is a variant of

```
type (*)(T)
```

where type is `X<int>` and `T` is `char[6]`. — *end example*]

12 Template arguments cannot be deduced from function arguments involving constructs other than the ones specified above.

13 A template type argument cannot be deduced from the type of a non-type *template-argument*.

14 [*Example:*

```
template<class T, T i> void f(double a[10][i]);
int v[10][20];
f(v);               // error: argument for template-parameter T cannot be deduced
```

— *end example*]

15 [*Note:* Except for reference and pointer types, a major array bound is not part of a function parameter type and cannot be deduced from an argument:

```
template<int i> void f1(int a[10][i]);
template<int i> void f2(int a[i][20]);
template<int i> void f3(int (&a)[i][20]);

void g() {
  int v[10][20];
  f1(v);            // OK: i deduced to be 20
  f1<20>(v);        // OK
  f2(v);            // error: cannot deduce template-argument i
  f2<10>(v);        // OK
  f3(v);            // OK: i deduced to be 10
}
```

16 If, in the declaration of a function template with a non-type template parameter, the non-type template parameter is used in a subexpression in the function parameter list, the expression is a non-deduced context as specified above. [*Example:*

```
template <int i> class A { /* ... */ };
template <int i> void g(A<i+1>);
template <int i> void f(A<i>, A<i+1>);
void k() {
  A<1> a1;
  A<2> a2;
  g(a1);            // error: deduction fails for expression i+1
  g<0>(a1);         // OK
  f(a1, a2);        // OK
}
```

— *end example*] — *end note*] [*Note:* Template parameters do not participate in template argument deduction if they are used only in non-deduced contexts. For example,

```
template<int i, typename T>
T deduce(typename A<T>::X x,    // T is not deduced here
  T t,                          // but T is deduced here
  typename B<i>::Y y);          // i is not deduced here
A<int> a;
B<77>  b;

int    x = deduce<77>(a.xm, 62, b.ym);
// T is deduced to be int, a.xm must be convertible to
// A<int>::X
// i is explicitly specified to be 77, b.ym must be convertible
// to B<77>::Y
```

— *end note*]

17 If `P` has a form that contains `<i>`, and if the type of the corresponding value of `A` differs from the type of `i`, deduction fails. If `P` has a form that contains `[i]`, and if the type of `i` is not an integral type, deduction

fails.[144] [*Example:*

```
template<int i> class A { /* ... */ };
template<short s> void f(A<s>);
void k1() {
  A<1> a;
  f(a);             // error: deduction fails for conversion from int to short
  f<1>(a);          // OK
}

template<const short cs> class B { };
template<short s> void g(B<s>);
void k2() {
  B<1> b;
  g(b);             // OK: cv-qualifiers are ignored on template parameter types
}
```

— *end example*]

18 A *template-argument* can be deduced from a function, pointer to function, or pointer to member function type.

[*Example:*

```
template<class T> void f(void(*)(T,int));
template<class T> void foo(T,int);
void g(int,int);
void g(char,int);

void h(int,int,int);
void h(char,int);
int m() {
  f(&g);            // error: ambiguous
  f(&h);            // OK: void h(char,int) is a unique match
  f(&foo);          // error: type deduction fails because foo is a template
}
```

— *end example*]

19 A template *type-parameter* cannot be deduced from the type of a function default argument. [*Example:*

```
template <class T> void f(T = 5, T = 7);
void g() {
  f(1);             // OK: call f<int>(1,7)
  f();              // error: cannot deduce T
  f<int>();         // OK: call f<int>(5,7)
}
```

— *end example*]

20 The *template-argument* corresponding to a template *template-parameter* is deduced from the type of the *template-argument* of a class template specialization used in the argument list of a function call. [*Example:*

```
template <template <class T> class X> struct A { };
template <template <class T> class X> void f(A<X>) { }
template<class T> struct B { };
```

144) Although the *template-argument* corresponding to a *template-parameter* of type `bool` may be deduced from an array bound, the resulting value will always be `true` because the array bound will be non-zero.

```
A<B> ab;
f(ab);              // calls f(A<B>)
```

— *end example*]

21 [*Note:* Template argument deduction involving parameter packs (14.5.3) can deduce zero or more arguments for each parameter pack. — *end note*] [*Example:*

```
template<class> struct X { };
template<class R, class ... ArgTypes> struct X<R(int, ArgTypes ...)> { };
template<class ... Types> struct Y { };
template<class T, class ... Types> struct Y<T, Types& ...> { };

template<class ... Types> int f(void (*)(Types ...));
void g(int, float);

X<int> x1;                      // uses primary template
X<int(int, float, double)> x2;  // uses partial specialization; ArgTypes contains float, double
X<int(float, int)> x3;          // uses primary template
Y<> y1;                         // use primary template; Types is empty
Y<int&, float&, double&> y2;    // uses partial specialization; T is int&, Types contains float, double
Y<int, float, double> y3;       // uses primary template; Types contains int, float, double
int fv = f(g);                  // OK; Types contains int, float
```

— *end example*]

14.8.2.6 Deducing template arguments from a function declaration [temp.deduct.decl]

1 In a declaration whose *declarator-id* refers to a specialization of a function template, template argument deduction is performed to identify the specialization to which the declaration refers. Specifically, this is done for explicit instantiations (14.7.2), explicit specializations (14.7.3), and certain friend declarations (14.5.4). This is also done to determine whether a deallocation function template specialization matches a placement `operator new` (3.7.4.2, 5.3.4). In all these cases, `P` is the type of the function template being considered as a potential match and `A` is either the function type from the declaration or the type of the deallocation function that would match the placement `operator new` as described in 5.3.4. The deduction is done as described in 14.8.2.5.

2 If, for the set of function templates so considered, there is either no match or more than one match after partial ordering has been considered (14.5.6.2), deduction fails and, in the declaration cases, the program is ill-formed.

14.8.3 Overload resolution [temp.over]

1 A function template can be overloaded either by (non-template) functions of its name or by (other) function templates of the same name. When a call to that name is written (explicitly, or implicitly using the operator notation), template argument deduction (14.8.2) and checking of any explicit template arguments (14.3) are performed for each function template to find the template argument values (if any) that can be used with that function template to instantiate a function template specialization that can be invoked with the call arguments. For each function template, if the argument deduction and checking succeeds, the *template-argument*s (deduced and/or explicit) are used to synthesize the declaration of a single function template specialization which is added to the candidate functions set to be used in overload resolution. If, for a given function template, argument deduction fails, no such function is added to the set of candidate functions for that template. The complete set of candidate functions includes all the synthesized declarations and all of the non-template overloaded functions of the same name. The synthesized declarations are treated like any

other functions in the remainder of overload resolution, except as explicitly noted in 13.3.3.[145]

[*Example:*

```
template<class T> T max(T a, T b) { return a>b?a:b; }

void f(int a, int b, char c, char d) {
  int m1 = max(a,b);            // max(int a, int b)
  char m2 = max(c,d);           // max(char a, char b)
  int m3 = max(a,c);            // error: cannot generate max(int,char)
}
```

2 Adding the non-template function

```
int max(int,int);
```

to the example above would resolve the third call, by providing a function that could be called for `max(a,c)` after using the standard conversion of `char` to `int` for `c`.

3 Here is an example involving conversions on a function argument involved in *template-argument* deduction:

```
template<class T> struct B { /* ... */ };
template<class T> struct D : public B<T> { /* ... */ };
template<class T> void f(B<T>&);

void g(B<int>& bi, D<int>& di) {
  f(bi);            // f(bi)
  f(di);            // f((B<int>&)di)
}
```

4 Here is an example involving conversions on a function argument not involved in *template-parameter* deduction:

```
template<class T> void f(T*,int);       // #1
template<class T> void f(T,char);       // #2

void h(int* pi, int i, char c) {
  f(pi,i);          // #1: f<int>(pi,i)
  f(pi,c);          // #2: f<int*>(pi,c)

  f(i,c);           // #2: f<int>(i,c);
  f(i,i);           // #2: f<int>(i,char(i))
}
```

— *end example*]

5 Only the signature of a function template specialization is needed to enter the specialization in a set of candidate functions. Therefore only the function template declaration is needed to resolve a call for which a template specialization is a candidate. [*Example:*

```
template<class T> void f(T);    // declaration

void g() {
  f("Annemarie");               // call of f<const char*>
}
```

145) The parameters of function template specializations contain no template parameter types. The set of conversions allowed on deduced arguments is limited, because the argument deduction process produces function templates with parameters that either match the call arguments exactly or differ only in ways that can be bridged by the allowed limited conversions. Non-deduced arguments allow the full range of conversions. Note also that 13.3.3 specifies that a non-template function will be given preference over a template specialization if the two functions are otherwise equally good candidates for an overload match.

[6] The call of `f` is well-formed even if the template `f` is only declared and not defined at the point of the call. The program will be ill-formed unless a specialization for `f<const char*>`, either implicitly or explicitly generated, is present in some translation unit. — *end example*]

15 Exception handling [except]

1 Exception handling provides a way of transferring control and information from a point in the execution of a thread to an exception handler associated with a point previously passed by the execution. A handler will be invoked only by throwing an exception in code executed in the handler's try block or in functions called from the handler's try block.

try-block:
`try` *compound-statement handler-seq*

function-try-block:
`try` *ctor-initializer*$_{opt}$ *compound-statement handler-seq*

handler-seq:
handler handler-seq$_{opt}$

handler:
`catch (` *exception-declaration* `)` *compound-statement*

exception-declaration:
attribute-specifier-seq$_{opt}$ *type-specifier-seq declarator*
attribute-specifier-seq$_{opt}$ *type-specifier-seq abstract-declarator*$_{opt}$
`...`

The optional *attribute-specifier-seq* in an *exception-declaration* appertains to the parameter of the catch clause (15.3).

2 A *try-block* is a *statement* (Clause 6). [*Note:* Within this Clause "try block" is taken to mean both *try-block* and *function-try-block*. — *end note*]

3 A `goto` or `switch` statement shall not be used to transfer control into a try block or into a handler. [*Example:*

```
void f() {
  goto l1;          // Ill-formed
  goto l2;          // Ill-formed
  try {
    goto l1;        // OK
    goto l2;        // Ill-formed
    l1: ;
  } catch (...) {
    l2: ;
    goto l1;        // Ill-formed
    goto l2;        // OK
  }
}
```

— *end example*] A `goto`, `break`, `return`, or `continue` statement can be used to transfer control out of a try block or handler. When this happens, each variable declared in the try block will be destroyed in the context that directly contains its declaration. [*Example:*

```
lab:  try {
  T1 t1;
  try {
    T2 t2;
    if (condition)
      goto lab;
```

```
    } catch(...) { /* handler 2 */ }
  } catch(...) { /* handler 1 */ }
```

Here, executing `goto lab;` will destroy first `t2`, then `t1`, assuming the *condition* does not declare a variable. Any exception raised while destroying `t2` will result in executing *handler 2*; any exception raised while destroying `t1` will result in executing *handler 1*. — *end example*]

4 A *function-try-block* associates a *handler-seq* with the *ctor-initializer*, if present, and the *compound-statement*. An exception thrown during the execution of the *compound-statement* or, for constructors and destructors, during the initialization or destruction, respectively, of the class's subobjects, transfers control to a handler in a *function-try-block* in the same way as an exception thrown during the execution of a *try-block* transfers control to other handlers. [*Example:*

```
int f(int);
class C {
  int i;
  double d;
public:
  C(int, double);
};

C::C(int ii, double id)
try : i(f(ii)), d(id) {
    // constructor statements
}
catch (...) {
    // handles exceptions thrown from the ctor-initializer
    // and from the constructor statements
}
```

— *end example*]

15.1 Throwing an exception [except.throw]

1 Throwing an exception transfers control to a handler. [*Note:* An exception can be thrown from one of the following contexts: *throw-expression*s (5.17), allocation functions (3.7.4.1), `dynamic_cast` (5.2.7), `typeid` (5.2.8), *new-expression*s (5.3.4), and standard library functions (17.5.1.4). — *end note*] An object is passed and the type of that object determines which handlers can catch it. [*Example:*

```
throw "Help!";
```

can be caught by a *handler* of `const char*` type:

```
try {
    // ...
}
catch(const char* p) {
    // handle character string exceptions here
}
```

and

```
class Overflow {
public:
    Overflow(char,double,double);
};
```

```
void f(double x) {
    throw Overflow('+',x,3.45e107);
}
```

can be caught by a handler for exceptions of type `Overflow`

```
try {
    f(1.2);
} catch(Overflow& oo) {
    // handle exceptions of type Overflow here
}
```

— *end example*]

2 When an exception is thrown, control is transferred to the nearest handler with a matching type (15.3); "nearest" means the handler for which the *compound-statement* or *ctor-initializer* following the `try` keyword was most recently entered by the thread of control and not yet exited.

3 Throwing an exception copy-initializes (8.5, 12.8) a temporary object, called the *exception object*. The temporary is an lvalue and is used to initialize the variable declared in the matching *handler* (15.3). If the type of the exception object would be an incomplete type or a pointer to an incomplete type other than (possibly cv-qualified) `void` the program is ill-formed.

4 The memory for the exception object is allocated in an unspecified way, except as noted in 3.7.4.1. If a handler exits by rethrowing, control is passed to another handler for the same exception. The exception object is destroyed after either the last remaining active handler for the exception exits by any means other than rethrowing, or the last object of type `std::exception_ptr` (18.8.5) that refers to the exception object is destroyed, whichever is later. In the former case, the destruction occurs when the handler exits, immediately after the destruction of the object declared in the *exception-declaration* in the handler, if any. In the latter case, the destruction occurs before the destructor of `std::exception_ptr` returns. The implementation may then deallocate the memory for the exception object; any such deallocation is done in an unspecified way. [*Note:* a thrown exception does not propagate to other threads unless caught, stored, and rethrown using appropriate library functions; see 18.8.5 and 30.6. — *end note*]

5 When the thrown object is a class object, the constructor selected for the copy-initialization and the destructor shall be accessible, even if the copy/move operation is elided (12.8).

6 An exception is considered caught when a handler for that exception becomes active (15.3). [*Note:* An exception can have active handlers and still be considered uncaught if it is rethrown. — *end note*]

7 If the exception handling mechanism, after completing the initialization of the exception object but before the activation of a handler for the exception, calls a function that exits via an exception, `std::terminate` is called (15.5.1). [*Example:*

```
struct C {
  C() { }
  C(const C&) {
    if (std::uncaught_exceptions()) {
      throw 0;       // throw during copy to handler's exception-declaration object (15.3)
    }
  }
};

int main() {
  try {
    throw C();       // calls std::terminate() if construction of the handler's
                     // exception-declaration object is not elided (12.8)
  } catch(C) { }
```

```
}
```

— *end example*]

15.2 Constructors and destructors [except.ctor]

1 As control passes from the point where an exception is thrown to a handler, destructors are invoked by a process, specified in this section, called *stack unwinding*. If a destructor directly invoked by stack unwinding exits with an exception, `std::terminate` is called (15.5.1). [*Note:* Consequently, destructors should generally catch exceptions and not let them propagate out of the destructor. — *end note*]

2 The destructor is invoked for each automatic object of class type constructed since the try block was entered. The automatic objects are destroyed in the reverse order of the completion of their construction.

3 For an object of class type of any storage duration whose initialization or destruction is terminated by an exception, the destructor is invoked for each of the object's fully constructed subobjects, that is, for each subobject for which the principal constructor (12.6.2) has completed execution and the destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. The subobjects are destroyed in the reverse order of the completion of their construction. Such destruction is sequenced before entering a handler of the *function-try-block* of the constructor or destructor, if any.

4 Similarly, if the non-delegating constructor for an object has completed execution and a delegating constructor for that object exits with an exception, the object's destructor is invoked. Such destruction is sequenced before entering a handler of the *function-try-block* of a delegating constructor for that object, if any.

5 [*Note:* If the object was allocated by a *new-expression* (5.3.4), the matching deallocation function (3.7.4.2), if any, is called to free the storage occupied by the object. — *end note*]

15.3 Handling an exception [except.handle]

1 The *exception-declaration* in a *handler* describes the type(s) of exceptions that can cause that *handler* to be entered. The *exception-declaration* shall not denote an incomplete type, an abstract class type, or an rvalue reference type. The *exception-declaration* shall not denote a pointer or reference to an incomplete type, other than `void*`, `const void*`, `volatile void*`, or `const volatile void*`.

2 A handler of type "array of `T`" or "function returning `T`" is adjusted to be of type "pointer to `T`" or "pointer to function returning `T`", respectively.

3 A *handler* is a match for an exception object of type `E` if

(3.1) — The *handler* is of type *cv* `T` or *cv* `T&` and `E` and `T` are the same type (ignoring the top-level *cv-qualifiers*), or

(3.2) — the *handler* is of type *cv* `T` or *cv* `T&` and `T` is an unambiguous public base class of `E`, or

(3.3) — the *handler* is of type *cv* `T` or `const T&` where `T` is a pointer type and `E` is a pointer type that can be converted to `T` by either or both of

(3.3.1) — a standard pointer conversion (4.10) not involving conversions to pointers to private or protected or ambiguous classes

(3.3.2) — a qualification conversion, or

(3.4) — the *handler* is of type *cv* `T` or `const T&` where `T` is a pointer or pointer to member type and `E` is `std::nullptr_t`.

[*Note:* A *throw-expression* whose operand is an integer literal with value zero does not match a handler of pointer or pointer to member type. — *end note*]

[*Example:*

```
class Matherr { /* ... */ virtual void vf(); };
class Overflow: public Matherr { /* ... */ };
class Underflow: public Matherr { /* ... */ };
class Zerodivide: public Matherr { /* ... */ };

void f() {
  try {
    g();
  } catch (Overflow oo) {
        // ...
  } catch (Matherr mm) {
        // ...
  }
}
```

Here, the `Overflow` handler will catch exceptions of type `Overflow` and the `Matherr` handler will catch exceptions of type `Matherr` and of all types publicly derived from `Matherr` including exceptions of type `Underflow` and `Zerodivide`. — *end example*]

4 The handlers for a try block are tried in order of appearance. That makes it possible to write handlers that can never be executed, for example by placing a handler for a derived class after a handler for a corresponding base class.

5 A `...` in a handler's *exception-declaration* functions similarly to `...` in a function parameter declaration; it specifies a match for any exception. If present, a `...` handler shall be the last handler for its try block.

6 If no match is found among the handlers for a try block, the search for a matching handler continues in a dynamically surrounding try block of the same thread.

7 A handler is considered active when initialization is complete for the parameter (if any) of the catch clause. [*Note:* The stack will have been unwound at that point. — *end note*] Also, an implicit handler is considered active when `std::terminate()` or `std::unexpected()` is entered due to a throw. A handler is no longer considered active when the catch clause exits or when `std::unexpected()` exits after being entered due to a throw.

8 The exception with the most recently activated handler that is still active is called the *currently handled exception*.

9 If no matching handler is found, the function `std::terminate()` is called; whether or not the stack is unwound before this call to `std::terminate()` is implementation-defined (15.5.1).

10 Referring to any non-static member or base class of an object in the handler for a *function-try-block* of a constructor or destructor for that object results in undefined behavior.

11 The scope and lifetime of the parameters of a function or constructor extend into the handlers of a *function-try-block*.

12 Exceptions thrown in destructors of objects with static storage duration or in constructors of namespace-scope objects with static storage duration are not caught by a *function-try-block* on `main()`. Exceptions thrown in destructors of objects with thread storage duration or in constructors of namespace-scope objects with thread storage duration are not caught by a *function-try-block* on the initial function of the thread.

13 If a return statement appears in a handler of the *function-try-block* of a constructor, the program is ill-formed.

14 The currently handled exception is rethrown if control reaches the end of a handler of the *function-try-block* of a constructor or destructor. Otherwise, a function returns when control reaches the end of a handler for the *function-try-block* (6.6.3). Flowing off the end of a *function-try-block* is equivalent to a `return` with no value; this results in undefined behavior in a value-returning function (6.6.3).

15 The variable declared by the *exception-declaration*, of type *cv* `T` or *cv* `T&`, is initialized from the exception object, of type `E`, as follows:

(15.1) — if `T` is a base class of `E`, the variable is copy-initialized (8.5) from the corresponding base class subobject of the exception object;

(15.2) — otherwise, the variable is copy-initialized (8.5) from the exception object.

The lifetime of the variable ends when the handler exits, after the destruction of any automatic objects initialized within the handler.

16 When the handler declares an object, any changes to that object will not affect the exception object. When the handler declares a reference to an object, any changes to the referenced object are changes to the exception object and will have effect should that object be rethrown.

15.4 Exception specifications [except.spec]

1 The *exception specification* of a function is a (possibly empty) set of types, indicating that the function might exit via an exception that matches a handler of one of the types in the set; the (conceptual) set of all types is used to denote that the function might exit via an exception of arbitrary type. If the set is empty, the function is said to have a *non-throwing exception specification*. The exception specification is either defined explicitly by using an *exception-specification* as a suffix of a function declaration's declarator (8.3.5) or implicitly.

exception-specification:
dynamic-exception-specification
noexcept-specification

dynamic-exception-specification:
`throw (` *type-id-list*$_{opt}$ `)`

type-id-list:
type-id `...`$_{opt}$
type-id-list `,` *type-id* `...`$_{opt}$

noexcept-specification:
`noexcept (` *constant-expression* `)`
`noexcept`

In a *noexcept-specification*, the *constant-expression*, if supplied, shall be a constant expression (5.20) that is contextually converted to `bool` (Clause 4). A `(` token that follows `noexcept` is part of the *noexcept-specification* and does not commence an initializer (8.5).

2 An *exception-specification* shall appear only on a function declarator for a function type, pointer to function type, reference to function type, or pointer to member function type that is the top-level type of a declaration or definition, or on such a type appearing as a parameter or return type in a function declarator. An *exception-specification* shall not appear in a typedef declaration or *alias-declaration*. [*Example:*

```
void f() throw(int);                    // OK
void (*fp)() throw (int);               // OK
void g(void pfa() throw(int));          // OK
typedef int (*pf)() throw(int);         // ill-formed
```

— *end example*]

A type denoted in a *dynamic-exception-specification* shall not denote an incomplete type or an rvalue reference type. A type denoted in a *dynamic-exception-specification* shall not denote a pointer or reference to an incomplete type, other than "pointer to *cv* `void`". A type *cv* `T`, "array of `T`", or "function returning `T`" denoted in a *dynamic-exception-specification* is adjusted to type `T`, "pointer to `T`", or "pointer to function returning `T`", respectively. A *dynamic-exception-specification* denotes an exception specification that is the set of adjusted types specified thereby.

3 The *exception-specification* `noexcept` or `noexcept(`*constant-expression*`)`, where the *constant-expression* yields `true`, denotes an exception specification that is the empty set. The *exception-specification* `noexcept(`*constant-expression*`)`, where the *constant-expression* yields `false`, or the absence of an *exception-specification* in a function declarator other than that for a destructor (12.4) or a deallocation function (3.7.4.2) denotes an exception specification that is the set of all types.

4 Two *exception-specification*s are *compatible* if the sets of types they denote are the same.

5 If any declaration of a function has an *exception-specification* that is not a *noexcept-specification* allowing all exceptions, all declarations, including the definition and any explicit specialization, of that function shall have a compatible *exception-specification*. If any declaration of a pointer to function, reference to function, or pointer to member function has an *exception-specification*, all occurrences of that declaration shall have a compatible *exception-specification*. If a declaration of a function has an implicit exception specification, other declarations of the function shall not specify an *exception-specification*. In an explicit instantiation an *exception-specification* may be specified, but is not required. If an *exception-specification* is specified in an explicit instantiation directive, it shall be compatible with the *exception-specification*s of other declarations of that function. A diagnostic is required only if the *exception-specification*s are not compatible within a single translation unit.

6 If a virtual function has an exception specification, all declarations, including the definition, of any function that overrides that virtual function in any derived class shall only allow exceptions that are allowed by the exception specification of the base class virtual function, unless the overriding function is defined as deleted. [*Example:*

```
struct B {
  virtual void f() throw (int, double);
  virtual void g();
};

struct D: B {
  void f();                       // ill-formed
  void g() throw (int);           // OK
};
```

The declaration of `D::f` is ill-formed because it allows all exceptions, whereas `B::f` allows only `int` and `double`. — *end example*] A similar restriction applies to assignment to and initialization of pointers to functions, pointers to member functions, and references to functions: the target entity shall allow at least the exceptions allowed by the source value in the assignment or initialization. [*Example:*

```
class A { /* ... */ };
void (*pf1)();      // no exception specification
void (*pf2)() throw(A);

void f() {
  pf1 = pf2;        // OK: pf1 is less restrictive
  pf2 = pf1;        // error: pf2 is more restrictive
}
```

— *end example*]

7 In such an assignment or initialization, *exception-specification*s on return types and parameter types shall be compatible. In other assignments or initializations, *exception-specification*s shall be compatible.

8 An *exception-specification* can include the same type more than once and can include classes that are related by inheritance, even though doing so is redundant. [*Note:* An *exception-specification* can also include the class `std::bad_exception` (18.8.2). — *end note*]

9 A function is said to *allow an exception* of type `E` if its exception specification contains a type `T` for which a handler of type `T` would be a match (15.3) for an exception of type `E`. A function is said to *allow all exceptions* if its exception specification is the set of all types.

10 Whenever an exception of type `E` is thrown and the search for a handler (15.3) encounters the outermost block of a function with an exception specification that does not allow `E`, then,

(10.1) — if the function definition has a *dynamic-exception-specification*, the function `std::unexpected()` is called (15.5.2),

(10.2) — otherwise, the function `std::terminate()` is called (15.5.1).

[*Example:*

```
class X { };
class Y { };
class Z: public X { };
class W { };

void f() throw (X, Y) {
  int n = 0;
  if (n) throw X();             // OK
  if (n) throw Z();             // also OK
  throw W();                    // will call std::unexpected()
}
```

— *end example*]

[*Note:* A function can have multiple declarations with different non-throwing *exception-specification*s; for this purpose, the one on the function definition is used. — *end note*]

11 An implementation shall not reject an expression merely because when executed it throws or might throw an exception that the containing function does not allow. [*Example:*

```
extern void f() throw(X, Y);

void g() throw(X) {
  f();                          // OK
}
```

the call to `f` is well-formed even though when called, `f` might throw exception `Y` that `g` does not allow. — *end example*]

12 [*Note:* An exception specification is not considered part of a function's type; see 8.3.5. — *end note*]

13 A *potential exception* of a given context is either a type that might be thrown as an exception or a pseudo-type, denoted by "any", that represents the situation where an exception of an arbitrary type might be thrown. A subexpression `e1` of an expression `e` is an *immediate subexpression* if there is no subexpression `e2` of `e` such that `e1` is a subexpression of `e2`.

14 The *set of potential exceptions of a function, function pointer, or member function pointer* `f` is defined as follows:

(14.1) — If the exception specification of `f` is the set of all types, the set consists of the pseudo-type "any".

(14.2) — Otherwise, the set consists of every type in the exception specification of `f`.

15 The *set of potential exceptions of an expression* `e` is empty if `e` is a core constant expression (5.20). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of `e`, including default argument expressions used in a function call, combined with a set S defined by the form of `e`, as follows:

(15.1) — If `e` is a function call (5.2.2):

(15.1.1) — If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1), class member access (5.2.5), or pointer-to-member operation (5.5) whose *cast-expression* is an *id-expression*, S is the set of potential exceptions of the entity selected by the contained *id-expression* (after overload resolution, if applicable).

(15.1.2) — Otherwise, S contains the pseudo-type "any".

(15.2) — If `e` implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if `e` is a full-expression (1.9)), S is the set of potential exceptions of the function.

(15.3) — if `e` is a *throw-expression* (5.17), S consists of the type of the exception object that would be initialized by the operand, if present, or the pseudo-type "any" otherwise.

(15.4) — if `e` is a `dynamic_cast` expression that casts to a reference type and requires a run-time check (5.2.7), S consists of the type `std::bad_cast`.

(15.5) — if `e` is a `typeid` expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8), S consists of the type `std::bad_typeid`.

(15.6) — if `e` is a *new-expression* with a non-constant *expression* in the *noptr-new-declarator* (5.3.4), S consists of the type `std::bad_array_new_length`.

[*Example:* Given the following declarations

```
void f() throw(int);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw (double); };
```

the set of potential exceptions for some sample expressions is:

(15.7) — for `f()`, the set consists of `int`;

(15.8) — for `g()`, the set consists of "any";

(15.9) — for `new A`, the set consists of "any";

(15.10) — for `B()`, the set is empty;

(15.11) — for `new D`, the set consists of "any" and `double`.

— *end example*]

16 Given a member function `f` of some class `X`, where `f` is an inheriting constructor (12.9) or an implicitly-declared special member function, the *set of potential exceptions of the implicitly-declared member function* `f` consists of all the members from the following sets:

(16.1) — if `f` is a constructor,

(16.1.1) — the sets of potential exceptions of the constructor invocations

(16.1.1.1) — for `X`'s non-variant non-static data members,

(16.1.1.2) — for `X`'s direct base classes, and

(16.1.1.3) — if `X` is non-abstract (10.4), for `X`'s virtual base classes,

(including default argument expressions used in such invocations) as selected by overload resolution for the implicit definition of `f` (12.1). [*Note:* Even though destructors for fully-constructed subobjects are invoked when an exception is thrown during the execution of a constructor (15.2), their exception specifications do not contribute to the exception specification of the constructor, because an exception thrown from such a destructor could never escape the constructor (15.1, 15.5.1). — *end note*]

(16.1.2) — the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializer*s that are not ignored (12.6.2);

(16.2) — if `f` is an assignment operator, the sets of potential exceptions of the assignment operator invocations for `X`'s non-variant non-static data members and for `X`'s direct base classes (including default argument expressions used in such invocations), as selected by overload resolution for the implicit definition of `f` (12.8);

(16.3) — if `f` is a destructor, the sets of potential exceptions of the destructor invocations for `X`'s non-variant non-static data members and for `X`'s virtual and direct base classes.

17 An inheriting constructor (12.9) and an implicitly-declared special member function (Clause 12) are considered to have an implicit exception specification, as follows, where S is the set of potential exceptions of the implicitly-declared member function:

(17.1) — if S contains the pseudo-type "any", the implicit exception specification is the set of all types;

(17.2) — otherwise, the implicit exception specification contains all the types in S.

[*Note:* An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — *end note*] [*Example:*

```
struct A {
  A(int = (A(5), 0)) noexcept;
  A(const A&) throw();
  A(A&&) throw();
  ~A() throw(X);
};
struct B {
  B() throw();
  B(const B&) = default; // exception specification contains no types
  B(B&&, int = (throw Y(), 0)) noexcept;
  ~B() throw(Y);
};
int n = 7;
struct D : public A, public B {
    int * p = new (std::nothrow) int[n];
    // exception specification of D::D() contains X and std::bad_array_new_length
    // exception specification of D::D(const D&) contains no types
    // exception specification of D::D(D&&) contains Y
    // exception specification of D::~D() contains X and Y
};
```

Furthermore, if `A::~A()` or `B::~B()` were virtual, `D::~D()` would not be as restrictive as that of `A::~A`, and the program would be ill-formed since a function that overrides a virtual function from a base class shall have an *exception-specification* at least as restrictive as that in the base class. — *end example*]

18 A deallocation function (3.7.4.2) with no explicit *exception-specification* has an exception specification that is the empty set.

19 An *exception-specification* is considered to be *needed* when:

(19.1) — in an expression, the function is the unique lookup result or the selected member of a set of overloaded functions (3.4, 13.3, 13.4);

(19.2) — the function is odr-used (3.2) or, if it appears in an unevaluated operand, would be odr-used if the expression were potentially-evaluated;

(19.3) — the *exception-specification* is compared to that of another declaration (e.g., an explicit specialization or an overriding virtual function);

(19.4) — the function is defined; or

(19.5) — the *exception-specification* is needed for a defaulted special member function that calls the function. [*Note:* A defaulted declaration does not require the *exception-specification* of a base member function to be evaluated until the implicit *exception-specification* of the derived function is needed, but an explicit *exception-specification* needs the implicit *exception-specification* to compare against. — *end note*]

The *exception-specification* of a defaulted special member function is evaluated as described above only when needed; similarly, the *exception-specification* of a specialization of a function template or member function of a class template is instantiated only when needed.

20 In a *dynamic-exception-specification*, a *type-id* followed by an ellipsis is a pack expansion (14.5.3).

21 [*Note:* The use of *dynamic-exception-specification*s is deprecated (see Annex D). — *end note*]

15.5 Special functions [except.special]

1 The functions `std::terminate()` (15.5.1) and `std::unexpected()` (15.5.2) are used by the exception handling mechanism for coping with errors related to the exception handling mechanism itself. The function `std::current_exception()` (18.8.5) and the class `std::nested_exception` (18.8.6) can be used by a program to capture the currently handled exception.

15.5.1 The `std::terminate()` function [except.terminate]

1 In some situations exception handling must be abandoned for less subtle error handling techniques. [*Note:* These situations are:

(1.1) — when the exception handling mechanism, after completing the initialization of the exception object but before activation of a handler for the exception (15.1), calls a function that exits via an exception, or

(1.2) — when the exception handling mechanism cannot find a handler for a thrown exception (15.3), or

(1.3) — when the search for a handler (15.3) encounters the outermost block of a function with a *noexcept-specification* that does not allow the exception (15.4), or

(1.4) — when the destruction of an object during stack unwinding (15.2) terminates by throwing an exception, or

(1.5) — when initialization of a non-local variable with static or thread storage duration (3.6.2) exits via an exception, or

(1.6) — when destruction of an object with static or thread storage duration exits via an exception (3.6.3), or

(1.7) — when execution of a function registered with `std::atexit` or `std::at_quick_exit` exits via an exception (18.5), or

(1.8) — when a *throw-expression* (5.17) with no operand attempts to rethrow an exception and no exception is being handled (15.1), or

(1.9) — when `std::unexpected` exits via an exception of a type that is not allowed by the previously violated exception specification, and `std::bad_exception` is not included in that exception specification (15.5.2), or

(1.10) — when the implementation's default unexpected exception handler is called (D.8.1), or

(1.11) — when the function `std::nested_exception::rethrow_nested` is called for an object that has captured no exception (18.8.6), or

(1.12) — when execution of the initial function of a thread exits via an exception (30.3.1.2), or

(1.13) — when the destructor or the copy assignment operator is invoked on an object of type `std::thread` that refers to a joinable thread (30.3.1.3, 30.3.1.4), or

(1.14) — when a call to a `wait()`, `wait_until()`, or `wait_for()` function on a condition variable (30.5.1, 30.5.2) fails to meet a postcondition.

— *end note*]

2 In such cases, `std::terminate()` is called (18.8.3). In the situation where no matching handler is found, it is implementation-defined whether or not the stack is unwound before `std::terminate()` is called. In the situation where the search for a handler (15.3) encounters the outermost block of a function with a *noexcept-specification* that does not allow the exception (15.4), it is implementation-defined whether the stack is unwound, unwound partially, or not unwound at all before `std::terminate()` is called. In all other situations, the stack shall not be unwound before `std::terminate()` is called. An implementation is not permitted to finish stack unwinding prematurely based on a determination that the unwind process will eventually cause a call to `std::terminate()`.

15.5.2 The `std::unexpected()` function [except.unexpected]

1 If a function with a *dynamic-exception-specification* exits via an exception of a type that is not allowed by its exception specification, the function `std::unexpected()` is called (D.8) immediately after completing the stack unwinding for the former function.

2 [*Note:* By default, `std::unexpected()` calls `std::terminate()`, but a program can install its own handler function (D.8.2). In either case, the constraints in the following paragraph apply. — *end note*]

3 The `std::unexpected()` function shall not return, but it can throw (or rethrow) an exception. If it throws a new exception which is allowed by the exception specification which previously was violated, then the search for another handler will continue at the call of the function whose exception specification was violated. If it exits via an exception of a type that the *dynamic-exception-specification* does not allow, then the following happens: If the *dynamic-exception-specification* does not include the class `std::bad_exception` (18.8.2) then the function `std::terminate()` is called, otherwise the thrown exception is replaced by an implementation-defined object of type `std::bad_exception` and the search for another handler will continue at the call of the function whose *dynamic-exception-specification* was violated.

4 [*Note:* Thus, a *dynamic-exception-specification* guarantees that a function exits only via an exception of one of the listed types. If the *dynamic-exception-specification* includes the type `std::bad_exception` then any exception type not on the list may be replaced by `std::bad_exception` within the function `std::unexpected()`. — *end note*]

15.5.3 The `std::uncaught_exceptions()` function [except.uncaught]

1 An exception is considered uncaught after completing the initialization of the exception object (15.1) until completing the activation of a handler for the exception (15.3). This includes stack unwinding. If the exception is rethrown (5.17), it is considered uncaught from the point of rethrow until the rethrown exception is caught again. The function `std::uncaught_exceptions()` (18.8.4) returns the number of uncaught exceptions.

16 Preprocessing directives [cpp]

1 A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: The first token in the sequence is a **#** preprocessing token that (at the start of translation phase 4) is either the first character in the source file (optionally after white space containing no new-line characters) or that follows white space containing at least one new-line character. The last token in the sequence is the first new-line character that follows the first token in the sequence.[146] A new-line character ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro.

preprocessing-file:
group$_{opt}$

group:
group-part
group group-part

group-part:
if-section
control-line
text-line
*non-directive*

if-section:
if-group elif-groups$_{opt}$ *else-group*$_{opt}$ *endif-line*

if-group:
if *constant-expression new-line group*$_{opt}$
ifdef *identifier new-line group*$_{opt}$
ifndef *identifier new-line group*$_{opt}$

elif-groups:
elif-group
elif-groups elif-group

elif-group:
elif *constant-expression new-line group*$_{opt}$

else-group:
else *new-line group*$_{opt}$

endif-line:
endif *new-line*

control-line:
include *pp-tokens new-line*
define *identifier replacement-list new-line*
define *identifier lparen identifier-list*$_{opt}$**)** *replacement-list new-line*
define *identifier lparen* **...** **)** *replacement-list new-line*
define *identifier lparen identifier-list,* **...** **)** *replacement-list new-line*
undef *identifier new-line*
line *pp-tokens new-line*
error *pp-tokens*$_{opt}$ *new-line*
pragma *pp-tokens*$_{opt}$ *new-line*
*new-line*

146) Thus, preprocessing directives are commonly called "lines." These "lines" have no other syntactic significance, as all white space is equivalent except in certain situations during preprocessing (see the **#** character string literal creation operator in 16.3.2, for example).

text-line:
pp-tokens$_{opt}$ *new-line*
non-directive:
pp-tokens new-line
lparen:
a `(` character not immediately preceded by white-space
identifier-list:
identifier
identifier-list `,` *identifier*
replacement-list:
pp-tokens$_{opt}$
pp-tokens:
preprocessing-token
pp-tokens preprocessing-token
new-line:
the new-line character

2 A text line shall not begin with a `#` preprocessing token. A non-directive shall not begin with any of the directive names appearing in the syntax.

3 When in a group that is skipped (16.1), the directive syntax is relaxed to allow any sequence of preprocessing tokens to occur between the directive name and the following new-line character.

4 The only white-space characters that shall appear between preprocessing tokens within a preprocessing directive (from just after the introducing `#` preprocessing token through just before the terminating new-line character) are space and horizontal-tab (including spaces that have replaced comments or possibly other white-space characters in translation phase 3).

5 The implementation can process and skip sections of source files conditionally, include other source files, and replace macros. These capabilities are called *preprocessing*, because conceptually they occur before translation of the resulting translation unit.

6 The preprocessing tokens within a preprocessing directive are not subject to macro expansion unless otherwise stated.

[*Example:* In:

```
#define EMPTY
EMPTY   #   include <file.h>
```

the sequence of preprocessing tokens on the second line is *not* a preprocessing directive, because it does not begin with a # at the start of translation phase 4, even though it will do so after the macro `EMPTY` has been replaced. — *end example*]

16.1 Conditional inclusion [cpp.cond]

1 The expression that controls conditional inclusion shall be an integral constant expression except that identifiers (including those lexically identical to keywords) are interpreted as described below[147] and it may contain unary operator expressions of the form

`defined` *identifier*

or

`defined (` *identifier* `)`

147) Because the controlling constant expression is evaluated during translation phase 4, all identifiers either are or are not macro names — there simply are no keywords, enumeration constants, etc.

which evaluate to `1` if the identifier is currently defined as a macro name (that is, if it is predefined or if it has been the subject of a `#define` preprocessing directive without an intervening `#undef` directive with the same subject identifier), `0` if it is not.

2 Each preprocessing token that remains (in the list of preprocessing tokens that will become the controlling expression) after all macro replacements have occurred shall be in the lexical form of a token (2.6).

3 Preprocessing directives of the forms

`# if` *constant-expression new-line* $group_{opt}$
`# elif` *constant-expression new-line* $group_{opt}$

check whether the controlling constant expression evaluates to nonzero.

4 Prior to evaluation, macro invocations in the list of preprocessing tokens that will become the controlling constant expression are replaced (except for those macro names modified by the `defined` unary operator), just as in normal text. If the token `defined` is generated as a result of this replacement process or use of the `defined` unary operator does not match one of the two specified forms prior to macro replacement, the behavior is undefined. After all replacements due to macro expansion and the `defined` unary operator have been performed, all remaining identifiers and keywords[148], except for `true` and `false`, are replaced with the pp-number `0`, and then each preprocessing token is converted into a token. The resulting tokens comprise the controlling constant expression which is evaluated according to the rules of 5.20 using arithmetic that has at least the ranges specified in 18.3. For the purposes of this token conversion and evaluation all signed and unsigned integer types act as if they have the same representation as, respectively, `intmax_t` or `uintmax_t` (18.4).[149] This includes interpreting character literals, which may involve converting escape sequences into execution character set members. Whether the numeric value for these character literals matches the value obtained when an identical character literal occurs in an expression (other than within a `#if` or `#elif` directive) is implementation-defined.[150] Also, whether a single-character character literal may have a negative value is implementation-defined. Each subexpression with type `bool` is subjected to integral promotion before processing continues.

5 Preprocessing directives of the forms

`# ifdef` *identifier new-line* $group_{opt}$
`# ifndef` *identifier new-line* $group_{opt}$

check whether the identifier is or is not currently defined as a macro name. Their conditions are equivalent to `#if defined` *identifier* and `#if !defined` *identifier* respectively.

6 Each directive's condition is checked in order. If it evaluates to false (zero), the group that it controls is skipped: directives are processed only through the name that determines the directive in order to keep track of the level of nested conditionals; the rest of the directives' preprocessing tokens are ignored, as are the other preprocessing tokens in the group. Only the first group whose control condition evaluates to true (nonzero) is processed. If none of the conditions evaluates to true, and there is a `#else` directive, the group controlled by the `#else` is processed; lacking a `#else` directive, all the groups until the `#endif` are skipped.[151]

148) An alternative token (2.5) is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not subject to this replacement.

149) Thus on an implementation where `std::numeric_limits<int>::max()` is `0x7FFF` and `std::numeric_limits<unsigned int>::max()` is `0xFFFF`, the integer literal `0x8000` is signed and positive within a `#if` expression even though it is unsigned in translation phase 7 (2.2).

150) Thus, the constant expression in the following `#if` directive and `if` statement is not guaranteed to evaluate to the same value in these two contexts.

```
#if 'z' - 'a' == 25
if ('z' - 'a' == 25)
```

151) As indicated by the syntax, a preprocessing token shall not follow a `#else` or `#endif` directive before the terminating new-line character. However, comments may appear anywhere in a source file, including within a preprocessing directive.

16.2 Source file inclusion [cpp.include]

1 A `#include` directive shall identify a header or source file that can be processed by the implementation.

2 A preprocessing directive of the form

`# include <` *h-char-sequence*`>` *new-line*

searches a sequence of implementation-defined places for a header identified uniquely by the specified sequence between the `<` and `>` delimiters, and causes the replacement of that directive by the entire contents of the header. How the places are specified or the header identified is implementation-defined.

3 A preprocessing directive of the form

`# include "` *q-char-sequence*`"` *new-line*

causes the replacement of that directive by the entire contents of the source file identified by the specified sequence between the `"` delimiters. The named source file is searched for in an implementation-defined manner. If this search is not supported, or if the search fails, the directive is reprocessed as if it read

`# include <` *h-char-sequence*`>` *new-line*

with the identical contained sequence (including `>` characters, if any) from the original directive.

4 A preprocessing directive of the form

`# include` *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after `include` in the directive are processed just as in normal text (i.e., each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). If the directive resulting after all replacements does not match one of the two previous forms, the behavior is undefined.[152] The method by which a sequence of preprocessing tokens between a `<` and a `>` preprocessing token pair or a pair of `"` characters is combined into a single header name preprocessing token is implementation-defined.

5 The implementation shall provide unique mappings for sequences consisting of one or more *nondigit*s or *digit*s (2.10) followed by a period (`.`) and a single *nondigit*. The first character shall not be a *digit*. The implementation may ignore distinctions of alphabetical case.

6 A `#include` preprocessing directive may appear in a source file that has been read because of a `#include` directive in another file, up to an implementation-defined nesting limit.

7 [*Note:* Although an implementation may provide a mechanism for making arbitrary source files available to the `< >` search, in general programmers should use the `< >` form for headers provided with the implementation, and the `" "` form for sources outside the control of the implementation. For instance:

```
#include <stdio.h>
#include <unistd.h>
#include "usefullib.h"
#include "myprog.h"
```

— *end note*]

8 [*Example:* This illustrates macro-replaced `#include` directives:

```
#if VERSION == 1
    #define INCFILE  "vers1.h"
#elif VERSION == 2
    #define INCFILE  "vers2.h"   // and so on
#else
    #define INCFILE  "versN.h"
```

152) Note that adjacent string literals are not concatenated into a single string literal (see the translation phases in 2.2); thus, an expansion that results in two string literals is an invalid directive.

```
#endif
#include INCFILE
```

— *end example*]

16.3 Macro replacement [cpp.replace]

1 Two replacement lists are identical if and only if the preprocessing tokens in both have the same number, ordering, spelling, and white-space separation, where all white-space separations are considered identical.

2 An identifier currently defined as an *object-like* macro may be redefined by another `#define` preprocessing directive provided that the second definition is an object-like macro definition and the two replacement lists are identical, otherwise the program is ill-formed. Likewise, an identifier currently defined as a *function-like* macro may be redefined by another `#define` preprocessing directive provided that the second definition is a function-like macro definition that has the same number and spelling of parameters, and the two replacement lists are identical, otherwise the program is ill-formed.

3 There shall be white-space between the identifier and the replacement list in the definition of an object-like macro.

4 If the *identifier-list* in the macro definition does not end with an ellipsis, the number of arguments (including those arguments consisting of no preprocessing tokens) in an invocation of a function-like macro shall equal the number of parameters in the macro definition. Otherwise, there shall be more arguments in the invocation than there are parameters in the macro definition (excluding the `...`). There shall exist a `)` preprocessing token that terminates the invocation.

5 The identifier `__VA_ARGS__` shall occur only in the replacement-list of a function-like macro that uses the ellipsis notation in the parameters.

6 A parameter identifier in a function-like macro shall be uniquely declared within its scope.

7 The identifier immediately following the `define` is called the *macro name*. There is one name space for macro names. Any white-space characters preceding or following the replacement list of preprocessing tokens are not considered part of the replacement list for either form of macro.

8 If a `#` preprocessing token, followed by an identifier, occurs lexically at the point at which a preprocessing directive could begin, the identifier is not subject to macro replacement.

9 A preprocessing directive of the form

`# define` *identifier replacement-list new-line*

defines an *object-like macro* that causes each subsequent instance of the macro name[153] to be replaced by the replacement list of preprocessing tokens that constitute the remainder of the directive.[154] The replacement list is then rescanned for more macro names as specified below.

10 A preprocessing directive of the form

`# define` *identifier lparen identifier-list*$_{opt}$`)` *replacement-list new-line*
`# define` *identifier lparen* `...` `)` *replacement-list new-line*
`# define` *identifier lparen identifier-list* `,` `...` `)` *replacement-list new-line*

defines a *function-like macro* with parameters, whose use is similar syntactically to a function call. The parameters are specified by the optional list of identifiers, whose scope extends from their declaration in the identifier list until the new-line character that terminates the `#define` preprocessing directive. Each subsequent instance of the function-like macro name followed by a `(` as the next preprocessing token introduces

153) Since, by macro-replacement time, all character literals and string literals are preprocessing tokens, not sequences possibly containing identifier-like subsequences (see 2.2, translation phases), they are never scanned for macro names or parameters.
154) An alternative token (2.5) is not an identifier, even when its spelling consists entirely of letters and underscores. Therefore it is not possible to define a macro whose name is the same as that of an alternative token.

the sequence of preprocessing tokens that is replaced by the replacement list in the definition (an invocation of the macro). The replaced sequence of preprocessing tokens is terminated by the matching `)` preprocessing token, skipping intervening matched pairs of left and right parenthesis preprocessing tokens. Within the sequence of preprocessing tokens making up an invocation of a function-like macro, new-line is considered a normal white-space character.

11 The sequence of preprocessing tokens bounded by the outside-most matching parentheses forms the list of arguments for the function-like macro. The individual arguments within the list are separated by comma preprocessing tokens, but comma preprocessing tokens between matching inner parentheses do not separate arguments. If there are sequences of preprocessing tokens within the list of arguments that would otherwise act as preprocessing directives,[155] the behavior is undefined.

12 If there is a `...` immediately preceding the `)` in the function-like macro definition, then the trailing arguments, including any separating comma preprocessing tokens, are merged to form a single item: the *variable arguments*. The number of arguments so combined is such that, following merger, the number of arguments is one more than the number of parameters in the macro definition (excluding the `...`).

16.3.1 Argument substitution [cpp.subst]

1 After the arguments for the invocation of a function-like macro have been identified, argument substitution takes place. A parameter in the replacement list, unless preceded by a `#` or `##` preprocessing token or followed by a `##` preprocessing token (see below), is replaced by the corresponding argument after all macros contained therein have been expanded. Before being substituted, each argument's preprocessing tokens are completely macro replaced as if they formed the rest of the preprocessing file; no other preprocessing tokens are available.

2 An identifier `__VA_ARGS__` that occurs in the replacement list shall be treated as if it were a parameter, and the variable arguments shall form the preprocessing tokens used to replace it.

16.3.2 The # operator [cpp.stringize]

1 Each `#` preprocessing token in the replacement list for a function-like macro shall be followed by a parameter as the next preprocessing token in the replacement list.

2 A *character string literal* is a *string-literal* with no prefix. If, in the replacement list, a parameter is immediately preceded by a `#` preprocessing token, both are replaced by a single character string literal preprocessing token that contains the spelling of the preprocessing token sequence for the corresponding argument. Each occurrence of white space between the argument's preprocessing tokens becomes a single space character in the character string literal. White space before the first preprocessing token and after the last preprocessing token comprising the argument is deleted. Otherwise, the original spelling of each preprocessing token in the argument is retained in the character string literal, except for special handling for producing the spelling of string literals and character literals: a `\` character is inserted before each `"` and `\` character of a character literal or string literal (including the delimiting `"` characters). If the replacement that results is not a valid character string literal, the behavior is undefined. The character string literal corresponding to an empty argument is `""`. The order of evaluation of `#` and `##` operators is unspecified.

16.3.3 The ## operator [cpp.concat]

1 A `##` preprocessing token shall not occur at the beginning or at the end of a replacement list for either form of macro definition.

2 If, in the replacement list of a function-like macro, a parameter is immediately preceded or followed by a `##` preprocessing token, the parameter is replaced by the corresponding argument's preprocessing token

155) Despite the name, a non-directive is a preprocessing directive.

sequence; however, if an argument consists of no preprocessing tokens, the parameter is replaced by a placemarker preprocessing token instead.[156]

3 For both object-like and function-like macro invocations, before the replacement list is reexamined for more macro names to replace, each instance of a `##` preprocessing token in the replacement list (not from an argument) is deleted and the preceding preprocessing token is concatenated with the following preprocessing token. Placemarker preprocessing tokens are handled specially: concatenation of two placemarkers results in a single placemarker preprocessing token, and concatenation of a placemarker with a non-placemarker preprocessing token results in the non-placemarker preprocessing token. If the result is not a valid preprocessing token, the behavior is undefined. The resulting token is available for further macro replacement. The order of evaluation of `##` operators is unspecified.

[*Example:* In the following fragment:

```
#define hash_hash # ## #
#define mkstr(a) # a
#define in_between(a) mkstr(a)
#define join(c, d) in_between(c hash_hash d)
char p[] = join(x, y);          // equivalent to
                                // char p[] = "x ## y";
```

The expansion produces, at various stages:

```
join(x, y)
in_between(x hash_hash y)
in_between(x ## y)
mkstr(x ## y)
"x ## y"
```

In other words, expanding `hash_hash` produces a new token, consisting of two adjacent sharp signs, but this new token is not the `##` operator. — *end example*]

16.3.4 Rescanning and further replacement [cpp.rescan]

1 After all parameters in the replacement list have been substituted and `#` and `##` processing has taken place, all placemarker preprocessing tokens are removed. Then the resulting preprocessing token sequence is rescanned, along with all subsequent preprocessing tokens of the source file, for more macro names to replace.

2 If the name of the macro being replaced is found during this scan of the replacement list (not including the rest of the source file's preprocessing tokens), it is not replaced. Furthermore, if any nested replacements encounter the name of the macro being replaced, it is not replaced. These nonreplaced macro name preprocessing tokens are no longer available for further replacement even if they are later (re)examined in contexts in which that macro name preprocessing token would otherwise have been replaced.

3 The resulting completely macro-replaced preprocessing token sequence is not processed as a preprocessing directive even if it resembles one, but all pragma unary operator expressions within it are then processed as specified in 16.9 below.

16.3.5 Scope of macro definitions [cpp.scope]

1 A macro definition lasts (independent of block structure) until a corresponding `#undef` directive is encountered or (if none is encountered) until the end of the translation unit. Macro definitions have no significance after translation phase 4.

2 A preprocessing directive of the form

156) Placemarker preprocessing tokens do not appear in the syntax because they are temporary entities that exist only within translation phase 4.

`# undef` *identifier new-line*

causes the specified identifier no longer to be defined as a macro name. It is ignored if the specified identifier is not currently defined as a macro name.

3 [*Example:* The simplest use of this facility is to define a "manifest constant," as in

```
#define TABSIZE 100
int table[TABSIZE];
```

— *end example*]

4 [*Example:* The following defines a function-like macro whose value is the maximum of its arguments. It has the advantages of working for any compatible types of the arguments and of generating in-line code without the overhead of function calling. It has the disadvantages of evaluating one or the other of its arguments a second time (including side effects) and generating more code than a function if invoked several times. It also cannot have its address taken, as it has none.

```
#define max(a, b) ((a) > (b) ? (a) : (b))
```

The parentheses ensure that the arguments and the resulting expression are bound properly. — *end example*]

5 [*Example:* To illustrate the rules for redefinition and reexamination, the sequence

```
#define x       3
#define f(a)    f(x * (a))
#undef  x
#define x       2
#define g       f
#define z       z[0]
#define h       g(~
#define m(a)    a(w)
#define w       0,1
#define t(a)    a
#define p()     int
#define q(x)    x
#define r(x,y)  x ## y
#define str(x)  # x

f(y+1) + f(f(z)) % t(t(g)(0) + t)(1);
g(x+(3,4)-w) | h 5) & m
    (f)^m(m);
p() i[q()] = { q(1), r(2,3), r(4,), r(,5), r(,) };
char c[2][6] = { str(hello), str() };
```

results in

```
f(2 * (y+1)) + f(2 * (f(2 * (z[0])))) % f(2 * (0)) + t(1);
f(2 * (2+(3,4)-0,1)) | f(2 * (~ 5)) & f(2 * (0,1))^m(0,1);
int i[] = { 1, 23, 4, 5, };
char c[2][6] = { "hello", "" };
```

— *end example*]

6 [*Example:* To illustrate the rules for creating character string literals and concatenating tokens, the sequence

```
#define str(s)      # s
#define xstr(s)     str(s)
#define debug(s, t) printf("x" # s "= %d, x" # t "= %s", \
                x ## s, x ## t)
```

```
#define INCFILE(n)  vers ## n
#define glue(a, b)  a ## b
#define xglue(a, b) glue(a, b)
#define HIGHLOW     "hello"
#define LOW         LOW ", world"

debug(1, 2);
fputs(str(strncmp("abc\0d", "abc", '\4')  // this goes away
    == 0) str(: @\n), s);
#include xstr(INCFILE(2).h)
glue(HIGH, LOW);
xglue(HIGH, LOW)
```

results in

```
printf("x" "1" "= %d, x" "2" "= %s", x1, x2);
fputs("strncmp(\"abc\\0d\", \"abc\", '\\4') == 0" ": @\n", s);
#include "vers2.h"    (after macro replacement, before file access)
"hello";
"hello" ", world"
```

or, after concatenation of the character string literals,

```
printf("x1= %d, x2= %s", x1, x2);
fputs("strncmp(\"abc\\0d\", \"abc\", '\\4') == 0: @\n", s);
#include "vers2.h"    (after macro replacement, before file access)
"hello";
"hello, world"
```

Space around the # and ## tokens in the macro definition is optional. — *end example*]

7 [*Example:* To illustrate the rules for placemarker preprocessing tokens, the sequence

```
#define t(x,y,z) x ## y ## z
int j[] = { t(1,2,3), t(,4,5), t(6,,7), t(8,9,),
  t(10,,), t(,11,), t(,,12), t(,,) };
```

results in

```
int j[] = { 123, 45, 67, 89,
  10, 11, 12, };
```

— *end example*]

8 [*Example:* To demonstrate the redefinition rules, the following sequence is valid.

```
#define OBJ_LIKE      (1-1)
#define OBJ_LIKE      /* white space */ (1-1) /* other */
#define FUNC_LIKE(a)   ( a )
#define FUNC_LIKE( a )(     /* note the white space */ \
                a /* other stuff on this line
                  */ )
```

But the following redefinitions are invalid:

```
#define OBJ_LIKE    (0)       // different token sequence
#define OBJ_LIKE    (1 - 1)   // different white space
#define FUNC_LIKE(b) ( a )    // different parameter usage
#define FUNC_LIKE(b) ( b )    // different parameter spelling
```

— *end example*]

9 [*Example:* Finally, to show the variable argument list macro facilities:

```
#define debug(...) fprintf(stderr, __VA_ARGS__)
#define showlist(...) puts(#__VA_ARGS__)
#define report(test, ...) ((test) ? puts(#test) : printf(__VA_ARGS__))
debug("Flag");
debug("X = %d\n", x);
showlist(The first, second, and third items.);
report(x>y, "x is %d but y is %d", x, y);
```

results in

```
fprintf(stderr, "Flag");
fprintf(stderr, "X = %d\n", x);
puts("The first, second, and third items.");
((x>y) ? puts("x>y") : printf("x is %d but y is %d", x, y));
```

— *end example*]

16.4 Line control [cpp.line]

1 The string literal of a `#line` directive, if present, shall be a character string literal.

2 The *line number* of the current source line is one greater than the number of new-line characters read or introduced in translation phase 1 (2.2) while processing the source file to the current token.

3 A preprocessing directive of the form

`# line` *digit-sequence new-line*

causes the implementation to behave as if the following sequence of source lines begins with a source line that has a line number as specified by the digit sequence (interpreted as a decimal integer). If the digit sequence specifies zero or a number greater than 2147483647, the behavior is undefined.

4 A preprocessing directive of the form

`# line` *digit-sequence* `"` *s-char-sequence*$_{opt}$`"` *new-line*

sets the presumed line number similarly and changes the presumed name of the source file to be the contents of the character string literal.

5 A preprocessing directive of the form

`# line` *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after `line` on the directive are processed just as in normal text (each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens). If the directive resulting after all replacements does not match one of the two previous forms, the behavior is undefined; otherwise, the result is processed as appropriate.

16.5 Error directive [cpp.error]

1 A preprocessing directive of the form

`# error` *pp-tokens*$_{opt}$ *new-line*

causes the implementation to produce a diagnostic message that includes the specified sequence of preprocessing tokens, and renders the program ill-formed.

16.6 Pragma directive [cpp.pragma]

1 A preprocessing directive of the form

`# pragma` *pp-tokens*$_{opt}$ *new-line*

causes the implementation to behave in an implementation-defined manner. The behavior might cause translation to fail or cause the translator or the resulting program to behave in a non-conforming manner. Any pragma that is not recognized by the implementation is ignored.

16.7 Null directive [cpp.null]

1 A preprocessing directive of the form

`#` *new-line*

has no effect.

16.8 Predefined macro names [cpp.predefined]

1 The following macro names shall be defined by the implementation:

`__cplusplus`
The name `__cplusplus` is defined to the value `201402L` when compiling a C++ translation unit.[157]

`__DATE__`
The date of translation of the source file: a character string literal of the form `"Mmm dd yyyy"`, where the names of the months are the same as those generated by the `asctime` function, and the first character of `dd` is a space character if the value is less than 10. If the date of translation is not available, an implementation-defined valid date shall be supplied.

`__FILE__`
The presumed name of the current source file (a character string literal).[158]

`__LINE__`
The presumed line number (within the current source file) of the current source line (an integer literal).[159]

`__STDC_HOSTED__`
The integer literal `1` if the implementation is a hosted implementation or the integer literal `0` if it is not.

`__TIME__`
The time of translation of the source file: a character string literal of the form `"hh:mm:ss"` as in the time generated by the `asctime` function. If the time of translation is not available, an implementation-defined valid time shall be supplied.

2 The following macro names are conditionally defined by the implementation:

`__STDC__`
Whether `__STDC__` is predefined and if so, what its value is, are implementation-defined.

`__STDC_MB_MIGHT_NEQ_WC__`
The integer literal `1`, intended to indicate that, in the encoding for `wchar_t`, a member of the basic character set need not have a code value equal to its value when used as the lone character in an ordinary character literal.

`__STDC_VERSION__`
Whether `__STDC_VERSION__` is predefined and if so, what its value is, are implementation-defined.

157) It is intended that future versions of this standard will replace the value of this macro with a greater value. Non-conforming compilers should use a value with at most five decimal digits.
158) The presumed source file name can be changed by the `#line` directive.
159) The presumed line number can be changed by the `#line` directive.

`__STDC_ISO_10646__`
An integer literal of the form `yyyymmL` (for example, `199712L`). If this symbol is defined, then every character in the Unicode required set, when stored in an object of type `wchar_t`, has the same value as the short identifier of that character. The *Unicode required set* consists of all the characters that are defined by ISO/IEC 10646, along with all amendments and technical corrigenda as of the specified year and month.

`__STDCPP_STRICT_POINTER_SAFETY__`
Defined, and has the value integer literal 1, if and only if the implementation has strict pointer safety (3.7.4.3).

`__STDCPP_THREADS__`
Defined, and has the value integer literal 1, if and only if a program can have more than one thread of execution (1.10).

3 The values of the predefined macros (except for `__FILE__` and `__LINE__`) remain constant throughout the translation unit.

4 If any of the pre-defined macro names in this subclause, or the identifier `defined`, is the subject of a `#define` or a `#undef` preprocessing directive, the behavior is undefined. Any other predefined macro names shall begin with a leading underscore followed by an uppercase letter or a second underscore.

16.9 Pragma operator [cpp.pragma.op]

A unary operator expression of the form:

`_Pragma (` *string-literal* `)`

is processed as follows: The string literal is *destringized* by deleting the `L` prefix, if present, deleting the leading and trailing double-quotes, replacing each escape sequence `\"` by a double-quote, and replacing each escape sequence `\\` by a single backslash. The resulting sequence of characters is processed through translation phase 3 to produce preprocessing tokens that are executed as if they were the *pp-tokens* in a pragma directive. The original four preprocessing tokens in the unary operator expression are removed.

[*Example:*

```
#pragma listing on "..\listing.dir"
```

can also be expressed as:

```
_Pragma ( "listing on \"..\\listing.dir\"" )
```

The latter form is processed in the same way whether it appears literally as shown, or results from macro replacement, as in:

```
#define LISTING(x) PRAGMA(listing on #x)
#define PRAGMA(x) _Pragma(#x)

LISTING( ..\listing.dir )
```

— *end example*]

17 Library introduction [library]

17.1 General [library.general]

1 This Clause describes the contents of the *C++ standard library*, how a well-formed C++ program makes use of the library, and how a conforming implementation may provide the entities in the library.

2 The following subclauses describe the definitions (17.3), method of description (17.5), and organization (17.6.1) of the library. Clause 17.6, Clauses 18 through 30, and Annex D specify the contents of the library, as well as library requirements and constraints on both well-formed C++ programs and conforming implementations.

3 Detailed specifications for each of the components in the library are in Clauses 18–30, as shown in Table 13.

Table 13 — Library categories

Clause	Category
18	Language support library
19	Diagnostics library
20	General utilities library
21	Strings library
22	Localization library
23	Containers library
24	Iterators library
25	Algorithms library
26	Numerics library
27	Input/output library
28	Regular expressions library
29	Atomic operations library
30	Thread support library

4 The language support library (Clause 18) provides components that are required by certain parts of the C++ language, such as memory allocation (5.3.4, 5.3.5) and exception processing (Clause 15).

5 The diagnostics library (Clause 19) provides a consistent framework for reporting errors in a C++ program, including predefined exception classes.

6 The general utilities library (Clause 20) includes components used by other library elements, such as a predefined storage allocator for dynamic storage management (3.7.4), and components used as infrastructure in C++ programs, such as a tuples, function wrappers, and time facilities.

7 The strings library (Clause 21) provides support for manipulating text represented as sequences of type `char`, sequences of type `char16_t`, sequences of type `char32_t`, sequences of type `wchar_t`, and sequences of any other character-like type.

8 The localization library (Clause 22) provides extended internationalization support for text processing.

9 The containers (Clause 23), iterators (Clause 24), and algorithms (Clause 25) libraries provide a C++ program with access to a subset of the most widely used algorithms and data structures.

10 The numerics library (Clause 26) provides numeric algorithms and complex number components that extend support for numeric processing. The `valarray` component provides support for *n*-at-a-time processing, potentially implemented as parallel operations on platforms that support such processing. The random number component provides facilities for generating pseudo-random numbers.

11 The input/output library (Clause 27) provides the `iostream` components that are the primary mechanism for C++ program input and output. They can be used with other elements of the library, particularly strings, locales, and iterators.

12 The regular expressions library (Clause 28) provides regular expression matching and searching.

13 The atomic operations library (Clause 29) allows more fine-grained concurrent access to shared data than is possible with locks.

14 The thread support library (Clause 30) provides components to create and manage threads, including mutual exclusion and interthread communication.

17.2 The C standard library [library.c]

1 The C++ standard library also makes available the facilities of the C standard library, suitably adjusted to ensure static type safety.

2 The descriptions of many library functions rely on the C standard library for the signatures and semantics of those functions. In all such cases, any use of the `restrict` qualifier shall be omitted.

17.3 Definitions [definitions]

17.3.1 [defns.arbitrary.stream]

arbitrary-positional stream
a stream (described in Clause 27) that can seek to any integral position within the length of the stream
[*Note:* Every arbitrary-positional stream is also a repositional stream. — *end note*]

17.3.2 [defns.block]

block
place a thread in the blocked state

17.3.3 [defns.blocked]

blocked thread
a thread that is waiting for some condition (other than the availability of a processor) to be satisfied before it can continue execution[160]

17.3.4 [defns.character]

character
<Clauses 21, 22, 27, and 28> any object which, when treated sequentially, can represent text
[*Note:* The term does not mean only `char`, `char16_t`, `char32_t`, and `wchar_t` objects, but any value that can be represented by a type that provides the definitions specified in these Clauses. — *end note*]

17.3.5 [defns.character.container]

character container type
a class or a type used to represent a *character*
[*Note:* It is used for one of the template parameters of the string, iostream, and regular expression class templates. A character container type is a POD (3.9) type. — *end note*]

17.3.6 [defns.comparison]

comparison function
an operator function (13.5) for any of the equality (5.10) or relational (5.9) operators

160) This definition is taken from POSIX.

17.3.7 [defns.component]

component

a group of library entities directly related as members, parameters, or return types

[*Note:* For example, the class template `basic_string` and the non-member function templates that operate on strings are referred to as the *string component*. — *end note*]

17.3.8 [defns.deadlock]

deadlock

one or more threads are unable to continue execution because each is blocked waiting for one or more of the others to satisfy some condition

17.3.9 [defns.default.behavior.impl]

default behavior

<implementation> any specific behavior provided by the implementation, within the scope of the *required behavior*

17.3.10 [defns.default.behavior.func]

default behavior

<specification> a description of *replacement function* and *handler function* semantics

17.3.11 [defns.handler]

handler function

a *non-reserved function* whose definition may be provided by a C++ program

[*Note:* A C++ program may designate a handler function at various points in its execution by supplying a pointer to the function when calling any of the library functions that install handler functions (Clause 18). — *end note*]

17.3.12 [defns.iostream.templates]

iostream class templates

templates, defined in Clause 27, that take two template arguments

[*Note:* The arguments are named `charT` and `traits`. The argument `charT` is a character container class, and the argument `traits` is a class which defines additional characteristics and functions of the character type represented by `charT` necessary to implement the iostream class templates. — *end note*]

17.3.13 [defns.modifier]

modifier function

a class member function (9.3) other than a constructor, assignment operator, or destructor that alters the state of an object of the class

17.3.14 [defns.move.constr]

move construction

direct-initialization of an object of some type with an rvalue of the same type

17.3.15 [defns.move.assign]

move assignment

assignment of an rvalue of some object type to a modifiable lvalue of the same type

17.3.16 [defns.obj.state]

object state

the current value of all non-static class members of an object (9.2)

[*Note:* The state of an object can be obtained by using one or more *observer functions*. — *end note*]

17.3.17 [defns.ntcts]
NTCTS
a sequence of values that have *character type* that precede the terminating null character type value `charT()`

17.3.18 [defns.observer]
observer function
a class member function (9.3) that accesses the state of an object of the class but does not alter that state
[*Note:* Observer functions are specified as `const` member functions (9.3.2). — *end note*]

17.3.19 [defns.referenceable]
referenceable type
An object type, a function type that does not have cv-qualifiers or a *ref-qualifier*, or a reference type. [*Note:* The term describes a type to which a reference can be created, including reference types. — *end note*]

17.3.20 [defns.replacement]
replacement function
a *non-reserved function* whose definition is provided by a C++ program
[*Note:* Only one definition for such a function is in effect for the duration of the program's execution, as the result of creating the program (2.2) and resolving the definitions of all translation units (3.5). — *end note*]

17.3.21 [defns.repositional.stream]
repositional stream
a stream (described in Clause 27) that can seek to a position that was previously encountered

17.3.22 [defns.required.behavior]
required behavior
a description of *replacement function* and *handler function* semantics applicable to both the behavior provided by the implementation and the behavior of any such function definition in the program
[*Note:* If such a function defined in a C++ program fails to meet the required behavior when it executes, the behavior is undefined. — *end note*]

17.3.23 [defns.reserved.function]
reserved function
a function, specified as part of the C++ standard library, that must be defined by the implementation
[*Note:* If a C++ program provides a definition for any reserved function, the results are undefined. — *end note*]

17.3.24 [defns.stable]
stable algorithm
an algorithm that preserves, as appropriate to the particular algorithm, the order of elements
[*Note:* Requirements for stable algorithms are given in 17.6.5.7. — *end note*]

17.3.25 [defns.traits]
traits class
a class that encapsulates a set of types and functions necessary for class templates and function templates to manipulate objects of types for which they are instantiated
[*Note:* Traits classes defined in Clauses 21, 22 and 27 are *character traits*, which provide the character handling support needed by the string and iostream classes. — *end note*]

17.3.26 [defns.unblock]
unblock
place a thread in the unblocked state

17.3.27 [defns.valid]

valid but unspecified state

an object state that is not specified except that the object's invariants are met and operations on the object behave as specified for its type

[*Example:* If an object `x` of type `std::vector<int>` is in a valid but unspecified state, `x.empty()` can be called unconditionally, and `x.front()` can be called only if `x.empty()` returns `false`. — *end example*]

17.4 Additional definitions [defns.additional]

1 1.3 defines additional terms used elsewhere in this International Standard.

17.5 Method of description (Informative) [description]

1 This subclause describes the conventions used to specify the C++ standard library. 17.5.1 describes the structure of the normative Clauses 18 through 30 and Annex D. 17.5.2 describes other editorial conventions.

17.5.1 Structure of each clause [structure]

17.5.1.1 Elements [structure.elements]

1 Each library clause contains the following elements, as applicable:[161]

(1.1) — Summary

(1.2) — Requirements

(1.3) — Detailed specifications

(1.4) — References to the Standard C library

17.5.1.2 Summary [structure.summary]

1 The Summary provides a synopsis of the category, and introduces the first-level subclauses. Each subclause also provides a summary, listing the headers specified in the subclause and the library entities provided in each header.

2 Paragraphs labeled "Note(s):" or "Example(s):" are informative, other paragraphs are normative.

3 The contents of the summary and the detailed specifications include:

(3.1) — macros

(3.2) — values

(3.3) — types

(3.4) — classes and class templates

(3.5) — functions and function templates

(3.6) — objects

161) To save space, items that do not apply to a Clause are omitted. For example, if a Clause does not specify any requirements, there will be no "Requirements" subclause.

17.5.1.3 Requirements [structure.requirements]

1 Requirements describe constraints that shall be met by a C++ program that extends the standard library. Such extensions are generally one of the following:

(1.1) — Template arguments

(1.2) — Derived classes

(1.3) — Containers, iterators, and algorithms that meet an interface convention

2 The string and iostream components use an explicit representation of operations required of template arguments. They use a class template `char_traits` to define these constraints.

3 Interface convention requirements are stated as generally as possible. Instead of stating "class X has to define a member function `operator++()`," the interface requires "for any object `x` of class `X`, `++x` is defined." That is, whether the operator is a member is unspecified.

4 Requirements are stated in terms of well-defined expressions that define valid terms of the types that satisfy the requirements. For every set of well-defined expression requirements there is a table that specifies an initial set of the valid expressions and their semantics. Any generic algorithm (Clause 25) that uses the well-defined expression requirements is described in terms of the valid expressions for its template type parameters.

5 Template argument requirements are sometimes referenced by name. See 17.5.2.1.

6 In some cases the semantic requirements are presented as C++ code. Such code is intended as a specification of equivalence of a construct to another construct, not necessarily as the way the construct must be implemented.[162]

17.5.1.4 Detailed specifications [structure.specifications]

1 The detailed specifications each contain the following elements:

(1.1) — name and brief description

(1.2) — synopsis (class definition or function declaration, as appropriate)

(1.3) — restrictions on template arguments, if any

(1.4) — description of class invariants

(1.5) — description of function semantics

2 Descriptions of class member functions follow the order (as appropriate):[163]

(2.1) — constructor(s) and destructor

(2.2) — copying, moving & assignment functions

(2.3) — comparison functions

(2.4) — modifier functions

(2.5) — observer functions

(2.6) — operators and other non-member functions

162) Although in some cases the code given is unambiguously the optimum implementation.

163) To save space, items that do not apply to a class are omitted. For example, if a class does not specify any comparison functions, there will be no "Comparison functions" subclause.

3 Descriptions of function semantics contain the following elements (as appropriate):[164]

(3.1) — *Requires:* the preconditions for calling the function

(3.2) — *Effects:* the actions performed by the function

(3.3) — *Synchronization:* the synchronization operations (1.10) applicable to the function

(3.4) — *Postconditions:* the observable results established by the function

(3.5) — *Returns:* a description of the value(s) returned by the function

(3.6) — *Throws:* any exceptions thrown by the function, and the conditions that would cause the exception

(3.7) — *Complexity:* the time and/or space complexity of the function

(3.8) — *Remarks:* additional semantic constraints on the function

(3.9) — *Error conditions:* the error conditions for error codes reported by the function.

(3.10) — *Notes:* non-normative comments about the function

4 Whenever the *Effects:* element specifies that the semantics of some function `F` are *Equivalent to* some code sequence, then the various elements are interpreted as follows. If `F`'s semantics specifies a *Requires:* element, then that requirement is logically imposed prior to the *equivalent-to* semantics. Next, the semantics of the code sequence are determined by the *Requires:*, *Effects:*, *Postconditions:*, *Returns:*, *Throws:*, *Complexity:*, *Remarks:*, *Error conditions:*, and *Notes:* specified for the function invocations contained in the code sequence. The value returned from `F` is specified by `F`'s *Returns:* element, or if `F` has no *Returns:* element, a non-`void` return from `F` is specified by the *Returns:* elements in the code sequence. If `F`'s semantics contains a *Throws:*, *Postconditions:*, or *Complexity:* element, then that supersedes any occurrences of that element in the code sequence.

5 For non-reserved replacement and handler functions, Clause 18 specifies two behaviors for the functions in question: their required and default behavior. The *default behavior* describes a function definition provided by the implementation. The *required behavior* describes the semantics of a function definition provided by either the implementation or a C++ program. Where no distinction is explicitly made in the description, the behavior described is the required behavior.

6 If the formulation of a complexity requirement calls for a negative number of operations, the actual requirement is zero operations.[165]

7 Complexity requirements specified in the library clauses are upper bounds, and implementations that provide better complexity guarantees satisfy the requirements.

8 Error conditions specify conditions where a function may fail. The conditions are listed, together with a suitable explanation, as the `enum class errc` constants (19.5).

17.5.1.5 C library [structure.see.also]

1 Paragraphs labeled "See also:" contain cross-references to the relevant portions of this International Standard and the ISO C standard, which is incorporated into this International Standard by reference.

17.5.2 Other conventions [conventions]

1 This subclause describes several editorial conventions used to describe the contents of the C++ standard library. These conventions are for describing implementation-defined types (17.5.2.1), and member functions (17.5.2.2).

164) To save space, items that do not apply to a function are omitted. For example, if a function does not specify any further preconditions, there will be no "Requires" paragraph.

165) This simplifies the presentation of complexity requirements in some cases.

17.5.2.1 Type descriptions [type.descriptions]

17.5.2.1.1 General [type.descriptions.general]

1 The Requirements subclauses may describe names that are used to specify constraints on template arguments.[166] These names are used in library Clauses to describe the types that may be supplied as arguments by a C++ program when instantiating template components from the library.

2 Certain types defined in Clause 27 are used to describe implementation-defined types. They are based on other types, but with added constraints.

17.5.2.1.2 Enumerated types [enumerated.types]

1 Several types defined in Clause 27 are *enumerated types*. Each enumerated type may be implemented as an enumeration or as a synonym for an enumeration.[167]

2 The enumerated type *enumerated* can be written:

```
enum enumerated { V0, V1, V2, V3, ..... };

static const enumerated C0 (V0);
static const enumerated C1 (V1);
static const enumerated C2 (V2);
static const enumerated C3 (V3);
  .....
```

3 Here, the names *C0*, *C1*, etc. represent *enumerated elements* for this particular enumerated type. All such elements have distinct values.

17.5.2.1.3 Bitmask types [bitmask.types]

1 Several types defined in Clauses 18 through 30 and Annex D are *bitmask types*. Each bitmask type can be implemented as an enumerated type that overloads certain operators, as an integer type, or as a `bitset` (20.6).

2 The bitmask type *bitmask* can be written:

```
// For exposition only.
// int_type is an integral type capable of
// representing all values of the bitmask type.
enum bitmask : int_type {
  V0 = 1 << 0, V1 = 1 << 1, V2 = 1 << 2, V3 = 1 << 3, .....
};

constexpr bitmask C0(V0);
constexpr bitmask C1(V1);
constexpr bitmask C2(V2);
constexpr bitmask C3(V3);
  .....

constexpr bitmask operator&(bitmask X, bitmask Y) {
  return static_cast<bitmask>(
    static_cast<int_type>(X) & static_cast<int_type>(Y));
}
constexpr bitmask operator|(bitmask X, bitmask Y) {
  return static_cast<bitmask>(
```

166) Examples from 17.6.3 include: `EqualityComparable`, `LessThanComparable`, `CopyConstructible`. Examples from 24.2 include: `InputIterator`, `ForwardIterator`, `Function`, `Predicate`.
167) Such as an integer type, with constant integer values (3.9.1).

```
    static_cast<int_type>(X) | static_cast<int_type>(Y));
}
constexpr bitmask operator^(bitmask X, bitmask Y){
  return static_cast<bitmask>(
    static_cast<int_type>(X) ^ static_cast<int_type>(Y));
}
constexpr bitmask operator~(bitmask X){
  return static_cast<bitmask>(~static_cast<int_type>(X));
}
bitmask& operator&=(bitmask& X, bitmask Y){
  X = X & Y; return X;
}
bitmask& operator|=(bitmask& X, bitmask Y) {
  X = X | Y; return X;
}
bitmask& operator^=(bitmask& X, bitmask Y) {
  X = X ^ Y; return X;
}
```

3 Here, the names *C0*, *C1*, etc. represent *bitmask elements* for this particular bitmask type. All such elements have distinct, nonzero values such that, for any pair *Ci* and *Cj* where $i != j$, *Ci* & *Ci* is nonzero and *Ci* & *Cj* is zero. Additionally, the value 0 is used to represent an *empty bitmask*, in which no bitmask elements are set.

4 The following terms apply to objects and values of bitmask types:

(4.1) — To *set* a value *Y* in an object *X* is to evaluate the expression *X* `|=` *Y*.

(4.2) — To *clear* a value *Y* in an object *X* is to evaluate the expression *X* `&= ~` *Y*.

(4.3) — The value *Y* *is set* in the object *X* if the expression *X* `&` *Y* is nonzero.

17.5.2.1.4 Character sequences [character.seq]

1 The C standard library makes widespread use of characters and character sequences that follow a few uniform conventions:

(1.1) — A *letter* is any of the 26 lowercase or 26 uppercase letters in the basic execution character set.[168]

(1.2) — The *decimal-point character* is the (single-byte) character used by functions that convert between a (single-byte) character sequence and a value of one of the floating-point types. It is used in the character sequence to denote the beginning of a fractional part. It is represented in Clauses 18 through 30 and Annex D by a period, `'.'`, which is also its value in the `"C"` locale, but may change during program execution by a call to `setlocale(int, const char*)`,[169] or by a change to a `locale` object, as described in Clauses 22.3 and 27.

(1.3) — A *character sequence* is an array object (8.3.4) *A* that can be declared as *`T A[N]`*, where *T* is any of the types `char`, `unsigned char`, or `signed char` (3.9.1), optionally qualified by any combination of `const` or `volatile`. The initial elements of the array have defined contents up to and including an element determined by some predicate. A character sequence can be designated by a pointer value *S* that points to its first element.

168) Note that this definition differs from the definition in ISO C 7.1.1.
169) declared in `<clocale>` (22.6).

17.5.2.1.4.1 Byte strings [byte.strings]

1 A *null-terminated byte string*, or NTBS, is a character sequence whose highest-addressed element with defined content has the value zero (the *terminating null* character); no other element in the sequence has the value zero.[170]

2 The *length* of an NTBS is the number of elements that precede the terminating null character. An *empty* NTBS has a length of zero.

3 The *value* of an NTBS is the sequence of values of the elements up to and including the terminating null character.

4 A *static* NTBS is an NTBS with static storage duration.[171]

17.5.2.1.4.2 Multibyte strings [multibyte.strings]

1 A *null-terminated multibyte string,* or NTMBS, is an NTBS that constitutes a sequence of valid multibyte characters, beginning and ending in the initial shift state.[172]

2 A *static* NTMBS is an NTMBS with static storage duration.

17.5.2.2 Functions within classes [functions.within.classes]

1 For the sake of exposition, Clauses 18 through 30 and Annex D do not describe copy/move constructors, assignment operators, or (non-virtual) destructors with the same apparent semantics as those that can be generated by default (12.1, 12.4, 12.8). It is unspecified whether the implementation provides explicit definitions for such member function signatures, or for virtual destructors that can be generated by default.

2 For the sake of exposition, the library clauses sometimes annotate constructors with *`EXPLICIT`*. Such a constructor is conditionally declared as either explicit or non-explicit (12.3.1). [*Note:* This is typically implemented by declaring two such constructors, of which at most one participates in overload resolution. — *end note*]

17.5.2.3 Private members [objects.within.classes]

1 Clauses 18 through 30 and Annex D do not specify the representation of classes, and intentionally omit specification of class members (9.2). An implementation may define static or non-static class members, or both, as needed to implement the semantics of the member functions specified in Clauses 18 through 30 and Annex D.

2 Objects of certain classes are sometimes required by the external specifications of their classes to store data, apparently in member objects. For the sake of exposition, some subclauses provide representative declarations, and semantic requirements, for private member objects of classes that meet the external specifications of the classes. The declarations for such member objects and the definitions of related member types are followed by a comment that ends with *exposition only*, as in:

```
streambuf* sb;   // exposition only
```

3 An implementation may use any technique that provides equivalent external behavior.

17.6 Library-wide requirements [requirements]

1 This subclause specifies requirements that apply to the entire C++ standard library. Clauses 18 through 30 and Annex D specify the requirements of individual entities within the library.

170) Many of the objects manipulated by function signatures declared in `<cstring>` (21.8) are character sequences or NTBSs. The size of some of these character sequences is limited by a length value, maintained separately from the character sequence.
171) A string literal, such as `"abc"`, is a static NTBS.
172) An NTBS that contains characters only from the basic execution character set is also an NTMBS. Each multibyte character then consists of a single byte.

2 Requirements specified in terms of interactions between threads do not apply to programs having only a single thread of execution.

3 Within this subclause, 17.6.1 describes the library's contents and organization, 17.6.2 describes how well-formed C++ programs gain access to library entities, 17.6.3 describes constraints on types and functions used with the C++ standard library, 17.6.4 describes constraints on well-formed C++ programs, and 17.6.5 describes constraints on conforming implementations.

17.6.1 Library contents and organization [organization]

1 17.6.1.1 describes the entities defined in the C++ standard library. 17.6.1.2 lists the standard library headers and some constraints on those headers. 17.6.1.3 lists requirements for a freestanding implementation of the C++ standard library.

17.6.1.1 Library contents [contents]

1 The C++ standard library provides definitions for the following types of entities: macros, values, types, templates, classes, functions, objects.

2 All library entities except macros, `operator new` and `operator delete` are defined within the namespace `std` or namespaces nested within namespace `std`.[173] It is unspecified whether names declared in a specific namespace are declared directly in that namespace or in an inline namespace inside that namespace.[174]

3 Whenever a name `x` defined in the standard library is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless explicitly described otherwise. For example, if the Effects section for library function `F` is described as calling library function `G`, the function `::std::G` is meant.

17.6.1.2 Headers [headers]

1 Each element of the C++ standard library is declared or defined (as appropriate) in a *header*.[175]

2 The C++ standard library provides 53 *C++ library headers*, as shown in Table 14.

Table 14 — C++ library headers

`<algorithm>`	`<fstream>`	`<list>`	`<regex>`	`<tuple>`
`<array>`	`<functional>`	`<locale>`	`<scoped_allocator>`	`<type_traits>`
`<atomic>`	`<future>`	`<map>`	`<set>`	`<typeindex>`
`<bitset>`	`<initializer_list>`	`<memory>`	`<sstream>`	`<typeinfo>`
`<chrono>`	`<iomanip>`	`<mutex>`	`<stack>`	`<unordered_map>`
`<codecvt>`	`<ios>`	`<new>`	`<stdexcept>`	`<unordered_set>`
`<complex>`	`<iosfwd>`	`<numeric>`	`<streambuf>`	`<utility>`
`<condition_variable>`	`<iostream>`	`<ostream>`	`<string>`	`<valarray>`
`<deque>`	`<istream>`	`<queue>`	`<strstream>`	`<vector>`
`<exception>`	`<iterator>`	`<random>`	`<system_error>`	
`<forward_list>`	`<limits>`	`<ratio>`	`<thread>`	

3 The facilities of the C standard Library are provided in 26 additional headers, as shown in Table 15.

4 Except as noted in Clauses 18 through 30 and Annex D, the contents of each header `c`*`name`* shall be the same as that of the corresponding header *`name`*`.h`, as specified in the C standard library (1.2) or the C Unicode

173) The C standard library headers (Annex D.5) also define names within the global namespace, while the C++ headers for C library facilities (17.6.1.2) may also define names within the global namespace.
174) This gives implementers freedom to use inline namespaces to support multiple configurations of the library.
175) A header is not necessarily a source file, nor are the sequences delimited by `<` and `>` in header names necessarily valid source file names (16.2).

Table 15 — C++ headers for C library facilities

<cassert>	<cinttypes>	<csignal>	<cstdio>	<cwchar>
<ccomplex>	<ciso646>	<cstdalign>	<cstdlib>	<cwctype>
<cctype>	<climits>	<cstdarg>	<cstring>	
<cerrno>	<clocale>	<cstdbool>	<ctgmath>	
<cfenv>	<cmath>	<cstddef>	<ctime>	
<cfloat>	<csetjmp>	<cstdint>	<cuchar>	

TR, as appropriate, as if by inclusion. In the C++ standard library, however, the declarations (except for names which are defined as macros in C) are within namespace scope (3.3.6) of the namespace `std`. It is unspecified whether these names are first declared within the global namespace scope and are then injected into namespace `std` by explicit *using-declaration*s (7.3.3).

5 Names which are defined as macros in C shall be defined as macros in the C++ standard library, even if C grants license for implementation as functions. [*Note:* The names defined as macros in C include the following: `assert`, `offsetof`, `setjmp`, `va_arg`, `va_end`, and `va_start`. — *end note*]

6 Names that are defined as functions in C shall be defined as functions in the C++ standard library.[176]

7 Identifiers that are keywords or operators in C++ shall not be defined as macros in C++ standard library headers.[177]

8 D.5, C standard library headers, describes the effects of using the *`name`*`.h` (C header) form in a C++ program.[178]

17.6.1.3 Freestanding implementations [compliance]

1 Two kinds of implementations are defined: *hosted* and *freestanding* (1.4). For a hosted implementation, this International Standard describes the set of available headers.

2 A freestanding implementation has an implementation-defined set of headers. This set shall include at least the headers shown in Table 16.

3 The supplied version of the header `<cstdlib>` shall declare at least the functions `abort`, `atexit`, `at_quick_exit`, `exit`, and `quick_exit` (18.5). The other headers listed in this table shall meet the same requirements as for a hosted implementation.

17.6.2 Using the library [using]

17.6.2.1 Overview [using.overview]

1 This section describes how a C++ program gains access to the facilities of the C++ standard library. 17.6.2.2 describes effects during translation phase 4, while 17.6.2.3 describes effects during phase 8 (2.2).

17.6.2.2 Headers [using.headers]

1 The entities in the C++ standard library are defined in headers, whose contents are made available to a translation unit when it contains the appropriate `#include` preprocessing directive (16.2).

2 A translation unit may include library headers in any order (Clause 2). Each may be included more than once, with no effect different from being included exactly once, except that the effect of including either

176) This disallows the practice, allowed in C, of providing a masking macro in addition to the function prototype. The only way to achieve equivalent inline behavior in C++ is to provide a definition as an extern inline function.
177) In particular, including the standard header `<iso646.h>` or `<ciso646>` has no effect.
178) The `".h"` headers dump all their names into the global namespace, whereas the newer forms keep their names in namespace `std`. Therefore, the newer forms are the preferred forms for all uses except for C++ programs which are intended to be strictly compatible with C.

Table 16 — C++ headers for freestanding implementations

Subclause		Header(s)
		`<ciso646>`
18.2	Types	`<cstddef>`
18.3	Implementation properties	`<cfloat>` `<limits>` `<climits>`
18.4	Integer types	`<cstdint>`
18.5	Start and termination	`<cstdlib>`
18.6	Dynamic memory management	`<new>`
18.7	Type identification	`<typeinfo>`
18.8	Exception handling	`<exception>`
18.9	Initializer lists	`<initializer_list>`
18.10	Other runtime support	`<cstdalign>` `<cstdarg>` `<cstdbool>`
20.10	Type traits	`<type_traits>`
29	Atomics	`<atomic>`

`<cassert>` or `<assert.h>` depends each time on the lexically current definition of `NDEBUG`.[179]

3 A translation unit shall include a header only outside of any declaration or definition, and shall include the header lexically before the first reference in that translation unit to any of the entities declared in that header. No diagnostic is required.

17.6.2.3 Linkage [using.linkage]

1 Entities in the C++ standard library have external linkage (3.5). Unless otherwise specified, objects and functions have the default `extern "C++"` linkage (7.5).

2 Whether a name from the C standard library declared with external linkage has `extern "C"` or `extern "C++"` linkage is implementation-defined. It is recommended that an implementation use `extern "C++"` linkage for this purpose.[180]

3 Objects and functions defined in the library and required by a C++ program are included in the program prior to program startup.

See also: replacement functions (17.6.4.6), run-time changes (17.6.4.7).

17.6.3 Requirements on types and expressions [utility.requirements]

1 17.6.3.1 describes requirements on types and expressions used to instantiate templates defined in the C++ standard library. 17.6.3.2 describes the requirements on swappable types and swappable expressions. 17.6.3.3 describes the requirements on pointer-like types that support null values. 17.6.3.4 describes the requirements on hash function objects. 17.6.3.5 describes the requirements on storage allocators.

17.6.3.1 Template argument requirements [utility.arg.requirements]

1 The template definitions in the C++ standard library refer to various named requirements whose details are set out in tables 17–24. In these tables, `T` is an object or reference type to be supplied by a C++ program instantiating a template; `a`, `b`, and `c` are values of type (possibly `const`) `T`; `s` and `t` are modifiable lvalues of type `T`; `u` denotes an identifier; `rv` is an rvalue of type `T`; and `v` is an lvalue of type (possibly `const`) `T` or an rvalue of type `const T`.

179) This is the same as the Standard C library.

180) The only reliable way to declare an object or function signature from the Standard C library is by including the header that declares it, notwithstanding the latitude granted in 7.1.4 of the C Standard.

[2] In general, a default constructor is not required. Certain container class member function signatures specify `T()` as a default argument. `T()` shall be a well-defined expression (8.5) if one of those signatures is called using the default argument (8.3.6).

Table 17 — `EqualityComparable` requirements [**equalitycomparable**]

Expression	Return type	Requirement
`a == b`	convertible to `bool`	`==` is an equivalence relation, that is, it has the following properties: — For all `a`, `a == a`. — If `a == b`, then `b == a`. — If `a == b` and `b == c`, then `a == c`.

Table 18 — `LessThanComparable` requirements [**lessthancomparable**]

Expression	Return type	Requirement
`a < b`	convertible to `bool`	`<` is a strict weak ordering relation (25.4)

Table 19 — `DefaultConstructible` requirements [**defaultconstructible**]

Expression	Post-condition
`T t;`	object `t` is default-initialized
`T u{};`	object `u` is value-initialized or aggregate-initialized
`T()` `T{}`	a temporary object of type `T` is value-initialized or aggregate-initialized

Table 20 — `MoveConstructible` requirements [**moveconstructible**]

Expression	Post-condition
`T u = rv;`	`u` is equivalent to the value of `rv` before the construction
`T(rv)`	`T(rv)` is equivalent to the value of `rv` before the construction
`rv`'s state is unspecified [*Note:* `rv` must still meet the requirements of the library component that is using it. The operations listed in those requirements must work as specified whether `rv` has been moved from or not. — *end note*]	

Table 21 — `CopyConstructible` requirements (in addition to `MoveConstructible`) [**copyconstructible**]

Expression	Post-condition
`T u = v;`	the value of `v` is unchanged and is equivalent to `u`
`T(v)`	the value of `v` is unchanged and is equivalent to `T(v)`

Table 22 — `MoveAssignable` requirements **[moveassignable]**

Expression	Return type	Return value	Post-condition
`t = rv`	`T&`	`t`	`t` is equivalent to the value of `rv` before the assignment
`rv`'s state is unspecified. [*Note:* `rv` must still meet the requirements of the library component that is using it. The operations listed in those requirements must work as specified whether `rv` has been moved from or not. — *end note*]			

Table 23 — `CopyAssignable` requirements (in addition to `MoveAssignable`) **[copyassignable]**

Expression	Return type	Return value	Post-condition
`t = v`	`T&`	`t`	`t` is equivalent to `v`, the value of `v` is unchanged

Table 24 — `Destructible` requirements **[destructible]**

Expression	Post-condition
`u.~T()`	All resources owned by `u` are reclaimed, no exception is propagated.

17.6.3.2 `Swappable` requirements [swappable.requirements]

1 This subclause provides definitions for swappable types and expressions. In these definitions, let `t` denote an expression of type `T`, and let `u` denote an expression of type `U`.

2 An object `t` is *swappable with* an object `u` if and only if:

(2.1) — the expressions `swap(t, u)` and `swap(u, t)` are valid when evaluated in the context described below, and

(2.2) — these expressions have the following effects:

(2.2.1) — the object referred to by `t` has the value originally held by `u` and

(2.2.2) — the object referred to by `u` has the value originally held by `t`.

3 The context in which `swap(t, u)` and `swap(u, t)` are evaluated shall ensure that a binary non-member function named "swap" is selected via overload resolution (13.3) on a candidate set that includes:

(3.1) — the two `swap` function templates defined in `<utility>` (20.2) and

(3.2) — the lookup set produced by argument-dependent lookup (3.4.2).

[*Note:* If `T` and `U` are both fundamental types or arrays of fundamental types and the declarations from the header `<utility>` are in scope, the overall lookup set described above is equivalent to that of the qualified name lookup applied to the expression `std::swap(t, u)` or `std::swap(u, t)` as appropriate. — *end note*]

[*Note:* It is unspecified whether a library component that has a swappable requirement includes the header `<utility>` to ensure an appropriate evaluation context. — *end note*]

4 An rvalue or lvalue `t` is *swappable* if and only if `t` is swappable with any rvalue or lvalue, respectively, of type `T`.

5 A type `X` satisfying any of the iterator requirements (24.2) satisfies the requirements of `ValueSwappable` if, for any dereferenceable object `x` of type `X`, `*x` is swappable.

[*Example:* User code can ensure that the evaluation of `swap` calls is performed in an appropriate context under the various conditions as follows:

```
#include <utility>

// Requires: std::forward<T>(t) shall be swappable with std::forward<U>(u).
template <class T, class U>
void value_swap(T&& t, U&& u) {
  using std::swap;
  swap(std::forward<T>(t), std::forward<U>(u)); // OK: uses "swappable with" conditions
                                                // for rvalues and lvalues
}

// Requires: lvalues of T shall be swappable.
template <class T>
void lv_swap(T& t1, T& t2) {
  using std::swap;
  swap(t1, t2);                                 // OK: uses swappable conditions for
}                                               // lvalues of type T

namespace N {
  struct A { int m; };
  struct Proxy { A* a; };
  Proxy proxy(A& a) { return Proxy{ &a }; }

  void swap(A& x, Proxy p) {
    std::swap(x.m, p.a->m);                     // OK: uses context equivalent to swappable
                                                // conditions for fundamental types
  }
  void swap(Proxy p, A& x) { swap(x, p); }      // satisfy symmetry constraint
}

int main() {
  int i = 1, j = 2;
  lv_swap(i, j);
  assert(i == 2 && j == 1);

  N::A a1 = { 5 }, a2 = { -5 };
  value_swap(a1, proxy(a2));
  assert(a1.m == -5 && a2.m == 5);
}
```

— *end example*]

17.6.3.3 `NullablePointer` requirements [nullablepointer.requirements]

1 A `NullablePointer` type is a pointer-like type that supports null values. A type `P` meets the requirements of `NullablePointer` if:

(1.1) — `P` satisfies the requirements of `EqualityComparable`, `DefaultConstructible`, `CopyConstructible`, `CopyAssignable`, and `Destructible`,

(1.2) — lvalues of type `P` are swappable (17.6.3.2),

(1.3) — the expressions shown in Table 25 are valid and have the indicated semantics, and

(1.4) — `P` satisfies all the other requirements of this subclause.

2 A value-initialized object of type `P` produces the null value of the type. The null value shall be equivalent only to itself. A default-initialized object of type `P` may have an indeterminate value. [*Note:* Operations involving indeterminate values may cause undefined behavior. — *end note*]

3 An object `p` of type `P` can be contextually converted to `bool` (Clause 4). The effect shall be as if `p != nullptr` had been evaluated in place of `p`.

4 No operation which is part of the `NullablePointer` requirements shall exit via an exception.

5 In Table 25, `u` denotes an identifier, `t` denotes a non-`const` lvalue of type `P`, `a` and `b` denote values of type (possibly `const`) `P`, and `np` denotes a value of type (possibly `const`) `std::nullptr_t`.

Table 25 — `NullablePointer` requirements **[nullablepointer]**

Expression	Return type	Operational semantics
`P u(np);` `P u = np;`		post: `u == nullptr`
`P(np)`		post: `P(np) == nullptr`
`t = np`	`P&`	post: `t == nullptr`
`a != b`	contextually convertible to `bool`	`!(a == b)`
`a == np` `np == a`	contextually convertible to `bool`	`a == P()`
`a != np` `np != a`	contextually convertible to `bool`	`!(a == np)`

17.6.3.4 Hash requirements [hash.requirements]

1 A type `H` meets the `Hash` requirements if:

(1.1) — it is a function object type (20.9),

(1.2) — it satisfies the requirements of `CopyConstructible` and `Destructible` (17.6.3.1), and

(1.3) — the expressions shown in Table 26 are valid and have the indicated semantics.

2 Given `Key` is an argument type for function objects of type `H`, in Table 26 `h` is a value of type (possibly `const`) `H`, `u` is an lvalue of type `Key`, and `k` is a value of a type convertible to (possibly `const`) `Key`.

Table 26 — `Hash` requirements **[hash]**

Expression	Return type	Requirement
`h(k)`	`size_t`	The value returned shall depend only on the argument `k` for the duration of the program. [*Note:* Thus all evaluations of the expression `h(k)` with the same value for `k` yield the same result for a given execution of the program. — *end note*] [*Note:* For two different values `t1` and `t2`, the probability that `h(t1)` and `h(t2)` compare equal should be very small, approaching `1.0 / numeric_limits<size_t>::max()`. — *end note*]
`h(u)`	`size_t`	Shall not modify `u`.

17.6.3.5 Allocator requirements [allocator.requirements]

1 The library describes a standard set of requirements for *allocators*, which are class-type objects that encapsulate the information about an allocation model. This information includes the knowledge of pointer types, the type of their difference, the type of the size of objects in this allocation model, as well as the memory allocation and deallocation primitives for it. All of the string types (Clause 21), containers (Clause 23) (except array), string buffers and string streams (Clause 27), and `match_results` (Clause 28) are parameterized in terms of allocators.

[2] The class template `allocator_traits` (20.7.8) supplies a uniform interface to all allocator types. Table 27 describes the types manipulated through allocators. Table 28 describes the requirements on allocator types and thus on types used to instantiate `allocator_traits`. A requirement is optional if the last column of Table 28 specifies a default for a given expression. Within the standard library `allocator_traits` template, an optional requirement that is not supplied by an allocator is replaced by the specified default expression. A user specialization of `allocator_traits` may provide different defaults and may provide defaults for different requirements than the primary template. Within Tables 27 and 28, the use of `move` and `forward` always refers to `std::move` and `std::forward`, respectively.

Table 27 — Descriptive variable definitions

Variable	Definition
`T, U, C`	any non-const object type (3.9)
`X`	an Allocator class for type `T`
`Y`	the corresponding Allocator class for type `U`
`XX`	the type `allocator_traits<X>`
`YY`	the type `allocator_traits<Y>`
`a, a1, a2`	lvalues of type `X`
`u`	the name of a variable being declared
`b`	a value of type `Y`
`c`	a pointer of type `C*` through which indirection is valid
`p`	a value of type `XX::pointer`, obtained by calling `a1.allocate`, where `a1 == a`
`q`	a value of type `XX::const_pointer` obtained by conversion from a value `p`.
`w`	a value of type `XX::void_pointer` obtained by conversion from a value `p`
`x`	a value of type `XX::const_void_pointer` obtained by conversion from a value `q` or a value `w`
`y`	a value of type `XX:const_void_pointer` obtained by conversion from a result value of `YY::allocate`, or else a value of type (possibly `const`) `std::nullptr_t`.
`n`	a value of type `XX::size_type`.
`Args`	a template parameter pack
`args`	a function parameter pack with the pattern `Args&&`

Table 28 — Allocator requirements

Expression	Return type	Assertion/note pre-/post-condition	Default
`X::pointer`			`T*`
`X::const_pointer`		`X::pointer` is convertible to `X::const_pointer`	`pointer_traits<X::pointer>::rebind<const T>`

Table 28 — Allocator requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Default
`X::void_pointer` `Y::void_pointer`		`X::pointer` is convertible to `X::void_pointer`. `X::void_pointer` and `Y::void_pointer` are the same type.	`pointer_traits<X::pointer>::rebind<void>`
`X::const_void_pointer` `Y::const_void_pointer`		`X::pointer`, `X::const_pointer`, and `X::void_pointer` are convertible to `X::const_void_pointer`. `X::const_void_pointer` and `Y::const_void_pointer` are the same type.	`pointer_traits<X::pointer>::rebind<const void>`
`X::value_type`	Identical to `T`		
`X::size_type`	unsigned integer type	a type that can represent the size of the largest object in the allocation model.	`make_unsigned_t<X::difference_type>`
`X::difference_type`	signed integer type	a type that can represent the difference between any two pointers in the allocation model.	`pointer_traits<X::pointer>::difference_type`
`typename X::template rebind<U>::other`	`Y`	For all `U` (including `T`), `Y::template rebind<T>::other` is `X`.	See Note A, below.
`*p`	`T&`		
`*q`	`const T&`	`*q` refers to the same object as `*p`	
`p->m`	type of `T::m`	*pre:* `(*p).m` is well-defined. equivalent to `(*p).m`	
`q->m`	type of `T::m`	*pre:* `(*q).m` is well-defined. equivalent to `(*q).m`	
`static_cast<X::pointer>(w)`	`X::pointer`	`static_cast<X::pointer>(w) == p`	
`static_cast<X ::const_pointer>(x)`	`X::const_pointer`	`static_cast<X ::const_pointer>(x) == q`	
`a.allocate(n)`	`X::pointer`	Memory is allocated for `n` objects of type `T` but objects are not constructed. `allocate` may raise an appropriate exception.[181] [*Note:* If `n == 0`, the return value is unspecified. — *end note*]	

Table 28 — Allocator requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Default
`a.allocate(n, y)`	`X::pointer`	Same as `a.allocate(n)`. The use of `y` is unspecified, but it is intended as an aid to locality.	`a.allocate(n)`
`a.deallocate(p,n)`	(not used)	All `n` `T` objects in the area pointed to by `p` shall be destroyed prior to this call. `n` shall match the value passed to `allocate` to obtain this memory. Does not throw exceptions. [*Note:* `p` shall not be singular. — *end note*]	
`a.max_size()`	`X::size_type`	the largest value that can meaningfully be passed to `X::allocate()`	`numeric_limits<size_type>::max()`
`a1 == a2`	`bool`	returns `true` only if storage allocated from each can be deallocated via the other. `operator==` shall be reflexive, symmetric, and transitive, and shall not exit via an exception.	
`a1 != a2`	`bool`	same as `!(a1 == a2)`	
`a == b`	`bool`	same as `a == Y::rebind<T>::other(b)`	
`a != b`	`bool`	same as `!(a == b)`	
`X u(a);` `X u = a;`		Shall not exit via an exception. post: `a1 == a`	
`X u(b);`		Shall not exit via an exception. post: `Y(a) == b`, `a == X(b)`	
`X u(move(a));` `X u = move(a);`		Shall not exit via an exception. post: `a1` equals the prior value of `a`.	
`X u(move(b));`		Shall not exit via an exception. post: `a` equals the prior value of `X(b)`.	
`a.construct(c, args)`	(not used)	Effect: Constructs an object of type `C` at `c`	`::new ((void*)c) C(forward<Args> (args)...)`
`a.destroy(c)`	(not used)	Effect: Destroys the object at `c`	`c->~C()`
`a.select_on_container_copy_construction()`	`X`	Typically returns either `a` or `X()`	`return a;`

Table 28 — Allocator requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Default
`X::propagate_on_container_copy_assignment`	Identical to or derived from `true_type` or `false_type`	`true_type` only if an allocator of type `X` should be copied when the client container is copy-assigned. See Note B, below.	`false_type`
`X::propagate_on_container_move_assignment`	Identical to or derived from `true_type` or `false_type`	`true_type` only if an allocator of type `X` should be moved when the client container is move-assigned. See Note B, below.	`false_type`
`X::propagate_on_container_swap`	Identical to or derived from `true_type` or `false_type`	`true_type` only if an allocator of type `X` should be swapped when the client container is swapped. See Note B, below.	`false_type`
`X::is_always_equal`	Identical to or derived from `true_type` or `false_type`	`true_type` only if the expression `a1 == a2` is guaranteed to be true for any two (possibly `const`) values `a1`, `a2` of type `X`.	`is_empty<X>::type`

3 Note A: The member class template `rebind` in the table above is effectively a typedef template. [*Note:* In general, if the name `Allocator` is bound to `SomeAllocator<T>`, then `Allocator::rebind<U>::other` is the same type as `SomeAllocator<U>`, where `SomeAllocator<T>::value_type` is `T` and `SomeAllocator<U>::value_type` is `U`. — *end note*] If `Allocator` is a class template instantiation of the form `SomeAllocator<T, Args>`, where `Args` is zero or more type arguments, and `Allocator` does not supply a `rebind` member template, the standard `allocator_traits` template uses `SomeAllocator<U, Args>` in place of `Allocator::rebind<U>::other` by default. For allocator types that are not template instantiations of the above form, no default is provided.

4 Note B: If `X::propagate_on_container_copy_assignment::value` is `true`, `X` shall satisfy the `CopyAssignable` requirements (Table 23) and the copy operation shall not throw exceptions. If `X::propagate_on_container_move_assignment::value` is `true`, `X` shall satisfy the `MoveAssignable` requirements (Table 22) and the move operation shall not throw exceptions. If `X::propagate_on_container_swap::value` is `true`, lvalues of type `X` shall be swappable (17.6.3.2) and the `swap` operation shall not throw exceptions.

5 An allocator type `X` shall satisfy the requirements of `CopyConstructible` (17.6.3.1). The `X::pointer`, `X::const_pointer`, `X::void_pointer`, and `X::const_void_pointer` types shall satisfy the requirements of `NullablePointer` (17.6.3.3). No constructor, comparison operator, copy operation, move operation, or swap operation on these pointer types shall exit via an exception. `X::pointer` and `X::const_pointer` shall also satisfy the requirements for a random access iterator (24.2).

6 Let `x1` and `x2` denote objects of (possibly different) types `X::void_pointer`, `X::const_void_pointer`, `X::pointer`, or `X::const_pointer`. Then, `x1` and `x2` are *equivalently-valued* pointer values, if and only if both `x1` and `x2` can be explicitly converted to the two corresponding objects `px1` and `px2` of type `X::const_pointer`, using a sequence of `static_cast`s using only these four types, and the expression `px1 == px2` evaluates to `true`.

181) It is intended that `a.allocate` be an efficient means of allocating a single object of type `T`, even when `sizeof(T)` is small. That is, there is no need for a container to maintain its own free list.

7 Let `w1` and `w2` denote objects of type `X::void_pointer`. Then for the expressions

```
w1 == w2
w1 != w2
```

either or both objects may be replaced by an equivalently-valued object of type `X::const_void_pointer` with no change in semantics.

8 Let `p1` and `p2` denote objects of type `X::pointer`. Then for the expressions

```
p1 == p2
p1 != p2
p1 < p2
p1 <= p2
p1 >= p2
p1 > p2
p1 - p2
```

either or both objects may be replaced by an equivalently-valued object of type `X::const_pointer` with no change in semantics.

9 An allocator may constrain the types on which it can be instantiated and the arguments for which its `construct` or `destroy` members may be called. If a type cannot be used with a particular allocator, the allocator class or the call to `construct` or `destroy` may fail to instantiate.

[*Example:* the following is an allocator class template supporting the minimal interface that satisfies the requirements of Table 28:

```
template <class Tp>
struct SimpleAllocator {
  typedef Tp value_type;
  SimpleAllocator(ctor args);

  template <class T> SimpleAllocator(const SimpleAllocator<T>& other);

  Tp* allocate(std::size_t n);
  void deallocate(Tp* p, std::size_t n);
};

template <class T, class U>
bool operator==(const SimpleAllocator<T>&, const SimpleAllocator<U>&);
template <class T, class U>
bool operator!=(const SimpleAllocator<T>&, const SimpleAllocator<U>&);
```

— *end example*]

10 If the alignment associated with a specific over-aligned type is not supported by an allocator, instantiation of the allocator for that type may fail. The allocator also may silently ignore the requested alignment. [*Note:* Additionally, the member function `allocate` for that type may fail by throwing an object of type `std::bad_alloc`. — *end note*]

17.6.3.5.1 Allocator completeness requirements [allocator.requirements.completeness]

1 If `X` is an allocator class for type `T`, `X` additionally satisfies the allocator completeness requirements if, whether or not `T` is a complete type:

(1.1) — `X` is a complete type, and

(1.2) — all the member types of `allocator_traits<X>` 20.7.8 other than `value_type` are complete types.

17.6.4 Constraints on programs [constraints]

17.6.4.1 Overview [constraints.overview]

1 This section describes restrictions on C++ programs that use the facilities of the C++ standard library. The following subclauses specify constraints on the program's use of namespaces (17.6.4.2.1), its use of various reserved names (17.6.4.3), its use of headers (17.6.4.4), its use of standard library classes as base classes (17.6.4.5), its definitions of replacement functions (17.6.4.6), and its installation of handler functions during execution (17.6.4.7).

17.6.4.2 Namespace use [namespace.constraints]

17.6.4.2.1 Namespace `std` [namespace.std]

1 The behavior of a C++ program is undefined if it adds declarations or definitions to namespace `std` or to a namespace within namespace `std` unless otherwise specified. A program may add a template specialization for any standard library template to namespace `std` only if the declaration depends on a user-defined type and the specialization meets the standard library requirements for the original template and is not explicitly prohibited.[182]

2 The behavior of a C++ program is undefined if it declares

(2.1) — an explicit specialization of any member function of a standard library class template, or

(2.2) — an explicit specialization of any member function template of a standard library class or class template, or

(2.3) — an explicit or partial specialization of any member class template of a standard library class or class template.

A program may explicitly instantiate a template defined in the standard library only if the declaration depends on the name of a user-defined type and the instantiation meets the standard library requirements for the original template.

3 A translation unit shall not declare namespace `std` to be an inline namespace (7.3.1).

17.6.4.2.2 Namespace `posix` [namespace.posix]

1 The behavior of a C++ program is undefined if it adds declarations or definitions to namespace `posix` or to a namespace within namespace `posix` unless otherwise specified. The namespace `posix` is reserved for use by ISO/IEC 9945 and other POSIX standards.

17.6.4.3 Reserved names [reserved.names]

1 The C++ standard library reserves the following kinds of names:

(1.1) — macros

(1.2) — global names

(1.3) — names with external linkage

2 If a program declares or defines a name in a context where it is reserved, other than as explicitly allowed by this Clause, its behavior is undefined.

182) Any library code that instantiates other library templates must be prepared to work adequately with any user-supplied specialization that meets the minimum requirements of the Standard.

17.6.4.3.1 Macro names [macro.names]

1 A translation unit that includes a standard library header shall not `#define` or `#undef` names declared in any standard library header.

2 A translation unit shall not `#define` or `#undef` names lexically identical to keywords, to the identifiers listed in Table 2, or to the *attribute-token*s described in 7.6.

17.6.4.3.2 External linkage [extern.names]

1 Each name declared as an object with external linkage in a header is reserved to the implementation to designate that library object with external linkage,[183] both in namespace `std` and in the global namespace.

2 Each global function signature declared with external linkage in a header is reserved to the implementation to designate that function signature with external linkage. [184]

3 Each name from the Standard C library declared with external linkage is reserved to the implementation for use as a name with `extern "C"` linkage, both in namespace std and in the global namespace.

4 Each function signature from the Standard C library declared with external linkage is reserved to the implementation for use as a function signature with both `extern "C"` and `extern "C++"` linkage, [185] or as a name of namespace scope in the global namespace.

17.6.4.3.3 Types [extern.types]

1 For each type T from the Standard C library,[186] the types `::T` and `std::T` are reserved to the implementation and, when defined, `::T` shall be identical to `std::T`.

17.6.4.3.4 User-defined literal suffixes [usrlit.suffix]

1 Literal suffix identifiers (13.5.8) that do not start with an underscore are reserved for future standardization.

17.6.4.4 Headers [alt.headers]

1 If a file with a name equivalent to the derived file name for one of the C++ standard library headers is not provided as part of the implementation, and a file with that name is placed in any of the standard places for a source file to be included (16.2), the behavior is undefined.

17.6.4.5 Derived classes [derived.classes]

1 Virtual member function signatures defined for a base class in the C++ standard library may be overridden in a derived class defined in the program (10.3).

17.6.4.6 Replacement functions [replacement.functions]

1 Clauses 18 through 30 and Annex D describe the behavior of numerous functions defined by the C++ standard library. Under some circumstances, however, certain of these function descriptions also apply to replacement functions defined in the program (17.3).

2 A C++ program may provide the definition for any of twelve dynamic memory allocation function signatures declared in header `<new>` (3.7.4, 18.6):

(2.1) — `operator new(std::size_t)`

(2.2) — `operator new(std::size_t, const std::nothrow_t&)`

183) The list of such reserved names includes `errno`, declared or defined in `<cerrno>`.
184) The list of such reserved function signatures with external linkage includes `setjmp(jmp_buf)`, declared or defined in `<csetjmp>`, and `va_end(va_list)`, declared or defined in `<cstdarg>`.
185) The function signatures declared in `<cuchar>`, `<cwchar>`, and `<cwctype>` are always reserved, notwithstanding the restrictions imposed in subclause 4.5.1 of Amendment 1 to the C Standard for these headers.
186) These types are `clock_t`, `div_t`, `FILE`, `fpos_t`, `lconv`, `ldiv_t`, `mbstate_t`, `ptrdiff_t`, `sig_atomic_t`, `size_t`, `time_t`, `tm`, `va_list`, `wctrans_t`, `wctype_t`, and `wint_t`.

(2.3) — `operator new[](std::size_t)`

(2.4) — `operator new[](std::size_t, const std::nothrow_t&)`

(2.5) — `operator delete(void*)`

(2.6) — `operator delete(void*, const std::nothrow_t&)`

(2.7) — `operator delete[](void*)`

(2.8) — `operator delete[](void*, const std::nothrow_t&)`

(2.9) — `operator delete(void*, std::size_t)`

(2.10) — `operator delete(void*, std::size_t, const std::nothrow_t&)`

(2.11) — `operator delete[](void*, std::size_t)`

(2.12) — `operator delete[](void*, std::size_t, const std::nothrow_t&)`

3 The program's definitions are used instead of the default versions supplied by the implementation (18.6). Such replacement occurs prior to program startup (3.2, 3.6). The program's declarations shall not be specified as `inline`. No diagnostic is required.

17.6.4.7 Handler functions [handler.functions]

1 The C++ standard library provides default versions of the following handler functions (Clause 18):

(1.1) — `unexpected_handler`

(1.2) — `terminate_handler`

2 A C++ program may install different handler functions during execution, by supplying a pointer to a function defined in the program or the library as an argument to (respectively):

(2.1) — `set_new_handler`

(2.2) — `set_unexpected`

(2.3) — `set_terminate`

See also: subclauses 18.6.2, Storage allocation errors, and 18.8, Exception handling.

3 A C++ program can get a pointer to the current handler function by calling the following functions:

(3.1) — `get_new_handler`

(3.2) — `get_unexpected`

(3.3) — `get_terminate`

4 Calling the `set_*` and `get_*` functions shall not incur a data race. A call to any of the `set_*` functions shall synchronize with subsequent calls to the same `set_*` function and to the corresponding `get_*` function.

17.6.4.8 Other functions [res.on.functions]

1 In certain cases (replacement functions, handler functions, operations on types used to instantiate standard library template components), the C++ standard library depends on components supplied by a C++ program. If these components do not meet their requirements, the Standard places no requirements on the implementation.

2 In particular, the effects are undefined in the following cases:

(2.1) — for replacement functions (18.6.1), if the installed replacement function does not implement the semantics of the applicable *Required behavior:* paragraph.

(2.2) — for handler functions (18.6.2.3, 18.8.3.1, D.8.1), if the installed handler function does not implement the semantics of the applicable *Required behavior:* paragraph

(2.3) — for types used as template arguments when instantiating a template component, if the operations on the type do not implement the semantics of the applicable **Requirements** subclause (17.6.3.5, 23.2, 24.2, 26.2). Operations on such types can report a failure by throwing an exception unless otherwise specified.

(2.4) — if any replacement function or handler function or destructor operation exits via an exception, unless specifically allowed in the applicable *Required behavior:* paragraph.

(2.5) — if an incomplete type (3.9) is used as a template argument when instantiating a template component, unless specifically allowed for that component.

17.6.4.9 Function arguments [res.on.arguments]

1 Each of the following applies to all arguments to functions defined in the C++ standard library, unless explicitly stated otherwise.

(1.1) — If an argument to a function has an invalid value (such as a value outside the domain of the function or a pointer invalid for its intended use), the behavior is undefined.

(1.2) — If a function argument is described as being an array, the pointer actually passed to the function shall have a value such that all address computations and accesses to objects (that would be valid if the pointer did point to the first element of such an array) are in fact valid.

(1.3) — If a function argument binds to an rvalue reference parameter, the implementation may assume that this parameter is a unique reference to this argument. [*Note:* If the parameter is a generic parameter of the form `T&&` and an lvalue of type `A` is bound, the argument binds to an lvalue reference (14.8.2.1) and thus is not covered by the previous sentence. — *end note*] [*Note:* If a program casts an lvalue to an xvalue while passing that lvalue to a library function (e.g. by calling the function with the argument `move(x)`), the program is effectively asking that function to treat that lvalue as a temporary. The implementation is free to optimize away aliasing checks which might be needed if the argument was an lvalue. — *end note*]

17.6.4.10 Shared objects and the library [res.on.objects]

1 The behavior of a program is undefined if calls to standard library functions from different threads may introduce a data race. The conditions under which this may occur are specified in 17.6.5.9. [*Note:* Modifying an object of a standard library type that is shared between threads risks undefined behavior unless objects of that type are explicitly specified as being sharable without data races or the user supplies a locking mechanism. — *end note*]

2 [*Note:* In particular, the program is required to ensure that completion of the constructor of any object of a class type defined in the standard library happens before any other member function invocation on that object and, unless otherwise specified, to ensure that completion of any member function invocation other

than destruction on such an object happens before destruction of that object. This applies even to objects such as mutexes intended for thread synchronization. — *end note*]

17.6.4.11 Requires paragraph [res.on.required]

1 Violation of the preconditions specified in a function's *Requires:* paragraph results in undefined behavior unless the function's *Throws:* paragraph specifies throwing an exception when the precondition is violated.

17.6.5 Conforming implementations [conforming]

17.6.5.1 Overview [conforming.overview]

1 This section describes the constraints upon, and latitude of, implementations of the C++ standard library.

2 An implementation's use of headers is discussed in 17.6.5.2, its use of macros in 17.6.5.3, global functions in 17.6.5.4, member functions in 17.6.5.5, data race avoidance in 17.6.5.9, access specifiers in 17.6.5.10, class derivation in 17.6.5.11, and exceptions in 17.6.5.12.

17.6.5.2 Headers [res.on.headers]

1 A C++ header may include other C++ headers. A C++ header shall provide the declarations and definitions that appear in its synopsis. A C++ header shown in its synopsis as including other C++ headers shall provide the declarations and definitions that appear in the synopses of those other headers.

2 Certain types and macros are defined in more than one header. Every such entity shall be defined such that any header that defines it may be included after any other header that also defines it (3.2).

3 The C standard headers (D.5) shall include only their corresponding C++ standard header, as described in 17.6.1.2.

17.6.5.3 Restrictions on macro definitions [res.on.macro.definitions]

1 The names and global function signatures described in 17.6.1.1 are reserved to the implementation.

2 All object-like macros defined by the C standard library and described in this Clause as expanding to integral constant expressions are also suitable for use in `#if` preprocessing directives, unless explicitly stated otherwise.

17.6.5.4 Global and non-member functions [global.functions]

1 It is unspecified whether any global or non-member functions in the C++ standard library are defined as `inline` (7.1.2).

2 A call to a global or non-member function signature described in Clauses 18 through 30 and Annex D shall behave as if the implementation declared no additional global or non-member function signatures.[187]

3 An implementation shall not declare a global or non-member function signature with additional default arguments.

4 Unless otherwise specified, global and non-member functions in the standard library shall not use functions from another namespace which are found through *argument-dependent name lookup* (3.4.2). [*Note:* The phrase "unless otherwise specified" is intended to allow argument-dependent lookup in cases like that of `ostream_iterator::operator=` (24.6.2.2):

Effects:

```
*out_stream << value;
if (delim != 0)
  *out_stream << delim;
return *this;
```

187) A valid C++ program always calls the expected library global or non-member function. An implementation may also define additional global or non-member functions that would otherwise not be called by a valid C++ program.

— *end note*]

17.6.5.5 Member functions [member.functions]

1 It is unspecified whether any member functions in the C++ standard library are defined as `inline` (7.1.2).

2 An implementation may declare additional non-virtual member function signatures within a class:

(2.1) — by adding arguments with default values to a member function signature;[188] [*Note:* An implementation may not add arguments with default values to virtual, global, or non-member functions. — *end note*]

(2.2) — by replacing a member function signature with default values by two or more member function signatures with equivalent behavior; and

(2.3) — by adding a member function signature for a member function name.

3 A call to a member function signature described in the C++ standard library behaves as if the implementation declares no additional member function signatures.[189]

17.6.5.6 `constexpr` functions and constructors [constexpr.functions]

1 This standard explicitly requires that certain standard library functions are `constexpr` (7.1.5). An implementation shall not declare any standard library function signature as `constexpr` except for those where it is explicitly required. Within any header that provides any non-defining declarations of `constexpr` functions or constructors an implementation shall provide corresponding definitions.

17.6.5.7 Requirements for stable algorithms [algorithm.stable]

1 When the requirements for an algorithm state that it is "stable" without further elaboration, it means:

(1.1) — For the *sort* algorithms the relative order of equivalent elements is preserved.

(1.2) — For the *remove* and *copy* algorithms the relative order of the elements that are not removed is preserved.

(1.3) — For the *merge* algorithms, for equivalent elements in the original two ranges, the elements from the first range (preserving their original order) precede the elements from the second range (preserving their original order).

17.6.5.8 Reentrancy [reentrancy]

1 Except where explicitly specified in this standard, it is implementation-defined which functions in the Standard C++ library may be recursively reentered.

17.6.5.9 Data race avoidance [res.on.data.races]

1 This section specifies requirements that implementations shall meet to prevent data races (1.10). Every standard library function shall meet each requirement unless otherwise specified. Implementations may prevent data races in cases other than those specified below.

2 A C++ standard library function shall not directly or indirectly access objects (1.10) accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's arguments, including `this`.

3 A C++ standard library function shall not directly or indirectly modify objects (1.10) accessible by threads other than the current thread unless the objects are accessed directly or indirectly via the function's non-const arguments, including `this`.

188) Hence, the address of a member function of a class in the C++ standard library has an unspecified type.
189) A valid C++ program always calls the expected library member function, or one with equivalent behavior. An implementation may also define additional member functions that would otherwise not be called by a valid C++ program.

4 [*Note:* This means, for example, that implementations can't use a static object for internal purposes without synchronization because it could cause a data race even in programs that do not explicitly share objects between threads. — *end note*]

5 A C++ standard library function shall not access objects indirectly accessible via its arguments or via elements of its container arguments except by invoking functions required by its specification on those container elements.

6 Operations on iterators obtained by calling a standard library container or string member function may access the underlying container, but shall not modify it. [*Note:* In particular, container operations that invalidate iterators conflict with operations on iterators associated with that container. — *end note*]

7 Implementations may share their own internal objects between threads if the objects are not visible to users and are protected against data races.

8 Unless otherwise specified, C++ standard library functions shall perform all operations solely within the current thread if those operations have effects that are visible (1.10) to users.

9 [*Note:* This allows implementations to parallelize operations if there are no visible side effects. — *end note*]

17.6.5.10 Protection within classes [protection.within.classes]

1 It is unspecified whether any function signature or class described in Clauses 18 through 30 and Annex D is a `friend` of another class in the C++ standard library.

17.6.5.11 Derived classes [derivation]

1 An implementation may derive any class in the C++ standard library from a class with a name reserved to the implementation.

2 Certain classes defined in the C++ standard library are required to be derived from other classes in the C++ standard library. An implementation may derive such a class directly from the required base or indirectly through a hierarchy of base classes with names reserved to the implementation.

3 In any case:

(3.1) — Every base class described as `virtual` shall be virtual;

(3.2) — Every base class described as non-`virtual` shall not be virtual;

(3.3) — Unless explicitly stated otherwise, types with distinct names shall be distinct types.[190]

17.6.5.12 Restrictions on exception handling [res.on.exception.handling]

1 Any of the functions defined in the C++ standard library can report a failure by throwing an exception of a type described in its **Throws:** paragraph. An implementation may strengthen the exception specification for a non-virtual function by adding a non-throwing *noexcept-specification*.

2 A function may throw an object of a type not listed in its **Throws** clause if its type is derived from a type named in the **Throws** clause and would be caught by an exception handler for the base type.

3 Functions from the C standard library shall not throw exceptions[191] except when such a function calls a program-supplied function that throws an exception.[192]

4 Destructor operations defined in the C++ standard library shall not throw exceptions. Every destructor in the C++ standard library shall behave as if it had a non-throwing exception specification. Any other functions

190) There is an implicit exception to this rule for types that are described as synonyms for basic integral types, such as `size_t` (18.2) and `streamoff` (27.5.2).
191) That is, the C library functions can all be treated as if they are marked `noexcept`. This allows implementations to make performance optimizations based on the absence of exceptions at runtime.
192) The functions `qsort()` and `bsearch()` (25.5) meet this condition.

defined in the C++ standard library that do not have an *exception-specification* may throw implementation-defined exceptions unless otherwise specified.[193] An implementation may strengthen this implicit *exception-specification* by adding an explicit one.[194]

17.6.5.13 Restrictions on storage of pointers [res.on.pointer.storage]

1 Objects constructed by the standard library that may hold a user-supplied pointer value or an integer of type `std::intptr_t` shall store such values in a traceable pointer location (3.7.4.3). [*Note:* Other libraries are strongly encouraged to do the same, since not doing so may result in accidental use of pointers that are not safely derived. Libraries that store pointers outside the user's address space should make it appear that they are stored and retrieved from a traceable pointer location. — *end note*]

17.6.5.14 Value of error codes [value.error.codes]

1 Certain functions in the C++ standard library report errors via a `std::error_code` (19.5.2.1) object. That object's `category()` member shall return `std::system_category()` for errors originating from the operating system, or a reference to an implementation-defined `error_category` object for errors originating elsewhere. The implementation shall define the possible values of `value()` for each of these error categories. [*Example:* For operating systems that are based on POSIX, implementations are encouraged to define the `std::system_category()` values as identical to the POSIX `errno` values, with additional values as defined by the operating system's documentation. Implementations for operating systems that are not based on POSIX are encouraged to define values identical to the operating system's values. For errors that do not originate from the operating system, the implementation may provide enums for the associated values. — *end example*]

17.6.5.15 Moved-from state of library types [lib.types.movedfrom]

1 Objects of types defined in the C++ standard library may be moved from (12.8). Move operations may be explicitly specified or implicitly generated. Unless otherwise specified, such moved-from objects shall be placed in a valid but unspecified state.

193) In particular, they can report a failure to allocate storage by throwing an exception of type `bad_alloc`, or a class derived from `bad_alloc` (18.6.2.1). Library implementations should report errors by throwing exceptions of or derived from the standard exception classes (18.6.2.1, 18.8, 19.2).

194) That is, an implementation may provide an explicit *exception-specification* that defines the subset of "any" exceptions thrown by that function. This implies that the implementation may list implementation-defined types in such an *exception-specification*.

18 Language support library [language.support]

18.1 General [support.general]

1 This Clause describes the function signatures that are called implicitly, and the types of objects generated implicitly, during the execution of some C++ programs. It also describes the headers that declare these function signatures and define any related types.

2 The following subclauses describe common type definitions used throughout the library, characteristics of the predefined types, functions supporting start and termination of a C++ program, support for dynamic memory management, support for dynamic type identification, support for exception processing, support for initializer lists, and other runtime support, as summarized in Table 29.

Table 29 — Language support library summary

Subclause		Header(s)
18.2	Types	`<cstddef>`

18.3	Implementation properties	`<limits>`
		`<climits>`
		`<cfloat>`

18.4	Integer types	`<cstdint>`
18.5	Start and termination	`<cstdlib>`
18.6	Dynamic memory management	`<new>`
18.7	Type identification	`<typeinfo>`
18.8	Exception handling	`<exception>`
18.9	Initializer lists	`<initializer_list>`

18.10	Other runtime support	`<csignal>`
		`<csetjmp>`
		`<cstdalign>`
		`<cstdarg>`
		`<cstdbool>`
		`<cstdlib>`
		`<ctime>`

18.2 Types [support.types]

1 Table 30 describes the header `<cstddef>`.

Table 30 — Header `<cstddef>` synopsis

Type	Name(s)	
Macros:	`NULL`	`offsetof`
Types:	`ptrdiff_t`	`size_t`
	`max_align_t`	`nullptr_t`

2 The contents are the same as the Standard C library header `<stddef.h>`, with the following changes:

3 The macro `NULL` is an implementation-defined C++ null pointer constant in this International Standard (4.10).[195]

4 The macro `offsetof`(*type*, *member-designator*) accepts a restricted set of *type* arguments in this International Standard. If *type* is not a standard-layout class (Clause 9), the results are undefined.[196] The expression `offsetof`(*type*, *member-designator*) is never type-dependent (14.6.2.2) and it is value-dependent (14.6.2.3) if and only if *type* is dependent. The result of applying the `offsetof` macro to a field that is a static data member or a function member is undefined. No operation invoked by the `offsetof` macro shall throw an exception and `noexcept(offsetof(type, member-designator))` shall be `true`.

5 The type `ptrdiff_t` is an implementation-defined signed integer type that can hold the difference of two subscripts in an array object, as described in 5.7.

6 The type `size_t` is an implementation-defined unsigned integer type that is large enough to contain the size in bytes of any object.

7 [*Note:* It is recommended that implementations choose types for `ptrdiff_t` and `size_t` whose integer conversion ranks (4.13) are no greater than that of `signed long int` unless a larger size is necessary to contain all the possible values. — *end note*]

8 The type `max_align_t` is a POD type whose alignment requirement is at least as great as that of every scalar type, and whose alignment requirement is supported in every context.

9 `nullptr_t` is defined as follows:

```
namespace std {
  typedef decltype(nullptr) nullptr_t;
}
```

The type for which `nullptr_t` is a synonym has the characteristics described in 3.9.1 and 4.10. [*Note:* Although `nullptr`'s address cannot be taken, the address of another `nullptr_t` object that is an lvalue can be taken. — *end note*]

See also: Alignment (3.11), Sizeof (5.3.3), Additive operators (5.7), Free store (12.5), and ISO C 7.1.6.

18.3 Implementation properties [support.limits]

18.3.1 In general [support.limits.general]

1 The headers `<limits>` (18.3.2), `<climits>`, and `<cfloat>` (18.3.3) supply characteristics of implementation-dependent arithmetic types (3.9.1).

18.3.2 Numeric limits [limits]

18.3.2.1 Class template `numeric_limits` [limits.numeric]

1 The `numeric_limits` class template provides a C++ program with information about various properties of the implementation's representation of the arithmetic types.

2 Specializations shall be provided for each arithmetic type, both floating point and integer, including `bool`. The member `is_specialized` shall be `true` for all such specializations of `numeric_limits`.

3 For all members declared `static constexpr` in the `numeric_limits` template, specializations shall define these values in such a way that they are usable as constant expressions.

4 Non-arithmetic standard types, such as `complex<T>` (26.4.2), shall not have specializations.

18.3.2.2 Header `<limits>` synopsis [limits.syn]

195) Possible definitions include `0` and `0L`, but not `(void*)0`.

196) Note that `offsetof` is required to work as specified even if unary `operator&` is overloaded for any of the types involved.

```
namespace std {
  template<class T> class numeric_limits;
  enum float_round_style;
  enum float_denorm_style;

  template<> class numeric_limits<bool>;

  template<> class numeric_limits<char>;
  template<> class numeric_limits<signed char>;
  template<> class numeric_limits<unsigned char>;
  template<> class numeric_limits<char16_t>;
  template<> class numeric_limits<char32_t>;
  template<> class numeric_limits<wchar_t>;

  template<> class numeric_limits<short>;
  template<> class numeric_limits<int>;
  template<> class numeric_limits<long>;
  template<> class numeric_limits<long long>;
  template<> class numeric_limits<unsigned short>;
  template<> class numeric_limits<unsigned int>;
  template<> class numeric_limits<unsigned long>;
  template<> class numeric_limits<unsigned long long>;

  template<> class numeric_limits<float>;
  template<> class numeric_limits<double>;
  template<> class numeric_limits<long double>;
}
```

18.3.2.3 Class template `numeric_limits` [numeric.limits]

```
namespace std {
  template<class T> class numeric_limits {
  public:
    static constexpr bool is_specialized = false;
    static constexpr T min() noexcept { return T(); }
    static constexpr T max() noexcept { return T(); }
    static constexpr T lowest() noexcept { return T(); }

    static constexpr int  digits = 0;
    static constexpr int  digits10 = 0;
    static constexpr int  max_digits10 = 0;
    static constexpr bool is_signed = false;
    static constexpr bool is_integer = false;
    static constexpr bool is_exact = false;
    static constexpr int  radix = 0;
    static constexpr T epsilon() noexcept { return T(); }
    static constexpr T round_error() noexcept { return T(); }

    static constexpr int  min_exponent = 0;
    static constexpr int  min_exponent10 = 0;
    static constexpr int  max_exponent = 0;
    static constexpr int  max_exponent10 = 0;

    static constexpr bool has_infinity = false;
    static constexpr bool has_quiet_NaN = false;
    static constexpr bool has_signaling_NaN = false;
```

```
    static constexpr float_denorm_style has_denorm = denorm_absent;
    static constexpr bool has_denorm_loss = false;
    static constexpr T infinity() noexcept { return T(); }
    static constexpr T quiet_NaN() noexcept { return T(); }
    static constexpr T signaling_NaN() noexcept { return T(); }
    static constexpr T denorm_min() noexcept { return T(); }

    static constexpr bool is_iec559 = false;
    static constexpr bool is_bounded = false;
    static constexpr bool is_modulo = false;

    static constexpr bool traps = false;
    static constexpr bool tinyness_before = false;
    static constexpr float_round_style round_style = round_toward_zero;
  };

  template<class T> class numeric_limits<const T>;
  template<class T> class numeric_limits<volatile T>;
  template<class T> class numeric_limits<const volatile T>;
}
```

1 The default `numeric_limits<T>` template shall have all members, but with 0 or `false` values.

2 The value of each member of a specialization of `numeric_limits` on a *cv*-qualified type `cv T` shall be equal to the value of the corresponding member of the specialization on the unqualified type `T`.

18.3.2.4 `numeric_limits` members [numeric.limits.members]

`static constexpr T min() noexcept;`

1 Minimum finite value.[197]

2 For floating types with denormalization, returns the minimum positive normalized value.

3 Meaningful for all specializations in which `is_bounded != false`, or `is_bounded == false && is_signed == false`.

`static constexpr T max() noexcept;`

4 Maximum finite value.[198]

5 Meaningful for all specializations in which `is_bounded != false`.

`static constexpr T lowest() noexcept;`

6 A finite value `x` such that there is no other finite value `y` where `y < x`.[199]

7 Meaningful for all specializations in which `is_bounded != false`.

`static constexpr int digits;`

8 Number of `radix` digits that can be represented without change.

9 For integer types, the number of non-sign bits in the representation.

10 For floating point types, the number of `radix` digits in the mantissa.[200]

197) Equivalent to `CHAR_MIN`, `SHRT_MIN`, `FLT_MIN`, `DBL_MIN`, etc.
198) Equivalent to `CHAR_MAX`, `SHRT_MAX`, `FLT_MAX`, `DBL_MAX`, etc.
199) `lowest()` is necessary because not all floating-point representations have a smallest (most negative) value that is the negative of the largest (most positive) finite value.
200) Equivalent to `FLT_MANT_DIG`, `DBL_MANT_DIG`, `LDBL_MANT_DIG`.

`static constexpr int digits10;`

11 Number of base 10 digits that can be represented without change.[201]

12 Meaningful for all specializations in which `is_bounded != false`.

`static constexpr int max_digits10;`

13 Number of base 10 digits required to ensure that values which differ are always differentiated.

14 Meaningful for all floating point types.

`static constexpr bool is_signed;`

15 True if the type is signed.

16 Meaningful for all specializations.

`static constexpr bool is_integer;`

17 True if the type is integer.

18 Meaningful for all specializations.

`static constexpr bool is_exact;`

19 True if the type uses an exact representation. All integer types are exact, but not all exact types are integer. For example, rational and fixed-exponent representations are exact but not integer.

20 Meaningful for all specializations.

`static constexpr int radix;`

21 For floating types, specifies the base or radix of the exponent representation (often 2).[202]

22 For integer types, specifies the base of the representation.[203]

23 Meaningful for all specializations.

`static constexpr T epsilon() noexcept;`

24 Machine epsilon: the difference between 1 and the least value greater than 1 that is representable.[204]

25 Meaningful for all floating point types.

`static constexpr T round_error() noexcept;`

26 Measure of the maximum rounding error.[205]

`static constexpr int  min_exponent;`

27 Minimum negative integer such that `radix` raised to the power of one less than that integer is a normalized floating point number.[206]

28 Meaningful for all floating point types.

`static constexpr int  min_exponent10;`

---

201) Equivalent to `FLT_DIG`, `DBL_DIG`, `LDBL_DIG`.
202) Equivalent to `FLT_RADIX`.
203) Distinguishes types with bases other than 2 (e.g. BCD).
204) Equivalent to `FLT_EPSILON`, `DBL_EPSILON`, `LDBL_EPSILON`.
205) Rounding error is described in ISO/IEC 10967-1 Language independent arithmetic - Part 1 Section 5.2.8 and Annex A Rationale Section A.5.2.8 - Rounding constants.
206) Equivalent to `FLT_MIN_EXP`, `DBL_MIN_EXP`, `LDBL_MIN_EXP`.

29 Minimum negative integer such that 10 raised to that power is in the range of normalized floating point numbers.[207]

30 Meaningful for all floating point types.

```
static constexpr int  max_exponent;
```

31 Maximum positive integer such that `radix` raised to the power one less than that integer is a representable finite floating point number.[208]

32 Meaningful for all floating point types.

```
static constexpr int  max_exponent10;
```

33 Maximum positive integer such that 10 raised to that power is in the range of representable finite floating point numbers.[209]

34 Meaningful for all floating point types.

```
static constexpr bool has_infinity;
```

35 True if the type has a representation for positive infinity.

36 Meaningful for all floating point types.

37 Shall be `true` for all specializations in which `is_iec559 != false`.

```
static constexpr bool has_quiet_NaN;
```

38 True if the type has a representation for a quiet (non-signaling) "Not a Number."[210]

39 Meaningful for all floating point types.

40 Shall be `true` for all specializations in which `is_iec559 != false`.

```
static constexpr bool has_signaling_NaN;
```

41 True if the type has a representation for a signaling "Not a Number."[211]

42 Meaningful for all floating point types.

43 Shall be `true` for all specializations in which `is_iec559 != false`.

```
static constexpr float_denorm_style has_denorm;
```

44 `denorm_present` if the type allows denormalized values (variable number of exponent bits)[212], `denorm_absent` if the type does not allow denormalized values, and `denorm_indeterminate` if it is indeterminate at compile time whether the type allows denormalized values.

45 Meaningful for all floating point types.

```
static constexpr bool has_denorm_loss;
```

46 True if loss of accuracy is detected as a denormalization loss, rather than as an inexact result.[213]

```
static constexpr T infinity() noexcept;
```

207) Equivalent to `FLT_MIN_10_EXP`, `DBL_MIN_10_EXP`, `LDBL_MIN_10_EXP`.
208) Equivalent to `FLT_MAX_EXP`, `DBL_MAX_EXP`, `LDBL_MAX_EXP`.
209) Equivalent to `FLT_MAX_10_EXP`, `DBL_MAX_10_EXP`, `LDBL_MAX_10_EXP`.
210) Required by LIA-1.
211) Required by LIA-1.
212) Required by LIA-1.
213) See IEC 559.

47 Representation of positive infinity, if available.[214]

48 Meaningful for all specializations for which `has_infinity != false`. Required in specializations for which `is_iec559 != false`.

```
static constexpr T quiet_NaN() noexcept;
```

49 Representation of a quiet "Not a Number," if available.[215]

50 Meaningful for all specializations for which `has_quiet_NaN != false`. Required in specializations for which `is_iec559 != false`.

```
static constexpr T signaling_NaN() noexcept;
```

51 Representation of a signaling "Not a Number," if available.[216]

52 Meaningful for all specializations for which `has_signaling_NaN != false`. Required in specializations for which `is_iec559 != false`.

```
static constexpr T denorm_min() noexcept;
```

53 Minimum positive denormalized value.[217]

54 Meaningful for all floating point types.

55 In specializations for which `has_denorm == false`, returns the minimum positive normalized value.

```
static constexpr bool is_iec559;
```

56 True if and only if the type adheres to IEC 559 standard.[218]

57 Meaningful for all floating point types.

```
static constexpr bool is_bounded;
```

58 True if the set of values representable by the type is finite.[219] [*Note:* All fundamental types (3.9.1) are bounded. This member would be false for arbitrary precision types. — *end note*]

59 Meaningful for all specializations.

```
static constexpr bool is_modulo;
```

60 True if the type is modulo.[220] A type is modulo if, for any operation involving `+`, `-`, or `*` on values of that type whose result would fall outside the range `[min(),max()]`, the value returned differs from the true value by an integer multiple of `max() - min() + 1`.

61 On most machines, this is `false` for floating types, `true` for unsigned integers, and `true` for signed integers.

62 Meaningful for all specializations.

```
static constexpr bool traps;
```

63 `true` if, at program startup, there exists a value of the type that would cause an arithmetic operation using that value to trap.[221]

64 Meaningful for all specializations.

214) Required by LIA-1.
215) Required by LIA-1.
216) Required by LIA-1.
217) Required by LIA-1.
218) International Electrotechnical Commission standard 559 is the same as IEEE 754.
219) Required by LIA-1.
220) Required by LIA-1.
221) Required by LIA-1.

```
static constexpr bool tinyness_before;
```

65 `true` if tinyness is detected before rounding.[222]

66 Meaningful for all floating point types.

```
static constexpr float_round_style round_style;
```

67 The rounding style for the type.[223]

68 Meaningful for all floating point types. Specializations for integer types shall return `round_toward_zero`.

18.3.2.5 Type `float_round_style` [round.style]

```
namespace std {
  enum float_round_style {
    round_indeterminate       = -1,
    round_toward_zero         =  0,
    round_to_nearest          =  1,
    round_toward_infinity     =  2,
    round_toward_neg_infinity =  3
  };
}
```

1 The rounding mode for floating point arithmetic is characterized by the values:

(1.1) — `round_indeterminate` if the rounding style is indeterminable

(1.2) — `round_toward_zero` if the rounding style is toward zero

(1.3) — `round_to_nearest` if the rounding style is to the nearest representable value

(1.4) — `round_toward_infinity` if the rounding style is toward infinity

(1.5) — `round_toward_neg_infinity` if the rounding style is toward negative infinity

18.3.2.6 Type `float_denorm_style` [denorm.style]

```
namespace std {
  enum float_denorm_style {
    denorm_indeterminate = -1,
    denorm_absent = 0,
    denorm_present = 1
  };
}
```

1 The presence or absence of denormalization (variable number of exponent bits) is characterized by the values:

(1.1) — `denorm_indeterminate` if it cannot be determined whether or not the type allows denormalized values

(1.2) — `denorm_absent` if the type does not allow denormalized values

(1.3) — `denorm_present` if the type does allow denormalized values

222) Refer to IEC 559. Required by LIA-1.
223) Equivalent to `FLT_ROUNDS`. Required by LIA-1.

18.3.2.7 `numeric_limits` specializations [numeric.special]

1 All members shall be provided for all specializations. However, many values are only required to be meaningful under certain conditions (for example, `epsilon()` is only meaningful if `is_integer` is `false`). Any value that is not "meaningful" shall be set to 0 or `false`.

2 [*Example:*

```
namespace std {
  template<> class numeric_limits<float> {
  public:
    static constexpr bool is_specialized = true;

    inline static constexpr float min() noexcept { return 1.17549435E-38F; }
    inline static constexpr float max() noexcept { return 3.40282347E+38F; }
    inline static constexpr float lowest() noexcept { return -3.40282347E+38F; }

    static constexpr int digits   = 24;
    static constexpr int digits10 =  6;
    static constexpr int max_digits10 =  9;

    static constexpr bool is_signed  = true;
    static constexpr bool is_integer = false;
    static constexpr bool is_exact   = false;

    static constexpr int radix = 2;
    inline static constexpr float epsilon() noexcept     { return 1.19209290E-07F; }
    inline static constexpr float round_error() noexcept { return 0.5F; }

    static constexpr int min_exponent   = -125;
    static constexpr int min_exponent10 = - 37;
    static constexpr int max_exponent   = +128;
    static constexpr int max_exponent10 = + 38;

    static constexpr bool has_infinity             = true;
    static constexpr bool has_quiet_NaN            = true;
    static constexpr bool has_signaling_NaN        = true;
    static constexpr float_denorm_style has_denorm = denorm_absent;
    static constexpr bool has_denorm_loss          = false;

    inline static constexpr float infinity()      noexcept { return value; }
    inline static constexpr float quiet_NaN()     noexcept { return value; }
    inline static constexpr float signaling_NaN() noexcept { return value; }
    inline static constexpr float denorm_min()    noexcept { return min(); }

    static constexpr bool is_iec559  = true;
    static constexpr bool is_bounded = true;
    static constexpr bool is_modulo  = false;
    static constexpr bool traps      = true;
    static constexpr bool tinyness_before = true;

    static constexpr float_round_style round_style = round_to_nearest;
  };
}
```

— *end example*]

3 The specialization for `bool` shall be provided as follows:

```
namespace std {
   template<> class numeric_limits<bool> {
   public:
     static constexpr bool is_specialized = true;
     static constexpr bool min() noexcept { return false; }
     static constexpr bool max() noexcept { return true; }
     static constexpr bool lowest() noexcept { return false; }

     static constexpr int  digits = 1;
     static constexpr int  digits10 = 0;
     static constexpr int  max_digits10 = 0;

     static constexpr bool is_signed = false;
     static constexpr bool is_integer = true;
     static constexpr bool is_exact = true;
     static constexpr int  radix = 2;
     static constexpr bool epsilon() noexcept { return 0; }
     static constexpr bool round_error() noexcept { return 0; }

     static constexpr int  min_exponent = 0;
     static constexpr int  min_exponent10 = 0;
     static constexpr int  max_exponent = 0;
     static constexpr int  max_exponent10 = 0;

     static constexpr bool has_infinity = false;
     static constexpr bool has_quiet_NaN = false;
     static constexpr bool has_signaling_NaN = false;
     static constexpr float_denorm_style has_denorm = denorm_absent;
     static constexpr bool has_denorm_loss = false;
     static constexpr bool infinity() noexcept { return 0; }
     static constexpr bool quiet_NaN() noexcept { return 0; }
     static constexpr bool signaling_NaN() noexcept { return 0; }
     static constexpr bool denorm_min() noexcept { return 0; }

     static constexpr bool is_iec559 = false;
     static constexpr bool is_bounded = true;
     static constexpr bool is_modulo = false;

     static constexpr bool traps = false;
     static constexpr bool tinyness_before = false;
     static constexpr float_round_style round_style = round_toward_zero;
   };
}
```

18.3.3 C library [c.limits]

1 Table 31 describes the header `<climits>`.

2 The contents are the same as the Standard C library header `<limits.h>`. [*Note:* The types of the constants defined by macros in `<climits>` are not required to match the types to which the macros refer. — *end note*]

3 Table 32 describes the header `<cfloat>`.

4 The contents are the same as the Standard C library header `<float.h>`.

See also: ISO C 7.1.5, 5.2.4.2.2, 5.2.4.2.1.

Table 31 — Header `<climits>` synopsis

Type	Name(s)				
Values:					
CHAR_BIT	INT_MAX	LONG_MAX	SCHAR_MIN	SHRT_MIN	ULLONG_MAX
CHAR_MAX	LLONG_MAX	LONG_MIN	SCHAR_MAX	UCHAR_MAX	ULONG_MAX
CHAR_MIN	LLONG_MIN	MB_LEN_MAX	SHRT_MAX	UINT_MAX	USHRT_MAX
INT_MIN					

Table 32 — Header `<cfloat>` synopsis

Type	Name(s)		
Values:			
DBL_DIG	DBL_MIN_EXP	FLT_MAX_EXP	LDBL_MANT_DIG
DBL_EPSILON	DECIMAL_DIG	FLT_MIN	LDBL_MAX_10_EXP
DBL_MANT_DIG	FLT_DIG	FLT_MIN_10_EXP	LDBL_MAX_EXP
DBL_MAX	FLT_EPSILON	FLT_MIN_EXP	LDBL_MAX
DBL_MAX_10_EXP	FLT_EVAL_METHOD	FLT_RADIX	LDBL_MIN
DBL_MAX_EXP	FLT_MANT_DIG	FLT_ROUNDS	LDBL_MIN_10_EXP
DBL_MIN	FLT_MAX	LDBL_DIG	LDBL_MIN_EXP
DBL_MIN_10_EXP	FLT_MAX_10_EXP	LDBL_EPSILON	

18.4 Integer types [cstdint]

18.4.1 Header `<cstdint>` synopsis [cstdint.syn]

```
namespace std {
  typedef signed integer type int8_t;   // optional
  typedef signed integer type int16_t;  // optional
  typedef signed integer type int32_t;  // optional
  typedef signed integer type int64_t;  // optional

  typedef signed integer type int_fast8_t;
  typedef signed integer type int_fast16_t;
  typedef signed integer type int_fast32_t;
  typedef signed integer type int_fast64_t;

  typedef signed integer type int_least8_t;
  typedef signed integer type int_least16_t;
  typedef signed integer type int_least32_t;
  typedef signed integer type int_least64_t;

  typedef signed integer type intmax_t;
  typedef signed integer type intptr_t;         // optional

  typedef unsigned integer type uint8_t;        // optional
  typedef unsigned integer type uint16_t;       // optional
  typedef unsigned integer type uint32_t;       // optional
  typedef unsigned integer type uint64_t;       // optional

  typedef unsigned integer type uint_fast8_t;
  typedef unsigned integer type uint_fast16_t;
  typedef unsigned integer type uint_fast32_t;
  typedef unsigned integer type uint_fast64_t;
```

```
  typedef unsigned integer type uint_least8_t;
  typedef unsigned integer type uint_least16_t;
  typedef unsigned integer type uint_least32_t;
  typedef unsigned integer type uint_least64_t;

  typedef unsigned integer type uintmax_t;
  typedef unsigned integer type uintptr_t;      // optional
} // namespace std
```

1 The header also defines numerous macros of the form:

```
INT_[FAST LEAST]{8 16 32 64}_MIN
[U]INT_[FAST LEAST]{8 16 32 64}_MAX
INT{MAX PTR}_MIN
[U]INT{MAX PTR}_MAX
{PTRDIFF SIG_ATOMIC WCHAR WINT}{_MAX _MIN}
SIZE_MAX
```

plus function macros of the form:

```
[U]INT{8 16 32 64 MAX}_C
```

2 The header defines all functions, types, and macros the same as 7.18 in the C standard. [*Note:* The macros defined by `<cstdint>` are provided unconditionally. In particular, the symbols `__STDC_LIMIT_MACROS` and `__STDC_CONSTANT_MACROS` (mentioned in footnotes 219, 220, and 222 in the C standard) play no role in C++. — *end note*]

18.5 Start and termination [support.start.term]

1 Table 33 describes some of the contents of the header `<cstdlib>`.

Table 33 — Header `<cstdlib>` synopsis

Type	Name(s)		
Macros:	`EXIT_FAILURE`	`EXIT_SUCCESS`	
Functions:	`_Exit`	`abort`	`atexit`
	`at_quick_exit`	`exit`	`quick_exit`

2 The contents are the same as the Standard C library header `<stdlib.h>`, with the following changes:

```
[[noreturn]] void _Exit(int status) noexcept;
```

3 The function `_Exit(int status)` has additional behavior in this International Standard:

(3.1) — The program is terminated without executing destructors for objects of automatic, thread, or static storage duration and without calling functions passed to `atexit()` (3.6.3).

```
[[noreturn]] void abort(void) noexcept;
```

4 The function `abort()` has additional behavior in this International Standard:

(4.1) — The program is terminated without executing destructors for objects of automatic, thread, or static storage duration and without calling functions passed to `atexit()` (3.6.3).

```
extern "C" int atexit(void (*f)(void)) noexcept;
extern "C++" int atexit(void (*f)(void)) noexcept;
```

5 *Effects:* The `atexit()` functions register the function pointed to by `f` to be called without arguments at normal program termination. It is unspecified whether a call to `atexit()` that does not happen before (1.10) a call to `exit()` will succeed. [*Note:* The `atexit()` functions do not introduce a data race (17.6.5.9). — *end note*]

6 *Implementation limits:* The implementation shall support the registration of at least 32 functions.

7 *Returns:* The `atexit()` function returns zero if the registration succeeds, non-zero if it fails.

```
[[noreturn]] void exit(int status);
```

8 The function `exit()` has additional behavior in this International Standard:

(8.1) — First, objects with thread storage duration and associated with the current thread are destroyed. Next, objects with static storage duration are destroyed and functions registered by calling `atexit` are called.[224] See 3.6.3 for the order of destructions and calls. (Automatic objects are not destroyed as a result of calling `exit()`.)[225]

If control leaves a registered function called by `exit` because the function does not provide a handler for a thrown exception, `std::terminate()` shall be called (15.5.1).

(8.2) — Next, all open C streams (as mediated by the function signatures declared in `<cstdio>`) with unwritten buffered data are flushed, all open C streams are closed, and all files created by calling `tmpfile()` are removed.

(8.3) — Finally, control is returned to the host environment. If `status` is zero or `EXIT_SUCCESS`, an implementation-defined form of the status *successful termination* is returned. If `status` is `EXIT_FAILURE`, an implementation-defined form of the status *unsuccessful termination* is returned. Otherwise the status returned is implementation-defined.[226]

```
extern "C" int at_quick_exit(void (*f)(void)) noexcept;
extern "C++" int at_quick_exit(void (*f)(void)) noexcept;
```

9 *Effects:* The `at_quick_exit()` functions register the function pointed to by `f` to be called without arguments when `quick_exit` is called. It is unspecified whether a call to `at_quick_exit()` that does not happen before (1.10) all calls to `quick_exit` will succeed. [*Note:* The `at_quick_exit()` functions do not introduce a data race (17.6.5.9). — *end note*] [*Note:* The order of registration may be indeterminate if `at_quick_exit` was called from more than one thread. — *end note*] [*Note:* The `at_quick_exit` registrations are distinct from the `atexit` registrations, and applications may need to call both registration functions with the same argument. — *end note*]

10 *Implementation limits:* The implementation shall support the registration of at least 32 functions.

11 *Returns:* Zero if the registration succeeds, non-zero if it fails.

```
[[noreturn]] void quick_exit(int status) noexcept;
```

12 *Effects:* Functions registered by calls to `at_quick_exit` are called in the reverse order of their registration, except that a function shall be called after any previously registered functions that had already been called at the time it was registered. Objects shall not be destroyed as a result of calling `quick_exit`. If control leaves a registered function called by `quick_exit` because the function does not provide a handler for a thrown exception, `std::terminate()` shall be called. [*Note:* `at_quick_exit` may call a registered function from a different thread than the one that registered it, so registered functions should not rely on the identity of objects with thread storage duration. — *end note*] After calling registered functions, `quick_exit` shall call `_Exit(status)`. [*Note:* The standard file buffers are not flushed. See: ISO C 7.20.4.4. — *end note*]

224) A function is called for every time it is registered.

225) Objects with automatic storage duration are all destroyed in a program whose function `main()` contains no automatic objects and executes the call to `exit()`. Control can be transferred directly to such a `main()` by throwing an exception that is caught in `main()`.

226) The macros `EXIT_FAILURE` and `EXIT_SUCCESS` are defined in `<cstdlib>`.

See also: 3.6, 3.6.3, ISO C 7.10.4.

18.6 Dynamic memory management [support.dynamic]

1 The header `<new>` defines several functions that manage the allocation of dynamic storage in a program. It also defines components for reporting storage management errors.

Header `<new>` synopsis

```
namespace std {
  class bad_alloc;
  class bad_array_new_length;
  struct nothrow_t {};
  extern const nothrow_t nothrow;
  typedef void (*new_handler)();
  new_handler get_new_handler() noexcept;
  new_handler set_new_handler(new_handler new_p) noexcept;
}

void* operator new(std::size_t size);
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
void  operator delete(void* ptr) noexcept;
void  operator delete(void* ptr, const std::nothrow_t&) noexcept;
void  operator delete(void* ptr, std::size_t size) noexcept;
void* operator new[](std::size_t size);
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
void  operator delete[](void* ptr) noexcept;
void  operator delete[](void* ptr, const std::nothrow_t&) noexcept;
void  operator delete[](void* ptr, std::size_t size) noexcept;

void* operator new  (std::size_t size, void* ptr) noexcept;
void* operator new[](std::size_t size, void* ptr) noexcept;
void  operator delete  (void* ptr, void*) noexcept;
void  operator delete[](void* ptr, void*) noexcept;
```

See also: 1.7, 3.7.4, 5.3.4, 5.3.5, 12.5, 20.7.

18.6.1 Storage allocation and deallocation [new.delete]

1 Except where otherwise specified, the provisions of (3.7.4) apply to the library versions of `operator new` and `operator delete`.

18.6.1.1 Single-object forms [new.delete.single]

```
void* operator new(std::size_t size);
```

1 *Effects:* The *allocation function* (3.7.4.1) called by a *new-expression* (5.3.4) to allocate `size` bytes of storage suitably aligned to represent any object of that size.

2 *Replaceable:* a C++ program may define a function with this function signature that displaces the default version defined by the C++ standard library.

3 *Required behavior:* Return a non-null pointer to suitably aligned storage (3.7.4), or else throw a `bad_alloc` exception. This requirement is binding on a replacement version of this function.

4 *Default behavior:*

(4.1) — Executes a loop: Within the loop, the function first attempts to allocate the requested storage. Whether the attempt involves a call to the Standard C library function `malloc` is unspecified.

(4.2) — Returns a pointer to the allocated storage if the attempt is successful. Otherwise, if the current `new_handler` (18.6.2.5) is a null pointer value, throws `bad_alloc`.

(4.3) — Otherwise, the function calls the current `new_handler` function (18.6.2.3). If the called function returns, the loop repeats.

(4.4) — The loop terminates when an attempt to allocate the requested storage is successful or when a called `new_handler` function does not return.

```
void* operator new(std::size_t size, const std::nothrow_t&) noexcept;
```

5 *Effects:* Same as above, except that it is called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a `bad_alloc` exception.

6 *Replaceable:* a C++ program may define a function with this function signature that displaces the default version defined by the C++ standard library.

7 *Required behavior:* Return a non-null pointer to suitably aligned storage (3.7.4), or else return a null pointer. This nothrow version of `operator new` returns a pointer obtained as if acquired from the (possibly replaced) ordinary version. This requirement is binding on a replacement version of this function.

8 *Default behavior:* Calls `operator new(size)`. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

9 [*Example:*

```
T* p1 = new T;                  // throws bad_alloc if it fails
T* p2 = new(nothrow) T;         // returns nullptr if it fails
```

— *end example*]

```
void operator delete(void* ptr) noexcept;
void operator delete(void* ptr, std::size_t size) noexcept;
```

10 *Effects:* The *deallocation function* (3.7.4.2) called by a *delete-expression* to render the value of `ptr` invalid.

11 *Replaceable:* a C++ program may define a function with signature `void operator delete(void* ptr) noexcept` that displaces the default version defined by the C++ standard library. If this function (without `size` parameter) is defined, the program should also define `void operator delete(void* ptr, std::size_t size) noexcept`. If this function with `size` parameter is defined, the program shall also define the version without the `size` parameter. [*Note:* The default behavior below may change in the future, which will require replacing both deallocation functions when replacing the allocation function. — *end note*]

12 *Requires: ptr* shall be a null pointer or its value shall be a value returned by an earlier call to the (possibly replaced) `operator new(std::size_t)` or `operator new(std::size_t,const std::nothrow_t&)` which has not been invalidated by an intervening call to `operator delete(void*)` or `operator delete(void*, std::size_t)`.

13 *Requires:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

14 *Requires:* If present, the `std::size_t size` argument shall equal the size argument passed to the allocation function that returned `ptr`.

15 *Required behavior:* Calls to `operator delete(void* ptr, std::size_t size)` may be changed to calls to `operator delete(void* ptr)` without affecting memory allocation. [*Note:* A conforming implementation is for `operator delete(void* ptr, std::size_t size)` to simply call `operator delete(ptr)`. — *end note*]

16 *Default behavior:* the function `operator delete(void* ptr, std::size_t size)` calls `operator delete(ptr)`. [*Note:* See the note in the above *Replaceable* paragraph. — *end note*]

17 *Default behavior:* If `ptr` is null, does nothing. Otherwise, reclaims the storage allocated by the earlier call to `operator new`.

18 *Remarks:* It is unspecified under what conditions part or all of such reclaimed storage will be allocated by subsequent calls to `operator new` or any of `calloc`, `malloc`, or `realloc`, declared in `<cstdlib>`.

```
void operator delete(void* ptr, const std::nothrow_t&) noexcept;
```

19 *Effects:* The *deallocation function* (3.7.4.2) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the *new-expression* throws an exception.

20 *Replaceable:* a C++ program may define a function with signature `void operator delete(void* ptr, const std::nothrow_t&) noexcept` that displaces the default version defined by the C++ standard library.

21 *Requires:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

22 *Default behavior:* `operator delete(void* ptr, const std::nothrow_t&)` calls `operator delete(ptr)`.

18.6.1.2 Array forms [new.delete.array]

```
void* operator new[](std::size_t size);
```

1 *Effects:* The *allocation function* (3.7.4.1) called by the array form of a *new-expression* (5.3.4) to allocate `size` bytes of storage suitably aligned to represent any array object of that size or smaller.[227]

2 *Replaceable:* a C++ program can define a function with this function signature that displaces the default version defined by the C++ standard library.

3 *Required behavior:* Same as for `operator new(std::size_t)`. This requirement is binding on a replacement version of this function.

4 *Default behavior:* Returns `operator new(size)`.

```
void* operator new[](std::size_t size, const std::nothrow_t&) noexcept;
```

5 *Effects:* Same as above, except that it is called by a placement version of a *new-expression* when a C++ program prefers a null pointer result as an error indication, instead of a `bad_alloc` exception.

6 *Replaceable:* a C++ program can define a function with this function signature that displaces the default version defined by the C++ standard library.

7 *Required behavior:* Return a non-null pointer to suitably aligned storage (3.7.4), or return a null pointer. This requirement is binding on a replacement version of this function.

8 *Default behavior:* Calls `operator new[](size)`. If the call returns normally, returns the result of that call. Otherwise, returns a null pointer.

```
void operator delete[](void* ptr) noexcept;
void operator delete[](void* ptr, std::size_t size) noexcept;
```

227) It is not the direct responsibility of `operator new[](std::size_t)` or `operator delete[](void*)` to note the repetition count or element size of the array. Those operations are performed elsewhere in the array `new` and `delete` expressions. The array `new` expression, may, however, increase the `size` argument to `operator new[](std::size_t)` to obtain space to store supplemental information.

9 *Effects:* The *deallocation function* (3.7.4.2) called by the array form of a *delete-expression* to render the value of `ptr` invalid.

10 *Replaceable:* a C++ program can define a function with signature `void operator delete[](void* ptr) noexcept` that displaces the default version defined by the C++ standard library. If this function (without `size` parameter) is defined, the program should also define `void operator delete[](void* ptr, std::size_t size) noexcept`. If this function with `size` parameter is defined, the program shall also define the version without the `size` parameter. [*Note:* The default behavior below may change in the future, which will require replacing both deallocation functions when replacing the allocation function. — *end note*]

11 *Requires:* *ptr* shall be a null pointer or its value shall be the value returned by an earlier call to `operator new[](std::size_t)` or `operator new[](std::size_t,const std::nothrow_t&)` which has not been invalidated by an intervening call to `operator delete[](void*)` or `operator delete[](void*, std::size_t)`.

12 *Requires:* If present, the `std::size_t size` argument must equal the size argument passed to the allocation function that returned `ptr`.

13 *Required behavior:* Calls to `operator delete[](void* ptr, std::size_t size)` may be changed to calls to `operator delete[](void* ptr)` without affecting memory allocation. [*Note:* A conforming implementation is for `operator delete[](void* ptr, std::size_t size)` to simply call `operator delete[](void* ptr)`. — *end note*]

14 *Requires:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

15 *Default behavior:* `operator delete[](void* ptr, std::size_t size)` calls `operator delete[](ptr)`, and `operator delete[](void* ptr)` calls `operator delete(ptr)`.

```
void operator delete[](void* ptr, const std::nothrow_t&) noexcept;
```

16 *Effects:* The *deallocation function* (3.7.4.2) called by the implementation to render the value of `ptr` invalid when the constructor invoked from a nothrow placement version of the array *new-expression* throws an exception.

17 *Replaceable:* a C++ program may define a function with signature `void operator delete[](void* ptr, const std::nothrow_t&) noexcept` that displaces the default version defined by the C++ standard library.

18 *Requires:* If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

19 *Default behavior:* `operator delete[](void* ptr, const std::nothrow_t&)` calls `operator delete[](ptr)`.

18.6.1.3 Placement forms [new.delete.placement]

1 These functions are reserved, a C++ program may not define functions that displace the versions in the Standard C++ library (17.6.4). The provisions of (3.7.4) do not apply to these reserved placement forms of `operator new` and `operator delete`.

```
void* operator new(std::size_t size, void* ptr) noexcept;
```

2 *Returns:* `ptr`.

3 *Remarks:* Intentionally performs no other action.

4 [*Example:* This can be useful for constructing an object at a known address:

```
void* place = operator new(sizeof(Something));
Something* p = new (place) Something();
```

— *end example*]

```
void* operator new[](std::size_t size, void* ptr) noexcept;
```

Returns: `ptr`.

Remarks: Intentionally performs no other action.

```
void operator delete(void* ptr, void*) noexcept;
```

Effects: Intentionally performs no action.

Requires: If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

Remarks: Default function called when any part of the initialization in a placement *new-expression* that invokes the library's non-array placement operator new terminates by throwing an exception (5.3.4).

```
void operator delete[](void* ptr, void*) noexcept;
```

Effects: Intentionally performs no action.

Requires: If an implementation has strict pointer safety (3.7.4.3) then `ptr` shall be a safely-derived pointer.

Remarks: Default function called when any part of the initialization in a placement *new-expression* that invokes the library's array placement operator new terminates by throwing an exception (5.3.4).

18.6.1.4 Data races [new.delete.dataraces]

For purposes of determining the existence of data races, the library versions of `operator new`, user replacement versions of global `operator new`, the C standard library functions `calloc` and `malloc`, the library versions of `operator delete`, user replacement versions of `operator delete`, the C standard library function `free`, and the C standard library function `realloc` shall not introduce a data race (17.6.5.9). Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before (1.10) the next allocation (if any) in this order.

18.6.2 Storage allocation errors [alloc.errors]

18.6.2.1 Class `bad_alloc` [bad.alloc]

```
namespace std {
  class bad_alloc : public exception {
  public:
    bad_alloc() noexcept;
    bad_alloc(const bad_alloc&) noexcept;
    bad_alloc& operator=(const bad_alloc&) noexcept;
    virtual const char* what() const noexcept;
  };
}
```

The class `bad_alloc` defines the type of objects thrown as exceptions by the implementation to report a failure to allocate storage.

```
bad_alloc() noexcept;
```

Effects: Constructs an object of class `bad_alloc`.

```
bad_alloc(const bad_alloc&) noexcept;
bad_alloc& operator=(const bad_alloc&) noexcept;
```

Effects: Copies an object of class `bad_alloc`.

```
virtual const char* what() const noexcept;
```

4 *Returns:* An implementation-defined NTBS.

5 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4).

18.6.2.2 Class `bad_array_new_length` [new.badlength]

```
namespace std {
  class bad_array_new_length : public bad_alloc {
  public:
    bad_array_new_length() noexcept;
    virtual const char* what() const noexcept;
  };
}
```

1 The class `bad_array_new_length` defines the type of objects thrown as exceptions by the implementation to report an attempt to allocate an array of size less than zero or greater than an implementation-defined limit (5.3.4).

```
bad_array_new_length() noexcept;
```

2 *Effects:* constructs an object of class `bad_array_new_length`.

```
virtual const char* what() const noexcept;
```

3 *Returns:* An implementation-defined NTBS.

4 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4).

18.6.2.3 Type `new_handler` [new.handler]

```
typedef void (*new_handler)();
```

1 The type of a *handler function* to be called by `operator new()` or `operator new[]()` (18.6.1) when they cannot satisfy a request for additional storage.

2 *Required behavior:* A `new_handler` shall perform one of the following:

(2.1) — make more storage available for allocation and then return;

(2.2) — throw an exception of type `bad_alloc` or a class derived from `bad_alloc`;

(2.3) — terminate execution of the program without returning to the caller;

18.6.2.4 `set_new_handler` [set.new.handler]

```
new_handler set_new_handler(new_handler new_p) noexcept;
```

1 *Effects:* Establishes the function designated by `new_p` as the current `new_handler`.

2 *Returns:* The previous `new_handler`.

3 *Remarks:* The initial `new_handler` is a null pointer.

18.6.2.5 `get_new_handler` [get.new.handler]

```
new_handler get_new_handler() noexcept;
```

1 *Returns:* The current `new_handler`. [*Note:* This may be a null pointer value. — *end note*]

18.7 Type identification [support.rtti]

1 The header `<typeinfo>` defines a type associated with type information generated by the implementation. It also defines two types for reporting dynamic type identification errors.

Header `<typeinfo>` synopsis

```
namespace std {
  class type_info;
  class bad_cast;
  class bad_typeid;
}
```

See also: 5.2.7, 5.2.8.

18.7.1 Class `type_info` [type.info]

```
namespace std {
  class type_info {
  public:
    virtual ~type_info();
    bool operator==(const type_info& rhs) const noexcept;
    bool operator!=(const type_info& rhs) const noexcept;
    bool before(const type_info& rhs) const noexcept;
    size_t hash_code() const noexcept;
    const char* name() const noexcept;

    type_info(const type_info& rhs) = delete;            // cannot be copied
    type_info& operator=(const type_info& rhs) = delete; // cannot be copied
  };
}
```

1 The class `type_info` describes type information generated by the implementation. Objects of this class effectively store a pointer to a name for the type, and an encoded value suitable for comparing two types for equality or collating order. The names, encoding rule, and collating sequence for types are all unspecified and may differ between programs.

`bool operator==(const type_info& rhs) const noexcept;`

2 *Effects:* Compares the current object with `rhs`.

3 *Returns:* `true` if the two values describe the same type.

`bool operator!=(const type_info& rhs) const noexcept;`

4 *Returns:* `!(*this == rhs)`.

`bool before(const type_info& rhs) const noexcept;`

5 *Effects:* Compares the current object with `rhs`.

6 *Returns:* `true` if `*this` precedes `rhs` in the implementation's collation order.

`size_t hash_code() const noexcept;`

7 *Returns:* An unspecified value, except that within a single execution of the program, it shall return the same value for any two `type_info` objects which compare equal.

8 *Remark:* an implementation should return different values for two `type_info` objects which do not compare equal.

```
const char* name() const noexcept;
```

9 *Returns:* An implementation-defined NTBS.

10 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4)

18.7.2 Class `bad_cast` [bad.cast]

```
namespace std {
  class bad_cast : public exception {
  public:
    bad_cast() noexcept;
    bad_cast(const bad_cast&) noexcept;
    bad_cast& operator=(const bad_cast&) noexcept;
    virtual const char* what() const noexcept;
  };
}
```

1 The class `bad_cast` defines the type of objects thrown as exceptions by the implementation to report the execution of an invalid *dynamic-cast* expression (5.2.7).

```
bad_cast() noexcept;
```

2 *Effects:* Constructs an object of class `bad_cast`.

```
bad_cast(const bad_cast&) noexcept;
bad_cast& operator=(const bad_cast&) noexcept;
```

3 *Effects:* Copies an object of class `bad_cast`.

```
virtual const char* what() const noexcept;
```

4 *Returns:* An implementation-defined NTBS.

5 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4)

18.7.3 Class `bad_typeid` [bad.typeid]

```
namespace std {
  class bad_typeid : public exception {
  public:
    bad_typeid() noexcept;
    bad_typeid(const bad_typeid&) noexcept;
    bad_typeid& operator=(const bad_typeid&) noexcept;
    virtual const char* what() const noexcept;
  };
}
```

1 The class `bad_typeid` defines the type of objects thrown as exceptions by the implementation to report a null pointer in a *typeid* expression (5.2.8).

```
bad_typeid() noexcept;
```

2 *Effects:* Constructs an object of class `bad_typeid`.

```
bad_typeid(const bad_typeid&) noexcept;
bad_typeid& operator=(const bad_typeid&) noexcept;
```

3 *Effects:* Copies an object of class `bad_typeid`.

```
virtual const char* what() const noexcept;
```

4 *Returns:* An implementation-defined NTBS.

5 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4)

18.8 Exception handling [support.exception]

1 The header `<exception>` defines several types and functions related to the handling of exceptions in a C++ program.

Header `<exception>` synopsis

```
namespace std {
  class exception;
  class bad_exception;
  class nested_exception;

  typedef void (*unexpected_handler)();
  unexpected_handler get_unexpected() noexcept;
  unexpected_handler set_unexpected(unexpected_handler f) noexcept;
  [[noreturn]] void unexpected();

  typedef void (*terminate_handler)();
  terminate_handler get_terminate() noexcept;
  terminate_handler set_terminate(terminate_handler f) noexcept;
  [[noreturn]] void terminate() noexcept;

  int uncaught_exceptions() noexcept;
  // D.9, uncaught_exception (deprecated)
  bool uncaught_exception() noexcept;

  typedef unspecified  exception_ptr;

  exception_ptr current_exception() noexcept;
  [[noreturn]] void rethrow_exception(exception_ptr p);
  template<class E> exception_ptr make_exception_ptr(E e) noexcept;

  template <class T> [[noreturn]] void throw_with_nested(T&& t);
  template <class E> void rethrow_if_nested(const E& e);
}
```

See also: 15.5.

18.8.1 Class `exception` [exception]

```
namespace std {
  class exception {
  public:
    exception() noexcept;
    exception(const exception&) noexcept;
    exception& operator=(const exception&) noexcept;
    virtual ~exception();
    virtual const char* what() const noexcept;
  };
}
```

1 The class `exception` defines the base class for the types of objects thrown as exceptions by C++ standard library components, and certain expressions, to report errors detected during program execution.

2 Each standard library class `T` that derives from class `exception` shall have a publicly accessible copy constructor and a publicly accessible copy assignment operator that do not exit with an exception. These member functions shall meet the following postcondition: If two objects `lhs` and `rhs` both have dynamic type `T` and `lhs` is a copy of `rhs`, then `strcmp(lhs.what(), rhs.what())` shall equal 0.

```
exception() noexcept;
```

3 *Effects:* Constructs an object of class `exception`.

```
exception(const exception& rhs) noexcept;
exception& operator=(const exception& rhs) noexcept;
```

4 *Effects:* Copies an `exception` object.

5 *Postcondition:* If `*this` and `rhs` both have dynamic type `exception` then `strcmp(what(), rhs.what())` shall equal 0.

```
virtual ~exception();
```

6 *Effects:* Destroys an object of class `exception`.

```
virtual const char* what() const noexcept;
```

7 *Returns:* An implementation-defined NTBS.

8 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4). The return value remains valid until the exception object from which it is obtained is destroyed or a non-`const` member function of the exception object is called.

18.8.2 Class `bad_exception` [bad.exception]

```
namespace std {
  class bad_exception : public exception {
  public:
    bad_exception() noexcept;
    bad_exception(const bad_exception&) noexcept;
    bad_exception& operator=(const bad_exception&) noexcept;
    virtual const char* what() const noexcept;
  };
}
```

1 The class `bad_exception` defines the type of objects thrown as described in (15.5.2).

```
bad_exception() noexcept;
```

2 *Effects:* Constructs an object of class `bad_exception`.

```
bad_exception(const bad_exception&) noexcept;
bad_exception& operator=(const bad_exception&) noexcept;
```

3 *Effects:* Copies an object of class `bad_exception`.

```
virtual const char* what() const noexcept;
```

4 *Returns:* An implementation-defined NTBS.

5 *Remarks:* The message may be a null-terminated multibyte string (17.5.2.1.4.2), suitable for conversion and display as a `wstring` (21.3, 22.4.1.4).

18.8.3 Abnormal termination [exception.terminate]

18.8.3.1 Type `terminate_handler` [terminate.handler]

```
typedef void (*terminate_handler)();
```

1 The type of a *handler function* to be called by `std::terminate()` when terminating exception processing.

2 *Required behavior:* A `terminate_handler` shall terminate execution of the program without returning to the caller.

3 *Default behavior:* The implementation's default `terminate_handler` calls `abort()`.

18.8.3.2 `set_terminate` [set.terminate]

```
terminate_handler set_terminate(terminate_handler f) noexcept;
```

1 *Effects:* Establishes the function designated by `f` as the current handler function for terminating exception processing.

2 *Remarks:* It is unspecified whether a null pointer value designates the default `terminate_handler`.

3 *Returns:* The previous `terminate_handler`.

18.8.3.3 `get_terminate` [get.terminate]

```
terminate_handler get_terminate() noexcept;
```

1 *Returns:* The current `terminate_handler`. [*Note:* This may be a null pointer value. — *end note*]

18.8.3.4 `terminate` [terminate]

```
[[noreturn]] void terminate() noexcept;
```

1 *Remarks:* Called by the implementation when exception handling must be abandoned for any of several reasons (15.5.1), in effect immediately after throwing the exception. May also be called directly by the program.

2 *Effects:* Calls the current `terminate_handler` function. [*Note:* A default `terminate_handler` is always considered a callable handler in this context. — *end note*]

18.8.4 `uncaught_exceptions` [uncaught.exceptions]

```
int uncaught_exceptions() noexcept;
```

1 *Returns:* The number of uncaught exceptions (15.5.3).

2 *Remarks:* When `uncaught_exceptions() > 0`, throwing an exception can result in a call of `std::terminate()` (15.5.1).

18.8.5 Exception propagation [propagation]

```
typedef unspecified  exception_ptr;
```

1 The type exception_ptr can be used to refer to an exception object.

2 `exception_ptr` shall satisfy the requirements of `NullablePointer` (17.6.3.3).

3 Two non-null values of type `exception_ptr` are equivalent and compare equal if and only if they refer to the same exception.

4 The default constructor of `exception_ptr` produces the null value of the type.

5 `exception_ptr` shall not be implicitly convertible to any arithmetic, enumeration, or pointer type.

6 [*Note:* An implementation might use a reference-counted smart pointer as `exception_ptr`. — *end note*]

7 For purposes of determining the presence of a data race, operations on `exception_ptr` objects shall access and modify only the `exception_ptr` objects themselves and not the exceptions they refer to. Use of `rethrow_exception` on `exception_ptr` objects that refer to the same exception object shall not introduce a data race. [*Note:* if `rethrow_exception` rethrows the same exception object (rather than a copy), concurrent access to that rethrown exception object may introduce a data race. Changes in the number of `exception_ptr` objects that refer to a particular exception do not introduce a data race. — *end note*]

```
exception_ptr current_exception() noexcept;
```

8 *Returns:* An `exception_ptr` object that refers to the currently handled exception (15.3) or a copy of the currently handled exception, or a null `exception_ptr` object if no exception is being handled. The referenced object shall remain valid at least as long as there is an `exception_ptr` object that refers to it. If the function needs to allocate memory and the attempt fails, it returns an `exception_ptr` object that refers to an instance of `bad_alloc`. It is unspecified whether the return values of two successive calls to `current_exception` refer to the same exception object. [*Note:* That is, it is unspecified whether `current_exception` creates a new copy each time it is called. — *end note*] If the attempt to copy the current exception object throws an exception, the function returns an `exception_ptr` object that refers to the thrown exception or, if this is not possible, to an instance of `bad_exception`. [*Note:* The copy constructor of the thrown exception may also fail, so the implementation is allowed to substitute a `bad_exception` object to avoid infinite recursion. — *end note*]

```
[[noreturn]] void rethrow_exception(exception_ptr p);
```

9 *Requires:* `p` shall not be a null pointer.

10 *Throws:* the exception object to which `p` refers.

```
template<class E> exception_ptr make_exception_ptr(E e) noexcept;
```

11 *Effects:* Creates an `exception_ptr` object that refers to a copy of `e`, as if

```
try {
  throw e;
} catch(...) {
  return current_exception();
}
```

12 [*Note:* This function is provided for convenience and efficiency reasons. — *end note*]

18.8.6 `nested_exception` [except.nested]

```
namespace std {
  class nested_exception {
  public:
    nested_exception() noexcept;
    nested_exception(const nested_exception&) noexcept = default;
    nested_exception& operator=(const nested_exception&) noexcept = default;
    virtual ~nested_exception() = default;

    // access functions
    [[noreturn]] void rethrow_nested() const;
    exception_ptr nested_ptr() const noexcept;
  };
```

```
  template<class T> [[noreturn]] void throw_with_nested(T&& t);
  template <class E> void rethrow_if_nested(const E& e);
}
```

1 The class `nested_exception` is designed for use as a mixin through multiple inheritance. It captures the currently handled exception and stores it for later use.

2 [*Note:* `nested_exception` has a virtual destructor to make it a polymorphic class. Its presence can be tested for with `dynamic_cast`. — *end note*]

```
nested_exception() noexcept;
```

3 *Effects:* The constructor calls `current_exception()` and stores the returned value.

```
[[noreturn]] void rethrow_nested() const;
```

4 *Effects:* If `nested_ptr()` returns a null pointer, the function calls `std::terminate()`. Otherwise, it throws the stored exception captured by `*this`.

```
exception_ptr nested_ptr() const noexcept;
```

5 *Returns:* The stored exception captured by this `nested_exception` object.

```
template <class T> [[noreturn]] void throw_with_nested(T&& t);
```

6 Let `U` be `remove_reference_t<T>`.

7 *Requires:* `U` shall be `CopyConstructible`.

8 *Throws:* if `U` is a non-union class type not derived from `nested_exception`, an exception of unspecified type that is publicly derived from both `U` and `nested_exception` and constructed from `std::forward<T>(t)`, otherwise `std::forward<T>(t)`.

```
template <class E> void rethrow_if_nested(const E& e);
```

9 *Effects:* If the dynamic type of `e` is publicly and unambiguously derived from `nested_exception`, calls `dynamic_cast<const nested_exception&>(e).rethrow_nested()`.

18.9 Initializer lists [support.initlist]

1 The header `<initializer_list>` defines a class template and several support functions related to list-initialization (see 8.5.4).

Header `<initializer_list>` synopsis

```
namespace std {
  template<class E> class initializer_list {
  public:
    typedef E value_type;
    typedef const E& reference;
    typedef const E& const_reference;
    typedef size_t size_type;

    typedef const E* iterator;
    typedef const E* const_iterator;

    constexpr initializer_list() noexcept;

    constexpr size_t size() const noexcept;       // number of elements
    constexpr const E* begin() const noexcept;    // first element
```

```
  constexpr const E* end() const noexcept;      // one past the last element
};

// 18.9.3 initializer list range access
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
}
```

2 An object of type `initializer_list<E>` provides access to an array of objects of type `const E`. [*Note:* A pair of pointers or a pointer plus a length would be obvious representations for `initializer_list`. `initializer_list` is used to implement initializer lists as specified in 8.5.4. Copying an initializer list does not copy the underlying elements. — *end note*]

3 If an explicit specialization or partial specialization of `initializer_list` is declared, the program is ill-formed.

18.9.1 Initializer list constructors [support.initlist.cons]

```
constexpr initializer_list() noexcept;
```

1 *Effects:* constructs an empty `initializer_list` object.

2 *Postcondition:* `size() == 0`

18.9.2 Initializer list access [support.initlist.access]

```
constexpr const E* begin() const noexcept;
```

1 *Returns:* A pointer to the beginning of the array. If `size() == 0` the values of `begin()` and `end()` are unspecified but they shall be identical.

```
constexpr const E* end() const noexcept;
```

2 *Returns:* `begin() + size()`

```
constexpr size_t size() const noexcept;
```

3 *Returns:* The number of elements in the array.

4 *Complexity:* Constant time.

18.9.3 Initializer list range access [support.initlist.range]

```
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
```

1 *Returns:* `il.begin()`.

```
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
```

2 *Returns:* `il.end()`.

18.10 Other runtime support [support.runtime]

1 Headers `<csetjmp>` (nonlocal jumps), `<csignal>` (signal handling), `<cstdalign> (alignment)`, `<cstdarg>` (variable arguments), `<cstdbool>` (`__bool_true_false_are_defined`). `<cstdlib>` (runtime environment `getenv()`, `system()`), and `<ctime>` (system clock `clock()`, `time()`) provide further compatibility with C code.

2 The contents of these headers are the same as the Standard C library headers `<setjmp.h>`, `<signal.h>`, `<stdalign.h>`, `<stdarg.h>`, `<stdbool.h>`, `<stdlib.h>`, and `<time.h>`, respectively, with the following changes:

3 The restrictions that ISO C places on the second parameter to the `va_start()` macro in header `<stdarg.h>` are different in this International Standard. The parameter `parmN` is the identifier of the rightmost parameter in the variable parameter list of the function definition (the one just before the `...`).[228] If the parameter `parmN` is of a reference type, or of a type that is not compatible with the type that results when passing an argument for which there is no parameter, the behavior is undefined.

See also: ISO C 4.8.1.1.

4 The function signature `longjmp(jmp_buf jbuf, int val)` has more restricted behavior in this International Standard. A `setjmp/longjmp` call pair has undefined behavior if replacing the `setjmp` and `longjmp` by `catch` and `throw` would invoke any non-trivial destructors for any automatic objects.

See also: ISO C 7.10.4, 7.8, 7.6, 7.12.

5 Calls to the function `getenv` shall not introduce a data race (17.6.5.9) provided that nothing modifies the environment. [*Note:* Calls to the POSIX functions `setenv` and `putenv` modify the environment. — *end note*]

6 A call to the `setlocale` function may introduce a data race with other calls to the `setlocale` function or with calls to functions that are affected by the current C locale. The implementation shall behave as if no library function other than `locale::global()` calls the `setlocale` function.

7 The header `<cstdalign>` and the header `<stdalign.h>` shall not define a macro named `alignas`.

8 The header `<cstdbool>` and the header `<stdbool.h>` shall not define macros named `bool`, `true`, or `false`.

9 A call to the function `signal` synchronizes with any resulting invocation of the signal handler so installed.

10 The common subset of the C and C++ languages consists of all declarations, definitions, and expressions that may appear in a well formed C++ program and also in a conforming C program. A POF ("plain old function") is a function that uses only features from this common subset, and that does not directly or indirectly use any function that is not a POF, except that it may use plain lock-free atomic operations. A *plain lock-free atomic operation* is an invocation of a function f from Clause 29, such that f is not a member function, and either f is the function `atomic_is_lock_free`, or for every atomic argument `A` passed to f, `atomic_is_lock_free(A)` yields `true`. All signal handlers shall have C linkage. The behavior of any function other than a POF used as a signal handler in a C++ program is implementation-defined.[229]

Table 34 — Header `<csetjmp>` synopsis

Type	Name(s)
Macro:	`setjmp`
Type:	`jmp_buf`
Function:	`longjmp`

Table 35 — Header `<csignal>` synopsis

Type	Name(s)			
Macros:	`SIGABRT`	`SIGILL`	`SIGSEGV`	`SIG_DFL`
	`SIG_IGN`	`SIGFPE`	`SIGINT`	`SIGTERM`	`SIG_ERR`
Type:	`sig_atomic_t`			
Functions:	`raise`	`signal`		

228) Note that `va_start` is required to work as specified even if unary `operator&` is overloaded for the type of `parmN`.
229) In particular, a signal handler using exception handling is very likely to have problems. Also, invoking `std::exit` may cause destruction of objects, including those of the standard library implementation, which, in general, yields undefined behavior in a signal handler (see 1.9).

Table 36 — Header `<cstdalign>` synopsis

Type	Name(s)
Macro:	`__alignas_is_defined`

Table 37 — Header `<cstdarg>` synopsis

Type	Name(s)
Macros:	`va_arg` `va_end` `va_start` `va_copy`
Type:	`va_list`

Table 38 — Header `<cstdbool>` synopsis

Type	Name(s)
Macro:	`__bool_true_false_are_defined`

Table 39 — Header `<cstdlib>` synopsis

Type	Name(s)
Functions:	`getenv` `system`

Table 40 — Header `<ctime>` synopsis

Type	Name(s)
Macro:	`CLOCKS_PER_SEC`
Type:	`clock_t`
Function:	`clock`

19 Diagnostics library [diagnostics]

19.1 General [diagnostics.general]

1 This Clause describes components that C++ programs may use to detect and report error conditions.

2 The following subclauses describe components for reporting several kinds of exceptional conditions, documenting program assertions, and a global variable for error number codes, as summarized in Table 41.

Table 41 — Diagnostics library summary

	Subclause	Header(s)
19.2	Exception classes	`<stdexcept>`
19.3	Assertions	`<cassert>`
19.4	Error numbers	`<cerrno>`
19.5	System error support	`<system_error>`

19.2 Exception classes [std.exceptions]

1 The Standard C++ library provides classes to be used to report certain errors (17.6.5.12) in C++ programs. In the error model reflected in these classes, errors are divided into two broad categories: *logic* errors and *runtime* errors.

2 The distinguishing characteristic of logic errors is that they are due to errors in the internal logic of the program. In theory, they are preventable.

3 By contrast, runtime errors are due to events beyond the scope of the program. They cannot be easily predicted in advance. The header `<stdexcept>` defines several types of predefined exceptions for reporting errors in a C++ program. These exceptions are related by inheritance.

Header `<stdexcept>` synopsis

```
namespace std {
  class logic_error;
    class domain_error;
    class invalid_argument;
    class length_error;
    class out_of_range;
  class runtime_error;
    class range_error;
    class overflow_error;
    class underflow_error;
}
```

19.2.1 Class `logic_error` [logic.error]

```
namespace std {
  class logic_error : public exception {
  public:
    explicit logic_error(const string& what_arg);
    explicit logic_error(const char* what_arg);
  };
}
```

1 The class `logic_error` defines the type of objects thrown as exceptions to report errors presumably detectable before the program executes, such as violations of logical preconditions or class invariants.

```
logic_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `logic_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
logic_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `logic_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.2 Class `domain_error` [domain.error]

```
namespace std {
  class domain_error : public logic_error {
  public:
    explicit domain_error(const string& what_arg);
    explicit domain_error(const char* what_arg);
  };
}
```

1 The class `domain_error` defines the type of objects thrown as exceptions by the implementation to report domain errors.

```
domain_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `domain_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
domain_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `domain_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.3 Class `invalid_argument` [invalid.argument]

```
namespace std {
  class invalid_argument : public logic_error {
  public:
    explicit invalid_argument(const string& what_arg);
    explicit invalid_argument(const char* what_arg);
  };
}
```

1 The class `invalid_argument` defines the type of objects thrown as exceptions to report an invalid argument.

```
invalid_argument(const string& what_arg);
```

2 *Effects:* Constructs an object of class `invalid_argument`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
invalid_argument(const char* what_arg);
```

4 *Effects:* Constructs an object of class `invalid_argument`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.4 Class `length_error` [length.error]

```
namespace std {
  class length_error : public logic_error {
  public:
    explicit length_error(const string& what_arg);
    explicit length_error(const char* what_arg);
  };
}
```

1 The class `length_error` defines the type of objects thrown as exceptions to report an attempt to produce an object whose length exceeds its maximum allowable size.

```
length_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `length_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
length_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `length_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.5 Class `out_of_range` [out.of.range]

```
namespace std {
  class out_of_range : public logic_error {
  public:
    explicit out_of_range(const string& what_arg);
    explicit out_of_range(const char* what_arg);
  };
}
```

1 The class `out_of_range` defines the type of objects thrown as exceptions to report an argument value not in its expected range.

```
out_of_range(const string& what_arg);
```

2 *Effects:* Constructs an object of class `out_of_range`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
out_of_range(const char* what_arg);
```

4 *Effects:* Constructs an object of class `out_of_range`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.6 Class `runtime_error` [runtime.error]

```
namespace std {
  class runtime_error : public exception {
  public:
    explicit runtime_error(const string& what_arg);
    explicit runtime_error(const char* what_arg);
  };
}
```

1 The class `runtime_error` defines the type of objects thrown as exceptions to report errors presumably detectable only when the program executes.

```
runtime_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `runtime_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
runtime_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `runtime_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.7 Class `range_error` [range.error]

```
namespace std {
  class range_error : public runtime_error {
  public:
    explicit range_error(const string& what_arg);
    explicit range_error(const char* what_arg);
  };
}
```

1 The class `range_error` defines the type of objects thrown as exceptions to report range errors in internal computations.

```
range_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `range_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
range_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `range_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.8 Class `overflow_error` [overflow.error]

```
namespace std {
  class overflow_error : public runtime_error {
  public:
    explicit overflow_error(const string& what_arg);
    explicit overflow_error(const char* what_arg);
  };
}
```

1 The class `overflow_error` defines the type of objects thrown as exceptions to report an arithmetic overflow error.

```
overflow_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `overflow_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
overflow_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `overflow_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.2.9 Class `underflow_error` [underflow.error]

```
namespace std {
  class underflow_error : public runtime_error {
  public:
    explicit underflow_error(const string& what_arg);
    explicit underflow_error(const char* what_arg);
  };
}
```

1 The class `underflow_error` defines the type of objects thrown as exceptions to report an arithmetic underflow error.

```
underflow_error(const string& what_arg);
```

2 *Effects:* Constructs an object of class `underflow_error`.

3 *Postcondition:* `strcmp(what(), what_arg.c_str()) == 0`.

```
underflow_error(const char* what_arg);
```

4 *Effects:* Constructs an object of class `underflow_error`.

5 *Postcondition:* `strcmp(what(), what_arg) == 0`.

19.3 Assertions [assertions]

1 The header `<cassert>`, described in (Table 42), provides a macro for documenting C++ program assertions and a mechanism for disabling the assertion checks.

Table 42 — Header `<cassert>` synopsis

Type	Name(s)
Macro:	`assert`

2 The contents are the same as the Standard C library header `<assert.h>`.

See also: ISO C 7.2.

19.4 Error numbers [errno]

1 The header `<cerrno>` is described in Table 43. Its contents are the same as the POSIX header `<errno.h>`, except that `errno` shall be defined as a macro. [*Note:* The intent is to remain in close alignment with the POSIX standard. — *end note*] A separate `errno` value shall be provided for each thread.

19.5 System error support [syserr]

1 This subclause describes components that the standard library and C++ programs may use to report error conditions originating from the operating system or other low-level application program interfaces.

2 Components described in this subclause shall not change the value of `errno` (19.4). Implementations should leave the error states provided by other libraries unchanged.

Header `<system_error>` synopsis

```
namespace std {
  class error_category;
  const error_category& generic_category() noexcept;
  const error_category& system_category() noexcept;

  class error_code;
```

Table 43 — Header <cerrno> synopsis

Type	Name(s)				
Macros:	ECONNREFUSED	EIO	ENODEV	ENOTEMPTY	ERANGE
E2BIG	ECONNRESET	EISCONN	ENOENT	ENOTRECOVERABLE	EROFS
EACCES	EDEADLK	EISDIR	ENOEXEC	ENOTSOCK	ESPIPE
EADDRINUSE	EDESTADDRREQ	ELOOP	ENOLCK	ENOTSUP	ESRCH
EADDRNOTAVAIL	EDOM	EMFILE	ENOLINK	ENOTTY	ETIME
EAFNOSUPPORT	EEXIST	EMLINK	ENOMEM	ENXIO	ETIMEDOUT
EAGAIN	EFAULT	EMSGSIZE	ENOMSG	EOPNOTSUPP	ETXTBSY
EALREADY	EFBIG	ENAMETOOLONG	ENOPROTOOPT	EOVERFLOW	EWOULDBLOCK
EBADF	EHOSTUNREACH	ENETDOWN	ENOSPC	EOWNERDEAD	EXDEV
EBADMSG	EIDRM	ENETRESET	ENOSR	EPERM	errno
EBUSY	EILSEQ	ENETUNREACH	ENOSTR	EPIPE	
ECANCELED	EINPROGRESS	ENFILE	ENOSYS	EPROTO	
ECHILD	EINTR	ENOBUFS	ENOTCONN	EPROTONOSUPPORT	
ECONNABORTED	EINVAL	ENODATA	ENOTDIR	EPROTOTYPE	

```
class error_condition;
class system_error;

template <class T>
struct is_error_code_enum : public false_type {};

template <class T>
struct is_error_condition_enum : public false_type {};

enum class errc {
  address_family_not_supported,       // EAFNOSUPPORT
  address_in_use,                     // EADDRINUSE
  address_not_available,              // EADDRNOTAVAIL
  already_connected,                  // EISCONN
  argument_list_too_long,             // E2BIG
  argument_out_of_domain,             // EDOM
  bad_address,                        // EFAULT
  bad_file_descriptor,                // EBADF
  bad_message,                        // EBADMSG
  broken_pipe,                        // EPIPE
  connection_aborted,                 // ECONNABORTED
  connection_already_in_progress,     // EALREADY
  connection_refused,                 // ECONNREFUSED
  connection_reset,                   // ECONNRESET
  cross_device_link,                  // EXDEV
  destination_address_required,       // EDESTADDRREQ
  device_or_resource_busy,            // EBUSY
  directory_not_empty,                // ENOTEMPTY
  executable_format_error,            // ENOEXEC
  file_exists,                        // EEXIST
  file_too_large,                     // EFBIG
  filename_too_long,                  // ENAMETOOLONG
  function_not_supported,             // ENOSYS
  host_unreachable,                   // EHOSTUNREACH
  identifier_removed,                 // EIDRM
```

```
  illegal_byte_sequence,              // EILSEQ
  inappropriate_io_control_operation, // ENOTTY
  interrupted,                        // EINTR
  invalid_argument,                   // EINVAL
  invalid_seek,                       // ESPIPE
  io_error,                           // EIO
  is_a_directory,                     // EISDIR
  message_size,                       // EMSGSIZE
  network_down,                       // ENETDOWN
  network_reset,                      // ENETRESET
  network_unreachable,                // ENETUNREACH
  no_buffer_space,                    // ENOBUFS
  no_child_process,                   // ECHILD
  no_link,                            // ENOLINK
  no_lock_available,                  // ENOLCK
  no_message_available,               // ENODATA
  no_message,                         // ENOMSG
  no_protocol_option,                 // ENOPROTOOPT
  no_space_on_device,                 // ENOSPC
  no_stream_resources,                // ENOSR
  no_such_device_or_address,          // ENXIO
  no_such_device,                     // ENODEV
  no_such_file_or_directory,          // ENOENT
  no_such_process,                    // ESRCH
  not_a_directory,                    // ENOTDIR
  not_a_socket,                       // ENOTSOCK
  not_a_stream,                       // ENOSTR
  not_connected,                      // ENOTCONN
  not_enough_memory,                  // ENOMEM
  not_supported,                      // ENOTSUP
  operation_canceled,                 // ECANCELED
  operation_in_progress,              // EINPROGRESS
  operation_not_permitted,            // EPERM
  operation_not_supported,            // EOPNOTSUPP
  operation_would_block,              // EWOULDBLOCK
  owner_dead,                         // EOWNERDEAD
  permission_denied,                  // EACCES
  protocol_error,                     // EPROTO
  protocol_not_supported,             // EPROTONOSUPPORT
  read_only_file_system,              // EROFS
  resource_deadlock_would_occur,      // EDEADLK
  resource_unavailable_try_again,     // EAGAIN
  result_out_of_range,                // ERANGE
  state_not_recoverable,              // ENOTRECOVERABLE
  stream_timeout,                     // ETIME
  text_file_busy,                     // ETXTBSY
  timed_out,                          // ETIMEDOUT
  too_many_files_open_in_system,      // ENFILE
  too_many_files_open,                // EMFILE
  too_many_links,                     // EMLINK
  too_many_symbolic_link_levels,      // ELOOP
  value_too_large,                    // EOVERFLOW
  wrong_protocol_type,                // EPROTOTYPE
};
```

```
  template <> struct is_error_condition_enum<errc> : true_type { };

  error_code make_error_code(errc e) noexcept;
  error_condition make_error_condition(errc e) noexcept;

  // 19.5.4 Comparison operators:
  bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
  bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
  bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
  bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
  bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
  bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
  bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
  bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;

  // 19.5.5 Hash support
  template <class T> struct hash;
  template <> struct hash<error_code>;
} // namespace std
```

3 The value of each `enum errc` constant shall be the same as the value of the `<cerrno>` macro shown in the above synopsis. Whether or not the `<system_error>` implementation exposes the `<cerrno>` macros is unspecified.

4 The `is_error_code_enum` and `is_error_condition_enum` may be specialized for user-defined types to indicate that such types are eligible for `class error_code` and `class error_condition` automatic conversions, respectively.

19.5.1 Class `error_category` [syserr.errcat]

19.5.1.1 Class `error_category` overview [syserr.errcat.overview]

1 The class `error_category` serves as a base class for types used to identify the source and encoding of a particular category of error code. Classes may be derived from `error_category` to support categories of errors in addition to those defined in this International Standard. Such classes shall behave as specified in this subclause. [*Note:* `error_category` objects are passed by reference, and two such objects are equal if they have the same address. This means that applications using custom `error_category` types should create a single object of each such type. — *end note*]

```
namespace std {
  class error_category {
  public:
    constexpr error_category() noexcept;
    virtual ~error_category();
    error_category(const error_category&) = delete;
    error_category& operator=(const error_category&) = delete;
    virtual const char* name() const noexcept = 0;
    virtual error_condition default_error_condition(int ev) const noexcept;
    virtual bool equivalent(int code, const error_condition& condition) const noexcept;
    virtual bool equivalent(const error_code& code, int condition) const noexcept;
    virtual string message(int ev) const = 0;

    bool operator==(const error_category& rhs) const noexcept;
    bool operator!=(const error_category& rhs) const noexcept;
    bool operator<(const error_category& rhs) const noexcept;
  };
```

```
  const error_category& generic_category() noexcept;
  const error_category& system_category() noexcept;

}   // namespace std
```

19.5.1.2 Class error_category virtual members [syserr.errcat.virtuals]

```
virtual ~error_category();
```

1 *Effects:* Destroys an object of class `error_category`.

```
virtual const char* name() const noexcept = 0;
```

2 *Returns:* A string naming the error category.

```
virtual error_condition default_error_condition(int ev) const noexcept;
```

3 *Returns:* `error_condition(ev, *this)`.

```
virtual bool equivalent(int code, const error_condition& condition) const noexcept;
```

4 *Returns:* `default_error_condition(code) == condition`.

```
virtual bool equivalent(const error_code& code, int condition) const noexcept;
```

5 *Returns:* `*this == code.category() && code.value() == condition`.

```
virtual string message(int ev) const = 0;
```

6 *Returns:* A string that describes the error condition denoted by `ev`.

19.5.1.3 Class error_category non-virtual members [syserr.errcat.nonvirtuals]

```
constexpr error_category() noexcept;
```

1 *Effects:* Constructs an object of class `error_category`.

```
bool operator==(const error_category& rhs) const noexcept;
```

2 *Returns:* `this == &rhs`.

```
bool operator!=(const error_category& rhs) const noexcept;
```

3 *Returns:* `!(*this == rhs)`.

```
bool operator<(const error_category& rhs) const noexcept;
```

4 *Returns:* `less<const error_category*>()(this, &rhs)`.

[*Note:* `less` (20.9.6) provides a total ordering for pointers. — *end note*]

19.5.1.4 Program defined classes derived from error_category [syserr.errcat.derived]

```
virtual const char* name() const noexcept = 0;
```

1 *Returns:* A string naming the error category.

```
virtual error_condition default_error_condition(int ev) const noexcept;
```

2 *Returns:* An object of type `error_condition` that corresponds to `ev`.

```
virtual bool equivalent(int code, const error_condition& condition) const noexcept;
```

3 *Returns:* `true` if, for the category of error represented by `*this`, `code` is considered equivalent to `condition`; otherwise, `false`.

```
virtual bool equivalent(const error_code& code, int condition) const noexcept;
```

4 *Returns:* `true` if, for the category of error represented by `*this`, `code` is considered equivalent to `condition`; otherwise, `false`.

19.5.1.5 Error category objects [syserr.errcat.objects]

```
const error_category& generic_category() noexcept;
```

1 *Returns:* A reference to an object of a type derived from class `error_category`. All calls to this function shall return references to the same object.

2 *Remarks:* The object's `default_error_condition` and `equivalent` virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function shall return a pointer to the string `"generic"`.

```
const error_category& system_category() noexcept;
```

3 *Returns:* A reference to an object of a type derived from class `error_category`. All calls to this function shall return references to the same object.

4 *Remarks:* The object's `equivalent` virtual functions shall behave as specified for class `error_category`. The object's `name` virtual function shall return a pointer to the string `"system"`. The object's `default_error_condition` virtual function shall behave as follows:

If the argument `ev` corresponds to a POSIX `errno` value `posv`, the function shall return `error_condition(posv, generic_category())`. Otherwise, the function shall return `error_condition(ev, system_category())`. What constitutes correspondence for any given operating system is unspecified. [*Note:* The number of potential system error codes is large and unbounded, and some may not correspond to any POSIX `errno` value. Thus implementations are given latitude in determining correspondence. — *end note*]

19.5.2 Class `error_code` [syserr.errcode]

19.5.2.1 Class `error_code` overview [syserr.errcode.overview]

1 The class `error_code` describes an object used to hold error code values, such as those originating from the operating system or other low-level application program interfaces. [*Note:* Class `error_code` is an adjunct to error reporting by exception. — *end note*]

```
namespace std {
  class error_code {
  public:
    // 19.5.2.2 constructors:
    error_code() noexcept;
    error_code(int val, const error_category& cat) noexcept;
    template <class ErrorCodeEnum>
      error_code(ErrorCodeEnum e) noexcept;

    // 19.5.2.3 modifiers:
    void assign(int val, const error_category& cat) noexcept;
    template <class ErrorCodeEnum>
        error_code& operator=(ErrorCodeEnum e) noexcept;
    void clear() noexcept;

    // 19.5.2.4 observers:
```

```
    int value() const noexcept;
    const error_category& category() const noexcept;
    error_condition default_error_condition() const noexcept;
    string message() const;
    explicit operator bool() const noexcept;

  private:
    int val_;                   // exposition only
    const error_category* cat_; // exposition only
  };

  // 19.5.2.5 non-member functions:
  error_code make_error_code(errc e) noexcept;
  bool operator<(const error_code& lhs, const error_code& rhs) noexcept;

  template <class charT, class traits>
    basic_ostream<charT,traits>&
      operator<<(basic_ostream<charT,traits>& os, const error_code& ec);
}   // namespace std
```

19.5.2.2 Class `error_code` constructors [syserr.errcode.constructors]

```
error_code() noexcept;
```

1 *Effects:* Constructs an object of type `error_code`.

2 *Postconditions:* `val_ == 0` and `cat_ == &system_category()`.

```
error_code(int val, const error_category& cat) noexcept;
```

3 *Effects:* Constructs an object of type `error_code`.

4 *Postconditions:* `val_ == val` and `cat_ == &cat`.

```
template <class ErrorCodeEnum>
  error_code(ErrorCodeEnum e) noexcept;
```

5 *Effects:* Constructs an object of type `error_code`.

6 *Postconditions:* `*this == make_error_code(e)`.

7 *Remarks:* This constructor shall not participate in overload resolution unless `is_error_code_enum<ErrorCodeEnum>::value` is `true`.

19.5.2.3 Class `error_code` modifiers [syserr.errcode.modifiers]

```
void assign(int val, const error_category& cat) noexcept;
```

1 *Postconditions:* `val_ == val` and `cat_ == &cat`.

```
template <class ErrorCodeEnum>
    error_code& operator=(ErrorCodeEnum e) noexcept;
```

2 *Postconditions:* `*this == make_error_code(e)`.

3 *Returns:* `*this`.

4 *Remarks:* This operator shall not participate in overload resolution unless `is_error_code_enum<ErrorCodeEnum>::value` is `true`.

```
void clear() noexcept;
```

5 *Postconditions:* `value() == 0` and `category() == system_category()`.

19.5.2.4 Class `error_code` observers [syserr.errcode.observers]

```
int value() const noexcept;
```

1 *Returns:* `val_`.

```
const error_category& category() const noexcept;
```

2 *Returns:* `*cat_`.

```
error_condition default_error_condition() const noexcept;
```

3 *Returns:* `category().default_error_condition(value())`.

```
string message() const;
```

4 *Returns:* `category().message(value())`.

```
explicit operator bool() const noexcept;
```

5 *Returns:* `value() != 0`.

19.5.2.5 Class `error_code` non-member functions [syserr.errcode.nonmembers]

```
error_code make_error_code(errc e) noexcept;
```

1 *Returns:* `error_code(static_cast<int>(e), generic_category())`.

```
bool operator<(const error_code& lhs, const error_code& rhs) noexcept;
```

2 *Returns:* `lhs.category() < rhs.category() || lhs.category() == rhs.category() && lhs.value() < rhs.value()`.

```
template <class charT, class traits>
  basic_ostream<charT,traits>&
    operator<<(basic_ostream<charT,traits>& os, const error_code& ec);
```

3 *Effects:* `os << ec.category().name() << ':' << ec.value()`.

19.5.3 Class `error_condition` [syserr.errcondition]

19.5.3.1 Class `error_condition` overview [syserr.errcondition.overview]

1 The class `error_condition` describes an object used to hold values identifying error conditions. [*Note:* `error_condition` values are portable abstractions, while `error_code` values (19.5.2) are implementation specific. — *end note*]

```
namespace std {
  class error_condition {
  public:
    // 19.5.3.2 constructors:
    error_condition() noexcept;
    error_condition(int val, const error_category& cat) noexcept;
    template <class ErrorConditionEnum>
      error_condition(ErrorConditionEnum e) noexcept;

    // 19.5.3.3 modifiers:
    void assign(int val, const error_category& cat) noexcept;
    template<class ErrorConditionEnum>
        error_condition& operator=(ErrorConditionEnum e) noexcept;
    void clear() noexcept;
```

```
    // 19.5.3.4 observers:
    int value() const noexcept;
    const error_category& category() const noexcept;
    string message() const;
    explicit operator bool() const noexcept;

  private:
    int val_;                   // exposition only
    const error_category* cat_; // exposition only
  };

  // 19.5.3.5 non-member functions:
  bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;
} // namespace std
```

19.5.3.2 Class `error_condition` constructors [syserr.errcondition.constructors]

```
error_condition() noexcept;
```

1 *Effects:* Constructs an object of type `error_condition`.

2 *Postconditions:* `val_ == 0` and `cat_ == &generic_category()`.

```
error_condition(int val, const error_category& cat) noexcept;
```

3 *Effects:* Constructs an object of type `error_condition`.

4 *Postconditions:* `val_ == val` and `cat_ == &cat`.

```
template <class ErrorConditionEnum>
  error_condition(ErrorConditionEnum e) noexcept;
```

5 *Effects:* Constructs an object of type `error_condition`.

6 *Postcondition:* `*this == make_error_condition(e)`.

7 *Remarks:* This constructor shall not participate in overload resolution unless `is_error_condition_enum<ErrorConditionEnum>::value` is `true`.

19.5.3.3 Class `error_condition` modifiers [syserr.errcondition.modifiers]

```
void assign(int val, const error_category& cat) noexcept;
```

1 *Postconditions:* `val_ == val` and `cat_ == &cat`.

```
template <class ErrorConditionEnum>
    error_condition& operator=(ErrorConditionEnum e) noexcept;
```

2 *Postcondition:* `*this == make_error_condition(e)`.

3 *Returns:* `*this`.

4 *Remarks:* This operator shall not participate in overload resolution unless `is_error_condition_enum<ErrorConditionEnum>::value` is `true`.

```
void clear() noexcept;
```

Postconditions: `value() == 0` and `category() == generic_category()`.

19.5.3.4 Class error_condition observers [syserr.errcondition.observers]

```
int value() const noexcept;
```

1 *Returns:* val_.

```
const error_category& category() const noexcept;
```

2 *Returns:* *cat_.

```
string message() const;
```

3 *Returns:* category().message(value()).

```
explicit operator bool() const noexcept;
```

4 *Returns:* value() != 0.

19.5.3.5 Class error_condition non-member functions [syserr.errcondition.nonmembers]

```
error_condition make_error_condition(errc e) noexcept;
```

Returns: error_condition(static_cast<int>(e), generic_category()).

```
bool operator<(const error_condition& lhs, const error_condition& rhs) noexcept;
```

1 *Returns:* lhs.category() < rhs.category() || lhs.category() == rhs.category() && lhs.value() < rhs.value().

19.5.4 Comparison operators [syserr.compare]

```
bool operator==(const error_code& lhs, const error_code& rhs) noexcept;
```

1 *Returns:* lhs.category() == rhs.category() && lhs.value() == rhs.value().

```
bool operator==(const error_code& lhs, const error_condition& rhs) noexcept;
```

2 *Returns:* lhs.category().equivalent(lhs.value(), rhs) || rhs.category().equivalent(lhs, rhs.value()).

```
bool operator==(const error_condition& lhs, const error_code& rhs) noexcept;
```

3 *Returns:* rhs.category().equivalent(rhs.value(), lhs) || lhs.category().equivalent(rhs, lhs.value()).

```
bool operator==(const error_condition& lhs, const error_condition& rhs) noexcept;
```

4 *Returns:* lhs.category() == rhs.category() && lhs.value() == rhs.value().

```
bool operator!=(const error_code& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_code& lhs, const error_condition& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_code& rhs) noexcept;
bool operator!=(const error_condition& lhs, const error_condition& rhs) noexcept;
```

5 *Returns:* !(lhs == rhs).

19.5.5 System error hash support [syserr.hash]

```
template <> struct hash<error_code>;
```

1 The template specialization shall meet the requirements of class template hash (20.9.13).

19.5.6 Class `system_error` [syserr.syserr]

19.5.6.1 Class `system_error` overview [syserr.syserr.overview]

1 The class `system_error` describes an exception object used to report error conditions that have an associated error code. Such error conditions typically originate from the operating system or other low-level application program interfaces.

2 [*Note:* If an error represents an out-of-memory condition, implementations are encouraged to throw an exception object of type `bad_alloc` 18.6.2.1 rather than `system_error`. — *end note*]

```
namespace std {
  class system_error : public runtime_error {
  public:
    system_error(error_code ec, const string& what_arg);
    system_error(error_code ec, const char* what_arg);
    system_error(error_code ec);
    system_error(int ev, const error_category& ecat,
        const string& what_arg);
    system_error(int ev, const error_category& ecat,
        const char* what_arg);
    system_error(int ev, const error_category& ecat);
    const error_code& code() const noexcept;
    const char* what() const noexcept;
  };
}   // namespace std
```

19.5.6.2 Class `system_error` members [syserr.syserr.members]

```
system_error(error_code ec, const string& what_arg);
```

1 *Effects:* Constructs an object of class `system_error`.

2 *Postconditions:* `code() == ec`.

`string(what()).find(what_arg) != string::npos`.

```
system_error(error_code ec, const char* what_arg);
```

3 *Effects:* Constructs an object of class `system_error`.

4 *Postconditions:* `code() == ec`.

`string(what()).find(what_arg) != string::npos`.

```
system_error(error_code ec);
```

5 *Effects:* Constructs an object of class `system_error`.

6 *Postconditions:* `code() == ec`.

```
system_error(int ev, const error_category& ecat,
  const string& what_arg);
```

7 *Effects:* Constructs an object of class `system_error`.

8 *Postconditions:* `code() == error_code(ev, ecat)`.

`string(what()).find(what_arg) != string::npos`.

```
system_error(int ev, const error_category& ecat,
  const char* what_arg);
```

9 *Effects:* Constructs an object of class `system_error`.

10 *Postconditions:* `code() == error_code(ev, ecat)`.

`string(what()).find(what_arg) != string::npos`.

```
system_error(int ev, const error_category& ecat);
```

11 *Effects:* Constructs an object of class `system_error`.

12 *Postconditions:* `code() == error_code(ev, ecat)`.

```
const error_code& code() const noexcept;
```

13 *Returns:* `ec` or `error_code(ev, ecat)`, from the constructor, as appropriate.

```
const char* what() const noexcept;
```

14 *Returns:* An NTBS incorporating the arguments supplied in the constructor.

[*Note:* The returned NTBS might be the contents of `what_arg + ":  " + code.message()`. — *end note*]

20 General utilities library [utilities]

20.1 General [utilities.general]

1 This Clause describes utilities that are generally useful in C++ programs; some of these utilities are used by other elements of the C++ standard library. These utilities are summarized in Table 44.

Table 44 — General utilities library summary

Subclause		Header(s)
20.2	Utility components	`<utility>`
20.3	Pairs	`<utility>`
20.4	Tuples	`<tuple>`
20.5	Compile-time integer sequences	`<utility>`
20.6	Fixed-size sequences of bits	`<bitset>`

20.7	Memory	`<memory>`
		`<cstdlib>`
		`<cstring>`

20.8	Smart pointers	`<memory>`
20.9	Function objects	`<functional>`
20.10	Type traits	`<type_traits>`
20.11	Compile-time rational arithmetic	`<ratio>`
20.12	Time utilities	`<chrono>` `<ctime>`
20.13	Scoped allocators	`<scoped_allocator>`
20.14	Type indexes	`<typeindex>`

20.2 Utility components [utility]

1 This subclause contains some basic function and class templates that are used throughout the rest of the library.

Header `<utility>` synopsis

2 The header `<utility>` defines several types and function templates that are described in this Clause. It also defines the template `pair` and various function templates that operate on `pair` objects.

```
#include <initializer_list>

namespace std {
  // 20.2.1, operators:
  namespace rel_ops {
    template<class T> bool operator!=(const T&, const T&);
    template<class T> bool operator> (const T&, const T&);
    template<class T> bool operator<=(const T&, const T&);
    template<class T> bool operator>=(const T&, const T&);
  }

  // 20.2.2, swap:
  template<class T> void swap(T& a, T& b) noexcept(see below);
  template <class T, size_t N> void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));
```

```
// 20.2.3, exchange:
template <class T, class U=T> T exchange(T& obj, U&& new_val);

// 20.2.4, forward/move:
template <class T>
  constexpr T&& forward(remove_reference_t<T>& t) noexcept;
template <class T>
  constexpr T&& forward(remove_reference_t<T>&& t) noexcept;
template <class T>
  constexpr remove_reference_t<T>&& move(T&&) noexcept;
template <class T>
  constexpr conditional_t<
  !is_nothrow_move_constructible<T>::value && is_copy_constructible<T>::value,
  const T&, T&&> move_if_noexcept(T& x) noexcept;

// 20.2.5, declval:
template <class T>
  add_rvalue_reference_t<T> declval() noexcept;  // as unevaluated operand

// 20.3, pairs:
template <class T1, class T2> struct pair;

// 20.3.3, pair specialized algorithms:
template <class T1, class T2>
  constexpr bool operator==(const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  constexpr bool operator< (const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  constexpr bool operator!=(const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  constexpr bool operator> (const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  constexpr bool operator>=(const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  constexpr bool operator<=(const pair<T1,T2>&, const pair<T1,T2>&);
template <class T1, class T2>
  void swap(pair<T1,T2>& x, pair<T1,T2>& y) noexcept(noexcept(x.swap(y)));
template <class T1, class T2>
  constexpr see below make_pair(T1&&, T2&&);

// 20.3.4, tuple-like access to pair:
template <class T> class tuple_size;
template <size_t I, class T> class tuple_element;

template <class T1, class T2> struct tuple_size<pair<T1, T2> >;
template <class T1, class T2> struct tuple_element<0, pair<T1, T2> >;
template <class T1, class T2> struct tuple_element<1, pair<T1, T2> >;

template<size_t I, class T1, class T2>
  constexpr tuple_element_t<I, pair<T1, T2>>&
    get(pair<T1, T2>&) noexcept;
template<size_t I, class T1, class T2>
  constexpr tuple_element_t<I, pair<T1, T2>>&&
    get(pair<T1, T2>&&) noexcept;
```

```
    template<size_t I, class T1, class T2>
      constexpr const tuple_element_t<I, pair<T1, T2>>&
        get(const pair<T1, T2>&) noexcept;
    template <class T, class U>
      constexpr T& get(pair<T, U>& p) noexcept;
    template <class T, class U>
      constexpr const T& get(const pair<T, U>& p) noexcept;
    template <class T, class U>
      constexpr T&& get(pair<T, U>&& p) noexcept;
    template <class T, class U>
      constexpr T& get(pair<U, T>& p) noexcept;
    template <class T, class U>
      constexpr const T& get(const pair<U, T>& p) noexcept;
    template <class T, class U>
      constexpr T&& get(pair<U, T>&& p) noexcept;

    // 20.3.5, pair piecewise construction
    struct piecewise_construct_t { };
    constexpr piecewise_construct_t piecewise_construct{};
    template <class... Types> class tuple;  // defined in <tuple>

    // 20.5, Compile-time integer sequences
    template<class T, T...> struct integer_sequence;
    template<size_t... I>
      using index_sequence = integer_sequence<size_t, I...>;

    template<class T, T N>
      using make_integer_sequence = integer_sequence<T, see below>;
    template<size_t N>
      using make_index_sequence = make_integer_sequence<size_t, N>;

    template<class... T>
      using index_sequence_for = make_index_sequence<sizeof...(T)>;
  }
```

20.2.1 Operators [operators]

1 To avoid redundant definitions of `operator!=` out of `operator==` and operators `>`, `<=`, and `>=` out of `operator<`, the library provides the following:

```
template <class T> bool operator!=(const T& x, const T& y);
```

2 *Requires:* Type `T` is `EqualityComparable` (Table 17).

3 *Returns:* `!(x == y)`.

```
template <class T> bool operator>(const T& x, const T& y);
```

4 *Requires:* Type `T` is `LessThanComparable` (Table 18).

5 *Returns:* `y < x`.

```
template <class T> bool operator<=(const T& x, const T& y);
```

6 *Requires:* Type `T` is `LessThanComparable` (Table 18).

7 *Returns:* `!(y < x)`.

```
template <class T> bool operator>=(const T& x, const T& y);
```

8 *Requires:* Type `T` is `LessThanComparable` (Table 18).

9 *Returns:* `!(x < y)`.

10 In this library, whenever a declaration is provided for an `operator!=`, `operator>`, `operator>=`, or `operator<=`, and requirements and semantics are not explicitly provided, the requirements and semantics are as specified in this Clause.

20.2.2 swap [utility.swap]

```
template<class T> void swap(T& a, T& b) noexcept(see below);
```

1 *Remark:* The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_constructible<T>::value &&
is_nothrow_move_assignable<T>::value
```

2 *Requires:* Type `T` shall be `MoveConstructible` (Table 20) and `MoveAssignable` (Table 22).

3 *Effects:* Exchanges values stored in two locations.

```
template<class T, size_t N>
  void swap(T (&a)[N], T (&b)[N]) noexcept(noexcept(swap(*a, *b)));
```

4 *Requires:* `a[i]` shall be swappable with (17.6.3.2) `b[i]` for all `i` in the range `[0,N)`.

5 *Effects:* `swap_ranges(a, a + N, b)`

20.2.3 exchange [utility.exchange]

```
template <class T, class U=T> T exchange(T& obj, U&& new_val);
```

1 *Effects:* Equivalent to:

```
T old_val = std::move(obj);
obj = std::forward<U>(new_val);
return old_val;
```

20.2.4 forward/move helpers [forward]

1 The library provides templated helper functions to simplify applying move semantics to an lvalue and to simplify the implementation of forwarding functions.

```
template <class T> constexpr T&& forward(remove_reference_t<T>& t) noexcept;
template <class T> constexpr T&& forward(remove_reference_t<T>&& t) noexcept;
```

2 *Returns:* `static_cast<T&&>(t)`.

3 *Remark:* If the second form is instantiated with an lvalue reference type, the program is ill-formed.

4 [*Example:*

```
template <class T, class A1, class A2>
shared_ptr<T> factory(A1&& a1, A2&& a2) {
  return shared_ptr<T>(new T(std::forward<A1>(a1), std::forward<A2>(a2)));
}

struct A {
  A(int&, const double&);
};

void g() {
```

```
  shared_ptr<A> sp1 = factory<A>(2, 1.414); // error: 2 will not bind to int&
  int i = 2;
  shared_ptr<A> sp2 = factory<A>(i, 1.414); // OK
}
```

5 In the first call to `factory`, `A1` is deduced as `int`, so 2 is forwarded to `A`'s constructor as an rvalue. In the second call to `factory`, `A1` is deduced as `int&`, so `i` is forwarded to `A`'s constructor as an lvalue. In both cases, `A2` is deduced as `double`, so 1.414 is forwarded to `A`'s constructor as an rvalue.

— *end example*]

```
template <class T> constexpr remove_reference_t<T>&& move(T&& t) noexcept;
```

6 *Returns:* `static_cast<remove_reference_t<T>&&>(t)`.

7 [*Example:*

```
template <class T, class A1>
shared_ptr<T> factory(A1&& a1) {
  return shared_ptr<T>(new T(std::forward<A1>(a1)));
}

struct A {
  A();
  A(const A&);  // copies from lvalues
  A(A&&);       // moves from rvalues
};

void g() {
  A a;
  shared_ptr<A> sp1 = factory<A>(a);              // "a" binds to A(const A&)
  shared_ptr<A> sp1 = factory<A>(std::move(a));   // "a" binds to A(A&&)
}
```

8 In the first call to `factory`, `A1` is deduced as `A&`, so `a` is forwarded as a non-const lvalue. This binds to the constructor `A(const A&)`, which copies the value from `a`. In the second call to `factory`, because of the call `std::move(a)`, `A1` is deduced as `A`, so `a` is forwarded as an rvalue. This binds to the constructor `A(A&&)`, which moves the value from `a`.

— *end example*]

```
template <class T> constexpr conditional_t<
  !is_nothrow_move_constructible<T>::value && is_copy_constructible<T>::value,
  const T&, T&&> move_if_noexcept(T& x) noexcept;
```

9 *Returns:* `std::move(x)`

20.2.5 Function template `declval` [declval]

1 The library provides the function template `declval` to simplify the definition of expressions which occur as unevaluated operands (Clause 5).

```
template <class T>
  add_rvalue_reference_t<T> declval() noexcept;  // as unevaluated operand
```

2 *Remarks:* If this function is odr-used (3.2), the program is ill-formed.

3 *Remarks:* The template parameter `T` of `declval` may be an incomplete type.

[*Example:*

```
template <class To, class From>
  decltype(static_cast<To>(declval<From>())) convert(From&&);
```

declares a function template `convert` which only participates in overloading if the type `From` can be explicitly converted to type `To`. For another example see class template `common_type` (20.10.7.6). — *end example*]

20.3 Pairs [pairs]

20.3.1 In general [pairs.general]

1 The library provides a template for heterogeneous pairs of values. The library also provides a matching function template to simplify their construction and several templates that provide access to `pair` objects as if they were `tuple` objects (see 20.4.2.5 and 20.4.2.6).

20.3.2 Class template pair [pairs.pair]

```
// defined in header <utility>

namespace std {
  template <class T1, class T2>
  struct pair {
    typedef T1 first_type;
    typedef T2 second_type;

    T1 first;
    T2 second;
    pair(const pair&) = default;
    pair(pair&&) = default;
    constexpr pair();
    EXPLICIT constexpr pair(const T1& x, const T2& y);
    template<class U, class V> EXPLICIT constexpr pair(U&& x, V&& y);
    template<class U, class V> EXPLICIT constexpr pair(const pair<U, V>& p);
    template<class U, class V> EXPLICIT constexpr pair(pair<U, V>&& p);
    template <class... Args1, class... Args2>
      pair(piecewise_construct_t,
           tuple<Args1...> first_args, tuple<Args2...> second_args);

    pair& operator=(const pair& p);
    template<class U, class V> pair& operator=(const pair<U, V>& p);
    pair& operator=(pair&& p) noexcept(see below);
    template<class U, class V> pair& operator=(pair<U, V>&& p);

    void swap(pair& p) noexcept(see below);
  };
}
```

1 Constructors and member functions of `pair` shall not throw exceptions unless one of the element-wise operations specified to be called for that operation throws an exception.

2 The defaulted move and copy constructor, respectively, of pair shall be a `constexpr` function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a `constexpr` function.

`constexpr pair();`

3 *Requires:* `is_default_constructible<first_type>::value` is `true` and `is_default_constructible<second_type>::value` is `true`.

4 *Effects:* Value-initializes `first` and `second`.

```
EXPLICIT constexpr pair(const T1& x, const T2& y);
```

5 *Effects:* The constructor initializes `first` with `x` and `second` with `y`.

6 *Remarks:* This constructor shall not participate in overload resolution unless `is_copy_constructible<first_type>::value` is `true` and `is_copy_constructible<second_type>::value` is `true`. The constructor is explicit if and only if `is_convertible<const first_type&, first_type>::value` is `false` or `is_convertible<const second_type&, second_type>::value` is `false`.

```
template<class U, class V> EXPLICIT constexpr pair(U&& x, V&& y);
```

7 *Effects:* The constructor initializes `first` with `std::forward<U>(x)` and `second` with `std::forward<V>(y)`.

8 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<first_type, U&&>::value` is `true` and `is_constructible<second_type, V&&>::value` is `true`. The constructor is explicit if and only if `is_convertible<U&&, first_type>::value` is `false` or `is_convertible<V&&, second_type>::value` is `false`.

```
template<class U, class V> EXPLICIT constexpr pair(const pair<U, V>& p);
```

9 *Effects:* The constructor initializes members from the corresponding members of the argument.

10 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<first_type, const U&>::value` is `true` and `is_constructible<second_type, const V&>::value` is `true`. The constructor is explicit if and only if `is_convertible<const U&, first_type>::value` is `false` or `is_convertible<const V&, second_type>::value` is `false`.

```
template<class U, class V> EXPLICIT constexpr pair(pair<U, V>&& p);
```

11 *Effects:* The constructor initializes `first` with `std::forward<U>(p.first)` and `second` with `std::forward<V>(p.second)`.

12 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<first_type, U&&>::value` is `true` and `is_constructible<second_type, V&&>::value` is `true`. The constructor is explicit if and only if `is_convertible<U&&, first_type>::value` is `false` or `is_convertible<V&&, second_type>::value` is `false`.

```
template<class... Args1, class... Args2>
  pair(piecewise_construct_t,
       tuple<Args1...> first_args, tuple<Args2...> second_args);
```

13 *Requires:* `is_constructible<first_type, Args1&&...>::value` is `true` and `is_constructible<second_type, Args2&&...>::value` is `true`.

14 *Effects:* The constructor initializes `first` with arguments of types `Args1...` obtained by forwarding the elements of `first_args` and initializes `second` with arguments of types `Args2...` obtained by forwarding the elements of `second_args`. (Here, forwarding an element `x` of type `U` within a `tuple` object means calling `std::forward<U>(x)`.) This form of construction, whereby constructor arguments for `first` and `second` are each provided in a separate `tuple` object, is called *piecewise construction*.

```
pair& operator=(const pair& p);
```

15 *Requires:* `is_copy_assignable<first_type>::value` is `true` and `is_copy_assignable<second_type>::value` is `true`.

16 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

17 *Returns:* `*this`.

```
template<class U, class V> pair& operator=(const pair<U, V>& p);
```

18 *Requires:* `is_assignable<first_type&, const U&>::value` is `true` and `is_assignable<second_type&, const V&>::value` is `true`.

19 *Effects:* Assigns `p.first` to `first` and `p.second` to `second`.

20 *Returns:* `*this`.

```
pair& operator=(pair&& p) noexcept(see below);
```

21 *Remarks:* The expression inside `noexcept` is equivalent to:

```
is_nothrow_move_assignable<T1>::value &&
is_nothrow_move_assignable<T2>::value
```

22 *Requires:* `is_move_assignable<first_type>::value` is `true` and `is_move_assignable<second_type>::value` is `true`.

23 *Effects:* Assigns to `first` with `std::forward<first_type>(p.first)` and to `second` with `std::forward<second_type>(p.second)`.

24 *Returns:* `*this`.

```
template<class U, class V> pair& operator=(pair<U, V>&& p);
```

25 *Requires:* `is_assignable<first_type&, U&&>::value` is `true` and `is_assignable<second_type&, V&&>::value` is `true`.

26 *Effects:* Assigns to `first` with `std::forward<U>(p.first)` and to `second` with `std::forward<V>(p.second)`.

27 *Returns:* `*this`.

```
void swap(pair& p) noexcept(see below);
```

28 *Remarks:* The expression inside `noexcept` is equivalent to:

```
noexcept(swap(first, p.first)) &&
noexcept(swap(second, p.second))
```

29 *Requires:* `first` shall be swappable with (17.6.3.2) `p.first` and `second` shall be swappable with `p.second`.

30 *Effects:* Swaps `first` with `p.first` and `second` with `p.second`.

20.3.3 Specialized algorithms [pairs.spec]

```
template <class T1, class T2>
  constexpr bool operator==(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

1 *Returns:* `x.first == y.first && x.second == y.second`.

```
template <class T1, class T2>
  constexpr bool operator<(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

2 *Returns:* `x.first < y.first || (!(y.first < x.first) && x.second < y.second)`.

```
template <class T1, class T2>
  constexpr bool operator!=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

3 *Returns:* `!(x == y)`

```
template <class T1, class T2>
  constexpr bool operator>(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

4 *Returns:* `y < x`

```
template <class T1, class T2>
  constexpr bool operator>=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

5 *Returns:* `!(x < y)`

```
template <class T1, class T2>
  constexpr bool operator<=(const pair<T1, T2>& x, const pair<T1, T2>& y);
```

6 *Returns:* `!(y < x)`

```
template<class T1, class T2> void swap(pair<T1, T2>& x, pair<T1, T2>& y)
  noexcept(noexcept(x.swap(y)));
```

7 *Effects:* `x.swap(y)`

```
template <class T1, class T2>
  constexpr pair<V1, V2> make_pair(T1&& x, T2&& y);
```

8 *Returns:* `pair<V1, V2>(std::forward<T1>(x), std::forward<T2>(y))`;

where `V1` and `V2` are determined as follows: Let `Ui` be `decay_t<Ti>` for each `Ti`. Then each `Vi` is `X&` if `Ui` equals `reference_wrapper<X>`, otherwise `Vi` is `Ui`.

9 [*Example:* In place of:

```
return pair<int, double>(5, 3.1415926);    // explicit types
```

a C++ program may contain:

```
return make_pair(5, 3.1415926);            // types are deduced
```

— *end example*]

20.3.4 Tuple-like access to pair [pair.astuple]

```
template <class T1, class T2>
struct tuple_size<pair<T1, T2>>
  : integral_constant<size_t, 2> { };

tuple_element<0, pair<T1, T2> >::type
```

1 *Value:* the type `T1`.

```
tuple_element<1, pair<T1, T2> >::type
```

2 *Value:* the type T2.

```
template<size_t I, class T1, class T2>
  constexpr tuple_element_t<I, pair<T1, T2>>&
    get(pair<T1, T2>& p) noexcept;
template<size_t I, class T1, class T2>
  constexpr const tuple_element_t<I, pair<T1, T2>>&
    get(const pair<T1, T2>& p) noexcept;
```

3 *Returns:* If `I == 0` returns `p.first`; if `I == 1` returns `p.second`; otherwise the program is ill-formed.

```
template<size_t I, class T1, class T2>
  constexpr tuple_element_t<I, pair<T1, T2>>&&
    get(pair<T1, T2>&& p) noexcept;
```

4 *Returns:* If `I == 0` returns `std::forward<T1&&>(p.first)`; if `I == 1` returns `std::forward<T2&&>(p.second)`; otherwise the program is ill-formed.

```
template <class T, class U>
  constexpr T& get(pair<T, U>& p) noexcept;
template <class T, class U>
  constexpr const T& get(const pair<T, U>& p) noexcept;
```

5 *Requires:* `T` and `U` are distinct types. Otherwise, the program is ill-formed.

6 *Returns:* `get<0>(p);`

```
template <class T, class U>
  constexpr T&& get(pair<T, U>&& p) noexcept;
```

7 *Requires:* `T` and `U` are distinct types. Otherwise, the program is ill-formed.

8 *Returns:* `get<0>(std::move(p));`

```
template <class T, class U>
  constexpr T& get(pair<U, T>& p) noexcept;
template <class T, class U>
  constexpr const T& get(const pair<U, T>& p) noexcept;
```

9 *Requires:* `T` and `U` are distinct types. Otherwise, the program is ill-formed.

10 *Returns:* `get<1>(p);`

```
template <class T, class U>
  constexpr T&& get(pair<U, T>&& p) noexcept;
```

11 *Requires:* `T` and `U` are distinct types. Otherwise, the program is ill-formed.

12 *Returns:* `get<1>(std::move(p));`

20.3.5 Piecewise construction [pair.piecewise]

```
struct piecewise_construct_t { };
constexpr piecewise_construct_t piecewise_construct{};
```

1 The `struct piecewise_construct_t` is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, `pair` has a constructor with `piecewise_construct_t` as the first argument, immediately followed by two `tuple` (20.4) arguments used for piecewise construction of the elements of the `pair` object.

20.4 Tuples [tuple]

20.4.1 In general [tuple.general]

1 This subclause describes the tuple library that provides a tuple type as the class template `tuple` that can be instantiated with any number of arguments. Each template argument specifies the type of an element in the `tuple`. Consequently, tuples are heterogeneous, fixed-size collections of values. An instantiation of `tuple` with two arguments is similar to an instantiation of `pair` with the same two arguments. See 20.3.

2 **Header `<tuple>` synopsis**

```
namespace std {
  // 20.4.2, class template tuple:
  template <class... Types> class tuple;

  // 20.4.2.4, tuple creation functions:
  const unspecified  ignore;

  template <class... Types>
    constexpr tuple<VTypes...> make_tuple(Types&&...);
  template <class... Types>
    constexpr tuple<Types&&...> forward_as_tuple(Types&&...) noexcept;

  template<class... Types>
    constexpr tuple<Types&...> tie(Types&...) noexcept;

  template <class... Tuples>
    constexpr tuple<Ctypes...> tuple_cat(Tuples&&...);

  // 20.4.2.5, tuple helper classes:
  template <class T> class tuple_size;  // undefined
  template <class T> class tuple_size<const T>;
  template <class T> class tuple_size<volatile T>;
  template <class T> class tuple_size<const volatile T>;

  template <class... Types> class tuple_size<tuple<Types...> >;

  template <size_t I, class T> class tuple_element;    // undefined
  template <size_t I, class T> class tuple_element<I, const T>;
  template <size_t I, class T> class tuple_element<I, volatile T>;
  template <size_t I, class T> class tuple_element<I, const volatile T>;

  template <size_t I, class... Types> class tuple_element<I, tuple<Types...> >;

  template <size_t I, class T>
    using tuple_element_t = typename tuple_element<I, T>::type;

  // 20.4.2.6, element access:
  template <size_t I, class... Types>
    constexpr tuple_element_t<I, tuple<Types...>>&
      get(tuple<Types...>&) noexcept;
  template <size_t I, class... Types>
    constexpr tuple_element_t<I, tuple<Types...>>&&
      get(tuple<Types...>&&) noexcept;
  template <size_t I, class... Types>
    constexpr const tuple_element_t<I, tuple<Types...>>&
      get(const tuple<Types...>&) noexcept;
  template <class T, class... Types>
    constexpr T& get(tuple<Types...>& t) noexcept;
  template <class T, class... Types>
    constexpr T&& get(tuple<Types...>&& t) noexcept;
  template <class T, class... Types>
    constexpr const T& get(const tuple<Types...>& t) noexcept;

  // 20.4.2.7, relational operators:
  template<class... TTypes, class... UTypes>
```

```
    constexpr bool operator==(const tuple<TTypes...>&, const tuple<UTypes...>&);
  template<class... TTypes, class... UTypes>
    constexpr bool operator<(const tuple<TTypes...>&, const tuple<UTypes...>&);
  template<class... TTypes, class... UTypes>
    constexpr bool operator!=(const tuple<TTypes...>&, const tuple<UTypes...>&);
  template<class... TTypes, class... UTypes>
    constexpr bool operator>(const tuple<TTypes...>&, const tuple<UTypes...>&);
  template<class... TTypes, class... UTypes>
    constexpr bool operator<=(const tuple<TTypes...>&, const tuple<UTypes...>&);
  template<class... TTypes, class... UTypes>
    constexpr bool operator>=(const tuple<TTypes...>&, const tuple<UTypes...>&);

  // 20.4.2.8, allocator-related traits
  template <class... Types, class Alloc>
    struct uses_allocator<tuple<Types...>, Alloc>;

  // 20.4.2.9, specialized algorithms:
  template <class... Types>
    void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
}
```

20.4.2 Class template `tuple` [tuple.tuple]

```
namespace std {
  template <class... Types>
  class tuple  {
  public:

    // 20.4.2.1, tuple construction
    constexpr tuple();
    EXPLICIT constexpr tuple(const Types&...); // only if sizeof...(Types) >= 1
    template <class... UTypes>
      EXPLICIT constexpr tuple(UTypes&&...); // only if sizeof...(Types) >= 1

    tuple(const tuple&) = default;
    tuple(tuple&&) = default;

    template <class... UTypes>
      EXPLICIT constexpr tuple(const tuple<UTypes...>&);
    template <class... UTypes>
      EXPLICIT constexpr tuple(tuple<UTypes...>&&);

    template <class U1, class U2>
      EXPLICIT constexpr tuple(const pair<U1, U2>&);       // only if sizeof...(Types) == 2
    template <class U1, class U2>
      EXPLICIT constexpr tuple(pair<U1, U2>&&);            // only if sizeof...(Types) == 2

    // allocator-extended constructors
    template <class Alloc>
      tuple(allocator_arg_t, const Alloc& a);
    template <class Alloc>
      EXPLICIT tuple(allocator_arg_t, const Alloc& a, const Types&...);
    template <class Alloc, class... UTypes>
      EXPLICIT tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
    template <class Alloc>
```

```
    tuple(allocator_arg_t, const Alloc& a, const tuple&);
  template <class Alloc>
    tuple(allocator_arg_t, const Alloc& a, tuple&&);
  template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
  template <class Alloc, class... UTypes>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
  template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
  template <class Alloc, class U1, class U2>
    EXPLICIT tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);

  // 20.4.2.2, tuple assignment
  tuple& operator=(const tuple&);
  tuple& operator=(tuple&&) noexcept(see below);

  template <class... UTypes>
    tuple& operator=(const tuple<UTypes...>&);
  template <class... UTypes>
    tuple& operator=(tuple<UTypes...>&&);

  template <class U1, class U2>
    tuple& operator=(const pair<U1, U2>&);    // only if sizeof...(Types) == 2
  template <class U1, class U2>
    tuple& operator=(pair<U1, U2>&&);         // only if sizeof...(Types) == 2

  // 20.4.2.3, tuple swap
  void swap(tuple&) noexcept(see below);
};
}
```

20.4.2.1 Construction [tuple.cnstr]

1 For each `tuple` constructor, an exception is thrown only if the construction of one of the types in `Types` throws an exception.

2 The defaulted move and copy constructor, respectively, of `tuple` shall be a `constexpr` function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a `constexpr` function. The defaulted move and copy constructor of `tuple<>` shall be `constexpr` functions.

3 In the constructor descriptions that follow, let i be in the range `[0,sizeof...(Types))` in order, T_i be the ith type in `Types`, and U_i be the ith type in a template parameter pack named `UTypes`, where indexing is zero-based.

```
constexpr tuple();
```

4 *Requires:* `is_default_constructible<`T_i`>::value` is true for all i.

5 *Effects:* Value initializes each element.

```
EXPLICIT constexpr tuple(const Types&...);
```

6 *Effects:* The constructor initializes each element with the value of the corresponding parameter.

7 *Remarks:* This constructor shall not participate in overload resolution unless `sizeof...(Types) >= 1` and `is_copy_constructible<`T_i`>::value` is `true` for all i. The constructor is explicit if and only if `is_convertible<const` T_i`&,` T_i`>::value` is `false` for at least one i.

```
template <class... UTypes>
```

EXPLICIT `constexpr tuple(UTypes&&... u);`

8 *Requires:* `sizeof...(Types) == sizeof...(UTypes)`.

9 *Effects:* The constructor initializes the elements in the tuple with the corresponding value in `std::forward<UTypes>(u)`.

10 *Remarks:* This constructor shall not participate in overload resolution unless `sizeof...(Types) >= 1` and `is_constructible<`T_i`, `U_i`&&>::value` is `true` for all i. The constructor is explicit if and only if `is_convertible<`U_i`&&, `T_i`>::value` is `false` for at least one i.

`tuple(const tuple& u) = default;`

11 *Requires:* `is_copy_constructible<`T_i`>::value` is `true` for all i.

12 *Effects:* Initializes each element of `*this` with the corresponding element of `u`.

`tuple(tuple&& u) = default;`

13 *Requires:* `is_move_constructible<`T_i`>::value` is `true` for all i.

14 *Effects:* For all i, initializes the ith element of `*this` with `std::forward<`T_i`>(get<`i`>(u))`.

`template <class... UTypes> `*EXPLICIT*` constexpr tuple(const tuple<UTypes...>& u);`

15 *Requires:* `sizeof...(Types) == sizeof...(UTypes)`.

16 *Effects:* The constructor initializes each element of `*this` with the corresponding element of `u`.

17 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<`T_i`, const `U_i`&>::value` is `true` for all i. The constructor is explicit if and only if `is_convertible<const `U_i`&, `T_i`>::value` is `false` for at least one i.

`template <class... UTypes> `*EXPLICIT*` constexpr tuple(tuple<UTypes...>&& u);`

18 *Requires:* `sizeof...(Types) == sizeof...(UTypes)`.

19 *Effects:* For all i, the constructor initializes the ith element of `*this` with `std::forward<`U_i`>(get<`i`>(u))`.

20 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<`T_i`, `U_i`&&>::value` is `true` for all i. The constructor is explicit if and only if `is_convertible<`U_i`&&, `T_i`>::value` is `false` for at least one i.

`template <class U1, class U2> `*EXPLICIT*` constexpr tuple(const pair<U1, U2>& u);`

21 *Requires:* `sizeof...(Types) == 2`.

22 *Effects:* The constructor initializes the first element with `u.first` and the second element with `u.second`.

23 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<`T_0`, const U1&>::value` is `true` and `is_constructible<`T_1`, const U2&>::value` is `true`. The constructor is explicit if and only if `is_convertible<const U1&, `T_0`>::value` is `false` or `is_convertible<const U2&, `T_1`>::value` is `false`.

`template <class U1, class U2> `*EXPLICIT*` constexpr tuple(pair<U1, U2>&& u);`

24 *Requires:* `sizeof...(Types) == 2`.

25 *Effects:* The constructor initializes the first element with `std::forward<U1>(u.first)` and the second element with `std::forward<U2>(u.second)`.

26 *Remarks:* This constructor shall not participate in overload resolution unless `is_constructible<`T_0`, U1&&>::value` is `true` and `is_constructible<`T_1`, U2&&>::value` is `true`. The constructor is explicit if and only if `is_convertible<U1&&, `T_0`>::value` is `false` or `is_convertible<U2&&, `T_1`>::value` is `false`.

```
template <class Alloc>
  tuple(allocator_arg_t, const Alloc& a);
template <class Alloc>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, const Types&...);
template <class Alloc, class... UTypes>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, UTypes&&...);
template <class Alloc>
  tuple(allocator_arg_t, const Alloc& a, const tuple&);
template <class Alloc>
  tuple(allocator_arg_t, const Alloc& a, tuple&&);
template <class Alloc, class... UTypes>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, const tuple<UTypes...>&);
template <class Alloc, class... UTypes>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, tuple<UTypes...>&&);
template <class Alloc, class U1, class U2>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, const pair<U1, U2>&);
template <class Alloc, class U1, class U2>
  EXPLICIT tuple(allocator_arg_t, const Alloc& a, pair<U1, U2>&&);
```

27 *Requires:* `Alloc` shall meet the requirements for an `Allocator` (17.6.3.5).

28 *Effects:* Equivalent to the preceding constructors except that each element is constructed with uses-allocator construction (20.7.7.2).

20.4.2.2 Assignment [tuple.assign]

1 For each `tuple` assignment operator, an exception is thrown only if the assignment of one of the types in `Types` throws an exception. In the function descriptions that follow, let i be in the range `[0,sizeof...(Types))` in order, T_i be the ith type in `Types`, and U_i be the ith type in a template parameter pack named `UTypes`, where indexing is zero-based.

```
tuple& operator=(const tuple& u);
```

2 *Requires:* `is_copy_assignable<`T_i`>::value` is `true` for all i.

3 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

4 *Returns:* `*this`

```
tuple& operator=(tuple&& u) noexcept(see below);
```

5 *Remark:* The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

```
is_nothrow_move_assignable<Ti>::value
```

where T_i is the ith type in `Types`.

6 *Requires:* `is_move_assignable<`T_i`>::value` is `true` for all i.

7 *Effects:* For all i, assigns `std::forward<`T_i`>(get<`i`>(u))` to `get<`i`>(*this)`.

8 *Returns:* `*this`.

```
template <class... UTypes>
  tuple& operator=(const tuple<UTypes...>& u);
```

9 *Requires:* `sizeof...(Types) == sizeof...(UTypes)` and `is_assignable<`T_i`&, const `U_i`&>::value` is `true` for all i.

10 *Effects:* Assigns each element of `u` to the corresponding element of `*this`.

11 *Returns:* `*this`

```
template <class... UTypes>
  tuple& operator=(tuple<UTypes...>&& u);
```

12 *Requires:* `is_assignable<Ti&, Ui&&>::value == true` for all `i`. `sizeof...(Types) == sizeof...(UTypes)`.

13 *Effects:* For all i, assigns `std::forward<`U_i`>(get<`i`>(u))` to `get<`i`>(*this)`.

14 *Returns:* `*this`.

```
template <class U1, class U2> tuple& operator=(const pair<U1, U2>& u);
```

15 *Requires:* `sizeof...(Types) == 2`. `is_assignable<`T_0`&, const U1&>::value` is `true` for the first type T_0 in `Types` and `is_assignable<`T_1`&, const U2&>::value` is `true` for the second type T_1 in `Types`.

16 *Effects:* Assigns `u.first` to the first element of `*this` and `u.second` to the second element of `*this`.

17 *Returns:* `*this`

```
template <class U1, class U2> tuple& operator=(pair<U1, U2>&& u);
```

18 *Requires:* `sizeof...(Types) == 2`. `is_assignable<`T_0`&, U1&&>::value` is `true` for the first type T_0 in `Types` and `is_assignable<`T_1`&, U2&&>::value` is `true` for the second type T_1 in `Types`.

19 *Effects:* Assigns `std::forward<U1>(u.first)` to the first element of `*this` and `std::forward<U2>(u.second)` to the second element of `*this`.

20 *Returns:* `*this`.

20.4.2.3 `swap` [tuple.swap]

```
void swap(tuple& rhs) noexcept(see below);
```

1 *Remark:* The expression inside `noexcept` is equivalent to the logical AND of the following expressions:

```
noexcept(swap(declval<Ti&>(), declval<Ti&>()))
```

where T_i is the ith type in `Types`.

2 *Requires:* Each element in `*this` shall be swappable with (17.6.3.2) the corresponding element in `rhs`.

3 *Effects:* Calls `swap` for each element in `*this` and its corresponding element in `rhs`.

4 *Throws:* Nothing unless one of the element-wise `swap` calls throws an exception.

20.4.2.4 Tuple creation functions [tuple.creation]

1 In the function descriptions that follow, let i be in the range `[0,sizeof...(TTypes))` in order and let T_i be the ith type in a template parameter pack named `TTypes`; let j be in the range `[0,sizeof...(UTypes))` in order and U_j be the jth type in a template parameter pack named `UTypes`, where indexing is zero-based.

```
template<class... Types>
  constexpr tuple<VTypes...> make_tuple(Types&&... t);
```

2 Let U_i be `decay_t<`T_i`>` for each T_i in `Types`. Then each V_i in `VTypes` is `X&` if U_i equals `reference_wrapper<X>`, otherwise V_i is U_i.

3 *Returns:* `tuple<VTypes...>(std::forward<Types>(t)...)`.

4 [*Example:*

```
int i; float j;
make_tuple(1, ref(i), cref(j))
```

creates a tuple of type

```
tuple<int, int&, const float&>
```

— *end example*]

```
template<class... Types>
  constexpr tuple<Types&&...> forward_as_tuple(Types&&... t) noexcept;
```

5 *Effects:* Constructs a tuple of references to the arguments in `t` suitable for forwarding as arguments to a function. Because the result may contain references to temporary variables, a program shall ensure that the return value of this function does not outlive any of its arguments. (e.g., the program should typically not store the result in a named variable).

6 *Returns:* `tuple<Types&&...>(std::forward<Types>(t)...)`

```
template<class... Types>
  constexpr tuple<Types&...> tie(Types&... t) noexcept;
```

7 *Returns:* `tuple<Types&...>(t...)`. When an argument in `t` is `ignore`, assigning any value to the corresponding tuple element has no effect.

8 [*Example:* `tie` functions allow one to create tuples that unpack tuples into variables. `ignore` can be used for elements that are not needed:

```
int i; std::string s;
tie(i, ignore, s) = make_tuple(42, 3.14, "C++");
// i == 42, s == "C++"
```

— *end example*]

```
template <class... Tuples>
  constexpr tuple<CTypes...> tuple_cat(Tuples&&... tpls);
```

9 In the following paragraphs, let T_i be the ith type in `Tuples`, U_i be `remove_reference_t<Ti>`, and tp_i be the ith parameter in the function parameter pack `tpls`, where all indexing is zero-based.

10 *Requires:* For all i, U_i shall be the type cv_i `tuple<`$Args_i$`...>`, where cv_i is the (possibly empty) i^{th} cv-qualifier-seq and $Args_i$ is the parameter pack representing the element types in U_i. Let A_{ik} be the $k_i{}^{th}$ type in $Args_i$. For all A_{ik} the following requirements shall be satisfied: If T_i is deduced as an lvalue reference type, then `is_constructible<`A_{ik}`, `cv_i A_{ik}`&>::value == true`, otherwise `is_constructible<`A_{ik}`, `$cv_i A_{ik}$`&&>::value == true`.

11 *Remarks:* The types in *`CTypes`* shall be equal to the ordered sequence of the extended types $Args_0\ldots$, $Args_1\ldots$, ... $Args_{n-1}\ldots$, where n is equal to `sizeof...(Tuples)`. Let $e_i\ldots$ be the ith ordered sequence of tuple elements of the resulting `tuple` object corresponding to the type sequence $Args_i$.

12 *Returns:* A `tuple` object constructed by initializing the $k_i{}^{th}$ type element e_{ik} in $e_i\ldots$ with `get<`k_i`>(std::forward<`T_i`>(`tp_i`))` for each valid k_i and each group e_i in order.

13 *Note:* An implementation may support additional types in the parameter pack `Tuples` that support the `tuple`-like protocol, such as `pair` and `array`.

20.4.2.5 Tuple helper classes [tuple.helper]

```
template <class T> struct tuple_size;
```

Remarks: All specializations of `tuple_size<T>` shall meet the `UnaryTypeTrait` requirements (20.10.1) with a `BaseCharacteristic` of `integral_constant<size_t, N>` for some `N`.

```
template <class... Types>
class tuple_size<tuple<Types...> >
  : public integral_constant<size_t, sizeof...(Types)> { };
```

```
template <size_t I, class... Types>
class tuple_element<I, tuple<Types...> > {
public:
  typedef TI type;
};
```

1 *Requires:* `I < sizeof...(Types)`. The program is ill-formed if `I` is out of bounds.

2 *Type:* `TI` is the type of the `I`th element of `Types`, where indexing is zero-based.

```
template <class T> class tuple_size<const T>;
template <class T> class tuple_size<volatile T>;
template <class T> class tuple_size<const volatile T>;
```

3 Let *TS* denote `tuple_size<T>` of the *cv*-unqualified type `T`. Then each of the three templates shall meet the `UnaryTypeTrait` requirements (20.10.1) with a `BaseCharacteristic` of

```
integral_constant<size_t, TS::value>
```

4 In addition to being available via inclusion of the `<tuple>` header, the three templates are available when either of the headers `<array>` or `<utility>` are included.

```
template <size_t I, class T> class tuple_element<I, const T>;
template <size_t I, class T> class tuple_element<I, volatile T>;
template <size_t I, class T> class tuple_element<I, const volatile T>;
```

5 Let *TE* denote `tuple_element<I, T>` of the *cv*-unqualified type `T`. Then each of the three templates shall meet the `TransformationTrait` requirements (20.10.1) with a member typedef `type` that names the following type:

(5.1) — for the first specialization, `add_const_t<`*TE*`::type>`,

(5.2) — for the second specialization, `add_volatile_t<`*TE*`::type>`, and

(5.3) — for the third specialization, `add_cv_t<`*TE*`::type>`.

6 In addition to being available via inclusion of the `<tuple>` header, the three templates are available when either of the headers `<array>` or `<utility>` are included.

20.4.2.6 Element access [tuple.elem]

```
template <size_t I, class... Types>
  constexpr tuple_element_t<I, tuple<Types...> >& get(tuple<Types...>& t) noexcept;
```

1 *Requires:* `I < sizeof...(Types)`. The program is ill-formed if `I` is out of bounds.

2 *Returns:* A reference to the `I`th element of `t`, where indexing is zero-based.

```
template <size_t I, class... Types>
  constexpr tuple_element_t<I, tuple<Types...> >&& get(tuple<Types...>&& t) noexcept;
```

3 *Effects:* Equivalent to `return std::forward<typename tuple_element<I, tuple<Types...> >::type&&>(get<I>(t));`

4 *Note:* if a `T` in `Types` is some reference type `X&`, the return type is `X&`, not `X&&`. However, if the element type is a non-reference type `T`, the return type is `T&&`.

```
template <size_t I, class... Types>
  constexpr tuple_element_t<I, tuple<Types...> > const& get(const tuple<Types...>& t) noexcept;
```

5 *Requires:* `I < sizeof...(Types)`. The program is ill-formed if `I` is out of bounds.

6 *Returns:* A const reference to the `I`th element of `t`, where indexing is zero-based.

7 [*Note:* Constness is shallow. If a `T` in `Types` is some reference type `X&`, the return type is `X&`, not `const X&`. However, if the element type is non-reference type `T`, the return type is `const T&`. This is consistent with how constness is defined to work for member variables of reference type. — *end note*]

```
template <class T, class... Types>
  constexpr T& get(tuple<Types...>& t) noexcept;
template <class T, class... Types>
  constexpr T&& get(tuple<Types...>&& t) noexcept;
template <class T, class... Types>
  constexpr const T& get(const tuple<Types...>& t) noexcept;
```

8 *Requires:* The type `T` occurs exactly once in `Types...`. Otherwise, the program is ill-formed.

9 *Returns:* A reference to the element of `t` corresponding to the type `T` in `Types...`.

10 [*Example:*

```
const tuple<int, const int, double, double> t(1, 2, 3.4, 5.6);
const int &i1 = get<int>(t);        // OK. Not ambiguous. i1 == 1
const int &i2 = get<const int>(t);  // OK. Not ambiguous. i2 == 2
const double &d = get<double>(t);   // ERROR. ill-formed
```

— *end example*]

11 [*Note:* The reason `get` is a nonmember function is that if this functionality had been provided as a member function, code where the type depended on a template parameter would have required using the `template` keyword. — *end note*]

20.4.2.7 Relational operators [tuple.rel]

```
template<class... TTypes, class... UTypes>
  constexpr bool operator==(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

1 *Requires:* For all `i`, where `0 <= i` and `i < sizeof...(TTypes)`, `get<i>(t) == get<i>(u)` is a valid expression returning a type that is convertible to `bool`. `sizeof...(TTypes) == sizeof...(UTypes)`.

2 *Returns:* `true` if `get<i>(t) == get<i>(u)` for all `i`, otherwise `false`. For any two zero-length tuples `e` and `f`, `e == f` returns `true`.

3 *Effects:* The elementary comparisons are performed in order from the zeroth index upwards. No comparisons or element accesses are performed after the first equality comparison that evaluates to `false`.

```
template<class... TTypes, class... UTypes>
  constexpr bool operator<(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

4 *Requires:* For all `i`, where `0 <= i` and `i < sizeof...(TTypes)`, `get<i>(t) < get<i>(u)` and `get<i>(u) < get<i>(t)` are valid expressions returning types that are convertible to `bool`. `sizeof...(TTypes) == sizeof...(UTypes)`.

5 *Returns:* The result of a lexicographical comparison between `t` and `u`. The result is defined as: `(bool)(get<0>(t) < get<0>(u)) || (!(bool)(get<0>(u) < get<0>(t)) && t`$_{\text{tail}}$ `<` `u`$_{\text{tail}}$`)`, where `r`$_{\text{tail}}$ for some tuple `r` is a tuple containing all but the first element of `r`. For any two zero-length tuples `e` and `f`, `e < f` returns `false`.

```
template<class... TTypes, class... UTypes>
  constexpr bool operator!=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

6 *Returns:* `!(t == u)`.

```
template<class... TTypes, class... UTypes>
  constexpr bool operator>(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

7 *Returns:* `u < t`.

```
template<class... TTypes, class... UTypes>
  constexpr bool operator<=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

8 *Returns:* `!(u < t)`

```
template<class... TTypes, class... UTypes>
  constexpr bool operator>=(const tuple<TTypes...>& t, const tuple<UTypes...>& u);
```

9 *Returns:* `!(t < u)`

10 [*Note:* The above definitions for comparison operators do not require $\texttt{t}_{\text{tail}}$ (or $\texttt{u}_{\text{tail}}$) to be constructed. It may not even be possible, as `t` and `u` are not required to be copy constructible. Also, all comparison operators are short circuited; they do not perform element accesses beyond what is required to determine the result of the comparison. — *end note*]

20.4.2.8 Tuple traits [tuple.traits]

```
template <class... Types, class Alloc>
  struct uses_allocator<tuple<Types...>, Alloc> : true_type { };
```

Requires: `Alloc` shall be an `Allocator` (17.6.3.5).

1 [*Note:* Specialization of this trait informs other library components that `tuple` can be constructed with an allocator, even though it does not have a nested `allocator_type`. — *end note*]

20.4.2.9 Tuple specialized algorithms [tuple.special]

```
template <class... Types>
  void swap(tuple<Types...>& x, tuple<Types...>& y) noexcept(see below);
```

1 *Remark:* The expression inside `noexcept` is equivalent to:

```
noexcept(x.swap(y))
```

2 *Effects:* `x.swap(y)`

20.5 Compile-time integer sequences [intseq]

20.5.1 In general [intseq.general]

1 The library provides a class template that can represent an integer sequence. When used as an argument to a function template the parameter pack defining the sequence can be deduced and used in a pack expansion.

2 [*Example:*

```
template<class F, class Tuple, std::size_t... I>
  decltype(auto) apply_impl(F&& f, Tuple&& t, index_sequence<I...>) {
    return std::forward<F>(f)(std::get<I>(std::forward<Tuple>(t))...);
  }

template<class F, class Tuple>
  decltype(auto) apply(F&& f, Tuple&& t) {
    using Indices = make_index_sequence<std::tuple_size<std::decay_t<Tuple>>::value>;
    return apply_impl(std::forward<F>(f), std::forward<Tuple>(t), Indices());
  }
```

— *end example*] [*Note:* The `index_sequence` alias template is provided for the common case of an integer sequence of type `size_t`. — *end note*]

20.5.2 Class template `integer_sequence` [intseq.intseq]

```
namespace std {
  template<class T, T... I>
  struct integer_sequence {
    typedef T value_type;
    static constexpr size_t size() noexcept { return sizeof...(I); }
  };
}
```

1 `T` shall be an integer type.

20.5.3 Alias template `make_integer_sequence` [intseq.make]

```
template<class T, T N>
  using make_integer_sequence = integer_sequence<T, see below>;
```

1 If `N` is negative the program is ill-formed. The alias template `make_integer_sequence` denotes a specialization of `integer_sequence` with `N` template non-type arguments. The type `make_integer_sequence<T, N>` denotes the type `integer_sequence<T, 0, 1, ..., N-1>`. [*Note:* `make_integer_sequence<int, 0>` denotes the type `integer_sequence<int>` — *end note*]

20.6 Class template `bitset` [template.bitset]

Header `<bitset>` synopsis

```
#include <string>
#include <iosfwd>               // for istream, ostream
namespace std {
  template <size_t N> class bitset;

  // 20.6.4 bitset operators:
  template <size_t N>
    bitset<N> operator&(const bitset<N>&, const bitset<N>&) noexcept;
  template <size_t N>
    bitset<N> operator|(const bitset<N>&, const bitset<N>&) noexcept;
  template <size_t N>
    bitset<N> operator^(const bitset<N>&, const bitset<N>&) noexcept;
  template <class charT, class traits, size_t N>
    basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>& is, bitset<N>& x);
  template <class charT, class traits, size_t N>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);
}
```

1 The header `<bitset>` defines a class template and several related functions for representing and manipulating fixed-size sequences of bits.

```
namespace std {
  template<size_t N> class bitset {
  public:
    // bit reference:
    class reference {
      friend class bitset;
```

```
  reference() noexcept;
public:
 ~reference() noexcept;
  reference& operator=(bool x) noexcept;             // for b[i] = x;
  reference& operator=(const reference&) noexcept;   // for b[i] = b[j];
  bool operator~() const noexcept;                   // flips the bit
  operator bool() const noexcept;                    // for x = b[i];
  reference& flip() noexcept;                        // for b[i].flip();
};

// 20.6.1 constructors:
constexpr bitset() noexcept;
constexpr bitset(unsigned long long val) noexcept;
template<class charT, class traits, class Allocator>
  explicit bitset(
    const basic_string<charT,traits,Allocator>& str,
    typename basic_string<charT,traits,Allocator>::size_type pos = 0,
    typename basic_string<charT,traits,Allocator>::size_type n =
      basic_string<charT,traits,Allocator>::npos,
      charT zero = charT('0'), charT one = charT('1'));
template <class charT>
  explicit bitset(
    const charT* str,
    typename basic_string<charT>::size_type n = basic_string<charT>::npos,
    charT zero = charT('0'), charT one = charT('1'));

// 20.6.2 bitset operations:
bitset<N>& operator&=(const bitset<N>& rhs) noexcept;
bitset<N>& operator|=(const bitset<N>& rhs) noexcept;
bitset<N>& operator^=(const bitset<N>& rhs) noexcept;
bitset<N>& operator<<=(size_t pos) noexcept;
bitset<N>& operator>>=(size_t pos) noexcept;
bitset<N>& set() noexcept;
bitset<N>& set(size_t pos, bool val = true);
bitset<N>& reset() noexcept;
bitset<N>& reset(size_t pos);
bitset<N>  operator~() const noexcept;
bitset<N>& flip() noexcept;
bitset<N>& flip(size_t pos);

// element access:
constexpr bool operator[](size_t pos) const;       // for b[i];
reference operator[](size_t pos);                  // for b[i];

unsigned long to_ulong() const;
unsigned long long to_ullong() const;
template <class charT = char,
    class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
  basic_string<charT, traits, Allocator>
  to_string(charT zero = charT('0'), charT one = charT('1')) const;
size_t count() const noexcept;
constexpr size_t size() const noexcept;
bool operator==(const bitset<N>& rhs) const noexcept;
bool operator!=(const bitset<N>& rhs) const noexcept;
```

```
    bool test(size_t pos) const;
    bool all() const noexcept;
    bool any() const noexcept;
    bool none() const noexcept;
    bitset<N> operator<<(size_t pos) const noexcept;
    bitset<N> operator>>(size_t pos) const noexcept;
  };

  // 20.6.3 hash support
  template <class T> struct hash;
  template <size_t N> struct hash<bitset<N> >;
}
```

2 The class template `bitset<N>`describes an object that can store a sequence consisting of a fixed number of bits, `N`.

3 Each bit represents either the value zero (reset) or one (set). To *toggle* a bit is to change the value zero to one, or the value one to zero. Each bit has a non-negative position `pos`. When converting between an object of class `bitset<N>` and a value of some integral type, bit position `pos` corresponds to the *bit value* `1 <<pos`. The integral value corresponding to two or more bits is the sum of their bit values.

4 The functions described in this subclause can report three kinds of errors, each associated with a distinct exception:

(4.1) — an *invalid-argument* error is associated with exceptions of type `invalid_argument` (19.2.3);

(4.2) — an *out-of-range* error is associated with exceptions of type `out_of_range` (19.2.5);

(4.3) — an *overflow* error is associated with exceptions of type `overflow_error` (19.2.8).

20.6.1 `bitset` constructors [bitset.cons]

```
constexpr bitset() noexcept;
```

1 *Effects:* Constructs an object of class `bitset<N>`, initializing all bits to zero.

```
constexpr bitset(unsigned long long val) noexcept;
```

2 *Effects:* Constructs an object of class `bitset<N>`, initializing the first `M` bit positions to the corresponding bit values in `val`. `M` is the smaller of `N` and the number of bits in the value representation (3.9) of `unsigned long long`. If `M < N`, the remaining bit positions are initialized to zero.

```
template <class charT, class traits, class Allocator>
explicit
bitset(const basic_string<charT, traits, Allocator>& str,
       typename basic_string<charT, traits, Allocator>::size_type pos = 0,
       typename basic_string<charT, traits, Allocator>::size_type n =
         basic_string<charT, traits, Allocator>::npos,
         charT zero = charT('0'), charT one = charT('1'));
```

3 *Requires:* `pos <= str.size()`.

4 *Throws:* `out_of_range` if `pos > str.size()`.

5 *Effects:* Determines the effective length `rlen` of the initializing string as the smaller of `n` and `str.size() - pos`.

The function then throws `invalid_argument` if any of the `rlen` characters in `str` beginning at position `pos` is other than `zero` or `one`. The function uses `traits::eq()` to compare the character values.

Otherwise, the function constructs an object of class `bitset<N>`, initializing the first `M` bit positions to values determined from the corresponding characters in the string `str`. `M` is the smaller of `N` and `rlen`.

6 An element of the constructed object has value zero if the corresponding character in `str`, beginning at position `pos`, is `zero`. Otherwise, the element has the value one. Character position `pos + M - 1` corresponds to bit position zero. Subsequent decreasing character positions correspond to increasing bit positions.

7 If `M < N`, remaining bit positions are initialized to zero.

```
template <class charT>
  explicit bitset(
    const charT* str,
    typename basic_string<charT>::size_type n = basic_string<charT>::npos,
    charT zero = charT('0'), charT one = charT('1'));
```

8 *Effects:* Constructs an object of class `bitset<N>` as if by

```
bitset(
  n == basic_string<charT>::npos
    ? basic_string<charT>(str)
    : basic_string<charT>(str, n),
  0, n, zero, one)
```

20.6.2 `bitset` members [bitset.members]

```
bitset<N>& operator&=(const bitset<N>& rhs) noexcept;
```

1 *Effects:* Clears each bit in `*this` for which the corresponding bit in `rhs` is clear, and leaves all other bits unchanged.

2 *Returns:* `*this`.

```
bitset<N>& operator|=(const bitset<N>& rhs) noexcept;
```

3 *Effects:* Sets each bit in `*this` for which the corresponding bit in `rhs` is set, and leaves all other bits unchanged.

4 *Returns:* `*this`.

```
bitset<N>& operator^=(const bitset<N>& rhs) noexcept;
```

5 *Effects:* Toggles each bit in `*this` for which the corresponding bit in `rhs` is set, and leaves all other bits unchanged.

6 *Returns:* `*this`.

```
bitset<N>& operator<<=(size_t pos) noexcept;
```

7 *Effects:* Replaces each bit at position `I` in `*this` with a value determined as follows:

(7.1) — If `I < pos`, the new value is zero;

(7.2) — If `I >= pos`, the new value is the previous value of the bit at position `I - pos`.

8 *Returns:* `*this`.

```
bitset<N>& operator>>=(size_t pos) noexcept;
```

9 *Effects:* Replaces each bit at position `I` in `*this` with a value determined as follows:

(9.1) — If `pos >= N - I`, the new value is zero;

(9.2) — If `pos < N - I`, the new value is the previous value of the bit at position `I + pos`.

10 *Returns:* `*this`.

```
bitset<N>& set() noexcept;
```

11 *Effects:* Sets all bits in `*this`.

12 *Returns:* `*this`.

```
bitset<N>& set(size_t pos, bool val = true);
```

13 *Requires:* `pos` is valid

14 *Throws:* `out_of_range` if `pos` does not correspond to a valid bit position.

15 *Effects:* Stores a new value in the bit at position `pos` in `*this`. If `val` is nonzero, the stored value is one, otherwise it is zero.

16 *Returns:* `*this`.

```
bitset<N>& reset() noexcept;
```

17 *Effects:* Resets all bits in `*this`.

18 *Returns:* `*this`.

```
bitset<N>& reset(size_t pos);
```

19 *Requires:* `pos` is valid

20 *Throws:* `out_of_range` if `pos` does not correspond to a valid bit position.

21 *Effects:* Resets the bit at position `pos` in `*this`.

22 *Returns:* `*this`.

```
bitset<N> operator~() const noexcept;
```

23 *Effects:* Constructs an object `x` of class `bitset<N>` and initializes it with `*this`.

24 *Returns:* `x.flip()`.

```
bitset<N>& flip() noexcept;
```

25 *Effects:* Toggles all bits in `*this`.

26 *Returns:* `*this`.

```
bitset<N>& flip(size_t pos);
```

27 *Requires:* `pos` is valid

28 *Throws:* `out_of_range` if `pos` does not correspond to a valid bit position.

29 *Effects:* Toggles the bit at position `pos` in `*this`.

30 *Returns:* `*this`.

```
unsigned long to_ulong() const;
```

31 *Throws:* `overflow_error` if the integral value `x` corresponding to the bits in `*this` cannot be represented as type `unsigned long`.

32 *Returns:* `x`.

```
unsigned long long to_ullong() const;
```

33 *Throws:* `overflow_error` if the integral value `x` corresponding to the bits in `*this` cannot be represented as type `unsigned long long`.

34 *Returns:* `x`.

```
template <class charT = char,
    class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
  basic_string<charT, traits, Allocator>
  to_string(charT zero = charT('0'), charT one = charT('1')) const;
```

35 *Effects:* Constructs a string object of the appropriate type and initializes it to a string of length `N` characters. Each character is determined by the value of its corresponding bit position in `*this`. Character position `N - 1` corresponds to bit position zero. Subsequent decreasing character positions correspond to increasing bit positions. Bit value zero becomes the character `zero`, bit value one becomes the character `one`.

36 *Returns:* The created object.

```
size_t count() const noexcept;
```

37 *Returns:* A count of the number of bits set in `*this`.

```
constexpr size_t size() const noexcept;
```

38 *Returns:* `N`.

```
bool operator==(const bitset<N>& rhs) const noexcept;
```

39 *Returns:* `true` if the value of each bit in `*this` equals the value of the corresponding bit in `rhs`.

```
bool operator!=(const bitset<N>& rhs) const noexcept;
```

40 *Returns:* `true` if `!(*this == rhs)`.

```
bool test(size_t pos) const;
```

41 *Requires:* `pos` is valid

42 *Throws:* `out_of_range` if `pos` does not correspond to a valid bit position.

43 *Returns:* `true` if the bit at position `pos` in `*this` has the value one.

```
bool all() const noexcept;
```

44 *Returns:* `count() == size()`

```
bool any() const noexcept;
```

45 *Returns:* `count() != 0`

```
bool none() const noexcept;
```

46 *Returns:* `count() == 0`

```
bitset<N> operator<<(size_t pos) const noexcept;
```

47 *Returns:* `bitset<N>(*this) <<= pos`.

```
bitset<N> operator>>(size_t pos) const noexcept;
```

48 *Returns:* `bitset<N>(*this) >>= pos`.

```
constexpr bool operator[](size_t pos) const;
```

49 *Requires:* `pos` shall be valid.

50 *Returns:* `true` if the bit at position `pos` in `*this` has the value one, otherwise `false`.

51 *Throws:* Nothing.

```
bitset<N>::reference operator[](size_t pos);
```

52 *Requires:* `pos` shall be valid.

53 *Returns:* An object of type `bitset<N>::reference` such that `(*this)[pos] == this->test(pos)`, and such that `(*this)[pos] = val` is equivalent to `this->set(pos, val)`.

54 *Throws:* Nothing.

55 *Remark:* For the purpose of determining the presence of a data race (1.10), any access or update through the resulting reference potentially accesses or modifies, respectively, the entire underlying bitset.

20.6.3 `bitset` hash support [bitset.hash]

```
template <size_t N> struct hash<bitset<N> >;
```

1 The template specialization shall meet the requirements of class template `hash` (20.9.13).

20.6.4 `bitset` operators [bitset.operators]

```
bitset<N> operator&(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

1 *Returns:* `bitset<N>(lhs) &= rhs`.

```
bitset<N> operator|(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

2 *Returns:* `bitset<N>(lhs) |= rhs`.

```
bitset<N> operator^(const bitset<N>& lhs, const bitset<N>& rhs) noexcept;
```

3 *Returns:* `bitset<N>(lhs) ^= rhs`.

```
template <class charT, class traits, size_t N>
  basic_istream<charT, traits>&
  operator>>(basic_istream<charT, traits>& is, bitset<N>& x);
```

4 A formatted input function (27.7.2.2).

5 *Effects:* Extracts up to `N` characters from `is`. Stores these characters in a temporary object `str` of type `basic_string<charT, traits>`, then evaluates the expression `x = bitset<N>(str)`. Characters are extracted and stored until any of the following occurs:

(5.1) — `N` characters have been extracted and stored;

(5.2) — end-of-file occurs on the input sequence;

(5.3) — the next input character is neither `is.widen('0')` nor `is.widen('1')` (in which case the input character is not extracted).

6 If no characters are stored in `str`, calls `is.setstate(ios_base::failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

7 *Returns:* `is`.

```
template <class charT, class traits, size_t N>
  basic_ostream<charT, traits>&
  operator<<(basic_ostream<charT, traits>& os, const bitset<N>& x);
```

8 *Returns:*

```
os << x.template to_string<charT,traits,allocator<charT> >(
  use_facet<ctype<charT> >(os.getloc()).widen('0'),
  use_facet<ctype<charT> >(os.getloc()).widen('1'))
```

(see 27.7.3.6).

20.7 Memory [memory]

20.7.1 In general [memory.general]

1 This subclause describes the contents of the header <memory> (20.7.2) and some of the contents of the C headers <cstdlib> and <cstring> (20.7.13).

20.7.2 Header <memory> synopsis [memory.syn]

1 The header <memory> defines several types and function templates that describe properties of pointers and pointer-like types, manage memory for containers and other template types, and construct multiple objects in uninitialized memory buffers (20.7.3–20.7.12). The header also defines the templates unique_ptr, shared_ptr, weak_ptr, and various function templates that operate on objects of these types (20.8).

```
namespace std {
  // 20.7.3, pointer traits
  template <class Ptr> struct pointer_traits;
  template <class T> struct pointer_traits<T*>;

  // 20.7.4, pointer safety
  enum class pointer_safety { relaxed, preferred, strict };
  void declare_reachable(void* p);
  template <class T> T* undeclare_reachable(T* p);
  void declare_no_pointers(char* p, size_t n);
  void undeclare_no_pointers(char* p, size_t n);
  pointer_safety get_pointer_safety() noexcept;

  // 20.7.5, pointer alignment function
  void* align(std::size_t alignment, std::size_t size,
    void*& ptr, std::size_t& space);

  // 20.7.6, allocator argument tag
  struct allocator_arg_t { };
  constexpr allocator_arg_t allocator_arg{};

  // 20.7.7, uses_allocator
  template <class T, class Alloc> struct uses_allocator;

  // 20.7.8, allocator traits
  template <class Alloc> struct allocator_traits;

  // 20.7.9, the default allocator:
  template <class T> class allocator;
  template <> class allocator<void>;
  template <class T, class U>
    bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
  template <class T, class U>
    bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

  // 20.7.10, raw storage iterator:
```

```
template <class OutputIterator, class T> class raw_storage_iterator;

// 20.7.11, temporary buffers:
template <class T>
  pair<T*,ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
template <class T>
  void return_temporary_buffer(T* p);

// 20.7.12, specialized algorithms:
template <class T> T* addressof(T& r) noexcept;
template <class InputIterator, class ForwardIterator>
  ForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                     ForwardIterator result);
template <class InputIterator, class Size, class ForwardIterator>
  ForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                       ForwardIterator result);
template <class ForwardIterator, class T>
  void uninitialized_fill(ForwardIterator first, ForwardIterator last,
                          const T& x);
template <class ForwardIterator, class Size, class T>
  ForwardIterator uninitialized_fill_n(ForwardIterator first, Size n, const T& x);

// 20.8.1 class template unique_ptr:
template <class T> struct default_delete;
template <class T> struct default_delete<T[]>;
template <class T, class D = default_delete<T>> class unique_ptr;
template <class T, class D> class unique_ptr<T[], D>;

template <class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
template <class T> unique_ptr<T> make_unique(size_t n);
template <class T, class... Args> unspecified  make_unique(Args&&...) = delete;

template <class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

template <class T1, class D1, class T2, class D2>
  bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template <class T1, class D1, class T2, class D2>
  bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template <class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template <class T1, class D1, class T2, class D2>
  bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template <class T1, class D1, class T2, class D2>
  bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
template <class T1, class D1, class T2, class D2>
  bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

template <class T, class D>
  bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template <class T, class D>
  bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
template <class T, class D>
  bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template <class T, class D>
  bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
```

```
template <class T, class D>
  bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator<(nullptr_t, const unique_ptr<T, D>& y);
template <class T, class D>
  bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
template <class T, class D>
  bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator>(nullptr_t, const unique_ptr<T, D>& y);
template <class T, class D>
  bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

// 20.8.2.1, class bad_weak_ptr:
class bad_weak_ptr;

// 20.8.2.2, class template shared_ptr:
template<class T> class shared_ptr;

// 20.8.2.2.6, shared_ptr creation
template<class T, class... Args> shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
  shared_ptr<T> allocate_shared(const A& a, Args&&... args);

// 20.8.2.2.7, shared_ptr comparisons:
template<class T, class U>
  bool operator==(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;
template<class T, class U>
  bool operator!=(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;
template<class T, class U>
  bool operator<(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;
template<class T, class U>
  bool operator>(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;
template<class T, class U>
  bool operator<=(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;
template<class T, class U>
  bool operator>=(shared_ptr<T> const& a, shared_ptr<U> const& b) noexcept;

template <class T>
  bool operator==(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator==(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
  bool operator!=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator!=(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
  bool operator<(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator<(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
```

```
  bool operator<=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator<=(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
  bool operator>(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator>(nullptr_t, const shared_ptr<T>& y) noexcept;
template <class T>
  bool operator>=(const shared_ptr<T>& x, nullptr_t) noexcept;
template <class T>
  bool operator>=(nullptr_t, const shared_ptr<T>& y) noexcept;

// 20.8.2.2.8, shared_ptr specialized algorithms:
template<class T> void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

// 20.8.2.2.9, shared_ptr casts:
template<class T, class U>
  shared_ptr<T> static_pointer_cast(shared_ptr<U> const& r) noexcept;
template<class T, class U>
  shared_ptr<T> dynamic_pointer_cast(shared_ptr<U> const& r) noexcept;
template<class T, class U>
  shared_ptr<T> const_pointer_cast(shared_ptr<U> const& r) noexcept;

// 20.8.2.2.10, shared_ptr get_deleter:
template<class D, class T> D* get_deleter(shared_ptr<T> const& p) noexcept;

// 20.8.2.2.11, shared_ptr I/O:
template<class E, class T, class Y>
  basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, shared_ptr<Y> const& p);

// 20.8.2.3, class template weak_ptr:
template<class T> class weak_ptr;

// 20.8.2.3.6, weak_ptr specialized algorithms:
template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;

// 20.8.2.4, class template owner_less:
template<class T> class owner_less;

// 20.8.2.5, class template enable_shared_from_this:
template<class T> class enable_shared_from_this;

// 20.8.2.6, shared_ptr atomic access:
template<class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);

template<class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
template<class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);

template<class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
template<class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

```
  template<class T>
    shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
  template<class T>
    shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r,
                                           memory_order mo);

  template<class T>
    bool atomic_compare_exchange_weak(
      shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
  template<class T>
    bool atomic_compare_exchange_strong(
      shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
  template<class T>
    bool atomic_compare_exchange_weak_explicit(
      shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
      memory_order success, memory_order failure);
  template<class T>
    bool atomic_compare_exchange_strong_explicit(
      shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
      memory_order success, memory_order failure);

  // 20.8.2.7 hash support
  template <class T> struct hash;
  template <class T, class D> struct hash<unique_ptr<T, D> >;
  template <class T> struct hash<shared_ptr<T> >;
}
```

20.7.3 Pointer traits [pointer.traits]

1 The class template `pointer_traits` supplies a uniform interface to certain attributes of pointer-like types.

```
namespace std {
  template <class Ptr> struct pointer_traits {
    typedef Ptr       pointer;
    typedef see below element_type;
    typedef see below difference_type;

    template <class U> using rebind = see below;

    static pointer pointer_to(see below r);
  };

  template <class T> struct pointer_traits<T*> {
    typedef T*        pointer;
    typedef T         element_type;
    typedef ptrdiff_t difference_type;

    template <class U> using rebind = U*;

    static pointer pointer_to(see below r) noexcept;
  };
}
```

20.7.3.1 Pointer traits member types [pointer.traits.types]

```
typedef see below element_type;
```

1 *Type:* `Ptr::element_type` if the *qualified-id* `Ptr::element_type` is valid and denotes a type (14.8.2); otherwise, `T` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the specialization is ill-formed.

```
typedef see below difference_type;
```

2 *Type:* `Ptr::difference_type` if the *qualified-id* `Ptr::difference_type` is valid and denotes a type (14.8.2); otherwise, `std::ptrdiff_t`.

```
template <class U> using rebind = see below;
```

3 *Alias template:* `Ptr::rebind<U>` if the *qualified-id* `Ptr::rebind<U>` is valid and denotes a type (14.8.2); otherwise, `SomePointer<U, Args>` if `Ptr` is a class template instantiation of the form `SomePointer<T, Args>`, where `Args` is zero or more type arguments; otherwise, the instantiation of `rebind` is ill-formed.

20.7.3.2 Pointer traits member functions [pointer.traits.functions]

```
static pointer pointer_traits::pointer_to(see below r);
static pointer pointer_traits<T*>::pointer_to(see below r) noexcept;
```

1 *Remark:* If `element_type` is (possibly cv-qualified) `void`, the type of `r` is unspecified; otherwise, it is `element_type&`.

2 *Returns:* The first member function returns a pointer to `r` obtained by calling `Ptr::pointer_to(r)` through which indirection is valid; an instantiation of this function is ill-formed if `Ptr` does not have a matching `pointer_to` static member function. The second member function returns `std::addressof(r)`.

20.7.4 Pointer safety [util.dynamic.safety]

1 A complete object is *declared reachable* while the number of calls to `declare_reachable` with an argument referencing the object exceeds the number of calls to `undeclare_reachable` with an argument referencing the object.

```
void declare_reachable(void* p);
```

2 *Requires:* `p` shall be a safely-derived pointer (3.7.4.3) or a null pointer value.

3 *Effects:* If `p` is not null, the complete object referenced by `p` is subsequently declared reachable (3.7.4.3).

4 *Throws:* May throw `std::bad_alloc` if the system cannot allocate additional memory that may be required to track objects declared reachable.

```
template <class T> T* undeclare_reachable(T* p);
```

5 *Requires:* If `p` is not null, the complete object referenced by `p` shall have been previously declared reachable, and shall be live (3.8) from the time of the call until the last `undeclare_reachable(p)` call on the object.

6 *Returns:* A safely derived copy of `p` which shall compare equal to `p`.

7 *Throws:* Nothing.

8 [*Note:* It is expected that calls to `declare_reachable(p)` will consume a small amount of memory in addition to that occupied by the referenced object until the matching call to `undeclare_reachable(p)` is encountered. Long running programs should arrange that calls are matched. — *end note*]

```
void declare_no_pointers(char* p, size_t n);
```

9 *Requires:* No bytes in the specified range are currently registered with `declare_no_pointers()`. If the specified range is in an allocated object, then it must be entirely within a single allocated object. The object must be live until the corresponding `undeclare_no_pointers()` call. [*Note:* In a garbage-collecting implementation, the fact that a region in an object is registered with `declare_no_pointers()` should not prevent the object from being collected. — *end note*]

10 *Effects:* The `n` bytes starting at `p` no longer contain traceable pointer locations, independent of their type. Hence indirection through a pointer located there is undefined if the object it points to was created by global `operator new` and not previously declared reachable. [*Note:* This may be used to inform a garbage collector or leak detector that this region of memory need not be traced. — *end note*]

11 *Throws:* Nothing.

12 [*Note:* Under some conditions implementations may need to allocate memory. However, the request can be ignored if memory allocation fails. — *end note*]

```
void undeclare_no_pointers(char* p, size_t n);
```

13 *Requires:* The same range must previously have been passed to `declare_no_pointers()`.

14 *Effects:* Unregisters a range registered with `declare_no_pointers()` for destruction. It must be called before the lifetime of the object ends.

15 *Throws:* Nothing.

```
pointer_safety get_pointer_safety() noexcept;
```

16 *Returns:* `pointer_safety::strict` if the implementation has strict pointer safety (3.7.4.3). It is implementation defined whether `get_pointer_safety` returns `pointer_safety::relaxed` or `pointer_safety::preferred` if the implementation has relaxed pointer safety.[230]

20.7.5 Align [ptr.align]

```
void* align(std::size_t alignment, std::size_t size,
    void*& ptr, std::size_t& space);
```

1 *Effects:* If it is possible to fit `size` bytes of storage aligned by `alignment` into the buffer pointed to by `ptr` with length `space`, the function updates `ptr` to point to the first possible address of such storage and decreases `space` by the number of bytes used for alignment. Otherwise, the function does nothing.

2 *Requires:*

(2.1) — `alignment` shall be a power of two

(2.2) — `ptr` shall point to contiguous storage of at least `space` bytes

3 *Returns:* A null pointer if the requested aligned buffer would not fit into the available space, otherwise the adjusted value of `ptr`.

4 [*Note:* The function updates its `ptr` and `space` arguments so that it can be called repeatedly with possibly different `alignment` and `size` arguments for the same buffer. — *end note*]

20.7.6 Allocator argument tag [allocator.tag]

```
namespace std {
  struct allocator_arg_t { };
  constexpr allocator_arg_t allocator_arg{};
}
```

230) `pointer_safety::preferred` might be returned to indicate that a leak detector is running so that the program can avoid spurious leak reports.

1 The `allocator_arg_t` struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, several types (see `tuple` 20.4) have constructors with `allocator_arg_t` as the first argument, immediately followed by an argument of a type that satisfies the `Allocator` requirements (17.6.3.5).

20.7.7 `uses_allocator` [allocator.uses]

20.7.7.1 `uses_allocator` trait [allocator.uses.trait]

```
template <class T, class Alloc> struct uses_allocator;
```

1 *Remarks:* automatically detects whether `T` has a nested `allocator_type` that is convertible from `Alloc`. Meets the BinaryTypeTrait requirements (20.10.1). The implementation shall provide a definition that is derived from `true_type` if the *qualified-id* `T::allocator_type` is valid and denotes a type (14.8.2) and `is_convertible<Alloc, T::allocator_type>::value != false`, otherwise it shall be derived from `false_type`. A program may specialize this template to derive from `true_type` for a user-defined type `T` that does not have a nested `allocator_type` but nonetheless can be constructed with an allocator where either:

(1.1) — the first argument of a constructor has type `allocator_arg_t` and the second argument has type `Alloc` or

(1.2) — the last argument of a constructor has type `Alloc`.

20.7.7.2 uses-allocator construction [allocator.uses.construction]

1 *Uses-allocator construction* with allocator `Alloc` refers to the construction of an object `obj` of type `T`, using constructor arguments `v1, v2, ..., vN` of types `V1, V2, ..., VN`, respectively, and an allocator `alloc` of type `Alloc`, according to the following rules:

(1.1) — if `uses_allocator<T, Alloc>::value` is `false` and `is_constructible<T, V1, V2, ..., VN>::value` is `true`, then `obj` is initialized as `obj(v1, v2, ..., vN)`;

(1.2) — otherwise, if `uses_allocator<T, Alloc>::value` is `true` and `is_constructible<T, allocator_arg_t, Alloc, V1, V2, ..., VN>::value` is `true`, then `obj` is initialized as `obj(allocator_arg, alloc, v1, v2, ..., vN)`;

(1.3) — otherwise, if `uses_allocator<T, Alloc>::value` is `true` and `is_constructible<T, V1, V2, ..., VN, Alloc>::value` is `true`, then `obj` is initialized as `obj(v1, v2, ..., vN, alloc)`;

(1.4) — otherwise, the request for uses-allocator construction is ill-formed. [*Note:* An error will result if `uses_allocator<T, Alloc>::value` is `true` but the specific constructor does not take an allocator. This definition prevents a silent failure to pass the allocator to an element. — *end note*]

20.7.8 Allocator traits [allocator.traits]

1 The class template `allocator_traits` supplies a uniform interface to all allocator types. An allocator cannot be a non-class type, however, even if `allocator_traits` supplies the entire required interface. [*Note:* Thus, it is always possible to create a derived class from an allocator. — *end note*]

```
namespace std {
  template <class Alloc> struct allocator_traits {
    typedef Alloc allocator_type;

    typedef typename Alloc::value_type value_type;

    typedef see below pointer;
    typedef see below const_pointer;
    typedef see below void_pointer;
```

```
    typedef see below const_void_pointer;

    typedef see below difference_type;
    typedef see below size_type;

    typedef see below propagate_on_container_copy_assignment;
    typedef see below propagate_on_container_move_assignment;
    typedef see below propagate_on_container_swap;
    typedef see below is_always_equal;

    template <class T> using rebind_alloc = see below;
    template <class T> using rebind_traits = allocator_traits<rebind_alloc<T> >;

    static pointer allocate(Alloc& a, size_type n);
    static pointer allocate(Alloc& a, size_type n, const_void_pointer hint);

    static void deallocate(Alloc& a, pointer p, size_type n);

    template <class T, class... Args>
      static void construct(Alloc& a, T* p, Args&&... args);

    template <class T>
      static void destroy(Alloc& a, T* p);

    static size_type max_size(const Alloc& a) noexcept;

    static Alloc select_on_container_copy_construction(const Alloc& rhs);
  };
}
```

20.7.8.1 Allocator traits member types [allocator.traits.types]

`typedef see below pointer;`

1 *Type:* `Alloc::pointer` if the *qualified-id* `Alloc::pointer` is valid and denotes a type (14.8.2); otherwise, `value_type*`.

`typedef see below const_pointer;`

2 *Type:* `Alloc::const_pointer` if the *qualified-id* `Alloc::const_pointer` is valid and denotes a type (14.8.2); otherwise, `pointer_traits<pointer>::rebind<const value_type>`.

`typedef see below void_pointer;`

3 *Type:* `Alloc::void_pointer` if the *qualified-id* `Alloc::void_pointer` is valid and denotes a type (14.8.2); otherwise, `pointer_traits<pointer>::rebind<void>`.

`typedef see below const_void_pointer;`

4 *Type:* `Alloc::const_void_pointer` if the *qualified-id* `Alloc::const_void_pointer` is valid and denotes a type (14.8.2); otherwise, `pointer_traits<pointer>::rebind<const void>`.

`typedef see below difference_type;`

5 *Type:* `Alloc::difference_type` if the *qualified-id* `Alloc::difference_type` is valid and denotes a type (14.8.2); otherwise, `pointer_traits<pointer>::difference_type`.

```
typedef see below size_type;
```

6 *Type:* `Alloc::size_type` if the *qualified-id* `Alloc::size_type` is valid and denotes a type (14.8.2); otherwise, `make_unsigned_t<difference_type>`.

```
typedef see below propagate_on_container_copy_assignment;
```

7 *Type:* `Alloc::propagate_on_container_copy_assignment` if the *qualified-id* `Alloc::propagate_on_container_copy_assignment` is valid and denotes a type (14.8.2); otherwise `false_type`.

```
typedef see below propagate_on_container_move_assignment;
```

8 *Type:* `Alloc::propagate_on_container_move_assignment` if the *qualified-id* `Alloc::propagate_on_container_move_assignment` is valid and denotes a type (14.8.2); otherwise `false_type`.

```
typedef see below propagate_on_container_swap;
```

9 *Type:* `Alloc::propagate_on_container_swap` if the *qualified-id* `Alloc::propagate_on_container_swap` is valid and denotes a type (14.8.2); otherwise `false_type`.

```
typedef see below is_always_equal;
```

10 *Type:* `Alloc::is_always_equal` if the *qualified-id* `Alloc::is_always_equal` is valid and denotes a type (14.8.2); otherwise `is_empty<Alloc>::type`.

```
template <class T> using rebind_alloc = see below;
```

11 *Alias template:* `Alloc::rebind<T>::other` if the *qualified-id* `Alloc::rebind<T>::other` is valid and denotes a type (14.8.2); otherwise, `Alloc<T, Args>` if `Alloc` is a class template instantiation of the form `Alloc<U, Args>`, where `Args` is zero or more type arguments; otherwise, the instantiation of `rebind_alloc` is ill-formed.

20.7.8.2 Allocator traits static member functions [allocator.traits.members]

```
static pointer allocate(Alloc& a, size_type n);
```

1 *Returns:* `a.allocate(n)`.

```
static pointer allocate(Alloc& a, size_type n, const_void_pointer hint);
```

2 *Returns:* `a.allocate(n, hint)` if that expression is well-formed; otherwise, `a.allocate(n)`.

```
static void deallocate(Alloc& a, pointer p, size_type n);
```

3 *Effects:* calls `a.deallocate(p, n)`.

4 *Throws:* Nothing.

```
template <class T, class... Args>
  static void construct(Alloc& a, T* p, Args&&... args);
```

5 *Effects:* calls `a.construct(p, std::forward<Args>(args)...)` if that call is well-formed; otherwise, invokes `::new (static_cast<void*>(p)) T(std::forward<Args>(args)...)`.

```
template <class T>
  static void destroy(Alloc& a, T* p);
```

6 *Effects:* calls `a.destroy(p)` if that call is well-formed; otherwise, invokes `p->~T()`.

```
static size_type max_size(const Alloc& a) noexcept;
```

7 *Returns:* `a.max_size()` if that expression is well-formed; otherwise, `numeric_limits<size_type>::max()`.

```
static Alloc select_on_container_copy_construction(const Alloc& rhs);
```

8 *Returns:* `rhs.select_on_container_copy_construction()` if that expression is well-formed; otherwise, `rhs`.

20.7.9 The default allocator [default.allocator]

1 All specializations of the default allocator satisfy the allocator completeness requirements 17.6.3.5.1.

```
namespace std {
  template <class T> class allocator;

  // specialize for void:
  template <> class allocator<void> {
  public:
    typedef void*   pointer;
    typedef const void* const_pointer;
    // reference-to-void members are impossible.
    typedef void  value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
  };

  template <class T> class allocator {
   public:
    typedef size_t    size_type;
    typedef ptrdiff_t difference_type;
    typedef T*        pointer;
    typedef const T*  const_pointer;
    typedef T&        reference;
    typedef const T&  const_reference;
    typedef T         value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
    typedef true_type propagate_on_container_move_assignment;
    typedef true_type is_always_equal;

    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template <class U> allocator(const allocator<U>&) noexcept;
   ~allocator();

    pointer address(reference x) const noexcept;
    const_pointer address(const_reference x) const noexcept;

    pointer allocate(
      size_type, allocator<void>::const_pointer hint = 0);
    void deallocate(pointer p, size_type n);
    size_type max_size() const noexcept;

    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);
  };
}
```

20.7.9.1 `allocator` members [allocator.members]

1 Except for the destructor, member functions of the default allocator shall not introduce data races (1.10) as a result of concurrent calls to those member functions from different threads. Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before the next allocation (if any) in this order.

```
pointer address(reference x) const noexcept;
```

2 *Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded operator&.

```
const_pointer address(const_reference x) const noexcept;
```

3 *Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded operator&.

```
pointer allocate(size_type n, allocator<void>::const_pointer hint = 0);
```

4 [*Note:* In a container member function, the address of an adjacent element is often a good choice to pass for the `hint` argument. — *end note*]

5 *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (3.11).

6 *Remark:* the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called. The use of `hint` is unspecified, but intended as an aid to locality if an implementation so desires.

7 *Throws:* `bad_alloc` if the storage cannot be obtained.

```
void deallocate(pointer p, size_type n);
```

8 *Requires:* `p` shall be a pointer value obtained from `allocate()`. `n` shall equal the value passed as the first argument to the invocation of allocate which returned `p`.

9 *Effects:* Deallocates the storage referenced by `p` .

10 *Remarks:* Uses `::operator delete(void*, std::size_t)` (18.6.1), but it is unspecified when this function is called.

```
size_type max_size() const noexcept;
```

11 *Returns:* The largest value N for which the call `allocate(N,0)` might succeed.

```
template <class U, class... Args>
  void construct(U* p, Args&&... args);
```

12 *Effects:* `::new((void *)p) U(std::forward<Args>(args)...)`

```
template <class U>
  void destroy(U* p);
```

13 *Effects:* `p->~U()`

20.7.9.2 `allocator` globals [allocator.globals]

```
template <class T1, class T2>
  bool operator==(const allocator<T1>&, const allocator<T2>&) noexcept;
```

1 *Returns:* `true`.

```
template <class T1, class T2>
  bool operator!=(const allocator<T1>&, const allocator<T2>&) noexcept;
```

2 *Returns:* `false`.

20.7.10 Raw storage iterator [storage.iterator]

1 `raw_storage_iterator` is provided to enable algorithms to store their results into uninitialized memory. The template parameter `OutputIterator` is required to have its `operator*` return an object for which `operator&` is defined and returns a pointer to `T`, and is also required to satisfy the requirements of an output iterator (24.2.4).

```
namespace std {
  template <class OutputIterator, class T>
  class raw_storage_iterator {
  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;

    explicit raw_storage_iterator(OutputIterator x);

    raw_storage_iterator& operator*();
    raw_storage_iterator& operator=(const T& element);
    raw_storage_iterator& operator++();
    raw_storage_iterator  operator++(int);
    OutputIterator base() const;
  };
}
```

`explicit raw_storage_iterator(OutputIterator x);`

2 *Effects:* Initializes the iterator to point to the same value to which `x` points.

`raw_storage_iterator& operator*();`

3 *Returns:* `*this`

`raw_storage_iterator& operator=(const T& element);`

4 *Effects:* Constructs a value from `element` at the location to which the iterator points.

5 *Returns:* A reference to the iterator.

`raw_storage_iterator& operator++();`

6 *Effects:* Pre-increment: advances the iterator and returns a reference to the updated iterator.

`raw_storage_iterator operator++(int);`

7 *Effects:* Post-increment: advances the iterator and returns the old value of the iterator.

`OutputIterator base() const;`

8 *Returns:* An iterator of type `OutputIterator` that points to the same value as `*this` points to.

20.7.11 Temporary buffers [temporary.buffer]

```
template <class T>
  pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

1 *Effects:* Obtains a pointer to storage sufficient to store up to `n` adjacent `T` objects. It is implementation-defined whether over-aligned types are supported (3.11).

2 *Returns:* A `pair` containing the buffer's address and capacity (in the units of `sizeof(T)`), or a pair of 0 values if no storage can be obtained or if `n <= 0`.

```
template <class T> void return_temporary_buffer(T* p);
```

3 *Effects:* Deallocates the buffer to which `p` points.

4 *Requires:* The buffer shall have been previously allocated by `get_temporary_buffer`.

20.7.12 Specialized algorithms [specialized.algorithms]

1 In the algorithm `uninitialized_copy`, the template parameter `InputIterator` is required to satisfy the requirements of an input iterator (24.2.3). In all of the following algorithms, the template parameter `ForwardIterator` is required to satisfy the requirements of a forward iterator (24.2.5), and is required to have the property that no exceptions are thrown from increment, assignment, comparison, or indirection through valid iterators. In the following algorithms, if an exception is thrown there are no effects.

20.7.12.1 `addressof` [specialized.addressof]

```
template <class T> T* addressof(T& r) noexcept;
```

1 *Returns:* The actual address of the object or function referenced by `r`, even in the presence of an overloaded `operator&`.

20.7.12.2 `uninitialized_copy` [uninitialized.copy]

```
template <class InputIterator, class ForwardIterator>
  ForwardIterator uninitialized_copy(InputIterator first, InputIterator last,
                                     ForwardIterator result);
```

1 *Effects:*

```
for (; first != last; ++result, ++first)
  ::new (static_cast<void*>(addressof(*result)))
    typename iterator_traits<ForwardIterator>::value_type(*first);
```

2 *Returns:* `result`

```
template <class InputIterator, class Size, class ForwardIterator>
  ForwardIterator uninitialized_copy_n(InputIterator first, Size n,
                                       ForwardIterator result);
```

3 *Effects:*

```
for ( ; n > 0; ++result, ++first, --n) {
  ::new (static_cast<void*>(addressof(*result)))
    typename iterator_traits<ForwardIterator>::value_type(*first);
}
```

4 *Returns:* `result`

20.7.12.3 `uninitialized_fill` [uninitialized.fill]

```
template <class ForwardIterator, class T>
  void uninitialized_fill(ForwardIterator first, ForwardIterator last,
                          const T& x);
```

1 *Effects:*

```
for (; first != last; ++first)
  ::new (static_cast<void*>(addressof(*first)))
    typename iterator_traits<ForwardIterator>::value_type(x);
```

20.7.12.4 `uninitialized_fill_n` [uninitialized.fill.n]

```
template <class ForwardIterator, class Size, class T>
  ForwardIterator uninitialized_fill_n(ForwardIterator first, Size n, const T& x);
```

1 *Effects:*

```
for (; n--; ++first)
  ::new (static_cast<void*>(addressof(*first)))
    typename iterator_traits<ForwardIterator>::value_type(x);
return first;
```

20.7.13 C library [c.malloc]

1 Table 45 describes the header `<cstdlib>`.

Table 45 — Header `<cstdlib>` synopsis

Type	Name(s)	
Functions:	`calloc`	`malloc`
	`free`	`realloc`

2 The contents are the same as the Standard C library header `<stdlib.h>`, with the following changes:

3 The functions `calloc()`, `malloc()`, and `realloc()` do not attempt to allocate storage by calling `::operator new()` (18.6).

4 The function `free()` does not attempt to deallocate storage by calling `::operator delete()`.

See also: ISO C Clause 7.11.2.

5 Storage allocated directly with `malloc()`, `calloc()`, or `realloc()` is implicitly declared reachable (see 3.7.4.3) on allocation, ceases to be declared reachable on deallocation, and need not cease to be declared reachable as the result of an `undeclare_reachable()` call. [*Note:* This allows existing C libraries to remain unaffected by restrictions on pointers that are not safely derived, at the expense of providing far fewer garbage collection and leak detection options for `malloc()`-allocated objects. It also allows `malloc()` to be implemented with a separate allocation arena, bypassing the normal `declare_reachable()` implementation. The above functions should never intentionally be used as a replacement for `declare_reachable()`, and newly written code is strongly encouraged to treat memory allocated with these functions as though it were allocated with `operator new`. — *end note*]

6 Table 46 describes the header `<cstring>`.

Table 46 — Header `<cstring>` synopsis

Type	Name(s)	
Macro:	`NULL`	
Type:	`size_t`	
Functions:	`memchr`	`memcmp`
`memcpy`	`memmove`	`memset`

7 The contents are the same as the Standard C library header `<string.h>`, with the change to `memchr()` specified in 21.8.

See also: ISO C Clause 7.11.2.

20.8 Smart pointers [smartptr]

20.8.1 Class template `unique_ptr` [unique.ptr]

1 A *unique pointer* is an object that owns another object and manages that other object through a pointer. More precisely, a unique pointer is an object u that stores a pointer to a second object p and will dispose of p when u is itself destroyed (e.g., when leaving block scope (6.7)). In this context, u is said to *own* `p`.

2 The mechanism by which u disposes of p is known as p's associated *deleter*, a function object whose correct invocation results in p's appropriate disposition (typically its deletion).

3 Let the notation *u.p* denote the pointer stored by u, and let *u.d* denote the associated deleter. Upon request, u can *reset* (replace) *u.p* and *u.d* with another pointer and deleter, but must properly dispose of its owned object via the associated deleter before such replacement is considered completed.

4 Additionally, u can, upon request, *transfer ownership* to another unique pointer *u2*. Upon completion of such a transfer, the following postconditions hold:

(4.1) — *u2.p* is equal to the pre-transfer *u.p*,

(4.2) — *u.p* is equal to `nullptr`, and

(4.3) — if the pre-transfer *u.d* maintained state, such state has been transferred to *u2.d*.

As in the case of a reset, *u2* must properly dispose of its pre-transfer owned object via the pre-transfer associated deleter before the ownership transfer is considered complete. [*Note:* A deleter's state need never be copied, only moved or swapped as ownership is transferred. — *end note*]

5 Each object of a type `U` instantiated from the `unique_ptr` template specified in this subclause has the strict ownership semantics, specified above, of a unique pointer. In partial satisfaction of these semantics, each such `U` is `MoveConstructible` and `MoveAssignable`, but is not `CopyConstructible` nor `CopyAssignable`. The template parameter `T` of `unique_ptr` may be an incomplete type.

6 [*Note:* The uses of `unique_ptr` include providing exception safety for dynamically allocated memory, passing ownership of dynamically allocated memory to a function, and returning dynamically allocated memory from a function. — *end note*]

```
namespace std {
  template<class T> struct default_delete;
  template<class T> struct default_delete<T[]>;

  template<class T, class D = default_delete<T>> class unique_ptr;
  template<class T, class D> class unique_ptr<T[], D>;

  template<class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
  template<class T> unique_ptr<T> make_unique(size_t n);
  template<class T, class... Args> unspecified  make_unique(Args&&...) = delete;

  template<class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;

  template<class T1, class D1, class T2, class D2>
    bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
```

```
    bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
  template<class T1, class D1, class T2, class D2>
    bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);

  template <class T, class D>
    bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
  template <class T, class D>
    bool operator==(nullptr_t, const unique_ptr<T, D>& y) noexcept;
  template <class T, class D>
    bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
  template <class T, class D>
    bool operator!=(nullptr_t, const unique_ptr<T, D>& y) noexcept;
  template <class T, class D>
    bool operator<(const unique_ptr<T, D>& x, nullptr_t);
  template <class T, class D>
    bool operator<(nullptr_t, const unique_ptr<T, D>& y);
  template <class T, class D>
    bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
  template <class T, class D>
    bool operator<=(nullptr_t, const unique_ptr<T, D>& y);
  template <class T, class D>
    bool operator>(const unique_ptr<T, D>& x, nullptr_t);
  template <class T, class D>
    bool operator>(nullptr_t, const unique_ptr<T, D>& y);
  template <class T, class D>
    bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
  template <class T, class D>
    bool operator>=(nullptr_t, const unique_ptr<T, D>& y);

}
```

20.8.1.1 Default deleters [unique.ptr.dltr]

20.8.1.1.1 In general [unique.ptr.dltr.general]

1 The class template `default_delete` serves as the default deleter (destruction policy) for the class template `unique_ptr`.

2 The template parameter `T` of `default_delete` may be an incomplete type.

20.8.1.1.2 `default_delete` [unique.ptr.dltr.dflt]

```
namespace std {
  template <class T> struct default_delete {
    constexpr default_delete() noexcept = default;
    template <class U> default_delete(const default_delete<U>&) noexcept;
    void operator()(T*) const;
  };
}
```

```
template <class U> default_delete(const default_delete<U>& other) noexcept;
```

1 *Effects:* Constructs a `default_delete` object from another `default_delete<U>` object.

2 *Remarks:* This constructor shall not participate in overload resolution unless `U*` is implicitly convertible to `T*`.

```
void operator()(T* ptr) const;
```

3 *Effects:* calls `delete` on `ptr`.

4 *Remarks:* If `T` is an incomplete type, the program is ill-formed.

20.8.1.1.3 `default_delete<T[]>` [unique.ptr.dltr.dflt1]

```
namespace std {
  template <class T> struct default_delete<T[]> {
    constexpr default_delete() noexcept = default;
    template <class U> default_delete(const default_delete<U[]>&) noexcept;
    template <class U> void operator()(U* ptr) const;
  };
}
```

```
template <class U> default_delete(const default_delete<U[]>& other) noexcept;
```

1 *Effects:* constructs a `default_delete` object from another `default_delete<U[]>` object.

2 *Remarks:* This constructor shall not participate in overload resolution unless `U(*)[]` is convertible to `T(*)[]`.

```
template <class U> void operator()(U* ptr) const;
```

3 *Effects:* calls `delete[]` on `ptr`.

4 *Remarks:* If U is an incomplete type, the program is ill-formed. This function shall not participate in overload resolution unless `U(*)[]` is convertible to `T(*)[]`.

20.8.1.2 `unique_ptr` for single objects [unique.ptr.single]

```
namespace std {
  template <class T, class D = default_delete<T>> class unique_ptr {
  public:
    typedef see below pointer;
    typedef T element_type;
    typedef D deleter_type;

    // 20.8.1.2.1, constructors
    constexpr unique_ptr() noexcept;
    explicit unique_ptr(pointer p) noexcept;
    unique_ptr(pointer p, see below d1) noexcept;
    unique_ptr(pointer p, see below d2) noexcept;
    unique_ptr(unique_ptr&& u) noexcept;
    constexpr unique_ptr(nullptr_t) noexcept
      : unique_ptr() { }
    template <class U, class E>
      unique_ptr(unique_ptr<U, E>&& u) noexcept;

    // 20.8.1.2.2, destructor
    ~unique_ptr();

    // 20.8.1.2.3, assignment
    unique_ptr& operator=(unique_ptr&& u) noexcept;
    template <class U, class E> unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
    unique_ptr& operator=(nullptr_t) noexcept;

    // 20.8.1.2.4, observers
```

```
    add_lvalue_reference_t<T> operator*() const;
    pointer operator->() const noexcept;
    pointer get() const noexcept;
    deleter_type& get_deleter() noexcept;
    const deleter_type& get_deleter() const noexcept;
    explicit operator bool() const noexcept;

    // 20.8.1.2.5 modifiers
    pointer release() noexcept;
    void reset(pointer p = pointer()) noexcept;
    void swap(unique_ptr& u) noexcept;

    // disable copy from lvalue
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator=(const unique_ptr&) = delete;
  };
}
```

1 The default type for the template parameter `D` is `default_delete`. A client-supplied template argument `D` shall be a function object type (20.9), lvalue reference to function, or lvalue reference to function object type for which, given a value `d` of type `D` and a value `ptr` of type `unique_ptr<T, D>::pointer`, the expression `d(ptr)` is valid and has the effect of disposing of the pointer as appropriate for that deleter.

2 If the deleter's type `D` is not a reference type, `D` shall satisfy the requirements of `Destructible` (Table 24).

3 If the *qualified-id* `remove_reference_t<D>::pointer` is valid and denotes a type (14.8.2), then `unique_ptr<T, D>::pointer` shall be a synonym for `remove_reference_t<D>::pointer`. Otherwise `unique_ptr<T, D>::pointer` shall be a synonym for `element_type*`. The type `unique_ptr<T, D>::pointer` shall satisfy the requirements of `NullablePointer` (17.6.3.3).

4 [*Example:* Given an allocator type `X` (17.6.3.5) and letting `A` be a synonym for `allocator_traits<X>`, the types `A::pointer`, `A::const_pointer`, `A::void_pointer`, and `A::const_void_pointer` may be used as `unique_ptr<T, D>::pointer`. — *end example*]

20.8.1.2.1 `unique_ptr` constructors [unique.ptr.single.ctor]

`constexpr unique_ptr() noexcept;`

1 *Requires:* `D` shall satisfy the requirements of `DefaultConstructible` (Table 19), and that construction shall not throw an exception.

2 *Effects:* Constructs a `unique_ptr` object that owns nothing, value-initializing the stored pointer and the stored deleter.

3 *Postconditions:* `get() == nullptr`. `get_deleter()` returns a reference to the stored deleter.

4 *Remarks:* If this constructor is instantiated with a pointer type or reference type for the template argument `D`, the program is ill-formed.

`explicit unique_ptr(pointer p) noexcept;`

5 *Requires:* `D` shall satisfy the requirements of `DefaultConstructible` (Table 19), and that construction shall not throw an exception.

6 *Effects:* Constructs a `unique_ptr` which owns `p`, initializing the stored pointer with `p` and value-initializing the stored deleter.

7 *Postconditions:* `get() == p`. `get_deleter()` returns a reference to the stored deleter.

8 *Remarks:* If this constructor is instantiated with a pointer type or reference type for the template argument `D`, the program is ill-formed.

```
unique_ptr(pointer p, see below d1) noexcept;
unique_ptr(pointer p, see below d2) noexcept;
```

9 The signature of these constructors depends upon whether `D` is a reference type. If `D` is non-reference type `A`, then the signatures are:

```
unique_ptr(pointer p, const A& d);
unique_ptr(pointer p, A&& d);
```

10 If `D` is an lvalue reference type `A&`, then the signatures are:

```
unique_ptr(pointer p, A& d);
unique_ptr(pointer p, A&& d);
```

11 If `D` is an lvalue reference type `const A&`, then the signatures are:

```
unique_ptr(pointer p, const A& d);
unique_ptr(pointer p, const A&& d);
```

12 *Requires:*

(12.1) — If `D` is not an lvalue reference type then

(12.1.1) — If `d` is an lvalue or `const` rvalue then the first constructor of this pair will be selected. `D` shall satisfy the requirements of `CopyConstructible` (Table 21), and the copy constructor of `D` shall not throw an exception. This `unique_ptr` will hold a copy of `d`.

(12.1.2) — Otherwise, `d` is a non-const rvalue and the second constructor of this pair will be selected. `D` shall satisfy the requirements of `MoveConstructible` (Table 20), and the move constructor of `D` shall not throw an exception. This `unique_ptr` will hold a value move constructed from `d`.

(12.2) — Otherwise `D` is an lvalue reference type. `d` shall be reference-compatible with one of the constructors. If `d` is an rvalue, it will bind to the second constructor of this pair and the program is ill-formed. [*Note:* The diagnostic could be implemented using a `static_assert` which assures that `D` is not a reference type. — *end note*] Else `d` is an lvalue and will bind to the first constructor of this pair. The type which `D` references need not be `CopyConstructible` nor `MoveConstructible`. This `unique_ptr` will hold a `D` which refers to the lvalue `d`. [*Note:* `D` may not be an rvalue reference type. — *end note*]

13 *Effects:* Constructs a `unique_ptr` object which owns `p`, initializing the stored pointer with `p` and initializing the deleter as described above.

14 *Postconditions:* `get() == p`. `get_deleter()` returns a reference to the stored deleter. If `D` is a reference type then `get_deleter()` returns a reference to the lvalue `d`.

[*Example:*

```
D d;
unique_ptr<int, D> p1(new int, D());        // D must be MoveConstructible
unique_ptr<int, D> p2(new int, d);          // D must be CopyConstructible
unique_ptr<int, D&> p3(new int, d);         // p3 holds a reference to d
unique_ptr<int, const D&> p4(new int, D()); // error: rvalue deleter object combined
                                            // with reference deleter type
```

— *end example*]

```
unique_ptr(unique_ptr&& u) noexcept;
```

15 *Requires:* If `D` is not a reference type, `D` shall satisfy the requirements of `MoveConstructible` (Table 20). Construction of the deleter from an rvalue of type `D` shall not throw an exception.

16 *Effects:* Constructs a `unique_ptr` by transferring ownership from `u` to `*this`. If `D` is a reference type, this deleter is copy constructed from `u`'s deleter; otherwise, this deleter is move constructed from `u`'s deleter. [*Note:* The deleter constructor can be implemented with `std::forward<D>`. — *end note*]

17 *Postconditions:* `get()` yields the value `u.get()` yielded before the construction. `get_deleter()` returns a reference to the stored deleter that was constructed from `u.get_deleter()`. If `D` is a reference type then `get_deleter()` and `u.get_deleter()` both reference the same lvalue deleter.

```
template <class U, class E> unique_ptr(unique_ptr<U, E>&& u) noexcept;
```

18 *Requires:* If `E` is not a reference type, construction of the deleter from an rvalue of type `E` shall be well formed and shall not throw an exception. Otherwise, `E` is a reference type and construction of the deleter from an lvalue of type `E` shall be well formed and shall not throw an exception.

19 *Remarks:* This constructor shall not participate in overload resolution unless:

(19.1) — `unique_ptr<U, E>::pointer` is implicitly convertible to `pointer`,

(19.2) — `U` is not an array type, and

(19.3) — either `D` is a reference type and `E` is the same type as `D`, or `D` is not a reference type and `E` is implicitly convertible to `D`.

20 *Effects:* Constructs a `unique_ptr` by transferring ownership from `u` to `*this`. If `E` is a reference type, this deleter is copy constructed from `u`'s deleter; otherwise, this deleter is move constructed from `u`'s deleter. [*Note:* The deleter constructor can be implemented with `std::forward<E>`. — *end note*]

21 *Postconditions:* `get()` yields the value `u.get()` yielded before the construction. `get_deleter()` returns a reference to the stored deleter that was constructed from `u.get_deleter()`.

20.8.1.2.2 `unique_ptr` destructor [unique.ptr.single.dtor]

```
~unique_ptr();
```

1 *Requires:* The expression `get_deleter()(get())` shall be well formed, shall have well-defined behavior, and shall not throw exceptions. [*Note:* The use of `default_delete` requires `T` to be a complete type. — *end note*]

2 *Effects:* If `get() == nullptr` there are no effects. Otherwise `get_deleter()(get())`.

20.8.1.2.3 `unique_ptr` assignment [unique.ptr.single.asgn]

```
unique_ptr& operator=(unique_ptr&& u) noexcept;
```

1 *Requires:* If `D` is not a reference type, `D` shall satisfy the requirements of `MoveAssignable` (Table 22) and assignment of the deleter from an rvalue of type `D` shall not throw an exception. Otherwise, `D` is a reference type; `remove_reference_t<D>` shall satisfy the `CopyAssignable` requirements and assignment of the deleter from an lvalue of type `D` shall not throw an exception.

2 *Effects:* Transfers ownership from `u` to `*this` as if by calling `reset(u.release())` followed by `get_deleter() = std::forward<D>(u.get_deleter())`.

3 *Returns:* `*this`.

```
template <class U, class E> unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
```

4 *Requires:* If `E` is not a reference type, assignment of the deleter from an rvalue of type `E` shall be well-formed and shall not throw an exception. Otherwise, `E` is a reference type and assignment of the deleter from an lvalue of type `E` shall be well-formed and shall not throw an exception.

5 *Remarks:* This operator shall not participate in overload resolution unless:

(5.1) — `unique_ptr<U, E>::pointer` is implicitly convertible to `pointer`, and

(5.2) — `U` is not an array type, and

(5.3) — `is_assignable<D&, E&&>::value` is `true`.

6 *Effects:* Transfers ownership from `u` to `*this` as if by calling `reset(u.release())` followed by `get_deleter() = std::forward<E>(u.get_deleter())`.

7 *Returns:* `*this`.

```
unique_ptr& operator=(nullptr_t) noexcept;
```

8 *Effects:* `reset()`.

9 *Postcondition:* `get() == nullptr`

10 *Returns:* `*this`.

20.8.1.2.4 `unique_ptr` observers [unique.ptr.single.observers]

```
add_lvalue_reference_t<T> operator*() const;
```

1 *Requires:* `get() != nullptr`.

2 *Returns:* `*get()`.

```
pointer operator->() const noexcept;
```

3 *Requires:* `get() != nullptr`.

4 *Returns:* `get()`.

5 *Note:* use typically requires that `T` be a complete type.

```
pointer get() const noexcept;
```

6 *Returns:* The stored pointer.

```
deleter_type& get_deleter() noexcept;
const deleter_type& get_deleter() const noexcept;
```

7 *Returns:* A reference to the stored deleter.

```
explicit operator bool() const noexcept;
```

8 *Returns:* `get() != nullptr`.

20.8.1.2.5 `unique_ptr` modifiers [unique.ptr.single.modifiers]

```
pointer release() noexcept;
```

1 *Postcondition:* `get() == nullptr`.

2 *Returns:* The value `get()` had at the start of the call to `release`.

```
void reset(pointer p = pointer()) noexcept;
```

3 *Requires:* The expression `get_deleter()(get())` shall be well formed, shall have well-defined behavior, and shall not throw exceptions.

4 *Effects:* assigns `p` to the stored pointer, and then if the old value of the stored pointer, `old_p`, was not equal to `nullptr`, calls `get_deleter()(old_p)`. [*Note:* The order of these operations is significant because the call to `get_deleter()` may destroy `*this`. — *end note*]

5 *Postconditions:* `get() == p`. [*Note:* The postcondition does not hold if the call to `get_deleter()` destroys `*this` since `this->get()` is no longer a valid expression. — *end note*]

```
void swap(unique_ptr& u) noexcept;
```

6 *Requires:* `get_deleter()` shall be swappable (17.6.3.2) and shall not throw an exception under `swap`.

7 *Effects:* Invokes `swap` on the stored pointers and on the stored deleters of `*this` and `u`.

20.8.1.3 `unique_ptr` for array objects with a runtime length [unique.ptr.runtime]

```
namespace std {
  template <class T, class D> class unique_ptr<T[], D> {
  public:
    typedef see below pointer;
    typedef T element_type;
    typedef D deleter_type;

    // 20.8.1.3.1, constructors
    constexpr unique_ptr() noexcept;
    template <class U> explicit unique_ptr(U p) noexcept;
    template <class U> unique_ptr(U p, see below d) noexcept;
    template <class U> unique_ptr(U p, see below d) noexcept;
    unique_ptr(unique_ptr&& u) noexcept;
    template <class U, class E>
      unique_ptr(unique_ptr<U, E>&& u) noexcept;
    constexpr unique_ptr(nullptr_t) noexcept : unique_ptr() { }

    // destructor
    ~unique_ptr();

    // assignment
    unique_ptr& operator=(unique_ptr&& u) noexcept;
    template <class U, class E>
      unique_ptr& operator=(unique_ptr<U, E>&& u) noexcept;
    unique_ptr& operator=(nullptr_t) noexcept;

    // 20.8.1.3.3, observers
    T& operator[](size_t i) const;
    pointer get() const noexcept;
    deleter_type& get_deleter() noexcept;
    const deleter_type& get_deleter() const noexcept;
    explicit operator bool() const noexcept;

    // 20.8.1.3.4 modifiers
    pointer release() noexcept;
    template <class U> void reset(U p) noexcept;
    void reset(nullptr_t = nullptr) noexcept;
    void swap(unique_ptr& u) noexcept;

    // disable copy from lvalue
    unique_ptr(const unique_ptr&) = delete;
    unique_ptr& operator=(const unique_ptr&) = delete;
  };
}
```

1 A specialization for array types is provided with a slightly altered interface.

(1.1) — Conversions between different types of `unique_ptr<T[], D>` that would be disallowed for the corresponding pointer-to-array types, and conversions to or from the non-array forms of `unique_ptr`, produce an ill-formed program.

(1.2) — Pointers to types derived from `T` are rejected by the constructors, and by `reset`.

(1.3) — The observers `operator*` and `operator->` are not provided.

(1.4) — The indexing observer `operator[]` is provided.

(1.5) — The default deleter will call `delete[]`.

2 Descriptions are provided below only for members that differ from the primary template.

3 The template argument `T` shall be a complete type.

20.8.1.3.1 `unique_ptr` constructors [unique.ptr.runtime.ctor]

```
template <class U> explicit unique_ptr(U p) noexcept;
template <class U> unique_ptr(U p, see below d) noexcept;
template <class U> unique_ptr(U p, see below d) noexcept;
```

1 These constructors behave the same as the constructors that take a `pointer` parameter in the primary template except that they shall not participate in overload resolution unless either

(1.1) — `U` is the same type as `pointer`, or

(1.2) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, and `V(*)[]` is convertible to `element_type(*)[]`.

```
template <class U, class E>
  unique_ptr(unique_ptr<U, E>&& u) noexcept;
```

2 This constructor behaves the same as in the primary template, except that it shall not participate in overload resolution unless all of the following conditions hold, where `UP` is `unique_ptr<U, E>`:

(2.1) — `U` is an array type, and

(2.2) — `pointer` is the same type as element_type*, and

(2.3) — `UP::pointer` is the same type as `UP::element_type*`, and

(2.4) — `UP::element_type(*)[]` is convertible to `element_type(*)[]`, and

(2.5) — either `D` is a reference type and `E` is the same type as `D`, or `D` is not a reference type and `E` is implicitly convertible to `D`.

[*Note:* this replaces the overload-resolution specification of the primary template — *end note*]

20.8.1.3.2 `unique_ptr` assignment [unique.ptr.runtime.asgn]

```
template <class U, class E>
  unique_ptr& operator=(unique_ptr<U,E>&& u)noexcept;
```

1 This operator behaves the same as in the primary template, except that it shall not participate in overload resolution unless all of the following conditions hold, where `UP` is `unique_ptr<U, E>`:

(1.1) — `U` is an array type, and

(1.2) — `pointer` is the same type as `element_type*`, and

(1.3) — `UP::pointer` is the same type as `UP::element_type*`, and

(1.4) — `UP::element_type(*)[]` is convertible to `element_type(*)[]`, and

(1.5) — `is_assignable<D&, E&&>::value` is `true`.

[*Note:* this replaces the overload-resolution specification of the primary template — *end note*]

20.8.1.3.3 `unique_ptr` observers [unique.ptr.runtime.observers]

```
T& operator[](size_t i) const;
```

1 *Requires:* `i <` the number of elements in the array to which the stored pointer points.

2 *Returns:* `get()[i]`.

20.8.1.3.4 `unique_ptr` modifiers [unique.ptr.runtime.modifiers]

```
void reset(nullptr_t p = nullptr) noexcept;
```

1 *Effects:* Equivalent to `reset(pointer())`.

```
template <class U> void reset(U p) noexcept;
```

2 This function behaves the same as the `reset` member of the primary template, except that it shall not participate in overload resolution unless either

(2.1) — `U` is the same type as `pointer`, or

(2.2) — `pointer` is the same type as `element_type*`, `U` is a pointer type `V*`, and `V(*)[]` is convertible to `element_type(*)[]`.

20.8.1.4 `unique_ptr` creation [unique.ptr.create]

```
template <class T, class... Args> unique_ptr<T> make_unique(Args&&... args);
```

1 *Remarks:* This function shall not participate in overload resolution unless `T` is not an array.

2 *Returns:* `unique_ptr<T>(new T(std::forward<Args>(args)...))`.

```
template <class T> unique_ptr<T> make_unique(size_t n);
```

3 *Remarks:* This function shall not participate in overload resolution unless `T` is an array of unknown bound.

4 *Returns:* `unique_ptr<T>(new remove_extent_t<T>[n]())`.

```
template <class T, class... Args> unspecified  make_unique(Args&&...) = delete;
```

5 *Remarks:* This function shall not participate in overload resolution unless `T` is an array of known bound.

20.8.1.5 `unique_ptr` specialized algorithms [unique.ptr.special]

```
template <class T, class D> void swap(unique_ptr<T, D>& x, unique_ptr<T, D>& y) noexcept;
```

1 *Effects:* Calls `x.swap(y)`.

```
template <class T1, class D1, class T2, class D2>
  bool operator==(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

2 *Returns:* `x.get() == y.get()`.

```
template <class T1, class D1, class T2, class D2>
  bool operator!=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

3 *Returns:* `x.get() != y.get()`.

```
template <class T1, class D1, class T2, class D2>
  bool operator<(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

4 *Requires:* Let `CT` be `common_type<unique_ptr<T1, D1>::pointer, unique_ptr<T2, D2>::pointer>::type`. Then the specialization `less<CT>` shall be a function object type (20.9) that induces a strict weak ordering (25.4) on the pointer values.

5 *Returns:* `less<CT>()(x.get(), y.get())`.

6 *Remarks:* If `unique_ptr<T1, D1>::pointer` is not implicitly convertible to `CT` or `unique_ptr<T2, D2>::pointer` is not implicitly convertible to `CT`, the program is ill-formed.

```
template <class T1, class D1, class T2, class D2>
  bool operator<=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

7 *Returns:* `!(y < x)`.

```
template <class T1, class D1, class T2, class D2>
  bool operator>(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

8 *Returns:* `y < x`.

```
template <class T1, class D1, class T2, class D2>
  bool operator>=(const unique_ptr<T1, D1>& x, const unique_ptr<T2, D2>& y);
```

9 *Returns:* `!(x < y)`.

```
template <class T, class D>
  bool operator==(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template <class T, class D>
  bool operator==(nullptr_t, const unique_ptr<T, D>& x) noexcept;
```

10 *Returns:* `!x`.

```
template <class T, class D>
  bool operator!=(const unique_ptr<T, D>& x, nullptr_t) noexcept;
template <class T, class D>
  bool operator!=(nullptr_t, const unique_ptr<T, D>& x) noexcept;
```

11 *Returns:* `(bool)x`.

```
template <class T, class D>
  bool operator<(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator<(nullptr_t, const unique_ptr<T, D>& x);
```

12 *Requires:* The specialization `less<unique_ptr<T, D>::pointer>` shall be a function object type (20.9) that induces a strict weak ordering (25.4) on the pointer values.

13 *Returns:* The first function template returns `less<unique_ptr<T, D>::pointer>()(x.get(), nullptr)`. The second function template returns `less<unique_ptr<T, D>::pointer>()(nullptr, x.get())`.

```
template <class T, class D>
  bool operator>(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator>(nullptr_t, const unique_ptr<T, D>& x);
```

14 *Returns:* The first function template returns `nullptr < x`. The second function template returns `x < nullptr`.

```
template <class T, class D>
  bool operator<=(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator<=(nullptr_t, const unique_ptr<T, D>& x);
```

15 *Returns:* The first function template returns `!(nullptr < x)`. The second function template returns `!(x < nullptr)`.

```
template <class T, class D>
  bool operator>=(const unique_ptr<T, D>& x, nullptr_t);
template <class T, class D>
  bool operator>=(nullptr_t, const unique_ptr<T, D>& x);
```

16 *Returns:* The first function template returns `!(x < nullptr)`. The second function template returns `!(nullptr < x)`.

20.8.2 Shared-ownership pointers [util.smartptr]

20.8.2.1 Class `bad_weak_ptr` [util.smartptr.weakptr]

```
namespace std {
  class bad_weak_ptr: public std::exception {
  public:
    bad_weak_ptr() noexcept;
  };
} // namespace std
```

1 An exception of type `bad_weak_ptr` is thrown by the `shared_ptr` constructor taking a `weak_ptr`.

```
bad_weak_ptr() noexcept;
```

2 *Postconditions:* `what()` returns an implementation-defined NTBS.

20.8.2.2 Class template `shared_ptr` [util.smartptr.shared]

1 The `shared_ptr` class template stores a pointer, usually obtained via `new`. `shared_ptr` implements semantics of shared ownership; the last remaining owner of the pointer is responsible for destroying the object, or otherwise releasing the resources associated with the stored pointer. A `shared_ptr` object is *empty* if it does not own a pointer.

```
namespace std {
  template<class T> class shared_ptr {
  public:
    typedef T element_type;

    // 20.8.2.2.1, constructors:
    constexpr shared_ptr() noexcept;
    template<class Y> explicit shared_ptr(Y* p);
    template<class Y, class D> shared_ptr(Y* p, D d);
    template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
    template <class D> shared_ptr(nullptr_t p, D d);
    template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
    template<class Y> shared_ptr(const shared_ptr<Y>& r, T* p) noexcept;
    shared_ptr(const shared_ptr& r) noexcept;
    template<class Y> shared_ptr(const shared_ptr<Y>& r) noexcept;
    shared_ptr(shared_ptr&& r) noexcept;
    template<class Y> shared_ptr(shared_ptr<Y>&& r) noexcept;
    template<class Y> explicit shared_ptr(const weak_ptr<Y>& r);
    template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

```
  constexpr shared_ptr(nullptr_t) noexcept : shared_ptr() { }

  // 20.8.2.2.2, destructor:
  ~shared_ptr();

  // 20.8.2.2.3, assignment:
  shared_ptr& operator=(const shared_ptr& r) noexcept;
  template<class Y> shared_ptr& operator=(const shared_ptr<Y>& r) noexcept;
  shared_ptr& operator=(shared_ptr&& r) noexcept;
  template<class Y> shared_ptr& operator=(shared_ptr<Y>&& r) noexcept;
  template <class Y, class D> shared_ptr& operator=(unique_ptr<Y, D>&& r);

  // 20.8.2.2.4, modifiers:
  void swap(shared_ptr& r) noexcept;
  void reset() noexcept;
  template<class Y> void reset(Y* p);
  template<class Y, class D> void reset(Y* p, D d);
  template<class Y, class D, class A> void reset(Y* p, D d, A a);

  // 20.8.2.2.5, observers:
  T* get() const noexcept;
  T& operator*() const noexcept;
  T* operator->() const noexcept;
  long use_count() const noexcept;
  bool unique() const noexcept;
  explicit operator bool() const noexcept;
  template<class U> bool owner_before(shared_ptr<U> const& b) const;
  template<class U> bool owner_before(weak_ptr<U> const& b) const;
};

// 20.8.2.2.6, shared_ptr creation
template<class T, class... Args> shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
  shared_ptr<T> allocate_shared(const A& a, Args&&... args);

// 20.8.2.2.7, shared_ptr comparisons:
template<class T, class U>
  bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
  bool operator!=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
  bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
  bool operator>(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
  bool operator<=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
template<class T, class U>
  bool operator>=(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;

template <class T>
  bool operator==(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator==(nullptr_t, const shared_ptr<T>& b) noexcept;
template <class T>
  bool operator!=(const shared_ptr<T>& a, nullptr_t) noexcept;
```

```
  template <class T>
    bool operator!=(nullptr_t, const shared_ptr<T>& b) noexcept;
  template <class T>
    bool operator<(const shared_ptr<T>& a, nullptr_t) noexcept;
  template <class T>
    bool operator<(nullptr_t, const shared_ptr<T>& b) noexcept;
  template <class T>
    bool operator<=(const shared_ptr<T>& a, nullptr_t) noexcept;
  template <class T>
    bool operator<=(nullptr_t, const shared_ptr<T>& b) noexcept;
  template <class T>
    bool operator>(const shared_ptr<T>& a, nullptr_t) noexcept;
  template <class T>
    bool operator>(nullptr_t, const shared_ptr<T>& b) noexcept;
  template <class T>
    bool operator>=(const shared_ptr<T>& a, nullptr_t) noexcept;
  template <class T>
    bool operator>=(nullptr_t, const shared_ptr<T>& b) noexcept;

  // 20.8.2.2.8, shared_ptr specialized algorithms:
  template<class T> void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;

  // 20.8.2.2.9, shared_ptr casts:
  template<class T, class U>
    shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
  template<class T, class U>
    shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
  template<class T, class U>
    shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;

  // 20.8.2.2.10, shared_ptr get_deleter:
  template<class D, class T> D* get_deleter(const shared_ptr<T>& p) noexcept;

  // 20.8.2.2.11, shared_ptr I/O:
  template<class E, class T, class Y>
    basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, const shared_ptr<Y>& p);
} // namespace std
```

2 Specializations of `shared_ptr` shall be `CopyConstructible`, `CopyAssignable`, and `LessThanComparable`, allowing their use in standard containers. Specializations of `shared_ptr` shall be contextually convertible to `bool`, allowing their use in boolean expressions and declarations in conditions. The template parameter `T` of `shared_ptr` may be an incomplete type.

3 [*Example:*

```
if(shared_ptr<X> px = dynamic_pointer_cast<X>(py)) {
  // do something with px
}
```

— *end example*]

4 For purposes of determining the presence of a data race, member functions shall access and modify only the `shared_ptr` and `weak_ptr` objects themselves and not objects they refer to. Changes in `use_count()` do not reflect modifications that can introduce data races.

20.8.2.2.1 `shared_ptr` constructors [util.smartptr.shared.const]

```
constexpr shared_ptr() noexcept;
```

1 *Effects:* Constructs an *empty* `shared_ptr` object.

2 *Postconditions:* `use_count() == 0 && get() == nullptr`.

```
template<class Y> explicit shared_ptr(Y* p);
```

3 *Requires:* `p` shall be convertible to `T*`. `Y` shall be a complete type. The expression `delete p` shall be well formed, shall have well defined behavior, and shall not throw exceptions.

4 *Effects:* Constructs a `shared_ptr` object that *owns* the pointer `p`.

5 *Postconditions:* `use_count() == 1 && get() == p`.

6 *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory could not be obtained.

7 *Exception safety:* If an exception is thrown, `delete p` is called.

```
template<class Y, class D> shared_ptr(Y* p, D d);
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);
template <class D> shared_ptr(nullptr_t p, D d);
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

8 *Requires:* `p` shall be convertible to `T*`. `D` shall be `CopyConstructible`. The copy constructor and destructor of `D` shall not throw exceptions. The expression `d(p)` shall be well formed, shall have well defined behavior, and shall not throw exceptions. `A` shall be an allocator (17.6.3.5). The copy constructor and destructor of `A` shall not throw exceptions.

9 *Effects:* Constructs a `shared_ptr` object that *owns* the object `p` and the deleter `d`. The second and fourth constructors shall use a copy of `a` to allocate memory for internal use.

10 *Postconditions:* `use_count() == 1 && get() == p`.

11 *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory could not be obtained.

12 *Exception safety:* If an exception is thrown, `d(p)` is called.

```
template<class Y> shared_ptr(const shared_ptr<Y>& r, T* p) noexcept;
```

13 *Effects:* Constructs a `shared_ptr` instance that stores `p` and *shares ownership* with `r`.

14 *Postconditions:* `get() == p && use_count() == r.use_count()`

15 [*Note:* To avoid the possibility of a dangling pointer, the user of this constructor must ensure that `p` remains valid at least until the ownership group of `r` is destroyed. — *end note*]

16 [*Note:* This constructor allows creation of an *empty* `shared_ptr` instance with a non-null stored pointer. — *end note*]

```
shared_ptr(const shared_ptr& r) noexcept;
template<class Y> shared_ptr(const shared_ptr<Y>& r) noexcept;
```

17 *Remark:* The second constructor shall not participate in overload resolution unless `Y*` is implicitly convertible to `T*`.

18 *Effects:* If `r` is *empty*, constructs an *empty* `shared_ptr` object; otherwise, constructs a `shared_ptr` object that *shares ownership* with `r`.

19 *Postconditions:* `get() == r.get() && use_count() == r.use_count()`.

```
shared_ptr(shared_ptr&& r) noexcept;
template<class Y> shared_ptr(shared_ptr<Y>&& r) noexcept;
```

20 *Remark:* The second constructor shall not participate in overload resolution unless `Y*` is convertible to `T*`.

21 *Effects:* Move-constructs a `shared_ptr` instance from `r`.

22 *Postconditions:* `*this` shall contain the old value of `r`. `r` shall be *empty*. `r.get() == nullptr`.

```
template<class Y> explicit shared_ptr(const weak_ptr<Y>& r);
```

23 *Requires:* `Y*` shall be convertible to `T*`.

24 *Effects:* Constructs a `shared_ptr` object that *shares ownership* with `r` and stores a copy of the pointer stored in `r`.

25 *Postconditions:* `use_count() == r.use_count()`.

26 *Throws:* `bad_weak_ptr` when `r.expired()`.

27 *Exception safety:* If an exception is thrown, the constructor has no effect.

```
template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

28 *Remark:* This constructor shall not participate in overload resolution unless `unique_ptr<Y, D>::pointer` is convertible to `T*`.

29 *Effects:* If `r.get() == nullptr`, equivalent to `shared_ptr()`. Otherwise, if `D` is not a reference type, equivalent to `shared_ptr(r.release(), r.get_deleter())`. Otherwise, equivalent to `shared_ptr(r.release(), ref(r.get_deleter()))`.

30 *Exception safety:* If an exception is thrown, the constructor has no effect.

20.8.2.2.2 `shared_ptr` destructor [util.smartptr.shared.dest]

```
~shared_ptr();
```

1 *Effects:*

(1.1) — If `*this` is *empty* or shares ownership with another `shared_ptr` instance (`use_count() > 1`), there are no side effects.

(1.2) — Otherwise, if `*this` *owns* an object `p` and a deleter `d`, `d(p)` is called.

(1.3) — Otherwise, `*this` *owns* a pointer `p`, and `delete p` is called.

2 [*Note:* Since the destruction of `*this` decreases the number of instances that share ownership with `*this` by one, after `*this` has been destroyed all `shared_ptr` instances that shared ownership with `*this` will report a `use_count()` that is one less than its previous value. — *end note*]

20.8.2.2.3 `shared_ptr` assignment [util.smartptr.shared.assign]

```
shared_ptr& operator=(const shared_ptr& r) noexcept;
template<class Y> shared_ptr& operator=(const shared_ptr<Y>& r) noexcept;
```

1 *Effects:* Equivalent to `shared_ptr(r).swap(*this)`.

2 *Returns:* `*this`.

3 [*Note:* The use count updates caused by the temporary object construction and destruction are not observable side effects, so the implementation may meet the effects (and the implied guarantees) via different means, without creating a temporary. In particular, in the example:

```
shared_ptr<int> p(new int);
shared_ptr<void> q(p);
p = p;
q = p;
```

both assignments may be no-ops. — *end note*]

```
shared_ptr& operator=(shared_ptr&& r) noexcept;
template<class Y> shared_ptr& operator=(shared_ptr<Y>&& r) noexcept;
```

4 *Effects:* Equivalent to `shared_ptr(std::move(r)).swap(*this)`.

5 *Returns:* `*this`.

```
template <class Y, class D> shared_ptr& operator=(unique_ptr<Y, D>&& r);
```

6 *Effects:* Equivalent to `shared_ptr(std::move(r)).swap(*this)`.

7 *Returns:* `*this`

20.8.2.2.4 `shared_ptr` modifiers [util.smartptr.shared.mod]

```
void swap(shared_ptr& r) noexcept;
```

1 *Effects:* Exchanges the contents of `*this` and `r`.

```
void reset() noexcept;
```

2 *Effects:* Equivalent to `shared_ptr().swap(*this)`.

```
template<class Y> void reset(Y* p);
```

3 *Effects:* Equivalent to `shared_ptr(p).swap(*this)`.

```
template<class Y, class D> void reset(Y* p, D d);
```

4 *Effects:* Equivalent to `shared_ptr(p, d).swap(*this)`.

```
template<class Y, class D, class A> void reset(Y* p, D d, A a);
```

5 *Effects:* Equivalent to `shared_ptr(p, d, a).swap(*this)`.

20.8.2.2.5 `shared_ptr` observers [util.smartptr.shared.obs]

```
T* get() const noexcept;
```

1 *Returns:* the stored pointer.

```
T& operator*() const noexcept;
```

2 *Requires:* `get() != 0`.

3 *Returns:* `*get()`.

4 *Remarks:* When `T` is `void`, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

```
T* operator->() const noexcept;
```

5 *Requires:* `get() != 0`.

6 *Returns:* `get()`.

```
long use_count() const noexcept;
```

7 *Returns:* the number of `shared_ptr` objects, `*this` included, that *share ownership* with `*this`, or `0` when `*this` is *empty*.

```
bool unique() const noexcept;
```

8 *Returns:* `use_count() == 1`.

9 [*Note:* If you are using `unique()` to implement copy on write, do not rely on a specific value when `get() == nullptr`. — *end note*]

```
explicit operator bool() const noexcept;
```

10 *Returns:* `get() != 0`.

```
template<class U> bool owner_before(shared_ptr<U> const& b) const;
template<class U> bool owner_before(weak_ptr<U> const& b) const;
```

11 *Returns:* An unspecified value such that

(11.1) — `x.owner_before(y)` defines a strict weak ordering as defined in 25.4;

(11.2) — under the equivalence relation defined by `owner_before`, `!a.owner_before(b) && !b.owner_before(a)`, two `shared_ptr` or `weak_ptr` instances are equivalent if and only if they share ownership or are both empty.

20.8.2.2.6 `shared_ptr` creation [util.smartptr.shared.create]

```
template<class T, class... Args> shared_ptr<T> make_shared(Args&&... args);
template<class T, class A, class... Args>
  shared_ptr<T> allocate_shared(const A& a, Args&&... args);
```

1 *Requires:* The expression `::new (pv) T(std::forward<Args>(args)...)`, where `pv` has type `void*` and points to storage suitable to hold an object of type `T`, shall be well formed. `A` shall be an *allocator* (17.6.3.5). The copy constructor and destructor of `A` shall not throw exceptions.

2 *Effects:* Allocates memory suitable for an object of type `T` and constructs an object in that memory via the placement *new-expression* `::new (pv) T(std::forward<Args>(args)...)`. The template `allocate_shared` uses a copy of `a` to allocate memory. If an exception is thrown, the functions have no effect.

3 *Returns:* A `shared_ptr` instance that stores and owns the address of the newly constructed object of type `T`.

4 *Postconditions:* `get() != 0 && use_count() == 1`

5 *Throws:* `bad_alloc`, or an exception thrown from `A::allocate` or from the constructor of `T`.

6 *Remarks:* Implementations should perform no more than one memory allocation. [*Note:* This provides efficiency equivalent to an intrusive smart pointer. — *end note*]

7 [*Note:* These functions will typically allocate more memory than `sizeof(T)` to allow for internal bookkeeping structures such as the reference counts. — *end note*]

20.8.2.2.7 `shared_ptr` comparison [util.smartptr.shared.cmp]

```
template<class T, class U> bool operator==(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

1 *Returns:* `a.get() == b.get()`.

```
template<class T, class U> bool operator<(const shared_ptr<T>& a, const shared_ptr<U>& b) noexcept;
```

2 *Returns:* `less<V>()(a.get(), b.get())`, where `V` is the composite pointer type (Clause 5) of `T*` and `U*`.

3 [*Note:* Defining a comparison operator allows `shared_ptr` objects to be used as keys in associative containers. — *end note*]

```
template <class T>
  bool operator==(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator==(nullptr_t, const shared_ptr<T>& a) noexcept;
```

4 *Returns:* `!a`.

```
template <class T>
  bool operator!=(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator!=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

5 *Returns:* `(bool)a`.

```
template <class T>
  bool operator<(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator<(nullptr_t, const shared_ptr<T>& a) noexcept;
```

6 *Returns:* The first function template returns `less<T*>()(a.get(), nullptr)`. The second function template returns `less<T*>()(nullptr, a.get())`.

```
template <class T>
  bool operator>(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator>(nullptr_t, const shared_ptr<T>& a) noexcept;
```

7 *Returns:* The first function template returns `nullptr < a`. The second function template returns `a < nullptr`.

```
template <class T>
  bool operator<=(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator<=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

8 *Returns:* The first function template returns `!(nullptr < a)`. The second function template returns `!(a < nullptr)`.

```
template <class T>
  bool operator>=(const shared_ptr<T>& a, nullptr_t) noexcept;
template <class T>
  bool operator>=(nullptr_t, const shared_ptr<T>& a) noexcept;
```

9 *Returns:* The first function template returns `!(a < nullptr)`. The second function template returns `!(nullptr < a)`.

20.8.2.2.8 `shared_ptr` specialized algorithms [util.smartptr.shared.spec]

```
template<class T> void swap(shared_ptr<T>& a, shared_ptr<T>& b) noexcept;
```

1 *Effects:* Equivalent to `a.swap(b)`.

20.8.2.2.9 `shared_ptr` casts [util.smartptr.shared.cast]

```
template<class T, class U> shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
```

1 *Requires:* The expression `static_cast<T*>(r.get())` shall be well formed.

2 *Returns:* If `r` is *empty*, an *empty* `shared_ptr<T>`; otherwise, a `shared_ptr<T>` object that stores `static_cast<T*>(r.get())` and *shares ownership* with `r`.

3 *Postconditions:* `w.get() == static_cast<T*>(r.get())` and `w.use_count() == r.use_count()`, where `w` is the return value.

4 [*Note:* The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template<class T, class U> shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
```

5 *Requires:* The expression `dynamic_cast<T*>(r.get())` shall be well formed and shall have well defined behavior.

6 *Returns:*

(6.1) — When `dynamic_cast<T*>(r.get())` returns a nonzero value, a `shared_ptr<T>` object that stores a copy of it and *shares ownership* with `r`;

(6.2) — Otherwise, an *empty* `shared_ptr<T>` object.

7 *Postcondition:* `w.get() == dynamic_cast<T*>(r.get())`, where `w` is the return value.

8 [*Note:* The seemingly equivalent expression `shared_ptr<T>(dynamic_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

```
template<class T, class U> shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
```

9 *Requires:* The expression `const_cast<T*>(r.get())` shall be well formed.

10 *Returns:* If `r` is empty, an empty `shared_ptr<T>`; otherwise, a `shared_ptr<T>` object that stores `const_cast<T*>(r.get())` and shares ownership with `r`.

11 *Postconditions:* `w.get() == const_cast<T*>(r.get())` and `w.use_count() == r.use_count()`, where `w` is the return value.

12 [*Note:* The seemingly equivalent expression `shared_ptr<T>(const_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. — *end note*]

20.8.2.2.10 get_deleter [util.smartptr.getdeleter]

```
template<class D, class T> D* get_deleter(const shared_ptr<T>& p) noexcept;
```

1 *Returns:* If `p` *owns* a deleter `d` of type cv-unqualified `D`, returns `std:addressof(d)`; otherwise returns `nullptr`. The returned pointer remains valid as long as there exists a `shared_ptr` instance that owns `d`. [*Note:* It is unspecified whether the pointer remains valid longer than that. This can happen if the implementation doesn't destroy the deleter until all `weak_ptr` instances that share ownership with `p` have been destroyed. — *end note*]

20.8.2.2.11 shared_ptr I/O [util.smartptr.shared.io]

```
template<class E, class T, class Y>
  basic_ostream<E, T>& operator<< (basic_ostream<E, T>& os, shared_ptr<Y> const& p);
```

1 *Effects:* `os << p.get();`.

2 *Returns:* `os`.

20.8.2.3 Class template weak_ptr [util.smartptr.weak]

1 The `weak_ptr` class template stores a weak reference to an object that is already managed by a `shared_ptr`. To access the object, a `weak_ptr` can be converted to a `shared_ptr` using the member function `lock`.

```
namespace std {
  template<class T> class weak_ptr {
  public:
    typedef T element_type;

    // 20.8.2.3.1, constructors
    constexpr weak_ptr() noexcept;
    template<class Y> weak_ptr(shared_ptr<Y> const& r) noexcept;
    weak_ptr(weak_ptr const& r) noexcept;
    template<class Y> weak_ptr(weak_ptr<Y> const& r) noexcept;
    weak_ptr(weak_ptr&& r) noexcept;
    template<class Y> weak_ptr(weak_ptr<Y>&& r) noexcept;

    // 20.8.2.3.2, destructor
    ~weak_ptr();

    // 20.8.2.3.3, assignment
    weak_ptr& operator=(weak_ptr const& r) noexcept;
    template<class Y> weak_ptr& operator=(weak_ptr<Y> const& r) noexcept;
    template<class Y> weak_ptr& operator=(shared_ptr<Y> const& r) noexcept;
    weak_ptr& operator=(weak_ptr&& r) noexcept;
    template<class Y> weak_ptr& operator=(weak_ptr<Y>&& r) noexcept;

    // 20.8.2.3.4, modifiers
    void swap(weak_ptr& r) noexcept;
    void reset() noexcept;

    // 20.8.2.3.5, observers
    long use_count() const noexcept;
    bool expired() const noexcept;
    shared_ptr<T> lock() const noexcept;
    template<class U> bool owner_before(shared_ptr<U> const& b) const;
    template<class U> bool owner_before(weak_ptr<U> const& b) const;
  };

  // 20.8.2.3.6, specialized algorithms
  template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;
} // namespace std
```

2 Specializations of `weak_ptr` shall be `CopyConstructible` and `CopyAssignable`, allowing their use in standard containers. The template parameter `T` of `weak_ptr` may be an incomplete type.

20.8.2.3.1 `weak_ptr` constructors [util.smartptr.weak.const]

```
constexpr weak_ptr() noexcept;
```

1 *Effects:* Constructs an *empty* `weak_ptr` object.

2 *Postconditions:* `use_count() == 0`.

```
weak_ptr(const weak_ptr& r) noexcept;
template<class Y> weak_ptr(const weak_ptr<Y>& r) noexcept;
template<class Y> weak_ptr(const shared_ptr<Y>& r) noexcept;
```

3 *Remark:* The second and third constructors shall not participate in overload resolution unless `Y*` is implicitly convertible to `T*`.

4 *Effects:* If `r` is *empty*, constructs an *empty* `weak_ptr` object; otherwise, constructs a `weak_ptr` object that *shares ownership* with `r` and stores a copy of the pointer stored in `r`.

5 *Postconditions:* `use_count() == r.use_count()`.

```
weak_ptr(weak_ptr&& r) noexcept;
template<class Y> weak_ptr(weak_ptr<Y>&& r) noexcept;
```

6 *Remark:* The second constructor shall not participate in overload resolution unless `Y*` is implicitly convertible to `T*`.

7 *Effects:* Move-constructs a `weak_ptr` instance from `r`.

8 *Postconditions:* `*this` shall contain the old value of `r`. `r` shall be *empty*. `r.use_count() == 0`.

20.8.2.3.2 `weak_ptr` destructor [util.smartptr.weak.dest]

```
~weak_ptr();
```

1 *Effects:* Destroys this `weak_ptr` object but has no effect on the object its stored pointer points to.

20.8.2.3.3 `weak_ptr` assignment [util.smartptr.weak.assign]

```
weak_ptr& operator=(const weak_ptr& r) noexcept;
template<class Y> weak_ptr& operator=(const weak_ptr<Y>& r) noexcept;
template<class Y> weak_ptr& operator=(const shared_ptr<Y>& r) noexcept;
```

1 *Effects:* Equivalent to `weak_ptr(r).swap(*this)`.

2 *Remarks:* The implementation may meet the effects (and the implied guarantees) via different means, without creating a temporary.

3 *Returns:* `*this`.

```
weak_ptr& operator=(weak_ptr&& r) noexcept;
template<class Y> weak_ptr& operator=(weak_ptr<Y>&& r) noexcept;
```

4 *Effects:* Equivalent to `weak_ptr(std::move(r)).swap(*this)`.

5 *Returns:* `*this`.

20.8.2.3.4 `weak_ptr` modifiers [util.smartptr.weak.mod]

```
void swap(weak_ptr& r) noexcept;
```

1 *Effects:* Exchanges the contents of `*this` and `r`.

```
void reset() noexcept;
```

2 *Effects:* Equivalent to `weak_ptr().swap(*this)`.

20.8.2.3.5 `weak_ptr` observers [util.smartptr.weak.obs]

```
long use_count() const noexcept;
```

1 *Returns:* `0` if `*this` is *empty*; otherwise, the number of `shared_ptr` instances that *share ownership* with `*this`.

```
bool expired() const noexcept;
```

2 *Returns:* `use_count() == 0`.

```
shared_ptr<T> lock() const noexcept;
```

3 *Returns:* `expired() ? shared_ptr<T>() : shared_ptr<T>(*this)`, executed atomically.

```
template<class U> bool owner_before(shared_ptr<U> const& b) const;
template<class U> bool owner_before(weak_ptr<U> const& b) const;
```

4 *Returns:* An unspecified value such that

(4.1) — `x.owner_before(y)` defines a strict weak ordering as defined in 25.4;

(4.2) — under the equivalence relation defined by `owner_before`, `!a.owner_before(b) && !b.owner_before(a)`, two `shared_ptr` or `weak_ptr` instances are equivalent if and only if they share ownership or are both empty.

20.8.2.3.6 `weak_ptr` specialized algorithms [util.smartptr.weak.spec]

```
template<class T> void swap(weak_ptr<T>& a, weak_ptr<T>& b) noexcept;
```

1 *Effects:* Equivalent to `a.swap(b)`.

20.8.2.4 Class template `owner_less` [util.smartptr.ownerless]

1 The class template `owner_less` allows ownership-based mixed comparisons of shared and weak pointers.

```
namespace std {
  template<class T> struct owner_less;

  template<class T> struct owner_less<shared_ptr<T> > {
    typedef bool result_type;
    typedef shared_ptr<T> first_argument_type;
    typedef shared_ptr<T> second_argument_type;
    bool operator()(shared_ptr<T> const&, shared_ptr<T> const&) const;
    bool operator()(shared_ptr<T> const&, weak_ptr<T> const&) const;
    bool operator()(weak_ptr<T> const&, shared_ptr<T> const&) const;
  };

  template<class T> struct owner_less<weak_ptr<T> > {
    typedef bool result_type;
    typedef weak_ptr<T> first_argument_type;
    typedef weak_ptr<T> second_argument_type;
    bool operator()(weak_ptr<T> const&, weak_ptr<T> const&) const;
    bool operator()(shared_ptr<T> const&, weak_ptr<T> const&) const;
    bool operator()(weak_ptr<T> const&, shared_ptr<T> const&) const;
  };
}
```

2 `operator()(x,y)` shall return `x.owner_before(y)`. [*Note:* Note that

(2.1) — `operator()` defines a strict weak ordering as defined in 25.4;

(2.2) — under the equivalence relation defined by `operator()`, `!operator()(a, b) && !operator()(b, a)`, two `shared_ptr` or `weak_ptr` instances are equivalent if and only if they share ownership or are both empty.

— *end note*]

20.8.2.5 Class template `enable_shared_from_this` [util.smartptr.enab]

1 A class `T` can inherit from `enable_shared_from_this<T>` to inherit the `shared_from_this` member functions that obtain a *shared_ptr* instance pointing to `*this`.

2 [*Example:*

```
struct X: public enable_shared_from_this<X> {
};

int main() {
  shared_ptr<X> p(new X);
  shared_ptr<X> q = p->shared_from_this();
  assert(p == q);
  assert(!p.owner_before(q) && !q.owner_before(p)); // p and q share ownership
}
```

— *end example*]

```
namespace std {
  template<class T> class enable_shared_from_this {
  protected:
    constexpr enable_shared_from_this() noexcept;
    enable_shared_from_this(enable_shared_from_this const&) noexcept;
    enable_shared_from_this& operator=(enable_shared_from_this const&) noexcept;
    ~enable_shared_from_this();
  public:
    shared_ptr<T> shared_from_this();
    shared_ptr<T const> shared_from_this() const;
  };
} // namespace std
```

3 The template parameter `T` of `enable_shared_from_this` may be an incomplete type.

```
constexpr enable_shared_from_this() noexcept;
enable_shared_from_this(const enable_shared_from_this<T>&) noexcept;
```

4 *Effects:* Constructs an `enable_shared_from_this<T>` object.

```
enable_shared_from_this<T>& operator=(const enable_shared_from_this<T>&) noexcept;
```

5 *Returns:* `*this`.

```
~enable_shared_from_this();
```

6 *Effects:* Destroys `*this`.

```
shared_ptr<T>       shared_from_this();
shared_ptr<T const> shared_from_this() const;
```

7 *Requires:* `enable_shared_from_this<T>` shall be an accessible base class of `T`. `*this` shall be a subobject of an object `t` of type `T`. There shall be at least one `shared_ptr` instance `p` that *owns* `&t`.

8 *Returns:* A `shared_ptr<T>` object `r` that *shares ownership with* `p`.

9 *Postconditions:* `r.get() == this`.

10 [*Note:* A possible implementation is shown below:

```
template<class T> class enable_shared_from_this {
private:
  weak_ptr<T> __weak_this;
protected:
  constexpr enable_shared_from_this() : __weak_this() { }
  enable_shared_from_this(enable_shared_from_this const &) { }
  enable_shared_from_this& operator=(enable_shared_from_this const &) { return *this; }
  ~enable_shared_from_this() { }
```

```
  public:
    shared_ptr<T> shared_from_this() { return shared_ptr<T>(__weak_this); }
    shared_ptr<T const> shared_from_this() const { return shared_ptr<T const>(__weak_this); }
  };
```

11 The `shared_ptr` constructors that create unique pointers can detect the presence of an `enable_shared_from_this` base and assign the newly created `shared_ptr` to its `__weak_this` member. — *end note*]

20.8.2.6 `shared_ptr` atomic access [util.smartptr.shared.atomic]

1 Concurrent access to a `shared_ptr` object from multiple threads does not introduce a data race if the access is done exclusively via the functions in this section and the instance is passed as their first argument.

2 The meaning of the arguments of type `memory_order` is explained in 29.3.

```
template<class T>
  bool atomic_is_lock_free(const shared_ptr<T>* p);
```

3 *Requires:* `p` shall not be null.

4 *Returns:* `true` if atomic access to `*p` is lock-free, `false` otherwise.

5 *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load(const shared_ptr<T>* p);
```

6 *Requires:* `p` shall not be null.

7 *Returns:* `atomic_load_explicit(p, memory_order_seq_cst)`.

8 *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_load_explicit(const shared_ptr<T>* p, memory_order mo);
```

9 *Requires:* `p` shall not be null.

10 *Requires:* `mo` shall not be `memory_order_release` or `memory_order_acq_rel`.

11 *Returns:* `*p`.

12 *Throws:* Nothing.

```
template<class T>
  void atomic_store(shared_ptr<T>* p, shared_ptr<T> r);
```

13 *Requires:* `p` shall not be null.

14 *Effects:* `atomic_store_explicit(p, r, memory_order_seq_cst)`.

15 *Throws:* Nothing.

```
template<class T>
  void atomic_store_explicit(shared_ptr<T>* p, shared_ptr<T> r, memory_order mo);
```

16 *Requires:* `p` shall not be null.

17 *Requires:* `mo` shall not be `memory_order_acquire` or `memory_order_acq_rel`.

18 *Effects:* `p->swap(r)`.

19 *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange(shared_ptr<T>* p, shared_ptr<T> r);
```

20 *Requires:* `p` shall not be null.

21 *Returns:* `atomic_exchange_explicit(p, r, memory_order_seq_cst)`.

22 *Throws:* Nothing.

```
template<class T>
  shared_ptr<T> atomic_exchange_explicit(shared_ptr<T>* p, shared_ptr<T> r,
                                         memory_order mo);
```

23 *Requires:* `p` shall not be null.

24 *Effects:* `p->swap(r)`.

25 *Returns:* The previous value of `*p`.

26 *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_weak(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

27 *Requires:* `p` shall not be null and `v` shall not be null.

28 *Returns:* `atomic_compare_exchange_weak_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst)`.

29 *Throws:* Nothing.

```
template<class T>
  bool atomic_compare_exchange_strong(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w);
```

30 *Returns:* `atomic_compare_exchange_strong_explicit(p, v, w, memory_order_seq_cst, memory_order_seq_cst)`.

```
template<class T>
  bool atomic_compare_exchange_weak_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
template<class T>
  bool atomic_compare_exchange_strong_explicit(
    shared_ptr<T>* p, shared_ptr<T>* v, shared_ptr<T> w,
    memory_order success, memory_order failure);
```

31 *Requires:* `p` shall not be null and `v` shall not be null.

32 *Requires:* `failure` shall not be `memory_order_release`, `memory_order_acq_rel`, or stronger than `success`.

33 *Effects:* If `*p` is equivalent to `*v`, assigns `w` to `*p` and has synchronization semantics corresponding to the value of `success`, otherwise assigns `*p` to `*v` and has synchronization semantics corresponding to the value of `failure`.

34 *Returns:* `true` if `*p` was equivalent to `*v`, `false` otherwise.

35 *Throws:* Nothing.

36 *Remarks:* two `shared_ptr` objects are equivalent if they store the same pointer value and share ownership.

37 *Remarks:* the weak forms may fail spuriously. See 29.6.

20.8.2.7 Smart pointer hash support [util.smartptr.hash]

```
template <class T, class D> struct hash<unique_ptr<T, D> >;
```

1 The template specialization shall meet the requirements of class template `hash` (20.9.13). For an object `p` of type `UP`, where `UP` is `unique_ptr<T, D>`, `hash<UP>()(p)` shall evaluate to the same value as `hash<typename UP::pointer>()(p.get())`.

2 *Requires:* The specialization `hash<typename UP::pointer>` shall be well-formed and well-defined, and shall meet the requirements of class template `hash` (20.9.13).

```
template <class T> struct hash<shared_ptr<T> >;
```

3 The template specialization shall meet the requirements of class template `hash` (20.9.13). For an object `p` of type `shared_ptr<T>`, `hash<shared_ptr<T> >()(p)` shall evaluate to the same value as `hash<T*>()(p.get())`.

20.9 Function objects [function.objects]

1 A *function object type* is an object type (3.9) that can be the type of the *postfix-expression* in a function call (5.2.2, 13.3.1.1).[231] A *function object* is an object of a function object type. In the places where one would expect to pass a pointer to a function to an algorithmic template (Clause 25), the interface is specified to accept a function object. This not only makes algorithmic templates work with pointers to functions, but also enables them to work with arbitrary function objects.

2 **Header `<functional>` synopsis**

```
namespace std {
  // 20.9.3, invoke:
  template <class F, class... Args> result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args);

  // 20.9.4, reference_wrapper:
  template <class T> class reference_wrapper;

  template <class T> reference_wrapper<T> ref(T&) noexcept;
  template <class T> reference_wrapper<const T> cref(const T&) noexcept;
  template <class T> void ref(const T&&) = delete;
  template <class T> void cref(const T&&) = delete;

  template <class T> reference_wrapper<T> ref(reference_wrapper<T>) noexcept;
  template <class T> reference_wrapper<const T> cref(reference_wrapper<T>) noexcept;

  // 20.9.5, arithmetic operations:
  template <class T = void> struct plus;
  template <class T = void> struct minus;
  template <class T = void> struct multiplies;
  template <class T = void> struct divides;
  template <class T = void> struct modulus;
  template <class T = void> struct negate;
  template <> struct plus<void>;
  template <> struct minus<void>;
  template <> struct multiplies<void>;
  template <> struct divides<void>;
  template <> struct modulus<void>;
  template <> struct negate<void>;
```

231) Such a type is a function pointer or a class type which has a member `operator()` or a class type which has a conversion to a pointer to function.

```
// 20.9.6, comparisons:
template <class T = void> struct equal_to;
template <class T = void> struct not_equal_to;
template <class T = void> struct greater;
template <class T = void> struct less;
template <class T = void> struct greater_equal;
template <class T = void> struct less_equal;
template <> struct equal_to<void>;
template <> struct not_equal_to<void>;
template <> struct greater<void>;
template <> struct less<void>;
template <> struct greater_equal<void>;
template <> struct less_equal<void>;

// 20.9.7, logical operations:
template <class T = void> struct logical_and;
template <class T = void> struct logical_or;
template <class T = void> struct logical_not;
template <> struct logical_and<void>;
template <> struct logical_or<void>;
template <> struct logical_not<void>;

// 20.9.8, bitwise operations:
template <class T = void> struct bit_and;
template <class T = void> struct bit_or;
template <class T = void> struct bit_xor;
template <class T = void> struct bit_not;
template <> struct bit_and<void>;
template <> struct bit_or<void>;
template <> struct bit_xor<void>;
template <> struct bit_not<void>;

// 20.9.9, negators:
template <class Predicate> class unary_negate;
template <class Predicate>
  constexpr unary_negate<Predicate> not1(const Predicate&);
template <class Predicate> class binary_negate;
template <class Predicate>
  constexpr binary_negate<Predicate> not2(const Predicate&);

// 20.9.10, bind:
template<class T> struct is_bind_expression;
template<class T> struct is_placeholder;

template<class F, class... BoundArgs>
  unspecified  bind(F&&, BoundArgs&&...);
template<class R, class F, class... BoundArgs>
  unspecified  bind(F&&, BoundArgs&&...);

namespace placeholders {
  // M is the implementation-defined number of placeholders
  see below _1;
  see below _2;
              .
```

```
                .
                .
  see below _M;
}

// 20.9.11, member function adaptors:
template<class R, class T> unspecified  mem_fn(R T::*);

// 20.9.12 polymorphic function wrappers:
class bad_function_call;

template<class> class function; // undefined
template<class R, class... ArgTypes> class function<R(ArgTypes...)>;

template<class R, class... ArgTypes>
  void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&);

template<class R, class... ArgTypes>
  bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
  bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;
template<class R, class... ArgTypes>
  bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;
template<class R, class... ArgTypes>
  bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

// 20.9.13, hash function primary template:
template <class T> struct hash;

// Hash function specializations
template <> struct hash<bool>;
template <> struct hash<char>;
template <> struct hash<signed char>;
template <> struct hash<unsigned char>;
template <> struct hash<char16_t>;
template <> struct hash<char32_t>;
template <> struct hash<wchar_t>;
template <> struct hash<short>;
template <> struct hash<unsigned short>;
template <> struct hash<int>;
template <> struct hash<unsigned int>;
template <> struct hash<long>;
template <> struct hash<long long>;
template <> struct hash<unsigned long>;
template <> struct hash<unsigned long long>;

template <> struct hash<float>;
template <> struct hash<double>;
template <> struct hash<long double>;

template<class T> struct hash<T*>;
}
```

3 [*Example:* If a C++ program wants to have a by-element addition of two vectors `a` and `b` containing `double` and put the result into `a`, it can do:

```
transform(a.begin(), a.end(), b.begin(), a.begin(), plus<double>());
```

— *end example*]

4 [*Example:* To negate every element of `a`:

```
transform(a.begin(), a.end(), a.begin(), negate<double>());
```

— *end example*]

5 [*Note:* To enable adaptors and other components to manipulate function objects that take one or two arguments many of the function objects in this clause correspondingly provide typedefs `argument_type` and `result_type` for function objects that take one argument and `first_argument_type`, `second_argument_type`, and `result_type` for function objects that take two arguments. — *end note*]

20.9.1 Definitions [func.def]

1 The following definitions apply to this Clause:

2 A *call signature* is the name of a return type followed by a parenthesized comma-separated list of zero or more argument types.

3 A *callable type* is a function object type (20.9) or a pointer to member.

4 A *callable object* is an object of a callable type.

5 A *call wrapper type* is a type that holds a callable object and supports a call operation that forwards to that object.

6 A *call wrapper* is an object of a call wrapper type.

7 A *target object* is the callable object held by a call wrapper.

20.9.2 Requirements [func.require]

1 Define *INVOKE*`(f, t1, t2, ..., tN)` as follows:

(1.1) — `(t1.*f)(t2, ..., tN)` when `f` is a pointer to a member function of a class `T` and `t1` is an object of type `T` or a reference to an object of type `T` or a reference to an object of a type derived from `T`;

(1.2) — `((*t1).*f)(t2, ..., tN)` when `f` is a pointer to a member function of a class `T` and `t1` is not one of the types described in the previous item;

(1.3) — `t1.*f` when `N == 1` and `f` is a pointer to member data of a class `T` and `t1` is an object of type `T` or a reference to an object of type `T` or a reference to an object of a type derived from `T`;

(1.4) — `(*t1).*f` when `N == 1` and `f` is a pointer to member data of a class `T` and `t1` is not one of the types described in the previous item;

(1.5) — `f(t1, t2, ..., tN)` in all other cases.

2 Define *INVOKE*`(f, t1, t2, ..., tN, R)` as `static_cast<void>(`*INVOKE*`(f, t1, t2, ..., tN))` if `R` is *cv* `void`, otherwise *INVOKE*`(f, t1, t2, ..., tN)` implicitly converted to `R`.

3 If a call wrapper (20.9.1) has a *weak result type* the type of its member type `result_type` is based on the type `T` of the wrapper's target object (20.9.1):

(3.1) — if `T` is a pointer to function type, `result_type` shall be a synonym for the return type of `T`;

(3.2) — if `T` is a pointer to member function, `result_type` shall be a synonym for the return type of `T`;

(3.3) — if `T` is a class type and the *qualified-id* `T::result_type` is valid and denotes a type (14.8.2), then `result_type` shall be a synonym for `T::result_type`;

(3.4) — otherwise `result_type` shall not be defined.

4 Every call wrapper (20.9.1) shall be `MoveConstructible`. A *simple call wrapper* is a call wrapper that is `CopyConstructible` and `CopyAssignable` and whose copy constructor, move constructor, and assignment operator do not throw exceptions. A *forwarding call wrapper* is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step shall ensure that rvalue arguments are delivered as rvalue references and lvalue arguments are delivered as lvalue references. [*Note:* In a typical implementation forwarding call wrappers have an overloaded function call operator of the form

```
template<class... UnBoundArgs>
R operator()(UnBoundArgs&&... unbound_args) cv-qual;
```

— *end note*]

20.9.3 Function template `invoke` [func.invoke]

```
template <class F, class... Args>
  result_of_t<F&&(Args&&...)> invoke(F&& f, Args&&... args);
```

1 *Returns:* *INVOKE*`(std::forward<F>(f), std::forward<Args>(args)...)` (20.9.2).

20.9.4 Class template `reference_wrapper` [refwrap]

```
namespace std {
  template <class T> class reference_wrapper {
  public :
    // types
    typedef T type;
    typedef see below result_type;                // not always defined
    typedef see below argument_type;              // not always defined
    typedef see below first_argument_type;        // not always defined
    typedef see below second_argument_type;       // not always defined

    // construct/copy/destroy
    reference_wrapper(T&) noexcept;
    reference_wrapper(T&&) = delete;      // do not bind to temporary objects
    reference_wrapper(const reference_wrapper& x) noexcept;

    // assignment
    reference_wrapper& operator=(const reference_wrapper& x) noexcept;

    // access
    operator T& () const noexcept;
    T& get() const noexcept;

    // invocation
    template <class... ArgTypes>
    result_of_t<T&(ArgTypes&&...)>
    operator() (ArgTypes&&...) const;
  };
}
```

1 `reference_wrapper<T>` is a `CopyConstructible` and `CopyAssignable` wrapper around a reference to an object or function of type `T`.

2 `reference_wrapper<T>` shall be a trivially copyable type (3.9).

3 `reference_wrapper<T>` has a weak result type (20.9.2). If `T` is a function type, `result_type` shall be a synonym for the return type of `T`.

4 The template specialization `reference_wrapper<T>` shall define a nested type named `argument_type` as a synonym for `T1` only if the type `T` is any of the following:

(4.1) — a function type or a pointer to function type taking one argument of type `T1`

(4.2) — a pointer to member function `R T0::f` *cv* (where *cv* represents the member function's cv-qualifiers); the type `T1` is *cv* `T0*`

(4.3) — a class type where the *qualified-id* `T::argument_type` is valid and denotes a type (14.8.2); the type `T1` is `T::argument_type`.

5 The template instantiation `reference_wrapper<T>` shall define two nested types named `first_argument_type` and `second_argument_type` as synonyms for `T1` and `T2`, respectively, only if the type `T` is any of the following:

(5.1) — a function type or a pointer to function type taking two arguments of types `T1` and `T2`

(5.2) — a pointer to member function `R T0::f(T2)` *cv* (where *cv* represents the member function's cv-qualifiers); the type `T1` is *cv* `T0*`

(5.3) — a class type where the *qualified-id*s `T::first_argument_type` and `T::second_argument_type` are both valid and both denote types (14.8.2); the type `T1` is `T::first_argument_type` and the type `T2` is `T::second_argument_type`.

20.9.4.1 `reference_wrapper` construct/copy/destroy [refwrap.const]

```
reference_wrapper(T& t) noexcept;
```

1 *Effects:* Constructs a `reference_wrapper` object that stores a reference to `t`.

```
reference_wrapper(const reference_wrapper& x) noexcept;
```

2 *Effects:* Constructs a `reference_wrapper` object that stores a reference to `x.get()`.

20.9.4.2 `reference_wrapper` assignment [refwrap.assign]

```
reference_wrapper& operator=(const reference_wrapper& x) noexcept;
```

1 *Postconditions:* `*this` stores a reference to `x.get()`.

20.9.4.3 `reference_wrapper` access [refwrap.access]

```
operator T& () const noexcept;
```

1 *Returns:* The stored reference.

```
T& get() const noexcept;
```

2 *Returns:* The stored reference.

20.9.4.4 reference_wrapper invocation [refwrap.invoke]

```
template <class... ArgTypes>
  result_of_t<T&(ArgTypes&&... )>
    operator()(ArgTypes&&... args) const;
```

1 *Returns:* *INVOKE*`(get(), std::forward<ArgTypes>(args)...)`. (20.9.2)

2 *Remark:* `operator()` is described for exposition only. Implementations are not required to provide an actual `reference_wrapper::operator()`. Implementations are permitted to support `reference_wrapper` function invocation through multiple overloaded operators or through other means.

20.9.4.5 reference_wrapper helper functions [refwrap.helpers]

```
template <class T> reference_wrapper<T> ref(T& t) noexcept;
```

1 *Returns:* `reference_wrapper<T>(t)`

```
template <class T> reference_wrapper<T> ref(reference_wrapper<T> t) noexcept;
```

2 *Returns:* `ref(t.get())`

```
template <class T> reference_wrapper<const T> cref(const T& t) noexcept;
```

3 *Returns:* `reference_wrapper <const T>(t)`

```
template <class T> reference_wrapper<const T> cref(reference_wrapper<T> t) noexcept;
```

4 *Returns:* `cref(t.get());`

20.9.5 Arithmetic operations [arithmetic.operations]

1 The library provides basic function object classes for all of the arithmetic operators in the language (5.6, 5.7).

```
template <class T = void> struct plus {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

2 `operator()` returns `x + y`.

```
template <class T = void> struct minus {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

3 `operator()` returns `x - y`.

```
template <class T = void> struct multiplies {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

4 `operator()` returns `x * y`.

```
template <class T = void> struct divides {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

5 `operator()` returns `x / y`.

```
template <class T = void> struct modulus {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

6 `operator()` returns `x % y`.

```
template <class T = void> struct negate {
  constexpr T operator()(const T& x) const;
  typedef T argument_type;
  typedef T result_type;
};
```

7 `operator()` returns `-x`.

```
template <> struct plus<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) + std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

8 `operator()` returns `std::forward<T>(t) + std::forward<U>(u)`.

```
template <> struct minus<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) - std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

9 `operator()` returns `std::forward<T>(t) - std::forward<U>(u)`.

```
template <> struct multiplies<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) * std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

10 `operator()` returns `std::forward<T>(t) * std::forward<U>(u)`.

```
template <> struct divides<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) / std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

11 `operator()` returns `std::forward<T>(t) / std::forward<U>(u)`.

```
template <> struct modulus<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) % std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

12 operator() returns std::forward<T>(t) % std::forward<U>(u).

```
template <> struct negate<void> {
  template <class T> constexpr auto operator()(T&& t) const
    -> decltype(-std::forward<T>(t));

  typedef unspecified  is_transparent;
};
```

13 operator() returns -std::forward<T>(t).

20.9.6 Comparisons [comparisons]

1 The library provides basic function object classes for all of the comparison operators in the language (5.9, 5.10).

```
template <class T = void> struct equal_to {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

2 operator() returns x == y.

```
template <class T = void> struct not_equal_to {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

3 operator() returns x != y.

```
template <class T = void> struct greater {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

4 operator() returns x > y.

```
template <class T = void> struct less {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

5 operator() returns x < y.

```
template <class T = void> struct greater_equal {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

6 operator() returns x >= y.

```
template <class T = void> struct less_equal {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

7 operator() returns x <= y.

```
template <> struct equal_to<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) == std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

8 operator() returns std::forward<T>(t) == std::forward<U>(u).

```
template <> struct not_equal_to<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) != std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

9 operator() returns std::forward<T>(t) != std::forward<U>(u).

```
template <> struct greater<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) > std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

10 operator() returns std::forward<T>(t) > std::forward<U>(u).

```
template <> struct less<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) < std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

11 operator() returns std::forward<T>(t) < std::forward<U>(u).

```
template <> struct greater_equal<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) >= std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

12 operator() returns std::forward<T>(t) >= std::forward<U>(u).

```
template <> struct less_equal<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) <= std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

13 `operator()` returns `std::forward<T>(t) <= std::forward<U>(u)`.

14 For templates `greater`, `less`, `greater_equal`, and `less_equal`, the specializations for any pointer type yield a total order, even if the built-in operators `<`, `>`, `<=`, `>=` do not.

20.9.7 Logical operations [logical.operations]

1 The library provides basic function object classes for all of the logical operators in the language (5.14, 5.15, 5.3.1).

```
template <class T = void> struct logical_and {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

2 `operator()` returns `x && y`.

```
template <class T = void> struct logical_or {
  constexpr bool operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef bool result_type;
};
```

3 `operator()` returns `x || y`.

```
template <class T = void> struct logical_not {
  constexpr bool operator()(const T& x) const;
  typedef T argument_type;
  typedef bool result_type;
};
```

4 `operator()` returns `!x`.

```
template <> struct logical_and<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) && std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

5 `operator()` returns `std::forward<T>(t) && std::forward<U>(u)`.

```
template <> struct logical_or<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) || std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

6 operator() returns std::forward<T>(t) || std::forward<U>(u).

```
template <> struct logical_not<void> {
  template <class T> constexpr auto operator()(T&& t) const
    -> decltype(!std::forward<T>(t));

  typedef unspecified  is_transparent;
};
```

7 operator() returns !std::forward<T>(t).

20.9.8 Bitwise operations [bitwise.operations]

1 The library provides basic function object classes for all of the bitwise operators in the language (5.11, 5.13, 5.12, 5.3.1).

```
template <class T = void> struct bit_and {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

2 operator() returns x & y.

```
template <class T = void> struct bit_or {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

3 operator() returns x | y.

```
template <class T = void> struct bit_xor {
  constexpr T operator()(const T& x, const T& y) const;
  typedef T first_argument_type;
  typedef T second_argument_type;
  typedef T result_type;
};
```

4 operator() returns x ^ y.

```
template <class T = void> struct bit_not {
  constexpr T operator()(const T& x) const;
  typedef T argument_type;
  typedef T result_type;
};
```

5 operator() returns ~x.

```
template <> struct bit_and<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) & std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

6 operator() returns std::forward<T>(t) & std::forward<U>(u).

```
template <> struct bit_or<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) | std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

7 operator() returns std::forward<T>(t) | std::forward<U>(u).

```
template <> struct bit_xor<void> {
  template <class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) ^ std::forward<U>(u));

  typedef unspecified  is_transparent;
};
```

8 operator() returns std::forward<T>(t) ^ std::forward<U>(u).

```
template <> struct bit_not<void> {
  template <class T> constexpr auto operator()(T&& t) const
    -> decltype(~std::forward<T>(t));

  typedef unspecified  is_transparent;
};
```

9 operator() returns ~std::forward<T>(t).

20.9.9 Negators [negators]

1 Negators not1 and not2 take a unary and a binary predicate, respectively, and return their complements (5.3.1).

```
template <class Predicate>
  class unary_negate {
public:
  constexpr explicit unary_negate(const Predicate& pred);
  constexpr bool operator()(const typename Predicate::argument_type& x) const;
  typedef typename Predicate::argument_type argument_type;
  typedef bool result_type;
};
```

2 operator() returns !pred(x).

```
template <class Predicate>
  constexpr unary_negate<Predicate> not1(const Predicate& pred);
```

3 *Returns:* unary_negate<Predicate>(pred).

```
template <class Predicate>
  class binary_negate {
  public:
    constexpr explicit binary_negate(const Predicate& pred);
    constexpr bool operator()(const typename Predicate::first_argument_type& x,
        const typename Predicate::second_argument_type& y) const;
  typedef typename Predicate::first_argument_type first_argument_type;
  typedef typename Predicate::second_argument_type second_argument_type;
  typedef bool result_type;
  };
```

4 operator() returns !pred(x,y).

```
template <class Predicate>
  constexpr binary_negate<Predicate> not2(const Predicate& pred);
```

5 *Returns:* binary_negate<Predicate>(pred).

20.9.10 Function object binders [func.bind]

1 This subclause describes a uniform mechanism for binding arguments of callable objects.

20.9.10.1 Class template is_bind_expression [func.bind.isbind]

```
namespace std {
  template<class T> struct is_bind_expression; // see below
}
```

1 is_bind_expression can be used to detect function objects generated by bind. bind uses is_bind_expression to detect subexpressions.

2 Instantiations of the is_bind_expression template shall meet the UnaryTypeTrait requirements (20.10.1). The implementation shall provide a definition that has a BaseCharacteristic of true_type if T is a type returned from bind, otherwise it shall have a BaseCharacteristic of false_type. A program may specialize this template for a user-defined type T to have a BaseCharacteristic of true_type to indicate that T should be treated as a subexpression in a bind call.

20.9.10.2 Class template is_placeholder [func.bind.isplace]

```
namespace std {
  template<class T> struct is_placeholder; // see below
}
```

1 is_placeholder can be used to detect the standard placeholders _1, _2, and so on. bind uses is_placeholder to detect placeholders.

2 Instantiations of the is_placeholder template shall meet the UnaryTypeTrait requirements (20.10.1). The implementation shall provide a definition that has the BaseCharacteristic of integral_constant<int, J> if T is the type of std::placeholders::_J, otherwise it shall have a BaseCharacteristic of integral_constant<int, 0>. A program may specialize this template for a user-defined type T to have a BaseCharacteristic of integral_constant<int, *N*> with *N* > 0 to indicate that T should be treated as a placeholder type.

20.9.10.3 Function template bind [func.bind.bind]

1 In the text that follows, the following names have the following meanings:

(1.1) — FD is the type decay_t<F>,

(1.2) — fd is an lvalue of type FD constructed from std::forward<F>(f),

(1.3) — Ti is the ith type in the template parameter pack BoundArgs,

(1.4) — TiD is the type decay_t<Ti>,

(1.5) — ti is the ith argument in the function parameter pack bound_args,

(1.6) — tid is an lvalue of type TiD constructed from std::forward<Ti>(ti),

(1.7) — Uj is the jth deduced type of the UnBoundArgs&&... parameter of the forwarding call wrapper, and

(1.8) — uj is the jth argument associated with Uj.

```
template<class F, class... BoundArgs>
  unspecified  bind(F&& f, BoundArgs&&... bound_args);
```

2 *Requires:* `is_constructible<FD, F>::value` shall be `true`. For each `Ti` in `BoundArgs`, `is_constructible<TiD, Ti>::value` shall be `true`. *INVOKE* `(fd, w1, w2, ..., wN)` (20.9.2) shall be a valid expression for some values *w1, w2, ..., wN*, where `N == sizeof...(bound_args)`.

3 *Returns:* A forwarding call wrapper `g` with a weak result type (20.9.2). The effect of `g(u1, u2, ..., uM)` shall be *INVOKE*`(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), result_of_t<FD cv & (V1, V2, ..., VN)>)`, where *cv* represents the *cv*-qualifiers of `g` and the values and types of the bound arguments `v1, v2, ..., vN` are determined as specified below. The copy constructor and move constructor of the forwarding call wrapper shall throw an exception if and only if the corresponding constructor of `FD` or of any of the types `TiD` throws an exception.

4 *Throws:* Nothing unless the construction of `fd` or of one of the values `tid` throws an exception.

5 *Remarks:* The return type shall satisfy the requirements of `MoveConstructible`. If all of `FD` and `TiD` satisfy the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note:* This implies that all of `FD` and `TiD` are `MoveConstructible`. — *end note*]

```
template<class R, class F, class... BoundArgs>
  unspecified  bind(F&& f, BoundArgs&&... bound_args);
```

6 *Requires:* `is_constructible<FD, F>::value` shall be `true`. For each `Ti` in `BoundArgs`, `is_constructible<TiD, Ti>::value` shall be `true`. *INVOKE*`(fd, w1, w2, ..., wN)` shall be a valid expression for some values *w1, w2, ..., wN*, where `N == sizeof...(bound_args)`.

7 *Returns:* A forwarding call wrapper `g` with a nested type `result_type` defined as a synonym for `R`. The effect of `g(u1, u2, ..., uM)` shall be *INVOKE*`(fd, std::forward<V1>(v1), std::forward<V2>(v2), ..., std::forward<VN>(vN), R)`, where the values and types of the bound arguments `v1, v2, ..., vN` are determined as specified below. The copy constructor and move constructor of the forwarding call wrapper shall throw an exception if and only if the corresponding constructor of `FD` or of any of the types `TiD` throws an exception.

8 *Throws:* Nothing unless the construction of `fd` or of one of the values `tid` throws an exception.

9 *Remarks:* The return type shall satisfy the requirements of `MoveConstructible`. If all of `FD` and `TiD` satisfy the requirements of `CopyConstructible`, then the return type shall satisfy the requirements of `CopyConstructible`. [*Note:* This implies that all of `FD` and `TiD` are `MoveConstructible`. — *end note*]

10 The values of the *bound arguments* `v1, v2, ..., vN` and their corresponding types `V1, V2, ..., VN` depend on the types `TiD` derived from the call to `bind` and the *cv*-qualifiers *cv* of the call wrapper `g` as follows:

(10.1) — if `TiD` is `reference_wrapper<T>`, the argument is `tid.get()` and its type `Vi` is `T&`;

(10.2) — if the value of `is_bind_expression<TiD>::value` is `true`, the argument is `tid(std::forward<Uj>(uj)...)` and its type `Vi` is `result_of_t<TiD cv & (Uj&&...)>&&`;

(10.3) — if the value `j` of `is_placeholder<TiD>::value` is not zero, the argument is `std::forward<Uj>(uj)` and its type `Vi` is `Uj&&`;

(10.4) — otherwise, the value is `tid` and its type `Vi` is `TiD cv &`.

20.9.10.4 Placeholders [func.bind.place]

```
namespace std::placeholders {
  // M is the implementation-defined number of placeholders
  see below _1;
  see below _2;
              .
              .
              .
  see below _M;
}
```

1 All placeholder types shall be `DefaultConstructible` and `CopyConstructible`, and their default constructors and copy/move constructors shall not throw exceptions. It is implementation-defined whether placeholder types are `CopyAssignable`. `CopyAssignable` placeholders' copy assignment operators shall not throw exceptions.

2 Placeholders should be defined as:

```
constexpr unspecified  _1{};
```

If they are not, they shall be declared as:

```
extern unspecified  _1;
```

20.9.11 Function template `mem_fn` [func.memfn]

```
template<class R, class T> unspecified  mem_fn(R T::* pm);
```

1 *Returns:* A simple call wrapper (20.9.1) `fn` such that the expression `fn(t, a2, ..., aN)` is equivalent to *INVOKE*`(pm, t, a2, ..., aN)` (20.9.2). `fn` shall have a nested type `result_type` that is a synonym for the return type of `pm` when `pm` is a pointer to member function.

2 The simple call wrapper shall define two nested types named `argument_type` and `result_type` as synonyms for *cv* `T*` and `Ret`, respectively, when `pm` is a pointer to member function with cv-qualifier *cv* and taking no arguments, where *Ret* is `pm`'s return type.

3 The simple call wrapper shall define three nested types named `first_argument_type`, `second_argument_type`, and `result_type` as synonyms for *cv* `T*`, `T1`, and `Ret`, respectively, when `pm` is a pointer to member function with cv-qualifier *cv* and taking one argument of type `T1`, where *Ret* is `pm`'s return type.

4 *Throws:* Nothing.

20.9.12 Polymorphic function wrappers [func.wrap]

1 This subclause describes a polymorphic wrapper class that encapsulates arbitrary callable objects.

20.9.12.1 Class `bad_function_call` [func.wrap.badcall]

1 An exception of type `bad_function_call` is thrown by `function::operator()` (20.9.12.2.4) when the function wrapper object has no target.

```
namespace std {
  class bad_function_call : public std::exception {
  public:
    // 20.9.12.1.1, constructor:
    bad_function_call() noexcept;
  };
} // namespace std
```

20.9.12.1.1 `bad_function_call` constructor [func.wrap.badcall.const]

```
bad_function_call() noexcept;
```

1 *Effects:* constructs a `bad_function_call` object.

2 *Postconditions:* `what()` returns an implementation-defined NTBS.

20.9.12.2 Class template `function` [func.wrap.func]

```
namespace std {
  template<class> class function; // undefined

  template<class R, class... ArgTypes>
  class function<R(ArgTypes...)> {
  public:
    typedef R result_type;
    typedef T1 argument_type;           // only if sizeof...(ArgTypes) == 1 and
                                        // the type in ArgTypes is T1
    typedef T1 first_argument_type;     // only if sizeof...(ArgTypes) == 2 and
                                        // ArgTypes contains T1 and T2
    typedef T2 second_argument_type;    // only if sizeof...(ArgTypes) == 2 and
                                        // ArgTypes contains T1 and T2

    // 20.9.12.2.1, construct/copy/destroy:
    function() noexcept;
    function(nullptr_t) noexcept;
    function(const function&);
    function(function&&);
    template<class F> function(F);
    template<class A> function(allocator_arg_t, const A&) noexcept;
    template<class A> function(allocator_arg_t, const A&,
      nullptr_t) noexcept;
    template<class A> function(allocator_arg_t, const A&,
      const function&);
    template<class A> function(allocator_arg_t, const A&,
      function&&);
    template<class F, class A> function(allocator_arg_t, const A&, F);

    function& operator=(const function&);
    function& operator=(function&&);
    function& operator=(nullptr_t) noexcept;
    template<class F> function& operator=(F&&);
    template<class F> function& operator=(reference_wrapper<F>) noexcept;

    ~function();

    // 20.9.12.2.2, function modifiers:
    void swap(function&) noexcept;
    template<class F, class A> void assign(F&&, const A&);

    // 20.9.12.2.3, function capacity:
    explicit operator bool() const noexcept;

    // 20.9.12.2.4, function invocation:
    R operator()(ArgTypes...) const;
```

```
    // 20.9.12.2.5, function target access:
    const std::type_info& target_type() const noexcept;
    template<class T>       T* target() noexcept;
    template<class T> const T* target() const noexcept;

  };

  // 20.9.12.2.6, Null pointer comparisons:
  template <class R, class... ArgTypes>
    bool operator==(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

  template <class R, class... ArgTypes>
    bool operator==(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

  template <class R, class... ArgTypes>
    bool operator!=(const function<R(ArgTypes...)>&, nullptr_t) noexcept;

  template <class R, class... ArgTypes>
    bool operator!=(nullptr_t, const function<R(ArgTypes...)>&) noexcept;

  // 20.9.12.2.7, specialized algorithms:
  template <class R, class... ArgTypes>
    void swap(function<R(ArgTypes...)>&, function<R(ArgTypes...)>&);

  template<class R, class... ArgTypes, class Alloc>
    struct uses_allocator<function<R(ArgTypes...)>, Alloc>
      : true_type { };
}
```

1 The `function` class template provides polymorphic wrappers that generalize the notion of a function pointer. Wrappers can store, copy, and call arbitrary callable objects (20.9.1), given a call signature (20.9.1), allowing functions to be first-class objects.

2 A callable object `f` of type `F` is *Callable* for argument types `ArgTypes` and return type `R` if the expression *INVOKE*`(f, declval<ArgTypes>()..., R)`, considered as an unevaluated operand (Clause 5), is well formed (20.9.2).

3 The `function` class template is a call wrapper (20.9.1) whose call signature (20.9.1) is `R(ArgTypes...)`.

20.9.12.2.1 `function` construct/copy/destroy [func.wrap.func.con]

1 When any `function` constructor that takes a first argument of type `allocator_arg_t` is invoked, the second argument shall have a type that conforms to the requirements for Allocator (Table 17.6.3.5). A copy of the allocator argument is used to allocate memory, if necessary, for the internal data structures of the constructed `function` object.

```
function() noexcept;
template <class A> function(allocator_arg_t, const A& a) noexcept;
```

2 *Postconditions:* `!*this`.

```
function(nullptr_t) noexcept;
template <class A> function(allocator_arg_t, const A& a, nullptr_t) noexcept;
```

3 *Postconditions:* `!*this`.

```
function(const function& f);
template <class A> function(allocator_arg_t, const A& a, const function& f);
```

4 *Postconditions:* `!*this` if `!f`; otherwise, `*this` targets a copy of `f.target()`.

5 *Throws:* shall not throw exceptions if `f`'s target is a callable object passed via `reference_wrapper` or a function pointer. Otherwise, may throw `bad_alloc` or any exception thrown by the copy constructor of the stored callable object. [*Note:* Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where `f`'s target is an object holding only a pointer or reference to an object and a member function pointer. — *end note*]

```
function(function&& f);
template <class A> function(allocator_arg_t, const A& a, function&& f);
```

6 *Effects:* If `!f`, `*this` has no target; otherwise, move-constructs the target of `f` into the target of `*this`, leaving `f` in a valid state with an unspecified value.

```
template<class F> function(F f);
template <class F, class A> function(allocator_arg_t, const A& a, F f);
```

7 *Requires:* `F` shall be `CopyConstructible`.

8 *Remarks:* These constructors shall not participate in overload resolution unless `f` is Callable (20.9.12.2) for argument types `ArgTypes...` and return type `R`.

9 *Postconditions:* `!*this` if any of the following hold:

(9.1) — `f` is a null function pointer value.

(9.2) — `f` is a null member pointer value.

(9.3) — `F` is an instance of the `function` class template, and `!f`.

10 Otherwise, `*this` targets a copy of `f` initialized with `std::move(f)`. [*Note:* Implementations are encouraged to avoid the use of dynamically allocated memory for small callable objects, for example, where `f`'s target is an object holding only a pointer or reference to an object and a member function pointer. — *end note*]

11 *Throws:* shall not throw exceptions when `f` is a function pointer or a `reference_wrapper<T>` for some `T`. Otherwise, may throw `bad_alloc` or any exception thrown by `F`'s copy or move constructor.

```
function& operator=(const function& f);
```

12 *Effects:* `function(f).swap(*this);`

13 *Returns:* `*this`

```
function& operator=(function&& f);
```

14 *Effects:* Replaces the target of `*this` with the target of `f`.

15 *Returns:* `*this`

```
function& operator=(nullptr_t) noexcept;
```

16 *Effects:* If `*this != nullptr`, destroys the target of `this`.

17 *Postconditions:* `!(*this)`.

18 *Returns:* `*this`

```
template<class F> function& operator=(F&& f);
```

19 *Effects:* `function(std::forward<F>(f)).swap(*this);`

20 *Returns:* `*this`

21 *Remarks:* This assignment operator shall not participate in overload resolution unless `declval<typename decay<F>::type&>()` is Callable (20.9.12.2) for argument types `ArgTypes...` and return type `R`.

```
template<class F> function& operator=(reference_wrapper<F> f) noexcept;
```

22 *Effects:* `function(f).swap(*this);`

23 *Returns:* `*this`

```
~function();
```

24 *Effects:* If `*this != nullptr`, destroys the target of `this`.

20.9.12.2.2 `function` modifiers [func.wrap.func.mod]

```
void swap(function& other) noexcept;
```

1 *Effects:* interchanges the targets of `*this` and `other`.

```
template<class F, class A>
  void assign(F&& f, const A& a);
```

2 *Effects:* `function(allocator_arg, a, std::forward<F>(f)).swap(*this)`

20.9.12.2.3 `function` capacity [func.wrap.func.cap]

```
explicit operator bool() const noexcept;
```

1 *Returns:* `true` if `*this` has a target, otherwise `false`.

20.9.12.2.4 `function` invocation [func.wrap.func.inv]

```
R operator()(ArgTypes... args) const;
```

1 *Returns:* *INVOKE*`(f, std::forward<ArgTypes>(args)..., R)` (20.9.2), where `f` is the target object (20.9.1) of `*this`.

2 *Throws:* `bad_function_call` if `!*this`; otherwise, any exception thrown by the wrapped callable object.

20.9.12.2.5 function target access [func.wrap.func.targ]

```
const std::type_info& target_type() const noexcept;
```

1 *Returns:* If `*this` has a target of type `T`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<class T>       T* target() noexcept;
template<class T> const T* target() const noexcept;
```

2 *Requires:* `T` shall be a type that is Callable (20.9.12.2) for parameter types `ArgTypes` and return type `R`.

3 *Returns:* If `target_type() == typeid(T)` a pointer to the stored function target; otherwise a null pointer.

20.9.12.2.6 null pointer comparison operators [func.wrap.func.nullptr]

```
template <class R, class... ArgTypes>
  bool operator==(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
template <class R, class... ArgTypes>
  bool operator==(nullptr_t, const function<R(ArgTypes...)>& f) noexcept;
```

1 *Returns:* `!f`.

```
template <class R, class... ArgTypes>
  bool operator!=(const function<R(ArgTypes...)>& f, nullptr_t) noexcept;
template <class R, class... ArgTypes>
  bool operator!=(nullptr_t, const function<R(ArgTypes...)>& f) noexcept;
```

2 *Returns:* `(bool) f`.

20.9.12.2.7 specialized algorithms [func.wrap.func.alg]

```
template<class R, class... ArgTypes>
  void swap(function<R(ArgTypes...)>& f1, function<R(ArgTypes...)>& f2);
```

1 *Effects:* `f1.swap(f2);`

20.9.13 Class template `hash` [unord.hash]

1 The unordered associative containers defined in 23.5 use specializations of the class template `hash` as the default hash function. For all object types `Key` for which there exists a specialization `hash<Key>`, and for all enumeration types (7.2) `Key`, the instantiation `hash<Key>` shall:

(1.1) — satisfy the `Hash` requirements (17.6.3.4), with `Key` as the function call argument type, the `Default-Constructible` requirements (Table 19), the `CopyAssignable` requirements (Table 23),

(1.2) — be swappable (17.6.3.2) for lvalues,

(1.3) — provide two nested types `result_type` and `argument_type` which shall be synonyms for `size_t` and `Key`, respectively,

(1.4) — satisfy the requirement that if `k1 == k2` is true, `h(k1) == h(k2)` is also true, where `h` is an object of type `hash<Key>` and `k1` and `k2` are objects of type `Key`;

(1.5) — satisfy the requirement that the expression `h(k)`, where `h` is an object of type `hash<Key>` and `k` is an object of type `Key`, shall not throw an exception unless `hash<Key>` is a user-defined specialization that depends on at least one user-defined type.

```
template <> struct hash<bool>;
template <> struct hash<char>;
template <> struct hash<signed char>;
template <> struct hash<unsigned char>;
template <> struct hash<char16_t>;
template <> struct hash<char32_t>;
template <> struct hash<wchar_t>;
template <> struct hash<short>;
template <> struct hash<unsigned short>;
template <> struct hash<int>;
template <> struct hash<unsigned int>;
template <> struct hash<long>;
template <> struct hash<unsigned long>;
template <> struct hash<long long>;
template <> struct hash<unsigned long long>;
template <> struct hash<float>;
template <> struct hash<double>;
template <> struct hash<long double>;
template <class T> struct hash<T*>;
```

2 The template specializations shall meet the requirements of class template `hash` (20.9.13).

20.10 Metaprogramming and type traits [meta]

1 This subclause describes components used by C++ programs, particularly in templates, to support the widest possible range of types, optimise template code usage, detect type related user errors, and perform type inference and transformation at compile time. It includes type classification traits, type property inspection traits, and type transformations. The type classification traits describe a complete taxonomy of all possible C++ types, and state where in that taxonomy a given type belongs. The type property inspection traits allow important characteristics of types or of combinations of types to be inspected. The type transformations allow certain properties of types to be manipulated.

20.10.1 Requirements [meta.rqmts]

1 A *UnaryTypeTrait* describes a property of a type. It shall be a class template that takes one template type argument and, optionally, additional arguments that help define the property being described. It shall be `DefaultConstructible`, `CopyConstructible`, and publicly and unambiguously derived, directly or indirectly, from its *BaseCharacteristic*, which is a specialization of the template `integral_constant` (20.10.3), with the arguments to the template `integral_constant` determined by the requirements for the particular property being described. The member names of the BaseCharacteristic shall not be hidden and shall be unambiguously available in the UnaryTypeTrait.

2 A *BinaryTypeTrait* describes a relationship between two types. It shall be a class template that takes two template type arguments and, optionally, additional arguments that help define the relationship being described. It shall be `DefaultConstructible`, `CopyConstructible`, and publicly and unambiguously derived, directly or indirectly, from its *BaseCharacteristic*, which is a specialization of the template `integral_constant` (20.10.3), with the arguments to the template `integral_constant` determined by the requirements for the particular relationship being described. The member names of the BaseCharacteristic shall not be hidden and shall be unambiguously available in the BinaryTypeTrait.

3 A *TransformationTrait* modifies a property of a type. It shall be a class template that takes one template type argument and, optionally, additional arguments that help define the modification. It shall define a publicly accessible nested type named `type`, which shall be a synonym for the modified type.

20.10.2 Header <type_traits> synopsis [meta.type.synop]

```
namespace std {
  // 20.10.3, helper class:
  template <class T, T v> struct integral_constant;

  template <bool B>
    using bool_constant = integral_constant<bool, B>;
  typedef bool_constant<true> true_type;
  typedef bool_constant<false> false_type;

  // 20.10.4.1, primary type categories:
  template <class T> struct is_void;
  template <class T> struct is_null_pointer;
  template <class T> struct is_integral;
  template <class T> struct is_floating_point;
  template <class T> struct is_array;
  template <class T> struct is_pointer;
  template <class T> struct is_lvalue_reference;
  template <class T> struct is_rvalue_reference;
  template <class T> struct is_member_object_pointer;
  template <class T> struct is_member_function_pointer;
  template <class T> struct is_enum;
  template <class T> struct is_union;
```

```
template <class T> struct is_class;
template <class T> struct is_function;

// 20.10.4.2, composite type categories:
template <class T> struct is_reference;
template <class T> struct is_arithmetic;
template <class T> struct is_fundamental;
template <class T> struct is_object;
template <class T> struct is_scalar;
template <class T> struct is_compound;
template <class T> struct is_member_pointer;

// 20.10.4.3, type properties:
template <class T> struct is_const;
template <class T> struct is_volatile;
template <class T> struct is_trivial;
template <class T> struct is_trivially_copyable;
template <class T> struct is_standard_layout;
template <class T> struct is_pod;
template <class T> struct is_literal_type;
template <class T> struct is_empty;
template <class T> struct is_polymorphic;
template <class T> struct is_abstract;
template <class T> struct is_final;

template <class T> struct is_signed;
template <class T> struct is_unsigned;

template <class T, class... Args> struct is_constructible;
template <class T> struct is_default_constructible;
template <class T> struct is_copy_constructible;
template <class T> struct is_move_constructible;

template <class T, class U> struct is_assignable;
template <class T> struct is_copy_assignable;
template <class T> struct is_move_assignable;

template <class T> struct is_destructible;

template <class T, class... Args> struct is_trivially_constructible;
template <class T> struct is_trivially_default_constructible;
template <class T> struct is_trivially_copy_constructible;
template <class T> struct is_trivially_move_constructible;

template <class T, class U> struct is_trivially_assignable;
template <class T> struct is_trivially_copy_assignable;
template <class T> struct is_trivially_move_assignable;
template <class T> struct is_trivially_destructible;

template <class T, class... Args> struct is_nothrow_constructible;
template <class T> struct is_nothrow_default_constructible;
template <class T> struct is_nothrow_copy_constructible;
template <class T> struct is_nothrow_move_constructible;

template <class T, class U> struct is_nothrow_assignable;
```

```
template <class T> struct  is_nothrow_copy_assignable;
template <class T> struct is_nothrow_move_assignable;

template <class T> struct is_nothrow_destructible;
template <class T> struct has_virtual_destructor;

// 20.10.5, type property queries:
template <class T> struct alignment_of;
template <class T> struct rank;
template <class T, unsigned I = 0> struct extent;

// 20.10.6, type relations:
template <class T, class U> struct is_same;
template <class Base, class Derived> struct is_base_of;
template <class From, class To> struct is_convertible;

// 20.10.7.1, const-volatile modifications:
template <class T> struct remove_const;
template <class T> struct remove_volatile;
template <class T> struct remove_cv;
template <class T> struct add_const;
template <class T> struct add_volatile;
template <class T> struct add_cv;

template <class T>
  using remove_const_t    = typename remove_const<T>::type;
template <class T>
  using remove_volatile_t = typename remove_volatile<T>::type;
template <class T>
  using remove_cv_t       = typename remove_cv<T>::type;
template <class T>
  using add_const_t       = typename add_const<T>::type;
template <class T>
  using add_volatile_t    = typename add_volatile<T>::type;
template <class T>
  using add_cv_t          = typename add_cv<T>::type;

// 20.10.7.2, reference modifications:
template <class T> struct remove_reference;
template <class T> struct add_lvalue_reference;
template <class T> struct add_rvalue_reference;

template <class T>
  using remove_reference_t     = typename remove_reference<T>::type;
template <class T>
  using add_lvalue_reference_t = typename add_lvalue_reference<T>::type;
template <class T>
  using add_rvalue_reference_t = typename add_rvalue_reference<T>::type;

// 20.10.7.3, sign modifications:
template <class T> struct make_signed;
template <class T> struct make_unsigned;

template <class T>
  using make_signed_t   = typename make_signed<T>::type;
```

```
  template <class T>
    using make_unsigned_t = typename make_unsigned<T>::type;

  // 20.10.7.4, array modifications:
  template <class T> struct remove_extent;
  template <class T> struct remove_all_extents;

  template <class T>
    using remove_extent_t      = typename remove_extent<T>::type;
  template <class T>
    using remove_all_extents_t = typename remove_all_extents<T>::type;

  // 20.10.7.5, pointer modifications:
  template <class T> struct remove_pointer;
  template <class T> struct add_pointer;

  template <class T>
    using remove_pointer_t = typename remove_pointer<T>::type;
  template <class T>
    using add_pointer_t    = typename add_pointer<T>::type;

  // 20.10.7.6, other transformations:
  template <std::size_t Len,
            std::size_t Align = default-alignment>   // see 20.10.7.6
    struct aligned_storage;
  template <std::size_t Len, class... Types> struct aligned_union;
  template <class T> struct decay;
  template <bool, class T = void> struct enable_if;
  template <bool, class T, class F> struct conditional;
  template <class... T> struct common_type;
  template <class T> struct underlying_type;
  template <class> class result_of;   // not defined
  template <class F, class... ArgTypes> class result_of<F(ArgTypes...)>;

  template <std::size_t Len,
            std::size_t Align = default-alignment > // see 20.10.7.6
    using aligned_storage_t = typename aligned_storage<Len,Align>::type;
  template <std::size_t Len, class... Types>
    using aligned_union_t   = typename aligned_union<Len,Types...>::type;
  template <class T>
    using decay_t           = typename decay<T>::type;
  template <bool b, class T = void>
    using enable_if_t       = typename enable_if<b,T>::type;
  template <bool b, class T, class F>
    using conditional_t     = typename conditional<b,T,F>::type;
  template <class... T>
    using common_type_t     = typename common_type<T...>::type;
  template <class T>
    using underlying_type_t = typename underlying_type<T>::type;
  template <class T>
    using result_of_t       = typename result_of<T>::type;
  template <class...>
    using void_t            = void;
} // namespace std
```

1 The behavior of a program that adds specializations for any of the class templates defined in this subclause is undefined unless otherwise specified.

20.10.3 Helper classes [meta.help]

```
namespace std {
  template <class T, T v>
  struct integral_constant {
    static constexpr T value = v;
    typedef T value_type;
    typedef integral_constant<T,v> type;
    constexpr operator value_type() const noexcept { return value; }
    constexpr value_type operator()() const noexcept { return value; }
  };
}
```

1 The class template `integral_constant`, alias template `bool_constant`, and its associated typedefs `true_type` and `false_type` are used as base classes to define the interface for various type traits.

20.10.4 Unary type traits [meta.unary]

1 This sub-clause contains templates that may be used to query the properties of a type at compile time.

2 Each of these templates shall be a UnaryTypeTrait (20.10.1) with a BaseCharacteristic of `true_type` if the corresponding condition is true, otherwise `false_type`.

20.10.4.1 Primary type categories [meta.unary.cat]

1 The primary type categories correspond to the descriptions given in section 3.9 of the C++ standard.

2 For any given type `T`, the result of applying one of these templates to `T` and to *cv-qualified* `T` shall yield the same result.

3 [*Note:* For any given type `T`, exactly one of the primary type categories has a `value` member that evaluates to `true`. — *end note*]

Table 47 — Primary type category predicates

Template	Condition	Comments
`template <class T>` `struct is_void;`	`T` is `void`	
`template <class T>` `struct is_null_pointer;`	`T` is `std::nullptr_t` (3.9.1)	
`template <class T>` `struct is_integral;`	`T` is an integral type (3.9.1)	
`template <class T>` `struct is_floating_point;`	`T` is a floating point type (3.9.1)	
`template <class T>` `struct is_array;`	`T` is an array type (3.9.2) of known or unknown extent	Class template `array` (23.3.2) is not an array type.
`template <class T>` `struct is_pointer;`	`T` is a pointer type (3.9.2)	Includes pointers to functions but not pointers to non-static members.
`template <class T>` `struct is_lvalue_reference;`	`T` is an lvalue reference type (8.3.2)	
`template <class T>` `struct is_rvalue_reference;`	`T` is an rvalue reference type (8.3.2)	

Table 47 — Primary type category predicates (continued)

Template	Condition	Comments
`template <class T> struct is_member_object_pointer;`	`T` is a pointer to non-static data member	
`template <class T> struct is_member_function_pointer;`	`T` is a pointer to non-static member function	
`template <class T> struct is_enum;`	`T` is an enumeration type (3.9.2)	
`template <class T> struct is_union;`	`T` is a union type (3.9.2)	
`template <class T> struct is_class;`	`T` is a class type but not a union type (3.9.2)	
`template <class T> struct is_function;`	`T` is a function type (3.9.2)	

20.10.4.2 Composite type traits [meta.unary.comp]

1 These templates provide convenient compositions of the primary type categories, corresponding to the descriptions given in section 3.9.

2 For any given type `T`, the result of applying one of these templates to `T`, and to *cv-qualified* `T` shall yield the same result.

Table 48 — Composite type category predicates

Template	Condition	Comments
`template <class T> struct is_reference;`	`T` is an lvalue reference or an rvalue reference	
`template <class T> struct is_arithmetic;`	`T` is an arithmetic type (3.9.1)	
`template <class T> struct is_fundamental;`	`T` is a fundamental type (3.9.1)	
`template <class T> struct is_object;`	`T` is an object type (3.9)	
`template <class T> struct is_scalar;`	`T` is a scalar type (3.9)	
`template <class T> struct is_compound;`	`T` is a compound type (3.9.2)	
`template <class T> struct is_member_pointer;`	`T` is a pointer to non-static data member or non-static member function	

20.10.4.3 Type properties [meta.unary.prop]

1 These templates provide access to some of the more important properties of types.

2 It is unspecified whether the library defines any full or partial specializations of any of these templates.

3 For all of the class templates `X` declared in this Clause, instantiating that template with a template-argument that is a class template specialization may result in the implicit instantiation of the template argument if and only if the semantics of `X` require that the argument must be a complete type.

Table 49 — Type property predicates

Template	Condition	Preconditions
`template <class T> struct is_const;`	`T` is const-qualified (3.9.3)	
`template <class T> struct is_volatile;`	`T` is volatile-qualified (3.9.3)	
`template <class T> struct is_trivial;`	`T` is a trivial type (3.9)	`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) void.
`template <class T> struct is_trivially_copyable;`	`T` is a trivially copyable type (3.9)	`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) `void`.
`template <class T> struct is_standard_layout;`	`T` is a standard-layout type (3.9)	`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) void.
`template <class T> struct is_pod;`	`T` is a POD type (3.9)	`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) void.
`template <class T> struct is_literal_type;`	`T` is a literal type (3.9)	`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) `void`.
`template <class T> struct is_empty;`	`T` is a class type, but not a union type, with no non-static data members other than bit-fields of length 0, no virtual member functions, no virtual base classes, and no base class `B` for which `is_empty<B>::value` is false.	If `T` is a non-union class type, `T` shall be a complete type.
`template <class T> struct is_polymorphic;`	`T` is a polymorphic class (10.3)	If `T` is a non-union class type, `T` shall be a complete type.
`template <class T> struct is_abstract;`	`T` is an abstract class (10.4)	If `T` is a non-union class type, `T` shall be a complete type.
`template <class T> struct is_final;`	`T` is a class type marked with the *class-virt-specifier* `final` (Clause 9). [*Note:* A union is a class type that can be marked with `final`. — *end note*]	If `T` is a class type, `T` shall be a complete type.

Table 49 — Type property predicates (continued)

Template	Condition	Preconditions
`template <class T> struct is_signed;`	If `is_arithmetic<T>::value` is `true`, the same result as `bool_constant<T(-1) < T(0)>::value`; otherwise, `false`	
`template <class T> struct is_unsigned;`	If `is_arithmetic<T>::value` is `true`, the same result as `bool_constant<T(0) < T(-1)>::value`; otherwise, `false`	
`template <class T, class... Args> struct is_constructible;`	*see below*	`T` and all types in the parameter pack `Args` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.
`template <class T> struct is_default_constructible;`	`is_constructible<T>::value` is `true`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.
`template <class T> struct is_copy_constructible;`	For a referenceable type `T`, the same result as `is_constructible<T, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.
`template <class T> struct is_move_constructible;`	For a referenceable type `T`, the same result as `is_constructible<T, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.

Table 49 — Type property predicates (continued)

Template	Condition	Preconditions
`template <class T, class U>` `struct is_assignable;`	The expression `declval<T>() = declval<U>()` is well-formed when treated as an unevaluated operand (Clause 5). Access checking is performed as if in a context unrelated to `T` and `U`. Only the validity of the immediate context of the assignment expression is considered. [*Note:* The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]	`T` and `U` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.
`template <class T>` `struct is_copy_assignable;`	For a referenceable type `T`, the same result as `is_assignable<T&, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.
`template <class T>` `struct is_move_assignable;`	For a referenceable type `T`, the same result as `is_assignable<T&, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.

Table 49 — Type property predicates (continued)

Template	Condition	Preconditions
`template <class T>` `struct is_destructible;`	For reference types, `is_destructible<T>::value` is `true`. For incomplete types and function types, `is_destructible<T>::value` is `false`. For object types and given `U` equal to `remove_all_extents_t<T>`, if the expression `std::declval<U&>().~U()` is well-formed when treated as an unevaluated operand (Clause 5), then `is_destructible<T>::value` is `true`, otherwise it is `false`.	`T` shall be a complete type, (possibly *cv*-qualified) `void`, or an array of unknown bound.

`template <class T, class... Args>`
`struct`
`is_trivially_constructible;`
	`is_constructible<T, Args...>::value` is `true` and the variable definition for `is_constructible`, as defined below, is known to call no operation that is not trivial (3.9, 12).	`T` and all types in the parameter pack `Args` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.

`template <class T>`
`struct`
`is_trivially_default_constructible;`
	`is_trivially_constructible<T>::value` is `true`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.

`template <class T>`
`struct`
`is_trivially_copy_constructible;`
	For a referenceable type `T`, the same result as `is_trivially_constructible<T, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.

`template <class T>`
`struct`
`is_trivially_move_constructible;`
	For a referenceable type `T`, the same result as `is_trivially_constructible<T, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.

Table 49 — Type property predicates (continued)

Template	Condition	Preconditions
`template <class T, class U> struct is_trivially_assignable;`	`is_assignable<T, U>::value` is `true` and the assignment, as defined by `is_assignable`, is known to call no operation that is not trivial (3.9, 12).	`T` and `U` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.
`template <class T> struct is_trivially_copy_assignable;`	For a referenceable type `T`, the same result as `is_trivially_assignable<T&, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_trivially_move_assignable;`	For a referenceable type `T`, the same result as `is_trivially_assignable<T&, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_trivially_destructible;`	`is_destructible<T>::value` is `true` and the indicated destructor is known to be trivial.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T, class... Args> struct is_nothrow_constructible;`	`is_constructible<T, Args...>::value` is `true` and the variable definition for `is_constructible`, as defined below, is known not to throw any exceptions (5.3.7).	`T` and all types in the parameter pack `Args` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.
`template <class T> struct is_nothrow_default_constructible;`	`is_nothrow_constructible<T>::value` is `true`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_nothrow_copy_constructible;`	For a referenceable type `T`, the same result as `is_nothrow_constructible<T, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_nothrow_move_constructible;`	For a referenceable type `T`, the same result as `is_nothrow_constructible<T, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.

Table 49 — Type property predicates (continued)

Template	Condition	Preconditions
`template <class T, class U> struct is_nothrow_assignable;`	`is_assignable<T, U>::value` is `true` and the assignment is known not to throw any exceptions (5.3.7).	`T` and `U` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.
`template <class T> struct is_nothrow_copy_assignable;`	For a referenceable type `T`, the same result as `is_nothrow_assignable<T&, const T&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_nothrow_move_assignable;`	For a referenceable type `T`, the same result as `is_nothrow_assignable<T&, T&&>::value`, otherwise `false`.	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct is_nothrow_destructible;`	`is_destructible<T>::value` is `true` and the indicated destructor is known not to throw any exceptions (5.3.7).	`T` shall be a complete type, (possibly cv-qualified) void, or an array of unknown bound.
`template <class T> struct has_virtual_destructor;`	`T` has a virtual destructor (12.4)	If `T` is a non-union class type, `T` shall be a complete type.

4 [*Example:*

```
is_const<const volatile int>::value     // true
is_const<const int*>::value             // false
is_const<const int&>::value             // false
is_const<int[3]>::value                 // false
is_const<const int[3]>::value           // true
```

— *end example*]

5 [*Example:*

```
remove_const_t<const volatile int>  // volatile int
remove_const_t<const int* const>    // const int*
remove_const_t<const int&>          // const int&
remove_const_t<const int[3]>        // int[3]
```

— *end example*]

6 [*Example:*

```
// Given:
struct P final { };
union U1 { };
union U2 final { };
```

```
// the following assertions hold:
static_assert(!is_final<int>::value, "Error!");
static_assert( is_final<P>::value, "Error!");
static_assert(!is_final<U1>::value, "Error!");
static_assert( is_final<U2>::value, "Error!");
```

— *end example*]

7 Given the following function declaration:

```
template <class T>
  add_rvalue_reference_t<T> create() noexcept;
```

the predicate condition for a template specialization `is_constructible<T, Args...>` shall be satisfied if and only if the following variable definition would be well-formed for some invented variable `t`:

```
T t(create<Args>()...);
```

[*Note:* These tokens are never interpreted as a function declaration. — *end note*] Access checking is performed as if in a context unrelated to `T` and any of the `Args`. Only the validity of the immediate context of the variable initialization is considered. [*Note:* The evaluation of the initialization can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

20.10.5 Type property queries [meta.unary.prop.query]

1 This sub-clause contains templates that may be used to query properties of types at compile time.

Table 50 — Type property queries

Template	Value
`template <class T>` `struct alignment_of;`	`alignof(T)`. *Requires:* `alignof(T)` shall be a valid expression (5.3.6)
`template <class T>` `struct rank;`	If `T` names an array type, an integer value representing the number of dimensions of `T`; otherwise, 0.

`template <class T,`
`unsigned I = 0>`
`struct extent;`
	If `T` is not an array type, or if it has rank less than or equal to `I`, or if `I` is 0 and `T` has type "array of unknown bound of `U`", then 0; otherwise, the bound (8.3.4) of the `I`'th dimension of `T`, where indexing of `I` is zero-based

2 Each of these templates shall be a `UnaryTypeTrait` (20.10.1) with a `BaseCharacteristic` of `integral_constant<size_t, Value>`.

3 [*Example:*

```
// the following assertions hold:
assert(rank<int>::value == 0);
assert(rank<int[2]>::value == 1);
assert(rank<int[][4]>::value == 2);
```

— *end example*]

4 [*Example:*

```
// the following assertions hold:
assert(extent<int>::value == 0);
```

```
assert(extent<int[2]>::value == 2);
assert(extent<int[2][4]>::value == 2);
assert(extent<int[][4]>::value == 0);
assert((extent<int, 1>::value) == 0);
assert((extent<int[2], 1>::value) == 0);
assert((extent<int[2][4], 1>::value) == 4);
assert((extent<int[][4], 1>::value) == 4);
```

— *end example*]

20.10.6 Relationships between types [meta.rel]

1 This sub-clause contains templates that may be used to query relationships between types at compile time.

2 Each of these templates shall be a BinaryTypeTrait (20.10.1) with a BaseCharacteristic of `true_type` if the corresponding condition is true, otherwise `false_type`.

Table 51 — Type relationship predicates

Template	Condition	Comments
`template <class T, class U> struct is_same;`	`T` and `U` name the same type with the same cv-qualifications	
`template <class Base, class Derived> struct is_base_of;`	`Base` is a base class of `Derived` (Clause 10) without regard to cv-qualifiers or `Base` and `Derived` are not unions and name the same class type without regard to cv-qualifiers	If `Base` and `Derived` are non-union class types and are different types (ignoring possible cv-qualifiers) then `Derived` shall be a complete type. [*Note:* Base classes that are private, protected, or ambiguous are, nonetheless, base classes. — *end note*]
`template <class From, class To> struct is_convertible;`	*see below*	`From` and `To` shall be complete types, arrays of unknown bound, or (possibly cv-qualified) `void` types.

3 [*Example:*

```
struct B {};
struct B1 : B {};
struct B2 : B {};
struct D : private B1, private B2 {};

is_base_of<B, D>::value         // true
is_base_of<const B, D>::value   // true
is_base_of<B, const D>::value   // true
is_base_of<B, const B>::value   // true
is_base_of<D, B>::value         // false
is_base_of<B&, D&>::value       // false
is_base_of<B[3], D[3]>::value   // false
is_base_of<int, int>::value     // false
```

— *end example*]

4 Given the following function declaration:

```
template <class T>
  add_rvalue_reference_t<T> create() noexcept;
```

the predicate condition for a template specialization `is_convertible<From, To>` shall be satisfied if and only if the return expression in the following code would be well-formed, including any implicit conversions to the return type of the function:

```
To test() {
  return create<From>();
}
```

[*Note:* This requirement gives well defined results for reference types, void types, array types, and function types. — *end note*] Access checking is performed as if in a context unrelated to `To` and `From`. Only the validity of the immediate context of the expression of the *return-statement* (including conversions to the return type) is considered. [*Note:* The evaluation of the conversion can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

20.10.7 Transformations between types [meta.trans]

1 This sub-clause contains templates that may be used to transform one type to another following some predefined rule.

2 Each of the templates in this subclause shall be a *TransformationTrait* (20.10.1).

20.10.7.1 Const-volatile modifications [meta.trans.cv]

Table 52 — Const-volatile modifications

Template	Comments
`template <class T>` `struct remove_const;`	The member typedef `type` shall name the same type as `T` except that any top-level const-qualifier has been removed. [*Example:* `remove_const_t<const volatile int>` evaluates to `volatile int`, whereas `remove_const_t<const int*>` evaluates to `const int*`. — *end example*]
`template <class T>` `struct remove_volatile;`	The member typedef `type` shall name the same type as `T` except that any top-level volatile-qualifier has been removed. [*Example:* `remove_volatile_t<const volatile int>` evaluates to `const int`, whereas `remove_volatile_t<volatile int*>` evaluates to `volatile int*`. — *end example*]
`template <class T>` `struct remove_cv;`	The member typedef `type` shall be the same as `T` except that any top-level cv-qualifier has been removed. [*Example:* `remove_cv_t<const volatile int>` evaluates to `int`, whereas `remove_cv_t<const volatile int*>` evaluates to `const volatile int*`. — *end example*]
`template <class T>` `struct add_const;`	If `T` is a reference, function, or top-level const-qualified type, then `type` shall name the same type as `T`, otherwise `T const`.
`template <class T>` `struct add_volatile;`	If `T` is a reference, function, or top-level volatile-qualified type, then `type` shall name the same type as `T`, otherwise `T volatile`.
`template <class T>` `struct add_cv;`	The member typedef `type` shall name the same type as `add_const_t<add_volatile_t<T>>`.

20.10.7.2 Reference modifications [meta.trans.ref]

Table 53 — Reference modifications

Template	Comments
`template <class T> struct remove_reference;`	If `T` has type "reference to `T1`" then the member typedef `type` shall name `T1`; otherwise, `type` shall name `T`.
`template <class T> struct add_lvalue_reference;`	If `T` names an object or function type then the member typedef `type` shall name `T&`; otherwise, if `T` names a type "rvalue reference to `T1`" then the member typedef `type` shall name `T1&`; otherwise, `type` shall name `T`.
`template <class T> struct add_rvalue_reference;`	If `T` names an object or function type then the member typedef `type` shall name `T&&`; otherwise, `type` shall name `T`. [*Note:* This rule reflects the semantics of reference collapsing (8.3.2). For example, when a type `T` names a type `T1&`, the type `add_rvalue_reference_t<T>` is not an rvalue reference. — *end note*]

20.10.7.3 Sign modifications [meta.trans.sign]

Table 54 — Sign modifications

Template	Comments
`template <class T> struct make_signed;`	If `T` names a (possibly cv-qualified) signed integer type (3.9.1) then the member typedef `type` shall name the type `T`; otherwise, if `T` names a (possibly cv-qualified) unsigned integer type then `type` shall name the corresponding signed integer type, with the same cv-qualifiers as `T`; otherwise, `type` shall name the signed integer type with smallest rank (4.13) for which `sizeof(T) == sizeof(type)`, with the same cv-qualifiers as `T`. *Requires:* `T` shall be a (possibly cv-qualified) integral type or enumeration but not a `bool` type.
`template <class T> struct make_unsigned;`	If `T` names a (possibly cv-qualified) unsigned integer type (3.9.1) then the member typedef `type` shall name the type `T`; otherwise, if `T` names a (possibly cv-qualified) signed integer type then `type` shall name the corresponding unsigned integer type, with the same cv-qualifiers as `T`; otherwise, `type` shall name the unsigned integer type with smallest rank (4.13) for which `sizeof(T) == sizeof(type)`, with the same cv-qualifiers as `T`. *Requires:* `T` shall be a (possibly cv-qualified) integral type or enumeration but not a `bool` type.

20.10.7.4 Array modifications [meta.trans.arr]

Table 55 — Array modifications

Template	Comments
`template <class T>` `struct remove_extent;`	If `T` names a type "array of `U`", the member typedef `type` shall be `U`, otherwise `T`. [*Note:* For multidimensional arrays, only the first array dimension is removed. For a type "array of `const U`", the resulting type is `const U`. — *end note*]
`template <class T>` `struct remove_all_extents;`	If `T` is "multi-dimensional array of `U`", the resulting member typedef `type` is `U`, otherwise `T`.

1 [*Example*

```
// the following assertions hold:
assert((is_same<remove_extent_t<int>, int>::value));
assert((is_same<remove_extent_t<int[2]>, int>::value));
assert((is_same<remove_extent_t<int[2][3]>, int[3]>::value));
assert((is_same<remove_extent_t<int[][3]>, int[3]>::value));
```

— *end example*]

2 [*Example*

```
// the following assertions hold:
assert((is_same<remove_all_extents_t<int>, int>::value));
assert((is_same<remove_all_extents_t<int[2]>, int>::value));
assert((is_same<remove_all_extents_t<int[2][3]>, int>::value));
assert((is_same<remove_all_extents_t<int[][3]>, int>::value));
```

— *end example*]

20.10.7.5 Pointer modifications [meta.trans.ptr]

Table 56 — Pointer modifications

Template	Comments
`template <class T>` `struct remove_pointer;`	If `T` has type "(possibly cv-qualified) pointer to `T1`" then the member typedef `type` shall name `T1`; otherwise, it shall name `T`.
`template <class T>` `struct add_pointer;`	The member typedef `type` shall name the same type as `remove_reference_t<T>*`.

20.10.7.6 Other transformations [meta.trans.other]

Table 57 — Other transformations

Template	Condition	Comments
`template <std::size_t Len, std::size_t Align = default-alignment> struct aligned_storage;`	`Len` shall not be zero. `Align` shall be equal to `alignof(T)` for some type `T` or to *default-alignment*.	The value of *default-alignment* shall be the most stringent alignment requirement for any C++ object type whose size is no greater than `Len` (3.9). The member typedef `type` shall be a POD type suitable for use as uninitialized storage for any object whose size is at most *Len* and whose alignment is a divisor of *Align*.
`template <std::size_t Len, class... Types> struct aligned_union;`	At least one type is provided.	The member typedef `type` shall be a POD type suitable for use as uninitialized storage for any object whose type is listed in `Types`; its size shall be at least `Len`. The static member `alignment_value` shall be an integral constant of type `std::size_t` whose value is the strictest alignment of all types listed in `Types`.
`template <class T> struct decay;`		Let `U` be `remove_reference_t<T>`. If `is_array<U>::value` is `true`, the member typedef `type` shall equal `remove_extent_t<U>*`. If `is_function<U>::value` is `true`, the member typedef `type` shall equal `add_pointer_t<U>`. Otherwise the member typedef `type` equals `remove_cv_t<U>`. [*Note:* This behavior is similar to the lvalue-to-rvalue (4.1), array-to-pointer (4.2), and function-to-pointer (4.3) conversions applied when an lvalue expression is used as an rvalue, but also strips *cv*-qualifiers from class types in order to more closely model by-value argument passing. — *end note*]
`template <bool B, class T = void> struct enable_if;`		If `B` is `true`, the member typedef `type` shall equal `T`; otherwise, there shall be no member `type`.
`template <bool B, class T, class F> struct conditional;`		If `B` is `true`, the member typedef `type` shall equal `T`. If `B` is `false`, the member typedef `type` shall equal `F`.

Table 57 — Other transformations (continued)

Template	Condition	Comments
`template <class... T> struct common_type;`		The member typedef `type` shall be defined or omitted as specified below. If it is omitted, there shall be no member `type`. All types in the parameter pack `T` shall be complete or (possibly *cv*) `void`. A program may specialize this trait if at least one template parameter in the specialization is a user-defined type. [*Note:* Such specializations are needed when only explicit conversions are desired among the template arguments. — *end note*]
`template <class T> struct underlying_type;`	`T` shall be a complete enumeration type (7.2)	The member typedef `type` shall name the underlying type of `T`.
`template <class Fn, class... ArgTypes> struct result_of<Fn(ArgTypes...)>;`	`Fn` and all types in the parameter pack `ArgTypes` shall be complete types, (possibly cv-qualified) `void`, or arrays of unknown bound.	If the expression *`INVOKE`*`(declval<Fn>(), declval<ArgTypes>()...)` is well formed when treated as an unevaluated operand (Clause 5), the member typedef `type` shall name the type `decltype(`*`INVOKE`*`(declval<Fn>(), declval<ArgTypes>()...));` otherwise, there shall be no member `type`. Access checking is performed as if in a context unrelated to `Fn` and `ArgTypes`. Only the validity of the immediate context of the expression is considered. [*Note:* The compilation of the expression can result in side effects such as the instantiation of class template specializations and function template specializations, the generation of implicitly-defined functions, and so on. Such side effects are not in the "immediate context" and can result in the program being ill-formed. — *end note*]

1 [*Note:* A typical implementation would define `aligned_storage` as:

```
template <std::size_t Len, std::size_t Alignment>
struct aligned_storage {
  typedef struct {
    alignas(Alignment) unsigned char __data[Len];
  } type;
};
```

— *end note*]

2 It is implementation-defined whether any extended alignment is supported (3.11).

3 For the `common_type` trait applied to a parameter pack `T` of types, the member `type` shall be either defined or not present as follows:

(3.1) — If `sizeof...(T)` is zero, there shall be no member `type`.

(3.2) — If `sizeof...(T)` is one, let `T0` denote the sole type in the pack `T`. The member typedef `type` shall denote the same type as `decay_t<T0>`.

(3.3) — If `sizeof...(T)` is greater than one, let `T1`, `T2`, and `R`, respectively, denote the first, second, and (pack of) remaining types comprising `T`. [*Note:* `sizeof...(R)` may be zero. — *end note*] Let `C` denote the type, if any, of an unevaluated conditional expression (5.16) whose first operand is an arbitrary value of type `bool`, whose second operand is an xvalue of type `T1`, and whose third operand is an xvalue of type `T2`. If there is such a type `C`, the member typedef `type` shall denote the same type, if any, as `common_type_t<C,R...>`. Otherwise, there shall be no member `type`.

4 [*Example:* Given these definitions:

```
typedef bool (&PF1)();
typedef short (*PF2)(long);

struct S {
  operator PF2() const;
  double operator()(char, int&);
  void fn(long) const;
  char data;
};

typedef void (S::*PMF)(long) const;
typedef char S::*PMD;
```

the following assertions will hold:

```
static_assert(is_same<result_of_t<S(int)>, short>::value, "Error!");
static_assert(is_same<result_of_t<S&(unsigned char, int&)>, double>::value, "Error!");
static_assert(is_same<result_of_t<PF1()>, bool>::value, "Error!");
static_assert(is_same<result_of_t<PMF(unique_ptr<S>, int)>, void>::value, "Error!");
static_assert(is_same<result_of_t<PMD(S)>, char&&>::value, "Error!");
static_assert(is_same<result_of_t<PMD(const S*)>, const char&>::value, "Error!");
```

— *end example*]

20.11 Compile-time rational arithmetic [ratio]

20.11.1 In general [ratio.general]

1 This subclause describes the ratio library. It provides a class template `ratio` which exactly represents any finite rational number with a numerator and denominator representable by compile-time constants of type `intmax_t`.

2 Throughout this subclause, the names of template parameters are used to express type requirements. If a template parameter is named `R1` or `R2`, and the template argument is not a specialization of the `ratio` template, the program is ill-formed.

20.11.2 Header <ratio> synopsis [ratio.syn]

```
namespace std {
  // 20.11.3, class template ratio
  template <intmax_t N, intmax_t D = 1> class ratio;

  // 20.11.4, ratio arithmetic
  template <class R1, class R2> using ratio_add = see below;
  template <class R1, class R2> using ratio_subtract = see below;
  template <class R1, class R2> using ratio_multiply = see below;
  template <class R1, class R2> using ratio_divide = see below;

  // 20.11.5, ratio comparison
  template <class R1, class R2> struct ratio_equal;
  template <class R1, class R2> struct ratio_not_equal;
  template <class R1, class R2> struct ratio_less;
  template <class R1, class R2> struct ratio_less_equal;
  template <class R1, class R2> struct ratio_greater;
  template <class R1, class R2> struct ratio_greater_equal;

  // 20.11.6, convenience SI typedefs
  typedef ratio<1, 1'000'000'000'000'000'000'000'000> yocto;  // see below
  typedef ratio<1,     1'000'000'000'000'000'000'000> zepto;  // see below
  typedef ratio<1,         1'000'000'000'000'000'000> atto;
  typedef ratio<1,             1'000'000'000'000'000> femto;
  typedef ratio<1,                 1'000'000'000'000> pico;
  typedef ratio<1,                     1'000'000'000> nano;
  typedef ratio<1,                         1'000'000> micro;
  typedef ratio<1,                             1'000> milli;
  typedef ratio<1,                               100> centi;
  typedef ratio<1,                                10> deci;
  typedef ratio<                                 10, 1> deca;
  typedef ratio<                                100, 1> hecto;
  typedef ratio<                              1'000, 1> kilo;
  typedef ratio<                          1'000'000, 1> mega;
  typedef ratio<                      1'000'000'000, 1> giga;
  typedef ratio<                  1'000'000'000'000, 1> tera;
  typedef ratio<              1'000'000'000'000'000, 1> peta;
  typedef ratio<          1'000'000'000'000'000'000, 1> exa;
  typedef ratio<      1'000'000'000'000'000'000'000, 1> zetta;  // see below
  typedef ratio<1'000'000'000'000'000'000'000'000, 1> yotta;  // see below
}
```

20.11.3 Class template ratio [ratio.ratio]

```
namespace std {
  template <intmax_t N, intmax_t D = 1>
  class ratio {
  public:
    static constexpr intmax_t num;
    static constexpr intmax_t den;
    typedef ratio<num, den> type;
  };
}
```

1 If the template argument D is zero or the absolute values of either of the template arguments N and D is not representable by type intmax_t, the program is ill-formed. [*Note:* These rules ensure that infinite ratios

are avoided and that for any negative input, there exists a representable value of its absolute value which is positive. In a two's complement representation, this excludes the most negative value. — *end note*]

2 The static data members `num` and `den` shall have the following values, where `gcd` represents the greatest common divisor of the absolute values of `N` and `D`:

(2.1) — `num` shall have the value `sign(N) * sign(D) * abs(N) / gcd`.

(2.2) — `den` shall have the value `abs(D) / gcd`.

20.11.4 Arithmetic on `ratios` [ratio.arithmetic]

1 Each of the alias templates `ratio_add`, `ratio_subtract`, `ratio_multiply`, and `ratio_divide` denotes the result of an arithmetic computation on two `ratios` `R1` and `R2`. With `X` and `Y` computed (in the absence of arithmetic overflow) as specified by Table 58, each alias denotes a `ratio<U, V>` such that `U` is the same as `ratio<X, Y>::num` and `V` is the same as `ratio<X, Y>::den`.

2 If it is not possible to represent `U` or `V` with `intmax_t`, the program is ill-formed. Otherwise, an implementation should yield correct values of `U` and `V`. If it is not possible to represent `X` or `Y` with `intmax_t`, the program is ill-formed unless the implementation yields correct values of `U` and `V`.

Table 58 — Expressions used to perform ratio arithmetic

Type	Value of `X`	Value of `Y`
`ratio_add<R1, R2>`	`R1::num * R2::den + R2::num * R1::den`	`R1::den * R2::den`
`ratio_subtract<R1, R2>`	`R1::num * R2::den - R2::num * R1::den`	`R1::den * R2::den`
`ratio_multiply<R1, R2>`	`R1::num * R2::num`	`R1::den * R2::den`
`ratio_divide<R1, R2>`	`R1::num * R2::den`	`R1::den * R2::num`

3 [*Example:*

```
static_assert(ratio_add<ratio<1,3>, ratio<1,6>>::num == 1, "1/3+1/6 == 1/2");
static_assert(ratio_add<ratio<1,3>, ratio<1,6>>::den == 2, "1/3+1/6 == 1/2");
static_assert(ratio_multiply<ratio<1,3>, ratio<3,2>>::num == 1, "1/3*3/2 == 1/2");
static_assert(ratio_multiply<ratio<1,3>, ratio<3,2>>::den == 2, "1/3*3/2 == 1/2");

// The following cases may cause the program to be ill-formed under some implementations
static_assert(ratio_add<ratio<1,INT_MAX>, ratio<1,INT_MAX>>::num == 2,
  "1/MAX+1/MAX == 2/MAX");
static_assert(ratio_add<ratio<1,INT_MAX>, ratio<1,INT_MAX>>::den == INT_MAX,
  "1/MAX+1/MAX == 2/MAX");
static_assert(ratio_multiply<ratio<1,INT_MAX>, ratio<INT_MAX,2>>::num == 1,
  "1/MAX * MAX/2 == 1/2");
static_assert(ratio_multiply<ratio<1,INT_MAX>, ratio<INT_MAX,2>>::den == 2,
  "1/MAX * MAX/2 == 1/2");
```

— *end example*]

20.11.5 Comparison of `ratios` [ratio.comparison]

```
template <class R1, class R2> struct ratio_equal
  : bool_constant<R1::num == R2::num && R1::den == R2::den> { };

template <class R1, class R2> struct ratio_not_equal
  : bool_constant<!ratio_equal<R1, R2>::value> { };
```

```
template <class R1, class R2> struct ratio_less
  : bool_constant<see below> { };
```

1 If `R1::num` × `R2::den` is less than `R2::num` × `R1::den`, `ratio_less<R1, R2>` shall be derived from `bool_constant<true>`; otherwise it shall be derived from `bool_constant<false>`. Implementations may use other algorithms to compute this relationship to avoid overflow. If overflow occurs, the program is ill-formed.

```
template <class R1, class R2> struct ratio_less_equal
  : bool_constant<!ratio_less<R2, R1>::value> { };

template <class R1, class R2> struct ratio_greater
  : bool_constant<ratio_less<R2, R1>::value> { };

template <class R1, class R2> struct ratio_greater_equal
  : bool_constant<!ratio_less<R1, R2>::value> { };
```

20.11.6 SI types for `ratio` [ratio.si]

1 For each of the typedefs `yocto`, `zepto`, `zetta`, and `yotta`, if both of the constants used in its specification are representable by `intmax_t`, the typedef shall be defined; if either of the constants is not representable by `intmax_t`, the typedef shall not be defined.

20.12 Time utilities [time]

20.12.1 In general [time.general]

1 This subclause describes the chrono library (20.12.2) and various C functions (20.12.8) that provide generally useful time utilities.

20.12.2 Header `<chrono>` synopsis [time.syn]

```
namespace std {
namespace chrono {

// 20.12.5, class template duration
template <class Rep, class Period = ratio<1> > class duration;

// 20.12.6, class template time_point
template <class Clock, class Duration = typename Clock::duration> class time_point;

}  // namespace chrono

// 20.12.4.3 common_type specializations
template <class Rep1, class Period1, class Rep2, class Period2>
  struct common_type<chrono::duration<Rep1, Period1>, chrono::duration<Rep2, Period2>>;

template <class Clock, class Duration1, class Duration2>
  struct common_type<chrono::time_point<Clock, Duration1>, chrono::time_point<Clock, Duration2>>;

namespace chrono {

// 20.12.4, customization traits
template <class Rep> struct treat_as_floating_point;
template <class Rep> struct duration_values;

// 20.12.5.5, duration arithmetic
template <class Rep1, class Period1, class Rep2, class Period2>
```

```
  common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  constexpr operator+(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  constexpr operator-(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period, class Rep2>
  duration<common_type_t<Rep1, Rep2>, Period>
  constexpr operator*(const duration<Rep1, Period>& d, const Rep2& s);
template <class Rep1, class Rep2, class Period>
  duration<common_type_t<Rep1, Rep2>, Period>
  constexpr operator*(const Rep1& s, const duration<Rep2, Period>& d);
template <class Rep1, class Period, class Rep2>
  duration<common_type_t<Rep1, Rep2>, Period>
  constexpr operator/(const duration<Rep1, Period>& d, const Rep2& s);
template <class Rep1, class Period1, class Rep2, class Period2>
  common_type_t<Rep1, Rep2>
  constexpr operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period, class Rep2>
  duration<common_type_t<Rep1, Rep2>, Period>
  constexpr operator%(const duration<Rep1, Period>& d, const Rep2& s);
template <class Rep1, class Period1, class Rep2, class Period2>
  common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  constexpr operator%(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);

// 20.12.5.6, duration comparisons
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator==(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator!=(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator< (const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator<=(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator> (const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator>=(const duration<Rep1, Period1>& lhs,
                            const duration<Rep2, Period2>& rhs);

// 20.12.5.7, duration_cast
template <class ToDuration, class Rep, class Period>
  constexpr ToDuration duration_cast(const duration<Rep, Period>& d);

// convenience typedefs
typedef duration<signed integer type of at least 64 bits,        nano> nanoseconds;
typedef duration<signed integer type of at least 55 bits,       micro> microseconds;
typedef duration<signed integer type of at least 45 bits,       milli> milliseconds;
typedef duration<signed integer type of at least 35 bits            > seconds;
typedef duration<signed integer type of at least 29 bits, ratio<  60>> minutes;
typedef duration<signed integer type of at least 23 bits, ratio<3600>> hours;
```

```
  // 20.12.6.5, time_point arithmetic
  template <class Clock, class Duration1, class Rep2, class Period2>
    constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
    operator+(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
  template <class Rep1, class Period1, class Clock, class Duration2>
    constexpr time_point<Clock, common_type_t<duration<Rep1, Period1>, Duration2>>
    operator+(const duration<Rep1, Period1>& lhs, const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Rep2, class Period2>
    constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
    operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
  template <class Clock, class Duration1, class Duration2>
    constexpr common_type_t<Duration1, Duration2>
    operator-(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);

  // 20.12.6.6 time_point comparisons
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator==(const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator!=(const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator< (const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator<=(const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator> (const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);
  template <class Clock, class Duration1, class Duration2>
     constexpr bool operator>=(const time_point<Clock, Duration1>& lhs,
                               const time_point<Clock, Duration2>& rhs);

  // 20.12.6.7, time_point_cast
  template <class ToDuration, class Clock, class Duration>
    constexpr time_point<Clock, ToDuration>
    time_point_cast(const time_point<Clock, Duration>& t);

  // 20.12.7, clocks
  class system_clock;
  class steady_clock;
  class high_resolution_clock;

  }  // namespace chrono

  inline namespace literals {
  inline namespace chrono_literals {

  //  20.12.5.8, suffixes for duration literals
  constexpr chrono::hours                                   operator "" h(unsigned long long);
  constexpr chrono::duration<unspecified , ratio<3600,1>> operator "" h(long double);
  constexpr chrono::minutes                                 operator "" min(unsigned long long);
  constexpr chrono::duration<unspecified , ratio<60,1>>   operator "" min(long double);
```

```
constexpr chrono::seconds                                operator "" s(unsigned long long);
constexpr chrono::duration<unspecified >                 operator "" s(long double);
constexpr chrono::milliseconds                           operator "" ms(unsigned long long);
constexpr chrono::duration<unspecified , milli>          operator "" ms(long double);
constexpr chrono::microseconds                           operator "" us(unsigned long long);
constexpr chrono::duration<unspecified , micro>          operator "" us(long double);
constexpr chrono::nanoseconds                            operator "" ns(unsigned long long);
constexpr chrono::duration<unspecified , nano>           operator "" ns(long double);

} // namespace chrono_literals
} // namespace literals

namespace chrono {

using namespace literals::chrono_literals;

} // namespace chrono

} // namespace std
```

20.12.3 Clock requirements [time.clock.req]

1 A clock is a bundle consisting of a `duration`, a `time_point`, and a function `now()` to get the current `time_point`. The origin of the clock's `time_point` is referred to as the clock's *epoch*. A clock shall meet the requirements in Table 59.

2 In Table 59 `C1` and `C2` denote clock types. `t1` and `t2` are values returned by `C1::now()` where the call returning `t1` happens before (1.10) the call returning `t2` and both of these calls occur before `C1::time_point::max()`. [*Note:* this means `C1` did not wrap around between `t1` and `t2`. — *end note*]

Table 59 — Clock requirements

Expression	Return type	Operational semantics
`C1::rep`	An arithmetic type or a class emulating an arithmetic type	The representation type of `C1::duration`.
`C1::period`	a specialization of `ratio`	The tick period of the clock in seconds.
`C1::duration`	`chrono::duration<C1::rep, C1::period>`	The `duration` type of the clock.
`C1::time_point`	`chrono::time_point<C1>` or `chrono::time_point<C2, C1::duration>`	The `time_point` type of the clock. `C1` and `C2` shall refer to the same epoch.
`C1::is_steady`	`const bool`	`true` if `t1 <= t2` is always `true` and the time between clock ticks is constant, otherwise `false`.
`C1::now()`	`C1::time_point`	Returns a `time_point` object representing the current point in time.

3 [*Note:* The relative difference in durations between those reported by a given clock and the SI definition is a measure of the quality of implementation. — *end note*]

4 A type `TC` meets the `TrivialClock` requirements if:

(4.1) — `TC` satisfies the `Clock` requirements (20.12.3),

(4.2) — the types `TC::rep`, `TC::duration`, and `TC::time_point` satisfy the requirements of `EqualityComparable` (Table 17), `LessThanComparable` (Table 18), `DefaultConstructible` (Table 19), `CopyConstructible` (Table 21), `CopyAssignable` (Table 23), `Destructible` (Table 24), and the requirements of numeric types (26.2). [*Note:* this means, in particular, that operations on these types will not throw exceptions. — *end note*]

(4.3) — lvalues of the types `TC::rep`, `TC::duration`, and `TC::time_point` are swappable (17.6.3.2),

(4.4) — the function `TC::now()` does not throw exceptions, and

(4.5) — the type `TC::time_point::clock` meets the `TrivialClock` requirements, recursively.

20.12.4 Time-related traits [time.traits]

20.12.4.1 `treat_as_floating_point` [time.traits.is_fp]

```
template <class Rep> struct treat_as_floating_point
  : is_floating_point<Rep> { };
```

1 The `duration` template uses the `treat_as_floating_point` trait to help determine if a `duration` object can be converted to another `duration` with a different tick `period`. If `treat_as_floating_point<Rep>::value` is true, then implicit conversions are allowed among `duration`s. Otherwise, the implicit convertibility depends on the tick `period`s of the `duration`s. [*Note:* The intention of this trait is to indicate whether a given class behaves like a floating-point type, and thus allows division of one value by another with acceptable loss of precision. If `treat_as_floating_point<Rep>::value` is `false`, `Rep` will be treated as if it behaved like an integral type for the purpose of these conversions. — *end note*]

20.12.4.2 `duration_values` [time.traits.duration_values]

```
template <class Rep>
struct duration_values {
public:
  static constexpr Rep zero();
  static constexpr Rep min();
  static constexpr Rep max();
};
```

1 The `duration` template uses the `duration_values` trait to construct special values of the durations representation (`Rep`). This is done because the representation might be a class type with behavior which requires some other implementation to return these special values. In that case, the author of that class type should specialize `duration_values` to return the indicated values.

```
static constexpr Rep zero();
```

2 *Returns:* `Rep(0)`. [*Note:* `Rep(0)` is specified instead of `Rep()` because `Rep()` may have some other meaning, such as an uninitialized value. — *end note*]

3 *Remark:* The value returned shall be the additive identity.

```
static constexpr Rep min();
```

4 *Returns:* `numeric_limits<Rep>::lowest()`.

5 *Remark:* The value returned shall compare less than or equal to `zero()`.

```
static constexpr Rep max();
```

6 *Returns:* `numeric_limits<Rep>::max()`.

7 *Remark:* The value returned shall compare greater than `zero()`.

20.12.4.3 Specializations of `common_type` [time.traits.specializations]

```
template <class Rep1, class Period1, class Rep2, class Period2>
struct common_type<chrono::duration<Rep1, Period1>, chrono::duration<Rep2, Period2>> {
  typedef chrono::duration<common_type_t<Rep1, Rep2>, see below> type;
};
```

1 The `period` of the `duration` indicated by this specialization of `common_type` shall be the greatest common divisor of `Period1` and `Period2`. [*Note:* This can be computed by forming a ratio of the greatest common divisor of `Period1::num` and `Period2::num` and the least common multiple of `Period1::den` and `Period2::den`. — *end note*]

2 [*Note:* The `typedef` name `type` is a synonym for the `duration` with the largest tick `period` possible where both `duration` arguments will convert to it without requiring a division operation. The representation of this type is intended to be able to hold any value resulting from this conversion with no truncation error, although floating-point durations may have round-off errors. — *end note*]

```
template <class Clock, class Duration1, class Duration2>
struct common_type<chrono::time_point<Clock, Duration1>, chrono::time_point<Clock, Duration2>> {
  typedef chrono::time_point<Clock, common_type_t<Duration1, Duration2>> type;
};
```

3 The common type of two `time_point` types is a `time_point` with the same clock as the two types and the common type of their two `duration`s.

20.12.5 Class template `duration` [time.duration]

1 A `duration` type measures time between two points in time (`time_point`s). A `duration` has a representation which holds a count of ticks and a tick period. The tick period is the amount of time which occurs from one tick to the next, in units of seconds. It is expressed as a rational constant using the template `ratio`.

```
template <class Rep, class Period = ratio<1>>
class duration {
public:
  typedef Rep    rep;
  typedef Period period;
private:
  rep rep_;  // exposition only
public:
  // 20.12.5.1, construct/copy/destroy:
  constexpr duration() = default;
  template <class Rep2>
      constexpr explicit duration(const Rep2& r);
  template <class Rep2, class Period2>
     constexpr duration(const duration<Rep2, Period2>& d);
  ~duration() = default;
  duration(const duration&) = default;
  duration& operator=(const duration&) = default;

  // 20.12.5.2, observer:
  constexpr rep count() const;

  // 20.12.5.3, arithmetic:
  constexpr duration  operator+() const;
  constexpr duration  operator-() const;
```

```
  duration& operator++();
  duration  operator++(int);
  duration& operator--();
  duration  operator--(int);

  duration& operator+=(const duration& d);
  duration& operator-=(const duration& d);

  duration& operator*=(const rep& rhs);
  duration& operator/=(const rep& rhs);
  duration& operator%=(const rep& rhs);
  duration& operator%=(const duration& rhs);

  // 20.12.5.4, special values:
  static constexpr duration zero();
  static constexpr duration min();
  static constexpr duration max();
};
```

2 *Requires:* `Rep` shall be an arithmetic type or a class emulating an arithmetic type.

3 *Remarks:* If `duration` is instantiated with a `duration` type for the template argument `Rep`, the program is ill-formed.

4 *Remarks:* If `Period` is not a specialization of `ratio`, the program is ill-formed.

5 *Remarks:* If `Period::num` is not positive, the program is ill-formed.

6 *Requires:* Members of `duration` shall not throw exceptions other than those thrown by the indicated operations on their representations.

7 *Remarks:* The defaulted copy constructor of duration shall be a `constexpr` function if and only if the required initialization of the member `rep_` for copy and move, respectively, would satisfy the requirements for a `constexpr` function.

[*Example:*

```
duration<long, ratio<60>> d0;       // holds a count of minutes using a long
duration<long long, milli> d1;      // holds a count of milliseconds using a long long
duration<double, ratio<1, 30>>  d2; // holds a count with a tick period of 1/30 of a second
                                    // (30 Hz) using a double
```

— *end example*]

20.12.5.1 `duration` constructors [time.duration.cons]

```
template <class Rep2>
  constexpr explicit duration(const Rep2& r);
```

1 *Remarks:* This constructor shall not participate in overload resolution unless `Rep2` is implicitly convertible to `rep` and

(1.1) — `treat_as_floating_point<rep>::value` is `true` or

(1.2) — `treat_as_floating_point<Rep2>::value` is `false`.

[*Example:*

```
duration<int, milli> d(3);          // OK
duration<int, milli> d(3.5);        // error
```

— *end example*]

2 *Effects:* Constructs an object of type `duration`.

3 *Postcondition:* `count() == static_cast<rep>(r)`.

```
template <class Rep2, class Period2>
  constexpr duration(const duration<Rep2, Period2>& d);
```

4 *Remarks:* This constructor shall not participate in overload resolution unless no overflow is induced in the conversion and `treat_as_floating_point<rep>::value` is `true` or both `ratio_divide<Period2, period>::den` is `1` and `treat_as_floating_point<Rep2>::value` is `false`. [*Note:* This requirement prevents implicit truncation error when converting between integral-based `duration` types. Such a construction could easily lead to confusion about the value of the `duration`. — *end note*] [*Example:*

```
duration<int, milli> ms(3);
duration<int, micro> us = ms;       // OK
duration<int, milli> ms2 = us;      // error
```

— *end example*]

5 *Effects:* Constructs an object of type `duration`, constructing `rep_` from `duration_cast<duration>(d).count()`.

20.12.5.2 `duration` observer [time.duration.observer]

```
constexpr rep count() const;
```

1 *Returns:* `rep_`.

20.12.5.3 `duration` arithmetic [time.duration.arithmetic]

```
constexpr duration operator+() const;
```

1 *Returns:* `*this`.

```
constexpr duration operator-() const;
```

2 *Returns:* `duration(-rep_);`.

```
duration& operator++();
```

3 *Effects:* `++rep_`.

4 *Returns:* `*this`.

```
duration operator++(int);
```

5 *Returns:* `duration(rep_++);`.

```
duration& operator--();
```

6 *Effects:* `--rep_`.

7 *Returns:* `*this`.

```
duration operator--(int);
```

8 *Returns:* `duration(rep_--);`.

```
duration& operator+=(const duration& d);
```

9 *Effects:* `rep_ += d.count()`.

10 *Returns:* `*this`.

```
duration& operator-=(const duration& d);
```

11 *Effects:* `rep_ -= d.count()`.

12 *Returns:* `*this`.

```
duration& operator*=(const rep& rhs);
```

13 *Effects:* `rep_ *= rhs`.

14 *Returns:* `*this`.

```
duration& operator/=(const rep& rhs);
```

15 *Effects:* `rep_ /= rhs`.

16 *Returns:* `*this`.

```
duration& operator%=(const rep& rhs);
```

17 *Effects:* `rep_ %= rhs`.

18 *Returns:* `*this`.

```
duration& operator%=(const duration& rhs);
```

19 *Effects:* `rep_ %= rhs.count()`.

20 *Returns:* `*this`.

20.12.5.4 `duration` special values [time.duration.special]

```
static constexpr duration zero();
```

1 *Returns:* `duration(duration_values<rep>::zero())`.

```
static constexpr duration min();
```

2 *Returns:* `duration(duration_values<rep>::min())`.

```
static constexpr duration max();
```

3 *Returns:* `duration(duration_values<rep>::max())`.

20.12.5.5 `duration` non-member arithmetic [time.duration.nonmember]

1 In the function descriptions that follow, `CD` represents the return type of the function. `CR(A,B)` represents `common_type_t<A, B>`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  operator+(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

2 *Returns:* CD(CD(lhs).count() + CD(rhs).count()).

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  operator-(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

3 *Returns:* CD(CD(lhs).count() - CD(rhs).count()).

```
template <class Rep1, class Period, class Rep2>
  constexpr duration<common_type_t<Rep1, Rep2>, Period>
  operator*(const duration<Rep1, Period>& d, const Rep2& s);
```

4 *Remarks:* This operator shall not participate in overload resolution unless `Rep2` is implicitly convertible to `CR(Rep1, Rep2)`.

5 *Returns:* CD(CD(d).count() * s).

```
template <class Rep1, class Rep2, class Period>
  constexpr duration<common_type_t<Rep1, Rep2>, Period>
  operator*(const Rep1& s, const duration<Rep2, Period>& d);
```

6 *Remarks:* This operator shall not participate in overload resolution unless `Rep1` is implicitly convertible to `CR(Rep1, Rep2)`.

7 *Returns:* `d * s`.

```
template <class Rep1, class Period, class Rep2>
  constexpr duration<common_type_t<Rep1, Rep2>, Period>
  operator/(const duration<Rep1, Period>& d, const Rep2& s);
```

8 *Remarks:* This operator shall not participate in overload resolution unless `Rep2` is implicitly convertible to `CR(Rep1, Rep2)` and `Rep2` is not an instantiation of `duration`.

9 *Returns:* CD(CD(d).count() / s).

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr common_type_t<Rep1, Rep2>
  operator/(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

10 *Returns:* `CD(lhs).count() / CD(rhs).count()`.

```
template <class Rep1, class Period, class Rep2>
  constexpr duration<common_type_t<Rep1, Rep2>, Period>
  operator%(const duration<Rep1, Period>& d, const Rep2& s);
```

11 *Remarks:* This operator shall not participate in overload resolution unless `Rep2` is implicitly convertible to `CR(Rep1, Rep2)` and `Rep2` is not an instantiation of `duration`.

12 *Returns:* CD(CD(d).count() % s).

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr common_type_t<duration<Rep1, Period1>, duration<Rep2, Period2>>
  operator%(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

13 *Returns:* CD(CD(lhs).count() % CD(rhs).count()).

20.12.5.6 `duration` comparisons [time.duration.comparisons]

1 In the function descriptions that follow, `CT` represents `common_type_t<A, B>`, where `A` and `B` are the types of the two arguments to the function.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator==(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

2 *Returns:* `CT(lhs).count() == CT(rhs).count()`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator!=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

3 *Returns:* `!(lhs == rhs)`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator<(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

4 *Returns:* `CT(lhs).count() < CT(rhs).count()`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator<=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

5 *Returns:* `!(rhs < lhs)`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator>(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

6 *Returns:* `rhs < lhs`.

```
template <class Rep1, class Period1, class Rep2, class Period2>
  constexpr bool operator>=(const duration<Rep1, Period1>& lhs, const duration<Rep2, Period2>& rhs);
```

7 *Returns:* `!(lhs < rhs)`.

20.12.5.7 `duration_cast` [time.duration.cast]

```
template <class ToDuration, class Rep, class Period>
  constexpr ToDuration duration_cast(const duration<Rep, Period>& d);
```

1 *Remarks:* This function shall not participate in overload resolution unless `ToDuration` is an instantiation of `duration`.

2 *Returns:* Let `CF` be `ratio_divide<Period, typename ToDuration::period>`, and `CR` be `common_type< typename ToDuration::rep, Rep, intmax_t>::type`.

(2.1) — If `CF::num == 1` and `CF::den == 1`, returns

```
ToDuration(static_cast<typename ToDuration::rep>(d.count()))
```

(2.2) — otherwise, if `CF::num != 1` and `CF::den == 1`, returns

```
ToDuration(static_cast<typename ToDuration::rep>(
  static_cast<CR>(d.count()) * static_cast<CR>(CF::num)))
```

(2.3) — otherwise, if `CF::num == 1` and `CF::den != 1`, returns

```
ToDuration(static_cast<typename ToDuration::rep>(
  static_cast<CR>(d.count()) / static_cast<CR>(CF::den)))
```

(2.4) — otherwise, returns

```
ToDuration(static_cast<typename ToDuration::rep>(
  static_cast<CR>(d.count()) * static_cast<CR>(CF::num) / static_cast<CR>(CF::den)))
```

Notes: This function does not use any implicit conversions; all conversions are done with `static_cast`. It avoids multiplications and divisions when it is known at compile time that one or more arguments is 1. Intermediate computations are carried out in the widest representation and only converted to the destination representation at the final step.

20.12.5.8 Suffixes for duration literals [time.duration.literals]

1 This section describes literal suffixes for constructing duration literals. The suffixes `h`, `min`, `s`, `ms`, `us`, `ns` denote duration values of the corresponding types `hours`, `minutes`, `seconds`, `milliseconds`, `microseconds`, and `nanoseconds` respectively if they are applied to integral literals.

2 If any of these suffixes are applied to a floating point literal the result is a `chrono::duration` literal with an unspecified floating point representation.

3 If any of these suffixes are applied to an integer literal and the resulting `chrono::duration` value cannot be represented in the result type because of overflow, the program is ill-formed.

4 [*Example:* The following code shows some duration literals.

```
using namespace std::chrono_literals;
auto constexpr aday=24h;
auto constexpr lesson=45min;
auto constexpr halfanhour=0.5h;
```

— *end example*]

```
constexpr chrono::hours                                   operator "" h(unsigned long long hours);
constexpr chrono::duration<unspecified , ratio<3600,1>> operator "" h(long double hours);
```

5 *Returns:* A `duration` literal representing `hours` hours.

```
constexpr chrono::minutes                                operator "" min(unsigned long long minutes);
constexpr chrono::duration<unspecified , ratio<60,1>> operator "" min(long double minutes);
```

6 *Returns:* A `duration` literal representing `minutes` minutes.

```
constexpr chrono::seconds                operator "" s(unsigned long long sec);
constexpr chrono::duration<unspecified > operator "" s(long double sec);
```

7 *Returns:* A `duration` literal representing `sec` seconds.

8 [*Note:* The same suffix `s` is used for `basic_string` but there is no conflict, since duration suffixes apply to numbers and string literal suffixes apply to character array literals. — *end note*]

```
constexpr chrono::milliseconds                  operator "" ms(unsigned long long msec);
constexpr chrono::duration<unspecified , milli> operator "" ms(long double msec);
```

9 *Returns:* A `duration` literal representing `msec` milliseconds.

```
constexpr chrono::microseconds                  operator "" us(unsigned long long usec);
constexpr chrono::duration<unspecified , micro> operator "" us(long double usec);
```

10 *Returns:* A `duration` literal representing `usec` microseconds.

```
constexpr chrono::nanoseconds                 operator "" ns(unsigned long long nsec);
constexpr chrono::duration<unspecified , nano> operator "" ns(long double nsec);
```

11 *Returns:* A `duration` literal representing `nsec` nanoseconds.

20.12.6 Class template `time_point` [time.point]

```
template <class Clock, class Duration = typename Clock::duration>
class time_point {
public:
  typedef Clock                    clock;
  typedef Duration                 duration;
  typedef typename duration::rep   rep;
```

```
    typedef typename duration::period period;
  private:
    duration d_;  // exposition only

  public:
    // 20.12.6.1, construct:
    constexpr time_point();  // has value epoch
    constexpr explicit time_point(const duration& d);  // same as time_point() + d
    template <class Duration2>
      constexpr time_point(const time_point<clock, Duration2>& t);

    // 20.12.6.2, observer:
    constexpr duration time_since_epoch() const;

    // 20.12.6.3, arithmetic:
    time_point& operator+=(const duration& d);
    time_point& operator-=(const duration& d);

    // 20.12.6.4, special values:
    static constexpr time_point min();
    static constexpr time_point max();
  };
```

1 `Clock` shall meet the Clock requirements (20.12.7).

2 If `Duration` is not an instance of `duration`, the program is ill-formed.

20.12.6.1 `time_point` constructors [time.point.cons]

```
constexpr time_point();
```

1 *Effects:* Constructs an object of type `time_point`, initializing `d_` with `duration::zero()`. Such a `time_point` object represents the epoch.

```
constexpr explicit time_point(const duration& d);
```

2 *Effects:* Constructs an object of type `time_point`, initializing `d_` with `d`. Such a `time_point` object represents the epoch `+ d`.

```
template <class Duration2>
  constexpr time_point(const time_point<clock, Duration2>& t);
```

3 *Remarks:* This constructor shall not participate in overload resolution unless `Duration2` is implicitly convertible to `duration`.

4 *Effects:* Constructs an object of type `time_point`, initializing `d_` with `t.time_since_epoch()`.

20.12.6.2 `time_point` observer [time.point.observer]

```
constexpr duration time_since_epoch() const;
```

1 *Returns:* `d_`.

20.12.6.3 `time_point` arithmetic [time.point.arithmetic]

```
time_point& operator+=(const duration& d);
```

1 *Effects:* `d_ += d`.

2 *Returns:* `*this`.

```
time_point& operator-=(const duration& d);
```

3 *Effects:* `d_ -= d`.

4 *Returns:* `*this`.

20.12.6.4 `time_point` special values [time.point.special]

```
static constexpr time_point min();
```

1 *Returns:* `time_point(duration::min())`.

```
static constexpr time_point max();
```

2 *Returns:* `time_point(duration::max())`.

20.12.6.5 `time_point` non-member arithmetic [time.point.nonmember]

```
template <class Clock, class Duration1, class Rep2, class Period2>
  constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
  operator+(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

1 *Returns:* `CT(lhs.time_since_epoch() + rhs)`, where `CT` is the type of the return value.

```
template <class Rep1, class Period1, class Clock, class Duration2>
  constexpr time_point<Clock, common_type_t<duration<Rep1, Period1>, Duration2>>
  operator+(const duration<Rep1, Period1>& lhs, const time_point<Clock, Duration2>& rhs);
```

2 *Returns:* `rhs + lhs`.

```
template <class Clock, class Duration1, class Rep2, class Period2>
  constexpr time_point<Clock, common_type_t<Duration1, duration<Rep2, Period2>>>
  operator-(const time_point<Clock, Duration1>& lhs, const duration<Rep2, Period2>& rhs);
```

3 *Returns:* `lhs + (-rhs)`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr common_type_t<Duration1, Duration2>
  operator-(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

4 *Returns:* `lhs.time_since_epoch() - rhs.time_since_epoch()`.

20.12.6.6 `time_point` comparisons [time.point.comparisons]

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator==(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

1 *Returns:* `lhs.time_since_epoch() == rhs.time_since_epoch()`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator!=(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

2 *Returns:* `!(lhs == rhs)`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator<(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

3 *Returns:* `lhs.time_since_epoch() < rhs.time_since_epoch()`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator<=(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

4 *Returns:* `!(rhs < lhs)`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator>(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

5 *Returns:* `rhs < lhs`.

```
template <class Clock, class Duration1, class Duration2>
  constexpr bool operator>=(const time_point<Clock, Duration1>& lhs, const time_point<Clock, Duration2>& rhs);
```

6 *Returns:* `!(lhs < rhs)`.

20.12.6.7 `time_point_cast` [time.point.cast]

```
template <class ToDuration, class Clock, class Duration>
  constexpr time_point<Clock, ToDuration>
  time_point_cast(const time_point<Clock, Duration>& t);
```

1 *Remarks:* This function shall not participate in overload resolution unless `ToDuration` is an instantiation of `duration`.

2 *Returns:* `time_point<Clock, ToDuration>(duration_cast<ToDuration>(t.time_since_epoch()))`.

20.12.7 Clocks [time.clock]

1 The types defined in this subclause shall satisfy the `TrivialClock` requirements (20.12.3).

20.12.7.1 Class `system_clock` [time.clock.system]

1 Objects of class `system_clock` represent wall clock time from the system-wide realtime clock.

```
class system_clock {
public:
  typedef see below                          rep;
  typedef ratio<unspecified , unspecified >  period;
  typedef chrono::duration<rep, period>      duration;
  typedef chrono::time_point<system_clock>   time_point;
  static constexpr bool is_steady =          unspecified;

  static time_point now() noexcept;

  // Map to C API
  static time_t      to_time_t  (const time_point& t) noexcept;
  static time_point  from_time_t(time_t t) noexcept;
};
```

```
typedef unspecified  system_clock::rep;
```

2 *Requires:* `system_clock::duration::min() < system_clock::duration::zero()` shall be `true`. [*Note:* This implies that `rep` is a signed type. — *end note*]

```
static time_t to_time_t(const time_point& t) noexcept;
```

3 *Returns:* A `time_t` object that represents the same point in time as `t` when both values are restricted to the coarser of the precisions of `time_t` and `time_point`. It is implementation defined whether values are rounded or truncated to the required precision.

```
static time_point from_time_t(time_t t) noexcept;
```

4 *Returns:* A `time_point` object that represents the same point in time as `t` when both values are restricted to the coarser of the precisions of `time_t` and `time_point`. It is implementation defined whether values are rounded or truncated to the required precision.

20.12.7.2 Class `steady_clock` [time.clock.steady]

1 Objects of class `steady_clock` represent clocks for which values of `time_point` never decrease as physical time advances and for which values of `time_point` advance at a steady rate relative to real time. That is, the clock may not be adjusted.

```
class steady_clock {
public:
  typedef unspecified                                rep;
  typedef ratio<unspecified , unspecified >          period;
  typedef chrono::duration<rep, period>              duration;
  typedef chrono::time_point<unspecified, duration>  time_point;
  static constexpr bool is_steady =               true;

  static time_point now() noexcept;
};
```

20.12.7.3 Class `high_resolution_clock` [time.clock.hires]

1 Objects of class `high_resolution_clock` represent clocks with the shortest tick period. `high_resolution_clock` may be a synonym for `system_clock` or `steady_clock`.

```
class high_resolution_clock {
public:
  typedef unspecified                                rep;
  typedef ratio<unspecified , unspecified >          period;
  typedef chrono::duration<rep, period>              duration;
  typedef chrono::time_point<unspecified , duration> time_point;
  static constexpr bool is_steady =               unspecified ;

  static time_point now() noexcept;
};
```

20.12.8 Date and time functions [date.time]

1 Table 60 describes the header `<ctime>`.

Table 60 — Header `<ctime>` synopsis

Type	Name(s)			
Macros:	`NULL`	`CLOCKS_PER_SEC`		
Types:	`size_t`	`clock_t`	`time_t`	
Struct:	`tm`			
Functions:				
`asctime`	`clock`	`difftime`	`localtime`	`strftime`
`ctime`	`gmtime`	`mktime`	`time`	

2 The contents are the same as the Standard C library header `<time.h>`.[232] The functions `asctime`, `ctime`, `gmtime`, and `localtime` are not required to avoid data races (17.6.5.9).

See also: ISO C Clause 7.12, Amendment 1 Clause 4.6.4.

20.13 Class template `scoped_allocator_adaptor` [allocator.adaptor]

20.13.1 Header `<scoped_allocator>` synopsis [allocator.adaptor.syn]

232) `strftime` supports the C conversion specifiers `C`, `D`, `e`, `F`, `g`, `G`, `h`, `r`, `R`, `t`, `T`, `u`, `V`, and `z`, and the modifiers `E` and `O`.

```
// scoped allocator adaptor
template <class OuterAlloc, class... InnerAlloc>
  class scoped_allocator_adaptor;
template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

1 The class template `scoped_allocator_adaptor` is an allocator template that specifies the memory resource (the outer allocator) to be used by a container (as any other allocator does) and also specifies an inner allocator resource to be passed to the constructor of every element within the container. This adaptor is instantiated with one outer and zero or more inner allocator types. If instantiated with only one allocator type, the inner allocator becomes the `scoped_allocator_adaptor` itself, thus using the same allocator resource for the container and every element within the container and, if the elements themselves are containers, each of their elements recursively. If instantiated with more than one allocator, the first allocator is the outer allocator for use by the container, the second allocator is passed to the constructors of the container's elements, and, if the elements themselves are containers, the third allocator is passed to the elements' elements, and so on. If containers are nested to a depth greater than the number of allocators, the last allocator is used repeatedly, as in the single-allocator case, for any remaining recursions. [*Note:* The `scoped_allocator_adaptor` is derived from the outer allocator type so it can be substituted for the outer allocator type in most expressions. — *end note*]

```
namespace std {
  template <class OuterAlloc, class... InnerAllocs>
    class scoped_allocator_adaptor : public OuterAlloc {
  private:
    typedef allocator_traits<OuterAlloc> OuterTraits; // exposition only
    scoped_allocator_adaptor<InnerAllocs...> inner;   // exposition only
  public:
    typedef OuterAlloc outer_allocator_type;
    typedef see below inner_allocator_type;

    typedef typename OuterTraits::value_type value_type;
    typedef typename OuterTraits::size_type size_type;
    typedef typename OuterTraits::difference_type difference_type;
    typedef typename OuterTraits::pointer pointer;
    typedef typename OuterTraits::const_pointer const_pointer;
    typedef typename OuterTraits::void_pointer void_pointer;
    typedef typename OuterTraits::const_void_pointer const_void_pointer;

    typedef see below propagate_on_container_copy_assignment;
    typedef see below propagate_on_container_move_assignment;
    typedef see below propagate_on_container_swap;
    typedef see below is_always_equal;

    template <class Tp>
      struct rebind {
        typedef scoped_allocator_adaptor<
          OuterTraits::template rebind_alloc<Tp>, InnerAllocs...> other;
      };

    scoped_allocator_adaptor();
    template <class OuterA2>
```

```
    scoped_allocator_adaptor(OuterA2&& outerAlloc,
                             const InnerAllocs&... innerAllocs) noexcept;

  scoped_allocator_adaptor(const scoped_allocator_adaptor& other) noexcept;
  scoped_allocator_adaptor(scoped_allocator_adaptor&& other) noexcept;

  template <class OuterA2>
    scoped_allocator_adaptor(
      const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& other) noexcept;
  template <class OuterA2>
    scoped_allocator_adaptor(
      scoped_allocator_adaptor<OuterA2, InnerAllocs...>&& other) noexcept;

  ~scoped_allocator_adaptor();

  inner_allocator_type& inner_allocator() noexcept;
  const inner_allocator_type& inner_allocator() const noexcept;
  outer_allocator_type& outer_allocator() noexcept;
  const outer_allocator_type& outer_allocator() const noexcept;

  pointer allocate(size_type n);
  pointer allocate(size_type n, const_void_pointer hint);
  void deallocate(pointer p, size_type n);
  size_type max_size() const;

  template <class T, class... Args>
    void construct(T* p, Args&&... args);
  template <class T1, class T2, class... Args1, class... Args2>
    void construct(pair<T1, T2>* p, piecewise_construct_t,
                   tuple<Args1...> x, tuple<Args2...> y);
  template <class T1, class T2>
    void construct(pair<T1, T2>* p);
  template <class T1, class T2, class U, class V>
    void construct(pair<T1, T2>* p, U&& x, V&& y);
  template <class T1, class T2, class U, class V>
    void construct(pair<T1, T2>* p, const pair<U, V>& x);
  template <class T1, class T2, class U, class V>
    void construct(pair<T1, T2>* p, pair<U, V>&& x);

  template <class T>
    void destroy(T* p);

  scoped_allocator_adaptor select_on_container_copy_construction() const;
};

template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
}
```

20.13.2 Scoped allocator adaptor member types [allocator.adaptor.types]

`typedef` *see below* `inner_allocator_type;`

1 *Type:* `scoped_allocator_adaptor<OuterAlloc>` if `sizeof...(InnerAllocs)` is zero; otherwise, `scoped_allocator_adaptor<InnerAllocs...>`.

`typedef` *see below* `propagate_on_container_copy_assignment;`

2 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_copy_assignment::value` is `true` for any `A` in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

`typedef` *see below* `propagate_on_container_move_assignment;`

3 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_move_assignment::value` is `true` for any `A` in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

`typedef` *see below* `propagate_on_container_swap;`

4 *Type:* `true_type` if `allocator_traits<A>::propagate_on_container_swap::value` is `true` for any `A` in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

`typedef` *see below* `is_always_equal;`

5 *Type:* `true_type` if `allocator_traits<A>::is_always_equal::value` is `true` for every `A` in the set of `OuterAlloc` and `InnerAllocs...`; otherwise, `false_type`.

20.13.3 Scoped allocator adaptor constructors [allocator.adaptor.cnstr]

```
scoped_allocator_adaptor();
```

1 *Effects:* value-initializes the `OuterAlloc` base class and the `inner` allocator object.

```
template <class OuterA2>
  scoped_allocator_adaptor(OuterA2&& outerAlloc,
                           const InnerAllocs&... innerAllocs) noexcept;
```

2 *Requires:* `OuterAlloc` shall be constructible from `OuterA2`.

3 *Effects:* initializes the `OuterAlloc` base class with `std::forward<OuterA2>(outerAlloc)` and `inner` with `innerAllocs...` (hence recursively initializing each allocator within the adaptor with the corresponding allocator from the argument list).

```
scoped_allocator_adaptor(const scoped_allocator_adaptor& other) noexcept;
```

4 *Effects:* initializes each allocator within the adaptor with the corresponding allocator from `other`.

```
scoped_allocator_adaptor(scoped_allocator_adaptor&& other) noexcept;
```

5 *Effects:* move constructs each allocator within the adaptor with the corresponding allocator from `other`.

```
template <class OuterA2>
  scoped_allocator_adaptor(const scoped_allocator_adaptor<OuterA2,
                                                          InnerAllocs...>& other) noexcept;
```

6 *Requires:* `OuterAlloc` shall be constructible from `OuterA2`.

7 *Effects:* initializes each allocator within the adaptor with the corresponding allocator from `other`.

```
template <class OuterA2>
  scoped_allocator_adaptor(scoped_allocator_adaptor<OuterA2,
                                                    InnerAllocs...>&& other) noexcept;
```

8 *Requires:* `OuterAlloc` shall be constructible from `OuterA2`.

9 *Effects:* initializes each allocator within the adaptor with the corresponding allocator rvalue from `other`.

20.13.4 Scoped allocator adaptor members [allocator.adaptor.members]

1 In the `construct` member functions, *OUTERMOST(x)* is `x` if `x` does not have an `outer_allocator()` member function and *OUTERMOST(x.outer_allocator())* otherwise; *OUTERMOST_ALLOC_TRAITS(x)* is `allocator_traits<decltype(`*OUTERMOST*`(x))>`. [*Note:* *OUTERMOST*(x) and *OUTERMOST_ALLOC_TRAITS*(x) are recursive operations. It is incumbent upon the definition of `outer_allocator()` to ensure that the recursion terminates. It will terminate for all instantiations of `scoped_allocator_adaptor`. — *end note*]

```
inner_allocator_type& inner_allocator() noexcept;
const inner_allocator_type& inner_allocator() const noexcept;
```

2 *Returns:* `*this` if `sizeof...(InnerAllocs)` is zero; otherwise, `inner`.

```
outer_allocator_type& outer_allocator() noexcept;
```

3 *Returns:* `static_cast<OuterAlloc&>(*this)`.

```
const outer_allocator_type& outer_allocator() const noexcept;
```

4 *Returns:* `static_cast<const OuterAlloc&>(*this)`.

```
pointer allocate(size_type n);
```

5 *Returns:* `allocator_traits<OuterAlloc>::allocate(outer_allocator(), n)`.

```
pointer allocate(size_type n, const_void_pointer hint);
```

6 *Returns:* `allocator_traits<OuterAlloc>::allocate(outer_allocator(), n, hint)`.

```
void deallocate(pointer p, size_type n) noexcept;
```

7 *Effects:* `allocator_traits<OuterAlloc>::deallocate(outer_allocator(), p, n);`

```
size_type max_size() const;
```

8 *Returns:* `allocator_traits<OuterAlloc>::max_size(outer_allocator())`.

```
template <class T, class... Args>
  void construct(T* p, Args&&... args);
```

9 *Effects:*

(9.1) — If `uses_allocator<T, inner_allocator_type>::value` is `false` and `is_constructible<T, Args...>::value` is `true`, calls *OUTERMOST_ALLOC_TRAITS*(`*this`)`::construct(`*OUTERMOST*`(*this), p, std::forward<Args>(args)...)`.

(9.2) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is `true` and `is_constructible<T, allocator_arg_t, inner_allocator_type, Args...>::value` is `true`, calls *OUTERMOST_ALLOC_TRAITS*(`*this`)`::construct(`*OUTERMOST*`(*this), p, allocator_arg, inner_allocator(), std::forward<Args>(args)...)`.

(9.3) — Otherwise, if `uses_allocator<T, inner_allocator_type>::value` is `true` and `is_constructible<T, Args..., inner_allocator_type>::value` is `true`, calls *OUTERMOST_ALLOC_TRAITS*(*this):: `construct(`*OUTERMOST*`(*this), p, std::forward<Args>(args)..., inner_allocator())`.

(9.4) — Otherwise, the program is ill-formed. [*Note:* An error will result if `uses_allocator` evaluates to true but the specific constructor does not take an allocator. This definition prevents a silent failure to pass an inner allocator to a contained element. — *end note*]

```
template <class T1, class T2, class... Args1, class... Args2>
  void construct(pair<T1, T2>* p, piecewise_construct_t,
                 tuple<Args1...> x, tuple<Args2...> y);
```

10 *Requires:* all of the types in `Args1` and `Args2` shall be `CopyConstructible` (Table 21).

11 *Effects:* Constructs a `tuple` object `xprime` from `x` by the following rules:

(11.1) — If `uses_allocator<T1, inner_allocator_type>::value` is `false` and `is_constructible<T1, Args1...>::value` is `true`, then `xprime` is `x`.

(11.2) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is `true` and `is_constructible<T1, allocator_arg_t, inner_allocator_type, Args1...>::value` is `true`, then `xprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>( allocator_arg, inner_allocator()), std::move(x))`.

(11.3) — Otherwise, if `uses_allocator<T1, inner_allocator_type>::value` is `true` and `is_constructible<T1, Args1..., inner_allocator_type>::value` is `true`, then `xprime` is `tuple_cat(std::move(x), tuple<inner_allocator_type&>(inner_allocator()))`.

(11.4) — Otherwise, the program is ill-formed.

and constructs a `tuple` object `yprime` from `y` by the following rules:

(11.5) — If `uses_allocator<T2, inner_allocator_type>::value` is `false` and `is_constructible<T2, Args2...>::value` is `true`, then `yprime` is `y`.

(11.6) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is `true` and `is_constructible<T2, allocator_arg_t, inner_allocator_type, Args2...>::value` is `true`, then `yprime` is `tuple_cat(tuple<allocator_arg_t, inner_allocator_type&>( allocator_arg, inner_allocator()), std::move(y))`.

(11.7) — Otherwise, if `uses_allocator<T2, inner_allocator_type>::value` is `true` and `is_constructible<T2, Args2..., inner_allocator_type>::value` is `true`, then `yprime` is `tuple_cat(std::move(y), tuple<inner_allocator_type&>(inner_allocator()))`.

(11.8) — Otherwise, the program is ill-formed.

then calls *`OUTERMOST_ALLOC_TRAITS`*`(*this)::construct(`*`OUTERMOST`*`(*this), p, piecewise_construct, std::move(xprime), std::move(yprime))`.

```
template <class T1, class T2>
  void construct(pair<T1, T2>* p);
```

12 *Effects:* Equivalent to `this->construct(p, piecewise_construct, tuple<>(), tuple<>())`.

```
template <class T1, class T2, class U, class V>
  void construct(pair<T1, T2>* p, U&& x, V&& y);
```

13 *Effects:* Equivalent to `this->construct(p, piecewise_construct, forward_as_tuple(std::forward<U>(x)), forward_as_tuple(std::forward<V>(y)))`.

```
template <class T1, class T2, class U, class V>
  void construct(pair<T1, T2>* p, const pair<U, V>& x);
```

14 *Effects:* Equivalent to `this->construct(p, piecewise_construct, forward_as_tuple(x.first), forward_as_tuple(x.second))`.

```
template <class T1, class T2, class U, class V>
  void construct(pair<T1, T2>* p, pair<U, V>&& x);
```

15 *Effects:* Equivalent to `this->construct(p, piecewise_construct, forward_as_tuple(std::forward<U>(x.first)), forward_as_tuple(std::forward<V>(x.second)))`.

```
template <class T>
  void destroy(T* p);
```

16 *Effects:* calls *OUTERMOST_ALLOC_TRAITS*(*this)::destroy(*OUTERMOST*(*this), p).

```
scoped_allocator_adaptor select_on_container_copy_construction() const;
```

17 *Returns:* A new scoped_allocator_adaptor object where each allocator A in the adaptor is initialized from the result of calling allocator_traits<A>::select_on_container_copy_construction() on the corresponding allocator in *this.

20.13.5 Scoped allocator operators [scoped.adaptor.operators]

```
template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator==(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

1 *Returns:* a.outer_allocator() == b.outer_allocator() if sizeof...(InnerAllocs) is zero; otherwise, a.outer_allocator() == b.outer_allocator() && a.inner_allocator() == b.inner_allocator().

```
template <class OuterA1, class OuterA2, class... InnerAllocs>
  bool operator!=(const scoped_allocator_adaptor<OuterA1, InnerAllocs...>& a,
                  const scoped_allocator_adaptor<OuterA2, InnerAllocs...>& b) noexcept;
```

2 *Returns:* !(a == b).

20.14 Class type_index [type.index]

20.14.1 Header <typeindex> synopsis [type.index.synopsis]

```
namespace std {
  class type_index;
  template <class T> struct hash;
  template<> struct hash<type_index>;
}
```

20.14.2 type_index overview [type.index.overview]

```
namespace std {
  class type_index {
  public:
    type_index(const type_info& rhs) noexcept;
    bool operator==(const type_index& rhs) const noexcept;
    bool operator!=(const type_index& rhs) const noexcept;
    bool operator< (const type_index& rhs) const noexcept;
    bool operator<= (const type_index& rhs) const noexcept;
    bool operator> (const type_index& rhs) const noexcept;
    bool operator>= (const type_index& rhs) const noexcept;
    size_t hash_code() const noexcept;
    const char* name() const noexcept;
  private:
    const type_info* target;     // exposition only
    // Note that the use of a pointer here, rather than a reference,
    // means that the default copy/move constructor and assignment
    // operators will be provided and work as expected.
  };
}
```

1 The class `type_index` provides a simple wrapper for `type_info` which can be used as an index type in associative containers (23.4) and in unordered associative containers (23.5).

20.14.3 `type_index` members [type.index.members]

```
type_index(const type_info& rhs) noexcept;
```

1 *Effects:* constructs a `type_index` object, the equivalent of `target = &rhs`.

```
bool operator==(const type_index& rhs) const noexcept;
```

2 *Returns:* `*target == *rhs.target`

```
bool operator!=(const type_index& rhs) const noexcept;
```

3 *Returns:* `*target != *rhs.target`

```
bool operator<(const type_index& rhs) const noexcept;
```

4 *Returns:* `target->before(*rhs.target)`

```
bool operator<=(const type_index& rhs) const noexcept;
```

5 *Returns:* `!rhs.target->before(*target)`

```
bool operator>(const type_index& rhs) const noexcept;
```

6 *Returns:* `rhs.target->before(*target)`

```
bool operator>=(const type_index& rhs) const noexcept;
```

7 *Returns:* `!target->before(*rhs.target)`

```
size_t hash_code() const noexcept;
```

8 *Returns:* `target->hash_code()`

```
const char* name() const noexcept;
```

9 *Returns:* `target->name()`

20.14.4 Hash support [type.index.hash]

```
template <> struct hash<type_index>;
```

1 The template specialization shall meet the requirements of class template `hash` (20.9.13). For an object `index` of type `type_index`, `hash<type_index>()(index)` shall evaluate to the same result as `index.hash_code()`.

21 Strings library [strings]

21.1 General [strings.general]

1 This Clause describes components for manipulating sequences of any non-array POD (3.9) type. In this Clause such types are called *char-like types*, and objects of char-like types are called *char-like objects* or simply *characters*.

2 The following subclauses describe a character traits class, a string class, and null-terminated sequence utilities, as summarized in Table 61.

Table 61 — Strings library summary

Subclause		Header(s)
21.2	Character traits	`<string>`
21.3	String classes	`<string>`

21.8	Null-terminated sequence utilities	`<cctype>`
		`<cwctype>`
		`<cstring>`
		`<cwchar>`
		`<cstdlib>`
		`<cuchar>`

21.2 Character traits [char.traits]

1 This subclause defines requirements on classes representing *character traits*, and defines a class template `char_traits<charT>`, along with four specializations, `char_traits<char>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, that satisfy those requirements.

2 Most classes specified in Clauses 21.3 and 27 need a set of related types and functions to complete the definition of their semantics. These types and functions are provided as a set of member typedefs and functions in the template parameter 'traits' used by each such template. This subclause defines the semantics of these members.

3 To specialize those templates to generate a string or iostream class to handle a particular character container type `CharT`, that and its related character traits class `Traits` are passed as a pair of parameters to the string or iostream template as parameters `charT` and `traits`. `Traits::char_type` shall be the same as `CharT`.

4 This subclause specifies a class template, `char_traits<charT>`, and four explicit specializations of it, `char_traits<char>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`, all of which appear in the header `<string>` and satisfy the requirements below.

21.2.1 Character traits requirements [char.traits.require]

1 In Table 62, `X` denotes a Traits class defining types and functions for the character container type `CharT`; `c` and `d` denote values of type `CharT`; `p` and `q` denote values of type `const CharT*`; `s` denotes a value of type `CharT*`; `n`, `i` and `j` denote values of type `std::size_t`; `e` and `f` denote values of type `X::int_type`; `pos` denotes a value of type `X::pos_type`; `state` denotes a value of type `X::state_type`; and `r` denotes an lvalue of type `CharT`. Operations on Traits shall not throw exceptions.

Table 62 — Character traits requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X::char_type`	`charT`	(described in 21.2.2)	compile-time
`X::int_type`		(described in 21.2.2)	compile-time
`X::off_type`		(described in 21.2.2)	compile-time
`X::pos_type`		(described in 21.2.2)	compile-time
`X::state_type`		(described in 21.2.2)	compile-time
`X::eq(c,d)`	`bool`	yields: whether `c` is to be treated as equal to `d`.	constant
`X::lt(c,d)`	`bool`	yields: whether `c` is to be treated as less than `d`.	constant
`X::compare(p,q,n)`	`int`	yields: `0` if for each `i` in `[0,n)`, `X::eq(p[i],q[i])` is true; else, a negative value if, for some `j` in `[0,n)`, `X::lt(p[j],q[j])` is true and for each `i` in `[0,j)` `X::eq(p[i],q[i])` is true; else a positive value.	linear
`X::length(p)`	`std::size_t`	yields: the smallest `i` such that `X::eq(p[i],charT())` is true.	linear
`X::find(p,n,c)`	`const X::char_type*`	yields: the smallest `q` in `[p,p+n)` such that `X::eq(*q,c)` is true, zero otherwise.	linear
`X::move(s,p,n)`	`X::char_type*`	for each `i` in `[0,n)`, performs `X::assign(s[i],p[i])`. Copies correctly even where the ranges `[p,p+n)` and `[s,s+n)` overlap. yields: `s`.	linear
`X::copy(s,p,n)`	`X::char_type*`	pre: `p` not in `[s,s+n)`. yields: `s`. for each `i` in `[0,n)`, performs `X::assign(s[i],p[i])`.	linear
`X::assign(r,d)`	(not used)	assigns `r=d`.	constant
`X::assign(s,n,c)`	`X::char_type*`	for each `i` in `[0,n)`, performs `X::assign(s[i],c)`. yields: `s`.	linear
`X::not_eof(e)`	`int_type`	yields: `e` if `X::eq_int_type(e,X::eof())` is false, otherwise a value `f` such that `X::eq_int_type(f,X::eof())` is false.	constant
`X::to_char_type(e)`	`X::char_type`	yields: if for some `c`, `X::eq_int_type(e,X::to_int_type(c))` is true, `c`; else some unspecified value.	constant

Table 62 — Character traits requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X::to_int_type(c)`	`X::int_type`	yields: some value `e`, constrained by the definitions of `to_char_type` and `eq_int_type`.	constant
`X::eq_int_type(e,f)`	`bool`	yields: for all `c` and `d`, `X::eq(c,d)` is equal to `X::eq_int_type(X::to_int_type(c), X::to_int_type(d))`; otherwise, yields true if `e` and `f` are both copies of `X::eof()`; otherwise, yields false if one of `e` and `f` is a copy of `X::eof()` and the other is not; otherwise the value is unspecified.	constant
`X::eof()`	`X::int_type`	yields: a value `e` such that `X::eq_int_type(e,X::to_int_type(c))` is false for all values `c`.	constant

2 The class template

```
template<class charT> struct char_traits;
```

shall be provided in the header `<string>` as a basis for explicit specializations.

21.2.2 traits typedefs [char.traits.typedefs]

```
typedef CHAR_T char_type;
```

1 The type `char_type` is used to refer to the character container type in the implementation of the library classes defined in 21.3 and Clause 27.

```
typedef INT_T int_type;
```

2 *Requires:* For a certain character container type `char_type`, a related container type `INT_T` shall be a type or class which can represent all of the valid characters converted from the corresponding `char_type` values, as well as an end-of-file value, `eof()`. The type `int_type` represents a character container type which can hold end-of-file to be used as a return type of the iostream class member functions.[233]

```
typedef implementation-defined off_type;
typedef implementation-defined pos_type;
```

3 *Requires:* Requirements for `off_type` and `pos_type` are described in 27.2.2 and 27.3.

```
typedef STATE_T state_type;
```

4 *Requires:* `state_type` shall meet the requirements of `CopyAssignable` (Table 23), `CopyConstructible` (Table 21), and `DefaultConstructible` (Table 19) types.

233) If `eof()` can be held in `char_type` then some iostreams operations may give surprising results.

21.2.3 `char_traits` specializations [char.traits.specializations]

```
namespace std {
  template<> struct char_traits<char>;
  template<> struct char_traits<char16_t>;
  template<> struct char_traits<char32_t>;
  template<> struct char_traits<wchar_t>;
}
```

1 The header `<string>` shall define four specializations of the class template `char_traits`: `char_traits<char>`, `char_traits<char16_t>`, `char_traits<char32_t>`, and `char_traits<wchar_t>`.

2 The requirements for the members of these specializations are given in Clause 21.2.1.

21.2.3.1 `struct char_traits<char>` [char.traits.specializations.char]

```
namespace std {
  template<> struct char_traits<char> {
    typedef char        char_type;
    typedef int         int_type;
    typedef streamoff   off_type;
    typedef streampos   pos_type;
    typedef mbstate_t   state_type;

    static void assign(char_type& c1, const char_type& c2) noexcept;
    static constexpr bool eq(char_type c1, char_type c2) noexcept;
    static constexpr bool lt(char_type c1, char_type c2) noexcept;

    static int compare(const char_type* s1, const char_type* s2, size_t n);
    static size_t length(const char_type* s);
    static const char_type* find(const char_type* s, size_t n,
                 const char_type& a);
    static char_type* move(char_type* s1, const char_type* s2, size_t n);
    static char_type* copy(char_type* s1, const char_type* s2, size_t n);
    static char_type* assign(char_type* s, size_t n, char_type a);

    static constexpr int_type not_eof(int_type c) noexcept;
    static constexpr char_type to_char_type(int_type c) noexcept;
    static constexpr int_type to_int_type(char_type c) noexcept;
    static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
    static constexpr int_type eof() noexcept;
  };
}
```

1 The defined types for `int_type`, `pos_type`, `off_type`, and `state_type` shall be `int`, `streampos`, `streamoff`, and `mbstate_t` respectively.

2 The type `streampos` shall be an implementation-defined type that satisfies the requirements for `pos_type` in 27.2.2 and 27.3.

3 The type `streamoff` shall be an implementation-defined type that satisfies the requirements for `off_type` in 27.2.2 and 27.3.

4 The type `mbstate_t` is defined in `<cwchar>` and can represent any of the conversion states that can occur in an implementation-defined set of supported multibyte character encoding rules.

5 The two-argument member `assign` shall be defined identically to the built-in operator `=`. The two-argument members `eq` and `lt` shall be defined identically to the built-in operators `==` and `<` for type `unsigned char`.

6 The member `eof()` shall return `EOF`.

21.2.3.2 `struct char_traits<char16_t>` [char.traits.specializations.char16_t]

```
namespace std {
  template<> struct char_traits<char16_t> {
    typedef char16_t        char_type;
    typedef uint_least16_t  int_type;
    typedef streamoff       off_type;
    typedef u16streampos    pos_type;
    typedef mbstate_t       state_type;

    static void assign(char_type& c1, const char_type& c2) noexcept;
    static constexpr bool eq(char_type c1, char_type c2) noexcept;
    static constexpr bool lt(char_type c1, char_type c2) noexcept;

    static int compare(const char_type* s1, const char_type* s2, size_t n);
    static size_t length(const char_type* s);
    static const char_type* find(const char_type* s, size_t n,
                                 const char_type& a);
    static char_type* move(char_type* s1, const char_type* s2, size_t n);
    static char_type* copy(char_type* s1, const char_type* s2, size_t n);
    static char_type* assign(char_type* s, size_t n, char_type a);

    static constexpr int_type not_eof(int_type c) noexcept;
    static constexpr char_type to_char_type(int_type c) noexcept;
    static constexpr int_type to_int_type(char_type c) noexcept;
    static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
    static constexpr int_type eof() noexcept;
  };
}
```

1 The type `u16streampos` shall be an implementation-defined type that satisfies the requirements for pos_type in 27.2.2 and 27.3.

2 The two-argument members `assign`, `eq`, and `lt` shall be defined identically to the built-in operators `=`, `==`, and `<` respectively.

3 The member `eof()` shall return an implementation-defined constant that cannot appear as a valid UTF-16 code unit.

21.2.3.3 `struct char_traits<char32_t>` [char.traits.specializations.char32_t]

```
namespace std {
  template<> struct char_traits<char32_t> {
    typedef char32_t        char_type;
    typedef uint_least32_t  int_type;
    typedef streamoff       off_type;
    typedef u32streampos    pos_type;
    typedef mbstate_t       state_type;

    static void assign(char_type& c1, const char_type& c2) noexcept;
    static constexpr bool eq(char_type c1, char_type c2) noexcept;
    static constexpr bool lt(char_type c1, char_type c2) noexcept;

    static int compare(const char_type* s1, const char_type* s2, size_t n);
    static size_t length(const char_type* s);
    static const char_type* find(const char_type* s, size_t n,
```

```
                const char_type& a);
    static char_type* move(char_type* s1, const char_type* s2, size_t n);
    static char_type* copy(char_type* s1, const char_type* s2, size_t n);
    static char_type* assign(char_type* s, size_t n, char_type a);

    static constexpr int_type not_eof(int_type c) noexcept;
    static constexpr char_type to_char_type(int_type c) noexcept;
    static constexpr int_type to_int_type(char_type c) noexcept;
    static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
    static constexpr int_type eof() noexcept;
  };
}
```

1 The type `u32streampos` shall be an implementation-defined type that satisfies the requirements for pos_type in 27.2.2 and 27.3.

2 The two-argument members `assign`, `eq`, and `lt` shall be defined identically to the built-in operators `=`, `==`, and `<` respectively.

3 The member `eof()` shall return an implementation-defined constant that cannot appear as a Unicode code point.

21.2.3.4 `struct char_traits<wchar_t>` [char.traits.specializations.wchar.t]

```
namespace std {
  template<> struct char_traits<wchar_t> {
    typedef wchar_t      char_type;
    typedef wint_t       int_type;
    typedef streamoff    off_type;
    typedef wstreampos   pos_type;
    typedef mbstate_t    state_type;

    static void assign(char_type& c1, const char_type& c2) noexcept;
    static constexpr bool eq(char_type c1, char_type c2) noexcept;
    static constexpr bool lt(char_type c1, char_type c2) noexcept;

    static int compare(const char_type* s1, const char_type* s2, size_t n);
    static size_t length(const char_type* s);
    static const char_type* find(const char_type* s, size_t n,
                const char_type& a);
    static char_type* move(char_type* s1, const char_type* s2, size_t n);
    static char_type* copy(char_type* s1, const char_type* s2, size_t n);
    static char_type* assign(char_type* s, size_t n, char_type a);

    static constexpr int_type not_eof(int_type c) noexcept;
    static constexpr char_type to_char_type(int_type c) noexcept;
    static constexpr int_type to_int_type(char_type c) noexcept;
    static constexpr bool eq_int_type(int_type c1, int_type c2) noexcept;
    static constexpr int_type eof() noexcept;
  };
}
```

1 The defined types for `int_type`, `pos_type`, and `state_type` shall be `wint_t`, `wstreampos`, and `mbstate_t` respectively.

2 The type `wstreampos` shall be an implementation-defined type that satisfies the requirements for pos_type in 27.2.2 and 27.3.

[3] The type `mbstate_t` is defined in `<cwchar>` and can represent any of the conversion states that can occur in an implementation-defined set of supported multibyte character encoding rules.

[4] The two-argument members `assign`, `eq`, and `lt` shall be defined identically to the built-in operators `=`, `==`, and `<` respectively.

[5] The member `eof()` shall return `WEOF`.

21.3 String classes [string.classes]

[1] The header `<string>` defines the `basic_string` class template for manipulating varying-length sequences of char-like objects and four typedefs, `string`, `u16string`, `u32string`, and `wstring`, that name the specializations `basic_string<char>`, `basic_string<char16_t>`, `basic_string<char32_t>`, and `basic_string<wchar_t>`, respectively.

Header `<string>` synopsis

```
#include <initializer_list>

namespace std {

  // 21.2, character traits:
  template<class charT> struct char_traits;
  template <> struct char_traits<char>;
  template <> struct char_traits<char16_t>;
  template <> struct char_traits<char32_t>;
  template <> struct char_traits<wchar_t>;

  // 21.4, basic_string:
  template<class charT, class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
      class basic_string;

  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(const basic_string<charT,traits,Allocator>& lhs,
                const basic_string<charT,traits,Allocator>& rhs);
  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(basic_string<charT,traits,Allocator>&& lhs,
                const basic_string<charT,traits,Allocator>& rhs);
  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(const basic_string<charT,traits,Allocator>& lhs,
                basic_string<charT,traits,Allocator>&& rhs);
  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(basic_string<charT,traits,Allocator>&& lhs,
                basic_string<charT,traits,Allocator>&& rhs);
  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(const charT* lhs,
                const basic_string<charT,traits,Allocator>& rhs);
  template<class charT, class traits, class Allocator>
    basic_string<charT,traits,Allocator>
      operator+(const charT* lhs,
                basic_string<charT,traits,Allocator>&& rhs);
  template<class charT, class traits, class Allocator>
```

```
  basic_string<charT,traits,Allocator>
    operator+(charT lhs, const basic_string<charT,traits,Allocator>& rhs);
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(charT lhs, basic_string<charT,traits,Allocator>&& rhs);
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs,
              const charT* rhs);
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs,
              const charT* rhs);
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs, charT rhs);
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs, charT rhs);

template<class charT, class traits, class Allocator>
  bool operator==(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator==(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
template<class charT, class traits, class Allocator>
  bool operator==(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
template<class charT, class traits, class Allocator>
  bool operator!=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator!=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
template<class charT, class traits, class Allocator>
  bool operator!=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);

template<class charT, class traits, class Allocator>
  bool operator< (const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator< (const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
template<class charT, class traits, class Allocator>
  bool operator< (const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
template<class charT, class traits, class Allocator>
  bool operator> (const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator> (const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
template<class charT, class traits, class Allocator>
```

```
  bool operator> (const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);

template<class charT, class traits, class Allocator>
  bool operator<=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator<=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
template<class charT, class traits, class Allocator>
  bool operator<=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
template<class charT, class traits, class Allocator>
  bool operator>=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
template<class charT, class traits, class Allocator>
  bool operator>=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
template<class charT, class traits, class Allocator>
  bool operator>=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);

// 21.4.8.8, swap:
template<class charT, class traits, class Allocator>
  void swap(basic_string<charT,traits,Allocator>& lhs,
            basic_string<charT,traits,Allocator>& rhs)
    noexcept(noexcept(lhs.swap(rhs)));

// 21.4.8.9, inserters and extractors:
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    operator>>(basic_istream<charT,traits>& is,
               basic_string<charT,traits,Allocator>& str);
template<class charT, class traits, class Allocator>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const basic_string<charT,traits,Allocator>& str);
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>& is,
            basic_string<charT,traits,Allocator>& str,
            charT delim);
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>&& is,
            basic_string<charT,traits,Allocator>& str,
            charT delim);
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>& is,
            basic_string<charT,traits,Allocator>& str);
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>&& is,
            basic_string<charT,traits,Allocator>& str);
```

```
  // basic_string typedef names
  typedef basic_string<char> string;
  typedef basic_string<char16_t> u16string;
  typedef basic_string<char32_t> u32string;
  typedef basic_string<wchar_t> wstring;

  // 21.5, numeric conversions:
  int stoi(const string& str, size_t* idx = 0, int base = 10);
  long stol(const string& str, size_t* idx = 0, int base = 10);
  unsigned long stoul(const string& str, size_t* idx = 0, int base = 10);
  long long stoll(const string& str, size_t* idx = 0, int base = 10);
  unsigned long long stoull(const string& str, size_t* idx = 0, int base = 10);
  float stof(const string& str, size_t* idx = 0);
  double stod(const string& str, size_t* idx = 0);
  long double stold(const string& str, size_t* idx = 0);
  string to_string(int val);
  string to_string(unsigned val);
  string to_string(long val);
  string to_string(unsigned long val);
  string to_string(long long val);
  string to_string(unsigned long long val);
  string to_string(float val);
  string to_string(double val);
  string to_string(long double val);

  int stoi(const wstring& str, size_t* idx = 0, int base = 10);
  long stol(const wstring& str, size_t* idx = 0, int base = 10);
  unsigned long stoul(const wstring& str, size_t* idx = 0, int base = 10);
  long long stoll(const wstring& str, size_t* idx = 0, int base = 10);
  unsigned long long stoull(const wstring& str, size_t* idx = 0, int base = 10);
  float stof(const wstring& str, size_t* idx = 0);
  double stod(const wstring& str, size_t* idx = 0);
  long double stold(const wstring& str, size_t* idx = 0);
  wstring to_wstring(int val);
  wstring to_wstring(unsigned val);
  wstring to_wstring(long val);
  wstring to_wstring(unsigned long val);
  wstring to_wstring(long long val);
  wstring to_wstring(unsigned long long val);
  wstring to_wstring(float val);
  wstring to_wstring(double val);
  wstring to_wstring(long double val);

  // 21.6, hash support:
  template <class T> struct hash;
  template <> struct hash<string>;
  template <> struct hash<u16string>;
  template <> struct hash<u32string>;
  template <> struct hash<wstring>;

inline namespace literals {
inline namespace string_literals {

  // 21.7, suffix for basic_string literals:
```

```
  string    operator "" s(const char* str, size_t len);
  u16string operator "" s(const char16_t* str, size_t len);
  u32string operator "" s(const char32_t* str, size_t len);
  wstring   operator "" s(const wchar_t* str, size_t len);

}
}
}
```

21.4 Class template `basic_string` [basic.string]

1 The class template `basic_string` describes objects that can store a sequence consisting of a varying number of arbitrary char-like objects with the first element of the sequence at position zero. Such a sequence is also called a "string" if the type of the char-like objects that it holds is clear from context. In the rest of this Clause, the type of the char-like objects held in a `basic_string` object is designated by `charT`.

2 The member functions of `basic_string` use an object of the `Allocator` class passed as a template parameter to allocate and free storage for the contained char-like objects.[234]

3 A `basic_string` is a contiguous container (23.2.1).

4 In all cases, `size() <= capacity()`.

5 The functions described in this Clause can report two kinds of errors, each associated with an exception type:

(5.1) — a *length* error is associated with exceptions of type `length_error` (19.2.4);

(5.2) — an *out-of-range* error is associated with exceptions of type `out_of_range` (19.2.5).

```
namespace std {
  template<class charT, class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
  class basic_string {
  public:
    // types:
    typedef          traits                                        traits_type;
    typedef typename traits::char_type                             value_type;
    typedef          Allocator                                     allocator_type;
    typedef typename allocator_traits<Allocator>::size_type        size_type;
    typedef typename allocator_traits<Allocator>::difference_type  difference_type;

    typedef value_type& reference;
    typedef const value_type&   const_reference;
    typedef typename allocator_traits<Allocator>::pointer          pointer;
    typedef typename allocator_traits<Allocator>::const_pointer    const_pointer;

    typedef implementation-defined              iterator;       // See 23.2
    typedef implementation-defined              const_iterator; // See 23.2
    typedef std::reverse_iterator<iterator> reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;
    static const size_type npos = -1;

    // 21.4.2, construct/copy/destroy:
    basic_string() noexcept(noexcept(Allocator())) : basic_string(Allocator()) { }
    explicit basic_string(const Allocator& a) noexcept;
```

234) `Allocator::value_type` must name the same type as `charT` (21.4.1).

```
  basic_string(const basic_string& str);
  basic_string(basic_string&& str) noexcept;
  basic_string(const basic_string& str, size_type pos, size_type n = npos,
               const Allocator& a = Allocator());
  basic_string(const charT* s,
               size_type n, const Allocator& a = Allocator());
  basic_string(const charT* s, const Allocator& a = Allocator());
  basic_string(size_type n, charT c, const Allocator& a = Allocator());
  template<class InputIterator>
    basic_string(InputIterator begin, InputIterator end,
                 const Allocator& a = Allocator());
  basic_string(initializer_list<charT>, const Allocator& = Allocator());
  basic_string(const basic_string&, const Allocator&);
  basic_string(basic_string&&, const Allocator&);

 ~basic_string();
  basic_string& operator=(const basic_string& str);
  basic_string& operator=(basic_string&& str)
    noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
             allocator_traits<Allocator>::is_always_equal::value);
  basic_string& operator=(const charT* s);
  basic_string& operator=(charT c);
  basic_string& operator=(initializer_list<charT>);

  // 21.4.3, iterators:
  iterator       begin() noexcept;
  const_iterator begin() const noexcept;
  iterator       end() noexcept;
  const_iterator end() const noexcept;

  reverse_iterator       rbegin() noexcept;
  const_reverse_iterator rbegin() const noexcept;
  reverse_iterator       rend() noexcept;
  const_reverse_iterator rend() const noexcept;

  const_iterator         cbegin() const noexcept;
  const_iterator         cend() const noexcept;
  const_reverse_iterator crbegin() const noexcept;
  const_reverse_iterator crend() const noexcept;

  // 21.4.4, capacity:
  size_type size() const noexcept;
  size_type length() const noexcept;
  size_type max_size() const noexcept;
  void resize(size_type n, charT c);
  void resize(size_type n);
  size_type capacity() const noexcept;
  void reserve(size_type res_arg = 0);
  void shrink_to_fit();
  void clear() noexcept;
  bool empty() const noexcept;

  // 21.4.5, element access:
  const_reference operator[](size_type pos) const;
  reference       operator[](size_type pos);
```

```
const_reference at(size_type n) const;
reference       at(size_type n);

const charT& front() const;
charT& front();
const charT& back() const;
charT& back();

// 21.4.6, modifiers:
basic_string& operator+=(const basic_string& str);
basic_string& operator+=(const charT* s);
basic_string& operator+=(charT c);
basic_string& operator+=(initializer_list<charT>);
basic_string& append(const basic_string& str);
basic_string& append(const basic_string& str, size_type pos,
                     size_type n = npos);
basic_string& append(const charT* s, size_type n);
basic_string& append(const charT* s);
basic_string& append(size_type n, charT c);
template<class InputIterator>
  basic_string& append(InputIterator first, InputIterator last);
basic_string& append(initializer_list<charT>);
void push_back(charT c);

basic_string& assign(const basic_string& str);
basic_string& assign(basic_string&& str)
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
basic_string& assign(const basic_string& str, size_type pos,
                     size_type n = npos);
basic_string& assign(const charT* s, size_type n);
basic_string& assign(const charT* s);
basic_string& assign(size_type n, charT c);
template<class InputIterator>
  basic_string& assign(InputIterator first, InputIterator last);
basic_string& assign(initializer_list<charT>);

basic_string& insert(size_type pos1, const basic_string& str);
basic_string& insert(size_type pos1, const basic_string& str,
                     size_type pos2, size_type n = npos);
basic_string& insert(size_type pos, const charT* s, size_type n);
basic_string& insert(size_type pos, const charT* s);
basic_string& insert(size_type pos, size_type n, charT c);
iterator insert(const_iterator p, charT c);
iterator insert(const_iterator p, size_type n, charT c);
template<class InputIterator>
  iterator insert(const_iterator p, InputIterator first, InputIterator last);
iterator insert(const_iterator p, initializer_list<charT>);

basic_string& erase(size_type pos = 0, size_type n = npos);
iterator erase(const_iterator p);
iterator erase(const_iterator first, const_iterator last);

void pop_back();
```

```
basic_string& replace(size_type pos1, size_type n1,
                      const basic_string& str);
basic_string& replace(size_type pos1, size_type n1,
                      const basic_string& str,
                      size_type pos2, size_type n2 = npos);
basic_string& replace(size_type pos, size_type n1, const charT* s,
                      size_type n2);
basic_string& replace(size_type pos, size_type n1, const charT* s);
basic_string& replace(size_type pos, size_type n1, size_type n2,
                      charT c);

basic_string& replace(const_iterator i1, const_iterator i2,
                      const basic_string& str);
basic_string& replace(const_iterator i1, const_iterator i2, const charT* s,
                      size_type n);
basic_string& replace(const_iterator i1, const_iterator i2, const charT* s);
basic_string& replace(const_iterator i1, const_iterator i2,
                      size_type n, charT c);
template<class InputIterator>
  basic_string& replace(const_iterator i1, const_iterator i2,
                        InputIterator j1, InputIterator j2);
basic_string& replace(const_iterator, const_iterator, initializer_list<charT>);

size_type copy(charT* s, size_type n, size_type pos = 0) const;
void swap(basic_string& str)
  noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
           allocator_traits<Allocator>::is_always_equal::value);

// 21.4.7, string operations:
const charT* c_str() const noexcept;
const charT* data() const noexcept;
allocator_type get_allocator() const noexcept;

size_type find (const basic_string& str, size_type pos = 0) const noexcept;
size_type find (const charT* s, size_type pos, size_type n) const;
size_type find (const charT* s, size_type pos = 0) const;
size_type find (charT c, size_type pos = 0) const;
size_type rfind(const basic_string& str, size_type pos = npos) const noexcept;
size_type rfind(const charT* s, size_type pos, size_type n) const;
size_type rfind(const charT* s, size_type pos = npos) const;
size_type rfind(charT c, size_type pos = npos) const;

size_type find_first_of(const basic_string& str,
                        size_type pos = 0) const noexcept;
size_type find_first_of(const charT* s,
                        size_type pos, size_type n) const;
size_type find_first_of(const charT* s, size_type pos = 0) const;
size_type find_first_of(charT c, size_type pos = 0) const;
size_type find_last_of (const basic_string& str,
                        size_type pos = npos) const noexcept;
size_type find_last_of (const charT* s,
                        size_type pos, size_type n) const;
size_type find_last_of (const charT* s, size_type pos = npos) const;
size_type find_last_of (charT c, size_type pos = npos) const;
```

```
    size_type find_first_not_of(const basic_string& str,
                size_type pos = 0) const noexcept;
    size_type find_first_not_of(const charT* s, size_type pos,
                                size_type n) const;
    size_type find_first_not_of(const charT* s, size_type pos = 0) const;
    size_type find_first_not_of(charT c, size_type pos = 0) const;
    size_type find_last_not_of (const basic_string& str,
                                size_type pos = npos) const noexcept;
    size_type find_last_not_of (const charT* s, size_type pos,
                                size_type n) const;
    size_type find_last_not_of (const charT* s,
                                size_type pos = npos) const;
    size_type find_last_not_of (charT c, size_type pos = npos) const;

    basic_string substr(size_type pos = 0, size_type n = npos) const;
    int compare(const basic_string& str) const noexcept;
    int compare(size_type pos1, size_type n1,
                const basic_string& str) const;
    int compare(size_type pos1, size_type n1,
                const basic_string& str,
                size_type pos2, size_type n2 = npos) const;
    int compare(const charT* s) const;
    int compare(size_type pos1, size_type n1,
                const charT* s) const;
    int compare(size_type pos1, size_type n1,
                const charT* s, size_type n2) const;
  };
}
```

21.4.1 `basic_string` general requirements [string.require]

1 If any operation would cause `size()` to exceed `max_size()`, that operation shall throw an exception object of type `length_error`.

2 If any member function or operator of `basic_string` throws an exception, that function or operator shall have no other effect.

3 In every specialization `basic_string<charT, traits, Allocator>`, the type `allocator_traits<Allocator>::value_type` shall name the same type as `charT`. Every object of type `basic_string<charT, traits, Allocator>` shall use an object of type `Allocator` to allocate and free storage for the contained `charT` objects as needed. The `Allocator` object used shall be obtained as described in 23.2.1.

4 References, pointers, and iterators referring to the elements of a `basic_string` sequence may be invalidated by the following uses of that `basic_string` object:

(4.1) — as an argument to any standard library function taking a reference to non-const `basic_string` as an argument.[235]

(4.2) — Calling non-const member functions, except `operator[]`, `at`, `front`, `back`, `begin`, `rbegin`, `end`, and `rend`.

21.4.2 `basic_string` constructors and assignment operators [string.cons]

```
explicit basic_string(const Allocator& a) noexcept;
```

235) For example, as an argument to non-member functions `swap()` (21.4.8.8), `operator>>()` (21.4.8.9), and `getline()` (21.4.8.9), or as an argument to `basic_string::swap()`

1 *Effects:* Constructs an object of class `basic_string`. The postconditions of this function are indicated in Table 63.

Table 63 — `basic_string(const Allocator&)` effects

Element	Value
`data()`	a non-null pointer that is copyable and can have 0 added to it
`size()`	0
`capacity()`	an unspecified value

```
basic_string(const basic_string& str);
basic_string(basic_string&& str) noexcept;
```

2 *Effects:* Constructs an object of class `basic_string` as indicated in Table 64. In the second form, `str` is left in a valid state with an unspecified value.

Table 64 — `basic_string(const basic_string&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of the array whose first element is pointed at by `str.data()`
`size()`	`str.size()`
`capacity()`	a value at least as large as `size()`

```
basic_string(const basic_string& str,
             size_type pos, size_type n = npos,
             const Allocator& a = Allocator());
```

3 *Requires:* `pos <= str.size()`

4 *Throws:* `out_of_range` if `pos > str.size()`.

5 *Effects:* Constructs an object of class `basic_string` and determines the effective length `rlen` of the initial string value as the smaller of `n` and `str.size() - pos`, as indicated in Table 65.

Table 65 — `basic_string(const basic_string&, size_type, size_type, const Allocator&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of `rlen` consecutive elements of the string controlled by `str` beginning at position `pos`
`size()`	`rlen`
`capacity()`	a value at least as large as `size()`

```
basic_string(const charT* s, size_type n,
             const Allocator& a = Allocator());
```

6 *Requires:* `s` points to an array of at least `n` elements of `charT`.

7 *Effects:* Constructs an object of class `basic_string` and determines its initial string value from the array of `charT` of length `n` whose first element is designated by `s`, as indicated in Table 66.

Table 66 — `basic_string(const charT*, size_type, const Allocator&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of the array whose first element is pointed at by `s`
`size()`	`n`
`capacity()`	a value at least as large as `size()`

```
basic_string(const charT* s, const Allocator& a = Allocator());
```

8 *Requires:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

9 *Effects:* Constructs an object of class `basic_string` and determines its initial string value from the array of `charT` of length `traits::length(s)` whose first element is designated by `s`, as indicated in Table 67.

Table 67 — `basic_string(const charT*, const Allocator&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of the array whose first element is pointed at by `s`
`size()`	`traits::length(s)`
`capacity()`	a value at least as large as `size()`

10 *Remarks:* Uses `traits::length()`.

```
basic_string(size_type n, charT c, const Allocator& a = Allocator());
```

11 *Requires:* `n < npos`

12 *Effects:* Constructs an object of class `basic_string` and determines its initial string value by repeating the char-like object `c` for all `n` elements, as indicated in Table 68.

Table 68 — `basic_string(size_t, charT, const Allocator&)` effects

Element	Value
`data()`	points at the first element of an allocated array of `n` elements, each storing the initial value `c`
`size()`	`n`
`capacity()`	a value at least as large as `size()`

```
template<class InputIterator>
  basic_string(InputIterator begin, InputIterator end,
               const Allocator& a = Allocator());
```

13 *Effects:* If `InputIterator` is an integral type, equivalent to

```
basic_string(static_cast<size_type>(begin), static_cast<value_type>(end), a)
```

14 Otherwise constructs a string from the values in the range [`begin`, `end`), as indicated in the Sequence Requirements table (see 23.2.3).

```
basic_string(initializer_list<charT> il, const Allocator& a = Allocator());
```

15 *Effects:* Same as `basic_string(il.begin(), il.end(), a)`.

```
basic_string(const basic_string& str, const Allocator& alloc);
basic_string(basic_string&& str, const Allocator& alloc);
```

16 *Effects:* Constructs an object of class `basic_string` as indicated in Table 69. The stored allocator is constructed from `alloc`. In the second form, `str` is left in a valid state with an unspecified value.

Table 69 — `basic_string(const basic_string&, const Allocator&)` and `basic_string(basic_string&&, const Allocator&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of the array whose first element is pointed at by the original value of `str.data()`.
`size()`	the original value of `str.size()`
`capacity()`	a value at least as large as `size()`
`get_allocator()`	`alloc`

17 *Throws:* The second form throws nothing if `alloc == str.get_allocator()`.

```
basic_string& operator=(const basic_string& str);
```

18 *Effects:* If `*this` and `str` are not the same object, modifies `*this` as shown in Table 70.

19 If `*this` and `str` are the same object, the member has no effect.

20 *Returns:* `*this`

Table 70 — `operator=(const basic_string&)` effects

Element	Value
`data()`	points at the first element of an allocated copy of the array whose first element is pointed at by `str.data()`
`size()`	`str.size()`
`capacity()`	a value at least as large as `size()`

```
basic_string& operator=(basic_string&& str)
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

21 *Effects:* Move assigns as a sequence container (23.2), except that iterators, pointers and references may be invalidated.

22 *Returns:* `*this`

```
basic_string& operator=(const charT* s);
```

23 *Returns:* `*this = basic_string(s)`.

24 *Remarks:* Uses `traits::length()`.

```
basic_string& operator=(charT c);
```

25 *Returns:* `*this = basic_string(1,c)`.

```
basic_string& operator=(initializer_list<charT> il);
```

26 *Effects:* `*this = basic_string(il)`.

27 *Returns:* `*this`.

21.4.3 `basic_string` iterator support [string.iterators]

```
iterator       begin() noexcept;
const_iterator begin() const noexcept;
const_iterator cbegin() const noexcept;
```

1 *Returns:* An iterator referring to the first character in the string.

```
iterator       end() noexcept;
const_iterator end() const noexcept;
const_iterator cend() const noexcept;
```

2 *Returns:* An iterator which is the past-the-end value.

```
reverse_iterator       rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
const_reverse_iterator crbegin() const noexcept;
```

3 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(end())`.

```
reverse_iterator       rend() noexcept;
const_reverse_iterator rend() const noexcept;
const_reverse_iterator crend() const noexcept;
```

4 *Returns:* An iterator which is semantically equivalent to `reverse_iterator(begin())`.

21.4.4 `basic_string` capacity [string.capacity]

```
size_type size() const noexcept;
```

1 *Returns:* A count of the number of char-like objects currently in the string.

2 *Complexity:* Constant time.

```
size_type length() const noexcept;
```

3 *Returns:* `size()`.

```
size_type max_size() const noexcept;
```

4 *Returns:* The size of the largest possible string.

5 *Complexity:* Constant time.

```
void resize(size_type n, charT c);
```

6 *Requires:* `n <= max_size()`

7 *Throws:* `length_error` if `n > max_size()`.

8 *Effects:* Alters the length of the string designated by `*this` as follows:

(8.1) — If `n <= size()`, the function replaces the string designated by `*this` with a string of length `n` whose elements are a copy of the initial elements of the original string designated by `*this`.

(8.2) — If `n > size()`, the function replaces the string designated by `*this` with a string of length `n` whose first `size()` elements are a copy of the original string designated by `*this`, and whose remaining elements are all initialized to `c`.

```
void resize(size_type n);
```

9 *Effects:* `resize(n,charT())`.

```
size_type capacity() const noexcept;
```

10 *Returns:* The size of the allocated storage in the string.

```
void reserve(size_type res_arg=0);
```

11 The member function `reserve()` is a directive that informs a `basic_string` object of a planned change in size, so that it can manage the storage allocation accordingly.

12 *Effects:* After `reserve()`, `capacity()` is greater or equal to the argument of `reserve`. [*Note:* Calling `reserve()` with a `res_arg` argument less than `capacity()` is in effect a non-binding shrink request. A call with `res_arg <= size()` is in effect a non-binding shrink-to-fit request. — *end note*]

13 *Throws:* `length_error` if `res_arg > max_size()`.[236]

```
void shrink_to_fit();
```

14 *Remarks:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*]

```
void clear() noexcept;
```

15 *Effects:* Behaves as if the function calls:

```
  erase(begin(), end());
```

```
bool empty() const noexcept;
```

16 *Returns:* `size() == 0`.

21.4.5 `basic_string` element access [string.access]

```
const_reference operator[](size_type pos) const;
reference       operator[](size_type pos);
```

1 *Requires:* `pos <= size()`.

2 *Returns:* `*(begin() + pos)` if `pos < size()`. Otherwise, returns a reference to an object of type `charT` with value `charT()`, where modifying the object leads to undefined behavior.

3 *Throws:* Nothing.

4 *Complexity:* Constant time.

```
const_reference at(size_type pos) const;
reference       at(size_type pos);
```

5 *Throws:* `out_of_range` if `pos >= size()`.

6 *Returns:* `operator[](pos)`.

```
const charT& front() const;
charT& front();
```

7 *Requires:* `!empty()`

8 *Effects:* Equivalent to `operator[](0)`.

```
const charT& back() const;
charT& back();
```

9 *Requires:* `!empty()`

10 *Effects:* Equivalent to `operator[](size() - 1)`.

236) `reserve()` uses `allocator_traits<Allocator>::allocate()` which may throw an appropriate exception.

21.4.6 basic_string modifiers [string.modifiers]

21.4.6.1 basic_string::operator+= [string::op+=]

```
basic_string&
  operator+=(const basic_string& str);
```

1 *Effects:* Calls `append(str)`.

2 *Returns:* `*this`.

```
basic_string& operator+=(const charT* s);
```

3 *Effects:* Calls `append(s)`.

4 *Returns:* `*this`.

```
basic_string& operator+=(charT c);
```

5 *Effects:* Calls `push_back(c)`;

6 *Returns:* `*this`.

```
basic_string& operator+=(initializer_list<charT> il);
```

7 *Effects:* Calls `append(il)`.

8 *Returns:* `*this`.

21.4.6.2 basic_string::append [string::append]

```
basic_string&
  append(const basic_string& str);
```

1 *Effects:* Calls `append(str.data(), str.size())`.

2 *Returns:* `*this`.

```
basic_string&
  append(const basic_string& str, size_type pos, size_type n = npos);
```

3 *Requires:* `pos <= str.size()`

4 *Throws:* `out_of_range` if `pos > str.size()`.

5 *Effects:* Determines the effective length `rlen` of the string to append as the smaller of `n` and `str.size() - pos` and calls `append(str.data() + pos, rlen)`.

6 *Returns:* `*this`.

```
basic_string&
  append(const charT* s, size_type n);
```

7 *Requires:* `s` points to an array of at least `n` elements of `charT`.

8 *Throws:* `length_error` if `size() + n > max_size()`.

9 *Effects:* The function replaces the string controlled by `*this` with a string of length `size() + n` whose first `size()` elements are a copy of the original string controlled by `*this` and whose remaining elements are a copy of the initial `n` elements of `s`.

10 *Returns:* `*this`.

```
basic_string& append(const charT* s);
```

11 *Requires:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

12 *Effects:* Calls `append(s, traits::length(s))`.

13 *Returns:* `*this`.

```
basic_string& append(size_type n, charT c);
```

14 *Effects:* Equivalent to `append(basic_string(n, c))`.

15 *Returns:* `*this`.

```
template<class InputIterator>
  basic_string& append(InputIterator first, InputIterator last);
```

16 *Requires:* `[first,last)` is a valid range.

17 *Effects:* Equivalent to `append(basic_string(first, last))`.

18 *Returns:* `*this`.

```
basic_string& append(initializer_list<charT> il);
```

19 *Effects:* Calls `append(il.begin(), il.size())`.

20 *Returns:* `*this`.

```
void push_back(charT c);
```

21 *Effects:* Equivalent to `append(static_cast<size_type>(1), c)`.

21.4.6.3 `basic_string::assign` [string::assign]

```
basic_string& assign(const basic_string& str);
```

1 *Effects:* Equivalent to `assign(str, 0, npos)`.

2 *Returns:* `*this`.

```
basic_string& assign(basic_string&& str)
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

3 *Effects:* Equivalent to `*this = std::move(str)`.

4 *Returns:* `*this`.

```
basic_string&
  assign(const basic_string& str, size_type pos,
         size_type n = npos);
```

5 *Requires:* `pos <= str.size()`

6 *Throws:* `out_of_range` if `pos > str.size()`.

7 *Effects:* Determines the effective length `rlen` of the string to assign as the smaller of `n` and `str.size() - pos` and calls `assign(str.data() + pos, rlen)`.

8 *Returns:* `*this`.

```
basic_string& assign(const charT* s, size_type n);
```

9 *Requires:* `s` points to an array of at least `n` elements of `charT`.

10 *Throws:* `length_error` if `n > max_size()`.

11 *Effects:* Replaces the string controlled by `*this` with a string of length `n` whose elements are a copy of those pointed to by `s`.

12 *Returns:* `*this`.

```
basic_string& assign(const charT* s);
```

13 *Requires:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

14 *Effects:* Calls `assign(s, traits::length(s))`.

15 *Returns:* `*this`.

```
basic_string& assign(initializer_list<charT> il);
```

16 *Effects:* Calls `assign(il.begin(), il.size())`.

17 `*this`.

```
basic_string& assign(size_type n, charT c);
```

18 *Effects:* Equivalent to `assign(basic_string(n, c))`.

19 *Returns:* `*this`.

```
template<class InputIterator>
  basic_string& assign(InputIterator first, InputIterator last);
```

20 *Effects:* Equivalent to `assign(basic_string(first, last))`.

21 *Returns:* `*this`.

21.4.6.4 `basic_string::insert` [string::insert]

```
basic_string&
  insert(size_type pos1,
         const basic_string& str);
```

1 *Requires:* `pos <= size()`.

2 *Throws:* `out_of_range` if `pos > size()`.

3 *Effects:* Calls `insert(pos, str.data(), str.size())`.

4 *Returns:* `*this`.

```
basic_string&
  insert(size_type pos1,
         const basic_string& str,
         size_type pos2, size_type n = npos);
```

5 *Requires:* `pos1 <= size()` and `pos2 <= str.size()`

6 *Throws:* `out_of_range` if `pos1 > size()` or `pos2 > str.size()`.

7 *Effects:* Determines the effective length `rlen` of the string to insert as the smaller of `n` and `str.size() - pos2` and calls `insert(pos1, str.data() + pos2, rlen)`.

8 *Returns:* `*this`.

```
basic_string&
  insert(size_type pos, const charT* s, size_type n);
```

9 *Requires:* `s` points to an array of at least `n` elements of `charT` and `pos <= size()`.

10 *Throws:* `out_of_range` if `pos > size()` or `length_error` if `size() + n > max_size()`.

11 *Effects:* Replaces the string controlled by `*this` with a string of length `size() + n` whose first `pos` elements are a copy of the initial elements of the original string controlled by `*this` and whose next `n` elements are a copy of the elements in `s` and whose remaining elements are a copy of the remaining elements of the original string controlled by `*this`.

12 *Returns:* `*this`.

```
basic_string&
  insert(size_type pos, const charT* s);
```

13 *Requires:* `pos <= size()` and `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

14 *Effects:* Equivalent to `insert(pos, s, traits::length(s))`.

15 *Returns:* `*this`.

```
basic_string&
  insert(size_type pos, size_type n, charT c);
```

16 *Effects:* Equivalent to `insert(pos, basic_string(n, c))`.

17 *Returns:* `*this`.

```
iterator insert(const_iterator p, charT c);
```

18 *Requires:* `p` is a valid iterator on `*this`.

19 *Effects:* inserts a copy of `c` before the character referred to by `p`.

20 *Returns:* An iterator which refers to the copy of the inserted character.

```
iterator insert(const_iterator p, size_type n, charT c);
```

21 *Requires:* `p` is a valid iterator on `*this`.

22 *Effects:* inserts `n` copies of `c` before the character referred to by `p`.

23 *Returns:* An iterator which refers to the copy of the first inserted character, or `p` if `n == 0`.

```
template<class InputIterator>
  iterator insert(const_iterator p, InputIterator first, InputIterator last);
```

24 *Requires:* `p` is a valid iterator on `*this`. `[first,last)` is a valid range.

25 *Effects:* Equivalent to `insert(p - begin(), basic_string(first, last))`.

26 *Returns:* An iterator which refers to the copy of the first inserted character, or `p` if `first == last`.

```
iterator insert(const_iterator p, initializer_list<charT> il);
```

27 *Effects:* `insert(p, il.begin(), il.end())`.

28 *Returns:* An iterator which refers to the copy of the first inserted character, or `p` if `i1` is empty.

21.4.6.5 `basic_string::erase` [**string::erase**]

```
basic_string& erase(size_type pos = 0, size_type n = npos);
```

1 *Requires:* `pos <= size()`

2 *Throws:* `out_of_range` if `pos > size()`.

3 *Effects:* Determines the effective length `xlen` of the string to be removed as the smaller of `n` and `size() - pos`.

4 The function then replaces the string controlled by `*this` with a string of length `size() - xlen` whose first `pos` elements are a copy of the initial elements of the original string controlled by `*this`, and whose remaining elements are a copy of the elements of the original string controlled by `*this` beginning at position `pos + xlen`.

5 *Returns:* `*this`.

```
iterator erase(const_iterator p);
```

6 *Throws:* Nothing.

7 *Effects:* removes the character referred to by `p`.

8 *Returns:* An iterator which points to the element immediately following `p` prior to the element being erased. If no such element exists, `end()` is returned.

```
iterator erase(const_iterator first, const_iterator last);
```

9 *Requires:* `first` and `last` are valid iterators on `*this`, defining a range `[first,last)`.

10 *Throws:* Nothing.

11 *Effects:* removes the characters in the range `[first,last)`.

12 *Returns:* An iterator which points to the element pointed to by `last` prior to the other elements being erased. If no such element exists, `end()` is returned.

```
void pop_back();
```

13 *Requires:* `!empty()`

14 *Throws:* Nothing.

15 *Effects:* Equivalent to `erase(size() - 1, 1)`.

21.4.6.6 `basic_string::replace` [**string::replace**]

```
basic_string&
  replace(size_type pos1, size_type n1,
          const basic_string& str);
```

1 *Requires:* `pos1 <= size()`.

2 *Throws:* `out_of_range` if `pos1 > size()`.

3 *Effects:* Calls `replace(pos1, n1, str.data(), str.size())`.

4 *Returns:* `*this`.

```
basic_string&
  replace(size_type pos1, size_type n1,
          const basic_string& str,
          size_type pos2, size_type n2 = npos);
```

5 *Requires:* `pos1 <= size()` and `pos2 <= str.size()`.

6 *Throws:* `out_of_range` if `pos1 > size()` or `pos2 > str.size()`.

7 *Effects:* Determines the effective length `rlen` of the string to be inserted as the smaller of `n2` and `str.size() - pos2` and calls `replace(pos1, n1, str.data() + pos2, rlen)`.

8 *Returns:* `*this`.

```
basic_string&
  replace(size_type pos1, size_type n1, const charT* s, size_type n2);
```

9 *Requires:* `pos1 <= size()` and `s` points to an array of at least `n2` elements of `charT`.

10 *Throws:* `out_of_range` if `pos1 > size()` or `length_error` if the length of the resulting string would exceed `max_size()` (see below).

11 *Effects:* Determines the effective length `xlen` of the string to be removed as the smaller of `n1` and `size() - pos1`. If `size() - xlen >= max_size() - n2` throws `length_error`. Otherwise, the function replaces the string controlled by `*this` with a string of length `size() - xlen + n2` whose first `pos1` elements are a copy of the initial elements of the original string controlled by `*this`, whose next `n2` elements are a copy of the initial `n2` elements of `s`, and whose remaining elements are a copy of the elements of the original string controlled by `*this` beginning at position `pos + xlen`.

12 *Returns:* `*this`.

```
basic_string&
  replace(size_type pos, size_type n, const charT* s);
```

13 *Requires:* `pos <= size()` and `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

14 *Effects:* Calls `replace(pos, n, s, traits::length(s))`.

15 *Returns:* `*this`.

```
basic_string&
  replace(size_type pos1, size_type n1,
          size_type n2, charT c);
```

16 *Effects:* Equivalent to `replace(pos1, n1, basic_string(n2, c))`.

17 *Returns:* `*this`.

```
basic_string& replace(const_iterator i1, const_iterator i2, const basic_string& str);
```

18 *Requires:* `[begin(),i1)` and `[i1,i2)` are valid ranges.

19 *Effects:* Calls `replace(i1 - begin(), i2 - i1, str)`.

20 *Returns:* `*this`.

```
basic_string&
  replace(const_iterator i1, const_iterator i2, const charT* s, size_type n);
```

21 *Requires:* `[begin(),i1)` and `[i1,i2)` are valid ranges and `s` points to an array of at least `n` elements of `charT`.

22 *Effects:* Calls `replace(i1 - begin(), i2 - i1, s, n)`.

23 *Returns:* `*this`.

```
basic_string& replace(const_iterator i1, const_iterator i2, const charT* s);
```

24 *Requires:* `[begin(),i1)` and `[i1,i2)` are valid ranges and `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

25 *Effects:* Calls `replace(i1 - begin(), i2 - i1, s, traits::length(s))`.

26 *Returns:* `*this`.

```
basic_string& replace(const_iterator i1, const_iterator i2, size_type n,
                      charT c);
```

27 *Requires:* `[begin(),i1)` and `[i1,i2)` are valid ranges.

28 *Effects:* Calls `replace(i1 - begin(), i2 - i1, basic_string(n, c))`.

29 *Returns:* `*this`.

```
template<class InputIterator>
  basic_string& replace(const_iterator i1, const_iterator i2,
                        InputIterator j1, InputIterator j2);
```

30 *Requires:* `[begin(),i1)`, `[i1,i2)` and `[j1,j2)` are valid ranges.

31 *Effects:* Calls `replace(i1 - begin(), i2 - i1, basic_string(j1, j2))`.

32 *Returns:* `*this`.

```
basic_string& replace(const_iterator i1, const_iterator i2,
                      initializer_list<charT> il);
```

33 *Requires:* `[begin(),i1)` and `[i1,i2)` are valid ranges.

34 *Effects:* Calls `replace(i1 - begin(), i2 - i1, il.begin(), il.size())`.

35 *Returns:* `*this`.

21.4.6.7 `basic_string::copy` [string::copy]

```
size_type copy(charT* s, size_type n, size_type pos = 0) const;
```

1 *Requires:* `pos <= size()`

2 *Throws:* `out_of_range` if `pos > size()`.

3 *Effects:* Determines the effective length `rlen` of the string to copy as the smaller of `n` and `size() - pos`. `s` shall designate an array of at least `rlen` elements.

The function then replaces the string designated by `s` with a string of length `rlen` whose elements are a copy of the string controlled by `*this` beginning at position `pos`.

The function does not append a null object to the string designated by `s`.

4 *Returns:* `rlen`.

21.4.6.8 `basic_string::swap` [string::swap]

```
void swap(basic_string& s)
  noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

1 *Postcondition:* `*this` contains the same sequence of characters that was in `s`, `s` contains the same sequence of characters that was in `*this`.

2 *Throws:* Nothing.

3 *Complexity:* Constant time.

21.4.7 basic_string string operations [string.ops]

21.4.7.1 basic_string accessors [string.accessors]

```
const charT* c_str() const noexcept;
const charT* data() const noexcept;
```

1 *Returns:* A pointer p such that p + i == &operator[](i) for each i in [0,size()].

2 *Complexity:* Constant time.

3 *Requires:* The program shall not alter any of the values stored in the character array.

```
allocator_type get_allocator() const noexcept;
```

4 *Returns:* A copy of the Allocator object used to construct the string or, if that allocator has been replaced, a copy of the most recent replacement.

21.4.7.2 basic_string::find [string::find]

```
size_type find(const basic_string& str,
               size_type pos = 0) const noexcept;
```

1 *Effects:* Determines the lowest position xpos, if possible, such that both of the following conditions obtain:

(1.1) — pos <= xpos and xpos + str.size() <= size();

(1.2) — traits::eq(at(xpos+I), str.at(I)) for all elements I of the string controlled by str.

2 *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

3 *Remarks:* Uses traits::eq().

```
size_type find(const charT* s, size_type pos, size_type n) const;
```

4 *Returns:* find(basic_string(s,n),pos).

```
size_type find(const charT* s, size_type pos = 0) const;
```

5 *Requires:* s points to an array of at least traits::length(s) + 1 elements of charT.

6 *Returns:* find(basic_string(s), pos).

```
size_type find(charT c, size_type pos = 0) const;
```

7 *Returns:* find(basic_string(1,c), pos).

21.4.7.3 basic_string::rfind [string::rfind]

```
size_type rfind(const basic_string& str,
                size_type pos = npos) const noexcept;
```

1 *Effects:* Determines the highest position xpos, if possible, such that both of the following conditions obtain:

(1.1) — xpos <= pos and xpos + str.size() <= size();

(1.2) — traits::eq(at(xpos+I), str.at(I)) for all elements I of the string controlled by str.

2 *Returns:* xpos if the function can determine such a value for xpos. Otherwise, returns npos.

3 *Remarks:* Uses traits::eq().

```
size_type rfind(const charT* s, size_type pos, size_type n) const;
```

4 *Returns:* rfind(basic_string(s, n), pos).

```
size_type rfind(const charT* s, size_type pos = npos) const;
```

Requires: s points to an array of at least `traits::length(s) + 1` elements of `charT`.

Returns: `rfind(basic_string(s), pos)`.

```
size_type rfind(charT c, size_type pos = npos) const;
```

Returns: `rfind(basic_string(1,c),pos)`.

21.4.7.4 `basic_string::find_first_of` [string::find.first.of]

```
size_type
  find_first_of(const basic_string& str,
                size_type pos = 0) const noexcept;
```

Effects: Determines the lowest position `xpos`, if possible, such that both of the following conditions obtain:

(1.1) — `pos <= xpos` and `xpos < size()`;

(1.2) — `traits::eq(at(xpos), str.at(I))` for some element `I` of the string controlled by `str`.

Returns: `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

Remarks: Uses `traits::eq()`.

```
size_type
  find_first_of(const charT* s, size_type pos, size_type n) const;
```

Returns: `find_first_of(basic_string(s, n), pos)`.

```
size_type find_first_of(const charT* s, size_type pos = 0) const;
```

Requires: `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

Returns: `find_first_of(basic_string(s), pos)`.

```
size_type find_first_of(charT c, size_type pos = 0) const;
```

Returns: `find_first_of(basic_string(1,c), pos)`.

21.4.7.5 `basic_string::find_last_of` [string::find.last.of]

```
size_type
  find_last_of(const basic_string& str,
               size_type pos = npos) const noexcept;
```

Effects: Determines the highest position `xpos`, if possible, such that both of the following conditions obtain:

(1.1) — `xpos <= pos` and `xpos < size()`;

(1.2) — `traits::eq(at(xpos), str.at(I))` for some element `I` of the string controlled by `str`.

Returns: `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

Remarks: Uses `traits::eq()`.

```
size_type find_last_of(const charT* s, size_type pos, size_type n) const;
```

Returns: `find_last_of(basic_string(s, n), pos)`.

```
size_type find_last_of(const charT* s, size_type pos = npos) const;
```

Requires: `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

Returns: `find_last_of(basic_string(s), pos)`.

```
size_type find_last_of(charT c, size_type pos = npos) const;
```

Returns: `find_last_of(basic_string(1,c),pos)`.

21.4.7.6 `basic_string::find_first_not_of` [string::find.first.not.of]

```
size_type
  find_first_not_of(const basic_string& str,
                    size_type pos = 0) const noexcept;
```

1 *Effects:* Determines the lowest position `xpos`, if possible, such that both of the following conditions obtain:

(1.1) — `pos <= xpos` and `xpos < size()`;

(1.2) — `traits::eq(at(xpos), str.at(I))` for no element `I` of the string controlled by `str`.

2 *Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

3 *Remarks:* Uses `traits::eq()`.

```
size_type
  find_first_not_of(const charT* s, size_type pos, size_type n) const;
```

4 *Returns:* `find_first_not_of(basic_string(s, n), pos)`.

```
size_type find_first_not_of(const charT* s, size_type pos = 0) const;
```

5 *Requires:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

6 *Returns:* `find_first_not_of(basic_string(s), pos)`.

```
size_type find_first_not_of(charT c, size_type pos = 0) const;
```

7 *Returns:* `find_first_not_of(basic_string(1, c), pos)`.

21.4.7.7 `basic_string::find_last_not_of` [string::find.last.not.of]

```
size_type
  find_last_not_of(const basic_string& str,
                   size_type pos = npos) const noexcept;
```

1 *Effects:* Determines the highest position `xpos`, if possible, such that both of the following conditions obtain:

(1.1) — `xpos <= pos` and `xpos < size()`;

(1.2) — `traits::eq(at(xpos), str.at(I))` for no element `I` of the string controlled by `str`.

2 *Returns:* `xpos` if the function can determine such a value for `xpos`. Otherwise, returns `npos`.

3 *Remarks:* Uses `traits::eq()`.

```
size_type find_last_not_of(const charT* s, size_type pos,
                           size_type n) const;
```

4 *Returns:* `find_last_not_of(basic_string(s, n), pos)`.

```
size_type find_last_not_of(const charT* s, size_type pos = npos) const;
```

5 *Requires:* `s` points to an array of at least `traits::length(s) + 1` elements of `charT`.

6 *Returns:* `find_last_not_of(basic_string(s), pos)`.

```
size_type find_last_not_of(charT c, size_type pos = npos) const;
```

7 *Returns:* `find_last_not_of(basic_string(1, c), pos)`.

21.4.7.8 `basic_string::substr` [string::substr]

```
basic_string substr(size_type pos = 0, size_type n = npos) const;
```

1 *Requires:* `pos <= size()`

2 *Throws:* `out_of_range` if `pos > size()`.

3 *Effects:* Determines the effective length `rlen` of the string to copy as the smaller of `n` and `size() - pos`.

4 *Returns:* `basic_string(data()+pos,rlen)`.

21.4.7.9 `basic_string::compare` [string::compare]

```
int compare(const basic_string& str) const noexcept;
```

1 *Effects:* Determines the effective length *rlen* of the strings to compare as the smallest of `size()` and `str.size()`. The function then compares the two strings by calling `traits::compare(data(), str.data(), rlen)`.

2 *Returns:* The nonzero result if the result of the comparison is nonzero. Otherwise, returns a value as indicated in Table 71.

Table 71 — `compare()` results

Condition	Return Value
`size() < str.size()`	`< 0`
`size() == str.size()`	`0`
`size() > str.size()`	`> 0`

```
int compare(size_type pos1, size_type n1,
            const basic_string& str) const;
```

3 *Returns:* `basic_string(*this,pos1,n1).compare(str)`.

```
int compare(size_type pos1, size_type n1,
            const basic_string& str,
            size_type pos2, size_type n2 = npos) const;
```

4 *Returns:* `basic_string(*this, pos1, n1).compare(basic_string(str, pos2, n2))`.

```
int compare(const charT* s) const;
```

5 *Returns:* `compare(basic_string(s))`.

```
int compare(size_type pos, size_type n1,
            const charT* s) const;
```

6 *Returns:* `basic_string(*this, pos, n1).compare(basic_string(s))`.

```
int compare(size_type pos, size_type n1,
            const charT* s, size_type n2) const;
```

7 *Returns:* `basic_string(*this, pos, n1).compare(basic_string(s, n2))`.

21.4.8 basic_string non-member functions [string.nonmembers]

21.4.8.1 operator+ [string::op+]

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs,
              const basic_string<charT,traits,Allocator>& rhs);
```

1 *Returns:* `basic_string<charT,traits,Allocator>(lhs).append(rhs)`

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs,
              const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `std::move(lhs.append(rhs))`

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs,
              basic_string<charT,traits,Allocator>&& rhs);
```

3 *Returns:* `std::move(rhs.insert(0, lhs))`

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs,
              basic_string<charT,traits,Allocator>&& rhs);
```

4 *Returns:* `std::move(lhs.append(rhs))` [*Note:* Or equivalently `std::move(rhs.insert(0, lhs))` — *end note*]

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const charT* lhs,
              const basic_string<charT,traits,Allocator>& rhs);
```

5 *Returns:* `basic_string<charT,traits,Allocator>(lhs) + rhs`.

6 *Remarks:* Uses `traits::length()`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const charT* lhs,
              basic_string<charT,traits,Allocator>&& rhs);
```

7 *Returns:* `std::move(rhs.insert(0, lhs))`.

8 *Remarks:* Uses `traits::length()`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(charT lhs,
              const basic_string<charT,traits,Allocator>& rhs);
```

9 *Returns:* `basic_string<charT,traits,Allocator>(1,lhs) + rhs`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(charT lhs,
              basic_string<charT,traits,Allocator>&& rhs);
```

10 *Returns:* `std::move(rhs.insert(0, 1, lhs))`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs,
              const charT* rhs);
```

11 *Returns:* `lhs + basic_string<charT,traits,Allocator>(rhs)`.

12 *Remarks:* Uses `traits::length()`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs,
              const charT* rhs);
```

13 *Returns:* `std::move(lhs.append(rhs))`.

14 *Remarks:* Uses `traits::length()`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(const basic_string<charT,traits,Allocator>& lhs,
              charT rhs);
```

15 *Returns:* `lhs + basic_string<charT,traits,Allocator>(1,rhs)`.

```
template<class charT, class traits, class Allocator>
  basic_string<charT,traits,Allocator>
    operator+(basic_string<charT,traits,Allocator>&& lhs,
              charT rhs);
```

16 *Returns:* `std::move(lhs.append(1, rhs))`.

21.4.8.2 `operator==` [string::operator==]

```
template<class charT, class traits, class Allocator>
  bool operator==(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `lhs.compare(rhs) == 0`.

```
template<class charT, class traits, class Allocator>
  bool operator==(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs == lhs`.

```
template<class charT, class traits, class Allocator>
  bool operator==(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Requires:* `rhs` points to an array of at least `traits::length(rhs) + 1` elements of `charT`.

4 *Returns:* `lhs.compare(rhs) == 0`.

21.4.8.3 `operator!=` [string::op!=]

```
template<class charT, class traits, class Allocator>
  bool operator!=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `!(lhs == rhs)`.

```
template<class charT, class traits, class Allocator>
  bool operator!=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs != lhs`.

```
template<class charT, class traits, class Allocator>
  bool operator!=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Requires:* `rhs` points to an array of at least `traits::length(rhs) + 1` elements of `charT`.

4 *Returns:* `lhs.compare(rhs) != 0`.

21.4.8.4 `operator<` [string::op<]

```
template<class charT, class traits, class Allocator>
  bool operator< (const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `lhs.compare(rhs) < 0`.

```
template<class charT, class traits, class Allocator>
  bool operator< (const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs.compare(lhs) > 0`.

```
template<class charT, class traits, class Allocator>
  bool operator< (const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Returns:* `lhs.compare(rhs) < 0`.

21.4.8.5 `operator>` [string::op>]

```
template<class charT, class traits, class Allocator>
  bool operator> (const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `lhs.compare(rhs) > 0`.

```
template<class charT, class traits, class Allocator>
  bool operator> (const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs.compare(lhs) < 0`.

```
template<class charT, class traits, class Allocator>
  bool operator> (const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Returns:* `lhs.compare(rhs) > 0`.

21.4.8.6 operator<= [string::op<=]

```
template<class charT, class traits, class Allocator>
  bool operator<=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `lhs.compare(rhs) <= 0`.

```
template<class charT, class traits, class Allocator>
  bool operator<=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs.compare(lhs) >= 0`.

```
template<class charT, class traits, class Allocator>
  bool operator<=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Returns:* `lhs.compare(rhs) <= 0`.

21.4.8.7 operator>= [string::op>=]

```
template<class charT, class traits, class Allocator>
  bool operator>=(const basic_string<charT,traits,Allocator>& lhs,
                  const basic_string<charT,traits,Allocator>& rhs) noexcept;
```

1 *Returns:* `lhs.compare(rhs) >= 0`.

```
template<class charT, class traits, class Allocator>
  bool operator>=(const charT* lhs,
                  const basic_string<charT,traits,Allocator>& rhs);
```

2 *Returns:* `rhs.compare(lhs) <= 0`.

```
template<class charT, class traits, class Allocator>
  bool operator>=(const basic_string<charT,traits,Allocator>& lhs,
                  const charT* rhs);
```

3 *Returns:* `lhs.compare(rhs) >= 0`.

21.4.8.8 swap [string.special]

```
template<class charT, class traits, class Allocator>
  void swap(basic_string<charT,traits,Allocator>& lhs,
            basic_string<charT,traits,Allocator>& rhs)
    noexcept(noexcept(lhs.swap(rhs)));
```

1 *Effects:* Equivalent to `lhs.swap(rhs);`

21.4.8.9 Inserters and extractors [string.io]

```
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    operator>>(basic_istream<charT,traits>& is,
               basic_string<charT,traits,Allocator>& str);
```

1 *Effects:* Behaves as a formatted input function (27.7.2.2.1). After constructing a `sentry` object, if the sentry converts to true, calls `str.erase()` and then extracts characters from `is` and appends them to `str` as if by calling `str.append(1,c)`. If `is.width()` is greater than zero, the maximum number `n` of characters appended is `is.width()`; otherwise `n` is `str.max_size()`. Characters are extracted and appended until any of the following occurs:

(1.1) — n characters are stored;

(1.2) — end-of-file occurs on the input sequence;

(1.3) — `isspace(c,is.getloc())` is true for the next available input character c.

2 After the last character (if any) is extracted, `is.width(0)` is called and the `sentry` object `k` is destroyed.

3 If the function extracts no characters, it calls `is.setstate(ios::failbit)`, which may throw `ios_base::failure` (27.5.5.4).

4 *Returns:* `is`

```
template<class charT, class traits, class Allocator>
  basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>& os,
               const basic_string<charT,traits,Allocator>& str);
```

5 *Effects:* Behaves as a formatted output function (27.7.3.6.1) of `os`. Forms a character sequence `seq`, initially consisting of the elements defined by the range `[str.begin(), str.end())`. Determines padding for `seq` as described in 27.7.3.6.1. Then inserts `seq` as if by calling `os.rdbuf()->sputn(seq, n)`, where `n` is the larger of `os.width()` and `str.size()`; then calls `os.width(0)`.

6 *Returns:* `os`

```
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>& is,
            basic_string<charT,traits,Allocator>& str,
            charT delim);
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>&& is,
            basic_string<charT,traits,Allocator>& str,
            charT delim);
```

7 *Effects:* Behaves as an unformatted input function (27.7.2.3), except that it does not affect the value returned by subsequent calls to `basic_istream<>::gcount()`. After constructing a `sentry` object, if the sentry converts to true, calls `str.erase()` and then extracts characters from `is` and appends them to `str` as if by calling `str.append(1, c)` until any of the following occurs:

(7.1) — end-of-file occurs on the input sequence (in which case, the `getline` function calls `is.setstate(ios_base::eofbit)`).

(7.2) — `traits::eq(c, delim)` for the next available input character c (in which case, c is extracted but not appended) (27.5.5.4)

(7.3) — `str.max_size()` characters are stored (in which case, the function calls `is.setstate(ios_base::failbit))` (27.5.5.4)

8 The conditions are tested in the order shown. In any case, after the last character is extracted, the `sentry` object `k` is destroyed.

9 If the function extracts no characters, it calls `is.setstate(ios_base::failbit)` which may throw `ios_base::failure` (27.5.5.4).

10 *Returns:* `is`.

```
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>& is,
            basic_string<charT,traits,Allocator>& str);
```

```
template<class charT, class traits, class Allocator>
  basic_istream<charT,traits>&
    getline(basic_istream<charT,traits>&& is,
            basic_string<charT,traits,Allocator>& str);
```

11 *Returns:* `getline(is,str,is.widen('\n'))`

21.5 Numeric conversions [string.conversions]

```
int stoi(const string& str, size_t* idx = 0, int base = 10);
long stol(const string& str, size_t* idx = 0, int base = 10);
unsigned long stoul(const string& str, size_t* idx = 0, int base = 10);
long long stoll(const string& str, size_t* idx = 0, int base = 10);
unsigned long long stoull(const string& str, size_t* idx = 0, int base = 10);
```

1 *Effects:* the first two functions call `strtol(str.c_str(), ptr, base)`, and the last three functions call `strtoul(str.c_str(), ptr, base)`, `strtoll(str.c_str(), ptr, base)`, and `strtoull(str.c_str(), ptr, base)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != 0`, the function stores in `*idx` the index of the first unconverted element of `str`.

2 *Returns:* The converted result.

3 *Throws:* `invalid_argument` if `strtol`, `strtoul`, `strtoll`, or `strtoull` reports that no conversion could be performed. Throws `out_of_range` if `strtol`, `strtoul`, `strtoll` or `strtoull` sets `errno` to `ERANGE`, or if the converted value is outside the range of representable values for the return type.

```
float stof(const string& str, size_t* idx = 0);
double stod(const string& str, size_t* idx = 0);
long double stold(const string& str, size_t* idx = 0);
```

4 *Effects:* These functions call `strtof(str.c_str(), ptr)`, `strtod(str.c_str(), ptr)`, and `strtold(str.c_str(), ptr)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != 0`, the function stores in `*idx` the index of the first unconverted element of `str`.

5 *Returns:* The converted result.

6 *Throws:* `invalid_argument` if `strtof`, `strtod`, or `strtold` reports that no conversion could be performed. Throws `out_of_range` if `strtof`, `strtod`, or `strtold` sets `errno` to `ERANGE` or if the converted value is outside the range of representable values for the return type.

```
string to_string(int val);
string to_string(unsigned val);
string to_string(long val);
string to_string(unsigned long val);
string to_string(long long val);
string to_string(unsigned long long val);
string to_string(float val);
string to_string(double val);
string to_string(long double val);
```

7 *Returns:* Each function returns a `string` object holding the character representation of the value of its argument that would be generated by calling `sprintf(buf, fmt, val)` with a format specifier of `"%d"`, `"%u"`, `"%ld"`, `"%lu"`, `"%lld"`, `"%llu"`, `"%f"`, `"%f"`, or `"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size.

```
int stoi(const wstring& str, size_t* idx = 0, int base = 10);
long stol(const wstring& str, size_t* idx = 0, int base = 10);
unsigned long stoul(const wstring& str, size_t* idx = 0, int base = 10);
long long stoll(const wstring& str, size_t* idx = 0, int base = 10);
unsigned long long stoull(const wstring& str, size_t* idx = 0, int base = 10);
```

8 *Effects:* the first two functions call `wcstol(str.c_str(), ptr, base)`, and the last three functions call `wcstoul(str.c_str(), ptr, base)`, `wcstoll(str.c_str(), ptr, base)`, and `wcstoull(str.c_str(), ptr, base)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != 0`, the function stores in `*idx` the index of the first unconverted element of `str`.

9 *Returns:* The converted result.

10 *Throws:* `invalid_argument` if `wcstol`, `wcstoul`, `wcstoll`, or `wcstoull` reports that no conversion could be performed. Throws `out_of_range` if the converted value is outside the range of representable values for the return type.

```
float stof(const wstring& str, size_t* idx = 0);
double stod(const wstring& str, size_t* idx = 0);
long double stold(const wstring& str, size_t* idx = 0);
```

11 *Effects:* These functions call `wcstof(str.c_str(), ptr)`, `wcstod(str.c_str(), ptr)`, and `wcstold(str.c_str(), ptr)`, respectively. Each function returns the converted result, if any. The argument `ptr` designates a pointer to an object internal to the function that is used to determine what to store at `*idx`. If the function does not throw an exception and `idx != 0`, the function stores in `*idx` the index of the first unconverted element of `str`.

12 *Returns:* The converted result.

13 *Throws:* `invalid_argument` if `wcstof`, `wcstod`, or `wcstold` reports that no conversion could be performed. Throws `out_of_range` if `wcstof`, `wcstod`, or `wcstold` sets `errno` to `ERANGE`.

```
wstring to_wstring(int val);
wstring to_wstring(unsigned val);
wstring to_wstring(long val);
wstring to_wstring(unsigned long val);
wstring to_wstring(long long val);
wstring to_wstring(unsigned long long val);
wstring to_wstring(float val);
wstring to_wstring(double val);
wstring to_wstring(long double val);
```

14 *Returns:* Each function returns a `wstring` object holding the character representation of the value of its argument that would be generated by calling `swprintf(buf, buffsz, fmt, val)` with a format specifier of `L"%d"`, `L"%u"`, `L"%ld"`, `L"%lu"`, `L"%lld"`, `L"%llu"`, `L"%f"`, `L"%f"`, or `L"%Lf"`, respectively, where `buf` designates an internal character buffer of sufficient size `buffsz`.

21.6 Hash support [basic.string.hash]

```
template <> struct hash<string>;
template <> struct hash<u16string>;
template <> struct hash<u32string>;
template <> struct hash<wstring>;
```

1 The template specializations shall meet the requirements of class template `hash` (20.9.13).

21.7 Suffix for `basic_string` literals [basic.string.literals]

```
string operator "" s(const char* str, size_t len);
```

1 *Returns:* `string{str,len}`

```
u16string operator "" s(const char16_t* str, size_t len);
```

2 *Returns:* `u16string{str,len}`

```
u32string operator "" s(const char32_t* str, size_t len);
```

3 *Returns:* `u32string{str,len}`

```
wstring operator "" s(const wchar_t* str, size_t len);
```

4 *Returns:* `wstring{str,len}`

5 [*Note:* The same suffix `s` is used for `chrono::duration` literals denoting seconds but there is no conflict, since duration suffixes apply to numbers and string literal suffixes apply to character array literals. — *end note*]

21.8 Null-terminated sequence utilities [c.strings]

1 Tables 73, 74, 75, 76, 77, and 78 describe headers `<cctype>`, `<cwctype>`, `<cstring>`, `<cwchar>`, `<cstdlib>` (character conversions), and `<cuchar>`, respectively.

2 The contents of these headers shall be the same as the Standard C Library headers `<ctype.h>`, `<wctype.h>`, `<string.h>`, `<wchar.h>`, and `<stdlib.h>` and the C Unicode TR header `<uchar.h>`, respectively, with the following modifications:

3 The headers shall not define the types `char16_t`, `char32_t`, and `wchar_t` (2.11).

4 The function signature `strchr(const char*, int)` shall be replaced by the two declarations:

```
const char* strchr(const char* s, int c);
      char* strchr(      char* s, int c);
```

both of which shall have the same behavior as the original declaration.

5 The function signature `strpbrk(const char*, const char*)` shall be replaced by the two declarations:

```
const char* strpbrk(const char* s1, const char* s2);
      char* strpbrk(      char* s1, const char* s2);
```

both of which shall have the same behavior as the original declaration.

6 The function signature `strrchr(const char*, int)` shall be replaced by the two declarations:

```
const char* strrchr(const char* s, int c);
      char* strrchr(      char* s, int c);
```

both of which shall have the same behavior as the original declaration.

7 The function signature `strstr(const char*, const char*)` shall be replaced by the two declarations:

```
const char* strstr(const char* s1, const char* s2);
      char* strstr(      char* s1, const char* s2);
```

both of which shall have the same behavior as the original declaration.

8 The function signature `memchr(const void*, int, size_t)` shall be replaced by the two declarations:

```
const void* memchr(const void* s, int c, size_t n);
      void* memchr(      void* s, int c, size_t n);
```

both of which shall have the same behavior as the original declaration.

9 The function signature `wcschr(const wchar_t*, wchar_t)` shall be replaced by the two declarations:

```
const wchar_t* wcschr(const wchar_t* s, wchar_t c);
      wchar_t* wcschr(      wchar_t* s, wchar_t c);
```

both of which shall have the same behavior as the original declaration.

10 The function signature `wcspbrk(const wchar_t*, const wchar_t*)` shall be replaced by the two declarations:

```
const wchar_t* wcspbrk(const wchar_t* s1, const wchar_t* s2);
      wchar_t* wcspbrk(      wchar_t* s1, const wchar_t* s2);
```

both of which shall have the same behavior as the original declaration.

11 The function signature `wcsrchr(const wchar_t*, wchar_t)` shall be replaced by the two declarations:

```
const wchar_t* wcsrchr(const wchar_t* s, wchar_t c);
      wchar_t* wcsrchr(      wchar_t* s, wchar_t c);
```

both of which shall have the same behavior as the original declaration.

12 The function signature `wcsstr(const wchar_t*, const wchar_t*)` shall be replaced by the two declarations:

```
const wchar_t* wcsstr(const wchar_t* s1, const wchar_t* s2);
      wchar_t* wcsstr(      wchar_t* s1, const wchar_t* s2);
```

both of which shall have the same behavior as the original declaration.

13 The function signature `wmemchr(const wwchar_t*, int, size_t)` shall be replaced by the two declarations:

```
const wchar_t* wmemchr(const wchar_t* s, wchar_t c, size_t n);
      wchar_t* wmemchr(      wchar_t* s, wchar_t c, size_t n);
```

both of which shall have the same behavior as the original declaration.

14 The functions `strerror` and `strtok` are not required to avoid data races (17.6.5.9).

15 Calling the functions listed in Table 72 with an `mbstate_t*` argument of `NULL` may introduce a data race (17.6.5.9) with other calls to these functions with an `mbstate_t*` argument of `NULL`.

Table 72 — Potential `mbstate_t` data races

`mbrlen`	`mbrtowc`	`mbsrtowcs`	`mbtowc`	`wcrtomb`
`wcsrtombs`	`wctomb`			

See also: ISO C 7.3, 7.10.7, 7.10.8, and 7.11. Amendment 1 4.4, 4.5, and 4.6.

Table 73 — Header `<cctype>` synopsis

Type	**Name(s)**			
Functions:				
`isalnum`	`isblank`	`isdigit`	`isprint`	`isupper`
`tolower`	`isalpha`	`isgraph`	`ispunct`	`isxdigit`
`toupper`	`iscntrl`	`islower`	`isspace`	

Table 74 — Header <cwctype> synopsis

Type	**Name(s)**			
Macro:	WEOF			
Types:	wctrans_t	wctype_t	wint_t	
Functions:				
iswalnum	iswctype	iswprint	iswxdigit	wctrans
iswalpha	iswdigit	iswpunct	towctrans	wctype
iswblank	iswgraph	iswspace	towlower	
iswcntrl	iswlower	iswupper	towupper	

Table 75 — Header <cstring> synopsis

Type	**Name(s)**			
Macro:	NULL <cstring>			
Type:	size_t <cstring>			
Functions:				
memchr	strcat	strcspn	strncpy	strtok
memcmp	strchr	strerror	strpbrk	strxfrm
memcpy	strcmp	strlen	strrchr	
memmove	strcoll	strncat	strspn	
memset	strcpy	strncmp	strstr	

Table 76 — Header <cwchar> synopsis

Type	**Name(s)**			
Macros:	NULL	WCHAR_MAX	WCHAR_MIN	WEOF
Types:	mbstate_t	wint_t	size_t	tm
Functions:				
btowc	mbsinit	vwscanf	wcsncpy	wcstoull
fgetwc	mbsrtowcs	wcrtomb	wcspbrk	wcstoul
fgetws	putwchar	wcscat	wcsrchr	wcsxfrm
fputwc	putwc	wcschr	wcsrtombs	wctob
fputws	swprintf	wcscmp	wcsspn	wmemchr
fwide	swscanf	wcscoll	wcsstr	wmemcmp
fwprintf	ungetwc	wcscpy	wcstod	wmemcpy
fwscanf	vfwprintf	wcscspn	wcstof	wmemmove
getwchar	vfwscanf	wcsftime	wcstok	wmemset
getwc	vswprintf	wcslen	wcstold	wprintf
mbrlen	vswscanf	wcsncat	wcstoll	wscanf
mbrtowc	vwprintf	wcsncmp	wcstol	

Table 77 — Header <cstdlib> synopsis

Type	**Name(s)**		
Macros:	MB_CUR_MAX		
Functions:			
atof	mblen	strtof	strtoul
atoi	mbtowc	strtol	strtoull
atol	mbstowcs	strtold	wctomb
atoll	strtod	strtoll	wcstombs

Table 78 — Header `<cuchar>` synopsis

Type	Name(s)
Macros:	`__STDC_UTF_16__` `__STDC_UTF_32__`
Functions:	`mbrtoc16` `c16rtomb` `mbrtoc32` `c32rtomb`

22 Localization library [localization]

22.1 General [localization.general]

1 This Clause describes components that C++ programs may use to encapsulate (and therefore be more portable when confronting) cultural differences. The locale facility includes internationalization support for character classification and string collation, numeric, monetary, and date/time formatting and parsing, and message retrieval.

2 The following subclauses describe components for locales themselves, the standard facets, and facilities from the ISO C library, as summarized in Table 79.

Table 79 — Localization library summary

Subclause		Header(s)
22.3	Locales	`<locale>`
22.4	Standard `locale` Categories	
22.5	Standard code conversion facets	`<codecvt>`
22.6	C library locales	`<clocale>`

22.2 Header `<locale>` synopsis [locale.syn]

```
namespace std {
  // 22.3.1, locale:
  class locale;
  template <class Facet> const Facet& use_facet(const locale&);
  template <class Facet> bool         has_facet(const locale&) noexcept;

  // 22.3.3, convenience interfaces:
  template <class charT> bool isspace (charT c, const locale& loc);
  template <class charT> bool isprint (charT c, const locale& loc);
  template <class charT> bool iscntrl (charT c, const locale& loc);
  template <class charT> bool isupper (charT c, const locale& loc);
  template <class charT> bool islower (charT c, const locale& loc);
  template <class charT> bool isalpha (charT c, const locale& loc);
  template <class charT> bool isdigit (charT c, const locale& loc);
  template <class charT> bool ispunct (charT c, const locale& loc);
  template <class charT> bool isxdigit(charT c, const locale& loc);
  template <class charT> bool isalnum (charT c, const locale& loc);
  template <class charT> bool isgraph (charT c, const locale& loc);
  template <class charT> bool isblank (charT c, const locale& loc);
  template <class charT> charT toupper(charT c, const locale& loc);
  template <class charT> charT tolower(charT c, const locale& loc);
  template <class Codecvt, class Elem = wchar_t,
    class Wide_alloc = std::allocator<Elem>,
    class Byte_alloc = std::allocator<char> > class wstring_convert;
  template <class Codecvt, class Elem = wchar_t,
     class Tr = char_traits<Elem>> class wbuffer_convert;

  // 22.4.1, ctype:
  class ctype_base;
```

```
  template <class charT> class ctype;
  template <>            class ctype<char>;              // specialization
  template <class charT> class ctype_byname;
  class codecvt_base;
  template <class internT, class externT, class stateT> class codecvt;
  template <class internT, class externT, class stateT> class codecvt_byname;

  // 22.4.2, numeric:
  template <class charT, class InputIterator = istreambuf_iterator<charT> >  class num_get;
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> > class num_put;
  template <class charT> class numpunct;
  template <class charT> class numpunct_byname;

  // 22.4.4, collation:
  template <class charT> class collate;
  template <class charT> class collate_byname;

  // 22.4.5, date and time:
  class time_base;
  template <class charT, class InputIterator = istreambuf_iterator<charT> >
    class time_get;
  template <class charT, class InputIterator = istreambuf_iterator<charT> >
    class time_get_byname;
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> >
    class time_put;
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> >
    class time_put_byname;

  // 22.4.6, money:
  class money_base;
  template <class charT, class InputIterator = istreambuf_iterator<charT> >  class money_get;
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> > class money_put;
  template <class charT, bool Intl = false> class moneypunct;
  template <class charT, bool Intl = false> class moneypunct_byname;

  // 22.4.7, message retrieval:
  class messages_base;
  template <class charT> class messages;
  template <class charT> class messages_byname;
}
```

1 The header `<locale>` defines classes and declares functions that encapsulate and manipulate the information peculiar to a locale.[237]

22.3 Locales [locales]

22.3.1 Class `locale` [locale]

```
namespace std {
  class locale {
  public:
    // types:
    class facet;
    class id;
    typedef int category;
```

237) In this subclause, the type name `struct tm` is an incomplete type that is defined in `<ctime>`.

```
    static const category   // values assigned here are for exposition only
      none     = 0,
      collate  = 0x010, ctype    = 0x020,
      monetary = 0x040, numeric  = 0x080,
      time     = 0x100, messages = 0x200,
      all = collate | ctype | monetary | numeric | time  | messages;

    // construct/copy/destroy:
    locale() noexcept;
    locale(const locale& other) noexcept;
    explicit locale(const char* std_name);
    explicit locale(const string& std_name);
    locale(const locale& other, const char* std_name, category);
    locale(const locale& other, const string& std_name, category);
    template <class Facet> locale(const locale& other, Facet* f);
    locale(const locale& other, const locale& one, category);
    ~locale();                  // not virtual
    const locale& operator=(const locale& other) noexcept;
    template <class Facet> locale combine(const locale& other) const;

    // locale operations:
    basic_string<char>                  name() const;

    bool operator==(const locale& other) const;
    bool operator!=(const locale& other) const;

    template <class charT, class traits, class Allocator>
      bool operator()(const basic_string<charT,traits,Allocator>& s1,
                      const basic_string<charT,traits,Allocator>& s2) const;

    // global locale objects:
    static       locale  global(const locale&);
    static const locale& classic();
  };
}
```

1 Class `locale` implements a type-safe polymorphic set of facets, indexed by facet *type*. In other words, a facet has a dual role: in one sense, it's just a class interface; at the same time, it's an index into a locale's set of facets.

2 Access to the facets of a `locale` is via two function templates, `use_facet<>` and `has_facet<>`.

3 [*Example:* An iostream `operator<<` might be implemented as:[238]

```
template <class charT, class traits>
basic_ostream<charT,traits>&
operator<< (basic_ostream<charT,traits>& s, Date d) {
  typename basic_ostream<charT,traits>::sentry cerberos(s);
  if (cerberos) {
    ios_base::iostate err = ios_base::iostate::goodbit;
    tm tmbuf; d.extract(tmbuf);
    use_facet< time_put<charT,ostreambuf_iterator<charT,traits> > >(
      s.getloc()).put(s, s, s.fill(), err, &tmbuf, 'x');
    s.setstate(err);            // might throw
  }
```

238) Note that in the call to `put` the stream is implicitly converted to an `ostreambuf_iterator<charT,traits>`.

```
  return s;
}
```

— *end example*]

4 In the call to `use_facet<Facet>(loc)`, the type argument chooses a facet, making available all members of the named type. If `Facet` is not present in a locale, it throws the standard exception `bad_cast`. A C++ program can check if a locale implements a particular facet with the function template `has_facet<Facet>()`. User-defined facets may be installed in a locale, and used identically as may standard facets (22.4.8).

5 [*Note:* All locale semantics are accessed via `use_facet<>` and `has_facet<>`, except that:

(5.1) — A member operator template `operator()(const basic_string<C, T, A>&, const basic_string< C, T, A>&)` is provided so that a locale may be used as a predicate argument to the standard collections, to collate strings.

(5.2) — Convenient global interfaces are provided for traditional `ctype` functions such as `isdigit()` and `isspace()`, so that given a locale object `loc` a C++ program can call `isspace(c,loc)`. (This eases upgrading existing extractors (27.7.2.2).) — *end note*]

6 Once a facet reference is obtained from a locale object by calling `use_facet<>`, that reference remains usable, and the results from member functions of it may be cached and re-used, as long as some locale object refers to that facet.

7 In successive calls to a locale facet member function on a facet object installed in the same locale, the returned result shall be identical.

8 A `locale` constructed from a name string (such as `"POSIX"`), or from parts of two named locales, has a name; all others do not. Named locales may be compared for equality; an unnamed locale is equal only to (copies of) itself. For an unnamed locale, `locale::name()` returns the string `"*"`.

9 Whether there is one global locale object for the entire program or one global locale object per thread is implementation-defined. Implementations should provide one global locale object per thread. If there is a single global locale object for the entire program, implementations are not required to avoid data races on it (17.6.5.9).

22.3.1.1 `locale` types [locale.types]

22.3.1.1.1 Type `locale::category` [locale.category]

```
typedef int category;
```

1 *Valid* `category` values include the `locale` member bitmask elements `collate`, `ctype`, `monetary`, `numeric`, `time`, and `messages`, each of which represents a single locale category. In addition, `locale` member bitmask constant `none` is defined as zero and represents no category. And `locale` member bitmask constant `all` is defined such that the expression

```
(collate | ctype | monetary | numeric | time | messages | all) == all
```

is `true`, and represents the union of all categories. Further, the expression `(X | Y)`, where `X` and `Y` each represent a single category, represents the union of the two categories.

2 `locale` member functions expecting a `category` argument require one of the `category` values defined above, or the union of two or more such values. Such a `category` value identifies a set of locale categories. Each locale category, in turn, identifies a set of locale facets, including at least those shown in Table 80.

3 For any locale `loc` either constructed, or returned by `locale::classic()`, and any facet `Facet` shown in Table 80, `has_facet<Facet>(loc)` is true. Each `locale` member function which takes a `locale::category` argument operates on the corresponding set of facets.

Table 80 — Locale category facets

Category	Includes facets
collate	`collate<char>`, `collate<wchar_t>`

ctype	`ctype<char>`, `ctype<wchar_t>`
	`codecvt<char,char,mbstate_t>`
	`codecvt<char16_t,char,mbstate_t>`
	`codecvt<char32_t,char,mbstate_t>`
	`codecvt<wchar_t,char,mbstate_t>`

monetary	`moneypunct<char>`, `moneypunct<wchar_t>`
	`moneypunct<char,true>`, `moneypunct<wchar_t,true>`
	`money_get<char>`, `money_get<wchar_t>`
	`money_put<char>`, `money_put<wchar_t>`

numeric	`numpunct<char>`, `numpunct<wchar_t>`
	`num_get<char>`, `num_get<wchar_t>`
	`num_put<char>`, `num_put<wchar_t>`

time	`time_get<char>`, `time_get<wchar_t>` `time_put<char>`, `time_put<wchar_t>`
messages	`messages<char>`, `messages<wchar_t>`

4 An implementation is required to provide those specializations for facet templates identified as members of a category, and for those shown in Table 81.

5 The provided implementation of members of facets `num_get<charT>` and `num_put<charT>` calls `use_facet <F> (l)` only for facet `F` of types `numpunct<charT>` and `ctype<charT>`, and for locale `l` the value obtained by calling member `getloc()` on the `ios_base&` argument to these functions.

6 In declarations of facets, a template parameter with name `InputIterator` or `OutputIterator` indicates the set of all possible specializations on parameters that satisfy the requirements of an Input Iterator or an Output Iterator, respectively (24.2). A template parameter with name `C` represents the set of types containing `char`, `wchar_t`, and any other implementation-defined character types that satisfy the requirements for a character on which any of the iostream components can be instantiated. A template parameter with name `International` represents the set of all possible specializations on a bool parameter.

22.3.1.1.2 Class `locale::facet` [locale.facet]

```
namespace std {
  class locale::facet {
  protected:
    explicit facet(size_t refs = 0);
    virtual ~facet();
    facet(const facet&) = delete;
    void operator=(const facet&) = delete;
  };
}
```

1 Template parameters in this Clause which are required to be facets are those named `Facet` in declarations. A program that passes a type that is *not* a facet, or a type that refers to a volatile-qualified facet, as an (explicit or deduced) template parameter to a locale function expecting a facet, is ill-formed. A const-qualified facet is a valid template argument to any locale function that expects a Facet template parameter.

2 The `refs` argument to the constructor is used for lifetime management.

(2.1) — For `refs == 0`, the implementation performs `delete static_cast<locale::facet*>(f)` (where `f` is a pointer to the facet) when the last `locale` object containing the facet is destroyed; for `refs == 1`,

Table 81 — Required specializations

Category	Includes facets
collate	`collate_byname<char>`, `collate_byname<wchar_t>`

ctype	`ctype_byname<char>`, `ctype_byname<wchar_t>`
	`codecvt_byname<char,char,mbstate_t>`
	`codecvt_byname<char16_t,char,mbstate_t>`
	`codecvt_byname<char32_t,char,mbstate_t>`
	`codecvt_byname<wchar_t,char,mbstate_t>`

monetary	`moneypunct_byname<char,International>`
	`moneypunct_byname<wchar_t,International>`
	`money_get<C,InputIterator>`
	`money_put<C,OutputIterator>`

numeric	`numpunct_byname<char>`, `numpunct_byname<wchar_t>` `num_get<C,InputIterator>`, `num_put<C,OutputIterator>`

time	`time_get<char,InputIterator>`
	`time_get_byname<char,InputIterator>`
	`time_get<wchar_t,InputIterator>`
	`time_get_byname<wchar_t,InputIterator>`
	`time_put<char,OutputIterator>`
	`time_put_byname<char,OutputIterator>`
	`time_put<wchar_t,OutputIterator>`
	`time_put_byname<wchar_t,OutputIterator>`

messages	`messages_byname<char>`, `messages_byname<wchar_t>`

the implementation never destroys the facet.

[3] Constructors of all facets defined in this Clause take such an argument and pass it along to their `facet` base class constructor. All one-argument constructors defined in this Clause are *explicit*, preventing their participation in automatic conversions.

[4] For some standard facets a standard "`..._byname`" class, derived from it, implements the virtual function semantics equivalent to that facet of the locale constructed by `locale(const char*)` with the same name. Each such facet provides a constructor that takes a `const char*` argument, which names the locale, and a `refs` argument, which is passed to the base class constructor. Each such facet also provides a constructor that takes a `string` argument `str` and a `refs` argument, which has the same effect as calling the first constructor with the two arguments `str.c_str()` and `refs`. If there is no "`..._byname`" version of a facet, the base class implements named locale semantics itself by reference to other facets.

22.3.1.1.3 Class `locale::id` [locale.id]

```
namespace std {
  class locale::id {
  public:
    id();
    void operator=(const id&) = delete;
    id(const id&) = delete;
  };
}
```

[1] The class locale::id provides identification of a locale facet interface, used as an index for lookup and to encapsulate initialization.

2 [*Note:* Because facets are used by iostreams, potentially while static constructors are running, their initialization cannot depend on programmed static initialization. One initialization strategy is for `locale` to initialize each facet's `id` member the first time an instance of the facet is installed into a locale. This depends only on static storage being zero before constructors run (3.6.2). — *end note*]

22.3.1.2 `locale` constructors and destructor [locale.cons]

```
locale() noexcept;
```

1 Default constructor: a snapshot of the current global locale.

2 *Effects:* Constructs a copy of the argument last passed to `locale::global(locale&)`, if it has been called; else, the resulting facets have virtual function semantics identical to those of `locale::classic()`. [*Note:* This constructor is commonly used as the default value for arguments of functions that take a `const locale&` argument. — *end note*]

```
locale(const locale& other) noexcept;
```

3 *Effects:* Constructs a locale which is a copy of `other`.

```
explicit locale(const char* std_name);
```

4 *Effects:* Constructs a locale using standard C locale names, e.g., `"POSIX"`. The resulting locale implements semantics defined to be associated with that name.

5 *Throws:* `runtime_error` if the argument is not valid, or is null.

6 *Remarks:* The set of valid string argument values is `"C"`, `""`, and any implementation-defined values.

```
explicit locale(const string& std_name);
```

7 *Effects:* The same as `locale(std_name.c_str())`.

```
locale(const locale& other, const char* std_name, category);
```

8 *Effects:* Constructs a locale as a copy of `other` except for the facets identified by the `category` argument, which instead implement the same semantics as `locale(std_name)`.

9 *Throws:* `runtime_error` if the argument is not valid, or is null.

10 *Remarks:* The locale has a name if and only if `other` has a name.

```
locale(const locale& other, const string& std_name, category cat);
```

11 *Effects:* The same as `locale(other, std_name.c_str(), cat)`.

```
template <class Facet> locale(const locale& other, Facet* f);
```

12 *Effects:* Constructs a locale incorporating all facets from the first argument except that of type `Facet`, and installs the second argument as the remaining facet. If `f` is null, the resulting object is a copy of `other`.

13 *Remarks:* The resulting locale has no name.

```
locale(const locale& other, const locale& one, category cats);
```

14 *Effects:* Constructs a locale incorporating all facets from the first argument except those that implement `cats`, which are instead incorporated from the second argument.

15 *Remarks:* The resulting locale has a name if and only if the first two arguments have names.

```
const locale& operator=(const locale& other) noexcept;
```

16 *Effects:* Creates a copy of `other`, replacing the current value.

17 *Returns:* `*this`

```
~locale();
```

18 A non-virtual destructor that throws no exceptions.

22.3.1.3 `locale` members [locale.members]

```
template <class Facet> locale combine(const locale& other) const;
```

1 *Effects:* Constructs a locale incorporating all facets from `*this` except for that one facet of `other` that is identified by `Facet`.

2 *Returns:* The newly created locale.

3 *Throws:* `runtime_error` if `has_facet<Facet>(other)` is false.

4 *Remarks:* The resulting locale has no name.

```
basic_string<char> name() const;
```

5 *Returns:* The name of `*this`, if it has one; otherwise, the string `"*"`. If `*this` has a name, then `locale(name().c_str())` is equivalent to `*this`. Details of the contents of the resulting string are otherwise implementation-defined.

22.3.1.4 `locale` operators [locale.operators]

```
bool operator==(const locale& other) const;
```

1 *Returns:* `true` if both arguments are the same locale, or one is a copy of the other, or each has a name and the names are identical; `false` otherwise.

```
bool operator!=(const locale& other) const;
```

2 *Returns:* The result of the expression: `!(*this == other)`.

```
template <class charT, class traits, class Allocator>
  bool operator()(const basic_string<charT,traits,Allocator>& s1,
                  const basic_string<charT,traits,Allocator>& s2) const;
```

3 *Effects:* Compares two strings according to the `collate<charT>` facet.

4 *Remarks:* This member operator template (and therefore `locale` itself) satisfies requirements for a comparator predicate template argument (Clause 25) applied to strings.

5 *Returns:* The result of the following expression:

```
use_facet< collate<charT> >(*this).compare
  (s1.data(), s1.data()+s1.size(), s2.data(), s2.data()+s2.size()) < 0;
```

6 [*Example:* A vector of strings `v` can be collated according to collation rules in locale `loc` simply by (25.4.1, 23.3.6):

```
std::sort(v.begin(), v.end(), loc);
```

— *end example*]

22.3.1.5 `locale` static members [locale.statics]

```
static locale global(const locale& loc);
```

1 Sets the global locale to its argument.

2 *Effects:* Causes future calls to the constructor `locale()` to return a copy of the argument. If the argument has a name, does

```
std::setlocale(LC_ALL, loc.name().c_str());
```

otherwise, the effect on the C locale, if any, is implementation-defined. No library function other than `locale::global()` shall affect the value returned by `locale()`. [*Note:* See 22.6 for data race considerations when `setlocale` is invoked. — *end note*]

3 *Returns:* The previous value of `locale()`.

```
static const locale& classic();
```

4 The `"C"` locale.

5 *Returns:* A locale that implements the classic `"C"` locale semantics, equivalent to the value `locale("C")`.

6 *Remarks:* This locale, its facets, and their member functions, do not change with time.

22.3.2 `locale` globals [locale.global.templates]

```
template <class Facet> const Facet& use_facet(const locale& loc);
```

1 *Requires:* `Facet` is a facet class whose definition contains the public static member `id` as defined in 22.3.1.1.2.

2 *Returns:* A reference to the corresponding facet of `loc`, if present.

3 *Throws:* `bad_cast` if `has_facet<Facet>(loc)` is `false`.

4 *Remarks:* The reference returned remains valid at least as long as any copy of `loc` exists.

```
template <class Facet> bool has_facet(const locale& loc) noexcept;
```

5 *Returns:* True if the facet requested is present in `loc`; otherwise false.

22.3.3 Convenience interfaces [locale.convenience]

22.3.3.1 Character classification [classification]

```
template <class charT> bool isspace (charT c, const locale& loc);
template <class charT> bool isprint (charT c, const locale& loc);
template <class charT> bool iscntrl (charT c, const locale& loc);
template <class charT> bool isupper (charT c, const locale& loc);
template <class charT> bool islower (charT c, const locale& loc);
template <class charT> bool isalpha (charT c, const locale& loc);
template <class charT> bool isdigit (charT c, const locale& loc);
template <class charT> bool ispunct (charT c, const locale& loc);
template <class charT> bool isxdigit(charT c, const locale& loc);
template <class charT> bool isalnum (charT c, const locale& loc);
template <class charT> bool isgraph (charT c, const locale& loc);
template <class charT> bool isblank (charT c, const locale& loc);
```

1 Each of these functions `is`***F*** returns the result of the expression:

```
use_facet< ctype<charT> >(loc).is(ctype_base::F, c)
```

where ***F*** is the `ctype_base::mask` value corresponding to that function (22.4.1).[239]

22.3.3.2 Conversions [conversions]

22.3.3.2.1 Character conversions [conversions.character]

```
template <class charT> charT toupper(charT c, const locale& loc);
```

1 *Returns:* `use_facet<ctype<charT> >(loc).toupper(c)`.

```
template <class charT> charT tolower(charT c, const locale& loc);
```

2 *Returns:* `use_facet<ctype<charT> >(loc).tolower(c)`.

22.3.3.2.2 `string` conversions [conversions.string]

1 Class template `wstring_convert` performs conversions between a wide string and a byte string. It lets you specify a code conversion facet (like class template `codecvt`) to perform the conversions, without affecting any streams or locales. [*Example:* If you want to use the code conversion facet `codecvt_utf8` to output to `cout` a UTF-8 multibyte sequence corresponding to a wide string, but you don't want to alter the locale for `cout`, you can write something like:

```
wstring_convert<std::codecvt_utf8<wchar_t>> myconv;
std::string mbstring = myconv.to_bytes(L"Hello\n");
std::cout << mbstring;
```

— *end example*]

2 **Class template `wstring_convert` synopsis**

```
namespace std {
template<class Codecvt, class Elem = wchar_t,
    class Wide_alloc = std::allocator<Elem>,
    class Byte_alloc = std::allocator<char> > class wstring_convert {
  public:
    typedef std::basic_string<char, char_traits<char>, Byte_alloc> byte_string;
    typedef std::basic_string<Elem, char_traits<Elem>, Wide_alloc> wide_string;
    typedef typename Codecvt::state_type state_type;
    typedef typename wide_string::traits_type::int_type int_type;

    explicit wstring_convert(Codecvt* pcvt = new Codecvt);
    wstring_convert(Codecvt* pcvt, state_type state);
    explicit wstring_convert(const byte_string& byte_err,
                             const wide_string& wide_err = wide_string());
    ~wstring_convert();

    wstring_convert(const wstring_convert&) = delete;
    wstring_convert& operator=(const wstring_convert&) = delete;

    wide_string from_bytes(char byte);
    wide_string from_bytes(const char* ptr);
    wide_string from_bytes(const byte_string& str);
    wide_string from_bytes(const char* first, const char* last);

    byte_string to_bytes(Elem wchar);
    byte_string to_bytes(const Elem* wptr);
    byte_string to_bytes(const wide_string& wstr);
    byte_string to_bytes(const Elem* first, const Elem* last);
```

239) When used in a loop, it is faster to cache the `ctype<>` facet and use it directly, or use the vector form of `ctype<>::is`.

```
    size_t converted() const noexcept;
    state_type state() const;
  private:
    byte_string byte_err_string;    // exposition only
    wide_string wide_err_string;    // exposition only
    Codecvt* cvtptr;                // exposition only
    state_type cvtstate;            // exposition only
    size_t cvtcount;                // exposition only
  };
}
```

3 The class template describes an object that controls conversions between wide string objects of class `std::basic_string<Elem, char_traits<Elem>, Wide_alloc>` and byte string objects of class `std::basic_string<char, char_traits<char>, Byte_alloc>`. The class template defines the types `wide_string` and `byte_string` as synonyms for these two types. Conversion between a sequence of `Elem` values (stored in a `wide_string` object) and multibyte sequences (stored in a `byte_string` object) is performed by an object of class `Codecvt`, which meets the requirements of the standard code-conversion facet `std::codecvt<Elem, char, std::mbstate_t>`.

4 An object of this class template stores:

(4.1) — `byte_err_string` — a byte string to display on errors

(4.2) — `wide_err_string` — a wide string to display on errors

(4.3) — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wstring_convert` object is destroyed)

(4.4) — `cvtstate` — a conversion state object

(4.5) — `cvtcount` — a conversion count

```
typedef std::basic_string<char, char_traits<char>, Byte_alloc> byte_string;
```

5 The type shall be a synonym for `std::basic_string<char, char_traits<char>, Byte_alloc>`

```
size_t converted() const noexcept;
```

6 *Returns:* `cvtcount`.

```
wide_string from_bytes(char byte);
wide_string from_bytes(const char* ptr);
wide_string from_bytes(const byte_string& str);
wide_string from_bytes(const char* first, const char* last);
```

7 *Effects:* The first member function shall convert the single-element sequence `byte` to a wide string. The second member function shall convert the null-terminated sequence beginning at `ptr` to a wide string. The third member function shall convert the sequence stored in `str` to a wide string. The fourth member function shall convert the sequence defined by the range `[first,last)` to a wide string.

8 In all cases:

(8.1) — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.

(8.2) — The number of input elements successfully converted shall be stored in `cvtcount`.

9 *Returns:* If no conversion error occurs, the member function shall return the converted wide string. Otherwise, if the object was constructed with a wide-error string, the member function shall return the wide-error string. Otherwise, the member function throws an object of class `std::range_error`.

```
typedef typename wide_string::traits_type::int_type int_type;
```

The type shall be a synonym for `wide_string::traits_type::int_type`.

```
state_type state() const;
```

10 returns `cvtstate`.

```
typedef typename Codecvt::state_type state_type;
```

11 The type shall be a synonym for `Codecvt::state_type`.

```
byte_string to_bytes(Elem wchar);
byte_string to_bytes(const Elem* wptr);
byte_string to_bytes(const wide_string& wstr);
byte_string to_bytes(const Elem* first, const Elem* last);
```

12 *Effects:* The first member function shall convert the single-element sequence `wchar` to a byte string. The second member function shall convert the null-terminated sequence beginning at `wptr` to a byte string. The third member function shall convert the sequence stored in `wstr` to a byte string. The fourth member function shall convert the sequence defined by the range `[first,last)` to a byte string.

13 In all cases:

(13.1) — If the `cvtstate` object was not constructed with an explicit value, it shall be set to its default value (the initial conversion state) before the conversion begins. Otherwise it shall be left unchanged.

(13.2) — The number of input elements successfully converted shall be stored in `cvtcount`.

14 *Returns:* If no conversion error occurs, the member function shall return the converted byte string. Otherwise, if the object was constructed with a byte-error string, the member function shall return the byte-error string. Otherwise, the member function shall throw an object of class `std::range_error`.

```
typedef std::basic_string<Elem, char_traits<Elem>, Wide_alloc> wide_string;
```

15 The type shall be a synonym for `std::basic_string<Elem, char_traits<Elem>, Wide_alloc>`.

```
explicit wstring_convert(Codecvt* pcvt = new Codecvt);
wstring_convert(Codecvt* pcvt, state_type state);
explicit wstring_convert(const byte_string& byte_err,
    const wide_string& wide_err = wide_string());
```

16 *Requires:* For the first and second constructors, `pcvt != nullptr`.

17 *Effects:* The first constructor shall store `pcvt` in `cvtptr` and default values in `cvtstate`, `byte_err_string`, and `wide_err_string`. The second constructor shall store `pcvt` in `cvtptr`, `state` in `cvtstate`, and default values in `byte_err_string` and `wide_err_string`; moreover the stored state shall be retained between calls to `from_bytes` and `to_bytes`. The third constructor shall store `new Codecvt` in `cvtptr`, `state_type()` in `cvtstate`, `byte_err` in `byte_err_string`, and `wide_err` in `wide_err_string`.

```
~wstring_convert();
```

18 *Effects:* The destructor shall delete `cvtptr`.

22.3.3.2.3 Buffer conversions [conversions.buffer]

1 Class template `wbuffer_convert` looks like a wide stream buffer, but performs all its I/O through an underlying byte stream buffer that you specify when you construct it. Like class template `wstring_convert`, it lets you specify a code conversion facet to perform the conversions, without affecting any streams or locales.

2 **Class template `wbuffer_convert` synopsis**

```
namespace std {
template<class Codecvt,
    class Elem = wchar_t,
    class Tr = std::char_traits<Elem> >
  class wbuffer_convert
    : public std::basic_streambuf<Elem, Tr> {
  public:
    typedef typename Codecvt::state_type state_type;

    explicit wbuffer_convert(std::streambuf* bytebuf = 0,
                             Codecvt* pcvt = new Codecvt,
                             state_type state = state_type());

    ~wbuffer_convert();

    wbuffer_convert(const wbuffer_convert&) = delete;
    wbuffer_convert& operator=(const wbuffer_convert&) = delete;

    std::streambuf* rdbuf() const;
    std::streambuf* rdbuf(std::streambuf* bytebuf);

    state_type state() const;

  private:
    std::streambuf* bufptr;          // exposition only
    Codecvt* cvtptr;                 // exposition only
    state_type cvtstate;             // exposition only
  };
}
```

3 The class template describes a stream buffer that controls the transmission of elements of type `Elem`, whose character traits are described by the class `Tr`, to and from a byte stream buffer of type `std::streambuf`. Conversion between a sequence of `Elem` values and multibyte sequences is performed by an object of class `Codecvt`, which shall meet the requirements of the standard code-conversion facet `std::codecvt<Elem, char, std::mbstate_t>`.

4 An object of this class template stores:

(4.1) — `bufptr` — a pointer to its underlying byte stream buffer

(4.2) — `cvtptr` — a pointer to the allocated conversion object (which is freed when the `wbuffer_convert` object is destroyed)

(4.3) — `cvtstate` — a conversion state object

```
state_type state() const;
```

5 *Returns:* `cvtstate`.

```
std::streambuf* rdbuf() const;
```

6 *Returns:* `bufptr`.

```
std::streambuf* rdbuf(std::streambuf* bytebuf);
```

7 *Effects:* stores `bytebuf` in `bufptr`.

8 *Returns:* The previous value of `bufptr`.

```
typedef typename Codecvt::state_type state_type;
```

9 The type shall be a synonym for `Codecvt::state_type`.

```
explicit wbuffer_convert(std::streambuf* bytebuf = 0,
    Codecvt* pcvt = new Codecvt, state_type state = state_type());
```

10 *Requires:* `pcvt != nullptr`.

11 *Effects:* The constructor constructs a stream buffer object, initializes `bufptr` to `bytebuf`, initializes `cvtptr` to `pcvt`, and initializes `cvtstate` to `state`.

```
~wbuffer_convert();
```

12 *Effects:* The destructor shall delete `cvtptr`.

22.4 Standard `locale` categories [locale.categories]

1 Each of the standard categories includes a family of facets. Some of these implement formatting or parsing of a datum, for use by standard or users' iostream operators `<<` and `>>`, as members `put()` and `get()`, respectively. Each such member function takes an `ios_base&` argument whose members `flags()`, `precision()`, and `width()`, specify the format of the corresponding datum (27.5.3). Those functions which need to use other facets call its member `getloc()` to retrieve the locale imbued there. Formatting facets use the character argument `fill` to fill out the specified width where necessary.

2 The `put()` members make no provision for error reporting. (Any failures of the OutputIterator argument must be extracted from the returned iterator.) The `get()` members take an `ios_base::iostate&` argument whose value they ignore, but set to `ios_base::failbit` in case of a parse error.

3 Within this clause it is unspecified whether one virtual function calls another virtual function.

22.4.1 The `ctype` category [category.ctype]

```
namespace std {
  class ctype_base {
  public:
    typedef T mask;

    // numeric values are for exposition only.
    static const mask space = 1 << 0;
    static const mask print = 1 << 1;
    static const mask cntrl = 1 << 2;
    static const mask upper = 1 << 3;
    static const mask lower = 1 << 4;
    static const mask alpha = 1 << 5;
    static const mask digit = 1 << 6;
    static const mask punct = 1 << 7;
    static const mask xdigit = 1 << 8;
    static const mask blank = 1 << 9;
    static const mask alnum = alpha | digit;
    static const mask graph = alnum | punct;
  };
}
```

1 The type `mask` is a bitmask type (17.5.2.1.3).

22.4.1.1 Class template `ctype` [locale.ctype]

```
namespace std {
  template <class charT>
  class ctype : public locale::facet, public ctype_base {
  public:
    typedef charT char_type;

    explicit ctype(size_t refs = 0);

    bool         is(mask m, charT c) const;
    const charT* is(const charT* low, const charT* high, mask* vec) const;
    const charT* scan_is(mask m,
                         const charT* low, const charT* high) const;
    const charT* scan_not(mask m,
                          const charT* low, const charT* high) const;
    charT        toupper(charT c) const;
    const charT* toupper(charT* low, const charT* high) const;
    charT        tolower(charT c) const;
    const charT* tolower(charT* low, const charT* high) const;

    charT        widen(char c) const;
    const char*  widen(const char* low, const char* high, charT* to) const;
    char         narrow(charT c, char dfault) const;
    const charT* narrow(const charT* low, const charT*, char dfault,
                        char* to) const;

    static locale::id id;

  protected:
   ~ctype();
    virtual bool         do_is(mask m, charT c) const;
    virtual const charT* do_is(const charT* low, const charT* high,
                               mask* vec) const;
    virtual const charT* do_scan_is(mask m,
                                    const charT* low, const charT* high) const;
    virtual const charT* do_scan_not(mask m,
                                     const charT* low, const charT* high) const;
    virtual charT        do_toupper(charT) const;
    virtual const charT* do_toupper(charT* low, const charT* high) const;
    virtual charT        do_tolower(charT) const;
    virtual const charT* do_tolower(charT* low, const charT* high) const;
    virtual charT        do_widen(char) const;
    virtual const char*  do_widen(const char* low, const char* high,
                                  charT* dest) const;
    virtual char         do_narrow(charT, char dfault) const;
    virtual const charT* do_narrow(const charT* low, const charT* high,
                                   char dfault, char* dest) const;
  };
}
```

1 Class `ctype` encapsulates the C library `<cctype>` features. `istream` members are required to use `ctype<>` for character classing during input parsing.

2 The specializations required in Table 80 (22.3.1.1.1), namely `ctype<char>` and `ctype<wchar_t>`, implement

character classing appropriate to the implementation's native character set.

22.4.1.1.1 `ctype` members [locale.ctype.members]

```
bool         is(mask m, charT c) const;
const charT* is(const charT* low, const charT* high,
                mask* vec) const;
```

1 *Returns:* `do_is(m,c)` or `do_is(low,high,vec)`

```
const charT* scan_is(mask m,
                     const charT* low, const charT* high) const;
```

2 *Returns:* `do_scan_is(m,low,high)`

```
const charT* scan_not(mask m,
                      const charT* low, const charT* high) const;
```

3 *Returns:* `do_scan_not(m,low,high)`

```
charT        toupper(charT) const;
const charT* toupper(charT* low, const charT* high) const;
```

4 *Returns:* `do_toupper(c)` or `do_toupper(low,high)`

```
charT        tolower(charT c) const;
const charT* tolower(charT* low, const charT* high) const;
```

5 *Returns:* `do_tolower(c)` or `do_tolower(low,high)`

```
charT       widen(char c) const;
const char* widen(const char* low, const char* high, charT* to) const;
```

6 *Returns:* `do_widen(c)` or `do_widen(low,high,to)`

```
char         narrow(charT c, char dfault) const;
const charT* narrow(const charT* low, const charT*, char dfault,
                    char* to) const;
```

7 *Returns:* `do_narrow(c,dfault)` or `do_narrow(low,high,dfault,to)`

22.4.1.1.2 `ctype` virtual functions [locale.ctype.virtuals]

```
bool         do_is(mask m, charT c) const;
const charT* do_is(const charT* low, const charT* high,
                   mask* vec) const;
```

1 *Effects:* Classifies a character or sequence of characters. For each argument character, identifies a value `M` of type `ctype_base::mask`. The second form identifies a value `M` of type `ctype_base::mask` for each `*p` where `(low<=p && p<high)`, and places it into `vec[p-low]`.

2 *Returns:* The first form returns the result of the expression `(M & m) != 0`; i.e., `true` if the character has the characteristics specified. The second form returns `high`.

```
const charT* do_scan_is(mask m,
                        const charT* low, const charT* high) const;
```

3 *Effects:* Locates a character in a buffer that conforms to a classification `m`.

4 *Returns:* The smallest pointer `p` in the range `[low, high)` such that `is(m,*p)` would return `true`; otherwise, returns `high`.

```
const charT* do_scan_not(mask m,
                         const charT* low, const charT* high) const;
```

Effects: Locates a character in a buffer that fails to conform to a classification `m`.

Returns: The smallest pointer `p`, if any, in the range `[low,high)` such that `is(m,*p)` would return `false`; otherwise, returns `high`.

```
charT        do_toupper(charT c) const;
const charT* do_toupper(charT* low, const charT* high) const;
```

Effects: Converts a character or characters to upper case. The second form replaces each character `*p` in the range `[low,high)` for which a corresponding upper-case character exists, with that character.

Returns: The first form returns the corresponding upper-case character if it is known to exist, or its argument if not. The second form returns `high`.

```
charT        do_tolower(charT c) const;
const charT* do_tolower(charT* low, const charT* high) const;
```

Effects: Converts a character or characters to lower case. The second form replaces each character `*p` in the range `[low,high)` and for which a corresponding lower-case character exists, with that character.

Returns: The first form returns the corresponding lower-case character if it is known to exist, or its argument if not. The second form returns `high`.

```
charT        do_widen(char c) const;
const char*  do_widen(const char* low, const char* high,
                      charT* dest) const;
```

Effects: Applies the simplest reasonable transformation from a `char` value or sequence of `char` values to the corresponding `charT` value or values.[240] The only characters for which unique transformations are required are those in the basic source character set (2.3).

For any named `ctype` category with a `ctype<charT>` facet `ctc` and valid `ctype_base::mask` value `M`, `(ctc.is(M, c) || !is(M, do_widen(c)) )` is `true`.[241]

The second form transforms each character `*p` in the range `[low,high)`, placing the result in `dest[p-low]`.

Returns: The first form returns the transformed value. The second form returns `high`.

```
char         do_narrow(charT c, char dfault) const;
const charT* do_narrow(const charT* low, const charT* high,
                       char dfault, char* dest) const;
```

Effects: Applies the simplest reasonable transformation from a `charT` value or sequence of `charT` values to the corresponding `char` value or values.

For any character `c` in the basic source character set (2.3) the transformation is such that

```
do_widen(do_narrow(c,0)) == c
```

For any named `ctype` category with a `ctype<char>` facet `ctc` however, and `ctype_base::mask` value `M`,

```
(is(M,c) || !ctc.is(M, do_narrow(c,dfault)) )
```

240) The char argument of `do_widen` is intended to accept values derived from character literals for conversion to the locale's encoding.

241) In other words, the transformed character is not a member of any character classification that `c` is not also a member of.

is `true` (unless `do_narrow` returns `dfault`). In addition, for any digit character `c`, the expression `(do_narrow(c, dfault) - '0')` evaluates to the digit value of the character. The second form transforms each character `*p` in the range `[low,high)`, placing the result (or `dfault` if no simple transformation is readily available) in `dest[p-low]`.

14 *Returns:* The first form returns the transformed value; or `dfault` if no mapping is readily available. The second form returns `high`.

22.4.1.2 Class template `ctype_byname` [locale.ctype.byname]

```
namespace std {
  template <class charT>
  class ctype_byname : public ctype<charT> {
  public:
    typedef typename ctype<charT>::mask mask;
    explicit ctype_byname(const char*, size_t refs = 0);
    explicit ctype_byname(const string&, size_t refs = 0);
  protected:
   ~ctype_byname();
  };
}
```

22.4.1.3 `ctype` specializations [facet.ctype.special]

```
namespace std {
  template <> class ctype<char>
    : public locale::facet, public ctype_base {
  public:
    typedef char char_type;

    explicit ctype(const mask* tab = 0, bool del = false,
                   size_t refs = 0);

    bool is(mask m, char c) const;
    const char* is(const char* low, const char* high, mask* vec) const;
    const char* scan_is (mask m,
                         const char* low, const char* high) const;
    const char* scan_not(mask m,
                         const char* low, const char* high) const;

    char        toupper(char c) const;
    const char* toupper(char* low, const char* high) const;
    char        tolower(char c) const;
    const char* tolower(char* low, const char* high) const;

    char  widen(char c) const;
    const char* widen(const char* low, const char* high, char* to) const;
    char  narrow(char c, char dfault) const;
    const char* narrow(const char* low, const char* high, char dfault,
                       char* to) const;

    static locale::id id;
    static const size_t table_size = implementation-defined;

    const mask* table() const noexcept;
    static const mask* classic_table() noexcept;
```

```
  protected:
   ~ctype();
    virtual char        do_toupper(char c) const;
    virtual const char* do_toupper(char* low, const char* high) const;
    virtual char        do_tolower(char c) const;
    virtual const char* do_tolower(char* low, const char* high) const;

    virtual char        do_widen(char c) const;
    virtual const char* do_widen(const char* low,
                                 const char* high,
                                 char* to) const;
    virtual char        do_narrow(char c, char dfault) const;
    virtual const char* do_narrow(const char* low,
                                  const char* high,
                                  char dfault, char* to) const;
  };
}
```

1 A specialization `ctype<char>` is provided so that the member functions on type `char` can be implemented `inline`.[242] The implementation-defined value of member `table_size` is at least 256.

22.4.1.3.1 `ctype<char>` destructor [facet.ctype.char.dtor]

```
~ctype();
```

1 *Effects:* If the constructor's first argument was nonzero, and its second argument was true, does `delete [] table()`.

22.4.1.3.2 `ctype<char>` members [facet.ctype.char.members]

1 In the following member descriptions, for `unsigned char` values `v` where `v >= table_size`, `table()[v]` is assumed to have an implementation-specific value (possibly different for each such value `v`) without performing the array lookup.

```
explicit ctype(const mask* tbl = 0, bool del = false,
               size_t refs = 0);
```

2 *Requires:* `tbl` either 0 or an array of at least `table_size` elements.

3 *Effects:* Passes its `refs` argument to its base class constructor.

```
bool        is(mask m, char c) const;
const char* is(const char* low, const char* high,
               mask* vec) const;
```

4 *Effects:* The second form, for all `*p` in the range `[low,high)`, assigns into `vec[p-low]` the value `table()[(unsigned char)*p]`.

5 *Returns:* The first form returns `table()[(unsigned char)c] & m`; the second form returns `high`.

```
const char* scan_is(mask m,
                    const char* low, const char* high) const;
```

6 *Returns:* The smallest `p` in the range `[low,high)` such that

```
table()[(unsigned char) *p] & m
```

is `true`.

242) Only the `char` (not `unsigned char` and `signed char`) form is provided. The specialization is specified in the standard, and not left as an implementation detail, because it affects the derivation interface for `ctype<char>`.

```
const char* scan_not(mask m,
                     const char* low, const char* high) const;
```

7 *Returns:* The smallest `p` in the range `[low,high)` such that

```
table()[(unsigned char) *p] & m
```

is `false`.

```
char        toupper(char c) const;
const char* toupper(char* low, const char* high) const;
```

8 *Returns:* `do_toupper(c)` or `do_toupper(low,high)`, respectively.

```
char        tolower(char c) const;
const char* tolower(char* low, const char* high) const;
```

9 *Returns:* `do_tolower(c)` or `do_tolower(low,high)`, respectively.

```
char  widen(char c) const;
const char* widen(const char* low, const char* high,
    char* to) const;
```

10 *Returns:* `do_widen(c)` or `do_widen(low, high, to)`, respectively.

```
char        narrow(char c, char dfault) const;
const char* narrow(const char* low, const char* high,
                   char dfault, char* to) const;
```

11 *Returns:* `do_narrow(c, dfault)` or `do_narrow(low, high, dfault, to)`, respectively.

```
const mask* table() const noexcept;
```

12 *Returns:* The first constructor argument, if it was non-zero, otherwise `classic_table()`.

22.4.1.3.3 `ctype<char>` static members [facet.ctype.char.statics]

```
static const mask* classic_table() noexcept;
```

1 *Returns:* A pointer to the initial element of an array of size `table_size` which represents the classifications of characters in the "C" locale.

22.4.1.3.4 `ctype<char>` virtual functions [facet.ctype.char.virtuals]

```
char        do_toupper(char) const;
const char* do_toupper(char* low, const char* high) const;
char        do_tolower(char) const;
const char* do_tolower(char* low, const char* high) const;

virtual char        do_widen(char c) const;
virtual const char* do_widen(const char* low,
                             const char* high,
                             char* to) const;
virtual char        do_narrow(char c, char dfault) const;
virtual const char* do_narrow(const char* low,
                              const char* high,
                              char dfault, char* to) const;
```

These functions are described identically as those members of the same name in the `ctype` class template (22.4.1.1.1).

22.4.1.4 Class template `codecvt` [locale.codecvt]

```
namespace std {
  class codecvt_base {
  public:
    enum result { ok, partial, error, noconv };
  };

  template <class internT, class externT, class stateT>
  class codecvt : public locale::facet, public codecvt_base {
  public:
    typedef internT  intern_type;
    typedef externT  extern_type;
    typedef stateT state_type;

    explicit codecvt(size_t refs = 0);

    result out(stateT& state,
               const internT* from, const internT* from_end, const internT*& from_next,
               externT*   to,       externT* to_end, externT*& to_next) const;
    result unshift(stateT& state,
                   externT*   to,        externT* to_end, externT*& to_next) const;
    result in(stateT& state,
              const externT* from, const externT* from_end, const externT*& from_next,
              internT*   to,       internT* to_end, internT*& to_next) const;
    int encoding() const noexcept;
    bool always_noconv() const noexcept;
    int length(stateT&, const externT* from, const externT* end,
               size_t max) const;
    int max_length() const noexcept;

    static locale::id id;

  protected:
    ~codecvt();
    virtual result do_out(stateT& state,
                          const internT* from, const internT* from_end, const internT*& from_next,
                          externT* to,         externT* to_end, externT*& to_next) const;
    virtual result do_in(stateT& state,
                         const externT* from, const externT* from_end, const externT*& from_next,
                         internT* to,         internT* to_end, internT*& to_next) const;
    virtual result do_unshift(stateT& state,
                              externT* to,         externT* to_end, externT*& to_next) const;
    virtual int do_encoding() const noexcept;
    virtual bool do_always_noconv() const noexcept;
    virtual int do_length(stateT&, const externT* from,
                          const externT* end, size_t max) const;
    virtual int do_max_length() const noexcept;
  };
}
```

1 The class `codecvt<internT,externT,stateT>` is for use when converting from one character encoding to another, such as from wide characters to multibyte characters or between wide character encodings such as Unicode and EUC.

2 The `stateT` argument selects the pair of character encodings being mapped between.

3 The specializations required in Table 80 (22.3.1.1.1) convert the implementation-defined native character set. `codecvt<char, char, mbstate_t>` implements a degenerate conversion; it does not convert at all. The specialization `codecvt<char16_t, char, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specialization `codecvt <char32_t, char, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms. `codecvt<wchar_t,char,mbstate_t>` converts between the native character sets for narrow and wide characters. Specializations on `mbstate_t` perform conversion between encodings known to the library implementer. Other encodings can be converted by specializing on a user-defined `stateT` type. Objects of type `stateT` can contain any state that is useful to communicate to or from the specialized `do_in` or `do_out` members.

22.4.1.4.1 `codecvt` members [locale.codecvt.members]

```
result out(stateT& state,
  const internT* from, const internT* from_end, const internT*& from_next,
        externT* to, externT* to_end, externT*& to_next) const;
```

1 *Returns:* `do_out(state, from, from_end, from_next, to, to_end, to_next)`

```
result unshift(stateT& state,
        externT* to, externT* to_end, externT*& to_next) const;
```

2 *Returns:* `do_unshift(state, to, to_end, to_next)`

```
result in(stateT& state,
  const externT* from, const externT* from_end, const externT*& from_next,
        internT* to, internT* to_end, internT*& to_next) const;
```

3 *Returns:* `do_in(state, from, from_end, from_next, to, to_end, to_next)`

```
int encoding() const noexcept;
```

4 *Returns:* `do_encoding()`

```
bool always_noconv() const noexcept;
```

5 *Returns:* `do_always_noconv()`

```
int length(stateT& state, const externT* from, const externT* from_end,
           size_t max) const;
```

6 *Returns:* `do_length(state, from,from_end,max)`

```
int max_length() const noexcept;
```

7 *Returns:* `do_max_length()`

22.4.1.4.2 `codecvt` virtual functions [locale.codecvt.virtuals]

```
result do_out(stateT& state,
  const internT* from, const internT* from_end, const internT*& from_next,
  externT* to, externT* to_end, externT*& to_next) const;

result do_in(stateT& state,
  const externT* from, const externT* from_end, const externT*& from_next,
        internT* to, internT* to_end, internT*& to_next) const;
```

1 *Requires:* `(from<=from_end && to<=to_end)` well-defined and `true`; `state` initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

2 *Effects:* Translates characters in the source range `[from,from_end)`, placing the results in sequential positions starting at destination `to`. Converts no more than `(from_end-from)` source elements, and stores no more than `(to_end-to)` destination elements.

Stops if it encounters a character it cannot convert. It always leaves the `from_next` and `to_next` pointers pointing one beyond the last element successfully converted. If returns `noconv`, `internT` and `externT` are the same type and the converted sequence is identical to the input sequence `[from, from_next)`. `to_next` is set equal to `to`, the value of `state` is unchanged, and there are no changes to the values in `[to, to_end)`.

3 A `codecvt` facet that is used by `basic_filebuf` (27.9) shall have the property that if

```
do_out(state, from, from_end, from_next, to, to_end, to_next)
```

would return `ok`, where `from != from_end`, then

```
do_out(state, from, from + 1, from_next, to, to_end, to_next)
```

shall also return `ok`, and that if

```
do_in(state, from, from_end, from_next, to, to_end, to_next)
```

would return `ok`, where `to != to_end`, then

```
do_in(state, from, from_end, from_next, to, to + 1, to_next)
```

shall also return `ok`.[243] [*Note:* As a result of operations on `state`, it can return `ok` or `partial` and set `from_next == from` and `to_next != to`. — *end note*]

4 *Remarks:* Its operations on `state` are unspecified. [*Note:* This argument can be used, for example, to maintain shift state, to specify conversion options (such as count only), or to identify a cache of seek offsets. — *end note*]

5 *Returns:* An enumeration value, as summarized in Table 82.

Table 82 — `do_in/do_out` result values

Value	Meaning
`ok`	completed the conversion
`partial`	not all source characters converted
`error`	encountered a character in `[from,from_end)` that it could not convert
`noconv`	`internT` and `externT` are the same type, and input sequence is identical to converted sequence

A return value of `partial`, if `(from_next==from_end)`, indicates that either the destination sequence has not absorbed all the available destination elements, or that additional source elements are needed before another destination element can be produced.

```
result do_unshift(stateT& state,
  externT* to, externT* to_end, externT*& to_next) const;
```

6 *Requires:* `(to <= to_end)` well defined and true; state initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

243) Informally, this means that `basic_filebuf` assumes that the mappings from internal to external characters is 1 to N: a `codecvt` facet that is used by `basic_filebuf` must be able to translate characters one internal character at a time.

7 *Effects:* Places characters starting at `to` that should be appended to terminate a sequence when the current `stateT` is given by `state`.[244] Stores no more than `(to_end-to)` destination elements, and leaves the `to_next` pointer pointing one beyond the last element successfully stored.

8 *Returns:* An enumeration value, as summarized in Table 83.

Table 83 — `do_unshift` result values

Value	Meaning
`ok`	completed the sequence
`partial`	space for more than `to_end-to` destination elements was needed to terminate a sequence given the value of `state`
`error`	an unspecified error has occurred
`noconv`	no termination is needed for this `state_type`

```
int do_encoding() const noexcept;
```

9 *Returns:* `-1` if the encoding of the `externT` sequence is state-dependent; else the constant number of `externT` characters needed to produce an internal character; or `0` if this number is not a constant.[245]

```
bool do_always_noconv() const noexcept;
```

10 *Returns:* `true` if `do_in()` and `do_out()` return `noconv` for all valid argument values. `codecvt<char, char, mbstate_t>` returns `true`.

```
int do_length(stateT& state, const externT* from, const externT* from_end,
              size_t max) const;
```

11 *Requires:* `(from<=from_end)` well-defined and `true`; `state` initialized, if at the beginning of a sequence, or else equal to the result of converting the preceding characters in the sequence.

12 *Effects:* The effect on the `state` argument is "as if" it called `do_in(state, from, from_end, from, to, to+max, to)` for `to` pointing to a buffer of at least `max` elements.

13 *Returns:* `(from_next-from)` where `from_next` is the largest value in the range `[from,from_end]` such that the sequence of values in the range `[from,from_next)` represents `max` or fewer valid complete characters of type `internT`. The specialization `codecvt<char, char, mbstate_t>`, returns the lesser of `max` and `(from_end-from)`.

```
int do_max_length() const noexcept;
```

14 *Returns:* The maximum value that `do_length(state, from, from_end, 1)` can return for any valid range `[from, from_end)` and `stateT` value `state`. The specialization `codecvt<char, char, mbstate_t>::do_max_length()` returns 1.

22.4.1.5 Class template `codecvt_byname` [locale.codecvt.byname]

```
namespace std {
  template <class internT, class externT, class stateT>
  class codecvt_byname : public codecvt<internT, externT, stateT> {
  public:
```

244) Typically these will be characters to return the state to `stateT()`

245) If `encoding()` yields `-1`, then more than `max_length()` `externT` elements may be consumed when producing a single `internT` character, and additional `externT` elements may appear at the end of a sequence after those that yield the final `internT` character.

```
    explicit codecvt_byname(const char*, size_t refs = 0);
    explicit codecvt_byname(const string&, size_t refs = 0);
  protected:
    ~codecvt_byname();
  };
}
```

22.4.2 The numeric category [category.numeric]

1 The classes `num_get<>` and `num_put<>` handle numeric formatting and parsing. Virtual functions are provided for several numeric types. Implementations may (but are not required to) delegate extraction of smaller types to extractors for larger types.[246]

2 All specifications of member functions for `num_put` and `num_get` in the subclauses of 22.4.2 only apply to the specializations required in Tables 80 and 81 (22.3.1.1.1), namely `num_get<char>`, `num_get<wchar_t>`, `num_get<C, InputIterator>`, `num_put<char>`, `num_put<wchar_t>`, and `num_put<C,OutputIterator>`. These specializations refer to the `ios_base&` argument for formatting specifications (22.4), and to its imbued locale for the `numpunct<>` facet to identify all numeric punctuation preferences, and also for the `ctype<>` facet to perform character classification.

3 Extractor and inserter members of the standard iostreams use `num_get<>` and `num_put<>` member functions for formatting and parsing numeric values (27.7.2.2.1, 27.7.3.6.1).

22.4.2.1 Class template `num_get` [locale.num.get]

```
namespace std {
  template <class charT, class InputIterator = istreambuf_iterator<charT> >
  class num_get : public locale::facet {
  public:
    typedef charT           char_type;
    typedef InputIterator   iter_type;

    explicit num_get(size_t refs = 0);

    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, bool& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, long& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, long long& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, unsigned short& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, unsigned int& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, unsigned long& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, unsigned long long& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, float& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, double& v) const;
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, long double& v) const;
```

246) Parsing `"-1"` correctly into, e.g., an `unsigned short` requires that the corresponding member `get()` at least extract the sign before delegating.

```
    iter_type get(iter_type in, iter_type end, ios_base&,
                  ios_base::iostate& err, void*& v) const;

    static locale::id id;

  protected:
    ~num_get();
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, bool& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, long& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, long long& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, unsigned short& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, unsigned int& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, unsigned long& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, unsigned long long& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, float& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, double& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, long double& v) const;
    virtual iter_type do_get(iter_type, iter_type, ios_base&,
                             ios_base::iostate& err, void*& v) const;
  };
}
```

1 The facet `num_get` is used to parse numeric values from an input sequence such as an istream.

22.4.2.1.1 `num_get` members [facet.num.get.members]

```
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, bool& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned short& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned int& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned long long& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, float& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, double& val) const;
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long double& val) const;
```

```
iter_type get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, void*& val) const;
```

1 *Returns:* `do_get(in, end, str, err, val)`.

22.4.2.1.2 `num_get` virtual functions [facet.num.get.virtuals]

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned short& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned int& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, unsigned long long& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, float& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, double& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, long double& val) const;
iter_type do_get(iter_type in, iter_type end, ios_base& str,
  ios_base::iostate& err, void*& val) const;
```

1 *Effects:* Reads characters from `in`, interpreting them according to `str.flags()`, `use_facet<ctype< charT> >(loc)`, and `use_facet< numpunct<charT> >(loc)`, where `loc` is `str.getloc()`.

2 The details of this operation occur in three stages

(2.1) — Stage 1: Determine a conversion specifier

(2.2) — Stage 2: Extract characters from `in` and determine a corresponding `char` value for the format expected by the conversion specification determined in stage 1.

(2.3) — Stage 3: Store results

3 The details of the stages are presented below.

Stage 1: The function initializes local variables via

```
fmtflags flags = str .flags();
fmtflags basefield = (flags & ios_base::basefield);
fmtflags uppercase = (flags & ios_base::uppercase);
fmtflags boolalpha = (flags & ios_base::boolalpha);
```

For conversion to an integral type, the function determines the integral conversion specifier as indicated in Table 84. The table is ordered. That is, the first line whose condition is true applies.

Table 84 — Integer conversions

State	`stdio` equivalent
`basefield == oct`	`%o`
`basefield == hex`	`%X`
`basefield == 0`	`%i`
`signed` integral type	`%d`
`unsigned` integral type	`%u`

For conversions to a floating type the specifier is `%g`.

For conversions to `void*` the specifier is `%p`.

A length modifier is added to the conversion specification, if needed, as indicated in Table 85.

Table 85 — Length modifier

Type	Length modifier
`short`	`h`
`unsigned short`	`h`
`long`	`l`
`unsigned long`	`l`
`long long`	`ll`
`unsigned long long`	`ll`
`double`	`l`
`long double`	`L`

Stage 2: If `in==end` then stage 2 terminates. Otherwise a `charT` is taken from `in` and local variables are initialized as if by

```
char_type ct = *in ;
char c = src[find(atoms, atoms + sizeof(src) - 1, ct) - atoms];
if (ct ==  use_facet<numpunct<charT> >(loc).decimal_point())
c = '.';
bool discard =
  ct == use_facet<numpunct<charT> >(loc).thousands_sep()
  && use_facet<numpunct<charT> >(loc).grouping().length() != 0;
```

where the values `src` and `atoms` are defined as if by:

```
static const char src[] = "0123456789abcdefxABCDEFX+-";
char_type atoms[sizeof(src)];
use_facet<ctype<charT> >(loc).widen(src, src + sizeof(src), atoms);
```

for this value of `loc`.

If `discard` is true, then if `'.'` has not yet been accumulated, then the position of the character is remembered, but the character is otherwise ignored. Otherwise, if `'.'` has already been accumulated, the character is discarded and Stage 2 terminates. If it is not discarded, then a check is made to determine if `c` is allowed as the next character of an input field of the conversion specifier returned by Stage 1. If so, it is accumulated.

If the character is either discarded or accumulated then `in` is advanced by `++in` and processing returns to the beginning of stage 2.

Stage 3: The sequence of `char`s accumulated in stage 2 (the field) is converted to a numeric value by the rules of one of the functions declared in the header `<cstdlib>`:

(3.1) — For a signed integer value, the function `strtoll`.

(3.2) — For an unsigned integer value, the function `strtoull`.

(3.3) — For a floating-point value, the function `strtold`.

The numeric value to be stored can be one of:

(3.4) — zero, if the conversion function fails to convert the entire field. `ios_base::failbit` is assigned to `err`.

(3.5) — the most positive representable value, if the field represents a value too large positive to be represented in `val`. `ios_base::failbit` is assigned to `err`.

(3.6) — the most negative representable value or zero for an unsigned integer type, if the field represents a value too large negative to be represented in `val`. `ios_base::failbit` is assigned to `err`.

(3.7) — the converted value, otherwise.

The resultant numeric value is stored in `val`.

4 Digit grouping is checked. That is, the positions of discarded separators is examined for consistency with `use_facet<numpunct<charT> >(loc).grouping()`. If they are not consistent then `ios_base::failbit` is assigned to `err`.

5 In any case, if stage 2 processing was terminated by the test for `in==end` then `err |=ios_base::eofbit` is performed.

```
iter_type do_get(iter_type in, iter_type end, ios_base& str,
                 ios_base::iostate& err, bool& val) const;
```

6 *Effects:* If `(str.flags()&ios_base::boolalpha)==0` then input proceeds as it would for a `long` except that if a value is being stored into `val`, the value is determined according to the following: If the value to be stored is 0 then `false` is stored. If the value is 1 then `true` is stored. Otherwise `true` is stored and `ios_base::failbit` is assigned to `err`.

7 Otherwise target sequences are determined "as if" by calling the members `falsename()` and `truename()` of the facet obtained by `use_facet<numpunct<charT> >(str.getloc())`. Successive characters in the range `[in,end)` (see 23.2.3) are obtained and matched against corresponding positions in the target sequences only as necessary to identify a unique match. The input iterator `in` is compared to `end` only when necessary to obtain a character. If a target sequence is uniquely matched, `val` is set to the corresponding value. Otherwise `false` is stored and `ios_base::failbit` is assigned to `err`.

8 The `in` iterator is always left pointing one position beyond the last character successfully matched. If `val` is set, then `err` is set to `str.goodbit`; or to `str.eofbit` if, when seeking another character to match, it is found that `(in == end)`. If `val` is not set, then `err` is set to `str.failbit`; or to `(str.failbit|str.eofbit)` if the reason for the failure was that `(in == end)`. [*Example:* For targets `true`: `"a"` and `false`: `"abb"`, the input sequence `"a"` yields `val == true` and `err == str.eofbit`; the input sequence `"abc"` yields `err = str.failbit`, with `in` ending at the `'c'` element. For targets `true`: `"1"` and `false`: `"0"`, the input sequence `"1"` yields `val == true` and `err == str.goodbit`. For empty targets `("")`, any input sequence yields `err == str.failbit`. — *end example*]

9 *Returns:* `in`.

22.4.2.2 Class template num_put [locale.nm.put]

```
namespace std {
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> >
  class num_put : public locale::facet {
  public:
    typedef charT            char_type;
    typedef OutputIterator   iter_type;

    explicit num_put(size_t refs = 0);

    iter_type put(iter_type s, ios_base& f, char_type fill, bool v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill, long v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill, long long v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  unsigned long v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  unsigned long long v) const;
```

```
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  double v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  long double v) const;
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  const void* v) const;

    static locale::id id;

  protected:
    ~num_put();
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             bool v) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             long v) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             long long v) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             unsigned long) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             unsigned long long) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             double v) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             long double v) const;
    virtual iter_type do_put(iter_type, ios_base&, char_type fill,
                             const void* v) const;
  };
}
```

1 The facet `num_put` is used to format numeric values to a character sequence such as an ostream.

22.4.2.2.1 `num_put` members [facet.num.put.members]

```
iter_type put(iter_type out, ios_base& str, char_type fill,
  bool val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  long long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  unsigned long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  unsigned long long val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  double val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  long double val) const;
iter_type put(iter_type out, ios_base& str, char_type fill,
  const void* val) const;
```

1 *Returns:* `do_put(out, str, fill, val)`.

22.4.2.2.2 `num_put` virtual functions [facet.num.put.virtuals]

```
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  long val) const;
```

```
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  long long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  unsigned long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  unsigned long long val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  double val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  long double val) const;
iter_type do_put(iter_type out, ios_base& str, char_type fill,
  const void* val) const;
```

1 *Effects:* Writes characters to the sequence `out`, formatting `val` as desired. In the following description, a local variable initialized with

```
locale loc = str.getloc();
```

2 The details of this operation occur in several stages:

(2.1) — Stage 1: Determine a printf conversion specifier `spec` and determining the characters that would be printed by `printf` (27.9.2) given this conversion specifier for

```
printf(spec, val )
```

assuming that the current locale is the `"C"` locale.

(2.2) — Stage 2: Adjust the representation by converting each `char` determined by stage 1 to a `charT` using a conversion and values returned by members of `use_facet< numpunct<charT> >(str.getloc())`

(2.3) — Stage 3: Determine where padding is required.

(2.4) — Stage 4: Insert the sequence into the `out`.

3 Detailed descriptions of each stage follow.

4 *Returns:* `out`.

5

Stage 1: The first action of stage 1 is to determine a conversion specifier. The tables that describe this determination use the following local variables

```
fmtflags flags = str.flags() ;
fmtflags basefield =  (flags & (ios_base::basefield));
fmtflags uppercase =  (flags & (ios_base::uppercase));
fmtflags floatfield = (flags & (ios_base::floatfield));
fmtflags showpos =    (flags & (ios_base::showpos));
fmtflags showbase =   (flags & (ios_base::showbase));
```

All tables used in describing stage 1 are ordered. That is, the first line whose condition is true applies. A line without a condition is the default behavior when none of the earlier lines apply.

For conversion from an integral type other than a character type, the function determines the integral conversion specifier as indicated in Table 86.

For conversion from a floating-point type, the function determines the floating-point conversion specifier as indicated in Table 87.

For conversions from an integral or floating-point type a length modifier is added to the conversion specifier as indicated in Table 88.

The conversion specifier has the following optional additional qualifiers prepended as indicated in Table 89.

Table 86 — Integer conversions

State	`stdio` equivalent
`basefield == ios_base::oct`	`%o`
`(basefield == ios_base::hex) && !uppercase`	`%x`
`(basefield == ios_base::hex)`	`%X`
for a `signed` integral type	`%d`
for an `unsigned` integral type	`%u`

Table 87 — Floating-point conversions

State	`stdio` equivalent
`floatfield == ios_base::fixed`	`%f`
`floatfield == ios_base::scientific && !uppercase`	`%e`
`floatfield == ios_base::scientific`	`%E`
`floatfield == (ios_base::fixed | ios_base::scientific) && !uppercase`	`%a`
`floatfield == (ios_base::fixed | ios_base::scientific)`	`%A`
`!uppercase`	`%g`
otherwise	`%G`

Table 88 — Length modifier

Type	Length modifier
`long`	`l`
`long long`	`ll`
`unsigned long`	`l`
`unsigned long long`	`ll`
`long double`	`L`
otherwise	*none*

For conversion from a floating-point type, if `floatfield != (ios_base::fixed | ios_base::scientific)`, `str.precision()` is specified as precision in the conversion specification. Otherwise, no precision is specified.

For conversion from `void*` the specifier is `%p`.

The representations at the end of stage 1 consists of the `char`'s that would be printed by a call of `printf(s, val)` where `s` is the conversion specifier determined above.

Stage 2: Any character `c` other than a decimal point(.) is converted to a `charT` via `use_facet<ctype<charT> >(loc).widen( c )`

A local variable `punct` is initialized via

```
const numpunct<charT>& punct = use_facet< numpunct<charT> >(str.getloc());
```

For arithmetic types, `punct.thousands_sep()` characters are inserted into the sequence as determined by the value returned by `punct.do_grouping()` using the method described in 22.4.3.1.2

Table 89 — Numeric conversions

Type(s)	**State**	`stdio` equivalent
an integral type	`flags & showpos`	`+`
	`flags & showbase`	`#`
a floating-point type	`flags & showpos`	`+`
	`flags & showpoint`	`#`

Decimal point characters(.) are replaced by `punct.decimal_point()`

Stage 3: A local variable is initialized as

```
fmtflags adjustfield=   (flags & (ios_base::adjustfield));
```

The location of any padding[247] is determined according to Table 90.

Table 90 — Fill padding

State	Location
`adjustfield == ios_base::left`	pad after
`adjustfield == ios_base::right`	pad before
`adjustfield == internal` and a sign occurs in the representation	pad after the sign
`adjustfield == internal` and representation after stage 1 began with 0x or 0X	pad after x or X
otherwise	pad before

If `str.width()` is nonzero and the number of `charT`'s in the sequence after stage 2 is less than `str.width()`, then enough `fill` characters are added to the sequence at the position indicated for padding to bring the length of the sequence to `str.width()`.

`str.width(0)` is called.

Stage 4: The sequence of `charT`'s at the end of stage 3 are output via

```
*out++ = c
```

```
iter_type do_put(iter_type out, ios_base& str, char_type fill,
                 bool val) const;
```

6 *Returns:* If `(str.flags() & ios_base::boolalpha) == 0` returns `do_put(out, str, fill, (int)val)`, otherwise obtains a string `s` as if by

```
string_type s =
  val ? use_facet<numpunct<charT>>(loc).truename()
    : use_facet<numpunct<charT>>(loc).falsename();
```

and then inserts each character `c` of `s` into `out` via `*out++ = c` and returns `out`.

22.4.3 The numeric punctuation facet [facet.numpunct]

22.4.3.1 Class template `numpunct` [locale.numpunct]

```
namespace std {
  template <class charT>
  class numpunct : public locale::facet {
  public:
    typedef charT              char_type;
    typedef basic_string<charT> string_type;

    explicit numpunct(size_t refs = 0);

    char_type    decimal_point()   const;
    char_type    thousands_sep()   const;
    string       grouping()        const;
```

247) The conversion specification `#o` generates a leading `0` which is *not* a padding character.

```
    string_type  truename()        const;
    string_type  falsename()       const;

    static locale::id id;

  protected:
   ~numpunct();                 // virtual
    virtual char_type    do_decimal_point() const;
    virtual char_type    do_thousands_sep() const;
    virtual string       do_grouping()      const;
    virtual string_type  do_truename()      const;      // for bool
    virtual string_type  do_falsename()     const;      // for bool
  };
}
```

1 `numpunct<>` specifies numeric punctuation. The specializations required in Table 80 (22.3.1.1.1), namely `numpunct<wchar_t>` and `numpunct<char>`, provide classic `"C"` numeric formats, i.e., they contain information equivalent to that contained in the `"C"` locale or their wide character counterparts as if obtained by a call to `widen`.

2 The syntax for number formats is as follows, where `digit` represents the radix set specified by the `fmtflags` argument value, and `thousands-sep` and `decimal-point` are the results of corresponding `numpunct<charT>` members. Integer values have the format:

```
integer   ::= [sign] units
sign      ::= plusminus
plusminus ::= '+' | '-'
units     ::= digits [thousands-sep units]
digits    ::= digit [digits]
```

and floating-point values have:

```
floatval ::= [sign] units [decimal-point [digits]] [e [sign] digits] |
             [sign]        decimal-point  digits   [e [sign] digits]
e        ::= 'e' | 'E'
```

where the number of digits between `thousands-sep`s is as specified by `do_grouping()`. For parsing, if the `digits` portion contains no thousands-separators, no grouping constraint is applied.

22.4.3.1.1 `numpunct` members [facet.numpunct.members]

```
char_type decimal_point() const;
```

1 *Returns:* `do_decimal_point()`

```
char_type thousands_sep() const;
```

2 *Returns:* `do_thousands_sep()`

```
string grouping()  const;
```

3 *Returns:* `do_grouping()`

```
string_type truename()  const;
string_type falsename() const;
```

4 *Returns:* `do_truename()` or `do_falsename()`, respectively.

22.4.3.1.2 `numpunct` virtual functions [facet.numpunct.virtuals]

```
char_type do_decimal_point() const;
```

1 *Returns:* A character for use as the decimal radix separator. The required specializations return `'.'` or `L'.'`.

```
char_type do_thousands_sep() const;
```

2 *Returns:* A character for use as the digit group separator. The required specializations return `','` or `L','`.

```
string do_grouping() const;
```

3 *Returns:* A basic_string<char> `vec` used as a vector of integer values, in which each element `vec[i]` represents the number of digits[248] in the group at position `i`, starting with position 0 as the rightmost group. If `vec.size() <= i`, the number is the same as group `(i-1)`; if `(i<0 || vec[i]<=0 || vec[i]==CHAR_MAX)`, the size of the digit group is unlimited.

4 The required specializations return the empty string, indicating no grouping.

```
string_type do_truename()  const;
string_type do_falsename() const;
```

5 *Returns:* A string representing the name of the boolean value `true` or `false`, respectively.

6 In the base class implementation these names are `"true"` and `"false"`, or `L"true"` and `L"false"`.

22.4.3.2 Class template `numpunct_byname` [locale.numpunct.byname]

```
namespace std {
  template <class charT>
  class numpunct_byname : public numpunct<charT> {
  // this class is specialized for char and wchar_t.
  public:
    typedef charT               char_type;
    typedef basic_string<charT>  string_type;
    explicit numpunct_byname(const char*, size_t refs = 0);
    explicit numpunct_byname(const string&, size_t refs = 0);
  protected:
   ~numpunct_byname();
  };
}
```

22.4.4 The collate category [category.collate]

22.4.4.1 Class template `collate` [locale.collate]

```
namespace std {
  template <class charT>
  class collate : public locale::facet {
  public:
    typedef charT              char_type;
    typedef basic_string<charT> string_type;

    explicit collate(size_t refs = 0);
```

248) Thus, the string `"\003"` specifies groups of 3 digits each, and `"3"` probably indicates groups of 51 (!) digits each, because 51 is the ASCII value of `"3"`.

```
    int compare(const charT* low1, const charT* high1,
                const charT* low2, const charT* high2) const;
    string_type transform(const charT* low, const charT* high) const;
    long hash(const charT* low, const charT* high) const;

    static locale::id id;

  protected:
    ~collate();
    virtual int do_compare(const charT* low1, const charT* high1,
                           const charT* low2, const charT* high2) const;
    virtual string_type do_transform(const charT* low, const charT* high) const;
    virtual long do_hash (const charT* low, const charT* high) const;
  };
}
```

1 The class `collate<charT>` provides features for use in the collation (comparison) and hashing of strings. A locale member function template, `operator()`, uses the collate facet to allow a locale to act directly as the predicate argument for standard algorithms (Clause 25) and containers operating on strings. The specializations required in Table 80 (22.3.1.1.1), namely `collate<char>` and `collate<wchar_t>`, apply lexicographic ordering (25.4.8).

2 Each function compares a string of characters `*p` in the range `[low,high)`.

22.4.4.1.1 `collate` members [locale.collate.members]

```
int compare(const charT* low1, const charT* high1,
            const charT* low2, const charT* high2) const;
```

1 *Returns:* `do_compare(low1, high1, low2, high2)`

```
string_type transform(const charT* low, const charT* high) const;
```

2 *Returns:* `do_transform(low, high)`

```
long hash(const charT* low, const charT* high) const;
```

3 *Returns:* `do_hash(low, high)`

22.4.4.1.2 `collate` virtual functions [locale.collate.virtuals]

```
int do_compare(const charT* low1, const charT* high1,
               const charT* low2, const charT* high2) const;
```

1 *Returns:* `1` if the first string is greater than the second, `-1` if less, zero otherwise. The specializations required in Table 80 (22.3.1.1.1), namely `collate<char>` and `collate<wchar_t>`, implement a lexicographical comparison (25.4.8).

```
string_type do_transform(const charT* low, const charT* high) const;
```

2 *Returns:* A `basic_string<charT>` value that, compared lexicographically with the result of calling `transform()` on another string, yields the same result as calling `do_compare()` on the same two strings.[249]

```
long do_hash(const charT* low, const charT* high) const;
```

249) This function is useful when one string is being compared to many other strings.

3 *Returns:* An integer value equal to the result of calling `hash()` on any other string for which `do_compare()` returns 0 (equal) when passed the two strings. [*Note:* The probability that the result equals that for another string which does not compare equal should be very small, approaching `(1.0/numeric_limits<unsigned long>::max())`. — *end note*]

22.4.4.2 Class template `collate_byname` [locale.collate.byname]

```
namespace std {
  template <class charT>
  class collate_byname : public collate<charT> {
  public:
    typedef basic_string<charT> string_type;
    explicit collate_byname(const char*, size_t refs = 0);
    explicit collate_byname(const string&, size_t refs = 0);
  protected:
    ~collate_byname();
  };
}
```

22.4.5 The time category [category.time]

1 Templates `time_get<charT,InputIterator>` and `time_put<charT,OutputIterator>` provide date and time formatting and parsing. All specifications of member functions for `time_put` and `time_get` in the subclauses of 22.4.5 only apply to the specializations required in Tables 80 and 81 (22.3.1.1.1). Their members use their `ios_base&`, `ios_base::iostate&`, and `fill` arguments as described in (22.4), and the `ctype<>` facet, to determine formatting details.

22.4.5.1 Class template `time_get` [locale.time.get]

```
namespace std {
  class time_base {
  public:
    enum dateorder { no_order, dmy, mdy, ymd, ydm };
  };

  template <class charT, class InputIterator = istreambuf_iterator<charT> >
  class time_get : public locale::facet, public time_base {
  public:
    typedef charT            char_type;
    typedef InputIterator    iter_type;

    explicit time_get(size_t refs = 0);

    dateorder date_order()  const { return do_date_order(); }
    iter_type get_time(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t)  const;
    iter_type get_date(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t)  const;
    iter_type get_weekday(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t) const;
    iter_type get_monthname(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t) const;
    iter_type get_year(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t) const;
    iter_type get(iter_type s, iter_type end, ios_base& f,
                       ios_base::iostate& err, tm* t, char format, char modifier = 0) const;
    iter_type get(iter_type s, iter_type end, ios_base& f,
```

```
                        ios_base::iostate& err, tm* t, const char_type* fmt,
                        const char_type* fmtend) const;

    static locale::id id;

  protected:
    ~time_get();
    virtual dateorder do_date_order()  const;
    virtual iter_type do_get_time(iter_type s, iter_type end, ios_base&,
                                  ios_base::iostate& err, tm* t) const;
    virtual iter_type do_get_date(iter_type s, iter_type end, ios_base&,
                                  ios_base::iostate& err, tm* t) const;
    virtual iter_type do_get_weekday(iter_type s, iter_type end, ios_base&,
                                     ios_base::iostate& err, tm* t) const;
    virtual iter_type do_get_monthname(iter_type s, iter_type end, ios_base&,
                                       ios_base::iostate& err, tm* t) const;
    virtual iter_type do_get_year(iter_type s, iter_type end, ios_base&,
                                  ios_base::iostate& err, tm* t) const;
    virtual iter_type do_get(iter_type s, iter_type end, ios_base& f,
                             ios_base::iostate& err, tm* t, char format, char modifier) const;
  };
}
```

1 `time_get` is used to parse a character sequence, extracting components of a time or date into a `struct tm` record. Each `get` member parses a format as produced by a corresponding format specifier to `time_put<>::put`. If the sequence being parsed matches the correct format, the corresponding members of the `struct tm` argument are set to the values used to produce the sequence; otherwise either an error is reported or unspecified values are assigned.[250]

2 If the end iterator is reached during parsing by any of the `get()` member functions, the member sets `ios_base::eofbit` in `err`.

22.4.5.1.1 `time_get` members [locale.time.get.members]

```
dateorder date_order() const;
```

1 *Returns:* `do_date_order()`

```
iter_type get_time(iter_type s, iter_type end, ios_base& str,
                   ios_base::iostate& err, tm* t) const;
```

2 *Returns:* `do_get_time(s, end, str, err, t)`

```
iter_type get_date(iter_type s, iter_type end, ios_base& str,
                   ios_base::iostate& err, tm* t) const;
```

3 *Returns:* `do_get_date(s, end, str, err, t)`

```
iter_type get_weekday(iter_type s, iter_type end, ios_base& str,
                      ios_base::iostate& err, tm* t) const;
iter_type get_monthname(iter_type s, iter_type end, ios_base& str,
                        ios_base::iostate& err, tm* t) const;
```

4 *Returns:* `do_get_weekday(s, end, str, err, t)` or `do_get_monthname(s, end, str, err, t)`

250) In other words, user confirmation is required for reliable parsing of user-entered dates and times, but machine-generated formats can be parsed reliably. This allows parsers to be aggressive about interpreting user variations on standard formats.

```
iter_type get_year(iter_type s, iter_type end, ios_base& str,
                   ios_base::iostate& err, tm* t) const;
```

5 *Returns:* `do_get_year(s, end, str, err, t)`

```
iter_type get(iter_type s, iter_type end, ios_base& f,
    ios_base::iostate& err, tm* t, char format, char modifier = 0) const;
```

6 *Returns:* `do_get(s, end, f, err, t, format, modifier)`

```
iter_type get(iter_type s, iter_type end, ios_base& f,
    ios_base::iostate& err, tm* t, const char_type* fmt, const char_type* fmtend) const;
```

7 *Requires:* `[fmt,fmtend)` shall be a valid range.

8 *Effects:* The function starts by evaluating `err = ios_base::goodbit`. It then enters a loop, reading zero or more characters from `s` at each iteration. Unless otherwise specified below, the loop terminates when the first of the following conditions holds:

(8.1) — The expression `fmt == fmtend` evaluates to true.

(8.2) — The expression `err == ios_base::goodbit` evaluates to false.

(8.3) — The expression `s == end` evaluates to true, in which case the function evaluates `err = ios_base::eofbit | ios_base::failbit`.

(8.4) — The next element of `fmt` is equal to `'%'`, optionally followed by a modifier character, followed by a conversion specifier character, `format`, together forming a conversion specification valid for the ISO/IEC 9945 function `strptime`. If the number of elements in the range `[fmt,fmtend)` is not sufficient to unambiguously determine whether the conversion specification is complete and valid, the function evaluates `err = ios_base::failbit`. Otherwise, the function evaluates `s = do_get(s, end, f, err, t, format, modifier)`, where the value of `modifier` is `'\0'` when the optional modifier is absent from the conversion specification. If `err == ios_base::goodbit` holds after the evaluation of the expression, the function increments `fmt` to point just past the end of the conversion specification and continues looping.

(8.5) — The expression `isspace(*fmt, f.getloc())` evaluates to true, in which case the function first increments `fmt` until `fmt == fmtend || !isspace(*fmt, f.getloc())` evaluates to true, then advances `s` until `s == end || !isspace(*s, f.getloc())` is true, and finally resumes looping.

(8.6) — The next character read from `s` matches the element pointed to by `fmt` in a case-insensitive comparison, in which case the function evaluates `++fmt, ++s` and continues looping. Otherwise, the function evaluates `err = ios_base::failbit`.

9 [*Note:* The function uses the `ctype<charT>` facet installed in `f`'s locale to determine valid whitespace characters. It is unspecified by what means the function performs case-insensitive comparison or whether multi-character sequences are considered while doing so. — *end note*]

10 *Returns:* `s`

22.4.5.1.2 `time_get` virtual functions [locale.time.get.virtuals]

```
dateorder do_date_order() const;
```

1 *Returns:* An enumeration value indicating the preferred order of components for those date formats that are composed of day, month, and year.[251] Returns `no_order` if the date format specified by `'x'` contains other variable components (e.g., Julian day, week number, week day).

```
iter_type do_get_time(iter_type s, iter_type end, ios_base& str,
                      ios_base::iostate& err, tm* t) const;
```

251) This function is intended as a convenience only, for common formats, and may return `no_order` in valid locales.

2 *Effects:* Reads characters starting at `s` until it has extracted those `struct tm` members, and remaining format characters, used by `time_put<>::put` to produce the format specified by `"%H:%M:%S"`, or until it encounters an error or end of sequence.

3 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid time.

```
iter_type do_get_date(iter_type s, iter_type end, ios_base& str,
                      ios_base::iostate& err, tm* t) const;
```

4 *Effects:* Reads characters starting at `s` until it has extracted those `struct tm` members and remaining format characters used by `time_put<>::put` to produce one of the following formats, or until it encounters an error. The format depends on the value returned by `date_order()` as shown in Table 91.

Table 91 — `do_get_date` effects

`date_order()`	**Format**
`no_order`	`"%m%d%y"`
`dmy`	`"%d%m%y"`
`mdy`	`"%m%d%y"`
`ymd`	`"%y%m%d"`
`ydm`	`"%y%d%m"`

5 An implementation may also accept additional implementation-defined formats.

6 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid date.

```
iter_type do_get_weekday(iter_type s, iter_type end, ios_base& str,
                         ios_base::iostate& err, tm* t) const;
iter_type do_get_monthname(iter_type s, iter_type end, ios_base& str,
                           ios_base::iostate& err, tm* t) const;
```

7 *Effects:* Reads characters starting at `s` until it has extracted the (perhaps abbreviated) name of a weekday or month. If it finds an abbreviation that is followed by characters that could match a full name, it continues reading until it matches the full name or fails. It sets the appropriate `struct tm` member accordingly.

8 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid name.

```
iter_type do_get_year(iter_type s, iter_type end, ios_base& str,
                      ios_base::iostate& err, tm* t) const;
```

9 *Effects:* Reads characters starting at `s` until it has extracted an unambiguous year identifier. It is implementation-defined whether two-digit year numbers are accepted, and (if so) what century they are assumed to lie in. Sets the `t->tm_year` member accordingly.

10 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid year identifier.

```
iter_type do_get(iter_type s, iter_type end, ios_base& f,
    ios_base::iostate& err, tm* t, char format, char modifier) const;
```

11 *Requires:* `t` shall point to an object.

12 *Effects:* The function starts by evaluating `err = ios_base::goodbit`. It then reads characters starting at `s` until it encounters an error, or until it has extracted and assigned those `struct tm` members,

and any remaining format characters, corresponding to a conversion directive appropriate for the ISO/IEC 9945 function `strptime`, formed by concatenating `'%'`, the `modifier` character, when non-NUL, and the `format` character. When the concatenation fails to yield a complete valid directive the function leaves the object pointed to by `t` unchanged and evaluates `err |= ios_base::failbit`. When `s == end` evaluates to true after reading a character the function evaluates `err |= ios_base::eofbit`.

13 For complex conversion directives such as `%c`, `%x`, or `%X`, or directives that involve the optional modifiers `E` or `O`, when the function is unable to unambiguously determine some or all `struct tm` members from the input sequence `[s,end)`, it evaluates `err |= ios_base::eofbit`. In such cases the values of those `struct tm` members are unspecified and may be outside their valid range.

14 *Remark:* It is unspecified whether multiple calls to `do_get()` with the address of the same `struct tm` object will update the current contents of the object or simply overwrite its members. Portable programs must zero out the object before invoking the function.

15 *Returns:* An iterator pointing immediately beyond the last character recognized as possibly part of a valid input sequence for the given `format` and `modifier`.

22.4.5.2 Class template `time_get_byname` [locale.time.get.byname]

```
namespace std {
  template <class charT, class InputIterator = istreambuf_iterator<charT> >
  class time_get_byname : public time_get<charT, InputIterator> {
  public:
    typedef time_base::dateorder dateorder;
    typedef InputIterator        iter_type;

    explicit time_get_byname(const char*, size_t refs = 0);
    explicit time_get_byname(const string&, size_t refs = 0);
  protected:
    ~time_get_byname();
  };
}
```

22.4.5.3 Class template `time_put` [locale.time.put]

```
namespace std {
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> >
  class time_put : public locale::facet {
  public:
    typedef charT            char_type;
    typedef OutputIterator   iter_type;

    explicit time_put(size_t refs = 0);

    // the following is implemented in terms of other member functions.
    iter_type put(iter_type s, ios_base& f, char_type fill, const tm* tmb,
                  const charT* pattern, const charT* pat_end) const;
    iter_type put(iter_type s, ios_base& f, char_type fill,
                  const tm* tmb, char format, char modifier = 0) const;

    static locale::id id;

  protected:
    ~time_put();
    virtual iter_type do_put(iter_type s, ios_base&, char_type, const tm* t,
```

```
                                char format, char modifier) const;
  };
}
```

22.4.5.3.1 time_put members [locale.time.put.members]

```
iter_type put(iter_type s, ios_base& str, char_type fill, const tm* t,
              const charT* pattern, const charT* pat_end) const;
iter_type put(iter_type s, ios_base& str, char_type fill, const tm* t,
              char format, char modifier = 0) const;
```

1 *Effects:* The first form steps through the sequence from `pattern` to `pat_end`, identifying characters that are part of a format sequence. Each character that is not part of a format sequence is written to `s` immediately, and each format sequence, as it is identified, results in a call to `do_put`; thus, format elements and other characters are interleaved in the output in the order in which they appear in the pattern. Format sequences are identified by converting each character `c` to a `char` value as if by `ct.narrow(c,0)`, where `ct` is a reference to `ctype<charT>` obtained from `str.getloc()`. The first character of each sequence is equal to `'%'`, followed by an optional modifier character `mod`[252] and a format specifier character `spec` as defined for the function `strftime`. If no modifier character is present, `mod` is zero. For each valid format sequence identified, calls `do_put(s, str, fill, t, spec, mod)`.

2 The second form calls `do_put(s, str, fill, t, format, modifier)`.

3 [*Note:* The `fill` argument may be used in the implementation-defined formats or by derivations. A space character is a reasonable default for this argument. — *end note*]

4 *Returns:* An iterator pointing immediately after the last character produced.

22.4.5.3.2 time_put virtual functions [locale.time.put.virtuals]

```
iter_type do_put(iter_type s, ios_base&, char_type fill, const tm* t,
                 char format, char modifier) const;
```

1 *Effects:* Formats the contents of the parameter `t` into characters placed on the output sequence `s`. Formatting is controlled by the parameters `format` and `modifier`, interpreted identically as the format specifiers in the string argument to the standard library function `strftime()`[253], except that the sequence of characters produced for those specifiers that are described as depending on the C locale are instead implementation-defined.[254]

2 *Returns:* An iterator pointing immediately after the last character produced. [*Note:* The `fill` argument may be used in the implementation-defined formats or by derivations. A space character is a reasonable default for this argument. — *end note*]

22.4.5.4 Class template time_put_byname [locale.time.put.byname]

```
namespace std {
  template <class charT, class OutputIterator = ostreambuf_iterator<charT> >
  class time_put_byname : public time_put<charT, OutputIterator>
  {
  public:
    typedef charT          char_type;
    typedef OutputIterator iter_type;

    explicit time_put_byname(const char*, size_t refs = 0);
```

252) Although the C programming language defines no modifiers, most vendors do.
253) Interpretation of the `modifier` argument is implementation-defined, but should follow POSIX conventions.
254) Implementations are encouraged to refer to other standards such as POSIX for these definitions.

```
    explicit time_put_byname(const string&, size_t refs = 0);
  protected:
    ~time_put_byname();
  };
}
```

22.4.6 The monetary category [category.monetary]

1 These templates handle monetary formats. A template parameter indicates whether local or international monetary formats are to be used.

2 All specifications of member functions for `money_put` and `money_get` in the subclauses of 22.4.6 only apply to the specializations required in Tables 80 and 81 (22.3.1.1.1). Their members use their `ios_base&`, `ios_base::iostate&`, and `fill` arguments as described in (22.4), and the `moneypunct<>` and `ctype<>` facets, to determine formatting details.

22.4.6.1 Class template `money_get` [locale.money.get]

```
namespace std {
  template <class charT,
    class InputIterator = istreambuf_iterator<charT> >
  class money_get : public locale::facet {
  public:
    typedef charT               char_type;
    typedef InputIterator       iter_type;
    typedef basic_string<charT> string_type;

    explicit money_get(size_t refs = 0);

    iter_type get(iter_type s, iter_type end, bool intl,
                  ios_base& f, ios_base::iostate& err,
                  long double& units) const;
    iter_type get(iter_type s, iter_type end, bool intl,
                  ios_base& f, ios_base::iostate& err,
                  string_type& digits) const;

    static locale::id id;

  protected:
    ~money_get();
    virtual iter_type do_get(iter_type, iter_type, bool, ios_base&,
                             ios_base::iostate& err, long double& units) const;
    virtual iter_type do_get(iter_type, iter_type, bool, ios_base&,
                             ios_base::iostate& err, string_type& digits) const;
  };
}
```

22.4.6.1.1 `money_get` members [locale.money.get.members]

```
iter_type get(iter_type s, iter_type end, bool intl,
              ios_base& f, ios_base::iostate& err,
              long double& quant) const;
iter_type get(s, iter_type end, bool intl, ios_base&f,
              ios_base::iostate& err, string_type& quant) const;
```

1 *Returns:* `do_get(s, end, intl, f, err, quant)`

22.4.6.1.2 `money_get` virtual functions [locale.money.get.virtuals]

```
iter_type do_get(iter_type s, iter_type end, bool intl,
                 ios_base& str, ios_base::iostate& err,
                 long double& units) const;
iter_type do_get(iter_type s, iter_type end, bool intl,
                 ios_base& str, ios_base::iostate& err,
                 string_type& digits) const;
```

1 *Effects:* Reads characters from `s` to parse and construct a monetary value according to the format specified by a `moneypunct<charT,Intl>` facet reference `mp` and the character mapping specified by a `ctype<charT>` facet reference `ct` obtained from the locale returned by `str.getloc()`, and `str.flags()`. If a valid sequence is recognized, does not change `err`; otherwise, sets `err` to `(err|str.failbit)`, or `(err|str.failbit|str.eofbit)` if no more characters are available, and does not change `units` or `digits`. Uses the pattern returned by `mp.neg_format()` to parse all values. The result is returned as an integral value stored in `units` or as a sequence of digits possibly preceded by a minus sign (as produced by `ct.widen(c)` where `c` is `'-'` or in the range from `'0'` through `'9'`, inclusive) stored in `digits`. [*Example:* The sequence `$1,056.23` in a common United States locale would yield, for `units`, `105623`, or, for `digits`, `"105623"`. — *end example*] If `mp.grouping()` indicates that no thousands separators are permitted, any such characters are not read, and parsing is terminated at the point where they first appear. Otherwise, thousands separators are optional; if present, they are checked for correct placement only after all format components have been read.

2 Where `money_base::space` or `money_base::none` appears as the last element in the format pattern, no white space is consumed. Otherwise, where `money_base::space` appears in any of the initial elements of the format pattern, at least one white space character is required. Where `money_base::none` appears in any of the initial elements of the format pattern, white space is allowed but not required. If `(str.flags() & str.showbase)` is false, the currency symbol is optional and is consumed only if other characters are needed to complete the format; otherwise, the currency symbol is required.

3 If the first character (if any) in the string `pos` returned by `mp.positive_sign()` or the string `neg` returned by `mp.negative_sign()` is recognized in the position indicated by `sign` in the format pattern, it is consumed and any remaining characters in the string are required after all the other format components. [*Example:* If `showbase` is off, then for a `neg` value of `"()"` and a currency symbol of `"L"`, in `"(100 L)"` the `"L"` is consumed; but if `neg` is `"-"`, the `"L"` in `"-100 L"` is not consumed. — *end example*] If `pos` or `neg` is empty, the sign component is optional, and if no sign is detected, the result is given the sign that corresponds to the source of the empty string. Otherwise, the character in the indicated position must match the first character of `pos` or `neg`, and the result is given the corresponding sign. If the first character of `pos` is equal to the first character of `neg`, or if both strings are empty, the result is given a positive sign.

4 Digits in the numeric monetary component are extracted and placed in `digits`, or into a character buffer `buf1` for conversion to produce a value for `units`, in the order in which they appear, preceded by a minus sign if and only if the result is negative. The value `units` is produced as if by[255]

```
for (int i = 0; i < n; ++i)
  buf2[i] = src[find(atoms, atoms+sizeof(src), buf1[i]) - atoms];
buf2[n] = 0;
sscanf(buf2, "%Lf", &units);
```

where `n` is the number of characters placed in `buf1`, `buf2` is a character buffer, and the values `src` and `atoms` are defined as if by

```
static const char src[] = "0123456789-";
charT atoms[sizeof(src)];
ct.widen(src, src + sizeof(src) - 1, atoms);
```

255) The semantics here are different from `ct.narrow`.

5 *Returns:* An iterator pointing immediately beyond the last character recognized as part of a valid monetary quantity.

22.4.6.2 Class template `money_put` [locale.money.put]

```
namespace std {
  template <class charT,
    class OutputIterator = ostreambuf_iterator<charT> >
  class money_put : public locale::facet {
  public:
    typedef charT               char_type;
    typedef OutputIterator      iter_type;
    typedef basic_string<charT> string_type;

    explicit money_put(size_t refs = 0);

    iter_type put(iter_type s, bool intl, ios_base& f,
                  char_type fill, long double units) const;
    iter_type put(iter_type s, bool intl, ios_base& f,
                  char_type fill, const string_type& digits) const;

    static locale::id id;

  protected:
    ~money_put();
    virtual iter_type do_put(iter_type, bool, ios_base&, char_type fill,
                             long double units) const;
    virtual iter_type do_put(iter_type, bool, ios_base&, char_type fill,
                             const string_type& digits) const;
  };
}
```

22.4.6.2.1 `money_put` members [locale.money.put.members]

```
iter_type put(iter_type s, bool intl, ios_base& f, char_type fill,
              long double quant) const;
iter_type put(iter_type s, bool intl, ios_base& f, char_type fill,
              const string_type& quant) const;
```

1 *Returns:* `do_put(s, intl, f, loc, quant)`

22.4.6.2.2 `money_put` virtual functions [locale.money.put.virtuals]

```
iter_type do_put(iter_type s, bool intl, ios_base& str,
                 char_type fill, long double units) const;
iter_type do_put(iter_type s, bool intl, ios_base& str,
                 char_type fill, const string_type& digits) const;
```

1 *Effects:* Writes characters to `s` according to the format specified by a `moneypunct<charT,Intl>` facet reference `mp` and the character mapping specified by a `ctype<charT>` facet reference `ct` obtained from the locale returned by `str.getloc()`, and `str.flags()`. The argument `units` is transformed into a sequence of wide characters as if by

```
ct.widen(buf1, buf1 + sprintf(buf1, "%.0Lf", units), buf2)
```

for character buffers `buf1` and `buf2`. If the first character in `digits` or `buf2` is equal to `ct.widen('-')`, then the pattern used for formatting is the result of `mp.neg_format()`; otherwise the pattern is the result of `mp.pos_format()`. Digit characters are written, interspersed with any thousands separators

and decimal point specified by the format, in the order they appear (after the optional leading minus sign) in `digits` or `buf2`. In `digits`, only the optional leading minus sign and the immediately subsequent digit characters (as classified according to `ct`) are used; any trailing characters (including digits appearing after a non-digit character) are ignored. Calls `str.width(0)`.

2 *Remarks:* The currency symbol is generated if and only if `(str.flags() & str.showbase)` is nonzero. If the number of characters generated for the specified format is less than the value returned by `str.width()` on entry to the function, then copies of `fill` are inserted as necessary to pad to the specified width. For the value `af` equal to `(str.flags() & str.adjustfield)`, if `(af == str.internal)` is true, the fill characters are placed where `none` or `space` appears in the formatting pattern; otherwise if `(af == str.left)` is true, they are placed after the other characters; otherwise, they are placed before the other characters. [*Note:* It is possible, with some combinations of format patterns and flag values, to produce output that cannot be parsed using `num_get<>::get`. — *end note*]

3 *Returns:* An iterator pointing immediately after the last character produced.

22.4.6.3 Class template `moneypunct` [locale.moneypunct]

```
namespace std {
  class money_base {
  public:
    enum part { none, space, symbol, sign, value };
    struct pattern { char field[4]; };
  };

  template <class charT, bool International = false>
  class moneypunct : public locale::facet, public money_base {
  public:
    typedef charT char_type;
    typedef basic_string<charT> string_type;

    explicit moneypunct(size_t refs = 0);

    charT        decimal_point() const;
    charT        thousands_sep() const;
    string       grouping()      const;
    string_type  curr_symbol()   const;
    string_type  positive_sign() const;
    string_type  negative_sign() const;
    int          frac_digits()   const;
    pattern      pos_format()    const;
    pattern      neg_format()    const;

    static locale::id id;
    static const bool intl = International;

  protected:
    ~moneypunct();
    virtual charT        do_decimal_point() const;
    virtual charT        do_thousands_sep() const;
    virtual string       do_grouping()      const;
    virtual string_type  do_curr_symbol()   const;
    virtual string_type  do_positive_sign() const;
    virtual string_type  do_negative_sign() const;
    virtual int          do_frac_digits()   const;
    virtual pattern      do_pos_format()    const;
```

```
    virtual pattern      do_neg_format()    const;
  };
}
```

1 The `moneypunct<>` facet defines monetary formatting parameters used by `money_get<>` and `money_put<>`. A monetary format is a sequence of four components, specified by a `pattern` value `p`, such that the `part` value `static_cast<part>(p.field[i])` determines the `i`th component of the format[256] In the `field` member of a `pattern` object, each value `symbol`, `sign`, `value`, and either `space` or `none` appears exactly once. The value `none`, if present, is not first; the value `space`, if present, is neither first nor last.

2 Where `none` or `space` appears, white space is permitted in the format, except where `none` appears at the end, in which case no white space is permitted. The value `space` indicates that at least one space is required at that position. Where `symbol` appears, the sequence of characters returned by `curr_symbol()` is permitted, and can be required. Where `sign` appears, the first (if any) of the sequence of characters returned by `positive_sign()` or `negative_sign()` (respectively as the monetary value is non-negative or negative) is required. Any remaining characters of the sign sequence are required after all other format components. Where `value` appears, the absolute numeric monetary value is required.

3 The format of the numeric monetary value is a decimal number:

```
value ::= units [ decimal-point [ digits ]] |
  decimal-point digits
```

if `frac_digits()` returns a positive value, or

```
value ::= units
```

otherwise. The symbol `decimal-point` indicates the character returned by `decimal_point()`. The other symbols are defined as follows:

```
units ::= digits [ thousands-sep units ]
digits ::= adigit [ digits ]
```

In the syntax specification, the symbol `adigit` is any of the values `ct.widen(c)` for `c` in the range `'0'` through `'9'`, inclusive, and `ct` is a reference of type `const ctype<charT>&` obtained as described in the definitions of `money_get<>` and `money_put<>`. The symbol `thousands-sep` is the character returned by `thousands_sep()`. The space character used is the value `ct.widen(' ')`. White space characters are those characters `c` for which `ci.is(space,c)` returns `true`. The number of digits required after the decimal point (if any) is exactly the value returned by `frac_digits()`.

4 The placement of thousands-separator characters (if any) is determined by the value returned by `grouping()`, defined identically as the member `numpunct<>::do_grouping()`.

22.4.6.3.1 `moneypunct` members [locale.moneypunct.members]

```
charT        decimal_point() const;
charT        thousands_sep() const;
string       grouping()      const;
string_type  curr_symbol()   const;
string_type  positive_sign() const;
string_type  negative_sign() const;
int          frac_digits()   const;
pattern      pos_format()    const;
pattern      neg_format()    const;
```

1 Each of these functions `F` returns the result of calling the corresponding virtual member function `do_F()`.

22.4.6.3.2 `moneypunct` virtual functions [locale.moneypunct.virtuals]

256) An array of `char`, rather than an array of `part`, is specified for `pattern::field` purely for efficiency.

```
charT do_decimal_point() const;
```

1 *Returns:* The radix separator to use in case `do_frac_digits()` is greater than zero.[257]

```
charT do_thousands_sep() const;
```

2 *Returns:* The digit group separator to use in case `do_grouping()` specifies a digit grouping pattern.[258]

```
string do_grouping() const;
```

3 *Returns:* A pattern defined identically as, but not necessarily equal to, the result of `numpunct<charT>::do_grouping()`.[259]

```
string_type do_curr_symbol() const;
```

4 *Returns:* A string to use as the currency identifier symbol.[260]

```
string_type do_positive_sign() const;
string_type do_negative_sign() const;
```

5 *Returns:* `do_positive_sign()` returns the string to use to indicate a positive monetary value;[261] `do_negative_sign()` returns the string to use to indicate a negative value.

```
int do_frac_digits() const;
```

6 *Returns:* The number of digits after the decimal radix separator, if any.[262]

```
pattern do_pos_format() const;
pattern do_neg_format() const;
```

7 *Returns:* The specializations required in Table 81 (22.3.1.1.1), namely `moneypunct<char>`, `moneypunct<wchar_t>`, `moneypunct<char,true>`, and `moneypunct<wchar_t,true>`, return an object of type `pattern` initialized to `{ symbol, sign, none, value }`.[263]

22.4.6.4 Class template `moneypunct_byname` [locale.moneypunct.byname]

```
namespace std {
  template <class charT, bool Intl = false>
  class moneypunct_byname : public moneypunct<charT, Intl> {
  public:
    typedef money_base::pattern pattern;
    typedef basic_string<charT> string_type;

    explicit moneypunct_byname(const char*, size_t refs = 0);
    explicit moneypunct_byname(const string&, size_t refs = 0);
  protected:
    ~moneypunct_byname();
  };
}
```

22.4.7 The message retrieval category [category.messages]

1 Class `messages<charT>` implements retrieval of strings from message catalogs.

257) In common U.S. locales this is `'.'`.
258) In common U.S. locales this is `','`.
259) To specify grouping by 3s, the value is `"\003"` *not* `"3"`.
260) For international specializations (second template parameter `true`) this is typically four characters long, usually three letters and a space.
261) This is usually the empty string.
262) In common U.S. locales, this is 2.
263) Note that the international symbol returned by `do_curr_sym()` usually contains a space, itself; for example, `"USD "`.

22.4.7.1 Class template messages [locale.messages]

```
namespace std {
  class messages_base {
  public:
    typedef unspecified signed integer type catalog;
  };

  template <class charT>
  class messages : public locale::facet, public messages_base {
  public:
    typedef charT char_type;
    typedef basic_string<charT> string_type;

    explicit messages(size_t refs = 0);

    catalog open(const basic_string<char>& fn, const locale&) const;
    string_type get(catalog c, int set, int msgid,
                    const string_type& dfault) const;
    void close(catalog c) const;

    static locale::id id;

  protected:
    ~messages();
    virtual catalog do_open(const basic_string<char>&, const locale&) const;
    virtual string_type do_get(catalog, int set, int msgid,
                               const string_type& dfault) const;
    virtual void do_close(catalog) const;
  };
}
```

1 Values of type `messages_base::catalog` usable as arguments to members `get` and `close` can be obtained only by calling member `open`.

22.4.7.1.1 messages members [locale.messages.members]

```
catalog open(const basic_string<char>& name, const locale& loc) const;
```

1 *Returns:* `do_open(name, loc)`.

```
string_type get(catalog cat, int set, int msgid,
                const string_type& dfault) const;
```

2 *Returns:* `do_get(cat, set, msgid, dfault)`.

```
void  close(catalog cat) const;
```

3 *Effects:* Calls `do_close(cat)`.

22.4.7.1.2 messages virtual functions [locale.messages.virtuals]

```
catalog do_open(const basic_string<char>& name,
                const locale& loc) const;
```

1 *Returns:* A value that may be passed to `get()` to retrieve a message from the message catalog identified by the string `name` according to an implementation-defined mapping. The result can be used until it is passed to `close()`.

2 Returns a value less than 0 if no such catalog can be opened.

3 *Remarks:* The locale argument `loc` is used for character set code conversion when retrieving messages, if needed.

```
string_type do_get(catalog cat, int set, int msgid,
              const string_type& dfault) const;
```

4 *Requires:* `cat` shall be a catalog obtained from `open()` and not yet closed.

5 *Returns:* A message identified by arguments `set`, `msgid`, and `dfault`, according to an implementation-defined mapping. If no such message can be found, returns `dfault`.

```
void do_close(catalog cat) const;
```

6 *Requires:* `cat` shall be a catalog obtained from `open()` and not yet closed.

7 *Effects:* Releases unspecified resources associated with `cat`.

8 *Remarks:* The limit on such resources, if any, is implementation-defined.

22.4.7.2 Class template `messages_byname` [locale.messages.byname]

```
namespace std {
  template <class charT>
  class messages_byname : public messages<charT> {
  public:
    typedef messages_base::catalog catalog;
    typedef basic_string<charT>    string_type;

    explicit messages_byname(const char*, size_t refs = 0);
    explicit messages_byname(const string&, size_t refs = 0);
  protected:
    ~messages_byname();
  };
}
```

22.4.8 Program-defined facets [facets.examples]

1 A C++ program may define facets to be added to a locale and used identically as the built-in facets. To create a new facet interface, C++ programs simply derive from `locale::facet` a class containing a static member: `static locale::id id`.

2 [*Note:* The locale member function templates verify its type and storage class. — *end note*]

3 [*Example:* Traditional global localization is still easy:

```
#include <iostream>
#include <locale>
int main(int argc, char** argv) {
  using namespace std;
  locale::global(locale(""));           // set the global locale
                                        // imbue it on all the std streams
  cin.imbue(locale());
  cout.imbue(locale());
  cerr.imbue(locale());
  wcin.imbue(locale());
  wcout.imbue(locale());
  wcerr.imbue(locale());

  return MyObject(argc, argv).doit();
}
```

— *end example*]

4 [*Example:* Greater flexibility is possible:

```
#include <iostream>
#include <locale>
int main() {
  using namespace std;
  cin.imbue(locale(""));        // the user's preferred locale
  cout.imbue(locale::classic());
  double f;
  while (cin >> f) cout << f << endl;
  return (cin.fail() != 0);
}
```

In a European locale, with input `3.456,78`, output is `3456.78`. — *end example*]

5 This can be important even for simple programs, which may need to write a data file in a fixed format, regardless of a user's preference.

6 [*Example:* Here is an example of the use of locales in a library interface.

// file: `Date.h`

```
#include <iosfwd>
#include <string>
#include <locale>

class Date {
public:
  Date(unsigned day, unsigned month, unsigned year);
  std::string asString(const std::locale& = std::locale());
};

std::istream& operator>>(std::istream& s, Date& d);
std::ostream& operator<<(std::ostream& s, Date d);
```

7 This example illustrates two architectural uses of class `locale`.

8 The first is as a default argument in `Date::asString()`, where the default is the global (presumably user-preferred) locale.

9 The second is in the operators `<<` and `>>`, where a locale "hitchhikes" on another object, in this case a stream, to the point where it is needed.

// file: `Date.C`

```
#include "Date"                 // includes <ctime>
#include <sstream>
std::string Date::asString(const std::locale& l) {
  using namespace std;
  ostringstream s; s.imbue(l);
  s << *this; return s.str();
}

std::istream& operator>>(std::istream& s, Date& d) {
  using namespace std;
  istream::sentry cerberos(s);
  if (cerberos) {
    ios_base::iostate err = goodbit;
    struct tm t;
```

```
    use_facet< time_get<char> >(s.getloc()).get_date(s, 0, s, err, &t);
    if (!err) d = Date(t.tm_day, t.tm_mon + 1, t.tm_year + 1900);
    s.setstate(err);
  }
  return s;
}
```

— *end example*]

10 A locale object may be extended with a new facet simply by constructing it with an instance of a class derived from `locale::facet`. The only member a C++ program must define is the static member `id`, which identifies your class interface as a new facet.

11 [*Example:* Classifying Japanese characters:

```
// file: <jctype>
#include <locale>
namespace My {
  using namespace std;
  class JCtype : public locale::facet {
  public:
    static locale::id id;       // required for use as a new locale facet
    bool is_kanji (wchar_t c) const;
    JCtype() { }
  protected:
    ~JCtype() { }
  };
}

// file: filt.C
#include <iostream>
#include <locale>
#include "jctype"               // above
std::locale::id My::JCtype::id; // the static JCtype member declared above.

int main() {
  using namespace std;
  typedef ctype<wchar_t> wctype;
  locale loc(locale(""),        // the user's preferred locale ...
         new My::JCtype);       // and a new feature ...
  wchar_t c = use_facet<wctype>(loc).widen('!');
  if (!use_facet<My::JCtype>(loc).is_kanji(c))
    cout << "no it isn't!" << endl;
  return 0;
}
```

12 The new facet is used exactly like the built-in facets. — *end example*]

13 [*Example:* Replacing an existing facet is even easier. The code does not define a member `id` because it is reusing the `numpunct<charT>` facet interface:

```
// file: my_bool.C
#include <iostream>
#include <locale>
#include <string>
namespace My {
  using namespace std;
  typedef numpunct_byname<char> cnumpunct;
```

```
  class BoolNames : public cnumpunct {
  protected:
    string do_truename()  const { return "Oui Oui!"; }
    string do_falsename() const { return "Mais Non!"; }
    ~BoolNames() { }
  public:
    BoolNames(const char* name) : cnumpunct(name) { }
  };
}

int main(int argc, char** argv) {
  using namespace std;
  // make the user's preferred locale, except for...
  locale loc(locale(""), new My::BoolNames(""));
  cout.imbue(loc);
  cout << boolalpha << "Any arguments today? " << (argc > 1) << endl;
  return 0;
}
```

— *end example*]

22.5 Standard code conversion facets [locale.stdcvt]

1 The header <codecvt> provides code conversion facets for various character encodings.

2 **Header <codecvt> synopsis**

```
namespace std {
  enum codecvt_mode {
    consume_header = 4,
    generate_header = 2,
    little_endian = 1
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf8
    : public codecvt<Elem, char, mbstate_t> {
  public:
    explicit codecvt_utf8(size_t refs = 0);
    ~codecvt_utf8();
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf16
    : public codecvt<Elem, char, mbstate_t> {
  public:
    explicit codecvt_utf16(size_t refs = 0);
    ~codecvt_utf16();
  };

  template<class Elem, unsigned long Maxcode = 0x10ffff,
    codecvt_mode Mode = (codecvt_mode)0>
  class codecvt_utf8_utf16
    : public codecvt<Elem, char, mbstate_t> {
  public:
```

```
    explicit codecvt_utf8_utf16(size_t refs = 0);
    ~codecvt_utf8_utf16();
  };
}
```

3 For each of the three code conversion facets `codecvt_utf8`, `codecvt_utf16`, and `codecvt_utf8_utf16`:

(3.1) — `Elem` is the wide-character type, such as `wchar_t`, `char16_t`, or `char32_t`.

(3.2) — `Maxcode` is the largest wide-character code that the facet will read or write without reporting a conversion error.

(3.3) — If `(Mode & consume_header)`, the facet shall consume an initial header sequence, if present, when reading a multibyte sequence to determine the endianness of the subsequent multibyte sequence to be read.

(3.4) — If `(Mode & generate_header)`, the facet shall generate an initial header sequence when writing a multibyte sequence to advertise the endianness of the subsequent multibyte sequence to be written.

(3.5) — If `(Mode & little_endian)`, the facet shall generate a multibyte sequence in little-endian order, as opposed to the default big-endian order.

4 For the facet `codecvt_utf8`:

(4.1) — The facet shall convert between UTF-8 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.

(4.2) — Endianness shall not affect how multibyte sequences are read or written.

(4.3) — The multibyte sequences may be written as either a text or a binary file.

5 For the facet `codecvt_utf16`:

(5.1) — The facet shall convert between UTF-16 multibyte sequences and UCS2 or UCS4 (depending on the size of `Elem`) within the program.

(5.2) — Multibyte sequences shall be read or written according to the `Mode` flag, as set out above.

(5.3) — The multibyte sequences may be written only as a binary file. Attempting to write to a text file produces undefined behavior.

6 For the facet `codecvt_utf8_utf16`:

(6.1) — The facet shall convert between UTF-8 multibyte sequences and UTF-16 (one or two 16-bit codes) within the program.

(6.2) — Endianness shall not affect how multibyte sequences are read or written.

(6.3) — The multibyte sequences may be written as either a text or a binary file.

See also: ISO/IEC 10646-1:1993.

22.6 C library locales [c.locales]

1 Table 92 describes header `<clocale>`.

2 The contents are the same as the Standard C library header `<locale.h>`.

3 Calls to the function `setlocale` may introduce a data race (17.6.5.9) with other calls to `setlocale` or with calls to the functions listed in Table 93.

See also: ISO C Clause 7.4.

Table 92 — Header `<clocale>` synopsis

Type	**Name(s)**		
Macros:	`LC_ALL`	`LC_COLLATE`	`LC_CTYPE`
	`LC_MONETARY`	`LC_NUMERIC`	`LC_TIME`
	`NULL`		
Struct:	`lconv`		
Functions:	`localeconv`	`setlocale`	

Table 93 — Potential `setlocale` data races

`fprintf`	`isprint`	`iswdigit`	`localeconv`	`tolower`
`fscanf`	`ispunct`	`iswgraph`	`mblen`	`toupper`
`isalnum`	`isspace`	`iswlower`	`mbstowcs`	`towlower`
`isalpha`	`isupper`	`iswprint`	`mbtowc`	`towupper`
`isblank`	`iswalnum`	`iswpunct`	`setlocale`	`wcscoll`
`iscntrl`	`iswalpha`	`iswspace`	`strcoll`	`wcstod`
`isdigit`	`iswblank`	`iswupper`	`strerror`	`wcstombs`
`isgraph`	`iswcntrl`	`iswxdigit`	`strtod`	`wcsxfrm`
`islower`	`iswctype`	`isxdigit`	`strxfrm`	`wctomb`

23 Containers library [containers]

23.1 General [containers.general]

1 This Clause describes components that C++ programs may use to organize collections of information.

2 The following subclauses describe container requirements, and components for sequence containers and associative containers, as summarized in Table 94.

Table 94 — Containers library summary

Subclause		Header(s)
23.2	Requirements	

23.3	Sequence containers	`<array>`
		`<deque>`
		`<forward_list>`
		`<list>`
		`<vector>`

23.4	Associative containers	`<map>` `<set>`
23.5	Unordered associative containers	`<unordered_map>` `<unordered_set>`
23.6	Container adaptors	`<queue>` `<stack>`

23.2 Container requirements [container.requirements]

23.2.1 General container requirements [container.requirements.general]

1 Containers are objects that store other objects. They control allocation and deallocation of these objects through constructors, destructors, insert and erase operations.

2 All of the complexity requirements in this Clause are stated solely in terms of the number of operations on the contained objects. [*Example:* the copy constructor of type `vector <vector<int> >` has linear complexity, even though the complexity of copying each contained `vector<int>` is itself linear. — *end example*]

3 For the components affected by this subclause that declare an `allocator_type`, objects stored in these components shall be constructed using the `allocator_traits<allocator_type>::construct` function and destroyed using the `allocator_traits<allocator_type>::destroy` function (20.7.8.2). These functions are called only for the container's element type, not for internal types used by the container. [*Note:* This means, for example, that a node-based container might need to construct nodes containing aligned buffers and call `construct` to place the element into the buffer. — *end note*]

4 In Tables 95, 96, and 97 `X` denotes a container class containing objects of type `T`, `a` and `b` denote values of type `X`, `u` denotes an identifier, `r` denotes a non-const value of type `X`, and `rv` denotes a non-const rvalue of type `X`.

Table 95 — Container requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
`X::value_type`	`T`		*Requires:* `T` is `Erasable` from `X` (see 23.2.1, below)	compile time
`X::reference`	`T&`			compile time
`X::const_reference`	`const T&`			compile time
`X::iterator`	iterator type whose value type is `T`		any iterator category that meets the forward iterator requirements. convertible to `X::const_iterator`.	compile time
`X::const_iterator`	constant iterator type whose value type is `T`		any iterator category that meets the forward iterator requirements.	compile time
`X::difference_type`	signed integer type		is identical to the difference type of `X::iterator` and `X::const_iterator`	compile time
`X::size_type`	unsigned integer type		`size_type` can represent any non-negative value of `difference_type`	compile time
`X u;`			post: `u.empty()`	constant
`X()`			post: `X().empty()`	constant
`X(a)`			*Requires:*`T` is `CopyInsertable` into `X` (see below). post: `a == X(a)`.	linear
`X u(a)` `X u = a;`			*Requires:*`T` is `CopyInsertable` into `X` (see below). post: `u == a`	linear
`X u(rv)` `X u = rv`			post: `u` shall be equal to the value that `rv` had before this construction	(Note B)
`a = rv`	`X&`	All existing elements of `a` are either move assigned to or destroyed	`a` shall be equal to the value that `rv` had before this assignment	linear

Table 95 — Container requirements (continued)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
`(&a)->~X()`	`void`		the destructor is applied to every element of `a`; any memory obtained is deallocated.	linear
`a.begin()`	`iterator`; `const_iterator` for constant `a`			constant
`a.end()`	`iterator`; `const_iterator` for constant `a`			constant
`a.cbegin()`	`const_iterator`	`const_cast<X const&>(a).begin();`		constant
`a.cend()`	`const_iterator`	`const_cast<X const&>(a).end();`		constant
`a == b`	convertible to `bool`	`==` is an equivalence relation. `equal(a.begin(), a.end(), b.begin(), b.end())`	*Requires:* `T` is `EqualityComparable`	Constant if `a.size() != b.size()`, linear otherwise
`a != b`	convertible to `bool`	Equivalent to: `!(a == b)`		linear
`a.swap(b)`	`void`		exchanges the contents of `a` and `b`	(Note A)
`swap(a, b)`	`void`	`a.swap(b)`		(Note A)
`r = a`	`X&`		post: `r == a`.	linear
`a.size()`	`size_type`	`distance(a.begin(), a.end())`		constant
`a.max_size()`	`size_type`	`distance(begin(), end())` for the largest possible container		constant
`a.empty()`	convertible to `bool`	`a.begin() == a.end()`		constant

Notes: the algorithm `equal()` is defined in Clause 25. Those entries marked "(Note A)" or "(Note B)" have linear complexity for `array` and have constant complexity for all other standard containers.

5 The member function `size()` returns the number of elements in the container. The number of elements is defined by the rules of constructors, inserts, and erases.

6 `begin()` returns an iterator referring to the first element in the container. `end()` returns an iterator which is the past-the-end value for the container. If the container is empty, then `begin() == end()`;

7 In the expressions

```
i == j
i != j
i < j
i <= j
i >= j
i > j
i - j
```

where `i` and `j` denote objects of a container's `iterator` type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

8 Unless otherwise specified, all containers defined in this clause obtain memory using an allocator (see 17.6.3.5). Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. Such move construction of the allocator shall not exit via an exception. All other constructors for these container types take a `const allocator_type&` argument. [*Note:* If an invocation of a constructor uses the default value of an optional allocator argument, then the `Allocator` type must support value initialization. — *end note*] A copy of this allocator is used for any memory allocation performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value`, `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value`, or `allocator_traits<allocator_type>::propagate_on_container_swap::value` is true within the implementation of the corresponding container operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement.

9 The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than `array`, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Lvalues of any `Compare`, `Pred`, or `Hash` types belonging to `a` and `b` shall be swappable and shall be exchanged by calling `swap` as described in 17.6.3.2. If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is `true`, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in 17.6.3.2. Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.

10 If the iterator type of a container belongs to the bidirectional or random access iterator categories (24.2), the container is called *reversible* and satisfies the additional requirements in Table 96.

Table 96 — Reversible container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X::reverse_iterator`	iterator type whose value type is `T`	`reverse_iterator<iterator>`	compile time
`X::const_reverse_iterator`	constant iterator type whose value type is `T`	`reverse_iterator<const_iterator>`	compile time

Table 96 — Reversible container requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a.rbegin()`	`reverse_iterator`; `const_reverse_iterator` for constant `a`	`reverse_iterator(end())`	constant
`a.rend()`	`reverse_iterator`; `const_reverse_iterator` for constant `a`	`reverse_iterator(begin())`	constant
`a.crbegin()`	`const_reverse_iterator`	`const_cast<X const&>(a).rbegin()`	constant
`a.crend()`	`const_reverse_iterator`	`const_cast<X const&>(a).rend()`	constant

11 Unless otherwise specified (see 23.2.4.1, 23.2.5.1, 23.3.3.4, and 23.3.6.5) all container types defined in this Clause meet the following additional requirements:

(11.1) — if an exception is thrown by an `insert()` or `emplace()` function while inserting a single element, that function has no effects.

(11.2) — if an exception is thrown by a `push_back()`, `push_front()`, `emplace_back()`, or `emplace_front()` function, that function has no effects.

(11.3) — no `erase()`, `clear()`, `pop_back()` or `pop_front()` function throws an exception.

(11.4) — no copy constructor or assignment operator of a returned iterator throws an exception.

(11.5) — no `swap()` function throws an exception.

(11.6) — no `swap()` function invalidates any references, pointers, or iterators referring to the elements of the containers being swapped. [*Note:* The `end()` iterator does not refer to any element, so it may be invalidated. — *end note*]

12 Unless otherwise specified (either explicitly or by defining a function in terms of other functions), invoking a container member function or passing a container as an argument to a library function shall not invalidate iterators to, or change the values of, objects within that container.

13 A *contiguous container* is a container that supports random access iterators (24.2.7) and whose member types `iterator` and `const_iterator` are contiguous iterators (24.2.1).

14 Table 97 lists operations that are provided for some types of containers but not others. Those containers for which the listed operations are provided shall implement the semantics described in Table 97 unless otherwise stated.

Table 97 — Optional container operations

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition	Complexity
`a < b`	convertible to `bool`	`lexicographical_compare( a.begin(), a.end(), b.begin(), b.end())`	pre: `<` is defined for values of `T`. `<` is a total ordering relationship.	linear
`a > b`	convertible to `bool`	`b < a`		linear
`a <= b`	convertible to `bool`	`!(a > b)`		linear
`a >= b`	convertible to `bool`	`!(a < b)`		linear

Note: the algorithm `lexicographical_compare()` is defined in Clause 25.

15 All of the containers defined in this Clause and in (21.4) except `array` meet the additional requirements of an allocator-aware container, as described in Table 98.

Given a container type `X` having an `allocator_type` identical to `A` and a `value_type` identical to `T` and given an lvalue `m` of type `A`, a pointer `p` of type `T*`, an expression `v` of type (possibly const) `T`, and an rvalue `rv` of type `T`, the following terms are defined. If `X` is not allocator-aware, the terms below are defined as if `A` were `std::allocator<T>` — no allocator object needs to be created and user specializations of `std::allocator<T>` are not instantiated:

(15.1) — `T` is *`DefaultInsertable` into `X`* means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p)
```

(15.2) — An element of `X` is *default-inserted* if it is initialized by evaluation of the expression

```
allocator_traits<A>::construct(m, p)
```

where `p` is the address of the uninitialized storage for the element allocated within `X`.

(15.3) — `T` is *`MoveInsertable` into `X`* means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, rv)
```

and its evaluation causes the following postcondition to hold: The value of `*p` is equivalent to the value of `rv` before the evaluation. [*Note:* rv remains a valid object. Its state is unspecified — *end note*]

(15.4) — `T` is *`CopyInsertable` into `X`* means that, in addition to `T` being `MoveInsertable` into `X`, the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, v)
```

and its evaluation causes the following postcondition to hold: The value of `v` is unchanged and is equivalent to `*p`.

(15.5) — `T` is *`EmplaceConstructible` into `X` from `args`*, for zero or more arguments `args`, means that the following expression is well-formed:

```
allocator_traits<A>::construct(m, p, args)
```

(15.6) — `T` is ***`Erasable`*** *from* ***`X`*** means that the following expression is well-formed:

```
allocator_traits<A>::destroy(m, p)
```

[*Note:* A container calls `allocator_traits<A>::construct(m, p, args)` to construct an element at `p` using `args`. The default `construct` in `std::allocator` will call `::new((void*)p) T(args)`, but specialized allocators may choose a different definition. — *end note*]

16 In Table 98, `X` denotes an allocator-aware container class with a `value_type` of `T` using allocator of type `A`, `u` denotes a variable, `a` and `b` denote non-const lvalues of type `X`, `t` denotes an lvalue or a const rvalue of type `X`, `rv` denotes a non-const rvalue of type `X`, and `m` is a value of type `A`.

Table 98 — Allocator-aware container requirements

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`allocator_type`	`A`	*Requires:* `allocator_type::value_type` is the same as `X::value_type`.	compile time
`get_allocator()`	`A`		constant
`X()` `X u;`		*Requires:* `A` is `DefaultConstructible`. post: `u.empty()` returns `true`, `u.get_allocator() == A()`	constant
`X(m)` `X u(m);`		post: `u.empty()` returns `true`, `u.get_allocator() == m`	constant
`X(t, m)` `X u(t, m);`		*Requires:* `T` is `CopyInsertable` into `X`. post: `u == t`, `u.get_allocator() == m`	linear
`X(rv)` `X u(rv)`		*Requires:* move construction of `A` shall not exit via an exception. post: `u` shall have the same elements as `rv` had before this construction; the value of `u.get_allocator()` shall be the same as the value of `rv.get_allocator()` before this construction.	constant
`X(rv, m)` `X u(rv, m);`		*Requires:* `T` is `MoveInsertable` into `X`. post: `u` shall have the same elements, or copies of the elements, that `rv` had before this construction, `u.get_allocator() == m`	constant if `m == rv.get_allocator()`, otherwise linear

Table 98 — Allocator-aware container requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a = t`	`X&`	*Requires:* `T` is `CopyInsertable` into `X` and `CopyAssignable`. post: `a == t`	linear
`a = rv`	`X&`	*Requires:* If `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value` is `false`, `T` is `MoveInsertable` into `X` and `MoveAssignable`. All existing elements of `a` are either move assigned to or destroyed. post: `a` shall be equal to the value that `rv` had before this assignment.	linear
`a.swap(b)`	`void`	exchanges the contents of `a` and `b`	constant

23.2.2 Container data races [container.requirements.dataraces]

1 For purposes of avoiding data races (17.6.5.9), implementations shall consider the following functions to be `const`: `begin`, `end`, `rbegin`, `rend`, `front`, `back`, `data`, `find`, `lower_bound`, `upper_bound`, `equal_range`, `at` and, except in associative or unordered associative containers, `operator[]`.

2 Notwithstanding (17.6.5.9), implementations are required to avoid data races when the contents of the contained object in different elements in the same container, excepting `vector<bool>`, are modified concurrently.

3 [*Note:* For a `vector<int> x` with a size greater than one, `x[1] = 5` and `*x.begin() = 10` can be executed concurrently without a data race, but `x[0] = 5` and `*x.begin() = 10` executed concurrently may result in a data race. As an exception to the general rule, for a `vector<bool> y`, `y[0] = true` may race with `y[1] = true`. — *end note*]

23.2.3 Sequence containers [sequence.reqmts]

1 A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides four basic kinds of sequence containers: `vector`, `forward_list`, `list`, and `deque`. In addition, `array` is provided as a sequence container which provides limited sequence operations because it has a fixed number of elements. The library also provides container adaptors that make it easy to construct abstract data types, such as `stack`s or `queue`s, out of the basic sequence container kinds (or out of other kinds of sequence containers that the user might define).

2 The sequence containers offer the programmer different complexity trade-offs and should be used accordingly. `vector` or `array` is the type of sequence container that should be used by default. `list` or `forward_list` should be used when there are frequent insertions and deletions from the middle of the sequence. `deque` is the data structure of choice when most insertions and deletions take place at the beginning or at the end of the sequence.

3 In Tables 99 and 100, `X` denotes a sequence container class, `a` denotes a value of type `X` containing elements of type `T`, `u` denotes the name of a variable being declared, `A` denotes `X::allocator_type` if the *qualified-id* `X::allocator_type` is valid and denotes a type (14.8.2) and `std::allocator<T>` if it doesn't, `i` and `j` denote

iterators satisfying input iterator requirements and refer to elements implicitly convertible to `value_type`, `[i, j)` denotes a valid range, `il` designates an object of type `initializer_list<value_type>`, `n` denotes a value of type `X::size_type`, `p` denotes a valid const iterator to `a`, `q` denotes a valid dereferenceable const iterator to `a`, `[q1, q2)` denotes a valid range of const iterators in `a`, `t` denotes an lvalue or a const rvalue of `X::value_type`, and `rv` denotes a non-const rvalue of `X::value_type`. `Args` denotes a template parameter pack; `args` denotes a function parameter pack with the pattern `Args&&`.

4 The complexities of the expressions are sequence dependent.

Table 99 — Sequence container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition
`X(n, t)` `X u(n, t)`		*Requires:* `T` shall be `CopyInsertable` into `X`. post: `distance(begin(), end()) == n` Constructs a sequence container with `n` copies of `t`
`X(i, j)` `X u(i, j)`		*Requires:* `T` shall be `EmplaceConstructible` into `X` from `*i`. For `vector`, if the iterator does not meet the forward iterator requirements (24.2.5), `T` shall also be `MoveInsertable` into `X`. Each iterator in the range `[i,j)` shall be dereferenced exactly once. post: `distance(begin(), end()) == distance(i, j)` Constructs a sequence container equal to the range `[i, j)`
`X(il);`		Equivalent to `X(il.begin(), il.end())`
`a = il;`	`X&`	*Requires:* `T` is `CopyInsertable` into `X` and `CopyAssignable`. Assigns the range `[il.begin(),il.end())` into `a`. All existing elements of `a` are either assigned to or destroyed. *Returns:* `*this`.
`a.emplace(p, args);`	`iterator`	*Requires:* `T` is `EmplaceConstructible` into `X` from `args`. For `vector` and `deque`, T is also `MoveInsertable` into `X` and `MoveAssignable`. *Effects:* Inserts an object of type `T` constructed with `std::forward<Args>(args)...` before `p`.
`a.insert(p,t)`	`iterator`	*Requires:* `T` shall be `CopyInsertable` into `X`. For `vector` and `deque`, `T` shall also be `CopyAssignable`. *Effects:* Inserts a copy of `t` before `p`.
`a.insert(p,rv)`	`iterator`	*Requires:* `T` shall be `MoveInsertable` into `X`. For `vector` and `deque`, `T` shall also be `MoveAssignable`. *Effects:* Inserts a copy of `rv` before `p`.

Table 99 — Sequence container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition
`a.insert(p,n,t)`	`iterator`	*Requires:* `T` shall be `CopyInsertable` into `X` and `CopyAssignable`. Inserts `n` copies of `t` before `p`.
`a.insert(p,i,j)`	`iterator`	*Requires:* `T` shall be `EmplaceConstructible` into `X` from `*i`. For `vector` and `deque`, `T` shall also be `MoveInsertable` into `X`, `MoveConstructible`, `MoveAssignable`, and swappable (17.6.3.2). Each iterator in the range `[i,j)` shall be dereferenced exactly once. pre: `i` and `j` are not iterators into `a`. Inserts copies of elements in `[i, j)` before `p`
`a.insert(p, il);`	`iterator`	`a.insert(p, il.begin(), il.end())`.
`a.erase(q)`	`iterator`	*Requires:* For `vector` and `deque`, `T` shall be `MoveAssignable`. *Effects:* Erases the element pointed to by `q`
`a.erase(q1,q2)`	`iterator`	*Requires:* For `vector` and `deque`, `T` shall be `MoveAssignable`. *Effects:* Erases the elements in the range `[q1, q2)`.
`a.clear()`	`void`	Destroys all elements in `a`. Invalidates all references, pointers, and iterators referring to the elements of `a` and may invalidate the past-the-end iterator. post: `a.empty()` returns `true`. *Complexity:* Linear.
`a.assign(i,j)`	`void`	*Requires:* `T` shall be `EmplaceConstructible` into `X` from `*i` and assignable from `*i`. For `vector`, if the iterator does not meet the forward iterator requirements (24.2.5), `T` shall also be `MoveInsertable` into `X`. Each iterator in the range `[i,j)` shall be dereferenced exactly once. pre: `i`, `j` are not iterators into `a`. Replaces elements in `a` with a copy of `[i, j)`.
`a.assign(il)`	`void`	`a.assign(il.begin(), il.end())`.
`a.assign(n,t)`	`void`	*Requires:* `T` shall be `CopyInsertable` into `X` and `CopyAssignable`. pre: `t` is not a reference into `a`. Replaces elements in `a` with `n` copies of `t`.

5 `iterator` and `const_iterator` types for sequence containers shall be at least of the forward iterator category.

6 The iterator returned from `a.insert(p, t)` points to the copy of `t` inserted into `a`.

7 The iterator returned from `a.insert(p, rv)` points to the copy of `rv` inserted into `a`.

8 The iterator returned from `a.insert(p, n, t)` points to the copy of the first element inserted into `a`, or `p` if `n == 0`.

9 The iterator returned from `a.insert(p, i, j)` points to the copy of the first element inserted into `a`, or `p` if `i == j`.

10 The iterator returned from `a.insert(p, il)` points to the copy of the first element inserted into `a`, or `p` if `il` is empty.

11 The iterator returned from `a.emplace(p, args)` points to the new element constructed from `args` into `a`.

12 The iterator returned from `a.erase(q)` points to the element immediately following `q` prior to the element being erased. If no such element exists, `a.end()` is returned.

13 The iterator returned by `a.erase(q1,q2)` points to the element pointed to by `q2` prior to any elements being erased. If no such element exists, `a.end()` is returned.

14 For every sequence container defined in this Clause and in Clause 21:

(14.1) — If the constructor

```
template <class InputIterator>
X(InputIterator first, InputIterator last,
  const allocator_type& alloc = allocator_type())
```

is called with a type `InputIterator` that does not qualify as an input iterator, then the constructor shall not participate in overload resolution.

(14.2) — If the member functions of the forms:

```
template <class InputIterator>          // such as insert()
  rt fx1(const_iterator p, InputIterator first, InputIterator last);

template <class InputIterator>          // such as append(), assign()
  rt fx2(InputIterator first, InputIterator last);

template <class InputIterator>          // such as replace()
  rt fx3(const_iterator i1, const_iterator i2, InputIterator first, InputIterator last);
```

are called with a type `InputIterator` that does not qualify as an input iterator, then these functions shall not participate in overload resolution.

15 The extent to which an implementation determines that a type cannot be an input iterator is unspecified, except that as a minimum integral types shall not qualify as input iterators.

16 Table 100 lists operations that are provided for some types of sequence containers but not others. An implementation shall provide these operations for all container types shown in the "container" column, and shall implement them so as to take amortized constant time.

Table 100 — Optional sequence container operations

Expression	Return type	Operational semantics	Container
`a.front()`	`reference; const_reference` for constant `a`	`*a.begin()`	`basic_string`, `array`, `deque`, `forward_list`, `list`, `vector`

Table 100 — Optional sequence container operations (continued)

Expression	Return type	Operational semantics	Container
`a.back()`	`reference; const_reference` for constant `a`	`{ auto tmp = a.end(); --tmp; return *tmp; }`	`basic_string`, `array`, `deque`, `list`, `vector`
`a.emplace_front(args)`	`void`	Prepends an object of type `T` constructed with `std::forward<Args>(args)...` *Requires:* `T` shall be `EmplaceConstructible` into `X` from `args`.	`deque`, `forward_list`, `list`
`a.emplace_back(args)`	`void`	Appends an object of type `T` constructed with `std::forward<Args>(args)...` *Requires:* `T` shall be `EmplaceConstructible` into `X` from `args`. For `vector`, `T` shall also be `MoveInsertable` into `X`.	`deque`, `list`, `vector`
`a.push_front(t)`	`void`	Prepends a copy of `t`. *Requires:* `T` shall be `CopyInsertable` into `X`.	`deque`, `forward_list`, `list`
`a.push_front(rv)`	`void`	Prepends a copy of `rv`. *Requires:* `T` shall be `MoveInsertable` into `X`.	`deque`, `forward_list`, `list`
`a.push_back(t)`	`void`	Appends a copy of `t`. *Requires:* `T` shall be `CopyInsertable` into `X`.	`basic_string`, `deque`, `list`, `vector`
`a.push_back(rv)`	`void`	Appends a copy of `rv`. *Requires:* `T` shall be `MoveInsertable` into `X`.	`basic_string`, `deque`, `list`, `vector`
`a.pop_front()`	`void`	Destroys the first element. *Requires:* `a.empty()` shall be `false`.	`deque`, `forward_list`, `list`
`a.pop_back()`	`void`	Destroys the last element. *Requires:* `a.empty()` shall be `false`.	`basic_string`, `deque`, `list`, `vector`
`a[n]`	`reference; const_reference` for constant `a`	`*(a.begin() + n)`	`basic_string`, `array`, `deque`, `vector`

Table 100 — Optional sequence container operations (continued)

Expression	Return type	Operational semantics	Container
`a.at(n)`	`reference; const_reference` for constant `a`	`*(a.begin() + n)`	`basic_string`, `array`, `deque`, `vector`

17 The member function `at()` provides bounds-checked access to container elements. `at()` throws `out_of_range` if `n >= a.size()`.

23.2.4 Associative containers [associative.reqmts]

1 Associative containers provide fast retrieval of data based on keys. The library provides four basic kinds of associative containers: `set`, `multiset`, `map` and `multimap`.

2 Each associative container is parameterized on `Key` and an ordering relation `Compare` that induces a strict weak ordering (25.4) on elements of `Key`. In addition, `map` and `multimap` associate an arbitrary *mapped type* `T` with the `Key`. The object of type `Compare` is called the *comparison object* of a container.

3 The phrase "equivalence of keys" means the equivalence relation imposed by the comparison and *not* the `operator==` on keys. That is, two keys `k1` and `k2` are considered to be equivalent if for the comparison object `comp`, `comp(k1, k2) == false && comp(k2, k1) == false`. For any two keys `k1` and `k2` in the same container, calling `comp(k1, k2)` shall always return the same value.

4 An associative container supports *unique keys* if it may contain at most one element for each key. Otherwise, it supports *equivalent keys*. The `set` and `map` classes support unique keys; the `multiset` and `multimap` classes support equivalent keys. For `multiset` and `multimap`, `insert`, `emplace`, and `erase` preserve the relative ordering of equivalent elements.

5 For `set` and `multiset` the value type is the same as the key type. For `map` and `multimap` it is equal to `pair<const Key, T>`.

6 `iterator` of an associative container is of the bidirectional iterator category. For associative containers where the value type is the same as the key type, both `iterator` and `const_iterator` are constant iterators. It is unspecified whether or not `iterator` and `const_iterator` are the same type. [*Note:* `iterator` and `const_iterator` have identical semantics in this case, and `iterator` is convertible to `const_iterator`. Users can avoid violating the one-definition rule by always using `const_iterator` in their function parameter lists. — *end note*]

7 The associative containers meet all the requirements of Allocator-aware containers (23.2.1), except that for `map` and `multimap`, the requirements placed on `value_type` in Table 95 apply instead to `key_type` and `mapped_type`. [*Note:* For example, in some cases `key_type` and `mapped_type` are required to be `CopyAssignable` even though the associated `value_type`, `pair<const key_type, mapped_type>`, is not `CopyAssignable`. — *end note*]

8 In Table 101, `X` denotes an associative container class, `a` denotes a value of type `X`, `u` denotes the name of a variable being declared, `a_uniq` denotes a value of type `X` when `X` supports unique keys, `a_eq` denotes a value of type `X` when `X` supports multiple keys, `a_tran` denotes a value of type `X` when the *qualified-id* `X::key_compare::is_transparent` is valid and denotes a type (14.8.2), `i` and `j` satisfy input iterator requirements and refer to elements implicitly convertible to `value_type`, `[i,j)` denotes a valid range, `p` denotes a valid const iterator to `a`, `q` denotes a valid dereferenceable const iterator to `a`, `r` denotes a valid dereferenceable iterator to `a`, `[q1, q2)` denotes a valid range of const iterators in `a`, `il` designates an object of type `initializer_list<value_type>`, `t` denotes a value of type `X::value_type`, `k` denotes a value of type `X::key_type` and `c` denotes a value of type `X::key_compare`; `kl` is a value such that `a` is

partitioned (25.4) with respect to `c(r, kl)`, with `r` the key value of `e` and `e` in `a`; `ku` is a value such that `a` is partitioned with respect to `!c(ku, r)`; `ke` is a value such that `a` is partitioned with respect to `c(r, ke)` and `!c(ke, r)`, with `c(r, ke)` implying `!c(ke, r)`. `A` denotes the storage allocator used by `X`, if any, or `std::allocator<X::value_type>` otherwise, and `m` denotes an allocator of a type convertible to `A`.

Table 101 — Associative container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X::key_type`	`Key`		compile time
`mapped_type` (`map` and `multimap` only)	`T`		compile time
`X::value_type` (`set` and `multiset` only)	`Key`	*Requires:* `value_type` is `Erasable` from `X`	compile time
`X::value_type` (`map` and `multimap` only)	`pair<const Key, T>`	*Requires:* `value_type` is `Erasable` from `X`	compile time
`X::key_compare`	`Compare`	defaults to `less<key_type>`	compile time
`X::value_compare`	a binary predicate type	is the same as `key_compare` for `set` and `multiset`; is an ordering relation on pairs induced by the first component (i.e., `Key`) for `map` and `multimap`.	compile time
`X(c)` `X u(c);`		*Requires:* `key_compare` is `CopyConstructible`. *Effects:* Constructs an empty container. Uses a copy of `c` as a comparison object.	constant
`X()` `X u;`		*Requires:* `key_compare` is `DefaultConstructible`. *Effects:* Constructs an empty container. Uses `Compare()` as a comparison object	constant

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X(i,j,c)` `X u(i,j,c);`		*Requires:* `key_compare` is `CopyConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Constructs an empty container and inserts elements from the range `[i, j)` into it; uses `c` as a comparison object.	$N \log N$ in general (N has the value `distance(i, j)`); linear if `[i, j)` is sorted with `value_comp()`
`X(i,j)` `X u(i,j);`		*Requires:* `key_compare` is `DefaultConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Same as above, but uses `Compare()` as a comparison object	same as above
`X(il);`		Same as `X(il.begin(), il.end())`.	Same as `X(il.begin(), il.end())`.
`X(il,c);`		Same as `X(il.begin(), il.end(), c)`.	Same as `X(il.begin(), il.end(), c)`.
`a = il`	`X&`	*Requires:* `value_type` is `CopyInsertable` into `X` and `CopyAssignable`. *Effects:* Assigns the range `[il.begin(),il.end())` into `a`. All existing elements of `a` are either assigned to or destroyed.	$NlogN$ in general (where N has the value `il.size() + a.size()`); linear if `[il.begin(),il.end())` is sorted with `value_comp()`.
`a.key_comp()`	`X::key_compare`	returns the comparison object out of which `a` was constructed.	constant
`a.value_comp()`	`X::value_compare`	returns an object of `value_compare` constructed out of the comparison object	constant

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a_uniq.emplace(args)`	`pair<iterator, bool>`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `args`. *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...` if and only if there is no element in the container with key equivalent to the key of `t`. The `bool` component of the returned pair is true if and only if the insertion takes place, and the iterator component of the pair points to the element with key equivalent to the key of `t`.	logarithmic
`a_eq.emplace(args)`	`iterator`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `args`. *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...` and returns the iterator pointing to the newly inserted element. If a range containing elements equivalent to `t` exists in `a_eq`, `t` is inserted at the end of that range.	logarithmic
`a.emplace_hint(p, args)`	`iterator`	equivalent to `a.emplace(std::forward<Args>(args)...)` Return value is an iterator pointing to the element with the key equivalent to the newly inserted element. The element is inserted as close as possible to the position just prior to `p`.	logarithmic in general, but amortized constant if the element is inserted right before `p`

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a_uniq.insert(t)`	`pair<iterator, bool>`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Inserts `t` if and only if there is no element in the container with key equivalent to the key of `t`. The `bool` component of the returned pair is true if and only if the insertion takes place, and the `iterator` component of the pair points to the element with key equivalent to the key of `t`.	logarithmic
`a_eq.insert(t)`	`iterator`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Inserts `t` and returns the iterator pointing to the newly inserted element. If a range containing elements equivalent to `t` exists in `a_eq`, `t` is inserted at the end of that range.	logarithmic
`a.insert(p, t)`	`iterator`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Inserts `t` if and only if there is no element with key equivalent to the key of `t` in containers with unique keys; always inserts `t` in containers with equivalent keys. Always returns the iterator pointing to the element with key equivalent to the key of `t`. `t` is inserted as close as possible to the position just prior to `p`.	logarithmic in general, but amortized constant if `t` is inserted right before `p`.

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a.insert( i, j)`	`void`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `*i`. pre: `i`, `j` are not iterators into `a`. inserts each element from the range `[i,j)` if and only if there is no element with key equivalent to the key of that element in containers with unique keys; always inserts that element in containers with equivalent keys.	$N \log(\text{a.size}() + N)$ (N has the value `distance(i, j)`
`a.insert(il)`	`void`	Equivalent to `a.insert(il.begin(), il.end())`.	
`a.erase(k)`	`size_type`	erases all elements in the container with key equivalent to `k`. returns the number of erased elements.	$\log(\text{a.size}()) + \text{a.count}(k)$
`a.erase(q)`	`iterator`	erases the element pointed to by `q`. Returns an iterator pointing to the element immediately following `q` prior to the element being erased. If no such element exists, returns `a.end()`.	amortized constant
`a.erase(r)`	`iterator`	erases the element pointed to by `r`. Returns an iterator pointing to the element immediately following `r` prior to the element being erased. If no such element exists, returns `a.end()`.	amortized constant
`a.erase( q1, q2)`	`iterator`	erases all the elements in the range `[q1,q2)`. Returns an iterator pointing to the element pointed to by q2 prior to any elements being erased. If no such element exists, `a.end()` is returned.	$\log(\text{a.size}()) + N$ where N has the value `distance(q1, q2)`.
`a.clear()`	`void`	`a.erase(a.begin(),a.end())` post: `a.empty()` returns `true`	linear in `a.size()`.

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a.find(k)`	`iterator`; `const_iterator` for constant `a`.	returns an iterator pointing to an element with the key equivalent to `k`, or `a.end()` if such an element is not found	logarithmic
`a_tran.find(ke)`	`iterator`; `const_iterator` for constant `a_tran`.	returns an iterator pointing to an element with key `r` such that `!c(r, ke) && !c(ke, r)`, or `a_tran.end()` if such an element is not found	logarithmic
`a.count(k)`	`size_type`	returns the number of elements with key equivalent to `k`	log(a.size()) + a.count(k)
`a_tran.count(ke)`	`size_type`	returns the number of elements with key `r` such that `!c(r, ke) && !c(ke, r)`	log(a_tran.size()) + a_tran.count(ke)
`a.lower_bound(k)`	`iterator`; `const_iterator` for constant `a`.	returns an iterator pointing to the first element with key not less than `k`, or `a.end()` if such an element is not found.	logarithmic
`a_tran.lower_bound(kl)`	`iterator`; `const_iterator` for constant `a_tran`.	returns an iterator pointing to the first element with key `r` such that `!c(r, kl)`, or `a_tran.end()` if such an element is not found.	logarithmic
`a.upper_bound(k)`	`iterator`; `const_iterator` for constant `a`.	returns an iterator pointing to the first element with key greater than `k`, or `a.end()` if such an element is not found.	logarithmic
`a_tran.upper_bound(ku)`	`iterator`; `const_iterator` for constant `a_tran`.	returns an iterator pointing to the first element with key `r` such that `c(ku, r)`, or `a_tran.end()` if such an element is not found.	logarithmic
`a.equal_range(k)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for constant `a`.	equivalent to `make_pair(a.lower_bound(k), a.upper_bound(k))`.	logarithmic

Table 101 — Associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a_tran.equal_range(ke)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for constant `a_tran`.	equivalent to `make_pair(a_tran.lower_bound(ke), a_tran.upper_bound(ke))`.	logarithmic

9 The `insert` and `emplace` members shall not affect the validity of iterators and references to the container, and the `erase` members shall invalidate only iterators and references to the erased elements.

10 The fundamental property of iterators of associative containers is that they iterate through the containers in the non-descending order of keys where non-descending is defined by the comparison that was used to construct them. For any two dereferenceable iterators `i` and `j` such that distance from `i` to `j` is positive,

```
value_comp(*j, *i) == false
```

11 For associative containers with unique keys the stronger condition holds,

```
value_comp(*i, *j) != false.
```

12 When an associative container is constructed by passing a comparison object the container shall not store a pointer or reference to the passed object, even if that object is passed by reference. When an associative container is copied, either through a copy constructor or an assignment operator, the target container shall then use the comparison object from the container being copied, as if that comparison object had been passed to the target container in its constructor.

13 The member function templates `find`, `count`, `lower_bound`, `upper_bound`, and `equal_range` shall not participate in overload resolution unless the *qualified-id* `Compare::is_transparent` is valid and denotes a type (14.8.2).

23.2.4.1 Exception safety guarantees [associative.reqmts.except]

1 For associative containers, no `clear()` function throws an exception. `erase(k)` does not throw an exception unless that exception is thrown by the container's `Compare` object (if any).

2 For associative containers, if an exception is thrown by any operation from within an `insert` or `emplace` function inserting a single element, the insertion has no effect.

3 For associative containers, no `swap` function throws an exception unless that exception is thrown by the swap of the container's `Compare` object (if any).

23.2.5 Unordered associative containers [unord.req]

1 Unordered associative containers provide an ability for fast retrieval of data based on keys. The worst-case complexity for most operations is linear, but the average case is much faster. The library provides four unordered associative containers: `unordered_set`, `unordered_map`, `unordered_multiset`, and `unordered_multimap`.

2 Unordered associative containers conform to the requirements for Containers (23.2), except that the expressions `a == b` and `a != b` have different semantics than for the other container types.

3 Each unordered associative container is parameterized by `Key`, by a function object type `Hash` that meets the `Hash` requirements (17.6.3.4) and acts as a hash function for argument values of type `Key`, and by a binary predicate `Pred` that induces an equivalence relation on values of type `Key`. Additionally, `unordered_map` and `unordered_multimap` associate an arbitrary *mapped type* `T` with the `Key`.

4 The container's object of type `Hash` — denoted by `hash` — is called the *hash function* of the container. The container's object of type `Pred` — denoted by `pred` — is called the *key equality predicate* of the container.

5 Two values `k1` and `k2` of type `Key` are considered equivalent if the container's key equality predicate returns `true` when passed those values. If `k1` and `k2` are equivalent, the container's hash function shall return the same value for both. [*Note:* Thus, when an unordered associative container is instantiated with a non-default `Pred` parameter it usually needs a non-default `Hash` parameter as well. — *end note*] For any two keys `k1` and `k2` in the same container, calling `pred(k1, k2)` shall always return the same value. For any key `k` in a container, calling `hash(k)` shall always return the same value.

6 An unordered associative container supports *unique keys* if it may contain at most one element for each key. Otherwise, it supports *equivalent keys*. `unordered_set` and `unordered_map` support unique keys. `unordered_multiset` and `unordered_multimap` support equivalent keys. In containers that support equivalent keys, elements with equivalent keys are adjacent to each other in the iteration order of the container. Thus, although the absolute order of elements in an unordered container is not specified, its elements are grouped into *equivalent-key group*s such that all elements of each group have equivalent keys. Mutating operations on unordered containers shall preserve the relative order of elements within each equivalent-key group unless otherwise specified.

7 For `unordered_set` and `unordered_multiset` the value type is the same as the key type. For `unordered_map` and `unordered_multimap` it is `std::pair<const Key, T>`.

8 For unordered containers where the value type is the same as the key type, both `iterator` and `const_iterator` are constant iterators. It is unspecified whether or not `iterator` and `const_iterator` are the same type. [*Note:* `iterator` and `const_iterator` have identical semantics in this case, and `iterator` is convertible to `const_iterator`. Users can avoid violating the one-definition rule by always using `const_iterator` in their function parameter lists. — *end note*]

9 The elements of an unordered associative container are organized into *buckets*. Keys with the same hash code appear in the same bucket. The number of buckets is automatically increased as elements are added to an unordered associative container, so that the average number of elements per bucket is kept below a bound. Rehashing invalidates iterators, changes ordering between elements, and changes which buckets elements appear in, but does not invalidate pointers or references to elements. For `unordered_multiset` and `unordered_multimap`, rehashing preserves the relative ordering of equivalent elements.

10 The unordered associative containers meet all the requirements of Allocator-aware containers (23.2.1), except that for `unordered_map` and `unordered_multimap`, the requirements placed on `value_type` in Table 95 apply instead to `key_type` and `mapped_type`. [*Note:* For example, `key_type` and `mapped_type` are sometimes required to be `CopyAssignable` even though the associated `value_type`, `pair<const key_type, mapped_type>`, is not `CopyAssignable`. — *end note*]

11 In Table 102: `X` denotes an unordered associative container class, `a` denotes a value of type `X`, `b` denotes a possibly const value of type `X`, `a_uniq` denotes a value of type `X` when `X` supports unique keys, `a_eq` denotes a value of type `X` when `X` supports equivalent keys, `i` and `j` denote input iterators that refer to `value_type`, `[i, j)` denotes a valid range, `p` and `q2` denote valid const iterators to `a`, `q` and `q1` denote valid dereferenceable const iterators to `a`, `r` denotes a valid dereferenceable iterator to `a`, `[q1, q2)` denotes a valid range in `a`, `il` designates an object of type `initializer_list<value_type>`, `t` denotes a value of type `X::value_type`, `k` denotes a value of type `key_type`, `hf` denotes a possibly const value of type `hasher`, `eq` denotes a possibly const value of type `key_equal`, `n` denotes a value of type `size_type`, and `z` denotes a value of type `float`.

Table 102 — Unordered associative container requirements (in addition to container)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X::key_type`	`Key`		compile time
`X::mapped_type` (`unordered_map` and `unordered_multimap` only)	`T`		compile time
`X::value_type` (`unordered_set` and `unordered_multiset` only)	`Key`	*Requires:* `value_type` is `Erasable` from `X`	compile time
`X::value_type` (`unordered_map` and `unordered_multimap` only)	`pair<const Key, T>`	*Requires:* `value_type` is `Erasable` from `X`	compile time
`X::hasher`	`Hash`	`Hash` shall be a unary function object type such that the expression `hf(k)` has type `std::size_t`.	compile time
`X::key_equal`	`Pred`	`Pred` shall be a binary predicate that takes two arguments of type `Key`. `Pred` is an equivalence relation.	compile time
`X::local_iterator`	An iterator type whose category, value type, difference type, and pointer and reference types are the same as `X::iterator`'s.	A `local_iterator` object may be used to iterate through a single bucket, but may not be used to iterate across buckets.	compile time
`X::const_local_iterator`	An iterator type whose category, value type, difference type, and pointer and reference types are the same as `X::const_iterator`'s.	A `const_local_iterator` object may be used to iterate through a single bucket, but may not be used to iterate across buckets.	compile time
`X(n, hf, eq)` `X a(n, hf, eq)`	X	*Requires:* `hasher` and `key_equal` are `CopyConstructible`. *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `eq` as the key equality predicate.	$\mathcal{O}(n)$

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X(n, hf)` `X a(n, hf)`	X	*Requires:* `hasher` is `CopyConstructible` and `key_equal` is `DefaultConstructible`. *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `key_equal()` as the key equality predicate.	$\mathcal{O}(n)$
`X(n)` `X a(n)`	X	*Requires:* `hasher` and `key_equal` are `DefaultConstructible`. *Effects:* Constructs an empty container with at least `n` buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate.	$\mathcal{O}(n)$
`X()` `X a`	X	*Requires:* `hasher` and `key_equal` are `DefaultConstructible`. *Effects:* Constructs an empty container with an unspecified number of buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate.	constant
`X(i, j, n, hf, eq)` `X a(i, j, n, hf, eq)`	X	*Requires:* `hasher` and `key_equal` are `CopyConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `eq` as the key equality predicate, and inserts elements from `[i, j)` into it.	Average case $\mathcal{O}(N)$ (N is `distance(i, j)`), worst case $\mathcal{O}(N^2)$

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X(i, j, n, hf)` `X a(i, j, n, hf)`	X	*Requires:* `hasher` is `CopyConstructible` and `key_equal` is `DefaultConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Constructs an empty container with at least `n` buckets, using `hf` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `[i, j)` into it.	Average case $\mathcal{O}(N)$ (N is `distance(i, j)`), worst case $\mathcal{O}(N^2)$
`X(i, j, n)` `X a(i, j, n)`	X	*Requires:* `hasher` and `key_equal` are `DefaultConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Constructs an empty container with at least `n` buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `[i, j)` into it.	Average case $\mathcal{O}(N)$ (N is `distance(i, j)`), worst case $\mathcal{O}(N^2)$
`X(i, j)` `X a(i, j)`	X	*Requires:* `hasher` and `key_equal` are `DefaultConstructible`. `value_type` is `EmplaceConstructible` into `X` from `*i`. *Effects:* Constructs an empty container with an unspecified number of buckets, using `hasher()` as the hash function and `key_equal()` as the key equality predicate, and inserts elements from `[i, j)` into it.	Average case $\mathcal{O}(N)$ (N is `distance(i, j)`), worst case $\mathcal{O}(N^2)$
`X(il)`	`X`	Same as `X(il.begin(), il.end())`.	Same as `X(il.begin(), il.end())`.
`X(il, n)`	`X`	Same as `X(il.begin(), il.end(), n)`.	Same as `X(il.begin(), il.end(), n)`.

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`X(il, n, hf)`	`X`	Same as `X(il.begin(), il.end(), n, hf)`.	Same as `X(il.begin(), il.end(), n, hf)`.
`X(il, n, hf, eq)`	`X`	Same as `X(il.begin(), il.end(), n, hf, eq)`.	Same as `X(il.begin(), il.end(), n, hf, eq)`.
`X(b)` `X a(b)`	`X`	Copy constructor. In addition to the requirements of Table 95, copies the hash function, predicate, and maximum load factor.	Average case linear in `b.size()`, worst case quadratic.
`a = b`	`X&`	Copy assignment operator. In addition to the requirements of Table 95, copies the hash function, predicate, and maximum load factor.	Average case linear in `b.size()`, worst case quadratic.
`a = il`	`X&`	*Requires:* `value_type` is `CopyInsertable` into `X` and `CopyAssignable`. *Effects:* Assigns the range `[il.begin(),il.end())` into `a`. All existing elements of `a` are either assigned to or destroyed.	Same as `a = X(il)`.
`b.hash_function()`	`hasher`	Returns `b`'s hash function.	constant
`b.key_eq()`	`key_equal`	Returns `b`'s key equality predicate.	constant
`a_uniq.` `emplace(args)`	`pair<iterator, bool>`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `args`. *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...` if and only if there is no element in the container with key equivalent to the key of `t`. The `bool` component of the returned pair is true if and only if the insertion takes place, and the iterator component of the pair points to the element with key equivalent to the key of `t`.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a_uniq.size()`).

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a_eq.emplace(args)`	`iterator`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `args`. *Effects:* Inserts a `value_type` object `t` constructed with `std::forward<Args>(args)...` and returns the iterator pointing to the newly inserted element.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a_eq.size()`).
`a.emplace_hint(p, args)`	`iterator`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `args`. *Effects:* Equivalent to `a.emplace( std::forward<Args>(args)...)`. Return value is an iterator pointing to the element with the key equivalent to the newly inserted element. The `const_iterator p` is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a.size()`).
`a_uniq.insert(t)`	`pair<iterator, bool>`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Inserts `t` if and only if there is no element in the container with key equivalent to the key of `t`. The `bool` component of the returned pair indicates whether the insertion takes place, and the `iterator` component points to the element with key equivalent to the key of `t`.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a_uniq.size()`).

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a_eq.insert(t)`	`iterator`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Inserts `t`, and returns an iterator pointing to the newly inserted element.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a_eq.size()`).
`a.insert(q, t)`	`iterator`	*Requires:* If `t` is a non-const rvalue expression, `value_type` shall be `MoveInsertable` into `X`; otherwise, `value_type` shall be `CopyInsertable` into `X`. *Effects:* Equivalent to a.insert(t). Return value is an iterator pointing to the element with the key equivalent to that of `t`. The iterator `q` is a hint pointing to where the search should start. Implementations are permitted to ignore the hint.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a.size()`).
`a.insert(i, j)`	`void`	*Requires:* `value_type` shall be `EmplaceConstructible` into `X` from `*i`. Pre: `i` and `j` are not iterators in `a`. Equivalent to `a.insert(t)` for each element in `[i,j)`.	Average case $\mathcal{O}(N)$, where N is `distance(i, j)`. Worst case $\mathcal{O}(N *$ `(a.size()) + N`).
`a.insert(il)`	`void`	Same as `a.insert(il.begin(), il.end())`.	Same as `a.insert( il.begin(), il.end())`.
`a.erase(k)`	`size_type`	Erases all elements with key equivalent to `k`. Returns the number of elements erased.	Average case $\mathcal{O}$(`a.count(k)`). Worst case $\mathcal{O}$(`a.size()`).
`a.erase(q)`	`iterator`	Erases the element pointed to by `q`. Returns the iterator immediately following `q` prior to the erasure.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a.size()`).
`a.erase(r)`	`iterator`	Erases the element pointed to by `r`. Returns the iterator immediately following `r` prior to the erasure.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`a.size()`).

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`a.erase(q1, q2)`	`iterator`	Erases all elements in the range `[q1, q2)`. Returns the iterator immediately following the erased elements prior to the erasure.	Average case linear in `distance(q1, q2)`, worst case $\mathcal{O}$(`a.size()`).
`a.clear()`	`void`	Erases all elements in the container. Post: `a.empty()` returns `true`	Linear.
`b.find(k)`	`iterator`; `const_iterator` for const `b`.	Returns an iterator pointing to an element with key equivalent to `k`, or `b.end()` if no such element exists.	Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`b.size()`).
`b.count(k)`	`size_type`	Returns the number of elements with key equivalent to `k`.	Average case $\mathcal{O}$(`b.count(k)`), worst case $\mathcal{O}$(`b.size()`).
`b.equal_range(k)`	`pair<iterator, iterator>`; `pair<const_iterator, const_iterator>` for const `b`.	Returns a range containing all elements with keys equivalent to `k`. Returns `make_pair(b.end(), b.end())` if no such elements exist.	Average case $\mathcal{O}$(`b.count(k)`). Worst case $\mathcal{O}$(`b.size()`).
`b.bucket_count()`	`size_type`	Returns the number of buckets that `b` contains.	Constant
`b.max_bucket_count()`	`size_type`	Returns an upper bound on the number of buckets that `b` might ever contain.	Constant
`b.bucket(k)`	`size_type`	Pre: `b.bucket_count() > 0`. Returns the index of the bucket in which elements with keys equivalent to `k` would be found, if any such element existed. Post: the return value shall be in the range `[0, b.bucket_count())`.	Constant
`b.bucket_size(n)`	`size_type`	Pre: `n` shall be in the range `[0, b.bucket_count())`. Returns the number of elements in the `n` th bucket.	$\mathcal{O}$(`b.bucket_size(n)`)
`b.begin(n)`	`local_iterator`; `const_local_iterator` for const `b`.	Pre: `n` shall be in the range `[0, b.bucket_count())`. `b.begin(n)` returns an iterator referring to the first element in the bucket. If the bucket is empty, then `b.begin(n) == b.end(n)`.	Constant

Table 102 — Unordered associative container requirements (in addition to container) (continued)

Expression	Return type	Assertion/note pre-/post-condition	Complexity
`b.end(n)`	`local_iterator`; `const_local_iterator` for const `b`.	Pre: `n` shall be in the range `[0, b.bucket_count())`. `b.end(n)` returns an iterator which is the past-the-end value for the bucket.	Constant
`b.cbegin(n)`	`const_local_iterator`	Pre: `n` shall be in the range `[0, b.bucket_count())`. Note: `[b.cbegin(n), b.cend(n))` is a valid range containing all of the elements in the `n` th bucket.	Constant
`b.cend(n)`	`const_local_iterator`	Pre: `n` shall be in the range `[0, b.bucket_count())`.	Constant
`b.load_factor()`	`float`	Returns the average number of elements per bucket.	Constant
`b.max_load_factor()`	`float`	Returns a positive number that the container attempts to keep the load factor less than or equal to. The container automatically increases the number of buckets as necessary to keep the load factor below this number.	Constant
`a.max_load_factor(z)`	`void`	Pre: `z` shall be positive. May change the container's maximum load factor, using `z` as a hint.	Constant
`a.rehash(n)`	`void`	Post: `a.bucket_count() > a.size() / a.max_load_factor()` and `a.bucket_count() >= n`.	Average case linear in `a.size()`, worst case quadratic.
`a.reserve(n)`	`void`	Same as `a.rehash(ceil(n / a.max_load_factor()))`.	Average case linear in `a.size()`, worst case quadratic.

[12] Two unordered containers `a` and `b` compare equal if `a.size() == b.size()` and, for every equivalent-key group `[Ea1,Ea2)` obtained from `a.equal_range(Ea1)`, there exists an equivalent-key group `[Eb1,Eb2)` obtained from `b.equal_range(Ea1)`, such that `is_permutation(Ea1, Ea2, Eb1, Eb2)` returns `true`. For `unordered_set` and `unordered_map`, the complexity of `operator==` (i.e., the number of calls to the `==` operator of the `value_type`, to the predicate returned by `key_equal()`, and to the hasher returned by `hash_function()`) is proportional to N in the average case and to N^2 in the worst case, where N is a.size(). For `unordered_multiset` and `unordered_multimap`, the complexity of `operator==` is proportional to $\sum E_i^2$ in the average case and to N^2 in the worst case, where N is `a.size()`, and E_i is the size of the ith equivalent-key group in `a`. However, if the respective elements of each corresponding pair of equivalent-key groups Ea_i

and Eb_i are arranged in the same order (as is commonly the case, e.g., if `a` and `b` are unmodified copies of the same container), then the average-case complexity for `unordered_multiset` and `unordered_multimap` becomes proportional to N (but worst-case complexity remains $\mathcal{O}(N^2)$, e.g., for a pathologically bad hash function). The behavior of a program that uses `operator==` or `operator!=` on unordered containers is undefined unless the `Hash` and `Pred` function objects respectively have the same behavior for both containers and the equality comparison operator for `Key` is a refinement[264] of the partition into equivalent-key groups produced by `Pred`.

13 The iterator types `iterator` and `const_iterator` of an unordered associative container are of at least the forward iterator category. For unordered associative containers where the key type and value type are the same, both `iterator` and `const_iterator` are const iterators.

14 The `insert` and `emplace` members shall not affect the validity of references to container elements, but may invalidate all iterators to the container. The `erase` members shall invalidate only iterators and references to the erased elements, and preserve the relative order of the elements that are not erased.

15 The `insert` and `emplace` members shall not affect the validity of iterators if `(N+n) < z * B`, where `N` is the number of elements in the container prior to the insert operation, `n` is the number of elements inserted, `B` is the container's bucket count, and `z` is the container's maximum load factor.

23.2.5.1 Exception safety guarantees [unord.req.except]

1 For unordered associative containers, no `clear()` function throws an exception. `erase(k)` does not throw an exception unless that exception is thrown by the container's `Hash` or `Pred` object (if any).

2 For unordered associative containers, if an exception is thrown by any operation other than the container's hash function from within an `insert` or `emplace` function inserting a single element, the insertion has no effect.

3 For unordered associative containers, no `swap` function throws an exception unless that exception is thrown by the swap of the container's Hash or Pred object (if any).

4 For unordered associative containers, if an exception is thrown from within a `rehash()` function other than by the container's hash function or comparison function, the `rehash()` function has no effect.

23.3 Sequence containers [sequences]

23.3.1 In general [sequences.general]

1 The headers `<array>`, `<deque>`, `<forward_list>`, `<list>`, and `<vector>` define template classes that meet the requirements for sequence containers.

Header `<array>` synopsis

```
#include <initializer_list>

namespace std {
  template <class T, size_t N> struct array;
  template <class T, size_t N>
    bool operator==(const array<T, N>& x, const array<T, N>& y);
  template <class T, size_t N>
    bool operator!=(const array<T, N>& x, const array<T, N>& y);
  template <class T, size_t N>
    bool operator< (const array<T, N>& x, const array<T, N>& y);
  template <class T, size_t N>
    bool operator> (const array<T, N>& x, const array<T, N>& y);
  template <class T, size_t N>
    bool operator<=(const array<T, N>& x, const array<T, N>& y);
```

264) Equality comparison is a refinement of partitioning if no two objects that compare equal fall into different partitions.

```
  template <class T, size_t N>
    bool operator>=(const array<T, N>& x, const array<T, N>& y);
  template <class T, size_t N>
    void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));

  template <class T> class tuple_size;
  template <size_t I, class T> class tuple_element;
  template <class T, size_t N>
    struct tuple_size<array<T, N> >;
  template <size_t I, class T, size_t N>
    struct tuple_element<I, array<T, N> >;
  template <size_t I, class T, size_t N>
    constexpr T& get(array<T, N>&) noexcept;
  template <size_t I, class T, size_t N>
    constexpr T&& get(array<T, N>&&) noexcept;
  template <size_t I, class T, size_t N>
    constexpr const T& get(const array<T, N>&) noexcept;
}
```

Header <deque> synopsis

```
#include <initializer_list>

namespace std {
  template <class T, class Allocator = allocator<T> > class deque;
  template <class T, class Allocator>
    bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

Header <forward_list> synopsis

```
#include <initializer_list>

namespace std {
  template <class T, class Allocator = allocator<T> > class forward_list;
  template <class T, class Allocator>
    bool operator==(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
```

```
  template <class T, class Allocator>
    bool operator>=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

Header <list> synopsis

```
#include <initializer_list>

namespace std {
  template <class T, class Allocator = allocator<T> > class list;
  template <class T, class Allocator>
    bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    void swap(list<T, Allocator>& x, list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

Header <vector> synopsis

```
#include <initializer_list>

namespace std {
  template <class T, class Allocator = allocator<T> > class vector;
  template <class T, class Allocator>
    bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Allocator> class vector<bool,Allocator>;
```

```
  // hash support
  template <class T> struct hash;
  template <class Allocator> struct hash<vector<bool, Allocator> >;
}
```

23.3.2 Class template array [array]

23.3.2.1 Class template array overview [array.overview]

1 The header <array> defines a class template for storing fixed-size sequences of objects. An array is a contiguous container (23.2.1). An instance of array<T, N> stores N elements of type T, so that size() == N is an invariant.

2 An array is an aggregate (8.5.1) that can be initialized with the syntax

```
array<T, N> a = { initializer-list };
```

where *initializer-list* is a comma-separated list of up to N elements whose types are convertible to T.

3 An array satisfies all of the requirements of a container and of a reversible container (23.2), except that a default constructed array object is not empty and that swap does not have constant complexity. An array satisfies some of the requirements of a sequence container (23.2.3). Descriptions are provided here only for operations on array that are not described in one of these tables and for operations where there is additional semantic information.

```
namespace std {
  template <class T, size_t N>
  struct array {
    // types:
    typedef T&                                    reference;
    typedef const T&                              const_reference;
    typedef implementation-defined                iterator;
    typedef implementation-defined                const_iterator;
    typedef size_t                                size_type;
    typedef ptrdiff_t                             difference_type;
    typedef T                                     value_type;
    typedef T*                                    pointer;
    typedef const T*                              const_pointer;
    typedef std::reverse_iterator<iterator>       reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    T       elems[N];           // exposition only

    // no explicit construct/copy/destroy for aggregate type

    void fill(const T& u);
    void swap(array&) noexcept(noexcept(swap(declval<T&>(), declval<T&>())));

    // iterators:
    iterator               begin() noexcept;
    const_iterator         begin() const noexcept;
    iterator               end() noexcept;
    const_iterator         end() const noexcept;

    reverse_iterator       rbegin() noexcept;
    const_reverse_iterator rbegin() const noexcept;
    reverse_iterator       rend() noexcept;
```

```
    const_reverse_iterator rend() const noexcept;

    const_iterator         cbegin() const noexcept;
    const_iterator         cend() const noexcept;
    const_reverse_iterator crbegin() const noexcept;
    const_reverse_iterator crend() const noexcept;

    // capacity:
    constexpr bool      empty() const noexcept;
    constexpr size_type size() const noexcept;
    constexpr size_type max_size() const noexcept;

    // element access:
    reference                 operator[](size_type n);
    constexpr const_reference operator[](size_type n) const;
    reference                 at(size_type n);
    constexpr const_reference at(size_type n) const;
    reference                 front();
    constexpr const_reference front() const;
    reference                 back();
    constexpr const_reference back() const;

    T *       data() noexcept;
    const T * data() const noexcept;
  };
}
```

4 [*Note:* The member variable `elems` is shown for exposition only, to emphasize that `array` is a class aggregate. The name `elems` is not part of `array`'s interface. — *end note*]

23.3.2.2 `array` constructors, copy, and assignment [array.cons]

1 The conditions for an aggregate (8.5.1) shall be met. Class `array` relies on the implicitly-declared special member functions (12.1, 12.4, and 12.8) to conform to the container requirements table in 23.2. In addition to the requirements specified in the container requirements table, the implicit move constructor and move assignment operator for `array` require that `T` be `MoveConstructible` or `MoveAssignable`, respectively.

23.3.2.3 `array` specialized algorithms [array.special]

```
template <class T, size_t N>
  void swap(array<T, N>& x, array<T, N>& y) noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

2 *Complexity:* Linear in `N`.

23.3.2.4 `array::size` [array.size]

```
template <class T, size_t N> constexpr size_type array<T, N>::size() const noexcept;
```

1 *Returns:* `N`

23.3.2.5 `array::data` [array.data]

```
T* data() noexcept;
const T* data() const noexcept;
```

1 *Returns:* `elems`.

23.3.2.6 `array::fill` [array.fill]

```
void fill(const T& u);
```

1 *Effects:* `fill_n(begin(), N, u)`

23.3.2.7 `array::swap` [array.swap]

```
void swap(array& y) noexcept(noexcept(swap(declval<T&>(), declval<T&>())));
```

1 *Effects:* `swap_ranges(begin(), end(), y.begin())`

2 *Throws:* Nothing unless one of the element-wise swap calls throws an exception.

3 *Note:* Unlike the `swap` function for other containers, `array::swap` takes linear time, may exit via an exception, and does not cause iterators to become associated with the other container.

23.3.2.8 Zero sized arrays [array.zero]

1 `array` shall provide support for the special case `N == 0`.

2 In the case that `N == 0`, `begin() == end() ==` unique value. The return value of `data()` is unspecified.

3 The effect of calling `front()` or `back()` for a zero-sized array is undefined.

4 Member function `swap()` shall have a *noexcept-specification* which is equivalent to `noexcept(true)`.

23.3.2.9 Tuple interface to class template `array` [array.tuple]

```
template <class T, size_t N>
  struct tuple_size<array<T, N>> : integral_constant<size_t, N> { };
```

```
tuple_element<I, array<T, N> >::type
```

1 *Requires:* `I < N`. The program is ill-formed if `I` is out of bounds.

2 *Value:* The type T.

```
template <size_t I, class T, size_t N>
  constexpr T& get(array<T, N>& a) noexcept;
```

3 *Requires:* `I < N`. The program is ill-formed if `I` is out of bounds.

4 *Returns:* A reference to the `I`th element of `a`, where indexing is zero-based.

```
template <size_t I, class T, size_t N>
  constexpr T&& get(array<T, N>&& a) noexcept;
```

5 *Effects:* Equivalent to `return std::move(get<I>(a));`

```
template <size_t I, class T, size_t N>
  constexpr const T& get(const array<T, N>& a) noexcept;
```

6 *Requires:* `I < N`. The program is ill-formed if `I` is out of bounds.

7 *Returns:* A const reference to the `I`th element of `a`, where indexing is zero-based.

23.3.3 Class template `deque` [deque]

23.3.3.1 Class template `deque` overview [deque.overview]

1 A `deque` is a sequence container that, like a `vector` (23.3.6), supports random access iterators. In addition, it supports constant time insert and erase operations at the beginning or the end; insert and erase in the middle take linear time. That is, a deque is especially optimized for pushing and popping elements at the beginning and end. As with vectors, storage management is handled automatically.

2 A `deque` satisfies all of the requirements of a container, of a reversible container (given in tables in 23.2), of a sequence container, including the optional sequence container requirements (23.2.3), and of an allocator-aware container (Table 98). Descriptions are provided here only for operations on `deque` that are not described in one of these tables or for operations where there is additional semantic information.

```
namespace std {
  template <class T, class Allocator = allocator<T> >
  class deque {
  public:
    // types:
    typedef value_type&                             reference;
    typedef const value_type&                       const_reference;
    typedef implementation-defined                  iterator;       // See 23.2
    typedef implementation-defined                  const_iterator; // See 23.2
    typedef implementation-defined                  size_type;      // See 23.2
    typedef implementation-defined                  difference_type;// See 23.2
    typedef T                                       value_type;
    typedef Allocator                               allocator_type;
    typedef typename allocator_traits<Allocator>::pointer           pointer;
    typedef typename allocator_traits<Allocator>::const_pointer     const_pointer;
    typedef std::reverse_iterator<iterator>         reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    // 23.3.3.2, construct/copy/destroy:
    deque() : deque(Allocator()) { }
    explicit deque(const Allocator&);
    explicit deque(size_type n, const Allocator& = Allocator());
    deque(size_type n, const T& value, const Allocator& = Allocator());
    template <class InputIterator>
      deque(InputIterator first, InputIterator last, const Allocator& = Allocator());
    deque(const deque& x);
    deque(deque&&);
    deque(const deque&, const Allocator&);
    deque(deque&&, const Allocator&);
    deque(initializer_list<T>, const Allocator& = Allocator());

    ~deque();
    deque& operator=(const deque& x);
    deque& operator=(deque&& x)
      noexcept(allocator_traits<Allocator>::is_always_equal::value);
    deque& operator=(initializer_list<T>);
    template <class InputIterator>
      void assign(InputIterator first, InputIterator last);
    void assign(size_type n, const T& t);
    void assign(initializer_list<T>);
    allocator_type get_allocator() const noexcept;

    // iterators:
    iterator               begin() noexcept;
    const_iterator         begin() const noexcept;
    iterator               end() noexcept;
    const_iterator         end() const noexcept;
    reverse_iterator       rbegin() noexcept;
    const_reverse_iterator rbegin() const noexcept;
    reverse_iterator       rend() noexcept;
```

```
  const_reverse_iterator rend() const noexcept;

  const_iterator         cbegin() const noexcept;
  const_iterator         cend() const noexcept;
  const_reverse_iterator crbegin() const noexcept;
  const_reverse_iterator crend() const noexcept;

  // 23.3.3.3, capacity:
  bool      empty() const noexcept;
  size_type size() const noexcept;
  size_type max_size() const noexcept;
  void      resize(size_type sz);
  void      resize(size_type sz, const T& c);
  void      shrink_to_fit();

  // element access:
  reference       operator[](size_type n);
  const_reference operator[](size_type n) const;
  reference       at(size_type n);
  const_reference at(size_type n) const;
  reference       front();
  const_reference front() const;
  reference       back();
  const_reference back() const;

  // 23.3.3.4, modifiers:
  template <class... Args> void emplace_front(Args&&... args);
  template <class... Args> void emplace_back(Args&&... args);
  template <class... Args> iterator emplace(const_iterator position, Args&&... args);

  void push_front(const T& x);
  void push_front(T&& x);
  void push_back(const T& x);
  void push_back(T&& x);

  iterator insert(const_iterator position, const T& x);
  iterator insert(const_iterator position, T&& x);
  iterator insert(const_iterator position, size_type n, const T& x);
  template <class InputIterator>
    iterator insert (const_iterator position, InputIterator first, InputIterator last);
  iterator insert(const_iterator position, initializer_list<T>);

  void pop_front();
  void pop_back();

  iterator erase(const_iterator position);
  iterator erase(const_iterator first, const_iterator last);
  void     swap(deque&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
  void     clear() noexcept;
};

template <class T, class Allocator>
  bool operator==(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
template <class T, class Allocator>
```

```
    bool operator< (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const deque<T, Allocator>& x, const deque<T, Allocator>& y);

  // specialized algorithms:
  template <class T, class Allocator>
    void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.3.3.2 `deque` constructors, copy, and assignment [deque.cons]

```
explicit deque(const Allocator&);
```

1 *Effects:* Constructs an empty `deque`, using the specified allocator.

2 *Complexity:* Constant.

```
explicit deque(size_type n, const Allocator& = Allocator());
```

3 *Effects:* Constructs a `deque` with `n` default-inserted elements using the specified allocator.

4 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

5 *Complexity:* Linear in `n`.

```
deque(size_type n, const T& value, const Allocator& = Allocator());
```

6 *Effects:* Constructs a `deque` with `n` copies of `value`, using the specified allocator.

7 *Requires:* `T` shall be `CopyInsertable` into `*this`.

8 *Complexity:* Linear in `n`.

```
template <class InputIterator>
  deque(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

9 *Effects:* Constructs a `deque` equal to the range `[first,last)`, using the specified allocator.

10 *Complexity:* Linear in `distance(first, last)`.

23.3.3.3 `deque` capacity [deque.capacity]

```
void resize(size_type sz);
```

1 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

2 *Requires:* `T` shall be `MoveInsertable` and `DefaultInsertable` into `*this`.

```
void resize(size_type sz, const T& c);
```

3 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

4 *Requires:* `T` shall be `CopyInsertable` into `*this`.

```
void shrink_to_fit();
```

5 *Requires:* `T` shall be `MoveInsertable` into `*this`.

6 *Complexity:* Linear in the size of the sequence.

7 *Remarks:* `shrink_to_fit` is a non-binding request to reduce memory use but does not change the size of the sequence. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*]

23.3.3.4 `deque` modifiers [deque.modifiers]

```
iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  iterator insert(const_iterator position,
                  InputIterator first, InputIterator last);
iterator insert(const_iterator position, initializer_list<T>);

template <class... Args> void emplace_front(Args&&... args);
template <class... Args> void emplace_back(Args&&... args);
template <class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
void push_back(const T& x);
void push_back(T&& x);
```

1 *Effects:* An insertion in the middle of the deque invalidates all the iterators and references to elements of the deque. An insertion at either end of the deque invalidates all the iterators to the deque, but has no effect on the validity of references to elements of the deque.

2 *Remarks:* If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `T` there are no effects. If an exception is thrown while inserting a single element at either end, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`CopyInsertable` `T`, the effects are unspecified.

3 *Complexity:* The complexity is linear in the number of elements inserted plus the lesser of the distances to the beginning and end of the deque. Inserting a single element either at the beginning or end of a deque always takes constant time and causes a single call to a constructor of `T`.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_front();
void pop_back();
```

4 *Effects:* An erase operation that erases the last element of a deque invalidates only the past-the-end iterator and all iterators and references to the erased elements. An erase operation that erases the first element of a deque but not the last element invalidates only iterators and references to the erased elements. An erase operation that erases neither the first element nor the last element of a deque invalidates the past-the-end iterator and all iterators and references to all the elements of the deque. [*Note:* `pop_front` and `pop_back` are erase operations. — *end note*]

5 *Complexity:* The number of calls to the destructor is the same as the number of elements erased, but the number of calls to the assignment operator is no more than the lesser of the number of elements before the erased elements and the number of elements after the erased elements.

6 *Throws:* Nothing unless an exception is thrown by the copy constructor, move constructor, assignment operator, or move assignment operator of `T`.

23.3.3.5 deque specialized algorithms [deque.special]

```
template <class T, class Allocator>
  void swap(deque<T, Allocator>& x, deque<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.3.4 Class template forward_list [forwardlist]

23.3.4.1 Class template forward_list overview [forwardlist.overview]

1 A `forward_list` is a container that supports forward iterators and allows constant time insert and erase operations anywhere within the sequence, with storage management handled automatically. Fast random access to list elements is not supported. [*Note:* It is intended that `forward_list` have zero space or time overhead relative to a hand-written C-style singly linked list. Features that would conflict with that goal have been omitted. — *end note*]

2 A `forward_list` satisfies all of the requirements of a container (Table 95), except that the `size()` member function is not provided and `operator==` has linear complexity. A `forward_list` also satisfies all of the requirements for an allocator-aware container (Table 98). In addition, a `forward_list` provides the `assign` member functions (Table 99) and several of the optional container requirements (Table 100). Descriptions are provided here only for operations on `forward_list` that are not described in that table or for operations where there is additional semantic information.

3 [*Note:* Modifying any list requires access to the element preceding the first element of interest, but in a `forward_list` there is no constant-time way to access a preceding element. For this reason, ranges that are modified, such as those supplied to `erase` and `splice`, must be open at the beginning. — *end note*]

```
namespace std {
  template <class T, class Allocator = allocator<T> >
  class forward_list {
  public:
    // types:
    typedef value_type&                                          reference;
    typedef const value_type&                                    const_reference;
    typedef implementation-defined iterator;       // See 23.2
    typedef implementation-defined const_iterator; // See 23.2
    typedef implementation-defined size_type;      // See 23.2
    typedef implementation-defined difference_type;// See 23.2
    typedef T value_type;
    typedef Allocator allocator_type;
    typedef typename allocator_traits<Allocator>::pointer         pointer;
    typedef typename allocator_traits<Allocator>::const_pointer   const_pointer;

    // 23.3.4.2, construct/copy/destroy:
    forward_list() : forward_list(Allocator()) { }
    explicit forward_list(const Allocator&);
    explicit forward_list(size_type n, const Allocator& = Allocator());
    forward_list(size_type n, const T& value,
                 const Allocator& = Allocator());
    template <class InputIterator>
      forward_list(InputIterator first, InputIterator last,
                   const Allocator& = Allocator());
    forward_list(const forward_list& x);
    forward_list(forward_list&& x);
```

```
forward_list(const forward_list& x, const Allocator&);
forward_list(forward_list&& x, const Allocator&);
forward_list(initializer_list<T>, const Allocator& = Allocator());
~forward_list();
forward_list& operator=(const forward_list& x);
forward_list& operator=(forward_list&& x)
  noexcept(allocator_traits<Allocator>::is_always_equal::value);
forward_list& operator=(initializer_list<T>);
template <class InputIterator>
  void assign(InputIterator first, InputIterator last);
void assign(size_type n, const T& t);
void assign(initializer_list<T>);
allocator_type get_allocator() const noexcept;

// 23.3.4.3, iterators:
iterator before_begin() noexcept;
const_iterator before_begin() const noexcept;
iterator begin() noexcept;
const_iterator begin() const noexcept;
iterator end() noexcept;
const_iterator end() const noexcept;

const_iterator cbegin() const noexcept;
const_iterator cbefore_begin() const noexcept;
const_iterator cend() const noexcept;

// capacity:
bool empty() const noexcept;
size_type max_size() const noexcept;

// 23.3.4.4, element access:
reference front();
const_reference front() const;

// 23.3.4.5, modifiers:
template <class... Args> void emplace_front(Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
void pop_front();

template <class... Args> iterator emplace_after(const_iterator position, Args&&... args);
iterator insert_after(const_iterator position, const T& x);
iterator insert_after(const_iterator position, T&& x);

iterator insert_after(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  iterator insert_after(const_iterator position, InputIterator first, InputIterator last);
iterator insert_after(const_iterator position, initializer_list<T> il);

iterator erase_after(const_iterator position);
iterator erase_after(const_iterator position, const_iterator last);
void swap(forward_list&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value);

void resize(size_type sz);
```

```
    void resize(size_type sz, const value_type& c);
    void clear() noexcept;

    // 23.3.4.6, forward_list operations:
    void splice_after(const_iterator position, forward_list& x);
    void splice_after(const_iterator position, forward_list&& x);
    void splice_after(const_iterator position, forward_list& x,
                      const_iterator i);
    void splice_after(const_iterator position, forward_list&& x,
                      const_iterator i);
    void splice_after(const_iterator position, forward_list& x,
                      const_iterator first, const_iterator last);
    void splice_after(const_iterator position, forward_list&& x,
                      const_iterator first, const_iterator last);

    void remove(const T& value);
    template <class Predicate> void remove_if(Predicate pred);

    void unique();
    template <class BinaryPredicate> void unique(BinaryPredicate binary_pred);

    void merge(forward_list& x);
    void merge(forward_list&& x);
    template <class Compare> void merge(forward_list& x, Compare comp);
    template <class Compare> void merge(forward_list&& x, Compare comp);

    void sort();
    template <class Compare> void sort(Compare comp);

    void reverse() noexcept;
  };

  // Comparison operators
  template <class T, class Allocator>
    bool operator==(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const forward_list<T, Allocator>& x, const forward_list<T, Allocator>& y);

  // 23.3.4.7, specialized algorithms:
  template <class T, class Allocator>
    void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

[4] An incomplete type `T` may be used when instantiating `forward_list` if the allocator satisfies the allocator completeness requirements 17.6.3.5.1. `T` shall be complete before any member of the resulting specialization of `forward_list` is referenced.

23.3.4.2 `forward_list` constructors, copy, assignment [forwardlist.cons]

```
explicit forward_list(const Allocator&);
```

1 *Effects:* Constructs an empty `forward_list` object using the specified allocator.

2 *Complexity:* Constant.

```
explicit forward_list(size_type n, const Allocator& = Allocator());
```

3 *Effects:* Constructs a `forward_list` object with `n` default-inserted elements using the specified allocator.

4 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

5 *Complexity:* Linear in `n`.

```
forward_list(size_type n, const T& value, const Allocator& = Allocator());
```

6 *Effects:* Constructs a `forward_list` object with `n` copies of `value` using the specified allocator.

7 *Requires:* `T` shall be `CopyInsertable` into `*this`.

8 *Complexity:* Linear in `n`.

```
template <class InputIterator>
  forward_list(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

9 *Effects:* Constructs a `forward_list` object equal to the range `[first,last)`.

10 *Complexity:* Linear in `distance(first, last)`.

23.3.4.3 `forward_list` iterators [forwardlist.iter]

```
iterator before_begin() noexcept;
const_iterator before_begin() const noexcept;
const_iterator cbefore_begin() const noexcept;
```

1 *Returns:* A non-dereferenceable iterator that, when incremented, is equal to the iterator returned by `begin()`.

2 *Effects:* `cbefore_begin()` is equivalent to `const_cast<forward_list const&>(*this).before_begin()`.

3 *Remarks:* `before_begin() == end()` shall equal `false`.

23.3.4.4 `forward_list` element access [forwardlist.access]

```
reference front();
const_reference front() const;
```

1 *Returns:* `*begin()`

23.3.4.5 `forward_list` modifiers [forwardlist.modifiers]

1 None of the overloads of `insert_after` shall affect the validity of iterators and references, and `erase_after` shall invalidate only iterators and references to the erased elements. If an exception is thrown during `insert_after` there shall be no effect. Inserting `n` elements into a `forward_list` is linear in `n`, and the number of calls to the copy or move constructor of `T` is exactly equal to `n`. Erasing `n` elements from a `forward_list` is linear in `n` and the number of calls to the destructor of type `T` is exactly equal to `n`.

```
template <class... Args> void emplace_front(Args&&... args);
```

2 *Effects:* Inserts an object of type `value_type` constructed with `value_type(std::forward<Args>(args)...)` at the beginning of the list.

```
void push_front(const T& x);
void push_front(T&& x);
```

3 *Effects:* Inserts a copy of `x` at the beginning of the list.

```
void pop_front();
```

4 *Effects:* `erase_after(before_begin())`

```
iterator insert_after(const_iterator position, const T& x);
iterator insert_after(const_iterator position, T&& x);
```

5 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`.

6 *Effects:* Inserts a copy of `x` after `position`.

7 *Returns:* An iterator pointing to the copy of `x`.

```
iterator insert_after(const_iterator position, size_type n, const T& x);
```

8 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`.

9 *Effects:* Inserts `n` copies of `x` after `position`.

10 *Returns:* An iterator pointing to the last inserted copy of `x` or `position` if `n == 0`.

```
template <class InputIterator>
  iterator insert_after(const_iterator position, InputIterator first, InputIterator last);
```

11 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`. `first` and `last` are not iterators in `*this`.

12 *Effects:* Inserts copies of elements in `[first,last)` after `position`.

13 *Returns:* An iterator pointing to the last inserted element or `position` if `first == last`.

```
iterator insert_after(const_iterator position, initializer_list<T> il);
```

14 *Effects:* `insert_after(p, il.begin(), il.end())`.

15 *Returns:* An iterator pointing to the last inserted element or `position` if `il` is empty.

```
template <class... Args>
  iterator emplace_after(const_iterator position, Args&&... args);
```

16 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`.

17 *Effects:* Inserts an object of type `value_type` constructed with `value_type(std::forward<Args>(args)...)` after `position`.

18 *Returns:* An iterator pointing to the new object.

```
iterator erase_after(const_iterator position);
```

19 *Requires:* The iterator following `position` is dereferenceable.

20 *Effects:* Erases the element pointed to by the iterator following `position`.

21 *Returns:* An iterator pointing to the element following the one that was erased, or `end()` if no such element exists.

22 *Throws:* Nothing.

```
iterator erase_after(const_iterator position, const_iterator last);
```

23 *Requires:* All iterators in the range `(position,last)` are dereferenceable.

24 *Effects:* Erases the elements in the range `(position,last)`.

25 *Returns:* `last`.

26 *Throws:* Nothing.

```
void resize(size_type sz);
```

27 *Effects:* If `sz < distance(begin(), end())`, erases the last `distance(begin(), end()) - sz` elements from the list. Otherwise, inserts `sz - distance(begin(), end())` default-inserted elements at the end of the list.

28 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

```
void resize(size_type sz, const value_type& c);
```

29 *Effects:* If `sz < distance(begin(), end())`, erases the last `distance(begin(), end()) - sz` elements from the list. Otherwise, inserts `sz - distance(begin(), end())` elements at the end of the list such that each new element, `e`, is initialized by a method equivalent to calling `allocator_traits<allocator_type>::construct(get_allocator(), std::addressof(e), c)`.

30 *Requires:* `T` shall be `CopyInsertable` into `*this`.

```
void clear() noexcept;
```

31 *Effects:* Erases all elements in the range `[begin(),end())`.

32 *Remarks:* Does not invalidate past-the-end iterators.

23.3.4.6 `forward_list` operations [forwardlist.ops]

```
void splice_after(const_iterator position, forward_list& x);
void splice_after(const_iterator position, forward_list&& x);
```

1 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`. `get_allocator() == x.get_allocator()`. `&x != this`.

2 *Effects:* Inserts the contents of `x` after `position`, and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

3 *Throws:* Nothing.

4 *Complexity:* $\mathcal{O}(distance(x.begin(), x.end()))$

```
void splice_after(const_iterator position, forward_list& x, const_iterator i);
void splice_after(const_iterator position, forward_list&& x, const_iterator i);
```

5 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`. The iterator following `i` is a dereferenceable iterator in `x`. `get_allocator() == x.get_allocator()`.

6 *Effects:* Inserts the element following `i` into `*this`, following `position`, and removes it from `x`. The result is unchanged if `position == i` or `position == ++i`. Pointers and references to `*++i` continue to refer to the same element but as a member of `*this`. Iterators to `*++i` continue to refer to the same element, but now behave as iterators into `*this`, not into `x`.

7 *Throws:* Nothing.

8 *Complexity:* $\mathcal{O}(1)$

```
void splice_after(const_iterator position, forward_list& x,
                  const_iterator first, const_iterator last);
void splice_after(const_iterator position, forward_list&& x,
                  const_iterator first, const_iterator last);
```

9 *Requires:* `position` is `before_begin()` or is a dereferenceable iterator in the range `[begin(),end())`. `(first,last)` is a valid range in `x`, and all iterators in the range `(first,last)` are dereferenceable. `position` is not an iterator in the range `(first,last)`. `get_allocator() == x.get_allocator()`.

10 *Effects:* Inserts elements in the range `(first,last)` after `position` and removes the elements from `x`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

11 *Complexity:* $\mathcal{O}(distance(first, last))$

```
void remove(const T& value);
template <class Predicate> void remove_if(Predicate pred);
```

12 *Effects:* Erases all the elements in the list referred by a list iterator `i` for which the following conditions hold: `*i == value` (for `remove()`), `pred(*i)` is true (for `remove_if()`). Invalidates only the iterators and references to the erased elements.

13 *Throws:* Nothing unless an exception is thrown by the equality comparison or the predicate.

14 *Remarks:* Stable (17.6.5.7).

15 *Complexity:* Exactly `distance(begin(), end())` applications of the corresponding predicate.

```
void unique();
template <class BinaryPredicate> void unique(BinaryPredicate pred);
```

16 *Effects:* Erases all but the first element from every consecutive group of equal elements referred to by the iterator `i` in the range `[first + 1,last)` for which `*i == *(i-1)` (for the version with no arguments) or `pred(*i, *(i - 1))` (for the version with a predicate argument) holds. Invalidates only the iterators and references to the erased elements.

17 *Throws:* Nothing unless an exception is thrown by the equality comparison or the predicate.

18 *Complexity:* If the range `[first,last)` is not empty, exactly `(last - first) - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
void merge(forward_list& x);
void merge(forward_list&& x);
template <class Compare> void merge(forward_list& x, Compare comp);
template <class Compare> void merge(forward_list&& x, Compare comp);
```

19 *Requires:* `comp` defines a strict weak ordering (25.4), and `*this` and `x` are both sorted according to this ordering. `get_allocator() == x.get_allocator()`.

20 *Effects:* Merges the two sorted ranges `[begin(), end())` and `[x.begin(), x.end())`. `x` is empty after the merge. If an exception is thrown other than by a comparison there are no effects. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

21 *Remarks:* Stable (17.6.5.7). The behavior is undefined if `this->get_allocator() != x.get_allocator()`.

22 *Complexity:* At most `distance(begin(), end()) + distance(x.begin(), x.end()) - 1` comparisons.

```
void sort();
template <class Compare> void sort(Compare comp);
```

23 *Requires:* `operator<` (for the version with no arguments) or `comp` (for the version with a comparison argument) defines a strict weak ordering (25.4).

24 *Effects:* Sorts the list according to the `operator<` or the `comp` function object. If an exception is thrown the order of the elements in `*this` is unspecified. Does not affect the validity of iterators and references.

25 *Remarks:* Stable (17.6.5.7).

26 *Complexity:* Approximately $N \log N$ comparisons, where N is `distance(begin(), end())`.

```
void reverse() noexcept;
```

27 *Effects:* Reverses the order of the elements in the list. Does not affect the validity of iterators and references.

28 *Complexity:* Linear time.

23.3.4.7 `forward_list` specialized algorithms [forwardlist.spec]

```
template <class T, class Allocator>
  void swap(forward_list<T, Allocator>& x, forward_list<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`

23.3.5 Class template `list` [list]

23.3.5.1 Class template `list` overview [list.overview]

1 A `list` is a sequence container that supports bidirectional iterators and allows constant time insert and erase operations anywhere within the sequence, with storage management handled automatically. Unlike vectors (23.3.6) and deques (23.3.3), fast random access to list elements is not supported, but many algorithms only need sequential access anyway.

2 A `list` satisfies all of the requirements of a container, of a reversible container (given in two tables in 23.2), of a sequence container, including most of the optional sequence container requirements (23.2.3), and of an allocator-aware container (Table 98). The exceptions are the `operator[]` and `at` member functions, which are not provided.[265] Descriptions are provided here only for operations on `list` that are not described in one of these tables or for operations where there is additional semantic information.

```
namespace std {
  template <class T, class Allocator = allocator<T> >
  class list {
  public:
    // types:
    typedef value_type&                                          reference;
    typedef const value_type&                                    const_reference;
    typedef implementation-defined                iterator;      // see 23.2
    typedef implementation-defined                const_iterator; // see 23.2
    typedef implementation-defined                size_type;     // see 23.2
    typedef implementation-defined                difference_type;// see 23.2
    typedef T                                     value_type;
    typedef Allocator                             allocator_type;
    typedef typename allocator_traits<Allocator>::pointer        pointer;
    typedef typename allocator_traits<Allocator>::const_pointer  const_pointer;
```

265) These member functions are only provided by containers whose iterators are random access iterators.

```
  typedef std::reverse_iterator<iterator>       reverse_iterator;
  typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

  // 23.3.5.2, construct/copy/destroy:
  list() : list(Allocator()) { }
  explicit list(const Allocator&);
  explicit list(size_type n, const Allocator& = Allocator());
  list(size_type n, const T& value, const Allocator& = Allocator());
  template <class InputIterator>
    list(InputIterator first, InputIterator last, const Allocator& = Allocator());
  list(const list& x);
  list(list&& x);
  list(const list&, const Allocator&);
  list(list&&, const Allocator&);
  list(initializer_list<T>, const Allocator& = Allocator());
 ~list();
  list& operator=(const list& x);
  list& operator=(list&& x)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
  list& operator=(initializer_list<T>);
  template <class InputIterator>
    void assign(InputIterator first, InputIterator last);
  void assign(size_type n, const T& t);
  void assign(initializer_list<T>);
  allocator_type get_allocator() const noexcept;

  // iterators:
  iterator               begin() noexcept;
  const_iterator         begin() const noexcept;
  iterator               end() noexcept;
  const_iterator         end() const noexcept;
  reverse_iterator       rbegin() noexcept;
  const_reverse_iterator rbegin() const noexcept;
  reverse_iterator       rend() noexcept;
  const_reverse_iterator rend() const noexcept;

  const_iterator         cbegin() const noexcept;
  const_iterator         cend() const noexcept;
  const_reverse_iterator crbegin() const noexcept;
  const_reverse_iterator crend() const noexcept;

  // 23.3.5.3, capacity:
  bool      empty() const noexcept;
  size_type size() const noexcept;
  size_type max_size() const noexcept;
  void      resize(size_type sz);
  void      resize(size_type sz, const T& c);

  // element access:
  reference       front();
  const_reference front() const;
  reference       back();
  const_reference back() const;

  // 23.3.5.4, modifiers:
```

```
  template <class... Args> void emplace_front(Args&&... args);
  void pop_front();
  template <class... Args> void emplace_back(Args&&... args);
  void push_front(const T& x);
  void push_front(T&& x);
  void push_back(const T& x);
  void push_back(T&& x);
  void pop_back();

  template <class... Args> iterator emplace(const_iterator position, Args&&... args);
  iterator insert(const_iterator position, const T& x);
  iterator insert(const_iterator position, T&& x);
  iterator insert(const_iterator position, size_type n, const T& x);
  template <class InputIterator>
    iterator insert(const_iterator position, InputIterator first,
                    InputIterator last);
  iterator insert(const_iterator position, initializer_list<T> il);

  iterator erase(const_iterator position);
  iterator erase(const_iterator position, const_iterator last);
  void     swap(list&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value);
  void     clear() noexcept;

  // 23.3.5.5, list operations:
  void splice(const_iterator position, list& x);
  void splice(const_iterator position, list&& x);
  void splice(const_iterator position, list& x, const_iterator i);
  void splice(const_iterator position, list&& x, const_iterator i);
  void splice(const_iterator position, list& x,
              const_iterator first, const_iterator last);
  void splice(const_iterator position, list&& x,
              const_iterator first, const_iterator last);

  void remove(const T& value);
  template <class Predicate> void remove_if(Predicate pred);

  void unique();
  template <class BinaryPredicate>
    void unique(BinaryPredicate binary_pred);

  void merge(list& x);
  void merge(list&& x);
  template <class Compare> void merge(list& x, Compare comp);
  template <class Compare> void merge(list&& x, Compare comp);

  void sort();
  template <class Compare> void sort(Compare comp);

  void reverse() noexcept;
};

template <class T, class Allocator>
  bool operator==(const list<T, Allocator>& x, const list<T, Allocator>& y);
template <class T, class Allocator>
```

```
    bool operator< (const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const list<T, Allocator>& x, const list<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const list<T, Allocator>& x, const list<T, Allocator>& y);

  // specialized algorithms:
  template <class T, class Allocator>
    void swap(list<T, Allocator>& x, list<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

3 An incomplete type `T` may be used when instantiating `list` if the allocator satisfies the allocator completeness requirements 17.6.3.5.1. `T` shall be complete before any member of the resulting specialization of `list` is referenced.

23.3.5.2 `list` constructors, copy, and assignment [list.cons]

```
explicit list(const Allocator&);
```

1 *Effects:* Constructs an empty list, using the specified allocator.

2 *Complexity:* Constant.

```
explicit list(size_type n, const Allocator& = Allocator());
```

3 *Effects:* Constructs a `list` with `n` default-inserted elements using the specified allocator.

4 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

5 *Complexity:* Linear in `n`.

```
list(size_type n, const T& value, const Allocator& = Allocator());
```

6 *Effects:* Constructs a `list` with `n` copies of `value`, using the specified allocator.

7 *Requires:* `T` shall be `CopyInsertable` into `*this`.

8 *Complexity:* Linear in `n`.

```
template <class InputIterator>
  list(InputIterator first, InputIterator last, const Allocator& = Allocator());
```

9 *Effects:* Constructs a `list` equal to the range `[first,last)`.

10 *Complexity:* Linear in `distance(first, last)`.

23.3.5.3 `list` capacity [list.capacity]

```
void resize(size_type sz);
```

1 *Effects:* If `size() < sz`, appends `sz - size()` default-inserted elements to the sequence. If `sz <= size()`, equivalent to

```
list<T>::iterator it = begin();
advance(it, sz);
erase(it, end());
```

2 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

```
void resize(size_type sz, const T& c);
```

3 *Effects:*

```
if (sz > size())
  insert(end(), sz-size(), c);
else if (sz < size()) {
  iterator i = begin();
  advance(i, sz);
  erase(i, end());
}
else
  ;                  // do nothing
```

4 *Requires:* `T` shall be `CopyInsertable` into `*this`.

23.3.5.4 `list` modifiers [list.modifiers]

```
iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  iterator insert(const_iterator position, InputIterator first,
                  InputIterator last);
iterator insert(const_iterator position, initializer_list<T>);

template <class... Args> void emplace_front(Args&&... args);
template <class... Args> void emplace_back(Args&&... args);
template <class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_front(const T& x);
void push_front(T&& x);
void push_back(const T& x);
void push_back(T&& x);
```

1 *Remarks:* Does not affect the validity of iterators and references. If an exception is thrown there are no effects.

2 *Complexity:* Insertion of a single element into a list takes constant time and exactly one call to a constructor of `T`. Insertion of multiple elements into a list is linear in the number of elements inserted, and the number of calls to the copy constructor or move constructor of `T` is exactly equal to the number of elements inserted.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);

void pop_front();
void pop_back();
void clear() noexcept;
```

3 *Effects:* Invalidates only the iterators and references to the erased elements.

4 *Throws:* Nothing.

5 *Complexity:* Erasing a single element is a constant time operation with a single call to the destructor of `T`. Erasing a range in a list is linear time in the size of the range and the number of calls to the destructor of type `T` is exactly equal to the size of the range.

23.3.5.5 `list` operations [list.ops]

1 Since lists allow fast insertion and erasing from the middle of a list, certain operations are provided specifically for them.[266]

2 `list` provides three splice operations that destructively move elements from one list to another. The behavior of splice operations is undefined if `get_allocator() != x.get_allocator()`.

```
void splice(const_iterator position, list& x);
void splice(const_iterator position, list&& x);
```

3 *Requires:* `&x != this`.

4 *Effects:* Inserts the contents of `x` before `position` and `x` becomes empty. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

5 *Throws:* Nothing.

6 *Complexity:* Constant time.

```
void splice(const_iterator position, list& x, const_iterator i);
void splice(const_iterator position, list&& x, const_iterator i);
```

7 *Effects:* Inserts an element pointed to by `i` from list `x` before `position` and removes the element from `x`. The result is unchanged if `position == i` or `position == ++i`. Pointers and references to `*i` continue to refer to this same element but as a member of `*this`. Iterators to `*i` (including `i` itself) continue to refer to the same element, but now behave as iterators into `*this`, not into `x`.

8 *Requires:* `i` is a valid dereferenceable iterator of `x`.

9 *Throws:* Nothing.

10 *Complexity:* Constant time.

```
void splice(const_iterator position, list& x, const_iterator first,
            const_iterator last);
void splice(const_iterator position, list&& x, const_iterator first,
            const_iterator last);
```

11 *Effects:* Inserts elements in the range `[first,last)` before `position` and removes the elements from `x`.

12 *Requires:* `[first, last)` is a valid range in `x`. The result is undefined if `position` is an iterator in the range `[first,last)`. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

13 *Throws:* Nothing.

14 *Complexity:* Constant time if `&x == this`; otherwise, linear time.

```
void remove(const T& value);
template <class Predicate> void remove_if(Predicate pred);
```

15 *Effects:* Erases all the elements in the list referred by a list iterator `i` for which the following conditions hold: `*i == value, pred(*i) != false`. Invalidates only the iterators and references to the erased elements.

16 *Throws:* Nothing unless an exception is thrown by `*i == value` or `pred(*i) != false`.

266) As specified in 17.6.3.5, the requirements in this Clause apply only to lists whose allocators compare equal.

17 *Remarks:* Stable (17.6.5.7).

18 *Complexity:* Exactly `size()` applications of the corresponding predicate.

```
void unique();
template <class BinaryPredicate> void unique(BinaryPredicate binary_pred);
```

19 *Effects:* Erases all but the first element from every consecutive group of equal elements referred to by the iterator `i` in the range `[first + 1,last)` for which `*i == *(i-1)` (for the version of `unique` with no arguments) or `pred(*i, *(i - 1))` (for the version of `unique` with a predicate argument) holds. Invalidates only the iterators and references to the erased elements.

20 *Throws:* Nothing unless an exception is thrown by `*i == *(i-1)` or `pred(*i, *(i - 1))`

21 *Complexity:* If the range `[first, last)` is not empty, exactly `(last - first) - 1` applications of the corresponding predicate, otherwise no applications of the predicate.

```
void merge(list& x);
void merge(list&& x);
template <class Compare> void merge(list& x, Compare comp);
template <class Compare> void merge(list&& x, Compare comp);
```

22 *Requires:* `comp` shall define a strict weak ordering (25.4), and both the list and the argument list shall be sorted according to this ordering.

23 *Effects:* If `(&x == this)` does nothing; otherwise, merges the two sorted ranges `[begin(), end())` and `[x.begin(), x.end())`. The result is a range in which the elements will be sorted in non-decreasing order according to the ordering defined by `comp`; that is, for every iterator `i`, in the range other than the first, the condition `comp(*i, *(i - 1))` will be false. Pointers and references to the moved elements of `x` now refer to those same elements but as members of `*this`. Iterators referring to the moved elements will continue to refer to their elements, but they now behave as iterators into `*this`, not into `x`.

24 *Remarks:* Stable (17.6.5.7). If `(&x != this)` the range `[x.begin(), x.end())` is empty after the merge. No elements are copied by this operation. The behavior is undefined if `this->get_allocator() != x.get_allocator()`.

25 *Complexity:* At most `size() + x.size() - 1` applications of `comp` if `(&x != this)`; otherwise, no applications of `comp` are performed. If an exception is thrown other than by a comparison there are no effects.

```
void reverse() noexcept;
```

26 *Effects:* Reverses the order of the elements in the list. Does not affect the validity of iterators and references.

27 *Complexity:* Linear time.

```
void sort();
template <class Compare> void sort(Compare comp);
```

28 *Requires:* `operator<` (for the first version) or `comp` (for the second version) shall define a strict weak ordering (25.4).

29 *Effects:* Sorts the list according to the `operator<` or a `Compare` function object. Does not affect the validity of iterators and references.

30 *Remarks:* Stable (17.6.5.7).

31 *Complexity:* Approximately $N \log(N)$ comparisons, where `N == size()`.

23.3.5.6 `list` specialized algorithms [list.special]

```
template <class T, class Allocator>
  void swap(list<T, Allocator>& x, list<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.3.6 Class template `vector` [vector]

23.3.6.1 Class template `vector` overview [vector.overview]

1 A `vector` is a sequence container that supports (amortized) constant time insert and erase operations at the end; insert and erase in the middle take linear time. Storage management is handled automatically, though hints can be given to improve efficiency.

2 A `vector` satisfies all of the requirements of a container and of a reversible container (given in two tables in 23.2), of a sequence container, including most of the optional sequence container requirements (23.2.3), of an allocator-aware container (Table 98), and, for an element type other than `bool`, of a contiguous container (23.2.1). The exceptions are the `push_front`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `vector` that are not described in one of these tables or for operations where there is additional semantic information.

```
namespace std {
  template <class T, class Allocator = allocator<T> >
  class vector {
  public:
    // types:
    typedef value_type&                              reference;
    typedef const value_type&                        const_reference;
    typedef implementation-defined                   iterator;       // see 23.2
    typedef implementation-defined                   const_iterator; // see 23.2
    typedef implementation-defined                   size_type;      // see 23.2
    typedef implementation-defined                   difference_type;// see 23.2
    typedef T                                        value_type;
    typedef Allocator                                allocator_type;
    typedef typename allocator_traits<Allocator>::pointer          pointer;
    typedef typename allocator_traits<Allocator>::const_pointer    const_pointer;
    typedef std::reverse_iterator<iterator>          reverse_iterator;
    typedef std::reverse_iterator<const_iterator>    const_reverse_iterator;

    // 23.3.6.2, construct/copy/destroy:
    vector() noexcept(noexcept(Allocator())) : vector(Allocator()) { }
    explicit vector(const Allocator&) noexcept;
    explicit vector(size_type n, const Allocator& = Allocator());
    vector(size_type n, const T& value, const Allocator& = Allocator());
    template <class InputIterator>
      vector(InputIterator first, InputIterator last, const Allocator& = Allocator());
    vector(const vector& x);
    vector(vector&&) noexcept;
    vector(const vector&, const Allocator&);
    vector(vector&&, const Allocator&);
    vector(initializer_list<T>, const Allocator& = Allocator());
   ~vector();
    vector& operator=(const vector& x);
    vector& operator=(vector&& x)
```

```
  noexcept(allocator_traits<Allocator>::propagate_on_container_move_assignment::value ||
           allocator_traits<Allocator>::is_always_equal::value);
vector& operator=(initializer_list<T>);
template <class InputIterator>
  void assign(InputIterator first, InputIterator last);
void assign(size_type n, const T& u);
void assign(initializer_list<T>);
allocator_type get_allocator() const noexcept;

// iterators:
iterator               begin() noexcept;
const_iterator         begin() const noexcept;
iterator               end() noexcept;
const_iterator         end() const noexcept;
reverse_iterator       rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator       rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator         cbegin() const noexcept;
const_iterator         cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// 23.3.6.3, capacity:
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
size_type capacity() const noexcept;
void      resize(size_type sz);
void      resize(size_type sz, const T& c);
void      reserve(size_type n);
void      shrink_to_fit();

// element access:
reference       operator[](size_type n);
const_reference operator[](size_type n) const;
const_reference at(size_type n) const;
reference       at(size_type n);
reference       front();
const_reference front() const;
reference       back();
const_reference back() const;

// 23.3.6.4, data access
T*       data() noexcept;
const T* data() const noexcept;

// 23.3.6.5, modifiers:
template <class... Args> void emplace_back(Args&&... args);
void push_back(const T& x);
void push_back(T&& x);
void pop_back();

template <class... Args> iterator emplace(const_iterator position, Args&&... args);
```

```
    iterator insert(const_iterator position, const T& x);
    iterator insert(const_iterator position, T&& x);
    iterator insert(const_iterator position, size_type n, const T& x);
    template <class InputIterator>
      iterator insert(const_iterator position, InputIterator first, InputIterator last);
    iterator insert(const_iterator position, initializer_list<T> il);
    iterator erase(const_iterator position);
    iterator erase(const_iterator first, const_iterator last);
    void     swap(vector&)
      noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
               allocator_traits<Allocator>::is_always_equal::value);
    void     clear() noexcept;
  };

  template <class T, class Allocator>
    bool operator==(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator< (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator!=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator> (const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator>=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);
  template <class T, class Allocator>
    bool operator<=(const vector<T, Allocator>& x, const vector<T, Allocator>& y);

  // 23.3.6.6, specialized algorithms:
  template <class T, class Allocator>
    void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

3 An incomplete type `T` may be used when instantiating `vector` if the allocator satisfies the allocator completeness requirements 17.6.3.5.1. `T` shall be complete before any member of the resulting specialization of `vector` is referenced.

23.3.6.2 `vector` constructors, copy, and assignment [vector.cons]

```
explicit vector(const Allocator&);
```

1 *Effects:* Constructs an empty `vector`, using the specified allocator.

2 *Complexity:* Constant.

```
explicit vector(size_type n, const Allocator& = Allocator());
```

3 *Effects:* Constructs a `vector` with `n` default-inserted elements using the specified allocator.

4 *Requires:* `T` shall be `DefaultInsertable` into `*this`.

5 *Complexity:* Linear in `n`.

```
vector(size_type n, const T& value,
       const Allocator& = Allocator());
```

6 *Effects:* Constructs a `vector` with `n` copies of `value`, using the specified allocator.

7 *Requires:* `T` shall be `CopyInsertable` into `*this`.

8 *Complexity:* Linear in `n`.

```
template <class InputIterator>
  vector(InputIterator first, InputIterator last,
         const Allocator& = Allocator());
```

9 *Effects:* Constructs a `vector` equal to the range `[first,last)`, using the specified allocator.

10 *Complexity:* Makes only N calls to the copy constructor of `T` (where N is the distance between `first` and `last`) and no reallocations if iterators first and last are of forward, bidirectional, or random access categories. It makes order `N` calls to the copy constructor of `T` and order $\log(N)$ reallocations if they are just input iterators.

23.3.6.3 `vector` capacity [vector.capacity]

```
size_type capacity() const noexcept;
```

1 *Returns:* The total number of elements that the vector can hold without requiring reallocation.

```
void reserve(size_type n);
```

2 *Requires:* `T` shall be `MoveInsertable` into `*this`.

3 *Effects:* A directive that informs a `vector` of a planned change in size, so that it can manage the storage allocation accordingly. After `reserve()`, `capacity()` is greater or equal to the argument of `reserve` if reallocation happens; and equal to the previous value of `capacity()` otherwise. Reallocation happens at this point if and only if the current capacity is less than the argument of `reserve()`. If an exception is thrown other than by the move constructor of a non-`CopyInsertable` type, there are no effects.

4 *Complexity:* It does not change the size of the sequence and takes at most linear time in the size of the sequence.

5 *Throws:* `length_error` if `n > max_size()`.[267]

6 *Remarks:* Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence. No reallocation shall take place during insertions that happen after a call to `reserve()` until the time when an insertion would make the size of the vector greater than the value of `capacity()`.

```
void shrink_to_fit();
```

7 *Requires:* `T` shall be `MoveInsertable` into `*this`.

8 *Complexity:* Linear in the size of the sequence.

9 *Remarks:* `shrink_to_fit` is a non-binding request to reduce `capacity()` to `size()`. [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations. — *end note*] If an exception is thrown other than by the move constructor of a non-`CopyInsertable` `T` there are no effects.

```
void swap(vector& x)
  noexcept(allocator_traits<Allocator>::propagate_on_container_swap::value ||
           allocator_traits<Allocator>::is_always_equal::value);
```

10 *Effects:* Exchanges the contents and `capacity()` of `*this` with that of `x`.

11 *Complexity:* Constant time.

```
void resize(size_type sz);
```

267) `reserve()` uses `Allocator::allocate()` which may throw an appropriate exception.

12 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.

13 *Requires:* `T` shall be `MoveInsertable` and `DefaultInsertable` into `*this`.

14 *Remarks:* If an exception is thrown other than by the move constructor of a non-`CopyInsertable` `T` there are no effects.

```
void resize(size_type sz, const T& c);
```

15 *Effects:* If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.

16 *Requires:* `T` shall be `CopyInsertable` into `*this`.

17 *Remarks:* If an exception is thrown there are no effects.

23.3.6.4 `vector` data [vector.data]

```
T*         data() noexcept;
const T*   data() const noexcept;
```

1 *Returns:* A pointer such that `[data(),data() + size())` is a valid range. For a non-empty vector, `data() == &front()`.

2 *Complexity:* Constant time.

23.3.6.5 `vector` modifiers [vector.modifiers]

```
iterator insert(const_iterator position, const T& x);
iterator insert(const_iterator position, T&& x);
iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  iterator insert(const_iterator position, InputIterator first, InputIterator last);
iterator insert(const_iterator position, initializer_list<T>);

template <class... Args> void emplace_back(Args&&... args);
template <class... Args> iterator emplace(const_iterator position, Args&&... args);
void push_back(const T& x);
void push_back(T&& x);
```

1 *Remarks:* Causes reallocation if the new size is greater than the old capacity. If no reallocation happens, all the iterators and references before the insertion point remain valid. If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `T` or by any `InputIterator` operation there are no effects. If an exception is thrown while inserting a single element at the end and `T` is `CopyInsertable` or `is_nothrow_move_constructible<T>::value` is `true`, there are no effects. Otherwise, if an exception is thrown by the move constructor of a non-`CopyInsertable` `T`, the effects are unspecified.

2 *Complexity:* The complexity is linear in the number of elements inserted plus the distance to the end of the vector.

```
iterator erase(const_iterator position);
iterator erase(const_iterator first, const_iterator last);
void pop_back();
```

3 *Effects:* Invalidates iterators and references at or after the point of the erase.

4 *Complexity:* The destructor of `T` is called the number of times equal to the number of the elements erased, but the move assignment operator of `T` is called the number of times equal to the number of elements in the vector after the erased elements.

5 *Throws:* Nothing unless an exception is thrown by the copy constructor, move constructor, assignment operator, or move assignment operator of `T`.

23.3.6.6 `vector` specialized algorithms [vector.special]

```
template <class T, class Allocator>
  void swap(vector<T, Allocator>& x, vector<T, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.3.7 Class `vector<bool>` [vector.bool]

1 To optimize space allocation, a specialization of vector for `bool` elements is provided:

```
namespace std {
  template <class Allocator> class vector<bool, Allocator> {
  public:
    // types:
    typedef bool                                 const_reference;
    typedef implementation-defined               iterator;       // see 23.2
    typedef implementation-defined               const_iterator; // see 23.2
    typedef implementation-defined               size_type;      // see 23.2
    typedef implementation-defined               difference_type;// see 23.2
    typedef bool                                 value_type;
    typedef Allocator                            allocator_type;
    typedef implementation-defined               pointer;
    typedef implementation-defined               const_pointer;
    typedef std::reverse_iterator<iterator>      reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    // bit reference:
    class reference {
      friend class vector;
      reference() noexcept;
    public:
      ~reference();
      operator bool() const noexcept;
      reference& operator=(const bool x) noexcept;
      reference& operator=(const reference& x) noexcept;
      void flip() noexcept;     // flips the bit
    };

    // construct/copy/destroy:
    vector() : vector(Allocator()) { }
    explicit vector(const Allocator&);
    explicit vector(size_type n, const Allocator& = Allocator());
    vector(size_type n, const bool& value,
           const Allocator& = Allocator());
    template <class InputIterator>
      vector(InputIterator first, InputIterator last,
             const Allocator& = Allocator());
    vector(const vector<bool, Allocator>& x);
    vector(vector<bool, Allocator>&& x);
    vector(const vector&, const Allocator&);
```

```
 vector(vector&&, const Allocator&);
 vector(initializer_list<bool>, const Allocator& = Allocator()));
~vector();
 vector<bool, Allocator>& operator=(const vector<bool, Allocator>& x);
 vector<bool, Allocator>& operator=(vector<bool, Allocator>&& x);
 vector& operator=(initializer_list<bool>);
 template <class InputIterator>
   void assign(InputIterator first, InputIterator last);
 void assign(size_type n, const bool& t);
 void assign(initializer_list<bool>);
 allocator_type get_allocator() const noexcept;

 // iterators:
 iterator               begin() noexcept;
 const_iterator         begin() const noexcept;
 iterator               end() noexcept;
 const_iterator         end() const noexcept;
 reverse_iterator       rbegin() noexcept;
 const_reverse_iterator rbegin() const noexcept;
 reverse_iterator       rend() noexcept;
 const_reverse_iterator rend() const noexcept;

 const_iterator         cbegin() const noexcept;
 const_iterator         cend() const noexcept;
 const_reverse_iterator crbegin() const noexcept;
 const_reverse_iterator crend() const noexcept;

 // capacity:
 bool      empty() const noexcept;
 size_type size() const noexcept;
 size_type max_size() const noexcept;
 size_type capacity() const noexcept;
 void      resize(size_type sz, bool c = false);
 void      reserve(size_type n);
 void      shrink_to_fit();

 // element access:
 reference       operator[](size_type n);
 const_reference operator[](size_type n) const;
 const_reference at(size_type n) const;
 reference       at(size_type n);
 reference       front();
 const_reference front() const;
 reference       back();
 const_reference back() const;

 // modifiers:
 template <class... Args> void emplace_back(Args&&... args);
 void push_back(const bool& x);
 void pop_back();
 template <class... Args> iterator emplace(const_iterator position, Args&&... args);
 iterator insert(const_iterator position, const bool& x);
 iterator insert (const_iterator position, size_type n, const bool& x);
 template <class InputIterator>
   iterator insert(const_iterator position,
```

```
                    InputIterator first, InputIterator last);
    iterator insert(const_iterator position, initializer_list<bool> il);

    iterator erase(const_iterator position);
    iterator erase(const_iterator first, const_iterator last);
    void swap(vector<bool, Allocator>&);
    static void swap(reference x, reference y) noexcept;
    void flip() noexcept;       // flips all bits
    void clear() noexcept;
  };
}
```

2 Unless described below, all operations have the same requirements and semantics as the primary `vector` template, except that operations dealing with the `bool` value type map to bit values in the container storage and `allocator_traits::construct` (20.7.8.2) is not used to construct these values.

3 There is no requirement that the data be stored as a contiguous allocation of `bool` values. A space-optimized representation of bits is recommended instead.

4 `reference` is a class that simulates the behavior of references of a single bit in `vector<bool>`. The conversion operator returns `true` when the bit is set, and `false` otherwise. The assignment operator sets the bit when the argument is (convertible to) `true` and clears it otherwise. `flip` reverses the state of the bit.

```
void flip() noexcept;
```

5 *Effects:* Replaces each element in the container with its complement.

```
static void swap(reference x, reference y) noexcept;
```

6 *Effects:* exchanges the contents of `x` and `y` as if by

```
bool b = x;
x = y;
y = b;
```

```
template <class Allocator> struct hash<vector<bool, Allocator> >;
```

7 The template specialization shall meet the requirements of class template `hash` (20.9.13).

23.4 Associative containers [associative]

23.4.1 In general [associative.general]

1 The header `<map>` defines the class templates `map` and `multimap`; the header `<set>` defines the class templates `set` and `multiset`.

23.4.2 Header <map> synopsis [associative.map.syn]

```
#include <initializer_list>

namespace std {

  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T> > >
    class map;
  template <class Key, class T, class Compare, class Allocator>
    bool operator==(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
```

```
    bool operator< (const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator!=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator> (const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator>=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator<=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    void swap(map<Key, T, Compare, Allocator>& x,
              map<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T> > >
    class multimap;
  template <class Key, class T, class Compare, class Allocator>
    bool operator==(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator< (const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator!=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator> (const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator>=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator<=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    void swap(multimap<Key, T, Compare, Allocator>& x,
              multimap<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.4.3 Header <set> synopsis [associative.set.syn]

```
#include <initializer_list>

namespace std {

  template <class Key, class Compare = less<Key>,
            class Allocator = allocator<Key> >
    class set;
  template <class Key, class Compare, class Allocator>
```

```
  bool operator==(const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator< (const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator!=(const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator> (const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator>=(const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator<=(const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  void swap(set<Key, Compare, Allocator>& x,
            set<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));

template <class Key, class Compare = less<Key>,
          class Allocator = allocator<Key> >
  class multiset;
template <class Key, class Compare, class Allocator>
  bool operator==(const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator< (const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator!=(const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator> (const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator>=(const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  bool operator<=(const multiset<Key, Compare, Allocator>& x,
                  const multiset<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
  void swap(multiset<Key, Compare, Allocator>& x,
            multiset<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
}
```

23.4.4 Class template `map` [map]

23.4.4.1 Class template `map` overview [map.overview]

1 A `map` is an associative container that supports unique keys (contains at most one of each key value) and provides for fast retrieval of values of another type `T` based on the keys. The `map` class supports bidirectional iterators.

2 A `map` satisfies all of the requirements of a container, of a reversible container (23.2), of an associative container (23.2.4), and of an allocator-aware container (Table 98). A `map` also provides most operations described in (23.2.4) for unique keys. This means that a `map` supports the `a_uniq` operations in (23.2.4) but not the `a_eq` operations. For a `map<Key,T>` the `key_type` is `Key` and the `value_type` is `pair<const Key,T>`. Descriptions are provided here only for operations on `map` that are not described in one of those tables or for operations where there is additional semantic information.

```
namespace std {
  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T> > >
  class map {
  public:
    // types:
    typedef Key                                   key_type;
    typedef T                                     mapped_type;
    typedef pair<const Key, T>                    value_type;
    typedef Compare                               key_compare;
    typedef Allocator                             allocator_type;
    typedef value_type&                           reference;
    typedef const value_type&                     const_reference;
    typedef implementation-defined                iterator;       // see 23.2
    typedef implementation-defined                const_iterator; // see 23.2
    typedef implementation-defined                size_type;      // see 23.2
    typedef implementation-defined                difference_type;// see 23.2
    typedef typename allocator_traits<Allocator>::pointer           pointer;
    typedef typename allocator_traits<Allocator>::const_pointer     const_pointer;
    typedef std::reverse_iterator<iterator>       reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    class value_compare {
    friend class map;
    protected:
      Compare comp;
      value_compare(Compare c) : comp(c) {}
    public:
      typedef bool result_type;
      typedef value_type first_argument_type;
      typedef value_type second_argument_type;
      bool operator()(const value_type& x, const value_type& y) const {
        return comp(x.first, y.first);
      }
    };

    // 23.4.4.2, construct/copy/destroy:
    map() : map(Compare()) { }
    explicit map(const Compare& comp, const Allocator& = Allocator());
    template <class InputIterator>
      map(InputIterator first, InputIterator last,
          const Compare& comp = Compare(), const Allocator& = Allocator());
    map(const map& x);
    map(map&& x);
    explicit map(const Allocator&);
    map(const map&, const Allocator&);
    map(map&&, const Allocator&);
    map(initializer_list<value_type>,
```

```
    const Compare& = Compare(),
    const Allocator& = Allocator());
  template <class InputIterator>
    map(InputIterator first, InputIterator last, const Allocator& a)
      : map(first, last, Compare(), a) { }
  map(initializer_list<value_type> il, const Allocator& a)
    : map(il, Compare(), a) { }
 ~map();
  map& operator=(const map& x);
  map& operator=(map&& x)
    noexcept(allocator_traits<Allocator>::is_always_equal::value &&
             is_nothrow_move_assignable<Compare>::value);
  map& operator=(initializer_list<value_type>);
  allocator_type get_allocator() const noexcept;

  // iterators:
  iterator               begin() noexcept;
  const_iterator         begin() const noexcept;
  iterator               end() noexcept;
  const_iterator         end() const noexcept;

  reverse_iterator       rbegin() noexcept;
  const_reverse_iterator rbegin() const noexcept;
  reverse_iterator       rend() noexcept;
  const_reverse_iterator rend() const noexcept;

  const_iterator         cbegin() const noexcept;
  const_iterator         cend() const noexcept;
  const_reverse_iterator crbegin() const noexcept;
  const_reverse_iterator crend() const noexcept;

  // capacity:
  bool      empty() const noexcept;
  size_type size() const noexcept;
  size_type max_size() const noexcept;

  // 23.4.4.3, element access:
  T& operator[](const key_type& x);
  T& operator[](key_type&& x);
  T&       at(const key_type& x);
  const T& at(const key_type& x) const;

  // 23.4.4.4, modifiers:
  template <class... Args> pair<iterator, bool> emplace(Args&&... args);
  template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
  pair<iterator, bool> insert(const value_type& x);
  pair<iterator, bool> insert(value_type&& x);
  template <class P> pair<iterator, bool> insert(P&& x);
  iterator insert(const_iterator position, const value_type& x);
  iterator insert(const_iterator position, value_type&& x);
  template <class P>
    iterator insert(const_iterator position, P&&);
  template <class InputIterator>
    void insert(InputIterator first, InputIterator last);
  void insert(initializer_list<value_type>);
```

```
template <class... Args>
  pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template <class... Args>
  pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template <class... Args>
  iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template <class... Args>
  iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template <class M>
  pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template <class M>
  pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template <class M>
  iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template <class M>
  iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& x);
iterator  erase(const_iterator first, const_iterator last);
void      swap(map&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Compare&>(), declval<Compare&>())));
void      clear() noexcept;

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// map operations:
iterator       find(const key_type& x);
const_iterator find(const key_type& x) const;
template <class K> iterator       find(const K& x);
template <class K> const_iterator find(const K& x) const;

size_type      count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

iterator       lower_bound(const key_type& x);
const_iterator lower_bound(const key_type& x) const;
template <class K> iterator       lower_bound(const K& x);
template <class K> const_iterator lower_bound(const K& x) const;

iterator       upper_bound(const key_type& x);
const_iterator upper_bound(const key_type& x) const;
template <class K> iterator       upper_bound(const K& x);
template <class K> const_iterator upper_bound(const K& x) const;

pair<iterator, iterator>               equal_range(const key_type& x);
pair<const_iterator, const_iterator>   equal_range(const key_type& x) const;
template <class K>
  pair<iterator, iterator>             equal_range(const K& x);
template <class K>
```

```
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
  };

  template <class Key, class T, class Compare, class Allocator>
    bool operator==(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator< (const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator!=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator> (const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator>=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator<=(const map<Key, T, Compare, Allocator>& x,
                    const map<Key, T, Compare, Allocator>& y);

  // specialized algorithms:
  template <class Key, class T, class Compare, class Allocator>
    void swap(map<Key, T, Compare, Allocator>& x,
              map<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.4.4.2 `map` constructors, copy, and assignment [map.cons]

```
explicit map(const Compare& comp, const Allocator& = Allocator());
```

1 *Effects:* Constructs an empty `map` using the specified comparison object and allocator.

2 *Complexity:* Constant.

```
template <class InputIterator>
  map(InputIterator first, InputIterator last,
      const Compare& comp = Compare(), const Allocator& = Allocator());
```

3 *Effects:* Constructs an empty `map` using the specified comparison object and allocator, and inserts elements from the range `[first,last)`.

4 *Complexity:* Linear in N if the range `[first,last)` is already sorted using `comp` and otherwise $N \log N$, where N is `last - first`.

23.4.4.3 `map` element access [map.access]

```
T& operator[](const key_type& x);
```

1 *Effects:* If there is no key equivalent to `x` in the map, inserts `value_type(x, T())` into the map.

2 *Requires:* `key_type` shall be `CopyInsertable` and `mapped_type` shall be `DefaultInsertable` into `*this`.

3 *Returns:* A reference to the `mapped_type` corresponding to `x` in `*this`.

4 *Complexity:* Logarithmic.

```
T& operator[](key_type&& x);
```

Effects: If there is no key equivalent to `x` in the map, inserts `value_type(std::move(x), T())` into the map.

Requires: `mapped_type` shall be `DefaultInsertable` into `*this`.

Returns: A reference to the `mapped_type` corresponding to `x` in `*this`.

Complexity: Logarithmic.

```
T&       at(const key_type& x);
const T& at(const key_type& x) const;
```

Returns: A reference to the `mapped_type` corresponding to `x` in `*this`.

Throws: An exception object of type `out_of_range` if no such element is present.

Complexity: Logarithmic.

23.4.4.4 `map` modifiers [map.modifiers]

```
template <class P> pair<iterator, bool> insert(P&& x);
template <class P> iterator insert(const_iterator position, P&& x);
template <class InputIterator>
  void insert(InputIterator first, InputIterator last);
```

Effects: The first form is equivalent to `return emplace(std::forward<P>(x))`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x))`.

Remarks: These signatures shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

```
template <class... Args> pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template <class... Args> iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
```

Requires: `value_type` shall be `EmplaceConstructible` into `map` from `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(forward<Args>(args)...)`.

Effects: If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(forward<Args>(args)...)`.

Returns: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

Complexity: The same as `emplace` and `emplace_hint`, respectively.

```
template <class... Args> pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template <class... Args> iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

Requires: `value_type` shall be `EmplaceConstructible` into `map` from `piecewise_construct`, `forward_as_tuple(move(k))`, `forward_as_tuple(forward<Args>(args)...)`.

Effects: If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct`, `forward_as_tuple(move(k))`, `forward_as_tuple(forward<Args>(args)...)`.

Returns: In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

Complexity: The same as `emplace` and `emplace_hint`, respectively.

```
template <class M> pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template <class M> iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
```

11 *Requires:* `is_assignable<mapped_type&, M&&>::value` shall be `true`. `value_type` shall be `EmplaceConstructible` into `map` from `k`, `forward<M>(obj)`.

12 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `k`, `forward<M>(obj)`.

13 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

14 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template <class M> pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template <class M> iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

15 *Requires:* `is_assignable<mapped_type&, M&&>::value` shall be `true`. `value_type` shall be `EmplaceConstructible` into `map` from `move(k)`, `forward<M>(obj)`.

16 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `move(k)`, `forward<M>(obj)`.

17 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

18 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

23.4.4.5 `map` specialized algorithms [map.special]

```
template <class Key, class T, class Compare, class Allocator>
  void swap(map<Key, T, Compare, Allocator>& x,
            map<Key, T, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.4.5 Class template `multimap` [multimap]

23.4.5.1 Class template `multimap` overview [multimap.overview]

1 A `multimap` is an associative container that supports equivalent keys (possibly containing multiple copies of the same key value) and provides for fast retrieval of values of another type `T` based on the keys. The `multimap` class supports bidirectional iterators.

2 A `multimap` satisfies all of the requirements of a container and of a reversible container (23.2), of an associative container (23.2.4), and of an allocator-aware container (Table 98). A `multimap` also provides most operations described in (23.2.4) for equal keys. This means that a `multimap` supports the `a_eq` operations in (23.2.4) but not the `a_uniq` operations. For a `multimap<Key,T>` the `key_type` is `Key` and the `value_type` is `pair<const Key,T>`. Descriptions are provided here only for operations on `multimap` that are not described in one of those tables or for operations where there is additional semantic information.

```
namespace std {
  template <class Key, class T, class Compare = less<Key>,
            class Allocator = allocator<pair<const Key, T> > >
  class multimap {
  public:
    // types:
    typedef Key                                   key_type;
```

```
typedef T                                       mapped_type;
typedef pair<const Key,T>                       value_type;
typedef Compare                                 key_compare;
typedef Allocator                               allocator_type;
typedef value_type&                             reference;
typedef const value_type&                       const_reference;
typedef implementation-defined                  iterator;       // see 23.2
typedef implementation-defined                  const_iterator; // see 23.2
typedef implementation-defined                  size_type;      // see 23.2
typedef implementation-defined                  difference_type;// see 23.2
typedef typename allocator_traits<Allocator>::pointer           pointer;
typedef typename allocator_traits<Allocator>::const_pointer     const_pointer;
typedef std::reverse_iterator<iterator>       reverse_iterator;
typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

class value_compare {
friend class multimap;
protected:
  Compare comp;
  value_compare(Compare c) : comp(c) { }
public:
  typedef bool result_type;
  typedef value_type first_argument_type;
  typedef value_type second_argument_type;
  bool operator()(const value_type& x, const value_type& y) const {
    return comp(x.first, y.first);
  }
};

// construct/copy/destroy:
multimap() : multimap(Compare()) { }
explicit multimap(const Compare& comp, const Allocator& = Allocator());
template <class InputIterator>
  multimap(InputIterator first, InputIterator last,
           const Compare& comp = Compare(),
           const Allocator& = Allocator());
multimap(const multimap& x);
multimap(multimap&& x);
explicit multimap(const Allocator&);
multimap(const multimap&, const Allocator&);
multimap(multimap&&, const Allocator&);
multimap(initializer_list<value_type>,
  const Compare& = Compare(),
  const Allocator& = Allocator());
template <class InputIterator>
  multimap(InputIterator first, InputIterator last, const Allocator& a)
    : multimap(first, last, Compare(), a) { }
multimap(initializer_list<value_type> il, const Allocator& a)
  : multimap(il, Compare(), a) { }
~multimap();
multimap& operator=(const multimap& x);
multimap& operator=(multimap&& x)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Compare>::value);
multimap& operator=(initializer_list<value_type>);
```

```
allocator_type get_allocator() const noexcept;

// iterators:
iterator               begin() noexcept;
const_iterator         begin() const noexcept;
iterator               end() noexcept;
const_iterator         end() const noexcept;

reverse_iterator       rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator       rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator         cbegin() const noexcept;
const_iterator         cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// capacity:
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
template <class... Args> iterator emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
iterator insert(const value_type& x);
iterator insert(value_type&& x);
template <class P> iterator insert(P&& x);
iterator insert(const_iterator position, const value_type& x);
iterator insert(const_iterator position, value_type&& x);
template <class P> iterator insert(const_iterator position, P&& x);
template <class InputIterator>
  void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& x);
iterator  erase(const_iterator first, const_iterator last);
void      swap(multimap&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Compare&>(), declval<Compare&>())));
void      clear() noexcept;

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// map operations:
iterator       find(const key_type& x);
const_iterator find(const key_type& x) const;
template <class K> iterator       find(const K& x);
template <class K> const_iterator find(const K& x) const;
```

```
    size_type      count(const key_type& x) const;
    template <class K> size_type count(const K& x) const;

    iterator       lower_bound(const key_type& x);
    const_iterator lower_bound(const key_type& x) const;
    template <class K> iterator       lower_bound(const K& x);
    template <class K> const_iterator lower_bound(const K& x) const;

    iterator       upper_bound(const key_type& x);
    const_iterator upper_bound(const key_type& x) const;
    template <class K> iterator       upper_bound(const K& x);
    template <class K> const_iterator upper_bound(const K& x) const;

    pair<iterator, iterator>               equal_range(const key_type& x);
    pair<const_iterator, const_iterator>   equal_range(const key_type& x) const;
    template <class K>
      pair<iterator, iterator>             equal_range(const K& x);
    template <class K>
      pair<const_iterator, const_iterator> equal_range(const K& x) const;
  };

  template <class Key, class T, class Compare, class Allocator>
    bool operator==(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator< (const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator!=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator> (const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator>=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);
  template <class Key, class T, class Compare, class Allocator>
    bool operator<=(const multimap<Key, T, Compare, Allocator>& x,
                    const multimap<Key, T, Compare, Allocator>& y);

  // specialized algorithms:
  template <class Key, class T, class Compare, class Allocator>
    void swap(multimap<Key, T, Compare, Allocator>& x,
              multimap<Key, T, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.4.5.2 `multimap` constructors [multimap.cons]

```
explicit multimap(const Compare& comp, const Allocator& = Allocator());
```

1 *Effects:* Constructs an empty `multimap` using the specified comparison object and allocator.

2 *Complexity:* Constant.

```
template <class InputIterator>
```

```
multimap(InputIterator first, InputIterator last,
         const Compare& comp = Compare(),
         const Allocator& = Allocator());
```

3 *Effects:* Constructs an empty `multimap` using the specified comparison object and allocator, and inserts elements from the range `[first,last)`.

4 *Complexity:* Linear in N if the range `[first,last)` is already sorted using `comp` and otherwise $N \log N$, where N is `last - first`.

23.4.5.3 `multimap` modifiers [multimap.modifiers]

```
template <class P> iterator insert(P&& x);
template <class P> iterator insert(const_iterator position, P&& x);
```

1 *Effects:* The first form is equivalent to `return emplace(std::forward<P>(x))`. The second form is equivalent to `return emplace_hint(position, std::forward<P>(x))`.

2 *Remarks:* These signatures shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

23.4.5.4 `multimap` specialized algorithms [multimap.special]

```
template <class Key, class T, class Compare, class Allocator>
  void swap(multimap<Key, T, Compare, Allocator>& x,
            multimap<Key, T, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.4.6 Class template `set` [set]

23.4.6.1 Class template `set` overview [set.overview]

1 A `set` is an associative container that supports unique keys (contains at most one of each key value) and provides for fast retrieval of the keys themselves. The `set` class supports bidirectional iterators.

2 A `set` satisfies all of the requirements of a container, of a reversible container (23.2), of an associative container (23.2.4), and of an allocator-aware container (Table 98). A `set` also provides most operations described in (23.2.4) for unique keys. This means that a `set` supports the `a_uniq` operations in (23.2.4) but not the `a_eq` operations. For a `set<Key>` both the `key_type` and `value_type` are `Key`. Descriptions are provided here only for operations on `set` that are not described in one of these tables and for operations where there is additional semantic information.

```
namespace std {
  template <class Key, class Compare = less<Key>,
            class Allocator = allocator<Key> >
  class set {
  public:
    // types:
    typedef Key                                  key_type;
    typedef Key                                  value_type;
    typedef Compare                              key_compare;
    typedef Compare                              value_compare;
    typedef Allocator                            allocator_type;
    typedef value_type&                          reference;
    typedef const value_type&                    const_reference;
    typedef implementation-defined               iterator;       // See 23.2
```

```
typedef implementation-defined                const_iterator; // See 23.2
typedef implementation-defined                size_type;      // See 23.2
typedef implementation-defined                difference_type;// See 23.2
typedef typename allocator_traits<Allocator>::pointer           pointer;
typedef typename allocator_traits<Allocator>::const_pointer     const_pointer;
typedef std::reverse_iterator<iterator>       reverse_iterator;
typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

// 23.4.6.2, construct/copy/destroy:
set() : set(Compare()) { }
explicit set(const Compare& comp, const Allocator& = Allocator());
template <class InputIterator>
  set(InputIterator first, InputIterator last,
      const Compare& comp = Compare(), const Allocator& = Allocator());
set(const set& x);
set(set&& x);
explicit set(const Allocator&);
set(const set&, const Allocator&);
set(set&&, const Allocator&);
set(initializer_list<value_type>, const Compare& = Compare(),
    const Allocator& = Allocator());
template <class InputIterator>
  set(InputIterator first, InputIterator last, const Allocator& a)
    : set(first, last, Compare(), a) { }
set(initializer_list<value_type> il, const Allocator& a)
  : set(il, Compare(), a) { }
~set();
set& operator=(const set& x);
set& operator=(set&& x)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Compare>::value);
set& operator=(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// iterators:
iterator               begin() noexcept;
const_iterator         begin() const noexcept;
iterator               end() noexcept;
const_iterator         end() const noexcept;

reverse_iterator       rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator       rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator         cbegin() const noexcept;
const_iterator         cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// capacity:
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;
```

```
  // modifiers:
  template <class... Args> pair<iterator, bool> emplace(Args&&... args);
  template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
  pair<iterator,bool> insert(const value_type& x);
  pair<iterator,bool> insert(value_type&& x);
  iterator insert(const_iterator position, const value_type& x);
  iterator insert(const_iterator position, value_type&& x);
  template <class InputIterator>
    void insert(InputIterator first, InputIterator last);
  void insert(initializer_list<value_type>);

  iterator  erase(iterator position);
  iterator  erase(const_iterator position);
  size_type erase(const key_type& x);
  iterator  erase(const_iterator first, const_iterator last);
  void      swap(set&)
    noexcept(allocator_traits<Allocator>::is_always_equal::value &&
             noexcept(swap(declval<Compare&>(), declval<Compare&>())));
  void      clear() noexcept;

  // observers:
  key_compare key_comp() const;
  value_compare value_comp() const;

  // set operations:
  iterator       find(const key_type& x);
  const_iterator find(const key_type& x) const;
  template <class K> iterator       find(const K& x);
  template <class K> const_iterator find(const K& x) const;

  size_type      count(const key_type& x) const;
  template <class K> size_type count(const K& x) const;

  iterator       lower_bound(const key_type& x);
  const_iterator lower_bound(const key_type& x) const;
  template <class K> iterator       lower_bound(const K& x);
  template <class K> const_iterator lower_bound(const K& x) const;

  iterator       upper_bound(const key_type& x);
  const_iterator upper_bound(const key_type& x) const;
  template <class K> iterator       upper_bound(const K& x);
  template <class K> const_iterator upper_bound(const K& x) const;

  pair<iterator, iterator>               equal_range(const key_type& x);
  pair<const_iterator, const_iterator>   equal_range(const key_type& x) const;
  template <class K>
    pair<iterator, iterator>             equal_range(const K& x);
  template <class K>
    pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template <class Key, class Compare, class Allocator>
  bool operator==(const set<Key, Compare, Allocator>& x,
                  const set<Key, Compare, Allocator>& y);
template <class Key, class Compare, class Allocator>
```

```
    bool operator< (const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator!=(const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator> (const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator>=(const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator<=(const set<Key, Compare, Allocator>& x,
                    const set<Key, Compare, Allocator>& y);

  // specialized algorithms:
  template <class Key, class Compare, class Allocator>
    void swap(set<Key, Compare, Allocator>& x,
              set<Key, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.4.6.2 `set` constructors, copy, and assignment [set.cons]

```
explicit set(const Compare& comp, const Allocator& = Allocator());
```

1 *Effects:* Constructs an empty set using the specified comparison objects and allocator.

2 *Complexity:* Constant.

```
template <class InputIterator>
  set(InputIterator first, InputIterator last,
      const Compare& comp = Compare(), const Allocator& = Allocator());
```

3 *Effects:* Constructs an empty `set` using the specified comparison object and allocator, and inserts elements from the range `[first,last)`.

4 *Complexity:* Linear in N if the range `[first,last)` is already sorted using `comp` and otherwise $N \log N$, where N is `last - first`.

23.4.6.3 `set` specialized algorithms [set.special]

```
template <class Key, class Compare, class Allocator>
  void swap(set<Key, Compare, Allocator>& x,
            set<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.4.7 Class template `multiset` [multiset]

23.4.7.1 Class template `multiset` overview [multiset.overview]

1 A `multiset` is an associative container that supports equivalent keys (possibly contains multiple copies of the same key value) and provides for fast retrieval of the keys themselves. The `multiset` class supports bidirectional iterators.

2 A `multiset` satisfies all of the requirements of a container, of a reversible container (23.2), of an associative container (23.2.4), and of an allocator-aware container (Table 98). `multiset` also provides most operations described in (23.2.4) for duplicate keys. This means that a `multiset` supports the `a_eq` operations in (23.2.4) but not the `a_uniq` operations. For a `multiset<Key>` both the `key_type` and `value_type` are `Key`. Descriptions are provided here only for operations on `multiset` that are not described in one of these tables and for operations where there is additional semantic information.

```
namespace std {
  template <class Key, class Compare = less<Key>,
            class Allocator = allocator<Key> >
  class multiset {
  public:
    // types:
    typedef Key                                                 key_type;
    typedef Key                                                 value_type;
    typedef Compare                                             key_compare;
    typedef Compare                                             value_compare;
    typedef Allocator                                           allocator_type;
    typedef value_type&                                         reference;
    typedef const value_type&                                   const_reference;
    typedef implementation-defined                iterator;       // see 23.2
    typedef implementation-defined                const_iterator; // see 23.2
    typedef implementation-defined                size_type;      // see 23.2
    typedef implementation-defined                difference_type;// see 23.2
    typedef typename allocator_traits<Allocator>::pointer       pointer;
    typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
    typedef std::reverse_iterator<iterator>       reverse_iterator;
    typedef std::reverse_iterator<const_iterator> const_reverse_iterator;

    // construct/copy/destroy:
    multiset() : multiset(Compare()) { }
    explicit multiset(const Compare& comp, const Allocator& = Allocator());
    template <class InputIterator>
      multiset(InputIterator first, InputIterator last,
               const Compare& comp = Compare(), const Allocator& = Allocator());
    multiset(const multiset& x);
    multiset(multiset&& x);
    explicit multiset(const Allocator&);
    multiset(const multiset&, const Allocator&);
    multiset(multiset&&, const Allocator&);
    multiset(initializer_list<value_type>, const Compare& = Compare(),
             const Allocator& = Allocator());
    template <class InputIterator>
      multiset(InputIterator first, InputIterator last, const Allocator& a)
        : multiset(first, last, Compare(), a) { }
    multiset(initializer_list<value_type> il, const Allocator& a)
      : multiset(il, Compare(), a) { }
   ~multiset();
    multiset& operator=(const multiset& x);
    multiset& operator=(multiset&& x)
      noexcept(allocator_traits<Allocator>::is_always_equal::value &&
               is_nothrow_move_assignable<Compare>::value);
    multiset& operator=(initializer_list<value_type>);
    allocator_type get_allocator() const noexcept;
```

```
// iterators:
iterator               begin() noexcept;
const_iterator         begin() const noexcept;
iterator               end() noexcept;
const_iterator         end() const noexcept;

reverse_iterator       rbegin() noexcept;
const_reverse_iterator rbegin() const noexcept;
reverse_iterator       rend() noexcept;
const_reverse_iterator rend() const noexcept;

const_iterator         cbegin() const noexcept;
const_iterator         cend() const noexcept;
const_reverse_iterator crbegin() const noexcept;
const_reverse_iterator crend() const noexcept;

// capacity:
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers:
template <class... Args> iterator emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
iterator insert(const value_type& x);
iterator insert(value_type&& x);
iterator insert(const_iterator position, const value_type& x);
iterator insert(const_iterator position, value_type&& x);
template <class InputIterator>
  void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& x);
iterator  erase(const_iterator first, const_iterator last);
void      swap(multiset&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Compare&>(), declval<Compare&>())));
void      clear() noexcept;

// observers:
key_compare key_comp() const;
value_compare value_comp() const;

// set operations:
iterator       find(const key_type& x);
const_iterator find(const key_type& x) const;
template <class K> iterator       find(const K& x);
template <class K> const_iterator find(const K& x) const;

size_type      count(const key_type& x) const;
template <class K> size_type count(const K& x) const;

iterator       lower_bound(const key_type& x);
```

```
    const_iterator lower_bound(const key_type& x) const;
    template <class K> iterator       lower_bound(const K& x);
    template <class K> const_iterator lower_bound(const K& x) const;

    iterator       upper_bound(const key_type& x);
    const_iterator upper_bound(const key_type& x) const;
    template <class K> iterator       upper_bound(const K& x);
    template <class K> const_iterator upper_bound(const K& x) const;

    pair<iterator, iterator>               equal_range(const key_type& x);
    pair<const_iterator, const_iterator>   equal_range(const key_type& x) const;
    template <class K>
      pair<iterator, iterator>             equal_range(const K& x);
    template <class K>
      pair<const_iterator, const_iterator> equal_range(const K& x) const;
  };

  template <class Key, class Compare, class Allocator>
    bool operator==(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator< (const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator!=(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator> (const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator>=(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);
  template <class Key, class Compare, class Allocator>
    bool operator<=(const multiset<Key, Compare, Allocator>& x,
                    const multiset<Key, Compare, Allocator>& y);

  // specialized algorithms:
  template <class Key, class Compare, class Allocator>
    void swap(multiset<Key, Compare, Allocator>& x,
              multiset<Key, Compare, Allocator>& y)
      noexcept(noexcept(x.swap(y)));
}
```

23.4.7.2 `multiset` constructors [multiset.cons]

```
explicit multiset(const Compare& comp, const Allocator& = Allocator());
```

1 *Effects:* Constructs an empty set using the specified comparison object and allocator.

2 *Complexity:* Constant.

```
template <class InputIterator>
  multiset(InputIterator first, InputIterator last,
           const Compare& comp = Compare(), const Allocator& = Allocator());
```

3 *Effects:* Constructs an empty `multiset` using the specified comparison object and allocator, and inserts elements from the range `[first,last)`.

4 *Complexity:* Linear in N if the range `[first,last)` is already sorted using `comp` and otherwise $N \log N$, where N is `last - first`.

23.4.7.3 `multiset` specialized algorithms [multiset.special]

```
template <class Key, class Compare, class Allocator>
  void swap(multiset<Key, Compare, Allocator>& x,
            multiset<Key, Compare, Allocator>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:*

```
x.swap(y);
```

23.5 Unordered associative containers [unord]

23.5.1 In general [unord.general]

1 The header `<unordered_map>` defines the class templates `unordered_map` and `unordered_multimap`; the header `<unordered_set>` defines the class templates `unordered_set` and `unordered_multiset`.

23.5.2 Header `<unordered_map>` synopsis [unord.map.syn]

```
#include <initializer_list>

namespace std {
  // 23.5.4, class template unordered_map:
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Alloc = std::allocator<std::pair<const Key, T> > >
    class unordered_map;

  // 23.5.5, class template unordered_multimap:
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Alloc = std::allocator<std::pair<const Key, T> > >
    class unordered_multimap;

  template <class Key, class T, class Hash, class Pred, class Alloc>
    void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,
              unordered_map<Key, T, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class T, class Hash, class Pred, class Alloc>
    void swap(unordered_multimap<Key, T, Hash, Pred, Alloc>& x,
              unordered_multimap<Key, T, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_map<Key, T, Hash, Pred, Alloc>& b);
  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_map<Key, T, Hash, Pred, Alloc>& b);
```

```
  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
} // namespace std
```

23.5.3 Header <unordered_set> synopsis [unord.set.syn]

```
#include <initializer_list>

namespace std {
  // 23.5.6, class template unordered_set:
  template <class Key,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Alloc = std::allocator<Key> >
    class unordered_set;

  // 23.5.7, class template unordered_multiset:
  template <class Key,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Alloc = std::allocator<Key> >
    class unordered_multiset;

  template <class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
              unordered_set<Key, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_multiset<Key, Hash, Pred, Alloc>& x,
              unordered_multiset<Key, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_set<Key, Hash, Pred, Alloc>& a,
                    const unordered_set<Key, Hash, Pred, Alloc>& b);
  template <class Key, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_set<Key, Hash, Pred, Alloc>& a,
                    const unordered_set<Key, Hash, Pred, Alloc>& b);
  template <class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
                    const unordered_multiset<Key, Hash, Pred, Alloc>& b);
  template <class Key, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
                    const unordered_multiset<Key, Hash, Pred, Alloc>& b);
} // namespace std
```

23.5.4 Class template unordered_map [unord.map]

23.5.4.1 Class template unordered_map overview [unord.map.overview]

1 An unordered_map is an unordered associative container that supports unique keys (an unordered_map contains at most one of each key value) and that associates values of another type mapped_type with the

keys. The `unordered_map` class supports forward iterators.

2 An `unordered_map` satisfies all of the requirements of a container, of an unordered associative container, and of an allocator-aware container (Table 98). It provides the operations described in the preceding requirements table for unique keys; that is, an `unordered_map` supports the `a_uniq` operations in that table, not the `a_eq` operations. For an `unordered_map<Key, T>` the `key type` is `Key`, the mapped type is `T`, and the value type is `std::pair<const Key, T>`.

3 This section only describes operations on `unordered_map` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Allocator = std::allocator<std::pair<const Key, T> > >
  class unordered_map
  {
  public:
    // types
    typedef Key                                                 key_type;
    typedef std::pair<const Key, T>                             value_type;
    typedef T                                                   mapped_type;
    typedef Hash                                                hasher;
    typedef Pred                                                key_equal;
    typedef Allocator                                           allocator_type;
    typedef typename allocator_traits<Allocator>::pointer       pointer;
    typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
    typedef value_type&                                         reference;
    typedef const value_type&                                   const_reference;
    typedef implementation-defined                              size_type;
    typedef implementation-defined                              difference_type;

    typedef implementation-defined                              iterator;
    typedef implementation-defined                              const_iterator;
    typedef implementation-defined                              local_iterator;
    typedef implementation-defined                              const_local_iterator;

    // construct/destroy/copy
    unordered_map();
    explicit unordered_map(size_type n,
                           const hasher& hf = hasher(),
                           const key_equal& eql = key_equal(),
                           const allocator_type& a = allocator_type());
    template <class InputIterator>
      unordered_map(InputIterator f, InputIterator l,
                    size_type n = see below,
                    const hasher& hf = hasher(),
                    const key_equal& eql = key_equal(),
                    const allocator_type& a = allocator_type());
    unordered_map(const unordered_map&);
    unordered_map(unordered_map&&);
    explicit unordered_map(const Allocator&);
    unordered_map(const unordered_map&, const Allocator&);
    unordered_map(unordered_map&&, const Allocator&);
```

```
unordered_map(initializer_list<value_type> il,
              size_type n = see below,
              const hasher& hf = hasher(),
              const key_equal& eql = key_equal(),
              const allocator_type& a = allocator_type());
unordered_map(size_type n, const allocator_type& a)
  : unordered_map(n, hasher(), key_equal(), a) { }
unordered_map(size_type n, const hasher& hf, const allocator_type& a)
  : unordered_map(n, hf, key_equal(), a) { }
template <class InputIterator>
  unordered_map(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
    : unordered_map(f, l, n, hasher(), key_equal(), a) { }
template <class InputIterator>
  unordered_map(InputIterator f, InputIterator l, size_type n, const hasher& hf,
                const allocator_type& a)
    : unordered_map(f, l, n, hf, key_equal(), a) { }
unordered_map(initializer_list<value_type> il, size_type n, const allocator_type& a)
  : unordered_map(il, n, hasher(), key_equal(), a) { }
unordered_map(initializer_list<value_type> il, size_type n, const hasher& hf,
              const allocator_type& a)
  : unordered_map(il, n, hf, key_equal(), a) { }
~unordered_map();
unordered_map& operator=(const unordered_map&);
unordered_map& operator=(unordered_map&&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Hash>::value &&
           is_nothrow_move_assignable<Pred>::value);
unordered_map& operator=(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// iterators
iterator       begin() noexcept;
const_iterator begin() const noexcept;
iterator       end() noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// size and capacity
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers
template <class... Args> pair<iterator, bool> emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
pair<iterator, bool> insert(const value_type& obj);
pair<iterator, bool> insert(value_type&& obj);
template <class P> pair<iterator, bool> insert(P&& obj);
iterator       insert(const_iterator hint, const value_type& obj);
iterator       insert(const_iterator hint, value_type&& obj);
template <class P> iterator insert(const_iterator hint, P&& obj);
template <class InputIterator> void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);
```

```
template <class... Args>
  pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template <class... Args>
  pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template <class... Args>
  iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
template <class... Args>
  iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
template <class M>
  pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template <class M>
  pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template <class M>
  iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
template <class M>
  iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& k);
iterator  erase(const_iterator first, const_iterator last);
void      swap(unordered_map&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
           noexcept(swap(declval<Pred&>(), declval<Pred&>())));
void      clear() noexcept;

// observers
hasher hash_function() const;
key_equal key_eq() const;

// lookup
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
size_type      count(const key_type& k) const;
std::pair<iterator, iterator>             equal_range(const key_type& k);
std::pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

mapped_type& operator[](const key_type& k);
mapped_type& operator[](key_type&& k);
mapped_type& at(const key_type& k);
const mapped_type& at(const key_type& k) const;

// bucket interface
size_type bucket_count() const noexcept;
size_type max_bucket_count() const noexcept;
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;
local_iterator begin(size_type n);
const_local_iterator begin(size_type n) const;
local_iterator end(size_type n);
const_local_iterator end(size_type n) const;
const_local_iterator cbegin(size_type n) const;
const_local_iterator cend(size_type n) const;
```

```
    // hash policy
    float load_factor() const noexcept;
    float max_load_factor() const noexcept;
    void max_load_factor(float z);
    void rehash(size_type n);
    void reserve(size_type n);
  };

  template <class Key, class T, class Hash, class Pred, class Alloc>
    void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,
              unordered_map<Key, T, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_map<Key, T, Hash, Pred, Alloc>& b);
  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_map<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_map<Key, T, Hash, Pred, Alloc>& b);
}
```

23.5.4.2 `unordered_map` constructors [unord.map.cnstr]

```
unordered_map() : unordered_map(size_type(see below)) { }
explicit unordered_map(size_type n,
                       const hasher& hf = hasher(),
                       const key_equal& eql = key_equal(),
                       const allocator_type& a = allocator_type());
```

1 *Effects:* Constructs an empty `unordered_map` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.

2 *Complexity:* Constant.

```
template <class InputIterator>
  unordered_map(InputIterator f, InputIterator l,
                size_type n = see below,
                const hasher& hf = hasher(),
                const key_equal& eql = key_equal(),
                const allocator_type& a = allocator_type());
unordered_map(initializer_list<value_type> il,
              size_type n = see below,
              const hasher& hf = hasher(),
              const key_equal& eql = key_equal(),
              const allocator_type& a = allocator_type());
```

3 *Effects:* Constructs an empty `unordered_map` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)` for the first form, or from the range `[il.begin(),il.end())` for the second form. `max_load_factor()` returns 1.0.

4 *Complexity:* Average case linear, worst case quadratic.

23.5.4.3 `unordered_map` element access [unord.map.elem]

```
mapped_type& operator[](const key_type& k);
mapped_type& operator[](key_type&& k);
```

1 *Requires:* `mapped_type` shall be `DefaultInsertable` into `*this`. For the first operator, `key_type` shall be `CopyInsertable` into `*this`. For the second operator, `key_type` shall be `MoveConstructible`.

2 *Effects:* If the `unordered_map` does not already contain an element whose key is equivalent to `k`, the first operator inserts the value `value_type(k, mapped_type())` and the second operator inserts the value `value_type(std::move(k), mapped_type())`.

3 *Returns:* A reference to `x.second`, where `x` is the (unique) element whose key is equivalent to `k`.

4 *Complexity:* Average case $\mathcal{O}(1)$, worst case $\mathcal{O}$(`size()`).

```
mapped_type& at(const key_type& k);
const mapped_type& at(const key_type& k) const;
```

5 *Returns:* A reference to `x.second`, where `x` is the (unique) element whose key is equivalent to `k`.

6 *Throws:* An exception object of type `out_of_range` if no such element is present.

23.5.4.4 `unordered_map` modifiers [unord.map.modifiers]

```
template <class P>
  pair<iterator, bool> insert(P&& obj);
```

1 *Effects:* Equivalent to `return emplace(std::forward<P>(obj))`.

2 *Remarks:* This signature shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

```
template <class P>
  iterator insert(const_iterator hint, P&& obj);
```

3 *Effects:* Equivalent to `return emplace_hint(hint, std::forward<P>(obj))`.

4 *Remarks:* This signature shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

```
template <class... Args> pair<iterator, bool> try_emplace(const key_type& k, Args&&... args);
template <class... Args> iterator try_emplace(const_iterator hint, const key_type& k, Args&&... args);
```

5 *Requires:* `value_type` shall be `EmplaceConstructible` into `unordered_map` from `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(forward<Args>(args)...)`.

6 *Effects:* If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct`, `forward_as_tuple(k)`, `forward_as_tuple(forward<Args>(args)...)`.

7 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

8 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template <class... Args> pair<iterator, bool> try_emplace(key_type&& k, Args&&... args);
template <class... Args> iterator try_emplace(const_iterator hint, key_type&& k, Args&&... args);
```

9 *Requires:* `value_type` shall be `EmplaceConstructible` into `unordered_map` from `piecewise_construct`, `forward_as_tuple(move(k))`, `forward_as_tuple(forward<Args>(args)...)`.

10 *Effects:* If the map already contains an element whose key is equivalent to `k`, there is no effect. Otherwise inserts an object of type `value_type` constructed with `piecewise_construct`, `forward_as_tuple(move(k))`, `forward_as_tuple(forward<Args>(args)...)`.

11 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

12 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template <class M> pair<iterator, bool> insert_or_assign(const key_type& k, M&& obj);
template <class M> iterator insert_or_assign(const_iterator hint, const key_type& k, M&& obj);
```

13 *Requires:* `is_assignable<mapped_type&, M&&>::value` shall be `true`. `value_type` shall be `EmplaceConstructible` into `unordered_map` from `k`, `forward<M>(obj)`.

14 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `k`, `forward<M>(obj)`.

15 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

16 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

```
template <class M> pair<iterator, bool> insert_or_assign(key_type&& k, M&& obj);
template <class M> iterator insert_or_assign(const_iterator hint, key_type&& k, M&& obj);
```

17 *Requires:* `is_assignable<mapped_type&, M&&>::value` shall be `true`. `value_type` shall be `EmplaceConstructible` into `unordered_map` from `move(k)`, `forward<M>(obj)`.

18 *Effects:* If the map already contains an element `e` whose key is equivalent to `k`, assigns `forward<M>(obj)` to `e.second`. Otherwise inserts an object of type `value_type` constructed with `move(k)`, `forward<M>(obj)`.

19 *Returns:* In the first overload, the `bool` component of the returned pair is `true` if and only if the insertion took place. The returned iterator points to the map element whose key is equivalent to `k`.

20 *Complexity:* The same as `emplace` and `emplace_hint`, respectively.

23.5.4.5 `unordered_map` swap [unord.map.swap]

```
template <class Key, class T, class Hash, class Pred, class Alloc>
  void swap(unordered_map<Key, T, Hash, Pred, Alloc>& x,
            unordered_map<Key, T, Hash, Pred, Alloc>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

23.5.5 Class template `unordered_multimap` [unord.multimap]

23.5.5.1 Class template `unordered_multimap` overview [unord.multimap.overview]

1 An `unordered_multimap` is an unordered associative container that supports equivalent keys (an instance of `unordered_multimap` may contain multiple copies of each key value) and that associates values of another type `mapped_type` with the keys. The `unordered_multimap` class supports forward iterators.

2 An `unordered_multimap` satisfies all of the requirements of a container, of an unordered associative container, and of an allocator-aware container (Table 98). It provides the operations described in the preceding requirements table for equivalent keys; that is, an `unordered_multimap` supports the `a_eq` operations in that table, not the `a_uniq` operations. For an `unordered_multimap<Key, T>` the `key type` is `Key`, the mapped type is `T`, and the value type is `std::pair<const Key, T>`.

3 This section only describes operations on `unordered_multimap` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template <class Key,
            class T,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Allocator = std::allocator<std::pair<const Key, T> > >
  class unordered_multimap
  {
```

```
public:
  // types
  typedef Key                                                 key_type;
  typedef std::pair<const Key, T>                             value_type;
  typedef T                                                   mapped_type;
  typedef Hash                                                hasher;
  typedef Pred                                                key_equal;
  typedef Allocator                                           allocator_type;
  typedef typename allocator_traits<Allocator>::pointer       pointer;
  typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
  typedef value_type&                                         reference;
  typedef const value_type&                                   const_reference;
  typedef implementation-defined                              size_type;
  typedef implementation-defined                              difference_type;

  typedef implementation-defined                              iterator;
  typedef implementation-defined                              const_iterator;
  typedef implementation-defined                              local_iterator;
  typedef implementation-defined                              const_local_iterator;

  // construct/destroy/copy
  unordered_multimap();
  explicit unordered_multimap(size_type n,
                              const hasher& hf = hasher(),
                              const key_equal& eql = key_equal(),
                              const allocator_type& a = allocator_type());
  template <class InputIterator>
    unordered_multimap(InputIterator f, InputIterator l,
                       size_type n = see below,
                       const hasher& hf = hasher(),
                       const key_equal& eql = key_equal(),
                       const allocator_type& a = allocator_type());
  unordered_multimap(const unordered_multimap&);
  unordered_multimap(unordered_multimap&&);
  explicit unordered_multimap(const Allocator&);
  unordered_multimap(const unordered_multimap&, const Allocator&);
  unordered_multimap(unordered_multimap&&, const Allocator&);
  unordered_multimap(initializer_list<value_type> il,
                     size_type n = see below,
                     const hasher& hf = hasher(),
                     const key_equal& eql = key_equal(),
                     const allocator_type& a = allocator_type());
  unordered_multimap(size_type n, const allocator_type& a)
    : unordered_multimap(n, hasher(), key_equal(), a) { }
  unordered_multimap(size_type n, const hasher& hf, const allocator_type& a)
    : unordered_multimap(n, hf, key_equal(), a) { }
  template <class InputIterator>
    unordered_multimap(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
      : unordered_multimap(f, l, n, hasher(), key_equal(), a) { }
  template <class InputIterator>
    unordered_multimap(InputIterator f, InputIterator l, size_type n, const hasher& hf,
                       const allocator_type& a)
      : unordered_multimap(f, l, n, hf, key_equal(), a) { }
  unordered_multimap(initializer_list<value_type> il, size_type n, const allocator_type& a)
    : unordered_multimap(il, n, hasher(), key_equal(), a) { }
```

```
unordered_multimap(initializer_list<value_type> il, size_type n, const hasher& hf,
                   const allocator_type& a)
  : unordered_multimap(il, n, hf, key_equal(), a) { }
~unordered_multimap();
unordered_multimap& operator=(const unordered_multimap&);
unordered_multimap& operator=(unordered_multimap&&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Hash>::value &&
           is_nothrow_move_assignable<Pred>::value);
unordered_multimap& operator=(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// iterators
iterator       begin() noexcept;
const_iterator begin() const noexcept;
iterator       end() noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// size and capacity
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers
template <class... Args> iterator emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
iterator insert(const value_type& obj);
iterator insert(value_type&& obj);
template <class P> iterator insert(P&& obj);
iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);
template <class P> iterator insert(const_iterator hint, P&& obj);
template <class InputIterator> void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& k);
iterator  erase(const_iterator first, const_iterator last);
void      swap(unordered_multimap&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
           noexcept(swap(declval<Pred&>(), declval<Pred&>())));
void      clear() noexcept;

// observers
hasher hash_function() const;
key_equal key_eq() const;

// lookup
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
size_type      count(const key_type& k) const;
```

```
    std::pair<iterator, iterator>             equal_range(const key_type& k);
    std::pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

    // bucket interface
    size_type bucket_count() const noexcept;
    size_type max_bucket_count() const noexcept;
    size_type bucket_size(size_type n) const;
    size_type bucket(const key_type& k) const;
    local_iterator begin(size_type n);
    const_local_iterator begin(size_type n) const;
    local_iterator end(size_type n);
    const_local_iterator end(size_type n) const;
    const_local_iterator cbegin(size_type n) const;
    const_local_iterator cend(size_type n) const;

    // hash policy
    float load_factor() const noexcept;
    float max_load_factor() const noexcept;
    void max_load_factor(float z);
    void rehash(size_type n);
    void reserve(size_type n);
  };

  template <class Key, class T, class Hash, class Pred, class Alloc>
    void swap(unordered_multimap<Key, T, Hash, Pred, Alloc>& x,
              unordered_multimap<Key, T, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
  template <class Key, class T, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_multimap<Key, T, Hash, Pred, Alloc>& a,
                    const unordered_multimap<Key, T, Hash, Pred, Alloc>& b);
}
```

23.5.5.2 `unordered_multimap` constructors [unord.multimap.cnstr]

```
unordered_multimap() : unordered_multimap(size_type(see below)) { }
explicit unordered_multimap(size_type n,
                            const hasher& hf = hasher(),
                            const key_equal& eql = key_equal(),
                            const allocator_type& a = allocator_type());
```

1 *Effects:* Constructs an empty `unordered_multimap` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.

2 *Complexity:* Constant.

```
template <class InputIterator>
  unordered_multimap(InputIterator f, InputIterator l,
                     size_type n = see below,
                     const hasher& hf = hasher(),
                     const key_equal& eql = key_equal(),
                     const allocator_type& a = allocator_type());
```

```
unordered_multimap(initializer_list<value_type> il,
                   size_type n = see below,
                   const hasher& hf = hasher(),
                   const key_equal& eql = key_equal(),
                   const allocator_type& a = allocator_type());
```

3 *Effects:* Constructs an empty `unordered_multimap` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)` for the first form, or from the range `[il.begin(),il.end())` for the second form. `max_load_factor()` returns 1.0.

4 *Complexity:* Average case linear, worst case quadratic.

23.5.5.3 `unordered_multimap` modifiers [unord.multimap.modifiers]

```
template <class P>
  iterator insert(P&& obj);
```

1 *Effects:* Equivalent to `return emplace(std::forward<P>(obj))`.

2 *Remarks:* This signature shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

```
template <class P>
  iterator insert(const_iterator hint, P&& obj);
```

3 *Effects:* Equivalent to `return emplace_hint(hint, std::forward<P>(obj))`.

4 *Remarks:* This signature shall not participate in overload resolution unless `std::is_constructible<value_type, P&&>::value` is `true`.

23.5.5.4 `unordered_multimap` swap [unord.multimap.swap]

```
template <class Key, class T, class Hash, class Pred, class Alloc>
  void swap(unordered_multimap<Key, T, Hash, Pred, Alloc>& x,
            unordered_multimap<Key, T, Hash, Pred, Alloc>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

23.5.6 Class template `unordered_set` [unord.set]

23.5.6.1 Class template `unordered_set` overview [unord.set.overview]

1 An `unordered_set` is an unordered associative container that supports unique keys (an `unordered_set` contains at most one of each key value) and in which the elements' keys are the elements themselves. The `unordered_set` class supports forward iterators.

2 An `unordered_set` satisfies all of the requirements of a container, of an unordered associative container, and of an allocator-aware container (Table 98). It provides the operations described in the preceding requirements table for unique keys; that is, an `unordered_set` supports the `a_uniq` operations in that table, not the `a_eq` operations. For an `unordered_set<Key>` the `key type` and the value type are both `Key`. The `iterator` and `const_iterator` types are both const iterator types. It is unspecified whether they are the same type.

3 This section only describes operations on `unordered_set` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template <class Key,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
```

```
          class Allocator = std::allocator<Key> >
class unordered_set
{
public:
  // types
  typedef Key                                                 key_type;
  typedef Key                                                 value_type;
  typedef Hash                                                hasher;
  typedef Pred                                                key_equal;
  typedef Allocator                                           allocator_type;
  typedef typename allocator_traits<Allocator>::pointer       pointer;
  typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
  typedef value_type&                                         reference;
  typedef const value_type&                                   const_reference;
  typedef implementation-defined                              size_type;
  typedef implementation-defined                              difference_type;

  typedef implementation-defined                              iterator;
  typedef implementation-defined                              const_iterator;
  typedef implementation-defined                              local_iterator;
  typedef implementation-defined                              const_local_iterator;

  // construct/destroy/copy
  unordered_set();
  explicit unordered_set(size_type n,
                         const hasher& hf = hasher(),
                         const key_equal& eql = key_equal(),
                         const allocator_type& a = allocator_type());
  template <class InputIterator>
    unordered_set(InputIterator f, InputIterator l,
                  size_type n = see below,
                  const hasher& hf = hasher(),
                  const key_equal& eql = key_equal(),
                  const allocator_type& a = allocator_type());
  unordered_set(const unordered_set&);
  unordered_set(unordered_set&&);
  explicit unordered_set(const Allocator&);
  unordered_set(const unordered_set&, const Allocator&);
  unordered_set(unordered_set&&, const Allocator&);
  unordered_set(initializer_list<value_type> il,
                size_type n = see below,
                const hasher& hf = hasher(),
                const key_equal& eql = key_equal(),
                const allocator_type& a = allocator_type());
  unordered_set(size_type n, const allocator_type& a)
    : unordered_set(n, hasher(), key_equal(), a) { }
  unordered_set(size_type n, const hasher& hf, const allocator_type& a)
    : unordered_set(n, hf, key_equal(), a) { }
  template <class InputIterator>
    unordered_set(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
      : unordered_set(f, l, n, hasher(), key_equal(), a) { }
  template <class InputIterator>
    unordered_set(InputIterator f, InputIterator l, size_type n, const hasher& hf,
                  const allocator_type& a)
    : unordered_set(f, l, n, hf, key_equal(), a) { }
```

```
unordered_set(initializer_list<value_type> il, size_type n, const allocator_type& a)
  : unordered_set(il, n, hasher(), key_equal(), a) { }
unordered_set(initializer_list<value_type> il, size_type n, const hasher& hf,
              const allocator_type& a)
  : unordered_set(il, n, hf, key_equal(), a) { }
~unordered_set();
unordered_set& operator=(const unordered_set&);
unordered_set& operator=(unordered_set&&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Hash>::value &&
           is_nothrow_move_assignable<Pred>::value);
unordered_set& operator=(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// iterators
iterator       begin() noexcept;
const_iterator begin() const noexcept;
iterator       end() noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// size and capacity
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers
template <class... Args> pair<iterator, bool> emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
pair<iterator, bool> insert(const value_type& obj);
pair<iterator, bool> insert(value_type&& obj);
iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);
template <class InputIterator> void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& k);
iterator  erase(const_iterator first, const_iterator last);
void      swap(unordered_set&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
           noexcept(swap(declval<Pred&>(), declval<Pred&>())));
void      clear() noexcept;

// observers
hasher hash_function() const;
key_equal key_eq() const;

// lookup
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
size_type      count(const key_type& k) const;
```

```
    std::pair<iterator, iterator>             equal_range(const key_type& k);
    std::pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

    // bucket interface
    size_type bucket_count() const noexcept;
    size_type max_bucket_count() const noexcept;
    size_type bucket_size(size_type n) const;
    size_type bucket(const key_type& k) const;
    local_iterator begin(size_type n);
    const_local_iterator begin(size_type n) const;
    local_iterator end(size_type n);
    const_local_iterator end(size_type n) const;
    const_local_iterator cbegin(size_type n) const;
    const_local_iterator cend(size_type n) const;

    // hash policy
    float load_factor() const noexcept;
    float max_load_factor() const noexcept;
    void max_load_factor(float z);
    void rehash(size_type n);
    void reserve(size_type n);
  };

  template <class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
              unordered_set<Key, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_set<Key, Hash, Pred, Alloc>& a,
                    const unordered_set<Key, Hash, Pred, Alloc>& b);
  template <class Key, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_set<Key, Hash, Pred, Alloc>& a,
                    const unordered_set<Key, Hash, Pred, Alloc>& b);
}
```

23.5.6.2 `unordered_set` constructors [unord.set.cnstr]

```
unordered_set() : unordered_set(size_type(see below)) { }
explicit unordered_set(size_type n,
                       const hasher& hf = hasher(),
                       const key_equal& eql = key_equal(),
                       const allocator_type& a = allocator_type());
```

1 *Effects:* Constructs an empty `unordered_set` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.

2 *Complexity:* Constant.

```
template <class InputIterator>
  unordered_set(InputIterator f, InputIterator l,
                size_type n = see below,
                const hasher& hf = hasher(),
                const key_equal& eql = key_equal(),
                const allocator_type& a = allocator_type());
```

```
unordered_set(initializer_list<value_type> il,
              size_type n = see below,
              const hasher& hf = hasher(),
              const key_equal& eql = key_equal(),
              const allocator_type& a = allocator_type());
```

3 *Effects:* Constructs an empty `unordered_set` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)` for the first form, or from the range `[il.begin(),il.end())` for the second form. `max_load_factor()` returns 1.0.

4 *Complexity:* Average case linear, worst case quadratic.

23.5.6.3 `unordered_set` swap [unord.set.swap]

```
template <class Key, class Hash, class Pred, class Alloc>
  void swap(unordered_set<Key, Hash, Pred, Alloc>& x,
            unordered_set<Key, Hash, Pred, Alloc>& y)
    noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

23.5.7 Class template `unordered_multiset` [unord.multiset]

23.5.7.1 Class template `unordered_multiset` overview [unord.multiset.overview]

1 An `unordered_multiset` is an unordered associative container that supports equivalent keys (an instance of `unordered_multiset` may contain multiple copies of the same key value) and in which each element's key is the element itself. The `unordered_multiset` class supports forward iterators.

2 An `unordered_multiset` satisfies all of the requirements of a container, of an unordered associative container, and of an allocator-aware container (Table 98). It provides the operations described in the preceding requirements table for equivalent keys; that is, an `unordered_multiset` supports the `a_eq` operations in that table, not the `a_uniq` operations. For an `unordered_multiset<Key>` the `key type` and the value type are both `Key`. The `iterator` and `const_iterator` types are both const iterator types. It is unspecified whether they are the same type.

3 This section only describes operations on `unordered_multiset` that are not described in one of the requirement tables, or for which there is additional semantic information.

```
namespace std {
  template <class Key,
            class Hash = hash<Key>,
            class Pred = std::equal_to<Key>,
            class Allocator = std::allocator<Key> >
  class unordered_multiset
  {
  public:
    // types
    typedef Key                                                key_type;
    typedef Key                                                value_type;
    typedef Hash                                               hasher;
    typedef Pred                                               key_equal;
    typedef Allocator                                          allocator_type;
    typedef typename allocator_traits<Allocator>::pointer      pointer;
    typedef typename allocator_traits<Allocator>::const_pointer const_pointer;
    typedef value_type&                                        reference;
    typedef const value_type&                                  const_reference;
    typedef implementation-defined                             size_type;
```

```
typedef implementation-defined                              difference_type;

typedef implementation-defined                              iterator;
typedef implementation-defined                              const_iterator;
typedef implementation-defined                              local_iterator;
typedef implementation-defined                              const_local_iterator;

// construct/destroy/copy
unordered_multiset();
explicit unordered_multiset(size_type n,
                            const hasher& hf = hasher(),
                            const key_equal& eql = key_equal(),
                            const allocator_type& a = allocator_type());
template <class InputIterator>
  unordered_multiset(InputIterator f, InputIterator l,
                     size_type n = see below,
                     const hasher& hf = hasher(),
                     const key_equal& eql = key_equal(),
                     const allocator_type& a = allocator_type());
unordered_multiset(const unordered_multiset&);
unordered_multiset(unordered_multiset&&);
explicit unordered_multiset(const Allocator&);
unordered_multiset(const unordered_multiset&, const Allocator&);
unordered_multiset(unordered_multiset&&, const Allocator&);
unordered_multiset(initializer_list<value_type> il,
                   size_type n = see below,
                   const hasher& hf = hasher(),
                   const key_equal& eql = key_equal(),
                   const allocator_type& a = allocator_type());
unordered_multiset(size_type n, const allocator_type& a)
  : unordered_multiset(n, hasher(), key_equal(), a) { }
unordered_multiset(size_type n, const hasher& hf, const allocator_type& a)
  : unordered_multiset(n, hf, key_equal(), a) { }
template <class InputIterator>
  unordered_multiset(InputIterator f, InputIterator l, size_type n, const allocator_type& a)
    : unordered_multiset(f, l, n, hasher(), key_equal(), a) { }
template <class InputIterator>
  unordered_multiset(InputIterator f, InputIterator l, size_type n, const hasher& hf,
                     const allocator_type& a)
  : unordered_multiset(f, l, n, hf, key_equal(), a) { }
unordered_multiset(initializer_list<value_type> il, size_type n, const allocator_type& a)
  : unordered_multiset(il, n, hasher(), key_equal(), a) { }
unordered_multiset(initializer_list<value_type> il, size_type n, const hasher& hf,
                   const allocator_type& a)
  : unordered_multiset(il, n, hf, key_equal(), a) { }
~unordered_multiset();
unordered_multiset& operator=(const unordered_multiset&);
unordered_multiset& operator=(unordered_multiset&&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           is_nothrow_move_assignable<Hash>::value &&
           is_nothrow_move_assignable<Pred>::value);
unordered_multiset& operator=(initializer_list<value_type>);
allocator_type get_allocator() const noexcept;

// iterators
```

```
iterator       begin() noexcept;
const_iterator begin() const noexcept;
iterator       end() noexcept;
const_iterator end() const noexcept;
const_iterator cbegin() const noexcept;
const_iterator cend() const noexcept;

// size and capacity
bool      empty() const noexcept;
size_type size() const noexcept;
size_type max_size() const noexcept;

// modifiers
template <class... Args> iterator emplace(Args&&... args);
template <class... Args> iterator emplace_hint(const_iterator position, Args&&... args);
iterator insert(const value_type& obj);
iterator insert(value_type&& obj);
iterator insert(const_iterator hint, const value_type& obj);
iterator insert(const_iterator hint, value_type&& obj);
template <class InputIterator> void insert(InputIterator first, InputIterator last);
void insert(initializer_list<value_type>);

iterator  erase(iterator position);
iterator  erase(const_iterator position);
size_type erase(const key_type& k);
iterator  erase(const_iterator first, const_iterator last);
void      swap(unordered_multiset&)
  noexcept(allocator_traits<Allocator>::is_always_equal::value &&
           noexcept(swap(declval<Hash&>(), declval<Hash&>())) &&
           noexcept(swap(declval<Pred&>(), declval<Pred&>())));
void      clear() noexcept;

// observers
hasher hash_function() const;
key_equal key_eq() const;

// lookup
iterator       find(const key_type& k);
const_iterator find(const key_type& k) const;
size_type      count(const key_type& k) const;
std::pair<iterator, iterator>             equal_range(const key_type& k);
std::pair<const_iterator, const_iterator> equal_range(const key_type& k) const;

// bucket interface
size_type bucket_count() const noexcept;
size_type max_bucket_count() const noexcept;
size_type bucket_size(size_type n) const;
size_type bucket(const key_type& k) const;
local_iterator begin(size_type n);
const_local_iterator begin(size_type n) const;
local_iterator end(size_type n);
const_local_iterator end(size_type n) const;
const_local_iterator cbegin(size_type n) const;
const_local_iterator cend(size_type n) const;
```

```
    // hash policy
    float load_factor() const noexcept;
    float max_load_factor() const noexcept;
    void max_load_factor(float z);
    void rehash(size_type n);
    void reserve(size_type n);
  };

  template <class Key, class Hash, class Pred, class Alloc>
    void swap(unordered_multiset<Key, Hash, Pred, Alloc>& x,
              unordered_multiset<Key, Hash, Pred, Alloc>& y)
      noexcept(noexcept(x.swap(y)));

  template <class Key, class Hash, class Pred, class Alloc>
    bool operator==(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
                    const unordered_multiset<Key, Hash, Pred, Alloc>& b);
  template <class Key, class Hash, class Pred, class Alloc>
    bool operator!=(const unordered_multiset<Key, Hash, Pred, Alloc>& a,
                    const unordered_multiset<Key, Hash, Pred, Alloc>& b);
}
```

23.5.7.2 `unordered_multiset` constructors [unord.multiset.cnstr]

```
unordered_multiset() : unordered_multiset(size_type(see below)) { }
explicit unordered_multiset(size_type n,
                            const hasher& hf = hasher(),
                            const key_equal& eql = key_equal(),
                            const allocator_type& a = allocator_type());
```

1 *Effects:* Constructs an empty `unordered_multiset` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. For the default constructor, the number of buckets is implementation-defined. `max_load_factor()` returns 1.0.

2 *Complexity:* Constant.

```
template <class InputIterator>
  unordered_multiset(InputIterator f, InputIterator l,
                     size_type n = see below,
                     const hasher& hf = hasher(),
                     const key_equal& eql = key_equal(),
                     const allocator_type& a = allocator_type());
unordered_multiset(initializer_list<value_type> il,
                   size_type n = see below,
                   const hasher& hf = hasher(),
                   const key_equal& eql = key_equal(),
                   const allocator_type& a = allocator_type());
```

3 *Effects:* Constructs an empty `unordered_multiset` using the specified hash function, key equality function, and allocator, and using at least `n` buckets. If `n` is not provided, the number of buckets is implementation-defined. Then inserts elements from the range `[f,l)` for the first form, or from the range `[il.begin(),il.end())` for the second form. `max_load_factor()` returns 1.0.

4 *Complexity:* Average case linear, worst case quadratic.

23.5.7.3 `unordered_multiset` swap [unord.multiset.swap]

```
template <class Key, class Hash, class Pred, class Alloc>
```

```
void swap(unordered_multiset<Key, Hash, Pred, Alloc>& x,
          unordered_multiset<Key, Hash, Pred, Alloc>& y)
  noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y);`

23.6 Container adaptors [container.adaptors]

23.6.1 In general [container.adaptors.general]

1 The headers `<queue>` and `<stack>` define the container adaptors `queue`, `priority_queue`, and `stack`.

2 The container adaptors each take a `Container` template parameter, and each constructor takes a `Container` reference argument. This container is copied into the `Container` member of each adaptor. If the container takes an allocator, then a compatible allocator may be passed in to the adaptor's constructor. Otherwise, normal copy or move construction is used for the container argument.

3 For container adaptors, no `swap` function throws an exception unless that exception is thrown by the swap of the adaptor's `Container` or `Compare` object (if any).

23.6.2 Header <queue> synopsis [queue.syn]

```
#include <initializer_list>

namespace std {
  template <class T, class Container = deque<T> > class queue;
  template <class T, class Container = vector<T>,
            class Compare = less<typename Container::value_type> >
    class priority_queue;

  template <class T, class Container>
    bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator>=(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);

  template <class T, class Container>
    void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));
  template <class T, class Container, class Compare>
    void swap(priority_queue<T, Container, Compare>& x,
              priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));
}
```

23.6.3 Class template queue [queue]

23.6.3.1 queue definition [queue.defn]

1 Any sequence container supporting operations `front()`, `back()`, `push_back()` and `pop_front()` can be used to instantiate `queue`. In particular, `list` (23.3.5) and `deque` (23.3.3) can be used.

```
namespace std {
  template <class T, class Container = deque<T> >
```

```
  class queue {
  public:
    typedef typename Container::value_type           value_type;
    typedef typename Container::reference            reference;
    typedef typename Container::const_reference      const_reference;
    typedef typename Container::size_type            size_type;
    typedef          Container                       container_type;

  protected:
    Container c;

  public:
    explicit queue(const Container&);
    explicit queue(Container&& = Container());
    template <class Alloc> explicit queue(const Alloc&);
    template <class Alloc> queue(const Container&, const Alloc&);
    template <class Alloc> queue(Container&&, const Alloc&);
    template <class Alloc> queue(const queue&, const Alloc&);
    template <class Alloc> queue(queue&&, const Alloc&);

    bool              empty() const     { return c.empty(); }
    size_type         size()  const     { return c.size(); }
    reference         front()           { return c.front(); }
    const_reference   front() const     { return c.front(); }
    reference         back()            { return c.back(); }
    const_reference   back() const      { return c.back(); }
    void push(const value_type& x)      { c.push_back(x); }
    void push(value_type&& x)           { c.push_back(std::move(x)); }
    template <class... Args>
      void emplace(Args&&... args)      { c.emplace_back(std::forward<Args>(args)...); }
    void pop()                          { c.pop_front(); }
    void swap(queue& q) noexcept(noexcept(swap(c, q.c)))
      { using std::swap; swap(c, q.c); }
  };

  template <class T, class Container>
    bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator!=(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator>=(const queue<T, Container>& x, const queue<T, Container>& y);
  template <class T, class Container>
    bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);

  template <class T, class Container>
    void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));

  template <class T, class Container, class Alloc>
    struct uses_allocator<queue<T, Container>, Alloc>
      : uses_allocator<Container, Alloc>::type { };
}
```

23.6.3.2 queue constructors [queue.cons]

```
explicit queue(const Container& cont);
```

1 *Effects:* Initializes `c` with `cont`.

```
explicit queue(Container&& cont = Container());
```

2 *Effects:* Initializes `c` with `std::move(cont)`.

23.6.3.3 queue constructors with allocators [queue.cons.alloc]

1 If `uses_allocator<container_type, Alloc>::value` is `false` the constructors in this subclause shall not participate in overload resolution.

```
template <class Alloc> explicit queue(const Alloc& a);
```

2 *Effects:* Initializes `c` with `a`.

```
template <class Alloc> queue(const container_type& cont, const Alloc& a);
```

3 *Effects:* Initializes `c` with `cont` as the first argument and `a` as the second argument.

```
template <class Alloc> queue(container_type&& cont, const Alloc& a);
```

4 *Effects:* Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument.

```
template <class Alloc> queue(const queue& q, const Alloc& a);
```

5 *Effects:* Initializes `c` with `q.c` as the first argument and `a` as the second argument.

```
template <class Alloc> queue(queue&& q, const Alloc& a);
```

6 *Effects:* Initializes `c` with `std::move(q.c)` as the first argument and `a` as the second argument.

23.6.3.4 queue operators [queue.ops]

```
template <class T, class Container>
  bool operator==(const queue<T, Container>& x, const queue<T, Container>& y);
```

1 *Returns:* `x.c == y.c`.

```
template <class T, class Container>
  bool operator!=(const queue<T, Container>& x,  const queue<T, Container>& y);
```

2 *Returns:* `x.c != y.c`.

```
template <class T, class Container>
  bool operator< (const queue<T, Container>& x, const queue<T, Container>& y);
```

3 *Returns:* `x.c < y.c`.

```
template <class T, class Container>
  bool operator<=(const queue<T, Container>& x, const queue<T, Container>& y);
```

4 *Returns:* `x.c <= y.c`.

```
template <class T, class Container>
  bool operator> (const queue<T, Container>& x, const queue<T, Container>& y);
```

5 *Returns:* `x.c > y.c`.

```
template <class T, class Container>
    bool operator>=(const queue<T, Container>& x,
                    const queue<T, Container>& y);
```

6 *Returns:* `x.c >= y.c`.

23.6.3.5 queue specialized algorithms [queue.special]

```
template <class T, class Container>
  void swap(queue<T, Container>& x, queue<T, Container>& y) noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

23.6.4 Class template `priority_queue` [priority.queue]

1 Any sequence container with random access iterator and supporting operations `front()`, `push_back()` and `pop_back()` can be used to instantiate `priority_queue`. In particular, `vector` (23.3.6) and `deque` (23.3.3) can be used. Instantiating `priority_queue` also involves supplying a function or function object for making priority comparisons; the library assumes that the function or function object defines a strict weak ordering (25.4).

```
namespace std {
  template <class T, class Container = vector<T>,
    class Compare = less<typename Container::value_type> >
  class priority_queue {
  public:
    typedef typename Container::value_type           value_type;
    typedef typename Container::reference            reference;
    typedef typename Container::const_reference      const_reference;
    typedef typename Container::size_type            size_type;
    typedef          Container                       container_type;

  protected:
    Container c;
    Compare comp;

  public:
    priority_queue(const Compare& x, const Container&);
    explicit priority_queue(const Compare& x = Compare(), Container&& = Container());
    template <class InputIterator>
      priority_queue(InputIterator first, InputIterator last,
             const Compare& x, const Container&);
    template <class InputIterator>
      priority_queue(InputIterator first, InputIterator last,
             const Compare& x = Compare(), Container&& = Container());
    template <class Alloc> explicit priority_queue(const Alloc&);
    template <class Alloc> priority_queue(const Compare&, const Alloc&);
    template <class Alloc> priority_queue(const Compare&, const Container&, const Alloc&);
    template <class Alloc> priority_queue(const Compare&, Container&&, const Alloc&);
    template <class Alloc> priority_queue(const priority_queue&, const Alloc&);
    template <class Alloc> priority_queue(priority_queue&&, const Alloc&);

    bool      empty() const       { return c.empty(); }
    size_type size()  const       { return c.size(); }
    const_reference   top() const { return c.front(); }
    void push(const value_type& x);
    void push(value_type&& x);
    template <class... Args> void emplace(Args&&... args);
    void pop();
    void swap(priority_queue& q) noexcept(noexcept(swap(c, q.c)) &&
                                          noexcept(swap(comp, q.comp)))
      { using std::swap; swap(c, q.c); swap(comp, q.comp); }
  };
```

```
  // no equality is provided

  template <class T, class Container, class Compare>
    void swap(priority_queue<T, Container, Compare>& x,
              priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));

  template <class T, class Container, class Compare, class Alloc>
    struct uses_allocator<priority_queue<T, Container, Compare>, Alloc>
      : uses_allocator<Container, Alloc>::type { };
}
```

23.6.4.1 `priority_queue` constructors [priqueue.cons]

```
priority_queue(const Compare& x,
               const Container& y);
explicit priority_queue(const Compare& x = Compare(),
               Container&& y = Container());
```

1 *Requires:* `x` shall define a strict weak ordering (25.4).

2 *Effects:* Initializes `comp` with `x` and `c` with `y` (copy constructing or move constructing as appropriate); calls `make_heap(c.begin(), c.end(), comp)`.

```
template <class InputIterator>
  priority_queue(InputIterator first, InputIterator last,
                 const Compare& x,
                 const Container& y);
template <class InputIterator>
  priority_queue(InputIterator first, InputIterator last,
                 const Compare& x = Compare(),
                 Container&& y = Container());
```

3 *Requires:* `x` shall define a strict weak ordering (25.4).

4 *Effects:* Initializes `comp` with `x` and `c` with `y` (copy constructing or move constructing as appropriate); calls `c.insert(c.end(), first, last)`; and finally calls `make_heap(c.begin(), c.end(), comp)`.

23.6.4.2 `priority_queue` constructors with allocators [priqueue.cons.alloc]

1 If `uses_allocator<container_type, Alloc>::value` is `false` the constructors in this subclause shall not participate in overload resolution.

```
template <class Alloc> explicit priority_queue(const Alloc& a);
```

2 *Effects:* Initializes `c` with `a` and value-initializes `comp`.

```
template <class Alloc> priority_queue(const Compare& compare, const Alloc& a);
```

3 *Effects:* Initializes `c` with `a` and initializes `comp` with `compare`.

```
template <class Alloc>
  priority_queue(const Compare& compare, const Container& cont, const Alloc& a);
```

4 *Effects:* Initializes `c` with `cont` as the first argument and `a` as the second argument, and initializes `comp` with `compare`.

```
template <class Alloc>
  priority_queue(const Compare& compare, Container&& cont, const Alloc& a);
```

5 *Effects:* Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument, and initializes `comp` with `compare`.

```
template <class Alloc> priority_queue(const priority_queue& q, const Alloc& a);
```

6 *Effects:* Initializes `c` with `q.c` as the first argument and `a` as the second argument, and initializes `comp` with `q.comp`.

```
template <class Alloc> priority_queue(priority_queue&& q, const Alloc& a);
```

7 *Effects:* Initializes `c` with `std::move(q.c)` as the first argument and `a` as the second argument, and initializes `comp` with `std::move(q.comp)`.

23.6.4.3 `priority_queue` members [priqueue.members]

```
void push(const value_type& x);
```

1 *Effects:*

```
c.push_back(x);
push_heap(c.begin(), c.end(), comp);
```

```
void push(value_type&& x);
```

2 *Effects:*

```
c.push_back(std::move(x));
push_heap(c.begin(), c.end(), comp);
```

```
template <class... Args> void emplace(Args&&... args)
```

3 *Effects:*

```
c.emplace_back(std::forward<Args>(args)...);
push_heap(c.begin(), c.end(), comp);
```

```
void pop();
```

4 *Effects:*

```
pop_heap(c.begin(), c.end(), comp);
c.pop_back();
```

23.6.4.4 `priority_queue` specialized algorithms [priqueue.special]

```
template <class T, class Container, class Compare>
  void swap(priority_queue<T, Container, Compare>& x,
            priority_queue<T, Container, Compare>& y) noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

23.6.5 Class template `stack` [stack]

1 Any sequence container supporting operations `back()`, `push_back()` and `pop_back()` can be used to instantiate `stack`. In particular, `vector` (23.3.6), `list` (23.3.5) and `deque` (23.3.3) can be used.

23.6.5.1 Header <stack> synopsis [stack.syn]

```
#include <initializer_list>

namespace std {
  template <class T, class Container = deque<T> > class stack;
  template <class T, class Container>
    bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator>=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator<=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)));
}
```

23.6.5.2 stack definition [stack.defn]

```
namespace std {
  template <class T, class Container = deque<T> >
  class stack {
  public:
    typedef typename Container::value_type           value_type;
    typedef typename Container::reference            reference;
    typedef typename Container::const_reference      const_reference;
    typedef typename Container::size_type            size_type;
    typedef          Container                       container_type;

  protected:
    Container c;

  public:
    explicit stack(const Container&);
    explicit stack(Container&& = Container());
    template <class Alloc> explicit stack(const Alloc&);
    template <class Alloc> stack(const Container&, const Alloc&);
    template <class Alloc> stack(Container&&, const Alloc&);
    template <class Alloc> stack(const stack&, const Alloc&);
    template <class Alloc> stack(stack&&, const Alloc&);

    bool      empty() const             { return c.empty(); }
    size_type size()  const             { return c.size(); }
    reference         top()             { return c.back(); }
    const_reference   top() const       { return c.back(); }
    void push(const value_type& x)      { c.push_back(x); }
    void push(value_type&& x)           { c.push_back(std::move(x)); }
    template <class... Args>
      void emplace(Args&&... args)      { c.emplace_back(std::forward<Args>(args)...); }
    void pop()                          { c.pop_back(); }
    void swap(stack& s) noexcept(noexcept(swap(c, s.c)))
      { using std::swap; swap(c, s.c); }
```

```
  };

  template <class T, class Container>
    bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator>=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    bool operator<=(const stack<T, Container>& x, const stack<T, Container>& y);
  template <class T, class Container>
    void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)));

  template <class T, class Container, class Alloc>
    struct uses_allocator<stack<T, Container>, Alloc>
      : uses_allocator<Container, Alloc>::type { };
}
```

23.6.5.3 `stack` constructors [stack.cons]

```
explicit stack(const Container& cont);
```

1 *Effects:* Initializes `c` with `cont`.

```
explicit stack(Container&& cont = Container());
```

2 *Effects:* Initializes `c` with `std::move(cont)`.

23.6.5.4 `stack` constructors with allocators [stack.cons.alloc]

1 If `uses_allocator<container_type, Alloc>::value` is `false` the constructors in this subclause shall not participate in overload resolution.

```
template <class Alloc> explicit stack(const Alloc& a);
```

2 *Effects:* Initializes `c` with `a`.

```
template <class Alloc> stack(const container_type& cont, const Alloc& a);
```

3 *Effects:* Initializes `c` with `cont` as the first argument and `a` as the second argument.

```
template <class Alloc> stack(container_type&& cont, const Alloc& a);
```

4 *Effects:* Initializes `c` with `std::move(cont)` as the first argument and `a` as the second argument.

```
template <class Alloc> stack(const stack& s, const Alloc& a);
```

5 *Effects:* Initializes `c` with `s.c` as the first argument and `a` as the second argument.

```
template <class Alloc> stack(stack&& s, const Alloc& a);
```

6 *Effects:* Initializes `c` with `std::move(s.c)` as the first argument and `a` as the second argument.

23.6.5.5 `stack` operators [stack.ops]

```
template <class T, class Container>
  bool operator==(const stack<T, Container>& x, const stack<T, Container>& y);
```

1 *Returns:* `x.c == y.c`.

```
template <class T, class Container>
  bool operator!=(const stack<T, Container>& x, const stack<T, Container>& y);
```

2 *Returns:* `x.c != y.c`.

```
template <class T, class Container>
  bool operator< (const stack<T, Container>& x, const stack<T, Container>& y);
```

3 *Returns:* `x.c < y.c`.

```
template <class T, class Container>
  bool operator<=(const stack<T, Container>& x, const stack<T, Container>& y);
```

4 *Returns:* `x.c <= y.c`.

```
template <class T, class Container>
  bool operator> (const stack<T, Container>& x, const stack<T, Container>& y);
```

5 *Returns:* `x.c > y.c`.

```
template <class T, class Container>
    bool operator>=(const stack<T, Container>& x, const stack<T, Container>& y);
```

6 *Returns:* `x.c >= y.c`.

23.6.5.6 `stack` specialized algorithms [stack.special]

```
template <class T, class Container>
  void swap(stack<T, Container>& x, stack<T, Container>& y) noexcept(noexcept(x.swap(y)));
```

1 *Effects:* `x.swap(y)`.

24 Iterators library [iterators]

24.1 General [iterators.general]

1 This Clause describes components that C++ programs may use to perform iterations over containers (Clause 23), streams (27.7), and stream buffers (27.6).

2 The following subclauses describe iterator requirements, and components for iterator primitives, predefined iterators, and stream iterators, as summarized in Table 103.

Table 103 — Iterators library summary

	Subclause	Header(s)
24.2	Requirements	
24.4	Iterator primitives	`<iterator>`
24.5	Predefined iterators	
24.6	Stream iterators	

24.2 Iterator requirements [iterator.requirements]

24.2.1 In general [iterator.requirements.general]

1 Iterators are a generalization of pointers that allow a C++ program to work with different data structures (containers) in a uniform manner. To be able to construct template algorithms that work correctly and efficiently on different types of data structures, the library formalizes not just the interfaces but also the semantics and complexity assumptions of iterators. All input iterators `i` support the expression `*i`, resulting in a value of some object type `T`, called the *value type* of the iterator. All output iterators support the expression `*i = o` where `o` is a value of some type that is in the set of types that are *writable* to the particular iterator type of `i`. All iterators `i` for which the expression `(*i).m` is well-defined, support the expression `i->m` with the same semantics as `(*i).m`. For every iterator type `X` for which equality is defined, there is a corresponding signed integer type called the *difference type* of the iterator.

2 Since iterators are an abstraction of pointers, their semantics is a generalization of most of the semantics of pointers in C++. This ensures that every function template that takes iterators works as well with regular pointers. This International Standard defines five categories of iterators, according to the operations defined on them: *input iterators*, *output iterators*, *forward iterators*, *bidirectional iterators* and *random access iterators*, as shown in Table 104.

Table 104 — Relations among iterator categories

Random Access	→ **Bidirectional**	→ **Forward**	→ **Input**
			→ **Output**

3 Forward iterators satisfy all the requirements of input iterators and can be used whenever an input iterator is specified; Bidirectional iterators also satisfy all the requirements of forward iterators and can be used whenever a forward iterator is specified; Random access iterators also satisfy all the requirements of bidirectional iterators and can be used whenever a bidirectional iterator is specified.

4 Iterators that further satisfy the requirements of output iterators are called *mutable iterator*s. Nonmutable iterators are referred to as *constant iterator*s.

5 Iterators that further satisfy the requirement that, for integral values `n` and dereferenceable iterator values `a` and `(a + n)`, `*(a + n)` is equivalent to `*(addressof(*a) + n)`, are called *contiguous iterators*. [*Note:* For example, the type "pointer to `int`" is a contiguous iterator, but `reverse_iterator<int *>` is not. For a valid iterator range [`a`,`b`) with dereferenceable `a`, the corresponding range denoted by pointers is [`addressof(*a)`,`addressof(*a) + (b - a)`); `b` might not be dereferenceable. — *end note*]

6 Just as a regular pointer to an array guarantees that there is a pointer value pointing past the last element of the array, so for any iterator type there is an iterator value that points past the last element of a corresponding sequence. These values are called *past-the-end* values. Values of an iterator `i` for which the expression `*i` is defined are called *dereferenceable*. The library never assumes that past-the-end values are dereferenceable. Iterators can also have singular values that are not associated with any sequence. [*Example:* After the declaration of an uninitialized pointer `x` (as with `int* x;`), `x` must always be assumed to have a singular value of a pointer. — *end example*] Results of most expressions are undefined for singular values; the only exceptions are destroying an iterator that holds a singular value, the assignment of a non-singular value to an iterator that holds a singular value, and, for iterators that satisfy the `DefaultConstructible` requirements, using a value-initialized iterator as the source of a copy or move operation. [*Note:* This guarantee is not offered for default initialization, although the distinction only matters for types with trivial default constructors such as pointers or aggregates holding pointers. — *end note*] In these cases the singular value is overwritten the same way as any other value. Dereferenceable values are always non-singular.

7 An iterator `j` is called *reachable* from an iterator `i` if and only if there is a finite sequence of applications of the expression `++i` that makes `i == j`. If `j` is reachable from `i`, they refer to elements of the same sequence.

8 Most of the library's algorithmic templates that operate on data structures have interfaces that use ranges. A *range* is a pair of iterators that designate the beginning and end of the computation. A range `[i,i)` is an empty range; in general, a range `[i,j)` refers to the elements in the data structure starting with the element pointed to by `i` and up to but not including the element pointed to by `j`. Range `[i,j)` is valid if and only if `j` is reachable from `i`. The result of the application of functions in the library to invalid ranges is undefined.

9 All the categories of iterators require only those functions that are realizable for a given category in constant time (amortized). Therefore, requirement tables for the iterators do not have a complexity column.

10 Destruction of an iterator may invalidate pointers and references previously obtained from that iterator.

11 An *invalid* iterator is an iterator that may be singular.[268]

12 In the following sections, `a` and `b` denote values of type `X` or `const X`, `difference_type` and `reference` refer to the types `iterator_traits<X>::difference_type` and `iterator_traits<X>::reference`, respectively, `n` denotes a value of `difference_type`, `u`, `tmp`, and `m` denote identifiers, `r` denotes a value of `X&`, `t` denotes a value of value type `T`, `o` denotes a value of some type that is writable to the output iterator. [*Note:* For an iterator type `X` there must be an instantiation of `iterator_traits<X>` (24.4.1). — *end note*]

24.2.2 Iterator [iterator.iterators]

1 The `Iterator` requirements form the basis of the iterator concept taxonomy; every iterator satisfies the `Iterator` requirements. This set of requirements specifies operations for dereferencing and incrementing an iterator. Most algorithms will require additional operations to read (24.2.3) or write (24.2.4) values, or to provide a richer set of iterator movements (24.2.5, 24.2.6, 24.2.7).)

2 A type `X` satisfies the `Iterator` requirements if:

(2.1) — `X` satisfies the `CopyConstructible`, `CopyAssignable`, and `Destructible` requirements (17.6.3.1) and lvalues of type `X` are swappable (17.6.3.2), and

(2.2) — the expressions in Table 105 are valid and have the indicated semantics.

268) This definition applies to pointers, since pointers are iterators. The effect of dereferencing an iterator that has been invalidated is undefined.

Table 105 — Iterator requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
`*r`	unspecified		pre: `r` is dereferenceable.
`++r`	`X&`		

24.2.3 Input iterators [input.iterators]

1 A class or pointer type `X` satisfies the requirements of an input iterator for the value type `T` if X satisfies the `Iterator` (24.2.2) and `EqualityComparable` (Table 17) requirements and the expressions in Table 106 are valid and have the indicated semantics.

2 In Table 106, the term *the domain of* `==` is used in the ordinary mathematical sense to denote the set of values over which `==` is (required to be) defined. This set can change over time. Each algorithm places additional requirements on the domain of `==` for the iterator values it uses. These requirements can be inferred from the uses that algorithm makes of `==` and `!=`. [*Example:* the call `find(a,b,x)` is defined only if the value of `a` has the property p defined as follows: `b` has property p and a value `i` has property p if `(*i==x)` or if `(*i!=x` and `++i` has property `p`). — *end example*]

Table 106 — Input iterator requirements (in addition to Iterator)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
`a != b`	contextually convertible to `bool`	`!(a == b)`	pre: `(a,b)` is in the domain of `==`.
`*a`	`reference`, convertible to `T`		pre: `a` is dereferenceable. The expression `(void)*a, *a` is equivalent to `*a`. If `a == b` and `(a,b)` is in the domain of `==` then `*a` is equivalent to `*b`.
`a->m`		`(*a).m`	pre: `a` is dereferenceable.
`++r`	`X&`		pre: `r` is dereferenceable. post: `r` is dereferenceable or `r` is past-the-end. post: any copies of the previous value of `r` are no longer required either to be dereferenceable or to be in the domain of `==`.
`(void)r++`			equivalent to `(void)++r`

`*r++`	convertible to `T`	`{ T tmp = *r;`
		`++r;`
		`return tmp; }`	

3 [*Note:* For input iterators, `a == b` does not imply `++a == ++b`. (Equality does not guarantee the substitution property or referential transparency.) Algorithms on input iterators should never attempt to pass through the same iterator twice. They should be *single pass* algorithms. Value type T is not required to be

a `CopyAssignable` type (Table 23). These algorithms can be used with istreams as the source of the input data through the `istream_iterator` class template. — *end note*]

24.2.4 Output iterators [output.iterators]

1 A class or pointer type `X` satisfies the requirements of an output iterator if `X` satisfies the `Iterator` requirements (24.2.2) and the expressions in Table 107 are valid and have the indicated semantics.

Table 107 — Output iterator requirements (in addition to Iterator)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
`*r = o`	result is not used		*Remark:* After this operation `r` is not required to be dereferenceable. post: `r` is incrementable.
`++r`	`X&`		`&r == &++r`. *Remark:* After this operation `r` is not required to be dereferenceable. post: `r` is incrementable.
`r++`	convertible to `const X&`	`{ X tmp = r; ++r; return tmp; }`	*Remark:* After this operation `r` is not required to be dereferenceable. post: `r` is incrementable.
`*r++ = o`	result is not used		*Remark:* After this operation `r` is not required to be dereferenceable. post: `r` is incrementable.

2 [*Note:* The only valid use of an `operator*` is on the left side of the assignment statement. *Assignment through the same value of the iterator happens only once.* Algorithms on output iterators should never attempt to pass through the same iterator twice. They should be *single pass* algorithms. Equality and inequality might not be defined. Algorithms that take output iterators can be used with ostreams as the destination for placing data through the `ostream_iterator` class as well as with insert iterators and insert pointers. — *end note*]

24.2.5 Forward iterators [forward.iterators]

1 A class or pointer type `X` satisfies the requirements of a forward iterator if

(1.1) — `X` satisfies the requirements of an input iterator (24.2.3),

(1.2) — X satisfies the `DefaultConstructible` requirements (17.6.3.1),

(1.3) — if `X` is a mutable iterator, `reference` is a reference to `T`; if `X` is a const iterator, `reference` is a reference to `const T`,

(1.4) — the expressions in Table 108 are valid and have the indicated semantics, and

(1.5) — objects of type `X` offer the multi-pass guarantee, described below.

2 The domain of == for forward iterators is that of iterators over the same underlying sequence. However, value-initialized iterators may be compared and shall compare equal to other value-initialized iterators of

the same type. [*Note:* value initialized iterators behave as if they refer past the end of the same empty sequence — *end note*]

3 Two dereferenceable iterators `a` and `b` of type `X` offer the *multi-pass guarantee* if:

(3.1) — `a == b` implies `++a == ++b` and

(3.2) — `X` is a pointer type or the expression `(void)++X(a), *a` is equivalent to the expression `*a`.

4 [*Note:* The requirement that `a == b` implies `++a == ++b` (which is not true for input and output iterators) and the removal of the restrictions on the number of the assignments through a mutable iterator (which applies to output iterators) allows the use of multi-pass one-directional algorithms with forward iterators. — *end note*]

Table 108 — Forward iterator requirements (in addition to input iterator)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition

`r++`	convertible to `const X&`	`{ X tmp = r;`
		`++r;`
		`return tmp; }`	

`*r++`	`reference`		

5 If `a` and `b` are equal, then either `a` and `b` are both dereferenceable or else neither is dereferenceable.

6 If `a` and `b` are both dereferenceable, then `a == b` if and only if `*a` and `*b` are bound to the same object.

24.2.6 Bidirectional iterators [bidirectional.iterators]

1 A class or pointer type `X` satisfies the requirements of a bidirectional iterator if, in addition to satisfying the requirements for forward iterators, the following expressions are valid as shown in Table 109.

Table 109 — Bidirectional iterator requirements (in addition to forward iterator)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
`--r`	`X&`		pre: there exists `s` such that `r == ++s`. post: `r` is dereferenceable. `--(++r) == r`. `--r == --s` implies `r == s`. `&r == &--r`.

`r--`	convertible to `const X&`	`{ X tmp = r;`
		`--r;`
		`return tmp; }`	

`*r--`	`reference`		

2 [*Note:* Bidirectional iterators allow algorithms to move iterators backward as well as forward. — *end note*]

24.2.7 Random access iterators [random.access.iterators]

1 A class or pointer type `X` satisfies the requirements of a random access iterator if, in addition to satisfying the requirements for bidirectional iterators, the following expressions are valid as shown in Table 110.

Table 110 — Random access iterator requirements (in addition to bidirectional iterator)

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition

`r += n`	`X&`	`{ difference_type m = n;`
		`if (m >= 0)`
		`while (m--)`
		`++r;`
		`else`
		`while (m++)`
		`--r;`
		`return r; }`	

`a + n` `n + a`	`X`	`{ X tmp = a;` `return tmp += n; }`	`a + n == n + a.`
`r -= n`	`X&`	`return r += -n;`	
`a - n`	`X`	`{ X tmp = a;` `return tmp -= n; }`	
`b - a`	`difference_type`	`return n`	pre: there exists a value `n` of type `difference_type` such that `a + n == b`. `b == a + (b - a).`
`a[n]`	convertible to `reference`	`*(a + n)`	
`a < b`	contextually convertible to `bool`	`b - a > 0`	`<` is a total ordering relation
`a > b`	contextually convertible to `bool`	`b < a`	`>` is a total ordering relation opposite to `<`.
`a >= b`	contextually convertible to `bool`	`!(a < b)`	
`a <= b`	contextually convertible to `bool`.	`!(a > b)`	

24.3 Header `<iterator>` synopsis [iterator.synopsis]

```
namespace std {
  // 24.4, primitives:
  template<class Iterator> struct iterator_traits;
  template<class T> struct iterator_traits<T*>;

  template<class Category, class T, class Distance = ptrdiff_t,
       class Pointer = T*, class Reference = T&> struct iterator;
```

```
struct input_iterator_tag { };
struct output_iterator_tag { };
struct forward_iterator_tag: public input_iterator_tag { };
struct bidirectional_iterator_tag: public forward_iterator_tag { };
struct random_access_iterator_tag: public bidirectional_iterator_tag { };

// 24.4.4, iterator operations:
template <class InputIterator, class Distance>
  void advance(InputIterator& i, Distance n);
template <class InputIterator>
  typename iterator_traits<InputIterator>::difference_type
  distance(InputIterator first, InputIterator last);
template <class ForwardIterator>
  ForwardIterator next(ForwardIterator x,
    typename std::iterator_traits<ForwardIterator>::difference_type n = 1);
template <class BidirectionalIterator>
  BidirectionalIterator prev(BidirectionalIterator x,
    typename std::iterator_traits<BidirectionalIterator>::difference_type n = 1);

// 24.5, predefined iterators:
template <class Iterator> class reverse_iterator;

template <class Iterator1, class Iterator2>
  bool operator==(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator<(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator!=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator>(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator<=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);

template <class Iterator1, class Iterator2>
  auto operator-(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y) ->decltype(y.base() - x.base());
template <class Iterator>
  reverse_iterator<Iterator>
    operator+(
  typename reverse_iterator<Iterator>::difference_type n,
```

```
  const reverse_iterator<Iterator>& x);

template <class Iterator>
  reverse_iterator<Iterator> make_reverse_iterator(Iterator i);

template <class Container> class back_insert_iterator;
template <class Container>
  back_insert_iterator<Container> back_inserter(Container& x);

template <class Container> class front_insert_iterator;
template <class Container>
  front_insert_iterator<Container> front_inserter(Container& x);

template <class Container> class insert_iterator;
template <class Container>
  insert_iterator<Container> inserter(Container& x, typename Container::iterator i);

template <class Iterator> class move_iterator;
template <class Iterator1, class Iterator2>
  bool operator==(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator!=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator<(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator<=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator>(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
template <class Iterator1, class Iterator2>
  bool operator>=(
    const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);

template <class Iterator1, class Iterator2>
  auto operator-(
  const move_iterator<Iterator1>& x,
  const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
template <class Iterator>
  move_iterator<Iterator> operator+(
    typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
template <class Iterator>
  move_iterator<Iterator> make_move_iterator(Iterator i);

// 24.6, stream iterators:
template <class T, class charT = char, class traits = char_traits<charT>,
    class Distance = ptrdiff_t>
class istream_iterator;
template <class T, class charT, class traits, class Distance>
  bool operator==(const istream_iterator<T,charT,traits,Distance>& x,
          const istream_iterator<T,charT,traits,Distance>& y);
template <class T, class charT, class traits, class Distance>
```

```
  bool operator!=(const istream_iterator<T,charT,traits,Distance>& x,
          const istream_iterator<T,charT,traits,Distance>& y);

template <class T, class charT = char, class traits = char_traits<charT> >
    class ostream_iterator;

template<class charT, class traits = char_traits<charT> >
  class istreambuf_iterator;
template <class charT, class traits>
  bool operator==(const istreambuf_iterator<charT,traits>& a,
          const istreambuf_iterator<charT,traits>& b);
template <class charT, class traits>
  bool operator!=(const istreambuf_iterator<charT,traits>& a,
          const istreambuf_iterator<charT,traits>& b);

template <class charT, class traits = char_traits<charT> >
  class ostreambuf_iterator;

// 24.7, range access:
template <class C> auto begin(C& c) -> decltype(c.begin());
template <class C> auto begin(const C& c) -> decltype(c.begin());
template <class C> auto end(C& c) -> decltype(c.end());
template <class C> auto end(const C& c) -> decltype(c.end());
template <class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
template <class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
template <class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
  -> decltype(std::begin(c));
template <class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
  -> decltype(std::end(c));
template <class C> auto rbegin(C& c) -> decltype(c.rbegin());
template <class C> auto rbegin(const C& c) -> decltype(c.rbegin());
template <class C> auto rend(C& c) -> decltype(c.rend());
template <class C> auto rend(const C& c) -> decltype(c.rend());
template <class T, size_t N> reverse_iterator<T*> rbegin(T (&array)[N]);
template <class T, size_t N> reverse_iterator<T*> rend(T (&array)[N]);
template <class E> reverse_iterator<const E*> rbegin(initializer_list<E> il);
template <class E> reverse_iterator<const E*> rend(initializer_list<E> il);
template <class C> auto crbegin(const C& c) -> decltype(std::rbegin(c));
template <class C> auto crend(const C& c) -> decltype(std::rend(c));

// 24.8, container access:
template <class C> constexpr auto size(const C& c) -> decltype(c.size());
template <class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept;
template <class C> constexpr auto empty(const C& c) -> decltype(c.empty());
template <class T, size_t N> constexpr bool empty(const T (&array)[N]) noexcept;
template <class E> constexpr bool empty(initializer_list<E> il) noexcept;
template <class C> constexpr auto data(C& c) -> decltype(c.data());
template <class C> constexpr auto data(const C& c) -> decltype(c.data());
template <class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
template <class E> constexpr const E* data(initializer_list<E> il) noexcept;
}
```

24.4 Iterator primitives [iterator.primitives]

1 To simplify the task of defining iterators, the library provides several classes and functions:

24.4.1 Iterator traits [iterator.traits]

1 To implement algorithms only in terms of iterators, it is often necessary to determine the value and difference types that correspond to a particular iterator type. Accordingly, it is required that if `Iterator` is the type of an iterator, the types

```
iterator_traits<Iterator>::difference_type
iterator_traits<Iterator>::value_type
iterator_traits<Iterator>::iterator_category
```

be defined as the iterator's difference type, value type and iterator category, respectively. In addition, the types

```
iterator_traits<Iterator>::reference
iterator_traits<Iterator>::pointer
```

shall be defined as the iterator's reference and pointer types, that is, for an iterator object `a`, the same type as the type of `*a` and `a->`, respectively. In the case of an output iterator, the types

```
iterator_traits<Iterator>::difference_type
iterator_traits<Iterator>::value_type
iterator_traits<Iterator>::reference
iterator_traits<Iterator>::pointer
```

may be defined as `void`.

2 If `Iterator` has valid (14.8.2) member types `difference_type`, `value_type`, `pointer`, `reference`, and `iterator_category`, `iterator_traits<Iterator>` shall have the following as publicly accessible members and shall have no other members:

```
typedef typename Iterator::difference_type difference_type;
typedef typename Iterator::value_type value_type;
typedef typename Iterator::pointer pointer;
typedef typename Iterator::reference reference;
typedef typename Iterator::iterator_category iterator_category;
```

Otherwise, `iterator_traits<Iterator>` shall have no members.

3 It is specialized for pointers as

```
namespace std {
  template<class T> struct iterator_traits<T*> {
    typedef ptrdiff_t difference_type;
    typedef T value_type;
    typedef T* pointer;
    typedef T& reference;
    typedef random_access_iterator_tag iterator_category;
  };
}
```

and for pointers to const as

```
namespace std {
  template<class T> struct iterator_traits<const T*> {
    typedef ptrdiff_t difference_type;
    typedef T value_type;
    typedef const T* pointer;
    typedef const T& reference;
    typedef random_access_iterator_tag iterator_category;
```

```
  };
}
```

4 [*Example:* To implement a generic `reverse` function, a C++ program can do the following:

```
template <class BidirectionalIterator>
void reverse(BidirectionalIterator first, BidirectionalIterator last) {
  typename iterator_traits<BidirectionalIterator>::difference_type n =
    distance(first, last);
  --n;
  while(n > 0) {
    typename iterator_traits<BidirectionalIterator>::value_type
     tmp = *first;
    *first++ = *--last;
    *last = tmp;
    n -= 2;
  }
}
```

— *end example*]

24.4.2 Basic iterator [iterator.basic]

1 The `iterator` template may be used as a base class to ease the definition of required types for new iterators.

```
namespace std {
  template<class Category, class T, class Distance = ptrdiff_t,
    class Pointer = T*, class Reference = T&>
  struct iterator {
    typedef T         value_type;
    typedef Distance  difference_type;
    typedef Pointer   pointer;
    typedef Reference reference;
    typedef Category  iterator_category;
  };
}
```

24.4.3 Standard iterator tags [std.iterator.tags]

1 It is often desirable for a function template specialization to find out what is the most specific category of its iterator argument, so that the function can select the most efficient algorithm at compile time. To facilitate this, the library introduces *category tag* classes which are used as compile time tags for algorithm selection. They are: `input_iterator_tag`, `output_iterator_tag`, `forward_iterator_tag`, `bidirectional_iterator_tag` and `random_access_iterator_tag`. For every iterator of type `Iterator`, `iterator_traits<Iterator>::iterator_category` shall be defined to be the most specific category tag that describes the iterator's behavior.

```
namespace std {
  struct input_iterator_tag { };
  struct output_iterator_tag { };
  struct forward_iterator_tag: public input_iterator_tag { };
  struct bidirectional_iterator_tag: public forward_iterator_tag { };
  struct random_access_iterator_tag: public bidirectional_iterator_tag { };
}
```

2 [*Example:* For a program-defined iterator `BinaryTreeIterator`, it could be included into the bidirectional iterator category by specializing the `iterator_traits` template:

```
template<class T> struct iterator_traits<BinaryTreeIterator<T> > {
  typedef std::ptrdiff_t difference_type;
  typedef T value_type;
  typedef T* pointer;
  typedef T& reference;
  typedef bidirectional_iterator_tag iterator_category;
};
```

Typically, however, it would be easier to derive `BinaryTreeIterator<T>` from `iterator<bidirectional_iterator_tag,T,ptrdiff_t,T*,T&>`. — *end example*]

3 [*Example:* If `evolve()` is well defined for bidirectional iterators, but can be implemented more efficiently for random access iterators, then the implementation is as follows:

```
template <class BidirectionalIterator>
inline void
evolve(BidirectionalIterator first, BidirectionalIterator last) {
  evolve(first, last,
    typename iterator_traits<BidirectionalIterator>::iterator_category());
}

template <class BidirectionalIterator>
void evolve(BidirectionalIterator first, BidirectionalIterator last,
  bidirectional_iterator_tag) {
  // more generic, but less efficient algorithm
}

template <class RandomAccessIterator>
void evolve(RandomAccessIterator first, RandomAccessIterator last,
  random_access_iterator_tag) {
  // more efficient, but less generic algorithm
}
```

— *end example*]

4 [*Example:* If a C++ program wants to define a bidirectional iterator for some data structure containing `double` and such that it works on a large memory model of the implementation, it can do so with:

```
class MyIterator :
  public iterator<bidirectional_iterator_tag, double, long, T*, T&> {
  // code implementing ++, etc.
};
```

5 Then there is no need to specialize the `iterator_traits` template. — *end example*]

24.4.4 Iterator operations [iterator.operations]

1 Since only random access iterators provide `+` and `-` operators, the library provides two function templates `advance` and `distance`. These function templates use `+` and `-` for random access iterators (and are, therefore, constant time for them); for input, forward and bidirectional iterators they use `++` to provide linear time implementations.

```
template <class InputIterator, class Distance>
  void advance(InputIterator& i, Distance n);
```

2 *Requires:* `n` shall be negative only for bidirectional and random access iterators.

3 *Effects:* Increments (or decrements for negative `n`) iterator reference `i` by `n`.

```
template <class InputIterator>
  typename iterator_traits<InputIterator>::difference_type
    distance(InputIterator first, InputIterator last);
```

4 *Effects:* If `InputIterator` meets the requirements of random access iterator, returns `(last - first)`; otherwise, returns the number of increments needed to get from `first` to `last`.

5 *Requires:* If `InputIterator` meets the requirements of random access iterator, `last` shall be reachable from `first` or `first` shall be reachable from `last`; otherwise, `last` shall be reachable from `first`.

```
template <class ForwardIterator>
  ForwardIterator next(ForwardIterator x,
    typename std::iterator_traits<ForwardIterator>::difference_type n = 1);
```

6 *Effects:* Equivalent to `advance(x, n); return x;`

```
template <class BidirectionalIterator>
  BidirectionalIterator prev(BidirectionalIterator x,
    typename std::iterator_traits<BidirectionalIterator>::difference_type n = 1);
```

7 *Effects:* Equivalent to `advance(x, -n); return x;`

24.5 Iterator adaptors [predef.iterators]

24.5.1 Reverse iterators [reverse.iterators]

1 Class template `reverse_iterator` is an iterator adaptor that iterates from the end of the sequence defined by its underlying iterator to the beginning of that sequence. The fundamental relation between a reverse iterator and its corresponding iterator `i` is established by the identity: `&*(reverse_iterator(i)) == &*(i - 1)`.

24.5.1.1 Class template `reverse_iterator` [reverse.iterator]

```
namespace std {
  template <class Iterator>
  class reverse_iterator {
  public:
    typedef Iterator                                                iterator_type;
    typedef typename iterator_traits<Iterator>::iterator_category iterator_category;
    typedef typename iterator_traits<Iterator>::value_type        value_type;
    typedef typename iterator_traits<Iterator>::difference_type   difference_type;
    typedef typename iterator_traits<Iterator>::pointer           pointer;
    typedef typename iterator_traits<Iterator>::reference         reference;

    reverse_iterator();
    explicit reverse_iterator(Iterator x);
    template <class U> reverse_iterator(const reverse_iterator<U>& u);
    template <class U> reverse_iterator& operator=(const reverse_iterator<U>& u);

    Iterator base() const;      // explicit
    reference operator*() const;
    pointer   operator->() const;

    reverse_iterator& operator++();
    reverse_iterator  operator++(int);
    reverse_iterator& operator--();
    reverse_iterator  operator--(int);

    reverse_iterator  operator+ (difference_type n) const;
```

```
    reverse_iterator& operator+=(difference_type n);
    reverse_iterator  operator- (difference_type n) const;
    reverse_iterator& operator-=(difference_type n);
    unspecified  operator[](difference_type n) const;
  protected:
    Iterator current;
  };

  template <class Iterator1, class Iterator2>
    bool operator==(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator<(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator!=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator>(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator>=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator<=(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    auto operator-(
      const reverse_iterator<Iterator1>& x,
      const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
  template <class Iterator>
    reverse_iterator<Iterator> operator+(
      typename reverse_iterator<Iterator>::difference_type n,
      const reverse_iterator<Iterator>& x);

  template <class Iterator>
    reverse_iterator<Iterator> make_reverse_iterator(Iterator i);
}
```

24.5.1.2 `reverse_iterator` requirements [reverse.iter.requirements]

1 The template parameter `Iterator` shall meet all the requirements of a Bidirectional Iterator (24.2.6).

2 Additionally, `Iterator` shall meet the requirements of a Random Access Iterator (24.2.7) if any of the members `operator+` (24.5.1.3.8), `operator-` (24.5.1.3.10), `operator+=` (24.5.1.3.9), `operator-=` (24.5.1.3.11), `operator []` (24.5.1.3.12), or the global operators `operator<` (24.5.1.3.14), `operator>` (24.5.1.3.16), `operator <=` (24.5.1.3.18), `operator>=` (24.5.1.3.17), `operator-` (24.5.1.3.19) or `operator+` (24.5.1.3.20) are referenced in a way that requires instantiation (14.7.1).

24.5.1.3 `reverse_iterator` operations [reverse.iter.ops]

24.5.1.3.1 `reverse_iterator` constructor [reverse.iter.cons]

```
reverse_iterator();
```

1 *Effects:* Value initializes `current`. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type `Iterator`.

```
explicit reverse_iterator(Iterator x);
```

2 *Effects:* Initializes `current` with `x`.

```
template <class U> reverse_iterator(const reverse_iterator<U> &u);
```

3 *Effects:* Initializes `current` with `u.current`.

24.5.1.3.2 `reverse_iterator::operator=` [reverse.iter.op=]

```
template <class U>
reverse_iterator&
  operator=(const reverse_iterator<U>& u);
```

1 *Effects:* Assigns `u.base()` to current.

2 *Returns:* `*this`.

24.5.1.3.3 Conversion [reverse.iter.conv]

```
Iterator base() const;          // explicit
```

1 *Returns:* `current`.

24.5.1.3.4 `operator*` [reverse.iter.op.star]

```
reference operator*() const;
```

1 *Effects:*

```
Iterator tmp = current;
return *--tmp;
```

24.5.1.3.5 `operator->` [reverse.iter.opref]

```
pointer operator->() const;
```

1 *Returns:* `std::addressof(operator*())`.

24.5.1.3.6 `operator++` [reverse.iter.op++]

```
reverse_iterator& operator++();
```

1 *Effects:* `--current;`

2 *Returns:* `*this`.

```
reverse_iterator operator++(int);
```

3 *Effects:*

```
reverse_iterator tmp = *this;
--current;
return tmp;
```

24.5.1.3.7 operator-- [reverse.iter.op--]

```
reverse_iterator& operator--();
```

1 *Effects:* ++current

2 *Returns:* *this.

```
reverse_iterator operator--(int);
```

3 *Effects:*

```
reverse_iterator tmp = *this;
++current;
return tmp;
```

24.5.1.3.8 operator+ [reverse.iter.op+]

```
reverse_iterator
operator+(typename reverse_iterator<Iterator>::difference_type n) const;
```

1 *Returns:* reverse_iterator(current-n).

24.5.1.3.9 operator+= [reverse.iter.op+=]

```
reverse_iterator&
operator+=(typename reverse_iterator<Iterator>::difference_type n);
```

1 *Effects:* current -= n;

2 *Returns:* *this.

24.5.1.3.10 operator- [reverse.iter.op-]

```
reverse_iterator
operator-(typename reverse_iterator<Iterator>::difference_type n) const;
```

1 *Returns:* reverse_iterator(current+n).

24.5.1.3.11 operator-= [reverse.iter.op-=]

```
reverse_iterator&
operator-=(typename reverse_iterator<Iterator>::difference_type n);
```

1 *Effects:* current += n;

2 *Returns:* *this.

24.5.1.3.12 operator[] [reverse.iter.opindex]

```
unspecified  operator[](
    typename reverse_iterator<Iterator>::difference_type n) const;
```

1 *Returns:* current[-n-1].

24.5.1.3.13 operator== [reverse.iter.op==]

```
template <class Iterator1, class Iterator2>
  bool operator==(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* x.current == y.current.

24.5.1.3.14 operator< [reverse.iter.op<]

```
template <class Iterator1, class Iterator2>
  bool operator<(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* `x.current > y.current`.

24.5.1.3.15 operator!= [reverse.iter.op!=]

```
template <class Iterator1, class Iterator2>
  bool operator!=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* `x.current != y.current`.

24.5.1.3.16 operator> [reverse.iter.op>]

```
template <class Iterator1, class Iterator2>
  bool operator>(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* `x.current < y.current`.

24.5.1.3.17 operator>= [reverse.iter.op>=]

```
template <class Iterator1, class Iterator2>
  bool operator>=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* `x.current <= y.current`.

24.5.1.3.18 operator<= [reverse.iter.op<=]

```
template <class Iterator1, class Iterator2>
  bool operator<=(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y);
```

1 *Returns:* `x.current >= y.current`.

24.5.1.3.19 operator- [reverse.iter.opdiff]

```
template <class Iterator1, class Iterator2>
    auto operator-(
    const reverse_iterator<Iterator1>& x,
    const reverse_iterator<Iterator2>& y) -> decltype(y.base() - x.base());
```

1 *Returns:* `y.current - x.current`.

24.5.1.3.20 operator+ [reverse.iter.opsum]

```
template <class Iterator>
  reverse_iterator<Iterator> operator+(
    typename reverse_iterator<Iterator>::difference_type n,
    const reverse_iterator<Iterator>& x);
```

1 *Returns:* `reverse_iterator<Iterator> (x.current - n)`.

24.5.1.3.21 Non-member function `make_reverse_iterator()` [reverse.iter.make]

```
template <class Iterator>
  reverse_iterator<Iterator> make_reverse_iterator(Iterator i);
```

1 *Returns:* `reverse_iterator<Iterator>(i)`.

24.5.2 Insert iterators [insert.iterators]

1 To make it possible to deal with insertion in the same way as writing into an array, a special kind of iterator adaptors, called *insert iterators*, are provided in the library. With regular iterator classes,

```
while (first != last) *result++ = *first++;
```

causes a range `[first,last)` to be copied into a range starting with result. The same code with `result` being an insert iterator will insert corresponding elements into the container. This device allows all of the copying algorithms in the library to work in the *insert mode* instead of the *regular overwrite* mode.

2 An insert iterator is constructed from a container and possibly one of its iterators pointing to where insertion takes place if it is neither at the beginning nor at the end of the container. Insert iterators satisfy the requirements of output iterators. `operator*` returns the insert iterator itself. The assignment `operator=(const T& x)` is defined on insert iterators to allow writing into them, it inserts `x` right before where the insert iterator is pointing. In other words, an insert iterator is like a cursor pointing into the container where the insertion takes place. `back_insert_iterator` inserts elements at the end of a container, `front_insert_iterator` inserts elements at the beginning of a container, and `insert_iterator` inserts elements where the iterator points to in a container. `back_inserter`, `front_inserter`, and `inserter` are three functions making the insert iterators out of a container.

24.5.2.1 Class template `back_insert_iterator` [back.insert.iterator]

```
namespace std {
  template <class Container>
  class back_insert_iterator {
  protected:
    Container* container;

  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;
    typedef Container container_type;
    explicit back_insert_iterator(Container& x);
    back_insert_iterator& operator=(const typename Container::value_type& value);
    back_insert_iterator& operator=(typename Container::value_type&& value);

    back_insert_iterator& operator*();
    back_insert_iterator& operator++();
    back_insert_iterator  operator++(int);
  };

  template <class Container>
    back_insert_iterator<Container> back_inserter(Container& x);
}
```

24.5.2.2 `back_insert_iterator` operations [back.insert.iter.ops]

24.5.2.2.1 `back_insert_iterator` constructor [back.insert.iter.cons]

```
explicit back_insert_iterator(Container& x);
```

1 *Effects:* Initializes `container` with `std::addressof(x)`.

24.5.2.2.2 `back_insert_iterator::operator=` [back.insert.iter.op=]

```
back_insert_iterator& operator=(const typename Container::value_type& value);
```

1 *Effects:* `container->push_back(value);`

2 *Returns:* `*this`.

```
back_insert_iterator& operator=(typename Container::value_type&& value);
```

3 *Effects:* `container->push_back(std::move(value));`

4 *Returns:* `*this`.

24.5.2.2.3 `back_insert_iterator::operator*` [back.insert.iter.op*]

```
back_insert_iterator& operator*();
```

1 *Returns:* `*this`.

24.5.2.2.4 `back_insert_iterator::operator++` [back.insert.iter.op++]

```
back_insert_iterator& operator++();
back_insert_iterator  operator++(int);
```

1 *Returns:* `*this`.

24.5.2.2.5 `back_inserter` [back.inserter]

```
template <class Container>
  back_insert_iterator<Container> back_inserter(Container& x);
```

1 *Returns:* `back_insert_iterator<Container>(x)`.

24.5.2.3 Class template `front_insert_iterator` [front.insert.iterator]

```
namespace std {
  template <class Container>
  class front_insert_iterator {
  protected:
    Container* container;

  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;
    typedef Container container_type;
    explicit front_insert_iterator(Container& x);
    front_insert_iterator& operator=(const typename Container::value_type& value);
    front_insert_iterator& operator=(typename Container::value_type&& value);

    front_insert_iterator& operator*();
    front_insert_iterator& operator++();
    front_insert_iterator  operator++(int);
  };
```

```
  template <class Container>
    front_insert_iterator<Container> front_inserter(Container& x);
}
```

24.5.2.4 front_insert_iterator operations [front.insert.iter.ops]

24.5.2.4.1 front_insert_iterator constructor [front.insert.iter.cons]

```
explicit front_insert_iterator(Container& x);
```

1 *Effects:* Initializes container with std::addressof(x).

24.5.2.4.2 front_insert_iterator::operator= [front.insert.iter.op=]

```
front_insert_iterator& operator=(const typename Container::value_type& value);
```

1 *Effects:* container->push_front(value);

2 *Returns:* *this.

```
front_insert_iterator& operator=(typename Container::value_type&& value);
```

3 *Effects:* container->push_front(std::move(value));

4 *Returns:* *this.

24.5.2.4.3 front_insert_iterator::operator* [front.insert.iter.op*]

```
front_insert_iterator& operator*();
```

1 *Returns:* *this.

24.5.2.4.4 front_insert_iterator::operator++ [front.insert.iter.op++]

```
front_insert_iterator& operator++();
front_insert_iterator  operator++(int);
```

1 *Returns:* *this.

24.5.2.4.5 front_inserter [front.inserter]

```
template <class Container>
  front_insert_iterator<Container> front_inserter(Container& x);
```

1 *Returns:* front_insert_iterator<Container>(x).

24.5.2.5 Class template insert_iterator [insert.iterator]

```
namespace std {
  template <class Container>
  class insert_iterator {
  protected:
    Container* container;
    typename Container::iterator iter;

  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;
    typedef Container container_type;
    insert_iterator(Container& x, typename Container::iterator i);
```

```
    insert_iterator& operator=(const typename Container::value_type& value);
    insert_iterator& operator=(typename Container::value_type&& value);

    insert_iterator& operator*();
    insert_iterator& operator++();
    insert_iterator& operator++(int);
  };

  template <class Container>
    insert_iterator<Container> inserter(Container& x, typename Container::iterator i);
}
```

24.5.2.6 insert_iterator operations [insert.iter.ops]

24.5.2.6.1 insert_iterator constructor [insert.iter.cons]

```
insert_iterator(Container& x, typename Container::iterator i);
```

1 *Effects:* Initializes `container` with `std::addressof(x)` and `iter` with `i`.

24.5.2.6.2 insert_iterator::operator= [insert.iter.op=]

```
insert_iterator& operator=(const typename Container::value_type& value);
```

1 *Effects:*

```
iter = container->insert(iter, value);
++iter;
```

2 *Returns:* `*this`.

```
insert_iterator& operator=(typename Container::value_type&& value);
```

3 *Effects:*

```
iter = container->insert(iter, std::move(value));
++iter;
```

4 *Returns:* `*this`.

24.5.2.6.3 insert_iterator::operator* [insert.iter.op*]

```
insert_iterator& operator*();
```

1 *Returns:* `*this`.

24.5.2.6.4 insert_iterator::operator++ [insert.iter.op++]

```
insert_iterator& operator++();
insert_iterator& operator++(int);
```

1 *Returns:* `*this`.

24.5.2.6.5 inserter [inserter]

```
template <class Container>
  insert_iterator<Container> inserter(Container& x, typename Container::iterator i);
```

1 *Returns:* `insert_iterator<Container>(x, i)`.

24.5.3 Move iterators [move.iterators]

1 Class template `move_iterator` is an iterator adaptor with the same behavior as the underlying iterator except that its indirection operator implicitly converts the value returned by the underlying iterator's indirection operator to an rvalue. Some generic algorithms can be called with move iterators to replace copying with moving.

2 [*Example:*

```
list<string> s;
// populate the list s
vector<string> v1(s.begin(), s.end());          // copies strings into v1
vector<string> v2(make_move_iterator(s.begin()),
                  make_move_iterator(s.end())); // moves strings into v2
```

— *end example*]

24.5.3.1 Class template `move_iterator` [move.iterator]

```
namespace std {
  template <class Iterator>
  class move_iterator {
  public:
    typedef Iterator                                                iterator_type;
    typedef typename iterator_traits<Iterator>::difference_type   difference_type;
    typedef Iterator                                                pointer;
    typedef typename iterator_traits<Iterator>::value_type        value_type;
    typedef typename iterator_traits<Iterator>::iterator_category iterator_category;
    typedef see below                                               reference;

    move_iterator();
    explicit move_iterator(Iterator i);
    template <class U> move_iterator(const move_iterator<U>& u);
    template <class U> move_iterator& operator=(const move_iterator<U>& u);

    iterator_type base() const;
    reference operator*() const;
    pointer operator->() const;

    move_iterator& operator++();
    move_iterator operator++(int);
    move_iterator& operator--();
    move_iterator operator--(int);

    move_iterator operator+(difference_type n) const;
    move_iterator& operator+=(difference_type n);
    move_iterator operator-(difference_type n) const;
    move_iterator& operator-=(difference_type n);
    unspecified  operator[](difference_type n) const;

  private:
    Iterator current;    // exposition only
  };

  template <class Iterator1, class Iterator2>
    bool operator==(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

```
  template <class Iterator1, class Iterator2>
    bool operator!=(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator<(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator<=(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator>(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
  template <class Iterator1, class Iterator2>
    bool operator>=(
      const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);

  template <class Iterator1, class Iterator2>
    auto operator-(
      const move_iterator<Iterator1>& x,
      const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
  template <class Iterator>
    move_iterator<Iterator> operator+(
      typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
  template <class Iterator>
    move_iterator<Iterator> make_move_iterator(Iterator i);
}
```

1 Let *R* be `iterator_traits<Iterator>::reference`. If `is_reference<`*R*`>::value` is `true`, the template specialization `move_iterator<Iterator>` shall define the nested type named `reference` as a synonym for `remove_reference<`*R*`>::type&&`, otherwise as a synonym for *R*.

24.5.3.2 `move_iterator` requirements [move.iter.requirements]

1 The template parameter `Iterator` shall meet the requirements for an Input Iterator (24.2.3). Additionally, if any of the bidirectional or random access traversal functions are instantiated, the template parameter shall meet the requirements for a Bidirectional Iterator (24.2.6) or a Random Access Iterator (24.2.7), respectively.

24.5.3.3 `move_iterator` operations [move.iter.ops]

24.5.3.3.1 `move_iterator` constructors [move.iter.op.const]

```
move_iterator();
```

1 *Effects:* Constructs a `move_iterator`, value initializing `current`. Iterator operations applied to the resulting iterator have defined behavior if and only if the corresponding operations are defined on a value-initialized iterator of type `Iterator`.

```
explicit move_iterator(Iterator i);
```

2 *Effects:* Constructs a `move_iterator`, initializing `current` with `i`.

```
template <class U> move_iterator(const move_iterator<U>& u);
```

3 *Effects:* Constructs a `move_iterator`, initializing `current` with `u.base()`.

4 *Requires:* `U` shall be convertible to `Iterator`.

24.5.3.3.2 `move_iterator::operator=` [move.iter.op=]

```
template <class U> move_iterator& operator=(const move_iterator<U>& u);
```

1 *Effects:* Assigns `u.base()` to `current`.

2 *Requires:* `U` shall be convertible to `Iterator`.

24.5.3.3.3 `move_iterator` conversion [move.iter.op.conv]

```
Iterator base() const;
```

1 *Returns:* `current`.

24.5.3.3.4 `move_iterator::operator*` [move.iter.op.star]

```
reference operator*() const;
```

1 *Returns:* `static_cast<reference>(*current)`.

24.5.3.3.5 `move_iterator::operator->` [move.iter.op.ref]

```
pointer operator->() const;
```

1 *Returns:* `current`.

24.5.3.3.6 `move_iterator::operator++` [move.iter.op.incr]

```
move_iterator& operator++();
```

1 *Effects:* `++current`.

2 *Returns:* `*this`.

```
move_iterator operator++(int);
```

3 *Effects:*

```
move_iterator tmp = *this;
++current;
return tmp;
```

24.5.3.3.7 `move_iterator::operator--` [move.iter.op.decr]

```
move_iterator& operator--();
```

1 *Effects:* `--current`.

2 *Returns:* `*this`.

```
move_iterator operator--(int);
```

3 *Effects:*

```
move_iterator tmp = *this;
--current;
return tmp;
```

24.5.3.3.8 `move_iterator::operator+` [move.iter.op.+]

```
move_iterator operator+(difference_type n) const;
```

1 *Returns:* `move_iterator(current + n)`.

24.5.3.3.9 `move_iterator::operator+=` [move.iter.op.+=]

```
move_iterator& operator+=(difference_type n);
```

1 *Effects:* `current += n`.

2 *Returns:* `*this`.

24.5.3.3.10 `move_iterator::operator-` [move.iter.op.-]

```
move_iterator operator-(difference_type n) const;
```

1 *Returns:* `move_iterator(current - n)`.

24.5.3.3.11 `move_iterator::operator-=` [move.iter.op.-=]

```
move_iterator& operator-=(difference_type n);
```

1 *Effects:* `current -= n`.

2 *Returns:* `*this`.

24.5.3.3.12 `move_iterator::operator[]` [move.iter.op.index]

```
unspecified  operator[](difference_type n) const;
```

1 *Returns:* `std::move(`current[n]`)`.

24.5.3.3.13 `move_iterator` comparisons [move.iter.op.comp]

```
template <class Iterator1, class Iterator2>
bool operator==(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

1 *Returns:* `x.base() == y.base()`.

```
template <class Iterator1, class Iterator2>
bool operator!=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

2 *Returns:* `!(x == y)`.

```
template <class Iterator1, class Iterator2>
bool operator<(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

3 *Returns:* `x.base() < y.base()`.

```
template <class Iterator1, class Iterator2>
bool operator<=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

4 *Returns:* `!(y < x)`.

```
template <class Iterator1, class Iterator2>
bool operator>(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

5 *Returns:* `y < x`.

```
template <class Iterator1, class Iterator2>
bool operator>=(const move_iterator<Iterator1>& x, const move_iterator<Iterator2>& y);
```

6 *Returns:* `!(x < y)`.

24.5.3.3.14 `move_iterator` non-member functions [move.iter.nonmember]

```
template <class Iterator1, class Iterator2>
    auto operator-(
    const move_iterator<Iterator1>& x,
    const move_iterator<Iterator2>& y) -> decltype(x.base() - y.base());
```

1 *Returns:* `x.base() - y.base()`.

```
template <class Iterator>
  move_iterator<Iterator> operator+(
    typename move_iterator<Iterator>::difference_type n, const move_iterator<Iterator>& x);
```

2 *Returns:* `x + n`.

```
template <class Iterator>
move_iterator<Iterator> make_move_iterator(Iterator i);
```

3 *Returns:* `move_iterator<Iterator>(i)`.

24.6 Stream iterators [stream.iterators]

1 To make it possible for algorithmic templates to work directly with input/output streams, appropriate iterator-like class templates are provided.

[*Example:*

```
partial_sum(istream_iterator<double, char>(cin),
  istream_iterator<double, char>(),
  ostream_iterator<double, char>(cout, "\n"));
```

reads a file containing floating point numbers from `cin`, and prints the partial sums onto `cout`. — *end example*]

24.6.1 Class template `istream_iterator` [istream.iterator]

1 The class template `istream_iterator` is an input iterator (24.2.3) that reads (using `operator>>`) successive elements from the input stream for which it was constructed. After it is constructed, and every time `++` is used, the iterator reads and stores a value of `T`. If the iterator fails to read and store a value of `T` (`fail()` on the stream returns `true`), the iterator becomes equal to the *end-of-stream* iterator value. The constructor with no arguments `istream_iterator()` always constructs an end-of-stream input iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-stream iterator is not defined. For any other iterator value a `const T&` is returned. The result of `operator->` on an end-of-stream iterator is not defined. For any other iterator value a `const T*` is returned. The behavior of a program that applies `operator++()` to an end-of-stream iterator is undefined. It is impossible to store things into istream iterators.

2 Two end-of-stream iterators are always equal. An end-of-stream iterator is not equal to a non-end-of-stream iterator. Two non-end-of-stream iterators are equal when they are constructed from the same stream.

```
namespace std {
  template <class T, class charT = char, class traits = char_traits<charT>,
      class Distance = ptrdiff_t>
  class istream_iterator {
  public:
    typedef input_iterator_tag iterator_category;
    typedef T value_type;
    typedef Distance difference_type;
    typedef const T* pointer;
```

```
    typedef const T& reference;
    typedef charT char_type;
    typedef traits traits_type;
    typedef basic_istream<charT,traits> istream_type;
    see below istream_iterator();
    istream_iterator(istream_type& s);
    istream_iterator(const istream_iterator& x) = default;
   ~istream_iterator() = default;

    const T& operator*() const;
    const T* operator->() const;
    istream_iterator& operator++();
    istream_iterator  operator++(int);
  private:
    basic_istream<charT,traits>* in_stream; // exposition only
    T value;                                // exposition only
  };

  template <class T, class charT, class traits, class Distance>
    bool operator==(const istream_iterator<T,charT,traits,Distance>& x,
            const istream_iterator<T,charT,traits,Distance>& y);
  template <class T, class charT, class traits, class Distance>
    bool operator!=(const istream_iterator<T,charT,traits,Distance>& x,
            const istream_iterator<T,charT,traits,Distance>& y);
}
```

24.6.1.1 `istream_iterator` constructors and destructor [istream.iterator.cons]

`see below istream_iterator();`

1 *Effects:* Constructs the end-of-stream iterator. If `T` is a literal type, then this constructor shall be a `constexpr` constructor.

2 *Postcondition:* `in_stream == 0`.

`istream_iterator(istream_type& s);`

3 *Effects:* Initializes *in_stream* with `&s`. *value* may be initialized during construction or the first time it is referenced.

4 *Postcondition:* `in_stream == &s`.

`istream_iterator(const istream_iterator& x) = default;`

5 *Effects:* Constructs a copy of `x`. If `T` is a literal type, then this constructor shall be a trivial copy constructor.

6 *Postcondition:* `in_stream == x.in_stream`.

`~istream_iterator() = default;`

7 *Effects:* The iterator is destroyed. If `T` is a literal type, then this destructor shall be a trivial destructor.

24.6.1.2 `istream_iterator` operations [istream.iterator.ops]

`const T& operator*() const;`

1 *Returns: value.*

`const T* operator->() const;`

2 *Returns:* `&(operator*())`.

```
istream_iterator& operator++();
```

3 *Requires:* `in_stream != 0`.

4 *Effects:* `*in_stream >>value`.

5 *Returns:* `*this`.

```
istream_iterator operator++(int);
```

6 *Requires:* `in_stream != 0`.

7 *Effects:*

```
istream_iterator tmp = *this;
*in_stream >> value;
return (tmp);
```

```
template <class T, class charT, class traits, class Distance>
  bool operator==(const istream_iterator<T,charT,traits,Distance> &x,
                  const istream_iterator<T,charT,traits,Distance> &y);
```

8 *Returns:* `x.in_stream == y.in_stream`.

```
template <class T, class charT, class traits, class Distance>
  bool operator!=(const istream_iterator<T,charT,traits,Distance> &x,
                  const istream_iterator<T,charT,traits,Distance> &y);
```

9 *Returns:* `!(x == y)`

24.6.2 Class template `ostream_iterator` [ostream.iterator]

1 `ostream_iterator` writes (using `operator<<`) successive elements onto the output stream from which it was constructed. If it was constructed with `charT*` as a constructor argument, this string, called a *delimiter string*, is written to the stream after every `T` is written. It is not possible to get a value out of the output iterator. Its only use is as an output iterator in situations like

```
while (first != last)
  *result++ = *first++;
```

2 `ostream_iterator` is defined as:

```
namespace std {
  template <class T, class charT = char, class traits = char_traits<charT> >
  class ostream_iterator {
  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;
    typedef charT char_type;
    typedef traits traits_type;
    typedef basic_ostream<charT,traits> ostream_type;
    ostream_iterator(ostream_type& s);
    ostream_iterator(ostream_type& s, const charT* delimiter);
    ostream_iterator(const ostream_iterator& x);
   ~ostream_iterator();
```

```
    ostream_iterator& operator=(const T& value);

    ostream_iterator& operator*();
    ostream_iterator& operator++();
    ostream_iterator& operator++(int);
  private:
    basic_ostream<charT,traits>* out_stream;  // exposition only
    const charT* delim;                       // exposition only
  };
}
```

24.6.2.1 `ostream_iterator` constructors and destructor [ostream.iterator.cons.des]

```
ostream_iterator(ostream_type& s);
```

1 *Effects:* Initializes *out_stream* with `&s` and *delim* with null.

```
ostream_iterator(ostream_type& s, const charT* delimiter);
```

2 *Effects:* Initializes *out_stream* with `&s` and *delim* with `delimiter`.

```
ostream_iterator(const ostream_iterator& x);
```

3 *Effects:* Constructs a copy of `x`.

```
~ostream_iterator();
```

4 *Effects:* The iterator is destroyed.

24.6.2.2 `ostream_iterator` operations [ostream.iterator.ops]

```
ostream_iterator& operator=(const T& value);
```

1 *Effects:*

```
*out_stream << value;
if (delim != 0)
  *out_stream << delim;
return *this;
```

```
ostream_iterator& operator*();
```

2 *Returns:* `*this`.

```
ostream_iterator& operator++();
ostream_iterator& operator++(int);
```

3 *Returns:* `*this`.

24.6.3 Class template `istreambuf_iterator` [istreambuf.iterator]

1 The class template `istreambuf_iterator` defines an input iterator (24.2.3) that reads successive *characters* from the streambuf for which it was constructed. `operator*` provides access to the current input character, if any. [*Note:* `operator->` may return a proxy. — *end note*] Each time `operator++` is evaluated, the iterator advances to the next input character. If the end of stream is reached (`streambuf_type::sgetc()` returns `traits::eof()`), the iterator becomes equal to the *end-of-stream* iterator value. The default constructor `istreambuf_iterator()` and the constructor `istreambuf_iterator(0)` both construct an end-of-stream iterator object suitable for use as an end-of-range. All specializations of `istreambuf_iterator` shall have a trivial copy constructor, a `constexpr` default constructor, and a trivial destructor.

[2] The result of `operator*()` on an end-of-stream iterator is undefined. For any other iterator value a `char_type` value is returned. It is impossible to assign a character via an input iterator.

```
namespace std {
  template<class charT, class traits = char_traits<charT> >
  class istreambuf_iterator {
  public:
    typedef input_iterator_tag            iterator_category;
    typedef charT                         value_type;
    typedef typename traits::off_type     difference_type;
    typedef unspecified                   pointer;
    typedef charT                         reference;
    typedef charT                         char_type;
    typedef traits                        traits_type;
    typedef typename traits::int_type     int_type;
    typedef basic_streambuf<charT,traits> streambuf_type;
    typedef basic_istream<charT,traits>   istream_type;

    class proxy;                          // exposition only

    constexpr istreambuf_iterator() noexcept;
    istreambuf_iterator(const istreambuf_iterator&) noexcept = default;
    ~istreambuf_iterator() = default;
    istreambuf_iterator(istream_type& s) noexcept;
    istreambuf_iterator(streambuf_type* s) noexcept;
    istreambuf_iterator(const proxy& p) noexcept;
    charT operator*() const;
    pointer operator->() const;
    istreambuf_iterator& operator++();
    proxy operator++(int);
    bool equal(const istreambuf_iterator& b) const;
  private:
    streambuf_type* sbuf_;                // exposition only
  };

  template <class charT, class traits>
    bool operator==(const istreambuf_iterator<charT,traits>& a,
            const istreambuf_iterator<charT,traits>& b);
  template <class charT, class traits>
    bool operator!=(const istreambuf_iterator<charT,traits>& a,
            const istreambuf_iterator<charT,traits>& b);
}
```

24.6.3.1 Class template `istreambuf_iterator::proxy` [istreambuf.iterator::proxy]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class istreambuf_iterator<charT, traits>::proxy { // exposition only
    charT keep_;
    basic_streambuf<charT,traits>* sbuf_;
    proxy(charT c, basic_streambuf<charT,traits>* sbuf)
      : keep_(c), sbuf_(sbuf) { }
  public:
    charT operator*() { return keep_; }
  };
}
```

1 Class `istreambuf_iterator<charT,traits>::proxy` is for exposition only. An implementation is permitted to provide equivalent functionality without providing a class with this name. Class `istreambuf_iterator<charT, traits>::proxy` provides a temporary placeholder as the return value of the post-increment operator (`operator++`). It keeps the character pointed to by the previous value of the iterator for some possible future access to get the character.

24.6.3.2 `istreambuf_iterator` constructors [istreambuf.iterator.cons]

```
constexpr istreambuf_iterator() noexcept;
```

1 *Effects:* Constructs the end-of-stream iterator.

```
istreambuf_iterator(basic_istream<charT,traits>& s) noexcept;
istreambuf_iterator(basic_streambuf<charT,traits>* s) noexcept;
```

2 *Effects:* Constructs an `istreambuf_iterator<>` that uses the `basic_streambuf<>` object `*(s.rdbuf())`, or `*s`, respectively. Constructs an end-of-stream iterator if `s.rdbuf()` is null.

```
istreambuf_iterator(const proxy& p) noexcept;
```

3 *Effects:* Constructs a `istreambuf_iterator<>` that uses the `basic_streambuf<>` object pointed to by the `proxy` object's constructor argument `p`.

24.6.3.3 `istreambuf_iterator::operator*` [istreambuf.iterator::op*]

```
charT operator*() const
```

1 *Returns:* The character obtained via the `streambuf` member `sbuf_->sgetc()`.

24.6.3.4 `istreambuf_iterator::operator++` [istreambuf.iterator::op++]

```
istreambuf_iterator& operator++();
```

1 *Effects:* `sbuf_->sbumpc()`.

2 *Returns:* `*this`.

```
proxy operator++(int);
```

3 *Returns:* `proxy(sbuf_->sbumpc(), sbuf_)`.

24.6.3.5 `istreambuf_iterator::equal` [istreambuf.iterator::equal]

```
bool equal(const istreambuf_iterator& b) const;
```

1 *Returns:* `true` if and only if both iterators are at end-of-stream, or neither is at end-of-stream, regardless of what `streambuf` object they use.

24.6.3.6 `operator==` [istreambuf.iterator::op==]

```
template <class charT, class traits>
  bool operator==(const istreambuf_iterator<charT,traits>& a,
                  const istreambuf_iterator<charT,traits>& b);
```

1 *Returns:* `a.equal(b)`.

24.6.3.7 `operator!=` [istreambuf.iterator::op!=]

```
template <class charT, class traits>
  bool operator!=(const istreambuf_iterator<charT,traits>& a,
                  const istreambuf_iterator<charT,traits>& b);
```

1 *Returns:* `!a.equal(b)`.

24.6.4 Class template `ostreambuf_iterator` [ostreambuf.iterator]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class ostreambuf_iterator {
  public:
    typedef output_iterator_tag          iterator_category;
    typedef void                         value_type;
    typedef void                         difference_type;
    typedef void                         pointer;
    typedef void                         reference;
    typedef charT                        char_type;
    typedef traits                       traits_type;
    typedef basic_streambuf<charT,traits> streambuf_type;
    typedef basic_ostream<charT,traits>   ostream_type;

    ostreambuf_iterator(ostream_type& s) noexcept;
    ostreambuf_iterator(streambuf_type* s) noexcept;
    ostreambuf_iterator& operator=(charT c);

    ostreambuf_iterator& operator*();
    ostreambuf_iterator& operator++();
    ostreambuf_iterator& operator++(int);
    bool failed() const noexcept;

  private:
    streambuf_type* sbuf_;               // exposition only
  };
}
```

1 The class template `ostreambuf_iterator` writes successive *characters* onto the output stream from which it was constructed. It is not possible to get a character value out of the output iterator.

24.6.4.1 `ostreambuf_iterator` constructors [ostreambuf.iter.cons]

```
ostreambuf_iterator(ostream_type& s) noexcept;
```

1 *Requires:* `s.rdbuf()` shall not be a null pointer.

2 *Effects:* Initializes `sbuf_` with `s.rdbuf()`.

```
ostreambuf_iterator(streambuf_type* s) noexcept;
```

3 *Requires:* `s` shall not be a null pointer.

4 *Effects:* Initializes `sbuf_` with `s`.

24.6.4.2 `ostreambuf_iterator` operations [ostreambuf.iter.ops]

```
ostreambuf_iterator& operator=(charT c);
```

1 *Effects:* If `failed()` yields `false`, calls `sbuf_->sputc(c)`; otherwise has no effect.

2 *Returns:* `*this`.

```
ostreambuf_iterator& operator*();
```

3 *Returns:* `*this`.

```
ostreambuf_iterator& operator++();
ostreambuf_iterator& operator++(int);
```

4 *Returns:* `*this`.

```
bool failed() const noexcept;
```

5 *Returns:* `true` if in any prior use of member `operator=`, the call to `sbuf_->sputc()` returned `traits::eof()`; or `false` otherwise.

24.7 Range access [iterator.range]

1 In addition to being available via inclusion of the `<iterator>` header, the function templates in 24.7 are available when any of the following headers are included: `<array>`, `<deque>`, `<forward_list>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<string>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

```
template <class C> auto begin(C& c) -> decltype(c.begin());
template <class C> auto begin(const C& c) -> decltype(c.begin());
```

2 *Returns:* `c.begin()`.

```
template <class C> auto end(C& c) -> decltype(c.end());
template <class C> auto end(const C& c) -> decltype(c.end());
```

3 *Returns:* `c.end()`.

```
template <class T, size_t N> constexpr T* begin(T (&array)[N]) noexcept;
```

4 *Returns:* `array`.

```
template <class T, size_t N> constexpr T* end(T (&array)[N]) noexcept;
```

5 *Returns:* `array + N`.

```
template <class C> constexpr auto cbegin(const C& c) noexcept(noexcept(std::begin(c)))
  -> decltype(std::begin(c));
```

6 *Returns:* `std::begin(c)`.

```
template <class C> constexpr auto cend(const C& c) noexcept(noexcept(std::end(c)))
  -> decltype(std::end(c));
```

7 *Returns:* `std::end(c)`.

```
template <class C> auto rbegin(C& c) -> decltype(c.rbegin());
template <class C> auto rbegin(const C& c) -> decltype(c.rbegin());
```

8 *Returns:* `c.rbegin()`.

```
template <class C> auto rend(C& c) -> decltype(c.rend());
template <class C> auto rend(const C& c) -> decltype(c.rend());
```

9 *Returns:* `c.rend()`.

```
template <class T, size_t N> reverse_iterator<T*> rbegin(T (&array)[N]);
```

10 *Returns:* `reverse_iterator<T*>(array + N)`.

```
template <class T, size_t N> reverse_iterator<T*> rend(T (&array)[N]);
```

11 *Returns:* `reverse_iterator<T*>(array)`.

```
template <class E> reverse_iterator<const E*> rbegin(initializer_list<E> il);
```

12 *Returns:* `reverse_iterator<const E*>(il.end())`.

```
template <class E> reverse_iterator<const E*> rend(initializer_list<E> il);
```

13 *Returns:* `reverse_iterator<const E*>(il.begin())`.

```
template <class C> auto crbegin(const C& c) -> decltype(std::rbegin(c));
```

14 *Returns:* `std::rbegin(c)`.

```
template <class C> auto crend(const C& c) -> decltype(std::rend(c));
```

15 *Returns:* `std::rend(c)`.

24.8 Container access [iterator.container]

1 In addition to being available via inclusion of the `<iterator>` header, the function templates in 24.8 are available when any of the following headers are included: `<array>`, `<deque>`, `<forward_list>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<string>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

```
template <class C> constexpr auto size(const C& c) -> decltype(c.size());
```

2 *Returns:* `c.size()`.

```
template <class T, size_t N> constexpr size_t size(const T (&array)[N]) noexcept;
```

3 *Returns:* `N`.

```
template <class C> constexpr auto empty(const C& c) -> decltype(c.empty());
```

4 *Returns:* `c.empty()`.

```
template <class T, size_t N> constexpr bool empty(const T (&array)[N]) noexcept;
```

5 *Returns:* `false`.

```
template <class E> constexpr bool empty(initializer_list<E> il) noexcept;
```

6 *Returns:* `il.size() == 0`.

```
template <class C> constexpr auto data(C& c) -> decltype(c.data());
template <class C> constexpr auto data(const C& c) -> decltype(c.data());
```

7 *Returns:* `c.data()`.

```
template <class T, size_t N> constexpr T* data(T (&array)[N]) noexcept;
```

8 *Returns:* `array`.

```
template <class E> constexpr const E* data(initializer_list<E> il) noexcept;
```

9 *Returns:* `il.begin()`.

25 Algorithms library [algorithms]

25.1 General [algorithms.general]

1 This Clause describes components that C++ programs may use to perform algorithmic operations on containers (Clause 23) and other sequences.

2 The following subclauses describe components for non-modifying sequence operation, modifying sequence operations, sorting and related operations, and algorithms from the ISO C library, as summarized in Table 111.

Table 111 — Algorithms library summary

Subclause		Header(s)
25.2	Non-modifying sequence operations	
25.3	Mutating sequence operations	`<algorithm>`
25.4	Sorting and related operations	
25.5	C library algorithms	`<cstdlib>`

Header `<algorithm>` synopsis

```
#include <initializer_list>

namespace std {

  // 25.2, non-modifying sequence operations:
  template <class InputIterator, class Predicate>
    bool all_of(InputIterator first, InputIterator last, Predicate pred);
  template <class InputIterator, class Predicate>
    bool any_of(InputIterator first, InputIterator last, Predicate pred);
  template <class InputIterator, class Predicate>
    bool none_of(InputIterator first, InputIterator last, Predicate pred);

  template<class InputIterator, class Function>
    Function for_each(InputIterator first, InputIterator last, Function f);
  template<class InputIterator, class T>
    InputIterator find(InputIterator first, InputIterator last,
                       const T& value);
  template<class InputIterator, class Predicate>
    InputIterator find_if(InputIterator first, InputIterator last,
                          Predicate pred);
  template<class InputIterator, class Predicate>
    InputIterator find_if_not(InputIterator first, InputIterator last,
                              Predicate pred);
  template<class ForwardIterator1, class ForwardIterator2>
    ForwardIterator1
      find_end(ForwardIterator1 first1, ForwardIterator1 last1,
               ForwardIterator2 first2, ForwardIterator2 last2);
  template<class ForwardIterator1, class ForwardIterator2,
     class BinaryPredicate>
    ForwardIterator1
      find_end(ForwardIterator1 first1, ForwardIterator1 last1,
```

```
            ForwardIterator2 first2, ForwardIterator2 last2,
            BinaryPredicate pred);

template<class InputIterator, class ForwardIterator>
  InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2);
template<class InputIterator, class ForwardIterator,
   class BinaryPredicate>
  InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2,
                  BinaryPredicate pred);

template<class ForwardIterator>
  ForwardIterator adjacent_find(ForwardIterator first,
                                ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
  ForwardIterator adjacent_find(ForwardIterator first,
                                ForwardIterator last,
                                BinaryPredicate pred);

template<class InputIterator, class T>
  typename iterator_traits<InputIterator>::difference_type
    count(InputIterator first, InputIterator last, const T& value);
template<class InputIterator, class Predicate>
  typename iterator_traits<InputIterator>::difference_type
    count_if(InputIterator first, InputIterator last, Predicate pred);

template<class InputIterator1, class InputIterator2>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2);
template
 <class InputIterator1, class InputIterator2, class BinaryPredicate>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2);

template
 <class InputIterator1, class InputIterator2, class BinaryPredicate>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2,
             BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2);
template
```

```
 <class InputIterator1, class InputIterator2, class BinaryPredicate>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2);

template
 <class InputIterator1, class InputIterator2, class BinaryPredicate>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2,
             BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2,
                 class BinaryPredicate>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
                    class BinaryPredicate>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, ForwardIterator2 last2,
                      BinaryPredicate pred);

template<class ForwardIterator1, class ForwardIterator2>
  ForwardIterator1 search(
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2,
   class BinaryPredicate>
  ForwardIterator1 search(
    ForwardIterator1 first1, ForwardIterator1 last1,
    ForwardIterator2 first2, ForwardIterator2 last2,
    BinaryPredicate pred);
template<class ForwardIterator, class Size, class T>
  ForwardIterator search_n(ForwardIterator first, ForwardIterator last,
                           Size count, const T& value);
template
 <class ForwardIterator, class Size, class T, class BinaryPredicate>
  ForwardIterator search_n(ForwardIterator first, ForwardIterator last,
                           Size count, const T& value,
                           BinaryPredicate pred);

// 25.3, modifying sequence operations:
// 25.3.1, copy:
template<class InputIterator, class OutputIterator>
  OutputIterator copy(InputIterator first, InputIterator last,
```

```
                    OutputIterator result);
template<class InputIterator, class Size, class OutputIterator>
  OutputIterator copy_n(InputIterator first, Size n,
                        OutputIterator result);
template<class InputIterator, class OutputIterator, class Predicate>
  OutputIterator copy_if(InputIterator first, InputIterator last,
                         OutputIterator result, Predicate pred);
template<class BidirectionalIterator1, class BidirectionalIterator2>
  BidirectionalIterator2 copy_backward(
    BidirectionalIterator1 first, BidirectionalIterator1 last,
    BidirectionalIterator2 result);

// 25.3.2, move:
template<class InputIterator, class OutputIterator>
  OutputIterator move(InputIterator first, InputIterator last,
                      OutputIterator result);
template<class BidirectionalIterator1, class BidirectionalIterator2>
  BidirectionalIterator2 move_backward(
    BidirectionalIterator1 first, BidirectionalIterator1 last,
    BidirectionalIterator2 result);

// 25.3.3, swap:
template<class ForwardIterator1, class ForwardIterator2>
  ForwardIterator2 swap_ranges(ForwardIterator1 first1,
  ForwardIterator1 last1, ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2>
  void iter_swap(ForwardIterator1 a, ForwardIterator2 b);

template<class InputIterator, class OutputIterator, class UnaryOperation>
  OutputIterator transform(InputIterator first, InputIterator last,
                           OutputIterator result, UnaryOperation op);
template<class InputIterator1, class InputIterator2, class OutputIterator,
   class BinaryOperation>
  OutputIterator transform(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, OutputIterator result,
                           BinaryOperation binary_op);

template<class ForwardIterator, class T>
  void replace(ForwardIterator first, ForwardIterator last,
               const T& old_value, const T& new_value);
template<class ForwardIterator, class Predicate, class T>
  void replace_if(ForwardIterator first, ForwardIterator last,
                  Predicate pred, const T& new_value);
template<class InputIterator, class OutputIterator, class T>
  OutputIterator replace_copy(InputIterator first, InputIterator last,
                              OutputIterator result,
                              const T& old_value, const T& new_value);
template<class InputIterator, class OutputIterator, class Predicate, class T>
  OutputIterator replace_copy_if(InputIterator first, InputIterator last,
                                 OutputIterator result,
                                 Predicate pred, const T& new_value);

template<class ForwardIterator, class T>
  void fill(ForwardIterator first, ForwardIterator last, const T& value);
template<class OutputIterator, class Size, class T>
```

```
  OutputIterator fill_n(OutputIterator first, Size n, const T& value);

template<class ForwardIterator, class Generator>
  void generate(ForwardIterator first, ForwardIterator last,
                Generator gen);
template<class OutputIterator, class Size, class Generator>
  OutputIterator generate_n(OutputIterator first, Size n, Generator gen);

template<class ForwardIterator, class T>
  ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                         const T& value);
template<class ForwardIterator, class Predicate>
  ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                            Predicate pred);
template<class InputIterator, class OutputIterator, class T>
  OutputIterator remove_copy(InputIterator first, InputIterator last,
                             OutputIterator result, const T& value);
template<class InputIterator, class OutputIterator, class Predicate>
  OutputIterator remove_copy_if(InputIterator first, InputIterator last,
                                OutputIterator result, Predicate pred);

template<class ForwardIterator>
  ForwardIterator unique(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class BinaryPredicate>
  ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                         BinaryPredicate pred);
template<class InputIterator, class OutputIterator>
  OutputIterator unique_copy(InputIterator first, InputIterator last,
                             OutputIterator result);
template<class InputIterator, class OutputIterator, class BinaryPredicate>
  OutputIterator unique_copy(InputIterator first, InputIterator last,
                             OutputIterator result, BinaryPredicate pred);

template<class BidirectionalIterator>
  void reverse(BidirectionalIterator first, BidirectionalIterator last);
template<class BidirectionalIterator, class OutputIterator>
  OutputIterator reverse_copy(BidirectionalIterator first,
                              BidirectionalIterator last,
                              OutputIterator result);

template<class ForwardIterator>
  ForwardIterator rotate(ForwardIterator first, ForwardIterator middle,
                         ForwardIterator last);
template<class ForwardIterator, class OutputIterator>
  OutputIterator rotate_copy(
    ForwardIterator first, ForwardIterator middle,
    ForwardIterator last, OutputIterator result);

// 25.3.12, shuffle:
template<class RandomAccessIterator, class UniformRandomNumberGenerator>
  void shuffle(RandomAccessIterator first,
                       RandomAccessIterator last,
                       UniformRandomNumberGenerator&& g);

// 25.3.13, partitions:
```

```
template <class InputIterator, class Predicate>
  bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);

template<class ForwardIterator, class Predicate>
  ForwardIterator partition(ForwardIterator first,
                            ForwardIterator last,
                            Predicate pred);
template<class BidirectionalIterator, class Predicate>
  BidirectionalIterator stable_partition(BidirectionalIterator first,
                                         BidirectionalIterator last,
                                         Predicate pred);
template <class InputIterator, class OutputIterator1,
          class OutputIterator2, class Predicate>
  pair<OutputIterator1, OutputIterator2>
  partition_copy(InputIterator first, InputIterator last,
                 OutputIterator1 out_true, OutputIterator2 out_false,
                 Predicate pred);
template<class ForwardIterator, class Predicate>
  ForwardIterator partition_point(ForwardIterator first,
                                  ForwardIterator last,
                                  Predicate pred);

// 25.4, sorting and related operations:
// 25.4.1, sorting:
template<class RandomAccessIterator>
  void sort(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void sort(RandomAccessIterator first, RandomAccessIterator last,
            Compare comp);

template<class RandomAccessIterator>
  void stable_sort(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void stable_sort(RandomAccessIterator first, RandomAccessIterator last,
                   Compare comp);

template<class RandomAccessIterator>
  void partial_sort(RandomAccessIterator first,
                    RandomAccessIterator middle,
                    RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void partial_sort(RandomAccessIterator first,
                    RandomAccessIterator middle,
                    RandomAccessIterator last, Compare comp);
template<class InputIterator, class RandomAccessIterator>
  RandomAccessIterator partial_sort_copy(
    InputIterator first, InputIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last);
template<class InputIterator, class RandomAccessIterator, class Compare>
  RandomAccessIterator partial_sort_copy(
    InputIterator first, InputIterator last,
    RandomAccessIterator result_first,
    RandomAccessIterator result_last,
    Compare comp);
```

```
template<class ForwardIterator>
  bool is_sorted(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  bool is_sorted(ForwardIterator first, ForwardIterator last,
                 Compare comp);
template<class ForwardIterator>
  ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last,
                                  Compare comp);

template<class RandomAccessIterator>
  void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                   RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                   RandomAccessIterator last, Compare comp);

// 25.4.3, binary search:
template<class ForwardIterator, class T>
  ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last,
                              const T& value);
template<class ForwardIterator, class T, class Compare>
  ForwardIterator lower_bound(ForwardIterator first, ForwardIterator last,
                              const T& value, Compare comp);

template<class ForwardIterator, class T>
  ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last,
                              const T& value);
template<class ForwardIterator, class T, class Compare>
  ForwardIterator upper_bound(ForwardIterator first, ForwardIterator last,
                              const T& value, Compare comp);

template<class ForwardIterator, class T>
  pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value);
template<class ForwardIterator, class T, class Compare>
  pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);

template<class ForwardIterator, class T>
  bool binary_search(ForwardIterator first, ForwardIterator last,
                     const T& value);
template<class ForwardIterator, class T, class Compare>
  bool binary_search(ForwardIterator first, ForwardIterator last,
                     const T& value, Compare comp);

// 25.4.4, merge:
template<class InputIterator1, class InputIterator2, class OutputIterator>
  OutputIterator merge(InputIterator1 first1, InputIterator1 last1,
                       InputIterator2 first2, InputIterator2 last2,
                       OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
```

```
    class Compare>
  OutputIterator merge(InputIterator1 first1, InputIterator1 last1,
                       InputIterator2 first2, InputIterator2 last2,
                       OutputIterator result, Compare comp);

template<class BidirectionalIterator>
  void inplace_merge(BidirectionalIterator first,
                     BidirectionalIterator middle,
                     BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
  void inplace_merge(BidirectionalIterator first,
                     BidirectionalIterator middle,
                     BidirectionalIterator last, Compare comp);

// 25.4.5, set operations:
template<class InputIterator1, class InputIterator2>
  bool includes(InputIterator1 first1, InputIterator1 last1,
                InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
  bool includes(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
  OutputIterator set_union(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2,
                           OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
    class Compare>
  OutputIterator set_union(InputIterator1 first1, InputIterator1 last1,
                           InputIterator2 first2, InputIterator2 last2,
                           OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
  OutputIterator set_intersection(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
    class Compare>
  OutputIterator set_intersection(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
  OutputIterator set_difference(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
    class Compare>
  OutputIterator set_difference(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
```

```
    OutputIterator result, Compare comp);

template<class InputIterator1, class InputIterator2, class OutputIterator>
  OutputIterator set_symmetric_difference(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    OutputIterator result);
template<class InputIterator1, class InputIterator2, class OutputIterator,
    class Compare>
  OutputIterator set_symmetric_difference(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    OutputIterator result, Compare comp);

// 25.4.6, heap operations:
template<class RandomAccessIterator>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);

template<class RandomAccessIterator>
  void pop_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void pop_heap(RandomAccessIterator first, RandomAccessIterator last,
                Compare comp);

template<class RandomAccessIterator>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);

template<class RandomAccessIterator>
  void sort_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);

template<class RandomAccessIterator>
  bool is_heap(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  bool is_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);
template<class RandomAccessIterator>
  RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last,
                                     Compare comp);

// 25.4.7, minimum and maximum:
template<class T> constexpr const T& min(const T& a, const T& b);
template<class T, class Compare>
  constexpr const T& min(const T& a, const T& b, Compare comp);
template<class T>
  constexpr T min(initializer_list<T> t);
```

```
template<class T, class Compare>
  constexpr T min(initializer_list<T> t, Compare comp);

template<class T> constexpr const T& max(const T& a, const T& b);
template<class T, class Compare>
  constexpr const T& max(const T& a, const T& b, Compare comp);
template<class T>
  constexpr T max(initializer_list<T> t);
template<class T, class Compare>
  constexpr T max(initializer_list<T> t, Compare comp);

template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);
template<class T, class Compare>
  constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);
template<class T>
  constexpr pair<T, T> minmax(initializer_list<T> t);
template<class T, class Compare>
  constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);

template<class ForwardIterator>
  constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last,
                                        Compare comp);
template<class ForwardIterator>
  constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last,
                                        Compare comp);
template<class ForwardIterator>
  constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);

template<class InputIterator1, class InputIterator2>
  bool lexicographical_compare(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2);
template<class InputIterator1, class InputIterator2, class Compare>
  bool lexicographical_compare(
    InputIterator1 first1, InputIterator1 last1,
    InputIterator2 first2, InputIterator2 last2,
    Compare comp);

// 25.4.9, permutations:
template<class BidirectionalIterator>
  bool next_permutation(BidirectionalIterator first,
                        BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
  bool next_permutation(BidirectionalIterator first,
                        BidirectionalIterator last, Compare comp);
template<class BidirectionalIterator>
  bool prev_permutation(BidirectionalIterator first,
```

```
                        BidirectionalIterator last);
  template<class BidirectionalIterator, class Compare>
    bool prev_permutation(BidirectionalIterator first,
                        BidirectionalIterator last, Compare comp);
}
```

3 All of the algorithms are separated from the particular implementations of data structures and are parameterized by iterator types. Because of this, they can work with program-defined data structures, as long as these data structures have iterator types satisfying the assumptions on the algorithms.

4 For purposes of determining the existence of data races, algorithms shall not modify objects referenced through an iterator argument unless the specification requires such modification.

5 Throughout this Clause, the names of template parameters are used to express type requirements.

(5.1) — If an algorithm's template parameter is named `InputIterator`, `InputIterator1`, or `InputIterator2`, the template argument shall satisfy the requirements of an input iterator (24.2.3).

(5.2) — If an algorithm's template parameter is named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, the template argument shall satisfy the requirements of an output iterator (24.2.4).

(5.3) — If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, or `ForwardIterator2`, the template argument shall satisfy the requirements of a forward iterator (24.2.5).

(5.4) — If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, or `BidirectionalIterator2`, the template argument shall satisfy the requirements of a bidirectional iterator (24.2.6).

(5.5) — If an algorithm's template parameter is named `RandomAccessIterator`, `RandomAccessIterator1`, or `RandomAccessIterator2`, the template argument shall satisfy the requirements of a random-access iterator (24.2.7).

6 If an algorithm's **Effects** section says that a value pointed to by any iterator passed as an argument is modified, then that algorithm has an additional type requirement: The type of that argument shall satisfy the requirements of a mutable iterator (24.2). [*Note:* This requirement does not affect arguments that are named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, because output iterators must always be mutable. — *end note*]

7 Both in-place and copying versions are provided for certain algorithms.[269] When such a version is provided for *algorithm* it is called *algorithm*`_copy`. Algorithms that take predicates end with the suffix `_if` (which follows the suffix `_copy`).

8 The `Predicate` parameter is used whenever an algorithm expects a function object (20.9) that, when applied to the result of dereferencing the corresponding iterator, returns a value testable as `true`. In other words, if an algorithm takes `Predicate pred` as its argument and `first` as its iterator argument, it should work correctly in the construct `pred(*first)` contextually converted to `bool` (Clause 4). The function object `pred` shall not apply any non-constant function through the dereferenced iterator.

9 The `BinaryPredicate` parameter is used whenever an algorithm expects a function object that when applied to the result of dereferencing two corresponding iterators or to dereferencing an iterator and type `T` when `T` is part of the signature returns a value testable as `true`. In other words, if an algorithm takes `BinaryPredicate binary_pred` as its argument and `first1` and `first2` as its iterator arguments, it should

269) The decision whether to include a copying version was usually based on complexity considerations. When the cost of doing the operation dominates the cost of copy, the copying version is not included. For example, `sort_copy` is not included because the cost of sorting is much more significant, and users might as well do `copy` followed by `sort`.

work correctly in the construct `binary_pred(*first1, *first2)` contextually converted to `bool` (Clause 4). `BinaryPredicate` always takes the first iterator's `value_type` as its first argument, that is, in those cases when `T value` is part of the signature, it should work correctly in the construct `binary_pred(*first1, value)` contextually converted to `bool` (Clause 4). `binary_pred` shall not apply any non-constant function through the dereferenced iterators.

10 [*Note:* Unless otherwise specified, algorithms that take function objects as arguments are permitted to copy those function objects freely. Programmers for whom object identity is important should consider using a wrapper class that points to a noncopied implementation object such as `reference_wrapper<T>` (20.9.4), or some equivalent solution. — *end note*]

11 When the description of an algorithm gives an expression such as `*first == value` for a condition, the expression shall evaluate to either true or false in boolean contexts.

12 In the description of the algorithms operators `+` and `-` are used for some of the iterator categories for which they do not have to be defined. In these cases the semantics of `a+n` is the same as that of

```
X tmp = a;
advance(tmp, n);
return tmp;
```

and that of `b-a` is the same as of

```
return distance(a, b);
```

25.2 Non-modifying sequence operations [alg.nonmodifying]

25.2.1 All of [alg.all_of]

```
template <class InputIterator, class Predicate>
  bool all_of(InputIterator first, InputIterator last, Predicate pred);
```

1 *Returns:* `true` if `[first,last)` is empty or if `pred(*i)` is `true` for every iterator `i` in the range `[first,last)`, and `false` otherwise.

2 *Complexity:* At most `last - first` applications of the predicate.

25.2.2 Any of [alg.any_of]

```
template <class InputIterator, class Predicate>
  bool any_of(InputIterator first, InputIterator last, Predicate pred);
```

1 *Returns:* `false` if `[first,last)` is empty or if there is no iterator `i` in the range `[first,last)` such that `pred(*i)` is `true`, and `true` otherwise.

2 *Complexity:* At most `last - first` applications of the predicate.

25.2.3 None of [alg.none_of]

```
template <class InputIterator, class Predicate>
  bool none_of(InputIterator first, InputIterator last, Predicate pred);
```

1 *Returns:* `true` if `[first,last)` is empty or if `pred(*i)` is `false` for every iterator `i` in the range `[first,last)`, and `false` otherwise.

2 *Complexity:* At most `last - first` applications of the predicate.

25.2.4 For each [alg.foreach]

```
template<class InputIterator, class Function>
  Function for_each(InputIterator first, InputIterator last, Function f);
```

1 *Requires:* `Function` shall meet the requirements of `MoveConstructible` (Table 20). [*Note:* `Function` need not meet the requirements of `CopyConstructible` (Table 21). — *end note*]

2 *Effects:* Applies `f` to the result of dereferencing every iterator in the range `[first,last)`, starting from `first` and proceeding to `last - 1`. [*Note:* If the type of `first` satisfies the requirements of a mutable iterator, `f` may apply nonconstant functions through the dereferenced iterator. — *end note*]

3 *Returns:* `std::move(f)`.

4 *Complexity:* Applies `f` exactly `last - first` times.

5 *Remarks:* If `f` returns a result, the result is ignored.

25.2.5 Find [alg.find]

```
template<class InputIterator, class T>
  InputIterator find(InputIterator first, InputIterator last,
                     const T& value);

template<class InputIterator, class Predicate>
  InputIterator find_if(InputIterator first, InputIterator last,
                        Predicate pred);
template<class InputIterator, class Predicate>
  InputIterator find_if_not(InputIterator first, InputIterator last,
                            Predicate pred);
```

1 *Returns:* The first iterator `i` in the range `[first,last)` for which the following corresponding conditions hold: `*i == value`, `pred(*i) != false`, `pred(*i) == false`. Returns `last` if no such iterator is found.

2 *Complexity:* At most `last - first` applications of the corresponding predicate.

25.2.6 Find end [alg.find.end]

```
template<class ForwardIterator1, class ForwardIterator2>
  ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
  ForwardIterator1
    find_end(ForwardIterator1 first1, ForwardIterator1 last1,
             ForwardIterator2 first2, ForwardIterator2 last2,
             BinaryPredicate pred);
```

1 *Effects:* Finds a subsequence of equal values in a sequence.

2 *Returns:* The last iterator `i` in the range `[first1,last1 - (last2 - first2))` such that for every non-negative integer `n < (last2 - first2)`, the following corresponding conditions hold: `*(i + n) == *(first2 + n)`, `pred(*(i + n), *(first2 + n)) != false`. Returns `last1` if `[first2,last2)` is empty or if no such iterator is found.

3 *Complexity:* At most `(last2 - first2) * (last1 - first1 - (last2 - first2) + 1)` applications of the corresponding predicate.

25.2.7 Find first [alg.find.first.of]

```
template<class InputIterator, class ForwardIterator>
  InputIterator
```

```
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2);

template<class InputIterator, class ForwardIterator,
          class BinaryPredicate>
  InputIterator
    find_first_of(InputIterator first1, InputIterator last1,
                  ForwardIterator first2, ForwardIterator last2,
                  BinaryPredicate pred);
```

1 *Effects:* Finds an element that matches one of a set of values.

2 *Returns:* The first iterator `i` in the range `[first1,last1)` such that for some iterator `j` in the range `[first2,last2)` the following conditions hold: `*i == *j, pred(*i,*j) != false`. Returns `last1` if `[first2,last2)` is empty or if no such iterator is found.

3 *Complexity:* At most `(last1-first1) * (last2-first2)` applications of the corresponding predicate.

25.2.8 Adjacent find [alg.adjacent.find]

```
template<class ForwardIterator>
  ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class BinaryPredicate>
  ForwardIterator adjacent_find(ForwardIterator first, ForwardIterator last,
                                BinaryPredicate pred);
```

1 *Returns:* The first iterator `i` such that both `i` and `i + 1` are in the range `[first,last)` for which the following corresponding conditions hold: `*i == *(i + 1), pred(*i, *(i + 1)) != false`. Returns `last` if no such iterator is found.

2 *Complexity:* For a nonempty range, exactly `min((i - first) + 1, (last - first) - 1)` applications of the corresponding predicate, where `i` is `adjacent_find`'s return value.

25.2.9 Count [alg.count]

```
template<class InputIterator, class T>
    typename iterator_traits<InputIterator>::difference_type
       count(InputIterator first, InputIterator last, const T& value);

template<class InputIterator, class Predicate>
    typename iterator_traits<InputIterator>::difference_type
      count_if(InputIterator first, InputIterator last, Predicate pred);
```

1 *Effects:* Returns the number of iterators `i` in the range `[first,last)` for which the following corresponding conditions hold: `*i == value, pred(*i) != false`.

2 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

25.2.10 Mismatch [mismatch]

```
template<class InputIterator1, class InputIterator2>
  pair<InputIterator1, InputIterator2>
      mismatch(InputIterator1 first1, InputIterator1 last1,
               InputIterator2 first2);

template<class InputIterator1, class InputIterator2,
          class BinaryPredicate>
```

```
  pair<InputIterator1, InputIterator2>
      mismatch(InputIterator1 first1, InputIterator1 last1,
               InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2);

template <class InputIterator1, class InputIterator2,
          class BinaryPredicate>
  pair<InputIterator1, InputIterator2>
    mismatch(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2,
             BinaryPredicate pred);
```

1 *Remarks:* If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.

2 *Returns:* A pair of iterators `i` and `j` such that `j == first2 + (i - first1)` and `i` is the first iterator in the range `[first1,last1)` for which the following corresponding conditions hold:

(2.1) — `j` is in the range `[first2, last2)`.

(2.2) — `!(*i == *(first2 + (i - first1)))`

(2.3) — `pred(*i, *(first2 + (i - first1))) == false`

Returns the pair `first1 + min(last1 - first1, last2 - first2)` and `first2 + min(last1 - first1, last2 - first2)` if such an iterator `i` is not found.

3 *Complexity:* At most `min(last1 - first1, last2 - first2)` applications of the corresponding predicate.

25.2.11 Equal [alg.equal]

```
template<class InputIterator1, class InputIterator2>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2);

template<class InputIterator1, class InputIterator2,
         class BinaryPredicate>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, BinaryPredicate pred);

template<class InputIterator1, class InputIterator2>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2,
          class BinaryPredicate>
  bool equal(InputIterator1 first1, InputIterator1 last1,
             InputIterator2 first2, InputIterator2 last2,
             BinaryPredicate pred);
```

1 *Remarks:* If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.

2 *Returns:* If `last1 - first1 != last2 - first2`, return `false`. Otherwise return `true` if for every iterator `i` in the range `[first1,last1)` the following corresponding conditions hold: `*i == *(first2 + (i - first1))`, `pred(*i, *(first2 + (i - first1))) != false`. Otherwise, returns `false`.

3 *Complexity:* No applications of the corresponding predicate if `InputIterator1` and `InputIterator2` meet the requirements of random access iterators and `last1 - first1 != last2 - first2`. Otherwise, at most `min(last1 - first1, last2 - first2)` applications of the corresponding predicate.

25.2.12 Is permutation [alg.is_permutation]

```
template<class ForwardIterator1, class ForwardIterator2>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2);
template<class ForwardIterator1, class ForwardIterator2,
                 class BinaryPredicate>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, BinaryPredicate pred);
template<class ForwardIterator1, class ForwardIterator2>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, ForwardIterator2 last2);
template<class ForwardIterator1, class ForwardIterator2,
                 class BinaryPredicate>
  bool is_permutation(ForwardIterator1 first1, ForwardIterator1 last1,
                      ForwardIterator2 first2, ForwardIterator2 last2,
                      BinaryPredicate pred);
```

1 *Requires:* `ForwardIterator1` and `ForwardIterator2` shall have the same value type. The comparison function shall be an equivalence relation.

2 *Remarks:* If `last2` was not given in the argument list, it denotes `first2 + (last1 - first1)` below.

3 *Returns:* If `last1 - first1 != last2 - first2`, return `false`. Otherwise return `true` if there exists a permutation of the elements in the range `[first2,first2 + (last1 - first1))`, beginning with `ForwardIterator2 begin`, such that `equal(first1, last1, begin)` returns `true` or `equal(first1, last1, begin, pred)` returns `true`; otherwise, returns `false`.

4 *Complexity:* No applications of the corresponding predicate if `ForwardIterator1` and `ForwardIterator2` meet the requirements of random access iterators and `last1 - first1 != last2 - first2`. Otherwise, exactly `distance(first1, last1)` applications of the corresponding predicate if `equal(first1, last1, first2, last2)` would return `true` if `pred` was not given in the argument list or `equal(first1, last1, first2, last2, pred)` would return `true` if pred was given in the argument list; otherwise, at worst $\mathcal{O}(N^2)$, where N has the value `distance(first1, last1)`.

25.2.13 Search [alg.search]

```
template<class ForwardIterator1, class ForwardIterator2>
  ForwardIterator1
    search(ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2);

template<class ForwardIterator1, class ForwardIterator2,
         class BinaryPredicate>
  ForwardIterator1
    search(ForwardIterator1 first1, ForwardIterator1 last1,
           ForwardIterator2 first2, ForwardIterator2 last2,
           BinaryPredicate pred);
```

1 *Effects:* Finds a subsequence of equal values in a sequence.

2 *Returns:* The first iterator `i` in the range `[first1,last1 - (last2-first2))` such that for every non-negative integer `n` less than `last2 - first2` the following corresponding conditions hold: `*(i +`

`n) == *(first2 + n), pred(*(i + n), *(first2 + n)) != false`. Returns `first1` if `[first2, last2)` is empty, otherwise returns `last1` if no such iterator is found.

3 *Complexity:* At most `(last1 - first1) * (last2 - first2)` applications of the corresponding predicate.

```
template<class ForwardIterator, class Size, class T>
  ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last, Size count,
           const T& value);

template<class ForwardIterator, class Size, class T,
         class BinaryPredicate>
  ForwardIterator
    search_n(ForwardIterator first, ForwardIterator last, Size count,
           const T& value, BinaryPredicate pred);
```

4 *Requires:* The type `Size` shall be convertible to integral type (4.7, 12.3).

5 *Effects:* Finds a subsequence of equal values in a sequence.

6 *Returns:* The first iterator `i` in the range `[first,last-count)` such that for every non-negative integer `n` less than `count` the following corresponding conditions hold: `*(i + n) == value, pred(*(i + n),value) != false`. Returns `last` if no such iterator is found.

7 *Complexity:* At most `last - first` applications of the corresponding predicate.

25.3 Mutating sequence operations [alg.modifying.operations]

25.3.1 Copy [alg.copy]

```
template<class InputIterator, class OutputIterator>
  OutputIterator copy(InputIterator first, InputIterator last,
                      OutputIterator result);
```

1 *Effects:* Copies elements in the range `[first,last)` into the range `[result,result + (last - first))` starting from `first` and proceeding to `last`. For each non-negative integer `n < (last - first)`, performs `*(result + n) = *(first + n)`.

2 *Returns:* `result + (last - first)`.

3 *Requires:* `result` shall not be in the range `[first,last)`.

4 *Complexity:* Exactly `last - first` assignments.

```
template<class InputIterator, class Size, class OutputIterator>
  OutputIterator copy_n(InputIterator first, Size n,
                        OutputIterator result);
```

5 *Effects:* For each non-negative integer $i < n$, performs `*(result + i) = *(first + i)`.

6 *Returns:* `result + n`.

7 *Complexity:* Exactly `n` assignments.

```
template<class InputIterator, class OutputIterator, class Predicate>
  OutputIterator copy_if(InputIterator first, InputIterator last,
                         OutputIterator result, Predicate pred);
```

8 *Requires:* The ranges `[first,last)` and `[result,result + (last - first))` shall not overlap.

9 *Effects:* Copies all of the elements referred to by the iterator `i` in the range `[first,last)` for which `pred(*i)` is `true`.

10 *Returns:* The end of the resulting range.

11 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

12 *Remarks:* Stable (17.6.5.7).

```
template<class BidirectionalIterator1, class BidirectionalIterator2>
  BidirectionalIterator2
    copy_backward(BidirectionalIterator1 first,
                  BidirectionalIterator1 last,
                  BidirectionalIterator2 result);
```

13 *Effects:* Copies elements in the range `[first,last)` into the range `[result - (last-first),result)` starting from `last - 1` and proceeding to `first`.[270] For each positive integer `n <= (last - first)`, performs `*(result - n) = *(last - n)`.

14 *Requires:* `result` shall not be in the range `(first,last]`.

15 *Returns:* `result - (last - first)`.

16 *Complexity:* Exactly `last - first` assignments.

25.3.2 Move [alg.move]

```
template<class InputIterator, class OutputIterator>
  OutputIterator move(InputIterator first, InputIterator last,
                      OutputIterator result);
```

1 *Effects:* Moves elements in the range `[first,last)` into the range `[result,result + (last - first))` starting from first and proceeding to last. For each non-negative integer `n < (last-first)`, performs `*(result + n) = std::move(*(first + n))`.

2 *Returns:* `result + (last - first)`.

3 *Requires:* `result` shall not be in the range `[first,last)`.

4 *Complexity:* Exactly `last - first` move assignments.

```
template<class BidirectionalIterator1, class BidirectionalIterator2>
  BidirectionalIterator2
    move_backward(BidirectionalIterator1 first,
                  BidirectionalIterator1 last,
                  BidirectionalIterator2 result);
```

5 *Effects:* Moves elements in the range `[first,last)` into the range `[result - (last-first),result)` starting from `last - 1` and proceeding to first.[271] For each positive integer `n <= (last - first)`, performs `*(result - n) = std::move(*(last - n))`.

6 *Requires:* `result` shall not be in the range `(first,last]`.

7 *Returns:* `result - (last - first)`.

8 *Complexity:* Exactly `last - first` assignments.

25.3.3 swap [alg.swap]

```
template<class ForwardIterator1, class ForwardIterator2>
  ForwardIterator2
    swap_ranges(ForwardIterator1 first1, ForwardIterator1 last1,
                ForwardIterator2 first2);
```

270) `copy_backward` should be used instead of copy when `last` is in the range `[result - (last - first),result)`.
271) `move_backward` should be used instead of move when last is in the range `[result - (last - first),result)`.

1 *Effects:* For each non-negative integer `n < (last1 - first1)` performs: `swap(*(first1 + n), *(first2 + n))`.

2 *Requires:* The two ranges `[first1,last1)` and `[first2,first2 + (last1 - first1))` shall not overlap. `*(first1 + n)` shall be swappable with (17.6.3.2) `*(first2 + n)`.

3 *Returns:* `first2 + (last1 - first1)`.

4 *Complexity:* Exactly `last1 - first1` swaps.

```
template<class ForwardIterator1, class ForwardIterator2>
  void iter_swap(ForwardIterator1 a, ForwardIterator2 b);
```

5 *Effects:* `swap(*a, *b)`.

6 *Requires:* `a` and `b` shall be dereferenceable. `*a` shall be swappable with (17.6.3.2) `*b`.

25.3.4 Transform [alg.transform]

```
template<class InputIterator, class OutputIterator,
         class UnaryOperation>
  OutputIterator
    transform(InputIterator first, InputIterator last,
              OutputIterator result, UnaryOperation op);

template<class InputIterator1, class InputIterator2,
         class OutputIterator, class BinaryOperation>
  OutputIterator
    transform(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, OutputIterator result,
              BinaryOperation binary_op);
```

1 *Effects:* Assigns through every iterator `i` in the range `[result,result + (last1 - first1))` a new corresponding value equal to `op(*(first1 + (i - result))` or `binary_op(*(first1 + (i - result)), *(first2 + (i - result)))`.

2 *Requires:* `op` and `binary_op` shall not invalidate iterators or subranges, or modify elements in the ranges `[first1,last1]`, `[first2,first2 + (last1 - first1)]`, and `[result,result + (last1 - first1)]`.[272]

3 *Returns:* `result + (last1 - first1)`.

4 *Complexity:* Exactly `last1 - first1` applications of `op` or `binary_op`.

5 *Remarks:* `result` may be equal to `first` in case of unary transform, or to `first1` or `first2` in case of binary transform.

25.3.5 Replace [alg.replace]

```
template<class ForwardIterator, class T>
  void replace(ForwardIterator first, ForwardIterator last,
               const T& old_value, const T& new_value);

template<class ForwardIterator, class Predicate, class T>
  void replace_if(ForwardIterator first, ForwardIterator last,
                  Predicate pred, const T& new_value);
```

1 *Requires:* The expression `*first = new_value` shall be valid.

272) The use of fully closed ranges is intentional.

2 *Effects:* Substitutes elements referred by the iterator `i` in the range `[first,last)` with `new_value`, when the following corresponding conditions hold: `*i == old_value`, `pred(*i) != false`.

3 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

```
template<class InputIterator, class OutputIterator, class T>
  OutputIterator
    replace_copy(InputIterator first, InputIterator last,
                 OutputIterator result,
                 const T& old_value, const T& new_value);

template<class InputIterator, class OutputIterator, class Predicate, class T>
  OutputIterator
    replace_copy_if(InputIterator first, InputIterator last,
                    OutputIterator result,
                    Predicate pred, const T& new_value);
```

4 *Requires:* The results of the expressions `*first` and `new_value` shall be writable to the `result` output iterator. The ranges `[first,last)` and `[result,result + (last - first))` shall not overlap.

5 *Effects:* Assigns to every iterator `i` in the range `[result,result + (last - first))` either `new_value` or `*(first + (i - result))` depending on whether the following corresponding conditions hold:

```
*(first + (i - result)) == old_value
pred(*(first + (i - result))) != false
```

6 *Returns:* `result + (last - first)`.

7 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

25.3.6 Fill [alg.fill]

```
template<class ForwardIterator, class T>
  void fill(ForwardIterator first, ForwardIterator last, const T& value);

template<class OutputIterator, class Size, class T>
  OutputIterator fill_n(OutputIterator first, Size n, const T& value);
```

1 *Requires:* The expression `value` shall be writable to the output iterator. The type `Size` shall be convertible to an integral type (4.7, 12.3).

2 *Effects:* The first algorithm assigns `value` through all the iterators in the range `[first,last)`. The second algorithm assigns `value` through all the iterators in the range `[first,first + n)` if `n` is positive, otherwise it does nothing.

3 *Returns:* `fill_n` returns `first + n` for non-negative values of `n` and `first` for negative values.

4 *Complexity:* Exactly `last - first`, `n`, or 0 assignments, respectively.

25.3.7 Generate [alg.generate]

```
template<class ForwardIterator, class Generator>
  void generate(ForwardIterator first, ForwardIterator last,
                Generator gen);

template<class OutputIterator, class Size, class Generator>
  OutputIterator generate_n(OutputIterator first, Size n, Generator gen);
```

1 *Effects:* The first algorithm invokes the function object `gen` and assigns the return value of `gen` through all the iterators in the range `[first,last)`. The second algorithm invokes the function object `gen` and assigns the return value of `gen` through all the iterators in the range `[first,first + n)` if `n` is positive, otherwise it does nothing.

2 *Requires:* `gen` takes no arguments, `Size` shall be convertible to an integral type (4.7, 12.3).

3 *Returns:* `generate_n` returns `first + n` for non-negative values of `n` and `first` for negative values.

4 *Complexity:* Exactly `last - first`, `n`, or 0 invocations of `gen` and assignments, respectively.

25.3.8 Remove [alg.remove]

```
template<class ForwardIterator, class T>
  ForwardIterator remove(ForwardIterator first, ForwardIterator last,
                         const T& value);

template<class ForwardIterator, class Predicate>
  ForwardIterator remove_if(ForwardIterator first, ForwardIterator last,
                            Predicate pred);
```

1 *Requires:* The type of `*first` shall satisfy the `MoveAssignable` requirements (Table 22).

2 *Effects:* Eliminates all the elements referred to by iterator `i` in the range `[first,last)` for which the following corresponding conditions hold: `*i == value, pred(*i) != false`.

3 *Returns:* The end of the resulting range.

4 *Remarks:* Stable (17.6.5.7).

5 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

6 *Note:* each element in the range `[ret,last)`, where `ret` is the returned value, has a valid but unspecified state, because the algorithms can eliminate elements by moving from elements that were originally in that range.

```
template<class InputIterator, class OutputIterator, class T>
  OutputIterator
    remove_copy(InputIterator first, InputIterator last,
                OutputIterator result, const T& value);

template<class InputIterator, class OutputIterator, class Predicate>
  OutputIterator
    remove_copy_if(InputIterator first, InputIterator last,
                   OutputIterator result, Predicate pred);
```

7 *Requires:* The ranges `[first,last)` and `[result,result + (last - first))` shall not overlap. The expression `*result = *first` shall be valid.

8 *Effects:* Copies all the elements referred to by the iterator `i` in the range `[first,last)` for which the following corresponding conditions do not hold: `*i == value, pred(*i) != false`.

9 *Returns:* The end of the resulting range.

10 *Complexity:* Exactly `last - first` applications of the corresponding predicate.

11 *Remarks:* Stable (17.6.5.7).

25.3.9 Unique [alg.unique]

```
template<class ForwardIterator>
  ForwardIterator unique(ForwardIterator first, ForwardIterator last);
```

```
template<class ForwardIterator, class BinaryPredicate>
  ForwardIterator unique(ForwardIterator first, ForwardIterator last,
                         BinaryPredicate pred);
```

1 *Effects:* For a nonempty range, eliminates all but the first element from every consecutive group of equivalent elements referred to by the iterator `i` in the range `[first + 1,last)` for which the following conditions hold: `*(i - 1) == *i` or `pred(*(i - 1), *i) != false`.

2 *Requires:* The comparison function shall be an equivalence relation. The type of `*first` shall satisfy the `MoveAssignable` requirements (Table 22).

3 *Returns:* The end of the resulting range.

4 *Complexity:* For nonempty ranges, exactly `(last - first) - 1` applications of the corresponding predicate.

```
template<class InputIterator, class OutputIterator>
  OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result);

template<class InputIterator, class OutputIterator,
         class BinaryPredicate>
  OutputIterator
    unique_copy(InputIterator first, InputIterator last,
                OutputIterator result, BinaryPredicate pred);
```

5 *Requires:* The comparison function shall be an equivalence relation. The ranges `[first,last)` and `[result,result+(last-first))` shall not overlap. The expression `*result = *first` shall be valid. Let `T` be the value type of `InputIterator`. If `InputIterator` meets the forward iterator requirements, then there are no additional requirements for `T`. Otherwise, if `OutputIterator` meets the forward iterator requirements and its value type is the same as `T`, then `T` shall be `CopyAssignable` (Table 23). Otherwise, `T` shall be both `CopyConstructible` (Table 21) and `CopyAssignable`.

6 *Effects:* Copies only the first element from every consecutive group of equal elements referred to by the iterator `i` in the range `[first,last)` for which the following corresponding conditions hold: `*i == *(i - 1)` or `pred(*i, *(i - 1)) != false`.

7 *Returns:* The end of the resulting range.

8 *Complexity:* For nonempty ranges, exactly `last - first - 1` applications of the corresponding predicate.

25.3.10 Reverse [alg.reverse]

```
template<class BidirectionalIterator>
  void reverse(BidirectionalIterator first, BidirectionalIterator last);
```

1 *Effects:* For each non-negative integer `i < (last - first)/2`, applies `iter_swap` to all pairs of iterators `first + i, (last - i) - 1`.

2 *Requires:* `*first` shall be swappable (17.6.3.2).

3 *Complexity:* Exactly `(last - first)/2` swaps.

```
template<class BidirectionalIterator, class OutputIterator>
  OutputIterator
    reverse_copy(BidirectionalIterator first,
                 BidirectionalIterator last, OutputIterator result);
```

4 *Effects:* Copies the range `[first,last)` to the range `[result,result+(last-first))` such that for every non-negative integer `i < (last - first)` the following assignment takes place: `*(result + (last - first) - 1 - i) = *(first + i)`.

5 *Requires:* The ranges `[first,last)` and `[result,result+(last-first))` shall not overlap.

6 *Returns:* `result + (last - first)`.

7 *Complexity:* Exactly `last - first` assignments.

25.3.11 Rotate [alg.rotate]

```
template<class ForwardIterator>
  ForwardIterator rotate(ForwardIterator first, ForwardIterator middle,
              ForwardIterator last);
```

1 *Effects:* For each non-negative integer `i < (last - first)`, places the element from the position `first + i` into position `first + (i + (last - middle)) % (last - first)`.

2 *Returns:* `first + (last - middle)`.

3 *Remarks:* This is a left rotate.

4 *Requires:* `[first,middle)` and `[middle,last)` shall be valid ranges. `ForwardIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and the requirements of `MoveAssignable` (Table 22).

5 *Complexity:* At most `last - first` swaps.

```
template<class ForwardIterator, class OutputIterator>
  OutputIterator
    rotate_copy(ForwardIterator first, ForwardIterator middle,
                ForwardIterator last, OutputIterator result);
```

6 *Effects:* Copies the range `[first,last)` to the range `[result,result + (last - first))` such that for each non-negative integer `i < (last - first)` the following assignment takes place: `*(result + i) = *(first + (i + (middle - first)) % (last - first))`.

7 *Returns:* `result + (last - first)`.

8 *Requires:* The ranges `[first,last)` and `[result,result + (last - first))` shall not overlap.

9 *Complexity:* Exactly `last - first` assignments.

25.3.12 Shuffle [alg.random.shuffle]

```
template<class RandomAccessIterator, class UniformRandomNumberGenerator>
  void shuffle(RandomAccessIterator first,
                       RandomAccessIterator last,
                       UniformRandomNumberGenerator&& g);
```

1 *Effects:* Permutes the elements in the range `[first,last)` such that each possible permutation of those elements has equal probability of appearance.

2 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type `UniformRandomNumberGenerator` shall meet the requirements of a uniform random number generator (26.5.1.3) type whose return type is convertible to `iterator_traits<RandomAccessIterator>::difference_type`.

3 *Complexity:* Exactly `(last - first) - 1` swaps.

4 *Remarks:* To the extent that the implementation of this function makes use of random numbers, the object `g` shall serve as the implementation's source of randomness.

25.3.13 Partitions [alg.partitions]

```
template <class InputIterator, class Predicate>
  bool is_partitioned(InputIterator first, InputIterator last, Predicate pred);
```

1 *Requires:* `InputIterator`'s value type shall be convertible to `Predicate`'s argument type.

2 *Returns:* `true` if `[first,last)` is empty or if `[first,last)` is partitioned by `pred`, i.e. if all elements that satisfy `pred` appear before those that do not.

3 *Complexity:* Linear. At most `last - first` applications of `pred`.

```
template<class ForwardIterator, class Predicate>
  ForwardIterator
    partition(ForwardIterator first,
              ForwardIterator last, Predicate pred);
```

4 *Effects:* Places all the elements in the range `[first,last)` that satisfy `pred` before all the elements that do not satisfy it.

5 *Returns:* An iterator `i` such that for every iterator `j` in the range `[first,i)` `pred(*j) != false`, and for every iterator `k` in the range `[i,last)`, `pred(*k) == false`.

6 *Requires:* `ForwardIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

7 *Complexity:* If ForwardIterator meets the requirements for a BidirectionalIterator, at most `(last - first) / 2` swaps are done; otherwise at most `last - first` swaps are done. Exactly `last - first` applications of the predicate are done.

```
template<class BidirectionalIterator, class Predicate>
  BidirectionalIterator
    stable_partition(BidirectionalIterator first,
                     BidirectionalIterator last, Predicate pred);
```

8 *Effects:* Places all the elements in the range `[first,last)` that satisfy `pred` before all the elements that do not satisfy it.

9 *Returns:* An iterator `i` such that for every iterator `j` in the range `[first,i)`, `pred(*j) != false`, and for every iterator `k` in the range `[i,last)`, `pred(*k) == false`. The relative order of the elements in both groups is preserved.

10 *Requires:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

11 *Complexity:* At most `(last - first) * log(last - first)` swaps, but only linear number of swaps if there is enough extra memory. Exactly `last - first` applications of the predicate.

```
template <class InputIterator, class OutputIterator1,
          class OutputIterator2, class Predicate>
  pair<OutputIterator1, OutputIterator2>
  partition_copy(InputIterator first, InputIterator last,
                 OutputIterator1 out_true, OutputIterator2 out_false,
                 Predicate pred);
```

12 *Requires:* `InputIterator`'s value type shall be `CopyAssignable`, and shall be writable to the `out_true` and `out_false` `OutputIterator`s, and shall be convertible to `Predicate`'s argument type. The input range shall not overlap with either of the output ranges.

13 *Effects:* For each iterator `i` in `[first,last)`, copies `*i` to the output range beginning with `out_true` if `pred(*i)` is `true`, or to the output range beginning with `out_false` otherwise.

14 *Returns:* A pair `p` such that `p.first` is the end of the output range beginning at `out_true` and `p.second` is the end of the output range beginning at `out_false`.

15 *Complexity:* Exactly `last - first` applications of `pred`.

```
template<class ForwardIterator, class Predicate>
  ForwardIterator partition_point(ForwardIterator first,
                                  ForwardIterator last,
                                  Predicate pred);
```

16 *Requires:* `ForwardIterator`'s value type shall be convertible to `Predicate`'s argument type. `[first, last)` shall be partitioned by `pred`, i.e. all elements that satisfy `pred` shall appear before those that do not.

17 *Returns:* An iterator `mid` such that `all_of(first, mid, pred)` and `none_of(mid, last, pred)` are both true.

18 *Complexity:* $\mathcal{O}(log(last - first))$ applications of `pred`.

25.4 Sorting and related operations [alg.sorting]

1 All the operations in 25.4 have two versions: one that takes a function object of type `Compare` and one that uses an `operator<`.

2 `Compare` is a function object type (20.9). The return value of the function call operation applied to an object of type `Compare`, when contextually converted to `bool` (Clause 4), yields `true` if the first argument of the call is less than the second, and `false` otherwise. `Compare comp` is used throughout for algorithms assuming an ordering relation. It is assumed that `comp` will not apply any non-constant function through the dereferenced iterator.

3 For all algorithms that take `Compare`, there is a version that uses `operator<` instead. That is, `comp(*i, *j) != false` defaults to `*i < *j != false`. For algorithms other than those described in 25.4.3 to work correctly, `comp` has to induce a strict weak ordering on the values.

4 The term *strict* refers to the requirement of an irreflexive relation (`!comp(x, x)` for all `x`), and the term *weak* to requirements that are not as strong as those for a total ordering, but stronger than those for a partial ordering. If we define `equiv(a, b)` as `!comp(a, b) && !comp(b, a)`, then the requirements are that `comp` and `equiv` both be transitive relations:

(4.1) — `comp(a, b) && comp(b, c)` implies `comp(a, c)`

(4.2) — `equiv(a, b) && equiv(b, c)` implies `equiv(a, c)`

[*Note:* Under these conditions, it can be shown that

(4.3) — `equiv` is an equivalence relation

(4.4) — `comp` induces a well-defined relation on the equivalence classes determined by `equiv`

(4.5) — The induced relation is a strict total ordering.

— *end note*]

5 A sequence is *sorted with respect to a comparator* `comp` if for every iterator `i` pointing to the sequence and every non-negative integer `n` such that `i + n` is a valid iterator pointing to an element of the sequence, `comp(*(i + n), *i) == false`.

6 A sequence `[start,finish)` is *partitioned with respect to an expression* `f(e)` if there exists an integer `n` such that for all `0 <= i < distance(start, finish)`, `f(*(start + i))` is true if and only if `i < n`.

7 In the descriptions of the functions that deal with ordering relationships we frequently use a notion of equivalence to describe concepts such as stability. The equivalence to which we refer is not necessarily an `operator==`, but an equivalence relation induced by the strict weak ordering. That is, two elements `a` and `b` are considered equivalent if and only if `!(a < b) && !(b < a)`.

25.4.1 Sorting [alg.sort]

25.4.1.1 `sort` [sort]

```
template<class RandomAccessIterator>
  void sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void sort(RandomAccessIterator first, RandomAccessIterator last,
            Compare comp);
```

1 *Effects:* Sorts the elements in the range `[first,last)`.

2 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

3 *Complexity:* $\mathcal{O}(N \log(N))$ (where N `== last - first`) comparisons.

25.4.1.2 `stable_sort` [stable.sort]

```
template<class RandomAccessIterator>
  void stable_sort(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void stable_sort(RandomAccessIterator first, RandomAccessIterator last,
                   Compare comp);
```

1 *Effects:* Sorts the elements in the range `[first,last)`.

2 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

3 *Complexity:* It does at most $N \log^2(N)$ (where N `== last - first`) comparisons; if enough extra memory is available, it is $N \log(N)$.

4 *Remarks:* Stable (17.6.5.7).

25.4.1.3 `partial_sort` [partial.sort]

```
template<class RandomAccessIterator>
  void partial_sort(RandomAccessIterator first,
                    RandomAccessIterator middle,
                    RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void partial_sort(RandomAccessIterator first,
                    RandomAccessIterator middle,
                    RandomAccessIterator last,
                    Compare comp);
```

1 *Effects:* Places the first `middle - first` sorted elements from the range `[first,last)` into the range `[first,middle)`. The rest of the elements in the range `[middle,last)` are placed in an unspecified order.

2 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

3 *Complexity:* It takes approximately `(last - first) * log(middle - first)` comparisons.

25.4.1.4 `partial_sort_copy` [partial.sort.copy]

```
template<class InputIterator, class RandomAccessIterator>
  RandomAccessIterator
    partial_sort_copy(InputIterator first, InputIterator last,
                      RandomAccessIterator result_first,
                      RandomAccessIterator result_last);

template<class InputIterator, class RandomAccessIterator,
         class Compare>
  RandomAccessIterator
    partial_sort_copy(InputIterator first, InputIterator last,
                      RandomAccessIterator result_first,
                      RandomAccessIterator result_last,
                      Compare comp);
```

1 *Effects:* Places the first `min(last - first, result_last - result_first)` sorted elements into the range `[result_first,result_first + min(last - first, result_last - result_first))`.

2 *Returns:* The smaller of: `result_last` or `result_first + (last - first)`.

3 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*result_first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

4 *Complexity:* Approximately `(last - first) * log(min(last - first, result_last - result_first))` comparisons.

25.4.1.5 `is_sorted` [is.sorted]

```
template<class ForwardIterator>
  bool is_sorted(ForwardIterator first, ForwardIterator last);
```

1 *Returns:* `is_sorted_until(first, last) == last`

```
template<class ForwardIterator, class Compare>
  bool is_sorted(ForwardIterator first, ForwardIterator last,
    Compare comp);
```

2 *Returns:* `is_sorted_until(first, last, comp) == last`

```
template<class ForwardIterator>
  ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  ForwardIterator is_sorted_until(ForwardIterator first, ForwardIterator last,
    Compare comp);
```

3 *Returns:* If `distance(first, last) < 2`, returns `last`. Otherwise, returns the last iterator `i` in `[first,last]` for which the range `[first,i)` is sorted.

4 *Complexity:* Linear.

25.4.2 Nth element [alg.nth.element]

```
template<class RandomAccessIterator>
  void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                   RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void nth_element(RandomAccessIterator first, RandomAccessIterator nth,
                   RandomAccessIterator last,  Compare comp);
```

1 After `nth_element` the element in the position pointed to by `nth` is the element that would be in that position if the whole range were sorted, unless `nth == last`. Also for every iterator `i` in the range `[first,nth)` and every iterator `j` in the range `[nth,last)` it holds that: `!(*j < *i)` or `comp(*j, *i) == false`.

2 *Requires:* `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

3 *Complexity:* Linear on average.

25.4.3 Binary search [alg.binary.search]

1 All of the algorithms in this section are versions of binary search and assume that the sequence being searched is partitioned with respect to an expression formed by binding the search key to an argument of the implied or explicit comparison function. They work on non-random access iterators minimizing the number of comparisons, which will be logarithmic for all types of iterators. They are especially appropriate for random access iterators, because these algorithms do a logarithmic number of steps through the data structure. For non-random access iterators they execute a linear number of steps.

25.4.3.1 `lower_bound` [lower.bound]

```
template<class ForwardIterator, class T>
  ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value);

template<class ForwardIterator, class T, class Compare>
  ForwardIterator
    lower_bound(ForwardIterator first, ForwardIterator last,
                const T& value, Compare comp);
```

1 *Requires:* The elements `e` of `[first,last)` shall be partitioned with respect to the expression `e < value` or `comp(e, value)`.

2 *Returns:* The furthermost iterator `i` in the range `[first,last]` such that for every iterator `j` in the range `[first,i)` the following corresponding conditions hold: `*j < value` or `comp(*j, value) != false`.

3 *Complexity:* At most $\log_2(\texttt{last - first}) + \mathcal{O}(1)$ comparisons.

25.4.3.2 `upper_bound` [upper.bound]

```
template<class ForwardIterator, class T>
  ForwardIterator
    upper_bound(ForwardIterator first, ForwardIterator last,
                const T& value);

template<class ForwardIterator, class T, class Compare>
```

```
ForwardIterator
  upper_bound(ForwardIterator first, ForwardIterator last,
              const T& value, Compare comp);
```

1 *Requires:* The elements `e` of `[first,last)` shall be partitioned with respect to the expression `!(value < e)` or `!comp(value, e)`.

2 *Returns:* The furthermost iterator `i` in the range `[first,last]` such that for every iterator `j` in the range `[first,i)` the following corresponding conditions hold: `!(value < *j)` or `comp(value, *j) == false`.

3 *Complexity:* At most $\log_2(\texttt{last - first}) + \mathcal{O}(1)$ comparisons.

25.4.3.3 `equal_range` [equal.range]

```
template<class ForwardIterator, class T>
  pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first,
                ForwardIterator last, const T& value);

template<class ForwardIterator, class T, class Compare>
  pair<ForwardIterator, ForwardIterator>
    equal_range(ForwardIterator first,
                ForwardIterator last, const T& value,
                Compare comp);
```

1 *Requires:* The elements `e` of `[first,last)` shall be partitioned with respect to the expressions `e < value` and `!(value < e)` or `comp(e, value)` and `!comp(value, e)`. Also, for all elements `e` of `[first, last)`, `e < value` shall imply `!(value < e)` or `comp(e, value)` shall imply `!comp(value, e)`.

2 *Returns:*

```
make_pair(lower_bound(first, last, value),
          upper_bound(first, last, value))
```

or

```
make_pair(lower_bound(first, last, value, comp),
          upper_bound(first, last, value, comp))
```

3 *Complexity:* At most $2 * \log_2(\texttt{last - first}) + \mathcal{O}(1)$ comparisons.

25.4.3.4 `binary_search` [binary.search]

```
template<class ForwardIterator, class T>
  bool binary_search(ForwardIterator first, ForwardIterator last,
                     const T& value);

template<class ForwardIterator, class T, class Compare>
  bool binary_search(ForwardIterator first, ForwardIterator last,
                     const T& value, Compare comp);
```

1 *Requires:* The elements `e` of `[first,last)` are partitioned with respect to the expressions `e < value` and `!(value < e)` or `comp(e, value)` and `!comp(value, e)`. Also, for all elements `e` of `[first, last)`, `e < value` implies `!(value < e)` or `comp(e, value)` implies `!comp(value, e)`.

2 *Returns:* `true` if there is an iterator `i` in the range `[first,last)` that satisfies the corresponding conditions: `!(*i < value) && !(value < *i)` or `comp(*i, value) == false && comp(value, *i) == false`.

3 *Complexity:* At most $\log_2(\texttt{last - first}) + \mathcal{O}(1)$ comparisons.

25.4.4 Merge [alg.merge]

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  OutputIterator
    merge(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, InputIterator2 last2,
          OutputIterator result);

template<class InputIterator1, class InputIterator2,
         class OutputIterator, class Compare>
  OutputIterator
    merge(InputIterator1 first1, InputIterator1 last1,
          InputIterator2 first2, InputIterator2 last2,
          OutputIterator result, Compare comp);
```

1 *Effects:* Copies all the elements of the two ranges `[first1,last1)` and `[first2,last2)` into the range `[result,result_last)`, where `result_last` is `result + (last1 - first1) + (last2 - first2)`, such that the resulting range satisfies `is_sorted(result, result_last)` or `is_sorted(result, result_last, comp)`, respectively.

2 *Requires:* The ranges `[first1,last1)` and `[first2,last2)` shall be sorted with respect to `operator<` or `comp`. The resulting range shall not overlap with either of the original ranges.

3 *Returns:* `result + (last1 - first1) + (last2 - first2)`.

4 *Complexity:* At most `(last1 - first1) + (last2 - first2) - 1` comparisons.

5 *Remarks:* Stable (17.6.5.7).

```
template<class BidirectionalIterator>
  void inplace_merge(BidirectionalIterator first,
                     BidirectionalIterator middle,
                     BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
  void inplace_merge(BidirectionalIterator first,
                     BidirectionalIterator middle,
                     BidirectionalIterator last, Compare comp);
```

6 *Effects:* Merges two sorted consecutive ranges `[first,middle)` and `[middle,last)`, putting the result of the merge into the range `[first,last)`. The resulting range will be in non-decreasing order; that is, for every iterator `i` in `[first,last)` other than `first`, the condition `*i < *(i - 1)` or, respectively, `comp(*i, *(i - 1))` will be false.

7 *Requires:* The ranges `[first,middle)` and `[middle,last)` shall be sorted with respect to `operator<` or `comp`. `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

8 *Complexity:* When enough additional memory is available, `(last - first) - 1` comparisons. If no additional memory is available, an algorithm with complexity $N \log(N)$ (where `N` is equal to `last - first`) may be used.

9 *Remarks:* Stable (17.6.5.7).

25.4.5 Set operations on sorted structures [alg.set.operations]

1 This section defines all the basic set operations on sorted structures. They also work with `multisets` (23.4.7) containing multiple copies of equivalent elements. The semantics of the set operations are generalized to `multisets` in a standard way by defining `set_union()` to contain the maximum number of occurrences of every element, `set_intersection()` to contain the minimum, and so on.

25.4.5.1 `includes` [includes]

```
template<class InputIterator1, class InputIterator2>
  bool includes(InputIterator1 first1, InputIterator1 last1,
                InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
  bool includes(InputIterator1 first1, InputIterator1 last1,
                InputIterator2 first2, InputIterator2 last2,
                Compare comp);
```

1 *Returns:* `true` if `[first2,last2)` is empty or if every element in the range `[first2,last2)` is contained in the range `[first1,last1)`. Returns `false` otherwise.

2 *Complexity:* At most `2 * ((last1 - first1) + (last2 - first2)) - 1` comparisons.

25.4.5.2 `set_union` [set.union]

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result);

template<class InputIterator1, class InputIterator2,
         class OutputIterator, class Compare>
  OutputIterator
    set_union(InputIterator1 first1, InputIterator1 last1,
              InputIterator2 first2, InputIterator2 last2,
              OutputIterator result, Compare comp);
```

1 *Effects:* Constructs a sorted union of the elements from the two ranges; that is, the set of elements that are present in one or both of the ranges.

2 *Requires:* The resulting range shall not overlap with either of the original ranges.

3 *Returns:* The end of the constructed range.

4 *Complexity:* At most `2 * ((last1 - first1) + (last2 - first2)) - 1` comparisons.

5 *Remarks:* If `[first1,last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then all m elements from the first range shall be copied to the output range, in order, and then $\max(n-m,0)$ elements from the second range shall be copied to the output range, in order.

25.4.5.3 `set_intersection` [set.intersection]

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2, InputIterator2 last2,
```

```
                    OutputIterator result);

template<class InputIterator1, class InputIterator2,
         class OutputIterator, class Compare>
  OutputIterator
    set_intersection(InputIterator1 first1, InputIterator1 last1,
                     InputIterator2 first2, InputIterator2 last2,
                     OutputIterator result, Compare comp);
```

1 *Effects:* Constructs a sorted intersection of the elements from the two ranges; that is, the set of elements that are present in both of the ranges.

2 *Requires:* The resulting range shall not overlap with either of the original ranges.

3 *Returns:* The end of the constructed range.

4 *Complexity:* At most `2 * ((last1 - first1) + (last2 - first2)) - 1` comparisons.

5 *Remarks:* If `[first1,last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, the first $\min(m, n)$ elements shall be copied from the first range to the output range, in order.

25.4.5.4 `set_difference` [set.difference]

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                   InputIterator2 first2, InputIterator2 last2,
                   OutputIterator result);

template<class InputIterator1, class InputIterator2,
         class OutputIterator, class Compare>
  OutputIterator
    set_difference(InputIterator1 first1, InputIterator1 last1,
                   InputIterator2 first2, InputIterator2 last2,
                   OutputIterator result, Compare comp);
```

1 *Effects:* Copies the elements of the range `[first1,last1)` which are not present in the range `[first2, last2)` to the range beginning at `result`. The elements in the constructed range are sorted.

2 *Requires:* The resulting range shall not overlap with either of the original ranges.

3 *Returns:* The end of the constructed range.

4 *Complexity:* At most `2 * ((last1 - first1) + (last2 - first2)) - 1` comparisons.

5 *Remarks:* If `[first1,last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, the last $\max(m-n, 0)$ elements from `[first1, last1)` shall be copied to the output range.

25.4.5.5 `set_symmetric_difference` [set.symmetric.difference]

```
template<class InputIterator1, class InputIterator2,
         class OutputIterator>
  OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2, InputIterator2 last2,
                             OutputIterator result);

template<class InputIterator1, class InputIterator2,
```

```
        class OutputIterator, class Compare>
  OutputIterator
    set_symmetric_difference(InputIterator1 first1, InputIterator1 last1,
                             InputIterator2 first2, InputIterator2 last2,
                             OutputIterator result, Compare comp);
```

1 *Effects:* Copies the elements of the range `[first1,last1)` that are not present in the range `[first2, last2)`, and the elements of the range `[first2,last2)` that are not present in the range `[first1, last1)` to the range beginning at `result`. The elements in the constructed range are sorted.

2 *Requires:* The resulting range shall not overlap with either of the original ranges.

3 *Returns:* The end of the constructed range.

4 *Complexity:* At most `2 * ((last1 - first1) + (last2 - first2)) - 1` comparisons.

5 *Remarks:* If `[first1,last1)` contains m elements that are equivalent to each other and `[first2, last2)` contains n elements that are equivalent to them, then $|m-n|$ of those elements shall be copied to the output range: the last $m-n$ of these elements from `[first1,last1)` if $m > n$, and the last $n-m$ of these elements from `[first2,last2)` if $m < n$.

25.4.6 Heap operations [alg.heap.operations]

1 A *heap* is a particular organization of elements in a range between two random access iterators `[a,b)`. Its two key properties are:

(1) There is no element greater than `*a` in the range and

(2) `*a` may be removed by `pop_heap()`, or a new element added by `push_heap()`, in $\mathcal{O}(\log(N))$ time.

2 These properties make heaps useful as priority queues.

3 `make_heap()` converts a range into a heap and `sort_heap()` turns a heap into a sorted sequence.

25.4.6.1 `push_heap` [push.heap]

```
template<class RandomAccessIterator>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void push_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);
```

1 *Effects:* Places the value in the location `last - 1` into the resulting heap `[first,last)`.

2 *Requires:* The range `[first,last - 1)` shall be a valid heap. The type of `*first` shall satisfy the `MoveConstructible` requirements (Table 20) and the `MoveAssignable` requirements (Table 22).

3 *Complexity:* At most `log(last - first)` comparisons.

25.4.6.2 `pop_heap` [pop.heap]

```
template<class RandomAccessIterator>
  void pop_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void pop_heap(RandomAccessIterator first, RandomAccessIterator last,
                Compare comp);
```

1 *Requires:* The range `[first,last)` shall be a valid non-empty heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

2 *Effects:* Swaps the value in the location `first` with the value in the location `last - 1` and makes `[first,last - 1)` into a heap.

3 *Complexity:* At most `2 * log(last - first)` comparisons.

25.4.6.3 `make_heap` [make.heap]

```
template<class RandomAccessIterator>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void make_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);
```

1 *Effects:* Constructs a heap out of the range `[first,last)`.

2 *Requires:* The type of `*first` shall satisfy the `MoveConstructible` requirements (Table 20) and the `MoveAssignable` requirements (Table 22).

3 *Complexity:* At most `3 * (last - first)` comparisons.

25.4.6.4 `sort_heap` [sort.heap]

```
template<class RandomAccessIterator>
  void sort_heap(RandomAccessIterator first, RandomAccessIterator last);

template<class RandomAccessIterator, class Compare>
  void sort_heap(RandomAccessIterator first, RandomAccessIterator last,
                 Compare comp);
```

1 *Effects:* Sorts elements in the heap `[first,last)`.

2 *Requires:* The range `[first,last)` shall be a valid heap. `RandomAccessIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2). The type of `*first` shall satisfy the requirements of `MoveConstructible` (Table 20) and of `MoveAssignable` (Table 22).

3 *Complexity:* At most $N \log(N)$ comparisons (where `N == last - first`).

25.4.6.5 `is_heap` [is.heap]

```
template<class RandomAccessIterator>
  bool is_heap(RandomAccessIterator first, RandomAccessIterator last);
```

1 *Returns:* `is_heap_until(first, last) == last`

```
template<class RandomAccessIterator, class Compare>
  bool is_heap(RandomAccessIterator first, RandomAccessIterator last, Compare comp);
```

2 *Returns:* `is_heap_until(first, last, comp) == last`

```
template<class RandomAccessIterator>
  RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last);
template<class RandomAccessIterator, class Compare>
  RandomAccessIterator is_heap_until(RandomAccessIterator first, RandomAccessIterator last,
    Compare comp);
```

3 *Returns:* If `distance(first, last) < 2`, returns `last`. Otherwise, returns the last iterator `i` in `[first,last]` for which the range `[first,i)` is a heap.

4 *Complexity:* Linear.

25.4.7 Minimum and maximum [alg.min.max]

```
template<class T> constexpr const T& min(const T& a, const T& b);
template<class T, class Compare>
  constexpr const T& min(const T& a, const T& b, Compare comp);
```

1 *Requires:* For the first form, type `T` shall be `LessThanComparable` (Table 18).

2 *Returns:* The smaller value.

3 *Remarks:* Returns the first argument when the arguments are equivalent.

4 *Complexity:* Exactly one comparison.

```
template<class T>
  constexpr T min(initializer_list<T> t);
template<class T, class Compare>
  constexpr T min(initializer_list<T> t, Compare comp);
```

5 *Requires:* `T` shall be `CopyConstructible` and `t.size() > 0`. For the first form, type `T` shall be `LessThanComparable`.

6 *Returns:* The smallest value in the initializer_list.

7 *Remarks:* Returns a copy of the leftmost argument when several arguments are equivalent to the smallest.

8 *Complexity:* Exactly `t.size() - 1` comparisons.

```
template<class T> constexpr const T& max(const T& a, const T& b);
template<class T, class Compare>
  constexpr const T& max(const T& a, const T& b, Compare comp);
```

9 *Requires:* For the first form, type `T` shall be `LessThanComparable` (Table 18).

10 *Returns:* The larger value.

11 *Remarks:* Returns the first argument when the arguments are equivalent.

12 *Complexity:* Exactly one comparison.

```
template<class T>
  constexpr T max(initializer_list<T> t);
template<class T, class Compare>
  constexpr T max(initializer_list<T> t, Compare comp);
```

13 *Requires:* `T` shall be `CopyConstructible` and `t.size() > 0`. For the first form, type `T` shall be `LessThanComparable`.

14 *Returns:* The largest value in the initializer_list.

15 *Remarks:* Returns a copy of the leftmost argument when several arguments are equivalent to the largest.

16 *Complexity:* Exactly `t.size() - 1` comparisons.

```
template<class T> constexpr pair<const T&, const T&> minmax(const T& a, const T& b);
template<class T, class Compare>
  constexpr pair<const T&, const T&> minmax(const T& a, const T& b, Compare comp);
```

17 *Requires:* For the first form, type `T` shall be `LessThanComparable` (Table 18).

18 *Returns:* `pair<const T&, const T&>(b, a)` if `b` is smaller than `a`, and `pair<const T&, const T&>(a, b)` otherwise.

19 *Remarks:* Returns `pair<const T&, const T&>(a, b)` when the arguments are equivalent.

20 *Complexity:* Exactly one comparison.

```
template<class T>
  constexpr pair<T, T> minmax(initializer_list<T> t);
template<class T, class Compare>
  constexpr pair<T, T> minmax(initializer_list<T> t, Compare comp);
```

21 *Requires:* `T` shall be `CopyConstructible` and `t.size() > 0`. For the first form, type `T` shall be `LessThanComparable`.

22 *Returns:* `pair<T, T>(x, y)`, where `x` has the smallest and `y` has the largest value in the initializer list.

23 *Remarks:* `x` is a copy of the leftmost argument when several arguments are equivalent to the smallest. `y` is a copy of the rightmost argument when several arguments are equivalent to the largest.

24 *Complexity:* At most `(3/2) * t.size()` applications of the corresponding predicate.

```
template<class ForwardIterator>
  constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last);

template<class ForwardIterator, class Compare>
  constexpr ForwardIterator min_element(ForwardIterator first, ForwardIterator last,
                                        Compare comp);
```

25 *Returns:* The first iterator `i` in the range `[first,last)` such that for every iterator `j` in the range `[first,last)` the following corresponding conditions hold: `!(*j < *i)` or `comp(*j, *i) == false`. Returns `last` if `first == last`.

26 *Complexity:* Exactly `max((last - first) - 1, 0)` applications of the corresponding comparisons.

```
template<class ForwardIterator>
  constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  constexpr ForwardIterator max_element(ForwardIterator first, ForwardIterator last,
                                        Compare comp);
```

27 *Returns:* The first iterator `i` in the range `[first,last)` such that for every iterator `j` in the range `[first,last)` the following corresponding conditions hold: `!(*i < *j)` or `comp(*i, *j) == false`. Returns `last` if `first == last`.

28 *Complexity:* Exactly `max((last - first) - 1, 0)` applications of the corresponding comparisons.

```
template<class ForwardIterator>
  constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last);
template<class ForwardIterator, class Compare>
  constexpr pair<ForwardIterator, ForwardIterator>
    minmax_element(ForwardIterator first, ForwardIterator last, Compare comp);
```

29 *Returns:* `make_pair(first, first)` if `[first,last)` is empty, otherwise `make_pair(m, M)`, where `m` is the first iterator in `[first,last)` such that no iterator in the range refers to a smaller element, and where `M` is the last iterator [273] in `[first,last)` such that no iterator in the range refers to a larger element.

30 *Complexity:* At most $max(\lfloor \frac{3}{2}(N-1) \rfloor, 0)$ applications of the corresponding predicate, where N is `distance(first, last)`.

273) This behavior intentionally differs from `max_element()`.

25.4.8 Lexicographical comparison [alg.lex.comparison]

```
template<class InputIterator1, class InputIterator2>
  bool
    lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, InputIterator2 last2);

template<class InputIterator1, class InputIterator2, class Compare>
  bool
    lexicographical_compare(InputIterator1 first1, InputIterator1 last1,
                            InputIterator2 first2, InputIterator2 last2,
                            Compare comp);
```

1 *Returns:* `true` if the sequence of elements defined by the range `[first1,last1)` is lexicographically less than the sequence of elements defined by the range `[first2,last2)` and `false` otherwise.

2 *Complexity:* At most `2*min((last1 - first1), (last2 - first2))` applications of the corresponding comparison.

3 *Remarks:* If two sequences have the same number of elements and their corresponding elements are equivalent, then neither sequence is lexicographically less than the other. If one sequence is a prefix of the other, then the shorter sequence is lexicographically less than the longer sequence. Otherwise, the lexicographical comparison of the sequences yields the same result as the comparison of the first corresponding pair of elements that are not equivalent.

```
for ( ; first1 != last1 && first2 != last2 ; ++first1, ++first2) {
  if (*first1 < *first2) return true;
  if (*first2 < *first1) return false;
}
return first1 == last1 && first2 != last2;
```

4 *Remarks:* An empty sequence is lexicographically less than any non-empty sequence, but not less than any empty sequence.

25.4.9 Permutation generators [alg.permutation.generators]

```
template<class BidirectionalIterator>
  bool next_permutation(BidirectionalIterator first,
                        BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
  bool next_permutation(BidirectionalIterator first,
                        BidirectionalIterator last, Compare comp);
```

1 *Effects:* Takes a sequence defined by the range `[first,last)` and transforms it into the next permutation. The next permutation is found by assuming that the set of all permutations is lexicographically sorted with respect to `operator<` or `comp`. If such a permutation exists, it returns `true`. Otherwise, it transforms the sequence into the smallest permutation, that is, the ascendingly sorted one, and returns `false`.

2 *Requires:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

3 *Complexity:* At most `(last - first)/2` swaps.

```
template<class BidirectionalIterator>
  bool prev_permutation(BidirectionalIterator first,
                        BidirectionalIterator last);

template<class BidirectionalIterator, class Compare>
```

```
bool prev_permutation(BidirectionalIterator first,
                      BidirectionalIterator last, Compare comp);
```

4 *Effects:* Takes a sequence defined by the range `[first,last)` and transforms it into the previous permutation. The previous permutation is found by assuming that the set of all permutations is lexicographically sorted with respect to `operator<` or `comp`.

5 *Returns:* `true` if such a permutation exists. Otherwise, it transforms the sequence into the largest permutation, that is, the descendingly sorted one, and returns `false`.

6 *Requires:* `BidirectionalIterator` shall satisfy the requirements of `ValueSwappable` (17.6.3.2).

7 *Complexity:* At most `(last - first)/2` swaps.

25.5 C library algorithms [alg.c.library]

1 Table 112 describes some of the contents of the header `<cstdlib>`.

Table 112 — Header `<cstdlib>` synopsis

Type	Name(s)
Type:	`size_t`
Functions:	`bsearch` `qsort`

2 The contents are the same as the Standard C library header `<stdlib.h>` with the following exceptions:

3 The function signature:

```
bsearch(const void *, const void *, size_t, size_t,
  int (*)(const void *, const void *));
```

is replaced by the two declarations:

```
extern "C" void* bsearch(const void* key, const void* base,
                         size_t nmemb, size_t size,
                         int (*compar)(const void*, const void*));
extern "C++" void* bsearch(const void* key, const void* base,
                           size_t nmemb, size_t size,
                           int (*compar)(const void*, const void*));
```

both of which have the same behavior as the original declaration.

4 The function signature:

```
qsort(void *, size_t, size_t,
  int (*)(const void *, const void *));
```

is replaced by the two declarations:

```
extern "C" void qsort(void* base, size_t nmemb, size_t size,
                      int (*compar)(const void*, const void*));
extern "C++" void qsort(void* base, size_t nmemb, size_t size,
                        int (*compar)(const void*, const void*));
```

both of which have the same behavior as the original declaration. The behavior is undefined unless the objects in the array pointed to by `base` are of trivial type.

[*Note:* Because the function argument `compar()` may throw an exception, `bsearch()` and `qsort()` are allowed to propagate the exception (17.6.5.12). — *end note*]

See also: ISO C 7.10.5.

26 Numerics library [numerics]

26.1 General [numerics.general]

1 This Clause describes components that C++ programs may use to perform seminumerical operations.

2 The following subclauses describe components for complex number types, random number generation, numeric (n-at-a-time) arrays, generalized numeric algorithms, and facilities included from the ISO C library, as summarized in Table 113.

Table 113 — Numerics library summary

Subclause		Header(s)
26.2	Requirements	
26.3	Floating-Point Environment	`<cfenv>`
26.4	Complex Numbers	`<complex>`
26.5	Random number generation	`<random>`
26.6	Numeric arrays	`<valarray>`
26.7	Generalized numeric operations	`<numeric>`

26.8	C library	`<cmath>`
		`<ctgmath>`
		`<tgmath.h>`
		`<cstdlib>`

26.2 Numeric type requirements [numeric.requirements]

1 The `complex` and `valarray` components are parameterized by the type of information they contain and manipulate. A C++ program shall instantiate these components only with a type `T` that satisfies the following requirements:[274]

(1.1) — `T` is not an abstract class (it has no pure virtual member functions);

(1.2) — `T` is not a reference type;

(1.3) — `T` is not cv-qualified;

(1.4) — If `T` is a class, it has a public default constructor;

(1.5) — If `T` is a class, it has a public copy constructor with the signature `T::T(const T&)`

(1.6) — If `T` is a class, it has a public destructor;

(1.7) — If `T` is a class, it has a public assignment operator whose signature is either `T& T::operator=(const T&)` or `T& T::operator=(T)`

(1.8) — If `T` is a class, its assignment operator, copy and default constructors, and destructor shall correspond to each other in the following sense: Initialization of raw storage using the default constructor, followed by assignment, is semantically equivalent to initialization of raw storage using the copy constructor.

274) In other words, value types. These include arithmetic types, pointers, the library class `complex`, and instantiations of `valarray` for value types.

Destruction of an object, followed by initialization of its raw storage using the copy constructor, is semantically equivalent to assignment to the original object.

[*Note:* This rule states that there shall not be any subtle differences in the semantics of initialization versus assignment. This gives an implementation considerable flexibility in how arrays are initialized.

[*Example:* An implementation is allowed to initialize a `valarray` by allocating storage using the `new` operator (which implies a call to the default constructor for each element) and then assigning each element its value. Or the implementation can allocate raw storage and use the copy constructor to initialize each element. — *end example*]

If the distinction between initialization and assignment is important for a class, or if it fails to satisfy any of the other conditions listed above, the programmer should use `vector` (23.3.6) instead of `valarray` for that class; — *end note*]

(1.9) — If `T` is a class, it does not overload unary `operator&`.

2 If any operation on `T` throws an exception the effects are undefined.

3 In addition, many member and related functions of `valarray<T>` can be successfully instantiated and will exhibit well-defined behavior if and only if `T` satisfies additional requirements specified for each such member or related function.

4 [*Example:* It is valid to instantiate `valarray<complex>`, but `operator>()` will not be successfully instantiated for `valarray<complex>` operands, since `complex` does not have any ordering operators. — *end example*]

26.3 The floating-point environment [cfenv]

26.3.1 Header <cfenv> synopsis [cfenv.syn]

```
namespace std {
  // types
  typedef object type  fenv_t;
  typedef integer type fexcept_t;

  // functions
  int feclearexcept(int except);
  int fegetexceptflag(fexcept_t* pflag, int except);
  int feraiseexcept(int except);
  int fesetexceptflag(const fexcept_t* pflag, int except);
  int fetestexcept(int except);

  int fegetround(void);
  int fesetround(int mode);

  int fegetenv(fenv_t* penv);
  int feholdexcept(fenv_t* penv);
  int fesetenv(const fenv_t* penv);
  int feupdateenv(const fenv_t* penv);
}
```

1 The header also defines the macros:

```
FE_ALL_EXCEPT
FE_DIVBYZERO
FE_INEXACT
FE_INVALID
FE_OVERFLOW
```

```
FE_UNDERFLOW

FE_DOWNWARD
FE_TONEAREST
FE_TOWARDZERO
FE_UPWARD

FE_DFL_ENV
```

2 The header defines all functions, types, and macros the same as Clause 7.6 of the C standard.

3 The floating-point environment has thread storage duration (3.7.2). The initial state for a thread's floating-point environment is the state of the floating-point environment of the thread that constructs the corresponding `std::thread` object (30.3.1) at the time it constructed the object. [*Note:* That is, the child thread gets the floating-point state of the parent thread at the time of the child's creation. — *end note*]

4 A separate floating-point environment shall be maintained for each thread. Each function accesses the environment corresponding to its calling thread.

26.4 Complex numbers [complex.numbers]

1 The header `<complex>` defines a class template, and numerous functions for representing and manipulating complex numbers.

2 The effect of instantiating the template `complex` for any type other than `float`, `double`, or `long double` is unspecified. The specializations `complex<float>`, `complex<double>`, and `complex<long double>` are literal types (3.9).

3 If the result of a function is not mathematically defined or not in the range of representable values for its type, the behavior is undefined.

4 If `z` is an lvalue expression of type *cv* `std::complex<T>` then:

(4.1) — the expression `reinterpret_cast<cv T(&)[2]>(z)` shall be well-formed,

(4.2) — `reinterpret_cast<cv T(&)[2]>(z)[0]` shall designate the real part of `z`, and

(4.3) — `reinterpret_cast<cv T(&)[2]>(z)[1]` shall designate the imaginary part of `z`.

Moreover, if `a` is an expression of type `cv std::complex<T>*` and the expression `a[i]` is well-defined for an integer expression `i`, then:

(4.4) — `reinterpret_cast<cv T*>(a)[2*i]` shall designate the real part of `a[i]`, and

(4.5) — `reinterpret_cast<cv T*>(a)[2*i + 1]` shall designate the imaginary part of `a[i]`.

26.4.1 Header <complex> synopsis [complex.syn]

```
namespace std {
  template<class T> class complex;
  template<> class complex<float>;
  template<> class complex<double>;
  template<> class complex<long double>;

  // 26.4.6, operators:
  template<class T>
    complex<T> operator+(const complex<T>&, const complex<T>&);
  template<class T> complex<T> operator+(const complex<T>&, const T&);
  template<class T> complex<T> operator+(const T&, const complex<T>&);
```

```
template<class T> complex<T> operator-(
  const complex<T>&, const complex<T>&);
template<class T> complex<T> operator-(const complex<T>&, const T&);
template<class T> complex<T> operator-(const T&, const complex<T>&);

template<class T> complex<T> operator*(
  const complex<T>&, const complex<T>&);
template<class T> complex<T> operator*(const complex<T>&, const T&);
template<class T> complex<T> operator*(const T&, const complex<T>&);

template<class T> complex<T> operator/(
  const complex<T>&, const complex<T>&);
template<class T> complex<T> operator/(const complex<T>&, const T&);
template<class T> complex<T> operator/(const T&, const complex<T>&);

template<class T> complex<T> operator+(const complex<T>&);
template<class T> complex<T> operator-(const complex<T>&);

template<class T> constexpr bool operator==(
  const complex<T>&, const complex<T>&);
template<class T> constexpr bool operator==(const complex<T>&, const T&);
template<class T> constexpr bool operator==(const T&, const complex<T>&);

template<class T> constexpr bool operator!=(const complex<T>&, const complex<T>&);
template<class T> constexpr bool operator!=(const complex<T>&, const T&);
template<class T> constexpr bool operator!=(const T&, const complex<T>&);

template<class T, class charT, class traits>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>&, complex<T>&);

template<class T, class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>&, const complex<T>&);

// 26.4.7, values:
template<class T> constexpr T real(const complex<T>&);
template<class T> constexpr T imag(const complex<T>&);

template<class T> T abs(const complex<T>&);
template<class T> T arg(const complex<T>&);
template<class T> T norm(const complex<T>&);

template<class T> complex<T> conj(const complex<T>&);
template <class T> complex<T> proj(const complex<T>&);
template<class T> complex<T> polar(const T&, const T& = 0);

// 26.4.8, transcendentals:
template<class T> complex<T> acos(const complex<T>&);
template<class T> complex<T> asin(const complex<T>&);
template<class T> complex<T> atan(const complex<T>&);

template<class T> complex<T> acosh(const complex<T>&);
template<class T> complex<T> asinh(const complex<T>&);
template<class T> complex<T> atanh(const complex<T>&);
```

```
  template<class T> complex<T> cos  (const complex<T>&);
  template<class T> complex<T> cosh (const complex<T>&);
  template<class T> complex<T> exp  (const complex<T>&);
  template<class T> complex<T> log  (const complex<T>&);
  template<class T> complex<T> log10(const complex<T>&);

  template<class T> complex<T> pow(const complex<T>&, const T&);
  template<class T> complex<T> pow(const complex<T>&, const complex<T>&);
  template<class T> complex<T> pow(const T&, const complex<T>&);

  template<class T> complex<T> sin  (const complex<T>&);
  template<class T> complex<T> sinh (const complex<T>&);
  template<class T> complex<T> sqrt (const complex<T>&);
  template<class T> complex<T> tan  (const complex<T>&);
  template<class T> complex<T> tanh (const complex<T>&);

  // 26.4.10, complex literals:
  inline namespace literals {
    inline namespace complex_literals {
      constexpr complex<long double> operator""il(long double);
      constexpr complex<long double> operator""il(unsigned long long);
      constexpr complex<double> operator""i(long double);
      constexpr complex<double> operator""i(unsigned long long);
      constexpr complex<float> operator""if(long double);
      constexpr complex<float> operator""if(unsigned long long);
    }
  }
}
```

26.4.2 Class template `complex` [complex]

```
namespace std {
  template<class T>
  class complex {
  public:
    typedef T value_type;

    constexpr complex(const T& re = T(), const T& im = T());
    constexpr complex(const complex&);
    template<class X> constexpr complex(const complex<X>&);

    constexpr T real() const;
    void real(T);
    constexpr T imag() const;
    void imag(T);

    complex<T>& operator= (const T&);
    complex<T>& operator+=(const T&);
    complex<T>& operator-=(const T&);
    complex<T>& operator*=(const T&);
    complex<T>& operator/=(const T&);

    complex& operator=(const complex&);
    template<class X> complex<T>& operator= (const complex<X>&);
    template<class X> complex<T>& operator+=(const complex<X>&);
```

```
    template<class X> complex<T>& operator-=(const complex<X>&);
    template<class X> complex<T>& operator*=(const complex<X>&);
    template<class X> complex<T>& operator/=(const complex<X>&);
  };
}
```

1 The class `complex` describes an object that can store the Cartesian components, `real()` and `imag()`, of a complex number.

26.4.3 `complex` specializations [complex.special]

```
namespace std {
  template<> class complex<float> {
  public:
    typedef float value_type;

    constexpr complex(float re = 0.0f, float im = 0.0f);
    constexpr explicit complex(const complex<double>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr float real() const;
    void real(float);
    constexpr float imag() const;
    void imag(float);

    complex<float>& operator= (float);
    complex<float>& operator+=(float);
    complex<float>& operator-=(float);
    complex<float>& operator*=(float);
    complex<float>& operator/=(float);

    complex<float>& operator=(const complex<float>&);
    template<class X> complex<float>& operator= (const complex<X>&);
    template<class X> complex<float>& operator+=(const complex<X>&);
    template<class X> complex<float>& operator-=(const complex<X>&);
    template<class X> complex<float>& operator*=(const complex<X>&);
    template<class X> complex<float>& operator/=(const complex<X>&);
  };

  template<> class complex<double> {
  public:
    typedef double value_type;

    constexpr complex(double re = 0.0, double im = 0.0);
    constexpr complex(const complex<float>&);
    constexpr explicit complex(const complex<long double>&);

    constexpr double real() const;
    void real(double);
    constexpr double imag() const;
    void imag(double);

    complex<double>& operator= (double);
    complex<double>& operator+=(double);
    complex<double>& operator-=(double);
    complex<double>& operator*=(double);
```

```
    complex<double>& operator/=(double);

    complex<double>& operator=(const complex<double>&);
    template<class X> complex<double>& operator= (const complex<X>&);
    template<class X> complex<double>& operator+=(const complex<X>&);
    template<class X> complex<double>& operator-=(const complex<X>&);
    template<class X> complex<double>& operator*=(const complex<X>&);
    template<class X> complex<double>& operator/=(const complex<X>&);
  };

  template<> class complex<long double> {
  public:
    typedef long double value_type;

    constexpr complex(long double re = 0.0L, long double im = 0.0L);
    constexpr complex(const complex<float>&);
    constexpr complex(const complex<double>&);

    constexpr long double real() const;
    void real(long double);
    constexpr long double imag() const;
    void imag(long double);

    complex<long double>& operator=(const complex<long double>&);
    complex<long double>& operator= (long double);
    complex<long double>& operator+=(long double);
    complex<long double>& operator-=(long double);
    complex<long double>& operator*=(long double);
    complex<long double>& operator/=(long double);

    template<class X> complex<long double>& operator= (const complex<X>&);
    template<class X> complex<long double>& operator+=(const complex<X>&);
    template<class X> complex<long double>& operator-=(const complex<X>&);
    template<class X> complex<long double>& operator*=(const complex<X>&);
    template<class X> complex<long double>& operator/=(const complex<X>&);
  };
}
```

26.4.4 `complex` member functions [complex.members]

```
template<class T> constexpr complex(const T& re = T(), const T& im = T());
```

1 *Effects:* Constructs an object of class `complex`.

2 *Postcondition:* `real() == re && imag() == im`.

```
constexpr T real() const;
```

Returns: The value of the real component.

```
void real(T val);
```

Effects: Assigns `val` to the real component.

```
constexpr T imag() const;
```

Returns: The value of the imaginary component.

```
void imag(T val);
```

Effects: Assigns `val` to the imaginary component.

26.4.5 `complex` member operators [complex.member.ops]

```
complex<T>& operator+=(const T& rhs);
```

1 *Effects:* Adds the scalar value `rhs` to the real part of the complex value `*this` and stores the result in the real part of `*this`, leaving the imaginary part unchanged.

2 *Returns:* `*this`.

```
complex<T>& operator-=(const T& rhs);
```

3 *Effects:* Subtracts the scalar value `rhs` from the real part of the complex value `*this` and stores the result in the real part of `*this`, leaving the imaginary part unchanged.

4 *Returns:* `*this`.

```
complex<T>& operator*=(const T& rhs);
```

5 *Effects:* Multiplies the scalar value `rhs` by the complex value `*this` and stores the result in `*this`.

6 *Returns:* `*this`.

```
complex<T>& operator/=(const T& rhs);
```

7 *Effects:* Divides the scalar value `rhs` into the complex value `*this` and stores the result in `*this`.

8 *Returns:* `*this`.

```
template<class X> complex<T>& operator+=(const complex<X>& rhs);
```

9 *Effects:* Adds the complex value `rhs` to the complex value `*this` and stores the sum in `*this`.

10 *Returns:* `*this`.

```
template<class X> complex<T>& operator-=(const complex<X>& rhs);
```

11 *Effects:* Subtracts the complex value `rhs` from the complex value `*this` and stores the difference in `*this`.

12 *Returns:* `*this`.

```
template<class X> complex<T>& operator*=(const complex<X>& rhs);
```

13 *Effects:* Multiplies the complex value `rhs` by the complex value `*this` and stores the product in `*this`.

Returns: `*this`.

```
template<class X> complex<T>& operator/=(const complex<X>& rhs);
```

14 *Effects:* Divides the complex value `rhs` into the complex value `*this` and stores the quotient in `*this`.

15 *Returns:* `*this`.

26.4.6 `complex` non-member operations [complex.ops]

```
template<class T> complex<T> operator+(const complex<T>& lhs);
```

1 *Remarks:* unary operator.

2 *Returns:* `complex<T>(lhs)`.

```
template<class T>
  complex<T> operator+(const complex<T>& lhs, const complex<T>& rhs);
template<class T> complex<T> operator+(const complex<T>& lhs, const T& rhs);
template<class T> complex<T> operator+(const T& lhs, const complex<T>& rhs);
```

3 *Returns:* `complex<T>(lhs) += rhs`.

```
template<class T> complex<T> operator-(const complex<T>& lhs);
```

4 *Remarks:* unary operator.

5 *Returns:* `complex<T>(-lhs.real(),-lhs.imag())`.

```
template<class T>
  complex<T> operator-(const complex<T>& lhs, const complex<T>& rhs);
template<class T> complex<T> operator-(const complex<T>& lhs, const T& rhs);
template<class T> complex<T> operator-(const T& lhs, const complex<T>& rhs);
```

6 *Returns:* `complex<T>(lhs) -= rhs`.

```
template<class T>
  complex<T> operator*(const complex<T>& lhs, const complex<T>& rhs);
template<class T> complex<T> operator*(const complex<T>& lhs, const T& rhs);
template<class T> complex<T> operator*(const T& lhs, const complex<T>& rhs);
```

7 *Returns:* `complex<T>(lhs) *= rhs`.

```
template<class T>
  complex<T> operator/(const complex<T>& lhs, const complex<T>& rhs);
template<class T> complex<T> operator/(const complex<T>& lhs, const T& rhs);
template<class T> complex<T> operator/(const T& lhs, const complex<T>& rhs);
```

8 *Returns:* `complex<T>(lhs) /= rhs`.

```
template<class T>
  constexpr bool operator==(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator==(const complex<T>& lhs, const T& rhs);
template<class T> constexpr bool operator==(const T& lhs, const complex<T>& rhs);
```

9 *Returns:* `lhs.real() == rhs.real() && lhs.imag() == rhs.imag()`.

10 *Remarks:* The imaginary part is assumed to be `T()`, or 0.0, for the `T` arguments.

```
template<class T>
  constexpr bool operator!=(const complex<T>& lhs, const complex<T>& rhs);
template<class T> constexpr bool operator!=(const complex<T>& lhs, const T& rhs);
template<class T> constexpr bool operator!=(const T& lhs, const complex<T>& rhs);
```

11 *Returns:* `rhs.real() != lhs.real() || rhs.imag() != lhs.imag()`.

```
template<class T, class charT, class traits>
basic_istream<charT, traits>&
operator>>(basic_istream<charT, traits>& is, complex<T>& x);
```

12 *Effects:* Extracts a complex number `x` of the form: `u`, `(u)`, or `(u,v)`, where `u` is the real part and `v` is the imaginary part (27.7.2.2).

13 *Requires:* The input values shall be convertible to `T`.

If bad input is encountered, calls `is.setstate(ios_base::failbit)` (which may throw `ios::failure` (27.5.5.4)).

14 *Returns:* `is`.

15 *Remarks:* This extraction is performed as a series of simpler extractions. Therefore, the skipping of whitespace is specified to be the same for each of the simpler extractions.

```
template<class T, class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& o, const complex<T>& x);
```

16 *Effects:* inserts the complex number `x` onto the stream `o` as if it were implemented as follows:

```
template<class T, class charT, class traits>
basic_ostream<charT, traits>&
operator<<(basic_ostream<charT, traits>& o, const complex<T>& x) {
  basic_ostringstream<charT, traits> s;
  s.flags(o.flags());
  s.imbue(o.getloc());
  s.precision(o.precision());
  s << '(' << x.real() << "," << x.imag() << ')';
  return o << s.str();
}
```

17 *Note:* In a locale in which comma is used as a decimal point character, the use of comma as a field separator can be ambiguous. Inserting `std::showpoint` into the output stream forces all outputs to show an explicit decimal point character; as a result, all inserted sequences of complex numbers can be extracted unambiguously.

26.4.7 `complex` value operations [complex.value.ops]

```
template<class T> constexpr T real(const complex<T>& x);
```

1 *Returns:* `x.real()`.

```
template<class T> constexpr T imag(const complex<T>& x);
```

2 *Returns:* `x.imag()`.

```
template<class T> T abs(const complex<T>& x);
```

3 *Returns:* The magnitude of `x`.

```
template<class T> T arg(const complex<T>& x);
```

4 *Returns:* The phase angle of `x`, or `atan2(imag(x), real(x))`.

```
template<class T> T norm(const complex<T>& x);
```

5 *Returns:* The squared magnitude of `x`.

```
template<class T> complex<T> conj(const complex<T>& x);
```

6 *Returns:* The complex conjugate of `x`.

```
template<class T> complex<T> proj(const complex<T>& x);
```

7 *Returns:* The projection of `x` onto the Riemann sphere.

8 *Remarks:* Behaves the same as the C function `cproj`, defined in 7.3.9.4.

```
template<class T> complex<T> polar(const T& rho, const T& theta = 0);
```

9 *Requires:* `rho` shall be non-negative and non-NaN. `theta` shall be finite.

10 *Returns:* The `complex` value corresponding to a complex number whose magnitude is `rho` and whose phase angle is `theta`.

26.4.8 `complex` transcendentals [complex.transcendentals]

```
template<class T> complex<T> acos(const complex<T>& x);
```

1 *Returns:* The complex arc cosine of `x`.

2 *Remarks:* Behaves the same as C function `cacos`, defined in 7.3.5.1.

```
template<class T> complex<T> asin(const complex<T>& x);
```

3 *Returns:* The complex arc sine of `x`.

4 *Remarks:* Behaves the same as C function `casin`, defined in 7.3.5.2.

```
template<class T> complex<T> atan(const complex<T>& x);
```

5 *Returns:* The complex arc tangent of `x`.

6 *Remarks:* Behaves the same as C function `catan`, defined in 7.3.5.3.

```
template<class T> complex<T> acosh(const complex<T>& x);
```

7 *Returns:* The complex arc hyperbolic cosine of `x`.

8 *Remarks:* Behaves the same as C function `cacosh`, defined in 7.3.6.1.

```
template<class T> complex<T> asinh(const complex<T>& x);
```

9 *Returns:* The complex arc hyperbolic sine of `x`.

10 *Remarks:* Behaves the same as C function `casinh`, defined in 7.3.6.2.

```
template<class T> complex<T> atanh(const complex<T>& x);
```

11 *Returns:* The complex arc hyperbolic tangent of `x`.

12 *Remarks:* Behaves the same as C function `catanh`, defined in 7.3.6.3.

```
template<class T> complex<T> cos(const complex<T>& x);
```

13 *Returns:* The complex cosine of `x`.

```
template<class T> complex<T> cosh(const complex<T>& x);
```

14 *Returns:* The complex hyperbolic cosine of `x`.

```
template<class T> complex<T> exp(const complex<T>& x);
```

15 *Returns:* The complex base e exponential of `x`.

```
template<class T> complex<T> log(const complex<T>& x);
```

16 *Remarks:* the branch cuts are along the negative real axis.

17 *Returns:* The complex natural (base e) logarithm of `x`, in the range of a strip mathematically unbounded along the real axis and in the interval `[-i times pi,i times pi]` along the imaginary axis. When `x` is a negative real number, `imag(log(x))` is pi.

```
template<class T> complex<T> log10(const complex<T>& x);
```

18 *Remarks:* the branch cuts are along the negative real axis.

19 *Returns:* The complex common (base 10) logarithm of `x`, defined as `log(x)/log(10)`.

```
template<class T>
  complex<T> pow(const complex<T>& x, const complex<T>& y);
template<class T> complex<T> pow  (const complex<T>& x, const T& y);
template<class T> complex<T> pow  (const T& x, const complex<T>& y);
```

20 *Remarks:* the branch cuts are along the negative real axis.

21 *Returns:* The complex power of base `x` raised to the `y`-th power, defined as `exp(y*log(x))`. The value returned for `pow(0,0)` is implementation-defined.

```
template<class T> complex<T> sin  (const complex<T>& x);
```

22 *Returns:* The complex sine of `x`.

```
template<class T> complex<T> sinh (const complex<T>& x);
```

23 *Returns:* The complex hyperbolic sine of `x`.

```
template<class T> complex<T> sqrt (const complex<T>& x);
```

24 *Remarks:* the branch cuts are along the negative real axis.

25 *Returns:* The complex square root of `x`, in the range of the right half-plane. If the argument is a negative real number, the value returned lies on the positive imaginary axis.

```
template<class T> complex<T> tan  (const complex<T>& x);
```

26 *Returns:* The complex tangent of `x`.

```
template<class T> complex<T> tanh (const complex<T>& x);
```

27 *Returns:* The complex hyperbolic tangent of `x`.

26.4.9 Additional overloads [cmplx.over]

1 The following function templates shall have additional overloads:

```
arg                 norm
conj                proj
imag                real
```

2 The additional overloads shall be sufficient to ensure:

1. If the argument has type `long double`, then it is effectively cast to `complex<long double>`.
2. Otherwise, if the argument has type `double` or an integer type, then it is effectively cast to `complex<double>`.
3. Otherwise, if the argument has type `float`, then it is effectively cast to `complex<float>`.

3 Function template `pow` shall have additional overloads sufficient to ensure, for a call with at least one argument of type `complex<T>`:

1. If either argument has type `complex<long double>` or type `long double`, then both arguments are effectively cast to `complex<long double>`.
2. Otherwise, if either argument has type `complex<double>`, `double`, or an integer type, then both arguments are effectively cast to `complex<double>`.
3. Otherwise, if either argument has type `complex<float>` or `float`, then both arguments are effectively cast to `complex<float>`.

26.4.10 Suffixes for complex number literals [complex.literals]

1 This section describes literal suffixes for constructing complex number literals. The suffixes `i`, `il`, and `if` create complex numbers of the types `complex<double>`, `complex<long double>`, and `complex<float>` respectively, with their imaginary part denoted by the given literal number and the real part being zero.

```
constexpr complex<long double> operator""il(long double d);
constexpr complex<long double> operator""il(unsigned long long d);
```

2 *Returns:* `complex<long double>{0.0L, static_cast<long double>(d)}`.

```
constexpr complex<double> operator""i(long double d);
constexpr complex<double> operator""i(unsigned long long d);
```

3 *Returns:* `complex<double>{0.0, static_cast<double>(d)}`.

```
constexpr complex<float> operator""if(long double d);
constexpr complex<float> operator""if(unsigned long long d);
```

4 *Returns:* `complex<float>{0.0f, static_cast<float>(d)}`.

26.4.11 Header <ccomplex> [ccmplx]

1 The header behaves as if it simply includes the header `<complex>`.

26.5 Random number generation [rand]

1 This subclause defines a facility for generating (pseudo-)random numbers.

2 In addition to a few utilities, four categories of entities are described: *uniform random number generators*, *random number engines*, *random number engine adaptors*, and *random number distributions*. These categorizations are applicable to types that satisfy the corresponding requirements, to objects instantiated from such types, and to templates producing such types when instantiated. [*Note:* These entities are specified in such a way as to permit the binding of any uniform random number generator object `e` as the argument to any random number distribution object `d`, thus producing a zero-argument function object such as given by `bind(d,e)`. — *end note*]

3 Each of the entities specified via this subclause has an associated arithmetic type (3.9.1) identified as `result_type`. With `T` as the `result_type` thus associated with such an entity, that entity is characterized:

a) as *boolean* or equivalently as *boolean-valued*, if `T` is `bool`;

b) otherwise as *integral* or equivalently as *integer-valued*, if `numeric_limits<T>::is_integer` is `true`;

c) otherwise as *floating* or equivalently as *real-valued*.

If integer-valued, an entity may optionally be further characterized as *signed* or *unsigned*, according to `numeric_limits<T>::is_signed`.

4 Unless otherwise specified, all descriptions of calculations in this subclause use mathematical real numbers.

5 Throughout this subclause, the operators bitand, bitor, and xor denote the respective conventional bitwise operations. Further:

a) the operator rshift denotes a bitwise right shift with zero-valued bits appearing in the high bits of the result, and

b) the operator lshift_w denotes a bitwise left shift with zero-valued bits appearing in the low bits of the result, and whose result is always taken modulo 2^w.

26.5.1 Requirements [rand.req]

26.5.1.1 General requirements [rand.req.genl]

1 Throughout this subclause 26.5, the effect of instantiating a template:

a) that has a template type parameter named `Sseq` is undefined unless the corresponding template argument is cv-unqualified and satisfies the requirements of seed sequence (26.5.1.2).

b) that has a template type parameter named `URNG` is undefined unless the corresponding template argument is cv-unqualified and satisfies the requirements of uniform random number generator (26.5.1.3).

c) that has a template type parameter named `Engine` is undefined unless the corresponding template argument is cv-unqualified and satisfies the requirements of random number engine (26.5.1.4).

d) that has a template type parameter named `RealType` is undefined unless the corresponding template argument is cv-unqualified and is one of `float`, `double`, or `long double`.

e) that has a template type parameter named `IntType` is undefined unless the corresponding template argument is cv-unqualified and is one of `short`, `int`, `long`, `long long`, `unsigned short`, `unsigned int`, `unsigned long`, or `unsigned long long`.

f) that has a template type parameter named `UIntType` is undefined unless the corresponding template argument is cv-unqualified and is one of `unsigned short`, `unsigned int`, `unsigned long`, or `unsigned long long`.

2 Throughout this subclause 26.5, phrases of the form "`x` is an iterator of a specific kind" shall be interpreted as equivalent to the more formal requirement that "`x` is a value of a type satisfying the requirements of the specified iterator type."

3 Throughout this subclause 26.5, any constructor that can be called with a single argument and that satisfies a requirement specified in this subclause shall be declared `explicit`.

26.5.1.2 Seed sequence requirements [rand.req.seedseq]

1 A *seed sequence* is an object that consumes a sequence of integer-valued data and produces a requested number of unsigned integer values i, $0 \leq i < 2^{32}$, based on the consumed data. [*Note:* Such an object provides a mechanism to avoid replication of streams of random variates. This can be useful, for example, in applications requiring large numbers of random number engines. — *end note*]

2 A class `S` satisfies the requirements of a seed sequence if the expressions shown in Table 114 are valid and have the indicated semantics, and if `S` also satisfies all other requirements of this section 26.5.1.2. In that Table and throughout this section:

a) `T` is the type named by `S`'s associated `result_type`;

b) `q` is a value of `S` and `r` is a possibly const value of `S`;

c) `ib` and `ie` are input iterators with an unsigned integer `value_type` of at least 32 bits;

d) `rb` and `re` are mutable random access iterators with an unsigned integer `value_type` of at least 32 bits;

e) `ob` is an output iterator; and

f) `il` is a value of `initializer_list<T>`.

Table 114 — Seed sequence requirements

Expression	Return type	Pre/post-condition	Complexity
`S::result_type`	`T`	`T` is an unsigned integer type (3.9.1) of at least 32 bits.	compile-time
`S()`		Creates a seed sequence with the same initial state as all other default-constructed seed sequences of type `S`.	constant
`S(ib,ie)`		Creates a seed sequence having internal state that depends on some or all of the bits of the supplied sequence [`ib`, `ie`).	$\mathcal{O}$(`ie` − `ib`)
`S(il)`		Same as `S(il.begin(), il.end())`.	same as `S(il.begin(), il.end())`
`q.generate(rb,re)`	`void`	Does nothing if `rb == re`. Otherwise, fills the supplied sequence [`rb`, `re`) with 32-bit quantities that depend on the sequence supplied to the constructor and possibly also depend on the history of `generate`'s previous invocations.	$\mathcal{O}$(`re` − `rb`)
`r.size()`	`size_t`	The number of 32-bit units that would be copied by a call to `r.param`.	constant
`r.param(ob)`	`void`	Copies to the given destination a sequence of 32-bit units that can be provided to the constructor of a second object of type `S`, and that would reproduce in that second object a state indistinguishable from the state of the first object.	$\mathcal{O}$(`r.size()`)

26.5.1.3 Uniform random number generator requirements [rand.req.urng]

1 A *uniform random number generator* `g` of type `G` is a function object returning unsigned integer values such that each value in the range of possible results has (ideally) equal probability of being returned. [*Note:* The degree to which `g`'s results approximate the ideal is often determined statistically. — *end note*]

2 A class `G` satisfies the requirements of a *uniform random number generator* if the expressions shown in Table 115 are valid and have the indicated semantics, and if `G` also satisfies all other requirements of this section 26.5.1.3. In that Table and throughout this section:

a) `T` is the type named by `G`'s associated `result_type`, and

b) `g` is a value of `G`.

Table 115 — Uniform random number generator requirements

Expression	Return type	Pre/post-condition	Complexity
`G::result_type`	`T`	`T` is an unsigned integer type (3.9.1).	compile-time
`g()`	`T`	Returns a value in the closed interval [`G::min()`, `G::max()`].	amortized constant
`G::min()`	`T`	Denotes the least value potentially returned by `operator()`.	compile-time
`G::max()`	`T`	Denotes the greatest value potentially returned by `operator()`.	compile-time

3 The following relation shall hold: `G::min() < G::max()`.

26.5.1.4 Random number engine requirements [rand.req.eng]

1 A *random number engine* (commonly shortened to *engine*) `e` of type `E` is a uniform random number generator that additionally meets the requirements (*e.g.*, for seeding and for input/output) specified in this section.

2 At any given time, `e` has a state $\mathtt{e}_i$ for some integer $i \geq 0$. Upon construction, `e` has an initial state $\mathtt{e}_0$. An engine's state may be established via a constructor, a `seed` function, assignment, or a suitable `operator>>`.

3 `E`'s specification shall define:

a) the size of `E`'s state in multiples of the size of `result_type`, given as an integral constant expression;

b) the *transition algorithm* TA by which `e`'s state $\mathtt{e}_i$ is advanced to its *successor state* $\mathtt{e}_{i+1}$; and

c) the *generation algorithm* GA by which an engine's state is mapped to a value of type `result_type`.

4 A class `E` that satisfies the requirements of a uniform random number generator (26.5.1.3) also satisfies the requirements of a *random number engine* if the expressions shown in Table 116 are valid and have the indicated semantics, and if `E` also satisfies all other requirements of this section 26.5.1.4. In that Table and throughout this section:

a) `T` is the type named by `E`'s associated `result_type`;

b) `e` is a value of `E`, `v` is an lvalue of `E`, `x` and `y` are (possibly `const`) values of `E`;

c) `s` is a value of `T`;

d) `q` is an lvalue satisfying the requirements of a seed sequence (26.5.1.2);

e) `z` is a value of type `unsigned long long`;

f) `os` is an lvalue of the type of some class template specialization `basic_ostream<charT, traits>`; and

g) `is` is an lvalue of the type of some class template specialization `basic_istream<charT, traits>`;

where `charT` and `traits` are constrained according to Clause 21 and Clause 27.

Table 116 — Random number engine requirements

Expression	Return type	Pre/post-condition	Complexity
`E()`		Creates an engine with the same initial state as all other default-constructed engines of type `E`.	$\mathcal{O}$(size of state)
`E(x)`		Creates an engine that compares equal to `x`.	$\mathcal{O}$(size of state)
`E(s)`		Creates an engine with initial state determined by `s`.	$\mathcal{O}$(size of state)
`E(q)`[275]		Creates an engine with an initial state that depends on a sequence produced by one call to `q.generate`.	same as complexity of `q.generate` called on a sequence whose length is size of state
`e.seed()`	`void`	post: `e == E()`.	same as `E()`
`e.seed(s)`	`void`	post: `e == E(s)`.	same as `E(s)`
`e.seed(q)`	`void`	post: `e == E(q)`.	same as `E(q)`
`e()`	`T`	Advances `e`'s state $\mathtt{e}_i$ to $\mathtt{e}_{i+1}$ = TA($\mathtt{e}_i$) and returns GA($\mathtt{e}_i$).	per Table 115
`e.discard(z)` [276]	`void`	Advances `e`'s state $\mathtt{e}_i$ to $\mathtt{e}_{i+\mathtt{z}}$ by any means equivalent to `z` consecutive calls `e()`.	no worse than the complexity of `z` consecutive calls `e()`
`x == y`	`bool`	This operator is an equivalence relation. With S_x and S_y as the infinite sequences of values that would be generated by repeated future calls to `x()` and `y()`, respectively, returns `true` if $S_x = S_y$; else returns `false`.	$\mathcal{O}$(size of state)
`x != y`	`bool`	`!(x == y)`.	$\mathcal{O}$(size of state)

275) This constructor (as well as the subsequent corresponding `seed()` function) may be particularly useful to applications requiring a large number of independent random sequences.
276) This operation is common in user code, and can often be implemented in an engine-specific manner so as to provide significant performance improvements over an equivalent naive loop that makes `z` consecutive calls `e()`.

Expression	Return type	Pre/post-condition	Complexity
`os << x`	reference to the type of `os`	With `os.`*fmtflags* set to `ios_base::dec|ios_base::left` and the fill character set to the space character, writes to `os` the textual representation of `x`'s current state. In the output, adjacent numbers are separated by one or more space characters. post: The `os.`*fmtflags* and fill character are unchanged.	$\mathcal{O}$(size of state)
`is >> v`	reference to the type of `is`	With `is.fmtflags` set to `ios_base::dec`, sets `v`'s state as determined by reading its textual representation from `is`. If bad input is encountered, ensures that `v`'s state is unchanged by the operation and calls `is.setstate(ios::failbit)` (which may throw `ios::failure` [27.5.5.4]). If a textual representation written via `os << x` was subsequently read via `is >> v`, then `x == v` provided that there have been no intervening invocations of `x` or of `v`. pre: `is` provides a textual representation that was previously written using an output stream whose imbued locale was the same as that of `is`, and whose type's template specialization arguments `charT` and `traits` were respectively the same as those of `is`. post: The `is.`*fmtflags* are unchanged.	$\mathcal{O}$(size of state)

5 `E` shall meet the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types. These operations shall each be of complexity no worse than $\mathcal{O}$(size of state).

26.5.1.5 Random number engine adaptor requirements [rand.req.adapt]

1 A *random number engine adaptor* (commonly shortened to *adaptor*) `a` of type `A` is a random number engine that takes values produced by some other random number engine, and applies an algorithm to those values in order to deliver a sequence of values with different randomness properties. An engine `b` of type `B` adapted in this way is termed a *base engine* in this context. The expression `a.base()` shall be valid and shall return a const reference to `a`'s base engine.

2 The requirements of a random number engine type shall be interpreted as follows with respect to a random number engine adaptor type.

```
A::A();
```

3 *Effects:* The base engine is initialized as if by its default constructor.

```
bool operator==(const A& a1, const A& a2);
```

4 *Returns:* `true` if `a1`'s base engine is equal to `a2`'s base engine. Otherwise returns `false`.

```
A::A(result_type s);
```

5 *Effects:* The base engine is initialized with `s`.

```
template<class Sseq> void A::A(Sseq& q);
```

6 *Effects:* The base engine is initialized with `q`.

```
void seed();
```

7 *Effects:* With `b` as the base engine, invokes `b.seed()`.

```
void seed(result_type s);
```

8 *Effects:* With `b` as the base engine, invokes `b.seed(s)`.

```
template<class Sseq> void seed(Sseq& q);
```

9 *Effects:* With `b` as the base engine, invokes `b.seed(q)`.

10 `A` shall also satisfy the following additional requirements:

a) The complexity of each function shall not exceed the complexity of the corresponding function applied to the base engine.

b) The state of `A` shall include the state of its base engine. The size of `A`'s state shall be no less than the size of the base engine.

c) Copying `A`'s state (*e.g.*, during copy construction or copy assignment) shall include copying the state of the base engine of `A`.

d) The textual representation of `A` shall include the textual representation of its base engine.

26.5.1.6 Random number distribution requirements [rand.req.dist]

1 A *random number distribution* (commonly shortened to *distribution*) `d` of type `D` is a function object returning values that are distributed according to an associated mathematical *probability density function* $p(z)$ or according to an associated *discrete probability function* $P(z_i)$. A distribution's specification identifies its associated probability function $p(z)$ or $P(z_i)$.

2 An associated probability function is typically expressed using certain externally-supplied quantities known as the *parameters of the distribution*. Such distribution parameters are identified in this context by writing, for example, $p(z \mid a, b)$ or $P(z_i \mid a, b)$, to name specific parameters, or by writing, for example, $p(z \mid \{\texttt{p}\})$ or $P(z_i \mid \{\texttt{p}\})$, to denote a distribution's parameters `p` taken as a whole.

3 A class `D` satisfies the requirements of a *random number distribution* if the expressions shown in Table 117 are valid and have the indicated semantics, and if `D` and its associated types also satisfy all other requirements of this section 26.5.1.6. In that Table and throughout this section,

a) `T` is the type named by `D`'s associated `result_type`;

b) `P` is the type named by `D`'s associated `param_type`;

c) `d` is a value of `D`, and `x` and `y` are (possibly `const`) values of `D`;

d) `glb` and `lub` are values of `T` respectively corresponding to the greatest lower bound and the least upper bound on the values potentially returned by `d`'s `operator()`, as determined by the current values of `d`'s parameters;

e) `p` is a (possibly `const`) value of `P`;

f) `g`, `g1`, and `g2` are lvalues of a type satisfying the requirements of a uniform random number generator [26.5.1.3];

g) `os` is an lvalue of the type of some class template specialization `basic_ostream<charT, traits>`; and

h) `is` is an lvalue of the type of some class template specialization `basic_istream<charT, traits>`;

where `charT` and `traits` are constrained according to Clauses 21 and 27.

Table 117 — Random number distribution requirements

Expression	Return type	Pre/post-condition	Complexity
`D::result_type`	`T`	`T` is an arithmetic type (3.9.1).	compile-time
`D::param_type`	`P`		compile-time
`D()`		Creates a distribution whose behavior is indistinguishable from that of any other newly default-constructed distribution of type `D`.	constant
`D(p)`		Creates a distribution whose behavior is indistinguishable from that of a distribution newly constructed directly from the values used to construct `p`.	same as `p`'s construction
`d.reset()`	`void`	Subsequent uses of `d` do not depend on values produced by any engine prior to invoking `reset`.	constant
`x.param()`	`P`	Returns a value `p` such that `D(p).param() == p`.	no worse than the complexity of `D(p)`
`d.param(p)`	`void`	post: `d.param() == p`.	no worse than the complexity of `D(p)`
`d(g)`	`T`	With `p` = `d.param()`, the sequence of numbers returned by successive invocations with the same object `g` is randomly distributed according to the associated $p(z \mid \{\texttt{p}\})$ or $P(z_i \mid \{\texttt{p}\})$ function.	amortized constant number of invocations of `g`

Expression	Return type	Pre/post-condition	Complexity
`d(g,p)`	`T`	The sequence of numbers returned by successive invocations with the same objects `g` and `p` is randomly distributed according to the associated $p(z \mid \{\texttt{p}\})$ or $P(z_i \mid \{\texttt{p}\})$ function.	amortized constant number of invocations of `g`
`x.min()`	`T`	Returns `glb`.	constant
`x.max()`	`T`	Returns `lub`.	constant
`x == y`	`bool`	This operator is an equivalence relation. Returns `true` if `x.param() == y.param()` and $S_1 = S_2$, where S_1 and S_2 are the infinite sequences of values that would be generated, respectively, by repeated future calls to `x(g1)` and `y(g2)` whenever `g1 == g2`. Otherwise returns `false`.	constant
`x != y`	`bool`	`!(x == y)`.	same as `x == y`.
`os << x`	reference to the type of `os`	Writes to `os` a textual representation for the parameters and the additional internal data of `x`. post: The `os`.*fmtflags* and fill character are unchanged.	
`is >> d`	reference to the type of `is`	Restores from `is` the parameters and additional internal data of the lvalue `d`. If bad input is encountered, ensures that `d` is unchanged by the operation and calls `is.setstate(ios::failbit)` (which may throw `ios::failure` [27.5.5.4]). pre: `is` provides a textual representation that was previously written using an `os` whose imbued locale and whose type's template specialization arguments `charT` and `traits` were the same as those of `is`. post: The `is`.*fmtflags* are unchanged.	

4 `D` shall satisfy the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types.

5 The sequence of numbers produced by repeated invocations of `d(g)` shall be independent of any invocation

of `os << d` or of any `const` member function of `D` between any of the invocations `d(g)`.

6 If a textual representation is written using `os << x` and that representation is restored into the same or a different object `y` of the same type using `is >> y`, repeated invocations of `y(g)` shall produce the same sequence of numbers as would repeated invocations of `x(g)`.

7 It is unspecified whether `D::param_type` is declared as a (nested) `class` or via a `typedef`. In this subclause 26.5, declarations of `D::param_type` are in the form of `typedef`s for convenience of exposition only.

8 `P` shall satisfy the requirements of `CopyConstructible` (Table 21), `CopyAssignable` (Table 23), and `EqualityComparable` (Table 17) types.

9 For each of the constructors of `D` taking arguments corresponding to parameters of the distribution, `P` shall have a corresponding constructor subject to the same requirements and taking arguments identical in number, type, and default values. Moreover, for each of the member functions of `D` that return values corresponding to parameters of the distribution, `P` shall have a corresponding member function with the identical name, type, and semantics.

10 `P` shall have a declaration of the form

```
typedef  D  distribution_type;
```

26.5.2 Header <random> synopsis [rand.synopsis]

```
#include <initializer_list>

namespace std {

 // 26.5.3.1, class template linear_congruential_engine
 template<class UIntType, UIntType a, UIntType c, UIntType m>
   class linear_congruential_engine;

 // 26.5.3.2, class template mersenne_twister_engine
 template<class UIntType, size_t w, size_t n, size_t m, size_t r,
          UIntType a, size_t u, UIntType d, size_t s,
          UIntType b, size_t t,
          UIntType c, size_t l, UIntType f>
   class mersenne_twister_engine;

 // 26.5.3.3, class template subtract_with_carry_engine
 template<class UIntType, size_t w, size_t s, size_t r>
   class subtract_with_carry_engine;

 // 26.5.4.2, class template discard_block_engine
 template<class Engine, size_t p, size_t r>
   class discard_block_engine;

 // 26.5.4.3, class template independent_bits_engine
 template<class Engine, size_t w, class UIntType>
   class independent_bits_engine;

 // 26.5.4.4, class template shuffle_order_engine
 template<class Engine, size_t k>
   class shuffle_order_engine;

 // 26.5.5, engines and engine adaptors with predefined parameters
 typedef see below minstd_rand0;
 typedef see below minstd_rand;
```

```
typedef see below mt19937;
typedef see below mt19937_64;
typedef see below ranlux24_base;
typedef see below ranlux48_base;
typedef see below ranlux24;
typedef see below ranlux48;
typedef see below knuth_b;
typedef see below default_random_engine;

// 26.5.6, class random_device
class random_device;

// 26.5.7.1, class seed_seq
class seed_seq;

// 26.5.7.2, function template generate_canonical
template<class RealType, size_t bits, class URNG>
  RealType generate_canonical(URNG& g);

// 26.5.8.2.1, class template uniform_int_distribution
template<class IntType = int>
  class uniform_int_distribution;

// 26.5.8.2.2, class template uniform_real_distribution
template<class RealType = double>
  class uniform_real_distribution;

// 26.5.8.3.1, class bernoulli_distribution
class bernoulli_distribution;

// 26.5.8.3.2, class template binomial_distribution
template<class IntType = int>
  class binomial_distribution;

// 26.5.8.3.3, class template geometric_distribution
template<class IntType = int>
  class geometric_distribution;

// 26.5.8.3.4, class template negative_binomial_distribution
template<class IntType = int>
  class negative_binomial_distribution;

// 26.5.8.4.1, class template poisson_distribution
template<class IntType = int>
  class poisson_distribution;

// 26.5.8.4.2, class template exponential_distribution
template<class RealType = double>
  class exponential_distribution;

// 26.5.8.4.3, class template gamma_distribution
template<class RealType = double>
  class gamma_distribution;

// 26.5.8.4.4, class template weibull_distribution
```

```
  template<class RealType = double>
    class weibull_distribution;

  // 26.5.8.4.5, class template extreme_value_distribution
  template<class RealType = double>
    class extreme_value_distribution;

  // 26.5.8.5.1, class template normal_distribution
  template<class RealType = double>
    class normal_distribution;

  // 26.5.8.5.2, class template lognormal_distribution
  template<class RealType = double>
    class lognormal_distribution;

  // 26.5.8.5.3, class template chi_squared_distribution
  template<class RealType = double>
    class chi_squared_distribution;

  // 26.5.8.5.4, class template cauchy_distribution
  template<class RealType = double>
    class cauchy_distribution;

  // 26.5.8.5.5, class template fisher_f_distribution
  template<class RealType = double>
    class fisher_f_distribution;

  // 26.5.8.5.6, class template student_t_distribution
  template<class RealType = double>
    class student_t_distribution;

  // 26.5.8.6.1, class template discrete_distribution
  template<class IntType = int>
    class discrete_distribution;

  // 26.5.8.6.2, class template piecewise_constant_distribution
  template<class RealType = double>
    class piecewise_constant_distribution;

  // 26.5.8.6.3, class template piecewise_linear_distribution
  template<class RealType = double>
    class piecewise_linear_distribution;

} // namespace std
```

26.5.3 Random number engine class templates [rand.eng]

1 Each type instantiated from a class template specified in this section 26.5.3 satisfies the requirements of a random number engine (26.5.1.4) type.

2 Except where specified otherwise, the complexity of each function specified in this section 26.5.3 is constant.

3 Except where specified otherwise, no function described in this section 26.5.3 throws an exception.

4 Descriptions are provided in this section 26.5.3 only for engine operations that are not described in 26.5.1.4 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators

are not shown in the synopses.

5 Each template specified in this section 26.5.3 requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

6 For every random number engine and for every random number engine adaptor `X` defined in this sub-clause (26.5.3) and in sub-clause 26.5.4:

(6.1) — if the constructor

```
template <class Sseq> explicit X(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this constructor shall not participate in overload resolution;

(6.2) — if the member function

```
template <class Sseq> void seed(Sseq& q);
```

is called with a type `Sseq` that does not qualify as a seed sequence, then this function shall not participate in overload resolution.

The extent to which an implementation determines that a type cannot be a seed sequence is unspecified, except that as a minimum a type shall not qualify as a seed sequence if it is implicitly convertible to `X::result_type`.

26.5.3.1 Class template `linear_congruential_engine` [rand.eng.lcong]

1 A `linear_congruential_engine` random number engine produces unsigned integer random numbers. The state $\mathtt{x}_i$ of a `linear_congruential_engine` object `x` is of size 1 and consists of a single integer. The transition algorithm is a modular linear function of the form $\mathsf{TA}(\mathtt{x}_i) = (a \cdot \mathtt{x}_i + c) \bmod m$; the generation algorithm is $\mathsf{GA}(\mathtt{x}_i) = \mathtt{x}_{i+1}$.

```
template<class UIntType, UIntType a, UIntType c, UIntType m>
 class linear_congruential_engine
{
public:
 // types
 typedef UIntType result_type;

 // engine characteristics
 static constexpr result_type multiplier = a;
 static constexpr result_type increment = c;
 static constexpr result_type modulus = m;
 static constexpr result_type min() { return c == 0u ? 1u: 0u; }
 static constexpr result_type max() { return m - 1u; }
 static constexpr result_type default_seed = 1u;

 // constructors and seeding functions
 explicit linear_congruential_engine(result_type s = default_seed);
 template<class Sseq> explicit linear_congruential_engine(Sseq& q);
 void seed(result_type s = default_seed);
 template<class Sseq> void seed(Sseq& q);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);
};
```

2 If the template parameter `m` is 0, the modulus m used throughout this section 26.5.3.1 is `numeric_limits<result_type>::max()` plus 1. [*Note:* m need not be representable as a value of type `result_type`. — *end note*]

3 If the template parameter `m` is not 0, the following relations shall hold: `a < m` and `c < m`.

4 The textual representation consists of the value of $\mathtt{x}_i$.

```
explicit linear_congruential_engine(result_type s = default_seed);
```

5 *Effects:* Constructs a `linear_congruential_engine` object. If $c \bmod m$ is 0 and `s` mod m is 0, sets the engine's state to 1, otherwise sets the engine's state to `s` mod m.

```
template<class Sseq> explicit linear_congruential_engine(Sseq& q);
```

6 *Effects:* Constructs a `linear_congruential_engine` object. With $k = \left\lceil \frac{\log_2 m}{32} \right\rceil$ and a an array (or equivalent) of length $k+3$, invokes `q.generate(`$a+0$`,` $a+k+3$`)` and then computes $S = \left(\sum_{j=0}^{k-1} a_{j+3} \cdot 2^{32j}\right) \bmod m$. If $c \bmod m$ is 0 and S is 0, sets the engine's state to 1, else sets the engine's state to S.

26.5.3.2 Class template `mersenne_twister_engine` [rand.eng.mers]

1 A `mersenne_twister_engine` random number engine[277] produces unsigned integer random numbers in the closed interval $[0, 2^w - 1]$. The state $\mathtt{x}_i$ of a `mersenne_twister_engine` object `x` is of size n and consists of a sequence X of n values of the type delivered by `x`; all subscripts applied to X are to be taken modulo n.

2 The transition algorithm employs a twisted generalized feedback shift register defined by shift values n and m, a twist value r, and a conditional xor-mask a. To improve the uniformity of the result, the bits of the raw shift register are additionally *tempered* (*i.e.*, scrambled) according to a bit-scrambling matrix defined by values u, d, s, b, t, c, and ℓ.

The state transition is performed as follows:

a) Concatenate the upper $w-r$ bits of X_{i-n} with the lower r bits of X_{i+1-n} to obtain an unsigned integer value Y.

b) With $\alpha = a \cdot (Y \mathsf{\ bitand\ } 1)$, set X_i to $X_{i+m-n} \mathsf{\ xor\ } (Y \mathsf{\ rshift\ } 1) \mathsf{\ xor\ } \alpha$.

The sequence X is initialized with the help of an initialization multiplier f.

3 The generation algorithm determines the unsigned integer values z_1, z_2, z_3, z_4 as follows, then delivers z_4 as its result:

a) Let $z_1 = X_i \mathsf{\ xor\ } \big((X_i \mathsf{\ rshift\ } u) \mathsf{\ bitand\ } d\big)$.

b) Let $z_2 = z_1 \mathsf{\ xor\ } \big((z_1 \mathsf{\ lshift}_w\ s) \mathsf{\ bitand\ } b\big)$.

c) Let $z_3 = z_2 \mathsf{\ xor\ } \big((z_2 \mathsf{\ lshift}_w\ t) \mathsf{\ bitand\ } c\big)$.

d) Let $z_4 = z_3 \mathsf{\ xor\ } (z_3 \mathsf{\ rshift\ } \ell)$.

```
template<class UIntType, size_t w, size_t n, size_t m, size_t r,
         UIntType a, size_t u, UIntType d, size_t s,
         UIntType b, size_t t,
```

277) The name of this engine refers, in part, to a property of its period: For properly-selected values of the parameters, the period is closely related to a large Mersenne prime number.

```
          UIntType c, size_t l, UIntType f>
 class mersenne_twister_engine
{
public:
 // types
 typedef UIntType result_type;

 // engine characteristics
 static constexpr size_t word_size = w;
 static constexpr size_t state_size = n;
 static constexpr size_t shift_size = m;
 static constexpr size_t mask_bits = r;
 static constexpr UIntType xor_mask = a;
 static constexpr size_t tempering_u = u;
 static constexpr UIntType tempering_d = d;
 static constexpr size_t tempering_s = s;
 static constexpr UIntType tempering_b = b;
 static constexpr size_t tempering_t = t;
 static constexpr UIntType tempering_c = c;
 static constexpr size_t tempering_l = l;
 static constexpr UIntType initialization_multiplier = f;
 static constexpr result_type min() { return 0; }
 static constexpr result_type max() { return  2^w − 1; }
 static constexpr result_type default_seed = 5489u;

 // constructors and seeding functions
 explicit mersenne_twister_engine(result_type value = default_seed);
 template<class Sseq> explicit mersenne_twister_engine(Sseq& q);
 void seed(result_type value = default_seed);
 template<class Sseq> void seed(Sseq& q);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);
};
```

4 The following relations shall hold: `0 < m`, `m <= n`, `2u < w`, `r <= w`, `u <= w`, `s <= w`, `t <= w`, `l <= w`, `w <= numeric_limits<UIntType>::digits`, `a <= (1u<<w) - 1u`, `b <= (1u<<w) - 1u`, `c <= (1u<<w) - 1u`, `d <= (1u<<w) - 1u`, and `f <= (1u<<w) - 1u`.

5 The textual representation of $\mathtt{x}_i$ consists of the values of $X_{i-n}, \ldots, X_{i-1}$, in that order.

`explicit mersenne_twister_engine(result_type value = default_seed);`

6 *Effects:* Constructs a `mersenne_twister_engine` object. Sets X_{-n} to `value` $\bmod 2^w$. Then, iteratively for $i = 1-n, \ldots, -1$, sets X_i to

$$\left[f \cdot \left(X_{i-1} \mathsf{xor} \left(X_{i-1} \mathsf{rshift} \left(w-2\right)\right)\right) + i \bmod n\right] \bmod 2^w \ .$$

7 *Complexity:* $\mathcal{O}(n)$.

`template<class Sseq> explicit mersenne_twister_engine(Sseq& q);`

8 *Effects:* Constructs a `mersenne_twister_engine` object. With $k = \lceil w/32 \rceil$ and a an array (or equivalent) of length $n \cdot k$, invokes `q.generate(`$a + 0$`,` $a + n \cdot k$`)` and then, iteratively for $i = -n, \ldots, -1$, sets X_i to $\left(\sum_{j=0}^{k-1} a_{k(i+n)+j} \cdot 2^{32j}\right) \bmod 2^w$. Finally, if the most significant $w - r$ bits of X_{-n} are zero, and if each of the other resulting X_i is 0, changes X_{-n} to 2^{w-1}.

26.5.3.3 Class template `subtract_with_carry_engine` [rand.eng.sub]

1 A `subtract_with_carry_engine` random number engine produces unsigned integer random numbers.

2 The state $\mathtt{x}_i$ of a `subtract_with_carry_engine` object `x` is of size $\mathcal{O}(r)$, and consists of a sequence X of r integer values $0 \leq X_i < m = 2^w$; all subscripts applied to X are to be taken modulo r. The state $\mathtt{x}_i$ additionally consists of an integer c (known as the *carry*) whose value is either 0 or 1.

3 The state transition is performed as follows:

a) Let $Y = X_{i-s} - X_{i-r} - c$.

b) Set X_i to $y = Y \bmod m$. Set c to 1 if $Y < 0$, otherwise set c to 0.

[*Note:* This algorithm corresponds to a modular linear function of the form $\mathsf{TA}(\mathtt{x}_i) = (a \cdot \mathtt{x}_i) \bmod b$, where b is of the form $m^r - m^s + 1$ and $a = b - (b-1)/m$. — *end note*]

4 The generation algorithm is given by $\mathsf{GA}(\mathtt{x}_i) = y$, where y is the value produced as a result of advancing the engine's state as described above.

```
template<class UIntType, size_t w, size_t s, size_t r>
 class subtract_with_carry_engine
{
public:
 // types
 typedef UIntType result_type;

 // engine characteristics
 static constexpr size_t word_size = w;
 static constexpr size_t short_lag = s;
 static constexpr size_t long_lag = r;
 static constexpr result_type min() { return 0; }
 static constexpr result_type max() { return m - 1; }
 static constexpr result_type default_seed = 19780503u;

 // constructors and seeding functions
 explicit subtract_with_carry_engine(result_type value = default_seed);
 template<class Sseq> explicit subtract_with_carry_engine(Sseq& q);
 void seed(result_type value = default_seed);
 template<class Sseq> void seed(Sseq& q);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);
};
```

5 The following relations shall hold: `0u < s`, `s < r`, `0 < w`, and `w <= numeric_limits<UIntType>::digits`.

6 The textual representation consists of the values of $X_{i-r}, \ldots, X_{i-1}$, in that order, followed by c.

```
explicit subtract_with_carry_engine(result_type value = default_seed);
```

7 *Effects:* Constructs a `subtract_with_carry_engine` object. Sets the values of $X_{-r}, \ldots, X_{-1}$, in that order, as specified below. If X_{-1} is then 0, sets c to 1; otherwise sets c to 0.

To set the values X_k, first construct `e`, a `linear_congruential_engine` object, as if by the following definition:

```
linear_congruential_engine<result_type,
                           40014u,0u,2147483563u> e(value == 0u ? default_seed : value);
```

Then, to set each X_k, obtain new values $z_0, \ldots, z_{n-1}$ from $n = \lceil w/32 \rceil$ successive invocations of `e` taken modulo 2^{32}. Set X_k to $\left(\sum_{j=0}^{n-1} z_j \cdot 2^{32j}\right) \bmod m$.

8 *Complexity:* Exactly $n \cdot$ `r` invocations of `e`.

```
template<class Sseq> explicit subtract_with_carry_engine(Sseq& q);
```

9 *Effects:* Constructs a `subtract_with_carry_engine` object. With $k = \lceil w/32 \rceil$ and a an array (or equivalent) of length $r \cdot k$, invokes `q.generate(`a`+0, `a`+`$r \cdot k$`)` and then, iteratively for $i = -r, \ldots, -1$, sets X_i to $\left(\sum_{j=0}^{k-1} a_{k(i+r)+j} \cdot 2^{32j}\right) \bmod m$. If X_{-1} is then 0, sets c to 1; otherwise sets c to 0.

26.5.4 Random number engine adaptor class templates [rand.adapt]

26.5.4.1 In general [rand.adapt.general]

1 Each type instantiated from a class template specified in this section 26.5.3 satisfies the requirements of a random number engine adaptor (26.5.1.5) type.

2 Except where specified otherwise, the complexity of each function specified in this section 26.5.4 is constant.

3 Except where specified otherwise, no function described in this section 26.5.4 throws an exception.

4 Descriptions are provided in this section 26.5.4 only for adaptor operations that are not described in section 26.5.1.5 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.

5 Each template specified in this section 26.5.4 requires one or more relationships, involving the value(s) of its non-type template parameter(s), to hold. A program instantiating any of these templates is ill-formed if any such required relationship fails to hold.

26.5.4.2 Class template `discard_block_engine` [rand.adapt.disc]

1 A `discard_block_engine` random number engine adaptor produces random numbers selected from those produced by some base engine e. The state `x`$_i$ of a `discard_block_engine` engine adaptor object `x` consists of the state `e`$_i$ of its base engine `e` and an additional integer n. The size of the state is the size of e's state plus 1.

2 The transition algorithm discards all but $r > 0$ values from each block of $p \geq r$ values delivered by e. The state transition is performed as follows: If $n \geq r$, advance the state of `e` from `e`$_i$ to `e`$_{i+p-r}$ and set n to 0. In any case, then increment n and advance `e`'s then-current state `e`$_j$ to `e`$_{j+1}$.

3 The generation algorithm yields the value returned by the last invocation of `e()` while advancing `e`'s state as described above.

```
template<class Engine, size_t p, size_t r>
 class discard_block_engine
{
public:
 // types
 typedef typename Engine::result_type result_type;

 // engine characteristics
 static constexpr size_t block_size = p;
 static constexpr size_t used_block = r;
 static constexpr result_type min() { return Engine::min(); }
 static constexpr result_type max() { return Engine::max(); }

 // constructors and seeding functions
 discard_block_engine();
```

```
  explicit discard_block_engine(const Engine& e);
  explicit discard_block_engine(Engine&& e);
  explicit discard_block_engine(result_type s);
  template<class Sseq> explicit discard_block_engine(Sseq& q);
  void seed();
  void seed(result_type s);
  template<class Sseq> void seed(Sseq& q);

  // generating functions
  result_type operator()();
  void discard(unsigned long long z);

  // property functions
  const Engine& base() const noexcept { return e; };

private:
  Engine e;    // exposition only
  int n;       // exposition only
};
```

4 The following relations shall hold: `0 < r` and `r <= p`.

5 The textual representation consists of the textual representation of `e` followed by the value of `n`.

6 In addition to its behavior pursuant to section 26.5.1.5, each constructor that is not a copy constructor sets `n` to 0.

26.5.4.3 Class template `independent_bits_engine` [rand.adapt.ibits]

1 An `independent_bits_engine` random number engine adaptor combines random numbers that are produced by some base engine e, so as to produce random numbers with a specified number of bits w. The state $\mathtt{x}_i$ of an `independent_bits_engine` engine adaptor object `x` consists of the state $\mathtt{e}_i$ of its base engine `e`; the size of the state is the size of e's state.

2 The transition and generation algorithms are described in terms of the following integral constants:

a) Let $R = \mathtt{e.max()} - \mathtt{e.min()} + 1$ and $m = \lfloor \log_2 R \rfloor$.

b) With n as determined below, let $w_0 = \lfloor w/n \rfloor$, $n_0 = n - w \bmod n$, $y_0 = 2^{w_0} \lfloor R/2^{w_0} \rfloor$, and $y_1 = 2^{w_0+1} \lfloor R/2^{w_0+1} \rfloor$.

c) Let $n = \lceil w/m \rceil$ if and only if the relation $R - y_0 \le \lfloor y_0/n \rfloor$ holds as a result. Otherwise let $n = 1 + \lceil w/m \rceil$.

[*Note:* The relation $w = n_0 w_0 + (n - n_0)(w_0 + 1)$ always holds. — *end note*]

3 The transition algorithm is carried out by invoking `e()` as often as needed to obtain n_0 values less than y_0 +`e.min()` and $n - n_0$ values less than $y_1 +$ `e.min()`.

4 The generation algorithm uses the values produced while advancing the state as described above to yield a quantity S obtained as if by the following algorithm:

```
S = 0;
for (k = 0; k ≠ n0; k += 1)  {
 do u = e() - e.min(); while (u ≥ y0);
 S = 2^w0 · S + u mod 2^w0;
}
for (k = n0; k ≠ n; k += 1)  {
```

```
 do u = e() - e.min(); while (u ≥ y1);
 S = 2^(w0+1) · S + u mod 2^(w0+1);
}

template<class Engine, size_t w, class UIntType>
class independent_bits_engine
{
public:
 // types
 typedef UIntType result_type;

 // engine characteristics
 static constexpr result_type min() { return 0; }
 static constexpr result_type max() { return 2^w − 1; }

 // constructors and seeding functions
 independent_bits_engine();
 explicit independent_bits_engine(const Engine& e);
 explicit independent_bits_engine(Engine&& e);
 explicit independent_bits_engine(result_type s);
 template<class Sseq> explicit independent_bits_engine(Sseq& q);
 void seed();
 void seed(result_type s);
 template<class Sseq> void seed(Sseq& q);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);

 // property functions
 const Engine& base() const noexcept { return e; };

private:
 Engine e;    // exposition only
};
```

5 The following relations shall hold: `0 < w` and `w <= numeric_limits<result_type>::digits`.

6 The textual representation consists of the textual representation of `e`.

26.5.4.4 Class template `shuffle_order_engine` [rand.adapt.shuf]

1 A `shuffle_order_engine` random number engine adaptor produces the same random numbers that are produced by some base engine e, but delivers them in a different sequence. The state $\mathtt{x}_i$ of a `shuffle_order_engine` engine adaptor object `x` consists of the state $\mathtt{e}_i$ of its base engine `e`, an additional value Y of the type delivered by `e`, and an additional sequence V of k values also of the type delivered by `e`. The size of the state is the size of e's state plus $k + 1$.

2 The transition algorithm permutes the values produced by e. The state transition is performed as follows:

a) Calculate an integer $j = \left\lfloor \frac{k \cdot (Y - e_{\min})}{e_{\max} - e_{\min} + 1} \right\rfloor$.

b) Set Y to V_j and then set V_j to `e()`.

3 The generation algorithm yields the last value of `Y` produced while advancing `e`'s state as described above.

```
template<class Engine, size_t k>
 class shuffle_order_engine
{
public:
 // types
 typedef typename Engine::result_type result_type;

 // engine characteristics
 static constexpr size_t table_size = k;
 static constexpr result_type min() { return Engine::min(); }
 static constexpr result_type max() { return Engine::max(); }

 // constructors and seeding functions
 shuffle_order_engine();
 explicit shuffle_order_engine(const Engine& e);
 explicit shuffle_order_engine(Engine&& e);
 explicit shuffle_order_engine(result_type s);
 template<class Sseq> explicit shuffle_order_engine(Sseq& q);
 void seed();
 void seed(result_type s);
 template<class Sseq> void seed(Sseq& q);

 // generating functions
 result_type operator()();
 void discard(unsigned long long z);

 // property functions
 const Engine& base() const noexcept { return e; };

private:
 Engine e;           // exposition only
 result_type Y;      // exposition only
 result_type V[k];   // exposition only
};
```

4 The following relation shall hold: `0 < k`.

5 The textual representation consists of the textual representation of `e`, followed by the `k` values of V, followed by the value of Y.

6 In addition to its behavior pursuant to section 26.5.1.5, each constructor that is not a copy constructor initializes `V[0]`, ..., `V[k-1]` and Y, in that order, with values returned by successive invocations of `e()`.

26.5.5 Engines and engine adaptors with predefined parameters [rand.predef]

```
typedef linear_congruential_engine<uint_fast32_t, 16807, 0, 2147483647>
      minstd_rand0;
```

1 *Required behavior:* The 10000th consecutive invocation of a default-constructed object of type `minstd_rand0` shall produce the value 1043618065.

```
typedef linear_congruential_engine<uint_fast32_t, 48271, 0, 2147483647>
      minstd_rand;
```

2 *Required behavior:* The 10000th consecutive invocation of a default-constructed object of type `minstd_rand` shall produce the value 399268537.

```
typedef mersenne_twister_engine<uint_fast32_t,
       32,624,397,31,0x9908b0df,11,0xffffffff,7,0x9d2c5680,15,0xefc60000,18,1812433253>
       mt19937;
```

3 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `mt19937` shall produce the value 4123659995.

```
typedef mersenne_twister_engine<uint_fast64_t,
       64,312,156,31,0xb5026f5aa96619e9,29,
       0x5555555555555555,17,
       0x71d67fffeda60000,37,
       0xfff7eee000000000,43,
       6364136223846793005>
       mt19937_64;
```

4 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `mt19937_64` shall produce the value 9981545732273789042.

```
typedef subtract_with_carry_engine<uint_fast32_t, 24, 10, 24>
       ranlux24_base;
```

5 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `ranlux24_base` shall produce the value 7937952.

```
typedef subtract_with_carry_engine<uint_fast64_t, 48, 5, 12>
       ranlux48_base;
```

6 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `ranlux48_base` shall produce the value 61839128582725.

```
typedef discard_block_engine<ranlux24_base, 223, 23>
       ranlux24;
```

7 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `ranlux24` shall produce the value 9901578.

```
typedef discard_block_engine<ranlux48_base, 389, 11>
       ranlux48;
```

8 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `ranlux48` shall produce the value 249142670248501.

```
typedef shuffle_order_engine<minstd_rand0,256>
       knuth_b;
```

9 *Required behavior:* The 10000 th consecutive invocation of a default-constructed object of type `knuth_b` shall produce the value 1112339016.

```
typedef implementation-defined
       default_random_engine;
```

10 *Remark:* The choice of engine type named by this `typedef` is implementation-defined. [*Note:* The implementation may select this type on the basis of performance, size, quality, or any combination of such factors, so as to provide at least acceptable engine behavior for relatively casual, inexpert, and/or lightweight use. Because different implementations may select different underlying engine types, code that uses this `typedef` need not generate identical sequences across implementations. — *end note*]

26.5.6 Class `random_device` [rand.device]

1 A `random_device` uniform random number generator produces non-deterministic random numbers.

2 If implementation limitations prevent generating non-deterministic random numbers, the implementation may employ a random number engine.

```
class random_device
{
public:
 // types
 typedef unsigned int result_type;

 // generator characteristics
 static constexpr result_type min() { return numeric_limits<result_type>::min(); }
 static constexpr result_type max() { return numeric_limits<result_type>::max(); }

 // constructors
 explicit random_device(const string& token = implementation-defined);

 // generating functions
 result_type operator()();

 // property functions
 double entropy() const noexcept;

 // no copy functions
 random_device(const random_device& ) = delete;
 void operator=(const random_device& ) = delete;
};
```

`explicit random_device(const string& token = implementation-defined);`

3 *Effects:* Constructs a `random_device` non-deterministic uniform random number generator object. The semantics and default value of the `token` parameter are implementation-defined.[278]

4 *Throws:* A value of an implementation-defined type derived from `exception` if the `random_device` could not be initialized.

`double entropy() const noexcept;`

5 *Returns:* If the implementation employs a random number engine, returns 0.0. Otherwise, returns an entropy estimate[279] for the random numbers returned by `operator()`, in the range `min()` to $\log_2(\texttt{max}() + 1)$.

`result_type operator()();`

6 *Returns:* A non-deterministic random value, uniformly distributed between `min()` and `max()`, inclusive. It is implementation-defined how these values are generated.

7 *Throws:* A value of an implementation-defined type derived from `exception` if a random number could not be obtained.

278) The parameter is intended to allow an implementation to differentiate between different sources of randomness.
279) If a device has n states whose respective probabilities are $P_0, \ldots, P_{n-1}$, the device entropy S is defined as $S = -\sum_{i=0}^{n-1} P_i \cdot \log P_i$.

26.5.7 Utilities [rand.util]

26.5.7.1 Class seed_seq [rand.util.seedseq]

```
class seed_seq
{
public:
 // types
 typedef uint_least32_t result_type;

 // constructors
 seed_seq();
 template<class T>
   seed_seq(initializer_list<T> il);
 template<class InputIterator>
   seed_seq(InputIterator begin, InputIterator end);

 // generating functions
 template<class RandomAccessIterator>
   void generate(RandomAccessIterator begin, RandomAccessIterator end);

 // property functions
 size_t size() const noexcept;
 template<class OutputIterator>
   void param(OutputIterator dest) const;

 // no copy functions
 seed_seq(const seed_seq& ) = delete;
 void operator=(const seed_seq& ) = delete;

private:
 vector<result_type> v;    // exposition only
};
```

```
seed_seq();
```

1 *Effects:* Constructs a `seed_seq` object as if by default-constructing its member `v`.

2 *Throws:* Nothing.

```
template<class T>
 seed_seq(initializer_list<T> il);
```

3 *Requires:* `T` shall be an integer type.

4 *Effects:* Same as `seed_seq(il.begin(), il.end())`.

```
template<class InputIterator>
  seed_seq(InputIterator begin, InputIterator end);
```

5 *Requires:* `InputIterator` shall satisfy the requirements of an input iterator (Table 106) type. Moreover, `iterator_traits<InputIterator>::value_type` shall denote an integer type.

6 *Effects:* Constructs a `seed_seq` object by the following algorithm:

```
for( InputIterator s = begin; s != end; ++s)
 v.push_back((*s) mod 2^32);
```

```
template<class RandomAccessIterator>
  void generate(RandomAccessIterator begin, RandomAccessIterator end);
```

7 *Requires:* `RandomAccessIterator` shall meet the requirements of a mutable random access iterator (Table 110) type. Moreover, `iterator_traits<RandomAccessIterator>::value_type` shall denote an unsigned integer type capable of accommodating 32-bit quantities.

8 *Effects:* Does nothing if `begin == end`. Otherwise, with $s =$ `v.size()` and $n =$ `end` $-$ `begin`, fills the supplied range [`begin`, `end`) according to the following algorithm in which each operation is to be carried out modulo 2^{32}, each indexing operator applied to `begin` is to be taken modulo n, and $T(x)$ is defined as x xor (x rshift 27):

a) By way of initialization, set each element of the range to the value `0x8b8b8b8b`. Additionally, for use in subsequent steps, let $p = (n - t)/2$ and let $q = p + t$, where

$$t = (n \geq 623)\ ?\ 11 : (n \geq 68)\ ?\ 7 : (n \geq 39)\ ?\ 5 : (n \geq 7)\ ?\ 3 : (n - 1)/2;$$

b) With m as the larger of $s + 1$ and n, transform the elements of the range: iteratively for $k = 0, \ldots, m - 1$, calculate values

$$\begin{aligned} r_1 &= 1664525 \cdot \mathtt{T}\,(\mathtt{begin}[k]\ \mathsf{xor}\ \mathtt{begin}[k+p]\ \mathsf{xor}\ \mathtt{begin}[k-1]) \\ r_2 &= r_1 + \begin{cases} s & , k = 0 \\ k \bmod n + \mathtt{v}[k-1] & , 0 < k \leq s \\ k \bmod n & , s < k \end{cases} \end{aligned}$$

and, in order, increment `begin`$[k + p]$ by r_1, increment `begin`$[k + q]$ by r_2, and set `begin[k]` to r_2.

c) Transform the elements of the range again, beginning where the previous step ended: iteratively for $k = m, \ldots, m+n-1$, calculate values

$$\begin{aligned} r_3 &= 1566083941 \cdot \mathtt{T}\,(\mathtt{begin}[k] + \mathtt{begin}[k+p] + \mathtt{begin}[k-1]) \\ r_4 &= r_3 - (k \bmod n) \end{aligned}$$

and, in order, update `begin`$[k + p]$ by xoring it with r_3, update `begin`$[k + q]$ by xoring it with r_4, and set `begin`$[k]$ to r_4.

9 *Throws:* What and when `RandomAccessIterator` operations of `begin` and `end` throw.

```
size_t size() const noexcept;
```

10 *Returns:* The number of 32-bit units that would be returned by a call to `param()`.

11 *Complexity:* Constant time.

```
template<class OutputIterator>
  void param(OutputIterator dest) const;
```

12 *Requires:* `OutputIterator` shall satisfy the requirements of an output iterator (Table 107) type. Moreover, the expression `*dest = rt` shall be valid for a value `rt` of type `result_type`.

13 *Effects:* Copies the sequence of prepared 32-bit units to the given destination, as if by executing the following statement:

```
copy(v.begin(), v.end(), dest);
```

14 *Throws:* What and when `OutputIterator` operations of `dest` throw.

26.5.7.2 Function template `generate_canonical` [rand.util.canonical]

1 Each function instantiated from the template described in this section 26.5.7.2 maps the result of one or more invocations of a supplied uniform random number generator `g` to one member of the specified `RealType` such that, if the values g_i produced by `g` are uniformly distributed, the instantiation's results t_j, $0 \le t_j < 1$, are distributed as uniformly as possible as specified below.

2 [*Note:* Obtaining a value in this way can be a useful step in the process of transforming a value generated by a uniform random number generator into a value that can be delivered by a random number distribution. — *end note*]

```
template<class RealType, size_t bits, class URNG>
 RealType generate_canonical(URNG& g);
```

3 *Complexity:* Exactly $k = \max(1, \lceil b/\log_2 R \rceil)$ invocations of `g`, where b[280] is the lesser of `numeric_limits<RealType>::digits` and `bits`, and R is the value of `g.max() - g.min() + 1`.

4 *Effects:* Invokes `g()` k times to obtain values $g_0, \ldots, g_{k-1}$, respectively. Calculates a quantity

$$S = \sum_{i=0}^{k-1} (g_i - \texttt{g.min()}) \cdot R^i$$

using arithmetic of type `RealType`.

5 *Returns:* S/R^k.

6 *Throws:* What and when `g` throws.

26.5.8 Random number distribution class templates [rand.dist]

26.5.8.1 In general [rand.dist.general]

1 Each type instantiated from a class template specified in this section 26.5.8 satisfies the requirements of a random number distribution (26.5.1.6) type.

2 Descriptions are provided in this section 26.5.8 only for distribution operations that are not described in 26.5.1.6 or for operations where there is additional semantic information. In particular, declarations for copy constructors, for copy assignment operators, for streaming operators, and for equality and inequality operators are not shown in the synopses.

3 The algorithms for producing each of the specified distributions are implementation-defined.

4 The value of each probability density function $p(z)$ and of each discrete probability function $P(z_i)$ specified in this section is 0 everywhere outside its stated domain.

26.5.8.2 Uniform distributions [rand.dist.uni]

26.5.8.2.1 Class template `uniform_int_distribution` [rand.dist.uni.int]

1 A `uniform_int_distribution` random number distribution produces random integers i, $a \le i \le b$, distributed according to the constant discrete probability function

$$P(i \mid a, b) = 1/(b - a + 1) \ .$$

```
template<class IntType = int>
 class uniform_int_distribution
{
public:
 // types
```

280) b is introduced to avoid any attempt to produce more bits of randomness than can be held in `RealType`.

```
  typedef IntType result_type;
  typedef unspecified param_type;

  // constructors and reset functions
  explicit uniform_int_distribution(IntType a = 0, IntType b = numeric_limits<IntType>::max());
  explicit uniform_int_distribution(const param_type& parm);
  void reset();

  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  result_type a() const;
  result_type b() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
};
```

```
explicit uniform_int_distribution(IntType a = 0, IntType b = numeric_limits<IntType>::max());
```

2 *Requires:* `a` $\leq$ `b`.

3 *Effects:* Constructs a `uniform_int_distribution` object; `a` and `b` correspond to the respective parameters of the distribution.

```
result_type a() const;
```

4 *Returns:* The value of the `a` parameter with which the object was constructed.

```
result_type b() const;
```

5 *Returns:* The value of the `b` parameter with which the object was constructed.

26.5.8.2.2 Class template `uniform_real_distribution` [rand.dist.uni.real]

1 A `uniform_real_distribution` random number distribution produces random numbers x, $a \leq x < b$, distributed according to the constant probability density function

$$p(x \mid a, b) = 1/(b - a) \ .$$

[*Note:* This implies that $p(x \mid a, b)$ is undefined when `a == b`. — *end note*]

```
template<class RealType = double>
 class uniform_real_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit uniform_real_distribution(RealType a = 0.0, RealType b = 1.0);
 explicit uniform_real_distribution(const param_type& parm);
```

```
  void reset();

  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  result_type a() const;
  result_type b() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
};
```

```
explicit uniform_real_distribution(RealType a = 0.0, RealType b = 1.0);
```

2 *Requires:* `a` $\leq$ `b` and `b` $-$ `a` $\leq$ `numeric_limits<RealType>::max()`.

3 *Effects:* Constructs a `uniform_real_distribution` object; `a` and `b` correspond to the respective parameters of the distribution.

```
result_type a() const;
```

4 *Returns:* The value of the `a` parameter with which the object was constructed.

```
result_type b() const;
```

5 *Returns:* The value of the `b` parameter with which the object was constructed.

26.5.8.3 Bernoulli distributions [rand.dist.bern]

26.5.8.3.1 Class `bernoulli_distribution` [rand.dist.bern.bernoulli]

1 A `bernoulli_distribution` random number distribution produces `bool` values b distributed according to the discrete probability function

$$P(b \,|\, p) = \begin{cases} p & \text{if} \quad b = \texttt{true} \\ 1 - p & \text{if} \quad b = \texttt{false} \end{cases} .$$

```
class bernoulli_distribution
{
public:
 // types
 typedef bool result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit bernoulli_distribution(double p = 0.5);
 explicit bernoulli_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
```

```
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  double p() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
};
```

```
explicit bernoulli_distribution(double p = 0.5);
```

2 *Requires:* $0 \le \mathtt{p} \le 1$.

3 *Effects:* Constructs a `bernoulli_distribution` object; `p` corresponds to the parameter of the distribution.

```
double p() const;
```

4 *Returns:* The value of the `p` parameter with which the object was constructed.

26.5.8.3.2 Class template `binomial_distribution` [rand.dist.bern.bin]

1 A `binomial_distribution` random number distribution produces integer values $i \ge 0$ distributed according to the discrete probability function

$$P(i \mid t, p) = \binom{t}{i} \cdot p^i \cdot (1-p)^{t-i} \, .$$

```
template<class IntType = int>
 class binomial_distribution
{
public:
 // types
 typedef IntType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit binomial_distribution(IntType t = 1, double p = 0.5);
 explicit binomial_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 IntType t() const;
 double p() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit binomial_distribution(IntType t = 1, double p = 0.5);
```

2 *Requires:* $0 \le$ `p` ≤ 1 and $0 \le$ `t`.

3 *Effects:* Constructs a `binomial_distribution` object; `t` and `p` correspond to the respective parameters of the distribution.

```
IntType t() const;
```

4 *Returns:* The value of the `t` parameter with which the object was constructed.

```
double p() const;
```

5 *Returns:* The value of the `p` parameter with which the object was constructed.

26.5.8.3.3 Class template `geometric_distribution` [rand.dist.bern.geo]

1 A `geometric_distribution` random number distribution produces integer values $i \ge 0$ distributed according to the discrete probability function

$$P(i \mid p) = p \cdot (1-p)^i \ .$$

```
template<class IntType = int>
 class geometric_distribution
{
public:
 // types
 typedef IntType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit geometric_distribution(double p = 0.5);
 explicit geometric_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 double p() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
 };
```

```
explicit geometric_distribution(double p = 0.5);
```

2 *Requires:* $0 <$ `p` < 1.

3 *Effects:* Constructs a `geometric_distribution` object; `p` corresponds to the parameter of the distribution.

```
double p() const;
```

4 *Returns:* The value of the `p` parameter with which the object was constructed.

26.5.8.3.4 Class template `negative_binomial_distribution` [rand.dist.bern.negbin]

1 A `negative_binomial_distribution` random number distribution produces random integers $i \geq 0$ distributed according to the discrete probability function

$$P(i \mid k, p) = \binom{k+i-1}{i} \cdot p^k \cdot (1-p)^i \ .$$

[*Note:* This implies that $P(i \mid k, p)$ is undefined when `p == 1`. — *end note*]

```
template<class IntType = int>
 class negative_binomial_distribution
{
public:
 // types
 typedef IntType  result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit negative_binomial_distribution(IntType k = 1, double p = 0.5);
 explicit negative_binomial_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 IntType k() const;
 double p() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit negative_binomial_distribution(IntType k = 1, double p = 0.5);
```

2 *Requires:* $0 < \mathtt{p} \leq 1$ and $0 < \mathtt{k}$.

3 *Effects:* Constructs a `negative_binomial_distribution` object; `k` and `p` correspond to the respective parameters of the distribution.

```
IntType k() const;
```

4 *Returns:* The value of the `k` parameter with which the object was constructed.

```
double p() const;
```

5 *Returns:* The value of the `p` parameter with which the object was constructed.

26.5.8.4 Poisson distributions [rand.dist.pois]

26.5.8.4.1 Class template `poisson_distribution` [rand.dist.pois.poisson]

1 A `poisson_distribution` random number distribution produces integer values $i \geq 0$ distributed according to the discrete probability function

$$P(i \mid \mu) = \frac{e^{-\mu}\mu^i}{i\,!} \ .$$

The distribution parameter μ is also known as this distribution's *mean* .

```
template<class IntType = int>
 class poisson_distribution
{
public:
 // types
 typedef IntType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit poisson_distribution(double mean = 1.0);
 explicit poisson_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 double mean() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit poisson_distribution(double mean = 1.0);
```

2 *Requires:* $0 <$ `mean`.

3 *Effects:* Constructs a `poisson_distribution` object; `mean` corresponds to the parameter of the distribution.

```
double mean() const;
```

4 *Returns:* The value of the `mean` parameter with which the object was constructed.

26.5.8.4.2 Class template `exponential_distribution` [rand.dist.pois.exp]

1 An `exponential_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function

$$p(x \mid \lambda) = \lambda e^{-\lambda x} .$$

```
template<class RealType = double>
 class exponential_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit exponential_distribution(RealType lambda = 1.0);
```

```
  explicit exponential_distribution(const param_type& parm);
  void reset();

  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  RealType lambda() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
};
```

```
explicit exponential_distribution(RealType lambda = 1.0);
```

2 *Requires:* $0 <$ `lambda`.

3 *Effects:* Constructs a `exponential_distribution` object; `lambda` corresponds to the parameter of the distribution.

```
RealType lambda() const;
```

4 *Returns:* The value of the `lambda` parameter with which the object was constructed.

26.5.8.4.3 Class template `gamma_distribution` [rand.dist.pois.gamma]

1 A `gamma_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function

$$p(x \mid \alpha, \beta) = \frac{e^{-x/\beta}}{\beta^{\alpha} \cdot \Gamma(\alpha)} \cdot x^{\alpha-1} .$$

```
template<class RealType = double>
 class gamma_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit gamma_distribution(RealType alpha = 1.0, RealType beta = 1.0);
 explicit gamma_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
```

```
  RealType alpha() const;
  RealType beta() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
 };

explicit gamma_distribution(RealType alpha = 1.0, RealType beta = 1.0);
```

2 *Requires:* $0 <$ `alpha` and $0 <$ `beta`.

3 *Effects:* Constructs a `gamma_distribution` object; `alpha` and `beta` correspond to the parameters of the distribution.

```
RealType alpha() const;
```

4 *Returns:* The value of the `alpha` parameter with which the object was constructed.

```
RealType beta() const;
```

5 *Returns:* The value of the `beta` parameter with which the object was constructed.

26.5.8.4.4 Class template `weibull_distribution` [rand.dist.pois.weibull]

1 A `weibull_distribution` random number distribution produces random numbers $x \geq 0$ distributed according to the probability density function

$$p(x \mid a, b) = \frac{a}{b} \cdot \left(\frac{x}{b}\right)^{a-1} \cdot \exp\left(-\left(\frac{x}{b}\right)^{a}\right) \ .$$

```
template<class RealType = double>
 class weibull_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit weibull_distribution(RealType a = 1.0, RealType b = 1.0);
 explicit weibull_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType a() const;
 RealType b() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit weibull_distribution(RealType a = 1.0, RealType b = 1.0);
```

2 *Requires:* $0 <$ `a` and $0 <$ `b`.

3 *Effects:* Constructs a `weibull_distribution` object; `a` and `b` correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the `a` parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the `b` parameter with which the object was constructed.

26.5.8.4.5 Class template `extreme_value_distribution` [rand.dist.pois.extreme]

1 An `extreme_value_distribution` random number distribution produces random numbers x distributed according to the probability density function[281]

$$p(x \mid a, b) = \frac{1}{b} \cdot \exp\left(\frac{a-x}{b} - \exp\left(\frac{a-x}{b}\right)\right) .$$

```
template<class RealType = double>
 class extreme_value_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit extreme_value_distribution(RealType a = 0.0, RealType b = 1.0);
 explicit extreme_value_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType a() const;
 RealType b() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit extreme_value_distribution(RealType a = 0.0, RealType b = 1.0);
```

281) The distribution corresponding to this probability density function is also known (with a possible change of variable) as the Gumbel Type I, the log-Weibull, or the Fisher-Tippett Type I distribution.

2 *Requires:* 0 < `b`.

3 *Effects:* Constructs an `extreme_value_distribution` object; `a` and `b` correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the `a` parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the `b` parameter with which the object was constructed.

26.5.8.5 Normal distributions [rand.dist.norm]

26.5.8.5.1 Class template `normal_distribution` [rand.dist.norm.normal]

1 A `normal_distribution` random number distribution produces random numbers x distributed according to the probability density function

$$p(x \mid \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \cdot \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

The distribution parameters μ and σ are also known as this distribution's *mean* and *standard deviation* .

```
template<class RealType = double>
 class normal_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructors and reset functions
 explicit normal_distribution(RealType mean = 0.0, RealType stddev = 1.0);
 explicit normal_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType mean() const;
 RealType stddev() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit normal_distribution(RealType mean = 0.0, RealType stddev = 1.0);
```

2 *Requires:* 0 < `stddev`.

3 *Effects:* Constructs a `normal_distribution` object; `mean` and `stddev` correspond to the respective parameters of the distribution.

```
RealType mean() const;
```

4 *Returns:* The value of the `mean` parameter with which the object was constructed.

```
RealType stddev() const;
```

5 *Returns:* The value of the `stddev` parameter with which the object was constructed.

26.5.8.5.2 Class template `lognormal_distribution` [rand.dist.norm.lognormal]

1 A `lognormal_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function

$$p(x \mid m, s) = \frac{1}{sx\sqrt{2\pi}} \cdot \exp\left(-\frac{(\ln x - m)^2}{2s^2}\right).$$

```
template<class RealType = double>
 class lognormal_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit lognormal_distribution(RealType m = 0.0, RealType s = 1.0);
 explicit lognormal_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType m() const;
 RealType s() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit lognormal_distribution(RealType m = 0.0, RealType s = 1.0);
```

2 *Requires:* $0 <$ `s`.

3 *Effects:* Constructs a `lognormal_distribution` object; `m` and `s` correspond to the respective parameters of the distribution.

```
RealType m() const;
```

4 *Returns:* The value of the `m` parameter with which the object was constructed.

```
RealType s() const;
```

5 *Returns:* The value of the `s` parameter with which the object was constructed.

26.5.8.5.3 Class template `chi_squared_distribution` [rand.dist.norm.chisq]

1 A `chi_squared_distribution` random number distribution produces random numbers $x > 0$ distributed according to the probability density function

$$p(x \mid n) = \frac{x^{(n/2)-1} \cdot e^{-x/2}}{\Gamma(n/2) \cdot 2^{n/2}} \ .$$

```
template<class RealType = double>
 class chi_squared_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit chi_squared_distribution(RealType n = 1);
 explicit chi_squared_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType n() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit chi_squared_distribution(RealType n = 1);
```

2 *Requires:* $0 <$ `n`.

3 *Effects:* Constructs a `chi_squared_distribution` object; `n` corresponds to the parameter of the distribution.

```
RealType n() const;
```

4 *Returns:* The value of the `n` parameter with which the object was constructed.

26.5.8.5.4 Class template `cauchy_distribution` [rand.dist.norm.cauchy]

1 A `cauchy_distribution` random number distribution produces random numbers x distributed according to the probability density function

$$p(x \mid a, b) = \left(\pi b \left(1 + \left(\frac{x-a}{b} \right)^2 \right) \right)^{-1} \ .$$

```
template<class RealType = double>
 class cauchy_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit cauchy_distribution(RealType a = 0.0, RealType b = 1.0);
 explicit cauchy_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 RealType a() const;
 RealType b() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
explicit cauchy_distribution(RealType a = 0.0, RealType b = 1.0);
```

2 *Requires:* $0 < \texttt{b}$.

3 *Effects:* Constructs a `cauchy_distribution` object; `a` and `b` correspond to the respective parameters of the distribution.

```
RealType a() const;
```

4 *Returns:* The value of the `a` parameter with which the object was constructed.

```
RealType b() const;
```

5 *Returns:* The value of the `b` parameter with which the object was constructed.

26.5.8.5.5 Class template `fisher_f_distribution` [rand.dist.norm.f]

1 A `fisher_f_distribution` random number distribution produces random numbers $x \geq 0$ distributed according to the probability density function

$$p(x \,|\, m, n) = \frac{\Gamma\big((m+n)/2\big)}{\Gamma(m/2)\,\Gamma(n/2)} \cdot \Big(\frac{m}{n}\Big)^{m/2} \cdot x^{(m/2)-1} \cdot \Big(1 + \frac{mx}{n}\Big)^{-(m+n)/2} .$$

```
template<class RealType = double>
 class fisher_f_distribution
{
public:
 // types
 typedef RealType result_type;
```

```
  typedef unspecified param_type;

  // constructor and reset functions
  explicit fisher_f_distribution(RealType m = 1, RealType n = 1);
  explicit fisher_f_distribution(const param_type& parm);
  void reset();

  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  RealType m() const;
  RealType n() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
};
```

```
explicit fisher_f_distribution(RealType m = 1, RealType n = 1);
```

2 *Requires:* $0 <$ `m` and $0 <$ `n`.

3 *Effects:* Constructs a `fisher_f_distribution` object; `m` and `n` correspond to the respective parameters of the distribution.

```
RealType m() const;
```

4 *Returns:* The value of the `m` parameter with which the object was constructed.

```
RealType n() const;
```

5 *Returns:* The value of the `n` parameter with which the object was constructed.

26.5.8.5.6 Class template `student_t_distribution` [rand.dist.norm.t]

1 A `student_t_distribution` random number distribution produces random numbers x distributed according to the probability density function

$$p(x \mid n) = \frac{1}{\sqrt{n\pi}} \cdot \frac{\Gamma\big((n+1)/2\big)}{\Gamma(n/2)} \cdot \left(1 + \frac{x^2}{n}\right)^{-(n+1)/2} .$$

```
template<class RealType = double>
 class student_t_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 explicit student_t_distribution(RealType n = 1);
 explicit student_t_distribution(const param_type& parm);
 void reset();
```

```
  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  RealType n() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
 };

explicit student_t_distribution(RealType n = 1);
```

2 *Requires:* $0 <$ `n`.

3 *Effects:* Constructs a `student_t_distribution` object; `n` corresponds to the parameter of the distribution.

```
RealType n() const;
```

4 *Returns:* The value of the `n` parameter with which the object was constructed.

26.5.8.6 Sampling distributions [rand.dist.samp]

26.5.8.6.1 Class template `discrete_distribution` [rand.dist.samp.discrete]

1 A `discrete_distribution` random number distribution produces random integers i, $0 \le i < n$, distributed according to the discrete probability function

$$P(i \,|\, p_0, \ldots, p_{n-1}) = p_i \ .$$

2 Unless specified otherwise, the distribution parameters are calculated as: $p_k = w_k/S$ for $k = 0, \ldots, n-1$, in which the values w_k, commonly known as the *weights* , shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold: $0 < S = w_0 + \cdots + w_{n-1}$.

```
template<class IntType = int>
 class discrete_distribution
{
public:
 // types
 typedef IntType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 discrete_distribution();
 template<class InputIterator>
   discrete_distribution(InputIterator firstW, InputIterator lastW);
 discrete_distribution(initializer_list<double> wl);
 template<class UnaryOperation>
   discrete_distribution(size_t nw, double xmin, double xmax, UnaryOperation fw);
 explicit discrete_distribution(const param_type& parm);
 void reset();
```

```
  // generating functions
  template<class URNG>
    result_type operator()(URNG& g);
  template<class URNG>
    result_type operator()(URNG& g, const param_type& parm);

  // property functions
  vector<double> probabilities() const;
  param_type param() const;
  void param(const param_type& parm);
  result_type min() const;
  result_type max() const;
 };

discrete_distribution();
```

3 *Effects:* Constructs a `discrete_distribution` object with $n = 1$ and $p_0 = 1$. [*Note:* Such an object will always deliver the value 0. — *end note*]

```
template<class InputIterator>
  discrete_distribution(InputIterator firstW, InputIterator lastW);
```

4 *Requires:* `InputIterator` shall satisfy the requirements of an input iterator (Table 106) type. Moreover, `iterator_traits<InputIterator>::value_type` shall denote a type that is convertible to `double`. If `firstW == lastW`, let $n = 1$ and $w_0 = 1$. Otherwise, $\big[$`firstW`, `lastW`$\big)$ shall form a sequence w of length $n > 0$.

5 *Effects:* Constructs a `discrete_distribution` object with probabilities given by the formula above.

```
discrete_distribution(initializer_list<double> wl);
```

6 *Effects:* Same as `discrete_distribution(wl.begin(), wl.end())`.

```
template<class UnaryOperation>
  discrete_distribution(size_t nw, double xmin, double xmax, UnaryOperation fw);
```

7 *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.9) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. If `nw` $= 0$, let $n = 1$, otherwise let $n =$ `nw`. The relation $0 < \delta = ($`xmax` $-$ `xmin`$)/n$ shall hold.

8 *Effects:* Constructs a `discrete_distribution` object with probabilities given by the formula above, using the following values: If `nw` $= 0$, let $w_0 = 1$. Otherwise, let $w_k =$ `fw`$($`xmin` $+ k \cdot \delta + \delta/2)$ for $k = 0, \ldots, n-1$.

9 *Complexity:* The number of invocations of `fw` shall not exceed n.

```
vector<double> probabilities() const;
```

10 *Returns:* A `vector<double>` whose `size` member returns n and whose `operator[]` member returns p_k when invoked with argument k for $k = 0, \ldots, n-1$.

26.5.8.6.2 Class template `piecewise_constant_distribution` [rand.dist.samp.pconst]

1 A `piecewise_constant_distribution` random number distribution produces random numbers x, $b_0 \le x < b_n$, uniformly distributed over each subinterval $[b_i, b_{i+1})$ according to the probability density function

$$p(x \mid b_0, \ldots, b_n,\ \rho_0, \ldots, \rho_{n-1}) = \rho_i \text{ , for } b_i \le x < b_{i+1} \text{ .}$$

2 The $n+1$ distribution parameters b_i, also known as this distribution's *interval boundaries* , shall satisfy the relation $b_i < b_{i+1}$ for $i = 0, \ldots, n-1$. Unless specified otherwise, the remaining n distribution parameters are calculated as:

$$\rho_k = \frac{w_k}{S \cdot (b_{k+1} - b_k)} \text{ for } k = 0, \ldots, n-1,$$

in which the values w_k, commonly known as the *weights* , shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold: $0 < S = w_0 + \cdots + w_{n-1}$.

```
template<class RealType = double>
 class piecewise_constant_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 piecewise_constant_distribution();
 template<class InputIteratorB, class InputIteratorW>
   piecewise_constant_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                   InputIteratorW firstW);
 template<class UnaryOperation>
   piecewise_constant_distribution(initializer_list<RealType> bl, UnaryOperation fw);
 template<class UnaryOperation>
   piecewise_constant_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);
 explicit piecewise_constant_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 vector<result_type> intervals() const;
 vector<result_type> densities() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
piecewise_constant_distribution();
```

3 *Effects:* Constructs a `piecewise_constant_distribution` object with $n = 1$, $\rho_0 = 1$, $b_0 = 0$, and $b_1 = 1$.

```
template<class InputIteratorB, class InputIteratorW>
 piecewise_constant_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                 InputIteratorW firstW);
```

4 *Requires:* `InputIteratorB` and `InputIteratorW` shall each satisfy the requirements of an input iterator (Table 106) type. Moreover, `iterator_traits<InputIteratorB>::value_type` and `iterator_traits<InputIteratorW>::value_type` shall each denote a type that is convertible to `double`. If `firstB == lastB` or `++firstB == lastB`, let $n = 1$, $w_0 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise,

$\big[$`firstB`, `lastB`$\big)$ shall form a sequence b of length $n+1$, the length of the sequence w starting from `firstW` shall be at least n, and any w_k for $k \geq n$ shall be ignored by the distribution.

5 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters as specified above.

```
template<class UnaryOperation>
 piecewise_constant_distribution(initializer_list<RealType> bl, UnaryOperation fw);
```

6 *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.9) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter.

7 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters taken or calculated from the following values: If `bl.size()` < 2, let $n = 1$, $w_0 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, let $\big[$`bl.begin()`, `bl.end()`$\big)$ form a sequence $b_0, \ldots, b_n$, and let $w_k =$ `fw`$\big((b_{k+1}+b_k)/2\big)$ for $k = 0, \ldots, n$–1.

8 *Complexity:* The number of invocations of `fw` shall not exceed n.

```
template<class UnaryOperation>
 piecewise_constant_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);
```

9 *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.9) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. If `nw` $= 0$, let $n = 1$, otherwise let $n =$ `nw`. The relation $0 < \delta =$ (`xmax` $-$ `xmin`)$/n$ shall hold.

10 *Effects:* Constructs a `piecewise_constant_distribution` object with parameters taken or calculated from the following values: Let $b_k =$ `xmin`$+k\cdot\delta$ for $k = 0, \ldots, n$, and $w_k =$ `fw`$(b_k+\delta/2)$ for $k = 0, \ldots, n$–1.

11 *Complexity:* The number of invocations of `fw` shall not exceed n.

```
vector<result_type> intervals() const;
```

12 *Returns:* A `vector<result_type>` whose `size` member returns $n+1$ and whose `operator[]` member returns b_k when invoked with argument k for $k = 0, \ldots, n$.

```
vector<result_type> densities() const;
```

13 *Returns:* A `vector<result_type>` whose `size` member returns n and whose `operator[]` member returns ρ_k when invoked with argument k for $k = 0, \ldots, n-1$.

26.5.8.6.3 Class template `piecewise_linear_distribution` [rand.dist.samp.plinear]

1 A `piecewise_linear_distribution` random number distribution produces random numbers x, $b_0 \leq x < b_n$, distributed over each subinterval $[b_i, b_{i+1})$ according to the probability density function

$$p(x \,|\, b_0, \ldots, b_n,\ \rho_0, \ldots, \rho_n) = \rho_i \cdot \frac{b_{i+1} - x}{b_{i+1} - b_i} + \rho_{i+1} \cdot \frac{x - b_i}{b_{i+1} - b_i}\ , \text{ for } b_i \leq x < b_{i+1}\ .$$

2 The $n+1$ distribution parameters b_i, also known as this distribution's *interval boundaries* , shall satisfy the relation $b_i < b_{i+1}$ for $i = 0, \ldots, n-1$. Unless specified otherwise, the remaining $n+1$ distribution parameters are calculated as $\rho_k = w_k/S$ for $k = 0, \ldots, n$, in which the values w_k, commonly known as the *weights at boundaries* , shall be non-negative, non-NaN, and non-infinity. Moreover, the following relation shall hold:

$$0 < S = \frac{1}{2} \cdot \sum_{k=0}^{n-1} (w_k + w_{k+1}) \cdot (b_{k+1} - b_k)\ .$$

```
template<class RealType = double>
 class piecewise_linear_distribution
{
public:
 // types
 typedef RealType result_type;
 typedef unspecified param_type;

 // constructor and reset functions
 piecewise_linear_distribution();
 template<class InputIteratorB, class InputIteratorW>
   piecewise_linear_distribution(InputIteratorB firstB, InputIteratorB lastB,
                                 InputIteratorW firstW);
 template<class UnaryOperation>
   piecewise_linear_distribution(initializer_list<RealType> bl, UnaryOperation fw);
 template<class UnaryOperation>
   piecewise_linear_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);
 explicit piecewise_linear_distribution(const param_type& parm);
 void reset();

 // generating functions
 template<class URNG>
   result_type operator()(URNG& g);
 template<class URNG>
   result_type operator()(URNG& g, const param_type& parm);

 // property functions
 vector<result_type> intervals() const;
 vector<result_type> densities() const;
 param_type param() const;
 void param(const param_type& parm);
 result_type min() const;
 result_type max() const;
};
```

```
piecewise_linear_distribution();
```

3 *Effects:* Constructs a `piecewise_linear_distribution` object with $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$.

```
template<class InputIteratorB, class InputIteratorW>
 piecewise_linear_distribution(InputIteratorB firstB, InputIteratorB lastB,
                               InputIteratorW firstW);
```

4 *Requires:* `InputIteratorB` and `InputIteratorW` shall each satisfy the requirements of an input iterator (Table 106) type. Moreover, `iterator_traits<InputIteratorB>::value_type` and `iterator_traits<InputIteratorW>::value_type` shall each denote a type that is convertible to `double`. If `firstB == lastB` or `++firstB == lastB`, let $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, $\big[$`firstB`, `lastB`$\big)$ shall form a sequence b of length $n + 1$, the length of the sequence w starting from `firstW` shall be at least $n + 1$, and any w_k for $k \geq n + 1$ shall be ignored by the distribution.

5 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters as specified above.

```
template<class UnaryOperation>
 piecewise_linear_distribution(initializer_list<RealType> bl, UnaryOperation fw);
```

6 *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.9) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter.

7 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters taken or calculated from the following values: If `bl.size()` < 2, let $n = 1$, $\rho_0 = \rho_1 = 1$, $b_0 = 0$, and $b_1 = 1$. Otherwise, let $\big($`bl.begin(),bl.end()`$\big)$ form a sequence $b_0, \ldots, b_n$, and let $w_k =$ `fw`(b_k) for $k = 0, \ldots, n$.

8 *Complexity:* The number of invocations of `fw` shall not exceed $n + 1$.

```
template<class UnaryOperation>
 piecewise_linear_distribution(size_t nw, RealType xmin, RealType xmax, UnaryOperation fw);
```

9 *Requires:* Each instance of type `UnaryOperation` shall be a function object (20.9) whose return type shall be convertible to `double`. Moreover, `double` shall be convertible to the type of `UnaryOperation`'s sole parameter. If `nw` $= 0$, let $n = 1$, otherwise let $n =$ `nw`. The relation $0 < \delta =$ (`xmax` $-$ `xmin`)$/n$ shall hold.

10 *Effects:* Constructs a `piecewise_linear_distribution` object with parameters taken or calculated from the following values: Let $b_k =$ `xmin` $+ k \cdot \delta$ for $k = 0, \ldots, n$, and $w_k =$ `fw`(b_k) for $k = 0, \ldots, n$.

11 *Complexity:* The number of invocations of `fw` shall not exceed $n + 1$.

```
vector<result_type> intervals() const;
```

12 *Returns:* A `vector<result_type>` whose `size` member returns $n+1$ and whose `operator[]` member returns b_k when invoked with argument k for $k = 0, \ldots, n$.

```
vector<result_type> densities() const;
```

13 *Returns:* A `vector<result_type>` whose `size` member returns n and whose `operator[]` member returns ρ_k when invoked with argument k for $k = 0, \ldots, n$.

26.6 Numeric arrays [numarray]

26.6.1 Header <valarray> synopsis [valarray.syn]

```
#include <initializer_list>

namespace std {

  template<class T> class valarray;         // An array of type T
  class slice;                              // a BLAS-like slice out of an array
  template<class T> class slice_array;
  class gslice;                             // a generalized slice out of an array
  template<class T> class gslice_array;
  template<class T> class mask_array;       // a masked array
  template<class T> class indirect_array;   // an indirected array

  template<class T> void swap(valarray<T>&, valarray<T>&) noexcept;

  template<class T> valarray<T> operator* (const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> operator* (const valarray<T>&, const T&);
  template<class T> valarray<T> operator* (const T&, const valarray<T>&);

  template<class T> valarray<T> operator/ (const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> operator/ (const valarray<T>&, const T&);
  template<class T> valarray<T> operator/ (const T&, const valarray<T>&);
```

```
template<class T> valarray<T> operator% (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&, const T&);
template<class T> valarray<T> operator% (const T&, const valarray<T>&);

template<class T> valarray<T> operator+ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&, const T&);
template<class T> valarray<T> operator+ (const T&, const valarray<T>&);

template<class T> valarray<T> operator- (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&, const T&);
template<class T> valarray<T> operator- (const T&, const valarray<T>&);

template<class T> valarray<T> operator^ (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&, const T&);
template<class T> valarray<T> operator^ (const T&, const valarray<T>&);

template<class T> valarray<T> operator& (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&, const T&);
template<class T> valarray<T> operator& (const T&, const valarray<T>&);

template<class T> valarray<T> operator| (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&, const T&);
template<class T> valarray<T> operator| (const T&, const valarray<T>&);

template<class T> valarray<T> operator<<(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator<<(const valarray<T>&, const T&);
template<class T> valarray<T> operator<<(const T&, const valarray<T>&);

template<class T> valarray<T> operator>>(const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator>>(const valarray<T>&, const T&);
template<class T> valarray<T> operator>>(const T&, const valarray<T>&);

template<class T> valarray<bool> operator&&(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&, const T&);
template<class T> valarray<bool> operator&&(const T&, const valarray<T>&);

template<class T> valarray<bool> operator||(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator||(const valarray<T>&, const T&);
template<class T> valarray<bool> operator||(const T&, const valarray<T>&);

template<class T>
  valarray<bool> operator==(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator==(const valarray<T>&, const T&);
template<class T> valarray<bool> operator==(const T&, const valarray<T>&);
template<class T>
  valarray<bool> operator!=(const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&, const T&);
template<class T> valarray<bool> operator!=(const T&, const valarray<T>&);

template<class T>
  valarray<bool> operator< (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&, const T&);
template<class T> valarray<bool> operator< (const T&, const valarray<T>&);
template<class T>
  valarray<bool> operator> (const valarray<T>&, const valarray<T>&);
```

```
  template<class T> valarray<bool> operator> (const valarray<T>&, const T&);
  template<class T> valarray<bool> operator> (const T&, const valarray<T>&);
  template<class T>
    valarray<bool> operator<=(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator<=(const valarray<T>&, const T&);
  template<class T> valarray<bool> operator<=(const T&, const valarray<T>&);
  template<class T>
    valarray<bool> operator>=(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<bool> operator>=(const valarray<T>&, const T&);
  template<class T> valarray<bool> operator>=(const T&, const valarray<T>&);

  template<class T> valarray<T> abs  (const valarray<T>&);
  template<class T> valarray<T> acos (const valarray<T>&);
  template<class T> valarray<T> asin (const valarray<T>&);
  template<class T> valarray<T> atan (const valarray<T>&);

  template<class T> valarray<T> atan2(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> atan2(const valarray<T>&, const T&);
  template<class T> valarray<T> atan2(const T&, const valarray<T>&);

  template<class T> valarray<T> cos  (const valarray<T>&);
  template<class T> valarray<T> cosh (const valarray<T>&);
  template<class T> valarray<T> exp  (const valarray<T>&);
  template<class T> valarray<T> log  (const valarray<T>&);
  template<class T> valarray<T> log10(const valarray<T>&);

  template<class T> valarray<T> pow(const valarray<T>&, const valarray<T>&);
  template<class T> valarray<T> pow(const valarray<T>&, const T&);
  template<class T> valarray<T> pow(const T&, const valarray<T>&);

  template<class T> valarray<T> sin  (const valarray<T>&);
  template<class T> valarray<T> sinh (const valarray<T>&);
  template<class T> valarray<T> sqrt (const valarray<T>&);
  template<class T> valarray<T> tan  (const valarray<T>&);
  template<class T> valarray<T> tanh (const valarray<T>&);

  template <class T> unspecified1 begin(valarray<T>& v);
  template <class T> unspecified2 begin(const valarray<T>& v);
  template <class T> unspecified1 end(valarray<T>& v);
  template <class T> unspecified2 end(const valarray<T>& v);
}
```

1 The header `<valarray>` defines five class templates (`valarray`, `slice_array`, `gslice_array`, `mask_array`, and `indirect_array`), two classes (`slice` and `gslice`), and a series of related function templates for representing and manipulating arrays of values.

2 The `valarray` array classes are defined to be free of certain forms of aliasing, thus allowing operations on these classes to be optimized.

3 Any function returning a `valarray<T>` is permitted to return an object of another type, provided all the const member functions of `valarray<T>` are also applicable to this type. This return type shall not add more than two levels of template nesting over the most deeply nested argument type.[282]

282) Annex B recommends a minimum number of recursively nested template instantiations. This requirement thus indirectly suggests a minimum allowable complexity for valarray expressions.

4 Implementations introducing such replacement types shall provide additional functions and operators as follows:

(4.1) — for every function taking a `const valarray<T>&` other than `begin` and `end` (26.6.10), identical functions taking the replacement types shall be added;

(4.2) — for every function taking two `const valarray<T>&` arguments, identical functions taking every combination of `const valarray<T>&` and replacement types shall be added.

5 In particular, an implementation shall allow a `valarray<T>` to be constructed from such replacement types and shall allow assignments and computed assignments of such types to `valarray<T>`, `slice_array<T>`, `gslice_array<T>`, `mask_array<T>` and `indirect_array<T>` objects.

6 These library functions are permitted to throw a `bad_alloc` (18.6.2.1) exception if there are not sufficient resources available to carry out the operation. Note that the exception is not mandated.

26.6.2 Class template `valarray` [template.valarray]

26.6.2.1 Class template `valarray` overview [template.valarray.overview]

```
namespace std {
  template<class T> class valarray {
  public:
    typedef T value_type;

    // 26.6.2.2 construct/destroy:
    valarray();
    explicit valarray(size_t);
    valarray(const T&, size_t);
    valarray(const T*, size_t);
    valarray(const valarray&);
    valarray(valarray&&) noexcept;
    valarray(const slice_array<T>&);
    valarray(const gslice_array<T>&);
    valarray(const mask_array<T>&);
    valarray(const indirect_array<T>&);
    valarray(initializer_list<T>);
    ~valarray();

    // 26.6.2.3 assignment:
    valarray& operator=(const valarray&);
    valarray& operator=(valarray&&) noexcept;
    valarray& operator=(initializer_list<T>);
    valarray& operator=(const T&);
    valarray& operator=(const slice_array<T>&);
    valarray& operator=(const gslice_array<T>&);
    valarray& operator=(const mask_array<T>&);
    valarray& operator=(const indirect_array<T>&);

    // 26.6.2.4 element access:
    const T&          operator[](size_t) const;
    T&                operator[](size_t);

    // 26.6.2.5 subset operations:
    valarray          operator[](slice) const;
    slice_array<T>    operator[](slice);
    valarray          operator[](const gslice&) const;
```

```
    gslice_array<T>   operator[](const gslice&);
    valarray          operator[](const valarray<bool>&) const;
    mask_array<T>     operator[](const valarray<bool>&);
    valarray          operator[](const valarray<size_t>&) const;
    indirect_array<T> operator[](const valarray<size_t>&);

    // 26.6.2.6 unary operators:
    valarray operator+() const;
    valarray operator-() const;
    valarray operator~() const;
    valarray<bool> operator!() const;

    // 26.6.2.7 computed assignment:
    valarray& operator*= (const T&);
    valarray& operator/= (const T&);
    valarray& operator%= (const T&);
    valarray& operator+= (const T&);
    valarray& operator-= (const T&);
    valarray& operator^= (const T&);
    valarray& operator&= (const T&);
    valarray& operator|= (const T&);
    valarray& operator<<=(const T&);
    valarray& operator>>=(const T&);

    valarray& operator*= (const valarray&);
    valarray& operator/= (const valarray&);
    valarray& operator%= (const valarray&);
    valarray& operator+= (const valarray&);
    valarray& operator-= (const valarray&);
    valarray& operator^= (const valarray&);
    valarray& operator|= (const valarray&);
    valarray& operator&= (const valarray&);
    valarray& operator<<=(const valarray&);
    valarray& operator>>=(const valarray&);

    // 26.6.2.8 member functions:
    void swap(valarray&) noexcept;

    size_t size() const;

    T sum() const;
    T min() const;
    T max() const;

    valarray shift (int) const;
    valarray cshift(int) const;
    valarray apply(T func(T)) const;
    valarray apply(T func(const T&)) const;
    void resize(size_t sz, T c = T());
  };
}
```

1 The class template `valarray<T>` is a one-dimensional smart array, with elements numbered sequentially from zero. It is a representation of the mathematical concept of an ordered set of values. The illusion of higher dimensionality may be produced by the familiar idiom of computed indices, together with the

powerful subsetting capabilities provided by the generalized subscript operators.[283]

2 An implementation is permitted to qualify any of the functions declared in `<valarray>` as `inline`.

26.6.2.2 `valarray` constructors [valarray.cons]

`valarray();`

1 *Effects:* Constructs an object of class `valarray<T>`[284] which has zero length.[285]

`explicit valarray(size_t);`

2 The array created by this constructor has a length equal to the value of the argument. The elements of the array are value-initialized (8.5).

`valarray(const T&, size_t);`

3 The array created by this constructor has a length equal to the second argument. The elements of the array are initialized with the value of the first argument.

`valarray(const T*, size_t);`

4 The array created by this constructor has a length equal to the second argument `n`. The values of the elements of the array are initialized with the first `n` values pointed to by the first argument.[286] If the value of the second argument is greater than the number of values pointed to by the first argument, the behavior is undefined.

`valarray(const valarray&);`

5 The array created by this constructor has the same length as the argument array. The elements are initialized with the values of the corresponding elements of the argument array.[287]

`valarray(valarray&& v) noexcept;`

6 The array created by this constructor has the same length as the argument array. The elements are initialized with the values of the corresponding elements of the argument array.

7 *Complexity:* Constant.

`valarray(initializer_list<T> il);`

8 *Effects:* Same as `valarray(il.begin(), il.size())`.

```
valarray(const slice_array<T>&);
valarray(const gslice_array<T>&);
valarray(const mask_array<T>&);
valarray(const indirect_array<T>&);
```

9 These conversion constructors convert one of the four reference templates to a `valarray`.

`~valarray();`

10 The destructor is applied to every element of `*this`; an implementation may return all allocated memory.

---

283) The intent is to specify an array template that has the minimum functionality necessary to address aliasing ambiguities and the proliferation of temporaries. Thus, the `valarray` template is neither a matrix class nor a field class. However, it is a very useful building block for designing such classes.

284) For convenience, such objects are referred to as "arrays" throughout the remainder of 26.6.

285) This default constructor is essential, since arrays of `valarray` may be useful. After initialization, the length of an empty array can be increased with the `resize` member function.

286) This constructor is the preferred method for converting a C array to a `valarray` object.

287) This copy constructor creates a distinct array rather than an alias. Implementations in which arrays share storage are permitted, but they shall implement a copy-on-reference mechanism to ensure that arrays are conceptually distinct.

26.6.2.3 `valarray` assignment [valarray.assign]

```
valarray& operator=(const valarray& v);
```

1 Each element of the `*this` array is assigned the value of the corresponding element of the argument array. If the length of `v` is not equal to the length of `*this` , resizes `*this` to make the two arrays the same length, as if by calling `resize(v.size())`, before performing the assignment.

2 *Postcondition:* `size() == v.size()`.

```
valarray& operator=(valarray&& v) noexcept;
```

3 *Effects:* `*this` obtains the value of `v`. The value of `v` after the assignment is not specified.

4 *Complexity:* Linear.

```
valarray& operator=(initializer_list<T> il);
```

5 *Effects:* `*this = valarray(il)`.

6 *Returns:* `*this`.

```
valarray& operator=(const T&);
```

7 The scalar assignment operator causes each element of the `*this` array to be assigned the value of the argument.

```
valarray& operator=(const slice_array<T>&);
valarray& operator=(const gslice_array<T>&);
valarray& operator=(const mask_array<T>&);
valarray& operator=(const indirect_array<T>&);
```

8 *Requires:* The length of the array to which the argument refers equals `size()`.

9 These operators allow the results of a generalized subscripting operation to be assigned directly to a `valarray`.

10 If the value of an element in the left-hand side of a valarray assignment operator depends on the value of another element in that left-hand side, the resulting behavior is undefined.

26.6.2.4 `valarray` element access [valarray.access]

```
const T&  operator[](size_t) const;
T& operator[](size_t);
```

1 The subscript operator returns a reference to the corresponding element of the array.

2 Thus, the expression `(a[i] = q, a[i]) == q` evaluates as true for any non-constant `valarray<T> a`, any `T q`, and for any `size_t i` such that the value of `i` is less than the length of `a`.

3 The expression `&a[i+j] == &a[i] + j` evaluates as true for all `size_t i` and `size_t j` such that `i+j` is less than the length of the array `a`.

4 Likewise, the expression `&a[i] != &b[j]` evaluates as `true` for any two arrays `a` and `b` and for any `size_t i` and `size_t j` such that `i` is less than the length of `a` and `j` is less than the length of `b`. This property indicates an absence of aliasing and may be used to advantage by optimizing compilers.[288]

5 The reference returned by the subscript operator for an array shall be valid until the member function `resize(size_t, T)` (26.6.2.8) is called for that array or until the lifetime of that array ends, whichever happens first.

6 If the subscript operator is invoked with a `size_t` argument whose value is not less than the length of the array, the behavior is undefined.

288) Compilers may take advantage of inlining, constant propagation, loop fusion, tracking of pointers obtained from `operator new`, and other techniques to generate efficient `valarrays`.

26.6.2.5 `valarray` subset operations [valarray.sub]

1 The member `operator[]` is overloaded to provide several ways to select sequences of elements from among those controlled by `*this`. Each of these operations returns a subset of the array. The const-qualified versions return this subset as a new `valarray` object. The non-const versions return a class template object which has reference semantics to the original array, working in conjunction with various overloads of `operator=` and other assigning operators to allow selective replacement (slicing) of the controlled sequence. In each case the selected element(s) must exist.

```
valarray operator[](slice slicearr) const;
```

2 *Returns:* An object of class `valarray<T>` containing those elements of the controlled sequence designated by `slicearr`. [*Example:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
// v0[slice(2, 5, 3)] returns valarray<char>("cfilo", 5)
```

— *end example*]

```
slice_array<T> operator[](slice slicearr);
```

3 *Returns:* An object that holds references to elements of the controlled sequence selected by `slicearr`. [*Example:*

```
valarray<char> v0("abcdefghijklmnop", 16);
valarray<char> v1("ABCDE", 5);
v0[slice(2, 5, 3)] = v1;
// v0 == valarray<char>("abAdeBghCjkDmnEp", 16);
```

— *end example*]

```
valarray operator[](const gslice& gslicearr) const;
```

4 *Returns:* An object of class `valarray<T>` containing those elements of the controlled sequence designated by `gslicearr`. [*Example:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
const size_t lv[] = { 2, 3 };
const size_t dv[] = { 7, 2 };
const valarray<size_t> len(lv, 2), str(dv, 2);
// v0[gslice(3, len, str)] returns
// valarray<char>("dfhkmo", 6)
```

— *end example*]

```
gslice_array<T> operator[](const gslice& gslicearr);
```

5 *Returns:* An object that holds references to elements of the controlled sequence selected by `gslicearr`. [*Example:*

```
valarray<char> v0("abcdefghijklmnop", 16);
valarray<char> v1("ABCDE", 5);
const size_t lv[] = { 2, 3 };
const size_t dv[] = { 7, 2 };
const valarray<size_t> len(lv, 2), str(dv, 2);
v0[gslice(3, len, str)] = v1;
// v0 == valarray<char>("abcAeBgCijDlEnFp", 16)
```

— *end example*]

```
valarray operator[](const valarray<bool>& boolarr) const;
```

6 *Returns:* An object of class `valarray<T>` containing those elements of the controlled sequence designated by `boolarr`. [*Example:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
const bool vb[] = { false, false, true, true, false, true };
// v0[valarray<bool>(vb, 6)] returns
// valarray<char>("cdf", 3)
```

— *end example*]

```
mask_array<T> operator[](const valarray<bool>& boolarr);
```

7 *Returns:* An object that holds references to elements of the controlled sequence selected by `boolarr`. [*Example:*

```
valarray<char> v0("abcdefghijklmnop", 16);
valarray<char> v1("ABC", 3);
const bool vb[] = { false, false, true, true, false, true };
v0[valarray<bool>(vb, 6)] = v1;
// v0 == valarray<char>("abABeCghijklmnop", 16)
```

— *end example*]

```
valarray operator[](const valarray<size_t>& indarr) const;
```

8 *Returns:* An object of class `valarray<T>` containing those elements of the controlled sequence designated by `indarr`. [*Example:*

```
const valarray<char> v0("abcdefghijklmnop", 16);
const size_t vi[] = { 7, 5, 2, 3, 8 };
// v0[valarray<size_t>(vi, 5)] returns
// valarray<char>("hfcdi", 5)
```

— *end example*]

```
indirect_array<T> operator[](const valarray<size_t>& indarr);
```

9 *Returns:* An object that holds references to elements of the controlled sequence selected by `indarr`. [*Example:*

```
valarray<char> v0("abcdefghijklmnop", 16);
valarray<char> v1("ABCDE", 5);
const size_t vi[] = { 7, 5, 2, 3, 8 };
v0[valarray<size_t>(vi, 5)] = v1;
// v0 == valarray<char>("abCDeBgAEjklmnop", 16)
```

— *end example*]

26.6.2.6 `valarray` unary operators [valarray.unary]

```
valarray operator+() const;
valarray operator-() const;
valarray operator~() const;
valarray<bool> operator!() const;
```

1 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied and for which the indicated operator returns a value which is of type `T` (`bool` for `operator!`) or which may be unambiguously implicitly converted to type `T` (`bool` for `operator!`).

2 Each of these operators returns an array whose length is equal to the length of the array. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array.

26.6.2.7 `valarray` computed assignment [valarray.cassign]

```
valarray& operator*= (const valarray&);
valarray& operator/= (const valarray&);
valarray& operator%= (const valarray&);
valarray& operator+= (const valarray&);
valarray& operator-= (const valarray&);
valarray& operator^= (const valarray&);
valarray& operator&= (const valarray&);
valarray& operator|= (const valarray&);
valarray& operator<<=(const valarray&);
valarray& operator>>=(const valarray&);
```

1 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied. Each of these operators performs the indicated operation on each of its elements and the corresponding element of the argument array.

2 The array is then returned by reference.

3 If the array and the argument array do not have the same length, the behavior is undefined. The appearance of an array on the left-hand side of a computed assignment does `not` invalidate references or pointers.

4 If the value of an element in the left-hand side of a valarray computed assignment operator depends on the value of another element in that left hand side, the resulting behavior is undefined.

```
valarray& operator*= (const T&);
valarray& operator/= (const T&);
valarray& operator%= (const T&);
valarray& operator+= (const T&);
valarray& operator-= (const T&);
valarray& operator^= (const T&);
valarray& operator&= (const T&);
valarray& operator|= (const T&);
valarray& operator<<=(const T&);
valarray& operator>>=(const T&);
```

5 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied.

6 Each of these operators applies the indicated operation to each element of the array and the non-array argument.

7 The array is then returned by reference.

8 The appearance of an array on the left-hand side of a computed assignment does *not* invalidate references or pointers to the elements of the array.

26.6.2.8 `valarray` member functions [valarray.members]

```
void swap(valarray& v) noexcept;
```

1 *Effects:* `*this` obtains the value of `v`. `v` obtains the value of `*this`.

2 *Complexity:* Constant.

```
size_t size() const;
```

3 *Returns:* The number of elements in the array.

4 *Complexity:* Constant time.

```
T sum() const;
```

This function may only be instantiated for a type `T` to which `operator+=` can be applied. This function returns the sum of all the elements of the array.

5 If the array has length 0, the behavior is undefined. If the array has length 1, `sum()` returns the value of element 0. Otherwise, the returned value is calculated by applying `operator+=` to a copy of an element of the array and all other elements of the array in an unspecified order.

```
T min() const;
```

6 This function returns the minimum value contained in `*this`. The value returned for an array of length 0 is undefined. For an array of length 1, the value of element 0 is returned. For all other array lengths, the determination is made using `operator<`.

```
T max() const;
```

7 This function returns the maximum value contained in `*this`. The value returned for an array of length 0 is undefined. For an array of length 1, the value of element 0 is returned. For all other array lengths, the determination is made using `operator<`.

```
valarray shift(int n) const;
```

8 This function returns an object of class `valarray<T>` of length `size()`, each of whose elements I is `(*this)[I + n]` if `I + n` is non-negative and less than `size()`, otherwise `T()`. Thus if element zero is taken as the leftmost element, a positive value of `n` shifts the elements left `n` places, with zero fill.

9 [*Example:* If the argument has the value -2, the first two elements of the result will be value-initialized (8.5); the third element of the result will be assigned the value of the first element of the argument; etc. — *end example*]

```
valarray cshift(int n) const;
```

10 This function returns an object of class `valarray<T>` of length `size()` that is a circular shift of `*this`. If element zero is taken as the leftmost element, a non-negative value of n shifts the elements circularly left n places and a negative value of n shifts the elements circularly right -n places.

```
valarray apply(T func(T)) const;
valarray apply(T func(const T&)) const;
```

11 These functions return an array whose length is equal to the array. Each element of the returned array is assigned the value returned by applying the argument function to the corresponding element of the array.

```
void resize(size_t sz, T c = T());
```

12 This member function changes the length of the `*this` array to `sz` and then assigns to each element the value of the second argument. Resizing invalidates all pointers and references to elements in the array.

26.6.3 `valarray` non-member operations [valarray.nonmembers]

26.6.3.1 `valarray` binary operators [valarray.binary]

```
template<class T> valarray<T> operator*
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator/
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator%
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator+
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator-
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator^
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator&
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator|
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator<<
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> operator>>
    (const valarray<T>&, const valarray<T>&);
```

1 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied and for which the indicated operator returns a value which is of type `T` or which can be unambiguously implicitly converted to type `T`.

2 Each of these operators returns an array whose length is equal to the lengths of the argument arrays. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding elements of the argument arrays.

3 If the argument arrays do not have the same length, the behavior is undefined.

```
template<class T> valarray<T> operator* (const valarray<T>&, const T&);
template<class T> valarray<T> operator* (const T&, const valarray<T>&);
template<class T> valarray<T> operator/ (const valarray<T>&, const T&);
template<class T> valarray<T> operator/ (const T&, const valarray<T>&);
template<class T> valarray<T> operator% (const valarray<T>&, const T&);
template<class T> valarray<T> operator% (const T&, const valarray<T>&);
template<class T> valarray<T> operator+ (const valarray<T>&, const T&);
template<class T> valarray<T> operator+ (const T&, const valarray<T>&);
template<class T> valarray<T> operator- (const valarray<T>&, const T&);
template<class T> valarray<T> operator- (const T&, const valarray<T>&);
template<class T> valarray<T> operator^ (const valarray<T>&, const T&);
template<class T> valarray<T> operator^ (const T&, const valarray<T>&);
template<class T> valarray<T> operator& (const valarray<T>&, const T&);
template<class T> valarray<T> operator& (const T&, const valarray<T>&);
template<class T> valarray<T> operator| (const valarray<T>&, const T&);
template<class T> valarray<T> operator| (const T&, const valarray<T>&);
template<class T> valarray<T> operator<<(const valarray<T>&, const T&);
template<class T> valarray<T> operator<<(const T&, const valarray<T>&);
template<class T> valarray<T> operator>>(const valarray<T>&, const T&);
template<class T> valarray<T> operator>>(const T&, const valarray<T>&);
```

4 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied and for which the indicated operator returns a value which is of type `T` or which can be

unambiguously implicitly converted to type `T`.

5 Each of these operators returns an array whose length is equal to the length of the array argument. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array argument and the non-array argument.

26.6.3.2 `valarray` logical operators [valarray.comparison]

```
template<class T> valarray<bool> operator==
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator!=
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator<
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator>
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator<=
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator>=
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator&&
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<bool> operator||
    (const valarray<T>&, const valarray<T>&);
```

1 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied and for which the indicated operator returns a value which is of type `bool` or which can be unambiguously implicitly converted to type `bool`.

2 Each of these operators returns a `bool` array whose length is equal to the length of the array arguments. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding elements of the argument arrays.

3 If the two array arguments do not have the same length, the behavior is undefined.

```
template<class T> valarray<bool> operator==(const valarray<T>&, const T&);
template<class T> valarray<bool> operator==(const T&, const valarray<T>&);
template<class T> valarray<bool> operator!=(const valarray<T>&, const T&);
template<class T> valarray<bool> operator!=(const T&, const valarray<T>&);
template<class T> valarray<bool> operator< (const valarray<T>&, const T&);
template<class T> valarray<bool> operator< (const T&, const valarray<T>&);
template<class T> valarray<bool> operator> (const valarray<T>&, const T&);
template<class T> valarray<bool> operator> (const T&, const valarray<T>&);
template<class T> valarray<bool> operator<=(const valarray<T>&, const T&);
template<class T> valarray<bool> operator<=(const T&, const valarray<T>&);
template<class T> valarray<bool> operator>=(const valarray<T>&, const T&);
template<class T> valarray<bool> operator>=(const T&, const valarray<T>&);
template<class T> valarray<bool> operator&&(const valarray<T>&, const T&);
template<class T> valarray<bool> operator&&(const T&, const valarray<T>&);
template<class T> valarray<bool> operator||(const valarray<T>&, const T&);
template<class T> valarray<bool> operator||(const T&, const valarray<T>&);
```

4 Each of these operators may only be instantiated for a type `T` to which the indicated operator can be applied and for which the indicated operator returns a value which is of type `bool` or which can be unambiguously implicitly converted to type `bool`.

5 Each of these operators returns a `bool` array whose length is equal to the length of the array argument. Each element of the returned array is initialized with the result of applying the indicated operator to the corresponding element of the array and the non-array argument.

26.6.3.3 `valarray` transcendentals [valarray.transcend]

```
template<class T> valarray<T> abs  (const valarray<T>&);
template<class T> valarray<T> acos (const valarray<T>&);
template<class T> valarray<T> asin (const valarray<T>&);
template<class T> valarray<T> atan (const valarray<T>&);
template<class T> valarray<T> atan2
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> atan2(const valarray<T>&, const T&);
template<class T> valarray<T> atan2(const T&, const valarray<T>&);
template<class T> valarray<T> cos  (const valarray<T>&);
template<class T> valarray<T> cosh (const valarray<T>&);
template<class T> valarray<T> exp  (const valarray<T>&);
template<class T> valarray<T> log  (const valarray<T>&);
template<class T> valarray<T> log10(const valarray<T>&);
template<class T> valarray<T> pow
    (const valarray<T>&, const valarray<T>&);
template<class T> valarray<T> pow  (const valarray<T>&, const T&);
template<class T> valarray<T> pow  (const T&, const valarray<T>&);
template<class T> valarray<T> sin  (const valarray<T>&);
template<class T> valarray<T> sinh (const valarray<T>&);
template<class T> valarray<T> sqrt (const valarray<T>&);
template<class T> valarray<T> tan  (const valarray<T>&);
template<class T> valarray<T> tanh (const valarray<T>&);
```

1 Each of these functions may only be instantiated for a type `T` to which a unique function with the indicated name can be applied (unqualified). This function shall return a value which is of type `T` or which can be unambiguously implicitly converted to type `T`.

26.6.3.4 `valarray` specialized algorithms [valarray.special]

```
template <class T> void swap(valarray<T>& x, valarray<T>& y) noexcept;
```

1 *Effects:* `x.swap(y)`.

26.6.4 Class `slice` [class.slice]

26.6.4.1 Class `slice` overview [class.slice.overview]

```
namespace std {
  class slice {
  public:
    slice();
    slice(size_t, size_t, size_t);

    size_t start() const;
    size_t size() const;
    size_t stride() const;
  };
}
```

1 The `slice` class represents a BLAS-like slice from an array. Such a slice is specified by a starting index, a length, and a stride.[289]

26.6.4.2 `slice` constructors [cons.slice]

289) BLAS stands for *Basic Linear Algebra Subprograms.* C++ programs may instantiate this class. See, for example, Dongarra, Du Croz, Duff, and Hammerling: *A set of Level 3 Basic Linear Algebra Subprograms*; Technical Report MCS-P1-0888, Argonne National Laboratory (USA), Mathematics and Computer Science Division, August, 1988.

```
slice();
slice(size_t start, size_t length, size_t stride);
slice(const slice&);
```

1 The default constructor is equivalent to `slice(0, 0, 0)`. A default constructor is provided only to permit the declaration of arrays of slices. The constructor with arguments for a slice takes a start, length, and stride parameter.

2 [*Example:* `slice(3, 8, 2)` constructs a slice which selects elements 3, 5, 7, ... 17 from an array. — *end example*]

26.6.4.3 `slice` access functions [slice.access]

```
size_t start() const;
size_t size() const;
size_t stride() const;
```

1 *Returns:* The start, length, or stride specified by a `slice` object.

2 *Complexity:* Constant time.

26.6.5 Class template `slice_array` [template.slice.array]

26.6.5.1 Class template `slice_array` overview [template.slice.array.overview]

```
namespace std {
  template <class T> class slice_array {
  public:
    typedef T value_type;

    void operator=  (const valarray<T>&) const;
    void operator*= (const valarray<T>&) const;
    void operator/= (const valarray<T>&) const;
    void operator%= (const valarray<T>&) const;
    void operator+= (const valarray<T>&) const;
    void operator-= (const valarray<T>&) const;
    void operator^= (const valarray<T>&) const;
    void operator&= (const valarray<T>&) const;
    void operator|= (const valarray<T>&) const;
    void operator<<=(const valarray<T>&) const;
    void operator>>=(const valarray<T>&) const;

    slice_array(const slice_array&);
    ~slice_array();
    const slice_array& operator=(const slice_array&) const;
  void operator=(const T&) const;

    slice_array() = delete;        // as implied by declaring copy constructor above
  };
}
```

1 The `slice_array` template is a helper template used by the `slice` subscript operator

```
slice_array<T> valarray<T>::operator[](slice);
```

It has reference semantics to a subset of an array specified by a `slice` object.

2 [*Example:* The expression `a[slice(1, 5, 3)] = b;` has the effect of assigning the elements of `b` to a slice of the elements in `a`. For the slice shown, the elements selected from `a` are 1, 4, ..., 13. — *end example*]

26.6.5.2 `slice_array` assignment [slice.arr.assign]

```
void operator=(const valarray<T>&) const;
const slice_array& operator=(const slice_array&) const;
```

1 These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which the `slice_array` object refers.

26.6.5.3 `slice_array` computed assignment [slice.arr.comp.assign]

```
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%= (const valarray<T>&) const;
void operator+= (const valarray<T>&) const;
void operator-= (const valarray<T>&) const;
void operator^= (const valarray<T>&) const;
void operator&= (const valarray<T>&) const;
void operator|= (const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;
```

1 These computed assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `slice_array` object refers.

26.6.5.4 `slice_array` fill function [slice.arr.fill]

```
void operator=(const T&) const;
```

1 This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `slice_array` object refers.

26.6.6 The `gslice` class [class.gslice]

26.6.6.1 The `gslice` class overview [class.gslice.overview]

```
namespace std {
  class gslice {
  public:
    gslice();
    gslice(size_t s, const valarray<size_t>& l, const valarray<size_t>& d);

    size_t           start() const;
    valarray<size_t> size() const;
    valarray<size_t> stride() const;
  };
}
```

1 This class represents a generalized slice out of an array. A `gslice` is defined by a starting offset (s), a set of lengths (l_j), and a set of strides (d_j). The number of lengths shall equal the number of strides.

2 A `gslice` represents a mapping from a set of indices (i_j), equal in number to the number of strides, to a single index k. It is useful for building multidimensional array classes using the `valarray` template, which is one-dimensional. The set of one-dimensional index values specified by a `gslice` are

$$k = s + \sum_j i_j d_j$$

where the multidimensional indices i_j range in value from 0 to $l_{ij} - 1$.

3 [*Example:* The `gslice` specification

```
start  = 3
length = {2, 4, 3}
stride = {19, 4, 1}
```

yields the sequence of one-dimensional indices

$$k = 3 + (0,1) \times 19 + (0,1,2,3) \times 4 + (0,1,2) \times 1$$

which are ordered as shown in the following table:

$$\begin{array}{cccc} (i_0, & i_1, & i_2, & k) = \\ (0, & 0, & 0, & 3), \\ (0, & 0, & 1, & 4), \\ (0, & 0, & 2, & 5), \\ (0, & 1, & 0, & 7), \\ (0, & 1, & 1, & 8), \\ (0, & 1, & 2, & 9), \\ (0, & 2, & 0, & 11), \\ (0, & 2, & 1, & 12), \\ (0, & 2, & 2, & 13), \\ (0, & 3, & 0, & 15), \\ (0, & 3, & 1, & 16), \\ (0, & 3, & 2, & 17), \\ (1, & 0, & 0, & 22), \\ (1, & 0, & 1, & 23), \\ \dots & & & \\ (1, & 3, & 2, & 36) \end{array}$$

That is, the highest-ordered index turns fastest. — *end example*]

4 It is possible to have degenerate generalized slices in which an address is repeated.

5 [*Example:* If the stride parameters in the previous example are changed to {1, 1, 1}, the first few elements of the resulting sequence of indices will be

$$\begin{array}{cccc} (0, & 0, & 0, & 3), \\ (0, & 0, & 1, & 4), \\ (0, & 0, & 2, & 5), \\ (0, & 1, & 0, & 4), \\ (0, & 1, & 1, & 5), \\ (0, & 1, & 2, & 6), \\ \dots & & & \end{array}$$

— *end example*]

6 If a degenerate slice is used as the argument to the non-`const` version of `operator[](const gslice&)`, the resulting behavior is undefined.

26.6.6.2 `gslice` constructors [gslice.cons]

```
gslice();
gslice(size_t start, const valarray<size_t>& lengths,
         const valarray<size_t>& strides);
gslice(const gslice&);
```

1 The default constructor is equivalent to `gslice(0, valarray<size_t>(), valarray<size_t>())`. The constructor with arguments builds a `gslice` based on a specification of start, lengths, and strides, as explained in the previous section.

26.6.6.3 `gslice` access functions [gslice.access]

```
size_t           start()  const;
valarray<size_t> size() const;
valarray<size_t> stride() const;
```

1 *Returns:* The representation of the start, lengths, or strides specified for the `gslice`.

2 *Complexity:* `start()` is constant time. `size()` and `stride()` are linear in the number of strides.

26.6.7 Class template `gslice_array` [template.gslice.array]

26.6.7.1 Class template `gslice_array` overview [template.gslice.array.overview]

```
namespace std {
  template <class T> class gslice_array {
  public:
    typedef T value_type;

    void operator=  (const valarray<T>&) const;
    void operator*= (const valarray<T>&) const;
    void operator/= (const valarray<T>&) const;
    void operator%= (const valarray<T>&) const;
    void operator+= (const valarray<T>&) const;
    void operator-= (const valarray<T>&) const;
    void operator^= (const valarray<T>&) const;
    void operator&= (const valarray<T>&) const;
    void operator|= (const valarray<T>&) const;
    void operator<<=(const valarray<T>&) const;
    void operator>>=(const valarray<T>&) const;

    gslice_array(const gslice_array&);
    ~gslice_array();
    const gslice_array& operator=(const gslice_array&) const;
    void operator=(const T&) const;

    gslice_array() = delete;       // as implied by declaring copy constructor above
  };
}
```

1 This template is a helper template used by the `slice` subscript operator

```
gslice_array<T> valarray<T>::operator[](const gslice&);
```

2 It has reference semantics to a subset of an array specified by a `gslice` object.

3 Thus, the expression `a[gslice(1, length, stride)] = b` has the effect of assigning the elements of `b` to a generalized slice of the elements in `a`.

26.6.7.2 `gslice_array` assignment [gslice.array.assign]

```
void operator=(const valarray<T>&) const;
const gslice_array& operator=(const gslice_array&) const;
```

1 These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which the `gslice_array` refers.

26.6.7.3 gslice_array [gslice.array.comp.assign]

```
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%= (const valarray<T>&) const;
void operator+= (const valarray<T>&) const;
void operator-= (const valarray<T>&) const;
void operator^= (const valarray<T>&) const;
void operator&= (const valarray<T>&) const;
void operator|= (const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;
```

1 These computed assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `gslice_array` object refers.

26.6.7.4 gslice_array fill function [gslice.array.fill]

```
void operator=(const T&) const;
```

1 This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `gslice_array` object refers.

26.6.8 Class template mask_array [template.mask.array]

26.6.8.1 Class template mask_array overview [template.mask.array.overview]

```
namespace std {
  template <class T> class mask_array {
  public:
    typedef T value_type;

    void operator=  (const valarray<T>&) const;
    void operator*= (const valarray<T>&) const;
    void operator/= (const valarray<T>&) const;
    void operator%= (const valarray<T>&) const;
    void operator+= (const valarray<T>&) const;
    void operator-= (const valarray<T>&) const;
    void operator^= (const valarray<T>&) const;
    void operator&= (const valarray<T>&) const;
    void operator|= (const valarray<T>&) const;
    void operator<<=(const valarray<T>&) const;
    void operator>>=(const valarray<T>&) const;

    mask_array(const mask_array&);
   ~mask_array();
    const mask_array& operator=(const mask_array&) const;
    void operator=(const T&) const;

    mask_array() = delete;          // as implied by declaring copy constructor above
  };
}
```

1 This template is a helper template used by the mask subscript operator:

`mask_array<T> valarray<T>::operator[](const valarray<bool>&).`

2 It has reference semantics to a subset of an array specified by a boolean mask. Thus, the expression `a[mask] = b;` has the effect of assigning the elements of `b` to the masked elements in `a` (those for which the corresponding element in `mask` is `true`.)

26.6.8.2 `mask_array` assignment [mask.array.assign]

```
void operator=(const valarray<T>&) const;
const mask_array& operator=(const mask_array&) const;
```

1 These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which it refers.

26.6.8.3 `mask_array` computed assignment [mask.array.comp.assign]

```
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%= (const valarray<T>&) const;
void operator+= (const valarray<T>&) const;
void operator-= (const valarray<T>&) const;
void operator^= (const valarray<T>&) const;
void operator&= (const valarray<T>&) const;
void operator|= (const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;
```

1 These computed assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the mask object refers.

26.6.8.4 `mask_array` fill function [mask.array.fill]

```
void operator=(const T&) const;
```

1 This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `mask_array` object refers.

26.6.9 Class template `indirect_array` [template.indirect.array]

26.6.9.1 Class template `indirect_array` overview [template.indirect.array.overview]

```
namespace std {
  template <class T> class indirect_array {
  public:
    typedef T value_type;

    void operator=  (const valarray<T>&) const;
    void operator*= (const valarray<T>&) const;
    void operator/= (const valarray<T>&) const;
    void operator%= (const valarray<T>&) const;
    void operator+= (const valarray<T>&) const;
    void operator-= (const valarray<T>&) const;
    void operator^= (const valarray<T>&) const;
    void operator&= (const valarray<T>&) const;
    void operator|= (const valarray<T>&) const;
    void operator<<=(const valarray<T>&) const;
    void operator>>=(const valarray<T>&) const;

    indirect_array(const indirect_array&);
   ~indirect_array();
    const indirect_array& operator=(const indirect_array&) const;
```

```
    void operator=(const T&) const;

    indirect_array() = delete;          // as implied by declaring copy constructor above
  };
}
```

1 This template is a helper template used by the indirect subscript operator

`indirect_array<T> valarray<T>::operator[](const valarray<size_t>&).`

2 It has reference semantics to a subset of an array specified by an `indirect_array`. Thus the expression `a[indirect] = b;` has the effect of assigning the elements of `b` to the elements in `a` whose indices appear in `indirect`.

26.6.9.2 `indirect_array` assignment [indirect.array.assign]

```
void operator=(const valarray<T>&) const;
const indirect_array& operator=(const indirect_array&) const;
```

1 These assignment operators have reference semantics, assigning the values of the argument array elements to selected elements of the `valarray<T>` object to which it refers.

2 If the `indirect_array` specifies an element in the `valarray<T>` object to which it refers more than once, the behavior is undefined.

3 [*Example:*

```
int addr[] = {2, 3, 1, 4, 4};
valarray<size_t> indirect(addr, 5);
valarray<double> a(0., 10), b(1., 5);
a[indirect] = b;
```

results in undefined behavior since element 4 is specified twice in the indirection. — *end example*]

26.6.9.3 `indirect_array` computed assignment [indirect.array.comp.assign]

```
void operator*= (const valarray<T>&) const;
void operator/= (const valarray<T>&) const;
void operator%= (const valarray<T>&) const;
void operator+= (const valarray<T>&) const;
void operator-= (const valarray<T>&) const;
void operator^= (const valarray<T>&) const;
void operator&= (const valarray<T>&) const;
void operator|= (const valarray<T>&) const;
void operator<<=(const valarray<T>&) const;
void operator>>=(const valarray<T>&) const;
```

1 These computed assignments have reference semantics, applying the indicated operation to the elements of the argument array and selected elements of the `valarray<T>` object to which the `indirect_array` object refers.

2 If the `indirect_array` specifies an element in the `valarray<T>` object to which it refers more than once, the behavior is undefined.

26.6.9.4 `indirect_array` fill function [indirect.array.fill]

```
void operator=(const T&) const;
```

1 This function has reference semantics, assigning the value of its argument to the elements of the `valarray<T>` object to which the `indirect_array` object refers.

26.6.10 valarray range access [valarray.range]

1 In the `begin` and `end` function templates that follow, *unspecified*1 is a type that meets the requirements of a mutable random access iterator (24.2.7) and of a contiguous iterator (24.2.1) whose `value_type` is the template parameter `T` and whose `reference` type is `T&`. *unspecified*2 is a type that meets the requirements of a constant random access iterator (24.2.7) and of a contiguous iterator (24.2.1) whose `value_type` is the template parameter `T` and whose `reference` type is `const T&`.

2 The iterators returned by `begin` and `end` for an array are guaranteed to be valid until the member function `resize(size_t, T)` (26.6.2.8) is called for that array or until the lifetime of that array ends, whichever happens first.

```
template <class T> unspecified1 begin(valarray<T>& v);
template <class T> unspecified2 begin(const valarray<T>& v);
```

3 *Returns:* An iterator referencing the first value in the numeric array.

```
template <class T> unspecified1 end(valarray<T>& v);
template <class T> unspecified2 end(const valarray<T>& v);
```

4 *Returns:* An iterator referencing one past the last value in the numeric array.

26.7 Generalized numeric operations [numeric.ops]

26.7.1 Header <numeric> synopsis [numeric.ops.overview]

```
namespace std {
  template <class InputIterator, class T>
    T accumulate(InputIterator first, InputIterator last, T init);
  template <class InputIterator, class T, class BinaryOperation>
    T accumulate(InputIterator first, InputIterator last, T init,
                 BinaryOperation binary_op);

  template <class InputIterator1, class InputIterator2, class T>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, T init);
  template <class InputIterator1, class InputIterator2, class T,
            class BinaryOperation1, class BinaryOperation2>
    T inner_product(InputIterator1 first1, InputIterator1 last1,
                    InputIterator2 first2, T init,
                    BinaryOperation1 binary_op1,
                    BinaryOperation2 binary_op2);

  template <class InputIterator, class OutputIterator>
    OutputIterator partial_sum(InputIterator first,
                               InputIterator last,
                               OutputIterator result);
  template <class InputIterator, class OutputIterator,
            class BinaryOperation>
    OutputIterator partial_sum(InputIterator first,
                               InputIterator last,
                               OutputIterator result,
                               BinaryOperation binary_op);

  template <class InputIterator, class OutputIterator>
    OutputIterator adjacent_difference(InputIterator first,
                                       InputIterator last,
                                       OutputIterator result);
```

```
  template <class InputIterator, class OutputIterator,
            class BinaryOperation>
    OutputIterator adjacent_difference(InputIterator first,
                                       InputIterator last,
                                       OutputIterator result,
                                       BinaryOperation binary_op);

  template <class ForwardIterator, class T>
    void iota(ForwardIterator first, ForwardIterator last, T value);
}
```

1 The requirements on the types of algorithms' arguments that are described in the introduction to Clause 25 also apply to the following algorithms.

26.7.2 Accumulate [accumulate]

```
template <class InputIterator, class T>
  T accumulate(InputIterator first, InputIterator last, T init);
template <class InputIterator, class T, class BinaryOperation>
  T accumulate(InputIterator first, InputIterator last, T init,
               BinaryOperation binary_op);
```

1 *Effects:* Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifies it with `acc = acc + *i` or `acc = binary_op(acc, *i)` for every iterator `i` in the range `[first,last)` in order.[290]

2 *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types. In the range `[first,last]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.[291]

26.7.3 Inner product [inner.product]

```
template <class InputIterator1, class InputIterator2, class T>
  T inner_product(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, T init);
template <class InputIterator1, class InputIterator2, class T,
    class BinaryOperation1, class BinaryOperation2>
  T inner_product(InputIterator1 first1, InputIterator1 last1,
                  InputIterator2 first2, T init,
                  BinaryOperation1 binary_op1,
                  BinaryOperation2 binary_op2);
```

1 *Effects:* Computes its result by initializing the accumulator `acc` with the initial value `init` and then modifying it with `acc = acc + (*i1) * (*i2)` or `acc = binary_op1(acc, binary_op2(*i1, *i2))` for every iterator `i1` in the range `[first1,last1)` and iterator `i2` in the range `[first2,first2 + (last1 - first1))` in order.

2 *Requires:* `T` shall meet the requirements of `CopyConstructible` (Table 21) and `CopyAssignable` (Table 23) types. In the ranges `[first1,last1]` and `[first2,first2 + (last1 - first1)]` `binary_op1` and `binary_op2` shall neither modify elements nor invalidate iterators or subranges.[292]

26.7.4 Partial sum [partial.sum]

290) `accumulate` is similar to the APL reduction operator and Common Lisp reduce function, but it avoids the difficulty of defining the result of reduction on an empty sequence by always requiring an initial value.
291) The use of fully closed ranges is intentional
292) The use of fully closed ranges is intentional

```
template <class InputIterator, class OutputIterator>
  OutputIterator partial_sum(
    InputIterator first, InputIterator last,
    OutputIterator result);
template
  <class InputIterator, class OutputIterator, class BinaryOperation>
    OutputIterator partial_sum(
      InputIterator first, InputIterator last,
      OutputIterator result, BinaryOperation binary_op);
```

1 *Effects:* For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1,last)` in order, `acc` is then modified by `acc = acc + *i` or `acc = binary_op(acc, *i)` and the result is assigned to `*(result + (i - first))`.

2 *Returns:* `result + (last - first)`.

3 *Complexity:* Exactly `(last - first) - 1` applications of the binary operation.

4 *Requires:* `InputIterator`'s value type shall be constructible from the type of `*first`. The result of the expression `acc + *i` or `binary_op(acc, *i)` shall be implicitly convertible to `InputIterator`'s value type. `acc` shall be writable to the `result` output iterator. In the ranges `[first,last]` and `[result,result + (last - first)]` `binary_op` shall neither modify elements nor invalidate iterators or subranges.[293]

5 *Remarks:* `result` may be equal to `first`.

26.7.5 Adjacent difference [adjacent.difference]

```
template <class InputIterator, class OutputIterator>
  OutputIterator adjacent_difference(
    InputIterator first, InputIterator last,
    OutputIterator result);
template <class InputIterator, class OutputIterator, class BinaryOperation>
  OutputIterator adjacent_difference(
    InputIterator first, InputIterator last,
    OutputIterator result,
    BinaryOperation binary_op);
```

1 *Effects:* For a non-empty range, the function creates an accumulator `acc` whose type is `InputIterator`'s value type, initializes it with `*first`, and assigns the result to `*result`. For every iterator `i` in `[first + 1,last)` in order, creates an object `val` whose type is `InputIterator`'s value type, initializes it with `*i`, computes `val - acc` or `binary_op(val, acc)`, assigns the result to `*(result + (i - first))`, and move assigns from `val` to `acc`.

2 *Requires:* `InputIterator`'s value type shall be `MoveAssignable` (Table 22) and shall be constructible from the type of `*first`. `acc` shall be writable to the `result` output iterator. The result of the expression `val - acc` or `binary_op(val, acc)` shall be writable to the `result` output iterator. In the ranges `[first,last]` and `[result,result + (last - first)]`, `binary_op` shall neither modify elements nor invalidate iterators or subranges.[294]

3 *Remarks:* `result` may be equal to `first`.

4 *Returns:* `result + (last - first)`.

5 *Complexity:* Exactly `(last - first) - 1` applications of the binary operation.

293) The use of fully closed ranges is intentional.
294) The use of fully closed ranges is intentional.

26.7.6 Iota [numeric.iota]

```
template <class ForwardIterator, class T>
  void iota(ForwardIterator first, ForwardIterator last, T value);
```

1 *Requires:* `T` shall be convertible to `ForwardIterator`'s value type. The expression `++val`, where `val` has type `T`, shall be well formed.

2 *Effects:* For each element referred to by the iterator `i` in the range `[first,last)`, assigns `*i = value` and increments `value` as if by `++value`.

3 *Complexity:* Exactly `last - first` increments and assignments.

26.8 C library [c.math]

1 The header `<ctgmath>` simply includes the headers `<ccomplex>` and `<cmath>`.

2 [*Note:* The overloads provided in C by type-generic macros are already provided in `<ccomplex>` and `<cmath>` by "sufficient" additional overloads. — *end note*]

3 Tables 118 and 119 describe headers `<cmath>` and `<cstdlib>`, respectively.

Table 118 — Header `<cmath>` synopsis

Type	Name(s)			
Macros:				
FP_FAST_FMA	FP_ILOGBNAN	FP_SUBNORMAL	HUGE_VALL	MATH_ERRNO
FP_FAST_FMAF	FP_INFINITE	FP_ZERO	INFINITY	MATH_ERREXCEPT
FP_FAST_FMAL	FP_NAN	HUGE_VAL	NAN	math_errhandling
FP_ILOGB0	FP_NORMAL	HUGE_VALF		
Types:	double_t	float_t		
Math Functions:				
abs	cosh	fmod	logb	remquo
acos	erf	frexp	lrint	rint
acosh	erfc	hypot	lround	round
asin	exp2	ilogb	modf	scalbln
asinh	exp	ldexp	nan	scalbn
atan	expm1	lgamma	nanf	sin
atan2	fabs	llrint	nanl	sinh
atanh	fdim	llround	nearbyint	sqrt
cbrt	floor	log	nextafter	tan
ceil	fma	log10	nexttoward	tanh
copysign	fmax	log1p	pow	tgamma
cos	fmin	log2	remainder	trunc
Classification/comparison Functions:				
fpclassify	isgreaterequal	islessequal	isnan	isunordered
isfinite	isinf	islessgreater	isnormal	signbit
isgreater	isless			

4 The contents of these headers are the same as the Standard C library headers `<math.h>` and `<stdlib.h>` respectively, with the following changes:

5 The `rand` function has the semantics specified in the C standard, except that the implementation may specify that particular library functions may call `rand`. It is implementation-defined whether the `rand` function may introduce data races (17.6.5.9). [*Note:* The random number generation (26.5) facilities in this standard are

Table 119 — Header `<cstdlib>` synopsis

Type	Name(s)	
Macro:	`RAND_MAX`	
Types:		
`div_t`	`ldiv_t`	`lldiv_t`
Functions:		
`abs`	`ldiv`	`rand`
`div`	`llabs`	`srand`
`labs`	`lldiv`	

often preferable to `rand`, because `rand`'s underlying algorithm is unspecified. Use of `rand` therefore continues to be nonportable, with unpredictable and oft-questionable quality and performance. — *end note*]

6 In addition to the `int` versions of certain math functions in `<cstdlib>`, C++ adds `long` and `long long` overloaded versions of these functions, with the same semantics.

7 The added signatures are:

```
long abs(long);                     // labs()
long long abs(long long);           // llabs()
ldiv_t div(long, long);             // ldiv()
lldiv_t div(long long, long long);  // lldiv()
```

8 In addition to the `double` versions of the math functions in `<cmath>`, C++ adds `float` and `long double` overloaded versions of these functions, with the same semantics.

9 The added signatures are:

```
float abs(float);
float acos(float);
float acosh(float);
float asin(float);
float asinh(float);
float atan(float);
float atan2(float, float);
float atanh(float);
float cbrt(float);
float ceil(float);
float copysign(float, float);
float cos(float);
float cosh(float);
float erf(float);
float erfc(float);
float exp(float);
float exp2(float);
float expm1(float);
float fabs(float);
float fdim(float, float);
float floor(float);
float fma(float, float, float);
float fmax(float, float);
float fmin(float, float);
float fmod(float, float);
float frexp(float, int*);
float hypot(float, float);
```

```
int ilogb(float);
float ldexp(float, int);
float lgamma(float);
long long llrint(float);
long long llround(float);
float log(float);
float log10(float);
float log1p(float);
float log2(float);
float logb(float);
long lrint(float);
long lround(float);
float modf(float, float*);
float nearbyint(float);
float nextafter(float, float);
float nexttoward(float, long double);
float pow(float, float);
float remainder(float, float);
float remquo(float, float, int *);
float rint(float);
float round(float);
float scalbln(float, long);
float scalbn(float, int);
float sin(float);
float sinh(float);
float sqrt(float);
float tan(float);
float tanh(float);
float tgamma(float);
float trunc(float);

double abs(double);            // fabs()

long double abs(long double);
long double acos(long double);
long double acosh(long double);
long double asin(long double);
long double asinh(long double);
long double atan(long double);
long double atan2(long double, long double);
long double atanh(long double);
long double cbrt(long double);
long double ceil(long double);
long double copysign(long double, long double);
long double cos(long double);
long double cosh(long double);
long double erf(long double);
long double erfc(long double);
long double exp(long double);
long double exp2(long double);
long double expm1(long double);
long double fabs(long double);
long double fdim(long double, long double);
long double floor(long double);
long double fma(long double, long double, long double);
```

```
long double fmax(long double, long double);
long double fmin(long double, long double);
long double fmod(long double, long double);
long double frexp(long double, int*);
long double hypot(long double, long double);
int ilogb(long double);
long double ldexp(long double, int);
long double lgamma(long double);
long long llrint(long double);
long long llround(long double);
long double log(long double);
long double log10(long double);
long double log1p(long double);
long double log2(long double);
long double logb(long double);
long lrint(long double);
long lround(long double);
long double modf(long double, long double*);
long double nearbyint(long double);
long double nextafter(long double, long double);
long double nexttoward(long double, long double);
long double pow(long double, long double);
long double remainder(long double, long double);
long double remquo(long double, long double, int *);
long double rint(long double);
long double round(long double);
long double scalbln(long double, long);
long double scalbn(long double, int);
long double sin(long double);
long double sinh(long double);
long double sqrt(long double);
long double tan(long double);
long double tanh(long double);
long double tgamma(long double);
long double trunc(long double);
```

10 The classification/comparison functions behave the same as the C macros with the corresponding names defined in 7.12.3, Classification macros, and 7.12.14, Comparison macros in the C Standard. Each function is overloaded for the three floating-point types, as follows:

```
int fpclassify(float x);
bool isfinite(float x);
bool isinf(float x);
bool isnan(float x);
bool isnormal(float x);
bool signbit(float x);

bool isgreater(float x, float y);
bool isgreaterequal(float x, float y);
bool isless(float x, float y);
bool islessequal(float x, float y);
bool islessgreater(float x, float y);
bool isunordered(float x, float y);

int fpclassify(double x);
bool isfinite(double x);
```

```
bool isinf(double x);
bool isnan(double x);
bool isnormal(double x);
bool signbit(double x);

bool isgreater(double x, double y);
bool isgreaterequal(double x, double y);
bool isless(double x, double y);
bool islessequal(double x, double y);
bool islessgreater(double x, double y);
bool isunordered(double x, double y);

int fpclassify(long double x);
bool isfinite(long double x);
bool isinf(long double x);
bool isnan(long double x);
bool isnormal(long double x);
bool signbit(long double x);

bool isgreater(long double x, long double y);
bool isgreaterequal(long double x, long double y);
bool isless(long double x, long double y);
bool islessequal(long double x, long double y);
bool islessgreater(long double x, long double y);
bool isunordered(long double x, long double y);
```

11 Moreover, there shall be additional overloads sufficient to ensure:

1. If any argument of arithmetic type corresponding to a `double` parameter has type `long double`, then all arguments of arithmetic type (3.9.1) corresponding to `double` parameters are effectively cast to `long double`.
2. Otherwise, if any argument of arithmetic type corresponding to a `double` parameter has type `double` or an integer type, then all arguments of arithmetic type corresponding to `double` parameters are effectively cast to `double`.
3. Otherwise, all arguments of arithmetic type corresponding to `double` parameters have type `float`.

See also: ISO C 7.5, 7.10.2, 7.10.6.

27 Input/output library [input.output]

27.1 General [input.output.general]

1 This Clause describes components that C++ programs may use to perform input/output operations.

2 The following subclauses describe requirements for stream parameters, and components for forward declarations of iostreams, predefined iostreams objects, base iostreams classes, stream buffering, stream formatting and manipulators, string streams, and file streams, as summarized in Table 120.

Table 120 — Input/output library summary

Subclause		Header(s)
27.2	Requirements	
27.3	Forward declarations	`<iosfwd>`
27.4	Standard iostream objects	`<iostream>`
27.5	Iostreams base classes	`<ios>`
27.6	Stream buffers	`<streambuf>`

27.7	Formatting and manipulators	`<istream>`
		`<ostream>`
		`<iomanip>`

27.8	String streams	`<sstream>`

27.9	File streams	`<fstream>`
		`<cstdio>`
		`<cinttypes>`

3 Figure 7 illustrates relationships among various types described in this clause. A line from **A** to **B** indicates that **A** is an alias (e.g. a typedef) for **B** or that **A** is defined in terms of **B**.

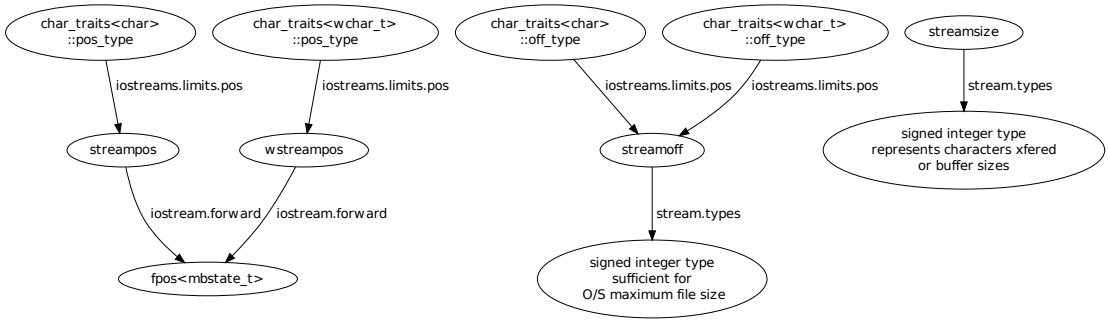


Figure 7 — Stream position, offset, and size types [non-normative]

27.2 Iostreams requirements [iostreams.requirements]

27.2.1 Imbue limitations [iostream.limits.imbue]

1 No function described in Clause 27 except for `ios_base::imbue` and `basic_filebuf::pubimbue` causes any instance of `basic_ios::imbue` or `basic_streambuf::imbue` to be called. If any user function called from a function declared in Clause 27 or as an overriding virtual function of any class declared in Clause 27 calls `imbue`, the behavior is undefined.

27.2.2 Positioning type limitations [iostreams.limits.pos]

1 The classes of Clause 27 with template arguments `charT` and `traits` behave as described if `traits::pos_type` and `traits::off_type` are `streampos` and `streamoff` respectively. Except as noted explicitly below, their behavior when `traits::pos_type` and `traits::off_type` are other types is implementation-defined.

2 In the classes of Clause 27, a template parameter with name `charT` represents a member of the set of types containing `char`, `wchar_t`, and any other implementation-defined character types that satisfy the requirements for a character on which any of the iostream components can be instantiated.

27.2.3 Thread safety [iostreams.threadsafety]

1 Concurrent access to a stream object (27.8, 27.9), stream buffer object (27.6), or C Library stream (27.9.2) by multiple threads may result in a data race (1.10) unless otherwise specified (27.4). [*Note:* Data races result in undefined behavior (1.10). — *end note*]

2 If one thread makes a library call a that writes a value to a stream and, as a result, another thread reads this value from the stream through a library call b such that this does not result in a data race, then a's write synchronizes with b's read.

27.3 Forward declarations [iostream.forward]

Header `<iosfwd>` synopsis

```
namespace std {
  template<class charT> class char_traits;
  template<> class char_traits<char>;
  template<> class char_traits<char16_t>;
  template<> class char_traits<char32_t>;
  template<> class char_traits<wchar_t>;

  template<class T> class allocator;

  template <class charT, class traits = char_traits<charT> >
    class basic_ios;
  template <class charT, class traits = char_traits<charT> >
    class basic_streambuf;
  template <class charT, class traits = char_traits<charT> >
    class basic_istream;
  template <class charT, class traits = char_traits<charT> >
    class basic_ostream;
  template <class charT, class traits = char_traits<charT> >
    class basic_iostream;

  template <class charT, class traits = char_traits<charT>,
      class Allocator = allocator<charT> >
    class basic_stringbuf;
  template <class charT, class traits = char_traits<charT>,
      class Allocator = allocator<charT> >
    class basic_istringstream;
```

```
template <class charT, class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
  class basic_ostringstream;
template <class charT, class traits = char_traits<charT>,
    class Allocator = allocator<charT> >
  class basic_stringstream;

template <class charT, class traits = char_traits<charT> >
  class basic_filebuf;
template <class charT, class traits = char_traits<charT> >
  class basic_ifstream;
template <class charT, class traits = char_traits<charT> >
  class basic_ofstream;
template <class charT, class traits = char_traits<charT> >
  class basic_fstream;

template <class charT, class traits = char_traits<charT> >
  class istreambuf_iterator;
template <class charT, class traits = char_traits<charT> >
  class ostreambuf_iterator;

typedef basic_ios<char>       ios;
typedef basic_ios<wchar_t>    wios;

typedef basic_streambuf<char> streambuf;
typedef basic_istream<char>   istream;
typedef basic_ostream<char>   ostream;
typedef basic_iostream<char>  iostream;

typedef basic_stringbuf<char>     stringbuf;
typedef basic_istringstream<char> istringstream;
typedef basic_ostringstream<char> ostringstream;
typedef basic_stringstream<char>  stringstream;

typedef basic_filebuf<char>  filebuf;
typedef basic_ifstream<char> ifstream;
typedef basic_ofstream<char> ofstream;
typedef basic_fstream<char>  fstream;

typedef basic_streambuf<wchar_t> wstreambuf;
typedef basic_istream<wchar_t>   wistream;
typedef basic_ostream<wchar_t>   wostream;
typedef basic_iostream<wchar_t>  wiostream;

typedef basic_stringbuf<wchar_t>     wstringbuf;
typedef basic_istringstream<wchar_t> wistringstream;
typedef basic_ostringstream<wchar_t> wostringstream;
typedef basic_stringstream<wchar_t>  wstringstream;

typedef basic_filebuf<wchar_t>  wfilebuf;
typedef basic_ifstream<wchar_t> wifstream;
typedef basic_ofstream<wchar_t> wofstream;
typedef basic_fstream<wchar_t>  wfstream;

template <class state> class fpos;
```

```
  typedef fpos<char_traits<char>::state_type>    streampos;
  typedef fpos<char_traits<wchar_t>::state_type> wstreampos;
}
```

1 Default template arguments are described as appearing both in `<iosfwd>` and in the synopsis of other headers but it is well-formed to include both `<iosfwd>` and one or more of the other headers.[295]

2 [*Note:* The class template specialization `basic_ios<charT,traits>` serves as a virtual base class for the class templates `basic_istream`, `basic_ostream`, and class templates derived from them. `basic_iostream` is a class template derived from both `basic_istream<charT,traits>` and `basic_ostream<charT,traits>`.

3 The class template specialization `basic_streambuf<charT,traits>` serves as a base class for class templates `basic_stringbuf` and `basic_filebuf`.

4 The class template specialization `basic_istream<charT,traits>` serves as a base class for class templates `basic_istringstream` and `basic_ifstream`.

5 The class template specialization `basic_ostream<charT,traits>` serves as a base class for class templates `basic_ostringstream` and `basic_ofstream`.

6 The class template specialization `basic_iostream<charT,traits>` serves as a base class for class templates `basic_stringstream` and `basic_fstream`.

7 Other typedefs define instances of class templates specialized for `char` or `wchar_t` types.

8 Specializations of the class template `fpos` are used for specifying file position information.

9 The types `streampos` and `wstreampos` are used for positioning streams specialized on `char` and `wchar_t` respectively.

10 This synopsis suggests a circularity between `streampos` and `char_traits<char>`. An implementation can avoid this circularity by substituting equivalent types. One way to do this might be

```
template<class stateT> class fpos { ... };       // depends on nothing
typedef ... _STATE;              // implementation private declaration of stateT

typedef fpos<_STATE> streampos;

template<> struct char_traits<char> {
  typedef streampos
  pos_type;
}
```

— *end note*]

27.4 Standard iostream objects [iostream.objects]

27.4.1 Overview [iostream.objects.overview]

Header `<iostream>` synopsis

```
#include <ios>
#include <streambuf>
#include <istream>
#include <ostream>

namespace std {
  extern istream cin;
```

295) It is the implementation's responsibility to implement headers so that including `<iosfwd>` and other headers does not violate the rules about multiple occurrences of default arguments.

```
  extern ostream cout;
  extern ostream cerr;
  extern ostream clog;

  extern wistream wcin;
  extern wostream wcout;
  extern wostream wcerr;
  extern wostream wclog;
}
```

1 The header <iostream> declares objects that associate objects with the standard C streams provided for by the functions declared in <cstdio> (27.9.2), and includes all the headers necessary to use these objects.

2 The objects are constructed and the associations are established at some time prior to or during the first time an object of class ios_base::Init is constructed, and in any case before the body of main begins execution.[296] The objects are not destroyed during program execution.[297] The results of including <iostream> in a translation unit shall be as if <iostream> defined an instance of ios_base::Init with static storage duration. Similarly, the entire program shall behave as if there were at least one instance of ios_base::Init with static storage duration.

3 Mixing operations on corresponding wide- and narrow-character streams follows the same semantics as mixing such operations on FILEs, as specified in Amendment 1 of the ISO C standard.

4 Concurrent access to a synchronized (27.5.3.4) standard iostream object's formatted and unformatted input (27.7.2.1) and output (27.7.3.1) functions or a standard C stream by multiple threads shall not result in a data race (1.10). [*Note:* Users must still synchronize concurrent use of these objects and streams by multiple threads if they wish to avoid interleaved characters. — *end note*]

27.4.2 Narrow stream objects [narrow.stream.objects]

`istream cin;`

1 The object cin controls input from a stream buffer associated with the object stdin, declared in <cstdio>.

2 After the object cin is initialized, cin.tie() returns &cout. Its state is otherwise the same as required for basic_ios<char>::init (27.5.5.2).

`ostream cout;`

3 The object cout controls output to a stream buffer associated with the object stdout, declared in <cstdio> (27.9.2).

`ostream cerr;`

4 The object cerr controls output to a stream buffer associated with the object stderr, declared in <cstdio> (27.9.2).

5 After the object cerr is initialized, cerr.flags() & unitbuf is nonzero and cerr.tie() returns &cout. Its state is otherwise the same as required for basic_ios<char>::init (27.5.5.2).

`ostream clog;`

6 The object clog controls output to a stream buffer associated with the object stderr, declared in <cstdio> (27.9.2).

296) If it is possible for them to do so, implementations are encouraged to initialize the objects earlier than required.
297) Constructors and destructors for static objects can access these objects to read input from stdin or write output to stdout or stderr.

27.4.3 Wide stream objects [wide.stream.objects]

`wistream wcin;`

1 The object `wcin` controls input from a stream buffer associated with the object `stdin`, declared in `<cstdio>`.

2 After the object `wcin` is initialized, `wcin.tie()` returns `&wcout`. Its state is otherwise the same as required for `basic_ios<wchar_t>::init` (27.5.5.2).

`wostream wcout;`

3 The object `wcout` controls output to a stream buffer associated with the object `stdout`, declared in `<cstdio>` (27.9.2).

`wostream wcerr;`

4 The object `wcerr` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (27.9.2).

5 After the object `wcerr` is initialized, `wcerr.flags() & unitbuf` is nonzero and `wcerr.tie()` returns `&wcout`. Its state is otherwise the same as required for `basic_ios<wchar_t>::init` (27.5.5.2).

`wostream wclog;`

6 The object `wclog` controls output to a stream buffer associated with the object `stderr`, declared in `<cstdio>` (27.9.2).

27.5 Iostreams base classes [iostreams.base]

27.5.1 Overview [iostreams.base.overview]

Header `<ios>` synopsis

```
#include <iosfwd>

namespace std {
  typedef implementation-defined streamoff;
  typedef implementation-defined streamsize;
  template <class stateT> class fpos;

  class ios_base;
  template <class charT, class traits = char_traits<charT> >
    class basic_ios;

  // 27.5.6, manipulators:
  ios_base& boolalpha  (ios_base& str);
  ios_base& noboolalpha(ios_base& str);

  ios_base& showbase   (ios_base& str);
  ios_base& noshowbase (ios_base& str);

  ios_base& showpoint  (ios_base& str);
  ios_base& noshowpoint(ios_base& str);

  ios_base& showpos    (ios_base& str);
  ios_base& noshowpos  (ios_base& str);

  ios_base& skipws     (ios_base& str);
  ios_base& noskipws   (ios_base& str);
```

```
    ios_base& uppercase  (ios_base& str);
    ios_base& nouppercase(ios_base& str);

    ios_base& unitbuf    (ios_base& str);
    ios_base& nounitbuf  (ios_base& str);

    // 27.5.6.2 adjustfield:
    ios_base& internal   (ios_base& str);
    ios_base& left       (ios_base& str);
    ios_base& right      (ios_base& str);

    // 27.5.6.3 basefield:
    ios_base& dec        (ios_base& str);
    ios_base& hex        (ios_base& str);
    ios_base& oct        (ios_base& str);

    // 27.5.6.4 floatfield:
    ios_base& fixed      (ios_base& str);
    ios_base& scientific (ios_base& str);
    ios_base& hexfloat   (ios_base& str);
    ios_base& defaultfloat(ios_base& str);

    // 27.5.6.5 error reporting:
    enum class io_errc {
      stream = 1
    };

    template <> struct is_error_code_enum<io_errc> : public true_type { };
    error_code make_error_code(io_errc e) noexcept;
    error_condition make_error_condition(io_errc e) noexcept;
    const error_category& iostream_category() noexcept;
  }
```

27.5.2 Types [stream.types]

```
typedef implementation-defined streamoff;
```

1 The type `streamoff` is a synonym for one of the signed basic integral types of sufficient size to represent the maximum possible file size for the operating system.[298]

```
typedef implementation-defined streamsize;
```

2 The type `streamsize` is a synonym for one of the signed basic integral types. It is used to represent the number of characters transferred in an I/O operation, or the size of I/O buffers.[299]

27.5.3 Class `ios_base` [ios.base]

```
  namespace std {
    class ios_base {
    public:
      class failure;
```

298) Typically `long long`.

299) `streamsize` is used in most places where ISO C would use `size_t`. Most of the uses of `streamsize` could use `size_t`, except for the `strstreambuf` constructors, which require negative values. It should probably be the signed type corresponding to `size_t` (which is what Posix.2 calls `ssize_t`).

```
// 27.5.3.1.2 fmtflags
typedef T1 fmtflags;
static constexpr fmtflags boolalpha = unspecified ;
static constexpr fmtflags dec = unspecified ;
static constexpr fmtflags fixed = unspecified ;
static constexpr fmtflags hex = unspecified ;
static constexpr fmtflags internal = unspecified ;
static constexpr fmtflags left = unspecified ;
static constexpr fmtflags oct = unspecified ;
static constexpr fmtflags right = unspecified ;
static constexpr fmtflags scientific = unspecified ;
static constexpr fmtflags showbase = unspecified ;
static constexpr fmtflags showpoint = unspecified ;
static constexpr fmtflags showpos = unspecified ;
static constexpr fmtflags skipws = unspecified ;
static constexpr fmtflags unitbuf = unspecified ;
static constexpr fmtflags uppercase = unspecified ;
static constexpr fmtflags adjustfield = see below;
static constexpr fmtflags basefield = see below;
static constexpr fmtflags floatfield = see below;

// 27.5.3.1.3 iostate
typedef T2 iostate;
static constexpr iostate badbit = unspecified ;
static constexpr iostate eofbit = unspecified ;
static constexpr iostate failbit = unspecified ;
static constexpr iostate goodbit = see below;

// 27.5.3.1.4 openmode
typedef T3 openmode;
static constexpr openmode app = unspecified ;
static constexpr openmode ate = unspecified ;
static constexpr openmode binary = unspecified ;
static constexpr openmode in = unspecified ;
static constexpr openmode out = unspecified ;
static constexpr openmode trunc = unspecified ;

// 27.5.3.1.5 seekdir
typedef T4 seekdir;
static constexpr seekdir beg = unspecified ;
static constexpr seekdir cur = unspecified ;
static constexpr seekdir end = unspecified ;

class Init;

// 27.5.3.2 fmtflags state:
fmtflags flags() const;
fmtflags flags(fmtflags fmtfl);
fmtflags setf(fmtflags fmtfl);
fmtflags setf(fmtflags fmtfl, fmtflags mask);
void unsetf(fmtflags mask);

streamsize precision() const;
streamsize precision(streamsize prec);
```

```
    streamsize width() const;
    streamsize width(streamsize wide);

    // 27.5.3.3 locales:
    locale imbue(const locale& loc);
    locale getloc() const;

    // 27.5.3.5 storage:
    static int xalloc();
    long&  iword(int index);
    void*& pword(int index);

    // destructor
    virtual ~ios_base();

    // 27.5.3.6 callbacks;
    enum event { erase_event, imbue_event, copyfmt_event };
    typedef void (*event_callback)(event, ios_base&, int index);
    void register_callback(event_callback fn, int index);

    ios_base(const ios_base&) = delete;
    ios_base& operator=(const ios_base&) = delete;

    static bool sync_with_stdio(bool sync = true);

  protected:
    ios_base();

  private:
    static int index;  // exposition only
    long* iarray;      // exposition only
    void** parray;     // exposition only
  };
}
```

1 `ios_base` defines several member types:

(1.1) — a class `failure` derived from `system_error`;

(1.2) — a class `Init`;

(1.3) — three bitmask types, `fmtflags`, `iostate`, and `openmode`;

(1.4) — an enumerated type, `seekdir`.

2 It maintains several kinds of data:

(2.1) — state information that reflects the integrity of the stream buffer;

(2.2) — control information that influences how to interpret (format) input sequences and how to generate (format) output sequences;

(2.3) — additional information that is stored by the program for its private use.

3 [*Note:* For the sake of exposition, the maintained data is presented here as:

(3.1) — `static int index`, specifies the next available unique index for the integer or pointer arrays maintained for the private use of the program, initialized to an unspecified value;

(3.2) — `long* iarray`, points to the first element of an arbitrary-length `long` array maintained for the private use of the program;

(3.3) — `void** parray`, points to the first element of an arbitrary-length pointer array maintained for the private use of the program. — *end note*]

27.5.3.1 Types [ios.types]

27.5.3.1.1 Class `ios_base::failure` [ios::failure]

```
namespace std {
  class ios_base::failure : public system_error {
  public:
    explicit failure(const string& msg, const error_code& ec = io_errc::stream);
    explicit failure(const char* msg, const error_code& ec = io_errc::stream);
  };
}
```

1 The class `failure` defines the base class for the types of all objects thrown as exceptions, by functions in the iostreams library, to report errors detected during stream buffer operations.

2 When throwing `ios_base::failure` exceptions, implementations should provide values of `ec` that identify the specific reason for the failure. [*Note:* Errors arising from the operating system would typically be reported as `system_category()` errors with an error value of the error number reported by the operating system. Errors arising from within the stream library would typically be reported as `error_code(io_errc::stream, iostream_category())`. — *end note*]

```
explicit failure(const string& msg, const error_code& ec = io_errc::stream);
```

3 *Effects:* Constructs an object of class `failure` by constructing the base class with `msg` and `ec`.

```
explicit failure(const char* msg, const error_code& ec = io_errc::stream);
```

4 *Effects:* Constructs an object of class `failure` by constructing the base class with `msg` and `ec`.

27.5.3.1.2 Type `ios_base::fmtflags` [ios::fmtflags]

```
typedef T1 fmtflags;
```

1 The type `fmtflags` is a bitmask type (17.5.2.1.3). Setting its elements has the effects indicated in Table 121.

2 Type `fmtflags` also defines the constants indicated in Table 122.

27.5.3.1.3 Type `ios_base::iostate` [ios::iostate]

```
typedef T2 iostate;
```

1 The type `iostate` is a bitmask type (17.5.2.1.3) that contains the elements indicated in Table 123.

2 Type `iostate` also defines the constant:

(2.1) — `goodbit`, the value zero.

27.5.3.1.4 Type `ios_base::openmode` [ios::openmode]

```
typedef T3 openmode;
```

1 The type `openmode` is a bitmask type (17.5.2.1.3). It contains the elements indicated in Table 124.

Table 121 — `fmtflags` effects

Element	Effect(s) if set
`boolalpha`	insert and extract `bool` type in alphabetic format
`dec`	converts integer input or generates integer output in decimal base
`fixed`	generate floating-point output in fixed-point notation
`hex`	converts integer input or generates integer output in hexadecimal base
`internal`	adds fill characters at a designated internal point in certain generated output, or identical to `right` if no such point is designated
`left`	adds fill characters on the right (final positions) of certain generated output
`oct`	converts integer input or generates integer output in octal base
`right`	adds fill characters on the left (initial positions) of certain generated output
`scientific`	generates floating-point output in scientific notation
`showbase`	generates a prefix indicating the numeric base of generated integer output
`showpoint`	generates a decimal-point character unconditionally in generated floating-point output
`showpos`	generates a `+` sign in non-negative generated numeric output
`skipws`	skips leading whitespace before certain input operations
`unitbuf`	flushes output after each output operation
`uppercase`	replaces certain lowercase letters with their uppercase equivalents in generated output

Table 122 — `fmtflags` constants

Constant	Allowable values
`adjustfield`	`left | right | internal`
`basefield`	`dec | oct | hex`
`floatfield`	`scientific | fixed`

Table 123 — `iostate` effects

Element	Effect(s) if set
`badbit`	indicates a loss of integrity in an input or output sequence (such as an irrecoverable read error from a file);
`eofbit`	indicates that an input operation reached the end of an input sequence;
`failbit`	indicates that an input operation failed to read the expected characters, or that an output operation failed to generate the desired characters.

Table 124 — `openmode` effects

Element	Effect(s) if set
`app`	seek to end before each write
`ate`	open and seek to end immediately after opening
`binary`	perform input and output in binary mode (as opposed to text mode)
`in`	open for input
`out`	open for output
`trunc`	truncate an existing stream when opening

27.5.3.1.5 Type `ios_base::seekdir` [ios::seekdir]

```
typedef T4 seekdir;
```

1 The type `seekdir` is an enumerated type (17.5.2.1.2) that contains the elements indicated in Table 125.

Table 125 — `seekdir` effects

Element	Meaning
`beg`	request a seek (for subsequent input or output) relative to the beginning of the stream
`cur`	request a seek relative to the current position within the sequence
`end`	request a seek relative to the current end of the sequence

27.5.3.1.6 Class `ios_base::Init` [ios::Init]

```
namespace std {
  class ios_base::Init {
  public:
    Init();
   ~Init();
  private:
    static int init_cnt; // exposition only
  };
}
```

1 The class `Init` describes an object whose construction ensures the construction of the eight objects declared in `<iostream>` (27.4) that associate file stream buffers with the standard C streams provided for by the functions declared in `<cstdio>` (27.9.2).

2 For the sake of exposition, the maintained data is presented here as:

(2.1) — `static int init_cnt`, counts the number of constructor and destructor calls for class `Init`, initialized to zero.

```
Init();
```

3 *Effects:* Constructs an object of class `Init`. Constructs and initializes the objects `cin`, `cout`, `cerr`, `clog`, `wcin`, `wcout`, `wcerr`, and `wclog` if they have not already been constructed and initialized.

```
~Init();
```

4 *Effects:* Destroys an object of class `Init`. If there are no other instances of the class still in existence, calls `cout.flush()`, `cerr.flush()`, `clog.flush()`, `wcout.flush()`, `wcerr.flush()`, `wclog.flush()`.

27.5.3.2 `ios_base` state functions [fmtflags.state]

```
fmtflags flags() const;
```

1 *Returns:* The format control information for both input and output.

```
fmtflags flags(fmtflags fmtfl);
```

2 *Postcondition:* `fmtfl == flags()`.

3 *Returns:* The previous value of `flags()`.

```
fmtflags setf(fmtflags fmtfl);
```

4 *Effects:* Sets `fmtfl` in `flags()`.

5 *Returns:* The previous value of `flags()`.

```
fmtflags setf(fmtflags fmtfl, fmtflags mask);
```

6 *Effects:* Clears `mask` in `flags()`, sets `fmtfl & mask` in `flags()`.

7 *Returns:* The previous value of `flags()`.

```
void unsetf(fmtflags mask);
```

8 *Effects:* Clears `mask` in `flags()`.

```
streamsize precision() const;
```

9 *Returns:* The precision to generate on certain output conversions.

```
streamsize precision(streamsize prec);
```

10 *Postcondition:* `prec == precision()`.

11 *Returns:* The previous value of `precision()`.

```
streamsize width() const;
```

12 *Returns:* The minimum field width (number of characters) to generate on certain output conversions.

```
streamsize width(streamsize wide);
```

13 *Postcondition:* `wide == width()`.

14 *Returns:* The previous value of `width()`.

27.5.3.3 `ios_base` functions [ios.base.locales]

```
locale imbue(const locale& loc);
```

1 *Effects:* Calls each registered callback pair `(fn,index)` (27.5.3.6) as `(*fn)(imbue_event,*this,index)` at such a time that a call to `ios_base::getloc()` from within `fn` returns the new locale value `loc`.

2 *Returns:* The previous value of `getloc()`.

3 *Postcondition:* `loc == getloc()`.

```
locale getloc() const;
```

4 *Returns:* If no locale has been imbued, a copy of the global C++ locale, `locale()`, in effect at the time of construction. Otherwise, returns the imbued locale, to be used to perform locale-dependent input and output operations.

27.5.3.4 `ios_base` static members [ios.members.static]

```
bool sync_with_stdio(bool sync = true);
```

1 *Returns:* `true` if the previous state of the standard iostream objects (27.4) was synchronized and otherwise returns `false`. The first time it is called, the function returns `true`.

2 *Effects:* If any input or output operation has occurred using the standard streams prior to the call, the effect is implementation-defined. Otherwise, called with a false argument, it allows the standard streams to operate independently of the standard C streams.

3 When a standard iostream object `str` is *synchronized* with a standard stdio stream `f`, the effect of inserting a character `c` by

```
fputc(f, c);
```

is the same as the effect of

```
str.rdbuf()->sputc(c);
```

for any sequences of characters; the effect of extracting a character `c` by

```
c = fgetc(f);
```

is the same as the effect of

```
c = str.rdbuf()->sbumpc();
```

for any sequences of characters; and the effect of pushing back a character `c` by

```
ungetc(c, f);
```

is the same as the effect of

```
str.rdbuf()->sputbackc(c);
```

for any sequence of characters.[300]

27.5.3.5 `ios_base` storage functions [ios.base.storage]

```
static int xalloc();
```

1 *Returns:* `index ++`.

2 *Remarks:* Concurrent access to this function by multiple threads shall not result in a data race (1.10).

```
long& iword(int idx);
```

3 *Effects:* If `iarray` is a null pointer, allocates an array of `long` of unspecified size and stores a pointer to its first element in `iarray`. The function then extends the array pointed at by `iarray` as necessary to include the element `iarray[idx]`. Each newly allocated element of the array is initialized to zero. The reference returned is invalid after any other operations on the object.[301] However, the value of the storage referred to is retained, so that until the next call to `copyfmt`, calling `iword` with the same index yields another reference to the same value. If the function fails[302] and `*this` is a base subobject of a `basic_ios<>` object or subobject, the effect is equivalent to calling `basic_ios<>::setstate(badbit)` on the derived object (which may throw `failure`).

4 *Returns:* On success `iarray[idx]`. On failure, a valid `long&` initialized to 0.

```
void*& pword(int idx);
```

5 *Effects:* If `parray` is a null pointer, allocates an array of pointers to `void` of unspecified size and stores a pointer to its first element in `parray`. The function then extends the array pointed at by `parray` as necessary to include the element `parray[idx]`. Each newly allocated element of the array is initialized to a null pointer. The reference returned is invalid after any other operations on the object. However, the value of the storage referred to is retained, so that until the next call to `copyfmt`, calling `pword` with the same index yields another reference to the same value. If the function fails[303]

300) This implies that operations on a standard iostream object can be mixed arbitrarily with operations on the corresponding stdio stream. In practical terms, synchronization usually means that a standard iostream object and a standard stdio object share a buffer.

301) An implementation is free to implement both the integer array pointed at by `iarray` and the pointer array pointed at by `parray` as sparse data structures, possibly with a one-element cache for each.

302) for example, because it cannot allocate space.

303) for example, because it cannot allocate space.

and `*this` is a base subobject of a `basic_ios<>` object or subobject, the effect is equivalent to calling `basic_ios<>::setstate(badbit)` on the derived object (which may throw `failure`).

6 *Returns:* On success `parray[idx]`. On failure a valid `void*&` initialized to 0.

7 *Remarks:* After a subsequent call to `pword(int)` for the same object, the earlier return value may no longer be valid.

27.5.3.6 `ios_base` callbacks [ios.base.callback]

```
void register_callback(event_callback fn, int index);
```

1 *Effects:* Registers the pair `(fn,index)` such that during calls to `imbue()` (27.5.3.3), `copyfmt()`, or `~ios_base()` (27.5.3.7), the function `fn` is called with argument `index`. Functions registered are called when an event occurs, in opposite order of registration. Functions registered while a callback function is active are not called until the next event.

2 *Requires:* The function `fn` shall not throw exceptions.

Remarks: Identical pairs are not merged. A function registered twice will be called twice.

27.5.3.7 `ios_base` constructors/destructor [ios.base.cons]

```
ios_base();
```

1 *Effects:* Each `ios_base` member has an indeterminate value after construction. The object's members shall be initialized by calling `basic_ios::init` before the object's first use or before it is destroyed, whichever comes first; otherwise the behavior is undefined.

```
~ios_base();
```

2 *Effects:* Destroys an object of class `ios_base`. Calls each registered callback pair `(fn, index)` (27.5.3.6) as `(*fn)(erase_event, *this, index)` at such time that any `ios_base` member function called from within `fn` has well defined results.

27.5.4 Class template `fpos` [fpos]

```
namespace std {
  template <class stateT> class fpos {
  public:
    // 27.5.4.1 Members
    stateT state() const;
    void state(stateT);
  private;
    stateT st; // exposition only
  };
}
```

27.5.4.1 `fpos` members [fpos.members]

```
void state(stateT s);
```

1 *Effects:* Assign `s` to `st`.

```
stateT state() const;
```

2 *Returns:* Current value of `st`.

27.5.4.2 `fpos` requirements [fpos.operations]

1 Operations specified in Table 126 are permitted. In that table,

(1.1) — `P` refers to an instance of `fpos`,

(1.2) — `p` and `q` refer to values of type `P`,

(1.3) — `O` refers to type `streamoff`,

(1.4) — `o` refers to a value of type `streamoff`,

(1.5) — `sz` refers to a value of type `streamsize` and

(1.6) — `i` refers to a value of type `int`.

Table 126 — Position type requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
`P(i)`			`p == P(i)` note: a destructor is assumed.
`P p(i);` `P p = i;`			post: `p == P(i)`.
`P(o)`	`fpos`	converts from `offset`	
`O(p)`	streamoff	converts to `offset`	`P(O(p)) == p`
`p == q`	convertible to `bool`		`==` is an equivalence relation
`p != q`	convertible to `bool`	`!(p == q)`	
`q = p + o` `p += o`	`fpos`	`+` offset	`q - o == p`
`q = p - o` `p -= o`	`fpos`	`-` offset	`q + o == p`
`o = p - q`	streamoff	distance	`q + o == p`
`streamsize(o)`	`streamsize`	converts	`streamsize(O(sz)) == sz`
`O(sz)`	streamoff	converts	`streamsize(O(sz)) == sz`

2 [*Note:* Every implementation is required to supply overloaded operators on `fpos` objects to satisfy the requirements of 27.5.4.2. It is unspecified whether these operators are members of `fpos`, global operators, or provided in some other way. — *end note*]

3 Stream operations that return a value of type `traits::pos_type` return `P(O(-1))` as an invalid value to signal an error. If this value is used as an argument to any `istream`, `ostream`, or `streambuf` member that accepts a value of type `traits::pos_type` then the behavior of that function is undefined.

27.5.5 Class template `basic_ios` [ios]

27.5.5.1 Overview [ios.overview]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_ios : public ios_base {
  public:

    // types:
```

```
    typedef charT                    char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                   traits_type;

    explicit operator bool() const;
    bool operator!() const;
    iostate rdstate() const;
    void clear(iostate state = goodbit);
    void setstate(iostate state);
    bool good() const;
    bool eof()  const;
    bool fail() const;
    bool bad()  const;

    iostate exceptions() const;
    void exceptions(iostate except);

    // 27.5.5.2 Constructor/destructor:
    explicit basic_ios(basic_streambuf<charT,traits>* sb);
    virtual ~basic_ios();

    // 27.5.5.3 Members:
    basic_ostream<charT,traits>* tie() const;
    basic_ostream<charT,traits>* tie(basic_ostream<charT,traits>* tiestr);

    basic_streambuf<charT,traits>* rdbuf() const;
    basic_streambuf<charT,traits>* rdbuf(basic_streambuf<charT,traits>* sb);

    basic_ios& copyfmt(const basic_ios& rhs);

    char_type fill() const;
    char_type fill(char_type ch);

    locale imbue(const locale& loc);

    char     narrow(char_type c, char dfault) const;
    char_type widen(char c) const;

    basic_ios(const basic_ios&) = delete;
    basic_ios& operator=(const basic_ios&) = delete;

  protected:
    basic_ios();
    void init(basic_streambuf<charT,traits>* sb);
    void move(basic_ios& rhs);
    void move(basic_ios&& rhs);
    void swap(basic_ios& rhs) noexcept;
    void set_rdbuf(basic_streambuf<charT, traits>* sb);

  };
}
```

27.5.5.2 `basic_ios` constructors [basic.ios.cons]

```
explicit basic_ios(basic_streambuf<charT,traits>* sb);
```

1 *Effects:* Constructs an object of class `basic_ios`, assigning initial values to its member objects by calling `init(sb)`.

```
basic_ios();
```

2 *Effects:* Constructs an object of class `basic_ios` (27.5.3.7) leaving its member objects uninitialized. The object shall be initialized by calling `basic_ios::init` before its first use or before it is destroyed, whichever comes first; otherwise the behavior is undefined.

```
~basic_ios();
```

3 *Remarks:* The destructor does not destroy `rdbuf()`.

```
void init(basic_streambuf<charT,traits>* sb);
```

Postconditions: The postconditions of this function are indicated in Table 127.

Table 127 — `basic_ios::init()` effects

Element	Value
`rdbuf()`	`sb`
`tie()`	`0`
`rdstate()`	`goodbit` if `sb` is not a null pointer, otherwise `badbit`.
`exceptions()`	`goodbit`
`flags()`	`skipws | dec`
`width()`	`0`
`precision()`	`6`
`fill()`	`widen(' ');`
`getloc()`	a copy of the value returned by `locale()`
`iarray`	a null pointer
`parray`	a null pointer

27.5.5.3 Member functions [basic.ios.members]

```
basic_ostream<charT,traits>* tie() const;
```

1 *Returns:* An output sequence that is *tied* to (synchronized with) the sequence controlled by the stream buffer.

```
basic_ostream<charT,traits>* tie(basic_ostream<charT,traits>* tiestr);
```

2 *Requires:* If `tiestr` is not null, `tiestr` must not be reachable by traversing the linked list of tied stream objects starting from `tiestr->tie()`.

3 *Postcondition:* `tiestr == tie()`.

4 *Returns:* The previous value of `tie()`.

```
basic_streambuf<charT,traits>* rdbuf() const;
```

5 *Returns:* A pointer to the `streambuf` associated with the stream.

```
basic_streambuf<charT,traits>* rdbuf(basic_streambuf<charT,traits>* sb);
```

6 *Postcondition:* `sb == rdbuf()`.

7 *Effects:* Calls `clear()`.

8 *Returns:* The previous value of `rdbuf()`.

```
locale imbue(const locale& loc);
```

9 *Effects:* Calls `ios_base::imbue(loc)` (27.5.3.3) and if `rdbuf()!=0` then `rdbuf()->pubimbue(loc)` (27.6.3.2.1).

10 *Returns:* The prior value of `ios_base::imbue()`.

```
char narrow(char_type c, char dfault) const;
```

11 *Returns:* `use_facet< ctype<char_type> >(getloc()).narrow(c,dfault)`

```
char_type widen(char c) const;
```

12 *Returns:* `use_facet< ctype<char_type> >(getloc()).widen(c)`

```
char_type fill() const;
```

13 *Returns:* The character used to pad (fill) an output conversion to the specified field width.

```
char_type fill(char_type fillch);
```

14 *Postcondition:* `traits::eq(fillch, fill())`

15 *Returns:* The previous value of `fill()`.

```
basic_ios& copyfmt(const basic_ios& rhs);
```

16 *Effects:* If `(this == &rhs)` does nothing. Otherwise assigns to the member objects of `*this` the corresponding member objects of `rhs` as follows:

1. calls each registered callback pair `(fn, index)` as `(*fn)(erase_event, *this, index)`;
2. assigns to the member objects of `*this` the corresponding member objects of `rhs`, except that

(16.1) — `rdstate()`, `rdbuf()`, and `exceptions()` are left unchanged;

(16.2) — the contents of arrays pointed at by `pword` and `iword` are copied, not the pointers themselves;[304] and

(16.3) — if any newly stored pointer values in `*this` point at objects stored outside the object `rhs` and those objects are destroyed when `rhs` is destroyed, the newly stored pointer values are altered to point at newly constructed copies of the objects;

3. calls each callback pair that was copied from `rhs` as `(*fn)(copyfmt_event, *this, index)`;
4. calls `exceptions(rhs.exceptions())`.

17 *Note:* The second pass through the callback pairs permits a copied `pword` value to be zeroed, or to have its referent deep copied or reference counted, or to have other special action taken.

18 *Postconditions:* The postconditions of this function are indicated in Table 128.

19 *Returns:* `*this`.

```
void move(basic_ios& rhs);
void move(basic_ios&& rhs);
```

304) This suggests an infinite amount of copying, but the implementation can keep track of the maximum element of the arrays that is non-zero.

Table 128 — `basic_ios::copyfmt()` effects

Element	Value
`rdbuf()`	*unchanged*
`tie()`	`rhs.tie()`
`rdstate()`	*unchanged*
`exceptions()`	`rhs.exceptions()`
`flags()`	`rhs.flags()`
`width()`	`rhs.width()`
`precision()`	`rhs.precision()`
`fill()`	`rhs.fill()`
`getloc()`	`rhs.getloc()`

20 *Postconditions:* `*this` shall have the state that `rhs` had before the function call, except that `rdbuf()` shall return 0. `rhs` shall be in a valid but unspecified state, except that `rhs.rdbuf()` shall return the same value as it returned before the function call, and `rhs.tie()` shall return 0.

```
void swap(basic_ios& rhs) noexcept;
```

21 *Effects:* The states of `*this` and `rhs` shall be exchanged, except that `rdbuf()` shall return the same value as it returned before the function call, and `rhs.rdbuf()` shall return the same value as it returned before the function call.

```
void set_rdbuf(basic_streambuf<charT, traits>* sb);
```

22 *Requires:* `sb != nullptr`.

23 *Effects:* Associates the `basic_streambuf` object pointed to by `sb` with this stream without calling `clear()`.

24 *Postconditions:* `rdbuf() == sb`.

25 *Throws:* Nothing.

27.5.5.4 `basic_ios` flags functions [iostate.flags]

```
explicit operator bool() const;
```

1 *Returns:* `!fail()`.

```
bool operator!() const;
```

2 *Returns:* `fail()`.

```
iostate rdstate() const;
```

3 *Returns:* The error state of the stream buffer.

```
void clear(iostate state = goodbit);
```

4 *Postcondition:* If `rdbuf()!=0` then `state == rdstate()`; otherwise `rdstate()==(state | ios_base::badbit)`.

5 *Effects:* If `((state | (rdbuf() ? goodbit : badbit)) & exceptions()) == 0`, returns. Otherwise, the function throws an object `fail` of class `basic_ios::failure` (27.5.3.1.1), constructed with implementation-defined argument values.

```
void setstate(iostate state);
```

6 *Effects:* Calls `clear(rdstate() | state)` (which may throw `basic_ios::failure` (27.5.3.1.1)).

```
bool good() const;
```

7 *Returns:* `rdstate() == 0`

```
bool eof() const;
```

8 *Returns:* `true` if `eofbit` is set in `rdstate()`.

```
bool fail() const;
```

9 *Returns:* `true` if `failbit` or `badbit` is set in `rdstate()`.[305]

```
bool bad() const;
```

10 *Returns:* `true` if `badbit` is set in `rdstate()`.

```
iostate exceptions() const;
```

11 *Returns:* A mask that determines what elements set in `rdstate()` cause exceptions to be thrown.

```
void exceptions(iostate except);
```

12 *Postcondition:* `except == exceptions()`.

13 *Effects:* Calls `clear(rdstate())`.

27.5.6 `ios_base` manipulators [std.ios.manip]

27.5.6.1 `fmtflags` manipulators [fmtflags.manip]

```
ios_base& boolalpha(ios_base& str);
```

1 *Effects:* Calls `str.setf(ios_base::boolalpha)`.

2 *Returns:* `str`.

```
ios_base& noboolalpha(ios_base& str);
```

3 *Effects:* Calls `str.unsetf(ios_base::boolalpha)`.

4 *Returns:* `str`.

```
ios_base& showbase(ios_base& str);
```

5 *Effects:* Calls `str.setf(ios_base::showbase)`.

6 *Returns:* `str`.

```
ios_base& noshowbase(ios_base& str);
```

7 *Effects:* Calls `str.unsetf(ios_base::showbase)`.

8 *Returns:* `str`.

```
ios_base& showpoint(ios_base& str);
```

9 *Effects:* Calls `str.setf(ios_base::showpoint)`.

10 *Returns:* `str`.

```
ios_base& noshowpoint(ios_base& str);
```

305) Checking `badbit` also for `fail()` is historical practice.

Effects: Calls `str.unsetf(ios_base::showpoint)`.
Returns: `str`.

`ios_base& showpos(ios_base& str);`
Effects: Calls `str.setf(ios_base::showpos)`.
Returns: `str`.

`ios_base& noshowpos(ios_base& str);`
Effects: Calls `str.unsetf(ios_base::showpos)`.
Returns: `str`.

`ios_base& skipws(ios_base& str);`
Effects: Calls `str.setf(ios_base::skipws)`.
Returns: `str`.

`ios_base& noskipws(ios_base& str);`
Effects: Calls `str.unsetf(ios_base::skipws)`.
Returns: `str`.

`ios_base& uppercase(ios_base& str);`
Effects: Calls `str.setf(ios_base::uppercase)`.
Returns: `str`.

`ios_base& nouppercase(ios_base& str);`
Effects: Calls `str.unsetf(ios_base::uppercase)`.
Returns: `str`.

`ios_base& unitbuf(ios_base& str);`
Effects: Calls `str.setf(ios_base::unitbuf)`.
Returns: `str`.

`ios_base& nounitbuf(ios_base& str);`
Effects: Calls `str.unsetf(ios_base::unitbuf)`.
Returns: `str`.

27.5.6.2 `adjustfield` manipulators [adjustfield.manip]

`ios_base& internal(ios_base& str);`
Effects: Calls `str.setf(ios_base::internal, ios_base::adjustfield)`.
Returns: `str`.

`ios_base& left(ios_base& str);`
Effects: Calls `str.setf(ios_base::left, ios_base::adjustfield)`.
Returns: `str`.

`ios_base& right(ios_base& str);`
Effects: Calls `str.setf(ios_base::right, ios_base::adjustfield)`.
Returns: `str`.

27.5.6.3 `basefield` manipulators [basefield.manip]

```
ios_base& dec(ios_base& str);
```

1 *Effects:* Calls `str.setf(ios_base::dec, ios_base::basefield)`.

2 *Returns:* `str`[306].

```
ios_base& hex(ios_base& str);
```

3 *Effects:* Calls `str.setf(ios_base::hex, ios_base::basefield)`.

4 *Returns:* `str`.

```
ios_base& oct(ios_base& str);
```

5 *Effects:* Calls `str.setf(ios_base::oct, ios_base::basefield)`.

6 *Returns:* `str`.

27.5.6.4 `floatfield` manipulators [floatfield.manip]

```
ios_base& fixed(ios_base& str);
```

1 *Effects:* Calls `str.setf(ios_base::fixed, ios_base::floatfield)`.

2 *Returns:* `str`.

```
ios_base& scientific(ios_base& str);
```

3 *Effects:* Calls `str.setf(ios_base::scientific, ios_base::floatfield)`.

4 *Returns:* `str`.

```
ios_base& hexfloat(ios_base& str);
```

5 *Effects:* Calls `str.setf(ios_base::fixed | ios_base::scientific, ios_base::floatfield)`.

6 *Returns:* `str`.

7 [*Note:* The more obvious use of `ios_base::hex` to specify hexadecimal floating-point format would change the meaning of existing well defined programs. C++2003 gives no meaning to the combination of `fixed` and `scientific`. — *end note*]

```
ios_base& defaultfloat(ios_base& str);
```

8 *Effects:* Calls `str.unsetf(ios_base::floatfield)`.

9 *Returns:* `str`.

27.5.6.5 Error reporting [error.reporting]

```
error_code make_error_code(io_errc e) noexcept;
```

1 *Returns:* `error_code(static_cast<int>(e), iostream_category())`.

```
error_condition make_error_condition(io_errc e) noexcept;
```

2 *Returns:* `error_condition(static_cast<int>(e), iostream_category())`.

```
const error_category& iostream_category() noexcept;
```

306) The function signature `dec(ios_base&)` can be called by the function signature `basic_ostream& stream::operator<<(ios_base& (*)(ios_base&))` to permit expressions of the form `cout <<dec` to change the format flags stored in `cout`.

3 *Returns:* A reference to an object of a type derived from class `error_category`.

4 The object's `default_error_condition` and `equivalent` virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function shall return a pointer to the string `"iostream"`.

27.6 Stream buffers [stream.buffers]

27.6.1 Overview [stream.buffers.overview]

Header `<streambuf>` synopsis

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
    class basic_streambuf;
  typedef basic_streambuf<char>     streambuf;
  typedef basic_streambuf<wchar_t> wstreambuf;
}
```

1 The header `<streambuf>` defines types that control input from and output to *character* sequences.

27.6.2 Stream buffer requirements [streambuf.reqts]

1 Stream buffers can impose various constraints on the sequences they control. Some constraints are:

(1.1) — The controlled input sequence can be not readable.

(1.2) — The controlled output sequence can be not writable.

(1.3) — The controlled sequences can be associated with the contents of other representations for character sequences, such as external files.

(1.4) — The controlled sequences can support operations *directly* to or from associated sequences.

(1.5) — The controlled sequences can impose limitations on how the program can read characters from a sequence, write characters to a sequence, put characters back into an input sequence, or alter the stream position.

2 Each sequence is characterized by three pointers which, if non-null, all point into the same `charT` array object. The array object represents, at any moment, a (sub)sequence of characters from the sequence. Operations performed on a sequence alter the values stored in these pointers, perform reads and writes directly to or from associated sequences, and alter "the stream position" and conversion state as needed to maintain this subsequence relationship. The three pointers are:

(2.1) — the *beginning pointer*, or lowest element address in the array (called `xbeg` here);

(2.2) — the *next pointer*, or next element address that is a current candidate for reading or writing (called `xnext` here);

(2.3) — the *end pointer*, or first element address beyond the end of the array (called `xend` here).

3 The following semantic constraints shall always apply for any set of three pointers for a sequence, using the pointer names given immediately above:

(3.1) — If `xnext` is not a null pointer, then `xbeg` and `xend` shall also be non-null pointers into the same `charT` array, as described above; otherwise, `xbeg` and `xend` shall also be null.

(3.2) — If `xnext` is not a null pointer and `xnext < xend` for an output sequence, then a *write position* is available. In this case, `*xnext` shall be assignable as the next element to write (to put, or to store a character value, into the sequence).

(3.3) — If `xnext` is not a null pointer and `xbeg < xnext` for an input sequence, then a *putback position* is available. In this case, `xnext[-1]` shall have a defined value and is the next (preceding) element to store a character that is put back into the input sequence.

(3.4) — If `xnext` is not a null pointer and `xnext < xend` for an input sequence, then a *read position* is available. In this case, `*xnext` shall have a defined value and is the next element to read (to get, or to obtain a character value, from the sequence).

27.6.3 Class template `basic_streambuf` [streambuf]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_streambuf {
  public:

    // types:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;

    virtual ~basic_streambuf();

    // 27.6.3.2.1 locales:
    locale   pubimbue(const locale& loc);
    locale   getloc() const;

    // 27.6.3.2.2 buffer and positioning:
    basic_streambuf* pubsetbuf(char_type* s, streamsize n);
    pos_type pubseekoff(off_type off, ios_base::seekdir way,
      ios_base::openmode which =
          ios_base::in | ios_base::out);
    pos_type pubseekpos(pos_type sp,
      ios_base::openmode which =
          ios_base::in | ios_base::out);
    int      pubsync();

    // Get and put areas:
    // 27.6.3.2.3 Get area:
    streamsize in_avail();
    int_type snextc();
    int_type sbumpc();
    int_type sgetc();
    streamsize sgetn(char_type* s, streamsize n);

    // 27.6.3.2.4 Putback:
    int_type sputbackc(char_type c);
    int_type sungetc();

    // 27.6.3.2.5 Put area:
    int_type   sputc(char_type c);
```

```
    streamsize sputn(const char_type* s, streamsize n);

  protected:
    basic_streambuf();
    basic_streambuf(const basic_streambuf& rhs);
    basic_streambuf& operator=(const basic_streambuf& rhs);

    void swap(basic_streambuf& rhs);

    // 27.6.3.3.2 Get area:
    char_type* eback() const;
    char_type* gptr()  const;
    char_type* egptr() const;
    void       gbump(int n);
    void       setg(char_type* gbeg, char_type* gnext, char_type* gend);

    // 27.6.3.3.3 Put area:
    char_type* pbase() const;
    char_type* pptr() const;
    char_type* epptr() const;
    void       pbump(int n);
    void       setp(char_type* pbeg, char_type* pend);

    // 27.6.3.4 virtual functions:
    // 27.6.3.4.1 Locales:
    virtual void imbue(const locale& loc);

    // 27.6.3.4.2 Buffer management and positioning:
    virtual basic_streambuf* setbuf(char_type* s, streamsize n);
    virtual pos_type seekoff(off_type off, ios_base::seekdir way,
        ios_base::openmode which = ios_base::in | ios_base::out);
    virtual pos_type seekpos(pos_type sp,
        ios_base::openmode which = ios_base::in | ios_base::out);
    virtual int      sync();

    // 27.6.3.4.3 Get area:
    virtual streamsize showmanyc();
    virtual streamsize xsgetn(char_type* s, streamsize n);
    virtual int_type   underflow();
    virtual int_type   uflow();

    // 27.6.3.4.4 Putback:
    virtual int_type   pbackfail(int_type c = traits::eof());

    // 27.6.3.4.5 Put area:
    virtual streamsize xsputn(const char_type* s, streamsize n);
    virtual int_type   overflow (int_type c = traits::eof());
  };
}
```

1 The class template `basic_streambuf` serves as an abstract base class for deriving various *stream buffers* whose objects each control two *character sequences*:

(1.1) — a character *input sequence*;

(1.2) — a character *output sequence*.

27.6.3.1 `basic_streambuf` constructors [streambuf.cons]

```
basic_streambuf();
```

1 *Effects:* Constructs an object of class `basic_streambuf<charT,traits>` and initializes:[307]

(1.1) — all its pointer member objects to null pointers,

(1.2) — the `getloc()` member to a copy the global locale, `locale()`, at the time of construction.

2 *Remarks:* Once the `getloc()` member is initialized, results of calling locale member functions, and of members of facets so obtained, can safely be cached until the next time the member `imbue` is called.

```
basic_streambuf(const basic_streambuf& rhs);
```

3 *Effects:* Constructs a copy of `rhs`.

4 *Postconditions:*

(4.1) — `eback() == rhs.eback()`

(4.2) — `gptr() == rhs.gptr()`

(4.3) — `egptr() == rhs.egptr()`

(4.4) — `pbase() == rhs.pbase()`

(4.5) — `pptr() == rhs.pptr()`

(4.6) — `epptr() == rhs.epptr()`

(4.7) — `getloc() == rhs.getloc()`

```
~basic_streambuf();
```

5 *Effects:* None.

27.6.3.2 `basic_streambuf` public member functions [streambuf.members]

27.6.3.2.1 Locales [streambuf.locales]

```
locale pubimbue(const locale& loc);
```

1 *Postcondition:* `loc == getloc()`.

2 *Effects:* Calls `imbue(loc)`.

3 *Returns:* Previous value of `getloc()`.

```
locale getloc() const;
```

4 *Returns:* If `pubimbue()` has ever been called, then the last value of `loc` supplied, otherwise the current global locale, `locale()`, in effect at the time of construction. If called after `pubimbue()` has been called but before `pubimbue` has returned (i.e., from within the call of `imbue()`) then it returns the previous value.

307) The default constructor is protected for class `basic_streambuf` to assure that only objects for classes derived from this class may be constructed.

27.6.3.2.2 Buffer management and positioning [streambuf.buffer]

```
basic_streambuf* pubsetbuf(char_type* s, streamsize n);
```

1 *Returns:* `setbuf(s, n)`.

```
pos_type pubseekoff(off_type off, ios_base::seekdir way,
                ios_base::openmode which = ios_base::in | ios_base::out);
```

2 *Returns:* `seekoff(off, way, which)`.

```
pos_type pubseekpos(pos_type sp,
                ios_base::openmode which = ios_base::in | ios_base::out);
```

3 *Returns:* `seekpos(sp, which)`.

```
int pubsync();
```

4 *Returns:* `sync()`.

27.6.3.2.3 Get area [streambuf.pub.get]

```
streamsize in_avail();
```

1 *Returns:* If a read position is available, returns `egptr() - gptr()`. Otherwise returns `showmanyc()` (27.6.3.4.3).

```
int_type snextc();
```

2 *Effects:* Calls `sbumpc()`.

3 *Returns:* If that function returns `traits::eof()`, returns `traits::eof()`. Otherwise, returns `sgetc()`.

```
int_type sbumpc();
```

4 *Returns:* If the input sequence read position is not available, returns `uflow()`. Otherwise, returns `traits::to_int_type(*gptr())` and increments the next pointer for the input sequence.

```
int_type sgetc();
```

5 *Returns:* If the input sequence read position is not available, returns `underflow()`. Otherwise, returns `traits::to_int_type(*gptr())`.

```
streamsize sgetn(char_type* s, streamsize n);
```

6 *Returns:* `xsgetn(s, n)`.

27.6.3.2.4 Putback [streambuf.pub.pback]

```
int_type sputbackc(char_type c);
```

1 *Returns:* If the input sequence putback position is not available, or if `traits::eq(c,gptr()[-1])` is false, returns `pbackfail(traits::to_int_type(c))`. Otherwise, decrements the next pointer for the input sequence and returns `traits::to_int_type(*gptr())`.

```
int_type sungetc();
```

2 *Returns:* If the input sequence putback position is not available, returns `pbackfail()`. Otherwise, decrements the next pointer for the input sequence and returns `traits::to_int_type(*gptr())`.

27.6.3.2.5 Put area [streambuf.pub.put]

```
int_type sputc(char_type c);
```

1 *Returns:* If the output sequence write position is not available, returns `overflow(traits::to_int_type(c))`. Otherwise, stores `c` at the next pointer for the output sequence, increments the pointer, and returns `traits::to_int_type(c)`.

```
streamsize sputn(const char_type* s, streamsize n);
```

2 *Returns:* `xsputn(s,n)`.

27.6.3.3 `basic_streambuf` protected member functions [streambuf.protected]

27.6.3.3.1 Assignment [streambuf.assign]

```
basic_streambuf& operator=(const basic_streambuf& rhs);
```

1 *Effects:* Assigns the data members of `rhs` to `*this`.

2 *Postconditions:*

(2.1) — `eback() == rhs.eback()`

(2.2) — `gptr() == rhs.gptr()`

(2.3) — `egptr() == rhs.egptr()`

(2.4) — `pbase() == rhs.pbase()`

(2.5) — `pptr() == rhs.pptr()`

(2.6) — `epptr() == rhs.epptr()`

(2.7) — `getloc() == rhs.getloc()`

3 *Returns:* `*this`.

```
void swap(basic_streambuf& rhs);
```

4 *Effects:* Swaps the data members of `rhs` and `*this`.

27.6.3.3.2 Get area access [streambuf.get.area]

```
char_type* eback() const;
```

1 *Returns:* The beginning pointer for the input sequence.

```
char_type* gptr() const;
```

2 *Returns:* The next pointer for the input sequence.

```
char_type* egptr() const;
```

3 *Returns:* The end pointer for the input sequence.

```
void gbump(int n);
```

4 *Effects:* Adds `n` to the next pointer for the input sequence.

```
void setg(char_type* gbeg, char_type* gnext, char_type* gend);
```

5 *Postconditions:* `gbeg == eback()`, `gnext == gptr()`, and `gend == egptr()`.

27.6.3.3.3 Put area access [streambuf.put.area]

```
char_type* pbase() const;
```

1 *Returns:* The beginning pointer for the output sequence.

```
char_type* pptr() const;
```

2 *Returns:* The next pointer for the output sequence.

```
char_type* epptr() const;
```

3 *Returns:* The end pointer for the output sequence.

```
void pbump(int n);
```

4 *Effects:* Adds `n` to the next pointer for the output sequence.

```
void setp(char_type* pbeg, char_type* pend);
```

5 *Postconditions:* `pbeg == pbase()`, `pbeg == pptr()`, and `pend == epptr()`.

27.6.3.4 `basic_streambuf` virtual functions [streambuf.virtuals]

27.6.3.4.1 Locales [streambuf.virt.locales]

```
void imbue(const locale&);
```

1 *Effects:* Change any translations based on locale.

2 *Remarks:* Allows the derived class to be informed of changes in locale at the time they occur. Between invocations of this function a class derived from streambuf can safely cache results of calls to locale functions and to members of facets so obtained.

3 *Default behavior:* Does nothing.

27.6.3.4.2 Buffer management and positioning [streambuf.virt.buffer]

```
basic_streambuf* setbuf(char_type* s, streamsize n);
```

1 *Effects:* Influences stream buffering in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (27.8.2.4, 27.9.1.5).

2 *Default behavior:* Does nothing. Returns `this`.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
                 ios_base::openmode which
                  = ios_base::in | ios_base::out);
```

3 *Effects:* Alters the stream positions within one or more of the controlled sequences in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (27.8.2.4, 27.9.1.5).

4 *Default behavior:* Returns `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp,
                 ios_base::openmode which
                  = ios_base::in | ios_base::out);
```

5 *Effects:* Alters the stream positions within one or more of the controlled sequences in a way that is defined separately for each class derived from `basic_streambuf` in this Clause (27.8.2, 27.9.1.1).

6 *Default behavior:* Returns `pos_type(off_type(-1))`.

```
int sync();
```

7 *Effects:* Synchronizes the controlled sequences with the arrays. That is, if `pbase()` is non-null the characters between `pbase()` and `pptr()` are written to the controlled sequence. The pointers may then be reset as appropriate.

8 *Returns:*-1 on failure. What constitutes failure is determined by each derived class (27.9.1.5).

9 *Default behavior:* Returns zero.

27.6.3.4.3 Get area [streambuf.virt.get]

```
streamsize showmanyc();
```[308]

1 *Returns:* An estimate of the number of characters available in the sequence, or -1. If it returns a positive value, then successive calls to `underflow()` will not return `traits::eof()` until at least that number of characters have been extracted from the stream. If `showmanyc()` returns -1, then calls to `underflow()` or `uflow()` will fail.[309]

2 *Default behavior:* Returns zero.

3 *Remarks:* Uses `traits::eof()`.

```
streamsize xsgetn(char_type* s, streamsize n);
```

4 *Effects:* Assigns up to `n` characters to successive elements of the array whose first element is designated by `s`. The characters assigned are read from the input sequence as if by repeated calls to `sbumpc()`. Assigning stops when either `n` characters have been assigned or a call to `sbumpc()` would return `traits::eof()`.

5 *Returns:* The number of characters assigned.[310]

6 *Remarks:* Uses `traits::eof()`.

```
int_type underflow();
```

7 *Remarks:* The public members of `basic_streambuf` call this virtual function only if `gptr()` is null or `gptr() >= egptr()`

8 *Returns:* `traits::to_int_type(c)`, where `c` is the first *character* of the *pending sequence*, without moving the input sequence position past it. If the pending sequence is null then the function returns `traits::eof()` to indicate failure.

9 The *pending sequence* of characters is defined as the concatenation of:

a) If `gptr()` is non-null, then the `egptr() - gptr()` characters starting at `gptr()`, otherwise the empty sequence.

b) Some sequence (possibly empty) of characters read from the input sequence.

10 The *result character* is

a) If the pending sequence is non-empty, the first character of the sequence.

b) If the pending sequence is empty then the next character that would be read from the input sequence.

11 The *backup sequence* is defined as the concatenation of:

308) The morphemes of `showmanyc` are "es-how-many-see", not "show-manic".
309) `underflow` or `uflow` might fail by throwing an exception prematurely. The intention is not only that the calls will not return `eof()` but that they will return "immediately."
310) Classes derived from `basic_streambuf` can provide more efficient ways to implement `xsgetn()` and `xsputn()` by overriding these definitions from the base class.

a) If `eback()` is null then empty,

b) Otherwise the `gptr() - eback()` characters beginning at `eback()`.

12 *Effects:* The function sets up the `gptr()` and `egptr()` satisfying one of:

a) If the pending sequence is non-empty, `egptr()` is non-null and `egptr() - gptr()` characters starting at `gptr()` are the characters in the pending sequence

b) If the pending sequence is empty, either `gptr()` is null or `gptr()` and `egptr()` are set to the same non-null pointer value.

13 If `eback()` and `gptr()` are non-null then the function is not constrained as to their contents, but the "usual backup condition" is that either:

a) If the backup sequence contains at least `gptr() - eback()` characters, then the `gptr() - eback()` characters starting at `eback()` agree with the last `gptr() - eback()` characters of the backup sequence.

b) Or the `n` characters starting at `gptr() - n` agree with the backup sequence (where `n` is the length of the backup sequence)

14 *Default behavior:* Returns `traits::eof()`.

```
int_type uflow();
```

15 *Requires:* The constraints are the same as for `underflow()`, except that the result character shall be transferred from the pending sequence to the backup sequence, and the pending sequence shall not be empty before the transfer.

16 *Default behavior:* Calls `underflow()`. If `underflow()` returns `traits::eof()`, returns `traits::eof()`. Otherwise, returns the value of `traits::to_int_type(*gptr())` and increment the value of the next pointer for the input sequence.

17 *Returns:* `traits::eof()` to indicate failure.

27.6.3.4.4 Putback [streambuf.virt.pback]

```
int_type pbackfail(int_type c = traits::eof());
```

1 *Remarks:* The public functions of `basic_streambuf` call this virtual function only when `gptr()` is null, `gptr() == eback()`, or `traits::eq(traits::to_char_type(c),gptr()[-1])` returns `false`. Other calls shall also satisfy that constraint.

The *pending sequence* is defined as for `underflow()`, with the modifications that

(1.1) — If `traits::eq_int_type(c,traits::eof())` returns `true`, then the input sequence is backed up one character before the pending sequence is determined.

(1.2) — If `traits::eq_int_type(c,traits::eof())` returns `false`, then `c` is prepended. Whether the input sequence is backed up or modified in any other way is unspecified.

2 *Postcondition:* On return, the constraints of `gptr()`, `eback()`, and `pptr()` are the same as for `underflow()`.

3 *Returns:* `traits::eof()` to indicate failure. Failure may occur because the input sequence could not be backed up, or if for some other reason the pointers could not be set consistent with the constraints. `pbackfail()` is called only when put back has really failed.

4 Returns some value other than `traits::eof()` to indicate success.

5 *Default behavior:* Returns `traits::eof()`.

27.6.3.4.5 Put area [streambuf.virt.put]

```
streamsize xsputn(const char_type* s, streamsize n);
```

1 *Effects:* Writes up to `n` characters to the output sequence as if by repeated calls to `sputc(c)`. The characters written are obtained from successive elements of the array whose first element is designated by `s`. Writing stops when either `n` characters have been written or a call to `sputc(c)` would return `traits::eof()`. Is is unspecified whether the function calls `overflow()` when `pptr() == epptr()` becomes true or whether it achieves the same effects by other means.

2 *Returns:* The number of characters written.

```
int_type overflow(int_type c = traits::eof());
```

3 *Effects:* Consumes some initial subsequence of the characters of the *pending sequence*. The pending sequence is defined as the concatenation of

a) if `pbase()` is null then the empty sequence otherwise, `pptr() - pbase()` characters beginning at `pbase()`.

b) if `traits::eq_int_type(c,traits::eof())` returns `true`, then the empty sequence otherwise, the sequence consisting of `c`.

4 *Remarks:* The member functions `sputc()` and `sputn()` call this function in case that no room can be found in the put buffer enough to accommodate the argument character sequence.

5 *Requires:* Every overriding definition of this virtual function shall obey the following constraints:

1) The effect of consuming a character on the associated output sequence is specified[311]

2) Let `r` be the number of characters in the pending sequence not consumed. If `r` is non-zero then `pbase()` and `pptr()` shall be set so that: `pptr() - pbase() == r` and the `r` characters starting at `pbase()` are the associated output stream. In case `r` is zero (all characters of the pending sequence have been consumed) then either `pbase()` is set to `nullptr`, or `pbase()` and `pptr()` are both set to the same non-null value.

3) The function may fail if either appending some character to the associated output stream fails or if it is unable to establish `pbase()` and `pptr()` according to the above rules.

6 *Returns:* `traits::eof()` or throws an exception if the function fails.

Otherwise, returns some value other than `traits::eof()` to indicate success.[312]

7 *Default behavior:* Returns `traits::eof()`.

27.7 Formatting and manipulators [iostream.format]

27.7.1 Overview [iostream.format.overview]

Header `<istream>` synopsis

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
    class basic_istream;
  typedef basic_istream<char>     istream;
  typedef basic_istream<wchar_t> wistream;
```

311) That is, for each class derived from an instance of `basic_streambuf` in this Clause (27.8.2, 27.9.1.1), a specification of how consuming a character effects the associated output sequence is given. There is no requirement on a program-defined class.
312) Typically, `overflow` returns `c` to indicate success, except when `traits::eq_int_type(c,traits::eof())` returns `true`, in which case it returns `traits::not_eof(c)`.

```
  template <class charT, class traits = char_traits<charT> >
    class basic_iostream;
  typedef basic_iostream<char>    iostream;
  typedef basic_iostream<wchar_t> wiostream;

  template <class charT, class traits>
    basic_istream<charT,traits>& ws(basic_istream<charT,traits>& is);

  template <class charT, class traits, class T>
    basic_istream<charT, traits>&
    operator>>(basic_istream<charT, traits>&& is, T& x);
}
```

Header <ostream> synopsis

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
    class basic_ostream;
  typedef basic_ostream<char>     ostream;
  typedef basic_ostream<wchar_t> wostream;

  template <class charT, class traits>
    basic_ostream<charT,traits>& endl(basic_ostream<charT,traits>& os);
  template <class charT, class traits>
    basic_ostream<charT,traits>& ends(basic_ostream<charT,traits>& os);
  template <class charT, class traits>
    basic_ostream<charT,traits>& flush(basic_ostream<charT,traits>& os);

  template <class charT, class traits, class T>
    basic_ostream<charT, traits>&
    operator<<(basic_ostream<charT, traits>&& os, const T& x);
}
```

Header <iomanip> synopsis

```
namespace std {
  // types T1, T2, ... are unspecified implementation types
  T1 resetiosflags(ios_base::fmtflags mask);
  T2 setiosflags  (ios_base::fmtflags mask);
  T3 setbase(int base);
  template<charT> T4 setfill(charT c);
  T5 setprecision(int n);
  T6 setw(int n);
  template <class moneyT> T7 get_money(moneyT& mon, bool intl = false);
  template <class moneyT> T8 put_money(const moneyT& mon, bool intl = false);
  template <class charT> T9 get_time(struct tm* tmb, const charT* fmt);
  template <class charT> T10 put_time(const struct tm* tmb, const charT* fmt);

  template <class charT>
    T11 quoted(const charT* s, charT delim=charT('"'), charT escape=charT('\\'));

  template <class charT, class traits, class Allocator>
    T12 quoted(const basic_string<charT, traits, Allocator>& s,
               charT delim=charT('"'), charT escape=charT('\\'));

  template <class charT, class traits, class Allocator>
```

```
    T13 quoted(basic_string<charT, traits, Allocator>& s,
               charT delim=charT('"'), charT escape=charT('\\'));
}
```

27.7.2 Input streams [input.streams]

1 The header <istream> defines two types and a function signature that control input from a stream buffer along with a function template that extracts from stream rvalues.

27.7.2.1 Class template basic_istream [istream]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_istream : virtual public basic_ios<charT,traits> {
  public:
    // types (inherited from basic_ios (27.5.5)):
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;

    // 27.7.2.1.1 Constructor/destructor:
    explicit basic_istream(basic_streambuf<charT,traits>* sb);
    virtual ~basic_istream();

    // 27.7.2.1.3 Prefix/suffix:
    class sentry;

    // 27.7.2.2 Formatted input:
    basic_istream<charT,traits>& operator>>(
      basic_istream<charT,traits>& (*pf)(basic_istream<charT,traits>&));
    basic_istream<charT,traits>& operator>>(
                        basic_ios<charT,traits>& (*pf)(basic_ios<charT,traits>&));
    basic_istream<charT,traits>& operator>>(
      ios_base& (*pf)(ios_base&));

    basic_istream<charT,traits>& operator>>(bool& n);
    basic_istream<charT,traits>& operator>>(short& n);
    basic_istream<charT,traits>& operator>>(unsigned short& n);
    basic_istream<charT,traits>& operator>>(int& n);
    basic_istream<charT,traits>& operator>>(unsigned int& n);
    basic_istream<charT,traits>& operator>>(long& n);
    basic_istream<charT,traits>& operator>>(unsigned long& n);
    basic_istream<charT,traits>& operator>>(long long& n);
    basic_istream<charT,traits>& operator>>(unsigned long long& n);
    basic_istream<charT,traits>& operator>>(float& f);
    basic_istream<charT,traits>& operator>>(double& f);
    basic_istream<charT,traits>& operator>>(long double& f);

    basic_istream<charT,traits>& operator>>(void*& p);
    basic_istream<charT,traits>& operator>>(
      basic_streambuf<char_type,traits>* sb);

    // 27.7.2.3 Unformatted input:
    streamsize gcount() const;
```

```
  int_type get();
  basic_istream<charT,traits>& get(char_type& c);
  basic_istream<charT,traits>& get(char_type* s, streamsize n);
  basic_istream<charT,traits>& get(char_type* s, streamsize n,
                                   char_type delim);
  basic_istream<charT,traits>& get(basic_streambuf<char_type,traits>& sb);
  basic_istream<charT,traits>& get(basic_streambuf<char_type,traits>& sb,
                                   char_type delim);

  basic_istream<charT,traits>& getline(char_type* s, streamsize n);
  basic_istream<charT,traits>& getline(char_type* s, streamsize n,
                                       char_type delim);

  basic_istream<charT,traits>& ignore(
    streamsize n = 1, int_type delim = traits::eof());
  int_type                     peek();
  basic_istream<charT,traits>& read    (char_type* s, streamsize n);
  streamsize                   readsome(char_type* s, streamsize n);

  basic_istream<charT,traits>& putback(char_type c);
  basic_istream<charT,traits>& unget();
  int sync();

  pos_type tellg();
  basic_istream<charT,traits>& seekg(pos_type);
  basic_istream<charT,traits>& seekg(off_type, ios_base::seekdir);

protected:
  basic_istream(const basic_istream& rhs) = delete;
  basic_istream(basic_istream&& rhs);

  // 27.7.2.1.2 Assign/swap:
  basic_istream& operator=(const basic_istream& rhs) = delete;
  basic_istream& operator=(basic_istream&& rhs);
  void swap(basic_istream& rhs);
};

// 27.7.2.2.3 character extraction templates:
template<class charT, class traits>
  basic_istream<charT,traits>& operator>>(basic_istream<charT,traits>&,
                                          charT&);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>&,
                                         unsigned char&);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>&,
                                         signed char&);

template<class charT, class traits>
  basic_istream<charT,traits>& operator>>(basic_istream<charT,traits>&,
                                          charT*);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>&,
                                         unsigned char*);
template<class traits>
```

```
    basic_istream<char,traits>& operator>>(basic_istream<char,traits>&,
                                           signed char*);
}
```

1 The class template `basic_istream` defines a number of member function signatures that assist in reading and interpreting input from sequences controlled by a stream buffer.

2 Two groups of member function signatures share common properties: the *formatted input functions* (or *extractors*) and the *unformatted input functions.* Both groups of input functions are described as if they obtain (or *extract*) input *characters* by calling `rdbuf()->sbumpc()` or `rdbuf()->sgetc()`. They may use other public members of `istream`.

3 If `rdbuf()->sbumpc()` or `rdbuf()->sgetc()` returns `traits::eof()`, then the input function, except as explicitly noted otherwise, completes its actions and does `setstate(eofbit)`, which may throw `ios_base::failure` (27.5.5.4), before returning.

4 If one of these called functions throws an exception, then unless explicitly noted otherwise, the input function sets `badbit` in error state. If `badbit` is on in `exceptions()`, the input function rethrows the exception without completing its actions, otherwise it does not throw anything and proceeds as if the called function had returned a failure indication.

27.7.2.1.1 `basic_istream` constructors [istream.cons]

```
explicit basic_istream(basic_streambuf<charT,traits>* sb);
```

1 *Effects:* Constructs an object of class `basic_istream`, assigning initial values to the base class by calling `basic_ios::init(sb)` (27.5.5.2).

2 *Postcondition:* `gcount() == 0`

```
basic_istream(basic_istream&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by default constructing the base class, copying the `gcount()` from `rhs`, calling `basic_ios<charT, traits>::move(rhs)` to initialize the base class, and setting the `gcount()` for `rhs` to 0.

```
virtual ~basic_istream();
```

4 *Effects:* Destroys an object of class `basic_istream`.

5 *Remarks:* Does not perform any operations of `rdbuf()`.

27.7.2.1.2 Class `basic_istream` assign and swap [istream.assign]

```
basic_istream& operator=(basic_istream&& rhs);
```

1 *Effects:* `swap(rhs);`.

2 *Returns:* `*this`.

```
void swap(basic_istream& rhs);
```

3 *Effects:* Calls `basic_ios<charT, traits>::swap(rhs)`. Exchanges the values returned by `gcount()` and `rhs.gcount()`.

27.7.2.1.3 Class `basic_istream::sentry` [istream::sentry]

```
namespace std {
  template <class charT,class traits = char_traits<charT> >
  class basic_istream<charT,traits>::sentry {
    typedef traits traits_type;
    bool ok_; // exposition only
```

```
  public:
    explicit sentry(basic_istream<charT,traits>& is, bool noskipws = false);
    ~sentry();
    explicit operator bool() const { return ok_; }
    sentry(const sentry&) = delete;
    sentry& operator=(const sentry&) = delete;
  };
}
```

1 The class `sentry` defines a class that is responsible for doing exception safe prefix and suffix operations.

`explicit sentry(basic_istream<charT,traits>& is, bool noskipws = false);`

2 *Effects:* If `is.good()` is `false`, calls `is.setstate(failbit)`. Otherwise, prepares for formatted or unformatted input. First, if `is.tie()` is not a null pointer, the function calls `is.tie()->flush()` to synchronize the output sequence with any associated external C stream. Except that this call can be suppressed if the put area of `is.tie()` is empty. Further an implementation is allowed to defer the call to `flush` until a call of `is.rdbuf()->underflow()` occurs. If no such call occurs before the `sentry` object is destroyed, the call to `flush` may be eliminated entirely.[313] If `noskipws` is zero and `is.flags() & ios_base::skipws` is nonzero, the function extracts and discards each character as long as the next available input character `c` is a whitespace character. If `is.rdbuf()->sbumpc()` or `is.rdbuf()->sgetc()` returns `traits::eof()`, the function calls `setstate(failbit | eofbit)` (which may throw `ios_base::failure`).

3 *Remarks:* The constructor `explicit sentry(basic_istream<charT,traits>& is, bool noskipws = false)` uses the currently imbued locale in `is`, to determine whether the next input character is whitespace or not.

4 To decide if the character `c` is a whitespace character, the constructor performs as if it executes the following code fragment:

```
const ctype<charT>& ctype = use_facet<ctype<charT> >(is.getloc());
if (ctype.is(ctype.space,c)!=0)
  // c is a whitespace character.
```

5 If, after any preparation is completed, `is.good()` is `true`, `ok_ != false` otherwise, `ok_ == false`. During preparation, the constructor may call `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4))[314]

`~sentry();`

6 *Effects:* None.

`explicit operator bool() const;`

7 *Effects:* Returns `ok_`.

27.7.2.2 Formatted input functions [istream.formatted]

27.7.2.2.1 Common requirements [istream.formatted.reqmts]

1 Each formatted input function begins execution by constructing an object of class `sentry` with the `noskipws` (second) argument `false`. If the `sentry` object returns `true`, when converted to a value of type `bool`, the function endeavors to obtain the requested input. If an exception is thrown during input then `ios::badbit`

313) This will be possible only in functions that are part of the library. The semantics of the constructor used in user code is as specified.

314) The sentry constructor and destructor can also perform additional implementation-dependent operations.

is turned on[315] in `*this`'s error state. If `(exceptions()&badbit) != 0` then the exception is rethrown. In any case, the formatted input function destroys the `sentry` object. If no exception has been thrown, it returns `*this`.

27.7.2.2.2 Arithmetic extractors [istream.formatted.arithmetic]

```
operator>>(unsigned short& val);
operator>>(unsigned int& val);
operator>>(long& val);
operator>>(unsigned long& val);
operator>>(long long& val);
operator>>(unsigned long long& val);
operator>>(float& val);
operator>>(double& val);
operator>>(long double& val);
operator>>(bool& val);
operator>>(void*& val);
```

1 As in the case of the inserters, these extractors depend on the locale's `num_get<>` (22.4.2.1) object to perform parsing the input stream data. These extractors behave as formatted input functions (as described in 27.7.2.2.1). After a sentry object is constructed, the conversion occurs as if performed by the following code fragment:

```
typedef num_get< charT,istreambuf_iterator<charT,traits> > numget;
iostate err = iostate::goodbit;
use_facet< numget >(loc).get(*this, 0, *this, err, val);
setstate(err);
```

In the above fragment, `loc` stands for the private member of the `basic_ios` class. [*Note:* The first argument provides an object of the `istreambuf_iterator` class which is an iterator pointed to an input stream. It bypasses istreams and uses streambufs directly. — *end note*] Class `locale` relies on this type as its interface to `istream`, so that it does not need to depend directly on `istream`.

```
operator>>(short& val);
```

2 The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
typedef num_get<charT,istreambuf_iterator<charT,traits> > numget;
iostate err = ios_base::goodbit;
long lval;
use_facet<numget>(loc).get(*this, 0, *this, err, lval);
if (lval < numeric_limits<short>::min()) {
  err |= ios_base::failbit;
  val = numeric_limits<short>::min();
} else if (numeric_limits<short>::max() < lval) {
  err |= ios_base::failbit;
  val = numeric_limits<short>::max();
}  else
  val = static_cast<short>(lval);
setstate(err);
```

```
operator>>(int& val);
```

315) This is done without causing an `ios::failure` to be thrown.

3 The conversion occurs as if performed by the following code fragment (using the same notation as for the preceding code fragment):

```
typedef num_get<charT,istreambuf_iterator<charT,traits> > numget;
iostate err = ios_base::goodbit;
long lval;
use_facet<numget>(loc).get(*this, 0, *this, err, lval);
if (lval < numeric_limits<int>::min()) {
  err |= ios_base::failbit;
  val = numeric_limits<int>::min();
} else if (numeric_limits<int>::max() < lval) {
  err |= ios_base::failbit;
  val = numeric_limits<int>::max();
}  else
  val = static_cast<int>(lval);
setstate(err);
```

27.7.2.2.3 `basic_istream::operator>>` [istream::extractors]

```
basic_istream<charT,traits>& operator>>
    (basic_istream<charT,traits>& (*pf)(basic_istream<charT,traits>&));
```

1 *Effects:* None. This extractor does not behave as a formatted input function (as described in 27.7.2.2.1.)

2 *Returns:* `pf(*this)`.[316]

```
basic_istream<charT,traits>& operator>>
    (basic_ios<charT,traits>& (*pf)(basic_ios<charT,traits>&));
```

3 *Effects:* Calls `pf(*this)`. This extractor does not behave as a formatted input function (as described in 27.7.2.2.1).

4 *Returns:* `*this`.

```
basic_istream<charT,traits>& operator>>
    (ios_base& (*pf)(ios_base&));
```

5 *Effects:* Calls `pf(*this)`.[317] This extractor does not behave as a formatted input function (as described in 27.7.2.2.1).

6 *Returns:* `*this`.

```
template<class charT, class traits>
  basic_istream<charT,traits>& operator>>(basic_istream<charT,traits>& in,
                                          charT* s);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>& in,
                                         unsigned char* s);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>& in,
                                         signed char* s);
```

7 *Effects:* Behaves like a formatted input member (as described in 27.7.2.2.1) of `in`. After a `sentry` object is constructed, `operator>>` extracts characters and stores them into successive locations of an array whose first element is designated by `s`. If `width()` is greater than zero, `n` is `width()`. Otherwise

316) See, for example, the function signature `ws(basic_istream&)` (27.7.2.4).
317) See, for example, the function signature `dec(ios_base&)` (27.5.6.3).

`n` is the number of elements of the largest array of `char_type` that can store a terminating `charT()`. `n` is the maximum number of characters stored.

8 Characters are extracted and stored until any of the following occurs:

(8.1) — `n-1` characters are stored;

(8.2) — end of file occurs on the input sequence;

(8.3) — `ct.is(ct.space,c)` is `true` for the next available input character `c`, where `ct` is `use_facet<ctype< charT> >(in.getloc())`.

9 `operator>>` then stores a null byte (`charT()`) in the next position, which may be the first position if no characters were extracted. `operator>>` then calls `width(0)`.

10 If the function extracted no characters, it calls `setstate(failbit)`, which may throw `ios_base::failure` (27.5.5.4).

11 *Returns:* `in`.

```
template<class charT, class traits>
  basic_istream<charT,traits>& operator>>(basic_istream<charT,traits>& in,
                                          charT& c);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>& in,
                                         unsigned char& c);
template<class traits>
  basic_istream<char,traits>& operator>>(basic_istream<char,traits>& in,
                                         signed char& c);
```

12 *Effects:* Behaves like a formatted input member (as described in 27.7.2.2.1) of `in`. After a `sentry` object is constructed a character is extracted from `in`, if one is available, and stored in `c`. Otherwise, the function calls `in.setstate(failbit)`.

13 *Returns:* `in`.

```
basic_istream<charT,traits>& operator>>
  (basic_streambuf<charT,traits>* sb);
```

14 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). If `sb` is null, calls `setstate(failbit)`, which may throw `ios_base::failure` (27.5.5.4). After a sentry object is constructed, extracts characters from `*this` and inserts them in the output sequence controlled by `sb`. Characters are extracted and inserted until any of the following occurs:

(14.1) — end-of-file occurs on the input sequence;

(14.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);

(14.3) — an exception occurs (in which case the exception is caught).

15 If the function inserts no characters, it calls `setstate(failbit)`, which may throw `ios_base::failure` (27.5.5.4). If it inserted no characters because it caught an exception thrown while extracting characters from `*this` and `failbit` is on in `exceptions()` (27.5.5.4), then the caught exception is rethrown.

16 *Returns:* `*this`.

27.7.2.3 Unformatted input functions [istream.unformatted]

1 Each unformatted input function begins execution by constructing an object of class `sentry` with the default argument `noskipws` (second) argument `true`. If the `sentry` object returns `true`, when converted to a value of type `bool`, the function endeavors to obtain the requested input. Otherwise, if the sentry constructor exits by throwing an exception or if the sentry object returns false, when converted to a value of type `bool`, the function returns without attempting to obtain any input. In either case the number of extracted characters is set to 0; unformatted input functions taking a character array of non-zero size as an argument shall also store a null character (using `charT()`) in the first location of the array. If an exception is thrown during input then `ios::badbit` is turned on[318] in `*this`'s error state. (Exceptions thrown from `basic_ios<>::clear()` are not caught or rethrown.) If `(exceptions()&badbit) != 0` then the exception is rethrown. It also counts the number of characters extracted. If no exception has been thrown it ends by storing the count in a member object and returning the value specified. In any event the `sentry` object is destroyed before leaving the unformatted input function.

```
streamsize gcount() const;
```

2 *Effects:* None. This member function does not behave as an unformatted input function (as described in 27.7.2.3, paragraph 1).

3 *Returns:* The number of characters extracted by the last unformatted input member function called for the object.

```
int_type get();
```

4 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts a character `c`, if one is available. Otherwise, the function calls `setstate(failbit)`, which may throw `ios_base::failure` (27.5.5.4),

5 *Returns:* `c` if available, otherwise `traits::eof()`.

```
basic_istream<charT,traits>& get(char_type& c);
```

6 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts a character, if one is available, and assigns it to `c`.[319] Otherwise, the function calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

7 *Returns:* `*this`.

```
basic_istream<charT,traits>& get(char_type* s, streamsize n,
                 char_type delim );
```

8 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts characters and stores them into successive locations of an array whose first element is designated by `s`.[320] Characters are extracted and stored until any of the following occurs:

(8.1) — `n` is less than one or `n - 1` characters are stored;

(8.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`);

(8.3) — `traits::eq(c, delim)` for the next available input character `c` (in which case `c` is not extracted).

9 If the function stores no characters, it calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)). In any case, if `n` is greater than zero it then stores a null character into the next successive location of the array.

10 *Returns:* `*this`.

318) This is done without causing an `ios::failure` to be thrown.
319) Note that this function is not overloaded on types `signed char` and `unsigned char`.
320) Note that this function is not overloaded on types `signed char` and `unsigned char`.

```
basic_istream<charT,traits>& get(char_type* s, streamsize n);
```

11 *Effects:* Calls `get(s,n,widen('\n'))`

12 *Returns:* Value returned by the call.

```
basic_istream<charT,traits>& get(basic_streambuf<char_type,traits>& sb,
                  char_type delim );
```

13 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts characters and inserts them in the output sequence controlled by `sb`. Characters are extracted and inserted until any of the following occurs:

(13.1) — end-of-file occurs on the input sequence;

(13.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);

(13.3) — `traits::eq(c, delim)` for the next available input character `c` (in which case `c` is not extracted);

(13.4) — an exception occurs (in which case, the exception is caught but not rethrown).

14 If the function inserts no characters, it calls `setstate(failbit)`, which may throw `ios_base::failure` (27.5.5.4).

15 *Returns:* `*this`.

```
basic_istream<charT,traits>& get(basic_streambuf<char_type,traits>& sb);
```

16 *Effects:* Calls `get(sb, widen('\n'))`

17 *Returns:* Value returned by the call.

```
basic_istream<charT,traits>& getline(char_type* s, streamsize n,
                     char_type delim);
```

18 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts characters and stores them into successive locations of an array whose first element is designated by `s`.[321] Characters are extracted and stored until one of the following occurs:

1. end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`);
2. `traits::eq(c, delim)` for the next available input character `c` (in which case the input character is extracted but not stored);[322]
3. `n` is less than one or `n - 1` characters are stored (in which case the function calls `setstate(failbit)`).

19 These conditions are tested in the order shown.[323]

20 If the function extracts no characters, it calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).[324]

21 In any case, if `n` is greater than zero, it then stores a null character (using `charT()`) into the next successive location of the array.

22 *Returns:* `*this`.

23 [*Example:*

321) Note that this function is not overloaded on types `signed char` and `unsigned char`.
322) Since the final input character is "extracted," it is counted in the `gcount()`, even though it is not stored.
323) This allows an input line which exactly fills the buffer, without setting `failbit`. This is different behavior than the historical AT&T implementation.
324) This implies an empty input line will not cause `failbit` to be set.

```
#include <iostream>

int main() {
  using namespace std;
  const int line_buffer_size = 100;

  char buffer[line_buffer_size];
  int line_number = 0;
  while (cin.getline(buffer, line_buffer_size, '\n') || cin.gcount()) {
    int count = cin.gcount();
    if (cin.eof())
      cout << "Partial final line";   // cin.fail() is false
    else if (cin.fail()) {
      cout << "Partial long line";
      cin.clear(cin.rdstate() & ~ios_base::failbit);
    } else {
      count--;                        // Don't include newline in count
      cout << "Line " << ++line_number;
    }
    cout << " (" << count << " chars): " << buffer << endl;
  }
}
```

— *end example*]

```
basic_istream<charT,traits>& getline(char_type* s, streamsize n);
```

24 *Returns:* `getline(s,n,widen('\n'))`

```
basic_istream<charT,traits>&
    ignore(streamsize n = 1, int_type delim = traits::eof());
```

25 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, extracts characters and discards them. Characters are extracted until any of the following occurs:

(25.1) — `n != numeric_limits<streamsize>::max()` (18.3.2) and `n` characters have been extracted so far

(25.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(eofbit)`, which may throw `ios_base::failure` (27.5.5.4));

(25.3) — `traits::eq_int_type(traits::to_int_type(c), delim)` for the next available input character `c` (in which case `c` is extracted).

26 *Remarks:* The last condition will never occur if `traits::eq_int_type(delim, traits::eof())`.

27 *Returns:* `*this`.

```
int_type peek();
```

28 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, reads but does not extract the current input character.

29 *Returns:* `traits::eof()` if `good()` is `false`. Otherwise, returns `rdbuf()->sgetc()`.

```
basic_istream<charT,traits>& read(char_type* s, streamsize n);
```

30 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. Otherwise extracts characters and stores them into successive locations of an array whose first element is designated by `s`.[325] Characters are extracted and stored until either of the following occurs:

(30.1) — `n` characters are stored;

(30.2) — end-of-file occurs on the input sequence (in which case the function calls `setstate(failbit | eofbit)`, which may throw `ios_base::failure` (27.5.5.4)).

31 *Returns:* `*this`.

```
streamsize readsome(char_type* s, streamsize n);
```

32 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1). After constructing a sentry object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. Otherwise extracts characters and stores them into successive locations of an array whose first element is designated by `s`. If `rdbuf()->in_avail() == -1`, calls `setstate(eofbit)` (which may throw `ios_base::failure` (27.5.5.4)), and extracts no characters;

(32.1) — If `rdbuf()->in_avail() == 0`, extracts no characters

(32.2) — If `rdbuf()->in_avail() > 0`, extracts `min(rdbuf()->in_avail(),n))`.

33 *Returns:* The number of characters extracted.

```
basic_istream<charT,traits>& putback(char_type c);
```

34 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that the function first clears `eofbit`. After constructing a sentry object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. If `rdbuf()` is not null, calls `rdbuf->sputbackc()`. If `rdbuf()` is null, or if `sputbackc()` returns `traits::eof()`, calls `setstate(badbit)` (which may throw `ios_base::failure` (27.5.5.4)). [*Note:* This function extracts no characters, so the value returned by the next call to `gcount()` is 0. — *end note*]

35 *Returns:* `*this`.

```
basic_istream<charT,traits>& unget();
```

36 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that the function first clears `eofbit`. After constructing a sentry object, if `!good()` calls `setstate(failbit)` which may throw an exception, and return. If `rdbuf()` is not null, calls `rdbuf()->sungetc()`. If `rdbuf()` is null, or if `sungetc()` returns `traits::eof()`, calls `setstate(badbit)` (which may throw `ios_base::failure` (27.5.5.4)). [*Note:* This function extracts no characters, so the value returned by the next call to `gcount()` is 0. — *end note*]

37 *Returns:* `*this`.

```
int sync();
```

38 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `gcount()`. After constructing a sentry object, if `rdbuf()` is a null pointer, returns -1 . Otherwise, calls `rdbuf()->pubsync()` and, if that function returns -1 calls `setstate(badbit)` (which may throw `ios_base::failure` (27.5.5.4), and returns `-1`. Otherwise, returns zero.

```
pos_type tellg();
```

325) Note that this function is not overloaded on types `signed char` and `unsigned char`.

39 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `gcount()`.

40 *Returns:* After constructing a sentry object, if `fail() != false`, returns `pos_type(-1)` to indicate failure. Otherwise, returns `rdbuf()->pubseekoff(0, cur, in)`.

```
basic_istream<charT,traits>& seekg(pos_type pos);
```

41 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that the function first clears `eofbit`, it does not count the number of characters extracted, and it does not affect the value returned by subsequent calls to `gcount()`. After constructing a sentry object, if `fail() != true`, executes `rdbuf()->pubseekpos(pos, ios_base::in)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

42 *Returns:* `*this`.

```
basic_istream<charT,traits>& seekg(off_type off, ios_base::seekdir dir);
```

43 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to `gcount()`. After constructing a sentry object, if `fail() != true`, executes `rdbuf()->pubseekoff(off, dir, ios_base::in)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

44 *Returns:* `*this`.

27.7.2.4 Standard `basic_istream` manipulators [istream.manip]

```
template <class charT, class traits>
  basic_istream<charT,traits>& ws(basic_istream<charT,traits>& is);
```

1 *Effects:* Behaves as an unformatted input function (as described in 27.7.2.3, paragraph 1), except that it does not count the number of characters extracted and does not affect the value returned by subsequent calls to is.gcount(). After constructing a sentry object extracts characters as long as the next available character `c` is whitespace or until there are no more characters in the sequence. Whitespace characters are distinguished with the same criterion as used by `sentry::sentry` (27.7.2.1.3). If `ws` stops extracting characters because there are no more available it sets `eofbit`, but not `failbit`.

2 *Returns:* `is`.

27.7.2.5 Class template `basic_iostream` [iostreamclass]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_iostream :
    public basic_istream<charT,traits>,
    public basic_ostream<charT,traits> {
  public:
    // types:
    typedef charT                    char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                   traits_type;

    // constructor/destructor
    explicit basic_iostream(basic_streambuf<charT,traits>* sb);
    virtual ~basic_iostream();
```

```
  protected:
    basic_iostream(const basic_iostream& rhs) = delete;
    basic_iostream(basic_iostream&& rhs);

    // assign/swap
    basic_iostream& operator=(const basic_iostream& rhs) = delete;
    basic_iostream& operator=(basic_iostream&& rhs);
    void swap(basic_iostream& rhs);
  };
}
```

1 The class template `basic_iostream` inherits a number of functions that allow reading input and writing output to sequences controlled by a stream buffer.

27.7.2.5.1 `basic_iostream` constructors [iostream.cons]

```
explicit basic_iostream(basic_streambuf<charT,traits>* sb);
```

1 *Effects:* Constructs an object of class `basic_iostream`, assigning initial values to the base classes by calling `basic_istream<charT,traits>(sb)` (27.7.2.1) and `basic_ostream<charT,traits>(sb)` (27.7.3.1)

2 *Postcondition:* `rdbuf()==sb` and `gcount()==0`.

```
basic_iostream(basic_iostream&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs` by constructing the `basic_istream` base class with `move(rhs)`.

27.7.2.5.2 `basic_iostream` destructor [iostream.dest]

```
virtual ~basic_iostream();
```

1 *Effects:* Destroys an object of class `basic_iostream`.

2 *Remarks:* Does not perform any operations on `rdbuf()`.

27.7.2.5.3 `basic_iostream` assign and swap [iostream.assign]

```
basic_iostream& operator=(basic_iostream&& rhs);
```

1 *Effects:* `swap(rhs)`.

```
void swap(basic_iostream& rhs);
```

2 *Effects:* Calls `basic_istream<charT, traits>::swap(rhs)`.

27.7.2.6 Rvalue stream extraction [istream.rvalue]

```
template <class charT, class traits, class T>
  basic_istream<charT, traits>&
  operator>>(basic_istream<charT, traits>&& is, T& x);
```

1 *Effects:* `is >>x`

2 *Returns:* `is`

27.7.3 Output streams [output.streams]

1 The header `<ostream>` defines a type and several function signatures that control output to a stream buffer along with a function template that inserts into stream rvalues.

27.7.3.1 Class template basic_ostream [ostream]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_ostream : virtual public basic_ios<charT,traits> {
  public:
    // types (inherited from basic_ios (27.5.5)):
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;

    // 27.7.3.2 Constructor/destructor:
    explicit basic_ostream(basic_streambuf<char_type,traits>* sb);
    virtual ~basic_ostream();

    // 27.7.3.4 Prefix/suffix:
    class sentry;

    // 27.7.3.6 Formatted output:
    basic_ostream<charT,traits>& operator<<(
      basic_ostream<charT,traits>& (*pf)(basic_ostream<charT,traits>&));
    basic_ostream<charT,traits>& operator<<(
      basic_ios<charT,traits>& (*pf)(basic_ios<charT,traits>&));
    basic_ostream<charT,traits>& operator<<(
      ios_base& (*pf)(ios_base&));

    basic_ostream<charT,traits>& operator<<(bool n);
    basic_ostream<charT,traits>& operator<<(short n);
    basic_ostream<charT,traits>& operator<<(unsigned short n);
    basic_ostream<charT,traits>& operator<<(int n);
    basic_ostream<charT,traits>& operator<<(unsigned int n);
    basic_ostream<charT,traits>& operator<<(long n);
    basic_ostream<charT,traits>& operator<<(unsigned long n);
    basic_ostream<charT,traits>& operator<<(long long n);
    basic_ostream<charT,traits>& operator<<(unsigned long long n);
    basic_ostream<charT,traits>& operator<<(float f);
    basic_ostream<charT,traits>& operator<<(double f);
    basic_ostream<charT,traits>& operator<<(long double f);

    basic_ostream<charT,traits>& operator<<(const void* p);
    basic_ostream<charT,traits>& operator<<(
      basic_streambuf<char_type,traits>* sb);

    // 27.7.3.7 Unformatted output:
    basic_ostream<charT,traits>& put(char_type c);
    basic_ostream<charT,traits>& write(const char_type* s, streamsize n);

    basic_ostream<charT,traits>& flush();

    // 27.7.3.5 seeks:
    pos_type tellp();
    basic_ostream<charT,traits>& seekp(pos_type);
    basic_ostream<charT,traits>& seekp(off_type, ios_base::seekdir);
  protected:
```

```
    basic_ostream(const basic_ostream& rhs) = delete;
    basic_ostream(basic_ostream&& rhs);

    // 27.7.3.3 Assign/swap
    basic_ostream& operator=(const basic_ostream& rhs) = delete;
    basic_ostream& operator=(basic_ostream&& rhs);
    void swap(basic_ostream& rhs);
  };

  // 27.7.3.6.4 character inserters
  template<class charT, class traits>
    basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>&,
                                            charT);
  template<class charT, class traits>
    basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>&,
                                            char);
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           char);

  // signed and unsigned
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           signed char);
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           unsigned char);

  template<class charT, class traits>
    basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>&,
                                            const charT*);
  template<class charT, class traits>
    basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>&,
                                            const char*);
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           const char*);

  // signed and unsigned
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           const signed char*);
  template<class traits>
    basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>&,
                                           const unsigned char*);
}
```

1 The class template `basic_ostream` defines a number of member function signatures that assist in formatting and writing output to output sequences controlled by a stream buffer.

2 Two groups of member function signatures share common properties: the *formatted output functions* (or *inserters*) and the *unformatted output functions*. Both groups of output functions generate (or *insert*) output *characters* by actions equivalent to calling `rdbuf()->sputc(int_type)`. They may use other public members of `basic_ostream` except that they shall not invoke any virtual members of `rdbuf()` except `overflow()`, `xsputn()`, and `sync()`.

3 If one of these called functions throws an exception, then unless explicitly noted otherwise the output function sets `badbit` in error state. If `badbit` is on in `exceptions()`, the output function rethrows the exception without completing its actions, otherwise it does not throw anything and treat as an error.

27.7.3.2 `basic_ostream` constructors [ostream.cons]

```
explicit basic_ostream(basic_streambuf<charT,traits>* sb);
```

1 *Effects:* Constructs an object of class `basic_ostream`, assigning initial values to the base class by calling `basic_ios<charT,traits>::init(sb)` (27.5.5.2).

2 *Postcondition:* `rdbuf() == sb`.

```
virtual ~basic_ostream();
```

3 *Effects:* Destroys an object of class `basic_ostream`.

4 *Remarks:* Does not perform any operations on `rdbuf()`.

```
basic_ostream(basic_ostream&& rhs);
```

5 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by default constructing the base class and calling `basic_ios<charT, traits>::move(rhs)` to initialize the base class.

27.7.3.3 Class `basic_ostream` assign and swap [ostream.assign]

```
basic_ostream& operator=(basic_ostream&& rhs);
```

1 *Effects:* `swap(rhs)`.

2 *Returns:* `*this`.

```
void swap(basic_ostream& rhs);
```

3 *Effects:* Calls `basic_ios<charT, traits>::swap(rhs)`.

27.7.3.4 Class `basic_ostream::sentry` [ostream::sentry]

```
namespace std {
  template <class charT,class traits = char_traits<charT> >
  class basic_ostream<charT,traits>::sentry {
    bool ok_; // exposition only
  public:
    explicit sentry(basic_ostream<charT,traits>& os);
    ~sentry();
    explicit operator bool() const { return ok_; }

    sentry(const sentry&) = delete;
    sentry& operator=(const sentry&) = delete;
  };
}
```

1 The class `sentry` defines a class that is responsible for doing exception safe prefix and suffix operations.

```
explicit sentry(basic_ostream<charT,traits>& os);
```

2 If `os.good()` is nonzero, prepares for formatted or unformatted output. If `os.tie()` is not a null pointer, calls `os.tie()->flush()`.[326]

326) The call `os.tie()->flush()` does not necessarily occur if the function can determine that no synchronization is necessary.

3 If, after any preparation is completed, `os.good()` is `true`, `ok_ == true` otherwise, `ok_ == false`. During preparation, the constructor may call `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4))[327]

`~sentry();`

4 If `(os.flags() & ios_base::unitbuf) && !uncaught_exceptions() && os.good()` is `true`, calls `os.rdbuf()->pubsync()`. If that function returns -1, sets `badbit` in `os.rdstate()` without propagating an exception.

`explicit operator bool() const;`

5 *Effects:* Returns `ok_`.

27.7.3.5 `basic_ostream` seek members [ostream.seeks]

1 Each seek member function begins execution by constructing an object of class `sentry`. It returns by destroying the `sentry` object.

`pos_type tellp();`

2 *Returns:* If `fail() != false`, returns `pos_type(-1)` to indicate failure. Otherwise, returns `rdbuf()->pubseekoff(0, cur, out)`.

`basic_ostream<charT,traits>& seekp(pos_type pos);`

3 *Effects:* If `fail() != true`, executes `rdbuf()->pubseekpos(pos, ios_base::out)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

4 *Returns:* `*this`.

`basic_ostream<charT,traits>& seekp(off_type off, ios_base::seekdir dir);`

5 *Effects:* If `fail() != true`, executes `rdbuf()->pubseekoff(off, dir, ios_base::out)`. In case of failure, the function calls `setstate(failbit)` (which may throw `ios_base::failure`).

6 *Returns:* `*this`.

27.7.3.6 Formatted output functions [ostream.formatted]

27.7.3.6.1 Common requirements [ostream.formatted.reqmts]

1 Each formatted output function begins execution by constructing an object of class `sentry`. If this object returns `true` when converted to a value of type `bool`, the function endeavors to generate the requested output. If the generation fails, then the formatted output function does `setstate(ios_base::failbit)`, which might throw an exception. If an exception is thrown during output, then `ios::badbit` is turned on[328] in `*this`'s error state. If `(exceptions()&badbit) != 0` then the exception is rethrown. Whether or not an exception is thrown, the `sentry` object is destroyed before leaving the formatted output function. If no exception is thrown, the result of the formatted output function is `*this`.

2 The descriptions of the individual formatted output functions describe how they perform output and do not mention the `sentry` object.

3 If a formatted output function of a stream `os` determines padding, it does so as follows. Given a `charT` character sequence `seq` where `charT` is the character type of the stream, if the length of `seq` is less than `os.width()`, then enough copies of `os.fill()` are added to this sequence as necessary to pad to a width of `os.width()` characters. If `(os.flags() & ios_base::adjustfield) == ios_base::left` is `true`, the fill characters are placed after the character sequence; otherwise, they are placed before the character sequence.

327) The `sentry` constructor and destructor can also perform additional implementation-dependent operations.
328) without causing an `ios::failure` to be thrown.

27.7.3.6.2 Arithmetic inserters [ostream.inserters.arithmetic]

```
operator<<(bool val);
operator<<(short val);
operator<<(unsigned short val);
operator<<(int val);
operator<<(unsigned int val);
operator<<(long val);
operator<<(unsigned long val);
operator<<(long long val);
operator<<(unsigned long long val);
operator<<(float val);
operator<<(double val);
operator<<(long double val);
operator<<(const void* val);
```

1 *Effects:* The classes `num_get<>` and `num_put<>` handle locale-dependent numeric formatting and parsing. These inserter functions use the imbued `locale` value to perform numeric formatting. When `val` is of type `bool`, `long`, `unsigned long`, `long long`, `unsigned long long`, `double`, `long double`, or `const void*`, the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT,ostreambuf_iterator<charT,traits> >
    >(getloc()).put(*this, *this, fill(), val).failed();
```

When `val` is of type `short` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<
  num_put<charT,ostreambuf_iterator<charT,traits> >
    >(getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
      ? static_cast<long>(static_cast<unsigned short>(val))
      : static_cast<long>(val)).failed();
```

When `val` is of type `int` the formatting conversion occurs as if it performed the following code fragment:

```
ios_base::fmtflags baseflags = ios_base::flags() & ios_base::basefield;
bool failed = use_facet<
  num_put<charT,ostreambuf_iterator<charT,traits> >
    >(getloc()).put(*this, *this, fill(),
    baseflags == ios_base::oct || baseflags == ios_base::hex
      ? static_cast<long>(static_cast<unsigned int>(val))
      : static_cast<long>(val)).failed();
```

When `val` is of type `unsigned short` or `unsigned int` the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT,ostreambuf_iterator<charT,traits> >
    >(getloc()).put(*this, *this, fill(),
      static_cast<unsigned long>(val)).failed();
```

When `val` is of type `float` the formatting conversion occurs as if it performed the following code fragment:

```
bool failed = use_facet<
  num_put<charT,ostreambuf_iterator<charT,traits> >
    >(getloc()).put(*this, *this, fill(),
      static_cast<double>(val)).failed();
```

2 The first argument provides an object of the `ostreambuf_iterator<>` class which is an iterator for class `basic_ostream<>`. It bypasses `ostream`s and uses `streambuf`s directly. Class `locale` relies on these types as its interface to iostreams, since for flexibility it has been abstracted away from direct dependence on `ostream`. The second parameter is a reference to the base subobject of type `ios_base`. It provides formatting specifications such as field width, and a locale from which to obtain other facets. If `failed` is `true` then does `setstate(badbit)`, which may throw an exception, and returns.

3 *Returns:* `*this`.

27.7.3.6.3 `basic_ostream::operator<<` [ostream.inserters]

```
basic_ostream<charT,traits>& operator<<
    (basic_ostream<charT,traits>& (*pf)(basic_ostream<charT,traits>&));
```

1 *Effects:* None. Does not behave as a formatted output function (as described in 27.7.3.6.1).

2 *Returns:* `pf(*this)`.[329]

```
basic_ostream<charT,traits>& operator<<
    (basic_ios<charT,traits>& (*pf)(basic_ios<charT,traits>&));
```

3 *Effects:* Calls `pf(*this)`. This inserter does not behave as a formatted output function (as described in 27.7.3.6.1).

4 *Returns:* `*this`.[330]

```
basic_ostream<charT,traits>& operator<<
    (ios_base& (*pf)(ios_base&));
```

5 *Effects:* Calls `pf(*this)`. This inserter does not behave as a formatted output function (as described in 27.7.3.6.1).

6 *Returns:* `*this`.

```
basic_ostream<charT,traits>& operator<<
    (basic_streambuf<charT,traits>* sb);
```

7 *Effects:* Behaves as an unformatted output function (as described in 27.7.3.7, paragraph 1). After the sentry object is constructed, if `sb` is null calls `setstate(badbit)` (which may throw `ios_base::failure`).

8 Gets characters from `sb` and inserts them in `*this`. Characters are read from `sb` and inserted until any of the following occurs:

(8.1) — end-of-file occurs on the input sequence;

(8.2) — inserting in the output sequence fails (in which case the character to be inserted is not extracted);

(8.3) — an exception occurs while getting a character from `sb`.

9 If the function inserts no characters, it calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)). If an exception was thrown while extracting a character, the function sets `failbit` in error state, and if `failbit` is on in `exceptions()` the caught exception is rethrown.

10 *Returns:* `*this`.

329) See, for example, the function signature `endl(basic_ostream&)` (27.7.3.8).

330) See, for example, the function signature `dec(ios_base&)` (27.5.6.3).

27.7.3.6.4 Character inserter function templates [ostream.inserters.character]

```
template<class charT, class traits>
  basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>& out,
                                          charT c);
template<class charT, class traits>
  basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>& out,
                                          char c);
  // specialization
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         char c);
  // signed and unsigned
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         signed char c);
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         unsigned char c);
```

1 *Effects:* Behaves as a formatted output function (27.7.3.6.1) of `out`. Constructs a character sequence `seq`. If `c` has type `char` and the character type of the stream is not `char`, then `seq` consists of `out.widen(c)`; otherwise `seq` consists of `c`. Determines padding for `seq` as described in 27.7.3.6.1. Inserts `seq` into `out`. Calls `os.width(0)`.

2 *Returns:* `out`.

```
template<class charT, class traits>
  basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>& out,
                                          const charT* s);
template<class charT, class traits>
  basic_ostream<charT,traits>& operator<<(basic_ostream<charT,traits>& out,
                                          const char* s);
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         const char* s);
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         const signed char* s);
template<class traits>
  basic_ostream<char,traits>& operator<<(basic_ostream<char,traits>& out,
                                         const unsigned char* s);
```

3 *Requires:* `s` shall not be a null pointer.

4 *Effects:* Behaves like a formatted inserter (as described in 27.7.3.6.1) of `out`. Creates a character sequence `seq` of `n` characters starting at `s`, each widened using `out.widen()` (27.5.5.3), where `n` is the number that would be computed as if by:

(4.1) — `traits::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const charT*`, and also for the overload where the first argument is of type `basic_ostream<char, traits>&` and the second is of type `const char*`,

(4.2) — `std::char_traits<char>::length(s)` for the overload where the first argument is of type `basic_ostream<charT, traits>&` and the second is of type `const char*`,

(4.3) — `traits::length(reinterpret_cast<const char*>(s))` for the other two overloads.

Determines padding for `seq` as described in 27.7.3.6.1. Inserts `seq` into `out`. Calls `width(0)`.

5 *Returns:* `out`.

27.7.3.7 Unformatted output functions [ostream.unformatted]

1 Each unformatted output function begins execution by constructing an object of class `sentry`. If this object returns `true`, while converting to a value of type `bool`, the function endeavors to generate the requested output. If an exception is thrown during output, then `ios::badbit` is turned on[331] in `*this`'s error state. If `(exceptions() & badbit) != 0` then the exception is rethrown. In any case, the unformatted output function ends by destroying the sentry object, then, if no exception was thrown, returning the value specified for the unformatted output function.

```
basic_ostream<charT,traits>& put(char_type c);
```

2 *Effects:* Behaves as an unformatted output function (as described in 27.7.3.7, paragraph 1). After constructing a sentry object, inserts the character `c`, if possible.[332]

3 Otherwise, calls `setstate(badbit)` (which may throw `ios_base::failure` (27.5.5.4)).

4 *Returns:* `*this`.

```
basic_ostream& write(const char_type* s, streamsize n);
```

5 *Effects:* Behaves as an unformatted output function (as described in 27.7.3.7, paragraph 1). After constructing a sentry object, obtains characters to insert from successive locations of an array whose first element is designated by `s`.[333] Characters are inserted until either of the following occurs:

(5.1) — `n` characters are inserted;

(5.2) — inserting in the output sequence fails (in which case the function calls `setstate(badbit)`, which may throw `ios_base::failure` (27.5.5.4)).

6 *Returns:* `*this`.

```
basic_ostream& flush();
```

7 *Effects:* Behaves as an unformatted output function (as described in 27.7.3.7, paragraph 1). If `rdbuf()` is not a null pointer, constructs a sentry object. If this object returns `true` when converted to a value of type `bool` the function calls `rdbuf()->pubsync()`. If that function returns -1 calls `setstate(badbit)` (which may throw `ios_base::failure` (27.5.5.4)). Otherwise, if the sentry object returns `false`, does nothing.

8 *Returns:* `*this`.

27.7.3.8 Standard `basic_ostream` manipulators [ostream.manip]

```
template <class charT, class traits>
  basic_ostream<charT,traits>& endl(basic_ostream<charT,traits>& os);
```

1 *Effects:* Calls `os.put(os.widen('\n'))`, then `os.flush()`.

2 *Returns:* `os`.

```
template <class charT, class traits>
  basic_ostream<charT,traits>& ends(basic_ostream<charT,traits>& os);
```

331) without causing an `ios::failure` to be thrown.
332) Note that this function is not overloaded on types `signed char` and `unsigned char`.
333) Note that this function is not overloaded on types `signed char` and `unsigned char`.

3 *Effects:* Inserts a null character into the output sequence: calls `os.put(charT())`.

4 *Returns:* `os`.

```
template <class charT, class traits>
  basic_ostream<charT,traits>& flush(basic_ostream<charT,traits>& os);
```

5 *Effects:* Calls `os.flush()`.

6 *Returns:* `os`.

27.7.3.9 Rvalue stream insertion [ostream.rvalue]

```
template <class charT, class traits, class T>
  basic_ostream<charT, traits>&
  operator<<(basic_ostream<charT, traits>&& os, const T& x);
```

1 *Effects:* `os << x`

2 *Returns:* `os`

27.7.4 Standard manipulators [std.manip]

1 The header `<iomanip>` defines several functions that support extractors and inserters that alter information maintained by class `ios_base` and its derived classes.

```
unspecified  resetiosflags(ios_base::fmtflags mask);
```

2 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << resetiosflags(mask)` behaves as if it called `f(out, mask)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> resetiosflags(mask)` behaves as if it called `f(in, mask)`, where the function `f` is defined as:[334]

```
void f(ios_base& str, ios_base::fmtflags mask) {
  // reset specified flags
  str.setf(ios_base::fmtflags(0), mask);
}
```

The expression `out << resetiosflags(mask)` shall have type `basic_ostream<charT,traits>&` and value `out`. The expression `in >> resetiosflags(mask)` shall have type `basic_istream<charT, traits>&` and value `in`.

```
unspecified  setiosflags(ios_base::fmtflags mask);
```

3 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setiosflags(mask)` behaves as if it called `f(out, mask)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setiosflags(mask)` behaves as if it called `f(in, mask)`, where the function `f` is defined as:

```
void f(ios_base& str, ios_base::fmtflags mask) {
  // set specified flags
  str.setf(mask);
}
```

The expression `out << setiosflags(mask)` shall have type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setiosflags(mask)` shall have type `basic_istream<charT, traits>&` and value `in`.

334) The expression `cin >>resetiosflags(ios_base::skipws)` clears `ios_base::skipws` in the format flags stored in the `basic_istream<charT,traits>` object `cin` (the same as `cin >>noskipws`), and the expression `cout <<resetiosflags(ios_base::showbase)` clears `ios_base::showbase` in the format flags stored in the `basic_ostream<charT,traits>` object `cout` (the same as `cout <<noshowbase`).

unspecified `setbase(int base);`

4 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setbase(base)` behaves as if it called `f(out, base)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setbase(base)` behaves as if it called `f(in, base)`, where the function `f` is defined as:

```
void f(ios_base& str, int base) {
  // set basefield
  str.setf(base ==  8 ? ios_base::oct :
      base == 10 ? ios_base::dec :
      base == 16 ? ios_base::hex :
      ios_base::fmtflags(0), ios_base::basefield);
}
```

The expression `out << setbase(base)` shall have type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setbase(base)` shall have type `basic_istream<charT, traits>&` and value `in`.

unspecified `setfill(char_type c);`

5 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` and `c` has type `charT` then the expression `out << setfill(c)` behaves as if it called `f(out, c)`, where the function `f` is defined as:

```
template<class charT, class traits>
void f(basic_ios<charT,traits>& str, charT c) {
  // set fill character
  str.fill(c);
}
```

The expression `out << setfill(c)` shall have type `basic_ostream<charT, traits>&` and value `out`.

unspecified `setprecision(int n);`

6 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << setprecision(n)` behaves as if it called `f(out, n)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setprecision(n)` behaves as if it called `f(in, n)`, where the function `f` is defined as:

```
void f(ios_base& str, int n) {
  // set precision
  str.precision(n);
}
```

The expression `out << setprecision(n)` shall have type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setprecision(n)` shall have type `basic_istream<charT, traits>&` and value `in`.

unspecified `setw(int n);`

7 *Returns:* An object of unspecified type such that if `out` is an instance of `basic_ostream<charT, traits>` then the expression `out << setw(n)` behaves as if it called `f(out, n)`, or if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> setw(n)` behaves as if it called `f(in, n)`, where the function `f` is defined as:

```
void f(ios_base& str, int n) {
  // set width
  str.width(n);
}
```

The expression `out << setw(n)` shall have type `basic_ostream<charT, traits>&` and value `out`. The expression `in >> setw(n)` shall have type `basic_istream<charT, traits>&` and value `in`.

27.7.5 Extended manipulators [ext.manip]

1 The header `<iomanip>` defines several functions that support extractors and inserters that allow for the parsing and formatting of sequences and values for money and time.

```
template <class moneyT> unspecified  get_money(moneyT& mon, bool intl = false);
```

2 *Requires:* The type `moneyT` shall be either `long double` or a specialization of the `basic_string` template (Clause 21).

3 *Effects:* The expression `in >>get_money(mon, intl)` described below behaves as a formatted input function (27.7.2.2.1).

4 *Returns:* An object of unspecified type such that if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> get_money(mon, intl)` behaves as if it called `f(in, mon, intl)`, where the function `f` is defined as:

```
template <class charT, class traits, class moneyT>
void f(basic_ios<charT, traits>& str, moneyT& mon, bool intl) {
  typedef istreambuf_iterator<charT, traits> Iter;
  typedef money_get<charT, Iter> MoneyGet;

  ios_base::iostate err = ios_base::goodbit;
  const MoneyGet &mg = use_facet<MoneyGet>(str.getloc());

  mg.get(Iter(str.rdbuf()), Iter(), intl, str, err, mon);

  if (ios_base::goodbit != err)
    str.setstate(err);
}
```

The expression `in >> get_money(mon, intl)` shall have type `basic_istream<charT, traits>&` and value `in`.

```
template <class moneyT> unspecified  put_money(const moneyT& mon, bool intl = false);
```

5 *Requires:* The type `moneyT` shall be either `long double` or a specialization of the `basic_string` template (Clause 21).

6 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << put_money(mon, intl)` behaves as a formatted input function that calls `f(out, mon, intl)`, where the function `f` is defined as:

```
template <class charT, class traits, class moneyT>
void f(basic_ios<charT, traits>& str, const moneyT& mon, bool intl) {
  typedef ostreambuf_iterator<charT, traits> Iter;
  typedef money_put<charT, Iter> MoneyPut;

  const MoneyPut& mp = use_facet<MoneyPut>(str.getloc());
  const Iter end = mp.put(Iter(str.rdbuf()), intl, str, str.fill(), mon);
```

```
  if (end.failed())
    str.setstate(ios::badbit);
}
```

The expression `out << put_money(mon, intl)` shall have type `basic_ostream<charT, traits>&` and value `out`.

```
template <class charT> unspecified  get_time(struct tm* tmb, const charT* fmt);
```

7 *Requires:* The argument `tmb` shall be a valid pointer to an object of type `struct tm`, and the argument `fmt` shall be a valid pointer to an array of objects of type `charT` with `char_traits<charT>::length(fmt)` elements.

8 *Returns:* An object of unspecified type such that if `in` is an object of type `basic_istream<charT, traits>` then the expression `in >> get_time(tmb, fmt)` behaves as if it called `f(in, tmb, fmt)`, where the function `f` is defined as:

```
template <class charT, class traits>
void f(basic_ios<charT, traits>& str, struct tm* tmb, const charT* fmt) {
  typedef istreambuf_iterator<charT, traits> Iter;
  typedef time_get<charT, Iter> TimeGet;

  ios_base::iostate err = ios_base::goodbit;
  const TimeGet& tg = use_facet<TimeGet>(str.getloc());

  tg.get(Iter(str.rdbuf()), Iter(), str, err, tmb,
    fmt, fmt + traits::length(fmt));

  if (err != ios_base::goodbit)
    str.setstate(err):
}
```

The expression `in >> get_time(tmb, fmt)` shall have type `basic_istream<charT, traits>&` and value `in`.

```
template <class charT> unspecified  put_time(const struct tm* tmb, const charT* fmt);
```

9 *Requires:* The argument `tmb` shall be a valid pointer to an object of type `struct tm`, and the argument `fmt` shall be a valid pointer to an array of objects of type `charT` with `char_traits<charT>::length(fmt)` elements.

10 *Returns:* An object of unspecified type such that if `out` is an object of type `basic_ostream<charT, traits>` then the expression `out << put_time(tmb, fmt)` behaves as if it called `f(out, tmb, fmt)`, where the function `f` is defined as:

```
template <class charT, class traits>
void f(basic_ios<charT, traits>& str, const struct tm* tmb, const charT* fmt) {
  typedef ostreambuf_iterator<charT, traits> Iter;
  typedef time_put<charT, Iter> TimePut;

  const TimePut& tp = use_facet<TimePut>(str.getloc());
  const Iter end = tp.put(Iter(str.rdbuf()), str, str.fill(), tmb,
    fmt, fmt + traits::length(fmt));

  if (end.failed())
    str.setstate(ios_base::badbit);
}
```

The expression `out << put_time(tmb, fmt)` shall have type `basic_ostream<charT, traits>&` and value `out`.

27.7.6 Quoted manipulators [quoted.manip]

1 [*Note:* Quoted manipulators provide string insertion and extraction of quoted strings (for example, XML and CSV formats). Quoted manipulators are useful in ensuring that the content of a string with embedded spaces remains unchanged if inserted and then extracted via stream I/O. — *end note*]

```
template <class charT>
  unspecified  quoted(const charT* s, charT delim=charT('"'), charT escape=charT('\\'));
template <class charT, class traits, class Allocator>
  unspecified  quoted(const basic_string<charT, traits, Allocator>& s,
                      charT delim=charT('"'), charT escape=charT('\\'));
```

2 *Returns:* An object of unspecified type such that if `out` is an instance of `basic_ostream` with member type `char_type` the same as `charT` and with member type `traits_type`, which in the second form is the same as `traits`, then the expression `out << quoted(s, delim, escape)` behaves as a formatted output function (27.7.3.6.1) of `out`. This forms a character sequence `seq`, initially consisting of the following elements:

(2.1) — `delim`.

(2.2) — Each character in `s`. If the character to be output is equal to `escape` or `delim`, as determined by `traits_type::eq`, first output `escape`.

(2.3) — `delim`.

Let `x` be the number of elements initially in `seq`. Then padding is determined for `seq` as described in 27.7.3.6.1, `seq` is inserted as if by calling `out.rdbuf()->sputn(seq, n)`, where `n` is the larger of `out.width()` and `x`, and `out.width(0)` is called. The expression `out << quoted(s, delim, escape)` shall have type `basic_ostream<charT, traits>&` and value `out`.

```
template <class charT, class traits, class Allocator>
  unspecified  quoted(basic_string<charT, traits, Allocator>& s,
                      charT delim=charT('"'), charT escape=charT('\\'));
```

3 *Returns:* An object of unspecified type such that:

(3.1) — If `in` is an instance of `basic_istream` with member types `char_type` and `traits_type` the same as `charT` and `traits`, respectively, then the expression `in >> quoted(s, delim, escape)` behaves as if it extracts the following characters from `in` using `basic_istream::operator>>` (27.7.2.2.3) which may throw `ios_base::failure` (27.5.3.1.1):

(3.1.1) — If the first character extracted is equal to `delim`, as determined by `traits_type::eq`, then:

(3.1.1.1) — Turn off the `skipws` flag.

(3.1.1.2) — `s.clear()`

(3.1.1.3) — Until an unescaped `delim` character is reached or `!in`, extract characters from `in` and append them to `s`, except that if an `escape` is reached, ignore it and append the next character to `s`.

(3.1.1.4) — Discard the final `delim` character.

(3.1.1.5) — Restore the `skipws` flag to its original value.

(3.1.2) — Otherwise, `in >> s`.

(3.2) — If `out` is an instance of `basic_ostream` with member types `char_type` and `traits_type` the same as `charT` and `traits`, respectively, then the expression `out << quoted(s, delim, escape)` behaves as specified for the `const basic_string<charT, traits, Allocator>&` overload of the `quoted` function.

The expression `in >> quoted(s, delim, escape)` shall have type `basic_istream<charT, traits>&` and value `in`. The expression `out << quoted(s, delim, escape)` shall have type `basic_ostream <charT, traits>&` and value `out`.

27.8 String-based streams [string.streams]

27.8.1 Overview [string.streams.overview]

1 The header `<sstream>` defines four class templates and eight types that associate stream buffers with objects of class `basic_string`, as described in 21.3.

Header `<sstream>` synopsis

```
namespace std {
  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
    class basic_stringbuf;

  typedef basic_stringbuf<char>     stringbuf;
  typedef basic_stringbuf<wchar_t> wstringbuf;

  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
    class basic_istringstream;

  typedef basic_istringstream<char>     istringstream;
  typedef basic_istringstream<wchar_t> wistringstream;

  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
    class basic_ostringstream;
  typedef basic_ostringstream<char>     ostringstream;
  typedef basic_ostringstream<wchar_t> wostringstream;

  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
    class basic_stringstream;
  typedef basic_stringstream<char>     stringstream;
  typedef basic_stringstream<wchar_t> wstringstream;
}
```

27.8.2 Class template `basic_stringbuf` [stringbuf]

```
namespace std {
  template <class charT, class traits = char_traits<charT>,
      class Allocator = allocator<charT> >
  class basic_stringbuf : public basic_streambuf<charT,traits> {
  public:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;
    typedef Allocator                 allocator_type;

    // 27.8.2.1 Constructors:
    explicit basic_stringbuf(ios_base::openmode which
```

```
                              = ios_base::in | ios_base::out);
    explicit basic_stringbuf
    (const basic_string<charT,traits,Allocator>& str,
     ios_base::openmode which = ios_base::in | ios_base::out);
    basic_stringbuf(const basic_stringbuf& rhs) = delete;
    basic_stringbuf(basic_stringbuf&& rhs);

    // 27.8.2.2 Assign and swap:
    basic_stringbuf& operator=(const basic_stringbuf& rhs) = delete;
    basic_stringbuf& operator=(basic_stringbuf&& rhs);
    void swap(basic_stringbuf& rhs);

    // 27.8.2.3 Get and set:
    basic_string<charT,traits,Allocator> str() const;
    void str(const basic_string<charT,traits,Allocator>& s);

  protected:
    // 27.8.2.4 Overridden virtual functions:
    virtual int_type   underflow();
    virtual int_type   pbackfail(int_type c = traits::eof());
    virtual int_type   overflow (int_type c = traits::eof());
    virtual  basic_streambuf<charT,traits>* setbuf(charT*, streamsize);


    virtual pos_type seekoff(off_type off, ios_base::seekdir way,
                             ios_base::openmode which
                               = ios_base::in | ios_base::out);
    virtual pos_type seekpos(pos_type sp,
                             ios_base::openmode which
                               = ios_base::in | ios_base::out);

  private:
    ios_base::openmode mode;  // exposition only
  };

  template <class charT, class traits, class Allocator>
  void swap(basic_stringbuf<charT, traits, Allocator>& x,
            basic_stringbuf<charT, traits, Allocator>& y);
}
```

1 The class `basic_stringbuf` is derived from `basic_streambuf` to associate possibly the input sequence and possibly the output sequence with a sequence of arbitrary *characters*. The sequence can be initialized from, or made available as, an object of class `basic_string`.

2 For the sake of exposition, the maintained data is presented here as:

(2.1) — `ios_base::openmode mode`, has `in` set if the input sequence can be read, and `out` set if the output sequence can be written.

27.8.2.1 `basic_stringbuf` constructors [stringbuf.cons]

```
explicit basic_stringbuf(ios_base::openmode which =
                         ios_base::in | ios_base::out);
```

1 *Effects:* Constructs an object of class `basic_stringbuf`, initializing the base class with `basic_streambuf()` (27.6.3.1), and initializing `mode` with `which`.

2 *Postcondition:* `str() == ""`.

```
explicit basic_stringbuf(const basic_string<charT,traits,Allocator>& s,
                         ios_base::openmode which = ios_base::in | ios_base::out);
```

3 *Effects:* Constructs an object of class `basic_stringbuf`, initializing the base class with `basic_streambuf()` (27.6.3.1), and initializing `mode` with `which`. Then calls `str(s)`.

```
basic_stringbuf(basic_stringbuf&& rhs);
```

4 *Effects:* Move constructs from the rvalue `rhs`. It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. Whether they do or not, `*this` and `rhs` reference separate buffers (if any at all) after the construction. The openmode, locale and any other state of `rhs` is also copied.

5 *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

(5.1) — `str() == rhs_p.str()`

(5.2) — `gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`

(5.3) — `egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`

(5.4) — `pptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`

(5.5) — `epptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`

(5.6) — `if (eback()) eback() != rhs_a.eback()`

(5.7) — `if (gptr()) gptr() != rhs_a.gptr()`

(5.8) — `if (egptr()) egptr() != rhs_a.egptr()`

(5.9) — `if (pbase()) pbase() != rhs_a.pbase()`

(5.10) — `if (pptr()) pptr() != rhs_a.pptr()`

(5.11) — `if (epptr()) epptr() != rhs_a.epptr()`

27.8.2.2 Assign and swap [stringbuf.assign]

```
basic_stringbuf& operator=(basic_stringbuf&& rhs);
```

1 *Effects:* After the move assignment `*this` has the observable state it would have had if it had been move constructed from `rhs` (see 27.8.2.1).

2 *Returns:* `*this`.

```
void swap(basic_stringbuf& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs`.

```
template <class charT, class traits, class Allocator>
void swap(basic_stringbuf<charT, traits, Allocator>& x,
          basic_stringbuf<charT, traits, Allocator>& y);
```

4 *Effects:* `x.swap(y)`.

27.8.2.3 Member functions [stringbuf.members]

```
basic_string<charT,traits,Allocator> str() const;
```

1 *Returns:* A `basic_string` object whose content is equal to the `basic_stringbuf` underlying character sequence. If the `basic_stringbuf` was created only in input mode, the resultant `basic_string` contains the character sequence in the range `[eback(),egptr())`. If the `basic_stringbuf` was created with `which & ios_base::out` being true then the resultant `basic_string` contains the character sequence in the range `[pbase(),high_mark)`, where `high_mark` represents the position one past the

highest initialized character in the buffer. Characters can be initialized by writing to the stream, by constructing the `basic_stringbuf` with a `basic_string`, or by calling the `str(basic_string)` member function. In the case of calling the `str(basic_string)` member function, all characters initialized prior to the call are now considered uninitialized (except for those characters re-initialized by the new `basic_string`). Otherwise the `basic_stringbuf` has been created in neither input nor output mode and a zero length `basic_string` is returned.

```
void str(const basic_string<charT,traits,Allocator>& s);
```

2 *Effects:* Copies the content of `s` into the `basic_stringbuf` underlying character sequence and initializes the input and output sequences according to `mode`.

3 *Postconditions:* If `mode & ios_base::out` is true, `pbase()` points to the first underlying character and `epptr() >= pbase() + s.size()` holds; in addition, if `mode & ios_base::ate` is true, `pptr() == pbase() + s.size()` holds, otherwise `pptr() == pbase()` is true. If `mode & ios_base::in` is true, `eback()` points to the first underlying character, and both `gptr() == eback()` and `egptr() == eback() + s.size()` hold.

27.8.2.4 Overridden virtual functions [stringbuf.virtuals]

```
int_type underflow();
```

1 *Returns:* If the input sequence has a read position available, returns `traits::to_int_type(*gptr())`. Otherwise, returns `traits::eof()`. Any character in the underlying buffer which has been initialized is considered to be part of the input sequence.

```
int_type pbackfail(int_type c = traits::eof());
```

2 *Effects:* Puts back the character designated by `c` to the input sequence, if possible, in one of three ways:

(2.1) — If `traits::eq_int_type(c,traits::eof())` returns `false` and if the input sequence has a putback position available, and if `traits::eq(to_char_type(c),gptr()[-1])` returns `true`, assigns `gptr() - 1` to `gptr()`.

Returns: `c`.

(2.2) — If `traits::eq_int_type(c,traits::eof())` returns `false` and if the input sequence has a putback position available, and if `mode & ios_base::out` is nonzero, assigns `c` to `*--gptr()`.

Returns: `c`.

(2.3) — If `traits::eq_int_type(c,traits::eof())` returns `true` and if the input sequence has a putback position available, assigns `gptr() - 1` to `gptr()`.

Returns: `traits::not_eof(c)`.

3 *Returns:* `traits::eof()` to indicate failure.

4 *Remarks:* If the function can succeed in more than one of these ways, it is unspecified which way is chosen.

```
int_type overflow(int_type c = traits::eof());
```

5 *Effects:* Appends the character designated by `c` to the output sequence, if possible, in one of two ways:

(5.1) — If `traits::eq_int_type(c,traits::eof())` returns `false` and if either the output sequence has a write position available or the function makes a write position available (as described below), the function calls `sputc(c )`.

Signals success by returning `c`.

(5.2) — If `traits::eq_int_type(c,traits::eof())` returns `true`, there is no character to append. Signals success by returning a value other than `traits::eof()`.

6 *Remarks:* The function can alter the number of write positions available as a result of any call.

7 *Returns:* `traits::eof()` to indicate failure.

8 The function can make a write position available only if `(mode & ios_base::out) != 0`. To make a write position available, the function reallocates (or initially allocates) an array object with a sufficient number of elements to hold the current array object (if any), plus at least one additional write position. If `(mode & ios_base::in) != 0`, the function alters the read end pointer `egptr()` to point just past the new write position.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
                 ios_base::openmode which
                   = ios_base::in | ios_base::out);
```

9 *Effects:* Alters the stream position within one of the controlled sequences, if possible, as indicated in Table 129.

Table 129 — `seekoff` positioning

Conditions	Result
`(which & ios_base::in) == ios_base::in`	positions the input sequence
`(which & ios_base::out) == ios_base::out`	positions the output sequence
`(which & (ios_base::in | ios_base::out)) == (ios_base::in | ios_base::out))` and `way ==` either `ios_base::beg` or `ios_base::end`	positions both the input and the output sequences
Otherwise	the positioning operation fails.

10 For a sequence to be positioned, if its next pointer (either `gptr()` or `pptr()`) is a null pointer and the new offset `newoff` is nonzero, the positioning operation fails. Otherwise, the function determines `newoff` as indicated in Table 130.

Table 130 — `newoff` values

Condition	`newoff` Value
`way == ios_base::beg`	0
`way == ios_base::cur`	the next pointer minus the beginning pointer (`xnext - xbeg`).
`way == ios_base::end`	the high mark pointer minus the beginning pointer (`high_mark - xbeg`).

11 If `(newoff + off) < 0`, or if `newoff + off` refers to an uninitialized character (as defined in 27.8.2.3 paragraph 1), the positioning operation fails. Otherwise, the function assigns `xbeg + newoff + off` to the next pointer `xnext`.

12 *Returns:* `pos_type(newoff)`, constructed from the resultant offset `newoff` (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp, ios_base::openmode which
                 = ios_base::in | ios_base::out);
```

13 *Effects:* Equivalent to `seekoff(off_type(sp), ios_base::beg, which)`.

14 *Returns:* `sp` to indicate success, or `pos_type(off_type(-1))` to indicate failure.

```
basic_streambuf<charT,traits>* setbuf(charT* s, streamsize n);
```

15 *Effects:* implementation-defined, except that `setbuf(0,0)` has no effect.

16 *Returns:* `this`.

27.8.3 Class template `basic_istringstream` [istringstream]

```
namespace std {
  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
  class basic_istringstream : public basic_istream<charT,traits> {
  public:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;
    typedef Allocator                 allocator_type;

    // 27.8.3.1 Constructors:
    explicit basic_istringstream(ios_base::openmode which = ios_base::in);
    explicit basic_istringstream(
               const basic_string<charT,traits,Allocator>& str,
               ios_base::openmode which = ios_base::in);
    basic_istringstream(const basic_istringstream& rhs) = delete;
    basic_istringstream(basic_istringstream&& rhs);

    // 27.8.3.2 Assign and swap:
    basic_istringstream& operator=(const basic_istringstream& rhs) = delete;
    basic_istringstream& operator=(basic_istringstream&& rhs);
    void swap(basic_istringstream& rhs);

    // 27.8.3.3 Members:
    basic_stringbuf<charT,traits,Allocator>* rdbuf() const;

    basic_string<charT,traits,Allocator> str() const;
    void str(const basic_string<charT,traits,Allocator>& s);
  private:
    basic_stringbuf<charT,traits,Allocator> sb; // exposition only
  };

  template <class charT, class traits, class Allocator>
  void swap(basic_istringstream<charT, traits, Allocator>& x,
            basic_istringstream<charT, traits, Allocator>& y);
}
```

1 The class `basic_istringstream<charT, traits, Allocator>` supports reading objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf<charT, traits, Allocator>` object to control the associated storage. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `stringbuf` object.

27.8.3.1 `basic_istringstream` constructors [istringstream.cons]

```
explicit basic_istringstream(ios_base::openmode which = ios_base::in);
```

1 *Effects:* Constructs an object of class `basic_istringstream<charT, traits>`, initializing the base class with `basic_istream(&sb)` and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::in))` (27.8.2.1).

```
explicit basic_istringstream(
               const basic_string<charT, traits, Allocator>& str,
               ios_base::openmode which = ios_base::in);
```

2 *Effects:* Constructs an object of class `basic_istringstream<charT, traits>`, initializing the base class with `basic_istream(&sb)` and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(str, which | ios_base::in))` (27.8.2.1).

```
basic_istringstream(basic_istringstream&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_istream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_stringbuf`.

27.8.3.2 Assign and swap [istringstream.assign]

```
basic_istringstream& operator=(basic_istringstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_istringstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_istream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits, class Allocator>
void swap(basic_istringstream<charT, traits, Allocator>& x,
          basic_istringstream<charT, traits, Allocator>& y);
```

4 *Effects:* `x.swap(y)`.

27.8.3.3 Member functions [istringstream.members]

```
basic_stringbuf<charT,traits,Allocator>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_stringbuf<charT,traits,Allocator>*>(&sb)`.

```
basic_string<charT,traits,Allocator> str() const;
```

2 *Returns:* `rdbuf()->str()`.

```
void str(const basic_string<charT,traits,Allocator>& s);
```

3 *Effects:* Calls `rdbuf()->str(s)`.

27.8.4 Class template `basic_ostringstream` [ostringstream]

```
namespace std {
  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
  class basic_ostringstream : public basic_ostream<charT,traits> {
  public:

    // types:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;
    typedef Allocator                 allocator_type;

    // 27.8.4.1 Constructors/destructor:
    explicit basic_ostringstream(ios_base::openmode which = ios_base::out);
    explicit basic_ostringstream(
             const basic_string<charT,traits,Allocator>& str,
             ios_base::openmode which = ios_base::out);
    basic_ostringstream(const basic_ostringstream& rhs) = delete;
    basic_ostringstream(basic_ostringstream&& rhs);

    // 27.8.4.2 Assign/swap:
    basic_ostringstream& operator=(const basic_ostringstream& rhs) = delete;
    basic_ostringstream& operator=(basic_ostringstream&& rhs);
    void swap(basic_ostringstream& rhs);

    // 27.8.4.3 Members:
    basic_stringbuf<charT,traits,Allocator>* rdbuf() const;

    basic_string<charT,traits,Allocator> str() const;
    void    str(const basic_string<charT,traits,Allocator>& s);
   private:
    basic_stringbuf<charT,traits,Allocator> sb; // exposition only
  };

  template <class charT, class traits, class Allocator>
  void swap(basic_ostringstream<charT, traits, Allocator>& x,
            basic_ostringstream<charT, traits, Allocator>& y);
}
```

1 The class `basic_ostringstream<charT, traits, Allocator>` supports writing objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf` object to control the associated storage. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `stringbuf` object.

27.8.4.1 `basic_ostringstream` constructors [ostringstream.cons]

```
explicit basic_ostringstream(ios_base::openmode which = ios_base::out);
```

1 *Effects:* Constructs an object of class `basic_ostringstream`, initializing the base class with `basic_ostream(&sb)` and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(which | ios_base::out))` (27.8.2.1).

```
explicit basic_ostringstream(
  const basic_string<charT,traits,Allocator>& str,
  ios_base::openmode which = ios_base::out);
```

2 *Effects:* Constructs an object of class `basic_ostringstream<charT, traits>`, initializing the base class with `basic_ostream(&sb)` and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(str, which | ios_base::out))` (27.8.2.1).

```
basic_ostringstream(basic_ostringstream&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_ostream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_stringbuf`.

27.8.4.2 Assign and swap [ostringstream.assign]

```
basic_ostringstream& operator=(basic_ostringstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_ostringstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_ostream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits, class Allocator>
void swap(basic_ostringstream<charT, traits, Allocator>& x,
          basic_ostringstream<charT, traits, Allocator>& y);
```

4 *Effects:* `x.swap(y)`.

27.8.4.3 Member functions [ostringstream.members]

```
basic_stringbuf<charT,traits,Allocator>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_stringbuf<charT,traits,Allocator>*>(&sb)`.

```
basic_string<charT,traits,Allocator> str() const;
```

2 *Returns:* `rdbuf()->str()`.

```
void str(const basic_string<charT,traits,Allocator>& s);
```

3 *Effects:* Calls `rdbuf()->str(s)`.

27.8.5 Class template `basic_stringstream` [stringstream]

```
namespace std {
  template <class charT, class traits = char_traits<charT>,
        class Allocator = allocator<charT> >
  class basic_stringstream
    : public basic_iostream<charT,traits> {
  public:

    // types:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
```

```
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;
    typedef Allocator                 allocator_type;

    // constructors/destructor
    explicit basic_stringstream(
    ios_base::openmode which = ios_base::out|ios_base::in);
    explicit basic_stringstream(
    const basic_string<charT,traits,Allocator>& str,
    ios_base::openmode which = ios_base::out|ios_base::in);
    basic_stringstream(const basic_stringstream& rhs) = delete;
    basic_stringstream(basic_stringstream&& rhs);

    // 27.8.5.2 Assign/swap:
    basic_stringstream& operator=(const basic_stringstream& rhs) = delete;
    basic_stringstream& operator=(basic_stringstream&& rhs);
    void swap(basic_stringstream& rhs);

    // Members:
    basic_stringbuf<charT,traits,Allocator>* rdbuf() const;
    basic_string<charT,traits,Allocator> str() const;
    void str(const basic_string<charT,traits,Allocator>& str);

  private:
    basic_stringbuf<charT, traits> sb;  // exposition only
  };

  template <class charT, class traits, class Allocator>
  void swap(basic_stringstream<charT, traits, Allocator>& x,
            basic_stringstream<charT, traits, Allocator>& y);
}
```

1 The class template `basic_stringstream<charT, traits>` supports reading and writing from objects of class `basic_string<charT, traits, Allocator>`. It uses a `basic_stringbuf<charT, traits, Allocator>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as

(1.1) — `sb`, the `stringbuf` object.

27.8.5.1 basic_stringstream constructors [stringstream.cons]

```
explicit basic_stringstream(
    ios_base::openmode which = ios_base::out|ios_base::in);
```

1 *Effects:* Constructs an object of class `basic_stringstream<charT,traits>`, initializing the base class with `basic_iostream(&sb)` and initializing `sb` with `basic_stringbuf<charT,traits,Allocator>(which)`.

```
explicit basic_stringstream(
    const basic_string<charT,traits,Allocator>& str,
    ios_base::openmode which = ios_base::out|ios_base::in);
```

2 *Effects:* Constructs an object of class `basic_stringstream<charT, traits>`, initializing the base class with `basic_iostream(&sb)` and initializing `sb` with `basic_stringbuf<charT, traits, Allocator>(str, which)`.

```
basic_stringstream(basic_stringstream&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_stringbuf`. Next `basic_istream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_stringbuf`.

27.8.5.2 Assign and swap [stringstream.assign]

```
basic_stringstream& operator=(basic_stringstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_stringstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_iostream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits, class Allocator>
void swap(basic_stringstream<charT, traits, Allocator>& x,
          basic_stringstream<charT, traits, Allocator>& y);
```

4 *Effects:* `x.swap(y)`.

27.8.5.3 Member functions [stringstream.members]

```
basic_stringbuf<charT,traits,Allocator>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_stringbuf<charT,traits,Allocator>*>(&sb)`

```
basic_string<charT,traits,Allocator> str() const;
```

2 *Returns:* `rdbuf()->str()`.

```
void str(const basic_string<charT,traits,Allocator>& str);
```

3 *Effects:* Calls `rdbuf()->str(str)`.

27.9 File-based streams [file.streams]

27.9.1 File streams [fstreams]

1 The header `<fstream>` defines four class templates and eight types that associate stream buffers with files and assist reading and writing files.

Header `<fstream>` synopsis

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
    class basic_filebuf;
  typedef basic_filebuf<char>    filebuf;
  typedef basic_filebuf<wchar_t> wfilebuf;

  template <class charT, class traits = char_traits<charT> >
    class basic_ifstream;
  typedef basic_ifstream<char>    ifstream;
  typedef basic_ifstream<wchar_t> wifstream;

  template <class charT, class traits = char_traits<charT> >
    class basic_ofstream;
  typedef basic_ofstream<char>    ofstream;
```

```
  typedef basic_ofstream<wchar_t> wofstream;

  template <class charT, class traits = char_traits<charT> >
    class basic_fstream;
  typedef basic_fstream<char>     fstream;
  typedef basic_fstream<wchar_t> wfstream;
}
```

2 In this subclause, the type name `FILE` refers to the type `FILE` declared in `<cstdio>` (27.9.2).

3 [*Note:* The class template `basic_filebuf` treats a file as a source or sink of bytes. In an environment that uses a large character set, the file typically holds multibyte character sequences and the `basic_filebuf` object converts those multibyte sequences into wide character sequences. — *end note*]

27.9.1.1 Class template `basic_filebuf` [filebuf]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_filebuf : public basic_streambuf<charT,traits> {
  public:
    typedef charT                    char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                   traits_type;

    // 27.9.1.2 Constructors/destructor:
    basic_filebuf();
    basic_filebuf(const basic_filebuf& rhs) = delete;
    basic_filebuf(basic_filebuf&& rhs);
    virtual ~basic_filebuf();

    // 27.9.1.3 Assign/swap:
    basic_filebuf& operator=(const basic_filebuf& rhs) = delete;
    basic_filebuf& operator=(basic_filebuf&& rhs);
    void swap(basic_filebuf& rhs);

     // 27.9.1.4 Members:
    bool is_open() const;
    basic_filebuf<charT,traits>* open(const char* s,
        ios_base::openmode mode);
    basic_filebuf<charT,traits>* open(const string& s,
        ios_base::openmode mode);
    basic_filebuf<charT,traits>* close();

  protected:
    // 27.9.1.5 Overridden virtual functions:
    virtual streamsize showmanyc();
    virtual int_type underflow();
    virtual int_type uflow();
    virtual int_type pbackfail(int_type c = traits::eof());
    virtual int_type overflow (int_type c = traits::eof());

    virtual basic_streambuf<charT,traits>*
        setbuf(char_type* s, streamsize n);
    virtual pos_type seekoff(off_type off, ios_base::seekdir way,
```

```
        ios_base::openmode which = ios_base::in | ios_base::out);
    virtual pos_type seekpos(pos_type sp,
        ios_base::openmode which = ios_base::in | ios_base::out);
    virtual int      sync();
    virtual void     imbue(const locale& loc);
  };

  template <class charT, class traits>
  void swap(basic_filebuf<charT, traits>& x,
            basic_filebuf<charT, traits>& y);
}
```

1 The class `basic_filebuf<charT,traits>` associates both the input sequence and the output sequence with a file.

2 The restrictions on reading and writing a sequence controlled by an object of class `basic_filebuf<charT, traits>` are the same as for reading and writing with the Standard C library `FILE`s.

3 In particular:

(3.1) — If the file is not open for reading the input sequence cannot be read.

(3.2) — If the file is not open for writing the output sequence cannot be written.

(3.3) — A joint file position is maintained for both the input sequence and the output sequence.

4 An instance of `basic_filebuf` behaves as described in 27.9.1.1 provided `traits::pos_type` is `fpos<traits::state_type>`. Otherwise the behavior is undefined.

5 In order to support file I/O and multibyte/wide character conversion, conversions are performed using members of a facet, referred to as `a_codecvt` in following sections, obtained as if by

```
const codecvt<charT,char,typename traits::state_type>& a_codecvt =
  use_facet<codecvt<charT,char,typename traits::state_type> >(getloc());
```

27.9.1.2 `basic_filebuf` constructors [filebuf.cons]

```
basic_filebuf();
```

1 *Effects:* Constructs an object of class `basic_filebuf<charT,traits>`, initializing the base class with `basic_streambuf<charT,traits>()` (27.6.3.1).

2 *Postcondition:* `is_open() == false`.

```
basic_filebuf(basic_filebuf&& rhs);
```

3 *Effects:* Move constructs from the rvalue `rhs`. It is implementation-defined whether the sequence pointers in `*this` (`eback()`, `gptr()`, `egptr()`, `pbase()`, `pptr()`, `epptr()`) obtain the values which `rhs` had. Whether they do or not, `*this` and `rhs` reference separate buffers (if any at all) after the construction. Additionally `*this` references the file which `rhs` did before the construction, and `rhs` references no file after the construction. The openmode, locale and any other state of `rhs` is also copied.

4 *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

(4.1) — `is_open() == rhs_p.is_open()`

(4.2) — `rhs_a.is_open() == false`

(4.3) — `gptr() - eback() == rhs_p.gptr() - rhs_p.eback()`

(4.4) — `egptr() - eback() == rhs_p.egptr() - rhs_p.eback()`

(4.5) — `pptr() - pbase() == rhs_p.pptr() - rhs_p.pbase()`

(4.6) — `epptr() - pbase() == rhs_p.epptr() - rhs_p.pbase()`

(4.7) — `if (eback()) eback() != rhs_a.eback()`

(4.8) — `if (gptr()) gptr() != rhs_a.gptr()`

(4.9) — `if (egptr()) egptr() != rhs_a.egptr()`

(4.10) — `if (pbase()) pbase() != rhs_a.pbase()`

(4.11) — `if (pptr()) pptr() != rhs_a.pptr()`

(4.12) — `if (epptr()) epptr() != rhs_a.epptr()`

```
virtual ~basic_filebuf();
```

5 *Effects:* Destroys an object of class `basic_filebuf<charT,traits>`. Calls `close()`. If an exception occurs during the destruction of the object, including the call to `close()`, the exception is caught but not rethrown (see 17.6.5.12).

27.9.1.3 Assign and swap [filebuf.assign]

```
basic_filebuf& operator=(basic_filebuf&& rhs);
```

1 *Effects:* Calls `this->close()` then move assigns from `rhs`. After the move assignment `*this` has the observable state it would have had if it had been move constructed from `rhs` (see 27.9.1.2).

2 *Returns:* `*this`.

```
void swap(basic_filebuf& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs`.

```
template <class charT, class traits>
void swap(basic_filebuf<charT, traits>& x,
          basic_filebuf<charT, traits>& y);
```

4 *Effects:* `x.swap(y)`.

27.9.1.4 Member functions [filebuf.members]

```
bool is_open() const;
```

1 *Returns:* `true` if a previous call to `open` succeeded (returned a non-null value) and there has been no intervening call to close.

```
basic_filebuf<charT,traits>* open(const char* s,
    ios_base::openmode mode);
```

2 *Effects:* If `is_open() != false`, returns a null pointer. Otherwise, initializes the `filebuf` as required. It then opens a file, if possible, whose name is the NTBS `s` (as if by calling `std::fopen(s,modstr)`). The NTBS `modstr` is determined from `mode & ~ios_base::ate` as indicated in Table 131. If `mode` is not some combination of flags shown in the table then the open fails.

3 If the open operation succeeds and `(mode & ios_base::ate) != 0`, positions the file to the end (as if by calling `std::fseek(file,0,SEEK_END)`).[335]

4 If the repositioning operation fails, calls `close()` and returns a null pointer to indicate failure.

5 *Returns:* `this` if successful, a null pointer otherwise.

335) The macro `SEEK_END` is defined, and the function signatures `fopen(const char*, const char*)` and `fseek(FILE*, long, int)` are declared, in `<cstdio>` (27.9.2).

Table 131 — File open modes

`ios_base` flag combination					`stdio` equivalent
`binary`	`in`	`out`	`trunc`	`app`	
		+			`"w"`
		+		+	`"a"`
				+	`"a"`
		+	+		`"w"`
	+				`"r"`
	+	+			`"r+"`
	+	+	+		`"w+"`
	+	+		+	`"a+"`
	+			+	`"a+"`
+		+			`"wb"`
+		+		+	`"ab"`
+				+	`"ab"`
+		+	+		`"wb"`
+	+				`"rb"`
+	+	+			`"r+b"`
+	+	+	+		`"w+b"`
+	+	+		+	`"a+b"`
+	+			+	`"a+b"`

```
basic_filebuf<charT,traits>* open(const string& s,
    ios_base::openmode mode);
```

Returns: `open(s.c_str(), mode);`

```
basic_filebuf<charT,traits>* close();
```

6 *Effects:* If `is_open() == false`, returns a null pointer. If a put area exists, calls `overflow(traits::eof())` to flush characters. If the last virtual member function called on `*this` (between `underflow`, `overflow`, `seekoff`, and `seekpos`) was `overflow` then calls `a_codecvt.unshift` (possibly several times) to determine a termination sequence, inserts those characters and calls `overflow(traits::eof())` again. Finally, regardless of whether any of the preceding calls fails or throws an exception, the function closes the file (as if by calling `std::fclose(file)`).[336] If any of the calls made by the function, including `std::fclose`, fails, `close` fails by returning a null pointer. If one of these calls throws an exception, the exception is caught and rethrown after closing the file.

7 *Returns:* `this` on success, a null pointer otherwise.

8 *Postcondition:* `is_open() == false`.

27.9.1.5 Overridden virtual functions [filebuf.virtuals]

```
streamsize showmanyc();
```

1 *Effects:* Behaves the same as `basic_streambuf::showmanyc()` (27.6.3.4).

2 *Remarks:* An implementation might well provide an overriding definition for this function signature if it can determine that more characters can be read from the input sequence.

```
int_type underflow();
```

336) The function signature `fclose(FILE*)` is declared in `<cstdio>` (27.9.2).

3 *Effects:* Behaves according to the description of `basic_streambuf<charT,traits>::underflow()`, with the specialization that a sequence of characters is read from the input sequence as if by reading from the associated file into an internal buffer (`extern_buf`) and then as if by doing

```
char   extern_buf[XSIZE];
char*  extern_end;
charT  intern_buf[ISIZE];
charT* intern_end;
codecvt_base::result r =
  a_codecvt.in(state, extern_buf, extern_buf+XSIZE, extern_end,
               intern_buf, intern_buf+ISIZE, intern_end);
```

This shall be done in such a way that the class can recover the position (`fpos_t`) corresponding to each character between `intern_buf` and `intern_end`. If the value of `r` indicates that `a_codecvt.in()` ran out of space in `intern_buf`, retry with a larger `intern_buf`.

```
int_type uflow();
```

4 *Effects:* Behaves according to the description of `basic_streambuf<charT,traits>::uflow()`, with the specialization that a sequence of characters is read from the input with the same method as used by `underflow`.

```
int_type pbackfail(int_type c = traits::eof());
```

5 *Effects:* Puts back the character designated by `c` to the input sequence, if possible, in one of three ways:

(5.1) — If `traits::eq_int_type(c,traits::eof())` returns `false` and if the function makes a putback position available and if `traits::eq(to_char_type(c),gptr()[-1])` returns `true`, decrements the next pointer for the input sequence, `gptr()`.

Returns: `c`.

(5.2) — If `traits::eq_int_type(c,traits::eof())` returns `false` and if the function makes a putback position available and if the function is permitted to assign to the putback position, decrements the next pointer for the input sequence, and stores `c` there.

Returns: `c`.

(5.3) — If `traits::eq_int_type(c,traits::eof())` returns `true`, and if either the input sequence has a putback position available or the function makes a putback position available, decrements the next pointer for the input sequence, `gptr()`.

Returns: `traits::not_eof(c)`.

6 *Returns:* `traits::eof()` to indicate failure.

7 *Remarks:* If `is_open() == false`, the function always fails.

8 The function does not put back a character directly to the input sequence.

9 If the function can succeed in more than one of these ways, it is unspecified which way is chosen. The function can alter the number of putback positions available as a result of any call.

```
int_type overflow(int_type c = traits::eof());
```

10 *Effects:* Behaves according to the description of `basic_streambuf<charT,traits>::overflow(c)`, except that the behavior of "consuming characters" is performed by first converting as if by:

```
charT* b = pbase();
charT* p = pptr();
charT* end;
char   xbuf[XSIZE];
char*  xbuf_end;
codecvt_base::result r =
  a_codecvt.out(state, b, p, end, xbuf, xbuf+XSIZE, xbuf_end);
```

and then

(10.1) — If `r == codecvt_base::error` then fail.

(10.2) — If `r == codecvt_base::noconv` then output characters from `b` up to (and not including) `p`.

(10.3) — If `r == codecvt_base::partial` then output to the file characters from `xbuf` up to `xbuf_end`, and repeat using characters from `end` to `p`. If output fails, fail (without repeating).

(10.4) — Otherwise output from `xbuf` to `xbuf_end`, and fail if output fails. At this point if `b != p` and `b == end` (`xbuf` isn't large enough) then increase `XSIZE` and repeat from the beginning.

11 *Returns:* `traits::not_eof(c)` to indicate success, and `traits::eof()` to indicate failure. If `is_open() == false`, the function always fails.

```
basic_streambuf* setbuf(char_type* s, streamsize n);
```

12 *Effects:* If `setbuf(0,0)` is called on a stream before any I/O has occurred on that stream, the stream becomes unbuffered. Otherwise the results are implementation-defined. "Unbuffered" means that `pbase()` and `pptr()` always return null and output to the file should appear as soon as possible.

```
pos_type seekoff(off_type off, ios_base::seekdir way,
    ios_base::openmode which = ios_base::in | ios_base::out);
```

13 *Effects:* Let `width` denote `a_codecvt.encoding()`. If `is_open() == false`, or `off != 0 && width <= 0`, then the positioning operation fails. Otherwise, if `way != basic_ios::cur` or `off != 0`, and if the last operation was output, then update the output sequence and write any unshift sequence. Next, seek to the new position: if `width > 0`, call `std::fseek(file, width * off, whence)`, otherwise call `std::fseek(file, 0, whence)`.

14 *Remarks:* "The last operation was output" means either the last virtual operation was overflow or the put buffer is non-empty. "Write any unshift sequence" means, if `width` if less than zero then call `a_codecvt.unshift(state, xbuf, xbuf+XSIZE, xbuf_end)` and output the resulting unshift sequence. The function determines one of three values for the argument `whence`, of type `int`, as indicated in Table 132.

Table 132 — `seekoff` effects

`way` Value	`stdio` Equivalent
`basic_ios::beg`	`SEEK_SET`
`basic_ios::cur`	`SEEK_CUR`
`basic_ios::end`	`SEEK_END`

15 *Returns:* A newly constructed `pos_type` object that stores the resultant stream position, if possible. If the positioning operation fails, or if the object cannot represent the resultant stream position, returns `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp,
    ios_base::openmode which = ios_base::in | ios_base::out);
```

16 Alters the file position, if possible, to correspond to the position stored in `sp` (as described below). Altering the file position performs as follows:

1. if `(om & ios_base::out) != 0`, then update the output sequence and write any unshift sequence;
2. set the file position to `sp`;
3. if `(om & ios_base::in) != 0`, then update the input sequence;

where `om` is the open mode passed to the last call to `open()`. The operation fails if `is_open()` returns false.

17 If `sp` is an invalid stream position, or if the function positions neither sequence, the positioning operation fails. If `sp` has not been obtained by a previous successful call to one of the positioning functions (`seekoff` or `seekpos`) on the same file the effects are undefined.

18 *Returns:* `sp` on success. Otherwise returns `pos_type(off_type(-1))`.

```
int sync();
```

19 *Effects:* If a put area exists, calls `filebuf::overflow` to write the characters to the file. If a get area exists, the effect is implementation-defined.

```
void imbue(const locale& loc);
```

20 *Requires:* If the file is not positioned at its beginning and the encoding of the current locale as determined by `a_codecvt.encoding()` is state-dependent (22.4.1.4.2) then that facet is the same as the corresponding facet of `loc`.

21 *Effects:* Causes characters inserted or extracted after this call to be converted according to `loc` until another call of `imbue`.

22 *Remark:* This may require reconversion of previously converted characters. This in turn may require the implementation to be able to reconstruct the original contents of the file.

27.9.1.6 Class template `basic_ifstream` [ifstream]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_ifstream : public basic_istream<charT,traits> {
  public:
    typedef charT                     char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                    traits_type;

    // 27.9.1.7 Constructors:
    basic_ifstream();
    explicit basic_ifstream(const char* s,
        ios_base::openmode mode = ios_base::in);
    explicit basic_ifstream(const string& s,
        ios_base::openmode mode = ios_base::in);
    basic_ifstream(const basic_ifstream& rhs) = delete;
    basic_ifstream(basic_ifstream&& rhs);

    // 27.9.1.8 Assign/swap:
    basic_ifstream& operator=(const basic_ifstream& rhs) = delete;
    basic_ifstream& operator=(basic_ifstream&& rhs);
    void swap(basic_ifstream& rhs);
```

```
    // 27.9.1.9 Members:
    basic_filebuf<charT,traits>* rdbuf() const;

    bool is_open() const;
    void open(const char* s, ios_base::openmode mode = ios_base::in);
    void open(const string& s, ios_base::openmode mode = ios_base::in);
    void close();
  private:
    basic_filebuf<charT,traits> sb; // exposition only
  };

  template <class charT, class traits>
  void swap(basic_ifstream<charT, traits>& x,
            basic_ifstream<charT, traits>& y);
}
```

1 The class `basic_ifstream<charT, traits>` supports reading from named files. It uses a `basic_filebuf<charT, traits>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `filebuf` object.

27.9.1.7 `basic_ifstream` constructors [ifstream.cons]

```
basic_ifstream();
```

1 *Effects:* Constructs an object of class `basic_ifstream<charT,traits>`, initializing the base class with `basic_istream(&sb)` and initializing `sb` with `basic_filebuf<charT,traits>()` (27.7.2.1.1, 27.9.1.2).

```
explicit basic_ifstream(const char* s,
    ios_base::openmode mode = ios_base::in);
```

2 *Effects:* Constructs an object of class `basic_ifstream`, initializing the base class with `basic_istream(&sb)` and initializing `sb` with `basic_filebuf<charT, traits>()` (27.7.2.1.1, 27.9.1.2), then calls `rdbuf()->open(s, mode | ios_base::in)`. If that function returns a null pointer, calls `setstate(failbit)`.

```
explicit basic_ifstream(const string& s,
    ios_base::openmode mode = ios_base::in);
```

3 *Effects:* the same as `basic_ifstream(s.c_str(), mode)`.

```
basic_ifstream(basic_ifstream&& rhs);
```

4 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_istream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_filebuf`.

27.9.1.8 Assign and swap [ifstream.assign]

```
basic_ifstream& operator=(basic_ifstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_ifstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_istream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits>
void swap(basic_ifstream<charT, traits>& x,
          basic_ifstream<charT, traits>& y);
```

4 *Effects:* `x.swap(y)`.

27.9.1.9 Member functions [ifstream.members]

```
basic_filebuf<charT,traits>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_filebuf<charT,traits>*>(&sb)`.

```
bool is_open() const;
```

2 *Returns:* `rdbuf()->is_open()`.

```
void open(const char* s, ios_base::openmode mode = ios_base::in);
```

3 *Effects:* Calls `rdbuf()->open(s, mode | ios_base::in)`. If that function does not return a null pointer calls `clear()`, otherwise calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

```
void open(const string& s, ios_base::openmode mode = ios_base::in);
```

4 *Effects:* calls `open(s.c_str(), mode)`.

```
void close();
```

5 *Effects:* Calls `rdbuf()->close()` and, if that function returns a null pointer, calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

27.9.1.10 Class template basic_ofstream [ofstream]

```
namespace std {
  template <class charT, class traits = char_traits<charT> >
  class basic_ofstream : public basic_ostream<charT,traits> {
  public:
    typedef charT                    char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                   traits_type;

    // 27.9.1.11 Constructors:
    basic_ofstream();
    explicit basic_ofstream(const char* s,
        ios_base::openmode mode = ios_base::out);
    explicit basic_ofstream(const string& s,
        ios_base::openmode mode = ios_base::out);
    basic_ofstream(const basic_ofstream& rhs) = delete;
    basic_ofstream(basic_ofstream&& rhs);

    // 27.9.1.12 Assign/swap:
    basic_ofstream& operator=(const basic_ofstream& rhs) = delete;
```

```
    basic_ofstream& operator=(basic_ofstream&& rhs);
    void swap(basic_ofstream& rhs);

    // 27.9.1.13 Members:
    basic_filebuf<charT,traits>* rdbuf() const;

    bool is_open() const;
    void open(const char* s, ios_base::openmode mode = ios_base::out);
    void open(const string& s, ios_base::openmode mode = ios_base::out);
    void close();
  private:
    basic_filebuf<charT,traits> sb; // exposition only
  };

  template <class charT, class traits>
  void swap(basic_ofstream<charT, traits>& x,
            basic_ofstream<charT, traits>& y);
}
```

1 The class `basic_ofstream<charT, traits>` supports writing to named files. It uses a `basic_filebuf<charT, traits>` object to control the associated sequence. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `filebuf` object.

27.9.1.11 `basic_ofstream` constructors [ofstream.cons]

```
basic_ofstream();
```

1 *Effects:* Constructs an object of class `basic_ofstream<charT,traits>`, initializing the base class with `basic_ostream(&sb)` and initializing `sb` with `basic_filebuf<charT,traits>()` (27.7.3.2, 27.9.1.2).

```
explicit basic_ofstream(const char* s,
    ios_base::openmode mode = ios_base::out);
```

2 *Effects:* Constructs an object of class `basic_ofstream<charT,traits>`, initializing the base class with `basic_ostream(&sb)` and initializing `sb` with `basic_filebuf<charT,traits>()` (27.7.3.2, 27.9.1.2), then calls `rdbuf()->open(s, mode|ios_base::out)`. If that function returns a null pointer, calls `setstate(failbit)`.

```
explicit basic_ofstream(const string& s,
    ios_base::openmode mode = ios_base::out);
```

3 *Effects:* the same as `basic_ofstream(s.c_str(), mode);`

```
basic_ofstream(basic_ofstream&& rhs);
```

4 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_ostream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_filebuf`.

27.9.1.12 Assign and swap [ofstream.assign]

```
basic_ofstream& operator=(basic_ofstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_ofstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_ostream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits>
void swap(basic_ofstream<charT, traits>& x,
          basic_ofstream<charT, traits>& y);
```

4 *Effects:* `x.swap(y)`.

27.9.1.13 Member functions [ofstream.members]

```
basic_filebuf<charT,traits>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_filebuf<charT,traits>*>(&sb)`.

```
bool is_open() const;
```

2 *Returns:* `rdbuf()->is_open()`.

```
void open(const char* s, ios_base::openmode mode = ios_base::out);
```

3 *Effects:* Calls `rdbuf()->open(s, mode | ios_base::out)`. If that function does not return a null pointer calls `clear()`, otherwise calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

```
void close();
```

4 *Effects:* Calls `rdbuf()->close()` and, if that function fails (returns a null pointer), calls `setstate(failbit)` (which may throw `ios_base::failure` (27.5.5.4)).

```
void open(const string& s, ios_base::openmode mode = ios_base::out);
```

5 *Effects:* calls `open(s.c_str(), mode);`

27.9.1.14 Class template `basic_fstream` [fstream]

```
namespace std {
  template <class charT, class traits=char_traits<charT> >
  class basic_fstream
    : public basic_iostream<charT,traits> {

  public:
    typedef charT                    char_type;
    typedef typename traits::int_type int_type;
    typedef typename traits::pos_type pos_type;
    typedef typename traits::off_type off_type;
    typedef traits                   traits_type;

    // constructors/destructor
    basic_fstream();
    explicit basic_fstream(const char* s,
        ios_base::openmode mode = ios_base::in|ios_base::out);
    explicit basic_fstream(const string& s,
        ios_base::openmode mode = ios_base::in|ios_base::out);
    basic_fstream(const basic_fstream& rhs) = delete;
    basic_fstream(basic_fstream&& rhs);
```

```
    // 27.9.1.16 Assign/swap:
    basic_fstream& operator=(const basic_fstream& rhs) = delete;
    basic_fstream& operator=(basic_fstream&& rhs);
    void swap(basic_fstream& rhs);

    // Members:
    basic_filebuf<charT,traits>* rdbuf() const;
    bool is_open() const;
    void open(const char* s,
        ios_base::openmode mode = ios_base::in|ios_base::out);
    void open(const string& s,
        ios_base::openmode mode = ios_base::in|ios_base::out);
    void close();

  private:
    basic_filebuf<charT,traits> sb; // exposition only
  };

  template <class charT, class traits>
  void swap(basic_fstream<charT, traits>& x,
            basic_fstream<charT, traits>& y);
}
```

1 The class template `basic_fstream<charT,traits>` supports reading and writing from named files. It uses a `basic_filebuf<charT,traits>` object to control the associated sequences. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `basic_filebuf` object.

27.9.1.15 `basic_fstream` constructors [fstream.cons]

```
basic_fstream();
```

1 *Effects:* Constructs an object of class `basic_fstream<charT,traits>`, initializing the base class with `basic_iostream(&sb)` and initializing `sb` with `basic_filebuf<charT,traits>()`.

```
explicit basic_fstream(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

2 *Effects:* Constructs an object of class `basic_fstream<charT, traits>`, initializing the base class with `basic_iostream(&sb)` and initializing `sb` with `basic_filebuf<charT, traits>()`. Then calls `rdbuf()->open(s, mode)`. If that function returns a null pointer, calls `setstate(failbit)`.

```
explicit basic_fstream(const string& s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

3 *Effects:* the same as `basic_fstream(s.c_str(), mode);`

```
basic_fstream(basic_fstream&& rhs);
```

4 *Effects:* Move constructs from the rvalue `rhs`. This is accomplished by move constructing the base class, and the contained `basic_filebuf`. Next `basic_istream<charT,traits>::set_rdbuf(&sb)` is called to install the contained `basic_filebuf`.

27.9.1.16 Assign and swap [fstream.assign]

```
basic_fstream& operator=(basic_fstream&& rhs);
```

1 *Effects:* Move assigns the base and members of `*this` from the base and corresponding members of `rhs`.

2 *Returns:* `*this`.

```
void swap(basic_fstream& rhs);
```

3 *Effects:* Exchanges the state of `*this` and `rhs` by calling `basic_iostream<charT,traits>::swap(rhs)` and `sb.swap(rhs.sb)`.

```
template <class charT, class traits>
void swap(basic_fstream<charT, traits>& x,
          basic_fstream<charT, traits>& y);
```

4 *Effects:* `x.swap(y)`.

27.9.1.17 Member functions [fstream.members]

```
basic_filebuf<charT,traits>* rdbuf() const;
```

1 *Returns:* `const_cast<basic_filebuf<charT,traits>*>(&sb)`.

```
bool is_open() const;
```

2 *Returns:* `rdbuf()->is_open()`.

```
void open(const char* s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

3 *Effects:* Calls `rdbuf()->open(s,mode)`. If that function does not return a null pointer calls `clear()`, otherwise calls `setstate(failbit)`, (which may throw `ios_base::failure`) (27.5.5.4).

```
void open(const string& s,
    ios_base::openmode mode = ios_base::in|ios_base::out);
```

4 *Effects:* calls `open(s.c_str(), mode);`

```
void close();
```

5 *Effects:* Calls `rdbuf()->close()` and, if that function returns returns a null pointer, calls `setstate(failbit)` (27.5.5.4) (which may throw `ios_base::failure`).

27.9.2 C library files [c.files]

1 Table 133 describes header `<cstdio>`. [*Note:* C++ does not define the function `gets`. — *end note*]

2 Calls to the function `tmpnam` with an argument of `NULL` may introduce a data race (17.6.5.9) with other calls to `tmpnam` with an argument of `NULL`.

See also: ISO C 7.9, Amendment 1 4.6.2.

3 Table 134 describes header `<cinttypes>`. [*Note:* The macros defined by `<cinttypes>` are provided unconditionally. In particular, the symbol `__STDC_FORMAT_MACROS`, mentioned in footnote 182 of the C standard, plays no role in C++. — *end note*]

4 The contents of header `<cinttypes>` are the same as the Standard C Library header `<inttypes.h>`, with the following changes:

(4.1) — the header `<cinttypes>` includes the header `<cstdint>` instead of `<stdint.h>`, and

(4.2) — if and only if the type `intmax_t` designates an extended integer type (3.9.1), the following function signatures are added:

Table 133 — Header `<cstdio>` synopsis

Type	Name(s)				
Macros:					
BUFSIZ	FOPEN_MAX	SEEK_CUR	TMP_MAX	_IONBF	stdout
EOF	L_tmpnam	SEEK_END	_IOFBF	stderr	
FILENAME_MAX	NULL <cstdio>	SEEK_SET	_IOLBF	stdin	
Types:	FILE	fpos_t	size_t <cstdio>		
Functions:					
clearerr	fopen	fsetpos	putchar	snprintf	vscanf
fclose	fprintf	ftell	puts	sprintf	vsnprintf
feof	fputc	fwrite	remove	sscanf	vsprintf
ferror	fputs	getc	rename	tmpfile	vsscanf
fflush	fread	getchar	rewind	tmpnam	
fgetc	freopen	perror	scanf	ungetc	
fgetpos	fscanf	printf	setbuf	vfprintf	
fgets	fseek	putc	setvbuf	vprintf	

Table 134 — Header `<cinttypes>` synopsis

Type	Name(s)		
Macros:			
PRI{d i o u x X}[FAST LEAST]{8 16 32 64}			
PRI{d i o u x X}{MAX PTR}			
SCN{d i o u x X}[FAST LEAST]{8 16 32 64}			
SCN{d i o u x X}{MAX PTR}			
Types: imaxdiv_t			
Functions:			
abs	imaxabs	strtoimax	wcstoimax
div	imaxdiv	strtoumax	wcstoumax

```
intmax_t abs(intmax_t);
imaxdiv_t div(intmax_t, intmax_t);
```

which shall have the same semantics as the function signatures `intmax_t imaxabs(intmax_t)` and `imaxdiv_t imaxdiv(intmax_t, intmax_t)`, respectively.

28 Regular expressions library [re]

28.1 General [re.general]

1 This Clause describes components that C++ programs may use to perform operations involving regular expression matching and searching.

2 The following subclauses describe a basic regular expression class template and its traits that can handle char-like template arguments, two specializations of this class template that handle sequences of `char` and `wchar_t`, a class template that holds the result of a regular expression match, a series of algorithms that allow a character sequence to be operated upon by a regular expression, and two iterator types for enumerating regular expression matches, as described in Table 135.

Table 135 — Regular expressions library summary

Subclause		Header(s)
28.2	Definitions	
28.3	Requirements	
28.5	Constants	
28.6	Exception type	
28.7	Traits	
28.8	Regular expression template	`<regex>`
28.9	Submatches	
28.10	Match results	
28.11	Algorithms	
28.12	Iterators	
28.13	Grammar	

28.2 Definitions [re.def]

1 The following definitions shall apply to this Clause:

28.2.1 [defns.regex.collating.element]

collating element
a sequence of one or more characters within the current locale that collate as if they were a single character.

28.2.2 [defns.regex.finite.state.machine]

finite state machine
an unspecified data structure that is used to represent a regular expression, and which permits efficient matches against the regular expression to be obtained.

28.2.3 [defns.regex.format.specifier]

format specifier
a sequence of one or more characters that is to be replaced with some part of a regular expression match.

28.2.4 [defns.regex.matched]

matched
a sequence of zero or more characters is matched by a regular expression when the characters in the sequence correspond to a sequence of characters defined by the pattern.

28.2.5 [defns.regex.primary.equivalence.class]

primary equivalence class

a set of one or more characters which share the same primary sort key: that is the sort key weighting that depends only upon character shape, and not accents, case, or locale specific tailorings.

28.2.6 [defns.regex.regular.expression]

regular expression

a pattern that selects specific strings from a set of character strings.

28.2.7 [defns.regex.subexpression]

sub-expression

a subset of a regular expression that has been marked by parenthesis.

28.3 Requirements [re.req]

1 This subclause defines requirements on classes representing regular expression traits. [*Note:* The class template `regex_traits`, defined in Clause 28.7, satisfies these requirements. — *end note*]

2 The class template `basic_regex`, defined in Clause 28.8, needs a set of related types and functions to complete the definition of its semantics. These types and functions are provided as a set of member typedefs and functions in the template parameter `traits` used by the `basic_regex` class template. This subclause defines the semantics of these members.

3 To specialize class template `basic_regex` for a character container `CharT` and its related regular expression traits class `Traits`, use `basic_regex<CharT, Traits>`.

4 In Table 136 `X` denotes a traits class defining types and functions for the character container type `charT`; `u` is an object of type `X`; `v` is an object of type `const X`; `p` is a value of type `const charT*`; `I1` and `I2` are input iterators (24.2.3); `F1` and `F2` are forward iterators (24.2.5); `c` is a value of type `const charT`; `s` is an object of type `X::string_type`; `cs` is an object of type `const X::string_type`; `b` is a value of type `bool`; `I` is a value of type `int`; `cl` is an object of type `X::char_class_type`, and `loc` is an object of type `X::locale_type`.

Table 136 — Regular expression traits class requirements

Expression	Return type	Assertion/note pre-/post-condition
`X::char_type`	`charT`	The character container type used in the implementation of class template `basic_regex`.
`X::string_type`	`std::basic_string<charT>`	
`X::locale_type`	A copy constructible type	A type that represents the locale used by the traits class.
`X::char_class_type`	A bitmask type (17.5.2.1.3).	A bitmask type representing a particular character classification.
`X::length(p)`	`std::size_t`	Yields the smallest `i` such that `p[i] == 0`. Complexity is linear in `i` .
`v.translate(c)`	`X::char_type`	Returns a character such that for any character `d` that is to be considered equivalent to `c` then `v.translate(c) == v.translate(d)`.

Table 136 — Regular expression traits class requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition
`v.translate_nocase(c)`	`X::char_type`	For all characters `C` that are to be considered equivalent to `c` when comparisons are to be performed without regard to case, then `v.translate_nocase(c) == v.translate_nocase(C)`.
`v.transform(F1, F2)`	`X::string_type`	Returns a sort key for the character sequence designated by the iterator range `[F1,F2)` such that if the character sequence `[G1,G2)` sorts before the character sequence `[H1,H2)` then `v.transform(G1, G2) < v.transform(H1, H2)`.
`v.transform_primary(F1, F2)`	`X::string_type`	Returns a sort key for the character sequence designated by the iterator range `[F1,F2)` such that if the character sequence `[G1,G2)` sorts before the character sequence `[H1,H2)` when character case is not considered then `v.transform_primary(G1, G2) < v.transform_primary(H1, H2)`.
`v.lookup_collatename(F1, F2)`	`X::string_type`	Returns a sequence of characters that represents the collating element consisting of the character sequence designated by the iterator range `[F1,F2)`. Returns an empty string if the character sequence is not a valid collating element.
`v.lookup_classname(F1, F2, b)`	`X::char_class_type`	Converts the character sequence designated by the iterator range `[F1,F2)` into a value of a bitmask type that can subsequently be passed to `isctype`. Values returned from `lookup_classname` can be bitwise or'ed together; the resulting value represents membership in either of the corresponding character classes. If `b` is true, the returned bitmask is suitable for matching characters without regard to their case. Returns 0 if the character sequence is not the name of a character class recognized by `X`. The value returned shall be independent of the case of the characters in the sequence.
`v.isctype(c, cl)`	`bool`	Returns `true` if character `c` is a member of one of the character classes designated by `cl`, `false` otherwise.
`v.value(c, I)`	`int`	Returns the value represented by the digit c in base I if the character c is a valid digit in base I; otherwise returns `-1`. [*Note:* The value of I will only be 8, 10, or 16. — *end note*]

Table 136 — Regular expression traits class requirements (continued)

Expression	Return type	Assertion/note pre-/post-condition
`u.imbue(loc)`	`X::locale_type`	Imbues `u` with the locale `loc` and returns the previous locale used by `u` if any.
`v.getloc()`	`X::locale_type`	Returns the current locale used by `v`, if any.

5 [*Note:* Class template `regex_traits` satisfies the requirements for a regular expression traits class when it is specialized for `char` or `wchar_t`. This class template is described in the header `<regex>`, and is described in Clause 28.7. — *end note*]

28.4 Header <regex> synopsis [re.syn]

```
#include <initializer_list>

namespace std {

  // 28.5, regex constants:
  namespace regex_constants {
    enum error_type;
  } // namespace regex_constants

  // 28.6, class regex_error:
  class regex_error;

  // 28.7, class template regex_traits:
  template <class charT> struct regex_traits;

  // 28.8, class template basic_regex:
  template <class charT, class traits = regex_traits<charT> > class basic_regex;

  typedef basic_regex<char>    regex;
  typedef basic_regex<wchar_t> wregex;

  // 28.8.6, basic_regex swap:
  template <class charT, class traits>
    void swap(basic_regex<charT, traits>& e1, basic_regex<charT, traits>& e2);

  // 28.9, class template sub_match:
  template <class BidirectionalIterator>
    class sub_match;

  typedef sub_match<const char*>             csub_match;
  typedef sub_match<const wchar_t*>          wcsub_match;
  typedef sub_match<string::const_iterator>  ssub_match;
  typedef sub_match<wstring::const_iterator> wssub_match;

  // 28.9.2, sub_match non-member operators:
  template <class BiIter>
    bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
  template <class BiIter>
    bool operator!=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
  template <class BiIter>
```

```
  bool operator<(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);


template <class BiIter, class ST, class SA>
  bool operator==(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
template <class BiIter, class ST, class SA>
  bool operator!=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
template <class BiIter, class ST, class SA>
  bool operator<(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
template <class BiIter, class ST, class SA>
  bool operator>(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
template <class BiIter, class ST, class SA>
  bool operator>=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
template <class BiIter, class ST, class SA>
  bool operator<=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);

template <class BiIter, class ST, class SA>
  bool operator==(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template <class BiIter, class ST, class SA>
  bool operator!=(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template <class BiIter, class ST, class SA>
  bool operator<(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

```
template <class BiIter, class ST, class SA>
  bool operator>(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template <class BiIter, class ST, class SA>
  bool operator>=(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
template <class BiIter, class ST, class SA>
  bool operator<=(
    const sub_match<BiIter>& lhs,
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);

template <class BiIter>
  bool operator==(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator!=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator<(typename iterator_traits<BiIter>::value_type const* lhs,
                 const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>(typename iterator_traits<BiIter>::value_type const* lhs,
                 const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator<=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);

template <class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
template <class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
template <class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const* rhs);
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const* rhs);
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);

template <class BiIter>
```

```
  bool operator==(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator!=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator<(typename iterator_traits<BiIter>::value_type const& lhs,
                 const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>(typename iterator_traits<BiIter>::value_type const& lhs,
                 const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator>=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
template <class BiIter>
  bool operator<=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);

template <class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
template <class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
template <class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const& rhs);
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const& rhs);
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);

template <class charT, class ST, class BiIter>
  basic_ostream<charT, ST>&
  operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);

// 28.10, class template match_results:
template <class BidirectionalIterator,
          class Allocator = allocator<sub_match<BidirectionalIterator> > >
  class match_results;

typedef match_results<const char*>             cmatch;
typedef match_results<const wchar_t*>          wcmatch;
typedef match_results<string::const_iterator>  smatch;
typedef match_results<wstring::const_iterator> wsmatch;

// match_results comparisons
template <class BidirectionalIterator, class Allocator>
  bool operator== (const match_results<BidirectionalIterator, Allocator>& m1,
                   const match_results<BidirectionalIterator, Allocator>& m2);
```

```
template <class BidirectionalIterator, class Allocator>
  bool operator!= (const match_results<BidirectionalIterator, Allocator>& m1,
                   const match_results<BidirectionalIterator, Allocator>& m2);

// 28.10.7, match_results swap:
template <class BidirectionalIterator, class Allocator>
  void swap(match_results<BidirectionalIterator, Allocator>& m1,
            match_results<BidirectionalIterator, Allocator>& m2);

// 28.11.2, function template regex_match:
template <class BidirectionalIterator, class Allocator,
    class charT, class traits>
  bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                   match_results<BidirectionalIterator, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
template <class BidirectionalIterator, class charT, class traits>
bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                 const basic_regex<charT, traits>& e,
                 regex_constants::match_flag_type flags =
                   regex_constants::match_default);
template <class charT, class Allocator, class traits>
  bool regex_match(const charT* str, match_results<const charT*, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                   match_results<
                     typename basic_string<charT, ST, SA>::const_iterator,
                     Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>&&,
                   match_results<
                     typename basic_string<charT, ST, SA>::const_iterator,
                     Allocator>&,
                   const basic_regex<charT, traits>&,
                   regex_constants::match_flag_type =
                     regex_constants::match_default) = delete;
template <class charT, class traits>
  bool regex_match(const charT* str,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
template <class ST, class SA, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);

// 28.11.3, function template regex_search:
```

```
template <class BidirectionalIterator, class Allocator,
    class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                    match_results<BidirectionalIterator, Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class BidirectionalIterator, class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class charT, class Allocator, class traits>
  bool regex_search(const charT* str,
                    match_results<const charT*, Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class charT, class traits>
  bool regex_search(const charT* str,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class ST, class SA, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    match_results<
                      typename basic_string<charT, ST, SA>::const_iterator,
                      Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>&&,
                    match_results<
                      typename basic_string<charT, ST, SA>::const_iterator,
                      Allocator>&,
                    const basic_regex<charT, traits>&,
                    regex_constants::match_flag_type =
                      regex_constants::match_default) = delete;

// 28.11.4, function template regex_replace:
template <class OutputIterator, class BidirectionalIterator,
    class traits, class charT, class ST, class SA>
  OutputIterator
  regex_replace(OutputIterator out,
                BidirectionalIterator first, BidirectionalIterator last,
                const basic_regex<charT, traits>& e,
                const basic_string<charT, ST, SA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
```

```
template <class OutputIterator, class BidirectionalIterator,
    class traits, class charT>
  OutputIterator
  regex_replace(OutputIterator out,
                BidirectionalIterator first, BidirectionalIterator last,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT, class ST, class SA,
    class FST, class FSA>
  basic_string<charT, ST, SA>
  regex_replace(const basic_string<charT, ST, SA>& s,
                const basic_regex<charT, traits>& e,
                const basic_string<charT, FST, FSA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT, class ST, class SA>
  basic_string<charT, ST, SA>
  regex_replace(const basic_string<charT, ST, SA>& s,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT, class ST, class SA>
  basic_string<charT>
  regex_replace(const charT* s,
                const basic_regex<charT, traits>& e,
                const basic_string<charT, ST, SA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT>
  basic_string<charT>
  regex_replace(const charT* s,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);

// 28.12.1, class template regex_iterator:
template <class BidirectionalIterator,
          class charT = typename iterator_traits<
            BidirectionalIterator>::value_type,
          class traits = regex_traits<charT> >
  class regex_iterator;

typedef regex_iterator<const char*>             cregex_iterator;
typedef regex_iterator<const wchar_t*>          wcregex_iterator;
typedef regex_iterator<string::const_iterator>  sregex_iterator;
typedef regex_iterator<wstring::const_iterator> wsregex_iterator;

// 28.12.2, class template regex_token_iterator:
template <class BidirectionalIterator,
          class charT = typename iterator_traits<
            BidirectionalIterator>::value_type,
```

```
          class traits = regex_traits<charT> >
    class regex_token_iterator;

  typedef regex_token_iterator<const char*>             cregex_token_iterator;
  typedef regex_token_iterator<const wchar_t*>          wcregex_token_iterator;
  typedef regex_token_iterator<string::const_iterator>  sregex_token_iterator;
  typedef regex_token_iterator<wstring::const_iterator> wsregex_token_iterator;
}
```

28.5 Namespace `std::regex_constants` [re.const]

1 The namespace `std::regex_constants` holds symbolic constants used by the regular expression library. This namespace provides three types, `syntax_option_type`, `match_flag_type`, and `error_type`, along with several constants of these types.

28.5.1 Bitmask type `syntax_option_type` [re.synopt]

```
namespace std::regex_constants {
  typedef T1 syntax_option_type;
  constexpr syntax_option_type icase = unspecified ;
  constexpr syntax_option_type nosubs = unspecified ;
  constexpr syntax_option_type optimize = unspecified ;
  constexpr syntax_option_type collate = unspecified ;
  constexpr syntax_option_type ECMAScript = unspecified ;
  constexpr syntax_option_type basic = unspecified ;
  constexpr syntax_option_type extended = unspecified ;
  constexpr syntax_option_type awk = unspecified ;
  constexpr syntax_option_type grep = unspecified ;
  constexpr syntax_option_type egrep = unspecified ;
}
```

1 The type `syntax_option_type` is an implementation-defined bitmask type (17.5.2.1.3). Setting its elements has the effects listed in table 137. A valid value of type `syntax_option_type` shall have at most one of the grammar elements `ECMAScript`, `basic`, `extended`, `awk`, `grep`, `egrep`, set. If no grammar element is set, the default grammar is `ECMAScript`.

28.5.2 Bitmask type `regex_constants::match_flag_type` [re.matchflag]

```
namespace std::regex_constants {
  typedef T2 match_flag_type;
  constexpr match_flag_type match_default = {};
  constexpr match_flag_type match_not_bol = unspecified ;
  constexpr match_flag_type match_not_eol = unspecified ;
  constexpr match_flag_type match_not_bow = unspecified ;
  constexpr match_flag_type match_not_eow = unspecified ;
  constexpr match_flag_type match_any = unspecified ;
  constexpr match_flag_type match_not_null = unspecified ;
  constexpr match_flag_type match_continuous = unspecified ;
  constexpr match_flag_type match_prev_avail = unspecified ;
  constexpr match_flag_type format_default = {};
  constexpr match_flag_type format_sed = unspecified ;
  constexpr match_flag_type format_no_copy = unspecified ;
  constexpr match_flag_type format_first_only = unspecified ;
}
```

1 The type `regex_constants::match_flag_type` is an implementation-defined bitmask type (17.5.2.1.3). The constants of that type, except for `match_default` and `format_default`, are bitmask elements. The

Table 137 — `syntax_option_type` effects

Element	Effect(s) if set
`icase`	Specifies that matching of regular expressions against a character container sequence shall be performed without regard to case.
`nosubs`	Specifies that no sub-expressions shall be considered to be marked, so that when a regular expression is matched against a character container sequence, no sub-expression matches shall be stored in the supplied `match_results` structure.
`optimize`	Specifies that the regular expression engine should pay more attention to the speed with which regular expressions are matched, and less to the speed with which regular expression objects are constructed. Otherwise it has no detectable effect on the program output.
`collate`	Specifies that character ranges of the form `"[a-b]"` shall be locale sensitive.
`ECMAScript`	Specifies that the grammar recognized by the regular expression engine shall be that used by ECMAScript in ECMA-262, as modified in 28.13.
`basic`	Specifies that the grammar recognized by the regular expression engine shall be that used by basic regular expressions in POSIX, Base Definitions and Headers, Section 9, Regular Expressions.
`extended`	Specifies that the grammar recognized by the regular expression engine shall be that used by extended regular expressions in POSIX, Base Definitions and Headers, Section 9, Regular Expressions.
`awk`	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility awk in POSIX.
`grep`	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility grep in POSIX.
`egrep`	Specifies that the grammar recognized by the regular expression engine shall be that used by the utility grep when given the -E option in POSIX.

`match_default` and `format_default` constants are empty bitmasks. Matching a regular expression against a sequence of characters `[first,last)` proceeds according to the rules of the grammar specified for the regular expression object, modified according to the effects listed in Table 138 for any bitmask elements set.

Table 138 — `regex_constants::match_flag_type` effects when obtaining a match against a character container sequence `[first, last)`.

Element	Effect(s) if set
`match_not_bol`	The first character in the sequence `[first,last)` shall be treated as though it is not at the beginning of a line, so the character `^` in the regular expression shall not match `[first,first)`.
`match_not_eol`	The last character in the sequence `[first,last)` shall be treated as though it is not at the end of a line, so the character `"$"` in the regular expression shall not match `[last,last)`.
`match_not_bow`	The expression `"\\b"` shall not match the sub-sequence `[first,first)`.
`match_not_eow`	The expression `"\\b"` shall not match the sub-sequence `[last,last)`.
`match_any`	If more than one match is possible then any match is an acceptable result.
`match_not_null`	The expression shall not match an empty sequence.

Table 138 — `regex_constants::match_flag_type` effects when obtaining a match against a character container sequence `[first, last)`. (continued)

Element	Effect(s) if set
`match_continuous`	The expression shall only match a sub-sequence that begins at `first`.
`match_prev_avail`	`--first` is a valid iterator position. When this flag is set the flags match_not_bol and match_not_bow shall be ignored by the regular expression algorithms 28.11 and iterators 28.12.
`format_default`	When a regular expression match is to be replaced by a new string, the new string shall be constructed using the rules used by the ECMAScript replace function in ECMA-262, part 15.5.4.11 String.prototype.replace. In addition, during search and replace operations all non-overlapping occurrences of the regular expression shall be located and replaced, and sections of the input that did not match the expression shall be copied unchanged to the output string.
`format_sed`	When a regular expression match is to be replaced by a new string, the new string shall be constructed using the rules used by the sed utility in POSIX.
`format_no_copy`	During a search and replace operation, sections of the character container sequence being searched that do not match the regular expression shall not be copied to the output string.
`format_first_only`	When specified during a search and replace operation, only the first occurrence of the regular expression shall be replaced.

28.5.3 Implementation-defined `error_type` [re.err]

```
namespace std::regex_constants {
  typedef T3 error_type;
  constexpr error_type error_collate = unspecified ;
  constexpr error_type error_ctype = unspecified ;
  constexpr error_type error_escape = unspecified ;
  constexpr error_type error_backref = unspecified ;
  constexpr error_type error_brack = unspecified ;
  constexpr error_type error_paren = unspecified ;
  constexpr error_type error_brace = unspecified ;
  constexpr error_type error_badbrace = unspecified ;
  constexpr error_type error_range = unspecified ;
  constexpr error_type error_space = unspecified ;
  constexpr error_type error_badrepeat = unspecified ;
  constexpr error_type error_complexity = unspecified ;
  constexpr error_type error_stack = unspecified ;
}
```

1 The type `error_type` is an implementation-defined enumerated type (17.5.2.1.2). Values of type `error_type` represent the error conditions described in Table 139:

Table 139 — `error_type` values in the C locale

Value	Error condition
`error_collate`	The expression contained an invalid collating element name.
`error_ctype`	The expression contained an invalid character class name.

Table 139 — `error_type` values in the C locale (continued)

Value	Error condition
`error_escape`	The expression contained an invalid escaped character, or a trailing escape.
`error_backref`	The expression contained an invalid back reference.
`error_brack`	The expression contained mismatched `[` and `]`.
`error_paren`	The expression contained mismatched `(` and `)`.
`error_brace`	The expression contained mismatched `{` and `}`
`error_badbrace`	The expression contained an invalid range in a `{}` expression.
`error_range`	The expression contained an invalid character range, such as `[b-a]` in most encodings.
`error_space`	There was insufficient memory to convert the expression into a finite state machine.
`error_badrepeat`	One of `*?+{` was not preceded by a valid regular expression.
`error_complexity`	The complexity of an attempted match against a regular expression exceeded a pre-set level.
`error_stack`	There was insufficient memory to determine whether the regular expression could match the specified character sequence.

28.6 Class `regex_error` [re.badexp]

```
class regex_error : public std::runtime_error {
  public:
    explicit regex_error(regex_constants::error_type ecode);
    regex_constants::error_type code() const;
};
```

1 The class `regex_error` defines the type of objects thrown as exceptions to report errors from the regular expression library.

```
regex_error(regex_constants::error_type ecode);
```

2 *Effects:* Constructs an object of class `regex_error`.

3 *Postcondition:* `ecode == code()`

```
regex_constants::error_type code() const;
```

4 *Returns:* The error code that was passed to the constructor.

28.7 Class template `regex_traits` [re.traits]

```
namespace std {
  template <class charT>
  struct regex_traits {
  public:
     typedef charT                       char_type;
     typedef std::basic_string<char_type> string_type;
     typedef std::locale                 locale_type;
     typedef bitmask_type                char_class_type;

     regex_traits();
     static std::size_t length(const char_type* p);
     charT translate(charT c) const;
     charT translate_nocase(charT c) const;
```

```
    template <class ForwardIterator>
      string_type transform(ForwardIterator first, ForwardIterator last) const;
    template <class ForwardIterator>
      string_type transform_primary(
        ForwardIterator first, ForwardIterator last) const;
    template <class ForwardIterator>
      string_type lookup_collatename(
        ForwardIterator first, ForwardIterator last) const;
    template <class ForwardIterator>
      char_class_type lookup_classname(
        ForwardIterator first, ForwardIterator last, bool icase = false) const;
    bool isctype(charT c, char_class_type f) const;
    int value(charT ch, int radix) const;
    locale_type imbue(locale_type l);
    locale_type getloc() const;
  };
}
```

1 The specializations `regex_traits<char>` and `regex_traits<wchar_t>` shall be valid and shall satisfy the requirements for a regular expression traits class (28.3).

`typedef` *`bitmask_type`* `char_class_type;`

2 The type `char_class_type` is used to represent a character classification and is capable of holding an implementation specific set returned by `lookup_classname`.

```
static std::size_t length(const char_type* p);
```

3 *Returns:* `char_traits<charT>::length(p);`

```
charT translate(charT c) const;
```

4 *Returns:* `(c)`.

```
charT translate_nocase(charT c) const;
```

5 *Returns:* `use_facet<ctype<charT> >(getloc()).tolower(c)`.

```
template <class ForwardIterator>
  string_type transform(ForwardIterator first, ForwardIterator last) const;
```

6 *Effects:*

```
  string_type str(first, last);
  return use_facet<collate<charT> >(
    getloc()).transform(&*str.begin(), &*str.begin() + str.length());
```

```
template <class ForwardIterator>
  string_type transform_primary(ForwardIterator first, ForwardIterator last) const;
```

7 *Effects:* if `typeid(use_facet<collate<charT> >) == typeid(collate_byname<charT>)` and the form of the sort key returned by `collate_byname<charT> ::transform(first, last)` is known and can be converted into a primary sort key then returns that key, otherwise returns an empty string.

```
template <class ForwardIterator>
  string_type lookup_collatename(ForwardIterator first, ForwardIterator last) const;
```

8 *Returns:* a sequence of one or more characters that represents the collating element consisting of the character sequence designated by the iterator range `[first,last)`. Returns an empty string if the character sequence is not a valid collating element.

```
template <class ForwardIterator>
  char_class_type lookup_classname(
    ForwardIterator first, ForwardIterator last, bool icase = false) const;
```

9 *Returns:* an unspecified value that represents the character classification named by the character sequence designated by the iterator range `[first,last)`. If the parameter `icase` is true then the returned mask identifies the character classification without regard to the case of the characters being matched, otherwise it does honor the case of the characters being matched.[337] The value returned shall be independent of the case of the characters in the character sequence. If the name is not recognized then returns `char_class_type()`.

10 *Remarks:* For `regex_traits<char>`, at least the narrow character names in Table 140 shall be recognized. For `regex_traits<wchar_t>`, at least the wide character names in Table 140 shall be recognized.

```
bool isctype(charT c, char_class_type f) const;
```

11 *Effects:* Determines if the character `c` is a member of the character classification represented by `f`.

12 *Returns:* Given the following function declaration:

```
// for exposition only
template<class C>
  ctype_base::mask convert(typename regex_traits<C>::char_class_type f);
```

that returns a value in which each `ctype_base::mask` value corresponding to a value in `f` named in Table 140 is set, then the result is determined as if by:

```
ctype_base::mask m = convert<charT>(f);
const ctype<charT>& ct = use_facet<ctype<charT>>(getloc());
if (ct.is(m, c)) {
  return true;
} else if (c == ct.widen('_')) {
  charT w[1] = { ct.widen('w') };
  char_class_type x = lookup_classname(w, w+1);

  return (f&x) == x;
} else {
  return false;
}
```

[*Example:*

```
regex_traits<char> t;
string d("d");
string u("upper");
regex_traits<char>::char_class_type f;
f = t.lookup_classname(d.begin(), d.end());
f |= t.lookup_classname(u.begin(), u.end());
ctype_base::mask m = convert<char>(f); // m == ctype_base::digit|ctype_base::upper
```

— *end example*] [*Example:*

337) For example, if the parameter `icase` is true then `[[:lower:]]` is the same as `[[:alpha:]]`.

```
regex_traits<char> t;
string w("w");
regex_traits<char>::char_class_type f;
f = t.lookup_classname(w.begin(), w.end());
t.isctype('A', f); // returns true
t.isctype('_', f); // returns true
t.isctype(' ', f); // returns false
```

— *end example*]

```
int value(charT ch, int radix) const;
```

13 *Requires:* The value of *radix* shall be 8, 10, or 16.

14 *Returns:* the value represented by the digit *ch* in base *radix* if the character *ch* is a valid digit in base *radix*; otherwise returns -1.

```
locale_type imbue(locale_type loc);
```

15 *Effects:* Imbues `this` with a copy of the locale `loc`. [*Note:* Calling `imbue` with a different locale than the one currently in use invalidates all cached data held by `*this`. — *end note*]

16 *Returns:* if no locale has been previously imbued then a copy of the global locale in effect at the time of construction of `*this`, otherwise a copy of the last argument passed to `imbue`.

17 *Postcondition:* `getloc() == loc`.

```
locale_type getloc() const;
```

18 *Returns:* if no locale has been imbued then a copy of the global locale in effect at the time of construction of `*this`, otherwise a copy of the last argument passed to `imbue`.

Table 140 — Character class names and corresponding `ctype` masks

Narrow character name	Wide character name	Corresponding `ctype_base::mask` value
`"alnum"`	`L"alnum"`	`ctype_base::alnum`
`"alpha"`	`L"alpha"`	`ctype_base::alpha`
`"blank"`	`L"blank"`	`ctype_base::blank`
`"cntrl"`	`L"cntrl"`	`ctype_base::cntrl`
`"digit"`	`L"digit"`	`ctype_base::digit`
`"d"`	`L"d"`	`ctype_base::digit`
`"graph"`	`L"graph"`	`ctype_base::graph`
`"lower"`	`L"lower"`	`ctype_base::lower`
`"print"`	`L"print"`	`ctype_base::print`
`"punct"`	`L"punct"`	`ctype_base::punct`
`"space"`	`L"space"`	`ctype_base::space`
`"s"`	`L"s"`	`ctype_base::space`
`"upper"`	`L"upper"`	`ctype_base::upper`
`"w"`	`L"w"`	`ctype_base::alnum`
`"xdigit"`	`L"xdigit"`	`ctype_base::xdigit`

28.8 Class template `basic_regex` [re.regex]

1 For a char-like type `charT`, specializations of class template `basic_regex` represent regular expressions constructed from character sequences of `charT` characters. In the rest of 28.8, `charT` denotes a given char-like type. Storage for a regular expression is allocated and freed as necessary by the member functions of class `basic_regex`.

2 Objects of type specialization of `basic_regex` are responsible for converting the sequence of `charT` objects to an internal representation. It is not specified what form this representation takes, nor how it is accessed by algorithms that operate on regular expressions. [*Note:* Implementations will typically declare some function templates as friends of `basic_regex` to achieve this — *end note*]

3 The functions described in this Clause report errors by throwing exceptions of type `regex_error`.

```
namespace std {
  template <class charT,
            class traits = regex_traits<charT> >
  class basic_regex {
  public:
    // types:
    typedef          charT                              value_type;
    typedef          traits                             traits_type;
    typedef typename traits::string_type                string_type;
    typedef          regex_constants::syntax_option_type flag_type;
    typedef typename traits::locale_type                locale_type;

    // 28.8.1, constants:
    static constexpr regex_constants::syntax_option_type
      icase = regex_constants::icase;
    static constexpr regex_constants::syntax_option_type
      nosubs = regex_constants::nosubs;
    static constexpr regex_constants::syntax_option_type
      optimize = regex_constants::optimize;
    static constexpr regex_constants::syntax_option_type
      collate = regex_constants::collate;
    static constexpr regex_constants::syntax_option_type
      ECMAScript = regex_constants::ECMAScript;
    static constexpr regex_constants::syntax_option_type
      basic = regex_constants::basic;
    static constexpr regex_constants::syntax_option_type
      extended = regex_constants::extended;
    static constexpr regex_constants::syntax_option_type
      awk = regex_constants::awk;
    static constexpr regex_constants::syntax_option_type
      grep = regex_constants::grep;
    static constexpr regex_constants::syntax_option_type
      egrep = regex_constants::egrep;

    // 28.8.2, construct/copy/destroy:
    basic_regex();
    explicit basic_regex(const charT* p,
      flag_type f = regex_constants::ECMAScript);
    basic_regex(const charT* p, size_t len, flag_type f = regex_constants::ECMAScript);
    basic_regex(const basic_regex&);
    basic_regex(basic_regex&&) noexcept;
    template <class ST, class SA>
      explicit basic_regex(const basic_string<charT, ST, SA>& p,
```

```
                        flag_type f = regex_constants::ECMAScript);
    template <class ForwardIterator>
      basic_regex(ForwardIterator first, ForwardIterator last,
                  flag_type f = regex_constants::ECMAScript);
    basic_regex(initializer_list<charT>,
      flag_type = regex_constants::ECMAScript);

    ~basic_regex();

    basic_regex& operator=(const basic_regex&);
    basic_regex& operator=(basic_regex&&) noexcept;
    basic_regex& operator=(const charT* ptr);
    basic_regex& operator=(initializer_list<charT> il);
    template <class ST, class SA>
      basic_regex& operator=(const basic_string<charT, ST, SA>& p);

    // 28.8.3, assign:
    basic_regex& assign(const basic_regex& that);
    basic_regex& assign(basic_regex&& that) noexcept;
    basic_regex& assign(const charT* ptr,
      flag_type f = regex_constants::ECMAScript);
    basic_regex& assign(const charT* p, size_t len, flag_type f);
    template <class string_traits, class A>
      basic_regex& assign(const basic_string<charT, string_traits, A>& s,
                          flag_type f = regex_constants::ECMAScript);
    template <class InputIterator>
      basic_regex& assign(InputIterator first, InputIterator last,
                          flag_type f = regex_constants::ECMAScript);
    basic_regex& assign(initializer_list<charT>,
                        flag_type = regex_constants::ECMAScript);

    // 28.8.4, const operations:
    unsigned mark_count() const;
    flag_type flags() const;

    // 28.8.5, locale:
    locale_type imbue(locale_type loc);
    locale_type getloc() const;

    // 28.8.6, swap:
    void swap(basic_regex&);
  };
}
```

28.8.1 basic_regex constants [re.regex.const]

```
static constexpr regex_constants::syntax_option_type
  icase = regex_constants::icase;
static constexpr regex_constants::syntax_option_type
  nosubs = regex_constants::nosubs;
static constexpr regex_constants::syntax_option_type
  optimize = regex_constants::optimize;
static constexpr regex_constants::syntax_option_type
  collate = regex_constants::collate;
static constexpr regex_constants::syntax_option_type
```

```
    ECMAScript = regex_constants::ECMAScript;
  static constexpr regex_constants::syntax_option_type
    basic = regex_constants::basic;
  static constexpr regex_constants::syntax_option_type
    extended = regex_constants::extended;
  static constexpr regex_constants::syntax_option_type
    awk = regex_constants::awk;
  static constexpr regex_constants::syntax_option_type
    grep = regex_constants::grep;
  static constexpr regex_constants::syntax_option_type
    egrep = regex_constants::egrep;
```

1 The static constant members are provided as synonyms for the constants declared in namespace `regex_constants`.

28.8.2 `basic_regex` constructors [re.regex.construct]

```
basic_regex();
```

1 *Effects:* Constructs an object of class `basic_regex` that does not match any character sequence.

```
explicit basic_regex(const charT* p, flag_type f = regex_constants::ECMAScript);
```

2 *Requires:* p shall not be a null pointer.

3 *Throws:* `regex_error` if p is not a valid regular expression.

4 *Effects:* Constructs an object of class `basic_regex`; the object's internal finite state machine is constructed from the regular expression contained in the array of `charT` of length `char_traits<charT>::length(p)` whose first element is designated by p, and interpreted according to the flags f.

5 *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

```
basic_regex(const charT* p, size_t len, flag_type f);
```

6 *Requires:* p shall not be a null pointer.

7 *Throws:* `regex_error` if p is not a valid regular expression.

8 *Effects:* Constructs an object of class `basic_regex`; the object's internal finite state machine is constructed from the regular expression contained in the sequence of characters `[p,p+len)`, and interpreted according the flags specified in f.

9 *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

```
basic_regex(const basic_regex& e);
```

10 *Effects:* Constructs an object of class `basic_regex` as a copy of the object `e`.

11 *Postconditions:* `flags()` and `mark_count()` return `e.flags()` and `e.mark_count()`, respectively.

```
basic_regex(basic_regex&& e) noexcept;
```

12 *Effects:* Move constructs an object of class `basic_regex` from `e`.

13 *Postconditions:* `flags()` and `mark_count()` return the values that `e.flags()` and `e.mark_count()`, respectively, had before construction. `e` is in a valid state with unspecified value.

```
template <class ST, class SA>
  explicit basic_regex(const basic_string<charT, ST, SA>& s,
                       flag_type f = regex_constants::ECMAScript);
```

14 *Throws:* `regex_error` if `s` is not a valid regular expression.

15 *Effects:* Constructs an object of class `basic_regex`; the object's internal finite state machine is constructed from the regular expression contained in the string `s`, and interpreted according to the flags specified in `f`.

16 *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

```
template <class ForwardIterator>
  basic_regex(ForwardIterator first, ForwardIterator last,
              flag_type f = regex_constants::ECMAScript);
```

17 *Throws:* `regex_error` if the sequence `[first,last)` is not a valid regular expression.

18 *Effects:* Constructs an object of class `basic_regex`; the object's internal finite state machine is constructed from the regular expression contained in the sequence of characters `[first,last)`, and interpreted according to the flags specified in `f`.

19 *Postconditions:* `flags()` returns `f`. `mark_count()` returns the number of marked sub-expressions within the expression.

```
basic_regex(initializer_list<charT> il,
            flag_type f = regex_constants::ECMAScript);
```

20 *Effects:* Same as `basic_regex(il.begin(), il.end(), f)`.

28.8.3 `basic_regex` assign [re.regex.assign]

```
basic_regex& operator=(const basic_regex& e);
```

1 *Effects:* returns `assign(e)`.

```
basic_regex& operator=(basic_regex&& e) noexcept;
```

2 *Effects:* returns `assign(std::move(e))`.

```
basic_regex& operator=(const charT* ptr);
```

3 *Requires:* `ptr` shall not be a null pointer.

4 *Effects:* returns `assign(ptr)`.

```
basic_regex& operator=(initializer_list<charT> il);
```

5 *Effects:* returns `assign(il.begin(), il.end())`.

```
template <class ST, class SA>
  basic_regex& operator=(const basic_string<charT, ST, SA>& p);
```

6 *Effects:* returns `assign(p)`.

```
basic_regex& assign(const basic_regex& that);
```

7 *Effects:* copies `that` into `*this` and returns `*this`.

8 *Postconditions:* `flags()` and `mark_count()` return `that.flags()` and `that.mark_count()`, respectively.

```
basic_regex& assign(basic_regex&& that) noexcept;
```

9 *Effects:* move assigns from `that` into `*this` and returns `*this`.

10 *Postconditions:* `flags()` and `mark_count()` return the values that `that.flags()` and `that.mark_count()`, respectively, had before assignment. `that` is in a valid state with unspecified value.

```
basic_regex& assign(const charT* ptr, flag_type f = regex_constants::ECMAScript);
```

11 *Returns:* `assign(string_type(ptr), f)`.

```
basic_regex& assign(const charT* ptr, size_t len,
  flag_type f = regex_constants::ECMAScript);
```

12 *Returns:* `assign(string_type(ptr, len), f)`.

```
template <class string_traits, class A>
  basic_regex& assign(const basic_string<charT, string_traits, A>& s,
                      flag_type f = regex_constants::ECMAScript);
```

13 *Throws:* `regex_error` if `s` is not a valid regular expression.

14 *Returns:* `*this`.

15 *Effects:* Assigns the regular expression contained in the string `s`, interpreted according the flags specified in `f`. If an exception is thrown, `*this` is unchanged.

16 *Postconditions:* If no exception is thrown, `flags()` returns `f` and `mark_count()` returns the number of marked sub-expressions within the expression.

```
template <class InputIterator>
  basic_regex& assign(InputIterator first, InputIterator last,
                      flag_type f = regex_constants::ECMAScript);
```

17 *Requires:* The type `InputIterator` shall satisfy the requirements for an Input Iterator (24.2.3).

18 *Returns:* `assign(string_type(first, last), f)`.

```
basic_regex& assign(initializer_list<charT> il,
                    flag_type f = regex_constants::ECMAScript);
```

19 *Effects:* Same as `assign(il.begin(), il.end(), f)`.

20 *Returns:* `*this`.

28.8.4 `basic_regex` constant operations [re.regex.operations]

```
unsigned mark_count() const;
```

1 *Effects:* Returns the number of marked sub-expressions within the regular expression.

```
flag_type flags() const;
```

2 *Effects:* Returns a copy of the regular expression syntax flags that were passed to the object's constructor or to the last call to `assign`.

28.8.5 `basic_regex` locale [re.regex.locale]

```
locale_type imbue(locale_type loc);
```

1 *Effects:* Returns the result of `traits_inst.imbue(loc)` where `traits_inst` is a (default initialized) instance of the template type argument `traits` stored within the object. After a call to `imbue` the `basic_regex` object does not match any character sequence.

```
locale_type getloc() const;
```

2 *Effects:* Returns the result of `traits_inst.getloc()` where `traits_inst` is a (default initialized) instance of the template parameter `traits` stored within the object.

28.8.6 `basic_regex` swap [re.regex.swap]

```
void swap(basic_regex& e);
```

1 *Effects:* Swaps the contents of the two regular expressions.

2 *Postcondition:* `*this` contains the regular expression that was in `e`, `e` contains the regular expression that was in `*this`.

3 *Complexity:* Constant time.

28.8.7 `basic_regex` non-member functions [re.regex.nonmemb]

28.8.7.1 `basic_regex` non-member swap [re.regex.nmswap]

```
template <class charT, class traits>
  void swap(basic_regex<charT, traits>& lhs, basic_regex<charT, traits>& rhs);
```

1 *Effects:* Calls `lhs.swap(rhs)`.

28.9 Class template `sub_match` [re.submatch]

1 Class template `sub_match` denotes the sequence of characters matched by a particular marked sub-expression.

```
namespace std {
  template <class BidirectionalIterator>
  class sub_match : public std::pair<BidirectionalIterator, BidirectionalIterator> {
  public:
     typedef typename iterator_traits<BidirectionalIterator>::
       value_type                                          value_type;
     typedef typename iterator_traits<BidirectionalIterator>::
       difference_type                                     difference_type;
     typedef BidirectionalIterator                         iterator;
     typedef basic_string<value_type>                      string_type;

     bool matched;

     constexpr sub_match();

     difference_type length() const;
     operator string_type() const;
     string_type str() const;

     int compare(const sub_match& s) const;
     int compare(const string_type& s) const;
     int compare(const value_type* s) const;
  };
}
```

28.9.1 `sub_match` members [re.submatch.members]

```
constexpr sub_match();
```

1 *Effects:* Value-initializes the `pair` base class subobject and the member `matched`.

```
difference_type length() const;
```

2 *Returns:* `(matched ?  distance(first, second) :  0)`.

```
operator string_type() const;
```

3 *Returns:* `matched ? string_type(first, second) : string_type()`.

```
string_type str() const;
```

4 *Returns:* `matched ? string_type(first, second) : string_type()`.

```
int compare(const sub_match& s) const;
```

5 *Returns:* `str().compare(s.str())`.

```
int compare(const string_type& s) const;
```

6 *Returns:* `str().compare(s)`.

```
int compare(const value_type* s) const;
```

7 *Returns:* `str().compare(s)`.

28.9.2 `sub_match` non-member operators [re.submatch.op]

```
template <class BiIter>
  bool operator==(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

1 *Returns:* `lhs.compare(rhs) == 0`.

```
template <class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

2 *Returns:* `lhs.compare(rhs) != 0`.

```
template <class BiIter>
  bool operator<(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

3 *Returns:* `lhs.compare(rhs) < 0`.

```
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

4 *Returns:* `lhs.compare(rhs) <= 0`.

```
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

5 *Returns:* `lhs.compare(rhs) >= 0`.

```
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs, const sub_match<BiIter>& rhs);
```

6 *Returns:* `lhs.compare(rhs) > 0`.

```
template <class BiIter, class ST, class SA>
  bool operator==(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

7 *Returns:* `rhs.compare(typename sub_match<BiIter>::string_type(lhs.data(), lhs.size())) == 0`.

```
template <class BiIter, class ST, class SA>
  bool operator!=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

8 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter, class ST, class SA>
  bool operator<(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

9 *Returns:* `rhs.compare(typename sub_match<BiIter>::string_type(lhs.data(), lhs.size())) > 0`.

```
template <class BiIter, class ST, class SA>
  bool operator>(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

10 *Returns:* `rhs < lhs`.

```
template <class BiIter, class ST, class SA>
  bool operator>=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

11 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter, class ST, class SA>
  bool operator<=(
    const basic_string<
      typename iterator_traits<BiIter>::value_type, ST, SA>& lhs,
    const sub_match<BiIter>& rhs);
```

12 *Returns:* `!(rhs < lhs)`.

```
template <class BiIter, class ST, class SA>
  bool operator==(const sub_match<BiIter>& lhs,
                  const basic_string<
                    typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

13 *Returns:* `lhs.compare(typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())) == 0`.

```
template <class BiIter, class ST, class SA>
  bool operator!=(const sub_match<BiIter>& lhs,
                  const basic_string<
                    typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

14 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter, class ST, class SA>
  bool operator<(const sub_match<BiIter>& lhs,
                 const basic_string<
                   typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

15 *Returns:* `lhs.compare(typename sub_match<BiIter>::string_type(rhs.data(), rhs.size())) < 0`.

```
template <class BiIter, class ST, class SA>
  bool operator>(const sub_match<BiIter>& lhs,
                 const basic_string<
                   typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

16 *Returns:* `rhs < lhs`.

```
template <class BiIter, class ST, class SA>
  bool operator>=(const sub_match<BiIter>& lhs,
                  const basic_string<
                    typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

17 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter, class ST, class SA>
  bool operator<=(const sub_match<BiIter>& lhs,
                  const basic_string<
                    typename iterator_traits<BiIter>::value_type, ST, SA>& rhs);
```

18 *Returns:* `!(rhs < lhs)`.

```
template <class BiIter>
  bool operator==(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
```

19 *Returns:* `rhs.compare(lhs) == 0`.

```
template <class BiIter>
  bool operator!=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
```

20 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter>
  bool operator<(typename iterator_traits<BiIter>::value_type const* lhs,
                 const sub_match<BiIter>& rhs);
```

21 *Returns:* `rhs.compare(lhs) > 0`.

```
template <class BiIter>
  bool operator>(typename iterator_traits<BiIter>::value_type const* lhs,
                 const sub_match<BiIter>& rhs);
```

22 *Returns:* `rhs < lhs`.

```
template <class BiIter>
  bool operator>=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
```

23 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter>
  bool operator<=(typename iterator_traits<BiIter>::value_type const* lhs,
                  const sub_match<BiIter>& rhs);
```

24 *Returns:* `!(rhs < lhs)`.

```
template <class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
```

25 *Returns:* `lhs.compare(rhs) == 0`.

```
template <class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
```

26 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const* rhs);
```

27 *Returns:* `lhs.compare(rhs) < 0`.

```
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const* rhs);
```

28 *Returns:* `rhs < lhs`.

```
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
```

29 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const* rhs);
```

30 *Returns:* `!(rhs < lhs)`.

```
template <class BiIter>
  bool operator==(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
```

31 *Returns:* `rhs.compare(typename sub_match<BiIter>::string_type(1, lhs)) == 0`.

```
template <class BiIter>
  bool operator!=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
```

32 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter>
  bool operator<(typename iterator_traits<BiIter>::value_type const& lhs,
                 const sub_match<BiIter>& rhs);
```

33 *Returns:* `rhs.compare(typename sub_match<BiIter>::string_type(1, lhs)) > 0`.

```
template <class BiIter>
  bool operator>(typename iterator_traits<BiIter>::value_type const& lhs,
                 const sub_match<BiIter>& rhs);
```

34 *Returns:* `rhs < lhs`.

```
template <class BiIter>
  bool operator>=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
```

35 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter>
  bool operator<=(typename iterator_traits<BiIter>::value_type const& lhs,
                  const sub_match<BiIter>& rhs);
```

36 *Returns:* `!(rhs < lhs)`.

```
template <class BiIter>
  bool operator==(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
```

37 *Returns:* `lhs.compare(typename sub_match<BiIter>::string_type(1, rhs)) == 0`.

```
template <class BiIter>
  bool operator!=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
```

38 *Returns:* `!(lhs == rhs)`.

```
template <class BiIter>
  bool operator<(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const& rhs);
```

39 *Returns:* `lhs.compare(typename sub_match<BiIter>::string_type(1, rhs)) < 0`.

```
template <class BiIter>
  bool operator>(const sub_match<BiIter>& lhs,
                 typename iterator_traits<BiIter>::value_type const& rhs);
```

40 *Returns:* `rhs < lhs`.

```
template <class BiIter>
  bool operator>=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
```

41 *Returns:* `!(lhs < rhs)`.

```
template <class BiIter>
  bool operator<=(const sub_match<BiIter>& lhs,
                  typename iterator_traits<BiIter>::value_type const& rhs);
```

42 *Returns:* `!(rhs < lhs)`.

```
template <class charT, class ST, class BiIter>
  basic_ostream<charT, ST>&
  operator<<(basic_ostream<charT, ST>& os, const sub_match<BiIter>& m);
```

43 *Returns:* `(os << m.str())`.

28.10 Class template `match_results` [re.results]

1 Class template `match_results` denotes a collection of character sequences representing the result of a regular expression match. Storage for the collection is allocated and freed as necessary by the member functions of class template `match_results`.

2 The class template `match_results` shall satisfy the requirements of an allocator-aware container and of a sequence container, as specified in 23.2.3, except that only operations defined for const-qualified sequence containers are supported.

3 A default-constructed `match_results` object has no fully established result state. A match result is *ready* when, as a consequence of a completed regular expression match modifying such an object, its result state becomes fully established. The effects of calling most member functions from a `match_results` object that is not ready are undefined.

4 The `sub_match` object stored at index 0 represents sub-expression 0, i.e., the whole match. In this case the `sub_match` member `matched` is always true. The `sub_match` object stored at index `n` denotes what matched the marked sub-expression `n` within the matched expression. If the sub-expression `n` participated in a regular expression match then the `sub_match` member `matched` evaluates to true, and members `first` and `second` denote the range of characters `[first,second)` which formed that match. Otherwise `matched` is false, and members `first` and `second` point to the end of the sequence that was searched. [*Note:* The `sub_match` objects representing different sub-expressions that did not participate in a regular expression match need not be distinct. — *end note*]

```
namespace std {
  template <class BidirectionalIterator,
            class Allocator = allocator<sub_match<BidirectionalIterator>>>
  class match_results {
  public:
     typedef sub_match<BidirectionalIterator>                      value_type;
     typedef const value_type&                                     const_reference;
     typedef value_type&                                           reference;
     typedef implementation-defined                                 const_iterator;
     typedef const_iterator                                        iterator;
     typedef typename
      iterator_traits<BidirectionalIterator>::difference_type      difference_type;
     typedef typename allocator_traits<Allocator>::size_type       size_type;
     typedef Allocator                                             allocator_type;
     typedef typename iterator_traits<BidirectionalIterator>::
       value_type                                                  char_type;
     typedef basic_string<char_type>                               string_type;

     // 28.10.1, construct/copy/destroy:
     explicit match_results(const Allocator& a = Allocator());
     match_results(const match_results& m);
     match_results(match_results&& m) noexcept;
     match_results& operator=(const match_results& m);
     match_results& operator=(match_results&& m);
     ~match_results();

     // 28.10.2, state:
     bool ready() const;

     // 28.10.3, size:
     size_type size() const;
     size_type max_size() const;
     bool empty() const;
```

```
    // 28.10.4, element access:
    difference_type length(size_type sub = 0) const;
    difference_type position(size_type sub = 0) const;
    string_type str(size_type sub = 0) const;
    const_reference operator[](size_type n) const;

    const_reference prefix() const;
    const_reference suffix() const;
    const_iterator begin() const;
    const_iterator end() const;
    const_iterator cbegin() const;
    const_iterator cend() const;

    // 28.10.5, format:
    template <class OutputIter>
     OutputIter
     format(OutputIter out,
            const char_type* fmt_first, const char_type* fmt_last,
            regex_constants::match_flag_type flags =
             regex_constants::format_default) const;
    template <class OutputIter, class ST, class SA>
      OutputIter
      format(OutputIter out,
             const basic_string<char_type, ST, SA>& fmt,
             regex_constants::match_flag_type flags =
               regex_constants::format_default) const;
    template <class ST, class SA>
     basic_string<char_type, ST, SA>
     format(const basic_string<char_type, ST, SA>& fmt,
            regex_constants::match_flag_type flags =
              regex_constants::format_default) const;
    string_type
    format(const char_type* fmt,
           regex_constants::match_flag_type flags =
             regex_constants::format_default) const;

    // 28.10.6, allocator:
    allocator_type get_allocator() const;

    // 28.10.7, swap:
    void swap(match_results& that);
  };
}
```

28.10.1 `match_results` constructors [re.results.const]

1 In all `match_results` constructors, a copy of the `Allocator` argument shall be used for any memory allocation performed by the constructor or member functions during the lifetime of the object.

```
match_results(const Allocator& a = Allocator());
```

2 *Effects:* Constructs an object of class `match_results`.

3 *Postconditions:* `ready()` returns `false`. `size()` returns `0`.

```
match_results(const match_results& m);
```

4 *Effects:* Constructs an object of class `match_results`, as a copy of `m`.

```
match_results(match_results&& m) noexcept;
```

5 *Effects:* Move-constructs an object of class `match_results` from `m` satisfying the same postconditions as Table 141. Additionally, the stored `Allocator` value is move constructed from `m.get_allocator()`.

6 *Throws:* Nothing.

```
match_results& operator=(const match_results& m);
```

7 *Effects:* Assigns `m` to `*this`. The postconditions of this function are indicated in Table 141.

```
match_results& operator=(match_results&& m);
```

8 *Effects:* Move-assigns `m` to `*this`. The postconditions of this function are indicated in Table 141.

Table 141 — `match_results` assignment operator effects

Element	Value
`ready()`	`m.ready()`
`size()`	`m.size()`
`str(n)`	`m.str(n)` for all integers `n < m.size()`
`prefix()`	`m.prefix()`
`suffix()`	`m.suffix()`
`(*this)[n]`	`m[n]` for all integers `n < m.size()`
`length(n)`	`m.length(n)` for all integers `n < m.size()`
`position(n)`	`m.position(n)` for all integers `n < m.size()`

28.10.2 `match_results` state [re.results.state]

```
bool ready() const;
```

1 *Returns:* `true` if `*this` has a fully established result state, otherwise `false`.

28.10.3 `match_results` size [re.results.size]

```
size_type size() const;
```

1 *Returns:* One plus the number of marked sub-expressions in the regular expression that was matched if `*this` represents the result of a successful match. Otherwise returns `0`. [*Note:* The state of a `match_results` object can be modified only by passing that object to `regex_match` or `regex_search`. Sections 28.11.2 and 28.11.3 specify the effects of those algorithms on their `match_results` arguments. — *end note*]

```
size_type max_size() const;
```

2 *Returns:* The maximum number of `sub_match` elements that can be stored in `*this`.

```
bool empty() const;
```

3 *Returns:* `size() == 0`.

28.10.4 `match_results` element access [re.results.acc]

```
difference_type length(size_type sub = 0) const;
```

1 *Requires:* `ready() == true`.

2 *Returns:* `(*this)[sub].length()`.

```
difference_type position(size_type sub = 0) const;
```

3 *Requires:* `ready() == true`.

4 *Returns:* The distance from the start of the target sequence to `(*this)[sub].first`.

```
string_type str(size_type sub = 0) const;
```

5 *Requires:* `ready() == true`.

6 *Returns:* `string_type((*this)[sub])`.

```
const_reference operator[](size_type n) const;
```

7 *Requires:* `ready() == true`.

8 *Returns:* A reference to the `sub_match` object representing the character sequence that matched marked sub-expression `n`. If `n == 0` then returns a reference to a `sub_match` object representing the character sequence that matched the whole regular expression. If `n >= size()` then returns a `sub_match` object representing an unmatched sub-expression.

```
const_reference prefix() const;
```

9 *Requires:* `ready() == true`.

10 *Returns:* A reference to the `sub_match` object representing the character sequence from the start of the string being matched/searched to the start of the match found.

```
const_reference suffix() const;
```

11 *Requires:* `ready() == true`.

12 *Returns:* A reference to the `sub_match` object representing the character sequence from the end of the match found to the end of the string being matched/searched.

```
const_iterator begin() const;
const_iterator cbegin() const;
```

13 *Returns:* A starting iterator that enumerates over all the sub-expressions stored in `*this`.

```
const_iterator end() const;
const_iterator cend() const;
```

14 *Returns:* A terminating iterator that enumerates over all the sub-expressions stored in `*this`.

28.10.5 `match_results` formatting [re.results.form]

```
template <class OutputIter>
  OutputIter format(OutputIter out,
                    const char_type* fmt_first, const char_type* fmt_last,
                    regex_constants::match_flag_type flags =
                      regex_constants::format_default) const;
```

1 *Requires:* ready() == true and `OutputIter` shall satisfy the requirements for an Output Iterator (24.2.4).

2 *Effects:* Copies the character sequence `[fmt_first,fmt_last)` to OutputIter `out`. Replaces each format specifier or escape sequence in the copied range with either the character(s) it represents or the sequence of characters within `*this` to which it refers. The bitmasks specified in `flags` determine which format specifiers and escape sequences are recognized.

3 *Returns:* `out`.

```
template <class OutputIter, class ST, class SA>
  OutputIter format(OutputIter out,
                    const basic_string<char_type, ST, SA>& fmt,
                    regex_constants::match_flag_type flags =
                      regex_constants::format_default) const;
```

4 *Effects:* Equivalent to `return format(out, fmt.data(), fmt.data() + fmt.size(), flags)`.

```
template <class ST, class SA>
  basic_string<char_type, ST, SA>
  format(const basic_string<char_type, ST, SA>& fmt,
         regex_constants::match_flag_type flags =
           regex_constants::format_default) const;
```

5 *Requires:* `ready() == true`.

6 *Effects:* Constructs an empty string `result` of type `basic_string<char_type, ST, SA>` and calls `format(back_inserter(result), fmt, flags)`.

7 *Returns:* `result`.

```
string_type
  format(const char_type* fmt,
         regex_constants::match_flag_type flags =
           regex_constants::format_default) const;
```

8 *Requires:* `ready() == true`.

9 *Effects:* Constructs an empty string `result` of type `string_type` and calls `format(back_inserter(result), fmt, fmt + char_traits<char_type>::length(fmt), flags)`.

10 *Returns:* `result`.

28.10.6 `match_results` allocator [re.results.all]

```
allocator_type get_allocator() const;
```

1 *Returns:* A copy of the Allocator that was passed to the object's constructor or, if that allocator has been replaced, a copy of the most recent replacement.

28.10.7 `match_results` swap [re.results.swap]

```
void swap(match_results& that);
```

1 *Effects:* Swaps the contents of the two sequences.

2 *Postcondition:* `*this` contains the sequence of matched sub-expressions that were in `that`, `that` contains the sequence of matched sub-expressions that were in `*this`.

3 *Complexity:* Constant time.

```
template <class BidirectionalIterator, class Allocator>
  void swap(match_results<BidirectionalIterator, Allocator>& m1,
            match_results<BidirectionalIterator, Allocator>& m2);
```

4 *Effects:* `m1.swap(m2)`.

28.10.8 `match_results` non-member functions [re.results.nonmember]

```
template <class BidirectionalIterator, class Allocator>
bool operator==(const match_results<BidirectionalIterator, Allocator>& m1,
                const match_results<BidirectionalIterator, Allocator>& m2);
```

1 *Returns:* `true` if neither match result is ready, `false` if one match result is ready and the other is not. If both match results are ready, returns `true` only if:

(1.1) — `m1.empty() && m2.empty()`, or

(1.2) — `!m1.empty() && !m2.empty()`, and the following conditions are satisfied:

(1.2.1) — `m1.prefix() == m2.prefix()`,

(1.2.2) — `m1.size() == m2.size() && equal(m1.begin(), m1.end(), m2.begin())`, and

(1.2.3) — `m1.suffix() == m2.suffix()`.

[*Note:* The algorithm `equal` is defined in Clause 25. — *end note*]

```
template <class BidirectionalIterator, class Allocator>
bool operator!=(const match_results<BidirectionalIterator, Allocator>& m1,
                const match_results<BidirectionalIterator, Allocator>& m2);
```

2 *Returns:* `!(m1 == m2)`.

28.11 Regular expression algorithms [re.alg]

28.11.1 exceptions [re.except]

1 The algorithms described in this subclause may throw an exception of type `regex_error`. If such an exception `e` is thrown, `e.code()` shall return either `regex_constants::error_complexity` or `regex_constants::error_stack`.

28.11.2 `regex_match` [re.alg.match]

```
template <class BidirectionalIterator, class Allocator, class charT, class traits>
  bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                   match_results<BidirectionalIterator, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
```

1 *Requires:* The type `BidirectionalIterator` shall satisfy the requirements of a Bidirectional Iterator (24.2.6).

2 *Effects:* Determines whether there is a match between the regular expression `e`, and all of the character sequence `[first,last)`. The parameter `flags` is used to control how the expression is matched against the character sequence. Returns `true` if such a match exists, `false` otherwise.

3 *Postconditions:* `m.ready() == true` in all cases. If the function returns `false`, then the effect on parameter `m` is unspecified except that `m.size()` returns `0` and `m.empty()` returns `true`. Otherwise the effects on parameter `m` are given in Table 142.

Table 142 — Effects of `regex_match` algorithm

Element	Value
`m.size()`	`1 + e.mark_count()`
`m.empty()`	false
`m.prefix().first`	first

Table 142 — Effects of `regex_match` algorithm (continued)

Element	Value
`m.prefix().second`	first
`m.prefix().matched`	false
`m.suffix().first`	last
`m.suffix().second`	last
`m.suffix().matched`	false
`m[0].first`	first
`m[0].second`	last
`m[0].matched`	`true`
`m[n].first`	For all integers `0 < n < m.size()`, the start of the sequence that matched sub-expression `n`. Alternatively, if sub-expression `n` did not participate in the match, then `last`.
`m[n].second`	For all integers `0 < n < m.size()`, the end of the sequence that matched sub-expression `n`. Alternatively, if sub-expression `n` did not participate in the match, then `last`.
`m[n].matched`	For all integers `0 < n < m.size()`, `true` if sub-expression `n` participated in the match, `false` otherwise.

```
template <class BidirectionalIterator, class charT, class traits>
  bool regex_match(BidirectionalIterator first, BidirectionalIterator last,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
```

4 *Effects:* Behaves "as if" by constructing an instance of `match_results<BidirectionalIterator> what`, and then returning the result of `regex_match(first, last, what, e, flags)`.

```
template <class charT, class Allocator, class traits>
  bool regex_match(const charT* str,
                   match_results<const charT*, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
```

5 *Returns:* `regex_match(str, str + char_traits<charT>::length(str), m, e, flags)`.

```
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                   match_results<
                     typename basic_string<charT, ST, SA>::const_iterator,
                     Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
```

6 *Returns:* `regex_match(s.begin(), s.end(), m, e, flags)`.

```
template <class charT, class traits>
  bool regex_match(const charT* str,
```

```
                  const basic_regex<charT, traits>& e,
                  regex_constants::match_flag_type flags =
                    regex_constants::match_default);
```

7 *Returns:* `regex_match(str, str + char_traits<charT>::length(str), e, flags)`

```
template <class ST, class SA, class charT, class traits>
  bool regex_match(const basic_string<charT, ST, SA>& s,
                  const basic_regex<charT, traits>& e,
                  regex_constants::match_flag_type flags =
                    regex_constants::match_default);
```

8 *Returns:* `regex_match(s.begin(), s.end(), e, flags)`.

28.11.3 `regex_search` [re.alg.search]

```
template <class BidirectionalIterator, class Allocator, class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                   match_results<BidirectionalIterator, Allocator>& m,
                   const basic_regex<charT, traits>& e,
                   regex_constants::match_flag_type flags =
                     regex_constants::match_default);
```

1 *Requires:* Type `BidirectionalIterator` shall satisfy the requirements of a Bidirectional Iterator (24.2.6).

2 *Effects:* Determines whether there is some sub-sequence within `[first,last)` that matches the regular expression `e`. The parameter `flags` is used to control how the expression is matched against the character sequence. Returns `true` if such a sequence exists, `false` otherwise.

3 *Postconditions:* `m.ready() == true` in all cases. If the function returns `false`, then the effect on parameter `m` is unspecified except that `m.size()` returns `0` and `m.empty()` returns `true`. Otherwise the effects on parameter `m` are given in Table 143.

Table 143 — Effects of `regex_search` algorithm

Element	Value
`m.size()`	`1 + e.mark_count()`
`m.empty()`	`false`
`m.prefix().first`	`first`
`m.prefix().second`	`m[0].first`
`m.prefix().matched`	`m.prefix().first != m.prefix().second`
`m.suffix().first`	`m[0].second`
`m.suffix().second`	`last`
`m.suffix().matched`	`m.suffix().first != m.suffix().second`
`m[0].first`	The start of the sequence of characters that matched the regular expression
`m[0].second`	The end of the sequence of characters that matched the regular expression
`m[0].matched`	`true`
`m[n].first`	For all integers `0 < n < m.size()`, the start of the sequence that matched sub-expression `n`. Alternatively, if sub-expression `n` did not participate in the match, then `last`.

Table 143 — Effects of `regex_search` algorithm (continued)

Element	Value
`m[n].second`	For all integers `0 < n < m.size()`, the end of the sequence that matched sub-expression `n`. Alternatively, if sub-expression `n` did not participate in the match, then `last` .
`m[n].matched`	For all integers `0 < n < m.size()`, `true` if sub-expression `n` participated in the match, `false` otherwise.

```
template <class charT, class Allocator, class traits>
bool regex_search(const charT* str, match_results<const charT*, Allocator>& m,
                  const basic_regex<charT, traits>& e,
                  regex_constants::match_flag_type flags =
                    regex_constants::match_default);
```

4 *Returns:* The result of `regex_search(str, str + char_traits<charT>::length(str), m, e, flags)`.

```
template <class ST, class SA, class Allocator, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    match_results<
                      typename basic_string<charT, ST, SA>::const_iterator,
                      Allocator>& m,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
```

5 *Returns:* The result of `regex_search(s.begin(), s.end(), m, e, flags)`.

```
template <class BidirectionalIterator, class charT, class traits>
  bool regex_search(BidirectionalIterator first, BidirectionalIterator last,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
```

6 *Effects:* Behaves "as if" by constructing an object `what` of type `match_results<BidirectionalIterator>` and then returning the result of `regex_search(first, last, what, e, flags)`.

```
template <class charT, class traits>
  bool regex_search(const charT* str,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
```

7 *Returns:* `regex_search(str, str + char_traits<charT>::length(str), e, flags)`

```
template <class ST, class SA, class charT, class traits>
  bool regex_search(const basic_string<charT, ST, SA>& s,
                    const basic_regex<charT, traits>& e,
                    regex_constants::match_flag_type flags =
                      regex_constants::match_default);
```

8 *Returns:* `regex_search(s.begin(), s.end(), e, flags)`.

28.11.4 `regex_replace` [re.alg.replace]

```
template <class OutputIterator, class BidirectionalIterator,
    class traits, class charT, class ST, class SA>
  OutputIterator
  regex_replace(OutputIterator out,
                BidirectionalIterator first, BidirectionalIterator last,
                const basic_regex<charT, traits>& e,
                const basic_string<charT, ST, SA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class OutputIterator, class BidirectionalIterator,
    class traits, class charT>
  OutputIterator
  regex_replace(OutputIterator out,
                BidirectionalIterator first, BidirectionalIterator last,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
```

1 *Effects:* Constructs a `regex_iterator` object `i` as if by

```
regex_iterator<BidirectionalIterator, charT, traits> i(first, last, e, flags)
```

and uses `i` to enumerate through all of the matches `m` of type `match_results<BidirectionalIterator>` that occur within the sequence `[first,last)`. If no such matches are found and `!(flags & regex_constants::format_no_copy)` then calls

```
out = std::copy(first, last, out)
```

If any matches are found then, for each such match:

(1.1) — If `!(flags & regex_constants::format_no_copy)`, calls

```
out = std::copy(m.prefix().first, m.prefix().second, out)
```

(1.2) — Then calls

```
out = m.format(out, fmt, flags)
```

for the first form of the function and

```
out = m.format(out, fmt, fmt + char_traits<charT>::length(fmt), flags)
```

for the second.

Finally, if such a match is found and `!(flags & regex_constants::format_no_copy)`, calls

```
out = std::copy(last_m.suffix().first, last_m.suffix().second, out)
```

where `last_m` is a copy of the last match found. If `flags & regex_constants::format_first_only` is non-zero then only the first match found is replaced.

2 *Returns:* `out`.

```
template <class traits, class charT, class ST, class SA, class FST, class FSA>
  basic_string<charT, ST, SA>
  regex_replace(const basic_string<charT, ST, SA>& s,
                const basic_regex<charT, traits>& e,
```

```
                const basic_string<charT, FST, FSA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT, class ST, class SA>
  basic_string<charT, ST, SA>
  regex_replace(const basic_string<charT, ST, SA>& s,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
```

3 *Effects:* Constructs an empty string `result` of type `basic_string<charT, ST, SA>` and calls `regex_replace(back_inserter(result), s.begin(), s.end(), e, fmt, flags)`.

4 *Returns:* `result`.

```
template <class traits, class charT, class ST, class SA>
  basic_string<charT>
  regex_replace(const charT* s,
                const basic_regex<charT, traits>& e,
                const basic_string<charT, ST, SA>& fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
template <class traits, class charT>
  basic_string<charT>
  regex_replace(const charT* s,
                const basic_regex<charT, traits>& e,
                const charT* fmt,
                regex_constants::match_flag_type flags =
                  regex_constants::match_default);
```

5 *Effects:* Constructs an empty string `result` of type `basic_string<charT>` and calls `regex_replace(back_inserter(result), s, s + char_traits<charT>::length(s), e, fmt, flags)`.

6 *Returns:* `result`.

28.12 Regular expression iterators [re.iter]

28.12.1 Class template `regex_iterator` [re.regiter]

1 The class template `regex_iterator` is an iterator adaptor. It represents a new view of an existing iterator sequence, by enumerating all the occurrences of a regular expression within that sequence. A `regex_iterator` uses `regex_search` to find successive regular expression matches within the sequence from which it was constructed. After the iterator is constructed, and every time `operator++` is used, the iterator finds and stores a value of `match_results<BidirectionalIterator>`. If the end of the sequence is reached (`regex_search` returns `false`), the iterator becomes equal to the end-of-sequence iterator value. The default constructor constructs an end-of-sequence iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-sequence iterator is not defined. For any other iterator value a const `match_results<BidirectionalIterator>&` is returned. The result of `operator->` on an end-of-sequence iterator is not defined. For any other iterator value a `const match_results<BidirectionalIterator>*` is returned. It is impossible to store things into `regex_iterator`s. Two end-of-sequence iterators are always equal. An end-of-sequence iterator is not equal to a non-end-of-sequence iterator. Two non-end-of-sequence iterators are equal when they are constructed from the same arguments.

```
namespace std {
  template <class BidirectionalIterator,
```

```
            class charT = typename iterator_traits<
              BidirectionalIterator>::value_type,
              class traits = regex_traits<charT> >
    class regex_iterator {
    public:
       typedef basic_regex<charT, traits>           regex_type;
       typedef match_results<BidirectionalIterator> value_type;
       typedef std::ptrdiff_t                       difference_type;
       typedef const value_type*                    pointer;
       typedef const value_type&                    reference;
       typedef std::forward_iterator_tag            iterator_category;

       regex_iterator();
       regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
                      const regex_type& re,
                      regex_constants::match_flag_type m =
                        regex_constants::match_default);
       regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
                      const regex_type&& re,
                      regex_constants::match_flag_type m =
                        regex_constants::match_default) = delete;
       regex_iterator(const regex_iterator&);
       regex_iterator& operator=(const regex_iterator&);
       bool operator==(const regex_iterator&) const;
       bool operator!=(const regex_iterator&) const;
       const value_type& operator*() const;
       const value_type* operator->() const;
       regex_iterator& operator++();
       regex_iterator operator++(int);
    private:
       BidirectionalIterator                begin;  // exposition only
       BidirectionalIterator                end;    // exposition only
       const regex_type*                    pregex; // exposition only
       regex_constants::match_flag_type     flags;  // exposition only
       match_results<BidirectionalIterator> match;  // exposition only
    };
  }
```

2 An object of type `regex_iterator` that is not an end-of-sequence iterator holds a *zero-length match* if `match[0].matched == true` and `match[0].first == match[0].second`. [*Note:* For example, this can occur when the part of the regular expression that matched consists only of an assertion (such as `'^'`, `'$'`, `'\b'`, `'\B'`). — *end note*]

28.12.1.1 `regex_iterator` constructors [re.regiter.cnstr]

```
regex_iterator();
```

1 *Effects:* Constructs an end-of-sequence iterator.

```
regex_iterator(BidirectionalIterator a, BidirectionalIterator b,
               const regex_type& re,
               regex_constants::match_flag_type m = regex_constants::match_default);
```

2 *Effects:* Initializes `begin` and `end` to `a` and `b`, respectively, sets `pregex` to `&re`, sets `flags` to `m`, then calls `regex_search(begin, end, match, *pregex, flags)`. If this call returns `false` the constructor sets `*this` to the end-of-sequence iterator.

28.12.1.2 `regex_iterator` comparisons [re.regiter.comp]

```
bool operator==(const regex_iterator& right) const;
```

1 *Returns:* `true` if `*this` and `right` are both end-of-sequence iterators or if the following conditions all hold:

(1.1) — `begin == right.begin`,

(1.2) — `end == right.end`,

(1.3) — `pregex == right.pregex`,

(1.4) — `flags == right.flags`, and

(1.5) — `match[0] == right.match[0]`;

otherwise `false`.

```
bool operator!=(const regex_iterator& right) const;
```

2 *Returns:* `!(*this == right)`.

28.12.1.3 `regex_iterator` indirection [re.regiter.deref]

```
const value_type& operator*() const;
```

1 *Returns:* `match`.

```
const value_type* operator->() const;
```

2 *Returns:* `&match`.

28.12.1.4 `regex_iterator` increment [re.regiter.incr]

```
regex_iterator& operator++();
```

1 *Effects:* Constructs a local variable `start` of type `BidirectionalIterator` and initializes it with the value of `match[0].second`.

2 If the iterator holds a zero-length match and `start == end` the operator sets `*this` to the end-of-sequence iterator and returns `*this`.

3 Otherwise, if the iterator holds a zero-length match the operator calls `regex_search(start, end, match, *pregex, flags | regex_constants::match_not_null | regex_constants::match_continuous)`. If the call returns `true` the operator returns `*this`. Otherwise the operator increments `start` and continues as if the most recent match was not a zero-length match.

4 If the most recent match was not a zero-length match, the operator sets `flags` to `flags | regex_constants ::match_prev_avail` and calls `regex_search(start, end, match, *pregex, flags)`. If the call returns `false` the iterator sets `*this` to the end-of-sequence iterator. The iterator then returns `*this`.

5 In all cases in which the call to `regex_search` returns `true`, `match.prefix().first` shall be equal to the previous value of `match[0].second`, and for each index `i` in the half-open range `[0, match.size())` for which `match[i].matched` is true, `match.position(i)` shall return `distance(begin, match[i].first)`.

6 [*Note:* This means that `match.position(i)` gives the offset from the beginning of the target sequence, which is often not the same as the offset from the sequence passed in the call to `regex_search`. — *end note*]

7 It is unspecified how the implementation makes these adjustments.

8 [*Note:* This means that a compiler may call an implementation-specific search function, in which case a user-defined specialization of `regex_search` will not be called. — *end note*]

```
regex_iterator operator++(int);
```

9 *Effects:*

```
regex_iterator tmp = *this;
++(*this);
return tmp;
```

28.12.2 Class template `regex_token_iterator` [re.tokiter]

1 The class template `regex_token_iterator` is an iterator adaptor; that is to say it represents a new view of an existing iterator sequence, by enumerating all the occurrences of a regular expression within that sequence, and presenting one or more sub-expressions for each match found. Each position enumerated by the iterator is a `sub_match` class template instance that represents what matched a particular sub-expression within the regular expression.

2 When class `regex_token_iterator` is used to enumerate a single sub-expression with index -1 the iterator performs field splitting: that is to say it enumerates one sub-expression for each section of the character container sequence that does not match the regular expression specified.

3 After it is constructed, the iterator finds and stores a value `regex_iterator<BidirectionalIterator> position` and sets the internal count `N` to zero. It also maintains a sequence `subs` which contains a list of the sub-expressions which will be enumerated. Every time `operator++` is used the count `N` is incremented; if `N` exceeds or equals `subs.size()`, then the iterator increments member `position` and sets count `N` to zero.

4 If the end of sequence is reached (`position` is equal to the end of sequence iterator), the iterator becomes equal to the end-of-sequence iterator value, unless the sub-expression being enumerated has index -1, in which case the iterator enumerates one last sub-expression that contains all the characters from the end of the last regular expression match to the end of the input sequence being enumerated, provided that this would not be an empty sub-expression.

5 The default constructor constructs an end-of-sequence iterator object, which is the only legitimate iterator to be used for the end condition. The result of `operator*` on an end-of-sequence iterator is not defined. For any other iterator value a `const sub_match<BidirectionalIterator>&` is returned. The result of `operator->` on an end-of-sequence iterator is not defined. For any other iterator value a `const sub_match<BidirectionalIterator>*` is returned.

6 It is impossible to store things into `regex_token_iterator`s. Two end-of-sequence iterators are always equal. An end-of-sequence iterator is not equal to a non-end-of-sequence iterator. Two non-end-of-sequence iterators are equal when they are constructed from the same arguments.

```
namespace std {
  template <class BidirectionalIterator,
            class charT = typename iterator_traits<
              BidirectionalIterator>::value_type,
              class traits = regex_traits<charT> >
  class regex_token_iterator  {
  public:
    typedef basic_regex<charT, traits>       regex_type;
    typedef sub_match<BidirectionalIterator> value_type;
    typedef std::ptrdiff_t                   difference_type;
    typedef const value_type*                pointer;
    typedef const value_type&                reference;
    typedef std::forward_iterator_tag        iterator_category;

    regex_token_iterator();
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                         const regex_type& re,
```

```
                      int submatch = 0,
                      regex_constants::match_flag_type m =
                        regex_constants::match_default);
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                      const regex_type& re,
                      const std::vector<int>& submatches,
                      regex_constants::match_flag_type m =
                        regex_constants::match_default);
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                      const regex_type& re,
                      initializer_list<int> submatches,
                      regex_constants::match_flag_type m =
                        regex_constants::match_default);
    template <std::size_t N>
      regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                      const regex_type& re,
                      const int (&submatches)[N],
                      regex_constants::match_flag_type m =
                        regex_constants::match_default);
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                       const regex_type&& re,
                       int submatch = 0,
                       regex_constants::match_flag_type m =
                         regex_constants::match_default) = delete;
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                       const regex_type&& re,
                       const std::vector<int>& submatches,
                       regex_constants::match_flag_type m =
                         regex_constants::match_default) = delete;
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                       const regex_type&& re,
                       initializer_list<int> submatches,
                       regex_constants::match_flag_type m =
                         regex_constants::match_default) = delete;
    template <std::size_t N>
    regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                       const regex_type&& re,
                       const int (&submatches)[N],
                       regex_constants::match_flag_type m =
                         regex_constants::match_default) = delete;
    regex_token_iterator(const regex_token_iterator&);
    regex_token_iterator& operator=(const regex_token_iterator&);
    bool operator==(const regex_token_iterator&) const;
    bool operator!=(const regex_token_iterator&) const;
    const value_type& operator*() const;
    const value_type* operator->() const;
    regex_token_iterator& operator++();
    regex_token_iterator operator++(int);
  private:
    typedef
      regex_iterator<BidirectionalIterator, charT, traits> position_iterator; // exposition only
    position_iterator position;                                               // exposition only
    const value_type* result;                                                 // exposition only
    value_type suffix;                                                        // exposition only
    std::size_t N;                                                            // exposition only
```

```
    std::vector<int> subs;                                    // exposition only
  };
}
```

7 A *suffix iterator* is a `regex_token_iterator` object that points to a final sequence of characters at the end of the target sequence. In a suffix iterator the member `result` holds a pointer to the data member `suffix`, the value of the member `suffix.match` is `true`, `suffix.first` points to the beginning of the final sequence, and `suffix.second` points to the end of the final sequence.

8 [*Note:* For a suffix iterator, data member `suffix.first` is the same as the end of the last match found, and `suffix.second` is the same as the end of the target sequence — *end note*]

9 The *current match* is `(*position).prefix()` if `subs[N] == -1`, or `(*position)[subs[N]]` for any other value of `subs[N]`.

28.12.2.1 `regex_token_iterator` constructors [re.tokiter.cnstr]

```
regex_token_iterator();
```

1 *Effects:* Constructs the end-of-sequence iterator.

```
regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                     const regex_type& re,
                     int submatch = 0,
                     regex_constants::match_flag_type m =
                      regex_constants::match_default);

regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                     const regex_type& re,
                     const std::vector<int>& submatches,
                     regex_constants::match_flag_type m =
                      regex_constants::match_default);

regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                     const regex_type& re,
                     initializer_list<int> submatches,
                     regex_constants::match_flag_type m =
                       regex_constants::match_default);

template <std::size_t N>
  regex_token_iterator(BidirectionalIterator a, BidirectionalIterator b,
                     const regex_type& re,
                     const int (&submatches)[N],
                     regex_constants::match_flag_type m =
                      regex_constants::match_default);
```

2 *Requires:* Each of the initialization values of `submatches` shall be `>= -1`.

3 *Effects:* The first constructor initializes the member `subs` to hold the single value `submatch`. The second constructor initializes the member `subs` to hold a copy of the argument `submatches`. The third and fourth constructors initialize the member `subs` to hold a copy of the sequence of integer values pointed to by the iterator range `[submatches.begin(),submatches.end())` and `[&submatches,&submatches + N)`, respectively.

4 Each constructor then sets `N` to 0, and `position` to `position_iterator(a, b, re, m)`. If `position` is not an end-of-sequence iterator the constructor sets `result` to the address of the current match. Otherwise if any of the values stored in `subs` is equal to -1 the constructor sets `*this` to a suffix

iterator that points to the range `[a,b)`, otherwise the constructor sets `*this` to an end-of-sequence iterator.

28.12.2.2 `regex_token_iterator` comparisons [re.tokiter.comp]

```
bool operator==(const regex_token_iterator& right) const;
```

1 *Returns:* `true` if `*this` and `right` are both end-of-sequence iterators, or if `*this` and `right` are both suffix iterators and `suffix == right.suffix`; otherwise returns `false` if `*this` or `right` is an end-of-sequence iterator or a suffix iterator. Otherwise returns `true` if `position == right.position`, `N == right.N`, and `subs == right.subs`. Otherwise returns `false`.

```
bool operator!=(const regex_token_iterator& right) const;
```

2 *Returns:* `!(*this == right)`.

28.12.2.3 `regex_token_iterator` indirection [re.tokiter.deref]

```
const value_type& operator*() const;
```

1 *Returns:* `*result`.

```
const value_type* operator->() const;
```

2 *Returns:* `result`.

28.12.2.4 `regex_token_iterator` increment [re.tokiter.incr]

```
regex_token_iterator& operator++();
```

1 *Effects:* Constructs a local variable `prev` of type `position_iterator`, initialized with the value of `position`.

2 If `*this` is a suffix iterator, sets `*this` to an end-of-sequence iterator.

3 Otherwise, if `N + 1 < subs.size()`, increments `N` and sets `result` to the address of the current match.

4 Otherwise, sets `N` to 0 and increments `position`. If `position` is not an end-of-sequence iterator the operator sets `result` to the address of the current match.

5 Otherwise, if any of the values stored in `subs` is equal to -1 and `prev->suffix().length()` is not 0 the operator sets `*this` to a suffix iterator that points to the range `[prev->suffix().first, prev->suffix().second)`.

6 Otherwise, sets `*this` to an end-of-sequence iterator.

Returns: `*this`

```
regex_token_iterator& operator++(int);
```

7 *Effects:* Constructs a copy `tmp` of `*this`, then calls `++(*this)`.

8 *Returns:* `tmp`.

28.13 Modified ECMAScript regular expression grammar [re.grammar]

1 The regular expression grammar recognized by `basic_regex` objects constructed with the ECMAScript flag is that specified by ECMA-262, except as specified below.

2 Objects of type specialization of `basic_regex` store within themselves a default-constructed instance of their `traits` template parameter, henceforth referred to as `traits_inst`. This `traits_inst` object is used to support localization of the regular expression; `basic_regex` member functions shall not call any locale dependent C or C++ API, including the formatted string input functions. Instead they shall call the appropriate traits member function to achieve the required effect.

3 The following productions within the ECMAScript grammar are modified as follows:

```
ClassAtom ::
  -
  ClassAtomNoDash
  ClassAtomExClass
  ClassAtomCollatingElement
  ClassAtomEquivalence
```

4 The following new productions are then added:

```
ClassAtomExClass ::
  [: ClassName :]

ClassAtomCollatingElement ::
  [. ClassName .]

ClassAtomEquivalence ::
  [= ClassName =]

ClassName ::
  ClassNameCharacter
  ClassNameCharacter ClassName

ClassNameCharacter ::
  SourceCharacter but not one of "." "=" ":"
```

5 The productions `ClassAtomExClass`, `ClassAtomCollatingElement` and `ClassAtomEquivalence` provide functionality equivalent to that of the same features in regular expressions in POSIX.

6 The regular expression grammar may be modified by any `regex_constants::syntax_option_type` flags specified when constructing an object of type specialization of `basic_regex` according to the rules in Table 137.

7 A `ClassName` production, when used in `ClassAtomExClass`, is not valid if `traits_inst.lookup_classname` returns zero for that name. The names recognized as valid `ClassName`s are determined by the type of the traits class, but at least the following names shall be recognized: `alnum`, `alpha`, `blank`, `cntrl`, `digit`, `graph`, `lower`, `print`, `punct`, `space`, `upper`, `xdigit`, `d`, `s`, `w`. In addition the following expressions shall be equivalent:

```
\d and [[:digit:]]

\D and [^[:digit:]]

\s and [[:space:]]

\S and [^[:space:]]

\w and [_[:alnum:]]

\W and [^_[:alnum:]]
```

8 A `ClassName` production when used in a `ClassAtomCollatingElement` production is not valid if the value returned by `traits_inst.lookup_collatename` for that name is an empty string.

9 The results from multiple calls to `traits_inst.lookup_classname` can be bitwise OR'ed together and subsequently passed to `traits_inst.isctype`.

10 A `ClassName` production when used in a `ClassAtomEquivalence` production is not valid if the value returned by `traits_inst.lookup_collatename` for that name is an empty string or if the value returned by `traits_inst.transform_primary` for the result of the call to `traits_inst.lookup_collatename` is an empty string.

11 When the sequence of characters being transformed to a finite state machine contains an invalid class name the translator shall throw an exception object of type `regex_error`.

12 If the *CV* of a *UnicodeEscapeSequence* is greater than the largest value that can be held in an object of type `charT` the translator shall throw an exception object of type `regex_error`. [*Note:* This means that values of the form `"uxxxx"` that do not fit in a character are invalid. — *end note*]

13 Where the regular expression grammar requires the conversion of a sequence of characters to an integral value, this is accomplished by calling `traits_inst.value`.

14 The behavior of the internal finite state machine representation when used to match a sequence of characters is as described in ECMA-262. The behavior is modified according to any match_flag_type flags 28.5.2 specified when using the regular expression object in one of the regular expression algorithms 28.11. The behavior is also localized by interaction with the traits class template parameter as follows:

(14.1) — During matching of a regular expression finite state machine against a sequence of characters, two characters `c` and `d` are compared using the following rules:

1. if `(flags() & regex_constants::icase)` the two characters are equal if `traits_inst.translate_nocase(c) == traits_inst.translate_nocase(d)`;
2. otherwise, if `flags() & regex_constants::collate` the two characters are equal if `traits_inst.translate(c) == traits_inst.translate(d)`;
3. otherwise, the two characters are equal if `c == d`.

(14.2) — During matching of a regular expression finite state machine against a sequence of characters, comparison of a collating element range `c1-c2` against a character `c` is conducted as follows: if `flags() & regex_constants ::collate` is false then the character `c` is matched if `c1 <= c && c <= c2`, otherwise `c` is matched in accordance with the following algorithm:

```
string_type str1 = string_type(1,
  flags() & icase ?
    traits_inst.translate_nocase(c1) : traits_inst.translate(c1);
string_type str2 = string_type(1,
  flags() & icase ?
    traits_inst.translate_nocase(c2) : traits_inst.translate(c2);
string_type str = string_type(1,
  flags() & icase ?
    traits_inst.translate_nocase(c) : traits_inst.translate(c);
return traits_inst.transform(str1.begin(), str1.end())
      <= traits_inst.transform(str.begin(), str.end())
  && traits_inst.transform(str.begin(), str.end())
      <= traits_inst.transform(str2.begin(), str2.end());
```

(14.3) — During matching of a regular expression finite state machine against a sequence of characters, testing whether a collating element is a member of a primary equivalence class is conducted by first converting the collating element and the equivalence class to sort keys using `traits::transform_primary`, and then comparing the sort keys for equality.

(14.4) — During matching of a regular expression finite state machine against a sequence of characters, a character `c` is a member of a character class designated by an iterator range `[first,last)` if `traits_inst.isctype(c, traits_inst.lookup_classname(first, last, flags() & icase))` is `true`.

29 Atomic operations library [atomics]

29.1 General [atomics.general]

1 This Clause describes components for fine-grained atomic access. This access is provided via operations on atomic objects.

2 The following subclauses describe atomics requirements and components for types and operations, as summarized below.

Table 144 — Atomics library summary

Subclause		Header(s)
29.3	Order and Consistency	
29.4	Lock-free Property	
29.5	Atomic Types	`<atomic>`
29.6	Operations on Atomic Types	
29.7	Flag Type and Operations	
29.8	Fences	

29.2 Header `<atomic>` synopsis [atomics.syn]

```
namespace std {
  // 29.3, order and consistency
  enum memory_order;
  template <class T>
    T kill_dependency(T y) noexcept;

  // 29.4, lock-free property
  #define ATOMIC_BOOL_LOCK_FREE unspecified
  #define ATOMIC_CHAR_LOCK_FREE unspecified
  #define ATOMIC_CHAR16_T_LOCK_FREE unspecified
  #define ATOMIC_CHAR32_T_LOCK_FREE unspecified
  #define ATOMIC_WCHAR_T_LOCK_FREE unspecified
  #define ATOMIC_SHORT_LOCK_FREE unspecified
  #define ATOMIC_INT_LOCK_FREE unspecified
  #define ATOMIC_LONG_LOCK_FREE unspecified
  #define ATOMIC_LLONG_LOCK_FREE unspecified
  #define ATOMIC_POINTER_LOCK_FREE unspecified

  // 29.5, generic types
  template<class T> struct atomic;
  template<> struct atomic<integral>;
  template<class T> struct atomic<T*>;

  // 29.6.1, general operations on atomic types
  // In the following declarations, atomic-type is either
  // atomic<T> or a named base class for T from
  // Table 145 or inferred from Table 146 or from bool.
  // If it is atomic<T>, then the declaration is a template
  // declaration prefixed with template <class T>.
```

```
bool atomic_is_lock_free(const volatile atomic-type*) noexcept;
bool atomic_is_lock_free(const atomic-type*) noexcept;
void atomic_init(volatile atomic-type*, T) noexcept;
void atomic_init(atomic-type*, T) noexcept;
void atomic_store(volatile atomic-type*, T) noexcept;
void atomic_store(atomic-type*, T) noexcept;
void atomic_store_explicit(volatile atomic-type*, T, memory_order) noexcept;
void atomic_store_explicit(atomic-type*, T, memory_order) noexcept;
T atomic_load(const volatile atomic-type*) noexcept;
T atomic_load(const atomic-type*) noexcept;
T atomic_load_explicit(const volatile atomic-type*, memory_order) noexcept;
T atomic_load_explicit(const atomic-type*, memory_order) noexcept;
T atomic_exchange(volatile atomic-type*, T) noexcept;
T atomic_exchange(atomic-type*, T) noexcept;
T atomic_exchange_explicit(volatile atomic-type*, T, memory_order) noexcept;
T atomic_exchange_explicit(atomic-type*, T, memory_order) noexcept;
bool atomic_compare_exchange_weak(volatile atomic-type*, T*, T) noexcept;
bool atomic_compare_exchange_weak(atomic-type*, T*, T) noexcept;
bool atomic_compare_exchange_strong(volatile atomic-type*, T*, T) noexcept;
bool atomic_compare_exchange_strong(atomic-type*, T*, T) noexcept;
bool atomic_compare_exchange_weak_explicit(volatile atomic-type*, T*, T,
  memory_order, memory_order) noexcept;
bool atomic_compare_exchange_weak_explicit(atomic-type*, T*, T,
  memory_order, memory_order) noexcept;
bool atomic_compare_exchange_strong_explicit(volatile atomic-type*, T*, T,
  memory_order, memory_order) noexcept;
bool atomic_compare_exchange_strong_explicit(atomic-type*, T*, T,
  memory_order, memory_order) noexcept;

// 29.6.2, templated operations on atomic types
template <class T>
  T atomic_fetch_add(volatile atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_add(atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_add_explicit(volatile atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_add_explicit(atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_sub(volatile atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_sub(atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_sub_explicit(volatile atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_sub_explicit(atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_and(volatile atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_and(atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_and_explicit(volatile atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_and_explicit(atomic<T>*, T, memory_order) noexcept;
template <class T>
```

```
  T atomic_fetch_or(volatile atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_or(atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_or_explicit(volatile atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_or_explicit(atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_xor(volatile atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_xor(atomic<T>*, T) noexcept;
template <class T>
  T atomic_fetch_xor_explicit(volatile atomic<T>*, T, memory_order) noexcept;
template <class T>
  T atomic_fetch_xor_explicit(atomic<T>*, T, memory_order) noexcept;

// 29.6.3, arithmetic operations on atomic types
// In the following declarations, atomic-integral is either
// atomic<T> or a named base class for T from
// Table 145 or inferred from Table 146.
// If it is atomic<T>, then the declaration is a template
// specialization declaration prefixed with template <>.

integral atomic_fetch_add(volatile atomic-integral*, integral) noexcept;
integral atomic_fetch_add(atomic-integral*, integral) noexcept;
integral atomic_fetch_add_explicit(volatile atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_add_explicit(atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_sub(volatile atomic-integral*, integral) noexcept;
integral atomic_fetch_sub(atomic-integral*, integral) noexcept;
integral atomic_fetch_sub_explicit(volatile atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_sub_explicit(atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_and(volatile atomic-integral*, integral) noexcept;
integral atomic_fetch_and(atomic-integral*, integral) noexcept;
integral atomic_fetch_and_explicit(volatile atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_and_explicit(atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_or(volatile atomic-integral*, integral) noexcept;
integral atomic_fetch_or(atomic-integral*, integral) noexcept;
integral atomic_fetch_or_explicit(volatile atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_or_explicit(atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_xor(volatile atomic-integral*, integral) noexcept;
integral atomic_fetch_xor(atomic-integral*, integral) noexcept;
integral atomic_fetch_xor_explicit(volatile atomic-integral*, integral, memory_order) noexcept;
integral atomic_fetch_xor_explicit(atomic-integral*, integral, memory_order) noexcept;

// 29.6.4, partial specializations for pointers

template <class T>
  T* atomic_fetch_add(volatile atomic<T*>*, ptrdiff_t) noexcept;
template <class T>
  T* atomic_fetch_add(atomic<T*>*, ptrdiff_t) noexcept;
template <class T>
  T* atomic_fetch_add_explicit(volatile atomic<T*>*, ptrdiff_t, memory_order) noexcept;
template <class T>
  T* atomic_fetch_add_explicit(atomic<T*>*, ptrdiff_t, memory_order) noexcept;
template <class T>
```

```
    T* atomic_fetch_sub(volatile atomic<T*>*, ptrdiff_t) noexcept;
  template <class T>
    T* atomic_fetch_sub(atomic<T*>*, ptrdiff_t) noexcept;
  template <class T>
    T* atomic_fetch_sub_explicit(volatile atomic<T*>*, ptrdiff_t, memory_order) noexcept;
  template <class T>
    T* atomic_fetch_sub_explicit(atomic<T*>*, ptrdiff_t, memory_order) noexcept;

  // 29.6.5, initialization
  #define ATOMIC_VAR_INIT(value) see below

  // 29.7, flag type and operations
  struct atomic_flag;
  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
  #define ATOMIC_FLAG_INIT see below

  // 29.8, fences
  extern "C" void atomic_thread_fence(memory_order) noexcept;
  extern "C" void atomic_signal_fence(memory_order) noexcept;
}
```

29.3 Order and consistency [atomics.order]

```
namespace std {
  typedef enum memory_order {
    memory_order_relaxed, memory_order_consume, memory_order_acquire,
    memory_order_release, memory_order_acq_rel, memory_order_seq_cst
  } memory_order;
}
```

1 The enumeration `memory_order` specifies the detailed regular (non-atomic) memory synchronization order as defined in 1.10 and may provide for operation ordering. Its enumerated values and their meanings are as follows:

(1.1) — `memory_order_relaxed`: no operation orders memory.

(1.2) — `memory_order_release`, `memory_order_acq_rel`, and `memory_order_seq_cst`: a store operation performs a release operation on the affected memory location.

(1.3) — `memory_order_consume`: a load operation performs a consume operation on the affected memory location.

(1.4) — `memory_order_acquire`, `memory_order_acq_rel`, and `memory_order_seq_cst`: a load operation performs an acquire operation on the affected memory location.

[*Note:* Atomic operations specifying `memory_order_relaxed` are relaxed with respect to memory ordering. Implementations must still guarantee that any given atomic access to a particular atomic object be indivisible with respect to all other atomic accesses to that object. — *end note*]

2 An atomic operation A that performs a release operation on an atomic object M synchronizes with an atomic operation B that performs an acquire operation on M and takes its value from any side effect in the release sequence headed by A.

3 There shall be a single total order S on all `memory_order_seq_cst` operations, consistent with the "happens before" order and modification orders for all affected locations, such that each `memory_order_seq_cst` operation B that loads a value from an atomic object M observes one of the following values:

(3.1) — the result of the last modification A of M that precedes B in S, if it exists, or

(3.2) — if A exists, the result of some modification of M that is not `memory_order_seq_cst` and that does not happen before A, or

(3.3) — if A does not exist, the result of some modification of M that is not `memory_order_seq_cst`.

[*Note:* Although it is not explicitly required that S include locks, it can always be extended to an order that does include lock and unlock operations, since the ordering between those is already included in the "happens before" ordering. — *end note*]

4 For an atomic operation B that reads the value of an atomic object M, if there is a `memory_order_seq_cst` fence X sequenced before B, then B observes either the last `memory_order_seq_cst` modification of M preceding X in the total order S or a later modification of M in its modification order.

5 For atomic operations A and B on an atomic object M, where A modifies M and B takes its value, if there is a `memory_order_seq_cst` fence X such that A is sequenced before X and B follows X in S, then B observes either the effects of A or a later modification of M in its modification order.

6 For atomic operations A and B on an atomic object M, where A modifies M and B takes its value, if there are `memory_order_seq_cst` fences X and Y such that A is sequenced before X, Y is sequenced before B, and X precedes Y in S, then B observes either the effects of A or a later modification of M in its modification order.

7 For atomic modifications A and B of an atomic object M, B occurs later than A in the modification order of M if:

(7.1) — there is a `memory_order_seq_cst` fence X such that A is sequenced before X, and X precedes B in S, or

(7.2) — there is a `memory_order_seq_cst` fence Y such that Y is sequenced before B, and A precedes Y in S, or

(7.3) — there are `memory_order_seq_cst` fences X and Y such that A is sequenced before X, Y is sequenced before B, and X precedes Y in S.

8 [*Note:* `memory_order_seq_cst` ensures sequential consistency only for a program that is free of data races and uses exclusively `memory_order_seq_cst` operations. Any use of weaker ordering will invalidate this guarantee unless extreme care is used. In particular, `memory_order_seq_cst` fences ensure a total order only for the fences themselves. Fences cannot, in general, be used to restore sequential consistency for atomic operations with weaker ordering specifications. — *end note*]

9 Implementations should ensure that no "out-of-thin-air" values are computed that circularly depend on their own computation.

[*Note:* For example, with `x` and `y` initially zero,

```
// Thread 1:
r1 = y.load(memory_order_relaxed);
x.store(r1, memory_order_relaxed);
```

```
// Thread 2:
r2 = x.load(memory_order_relaxed);
y.store(r2, memory_order_relaxed);
```

should not produce `r1 == r2 == 42`, since the store of 42 to `y` is only possible if the store to `x` stores `42`, which circularly depends on the store to `y` storing `42`. Note that without this restriction, such an execution is possible. — *end note*]

10 [*Note:* The recommendation similarly disallows `r1 == r2 == 42` in the following example, with `x` and `y` again initially zero:

```
// Thread 1:
r1 = x.load(memory_order_relaxed);
if (r1 == 42) y.store(42, memory_order_relaxed);

// Thread 2:
r2 = y.load(memory_order_relaxed);
if (r2 == 42) x.store(42, memory_order_relaxed);
```

— *end note*]

11 Atomic read-modify-write operations shall always read the last value (in the modification order) written before the write associated with the read-modify-write operation.

12 Implementations should make atomic stores visible to atomic loads within a reasonable amount of time.

```
template <class T>
  T kill_dependency(T y) noexcept;
```

13 *Effects:* The argument does not carry a dependency to the return value (1.10).

14 *Returns:* `y`.

29.4 Lock-free property [atomics.lockfree]

```
#define ATOMIC_BOOL_LOCK_FREE unspecified
#define ATOMIC_CHAR_LOCK_FREE unspecified
#define ATOMIC_CHAR16_T_LOCK_FREE unspecified
#define ATOMIC_CHAR32_T_LOCK_FREE unspecified
#define ATOMIC_WCHAR_T_LOCK_FREE unspecified
#define ATOMIC_SHORT_LOCK_FREE unspecified
#define ATOMIC_INT_LOCK_FREE unspecified
#define ATOMIC_LONG_LOCK_FREE unspecified
#define ATOMIC_LLONG_LOCK_FREE unspecified
#define ATOMIC_POINTER_LOCK_FREE unspecified
```

1 The `ATOMIC_..._LOCK_FREE` macros indicate the lock-free property of the corresponding atomic types, with the signed and unsigned variants grouped together. The properties also apply to the corresponding (partial) specializations of the `atomic` template. A value of 0 indicates that the types are never lock-free. A value of 1 indicates that the types are sometimes lock-free. A value of 2 indicates that the types are always lock-free.

2 The function `atomic_is_lock_free` (29.6) indicates whether the object is lock-free. In any given program execution, the result of the lock-free query shall be consistent for all pointers of the same type.

3 [*Note:* Operations that are lock-free should also be address-free. That is, atomic operations on the same memory location via two different addresses will communicate atomically. The implementation should not depend on any per-process state. This restriction enables communication by memory that is mapped into a process more than once and by memory that is shared between two processes. — *end note*]

29.5 Atomic types [atomics.types.generic]

```
namespace std {
  template <class T> struct atomic {
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T, memory_order = memory_order_seq_cst) volatile noexcept;
    void store(T, memory_order = memory_order_seq_cst) noexcept;
    T load(memory_order = memory_order_seq_cst) const volatile noexcept;
    T load(memory_order = memory_order_seq_cst) const noexcept;
    operator T() const volatile noexcept;
    operator T() const noexcept;
    T exchange(T, memory_order = memory_order_seq_cst) volatile noexcept;
    T exchange(T, memory_order = memory_order_seq_cst) noexcept;
    bool compare_exchange_weak(T&, T, memory_order, memory_order) volatile noexcept;
    bool compare_exchange_weak(T&, T, memory_order, memory_order) noexcept;
    bool compare_exchange_strong(T&, T, memory_order, memory_order) volatile noexcept;
    bool compare_exchange_strong(T&, T, memory_order, memory_order) noexcept;
    bool compare_exchange_weak(T&, T, memory_order = memory_order_seq_cst) volatile noexcept;
    bool compare_exchange_weak(T&, T, memory_order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(T&, T, memory_order = memory_order_seq_cst) volatile noexcept;
    bool compare_exchange_strong(T&, T, memory_order = memory_order_seq_cst) noexcept;

    atomic() noexcept = default;
    constexpr atomic(T) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T operator=(T) volatile noexcept;
    T operator=(T) noexcept;
  };

  template <> struct atomic<integral> {
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(integral, memory_order = memory_order_seq_cst) volatile noexcept;
    void store(integral, memory_order = memory_order_seq_cst) noexcept;
    integral load(memory_order = memory_order_seq_cst) const volatile noexcept;
    integral load(memory_order = memory_order_seq_cst) const noexcept;
    operator integral() const volatile noexcept;
    operator integral() const noexcept;
    integral exchange(integral, memory_order = memory_order_seq_cst) volatile noexcept;
    integral exchange(integral, memory_order = memory_order_seq_cst) noexcept;
    bool compare_exchange_weak(integral&, integral, memory_order, memory_order) volatile noexcept;
    bool compare_exchange_weak(integral&, integral, memory_order, memory_order) noexcept;
    bool compare_exchange_strong(integral&, integral, memory_order, memory_order) volatile noexcept;
    bool compare_exchange_strong(integral&, integral, memory_order, memory_order) noexcept;
    bool compare_exchange_weak(integral&, integral, memory_order = memory_order_seq_cst) volatile noexcept;
    bool compare_exchange_weak(integral&, integral, memory_order = memory_order_seq_cst) noexcept;
    bool compare_exchange_strong(integral&, integral, memory_order = memory_order_seq_cst) volatile noexcept;
    bool compare_exchange_strong(integral&, integral, memory_order = memory_order_seq_cst) noexcept;
    integral fetch_add(integral, memory_order = memory_order_seq_cst) volatile noexcept;
    integral fetch_add(integral, memory_order = memory_order_seq_cst) noexcept;
    integral fetch_sub(integral, memory_order = memory_order_seq_cst) volatile noexcept;
    integral fetch_sub(integral, memory_order = memory_order_seq_cst) noexcept;
    integral fetch_and(integral, memory_order = memory_order_seq_cst) volatile noexcept;
    integral fetch_and(integral, memory_order = memory_order_seq_cst) noexcept;
```

```
  integral fetch_or(integral, memory_order = memory_order_seq_cst) volatile noexcept;
  integral fetch_or(integral, memory_order = memory_order_seq_cst) noexcept;
  integral fetch_xor(integral, memory_order = memory_order_seq_cst) volatile noexcept;
  integral fetch_xor(integral, memory_order = memory_order_seq_cst) noexcept;

  atomic() noexcept = default;
  constexpr atomic(integral) noexcept;
  atomic(const atomic&) = delete;
  atomic& operator=(const atomic&) = delete;
  atomic& operator=(const atomic&) volatile = delete;
  integral operator=(integral) volatile noexcept;
  integral operator=(integral) noexcept;

  integral operator++(int) volatile noexcept;
  integral operator++(int) noexcept;
  integral operator--(int) volatile noexcept;
  integral operator--(int) noexcept;
  integral operator++() volatile noexcept;
  integral operator++() noexcept;
  integral operator--() volatile noexcept;
  integral operator--() noexcept;
  integral operator+=(integral) volatile noexcept;
  integral operator+=(integral) noexcept;
  integral operator-=(integral) volatile noexcept;
  integral operator-=(integral) noexcept;
  integral operator&=(integral) volatile noexcept;
  integral operator&=(integral) noexcept;
  integral operator|=(integral) volatile noexcept;
  integral operator|=(integral) noexcept;
  integral operator^=(integral) volatile noexcept;
  integral operator^=(integral) noexcept;
};

template <class T> struct atomic<T*> {
  bool is_lock_free() const volatile noexcept;
  bool is_lock_free() const noexcept;
  void store(T*, memory_order = memory_order_seq_cst) volatile noexcept;
  void store(T*, memory_order = memory_order_seq_cst) noexcept;
  T* load(memory_order = memory_order_seq_cst) const volatile noexcept;
  T* load(memory_order = memory_order_seq_cst) const noexcept;
  operator T*() const volatile noexcept;
  operator T*() const noexcept;
  T* exchange(T*, memory_order = memory_order_seq_cst) volatile noexcept;
  T* exchange(T*, memory_order = memory_order_seq_cst) noexcept;
  bool compare_exchange_weak(T*&, T*, memory_order, memory_order) volatile noexcept;
  bool compare_exchange_weak(T*&, T*, memory_order, memory_order) noexcept;
  bool compare_exchange_strong(T*&, T*, memory_order, memory_order) volatile noexcept;
  bool compare_exchange_strong(T*&, T*, memory_order, memory_order) noexcept;
  bool compare_exchange_weak(T*&, T*, memory_order = memory_order_seq_cst) volatile noexcept;
  bool compare_exchange_weak(T*&, T*, memory_order = memory_order_seq_cst) noexcept;
  bool compare_exchange_strong(T*&, T*, memory_order = memory_order_seq_cst) volatile noexcept;
  bool compare_exchange_strong(T*&, T*, memory_order = memory_order_seq_cst) noexcept;
  T* fetch_add(ptrdiff_t, memory_order = memory_order_seq_cst) volatile noexcept;
  T* fetch_add(ptrdiff_t, memory_order = memory_order_seq_cst) noexcept;
  T* fetch_sub(ptrdiff_t, memory_order = memory_order_seq_cst) volatile noexcept;
```

```
    T* fetch_sub(ptrdiff_t, memory_order = memory_order_seq_cst) noexcept;

    atomic() noexcept = default;
    constexpr atomic(T*) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T* operator=(T*) volatile noexcept;
    T* operator=(T*) noexcept;

    T* operator++(int) volatile noexcept;
    T* operator++(int) noexcept;
    T* operator--(int) volatile noexcept;
    T* operator--(int) noexcept;
    T* operator++() volatile noexcept;
    T* operator++() noexcept;
    T* operator--() volatile noexcept;
    T* operator--() noexcept;
    T* operator+=(ptrdiff_t) volatile noexcept;
    T* operator+=(ptrdiff_t) noexcept;
    T* operator-=(ptrdiff_t) volatile noexcept;
    T* operator-=(ptrdiff_t) noexcept;
  };
}
```

1 There is a generic class template `atomic<T>`. The type of the template argument `T` shall be trivially copyable (3.9). [*Note:* Type arguments that are not also statically initializable may be difficult to use. — *end note*]

2 The semantics of the operations on specializations of `atomic` are defined in 29.6.

3 Specializations and instantiations of the `atomic` template shall have a deleted copy constructor, a deleted copy assignment operator, and a constexpr value constructor.

4 There shall be explicit specializations of the `atomic` template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<cstdint>`. For each integral type *integral*, the specialization `atomic<integral>` provides additional atomic operations appropriate to integral types. There shall be a specialization `atomic<bool>` which provides the general atomic operations as specified in 29.6.1.

5 The atomic integral specializations and the specialization `atomic<bool>` shall have standard layout. They shall each have a trivial default constructor and a trivial destructor. They shall each support aggregate initialization syntax.

6 There shall be pointer partial specializations of the `atomic` class template. These specializations shall have standard layout, trivial default constructors, and trivial destructors. They shall each support aggregate initialization syntax.

7 There shall be named types corresponding to the integral specializations of `atomic`, as specified in Table 145, and a named type `atomic_bool` corresponding to the specified `atomic<bool>`. Each named type is either a typedef to the corresponding specialization or a base class of the corresponding specialization. If it is a base class, it shall support the same member functions as the corresponding specialization.

8 There shall be atomic typedefs corresponding to the typedefs in the header `<inttypes.h>` as specified in Table 146.

9 [*Note:* The representation of an atomic specialization need not have the same size as its corresponding

Table 145 — `atomic` integral typedefs

Named type	Integral argument type
`atomic_char`	`char`
`atomic_schar`	`signed char`
`atomic_uchar`	`unsigned char`
`atomic_short`	`short`
`atomic_ushort`	`unsigned short`
`atomic_int`	`int`
`atomic_uint`	`unsigned int`
`atomic_long`	`long`
`atomic_ulong`	`unsigned long`
`atomic_llong`	`long long`
`atomic_ullong`	`unsigned long long`
`atomic_char16_t`	`char16_t`
`atomic_char32_t`	`char32_t`
`atomic_wchar_t`	`wchar_t`

argument type. Specializations should have the same size whenever possible, as this reduces the effort required to port existing code. — *end note*]

29.6 Operations on atomic types [atomics.types.operations]

29.6.1 General operations on atomic types [atomics.types.operations.general]

1 The implementation shall provide the functions and function templates identified as "general operations on atomic types" in 29.2.

2 In the declarations of these functions and function templates, the name *atomic-type* refers to either `atomic<T>` or to a named base class for `T` from Table 145 or inferred from Table 146.

29.6.2 Templated operations on atomic types [atomics.types.operations.templ]

1 The implementation shall declare but not define the function templates identified as "templated operations on atomic types" in 29.2.

29.6.3 Arithmetic operations on atomic types [atomics.types.operations.arith]

1 The implementation shall provide the functions and function template specializations identified as "arithmetic operations on atomic types" in 29.2.

2 In the declarations of these functions and function template specializations, the name *integral* refers to an integral type and the name *atomic-integral* refers to either `atomic<`*`integral`*`>` or to a named base class for *`integral`* from Table 145 or inferred from Table 146.

29.6.4 Operations on atomic pointer types [atomics.types.operations.pointer]

1 The implementation shall provide the function template specializations identified as "partial specializations for pointers" in 29.2.

29.6.5 Requirements for operations on atomic types [atomics.types.operations.req]

1 There are only a few kinds of operations on atomic types, though there are many instances on those kinds. This section specifies each general kind. The specific instances are defined in 29.5, 29.6.1, 29.6.3, and 29.6.4.

2 In the following operation definitions:

(2.1) — an *A* refers to one of the atomic types.

Table 146 — `atomic <inttypes.h>` typedefs

Atomic typedef	<inttypes.h> type
atomic_int_least8_t	int_least8_t
atomic_uint_least8_t	uint_least8_t
atomic_int_least16_t	int_least16_t
atomic_uint_least16_t	uint_least16_t
atomic_int_least32_t	int_least32_t
atomic_uint_least32_t	uint_least32_t
atomic_int_least64_t	int_least64_t
atomic_uint_least64_t	uint_least64_t
atomic_int_fast8_t	int_fast8_t
atomic_uint_fast8_t	uint_fast8_t
atomic_int_fast16_t	int_fast16_t
atomic_uint_fast16_t	uint_fast16_t
atomic_int_fast32_t	int_fast32_t
atomic_uint_fast32_t	uint_fast32_t
atomic_int_fast64_t	int_fast64_t
atomic_uint_fast64_t	uint_fast64_t
atomic_intptr_t	intptr_t
atomic_uintptr_t	uintptr_t
atomic_size_t	size_t
atomic_ptrdiff_t	ptrdiff_t
atomic_intmax_t	intmax_t
atomic_uintmax_t	uintmax_t

(2.2) — a *C* refers to its corresponding non-atomic type.

(2.3) — an *M* refers to type of the other argument for arithmetic operations. For integral atomic types, *M* is *C*. For atomic address types, *M* is `std::ptrdiff_t`.

(2.4) — the non-member functions not ending in `_explicit` have the semantics of their corresponding `_explicit` functions with `memory_order` arguments of `memory_order_seq_cst`.

3 [*Note:* Many operations are volatile-qualified. The "volatile as device register" semantics have not changed in the standard. This qualification means that volatility is preserved when applying these operations to volatile objects. It does not mean that operations on non-volatile objects become volatile. Thus, volatile qualified operations on non-volatile objects may be merged under some conditions. — *end note*]

```
A::A() noexcept = default;
```

4 *Effects:* leaves the atomic object in an uninitialized state. [*Note:* These semantics ensure compatibility with C. — *end note*]

```
constexpr A::A(C desired) noexcept;
```

5 *Effects:* Initializes the object with the value `desired`. Initialization is not an atomic operation (1.10). [*Note:* it is possible to have an access to an atomic object `A` race with its construction, for example by communicating the address of the just-constructed object `A` to another thread via `memory_order_relaxed` operations on a suitable atomic pointer variable, and then immediately accessing `A` in the receiving thread. This results in undefined behavior. — *end note*]

```
#define ATOMIC_VAR_INIT(value) see below
```

6 The macro expands to a token sequence suitable for constant initialization of an atomic variable of static storage duration of a type that is initialization-compatible with *value*. [*Note:* This operation may need to initialize locks. — *end note*] Concurrent access to the variable being initialized, even via an atomic operation, constitutes a data race. [*Example:*

```
atomic<int> v = ATOMIC_VAR_INIT(5);
```

— *end example*]

```
bool atomic_is_lock_free(const volatile A* object) noexcept;
bool atomic_is_lock_free(const A* object) noexcept;
bool A::is_lock_free() const volatile noexcept;
bool A::is_lock_free() const noexcept;
```

7 *Returns:* True if the object's operations are lock-free, false otherwise.

```
void atomic_init(volatile A* object, C desired) noexcept;
void atomic_init(A* object, C desired) noexcept;
```

8 *Effects:* Non-atomically initializes `*object` with value `desired`. This function shall only be applied to objects that have been default constructed, and then only once. [*Note:* These semantics ensure compatibility with C. — *end note*] [*Note:* Concurrent access from another thread, even via an atomic operation, constitutes a data race. — *end note*]

```
void atomic_store(volatile A* object, C desired) noexcept;
void atomic_store(A* object, C desired) noexcept;
void atomic_store_explicit(volatile A* object, C desired, memory_order order) noexcept;
void atomic_store_explicit(A* object, C desired, memory_order order) noexcept;
void A::store(C desired, memory_order order = memory_order_seq_cst) volatile noexcept;
void A::store(C desired, memory_order order = memory_order_seq_cst) noexcept;
```

9 *Requires:* The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

10 *Effects:* Atomically replaces the value pointed to by `object` or by `this` with the value of `desired`. Memory is affected according to the value of `order`.

```
C A::operator=(C desired) volatile noexcept;
C A::operator=(C desired) noexcept;
```

11 *Effects:* `store(desired)`

12 *Returns:* `desired`

```
C atomic_load(const volatile A* object) noexcept;
C atomic_load(const A* object) noexcept;
C atomic_load_explicit(const volatile A* object, memory_order) noexcept;
C atomic_load_explicit(const A* object, memory_order) noexcept;
C A::load(memory_order order = memory_order_seq_cst) const volatile noexcept;
C A::load(memory_order order = memory_order_seq_cst) const noexcept;
```

13 *Requires:* The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

14 *Effects:* Memory is affected according to the value of `order`.

15 *Returns:* Atomically returns the value pointed to by `object` or by `this`.

```
A::operator C() const volatile noexcept;
A::operator C() const noexcept;
```

16 *Effects:* `load()`

17 *Returns:* The result of `load()`.

```
C atomic_exchange(volatile A* object, C desired) noexcept;
C atomic_exchange(A* object, C desired) noexcept;
C atomic_exchange_explicit(volatile A* object, C desired, memory_order) noexcept;
C atomic_exchange_explicit(A* object, C desired, memory_order) noexcept;
C A::exchange(C desired, memory_order order = memory_order_seq_cst) volatile noexcept;
C A::exchange(C desired, memory_order order = memory_order_seq_cst) noexcept;
```

18 *Effects:* Atomically replaces the value pointed to by `object` or by `this` with `desired`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (1.10).

19 *Returns:* Atomically returns the value pointed to by `object` or by `this` immediately before the effects.

```
bool atomic_compare_exchange_weak(volatile A* object, C* expected, C desired) noexcept;
bool atomic_compare_exchange_weak(A* object, C* expected, C desired) noexcept;
bool atomic_compare_exchange_strong(volatile A* object, C* expected, C desired) noexcept;
bool atomic_compare_exchange_strong(A* object, C* expected, C desired) noexcept;
bool atomic_compare_exchange_weak_explicit(volatile A* object, C* expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool atomic_compare_exchange_weak_explicit(A* object, C* expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool atomic_compare_exchange_strong_explicit(volatile A* object, C* expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool atomic_compare_exchange_strong_explicit(A* object, C* expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool A::compare_exchange_weak(C& expected, C desired,
    memory_order success, memory_order failure) volatile noexcept;
bool A::compare_exchange_weak(C& expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool A::compare_exchange_strong(C& expected, C desired,
    memory_order success, memory_order failure) volatile noexcept;
bool A::compare_exchange_strong(C& expected, C desired,
    memory_order success, memory_order failure) noexcept;
bool A::compare_exchange_weak(C& expected, C desired,
    memory_order order = memory_order_seq_cst) volatile noexcept;
bool A::compare_exchange_weak(C& expected, C desired,
    memory_order order = memory_order_seq_cst) noexcept;
bool A::compare_exchange_strong(C& expected, C desired,
    memory_order order = memory_order_seq_cst) volatile noexcept;
bool A::compare_exchange_strong(C& expected, C desired,
    memory_order order = memory_order_seq_cst) noexcept;
```

20 *Requires:* The `failure` argument shall not be `memory_order_release` nor `memory_order_acq_rel`. The `failure` argument shall be no stronger than the `success` argument.

21 *Effects:* Atomically, compares the contents of the memory pointed to by `object` or by `this` for equality with that in `expected`, and if true, replaces the contents of the memory pointed to by `object` or by `this` with that in `desired`, and if false, updates the contents of the memory in `expected` with the contents of the memory pointed to by `object` or by `this`. Further, if the comparison is true, memory is affected according to the value of `success`, and if the comparison is false, memory is affected according to the value of `failure`. When only one `memory_order` argument is supplied, the value of `success` is `order`, and the value of `failure` is `order` except that a value of `memory_order_acq_rel` shall be replaced by the value `memory_order_acquire` and a value of `memory_order_release` shall

be replaced by the value `memory_order_relaxed`. If the operation returns `true`, these operations are atomic read-modify-write operations (1.10). Otherwise, these operations are atomic load operations.

22 *Returns:* The result of the comparison.

23 [*Note:* For example, the effect of `atomic_compare_exchange_strong` is

```
if (memcmp(object, expected, sizeof(*object)) == 0)
  memcpy(object, &desired, sizeof(*object));
else
  memcpy(expected, object, sizeof(*object));
```

— *end note*] [*Example:* the expected use of the compare-and-exchange operations is as follows. The compare-and-exchange operations will update `expected` when another iteration of the loop is needed.

```
expected = current.load();
do {
  desired = function(expected);
} while (!current.compare_exchange_weak(expected, desired));
```

— *end example*]

24 Implementations should ensure that weak compare-and-exchange operations do not consistently return `false` unless either the atomic object has value different from `expected` or there are concurrent modifications to the atomic object.

25 *Remark:* A weak compare-and-exchange operation may fail spuriously. That is, even when the contents of memory referred to by `expected` and `object` are equal, it may return false and store back to `expected` the same memory contents that were originally there. [*Note:* This spurious failure enables implementation of compare-and-exchange on a broader class of machines, e.g., load-locked store-conditional machines. A consequence of spurious failure is that nearly all uses of weak compare-and-exchange will be in a loop.

When a compare-and-exchange is in a loop, the weak version will yield better performance on some platforms. When a weak compare-and-exchange would require a loop and a strong one would not, the strong one is preferable. — *end note*]

26 [*Note:* The `memcpy` and `memcmp` semantics of the compare-and-exchange operations may result in failed comparisons for values that compare equal with `operator==` if the underlying type has padding bits, trap bits, or alternate representations of the same value. Thus, `compare_exchange_strong` should be used with extreme care. On the other hand, `compare_exchange_weak` should converge rapidly. — *end note*]

27 The following operations perform arithmetic computations. The key, operator, and computation correspondence is:

Table 147 — Atomic arithmetic computations

Key	Op	Computation	Key	Op	Computation
`add`	`+`	addition	`sub`	`-`	subtraction
`or`	`|`	bitwise inclusive or	`xor`	`^`	bitwise exclusive or
`and`	`&`	bitwise and			

```
C atomic_fetch_key(volatile A* object, M operand) noexcept;
C atomic_fetch_key(A* object, M operand) noexcept;
C atomic_fetch_key_explicit(volatile A* object, M operand, memory_order order) noexcept;
C atomic_fetch_key_explicit(A* object, M operand, memory_order order) noexcept;
```

C A::fetch_*key*(*M* operand, memory_order order = memory_order_seq_cst) volatile noexcept;
C A::fetch_*key*(*M* operand, memory_order order = memory_order_seq_cst) noexcept;

28 *Effects:* Atomically replaces the value pointed to by `object` or by `this` with the result of the *computation* applied to the value pointed to by `object` or by `this` and the given `operand`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (1.10).

29 *Returns:* Atomically, the value pointed to by `object` or by `this` immediately before the effects.

30 *Remark:* For signed integer types, arithmetic is defined to use two's complement representation. There are no undefined results. For address types, the result may be an undefined address, but the operations otherwise have no undefined behavior.

C A::operator *op*=(*M* operand) volatile noexcept;
C A::operator *op*=(*M* operand) noexcept;

31 *Effects:* `fetch_`*key*`(operand)`

32 *Returns:* `fetch_`*key*`(operand) op operand`

C A::operator++(int) volatile noexcept;
C A::operator++(int) noexcept;

33 *Returns:* `fetch_add(1)`

C A::operator--(int) volatile noexcept;
C A::operator--(int) noexcept;

34 *Returns:* `fetch_sub(1)`

C A::operator++() volatile noexcept;
C A::operator++() noexcept;

35 *Effects:* `fetch_add(1)`

36 *Returns:* `fetch_add(1) + 1`

C A::operator--() volatile noexcept;
C A::operator--() noexcept;

37 *Effects:* `fetch_sub(1)`

38 *Returns:* `fetch_sub(1) - 1`

29.7 Flag type and operations [atomics.flag]

```
namespace std {
  typedef struct atomic_flag {
    bool test_and_set(memory_order = memory_order_seq_cst) volatile noexcept;
    bool test_and_set(memory_order = memory_order_seq_cst) noexcept;
    void clear(memory_order = memory_order_seq_cst) volatile noexcept;
    void clear(memory_order = memory_order_seq_cst) noexcept;

    atomic_flag() noexcept = default;
    atomic_flag(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) volatile = delete;
  } atomic_flag;

  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
```

```
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_order) noexcept;
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexcept;
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noexcept;
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;

  #define ATOMIC_FLAG_INIT see below
}
```

1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.

2 Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. No other type requires lock-free operations, so the `atomic_flag` type is the minimum hardware-implemented type needed to conform to this International standard. The remaining types can be emulated with `atomic_flag`, though with less than ideal properties. — *end note*]

3 The `atomic_flag` type shall have standard layout. It shall have a trivial default constructor, a deleted copy constructor, a deleted copy assignment operator, and a trivial destructor.

4 The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object, memory_order order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order order) noexcept;
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst) volatile noexcept;
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst) noexcept;
```

5 *Effects:* Atomically sets the value pointed to by `object` or by `this` to true. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (1.10).

6 *Returns:* Atomically, the value of the object immediately before the effects.

```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object, memory_order order) noexcept;
void atomic_flag_clear_explicit(atomic_flag* object, memory_order order) noexcept;
void atomic_flag::clear(memory_order order = memory_order_seq_cst) volatile noexcept;
void atomic_flag::clear(memory_order order = memory_order_seq_cst) noexcept;
```

7 *Requires:* The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

8 *Effects:* Atomically sets the value pointed to by `object` or by `this` to false. Memory is affected according to the value of `order`.

29.8 Fences [atomics.fences]

1 This section introduces synchronization primitives called *fences*. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an *acquire fence*. A fence with release semantics is called a *release fence*.

2 A release fence A synchronizes with an acquire fence B if there exist atomic operations X and Y, both operating on some atomic object M, such that A is sequenced before X, X modifies M, Y is sequenced before B, and Y reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.

3 A release fence A synchronizes with an atomic operation B that performs an acquire operation on an atomic object M if there exists an atomic operation X such that A is sequenced before X, X modifies M, and B reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.

4 An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A.

```
extern "C" void atomic_thread_fence(memory_order order) noexcept;
```

5 *Effects:* depending on the value of `order`, this operation:

(5.1) — has no effects, if `order == memory_order_relaxed`;

(5.2) — is an acquire fence, if `order == memory_order_acquire || order == memory_order_consume`;

(5.3) — is a release fence, if `order == memory_order_release`;

(5.4) — is both an acquire fence and a release fence, if `order == memory_order_acq_rel`;

(5.5) — is a sequentially consistent acquire and release fence, if `order == memory_order_seq_cst`.

```
extern "C" void atomic_signal_fence(memory_order order) noexcept;
```

6 *Effects:* Equivalent to `atomic_thread_fence(order)`, except that the resulting ordering constraints are established only between a thread and a signal handler executed in the same thread.

7 *Note:* `atomic_signal_fence` can be used to specify the order in which actions performed by the thread become visible to the signal handler.

8 *Note:* compiler optimizations and reorderings of loads and stores are inhibited in the same way as with `atomic_thread_fence`, but the hardware fence instructions that `atomic_thread_fence` would have inserted are not emitted.

30 Thread support library [thread]

30.1 General [thread.general]

1 The following subclauses describe components to create and manage threads (1.10), perform mutual exclusion, and communicate conditions and values between threads, as summarized in Table 148.

Table 148 — Thread support library summary

	Subclause	Header(s)
30.2	Requirements	
30.3	Threads	`<thread>`
30.4	Mutual exclusion	`<mutex>` `<shared_mutex>`
30.5	Condition variables	`<condition_variable>`
30.6	Futures	`<future>`

30.2 Requirements [thread.req]

30.2.1 Template parameter names [thread.req.paramname]

1 Throughout this Clause, the names of template parameters are used to express type requirements. If a template parameter is named `Predicate`, `operator()` applied to the template argument shall return a value that is convertible to `bool`.

30.2.2 Exceptions [thread.req.exception]

1 Some functions described in this Clause are specified to throw exceptions of type `system_error` (19.5.6). Such exceptions shall be thrown if any of the function's error conditions is detected or a call to an operating system or other underlying API results in an error that prevents the library function from meeting its specifications. Failure to allocate storage shall be reported as described in 17.6.5.12.

[*Example:* Consider a function in this clause that is specified to throw exceptions of type `system_error` and specifies error conditions that include `operation_not_permitted` for a thread that does not have the privilege to perform the operation. Assume that, during the execution of this function, an `errno` of `EPERM` is reported by a POSIX API call used by the implementation. Since POSIX specifies an `errno` of `EPERM` when "the caller does not have the privilege to perform the operation", the implementation maps `EPERM` to an `error_condition` of `operation_not_permitted` (19.5) and an exception of type `system_error` is thrown. — *end example*]

2 The `error_code` reported by such an exception's `code()` member function shall compare equal to one of the conditions specified in the function's error condition element.

30.2.3 Native handles [thread.req.native]

1 Several classes described in this Clause have members `native_handle_type` and `native_handle`. The presence of these members and their semantics is implementation-defined. [*Note:* These members allow implementations to provide access to implementation details. Their names are specified to facilitate portable compile-time detection. Actual use of these members is inherently non-portable. — *end note*]

30.2.4 Timing specifications [thread.req.timing]

1 Several functions described in this Clause take an argument to specify a timeout. These timeouts are specified as either a `duration` or a `time_point` type as specified in 20.12.

2 Implementations necessarily have some delay in returning from a timeout. Any overhead in interrupt response, function return, and scheduling induces a "quality of implementation" delay, expressed as duration D_i. Ideally, this delay would be zero. Further, any contention for processor and memory resources induces a "quality of management" delay, expressed as duration D_m. The delay durations may vary from timeout to timeout, but in all cases shorter is better.

3 The member functions whose names end in `_for` take an argument that specifies a duration. These functions produce relative timeouts. Implementations should use a steady clock to measure time for these functions.[338] Given a duration argument D_t, the real-time duration of the timeout is $D_t + D_i + D_m$.

4 The member functions whose names end in `_until` take an argument that specifies a time point. These functions produce absolute timeouts. Implementations should use the clock specified in the time point to measure time for these functions. Given a clock time point argument C_t, the clock time point of the return from timeout should be $C_t + D_i + D_m$ when the clock is not adjusted during the timeout. If the clock is adjusted to the time C_a during the timeout, the behavior should be as follows:

(4.1) — if $C_a > C_t$, the waiting function should wake as soon as possible, i.e. $C_a + D_i + D_m$, since the timeout is already satisfied. [*Note:* This specification may result in the total duration of the wait decreasing when measured against a steady clock. — *end note*]

(4.2) — if $C_a <= C_t$, the waiting function should not time out until `Clock::now()` returns a time $C_n >= C_t$, i.e. waking at $C_t + D_i + D_m$. [*Note:* When the clock is adjusted backwards, this specification may result in the total duration of the wait increasing when measured against a steady clock. When the clock is adjusted forwards, this specification may result in the total duration of the wait decreasing when measured against a steady clock. — *end note*]

An implementation shall return from such a timeout at any point from the time specified above to the time it would return from a steady-clock relative timeout on the difference between C_t and the time point of the call to the `_until` function. [*Note:* Implementations should decrease the duration of the wait when the clock is adjusted forwards. — *end note*]

5 [*Note:* If the clock is not synchronized with a steady clock, e.g., a CPU time clock, these timeouts might not provide useful functionality. — *end note*]

6 The resolution of timing provided by an implementation depends on both operating system and hardware. The finest resolution provided by an implementation is called the *native resolution*.

7 Implementation-provided clocks that are used for these functions shall meet the `TrivialClock` requirements (20.12.3).

8 A function that takes an argument which specifies a timeout will throw if, during its execution, a clock, time point, or time duration throws an exception. Such exceptions are referred to as *timeout-related exceptions*. [*Note:* instantiations of clock, time point and duration types supplied by the implementation as specified in 20.12.7 do not throw exceptions. — *end note*]

30.2.5 Requirements for Lockable types [thread.req.lockable]

30.2.5.1 In general [thread.req.lockable.general]

1 An *execution agent* is an entity such as a thread that may perform work in parallel with other execution agents. [*Note:* Implementations or users may introduce other kinds of agents such as processes or thread-

338) All implementations for which standard time units are meaningful must necessarily have a steady clock within their hardware implementation.

pool tasks. — *end note*] The calling agent is determined by context, e.g. the calling thread that contains the call, and so on.

2 [*Note:* Some lockable objects are "agent oblivious" in that they work for any execution agent model because they do not determine or store the agent's ID (e.g., an ordinary spin lock). — *end note*]

3 The standard library templates `unique_lock` (30.4.2.2), `lock_guard` (30.4.2.1), `lock`, `try_lock` (30.4.3), and `condition_variable_any` (30.5.2) all operate on user-supplied lockable objects. The `BasicLockable` requirements, the `Lockable` requirements, and the `TimedLockable` requirements list the requirements imposed by these library types in order to acquire or release ownership of a `lock` by a given execution agent. [*Note:* The nature of any lock ownership and any synchronization it may entail are not part of these requirements. — *end note*]

30.2.5.2 `BasicLockable` requirements [thread.req.lockable.basic]

1 A type `L` meets the `BasicLockable` requirements if the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`).

`m.lock()`

2 *Effects:* Blocks until a lock can be acquired for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

`m.unlock()`

3 *Requires:* The current execution agent shall hold a lock on `m`.

4 *Effects:* Releases a lock on `m` held by the current execution agent.

5 *Throws:* Nothing.

30.2.5.3 `Lockable` requirements [thread.req.lockable.req]

1 A type `L` meets the `Lockable` requirements if it meets the `BasicLockable` requirements and the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`).

`m.try_lock()`

2 *Effects:* attempts to acquire a lock for the current execution agent without blocking. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

3 *Return type:* `bool`.

4 *Returns:* `true` if the lock was acquired, `false` otherwise.

30.2.5.4 `TimedLockable` requirements [thread.req.lockable.timed]

1 A type `L` meets the `TimedLockable` requirements if it meets the `Lockable` requirements and the following expressions are well-formed and have the specified semantics (`m` denotes a value of type `L`, `rel_time` denotes a value of an instantiation of `duration` (20.12.5), and `abs_time` denotes a value of an instantiation of `time_point` (20.12.6)).

`m.try_lock_for(rel_time)`

2 *Effects:* attempts to acquire a lock for the current execution agent within the relative timeout (30.2.4) specified by `rel_time`. The function shall not return within the timeout specified by `rel_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

3 *Return type:* `bool`.

4 *Returns:* `true` if the lock was acquired, `false` otherwise.

`m.try_lock_until(abs_time)`

5 *Effects:* attempts to acquire a lock for the current execution agent before the absolute timeout (30.2.4) specified by `abs_time`. The function shall not return before the timeout specified by `abs_time` unless it has obtained a lock on `m` for the current execution agent. If an exception is thrown then a lock shall not have been acquired for the current execution agent.

6 *Return type:* `bool`.

7 *Returns:* `true` if the lock was acquired, `false` otherwise.

30.2.6 `decay_copy` [thread.decaycopy]

1 In several places in this Clause the operation *`DECAY_COPY`*`(x)` is used. All such uses mean call the function `decay_copy(x)` and use the result, where `decay_copy` is defined as follows:

```
template <class T> decay_t<T> decay_copy(T&& v)
  { return std::forward<T>(v); }
```

30.3 Threads [thread.threads]

1 30.3 describes components that can be used to create and manage threads. [*Note:* These threads are intended to map one-to-one with operating system threads. — *end note*]

Header `<thread>` synopsis

```
namespace std {
  class thread;

  void swap(thread& x, thread& y) noexcept;

  namespace this_thread {
    thread::id get_id() noexcept;

    void yield() noexcept;
    template <class Clock, class Duration>
      void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
    template <class Rep, class Period>
      void sleep_for(const chrono::duration<Rep, Period>& rel_time);
  }
}
```

30.3.1 Class `thread` [thread.thread.class]

1 The class `thread` provides a mechanism to create a new thread of execution, to join with a thread (i.e., wait for a thread to complete), and to perform other operations that manage and query the state of a thread. A `thread` object uniquely represents a particular thread of execution. That representation may be transferred to other `thread` objects in such a way that no two `thread` objects simultaneously represent the same thread of execution. A thread of execution is *detached* when no `thread` object represents that thread. Objects of class `thread` can be in a state that does not represent a thread of execution. [*Note:* A `thread` object does not represent a thread of execution after default construction, after being moved from, or after a successful call to `detach` or `join`. — *end note*]

```
namespace std {
  class thread {
  public:
    // types:
    class id;
```

```
    typedef implementation-defined native_handle_type; // See 30.2.3

    // construct/copy/destroy:
    thread() noexcept;
    template <class F, class ...Args> explicit thread(F&& f, Args&&... args);
    ~thread();
    thread(const thread&) = delete;
    thread(thread&&) noexcept;
    thread& operator=(const thread&) = delete;
    thread& operator=(thread&&) noexcept;

    // members:
    void swap(thread&) noexcept;
    bool joinable() const noexcept;
    void join();
    void detach();
    id get_id() const noexcept;
    native_handle_type native_handle(); // See 30.2.3

    // static members:
    static unsigned hardware_concurrency() noexcept;
  };
}
```

30.3.1.1 Class `thread::id` [thread.thread.id]

```
namespace std {
  class thread::id {
  public:
      id() noexcept;
  };

  bool operator==(thread::id x, thread::id y) noexcept;
  bool operator!=(thread::id x, thread::id y) noexcept;
  bool operator<(thread::id x, thread::id y) noexcept;
  bool operator<=(thread::id x, thread::id y) noexcept;
  bool operator>(thread::id x, thread::id y) noexcept;
  bool operator>=(thread::id x, thread::id y) noexcept;

  template<class charT, class traits>
    basic_ostream<charT, traits>&
      operator<< (basic_ostream<charT, traits>& out, thread::id id);

  // Hash support
  template <class T> struct hash;
  template <> struct hash<thread::id>;
}
```

1 An object of type `thread::id` provides a unique identifier for each thread of execution and a single distinct value for all `thread` objects that do not represent a thread of execution (30.3.1). Each thread of execution has an associated `thread::id` object that is not equal to the `thread::id` object of any other thread of execution and that is not equal to the `thread::id` object of any `std::thread` object that does not represent threads of execution.

2 `thread::id` shall be a trivially copyable class (Clause 9). The library may reuse the value of a `thread::id` of a terminated thread that can no longer be joined.

3 [*Note:* Relational operators allow `thread::id` objects to be used as keys in associative containers. — *end note*]

```
id() noexcept;
```

4 *Effects:* Constructs an object of type `id`.

5 *Postconditions:* The constructed object does not represent a thread of execution.

```
bool operator==(thread::id x, thread::id y) noexcept;
```

6 *Returns:* `true` only if `x` and `y` represent the same thread of execution or neither `x` nor `y` represents a thread of execution.

```
bool operator!=(thread::id x, thread::id y) noexcept;
```

7 *Returns:* `!(x == y)`

```
bool operator<(thread::id x, thread::id y) noexcept;
```

8 *Returns:* A value such that `operator<` is a total ordering as described in 25.4.

```
bool operator<=(thread::id x, thread::id y) noexcept;
```

9 *Returns:* `!(y < x)`

```
bool operator>(thread::id x, thread::id y) noexcept;
```

10 *Returns:* `y < x`

```
bool operator>=(thread::id x, thread::id y) noexcept;
```

11 *Returns:* `!(x < y)`

```
template<class charT, class traits>
  basic_ostream<charT, traits>&
    operator<< (basic_ostream<charT, traits>&& out, thread::id id);
```

12 *Effects:* Inserts an unspecified text representation of `id` into `out`. For two objects of type `thread::id` `x` and `y`, if `x == y` the `thread::id` objects shall have the same text representation and if `x != y` the `thread::id` objects shall have distinct text representations.

13 *Returns:* `out`

```
template <> struct hash<thread::id>;
```

14 The template specialization shall meet the requirements of class template `hash` (20.9.13).

30.3.1.2 `thread` constructors [thread.thread.constr]

```
thread() noexcept;
```

1 *Effects:* Constructs a `thread` object that does not represent a thread of execution.

2 *Postcondition:* `get_id() == id()`

```
template <class F, class ...Args> explicit thread(F&& f, Args&&... args);
```

3 *Requires:* `F` and each `Ti` in `Args` shall satisfy the `MoveConstructible` requirements. *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) (20.9.2) shall be a valid expression.

4 *Remarks:* This constructor shall not participate in overload resolution if `decay_t<F>` is the same type as `std::thread`.

5 *Effects:* Constructs an object of type `thread`. The new thread of execution executes *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) with the calls to *DECAY_COPY* being evaluated in the constructing thread. Any return value from this invocation is ignored. [*Note:* This implies that any exceptions not thrown from the invocation of the copy of `f` will be thrown in the constructing thread, not the new thread. — *end note*] If the invocation of *INVOKE*(*DECAY_COPY*(`std::forward<F>(f)`), *DECAY_COPY*(`std::forward<Args>(args)`)...) terminates with an uncaught exception, `std::terminate` shall be called.

6 *Synchronization:* The completion of the invocation of the constructor synchronizes with the beginning of the invocation of the copy of `f`.

7 *Postconditions:* `get_id() != id()`. `*this` represents the newly started thread.

8 *Throws:* `system_error` if unable to start the new thread.

9 *Error conditions:*

(9.1) — `resource_unavailable_try_again` — the system lacked the necessary resources to create another thread, or the system-imposed limit on the number of threads in a process would be exceeded.

```
thread(thread&& x) noexcept;
```

10 *Effects:* Constructs an object of type `thread` from `x`, and sets `x` to a default constructed state.

11 *Postconditions:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the start of construction.

30.3.1.3 `thread` destructor [thread.thread.destr]

```
~thread();
```

1 If `joinable()`, calls `std::terminate()`. Otherwise, has no effects. [*Note:* Either implicitly detaching or joining a `joinable()` thread in its destructor could result in difficult to debug correctness (for detach) or performance (for join) bugs encountered only when an exception is raised. Thus the programmer must ensure that the destructor is never executed while the thread is still joinable. — *end note*]

30.3.1.4 `thread` assignment [thread.thread.assign]

```
thread& operator=(thread&& x) noexcept;
```

1 *Effects:* If `joinable()`, calls `std::terminate()`. Otherwise, assigns the state of `x` to `*this` and sets `x` to a default constructed state.

2 *Postconditions:* `x.get_id() == id()` and `get_id()` returns the value of `x.get_id()` prior to the assignment.

3 *Returns:* `*this`

30.3.1.5 `thread` members [thread.thread.member]

```
void swap(thread& x) noexcept;
```

1 *Effects:* Swaps the state of `*this` and `x`.

```
bool joinable() const noexcept;
```

2 *Returns:* `get_id() != id()`

```
void join();
```

3 *Requires:* `joinable()` is `true`.

4 *Effects:* Blocks until the thread represented by `*this` has completed.

5 *Synchronization:* The completion of the thread represented by `*this` synchronizes with (1.10) the corresponding successful `join()` return. [*Note:* Operations on `*this` are not synchronized. — *end note*]

6 *Postconditions:* The thread represented by `*this` has completed. `get_id() == id()`.

7 *Throws:* `system_error` when an exception is required (30.2.2).

8 *Error conditions:*

(8.1) — `resource_deadlock_would_occur` — if deadlock is detected or `this->get_id() == std::this_thread::get_id()`.

(8.2) — `no_such_process` — if the thread is not valid.

(8.3) — `invalid_argument` — if the thread is not joinable.

```
void detach();
```

9 *Requires:* `joinable()` is `true`.

10 *Effects:* The thread represented by `*this` continues execution without the calling thread blocking. When `detach()` returns, `*this` no longer represents the possibly continuing thread of execution. When the thread previously represented by `*this` ends execution, the implementation shall release any owned resources.

11 *Postcondition:* `get_id() == id()`.

12 *Throws:* `system_error` when an exception is required (30.2.2).

13 *Error conditions:*

(13.1) — `no_such_process` — if the thread is not valid.

(13.2) — `invalid_argument` — if the thread is not joinable.

```
id get_id() const noexcept;
```

14 *Returns:* A default constructed `id` object if `*this` does not represent a thread, otherwise `this_thread::get_id()` for the thread of execution represented by `*this`.

30.3.1.6 `thread` static members [thread.thread.static]

```
unsigned hardware_concurrency() noexcept;
```

1 *Returns:* The number of hardware thread contexts. [*Note:* This value should only be considered to be a hint. — *end note*] If this value is not computable or well defined an implementation should return 0.

30.3.1.7 `thread` specialized algorithms [thread.thread.algorithm]

```
void swap(thread& x, thread& y) noexcept;
```

1 *Effects:* `x.swap(y)`

30.3.2 Namespace `this_thread` [thread.thread.this]

```
namespace std::this_thread {
  thread::id get_id() noexcept;

  void yield() noexcept;
  template <class Clock, class Duration>
    void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
  template <class Rep, class Period>
    void sleep_for(const chrono::duration<Rep, Period>& rel_time);
}
```

```
thread::id this_thread::get_id() noexcept;
```

1 *Returns:* An object of type `thread::id` that uniquely identifies the current thread of execution. No other thread of execution shall have this id and this thread of execution shall always have this id. The object returned shall not compare equal to a default constructed `thread::id`.

```
void this_thread::yield() noexcept;
```

2 *Effects:* Offers the implementation the opportunity to reschedule.

3 *Synchronization:* None.

```
template <class Clock, class Duration>
  void sleep_until(const chrono::time_point<Clock, Duration>& abs_time);
```

4 *Effects:* Blocks the calling thread for the absolute timeout (30.2.4) specified by `abs_time`.

5 *Synchronization:* None.

6 *Throws:* Timeout-related exceptions (30.2.4).

```
template <class Rep, class Period>
  void sleep_for(const chrono::duration<Rep, Period>& rel_time);
```

7 *Effects:* Blocks the calling thread for the relative timeout (30.2.4) specified by `rel_time`.

8 *Synchronization:* None.

9 *Throws:* Timeout-related exceptions (30.2.4).

30.4 Mutual exclusion [thread.mutex]

1 This section provides mechanisms for mutual exclusion: mutexes, locks, and call once. These mechanisms ease the production of race-free programs (1.10).

Header `<mutex>` synopsis

```
namespace std {
  class mutex;
  class recursive_mutex;
  class timed_mutex;
  class recursive_timed_mutex;

  struct defer_lock_t { };
  struct try_to_lock_t { };
  struct adopt_lock_t { };

  constexpr defer_lock_t  defer_lock { };
  constexpr try_to_lock_t try_to_lock { };
  constexpr adopt_lock_t  adopt_lock { };
```

```
  template <class Mutex> class lock_guard;
  template <class Mutex> class unique_lock;

  template <class Mutex>
    void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y) noexcept;

  template <class L1, class L2, class... L3> int try_lock(L1&, L2&, L3&...);
  template <class L1, class L2, class... L3> void lock(L1&, L2&, L3&...);

  struct once_flag {
    constexpr once_flag() noexcept;

    once_flag(const once_flag&) = delete;
    once_flag& operator=(const once_flag&) = delete;
  };

  template<class Callable, class ...Args>
    void call_once(once_flag& flag, Callable&& func, Args&&... args);
}
```

Header <shared_mutex> synopsis

```
namespace std {
  class shared_mutex;
  class shared_timed_mutex;
  template <class Mutex> class shared_lock;
  template <class Mutex>
    void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y) noexcept;
}
```

30.4.1 Mutex requirements [thread.mutex.requirements]

30.4.1.1 In general [thread.mutex.requirements.general]

1 A mutex object facilitates protection against data races and allows safe synchronization of data between execution agents (30.2.5). An execution agent *owns* a mutex from the time it successfully calls one of the lock functions until it calls unlock. Mutexes can be either recursive or non-recursive, and can grant simultaneous ownership to one or many execution agents. Both recursive and non-recursive mutexes are supplied.

30.4.1.2 Mutex types [thread.mutex.requirements.mutex]

1 The *mutex types* are the standard library types `std::mutex`, `std::recursive_mutex`, `std::timed_mutex`, `std::recursive_timed_mutex`, `std::shared_mutex`, and `std::shared_timed_mutex`. They shall meet the requirements set out in this section. In this description, `m` denotes an object of a mutex type.

2 The mutex types shall meet the `Lockable` requirements (30.2.5.3).

3 The mutex types shall be `DefaultConstructible` and `Destructible`. If initialization of an object of a mutex type fails, an exception of type `system_error` shall be thrown. The mutex types shall not be copyable or movable.

4 The error conditions for error codes, if any, reported by member functions of the mutex types shall be:

(4.1) — `resource_unavailable_try_again` — if any native handle type manipulated is not available.

(4.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(4.3) — `device_or_resource_busy` — if any native handle type manipulated is already locked.

(4.4) — `invalid_argument` — if any native handle type manipulated as part of mutex construction is incorrect.

5 The implementation shall provide lock and unlock operations, as described below. For purposes of determining the existence of a data race, these behave as atomic operations (1.10). The lock and unlock operations on a single mutex shall appear to occur in a single total order. [*Note:* this can be viewed as the modification order (1.10) of the mutex. — *end note*] [*Note:* Construction and destruction of an object of a mutex type need not be thread-safe; other synchronization should be used to ensure that mutex objects are initialized and visible to other threads. — *end note*]

6 The expression `m.lock()` shall be well-formed and have the following semantics:

7 *Requires:* If `m` is of type `std::mutex`, `std::timed_mutex`, `std::shared_mutex`, or `std::shared_timed_mutex`, the calling thread does not own the mutex.

8 *Effects:* Blocks the calling thread until ownership of the mutex can be obtained for the calling thread.

9 *Postcondition:* The calling thread owns the mutex.

10 *Return type:* `void`

11 *Synchronization:* Prior `unlock()` operations on the same object shall *synchronize with* (1.10) this operation.

12 *Throws:* `system_error` when an exception is required (30.2.2).

13 *Error conditions:*

(13.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(13.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

(13.3) — `device_or_resource_busy` — if the mutex is already locked and blocking is not possible.

14 The expression `m.try_lock()` shall be well-formed and have the following semantics:

15 *Requires:* If `m` is of type `std::mutex`, `std::timed_mutex`, `std::shared_mutex`, or `std::shared_timed_mutex`, the calling thread does not own the mutex.

16 *Effects:* Attempts to obtain ownership of the mutex for the calling thread without blocking. If ownership is not obtained, there is no effect and `try_lock()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread. [*Note:* This spurious failure is normally uncommon, but allows interesting implementations based on a simple compare and exchange (Clause 29). — *end note*] An implementation should ensure that `try_lock()` does not consistently return `false` in the absence of contending mutex acquisitions.

17 *Return type:* `bool`

18 *Returns:* `true` if ownership of the mutex was obtained for the calling thread, otherwise `false`.

19 *Synchronization:* If `try_lock()` returns `true`, prior `unlock()` operations on the same object *synchronize with* (1.10) this operation. [*Note:* Since `lock()` does not synchronize with a failed subsequent `try_lock()`, the visibility rules are weak enough that little would be known about the state after a failure, even in the absence of spurious failures. — *end note*]

20 *Throws:* Nothing.

21 The expression `m.unlock()` shall be well-formed and have the following semantics:

22 *Requires:* The calling thread shall own the mutex.

23 *Effects:* Releases the calling thread's ownership of the mutex.

24 *Return type:* `void`

25 *Synchronization:* This operation synchronizes with (1.10) subsequent lock operations that obtain ownership on the same object.

26 *Throws:* Nothing.

30.4.1.2.1 Class `mutex` [thread.mutex.class]

```
namespace std {
  class mutex {
  public:
    constexpr mutex() noexcept;
    ~mutex();

    mutex(const mutex&) = delete;
    mutex& operator=(const mutex&) = delete;

    void lock();
    bool try_lock();
    void unlock();

    typedef implementation-defined native_handle_type; // See 30.2.3
    native_handle_type native_handle();                // See 30.2.3
  };
}
```

1 The class `mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a mutex object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the owning thread has released ownership with a call to `unlock()`.

2 [*Note:* After a thread `A` has called `unlock()`, releasing a mutex, it is possible for another thread `B` to lock the same mutex, observe that it is no longer in use, unlock it, and destroy it, before thread `A` appears to have returned from its unlock call. Implementations are required to handle such scenarios correctly, as long as thread `A` doesn't access the mutex after the unlock call returns. These cases typically occur when a reference-counted object contains a mutex that is used to protect the reference count. — *end note*]

3 The class `mutex` shall satisfy all the `Mutex` requirements (30.4.1). It shall be a standard-layout class (Clause 9).

4 [*Note:* A program may deadlock if the thread that owns a `mutex` object calls `lock()` on that object. If the implementation can detect the deadlock, a `resource_deadlock_would_occur` error condition may be observed. — *end note*]

5 The behavior of a program is undefined if it destroys a `mutex` object owned by any thread or a thread terminates while owning a `mutex` object.

30.4.1.2.2 Class `recursive_mutex` [thread.mutex.recursive]

```
namespace std {
  class recursive_mutex {
  public:
    recursive_mutex();
    ~recursive_mutex();

    recursive_mutex(const recursive_mutex&) = delete;
    recursive_mutex& operator=(const recursive_mutex&) = delete;

    void lock();
    bool try_lock() noexcept;
    void unlock();
```

```
    typedef implementation-defined native_handle_type; // See 30.2.3
    native_handle_type native_handle();                // See 30.2.3
  };
}
```

1 The class `recursive_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`) until the first thread has completely released ownership.

2 The class `recursive_mutex` shall satisfy all the Mutex requirements (30.4.1). It shall be a standard-layout class (Clause 9).

3 A thread that owns a `recursive_mutex` object may acquire additional levels of ownership by calling `lock()` or `try_lock()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_mutex` object, additional calls to `try_lock()` shall fail, and additional calls to `lock()` shall throw an exception of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()` and `try_lock()`. Only when all levels of ownership have been released may ownership be acquired by another thread.

4 The behavior of a program is undefined if:

(4.1) — it destroys a `recursive_mutex` object owned by any thread or

(4.2) — a thread terminates while owning a `recursive_mutex` object.

30.4.1.3 Timed mutex types [thread.timedmutex.requirements]

1 The *timed mutex types* are the standard library types `std::timed_mutex`, `std::recursive_timed_mutex`, and `std::shared_timed_mutex`. They shall meet the requirements set out below. In this description, `m` denotes an object of a mutex type, `rel_time` denotes an object of an instantiation of `duration` (20.12.5), and `abs_time` denotes an object of an instantiation of `time_point` (20.12.6).

2 The timed mutex types shall meet the `TimedLockable` requirements (30.2.5.4).

3 The expression `m.try_lock_for(rel_time)` shall be well-formed and have the following semantics:

4 *Requires:* If `m` is of type `std::timed_mutex` or `std::shared_timed_mutex`, the calling thread does not own the mutex.

5 *Effects:* The function attempts to obtain ownership of the mutex within the relative timeout (30.2.4) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function shall return within the timeout specified by `rel_time` only if it has obtained ownership of the mutex object. [*Note:* As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

6 *Return type:* `bool`

7 *Returns:* `true` if ownership was obtained, otherwise `false`.

8 *Synchronization:* If `try_lock_for()` returns `true`, prior `unlock()` operations on the same object *synchronize with* (1.10) this operation.

9 *Throws:* Timeout-related exceptions (30.2.4).

10 The expression `m.try_lock_until(abs_time)` shall be well-formed and have the following semantics:

11 *Requires:* If `m` is of type `std::timed_mutex` or `std::shared_timed_mutex`, the calling thread does not own the mutex.

12 *Effects:* The function attempts to obtain ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain ownership without blocking (as if by calling `try_lock()`). The function shall return before the absolute timeout (30.2.4) specified by `abs_time` only if it has obtained ownership of the mutex object. [*Note:* As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*]

13 *Return type:* `bool`

14 *Returns:* `true` if ownership was obtained, otherwise `false`.

15 *Synchronization:* If `try_lock_until()` returns `true`, prior `unlock()` operations on the same object *synchronize with* (1.10) this operation.

16 *Throws:* Timeout-related exceptions (30.2.4).

30.4.1.3.1 Class `timed_mutex` [thread.timedmutex.class]

```
namespace std {
  class timed_mutex {
  public:
    timed_mutex();
    ~timed_mutex();

    timed_mutex(const timed_mutex&) = delete;
    timed_mutex& operator=(const timed_mutex&) = delete;

    void lock();  // blocking
    bool try_lock();
    template <class Rep, class Period>
      bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
      bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
    void unlock();

    typedef implementation-defined native_handle_type; // See 30.2.3
    native_handle_type native_handle();                // See 30.2.3
  };
}
```

1 The class `timed_mutex` provides a non-recursive mutex with exclusive ownership semantics. If one thread owns a `timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has released ownership with a call to `unlock()` or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).

2 The class `timed_mutex` shall satisfy all of the `TimedMutex` requirements (30.4.1.3). It shall be a standard-layout class (Clause 9).

3 The behavior of a program is undefined if:

(3.1) — it destroys a `timed_mutex` object owned by any thread,

(3.2) — a thread that owns a `timed_mutex` object calls `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object, or

(3.3) — a thread terminates while owning a `timed_mutex` object.

30.4.1.3.2 Class `recursive_timed_mutex` [thread.timedmutex.recursive]

```
namespace std {
  class recursive_timed_mutex {
  public:
    recursive_timed_mutex();
    ~recursive_timed_mutex();

    recursive_timed_mutex(const recursive_timed_mutex&) = delete;
    recursive_timed_mutex& operator=(const recursive_timed_mutex&) = delete;

    void lock();  // blocking
    bool try_lock() noexcept;
    template <class Rep, class Period>
      bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
      bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
    void unlock();

    typedef implementation-defined native_handle_type; // See 30.2.3
    native_handle_type native_handle();                // See 30.2.3
  };
}
```

1 The class `recursive_timed_mutex` provides a recursive mutex with exclusive ownership semantics. If one thread owns a `recursive_timed_mutex` object, attempts by another thread to acquire ownership of that object will fail (for `try_lock()`) or block (for `lock()`, `try_lock_for()`, and `try_lock_until()`) until the owning thread has completely released ownership or the call to `try_lock_for()` or `try_lock_until()` times out (having failed to obtain ownership).

2 The class `recursive_timed_mutex` shall satisfy all of the `TimedMutex` requirements (30.4.1.3). It shall be a standard-layout class (Clause 9).

3 A thread that owns a `recursive_timed_mutex` object may acquire additional levels of ownership by calling `lock()`, `try_lock()`, `try_lock_for()`, or `try_lock_until()` on that object. It is unspecified how many levels of ownership may be acquired by a single thread. If a thread has already acquired the maximum level of ownership for a `recursive_timed_mutex` object, additional calls to `try_lock()`, `try_lock_for()`, or `try_lock_until()` shall fail, and additional calls to `lock()` shall throw an exception of type `system_error`. A thread shall call `unlock()` once for each level of ownership acquired by calls to `lock()`, `try_lock()`, `try_lock_for()`, and `try_lock_until()`. Only when all levels of ownership have been released may ownership of the object be acquired by another thread.

4 The behavior of a program is undefined if:

(4.1) — it destroys a `recursive_timed_mutex` object owned by any thread, or

(4.2) — a thread terminates while owning a `recursive_timed_mutex` object.

30.4.1.4 Shared mutex types [thread.sharedmutex.requirements]

1 The standard library types `std::shared_mutex` and `std::shared_timed_mutex` are *shared mutex types*. Shared mutex types shall meet the requirements of mutex types (30.4.1.2), and additionally shall meet the requirements set out below. In this description, `m` denotes an object of a shared mutex type.

2 In addition to the exclusive lock ownership mode specified in 30.4.1.2, shared mutex types provide a *shared lock* ownership mode. Multiple execution agents can simultaneously hold a shared lock ownership of a shared mutex type. But no execution agent shall hold a shared lock while another execution agent holds an exclusive lock on the same shared mutex type, and vice-versa. The maximum number of execution agents which can

share a shared lock on a single shared mutex type is unspecified, but shall be at least 10000. If more than the maximum number of execution agents attempt to obtain a shared lock, the excess execution agents shall block until the number of shared locks are reduced below the maximum amount by other execution agents releasing their shared lock.

3 The expression `m.lock_shared()` shall be well-formed and have the following semantics:

4 *Requires:* The calling thread has no ownership of the mutex.

5 *Effects:* Blocks the calling thread until shared ownership of the mutex can be obtained for the calling thread. If an exception is thrown then a shared lock shall not have been acquired for the current thread.

6 *Postcondition:* The calling thread has a shared lock on the mutex.

7 *Return type:* `void`.

8 *Synchronization:* Prior `unlock()` operations on the same object shall synchronize with (1.10) this operation.

9 *Throws:* `system_error` when an exception is required (30.2.2).

10 *Error conditions:*

(10.1) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

(10.2) — `resource_deadlock_would_occur` — if the implementation detects that a deadlock would occur.

(10.3) — `device_or_resource_busy` — if the mutex is already locked and blocking is not possible.

11 The expression `m.unlock_shared()` shall be well-formed and have the following semantics:

12 *Requires:* The calling thread shall hold a shared lock on the mutex.

13 *Effects:* Releases a shared lock on the mutex held by the calling thread.

14 *Return type:* `void`.

15 *Synchronization:* This operation synchronizes with (1.10) subsequent `lock()` operations that obtain ownership on the same object.

16 *Throws:* Nothing.

17 The expression `m.try_lock_shared()` shall be well-formed and have the following semantics:

18 *Requires:* The calling thread has no ownership of the mutex.

19 *Effects:* Attempts to obtain shared ownership of the mutex for the calling thread without blocking. If shared ownership is not obtained, there is no effect and `try_lock_shared()` immediately returns. An implementation may fail to obtain the lock even if it is not held by any other thread.

20 *Return type:* `bool`.

21 *Returns:* `true` if the shared ownership lock was acquired, `false` otherwise.

22 *Synchronization:* If `try_lock_shared()` returns `true`, prior `unlock()` operations on the same object synchronize with (1.10) this operation.

23 *Throws:* Nothing.

30.4.1.4.1 Class shared_mutex [thread.sharedmutex.class]

```
namespace std {
  class shared_mutex {
  public:
     shared_mutex();
     ~shared_mutex();

     shared_mutex(const shared_mutex&) = delete;
     shared_mutex& operator=(const shared_mutex&) = delete;

     // Exclusive ownership
     void lock(); // blocking
     bool try_lock();
     void unlock();

     // Shared ownership
     void lock_shared(); // blocking
     bool try_lock_shared();
     void unlock_shared();

     typedef implementation-defined native_handle_type; // See 30.2.3
     native_handle_type native_handle(); // See 30.2.3
  };
}
```

1 The class `shared_mutex` provides a non-recursive mutex with shared ownership semantics.

2 The class `shared_mutex` shall satisfy all of the requirements for shared mutexes (30.4.1.4). It shall be a standard-layout class (Clause 9).

3 The behavior of a program is undefined if:

(3.1) — it destroys a `shared_mutex` object owned by any thread,

(3.2) — a thread attempts to recursively gain any ownership of a `shared_mutex`, or

(3.3) — a thread terminates while possessing any ownership of a `shared_mutex`.

4 `shared_mutex` may be a synonym for `shared_timed_mutex`.

30.4.1.5 Shared timed mutex types [thread.sharedtimedmutex.requirements]

1 The standard library type `std::shared_timed_mutex` is a *shared timed mutex type*. Shared timed mutex types shall meet the requirements of timed mutex types (30.4.1.3), shared mutex types (30.4.1.4), and additionally shall meet the requirements set out below. In this description, `m` denotes an object of a shared timed mutex type, `rel_type` denotes an object of an instantiation of `duration` (20.12.5), and `abs_time` denotes an object of an instantiation of `time_point` (20.12.6).

2 The expression `m.try_lock_shared_for(rel_time)` shall be well-formed and have the following semantics:

3 *Requires:* The calling thread has no ownership of the mutex.

4 *Effects:* Attempts to obtain shared lock ownership for the calling thread within the relative timeout (30.2.4) specified by `rel_time`. If the time specified by `rel_time` is less than or equal to `rel_time.zero()`, the function attempts to obtain ownership without blocking (as if by calling `try_lock_shared()`). The function shall return within the timeout specified by `rel_time` only if it has obtained shared ownership of the mutex object. [*Note:* As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a

strong effort to do so. — *end note*] If an exception is thrown then a shared lock shall not have been acquired for the current thread.

Return type: `bool`.

Returns: `true` if the shared lock was acquired, `false` otherwise.

Synchronization: If `try_lock_shared_for()` returns `true`, prior `unlock()` operations on the same object synchronize with (1.10) this operation.

Throws: Timeout-related exceptions (30.2.4).

The expression `m.try_lock_shared_until(abs_time)` shall be well-formed and have the following semantics:

Requires: The calling thread has no ownership of the mutex.

Effects: The function attempts to obtain shared ownership of the mutex. If `abs_time` has already passed, the function attempts to obtain shared ownership without blocking (as if by calling `try_lock_shared()`). The function shall return before the absolute timeout (30.2.4) specified by `abs_time` only if it has obtained shared ownership of the mutex object. [*Note:* As with `try_lock()`, there is no guarantee that ownership will be obtained if the lock is available, but implementations are expected to make a strong effort to do so. — *end note*] If an exception is thrown then a shared lock shall not have been acquired for the current thread.

Return type: `bool`.

Returns: `true` if the shared lock was acquired, `false` otherwise.

Synchronization: If `try_lock_shared_until()` returns `true`, prior `unlock()` operations on the same object synchronize with (1.10) this operation.

Throws: Timeout-related exceptions (30.2.4).

30.4.1.5.1 Class `shared_timed_mutex` [thread.sharedtimedmutex.class]

```
namespace std {
  class shared_timed_mutex {
  public:
    shared_timed_mutex();
    ~shared_timed_mutex();

    shared_timed_mutex(const shared_timed_mutex&) = delete;
    shared_timed_mutex& operator=(const shared_timed_mutex&) = delete;

    // Exclusive ownership
    void lock();  // blocking
    bool try_lock();
    template <class Rep, class Period>
      bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
      bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
    void unlock();

    // Shared ownership
    void lock_shared();  // blocking
    bool try_lock_shared();
    template <class Rep, class Period>
      bool
      try_lock_shared_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
```

```
    bool
    try_lock_shared_until(const chrono::time_point<Clock, Duration>& abs_time);
  void unlock_shared();
};
}
```

1 The class `shared_timed_mutex` provides a non-recursive mutex with shared ownership semantics.

2 The class `shared_timed_mutex` shall satisfy all of the requirements for shared timed mutexes (30.4.1.5). It shall be a standard-layout class (Clause 9).

3 The behavior of a program is undefined if:

(3.1) — it destroys a `shared_timed_mutex` object owned by any thread,

(3.2) — a thread attempts to recursively gain any ownership of a `shared_timed_mutex`, or

(3.3) — a thread terminates while possessing any ownership of a `shared_timed_mutex`.

30.4.2 Locks [thread.lock]

1 A *lock* is an object that holds a reference to a lockable object and may unlock the lockable object during the lock's destruction (such as when leaving block scope). An execution agent may use a lock to aid in managing ownership of a lockable object in an exception safe manner. A lock is said to *own* a lockable object if it is currently managing the ownership of that lockable object for an execution agent. A lock does not manage the lifetime of the lockable object it references. [*Note:* Locks are intended to ease the burden of unlocking the lockable object under both normal and exceptional circumstances. — *end note*]

2 Some lock constructors take tag types which describe what should be done with the lockable object during the lock's construction.

```
namespace std {
  struct defer_lock_t  { };      // do not acquire ownership of the mutex
  struct try_to_lock_t { };      // try to acquire ownership of the mutex
                                 // without blocking
  struct adopt_lock_t  { };      // assume the calling thread has already
                                 // obtained mutex ownership and manage it

  constexpr defer_lock_t   defer_lock { };
  constexpr try_to_lock_t  try_to_lock { };
  constexpr adopt_lock_t   adopt_lock { };
}
```

30.4.2.1 Class template `lock_guard` [thread.lock.guard]

```
namespace std {
  template <class Mutex>
  class lock_guard {
  public:
    typedef Mutex mutex_type;

    explicit lock_guard(mutex_type& m);
    lock_guard(mutex_type& m, adopt_lock_t);
    ~lock_guard();

    lock_guard(lock_guard const&) = delete;
    lock_guard& operator=(lock_guard const&) = delete;
```

```
  private:
    mutex_type& pm; // exposition only
  };
}
```

1 An object of type `lock_guard` controls the ownership of a lockable object within a scope. A `lock_guard` object maintains ownership of a lockable object throughout the `lock_guard` object's lifetime (3.8). The behavior of a program is undefined if the lockable object referenced by `pm` does not exist for the entire lifetime of the `lock_guard` object. The supplied `Mutex` type shall meet the `BasicLockable` requirements (30.2.5.2).

```
explicit lock_guard(mutex_type& m);
```

2 *Requires:* If `mutex_type` is not a recursive mutex, the calling thread does not own the mutex `m`.

3 *Effects:* `m.lock()`

4 *Postcondition:* `&pm == &m`

```
lock_guard(mutex_type& m, adopt_lock_t);
```

5 *Requires:* The calling thread owns the mutex `m`.

6 *Postcondition:* `&pm == &m`

7 *Throws:* Nothing.

```
~lock_guard();
```

8 *Effects:* `pm.unlock()`

30.4.2.2 Class template `unique_lock` [thread.lock.unique]

```
namespace std {
  template <class Mutex>
  class unique_lock {
  public:
    typedef Mutex mutex_type;

    // 30.4.2.2.1, construct/copy/destroy:
    unique_lock() noexcept;
    explicit unique_lock(mutex_type& m);
    unique_lock(mutex_type& m, defer_lock_t) noexcept;
    unique_lock(mutex_type& m, try_to_lock_t);
    unique_lock(mutex_type& m, adopt_lock_t);
    template <class Clock, class Duration>
      unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);
    template <class Rep, class Period>
      unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
    ~unique_lock();

    unique_lock(unique_lock const&) = delete;
    unique_lock& operator=(unique_lock const&) = delete;

    unique_lock(unique_lock&& u) noexcept;
    unique_lock& operator=(unique_lock&& u);

    // 30.4.2.2.2, locking:
    void lock();
    bool try_lock();
```

```
    template <class Rep, class Period>
      bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
      bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);

    void unlock();

    // 30.4.2.2.3, modifiers:
    void swap(unique_lock& u) noexcept;
    mutex_type* release() noexcept;

    // 30.4.2.2.4, observers:
    bool owns_lock() const noexcept;
    explicit operator bool () const noexcept;
    mutex_type* mutex() const noexcept;

  private:
    mutex_type* pm; // exposition only
    bool owns;      // exposition only
  };

  template <class Mutex>
    void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y) noexcept;
}
```

1 An object of type `unique_lock` controls the ownership of a lockable object within a scope. Ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `unique_lock` object. Objects of type `unique_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime (3.8) of the `unique_lock` object. The supplied `Mutex` type shall meet the `BasicLockable` requirements (30.2.5.2).

2 [*Note:* `unique_lock<Mutex>` meets the `BasicLockable` requirements. If `Mutex` meets the `Lockable` requirements (30.2.5.3), `unique_lock<Mutex>` also meets the `Lockable` requirements; if `Mutex` meets the `TimedLockable` requirements (30.2.5.4), `unique_lock<Mutex>` also meets the `TimedLockable` requirements. — *end note*]

30.4.2.2.1 `unique_lock` constructors, destructor, and assignment [thread.lock.unique.cons]

```
unique_lock() noexcept;
```

1 *Effects:* Constructs an object of type `unique_lock`.

2 *Postconditions:* `pm == 0` and `owns == false`.

```
explicit unique_lock(mutex_type& m);
```

3 *Requires:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

4 *Effects:* Constructs an object of type `unique_lock` and calls `m.lock()`.

5 *Postconditions:* `pm == &m` and `owns == true`.

```
unique_lock(mutex_type& m, defer_lock_t) noexcept;
```

6 *Effects:* Constructs an object of type `unique_lock`.

7 *Postconditions:* `pm == &m` and `owns == false`.

```
unique_lock(mutex_type& m, try_to_lock_t);
```

8 *Requires:* The supplied `Mutex` type shall meet the `Lockable` requirements (30.2.5.3). If `mutex_type` is not a recursive mutex the calling thread does not own the mutex.

9 *Effects:* Constructs an object of type `unique_lock` and calls `m.try_lock()`.

10 *Postconditions:* `pm == &m` and `owns == res`, where `res` is the value returned by the call to `m.try_lock()`.

```
unique_lock(mutex_type& m, adopt_lock_t);
```

11 *Requires:* The calling thread own the mutex.

12 *Effects:* Constructs an object of type `unique_lock`.

13 *Postconditions:* `pm == &m` and `owns == true`.

14 *Throws:* Nothing.

```
template <class Clock, class Duration>
  unique_lock(mutex_type& m, const chrono::time_point<Clock, Duration>& abs_time);
```

15 *Requires:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

16 *Effects:* Constructs an object of type `unique_lock` and calls `m.try_lock_until(abs_time)`.

17 *Postconditions:* `pm == &m` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_until(abs_time)`.

```
template <class Rep, class Period>
  unique_lock(mutex_type& m, const chrono::duration<Rep, Period>& rel_time);
```

18 *Requires:* If `mutex_type` is not a recursive mutex the calling thread does not own the mutex. The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

19 *Effects:* Constructs an object of type `unique_lock` and calls `m.try_lock_for(rel_time)`.

20 *Postconditions:* `pm == &m` and `owns == res`, where `res` is the value returned by the call to `m.try_lock_for(rel_time)`.

```
unique_lock(unique_lock&& u) noexcept;
```

21 *Postconditions:* `pm == u_p.pm` and `owns == u_p.owns` (where `u_p` is the state of `u` just prior to this construction), `u.pm == 0` and `u.owns == false`.

```
unique_lock& operator=(unique_lock&& u);
```

22 *Effects:* If `owns` calls `pm->unlock()`.

23 *Postconditions:* `pm == u_p.pm` and `owns == u_p.owns` (where `u_p` is the state of `u` just prior to this construction), `u.pm == 0` and `u.owns == false`.

24 [*Note:* With a recursive mutex it is possible for both `*this` and `u` to own the same mutex before the assignment. In this case, `*this` will own the mutex after the assignment and `u` will not. — *end note*]

25 *Throws:* Nothing.

```
~unique_lock();
```

26 *Effects:* If `owns` calls `pm->unlock()`.

30.4.2.2.2 `unique_lock` locking [thread.lock.unique.locking]

```
void lock();
```

1 *Effects:* `pm->lock()`

2 *Postcondition:* `owns == true`

3 *Throws:* Any exception thrown by `pm->lock()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is 0. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
bool try_lock();
```

4 *Requires:* The supplied `Mutex` shall meet the `Lockable` requirements (30.2.5.3).

5 *Effects:* `pm->try_lock()`

6 *Returns:* The value returned by the call to `try_lock()`.

7 *Postcondition:* `owns == res`, where `res` is the value returned by the call to `try_lock()`.

8 *Throws:* Any exception thrown by `pm->try_lock()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is 0. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
template <class Clock, class Duration>
  bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
```

9 *Requires:* The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

10 *Effects:* `pm->try_lock_until(abs_time)`

11 *Returns:* The value returned by the call to `try_lock_until(abs_time)`.

12 *Postcondition:* `owns == res`, where `res` is the value returned by the call to `try_lock_until(abs_time)`.

13 *Throws:* Any exception thrown by `pm->try_lock_until()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is 0. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
template <class Rep, class Period>
  bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

14 *Requires:* The supplied `Mutex` type shall meet the `TimedLockable` requirements (30.2.5.4).

15 *Effects:* `pm->try_lock_for(rel_time)`.

16 *Returns:* The value returned by the call to `try_lock_until(rel_time)`.

17 *Postcondition:* `owns == res`, where `res` is the value returned by the call to `try_lock_for(rel_time)`.

18 *Throws:* Any exception thrown by `pm->try_lock_for()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is 0. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
void unlock();
```

19 *Effects:* `pm->unlock()`

20 *Postcondition:* `owns == false`

21 *Throws:* `system_error` when an exception is required (30.2.2).

22 *Error conditions:*

(22.1) — `operation_not_permitted` — if on entry `owns` is false.

30.4.2.2.3 `unique_lock` modifiers [thread.lock.unique.mod]

```
void swap(unique_lock& u) noexcept;
```

1 *Effects:* Swaps the data members of `*this` and `u`.

```
mutex_type* release() noexcept;
```

2 *Returns:* The previous value of `pm`.

3 *Postconditions:* `pm == 0` and `owns == false`.

```
template <class Mutex>
  void swap(unique_lock<Mutex>& x, unique_lock<Mutex>& y) noexcept;
```

4 *Effects:* `x.swap(y)`

30.4.2.2.4 `unique_lock` observers [thread.lock.unique.obs]

```
bool owns_lock() const noexcept;
```

1 *Returns:* `owns`

```
explicit operator bool() const noexcept;
```

2 *Returns:* `owns`

```
mutex_type *mutex() const noexcept;
```

3 *Returns:* `pm`

30.4.2.3 Class template `shared_lock` [thread.lock.shared]

```
namespace std {

template <class Mutex>
class shared_lock {
public:
  typedef Mutex mutex_type;

  // Shared locking
  shared_lock() noexcept;
  explicit shared_lock(mutex_type& m);  // blocking
  shared_lock(mutex_type& m, defer_lock_t) noexcept;
  shared_lock(mutex_type& m, try_to_lock_t);
  shared_lock(mutex_type& m, adopt_lock_t);
  template <class Clock, class Duration>
    shared_lock(mutex_type& m,
                const chrono::time_point<Clock, Duration>& abs_time);
  template <class Rep, class Period>
    shared_lock(mutex_type& m,
                const chrono::duration<Rep, Period>& rel_time);
  ~shared_lock();

  shared_lock(shared_lock const&) = delete;
  shared_lock& operator=(shared_lock const&) = delete;

  shared_lock(shared_lock&& u) noexcept;
  shared_lock& operator=(shared_lock&& u) noexcept;
```

```
    void lock();  // blocking
    bool try_lock();
    template <class Rep, class Period>
      bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
    template <class Clock, class Duration>
      bool try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
    void unlock();

    // Setters
    void swap(shared_lock& u) noexcept;
    mutex_type* release() noexcept;

    // Getters
    bool owns_lock() const noexcept;
    explicit operator bool () const noexcept;
    mutex_type* mutex() const noexcept;

  private:
    mutex_type* pm; // exposition only
    bool owns;      // exposition only
  };

  template <class Mutex>
    void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y) noexcept;

  } // std
```

1 An object of type `shared_lock` controls the shared ownership of a lockable object within a scope. Shared ownership of the lockable object may be acquired at construction or after construction, and may be transferred, after acquisition, to another `shared_lock` object. Objects of type `shared_lock` are not copyable but are movable. The behavior of a program is undefined if the contained pointer `pm` is not null and the lockable object pointed to by `pm` does not exist for the entire remaining lifetime (3.8) of the `shared_lock` object. The supplied `Mutex` type shall meet the shared mutex requirements (30.4.1.5).

2 [*Note:* `shared_lock<Mutex>` meets the `TimedLockable` requirements (30.2.5.4). — *end note*]

30.4.2.3.1 `shared_lock` constructors, destructor, and assignment [thread.lock.shared.cons]

```
shared_lock() noexcept;
```

1 *Effects:* Constructs an object of type `shared_lock`.

2 *Postconditions:* `pm == nullptr` and `owns == false`.

```
explicit shared_lock(mutex_type& m);
```

3 *Requires:* The calling thread does not own the mutex for any ownership mode.

4 *Effects:* Constructs an object of type `shared_lock` and calls `m.lock_shared()`.

5 *Postconditions:* `pm == &m` and `owns == true`.

```
shared_lock(mutex_type& m, defer_lock_t) noexcept;
```

6 *Effects:* Constructs an object of type `shared_lock`.

7 *Postconditions:* `pm == &m` and `owns == false`.

```
shared_lock(mutex_type& m, try_to_lock_t);
```

8 *Requires:* The calling thread does not own the mutex for any ownership mode.

9 *Effects:* Constructs an object of type `shared_lock` and calls `m.try_lock_shared()`.

10 *Postconditions:* `pm == &m` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared()`.

```
shared_lock(mutex_type& m, adopt_lock_t);
```

11 *Requires:* The calling thread has shared ownership of the mutex.

12 *Effects:* Constructs an object of type `shared_lock`.

13 *Postconditions:* `pm == &m` and `owns == true`.

```
template <class Clock, class Duration>
  shared_lock(mutex_type& m,
              const chrono::time_point<Clock, Duration>& abs_time);
```

14 *Requires:* The calling thread does not own the mutex for any ownership mode.

15 *Effects:* Constructs an object of type `shared_lock` and calls `m.try_lock_shared_until(abs_time)`.

16 *Postconditions:* `pm == &m` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_until(abs_time)`.

```
template <class Rep, class Period>
  shared_lock(mutex_type& m,
              const chrono::duration<Rep, Period>& rel_time);
```

17 *Requires:* The calling thread does not own the mutex for any ownership mode.

18 *Effects:* Constructs an object of type `shared_lock` and calls `m.try_lock_shared_for(rel_time)`.

19 *Postconditions:* `pm == &m` and `owns == res` where `res` is the value returned by the call to `m.try_lock_shared_for(rel_time)`.

```
~shared_lock();
```

20 *Effects:* If `owns` calls `pm->unlock_shared()`.

```
shared_lock(shared_lock&& sl) noexcept;
```

21 *Postconditions:* `pm == sl_p.pm` and `owns == sl_p.owns` (where `sl_p` is the state of `sl` just prior to this construction), `sl.pm == nullptr` and `sl.owns == false`.

```
shared_lock& operator=(shared_lock&& sl) noexcept;
```

22 *Effects:* If `owns` calls `pm->unlock_shared()`.

23 *Postconditions:* `pm == sl_p.pm` and `owns == sl_p.owns` (where `sl_p` is the state of `sl` just prior to this assignment), `sl.pm == nullptr` and `sl.owns == false`.

30.4.2.3.2 `shared_lock` locking [thread.lock.shared.locking]

```
void lock();
```

1 *Effects:* `pm->lock_shared()`.

2 *Postconditions:* `owns == true`.

3 *Throws:* Any exception thrown by `pm->lock_shared()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is `nullptr`. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
bool try_lock();
```

4 *Effects:* `pm->try_lock_shared()`.

5 *Returns:* The value returned by the call to `pm->try_lock_shared()`.

6 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared()`.

7 *Throws:* Any exception thrown by `pm->try_lock_shared()`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is `nullptr`. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
template <class Clock, class Duration>
  bool
  try_lock_until(const chrono::time_point<Clock, Duration>& abs_time);
```

8 *Effects:* `pm->try_lock_shared_until(abs_time)`.

9 *Returns:* The value returned by the call to `pm->try_lock_shared_until(abs_time)`.

10 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_until(abs_time)`.

11 *Throws:* Any exception thrown by `pm->try_lock_shared_until(abs_time)`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is `nullptr`. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
template <class Rep, class Period>
  bool try_lock_for(const chrono::duration<Rep, Period>& rel_time);
```

12 *Effects:* `pm->try_lock_shared_for(rel_time)`.

13 *Returns:* The value returned by the call to `pm->try_lock_shared_for(rel_time)`.

14 *Postconditions:* `owns == res`, where `res` is the value returned by the call to `pm->try_lock_shared_for(rel_time)`.

15 *Throws:* Any exception thrown by `pm->try_lock_shared_for(rel_time)`. `system_error` if an exception is required (30.2.2). `system_error` with an error condition of `operation_not_permitted` if `pm` is `nullptr`. `system_error` with an error condition of `resource_deadlock_would_occur` if on entry `owns` is `true`.

```
void unlock();
```

16 *Effects:* `pm->unlock_shared()`.

17 *Postconditions:* `owns == false`.

18 *Throws:* `system_error` when an exception is required (30.2.2).

19 *Error conditions:*

(19.1) — `operation_not_permitted` — if on entry `owns` is `false`.

30.4.2.3.3 `shared_lock` modifiers [thread.lock.shared.mod]

```
void swap(shared_lock& sl) noexcept;
```

1 *Effects:* Swaps the data members of `*this` and `sl`.

```
mutex_type* release() noexcept;
```

2 *Returns:* The previous value of `pm`.

3 *Postconditions:* `pm == nullptr` and `owns == false`.

```
template <class Mutex>
  void swap(shared_lock<Mutex>& x, shared_lock<Mutex>& y) noexcept;
```

4 *Effects:* `x.swap(y)`.

30.4.2.3.4 `shared_lock` observers [thread.lock.shared.obs]

```
bool owns_lock() const noexcept;
```

1 *Returns:* `owns`.

```
explicit operator bool () const noexcept;
```

2 *Returns:* `owns`.

```
mutex_type* mutex() const noexcept;
```

3 *Returns:* `pm`.

30.4.3 Generic locking algorithms [thread.lock.algorithm]

```
template <class L1, class L2, class... L3> int try_lock(L1&, L2&, L3&...);
```

1 *Requires:* Each template parameter type shall meet the `Lockable` requirements. [*Note:* The `unique_lock` class template meets these requirements when suitably instantiated. — *end note*]

2 *Effects:* Calls `try_lock()` for each argument in order beginning with the first until all arguments have been processed or a call to `try_lock()` fails, either by returning `false` or by throwing an exception. If a call to `try_lock()` fails, `unlock()` shall be called for all prior arguments and there shall be no further calls to `try_lock()`.

3 *Returns:* `-1` if all calls to `try_lock()` returned `true`, otherwise a 0-based index value that indicates the argument for which `try_lock()` returned `false`.

```
template <class L1, class L2, class... L3> void lock(L1&, L2&, L3&...);
```

4 *Requires:* Each template parameter type shall meet the `Lockable` requirements, [*Note:* The `unique_lock` class template meets these requirements when suitably instantiated. — *end note*]

5 *Effects:* All arguments are locked via a sequence of calls to `lock()`, `try_lock()`, or `unlock()` on each argument. The sequence of calls shall not result in deadlock, but is otherwise unspecified. [*Note:* A deadlock avoidance algorithm such as try-and-back-off must be used, but the specific algorithm is not specified to avoid over-constraining implementations. — *end note*] If a call to `lock()` or `try_lock()` throws an exception, `unlock()` shall be called for any argument that had been locked by a call to `lock()` or `try_lock()`.

30.4.4 Call once [thread.once]

The class `once_flag` is an opaque data structure that `call_once` uses to initialize data without causing a data race or deadlock.

30.4.4.1 Struct `once_flag` [thread.once.onceflag]

```
constexpr once_flag() noexcept;
```

1 *Effects:* Constructs an object of type `once_flag`.

2 *Synchronization:* The construction of a `once_flag` object is not synchronized.

3 *Postcondition:* The object's internal state is set to indicate to an invocation of `call_once` with the object as its initial argument that no function has been called.

30.4.4.2 Function `call_once` [thread.once.callonce]

```
template<class Callable, class ...Args>
  void call_once(once_flag& flag, Callable&& func, Args&&... args);
```

1 *Requires:* *INVOKE*`( std::forward<Callable>(func), std::forward<Args>(args)...)` (20.9.2) shall be a valid expression.

2 *Effects:* An execution of `call_once` that does not call its `func` is a *passive* execution. An execution of `call_once` that calls its `func` is an *active* execution. An active execution shall call *INVOKE*`( std::forward<Callable>(func), std::forward<Args>(args)...)`. If such a call to `func` throws an exception the execution is *exceptional*, otherwise it is *returning*. An exceptional execution shall propagate the exception to the caller of `call_once`. Among all executions of `call_once` for any given `once_flag`: at most one shall be a returning execution; if there is a returning execution, it shall be the last active execution; and there are passive executions only if there is a returning execution. [*Note:* passive executions allow other threads to reliably observe the results produced by the earlier returning execution. — *end note*]

3 *Synchronization:* For any given `once_flag`: all active executions occur in a total order; completion of an active execution synchronizes with (1.10) the start of the next one in this total order; and the returning execution synchronizes with the return from all passive executions.

4 *Throws:* `system_error` when an exception is required (30.2.2), or any exception thrown by `func`.

5 [*Example:*

```
// global flag, regular function
void init();
std::once_flag flag;

void f() {
  std::call_once(flag, init);
}

// function static flag, function object
struct initializer {
  void operator()();
};

void g() {
  static std::once_flag flag2;
  std::call_once(flag2, initializer());
}

// object flag, member function
class information {
  std::once_flag verified;
  void verifier();
public:
  void verify() { std::call_once(verified, &information::verifier, *this); }
};
```

— *end example*]

30.5 Condition variables [thread.condition]

1 Condition variables provide synchronization primitives used to block a thread until notified by some other thread that some condition is met or until a system time is reached. Class `condition_variable` provides

a condition variable that can only wait on an object of type `unique_lock<mutex>`, allowing maximum efficiency on some platforms. Class `condition_variable_any` provides a general condition variable that can wait on objects of user-supplied lock types.

2 Condition variables permit concurrent invocation of the `wait`, `wait_for`, `wait_until`, `notify_one` and `notify_all` member functions.

3 The execution of `notify_one` and `notify_all` shall be atomic. The execution of `wait`, `wait_for`, and `wait_until` shall be performed in three atomic parts:

1. the release of the mutex and entry into the waiting state;
2. the unblocking of the wait; and
3. the reacquisition of the lock.

4 The implementation shall behave as if all executions of `notify_one`, `notify_all`, and each part of the `wait`, `wait_for`, and `wait_until` executions are executed in a single unspecified total order consistent with the "happens before" order.

5 Condition variable construction and destruction need not be synchronized.

Header `condition_variable` synopsis

```
namespace std {
  class condition_variable;
  class condition_variable_any;

  void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);

  enum class cv_status { no_timeout, timeout };
}
```

```
void notify_all_at_thread_exit(condition_variable& cond, unique_lock<mutex> lk);
```

6 *Requires:* `lk` is locked by the calling thread and either

(6.1) — no other thread is waiting on `cond`, or

(6.2) — `lk.mutex()` returns the same value for each of the lock arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

7 *Effects:* transfers ownership of the lock associated with `lk` into internal storage and schedules `cond` to be notified when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed. This notification shall be as if

```
lk.unlock();
cond.notify_all();
```

8 *Synchronization:* The implied `lk.unlock()` call is sequenced after the destruction of all objects with thread storage duration associated with the current thread.

9 *Note:* The supplied lock will be held until the thread exits, and care must be taken to ensure that this does not cause deadlock due to lock ordering issues. After calling `notify_all_at_thread_exit` it is recommended that the thread should be exited as soon as possible, and that no blocking or time-consuming tasks are run on that thread.

10 *Note:* It is the user's responsibility to ensure that waiting threads do not erroneously assume that the thread has finished if they experience spurious wakeups. This typically requires that the condition being waited for is satisfied while holding the lock on `lk`, and that this lock is not released and reacquired prior to calling `notify_all_at_thread_exit`.

30.5.1 Class `condition_variable` [thread.condition.condvar]

```
namespace std {
  class condition_variable {
  public:

    condition_variable();
    ~condition_variable();

    condition_variable(const condition_variable&) = delete;
    condition_variable& operator=(const condition_variable&) = delete;

    void notify_one() noexcept;
    void notify_all() noexcept;
    void wait(unique_lock<mutex>& lock);
    template <class Predicate>
      void wait(unique_lock<mutex>& lock, Predicate pred);
    template <class Clock, class Duration>
      cv_status wait_until(unique_lock<mutex>& lock,
                           const chrono::time_point<Clock, Duration>& abs_time);
    template <class Clock, class Duration, class Predicate>
      bool wait_until(unique_lock<mutex>& lock,
                      const chrono::time_point<Clock, Duration>& abs_time,
                      Predicate pred);

    template <class Rep, class Period>
      cv_status wait_for(unique_lock<mutex>& lock,
                         const chrono::duration<Rep, Period>& rel_time);
    template <class Rep, class Period, class Predicate>
      bool wait_for(unique_lock<mutex>& lock,
                    const chrono::duration<Rep, Period>& rel_time,
                    Predicate pred);

    typedef implementation-defined native_handle_type; // See 30.2.3
    native_handle_type native_handle();                // See 30.2.3
  };
}
```

1 The class `condition_variable` shall be a standard-layout class (Clause 9).

`condition_variable();`

2 *Effects:* Constructs an object of type `condition_variable`.

3 *Throws:* `system_error` when an exception is required (30.2.2).

4 *Error conditions:*

(4.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

`~condition_variable();`

5 *Requires:* There shall be no thread blocked on `*this`. [*Note:* That is, all threads shall have been notified; they may subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait must happen before destruction. The user must take care to ensure that no threads wait on `*this` once the destructor has been started, especially when the waiting threads are calling the wait

functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

6 *Effects:* Destroys the object.

```
void notify_one() noexcept;
```

7 *Effects:* If any threads are blocked waiting for `*this`, unblocks one of those threads.

```
void notify_all() noexcept;
```

8 *Effects:* Unblocks all threads that are blocked waiting for `*this`.

```
void wait(unique_lock<mutex>& lock);
```

9 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(9.1) — no other thread is waiting on this `condition_variable` object or

(9.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

10 *Effects:*

(10.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(10.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(10.3) — The function will unblock when signaled by a call to `notify_one()` or a call to `notify_all()`, or spuriously.

11 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

12 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

13 *Throws:* Nothing.

```
template <class Predicate>
  void wait(unique_lock<mutex>& lock, Predicate pred);
```

14 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(14.1) — no other thread is waiting on this `condition_variable` object or

(14.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

15 *Effects:* Equivalent to:

```
while (!pred())
  wait(lock);
```

16 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

17 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

18 *Throws:* Timeout-related exceptions (30.2.4) or any exception thrown by `pred`.

```
template <class Clock, class Duration>
  cv_status wait_until(unique_lock<mutex>& lock,
                       const chrono::time_point<Clock, Duration>& abs_time);
```

19 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(19.1) — no other thread is waiting on this `condition_variable` object or

(19.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

20 *Effects:*

(20.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(20.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock), then returns.

(20.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout (30.2.4) specified by `abs_time`, or spuriously.

(20.4) — If the function exits via an exception, `lock.lock()` shall be called prior to exiting the function.

21 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

22 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

23 *Returns:* `cv_status::timeout` if the absolute timeout (30.2.4) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

24 *Throws:* Timeout-related exceptions (30.2.4).

```
template <class Rep, class Period>
  cv_status wait_for(unique_lock<mutex>& lock,
                     const chrono::duration<Rep, Period>& rel_time);
```

25 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(25.1) — no other thread is waiting on this `condition_variable` object or

(25.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

26 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time);
```

27 *Returns:* `cv_status::timeout` if the relative timeout (30.2.4) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

28 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

29 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

30 *Throws:* Timeout-related exceptions (30.2.4).

```
template <class Clock, class Duration, class Predicate>
  bool wait_until(unique_lock<mutex>& lock,
                  const chrono::time_point<Clock, Duration>& abs_time,
                  Predicate pred);
```

31 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(31.1) — no other thread is waiting on this `condition_variable` object or

(31.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

32 *Effects:* Equivalent to:

```
while (!pred())
  if (wait_until(lock, abs_time) == cv_status::timeout)
    return pred();
return true;
```

33 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

34 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

35 [*Note:* The returned value indicates whether the predicate evaluated to `true` regardless of whether the timeout was triggered. — *end note*]

36 *Throws:* Timeout-related exceptions (30.2.4) or any exception thrown by `pred`.

```
template <class Rep, class Period, class Predicate>
  bool wait_for(unique_lock<mutex>& lock,
                const chrono::duration<Rep, Period>& rel_time,
                Predicate pred);
```

37 *Requires:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread, and either

(37.1) — no other thread is waiting on this `condition_variable` object or

(37.2) — `lock.mutex()` returns the same value for each of the `lock` arguments supplied by all concurrently waiting (via `wait`, `wait_for`, or `wait_until`) threads.

38 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time, std::move(pred));
```

39 [*Note:* There is no blocking if `pred()` is initially `true`, even if the timeout has already expired. — *end note*]

40 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

41 *Postcondition:* `lock.owns_lock()` is `true` and `lock.mutex()` is locked by the calling thread.

42 [*Note:* The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered. — *end note*]

43 *Throws:* Timeout-related exceptions (30.2.4) or any exception thrown by `pred`.

30.5.2 Class `condition_variable_any` [thread.condition.condvarany]

1 A `Lock` type shall meet the `BasicLockable` requirements (30.2.5.2). [*Note:* All of the standard mutex types meet this requirement. If a `Lock` type other than one of the standard mutex types or a `unique_lock` wrapper for a standard mutex type is used with `condition_variable_any`, the user must ensure that any necessary synchronization is in place with respect to the predicate associated with the `condition_variable_any` instance. — *end note*]

```
namespace std {
  class condition_variable_any {
  public:
    condition_variable_any();
    ~condition_variable_any();

    condition_variable_any(const condition_variable_any&) = delete;
    condition_variable_any& operator=(const condition_variable_any&) = delete;
```

```
    void notify_one() noexcept;
    void notify_all() noexcept;
    template <class Lock>
      void wait(Lock& lock);
    template <class Lock, class Predicate>
      void wait(Lock& lock, Predicate pred);

    template <class Lock, class Clock, class Duration>
      cv_status wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time);
    template <class Lock, class Clock, class Duration, class Predicate>
      bool wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time,
        Predicate pred);
    template <class Lock, class Rep, class Period>
      cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
    template <class Lock, class Rep, class Period, class Predicate>
      bool wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time,
        Predicate pred);
  };
}
```

`condition_variable_any();`

2 *Effects:* Constructs an object of type `condition_variable_any`.

3 *Throws:* `bad_alloc` or `system_error` when an exception is required (30.2.2).

4 *Error conditions:*

(4.1) — `resource_unavailable_try_again` — if some non-memory resource limitation prevents initialization.

(4.2) — `operation_not_permitted` — if the thread does not have the privilege to perform the operation.

`~condition_variable_any();`

5 *Requires:* There shall be no thread blocked on `*this`. [*Note:* That is, all threads shall have been notified; they may subsequently block on the lock specified in the wait. This relaxes the usual rules, which would have required all wait calls to happen before destruction. Only the notification to unblock the wait must happen before destruction. The user must take care to ensure that no threads wait on `*this` once the destructor has been started, especially when the waiting threads are calling the wait functions in a loop or using the overloads of `wait`, `wait_for`, or `wait_until` that take a predicate. — *end note*]

6 *Effects:* Destroys the object.

`void notify_one() noexcept;`

7 *Effects:* If any threads are blocked waiting for `*this`, unblocks one of those threads.

`void notify_all() noexcept;`

8 *Effects:* Unblocks all threads that are blocked waiting for `*this`.

```
template <class Lock>
  void wait(Lock& lock);
```

9 *Note:* if any of the `wait` functions exits via an exception, it is unspecified whether the `Lock` is held. One can use a `Lock` type that allows to query that, such as the `unique_lock` wrapper.

10 *Effects:*

(10.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(10.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.

(10.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, or spuriously.

11 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

12 *Postcondition:* `lock` is locked by the calling thread.

13 *Throws:* Nothing.

```
template <class Lock, class Predicate>
  void wait(Lock& lock, Predicate pred);
```

14 *Effects:* Equivalent to:

```
while (!pred())
  wait(lock);
```

```
template <class Lock, class Clock, class Duration>
  cv_status wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time);
```

15 *Effects:*

(15.1) — Atomically calls `lock.unlock()` and blocks on `*this`.

(15.2) — When unblocked, calls `lock.lock()` (possibly blocking on the lock) and returns.

(15.3) — The function will unblock when signaled by a call to `notify_one()`, a call to `notify_all()`, expiration of the absolute timeout (30.2.4) specified by `abs_time`, or spuriously.

(15.4) — If the function exits via an exception, `lock.lock()` shall be called prior to exiting the function.

16 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

17 *Postcondition:* `lock` is locked by the calling thread.

18 *Returns:* `cv_status::timeout` if the absolute timeout (30.2.4) specified by `abs_time` expired, otherwise `cv_status::no_timeout`.

19 *Throws:* Timeout-related exceptions (30.2.4).

```
template <class Lock, class Rep, class Period>
  cv_status wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time);
```

20 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time);
```

21 *Returns:* `cv_status::timeout` if the relative timeout (30.2.4) specified by `rel_time` expired, otherwise `cv_status::no_timeout`.

22 *Remarks:* If the function fails to meet the postcondition, `std::terminate()` shall be called (15.5.1). [*Note:* This can happen if the re-locking of the mutex throws an exception. — *end note*]

23 *Postcondition:* `lock` is locked by the calling thread.

24 *Throws:* Timeout-related exceptions (30.2.4).

```
template <class Lock, class Clock, class Duration, class Predicate>
  bool wait_until(Lock& lock, const chrono::time_point<Clock, Duration>& abs_time, Predicate pred);
```

25 *Effects:* Equivalent to:

```
while (!pred())
  if (wait_until(lock, abs_time) == cv_status::timeout)
    return pred();
return true;
```

26 [*Note:* There is no blocking if `pred()` is initially `true`, or if the timeout has already expired. — *end note*]

27 [*Note:* The returned value indicates whether the predicate evaluates to `true` regardless of whether the timeout was triggered. — *end note*]

```
template <class Lock, class Rep, class Period, class Predicate>
  bool wait_for(Lock& lock, const chrono::duration<Rep, Period>& rel_time, Predicate pred);
```

28 *Effects:* Equivalent to:

```
return wait_until(lock, chrono::steady_clock::now() + rel_time, std::move(pred));
```

30.6 Futures [futures]

30.6.1 Overview [futures.overview]

1 30.6 describes components that a C++ program can use to retrieve in one thread the result (value or exception) from a function that has run in the same thread or another thread. [*Note:* These components are not restricted to multi-threaded programs but can be useful in single-threaded programs as well. — *end note*]

Header <future> synopsis

```
namespace std {
  enum class future_errc {
    broken_promise = implementation-defined,
    future_already_retrieved = implementation-defined,
    promise_already_satisfied = implementation-defined,
    no_state = implementation-defined
  };

  enum class launch : unspecified {
    async = unspecified,
    deferred = unspecified,
    implementation-defined
  };

  enum class future_status {
    ready,
    timeout,
    deferred
  };

  template <> struct is_error_code_enum<future_errc> : public true_type { };
  error_code make_error_code(future_errc e) noexcept;
  error_condition make_error_condition(future_errc e) noexcept;

  const error_category& future_category() noexcept;

  class future_error;
```

```
  template <class R> class promise;
  template <class R> class promise<R&>;
  template <> class promise<void>;

  template <class R>
    void swap(promise<R>& x, promise<R>& y) noexcept;

  template <class R, class Alloc>
    struct uses_allocator<promise<R>, Alloc>;

  template <class R> class future;
  template <class R> class future<R&>;
  template <> class future<void>;

  template <class R> class shared_future;
  template <class R> class shared_future<R&>;
  template <> class shared_future<void>;

  template <class> class packaged_task;   // undefined
  template <class R, class... ArgTypes>
    class packaged_task<R(ArgTypes...)>;

  template <class R, class... ArgTypes>
    void swap(packaged_task<R(ArgTypes...)>&, packaged_task<R(ArgTypes...)>&) noexcept;

  template <class R, class Alloc>
    struct uses_allocator<packaged_task<R>, Alloc>;

  template <class F, class... Args>
    future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(F&& f, Args&&... args);
  template <class F, class... Args>
    future<result_of_t<decay_t<F>(decay_t<Args>...)>>
    async(launch policy, F&& f, Args&&... args);
}
```

2 The `enum` type `launch` is a bitmask type (17.5.2.1.3) with `launch::async` and `launch::deferred` denoting individual bits. [*Note:* Implementations can provide bitmasks to specify restrictions on task interaction by functions launched by `async()` applicable to a corresponding subset of available launch policies. Implementations can extend the behavior of the first overload of `async()` by adding their extensions to the launch policy under the "as if" rule. — *end note*]

3 The enum values of `future_errc` are distinct and not zero.

30.6.2 Error handling [futures.errors]

```
const error_category& future_category() noexcept;
```

1 *Returns:* A reference to an object of a type derived from class `error_category`.

2 The object's `default_error_condition` and equivalent virtual functions shall behave as specified for the class `error_category`. The object's `name` virtual function shall return a pointer to the string `"future"`.

```
error_code make_error_code(future_errc e) noexcept;
```

3 *Returns:* `error_code(static_cast<int>(e), future_category())`.

```
error_condition make_error_condition(future_errc e) noexcept;
```

4 *Returns:* `error_condition(static_cast<int>(e), future_category())`.

30.6.3 Class `future_error` [futures.future_error]

```
namespace std {
  class future_error : public logic_error {
  public:
    future_error(error_code ec);  // exposition only

    const error_code& code() const noexcept;
    const char*       what() const noexcept;
  };
}
```

```
const error_code& code() const noexcept;
```

1 *Returns:* The value of `ec` that was passed to the object's constructor.

```
const char* what() const noexcept;
```

2 *Returns:* An NTBS incorporating `code().message()`.

30.6.4 Shared state [futures.state]

1 Many of the classes introduced in this sub-clause use some state to communicate results. This *shared state* consists of some state information and some (possibly not yet evaluated) *result*, which can be a (possibly void) value or an exception. [*Note:* Futures, promises, and tasks defined in this clause reference such shared state. — *end note*]

2 [*Note:* The result can be any kind of object including a function to compute that result, as used by `async` when `policy` is `launch::deferred`. — *end note*]

3 An *asynchronous return object* is an object that reads results from a shared state. A *waiting function* of an asynchronous return object is one that potentially blocks to wait for the shared state to be made ready. If a waiting function can return before the state is made ready because of a timeout (30.2.5), then it is a *timed waiting function*, otherwise it is a *non-timed waiting function*.

4 An *asynchronous provider* is an object that provides a result to a shared state. The result of a shared state is set by respective functions on the asynchronous provider. [*Note:* Such as promises or tasks. — *end note*] The means of setting the result of a shared state is specified in the description of those classes and functions that create such a state object.

5 When an asynchronous return object or an asynchronous provider is said to release its shared state, it means:

(5.1) — if the return object or provider holds the last reference to its shared state, the shared state is destroyed; and

(5.2) — the return object or provider gives up its reference to its shared state; and

(5.3) — these actions will not block for the shared state to become ready, except that it may block if all of the following are true: the shared state was created by a call to `std::async`, the shared state is not yet ready, and this was the last reference to the shared state.

6 When an asynchronous provider is said to make its shared state ready, it means:

(6.1) — first, the provider marks its shared state as ready; and

(6.2) — second, the provider unblocks any execution agents waiting for its shared state to become ready.

7 When an asynchronous provider is said to abandon its shared state, it means:

(7.1) — first, if that state is not ready, the provider

(7.1.1) — stores an exception object of type `future_error` with an error condition of `broken_promise` within its shared state; and then

(7.1.2) — makes its shared state ready;

(7.2) — second, the provider releases its shared state.

8 A shared state is *ready* only if it holds a value or an exception ready for retrieval. Waiting for a shared state to become ready may invoke code to compute the result on the waiting thread if so specified in the description of the class or function that creates the state object.

9 Calls to functions that successfully set the stored result of a shared state synchronize with (1.10) calls to functions successfully detecting the ready state resulting from that setting. The storage of the result (whether normal or exceptional) into the shared state synchronizes with (1.10) the successful return from a call to a waiting function on the shared state.

10 Some functions (e.g., `promise::set_value_at_thread_exit`) delay making the shared state ready until the calling thread exits. The destruction of each of that thread's objects with thread storage duration (3.7.2) is sequenced before making that shared state ready.

11 Access to the result of the same shared state may conflict (1.10). [*Note:* this explicitly specifies that the result of the shared state is visible in the objects that reference this state in the sense of data race avoidance (17.6.5.9). For example, concurrent accesses through references returned by `shared_future::get()` (30.6.7) must either use read-only operations or provide additional synchronization. — *end note*]

30.6.5 Class template `promise` [futures.promise]

```
namespace std {
  template <class R>
  class promise {
  public:
    promise();
    template <class Allocator>
      promise(allocator_arg_t, const Allocator& a);
    promise(promise&& rhs) noexcept;
    promise(const promise& rhs) = delete;
    ~promise();

    // assignment
    promise& operator=(promise&& rhs) noexcept;
    promise& operator=(const promise& rhs) = delete;
    void swap(promise& other) noexcept;

    // retrieving the result
    future<R> get_future();

    // setting the result
    void set_value(see below);
    void set_exception(exception_ptr p);

    // setting the result with deferred notification
    void set_value_at_thread_exit(const R& r);
    void set_value_at_thread_exit(see below);
```

```
    void set_exception_at_thread_exit(exception_ptr p);
  };
  template <class R>
    void swap(promise<R>& x, promise<R>& y) noexcept;
  template <class R, class Alloc>
    struct uses_allocator<promise<R>, Alloc>;
}
```

1 The implementation shall provide the template `promise` and two specializations, `promise<R&>` and `promise<void>`. These differ only in the argument type of the member function `set_value`, as set out in its description, below.

2 The `set_value`, `set_exception`, `set_value_at_thread_exit`, and `set_exception_at_thread_exit` member functions behave as though they acquire a single mutex associated with the promise object while updating the promise object.

```
template <class R, class Alloc>
  struct uses_allocator<promise<R>, Alloc>
    : true_type { };
```

3 *Requires:* `Alloc` shall be an Allocator (17.6.3.5).

```
promise();
template <class Allocator>
  promise(allocator_arg_t, const Allocator& a);
```

4 *Effects:* constructs a `promise` object and a shared state. The second constructor uses the allocator `a` to allocate memory for the shared state.

```
promise(promise&& rhs) noexcept;
```

5 *Effects:* constructs a new `promise` object and transfers ownership of the shared state of `rhs` (if any) to the newly-constructed object.

6 *Postcondition:* `rhs` has no shared state.

```
~promise();
```

7 *Effects:* Abandons any shared state (30.6.4).

```
promise& operator=(promise&& rhs) noexcept;
```

8 *Effects:* Abandons any shared state (30.6.4) and then as if `promise(std::move(rhs)).swap(*this)`.

9 *Returns:* `*this`.

```
void swap(promise& other) noexcept;
```

10 *Effects:* Exchanges the shared state of `*this` and `other`.

11 *Postcondition:* `*this` has the shared state (if any) that `other` had prior to the call to `swap`. `other` has the shared state (if any) that `*this` had prior to the call to `swap`.

```
future<R> get_future();
```

12 *Returns:* A `future<R>` object with the same shared state as `*this`.

13 *Throws:* `future_error` if `*this` has no shared state or if `get_future` has already been called on a `promise` with the same shared state as `*this`.

14 *Error conditions:*

(14.1) — `future_already_retrieved` if `get_future` has already been called on a `promise` with the same shared state as `*this`.

(14.2) — `no_state` if `*this` has no shared state.

```
void promise::set_value(const R& r);
void promise::set_value(R&& r);
void promise<R&>::set_value(R& r);
void promise<void>::set_value();
```

15 *Effects:* atomically stores the value `r` in the shared state and makes that state ready (30.6.4).

16 *Throws:*

(16.1) — `future_error` if its shared state already has a stored value or exception, or

(16.2) — for the first version, any exception thrown by the constructor selected to copy an object of `R`, or

(16.3) — for the second version, any exception thrown by the constructor selected to move an object of `R`.

17 *Error conditions:*

(17.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(17.2) — `no_state` if `*this` has no shared state.

```
void set_exception(exception_ptr p);
```

18 *Effects:* atomically stores the exception pointer `p` in the shared state and makes that state ready (30.6.4).

19 *Throws:* `future_error` if its shared state already has a stored value or exception.

20 *Error conditions:*

(20.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(20.2) — `no_state` if `*this` has no shared state.

```
void promise::set_value_at_thread_exit(const R& r);
void promise::set_value_at_thread_exit(R&& r);
void promise<R&>::set_value_at_thread_exit(R& r);
void promise<void>::set_value_at_thread_exit();
```

21 *Effects:* Stores the value `r` in the shared state without making that state ready immediately. Schedules that state to be made ready when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed.

22 *Throws:*

(22.1) — `future_error` if its shared state already has a stored value or exception, or

(22.2) — for the first version, any exception thrown by the constructor selected to copy an object of `R`, or

(22.3) — for the second version, any exception thrown by the constructor selected to move an object of `R`.

23 *Error conditions:*

(23.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(23.2) — `no_state` if `*this` has no shared state.

```
void set_exception_at_thread_exit(exception_ptr p);
```

24 *Effects:* Stores the exception pointer `p` in the shared state without making that state ready immediately. Schedules that state to be made ready when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed.

25 *Throws:* `future_error` if an error condition occurs.

26 *Error conditions:*

(26.1) — `promise_already_satisfied` if its shared state already has a stored value or exception.

(26.2) — `no_state` if `*this` has no shared state.

```
template <class R>
  void swap(promise<R>& x, promise<R>& y);
```

27 *Effects:* `x.swap(y)`.

30.6.6 Class template `future` [futures.unique_future]

1 The class template `future` defines a type for asynchronous return objects which do not share their shared state with other asynchronous return objects. A default-constructed `future` object has no shared state. A `future` object with shared state can be created by functions on asynchronous providers (30.6.4) or by the move constructor and shares its shared state with the original asynchronous provider. The result (value or exception) of a `future` object can be set by calling a respective function on an object that shares the same shared state.

2 [*Note:* Member functions of `future` do not synchronize with themselves or with member functions of `shared_future`. — *end note*]

3 The effect of calling any member function other than the destructor, the move-assignment operator, or `valid` on a `future` object for which `valid() == false` is undefined. [*Note:* Implementations are encouraged to detect this case and throw an object of type `future_error` with an error condition of `future_errc::no_state`. — *end note*]

```
namespace std {
  template <class R>
  class future {
  public:
    future() noexcept;
    future(future &&) noexcept;
    future(const future& rhs) = delete;
    ~future();
    future& operator=(const future& rhs) = delete;
    future& operator=(future&&) noexcept;
    shared_future<R> share();

    // retrieving the value
    see below get();

    // functions to check state
    bool valid() const noexcept;

    void wait() const;
    template <class Rep, class Period>
      future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
    template <class Clock, class Duration>
      future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
  };
}
```

4 The implementation shall provide the template `future` and two specializations, `future<R&>` and `future<void>`. These differ only in the return type and return value of the member function `get`, as set out in its description, below.

```
future() noexcept;
```

5 *Effects:* constructs an *empty* `future` object that does not refer to a shared state.

6 *Postcondition:* `valid() == false`.

```
future(future&& rhs) noexcept;
```

7 *Effects:* move constructs a `future` object that refers to the shared state that was originally referred to by `rhs` (if any).

8 *Postconditions:*

(8.1) — `valid()` returns the same value as `rhs.valid()` prior to the constructor invocation.

(8.2) — `rhs.valid() == false`.

```
~future();
```

9 *Effects:*

(9.1) — releases any shared state (30.6.4);

(9.2) — destroys `*this`.

```
future& operator=(future&& rhs) noexcept;
```

10 *Effects:*

(10.1) — releases any shared state (30.6.4).

(10.2) — move assigns the contents of `rhs` to `*this`.

11 *Postconditions:*

(11.1) — `valid()` returns the same value as `rhs.valid()` prior to the assignment.

(11.2) — `rhs.valid() == false`.

```
shared_future<R> share();
```

12 *Returns:* `shared_future<R>(std::move(*this))`.

13 *Postcondition:* `valid() == false`.

```
R future::get();
R& future<R&>::get();
void future<void>::get();
```

14 *Note:* as described above, the template and its two required specializations differ only in the return type and return value of the member function `get`.

15 *Effects:* `wait()`s until the shared state is ready, then retrieves the value stored in the shared state.

16 *Returns:*

(16.1) — `future::get()` returns the value `v` stored in the object's shared state as `std::move(v)`.

(16.2) — `future<R&>::get()` returns the reference stored as value in the object's shared state.

(16.3) — `future<void>::get()` returns nothing.

17 *Throws:* the stored exception, if an exception was stored in the shared state.

18 *Postcondition:* `valid() == false`.

```
bool valid() const noexcept;
```

19 *Returns:* `true` only if `*this` refers to a shared state.

```
void wait() const;
```

20 *Effects:* blocks until the shared state is ready.

```
template <class Rep, class Period>
  future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
```

21 *Effects:* none if the shared state contains a deferred function (30.6.8), otherwise blocks until the shared state is ready or until the relative timeout (30.2.4) specified by `rel_time` has expired.

22 *Returns:*

(22.1) — `future_status::deferred` if the shared state contains a deferred function.

(22.2) — `future_status::ready` if the shared state is ready.

(22.3) — `future_status::timeout` if the function is returning because the relative timeout (30.2.4) specified by `rel_time` has expired.

23 *Throws:* timeout-related exceptions (30.2.4).

```
template <class Clock, class Duration>
  future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
```

24 *Effects:* none if the shared state contains a deferred function (30.6.8), otherwise blocks until the shared state is ready or until the absolute timeout (30.2.4) specified by `abs_time` has expired.

25 *Returns:*

(25.1) — `future_status::deferred` if the shared state contains a deferred function.

(25.2) — `future_status::ready` if the shared state is ready.

(25.3) — `future_status::timeout` if the function is returning because the absolute timeout (30.2.4) specified by `abs_time` has expired.

26 *Throws:* timeout-related exceptions (30.2.4).

30.6.7 Class template `shared_future` [futures.shared_future]

1 The class template `shared_future` defines a type for asynchronous return objects which may share their shared state with other asynchronous return objects. A default-constructed `shared_future` object has no shared state. A `shared_future` object with shared state can be created by conversion from a `future` object and shares its shared state with the original asynchronous provider (30.6.4) of the shared state. The result (value or exception) of a `shared_future` object can be set by calling a respective function on an object that shares the same shared state.

2 [*Note:* Member functions of `shared_future` do not synchronize with themselves, but they synchronize with the shared shared state. — *end note*]

3 The effect of calling any member function other than the destructor, the move-assignment operator, or `valid()` on a `shared_future` object for which `valid() == false` is undefined. [*Note:* Implementations are encouraged to detect this case and throw an object of type `future_error` with an error condition of `future_errc::no_state`. — *end note*]

```
namespace std {
  template <class R>
  class shared_future {
  public:
    shared_future() noexcept;
    shared_future(const shared_future& rhs);
    shared_future(future<R>&&) noexcept;
```

```
    shared_future(shared_future&& rhs) noexcept;
    ~shared_future();
    shared_future& operator=(const shared_future& rhs);
    shared_future& operator=(shared_future&& rhs) noexcept;

    // retrieving the value
    see below get() const;

    // functions to check state
    bool valid() const noexcept;

    void wait() const;
    template <class Rep, class Period>
      future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
    template <class Clock, class Duration>
      future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
  };
}
```

4 The implementation shall provide the template `shared_future` and two specializations, `shared_future<R&>` and `shared_future<void>`. These differ only in the return type and return value of the member function `get`, as set out in its description, below.

`shared_future() noexcept;`

5 *Effects:* constructs an *empty* `shared_future` object that does not refer to a shared state.

6 *Postcondition:* `valid() == false`.

`shared_future(const shared_future& rhs);`

7 *Effects:* constructs a `shared_future` object that refers to the same shared state as `rhs` (if any).

8 *Postcondition:* `valid()` returns the same value as `rhs.valid()`.

```
shared_future(future<R>&& rhs) noexcept;
shared_future(shared_future&& rhs) noexcept;
```

9 *Effects:* move constructs a `shared_future` object that refers to the shared state that was originally referred to by `rhs` (if any).

10 *Postconditions:*

(10.1) — `valid()` returns the same value as `rhs.valid()` returned prior to the constructor invocation.

(10.2) — `rhs.valid() == false`.

`~shared_future();`

11 *Effects:*

(11.1) — releases any shared state (30.6.4);

(11.2) — destroys `*this`.

`shared_future& operator=(shared_future&& rhs) noexcept;`

12 *Effects:*

(12.1) — releases any shared state (30.6.4);

(12.2) — move assigns the contents of `rhs` to `*this`.

13 *Postconditions:*

(13.1) — `valid()` returns the same value as `rhs.valid()` returned prior to the assignment.

(13.2) — `rhs.valid() == false`.

```
shared_future& operator=(const shared_future& rhs);
```

14 *Effects:*

(14.1) — releases any shared state (30.6.4);

(14.2) — assigns the contents of `rhs` to `*this`. [*Note:* As a result, `*this` refers to the same shared state as `rhs` (if any). — *end note*]

15 *Postconditions:* `valid() == rhs.valid()`.

```
const R& shared_future::get() const;
R& shared_future<R&>::get() const;
void shared_future<void>::get() const;
```

16 *Note:* as described above, the template and its two required specializations differ only in the return type and return value of the member function `get`.

17 *Note:* access to a value object stored in the shared state is unsynchronized, so programmers should apply only those operations on `R` that do not introduce a data race (1.10).

18 *Effects:* `wait()`s until the shared state is ready, then retrieves the value stored in the shared state.

19 *Returns:*

(19.1) — `shared_future::get()` returns a const reference to the value stored in the object's shared state. [*Note:* Access through that reference after the shared state has been destroyed produces undefined behavior; this can be avoided by not storing the reference in any storage with a greater lifetime than the `shared_future` object that returned the reference. — *end note*]

(19.2) — `shared_future<R&>::get()` returns the reference stored as value in the object's shared state.

(19.3) — `shared_future<void>::get()` returns nothing.

20 *Throws:* the stored exception, if an exception was stored in the shared state.

```
bool valid() const noexcept;
```

21 *Returns:* `true` only if `*this` refers to a shared state.

```
void wait() const;
```

22 *Effects:* blocks until the shared state is ready.

```
template <class Rep, class Period>
  future_status wait_for(const chrono::duration<Rep, Period>& rel_time) const;
```

23 *Effects:* none if the shared state contains a deferred function (30.6.8), otherwise blocks until the shared state is ready or until the relative timeout (30.2.4) specified by `rel_time` has expired.

24 *Returns:*

(24.1) — `future_status::deferred` if the shared state contains a deferred function.

(24.2) — `future_status::ready` if the shared state is ready.

(24.3) — `future_status::timeout` if the function is returning because the relative timeout (30.2.4) specified by `rel_time` has expired.

25 *Throws:* timeout-related exceptions (30.2.4).

```
template <class Clock, class Duration>
  future_status wait_until(const chrono::time_point<Clock, Duration>& abs_time) const;
```

26 *Effects:* none if the shared state contains a deferred function (30.6.8), otherwise blocks until the shared state is ready or until the absolute timeout (30.2.4) specified by `abs_time` has expired.

27 *Returns:*

(27.1) — `future_status::deferred` if the shared state contains a deferred function.

(27.2) — `future_status::ready` if the shared state is ready.

(27.3) — `future_status::timeout` if the function is returning because the absolute timeout (30.2.4) specified by `abs_time` has expired.

28 *Throws:* timeout-related exceptions (30.2.4).

30.6.8 Function template `async` [futures.async]

1 The function template `async` provides a mechanism to launch a function potentially in a new thread and provides the result of the function in a `future` object with which it shares a shared state.

```
template <class F, class... Args>
  future<result_of_t<decay_t<F>(decay_t<Args>...)>> async(F&& f, Args&&... args);
template <class F, class... Args>
  future<result_of_t<decay_t<F>(decay_t<Args>...)>> async(launch policy, F&& f, Args&&... args);
```

2 *Requires:* `F` and each `Ti` in `Args` shall satisfy the `MoveConstructible` requirements. *INVOKE*`(`*DECAY_COPY*`(std::forward<F>(f)), `*DECAY_COPY*`(std::forward<Args>(args))...)` (20.9.2, 30.3.1.2) shall be a valid expression.

3 *Effects:* The first function behaves the same as a call to the second function with a `policy` argument of `launch::async | launch::deferred` and the same arguments for `F` and `Args`. The second function creates a shared state that is associated with the returned `future` object. The further behavior of the second function depends on the `policy` argument as follows (if more than one of these conditions applies, the implementation may choose any of the corresponding policies):

(3.1) — if `policy & launch::async` is non-zero — calls *INVOKE*`(`*DECAY_COPY*`(std::forward<F>(f)), `*DECAY_COPY*`(std::forward<Args>(args))...)` (20.9.2, 30.3.1.2) as if in a new thread of execution represented by a `thread` object with the calls to *DECAY_COPY*`()` being evaluated in the thread that called `async`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of *INVOKE*`(`*DECAY_COPY*`(std::forward<F>(f)), `*DECAY_COPY*`(std::forward<Args>(args))...)` is stored as the exceptional result in the shared state. The `thread` object is stored in the shared state and affects the behavior of any asynchronous return objects that reference that state.

(3.2) — if `policy & launch::deferred` is non-zero — Stores *DECAY_COPY*`(std::forward<F>(f))` and *DECAY_COPY*`(std::forward<Args>(args))...` in the shared state. These copies of `f` and `args` constitute a *deferred function*. Invocation of the deferred function evaluates *INVOKE*`(std::move(g), std::move(xyz))` where `g` is the stored value of *DECAY_COPY*`(std::forward<F>(f))` and `xyz` is the stored copy of *DECAY_COPY*`(std::forward<Args>(args))...`. Any return value is stored as the result in the shared state. Any exception propagated from the execution of the deferred function is stored as the exceptional result in the shared state. The shared state is not made ready until the function has completed. The first call to a non-timed waiting function (30.6.4) on an asynchronous return object referring to this shared state shall invoke the deferred function in the thread that called the waiting function. Once evaluation of *INVOKE*`(std::move(g),`

`std::move(xyz))` begins, the function is no longer considered deferred. [*Note:* If this policy is specified together with other policies, such as when using a `policy` value of `launch::async | launch::deferred`, implementations should defer invocation or the selection of the policy when no more concurrency can be effectively exploited. — *end note*]

(3.3) — If no value is set in the launch policy, or a value is set that is neither specified in this International Standard or by the implementation, the behavior is undefined.

4 *Returns:* An object of type `future<result_of_t<decay_t<F>(decay_t<Args>...)>>` that refers to the shared state created by this call to `async`. [*Note:* If a future obtained from std::async is moved outside the local scope, other code that uses the future must be aware that the future's destructor may block for the shared state to become ready. — *end note*]

5 *Synchronization:* Regardless of the provided `policy` argument,

(5.1) — the invocation of `async` synchronizes with (1.10) the invocation of `f`. [*Note:* This statement applies even when the corresponding `future` object is moved to another thread. — *end note*]; and

(5.2) — the completion of the function `f` is sequenced before (1.10) the shared state is made ready. [*Note:* `f` might not be called at all, so its completion might never happen. — *end note*]

If the implementation chooses the `launch::async` policy,

(5.3) — a call to a waiting function on an asynchronous return object that shares the shared state created by this `async` call shall block until the associated thread has completed, as if joined, or else time out (30.3.1.5);

(5.4) — the associated thread completion synchronizes with (1.10) the return from the first function that successfully detects the ready status of the shared state or with the return from the last function that releases the shared state, whichever happens first.

6 *Throws:* `system_error` if `policy == launch::async` and the implementation is unable to start a new thread.

7 *Error conditions:*

(7.1) — `resource_unavailable_try_again` — if `policy == launch::async` and the system is unable to start a new thread.

8 [*Example:*

```
int work1(int value);
int work2(int value);
int work(int value) {
  auto handle = std::async([=]{ return work2(value); });
  int tmp = work1(value);
  return tmp + handle.get();    // #1
}
```

[*Note:* Line #1 might not result in concurrency because the `async` call uses the default policy, which may use `launch::deferred`, in which case the lambda might not be invoked until the `get()` call; in that case, `work1` and `work2` are called on the same thread and there is no concurrency. — *end note*] — *end example*]

30.6.9 Class template `packaged_task` [futures.task]

1 The class template `packaged_task` defines a type for wrapping a function or callable object so that the return value of the function or callable object is stored in a future when it is invoked.

2 When the `packaged_task` object is invoked, its stored task is invoked and the result (whether normal or exceptional) stored in the shared state. Any futures that share the shared state will then be able to access the stored result.

```
namespace std {
  template<class> class packaged_task; // undefined

  template<class R, class... ArgTypes>
  class packaged_task<R(ArgTypes...)> {
  public:
    // construction and destruction
    packaged_task() noexcept;
    template <class F>
      explicit packaged_task(F&& f);
    template <class F, class Allocator>
      packaged_task(allocator_arg_t, const Allocator& a, F&& f);
    ~packaged_task();

    // no copy
    packaged_task(const packaged_task&) = delete;
    packaged_task& operator=(const packaged_task&) = delete;

    // move support
    packaged_task(packaged_task&& rhs) noexcept;
    packaged_task& operator=(packaged_task&& rhs) noexcept;
    void swap(packaged_task& other) noexcept;

    bool valid() const noexcept;

    // result retrieval
    future<R> get_future();

    // execution
    void operator()(ArgTypes... );
    void make_ready_at_thread_exit(ArgTypes...);

    void reset();
  };
  template <class R, class... ArgTypes>
    void swap(packaged_task<R(ArgTypes...)>& x, packaged_task<R(ArgTypes...)>& y) noexcept;
  template <class R, class Alloc>
    struct uses_allocator<packaged_task<R>, Alloc>;
}
```

30.6.9.1 `packaged_task` member functions [futures.task.members]

`packaged_task() noexcept;`

1 *Effects:* constructs a `packaged_task` object with no shared state and no stored task.

```
template <class F>
  packaged_task(F&& f);
template <class F, class Allocator>
  packaged_task(allocator_arg_t, const Allocator& a, F&& f);
```

2 *Requires:* *INVOKE*`(f, t1, t2, ..., tN, R)`, where `t1, t2, ..., tN` are values of the corresponding types in `ArgTypes...`, shall be a valid expression. Invoking a copy of `f` shall behave the same as invoking `f`.

3 *Remarks:* These constructors shall not participate in overload resolution if `decay_t<F>` is the same type as `std::packaged_task<R(ArgTypes...)>`.

4 *Effects:* constructs a new `packaged_task` object with a shared state and initializes the object's stored task with `std::forward<F>(f)`. The constructors that take an `Allocator` argument use it to allocate memory needed to store the internal data structures.

5 *Throws:* any exceptions thrown by the copy or move constructor of `f`, or `std::bad_alloc` if memory for the internal data structures could not be allocated.

```
packaged_task(packaged_task&& rhs) noexcept;
```

6 *Effects:* constructs a new `packaged_task` object and transfers ownership of `rhs`'s shared state to `*this`, leaving `rhs` with no shared state. Moves the stored task from `rhs` to `*this`.

7 *Postcondition:* `rhs` has no shared state.

```
packaged_task& operator=(packaged_task&& rhs) noexcept;
```

8 *Effects:*

(8.1) — releases any shared state (30.6.4).

(8.2) — `packaged_task(std::move(rhs)).swap(*this)`.

```
~packaged_task();
```

9 *Effects:* Abandons any shared state. (30.6.4).

```
void swap(packaged_task& other) noexcept;
```

10 *Effects:* exchanges the shared states and stored tasks of `*this` and `other`.

11 *Postcondition:* `*this` has the same shared state and stored task (if any) as `other` prior to the call to `swap`. `other` has the same shared state and stored task (if any) as `*this` prior to the call to `swap`.

```
bool valid() const noexcept;
```

12 *Returns:* `true` only if `*this` has a shared state.

```
future<R> get_future();
```

13 *Returns:* A `future` object that shares the same shared state as `*this`.

14 *Throws:* a `future_error` object if an error occurs.

15 *Error conditions:*

(15.1) — `future_already_retrieved` if `get_future` has already been called on a `packaged_task` object with the same shared state as `*this`.

(15.2) — `no_state` if `*this` has no shared state.

```
void operator()(ArgTypes... args);
```

16 *Effects:* *INVOKE*`(f, t1, t2, ..., tN, R)`, where `f` is the stored task of `*this` and `t1, t2, ..., tN` are the values in `args...`. If the task returns normally, the return value is stored as the asynchronous result in the shared state of `*this`, otherwise the exception thrown by the task is stored. The shared state of `*this` is made ready, and any threads blocked in a function waiting for the shared state of `*this` to become ready are unblocked.

17 *Throws:* a `future_error` exception object if there is no shared state or the stored task has already been invoked.

18 *Error conditions:*

(18.1) — `promise_already_satisfied` if the stored task has already been invoked.

(18.2) — `no_state` if `*this` has no shared state.

```
void make_ready_at_thread_exit(ArgTypes... args);
```

19 *Effects:* *INVOKE*`(f, t1, t2, ..., tN, R)`, where `f` is the stored task and `t1, t2, ..., tN` are the values in `args...`. If the task returns normally, the return value is stored as the asynchronous result in the shared state of `*this`, otherwise the exception thrown by the task is stored. In either case, this shall be done without making that state ready (30.6.4) immediately. Schedules the shared state to be made ready when the current thread exits, after all objects of thread storage duration associated with the current thread have been destroyed.

20 *Throws:* `future_error` if an error condition occurs.

21 *Error conditions:*

(21.1) — `promise_already_satisfied` if the stored task has already been invoked.

(21.2) — `no_state` if `*this` has no shared state.

```
void reset();
```

22 *Effects:* as if `*this = packaged_task(std::move(f))`, where `f` is the task stored in `*this`. [*Note:* This constructs a new shared state for `*this`. The old state is abandoned (30.6.4). — *end note*]

23 *Throws:*

(23.1) — `bad_alloc` if memory for the new shared state could not be allocated.

(23.2) — any exception thrown by the move constructor of the task stored in the shared state.

(23.3) — `future_error` with an error condition of `no_state` if `*this` has no shared state.

30.6.9.2 `packaged_task` globals [futures.task.nonmembers]

```
template <class R, class... ArgTypes>
  void swap(packaged_task<R(ArgTypes...)>& x, packaged_task<R(ArgTypes...)>& y) noexcept;
```

1 *Effects:* `x.swap(y)`

```
template <class R, class Alloc>
  struct uses_allocator<packaged_task<R>, Alloc>
    : true_type { };
```

2 *Requires:* `Alloc` shall be an Allocator (17.6.3.5).

Annex A (informative) Grammar summary [gram]

1 This summary of C++ syntax is intended to be an aid to comprehension. It is not an exact statement of the language. In particular, the grammar described here accepts a superset of valid C++ constructs. Disambiguation rules (6.8, 7.1, 10.2) must be applied to distinguish expressions from declarations. Further, access control, ambiguity, and type rules must be used to weed out syntactically valid but meaningless constructs.

A.1 Keywords [gram.key]

1 New context-dependent keywords are introduced into a program by `typedef` (7.1.3), `namespace` (7.3.1), class (Clause 9), enumeration (7.2), and `template` (Clause 14) declarations.

typedef-name:
identifier
namespace-name:
original-namespace-name
namespace-alias

original-namespace-name:
identifier

namespace-alias:
identifier

class-name:
identifier
simple-template-id
enum-name:
identifier

template-name:
identifier

Note that a *typedef-name* naming a class is also a *class-name* (9.1).

A.2 Lexical conventions [gram.lex]

hex-quad:
hexadecimal-digit hexadecimal-digit hexadecimal-digit hexadecimal-digit

universal-character-name:
`\u` *hex-quad*
`\U` *hex-quad hex-quad*

preprocessing-token:
header-name
identifier
pp-number
character-literal
user-defined-character-literal
string-literal
user-defined-string-literal
preprocessing-op-or-punc
each non-white-space character that cannot be one of the above

token:
identifier
keyword
literal
operator
punctuator

header-name:
`<` *h-char-sequence* `>`
`"` *q-char-sequence* `"`

h-char-sequence:
h-char
h-char-sequence h-char

h-char:
any member of the source character set except new-line and `>`

q-char-sequence:
q-char
q-char-sequence q-char

q-char:
any member of the source character set except new-line and `"`

pp-number:
digit
`.` *digit*
pp-number digit
pp-number identifier-nondigit
pp-number `'` *digit*
pp-number `'` *nondigit*
pp-number `e` *sign*
pp-number `E` *sign*
pp-number `.`

identifier:
identifier-nondigit
identifier identifier-nondigit
identifier digit

identifier-nondigit:
nondigit
universal-character-name

nondigit: one of
`a b c d e f g h i j k l m`
`n o p q r s t u v w x y z`
`A B C D E F G H I J K L M`
`N O P Q R S T U V W X Y Z _`

digit: one of
`0 1 2 3 4 5 6 7 8 9`

preprocessing-op-or-punc: one of

`{`	`}`	`[`	`]`	`#`	`##`	`(`	`)`	
`<:`	`:>`	`<%`	`%>`	`%:`	`%:%:`	`;`	`:`	`...`
`new`	`delete`	`?`	`::`	`.`	`.*`			
`+`	`-`	`*`	`/`	`%`	`^`	`&`	`|`	`~`
`!`	`=`	`<`	`>`	`+=`	`-=`	`*=`	`/=`	`%=`
`^=`	`&=`	`|=`	`<<`	`>>`	`>>=`	`<<=`	`==`	`!=`
`<=`	`>=`	`&&`	`||`	`++`	`--`	`,`	`->*`	`->`
`and`	`and_eq`	`bitand`	`bitor`	`compl`	`not`	`not_eq`		
`or`	`or_eq`	`xor`	`xor_eq`					

literal:
- *integer-literal*
- *character-literal*
- *floating-literal*
- *string-literal*
- *boolean-literal*
- *pointer-literal*
- *user-defined-literal*

integer-literal:
- *binary-literal integer-suffix*$_{opt}$
- *octal-literal integer-suffix*$_{opt}$
- *decimal-literal integer-suffix*$_{opt}$
- *hexadecimal-literal integer-suffix*$_{opt}$

binary-literal:
- `0b` *binary-digit*
- `0B` *binary-digit*
- *binary-literal* `'`$_{opt}$ *binary-digit*

octal-literal:
- `0`
- *octal-literal* `'`$_{opt}$ *octal-digit*

decimal-literal:
- *nonzero-digit*
- *decimal-literal* `'`$_{opt}$ *digit*

hexadecimal-literal:
- `0x` *hexadecimal-digit*
- `0X` *hexadecimal-digit*
- *hexadecimal-literal* `'`$_{opt}$ *hexadecimal-digit*

binary-digit:
- `0`
- `1`

octal-digit: one of
`0 1 2 3 4 5 6 7`

nonzero-digit: one of
`1 2 3 4 5 6 7 8 9`

hexadecimal-digit: one of
`0 1 2 3 4 5 6 7 8 9`
`a b c d e f`
`A B C D E F`

integer-suffix:
- *unsigned-suffix long-suffix*$_{opt}$
- *unsigned-suffix long-long-suffix*$_{opt}$
- *long-suffix unsigned-suffix*$_{opt}$
- *long-long-suffix unsigned-suffix*$_{opt}$

unsigned-suffix: one of
`u U`

long-suffix: one of
`l L`

long-long-suffix: one of
`ll LL`

character-literal:
- *encoding-prefix*$_{opt}$`'` *c-char-sequence* `'`

encoding-prefix: one of
`u8 u U L`

c-char-sequence:
c-char
c-char-sequence c-char

c-char:
any member of the source character set except
the single-quote ', backslash \, or new-line character
escape-sequence
universal-character-name

escape-sequence:
simple-escape-sequence
octal-escape-sequence
hexadecimal-escape-sequence

simple-escape-sequence: one of
`\'` `\"` `\?` `\\`
`\a` `\b` `\f` `\n` `\r` `\t` `\v`

octal-escape-sequence:
`\` *octal-digit*
`\` *octal-digit octal-digit*
`\` *octal-digit octal-digit octal-digit*

hexadecimal-escape-sequence:
`\x` *hexadecimal-digit*
hexadecimal-escape-sequence hexadecimal-digit

floating-literal:
fractional-constant exponent-part$_{opt}$ *floating-suffix*$_{opt}$
digit-sequence exponent-part floating-suffix$_{opt}$

fractional-constant:
digit-sequence$_{opt}$`.` *digit-sequence*
digit-sequence `.`

exponent-part:
`e` *sign*$_{opt}$ *digit-sequence*
`E` *sign*$_{opt}$ *digit-sequence*

sign: one of
`+ -`

digit-sequence:
digit
digit-sequence '$_{opt}$ *digit*

floating-suffix: one of
`f l F L`

string-literal:
encoding-prefix$_{opt}$`"` *s-char-sequence*$_{opt}$`"`
encoding-prefix$_{opt}$`R` *raw-string*

s-char-sequence:
s-char
s-char-sequence s-char

s-char:
any member of the source character set except
the double-quote ", backslash \, or new-line character
escape-sequence
universal-character-name

raw-string:
`"` *d-char-sequence*$_{opt}$`(` *r-char-sequence*$_{opt}$`)` *d-char-sequence*$_{opt}$`"`

r-char-sequence:
r-char
r-char-sequence r-char

r-char:
any member of the source character set, except
a right parenthesis `)` followed by the initial *d-char-sequence* (which may be empty) followed by a double quote `"`.

d-char-sequence:
d-char
d-char-sequence d-char

d-char:
any member of the basic source character set except:
space, the left parenthesis `(`, the right parenthesis `)`, the backslash `\`, and the control characters representing horizontal tab, vertical tab, form feed, and newline.

boolean-literal:
`false`
`true`

pointer-literal:
`nullptr`

user-defined-literal:
user-defined-integer-literal
user-defined-floating-literal
user-defined-string-literal
user-defined-character-literal

user-defined-integer-literal:
decimal-literal ud-suffix
octal-literal ud-suffix
hexadecimal-literal ud-suffix
binary-literal ud-suffix

user-defined-floating-literal:
fractional-constant exponent-part$_{opt}$ *ud-suffix*
digit-sequence exponent-part ud-suffix

user-defined-string-literal:
string-literal ud-suffix

user-defined-character-literal:
character-literal ud-suffix

ud-suffix:
identifier

A.3 Basic concepts [gram.basic]

translation-unit:
declaration-seq$_{opt}$

A.4 Expressions [gram.expr]

primary-expression:
literal
`this`
`(` *expression* `)`
id-expression
lambda-expression
fold-expression

id-expression:
unqualified-id
qualified-id

unqualified-id:
identifier
operator-function-id
conversion-function-id
literal-operator-id
`~` *class-name*
`~` *decltype-specifier*
template-id

qualified-id:
nested-name-specifier `template`$_{opt}$ *unqualified-id*

nested-name-specifier:
`::`
type-name `::`
namespace-name `::`
decltype-specifier `::`
nested-name-specifier identifier `::`
nested-name-specifier `template`$_{opt}$ *simple-template-id* `::`

lambda-expression:
lambda-introducer lambda-declarator$_{opt}$ *compound-statement*

lambda-introducer:
`[` *lambda-capture*$_{opt}$`]`

lambda-capture:
capture-default
capture-list
capture-default `,` *capture-list*

capture-default:
`&`
`=`

capture-list:
capture `...`$_{opt}$
capture-list `,` *capture* `...`$_{opt}$

capture:
simple-capture
init-capture

simple-capture:
identifier
`&` *identifier*
`this`

init-capture:
identifier initializer
`&` *identifier initializer*

lambda-declarator:
`(` *parameter-declaration-clause* `)` `mutable`$_{opt}$
exception-specification$_{opt}$ *attribute-specifier-seq*$_{opt}$ *trailing-return-type*$_{opt}$

fold-expression:
`(` *cast-expression fold-operator* `...` `)`
`(` `...` *fold-operator cast-expression* `)`
`(` *cast-expression fold-operator* `...` *fold-operator cast-expression* `)`

fold-operator: one of

```
+    -    *    /    %    ^    &    |    <<    >>
+=   -=   *=   /=   %=   ^=   &=   |=   <<=   >>=   =
==   !=   <    >    <=   >=   &&   ||   ,     .*    ->*
```

postfix-expression:

primary-expression
postfix-expression `[` *expression* `]`
postfix-expression `[` *braced-init-list* `]`
postfix-expression `(` *expression-list*$_{opt}$`)`
simple-type-specifier `(` *expression-list*$_{opt}$`)`
typename-specifier `(` *expression-list*$_{opt}$`)`
simple-type-specifier braced-init-list
typename-specifier braced-init-list
postfix-expression `.` `template`$_{opt}$ *id-expression*
postfix-expression `->` `template`$_{opt}$ *id-expression*
postfix-expression `.` *pseudo-destructor-name*
postfix-expression `->` *pseudo-destructor-name*
postfix-expression `++`
postfix-expression `--`
`dynamic_cast` `<` *type-id* `>` `(` *expression* `)`
`static_cast` `<` *type-id* `>` `(` *expression* `)`
`reinterpret_cast` `<` *type-id* `>` `(` *expression* `)`
`const_cast` `<` *type-id* `>` `(` *expression* `)`
`typeid` `(` *expression* `)`
`typeid` `(` *type-id* `)`

expression-list:

initializer-list

pseudo-destructor-name:

nested-name-specifier$_{opt}$ *type-name* `::` `~` *type-name*
nested-name-specifier `template` *simple-template-id* `::` `~` *type-name*
`~` *type-name*
`~` *decltype-specifier*

unary-expression:

postfix-expression
`++` *cast-expression*
`--` *cast-expression*
unary-operator cast-expression
`sizeof` *unary-expression*
`sizeof` `(` *type-id* `)`
`sizeof` `...` `(` *identifier* `)`
`alignof` `(` *type-id* `)`
noexcept-expression
new-expression
delete-expression

unary-operator: one of

```
*  &  +  -  !  ~
```

new-expression:

`::`$_{opt}$`new` *new-placement*$_{opt}$ *new-type-id new-initializer*$_{opt}$
`::`$_{opt}$`new` *new-placement*$_{opt}$`(` *type-id* `)` *new-initializer*$_{opt}$

new-placement:

`(` *expression-list* `)`

new-type-id:

type-specifier-seq new-declarator$_{opt}$

new-declarator:
ptr-operator new-declarator$_{opt}$
noptr-new-declarator

noptr-new-declarator:
`[` *expression* `]` *attribute-specifier-seq*$_{opt}$
noptr-new-declarator `[` *constant-expression* `]` *attribute-specifier-seq*$_{opt}$

new-initializer:
`(` *expression-list*$_{opt}$`)`
braced-init-list

delete-expression:
`::`$_{opt}$`delete` *cast-expression*
`::`$_{opt}$`delete [ ]` *cast-expression*

noexcept-expression:
`noexcept (` *expression* `)`

cast-expression:
unary-expression
`(` *type-id* `)` *cast-expression*

pm-expression:
cast-expression
pm-expression `.*` *cast-expression*
pm-expression `->*` *cast-expression*

multiplicative-expression:
pm-expression
multiplicative-expression `*` *pm-expression*
multiplicative-expression `/` *pm-expression*
multiplicative-expression `%` *pm-expression*

additive-expression:
multiplicative-expression
additive-expression `+` *multiplicative-expression*
additive-expression `-` *multiplicative-expression*

shift-expression:
additive-expression
shift-expression `<<` *additive-expression*
shift-expression `>>` *additive-expression*

relational-expression:
shift-expression
relational-expression `<` *shift-expression*
relational-expression `>` *shift-expression*
relational-expression `<=` *shift-expression*
relational-expression `>=` *shift-expression*

equality-expression:
relational-expression
equality-expression `==` *relational-expression*
equality-expression `!=` *relational-expression*

and-expression:
equality-expression
and-expression `&` *equality-expression*

exclusive-or-expression:
and-expression
exclusive-or-expression `^` *and-expression*

inclusive-or-expression:
exclusive-or-expression
inclusive-or-expression `|` *exclusive-or-expression*

logical-and-expression:
inclusive-or-expression
logical-and-expression `&&` *inclusive-or-expression*

logical-or-expression:
logical-and-expression
logical-or-expression || *logical-and-expression*

conditional-expression:
logical-or-expression
logical-or-expression `?` *expression* `:` *assignment-expression*

throw-expression:
`throw` *assignment-expression*$_{opt}$

assignment-expression:
conditional-expression
logical-or-expression assignment-operator initializer-clause
throw-expression

assignment-operator: one of
`= *= /= %= += -= >>= <<= &= ^= |=`

expression:
assignment-expression
expression `,` *assignment-expression*

constant-expression:
conditional-expression

A.5 Statements [gram.stmt]

statement:
labeled-statement
attribute-specifier-seq$_{opt}$ *expression-statement*
attribute-specifier-seq$_{opt}$ *compound-statement*
attribute-specifier-seq$_{opt}$ *selection-statement*
attribute-specifier-seq$_{opt}$ *iteration-statement*
attribute-specifier-seq$_{opt}$ *jump-statement*
declaration-statement
attribute-specifier-seq$_{opt}$ *try-block*

labeled-statement:
attribute-specifier-seq$_{opt}$ *identifier* `:` *statement*
attribute-specifier-seq$_{opt}$`case` *constant-expression* `:` *statement*
attribute-specifier-seq$_{opt}$`default` `:` *statement*

expression-statement:
expression$_{opt}$`;`

compound-statement:
`{` *statement-seq*$_{opt}$`}`

statement-seq:
statement
statement-seq statement

selection-statement:
`if` `(` *condition* `)` *statement*
`if` `(` *condition* `)` *statement* `else` *statement*
`switch` `(` *condition* `)` *statement*

condition:
expression
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator* `=` *initializer-clause*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator braced-init-list*

iteration-statement:
`while` `(` *condition* `)` *statement*
`do` *statement* `while` `(` *expression* `)` `;`
`for` `(` *for-init-statement* *condition*$_{opt}$`;` *expression*$_{opt}$`)` *statement*
`for` `(` *for-range-declaration* `:` *for-range-initializer* `)` *statement*

for-init-statement:
expression-statement
simple-declaration

for-range-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator*

for-range-initializer:
expression
braced-init-list

jump-statement:
`break` `;`
`continue` `;`
`return` *expression*$_{opt}$`;`
`return` *braced-init-list* `;`
`goto` *identifier* `;`

declaration-statement:
block-declaration

A.6 Declarations [gram.dcl]

declaration-seq:
declaration
declaration-seq declaration

declaration:
block-declaration
function-definition
template-declaration
explicit-instantiation
explicit-specialization
linkage-specification
namespace-definition
empty-declaration
attribute-declaration

block-declaration:
simple-declaration
asm-definition
namespace-alias-definition
using-declaration
using-directive
static_assert-declaration
alias-declaration
opaque-enum-declaration

alias-declaration:
`using` *identifier attribute-specifier-seq*$_{opt}$`=` *type-id* `;`

simple-declaration:
decl-specifier-seq$_{opt}$ *init-declarator-list*$_{opt}$`;`
attribute-specifier-seq decl-specifier-seq$_{opt}$ *init-declarator-list* `;`

static_assert-declaration:
`static_assert` `(` *constant-expression* `)` `;`
`static_assert` `(` *constant-expression* `,` *string-literal* `)` `;`

empty-declaration:
`;`
attribute-declaration:
attribute-specifier-seq `;`
decl-specifier:
storage-class-specifier
type-specifier
function-specifier
`friend`
`typedef`
`constexpr`
decl-specifier-seq:
decl-specifier attribute-specifier-seq$_{opt}$
decl-specifier decl-specifier-seq
storage-class-specifier:
`register`
`static`
`thread_local`
`extern`
`mutable`
function-specifier:
`inline`
`virtual`
`explicit`
typedef-name:
identifier
type-specifier:
trailing-type-specifier
class-specifier
enum-specifier
trailing-type-specifier:
simple-type-specifier
elaborated-type-specifier
typename-specifier
cv-qualifier
type-specifier-seq:
type-specifier attribute-specifier-seq$_{opt}$
type-specifier type-specifier-seq
trailing-type-specifier-seq:
trailing-type-specifier attribute-specifier-seq$_{opt}$
trailing-type-specifier trailing-type-specifier-seq

simple-type-specifier:
nested-name-specifier$_{opt}$ *type-name*
nested-name-specifier `template` *simple-template-id*
`char`
`char16_t`
`char32_t`
`wchar_t`
`bool`
`short`
`int`
`long`
`signed`
`unsigned`
`float`
`double`
`void`
`auto`
decltype-specifier

type-name:
class-name
enum-name
typedef-name
simple-template-id

decltype-specifier:
`decltype (` *expression* `)`
`decltype ( auto )`

elaborated-type-specifier:
class-key attribute-specifier-seq$_{opt}$ *nested-name-specifier*$_{opt}$ *identifier*
class-key simple-template-id
class-key nested-name-specifier `template`$_{opt}$ *simple-template-id*
`enum` *nested-name-specifier*$_{opt}$ *identifier*

enum-name:
identifier

enum-specifier:
enum-head `{` *enumerator-list*$_{opt}$`}`
enum-head `{` *enumerator-list* `,` `}`

enum-head:
enum-key attribute-specifier-seq$_{opt}$ *identifier*$_{opt}$ *enum-base*$_{opt}$
enum-key attribute-specifier-seq$_{opt}$ *nested-name-specifier identifier*
enum-base$_{opt}$

opaque-enum-declaration:
enum-key attribute-specifier-seq$_{opt}$ *identifier enum-base*$_{opt}$`;`

enum-key:
`enum`
`enum class`
`enum struct`

enum-base:
`:` *type-specifier-seq*

enumerator-list:
enumerator-definition
enumerator-list `,` *enumerator-definition*

enumerator-definition:
enumerator
enumerator `=` *constant-expression*

enumerator:
identifier attribute-specifier-seq$_{opt}$
namespace-name:
identifier
namespace-alias
namespace-definition:
named-namespace-definition
unnamed-namespace-definition
nested-namespace-definition
named-namespace-definition:
`inline`$_{opt}$ `namespace` *attribute-specifier-seq*$_{opt}$ *identifier* `{` *namespace-body* `}`
unnamed-namespace-definition:
`inline`$_{opt}$ `namespace` *attribute-specifier-seq*$_{opt}$`{` *namespace-body* `}`
nested-namespace-definition:
`namespace` *enclosing-namespace-specifier* `::` *identifier* `{` *namespace-body* `}`
enclosing-namespace-specifier:
identifier
enclosing-namespace-specifier `::` *identifier*
namespace-body:
declaration-seq$_{opt}$
namespace-alias:
identifier
namespace-alias-definition:
`namespace` *identifier* `=` *qualified-namespace-specifier* `;`
qualified-namespace-specifier:
nested-name-specifier$_{opt}$ *namespace-name*
using-declaration:
`using typename`$_{opt}$ *nested-name-specifier unqualified-id* `;`

using-directive:
attribute-specifier-seq$_{opt}$`using namespace` *nested-name-specifier*$_{opt}$ *namespace-name* `;`
asm-definition:
`asm (` *string-literal* `) ;`
linkage-specification:
`extern` *string-literal* `{` *declaration-seq*$_{opt}$`}`
`extern` *string-literal declaration*
attribute-specifier-seq:
attribute-specifier-seq$_{opt}$ *attribute-specifier*
attribute-specifier:
`[ [` *attribute-list* `] ]`
alignment-specifier
alignment-specifier:
`alignas (` *type-id* `...`$_{opt}$`)`
`alignas (` *constant-expression* `...`$_{opt}$`)`
attribute-list:
attribute$_{opt}$
attribute-list `,` *attribute*$_{opt}$
attribute `...`
attribute-list `,` *attribute* `...`
attribute:
attribute-token attribute-argument-clause$_{opt}$

attribute-token:
identifier
attribute-scoped-token

attribute-scoped-token:
attribute-namespace `::` *identifier*

attribute-namespace:
identifier

attribute-argument-clause:
`(` *balanced-token-seq* `)`

balanced-token-seq:
balanced-token$_{opt}$
balanced-token-seq balanced-token

balanced-token:
`(` *balanced-token-seq* `)`
`[` *balanced-token-seq* `]`
`{` *balanced-token-seq* `}`
any *token* other than a parenthesis, a bracket, or a brace

A.7 Declarators [gram.decl]

init-declarator-list:
init-declarator
init-declarator-list `,` *init-declarator*

init-declarator:
declarator initializer$_{opt}$

declarator:
ptr-declarator
noptr-declarator parameters-and-qualifiers trailing-return-type

ptr-declarator:
noptr-declarator
ptr-operator ptr-declarator

noptr-declarator:
declarator-id attribute-specifier-seq$_{opt}$
noptr-declarator parameters-and-qualifiers
noptr-declarator `[` *constant-expression*$_{opt}$`]` *attribute-specifier-seq*$_{opt}$
`(` *ptr-declarator* `)`

parameters-and-qualifiers:
`(` *parameter-declaration-clause* `)` *cv-qualifier-seq*$_{opt}$
ref-qualifier$_{opt}$ *exception-specification*$_{opt}$ *attribute-specifier-seq*$_{opt}$

trailing-return-type:
`->` *trailing-type-specifier-seq abstract-declarator*$_{opt}$

ptr-operator:
`*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$
`&` *attribute-specifier-seq*$_{opt}$
`&&` *attribute-specifier-seq*$_{opt}$
nested-name-specifier `*` *attribute-specifier-seq*$_{opt}$ *cv-qualifier-seq*$_{opt}$

cv-qualifier-seq:
cv-qualifier cv-qualifier-seq$_{opt}$

cv-qualifier:
`const`
`volatile`

ref-qualifier:
`&`
`&&`

declarator-id:
...$_{opt}$ *id-expression*

type-id:
type-specifier-seq abstract-declarator$_{opt}$

abstract-declarator:
ptr-abstract-declarator
noptr-abstract-declarator$_{opt}$ *parameters-and-qualifiers trailing-return-type*
abstract-pack-declarator

ptr-abstract-declarator:
noptr-abstract-declarator
ptr-operator ptr-abstract-declarator$_{opt}$

noptr-abstract-declarator:
noptr-abstract-declarator$_{opt}$ *parameters-and-qualifiers*
noptr-abstract-declarator$_{opt}$ `[` *constant-expression*$_{opt}$ `]` *attribute-specifier-seq*$_{opt}$
`(` *ptr-abstract-declarator* `)`

abstract-pack-declarator:
noptr-abstract-pack-declarator
ptr-operator abstract-pack-declarator

noptr-abstract-pack-declarator:
noptr-abstract-pack-declarator parameters-and-qualifiers
noptr-abstract-pack-declarator `[` *constant-expression*$_{opt}$ `]` *attribute-specifier-seq*$_{opt}$
...

parameter-declaration-clause:
parameter-declaration-list$_{opt}$ *...*$_{opt}$
parameter-declaration-list `,` *...*

parameter-declaration-list:
parameter-declaration
parameter-declaration-list `,` *parameter-declaration*

parameter-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq declarator* `=` *initializer-clause*
attribute-specifier-seq$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$
attribute-specifier-seq$_{opt}$ *decl-specifier-seq abstract-declarator*$_{opt}$`=` *initializer-clause*

function-definition:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *declarator virt-specifier-seq*$_{opt}$ *function-body*

function-body:
ctor-initializer$_{opt}$ *compound-statement*
function-try-block
`= default ;`
`= delete ;`

initializer:
brace-or-equal-initializer
`(` *expression-list* `)`

brace-or-equal-initializer:
`=` *initializer-clause*
braced-init-list

initializer-clause:
assignment-expression
braced-init-list

initializer-list:
initializer-clause ...$_{opt}$
initializer-list `,` *initializer-clause ...*$_{opt}$

braced-init-list:
`{` *initializer-list* `,`$_{opt}$ `}`
`{ }`

A.8 Classes [gram.class]

class-name:
identifier
simple-template-id

class-specifier:
class-head `{` *member-specification*$_{opt}$`}`

class-head:
class-key attribute-specifier-seq$_{opt}$ *class-head-name class-virt-specifier*$_{opt}$ *base-clause*$_{opt}$
class-key attribute-specifier-seq$_{opt}$ *base-clause*$_{opt}$

class-head-name:
nested-name-specifier$_{opt}$ *class-name*

class-virt-specifier:
`final`

class-key:
`class`
`struct`
`union`

member-specification:
member-declaration member-specification$_{opt}$
access-specifier `:` *member-specification*$_{opt}$

member-declaration:
attribute-specifier-seq$_{opt}$ *decl-specifier-seq*$_{opt}$ *member-declarator-list*$_{opt}$ `;`
function-definition
using-declaration
static_assert-declaration
template-declaration
alias-declaration
empty-declaration

member-declarator-list:
member-declarator
member-declarator-list `,` *member-declarator*

member-declarator:
declarator virt-specifier-seq$_{opt}$ *pure-specifier*$_{opt}$
declarator brace-or-equal-initializer$_{opt}$
identifier$_{opt}$ *attribute-specifier-seq*$_{opt}$`:` *constant-expression*

virt-specifier-seq:
virt-specifier
virt-specifier-seq virt-specifier

virt-specifier:
`override`
`final`

pure-specifier:
`= 0`

A.9 Derived classes [gram.derived]

base-clause:
`:` *base-specifier-list*

base-specifier-list:
base-specifier `...`$_{opt}$
base-specifier-list `,` *base-specifier* `...`$_{opt}$

base-specifier:
attribute-specifier-seq$_{opt}$ *base-type-specifier*
attribute-specifier-seq$_{opt}$ `virtual` *access-specifier*$_{opt}$ *base-type-specifier*
attribute-specifier-seq$_{opt}$ *access-specifier* `virtual`$_{opt}$ *base-type-specifier*

class-or-decltype:
nested-name-specifier$_{opt}$ *class-name*
decltype-specifier

base-type-specifier:
class-or-decltype

access-specifier:
`private`
`protected`
`public`

A.10 Special member functions [gram.special]

conversion-function-id:
`operator` *conversion-type-id*

conversion-type-id:
type-specifier-seq conversion-declarator$_{opt}$

conversion-declarator:
ptr-operator conversion-declarator$_{opt}$

ctor-initializer:
`:` *mem-initializer-list*

mem-initializer-list:
mem-initializer `...`$_{opt}$
mem-initializer-list `,` *mem-initializer* `...`$_{opt}$

mem-initializer:
mem-initializer-id `(` *expression-list*$_{opt}$`)`
mem-initializer-id braced-init-list

mem-initializer-id:
class-or-decltype
identifier

A.11 Overloading [gram.over]

operator-function-id:
`operator` *operator*

operator: one of

```
new   delete   new[]    delete[]
+     -        *        /        %        ^        &        |        ~
!     =        <        >        +=       -=       *=       /=       %=
^=    &=       |=       <<       >>       >>=      <<=      ==       !=
<=    >=       &&       ||       ++       --       ,        ->*      ->
()    []
```

literal-operator-id:
`operator` *string-literal identifier*
`operator` *user-defined-string-literal*

A.12 Templates [gram.temp]

template-declaration:
`template` `<` *template-parameter-list* `>` *declaration*

template-parameter-list:
template-parameter
template-parameter-list `,` *template-parameter*

template-parameter:
type-parameter
parameter-declaration

type-parameter:
type-parameter-key ...$_{opt}$ *identifier*$_{opt}$
type-parameter-key identifier$_{opt}$ `=` *type-id*
`template` `<` *template-parameter-list* `>` *type-parameter-key* ...$_{opt}$ *identifier*$_{opt}$
`template` `<` *template-parameter-list* `>` *type-parameter-key identifier*$_{opt}$ `=` *id-expression*

type-parameter-key:
`class`
`typename`

simple-template-id:
template-name `<` *template-argument-list*$_{opt}$ `>`

template-id:
simple-template-id
operator-function-id `<` *template-argument-list*$_{opt}$ `>`
literal-operator-id `<` *template-argument-list*$_{opt}$ `>`

template-name:
identifier

template-argument-list:
template-argument ...$_{opt}$
template-argument-list `,` *template-argument* ...$_{opt}$

template-argument:
constant-expression
type-id
id-expression

typename-specifier:
`typename` *nested-name-specifier identifier*
`typename` *nested-name-specifier* `template`$_{opt}$ *simple-template-id*

explicit-instantiation:
`extern`$_{opt}$ `template` *declaration*

explicit-specialization:
`template` `<` `>` *declaration*

A.13 Exception handling [gram.except]

try-block:
`try` *compound-statement handler-seq*

function-try-block:
`try` *ctor-initializer*$_{opt}$ *compound-statement handler-seq*

handler-seq:
handler handler-seq$_{opt}$

handler:
`catch` `(` *exception-declaration* `)` *compound-statement*

exception-declaration:
attribute-specifier-seq$_{opt}$ *type-specifier-seq declarator*
attribute-specifier-seq$_{opt}$ *type-specifier-seq abstract-declarator*$_{opt}$
...

exception-specification:
dynamic-exception-specification
noexcept-specification

dynamic-exception-specification:
`throw` `(` *type-id-list*$_{opt}$ `)`

type-id-list:
type-id `...`$_{opt}$
type-id-list `,` *type-id* `...`$_{opt}$

noexcept-specification:
`noexcept (` *constant-expression* `)`
`noexcept`

A.14 Preprocessing directives [gram.cpp]

preprocessing-file:
group$_{opt}$

group:
group-part
group group-part

group-part:
if-section
control-line
text-line
`#` *non-directive*

if-section:
if-group elif-groups$_{opt}$ *else-group*$_{opt}$ *endif-line*

if-group:
`# if` *constant-expression new-line group*$_{opt}$
`# ifdef` *identifier new-line group*$_{opt}$
`# ifndef` *identifier new-line group*$_{opt}$

elif-groups:
elif-group
elif-groups elif-group

elif-group:
`# elif` *constant-expression new-line group*$_{opt}$

else-group:
`# else` *new-line group*$_{opt}$

endif-line:
`# endif` *new-line*

control-line:
`# include` *pp-tokens new-line*
`# define` *identifier replacement-list new-line*
`# define` *identifier lparen identifier-list*$_{opt}$`)` *replacement-list new-line*
`# define` *identifier lparen* `...` `)` *replacement-list new-line*
`# define` *identifier lparen identifier-list,* `...` `)` *replacement-list new-line*
`# undef` *identifier new-line*
`# line` *pp-tokens new-line*
`# error` *pp-tokens*$_{opt}$ *new-line*
`# pragma` *pp-tokens*$_{opt}$ *new-line*
`#` *new-line*

text-line:
pp-tokens$_{opt}$ *new-line*

non-directive:
pp-tokens new-line

lparen:
a `(` character not immediately preceded by white-space

identifier-list:
identifier
identifier-list **,** *identifier*
replacement-list:
pp-tokens$_{opt}$
pp-tokens:
preprocessing-token
pp-tokens preprocessing-token
new-line:
the new-line character

Annex B (informative) Implementation quantities [implimits]

1 Because computers are finite, C++ implementations are inevitably limited in the size of the programs they can successfully process. Every implementation shall document those limitations where known. This documentation may cite fixed limits where they exist, say how to compute variable limits as a function of available resources, or say that fixed limits do not exist or are unknown.

2 The limits may constrain quantities that include those described below or others. The bracketed number following each quantity is recommended as the minimum for that quantity. However, these quantities are only guidelines and do not determine compliance.

(2.1) — Nesting levels of compound statements, iteration control structures, and selection control structures [256].

(2.2) — Nesting levels of conditional inclusion [256].

(2.3) — Pointer, array, and function declarators (in any combination) modifying a class, arithmetic, or incomplete type in a declaration [256].

(2.4) — Nesting levels of parenthesized expressions within a full-expression [256].

(2.5) — Number of characters in an internal identifier or macro name [1 024].

(2.6) — Number of characters in an external identifier [1 024].

(2.7) — External identifiers in one translation unit [65 536].

(2.8) — Identifiers with block scope declared in one block [1 024].

(2.9) — Macro identifiers simultaneously defined in one translation unit [65 536].

(2.10) — Parameters in one function definition [256].

(2.11) — Arguments in one function call [256].

(2.12) — Parameters in one macro definition [256].

(2.13) — Arguments in one macro invocation [256].

(2.14) — Characters in one logical source line [65 536].

(2.15) — Characters in a string literal (after concatenation) [65 536].

(2.16) — Size of an object [262 144].

(2.17) — Nesting levels for `#include` files [256].

(2.18) — Case labels for a `switch` statement (excluding those for any nested `switch` statements) [16 384].

(2.19) — Data members in a single class [16 384].

(2.20) — Enumeration constants in a single enumeration [4 096].

(2.21) — Levels of nested class definitions in a single *member-specification* [256].

(2.22) — Functions registered by `atexit()` [32].

(2.23) — Functions registered by `at_quick_exit()` [32].

(2.24) — Direct and indirect base classes [16 384].

(2.25) — Direct base classes for a single class [1 024].

(2.26) — Members declared in a single class [4 096].

(2.27) — Final overriding virtual functions in a class, accessible or not [16 384].

(2.28) — Direct and indirect virtual bases of a class [1 024].

(2.29) — Static members of a class [1 024].

(2.30) — Friend declarations in a class [4 096].

(2.31) — Access control declarations in a class [4 096].

(2.32) — Member initializers in a constructor definition [6 144].

(2.33) — Scope qualifications of one identifier [256].

(2.34) — Nested external specifications [1 024].

(2.35) — Recursive `constexpr` function invocations [512].

(2.36) — Full-expressions evaluated within a core constant expression [1 048 576].

(2.37) — Template arguments in a template declaration [1 024].

(2.38) — Recursively nested template instantiations, including substitution during template argument deduction (14.8.2) [1 024].

(2.39) — Handlers per `try` block [256].

(2.40) — Throw specifications on a single function declaration [256].

(2.41) — Number of placeholders (20.9.10.4) [10].

Annex C (informative) Compatibility [diff]

C.1 C++ and ISO C [diff.iso]

1 This subclause lists the differences between C++ and ISO C, by the chapters of this document.

C.1.1 Clause 2: lexical conventions [diff.lex]

2.11
Change: New Keywords
New keywords are added to C++; see 2.11.
Rationale: These keywords were added in order to implement the new semantics of C++.
Effect on original feature: Change to semantics of well-defined feature. Any ISO C programs that used any of these keywords as identifiers are not valid C++ programs.
Difficulty of converting: Syntactic transformation. Converting one specific program is easy. Converting a large collection of related programs takes more work.
How widely used: Common.

2.13.3
Change: Type of character literal is changed from `int` to `char`
Rationale: This is needed for improved overloaded function argument type matching. For example:

```
int function( int i );
int function( char c );

function( 'x' );
```

It is preferable that this call match the second version of function rather than the first.
Effect on original feature: Change to semantics of well-defined feature. ISO C programs which depend on

```
sizeof('x') == sizeof(int)
```

will not work the same as C++ programs.
Difficulty of converting: Simple.
How widely used: Programs which depend upon `sizeof('x')` are probably rare.

Subclause 2.13.5:
Change: String literals made const
The type of a string literal is changed from "array of `char`" to "array of `const char`". The type of a `char16_t` string literal is changed from "array of *some-integer-type*" to "array of `const char16_t`". The type of a `char32_t` string literal is changed from "array of *some-integer-type*" to "array of `const char32_t`". The type of a wide string literal is changed from "array of `wchar_t`" to "array of `const wchar_t`".
Rationale: This avoids calling an inappropriate overloaded function, which might expect to be able to modify its argument.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Syntactic transformation. The fix is to add a cast:

```
char* p = "abc";              // valid in C, invalid in C++
void f(char*) {
  char* p = (char*)"abc";     // OK: cast added
```

```
  f(p);
  f((char*)"def");              // OK: cast added
}
```

How widely used: Programs that have a legitimate reason to treat string literals as pointers to potentially modifiable memory are probably rare.

C.1.2 Clause 3: basic concepts [diff.basic]

3.1
Change: C++ does not have "tentative definitions" as in C E.g., at file scope,

```
int i;
int i;
```

is valid in C, invalid in C++. This makes it impossible to define mutually referential file-local static objects, if initializers are restricted to the syntactic forms of C. For example,

```
struct X { int i; struct X* next; };

static struct X a;
static struct X b = { 0, &a };
static struct X a = { 1, &b };
```

Rationale: This avoids having different initialization rules for fundamental types and user-defined types.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation.
Rationale: In C++, the initializer for one of a set of mutually-referential file-local static objects must invoke a function call to achieve the initialization.
How widely used: Seldom.

3.3
Change: A `struct` is a scope in C++, not in C
Rationale: Class scope is crucial to C++, and a struct is a class.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Semantic transformation.
How widely used: C programs use `struct` extremely frequently, but the change is only noticeable when `struct`, enumeration, or enumerator names are referred to outside the `struct`. The latter is probably rare.

3.5 [also 7.1.6]
Change: A name of file scope that is explicitly declared `const`, and not explicitly declared `extern`, has internal linkage, while in C it would have external linkage
Rationale: Because `const` objects can be used as compile-time values in C++, this feature urges programmers to provide explicit initializer values for each `const`. This feature allows the user to put `const`objects in header files that are included in many compilation units.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Semantic transformation
How widely used: Seldom

3.6
Change: Main cannot be called recursively and cannot have its address taken
Rationale: The main function may require special actions.
Effect on original feature: Deletion of semantically well-defined feature

Difficulty of converting: Trivial: create an intermediary function such as `mymain(argc, argv)`.
How widely used: Seldom

3.9
Change: C allows "compatible types" in several places, C++ does not For example, otherwise-identical `struct` types with different tag names are "compatible" in C but are distinctly different types in C++.
Rationale: Stricter type checking is essential for C++.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. The "typesafe linkage" mechanism will find many, but not all, of such problems. Those problems not found by typesafe linkage will continue to function properly, according to the "layout compatibility rules" of this International Standard.
How widely used: Common.

C.1.3 Clause 4: standard conversions [diff.conv]

4.10
Change: Converting `void*` to a pointer-to-object type requires casting

```
char a[10];
void* b=a;
void foo() {
  char* c=b;
}
```

ISO C will accept this usage of pointer to void being assigned to a pointer to object type. C++ will not.
Rationale: C++ tries harder than C to enforce compile-time type safety.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Could be automated. Violations will be diagnosed by the C++ translator. The fix is to add a cast. For example:

```
char* c = (char*) b;
```

How widely used: This is fairly widely used but it is good programming practice to add the cast when assigning pointer-to-void to pointer-to-object. Some ISO C translators will give a warning if the cast is not used.

C.1.4 Clause 5: expressions [diff.expr]

5.2.2
Change: Implicit declaration of functions is not allowed
Rationale: The type-safe nature of C++.
Effect on original feature: Deletion of semantically well-defined feature. Note: the original feature was labeled as "obsolescent" in ISO C.
Difficulty of converting: Syntactic transformation. Facilities for producing explicit function declarations are fairly widespread commercially.
How widely used: Common.

5.3.3, 5.4
Change: Types must be declared in declarations, not in expressions In C, a sizeof expression or cast expression may create a new type. For example,

```
p = (void*)(struct x {int i;} *)0;
```

declares a new type, struct x .
Rationale: This prohibition helps to clarify the location of declarations in the source code.

Effect on original feature: Deletion of a semantically well-defined feature.
Difficulty of converting: Syntactic transformation.
How widely used: Seldom.

5.16, 5.18, 5.19

Change: The result of a conditional expression, an assignment expression, or a comma expression may be an lvalue
Rationale: C++ is an object-oriented language, placing relatively more emphasis on lvalues. For example, functions may return lvalues.
Effect on original feature: Change to semantics of well-defined feature. Some C expressions that implicitly rely on lvalue-to-rvalue conversions will yield different results. For example,

```
char arr[100];
sizeof(0, arr)
```

yields `100` in C++ and `sizeof(char*)` in C.
Difficulty of converting: Programs must add explicit casts to the appropriate rvalue.
How widely used: Rare.

C.1.5 Clause 6: statements [diff.stat]

6.4.2, 6.6.4
Change: It is now invalid to jump past a declaration with explicit or implicit initializer (except across entire block not entered)
Rationale: Constructors used in initializers may allocate resources which need to be de-allocated upon leaving the block. Allowing jump past initializers would require complicated run-time determination of allocation. Furthermore, any use of the uninitialized object could be a disaster. With this simple compile-time rule, C++ assures that if an initialized variable is in scope, then it has assuredly been initialized.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation.
How widely used: Seldom.

6.6.3
Change: It is now invalid to return (explicitly or implicitly) from a function which is declared to return a value without actually returning a value
Rationale: The caller and callee may assume fairly elaborate return-value mechanisms for the return of class objects. If some flow paths execute a return without specifying any value, the implementation must embody many more complications. Besides, promising to return a value of a given type, and then not returning such a value, has always been recognized to be a questionable practice, tolerated only because very-old C had no distinction between void functions and int functions.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. Add an appropriate return value to the source code, such as zero.
How widely used: Seldom. For several years, many existing C implementations have produced warnings in this case.

C.1.6 Clause 7: declarations [diff.dcl]

7.1.1
Change: In C++, the `static` or `extern` specifiers can only be applied to names of objects or functions Using these specifiers with type declarations is illegal in C++. In C, these specifiers are ignored when used on type declarations.

Example:

```
static struct S {                // valid C, invalid in C++
  int i;
};
```

Rationale: Storage class specifiers don't have any meaning when associated with a type. In C++, class members can be declared with the `static` storage class specifier. Allowing storage class specifiers on type declarations could render the code confusing for users.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Syntactic transformation.
How widely used: Seldom.

7.1.3
Change: A C++ typedef name must be different from any class type name declared in the same scope (except if the typedef is a synonym of the class name with the same name). In C, a typedef name and a struct tag name declared in the same scope can have the same name (because they have different name spaces)

Example:

```
typedef struct name1 { /*...*/ } name1;          // valid C and C++
struct name { /*...*/ };
typedef int name;                // valid C, invalid C++
```

Rationale: For ease of use, C++ doesn't require that a type name be prefixed with the keywords `class`, `struct` or `union` when used in object declarations or type casts.

Example:

```
class name { /*...*/ };
name i;                          // i has type class name
```

Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. One of the 2 types has to be renamed.
How widely used: Seldom.

7.1.6 [see also 3.5]
Change: const objects must be initialized in C++ but can be left uninitialized in C
Rationale: A const object cannot be assigned to so it must be initialized to hold a useful value.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation.
How widely used: Seldom.

7.1.6
Change: Banning implicit int

In C++ a *decl-specifier-seq* must contain a *type-specifier*, unless it is followed by a declarator for a constructor, a destructor, or a conversion function. In the following example, the left-hand column presents valid C; the right-hand column presents equivalent C++:

```
void f(const parm);              void f(const int parm);
const n = 3;                     const int n = 3;
main()                           int main()
    /* ... */                       /* ... */
```

Rationale: In C++, implicit int creates several opportunities for ambiguity between expressions involving function-like casts and declarations. Explicit declaration is increasingly considered to be proper style. Liaison with WG14 (C) indicated support for (at least) deprecating implicit int in the next revision of C.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Syntactic transformation. Could be automated.
How widely used: Common.

7.1.6.4
Change: The keyword `auto` cannot be used as a storage class specifier.

```
void f() {
  auto int x;      // valid C, invalid C++
}
```

Rationale: Allowing the use of `auto` to deduce the type of a variable from its initializer results in undesired interpretations of `auto` as a storage class specifier in certain contexts.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Syntactic transformation.
How widely used: Rare.

7.2
Change: C++ objects of enumeration type can only be assigned values of the same enumeration type. In C, objects of enumeration type can be assigned values of any integral type

Example:

```
enum color { red, blue, green };
enum color c = 1;                // valid C, invalid C++
```

Rationale: The type-safe nature of C++.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Syntactic transformation. (The type error produced by the assignment can be automatically corrected by applying an explicit cast.)
How widely used: Common.

7.2
Change: In C++, the type of an enumerator is its enumeration. In C, the type of an enumerator is `int`.

Example:

```
enum e { A };
sizeof(A) == sizeof(int)         // in C
sizeof(A) == sizeof(e)           // in C++
/* and sizeof(int) is not necessarily equal to sizeof(e) */
```

Rationale: In C++, an enumeration is a distinct type.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Semantic transformation.
How widely used: Seldom. The only time this affects existing C code is when the size of an enumerator is taken. Taking the size of an enumerator is not a common C coding practice.

C.1.7 Clause 8: declarators [diff.decl]

8.3.5

Change: In C++, a function declared with an empty parameter list takes no arguments. In C, an empty parameter list means that the number and type of the function arguments are unknown.

Example:

```
int f();            // means int f(void) in C++
                    // int f( unknown ) in C
```

Rationale: This is to avoid erroneous function calls (i.e., function calls with the wrong number or type of arguments).
Effect on original feature: Change to semantics of well-defined feature. This feature was marked as "obsolescent" in C.
Difficulty of converting: Syntactic transformation. The function declarations using C incomplete declaration style must be completed to become full prototype declarations. A program may need to be updated further if different calls to the same (non-prototype) function have different numbers of arguments or if the type of corresponding arguments differed.
How widely used: Common.

8.3.5 [see 5.3.3]
Change: In C++, types may not be defined in return or parameter types. In C, these type definitions are allowed

Example:

```
void f( struct S { int a; } arg ) {}    // valid C, invalid C++
enum E { A, B, C } f() {}               // valid C, invalid C++
```

Rationale: When comparing types in different compilation units, C++ relies on name equivalence when C relies on structural equivalence. Regarding parameter types: since the type defined in an parameter list would be in the scope of the function, the only legal calls in C++ would be from within the function itself.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. The type definitions must be moved to file scope, or in header files.
How widely used: Seldom. This style of type definitions is seen as poor coding style.

8.4
Change: In C++, the syntax for function definition excludes the "old-style" C function. In C, "old-style" syntax is allowed, but deprecated as "obsolescent."
Rationale: Prototypes are essential to type safety.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Syntactic transformation.
How widely used: Common in old programs, but already known to be obsolescent.

8.5.2
Change: In C++, when initializing an array of character with a string, the number of characters in the string (including the terminating `'\0'`) must not exceed the number of elements in the array. In C, an array can be initialized with a string even if the array is not large enough to contain the string-terminating `'\0'`

Example:

```
char array[4] = "abcd";          // valid C, invalid C++
```

Rationale: When these non-terminated arrays are manipulated by standard string routines, there is potential for major catastrophe.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. The arrays must be declared one element bigger to contain the string terminating `'\0'`.
How widely used: Seldom. This style of array initialization is seen as poor coding style.

C.1.8 Clause 9: classes [diff.class]

9.1 [see also 7.1.3]
Change: In C++, a class declaration introduces the class name into the scope where it is declared and hides any object, function or other declaration of that name in an enclosing scope. In C, an inner scope declaration of a struct tag name never hides the name of an object or function in an outer scope

Example:

```
int x[99];
void f() {
  struct x { int a; };
  sizeof(x);  /* size of the array in C */
  /* size of the struct in C++ */
}
```

Rationale: This is one of the few incompatibilities between C and C++ that can be attributed to the new C++ name space definition where a name can be declared as a type and as a non-type in a single scope causing the non-type name to hide the type name and requiring that the keywords `class`, `struct`, `union` or `enum` be used to refer to the type name. This new name space definition provides important notational conveniences to C++ programmers and helps making the use of the user-defined types as similar as possible to the use of fundamental types. The advantages of the new name space definition were judged to outweigh by far the incompatibility with C described above.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Semantic transformation. If the hidden name that needs to be accessed is at global scope, the `::` C++ operator can be used. If the hidden name is at block scope, either the type or the struct tag has to be renamed.
How widely used: Seldom.

9.6
Change: Bit-fields of type plain `int` are signed.
Rationale: Leaving the choice of signedness to implementations could lead to inconsistent definitions of template specializations. For consistency, the implementation freedom was eliminated for non-dependent types, too.
Effect on original feature: The choise is implementation-defined in C, but not so in C++.
Difficulty of converting: Syntactic transformation.
How widely used: Seldom.

9.7
Change: In C++, the name of a nested class is local to its enclosing class. In C the name of the nested class belongs to the same scope as the name of the outermost enclosing class.

Example:

```
struct X {
  struct Y { /* ... */ } y;
};
```

```
struct Y yy;                    // valid C, invalid C++
```

Rationale: C++ classes have member functions which require that classes establish scopes. The C rule would leave classes as an incomplete scope mechanism which would prevent C++ programmers from maintaining locality within a class. A coherent set of scope rules for C++ based on the C rule would be very complicated and C++ programmers would be unable to predict reliably the meanings of nontrivial examples involving nested or local functions.
Effect on original feature: Change of semantics of well-defined feature.
Difficulty of converting: Semantic transformation. To make the struct type name visible in the scope of the enclosing struct, the struct tag could be declared in the scope of the enclosing struct, before the enclosing struct is defined. Example:

```
struct Y;                       // struct Y and struct X are at the same scope
struct X {
  struct Y { /* ... */ } y;
};
```

All the definitions of C struct types enclosed in other struct definitions and accessed outside the scope of the enclosing struct could be exported to the scope of the enclosing struct. Note: this is a consequence of the difference in scope rules, which is documented in 3.3.
How widely used: Seldom.

9.9
Change: In C++, a typedef name may not be redeclared in a class definition after being used in that definition

Example:

```
typedef int I;
struct S {
  I i;
  int I;                    // valid C, invalid C++
};
```

Rationale: When classes become complicated, allowing such a redefinition after the type has been used can create confusion for C++ programmers as to what the meaning of 'I' really is.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. Either the type or the struct member has to be renamed.
How widely used: Seldom.

C.1.9 Clause 12: special member functions [diff.special]

12.8
Change: Copying volatile objects

The implicitly-declared copy constructor and implicitly-declared copy assignment operator cannot make a copy of a volatile lvalue. For example, the following is valid in ISO C:

```
struct X { int i; };
volatile struct X x1 = {0};
struct X x2(x1);                // invalid C++
struct X x3;
x3 = x1;                        // also invalid C++
```

Rationale: Several alternatives were debated at length. Changing the parameter to `volatile const X&` would greatly complicate the generation of efficient code for class objects. Discussion of providing two alternative signatures for these implicitly-defined operations raised unanswered concerns about creating ambiguities and complicating the rules that specify the formation of these operators according to the bases and members.
Effect on original feature: Deletion of semantically well-defined feature.
Difficulty of converting: Semantic transformation. If volatile semantics are required for the copy, a user-declared constructor or assignment must be provided. [*Note:* This user-declared constructor may be explicitly defaulted. — *end note*] If non-volatile semantics are required, an explicit `const_cast` can be used.
How widely used: Seldom.

C.1.10 Clause 16: preprocessing directives [diff.cpp]

16.8
Change: Whether `__STDC__` is defined and if so, what its value is, are implementation-defined
Rationale: C++ is not identical to ISO C. Mandating that `__STDC__` be defined would require that translators make an incorrect claim. Each implementation must choose the behavior that will be most useful to its marketplace.
Effect on original feature: Change to semantics of well-defined feature.
Difficulty of converting: Semantic transformation.
How widely used: Programs and headers that reference `__STDC__` are quite common.

C.2 C++ and ISO C++ 2003 [diff.cpp03]

1 This subclause lists the differences between C++ and ISO C++ 2003 (ISO/IEC 14882:2003, *Programming Languages — C++*), by the chapters of this document.

C.2.1 Clause 2: lexical conventions [diff.cpp03.lex]

2.4
Change: New kinds of string literals
Rationale: Required for new features.
Effect on original feature: Valid C++ 2003 code may fail to compile or produce different results in this International Standard. Specifically, macros named `R`, `u8`, `u8R`, `u`, `uR`, `U`, `UR`, or `LR` will not be expanded when adjacent to a string literal but will be interpreted as part of the string literal. For example,

```
#define u8 "abc"
const char* s = u8"def";        // Previously "abcdef", now "def"
```

2.4
Change: User-defined literal string support
Rationale: Required for new features.
Effect on original feature: Valid C++ 2003 code may fail to compile or produce different results in this International Standard, as the following example illustrates.

```
#define _x "there"
"hello"_x         // #1
```

Previously, #1 would have consisted of two separate preprocessing tokens and the macro `_x` would have been expanded. In this International Standard, #1 consists of a single preprocessing tokens, so the macro is not expanded.

2.11
Change: New keywords

Rationale: Required for new features.
Effect on original feature: Added to Table 3, the following identifiers are new keywords: `alignas`, `alignof`, `char16_t`, `char32_t`, `constexpr`, `decltype`, `noexcept`, `nullptr`, `static_assert`, and `thread_local`. Valid C++ 2003 code using these identifiers is invalid in this International Standard.

2.13.2
Change: Type of integer literals
Rationale: C99 compatibility.
Effect on original feature: Certain integer literals larger than can be represented by `long` could change from an unsigned integer type to `signed long long`.

C.2.2 Clause 4: standard conversions [diff.cpp03.conv]

4.10
Change: Only literals are integer null pointer constants
Rationale: Removing surprising interactions with templates and constant expressions
Effect on original feature: Valid C++ 2003 code may fail to compile or produce different results in this International Standard, as the following example illustrates:

```
void f(void *);  // #1
void f(...);     // #2
template<int N> void g() {
  f(0*N);        // calls #2; used to call #1
}
```

C.2.3 Clause 5: expressions [diff.cpp03.expr]

5.6
Change: Specify rounding for results of integer `/` and `%`
Rationale: Increase portability, C99 compatibility.
Effect on original feature: Valid C++ 2003 code that uses integer division rounds the result toward 0 or toward negative infinity, whereas this International Standard always rounds the result toward 0.

C.2.4 Clause 7: declarations [diff.cpp03.dcl.dcl]

7.1
Change: Remove `auto` as a storage class specifier
Rationale: New feature.
Effect on original feature: Valid C++ 2003 code that uses the keyword `auto` as a storage class specifier may be invalid in this International Standard. In this International Standard, `auto` indicates that the type of a variable is to be deduced from its initializer expression.

C.2.5 Clause 8: declarators [diff.cpp03.dcl.decl]

8.5.4
Change: Narrowing restrictions in aggregate initializers
Rationale: Catches bugs.
Effect on original feature: Valid C++ 2003 code may fail to compile in this International Standard. For example, the following code is valid in C++ 2003 but invalid in this International Standard because `double` to `int` is a narrowing conversion:

```
int x[] = { 2.0 };
```

C.2.6 Clause 12: special member functions [diff.cpp03.special]

12.1, 12.4, 12.8
Change: Implicitly-declared special member functions are defined as deleted when the implicit definition

would have been ill-formed.
Rationale: Improves template argument deduction failure.
Effect on original feature: A valid C++ 2003 program that uses one of these special member functions in a context where the definition is not required (e.g., in an expression that is not potentially evaluated) becomes ill-formed.

12.4 (destructors)
Change: User-declared destructors have an implicit exception specification.
Rationale: Clarification of destructor requirements.
Effect on original feature: Valid C++ 2003 code may execute differently in this International Standard. In particular, destructors that throw exceptions will call `std::terminate()` (without calling `std::unexpected()`) if their exception specification is `noexcept` or `noexcept(true)`. For a throwing virtual destructor of a derived class, `std::terminate()` can be avoided only if the base class virtual destructor has an exception specification that is not `noexcept` and not `noexcept(true)`.

C.2.7 Clause 14: templates [diff.cpp03.temp]

14.1
Change: Remove `export`
Rationale: No implementation consensus.
Effect on original feature: A valid C++ 2003 declaration containing `export` is ill-formed in this International Standard.

14.3
Change: Remove whitespace requirement for nested closing template right angle brackets
Rationale: Considered a persistent but minor annoyance. Template aliases representing nonclass types would exacerbate whitespace issues.
Effect on original feature: Change to semantics of well-defined expression. A valid C++ 2003 expression containing a right angle bracket (“`>`”) followed immediately by another right angle bracket may now be treated as closing two templates. For example, the following code is valid in C++ 2003 because “`>>`” is a right-shift operator, but invalid in this International Standard because “`>>`” closes two templates.

```
template <class T> struct X { };
template <int N> struct Y { };
X< Y< 1 >> 2 > > x;
```

14.6.4.2
Change: Allow dependent calls of functions with internal linkage
Rationale: Overly constrained, simplify overload resolution rules.
Effect on original feature: A valid C++ 2003 program could get a different result than this International Standard.

C.2.8 Clause 17: library introduction [diff.cpp03.library]

17 – 30
Change: New reserved identifiers
Rationale: Required by new features.
Effect on original feature: Valid C++ 2003 code that uses any identifiers added to the C++ standard library by this International Standard may fail to compile or produce different results in This International Standard. A comprehensive list of identifiers used by the C++ standard library can be found in the Index of Library Names in this International Standard.

17.6.1.2
Change: New headers
Rationale: New functionality.

Effect on original feature: The following C++ headers are new: `<array>`, `<atomic>`, `<chrono>`, `<codecvt>`, `<condition_variable>`, `<forward_list>`, `<future>`, `<initializer_list>`, `<mutex>`, `<random>`, `<ratio>`, `<regex>`, `<scoped_allocator>`, `<system_error>`, `<thread>`, `<tuple>`, `<typeindex>`, `<type_traits>`, `<unordered_map>`, and `<unordered_set>`. In addition the following C compatibility headers are new: `<ccomplex>`, `<cfenv>`, `<cinttypes>`, `<cstdalign>`, `<cstdbool>`, `<cstdint>`, `<ctgmath>`, and `<cuchar>`. Valid C++ 2003 code that `#include`s headers with these names may be invalid in this International Standard.

17.6.3.2
Effect on original feature: Function `swap` moved to a different header
Rationale: Remove dependency on `<algorithm>` for `swap`.
Effect on original feature: Valid C++ 2003 code that has been compiled expecting swap to be in `<algorithm>` may have to instead include `<utility>`.

17.6.4.2.2
Change: New reserved namespace
Rationale: New functionality.
Effect on original feature: The global namespace `posix` is now reserved for standardization. Valid C++ 2003 code that uses a top-level namespace `posix` may be invalid in this International Standard.

17.6.5.3
Change: Additional restrictions on macro names
Rationale: Avoid hard to diagnose or non-portable constructs.
Effect on original feature: Names of attribute identifiers may not be used as macro names. Valid C++ 2003 code that defines `override`, `final`, `carries_dependency`, or `noreturn` as macros is invalid in this International Standard.

C.2.9 Clause 18: language support library [diff.cpp03.language.support]

18.6.1.1
Change: Linking `new` and `delete` operators
Rationale: The two throwing single-object signatures of `operator new` and `operator delete` are now specified to form the base functionality for the other operators. This clarifies that replacing just these two signatures changes others, even if they are not explicitly changed.
Effect on original feature: Valid C++ 2003 code that replaces global `new` or `delete` operators may execute differently in this International Standard. For example, the following program should write `"custom deallocation"` twice, once for the single-object delete and once for the array delete.

```
#include <cstdio>
#include <cstdlib>
#include <new>

void* operator new(std::size_t size) throw(std::bad_alloc) {
  return std::malloc(size);
}

void operator delete(void* ptr) throw() {
  std::puts("custom deallocation");
  std::free(ptr);
}

int main() {
  int* i = new int;
  delete i;                    // single-object delete
  int* a = new int[3];
  delete [] a;                 // array delete
```

```
  return 0;
}
```

18.6.1.1
Change: `operator new` may throw exceptions other than `std::bad_alloc`
Rationale: Consistent application of `noexcept`.
Effect on original feature: Valid C++ 2003 code that assumes that global `operator new` only throws `std::bad_alloc` may execute differently in this International Standard.

C.2.10 Clause 19: diagnostics library [diff.cpp03.diagnostics]

19.4
Change: Thread-local error numbers
Rationale: Support for new thread facilities.
Effect on original feature: Valid but implementation-specific C++ 2003 code that relies on `errno` being the same across threads may change behavior in this International Standard.

C.2.11 Clause 20: general utilities library [diff.cpp03.utilities]

20.7.4
Change: Minimal support for garbage-collected regions
Rationale: Required by new feature.
Effect on original feature: Valid C++ 2003 code, compiled without traceable pointer support, that interacts with newer C++ code using regions declared reachable may have different runtime behavior.

20.9.4, 20.9.5, 20.9.6, 20.9.7, 20.9.8, 20.9.9
Change: Standard function object types no longer derived from `std::unary_function` or `std::binary_function`
Rationale: Superseded by new feature; `unary_function` and `binary_function` are no longer defined.
Effect on original feature: Valid C++ 2003 code that depends on function object types being derived from `unary_function` or `binary_function` may fail to compile in this International Standard.

C.2.12 Clause 21: strings library [diff.cpp03.strings]

21.3
Change: `basic_string` requirements no longer allow reference-counted strings
Rationale: Invalidation is subtly different with reference-counted strings. This change regularizes behavior for this International Standard.
Effect on original feature: Valid C++ 2003 code may execute differently in this International Standard.

21.4.1
Change: Loosen `basic_string` invalidation rules
Rationale: Allow small-string optimization.
Effect on original feature: Valid C++ 2003 code may execute differently in this International Standard. Some `const` member functions, such as `data` and `c_str`, no longer invalidate iterators.

C.2.13 Clause 23: containers library [diff.cpp03.containers]

23.2
Change: Complexity of `size()` member functions now constant
Rationale: Lack of specification of complexity of `size()` resulted in divergent implementations with inconsistent performance characteristics.
Effect on original feature: Some container implementations that conform to C++ 2003 may not conform to the specified `size()` requirements in this International Standard. Adjusting containers such as `std::list` to the stricter requirements may require incompatible changes.

23.2
Change: Requirements change: relaxation
Rationale: Clarification.
Effect on original feature: Valid C++ 2003 code that attempts to meet the specified container requirements may now be over-specified. Code that attempted to be portable across containers may need to be adjusted as follows:

— not all containers provide `size()`; use `empty()` instead of `size() == 0`;

— not all containers are empty after construction (`array`);

— not all containers have constant complexity for `swap()` (`array`).

23.2
Change: Requirements change: default constructible
Rationale: Clarification of container requirements.
Effect on original feature: Valid C++ 2003 code that attempts to explicitly instantiate a container using a user-defined type with no default constructor may fail to compile.

23.2.3, 23.2.4
Change: Signature changes: from `void` return types
Rationale: Old signature threw away useful information that may be expensive to recalculate.
Effect on original feature: The following member functions have changed:

— `erase(iter)` for `set`, `multiset`, `map`, `multimap`

— `erase(begin, end)` for `set`, `multiset`, `map`, `multimap`

— `insert(pos, num, val)` for `vector`, `deque`, `list`, `forward_list`

— `insert(pos, beg, end)` for `vector`, `deque`, `list`, `forward_list`

Valid C++ 2003 code that relies on these functions returning `void` (e.g., code that creates a pointer to member function that points to one of these functions) will fail to compile with this International Standard.

23.2.3, 23.2.4
Change: Signature changes: from `iterator` to `const_iterator` parameters
Rationale: Overspecification. *Effects:* The signatures of the following member functions changed from taking an `iterator` to taking a `const_iterator`:

— `insert(iter, val)` for `vector`, `deque`, `list`, `set`, `multiset`, `map`, `multimap`

— `insert(pos, beg, end)` for `vector`, `deque`, `list`, `forward_list`

— `erase(begin, end)` for `set`, `multiset`, `map`, `multimap`

— all forms of `list::splice`

— all forms of `list::merge`

Valid C++ 2003 code that uses these functions may fail to compile with this International Standard.

23.2.3, 23.2.4
Change: Signature changes: `resize`
Rationale: Performance, compatibility with move semantics.
Effect on original feature: For `vector`, `deque`, and `list` the fill value passed to `resize` is now passed by reference instead of by value, and an additional overload of `resize` has been added. Valid C++ 2003 code that uses this function may fail to compile with this International Standard.

C.2.14 Clause 25: algorithms library [diff.cpp03.algorithms]

25.1

Change: Result state of inputs after application of some algorithms
Rationale: Required by new feature.
Effect on original feature: A valid C++ 2003 program may detect that an object with a valid but unspecified state has a different valid but unspecified state with this International Standard. For example, `std::remove` and `std::remove_if` may leave the tail of the input sequence with a different set of values than previously.

C.2.15 Clause 26: numerics library [diff.cpp03.numerics]

26.4

Change: Specified representation of complex numbers
Rationale: Compatibility with C99.
Effect on original feature: Valid C++ 2003 code that uses implementation-specific knowledge about the binary representation of the required template specializations of `std::complex` may not be compatible with this International Standard.

C.2.16 Clause 27: Input/output library [diff.cpp03.input.output]

27.7.2.1.3, 27.7.3.4, 27.5.5.4

Change: Specify use of explicit in existing boolean conversion operators
Rationale: Clarify intentions, avoid workarounds.
Effect on original feature: Valid C++ 2003 code that relies on implicit boolean conversions will fail to compile with this International Standard. Such conversions occur in the following conditions:

— passing a value to a function that takes an argument of type `bool`;

— using `operator==` to compare to `false` or `true`;

— returning a value from a function with a return type of `bool`;

— initializing members of type `bool` via aggregate initialization;

— initializing a `const bool&` which would bind to a temporary.

27.5.3.1.1

Change: Change base class of `std::ios_base::failure`
Rationale: More detailed error messages.
Effect on original feature: `std::ios_base::failure` is no longer derived directly from `std::exception`, but is now derived from `std::system_error`, which in turn is derived from `std::runtime_error`. Valid C++ 2003 code that assumes that `std::ios_base::failure` is derived directly from `std::exception` may execute differently in this International Standard.

27.5.3

Change: Flag types in `std::ios_base` are now bitmasks with values defined as constexpr static members
Rationale: Required for new features.
Effect on original feature: Valid C++ 2003 code that relies on `std::ios_base` flag types being represented as `std::bitset` or as an integer type may fail to compile with this International Standard. For example:

```
#include <iostream>

int main() {
  int flag = std::ios_base::hex;
  std::cout.setf(flag);          // error: setf does not take argument of type int
  return 0;
}
```

C.3 C++ and ISO C++ 2011 [diff.cpp11]

1 This subclause lists the differences between C++ and ISO C++ 2011 (ISO/IEC 14882:2011, *Programming Languages — C++*), by the chapters of this document.

C.3.1 Clause 2: lexical conventions [diff.cpp11.lex]

2.9

Change: *pp-number* can contain one or more single quotes.
Rationale: Necessary to enable single quotes as digit separators.
Effect on original feature: Valid C++ 2011 code may fail to compile or may change meaning in this International Standard. For example, the following code is valid both in C++ 2011 and in this International Standard, but the macro invocation produces different outcomes because the single quotes delimit a character literal in C++ 2011, whereas they are digit separators in this International Standard:

```
#define M(x, ...) __VA_ARGS__
int x[2] = { M(1'2,3'4) };
// int x[2] = {};       — C++ 2011
// int x[2] = { 3'4 }; — this International Standard
```

C.3.2 Clause 3: basic concepts [diff.cpp11.basic]

3.7.4.2

Change: New usual (non-placement) deallocator
Rationale: Required for sized deallocation.
Effect on original feature: Valid C++ 2011 code could declare a global placement allocation function and deallocation function as follows:

```
void operator new(std::size_t, std::size_t);
void operator delete(void*, std::size_t) noexcept;
```

In this International Standard, however, the declaration of `operator delete` might match a predefined usual (non-placement) `operator delete` (3.7.4). If so, the program is ill-formed, as it was for class member allocation functions and deallocation functions (5.3.4).

C.3.3 Clause 7: declarations [diff.cpp11.dcl.dcl]

7.1.5

Change: `constexpr` non-static member functions are not implicitly `const` member functions.
Rationale: Necessary to allow `constexpr` member functions to mutate the object.
Effect on original feature: Valid C++ 2011 code may fail to compile in this International Standard. For example, the following code is valid in C++ 2011 but invalid in this International Standard because it declares the same member function twice with different return types:

```
struct S {
  constexpr const int &f();
  int &f();
};
```

C.3.4 Clause 27: input/output library [diff.cpp11.input.output]

27.9.2

Change: `gets` is not defined.
Rationale: Use of `gets` is considered dangerous.
Effect on original feature: Valid C++ 2011 code that uses the `gets` function may fail to compile in this International Standard.

C.4 C++ and ISO C++ 2014 [diff.cpp14]

1 This subclause lists the differences between C++ and ISO C++ 2014 (ISO/IEC 14882:2014, *Programming Languages — C++*), by the chapters of this document.

C.4.1 Clause 2: lexical conventions [diff.cpp14.lex]

2.2

Change: Removal of trigraph support as a required feature.
Rationale: Prevents accidental uses of trigraphs in non-raw string literals and comments.
Effect on original feature: Valid C++ 2014 code that uses trigraphs may not be valid or may have different semantics in this International Standard. Implementations may choose to translate trigraphs as specified in C++ 2014 if they appear outside of a raw string literal, as part of the implementation-defined mapping from physical source file characters to the basic source character set.

C.4.2 Annex D: compatibility features [diff.cpp14.depr]

Change: The class templates `auto_ptr`, `unary_function`, and `binary_function`, the function templates `random_shuffle`, and the function templates (and their return types) `ptr_fun`, `mem_fun`, `mem_fun_ref`, `bind1st`, and `bind2nd` are not defined.
Rationale: Superseded by new features.
Effect on original feature: Valid C++ 2014 code that uses these class templates and function templates may fail to compile in this International Standard.

C.5 C standard library [diff.library]

1 This subclause summarizes the contents of the C++ standard library included from the Standard C library. It also summarizes the explicit changes in definitions, declarations, or behavior from the Standard C library noted in other subclauses (17.6.1.2, 18.2, 21.8).

2 The C++ standard library provides 57 standard macros from the C library, as shown in Table 149.

3 The header names (enclosed in < and >) indicate that the macro may be defined in more than one header. All such definitions are equivalent (3.2).

Table 149 — Standard macros

assert	HUGE_VAL	NULL <cstring>	SIGINT	va_end
BUFSIZ	LC_ALL	NULL <ctime>	SIGSEGV	va_start
CLOCKS_PER_SEC	LC_COLLATE	NULL <cwchar>	SIGTERM	WCHAR_MAX
EDOM	LC_CTYPE	offsetof	SIG_DFL	WCHAR_MIN
EILSEQ	LC_MONETARY	RAND_MAX	SIG_ERR	WEOF <cwchar>
EOF	LC_NUMERIC	SEEK_CUR	SIG_IGN	WEOF <cwctype>
ERANGE	LC_TIME	SEEK_END	stderr	_IOFBF
errno	L_tmpnam	SEEK_SET	stdin	_IOLBF
EXIT_FAILURE	MB_CUR_MAX	setjmp	stdout	_IONBF
EXIT_SUCCESS	NULL <clocale>	SIGABRT	TMP_MAX	
FILENAME_MAX	NULL <cstddef>	SIGFPE	va_arg	
FOPEN_MAX	NULL <cstdlib>	SIGILL	va_copy	

4 The C++ standard library provides 57 standard values from the C library, as shown in Table 150.

5 The C++ standard library provides 20 standard types from the C library, as shown in Table 151.

6 The C++ standard library provides 2 standard `struct`s from the C library, as shown in Table 152.

7 The C++ standard library provides 209 standard functions from the C library, as shown in Table 153.

Table 150 — Standard values

CHAR_BIT	FLT_DIG	INT_MIN	MB_LEN_MAX
CHAR_MAX	FLT_EPSILON	LDBL_DIG	SCHAR_MAX
CHAR_MIN	FLT_MANT_DIG	LDBL_EPSILON	SCHAR_MIN
DBL_DIG	FLT_MAX	LDBL_MANT_DIG	SHRT_MAX
DBL_EPSILON	FLT_MAX_10_EXP	LDBL_MAX	SHRT_MIN
DBL_MANT_DIG	FLT_MAX_EXP	LDBL_MAX_10_EXP	UCHAR_MAX
DBL_MAX	FLT_MIN	LDBL_MAX_EXP	UINT_MAX
DBL_MAX_10_EXP	FLT_MIN_10_EXP	LDBL_MIN	ULONG_MAX
DBL_MAX_EXP	FLT_MIN_EXP	LDBL_MIN_10_EXP	USHRT_MAX
DBL_MIN	FLT_RADIX	LDBL_MIN_EXP	
DBL_MIN_10_EXP	FLT_ROUNDS	LONG_MAX	
DBL_MIN_EXP	INT_MAX	LONG_MIN	

Table 151 — Standard types

clock_t	ldiv_t	size_t <cstdio>	va_list
div_t	mbstate_t	size_t <cstdlib>	wctrans_t
FILE	ptrdiff_t	size_t <cstring>	wctype_t
fpos_t	sig_atomic_t	size_t <ctime>	wint_t <cwchar>
jmp_buf	size_t <cstddef>	time_t	wint_t <cwctype>

Table 152 — Standard structs

lconv tm

C.5.1 Modifications to headers [diff.mods.to.headers]

1 For compatibility with the Standard C library, the C++ standard library provides the C headers enumerated in D.5, but their use is deprecated in C++.

C.5.2 Modifications to definitions [diff.mods.to.definitions]

C.5.2.1 Types `char16_t` and `char32_t` [diff.char16]

1 The types `char16_t` and `char32_t` are distinct types rather than typedefs to existing integral types.

C.5.2.2 Type `wchar_t` [diff.wchar.t]

1 `wchar_t` is a keyword in this International Standard (2.11). It does not appear as a type name defined in any of `<cstddef>`, `<cstdlib>`, or `<cwchar>` (21.8).

C.5.2.3 Header `<iso646.h>` [diff.header.iso646.h]

1 The tokens `and`, `and_eq`, `bitand`, `bitor`, `compl`, `not_eq`, `not`, `or`, `or_eq`, `xor`, and `xor_eq` are keywords in this International Standard (2.11). They do not appear as macro names defined in `<ciso646>`.

C.5.2.4 Macro `NULL` [diff.null]

1 The macro `NULL`, defined in any of `<clocale>`, `<cstddef>`, `<cstdio>`, `<cstdlib>`, `<cstring>`, `<ctime>`, or `<cwchar>`, is an implementation-defined C++ null pointer constant in this International Standard (18.2).

C.5.3 Modifications to declarations [diff.mods.to.declarations]

1 Header `<cstring>`: The following functions have different declarations:

Table 153 — Standard functions

abort	fmod	iswalnum	modf	strlen	wcscat
abs	fopen	iswalpha	perror	strncat	wcschr
acos	fprintf	iswcntrl	pow	strncmp	wcscmp
asctime	fputc	iswctype	printf	strncpy	wcscoll
asin	fputs	iswdigit	putc	strpbrk	wcscpy
atan	fputwc	iswgraph	putchar	strrchr	wcscspn
atan2	fputws	iswlower	puts	strspn	wcsftime
atexit	fread	iswprint	putwc	strstr	wcslen
atof	free	iswpunct	putwchar	strtod	wcsncat
atoi	freopen	iswspace	qsort	strtok	wcsncmp
atol	frexp	iswupper	raise	strtol	wcsncpy
bsearch	fscanf	iswxdigit	rand	strtoul	wcspbrk
btowc	fseek	isxdigit	realloc	strxfrm	wcsrchr
calloc	fsetpos	labs	remove	swprintf	wcsrtombs
ceil	ftell	ldexp	rename	swscanf	wcsspn
clearerr	fwide	ldiv	rewind	system	wcsstr
clock	fwprintf	localeconv	scanf	tan	wcstod
cos	fwrite	localtime	setbuf	tanh	wcstok
cosh	fwscanf	log	setlocale	time	wcstol
ctime	getc	log10	setvbuf	tmpfile	wcstombs
difftime	getchar	longjmp	signal	tmpnam	wcstoul
div	getenv	malloc	sin	tolower	wcsxfrm
exit	getwc	mblen	sinh	toupper	wctob
exp	getwchar	mbrlen	sprintf	towctrans	wctomb
fabs	gmtime	mbrtowc	sqrt	towlower	wctrans
fclose	isalnum	mbsinit	srand	towupper	wctype
feof	isalpha	mbsrtowcs	sscanf	ungetc	wmemchr
ferror	iscntrl	mbstowcs	strcat	ungetwc	wmemcmp
fflush	isdigit	mbtowc	strchr	vfprintf	wmemcpy
fgetc	isgraph	memchr	strcmp	vfwprintf	wmemmove
fgetpos	islower	memcmp	strcoll	vprintf	wmemset
fgets	isprint	memcpy	strcpy	vsprintf	wprintf
fgetwc	ispunct	memmove	strcspn	vswprintf	wscanf
fgetws	isspace	memset	strerror	vwprintf	
floor	isupper	mktime	strftime	wcrtomb	

(1.1) — `strchr`

(1.2) — `strpbrk`

(1.3) — `strrchr`

(1.4) — `strstr`

(1.5) — `memchr`

21.8 describes the changes.

C.5.4 Modifications to behavior [diff.mods.to.behavior]

1 Header `<cstdlib>`: The following functions have different behavior:

(1.1) — `atexit`

(1.2) — `exit`

(1.3) — `abort`

18.5 describes the changes.

2 Header `<csetjmp>`: The following functions have different behavior:

(2.1) — `longjmp`

18.10 describes the changes.

C.5.4.1 Macro `offsetof(type, member-designator)` [diff.offsetof]

1 The macro `offsetof`, defined in `<cstddef>`, accepts a restricted set of `type` arguments in this International Standard. 18.2 describes the change.

C.5.4.2 Memory allocation functions [diff.malloc]

1 The functions `calloc`, `malloc`, and `realloc` are restricted in this International Standard. 20.7.13 describes the changes.

Annex D (normative) Compatibility features [depr]

1 This Clause describes features of the C++ Standard that are specified for compatibility with existing implementations.

2 These are deprecated features, where *deprecated* is defined as: Normative for the current edition of the Standard, but having been identified as a candidate for removal from future revisions. An implementation may declare library names and entities described in this section with the `deprecated` attribute (7.6.5).

D.1 Increment operator with `bool` operand [depr.incr.bool]

1 The use of an operand of type `bool` with the `++` operator is deprecated (see 5.3.2 and 5.2.6).

D.2 `register` keyword [depr.register]

1 The use of the `register` keyword as a *storage-class-specifier* (7.1.1) is deprecated.

D.3 Implicit declaration of copy functions [depr.impldec]

1 The implicit definition of a copy constructor as defaulted is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor. The implicit definition of a copy assignment operator as defaulted is deprecated if the class has a user-declared copy constructor or a user-declared destructor (12.4, 12.8). In a future revision of this International Standard, these implicit definitions could become deleted (8.4).

D.4 Dynamic exception specifications [depr.except.spec]

1 The use of *dynamic-exception-specification*s is deprecated.

D.5 C standard library headers [depr.c.headers]

1 For compatibility with the C standard library and the C Unicode TR, the C++ standard library provides the 26 *C headers*, as shown in Table 154.

Table 154 — C headers

`<assert.h>`	`<inttypes.h>`	`<signal.h>`	`<stdio.h>`	`<wchar.h>`
`<complex.h>`	`<iso646.h>`	`<stdalign.h>`	`<stdlib.h>`	`<wctype.h>`
`<ctype.h>`	`<limits.h>`	`<stdarg.h>`	`<string.h>`	
`<errno.h>`	`<locale.h>`	`<stdbool.h>`	`<tgmath.h>`	
`<fenv.h>`	`<math.h>`	`<stddef.h>`	`<time.h>`	
`<float.h>`	`<setjmp.h>`	`<stdint.h>`	`<uchar.h>`	

2 Every C header, each of which has a name of the form `name.h`, behaves as if each name placed in the standard library namespace by the corresponding `cname` header is placed within the global namespace scope. It is unspecified whether these names are first declared or defined within namespace scope (3.3.6) of the namespace `std` and are then injected into the global namespace scope by explicit *using-declaration*s (7.3.3).

3 [*Example:* The header `<cstdlib>` assuredly provides its declarations and definitions within the namespace `std`. It may also provide these names within the global namespace. The header `<stdlib.h>` assuredly provides the same declarations and definitions within the global namespace, much as in the C Standard. It may also provide these names within the namespace `std`. — *end example*]

D.6 Old iostreams members [depr.ios.members]

1 The following member names are in addition to names specified in Clause 27:

```
namespace std {
  class ios_base {
  public:
    typedef T1 io_state;
    typedef T2 open_mode;
    typedef T3 seek_dir;
    typedef implementation-defined streamoff;
    typedef implementation-defined streampos;
    // remainder unchanged
  };
}
```

2 The type `io_state` is a synonym for an integer type (indicated here as `T1`) that permits certain member functions to overload others on parameters of type `iostate` and provide the same behavior.

3 The type `open_mode` is a synonym for an integer type (indicated here as `T2`) that permits certain member functions to overload others on parameters of type `openmode` and provide the same behavior.

4 The type `seek_dir` is a synonym for an integer type (indicated here as `T3`) that permits certain member functions to overload others on parameters of type `seekdir` and provide the same behavior.

5 The type `streamoff` is an implementation-defined type that satisfies the requirements of off_type in 27.2.2.

6 The type `streampos` is an implementation-defined type that satisfies the requirements of pos_type in 27.2.2.

7 An implementation may provide the following additional member function, which has the effect of calling `sbumpc()` (27.6.3.2.3):

```
namespace std {
  template<class charT, class traits = char_traits<charT> >
  class basic_streambuf {
  public:
    void stossc();
    // remainder unchanged
  };
}
```

8 An implementation may provide the following member functions that overload signatures specified in Clause 27:

```
namespace std {
  template<class charT, class traits> class basic_ios {
  public:
    void clear(io_state state);
    void setstate(io_state state);
    void exceptions(io_state);
    // remainder unchanged
  };

  class ios_base {
  public:
    // remainder unchanged
  };

  template<class charT, class traits = char_traits<charT> >
  class basic_streambuf {
  public:
```

```
    pos_type pubseekoff(off_type off, ios_base::seek_dir way,
              ios_base::open_mode which = ios_base::in | ios_base::out);
    pos_type pubseekpos(pos_type sp,
              ios_base::open_mode which);
    // remainder unchanged
  };

  template <class charT, class traits = char_traits<charT> >
  class basic_filebuf : public basic_streambuf<charT,traits> {
  public:
    basic_filebuf<charT,traits>* open
    (const char* s, ios_base::open_mode mode);
    // remainder unchanged
  };

  template <class charT, class traits = char_traits<charT> >
  class basic_ifstream : public basic_istream<charT,traits> {
  public:
    void open(const char* s, ios_base::open_mode mode);
    // remainder unchanged
  };

  template <class charT, class traits = char_traits<charT> >
  class basic_ofstream : public basic_ostream<charT,traits> {
  public:
    void open(const char* s, ios_base::open_mode mode);
    // remainder unchanged
  };
}
```

9 The effects of these functions is to call the corresponding member function specified in Clause 27.

D.7 `char*` streams [depr.str.strstreams]

1 The header `<strstream>` defines three types that associate stream buffers with character array objects and assist reading and writing such objects.

D.7.1 Class `strstreambuf` [depr.strstreambuf]

```
namespace std {
  class strstreambuf : public basic_streambuf<char> {
  public:
    explicit strstreambuf(streamsize alsize_arg = 0);
    strstreambuf(void* (*palloc_arg)(size_t), void (*pfree_arg)(void*));
    strstreambuf(char* gnext_arg, streamsize n, char* pbeg_arg = 0);
    strstreambuf(const char* gnext_arg, streamsize n);

    strstreambuf(signed char* gnext_arg, streamsize n,
                 signed char* pbeg_arg = 0);
    strstreambuf(const signed char* gnext_arg, streamsize n);
    strstreambuf(unsigned char* gnext_arg, streamsize n,
                 unsigned char* pbeg_arg = 0);
    strstreambuf(const unsigned char* gnext_arg, streamsize n);

    virtual ~strstreambuf();
```

```
    void  freeze(bool freezefl = true);
    char* str();
    int   pcount();

  protected:
    virtual int_type overflow (int_type c = EOF);
    virtual int_type pbackfail(int_type c = EOF);
    virtual int_type underflow();
    virtual pos_type seekoff(off_type off, ios_base::seekdir way,
                             ios_base::openmode which
                               = ios_base::in | ios_base::out);
    virtual pos_type seekpos(pos_type sp, ios_base::openmode which
                               = ios_base::in | ios_base::out);
    virtual streambuf* setbuf(char* s, streamsize n);

  private:
    typedef T1 strstate;              // exposition only
    static const strstate allocated;  // exposition only
    static const strstate constant;   // exposition only
    static const strstate dynamic;    // exposition only
    static const strstate frozen;     // exposition only
    strstate strmode;                 // exposition only
    streamsize alsize;                // exposition only
    void* (*palloc)(size_t);          // exposition only
    void (*pfree)(void*);             // exposition only
  };
}
```

1 The class `strstreambuf` associates the input sequence, and possibly the output sequence, with an object of some *character* array type, whose elements store arbitrary values. The array object has several attributes.

2 [*Note:* For the sake of exposition, these are represented as elements of a bitmask type (indicated here as `T1`) called `strstate`. The elements are:

(2.1) — `allocated`, set when a dynamic array object has been allocated, and hence should be freed by the destructor for the `strstreambuf` object;

(2.2) — `constant`, set when the array object has `const` elements, so the output sequence cannot be written;

(2.3) — `dynamic`, set when the array object is allocated (or reallocated) as necessary to hold a character sequence that can change in length;

(2.4) — `frozen`, set when the program has requested that the array object not be altered, reallocated, or freed.

— *end note*]

3 [*Note:* For the sake of exposition, the maintained data is presented here as:

(3.1) — `strstate strmode`, the attributes of the array object associated with the `strstreambuf` object;

(3.2) — `int alsize`, the suggested minimum size for a dynamic array object;

(3.3) — `void* (*palloc)(size_t)`, points to the function to call to allocate a dynamic array object;

(3.4) — `void (*pfree)(void*)`, points to the function to call to free a dynamic array object.

— *end note*]

4 Each object of class `strstreambuf` has a *seekable area*, delimited by the pointers `seeklow` and `seekhigh`. If `gnext` is a null pointer, the seekable area is undefined. Otherwise, `seeklow` equals `gbeg` and `seekhigh` is either `pend`, if `pend` is not a null pointer, or `gend`.

D.7.1.1 `strstreambuf` constructors [depr.strstreambuf.cons]

```
explicit strstreambuf(streamsize alsize_arg = 0);
```

1 *Effects:* Constructs an object of class `strstreambuf`, initializing the base class with `streambuf()`. The postconditions of this function are indicated in Table 155.

Table 155 — `strstreambuf(streamsize)` effects

Element	Value
`strmode`	`dynamic`
`alsize`	`alsize_arg`
`palloc`	a null pointer
`pfree`	a null pointer

```
strstreambuf(void* (*palloc_arg)(size_t), void (*pfree_arg)(void*));
```

2 *Effects:* Constructs an object of class `strstreambuf`, initializing the base class with `streambuf()`. The postconditions of this function are indicated in Table 156.

Table 156 — `strstreambuf(void* (*)(size_t), void (*)(void*))` effects

Element	Value
`strmode`	`dynamic`
`alsize`	an unspecified value
`palloc`	`palloc_arg`
`pfree`	`pfree_arg`

```
strstreambuf(char* gnext_arg, streamsize n, char* pbeg_arg = 0);
strstreambuf(signed char* gnext_arg, streamsize n,
             signed char* pbeg_arg = 0);
strstreambuf(unsigned char* gnext_arg, streamsize n,
             unsigned char* pbeg_arg = 0);
```

3 *Effects:* Constructs an object of class `strstreambuf`, initializing the base class with `streambuf()`. The postconditions of this function are indicated in Table 157.

Table 157 — `strstreambuf(charT*, streamsize, charT*)` effects

Element	Value
`strmode`	0
`alsize`	an unspecified value
`palloc`	a null pointer
`pfree`	a null pointer

4 `gnext_arg` shall point to the first element of an array object whose number of elements `N` is determined as follows:

(4.1) — If `n > 0`, `N` is `n`.

(4.2) — If `n == 0`, `N` is `std::strlen(gnext_arg)`.

(4.3) — If `n < 0`, `N` is `INT_MAX`.[339]

5 If `pbeg_arg` is a null pointer, the function executes:

```
setg(gnext_arg, gnext_arg, gnext_arg + N);
```

6 Otherwise, the function executes:

```
setg(gnext_arg, gnext_arg, pbeg_arg);
setp(pbeg_arg,  pbeg_arg + N);
```

```
strstreambuf(const char* gnext_arg, streamsize n);
strstreambuf(const signed char* gnext_arg, streamsize n);
strstreambuf(const unsigned char* gnext_arg, streamsize n);
```

7 *Effects:* Behaves the same as `strstreambuf((char*)gnext_arg,n)`, except that the constructor also sets `constant` in `strmode`.

```
virtual ~strstreambuf();
```

8 *Effects:* Destroys an object of class `strstreambuf`. The function frees the dynamically allocated array object only if `strmode & allocated != 0` and `strmode & frozen == 0`. (D.7.1.3 describes how a dynamically allocated array object is freed.)

D.7.1.2 Member functions [depr.strstreambuf.members]

```
void freeze(bool freezefl = true);
```

1 *Effects:* If `strmode` & `dynamic` is non-zero, alters the freeze status of the dynamic array object as follows:

(1.1) — If `freezefl` is `true`, the function sets `frozen` in `strmode`.

(1.2) — Otherwise, it clears `frozen` in `strmode`.

```
char* str();
```

2 *Effects:* Calls `freeze()`, then returns the beginning pointer for the input sequence, `gbeg`.

3 *Remarks:* The return value can be a null pointer.

```
int pcount() const;
```

4 *Effects:* If the next pointer for the output sequence, `pnext`, is a null pointer, returns zero. Otherwise, returns the current effective length of the array object as the next pointer minus the beginning pointer for the output sequence, `pnext - pbeg`.

D.7.1.3 `strstreambuf` overridden virtual functions [depr.strstreambuf.virtuals]

```
int_type overflow(int_type c = EOF);
```

1 *Effects:* Appends the character designated by `c` to the output sequence, if possible, in one of two ways:

(1.1) — If `c != EOF` and if either the output sequence has a write position available or the function makes a write position available (as described below), assigns `c` to `*pnext++`.

Returns `(unsigned char)c`.

339) The function signature `strlen(const char*)` is declared in `<cstring>`. (21.8). The macro `INT_MAX` is defined in `<climits>` (18.3).

(1.2) — If `c == EOF`, there is no character to append.
Returns a value other than `EOF`.

2 Returns `EOF` to indicate failure.

3 *Remarks:* The function can alter the number of write positions available as a result of any call.

4 To make a write position available, the function reallocates (or initially allocates) an array object with a sufficient number of elements `n` to hold the current array object (if any), plus at least one additional write position. How many additional write positions are made available is otherwise unspecified.[340] If `palloc` is not a null pointer, the function calls `(*palloc)(n)` to allocate the new dynamic array object. Otherwise, it evaluates the expression `new charT[n]`. In either case, if the allocation fails, the function returns `EOF`. Otherwise, it sets `allocated` in `strmode`.

5 To free a previously existing dynamic array object whose first element address is `p`: If `pfree` is not a null pointer, the function calls `(*pfree)(p)`. Otherwise, it evaluates the expression `delete[]p`.

6 If `strmode & dynamic == 0`, or if `strmode & frozen != 0`, the function cannot extend the array (reallocate it with greater length) to make a write position available.

```
int_type pbackfail(int_type c = EOF);
```

7 Puts back the character designated by `c` to the input sequence, if possible, in one of three ways:

(7.1) — If `c != EOF`, if the input sequence has a putback position available, and if `(char)c == gnext[-1]`, assigns `gnext - 1` to `gnext`.
Returns `c`.

(7.2) — If `c != EOF`, if the input sequence has a putback position available, and if `strmode` & `constant` is zero, assigns `c` to `*--gnext`.
Returns `c`.

(7.3) — If `c == EOF` and if the input sequence has a putback position available, assigns `gnext - 1` to `gnext`.
Returns a value other than `EOF`.

8 Returns `EOF` to indicate failure.

9 *Remarks:* If the function can succeed in more than one of these ways, it is unspecified which way is chosen. The function can alter the number of putback positions available as a result of any call.

```
int_type underflow();
```

10 *Effects:* Reads a character from the *input sequence*, if possible, without moving the stream position past it, as follows:

(10.1) — If the input sequence has a read position available, the function signals success by returning `(unsigned char)*gnext`.

(10.2) — Otherwise, if the current write next pointer `pnext` is not a null pointer and is greater than the current read end pointer `gend`, makes a *read position* available by assigning to `gend` a value greater than `gnext` and no greater than `pnext`.
Returns `(unsigned char*)gnext`.

11 Returns `EOF` to indicate failure.

12 *Remarks:* The function can alter the number of read positions available as a result of any call.

```
pos_type seekoff(off_type off, seekdir way, openmode which = in | out);
```

Table 158 — `seekoff` positioning

Conditions	Result
`(which & ios::in) != 0`	positions the input sequence
`(which & ios::out) != 0`	positions the output sequence
`(which & (ios::in | ios::out)) == (ios::in | ios::out))` and `way ==` either `ios::beg` or `ios::end`	positions both the input and the output sequences
Otherwise	the positioning operation fails.

13 *Effects:* Alters the stream position within one of the controlled sequences, if possible, as indicated in Table 158.

14 For a sequence to be positioned, if its next pointer is a null pointer, the positioning operation fails. Otherwise, the function determines `newoff` as indicated in Table 159.

Table 159 — `newoff` values

Condition	`newoff` Value
`way == ios::beg`	0
`way == ios::cur`	the next pointer minus the beginning pointer (`xnext - xbeg`).
`way == ios::end`	`seekhigh` minus the beginning pointer (`seekhigh - xbeg`).

15 If `(newoff + off) < (seeklow - xbeg)` or `(seekhigh - xbeg) < (newoff + off)`, the positioning operation fails. Otherwise, the function assigns `xbeg + newoff + off` to the next pointer `xnext`.

16 *Returns:* `pos_type(newoff)`, constructed from the resultant offset `newoff` (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
pos_type seekpos(pos_type sp, ios_base::openmode which
                 = ios_base::in | ios_base::out);
```

17 *Effects:* Alters the stream position within one of the controlled sequences, if possible, to correspond to the stream position stored in `sp` (as described below).

(17.1) — If `(which & ios::in) != 0`, positions the input sequence.

(17.2) — If `(which & ios::out) != 0`, positions the output sequence.

(17.3) — If the function positions neither sequence, the positioning operation fails.

18 For a sequence to be positioned, if its next pointer is a null pointer, the positioning operation fails. Otherwise, the function determines `newoff` from `sp.offset()`:

(18.1) — If `newoff` is an invalid stream position, has a negative value, or has a value greater than (`seekhigh - seeklow`), the positioning operation fails

340) An implementation should consider `alsize` in making this decision.

(18.2) — Otherwise, the function adds `newoff` to the beginning pointer `xbeg` and stores the result in the next pointer `xnext`.

19 *Returns:* `pos_type(newoff)`, constructed from the resultant offset `newoff` (of type `off_type`), that stores the resultant stream position, if possible. If the positioning operation fails, or if the constructed object cannot represent the resultant stream position, the return value is `pos_type(off_type(-1))`.

```
streambuf<char>* setbuf(char* s, streamsize n);
```

20 *Effects:* Implementation defined, except that `setbuf(0, 0)` has no effect.

D.7.2 Class `istrstream` [depr.istrstream]

```
namespace std {
  class istrstream : public basic_istream<char> {
  public:
    explicit istrstream(const char* s);
    explicit istrstream(char* s);
    istrstream(const char* s, streamsize n);
    istrstream(char* s, streamsize n);
    virtual ~istrstream();

    strstreambuf* rdbuf() const;
    char* str();
  private:
    strstreambuf sb;  // exposition only
  };
}
```

1 The class `istrstream` supports the reading of objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `strstreambuf` object.

D.7.2.1 `istrstream` constructors [depr.istrstream.cons]

```
explicit istrstream(const char* s);
explicit istrstream(char* s);
```

1 *Effects:* Constructs an object of class `istrstream`, initializing the base class with `istream(&sb)` and initializing `sb` with `strstreambuf(s,0))`. `s` shall designate the first element of an NTBS.

```
istrstream(const char* s, streamsize n);
```

2 *Effects:* Constructs an object of class `istrstream`, initializing the base class with `istream(&sb)` and initializing `sb` with `strstreambuf(s,n))`. `s` shall designate the first element of an array whose length is `n` elements, and `n` shall be greater than zero.

D.7.2.2 Member functions [depr.istrstream.members]

```
strstreambuf* rdbuf() const;
```

1 *Returns:* `const_cast<strstreambuf*>(&sb)`.

```
char* str();
```

2 *Returns:* `rdbuf()->str()`.

D.7.3 Class `ostrstream` [depr.ostrstream]

```
namespace std {
  class ostrstream : public basic_ostream<char> {
  public:
    ostrstream();
    ostrstream(char* s, int n, ios_base::openmode mode = ios_base::out);
    virtual ~ostrstream();

    strstreambuf* rdbuf() const;
    void freeze(bool freezefl = true);
    char* str();
    int pcount() const;
  private:
    strstreambuf sb;  // exposition only
  };
}
```

1 The class `ostrstream` supports the writing of objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `strstreambuf` object.

D.7.3.1 `ostrstream` constructors [depr.ostrstream.cons]

```
ostrstream();
```

1 *Effects:* Constructs an object of class `ostrstream`, initializing the base class with `ostream(&sb)` and initializing `sb` with `strstreambuf()`.

```
ostrstream(char* s, int n, ios_base::openmode mode = ios_base::out);
```

2 *Effects:* Constructs an object of class `ostrstream`, initializing the base class with `ostream(&sb)`, and initializing `sb` with one of two constructors:

(2.1) — If `(mode & app) == 0`, then `s` shall designate the first element of an array of `n` elements. The constructor is `strstreambuf(s, n, s)`.

(2.2) — If `(mode & app) != 0`, then `s` shall designate the first element of an array of `n` elements that contains an NTBS whose first element is designated by `s`. The constructor is `strstreambuf(s, n, s + std::strlen(s))`.[341]

D.7.3.2 Member functions [depr.ostrstream.members]

```
strstreambuf* rdbuf() const;
```

1 *Returns:* `(strstreambuf*)&sb` .

```
void freeze(bool freezefl = true);
```

2 *Effects:* Calls `rdbuf()->freeze(freezefl)`.

```
char* str();
```

3 *Returns:* `rdbuf()->str()`.

```
int pcount() const;
```

4 *Returns:* `rdbuf()->pcount()`.

341) The function signature `strlen(const char*)` is declared in `<cstring>` (21.8).

D.7.4 Class `strstream` [depr.strstream]

```
namespace std {
  class strstream
    : public basic_iostream<char> {
  public:
    // Types
    typedef char                                  char_type;
    typedef typename char_traits<char>::int_type int_type;
    typedef typename char_traits<char>::pos_type pos_type;
    typedef typename char_traits<char>::off_type off_type;

    // constructors/destructor
    strstream();
    strstream(char* s, int n,
              ios_base::openmode mode = ios_base::in|ios_base::out);
    virtual ~strstream();

    // Members:
    strstreambuf* rdbuf() const;
    void freeze(bool freezefl = true);
    int pcount() const;
    char* str();

  private:
  strstreambuf sb;  // exposition only
  };
}
```

1 The class `strstream` supports reading and writing from objects of class `strstreambuf`. It supplies a `strstreambuf` object to control the associated array object. For the sake of exposition, the maintained data is presented here as:

(1.1) — `sb`, the `strstreambuf` object.

D.7.4.1 `strstream` constructors [depr.strstream.cons]

```
strstream();
```

1 *Effects:* Constructs an object of class `strstream`, initializing the base class with `iostream(&sb)`.

```
strstream(char* s, int n,
          ios_base::openmode mode = ios_base::in|ios_base::out);
```

2 *Effects:* Constructs an object of class `strstream`, initializing the base class with `iostream(&sb)` and initializing `sb` with one of the two constructors:

(2.1) — If `(mode & app) == 0`, then `s` shall designate the first element of an array of `n` elements. The constructor is `strstreambuf(s,n,s)`.

(2.2) — If `(mode & app) != 0`, then `s` shall designate the first element of an array of `n` elements that contains an NTBS whose first element is designated by `s`. The constructor is `strstreambuf(s,n,s + std::strlen(s))`.

D.7.4.2 `strstream` destructor [depr.strstream.dest]

```
virtual ~strstream();
```

1 *Effects:* Destroys an object of class `strstream`.

```
strstreambuf* rdbuf() const;
```

2 *Returns:* `&sb`.

D.7.4.3 `strstream` operations [depr.strstream.oper]

```
void freeze(bool freezefl = true);
```

1 *Effects:* Calls `rdbuf()->freeze(freezefl)`.

```
char* str();
```

2 *Returns:* `rdbuf()->str()`.

```
int pcount() const;
```

3 *Returns:* `rdbuf()->pcount()`.

D.8 Violating *exception-specification*s [exception.unexpected]

D.8.1 Type `unexpected_handler` [unexpected.handler]

```
typedef void (*unexpected_handler)();
```

1 The type of a *handler function* to be called by `unexpected()` when a function attempts to throw an exception not listed in its *dynamic-exception-specification*.

2 *Required behavior:* An `unexpected_handler` shall not return. See also 15.5.2.

3 *Default behavior:* The implementation's default `unexpected_handler` calls `std::terminate()`.

D.8.2 `set_unexpected` [set.unexpected]

```
unexpected_handler set_unexpected(unexpected_handler f) noexcept;
```

1 *Effects:* Establishes the function designated by `f` as the current `unexpected_handler`.

2 *Remark:* It is unspecified whether a null pointer value designates the default `unexpected_handler`.

3 *Returns:* The previous `unexpected_handler`.

D.8.3 `get_unexpected` [get.unexpected]

```
unexpected_handler get_unexpected() noexcept;
```

1 *Returns:* The current `unexpected_handler`. [*Note:* This may be a null pointer value. — *end note*]

D.8.4 `unexpected` [unexpected]

```
[[noreturn]] void unexpected();
```

1 *Remarks:* Called by the implementation when a function exits via an exception not allowed by its *exception-specification* (15.5.2), in effect after evaluating the throw-expression (D.8.1). May also be called directly by the program.

2 *Effects:* Calls the current `unexpected_handler` function. [*Note:* A default `unexpected_handler` is always considered a callable handler in this context. — *end note*]

D.9 `uncaught_exception` [depr.uncaught]

```
bool uncaught_exception() noexcept;
```

1 *Returns:* `uncaught_exceptions() > 0`.

Annex E (normative)
Universal character names for identifier characters [charname]

E.1 Ranges of characters allowed [charname.allowed]

00A8, 00AA, 00AD, 00AF, 00B2-00B5, 00B7-00BA, 00BC-00BE, 00C0-00D6, 00D8-00F6, 00F8-00FF

0100-167F, 1681-180D, 180F-1FFF

200B-200D, 202A-202E, 203F-2040, 2054, 2060-206F

2070-218F, 2460-24FF, 2776-2793, 2C00-2DFF, 2E80-2FFF

3004-3007, 3021-302F, 3031-303F

3040-D7FF

F900-FD3D, FD40-FDCF, FDF0-FE44, FE47-FFFD

10000-1FFFD, 20000-2FFFD, 30000-3FFFD, 40000-4FFFD, 50000-5FFFD,
60000-6FFFD, 70000-7FFFD, 80000-8FFFD, 90000-9FFFD, A0000-AFFFD,
B0000-BFFFD, C0000-CFFFD, D0000-DFFFD, E0000-EFFFD

E.2 Ranges of characters disallowed initially [charname.disallowed]

0300-036F, 1DC0-1DFF, 20D0-20FF, FE20-FE2F

Annex F (informative)
Cross references [xref]

This annex lists each section label and the corresponding section number, in alphabetical order by label. All of the section labels are the same as in the 2003 standard, except:

— labels that begin with `lib.` in the 2003 standard have had the `lib.` removed so that they do not all appear in the same part of this list. For example, in the 2003 standard, the non-modifying sequence algorithms were found in a section with the label `[lib.alg.nonmodifying]`. The label for that section is now `[alg.nonmodifying]`.

— the label for Annex B has been changed from `[limits]` to `[implimits]`. The label `[limits]` refers to section 18.3.2.

A

accumulate 26.7.2
adjacent.difference 26.7.5
adjustfield.manip 27.5.6.2
alg.adjacent.find 25.2.8
alg.all_of 25.2.1
alg.any_of 25.2.2
alg.binary.search 25.4.3
alg.c.library 25.5
alg.copy 25.3.1
alg.count 25.2.9
alg.equal 25.2.11
alg.fill 25.3.6
alg.find 25.2.5
alg.find.end 25.2.6
alg.find.first.of 25.2.7
alg.foreach 25.2.4
alg.generate 25.3.7
alg.heap.operations 25.4.6
alg.is_permutation 25.2.12
alg.lex.comparison 25.4.8
alg.merge 25.4.4
alg.min.max 25.4.7
alg.modifying.operations 25.3
alg.move 25.3.2
alg.none_of 25.2.3
alg.nonmodifying 25.2
alg.nth.element 25.4.2
alg.partitions 25.3.13
alg.permutation.generators 25.4.9
alg.random.shuffle 25.3.12
alg.remove 25.3.8
alg.replace 25.3.5
alg.reverse 25.3.10
alg.rotate 25.3.11
alg.search 25.2.13
alg.set.operations 25.4.5
alg.sort 25.4.1
alg.sorting 25.4
alg.swap 25.3.3
alg.transform 25.3.4
alg.unique 25.3.9
algorithm.stable 17.6.5.7
algorithms 25
algorithms.general 25.1
alloc.errors 18.6.2
allocator.adaptor 20.13
allocator.adaptor.cnstr 20.13.3
allocator.adaptor.members 20.13.4
allocator.adaptor.syn 20.13.1
allocator.adaptor.types 20.13.2
allocator.globals 20.7.9.2
allocator.members 20.7.9.1
allocator.requirements 17.6.3.5
allocator.requirements.completeness 17.6.3.5.1
allocator.tag 20.7.6
allocator.traits 20.7.8
allocator.traits.members 20.7.8.2
allocator.traits.types 20.7.8.1
allocator.uses 20.7.7
allocator.uses.construction 20.7.7.2
allocator.uses.trait 20.7.7.1
alt.headers 17.6.4.4
arithmetic.operations 20.9.5
array 23.3.2

array.cons 23.3.2.2
array.data 23.3.2.5
array.fill 23.3.2.6
array.overview 23.3.2.1
array.size 23.3.2.4
array.special 23.3.2.3
array.swap 23.3.2.7
array.tuple 23.3.2.9
array.zero 23.3.2.8
assertions 19.3
associative 23.4
associative.general 23.4.1
associative.map.syn 23.4.2
associative.reqmts 23.2.4
associative.reqmts.except 23.2.4.1
associative.set.syn 23.4.3
atomics 29
atomics.fences 29.8
atomics.flag 29.7
atomics.general 29.1
atomics.lockfree 29.4
atomics.order 29.3
atomics.syn 29.2
atomics.types.generic 29.5
atomics.types.operations 29.6
atomics.types.operations.arith 29.6.3
atomics.types.operations.general 29.6.1
atomics.types.operations.pointer 29.6.4
atomics.types.operations.req 29.6.5
atomics.types.operations.templ 29.6.2

B

back.insert.iter.cons 24.5.2.2.1
back.insert.iter.op* 24.5.2.2.3
back.insert.iter.op++ 24.5.2.2.4
back.insert.iter.op= 24.5.2.2.2
back.insert.iter.ops 24.5.2.2
back.insert.iterator 24.5.2.1
back.inserter 24.5.2.2.5
bad.alloc 18.6.2.1
bad.cast 18.7.2
bad.exception 18.8.2
bad.typeid 18.7.3
basefield.manip 27.5.6.3
basic 3
basic.align 3.11
basic.compound 3.9.2
basic.def 3.1
basic.def.odr 3.2
basic.fundamental 3.9.1
basic.funscope 3.3.5
basic.ios.cons 27.5.5.2
basic.ios.members 27.5.5.3
basic.life 3.8
basic.link 3.5
basic.lookup 3.4
basic.lookup.argdep 3.4.2
basic.lookup.classref 3.4.5
basic.lookup.elab 3.4.4
basic.lookup.qual 3.4.3
basic.lookup.udir 3.4.6
basic.lookup.unqual 3.4.1
basic.lval 3.10
basic.namespace 7.3
basic.scope 3.3
basic.scope.block 3.3.3
basic.scope.class 3.3.7
basic.scope.declarative 3.3.1
basic.scope.enum 3.3.8
basic.scope.hiding 3.3.10
basic.scope.namespace 3.3.6
basic.scope.pdecl 3.3.2
basic.scope.proto 3.3.4
basic.scope.temp 3.3.9
basic.start 3.6
basic.start.init 3.6.2
basic.start.main 3.6.1
basic.start.term 3.6.3
basic.stc 3.7
basic.stc.auto 3.7.3
basic.stc.dynamic 3.7.4
basic.stc.dynamic.allocation 3.7.4.1
basic.stc.dynamic.deallocation 3.7.4.2
basic.stc.dynamic.safety 3.7.4.3
basic.stc.inherit 3.7.5
basic.stc.static 3.7.1
basic.stc.thread 3.7.2
basic.string 21.4
basic.string.hash 21.6
basic.string.literals 21.7
basic.type.qualifier 3.9.3
basic.types 3.9
bidirectional.iterators 24.2.6
binary.search 25.4.3.4
bitmask.types 17.5.2.1.3
bitset.cons 20.6.1
bitset.hash 20.6.3
bitset.members 20.6.2
bitset.operators 20.6.4
bitwise.operations 20.9.8
byte.strings 17.5.2.1.4.1

C

c.files 27.9.2
c.limits 18.3.3
c.locales 22.6
c.malloc 20.7.13
c.math 26.8
c.strings 21.8
category.collate 22.4.4
category.ctype 22.4.1
category.messages 22.4.7
category.monetary 22.4.6
category.numeric 22.4.2
category.time 22.4.5
ccmplx 26.4.11
cfenv 26.3
cfenv.syn 26.3.1
char.traits 21.2
char.traits.require 21.2.1
char.traits.specializations 21.2.3
char.traits.specializations.char 21.2.3.1
char.traits.specializations.char16_t 21.2.3.2
char.traits.specializations.char32_t 21.2.3.3
char.traits.specializations.wchar.t 21.2.3.4
char.traits.typedefs 21.2.2
character.seq 17.5.2.1.4
charname E
charname.allowed E.1
charname.disallowed E.2
class 9
class.abstract 10.4
class.access 11
class.access.base 11.2
class.access.nest 11.7
class.access.spec 11.1
class.access.virt 11.5
class.base.init 12.6.2
class.bit 9.6
class.cdtor 12.7
class.conv 12.3
class.conv.ctor 12.3.1
class.conv.fct 12.3.2
class.copy 12.8
class.ctor 12.1
class.derived 10
class.dtor 12.4
class.expl.init 12.6.1
class.free 12.5
class.friend 11.3
class.gslice 26.6.6
class.gslice.overview 26.6.6.1
class.inhctor 12.9
class.init 12.6
class.local 9.8
class.mem 9.2
class.member.lookup 10.2
class.mfct 9.3
class.mfct.non-static 9.3.1
class.mi 10.1
class.name 9.1
class.nest 9.7
class.nested.type 9.9
class.paths 11.6
class.protected 11.4
class.qual 3.4.3.1
class.slice 26.6.4
class.slice.overview 26.6.4.1
class.static 9.4
class.static.data 9.4.2
class.static.mfct 9.4.1
class.temporary 12.2
class.this 9.3.2
class.union 9.5
class.virtual 10.3
classification 22.3.3.1
cmplx.over 26.4.9
comparisons 20.9.6
complex 26.4.2
complex.literals 26.4.10
complex.member.ops 26.4.5
complex.members 26.4.4
complex.numbers 26.4
complex.ops 26.4.6
complex.special 26.4.3
complex.syn 26.4.1
complex.transcendentals 26.4.8
complex.value.ops 26.4.7
compliance 17.6.1.3
conforming 17.6.5
conforming.overview 17.6.5.1
cons.slice 26.6.4.2
constexpr.functions 17.6.5.6
constraints 17.6.4
constraints.overview 17.6.4.1
container.adaptors 23.6
container.adaptors.general 23.6.1
container.requirements 23.2
container.requirements.dataraces 23.2.2
container.requirements.general 23.2.1
containers 23
containers.general 23.1

contents 17.6.1.1
conv 4
conv.array 4.2
conv.bool 4.12
conv.double 4.8
conv.fpint 4.9
conv.fpprom 4.6
conv.func 4.3
conv.integral 4.7
conv.lval 4.1
conv.mem 4.11
conv.prom 4.5
conv.ptr 4.10
conv.qual 4.4
conv.rank 4.13
conventions 17.5.2
conversions 22.3.3.2
conversions.buffer 22.3.3.2.3
conversions.character 22.3.3.2.1
conversions.string 22.3.3.2.2
cpp 16
cpp.concat 16.3.3
cpp.cond 16.1
cpp.error 16.5
cpp.include 16.2
cpp.line 16.4
cpp.null 16.7
cpp.pragma 16.6
cpp.pragma.op 16.9
cpp.predefined 16.8
cpp.replace 16.3
cpp.rescan 16.3.4
cpp.scope 16.3.5
cpp.stringize 16.3.2
cpp.subst 16.3.1
cstdint 18.4
cstdint.syn 18.4.1

D

date.time 20.12.8
dcl.align 7.6.2
dcl.ambig.res 8.2
dcl.array 8.3.4
dcl.asm 7.4
dcl.attr 7.6
dcl.attr.depend 7.6.4
dcl.attr.deprecated 7.6.5
dcl.attr.grammar 7.6.1
dcl.attr.noreturn 7.6.3
dcl.constexpr 7.1.5
dcl.dcl 7
dcl.decl 8
dcl.enum 7.2
dcl.fct 8.3.5
dcl.fct.def 8.4
dcl.fct.def.default 8.4.2
dcl.fct.def.delete 8.4.3
dcl.fct.def.general 8.4.1
dcl.fct.default 8.3.6
dcl.fct.spec 7.1.2
dcl.friend 7.1.4
dcl.init 8.5
dcl.init.aggr 8.5.1
dcl.init.list 8.5.4
dcl.init.ref 8.5.3
dcl.init.string 8.5.2
dcl.link 7.5
dcl.meaning 8.3
dcl.mptr 8.3.3
dcl.name 8.1
dcl.ptr 8.3.1
dcl.ref 8.3.2
dcl.spec 7.1
dcl.spec.auto 7.1.6.4
dcl.stc 7.1.1
dcl.type 7.1.6
dcl.type.cv 7.1.6.1
dcl.type.elab 7.1.6.3
dcl.type.simple 7.1.6.2
dcl.typedef 7.1.3
declval 20.2.5
default.allocator 20.7.9
definitions 17.3
defns.access 1.3.1
defns.additional 17.4
defns.arbitrary.stream 17.3.1
defns.argument 1.3.2
defns.argument.macro 1.3.3
defns.argument.templ 1.3.5
defns.argument.throw 1.3.4
defns.block 17.3.2
defns.blocked 17.3.3
defns.character 17.3.4
defns.character.container 17.3.5
defns.comparison 17.3.6
defns.component 17.3.7
defns.cond.supp 1.3.6
defns.deadlock 17.3.8
defns.default.behavior.func 17.3.10
defns.default.behavior.impl 17.3.9
defns.diagnostic 1.3.7

defns.dynamic.type 1.3.8
defns.dynamic.type.prvalue 1.3.9
defns.handler 17.3.11
defns.ill.formed 1.3.10
defns.impl.defined 1.3.11
defns.impl.limits 1.3.12
defns.iostream.templates 17.3.12
defns.locale.specific 1.3.13
defns.modifier 17.3.13
defns.move.assign 17.3.15
defns.move.constr 17.3.14
defns.multibyte 1.3.14
defns.ntcts 17.3.17
defns.obj.state 17.3.16
defns.observer 17.3.18
defns.parameter 1.3.15
defns.parameter.macro 1.3.16
defns.parameter.templ 1.3.17
defns.referenceable 17.3.19
defns.regex.collating.element 28.2.1
defns.regex.finite.state.machine 28.2.2
defns.regex.format.specifier 28.2.3
defns.regex.matched 28.2.4
defns.regex.primary.equivalence.class 28.2.5
defns.regex.regular.expression 28.2.6
defns.regex.subexpression 28.2.7
defns.replacement 17.3.20
defns.repositional.stream 17.3.21
defns.required.behavior 17.3.22
defns.reserved.function 17.3.23
defns.signature 1.3.18
defns.signature.member 1.3.21
defns.signature.member.spec 1.3.23
defns.signature.member.templ 1.3.22
defns.signature.spec 1.3.20
defns.signature.templ 1.3.19
defns.stable 17.3.24
defns.static.type 1.3.24
defns.traits 17.3.25
defns.unblock 17.3.26
defns.undefined 1.3.25
defns.unspecified 1.3.26
defns.valid 17.3.27
defns.well.formed 1.3.27
denorm.style 18.3.2.6
depr D
depr.c.headers D.5
depr.except.spec D.4
depr.impldec D.3
depr.incr.bool D.1
depr.ios.members D.6
depr.istrstream D.7.2
depr.istrstream.cons D.7.2.1
depr.istrstream.members D.7.2.2
depr.ostrstream D.7.3
depr.ostrstream.cons D.7.3.1
depr.ostrstream.members D.7.3.2
depr.register D.2
depr.str.strstreams D.7
depr.strstream D.7.4
depr.strstream.cons D.7.4.1
depr.strstream.dest D.7.4.2
depr.strstream.oper D.7.4.3
depr.strstreambuf D.7.1
depr.strstreambuf.cons D.7.1.1
depr.strstreambuf.members D.7.1.2
depr.strstreambuf.virtuals D.7.1.3
depr.uncaught D.9
deque 23.3.3
deque.capacity 23.3.3.3
deque.cons 23.3.3.2
deque.modifiers 23.3.3.4
deque.overview 23.3.3.1
deque.special 23.3.3.5
derivation 17.6.5.11
derived.classes 17.6.4.5
description 17.5
diagnostics 19
diagnostics.general 19.1
diff C
diff.basic C.1.2
diff.char16 C.5.2.1
diff.class C.1.8
diff.conv C.1.3
diff.cpp C.1.10
diff.cpp03 C.2
diff.cpp03.algorithms C.2.14
diff.cpp03.containers C.2.13
diff.cpp03.conv C.2.2
diff.cpp03.dcl.dcl C.2.4
diff.cpp03.dcl.decl C.2.5
diff.cpp03.diagnostics C.2.10
diff.cpp03.expr C.2.3
diff.cpp03.input.output C.2.16
diff.cpp03.language.support C.2.9
diff.cpp03.lex C.2.1
diff.cpp03.library C.2.8
diff.cpp03.numerics C.2.15
diff.cpp03.special C.2.6
diff.cpp03.strings C.2.12
diff.cpp03.temp C.2.7
diff.cpp03.utilities C.2.11

diff.cpp11 C.3
diff.cpp11.basic C.3.2
diff.cpp11.dcl.dcl C.3.3
diff.cpp11.input.output C.3.4
diff.cpp11.lex C.3.1
diff.cpp14 C.4
diff.cpp14.depr C.4.2
diff.cpp14.lex C.4.1
diff.dcl C.1.6
diff.decl C.1.7
diff.expr C.1.4
diff.header.iso646.h C.5.2.3
diff.iso C.1
diff.lex C.1.1
diff.library C.5
diff.malloc C.5.4.2
diff.mods.to.behavior C.5.4
diff.mods.to.declarations C.5.3
diff.mods.to.definitions C.5.2
diff.mods.to.headers C.5.1
diff.null C.5.2.4
diff.offsetof C.5.4.1
diff.special C.1.9
diff.stat C.1.5
diff.wchar.t C.5.2.2
domain.error 19.2.2

E

enumerated.types 17.5.2.1.2
equal.range 25.4.3.3
errno 19.4
error.reporting 27.5.6.5
except 15
except.ctor 15.2
except.handle 15.3
except.nested 18.8.6
except.spec 15.4
except.special 15.5
except.terminate 15.5.1
except.throw 15.1
except.uncaught 15.5.3
except.unexpected 15.5.2
exception 18.8.1
exception.terminate 18.8.3
exception.unexpected D.8
expr 5
expr.add 5.7
expr.alignof 5.3.6
expr.ass 5.18
expr.bit.and 5.11
expr.call 5.2.2
expr.cast 5.4
expr.comma 5.19
expr.cond 5.16
expr.const 5.20
expr.const.cast 5.2.11
expr.delete 5.3.5
expr.dynamic.cast 5.2.7
expr.eq 5.10
expr.log.and 5.14
expr.log.or 5.15
expr.mptr.oper 5.5
expr.mul 5.6
expr.new 5.3.4
expr.or 5.13
expr.post 5.2
expr.post.incr 5.2.6
expr.pre.incr 5.3.2
expr.prim 5.1
expr.prim.fold 5.1.3
expr.prim.general 5.1.1
expr.prim.lambda 5.1.2
expr.pseudo 5.2.4
expr.ref 5.2.5
expr.reinterpret.cast 5.2.10
expr.rel 5.9
expr.shift 5.8
expr.sizeof 5.3.3
expr.static.cast 5.2.9
expr.sub 5.2.1
expr.throw 5.17
expr.type.conv 5.2.3
expr.typeid 5.2.8
expr.unary 5.3
expr.unary.noexcept 5.3.7
expr.unary.op 5.3.1
expr.xor 5.12
ext.manip 27.7.5
extern.names 17.6.4.3.2
extern.types 17.6.4.3.3

F

facet.ctype.char.dtor 22.4.1.3.1
facet.ctype.char.members 22.4.1.3.2
facet.ctype.char.statics 22.4.1.3.3
facet.ctype.char.virtuals 22.4.1.3.4
facet.ctype.special 22.4.1.3
facet.num.get.members 22.4.2.1.1
facet.num.get.virtuals 22.4.2.1.2
facet.num.put.members 22.4.2.2.1

facet.num.put.virtuals 22.4.2.2.2
facet.numpunct 22.4.3
facet.numpunct.members 22.4.3.1.1
facet.numpunct.virtuals 22.4.3.1.2
facets.examples 22.4.8
file.streams 27.9
filebuf 27.9.1.1
filebuf.assign 27.9.1.3
filebuf.cons 27.9.1.2
filebuf.members 27.9.1.4
filebuf.virtuals 27.9.1.5
floatfield.manip 27.5.6.4
fmtflags.manip 27.5.6.1
fmtflags.state 27.5.3.2
forward 20.2.4
forward.iterators 24.2.5
forwardlist 23.3.4
forwardlist.access 23.3.4.4
forwardlist.cons 23.3.4.2
forwardlist.iter 23.3.4.3
forwardlist.modifiers 23.3.4.5
forwardlist.ops 23.3.4.6
forwardlist.overview 23.3.4.1
forwardlist.spec 23.3.4.7
fpos 27.5.4
fpos.members 27.5.4.1
fpos.operations 27.5.4.2
front.insert.iter.cons 24.5.2.4.1
front.insert.iter.op* 24.5.2.4.3
front.insert.iter.op++ 24.5.2.4.4
front.insert.iter.op= 24.5.2.4.2
front.insert.iter.ops 24.5.2.4
front.insert.iterator 24.5.2.3
front.inserter 24.5.2.4.5
fstream 27.9.1.14
fstream.assign 27.9.1.16
fstream.cons 27.9.1.15
fstream.members 27.9.1.17
fstreams 27.9.1
func.bind 20.9.10
func.bind.bind 20.9.10.3
func.bind.isbind 20.9.10.1
func.bind.isplace 20.9.10.2
func.bind.place 20.9.10.4
func.def 20.9.1
func.invoke 20.9.3
func.memfn 20.9.11
func.require 20.9.2
func.wrap 20.9.12
func.wrap.badcall 20.9.12.1
func.wrap.badcall.const 20.9.12.1.1
func.wrap.func 20.9.12.2
func.wrap.func.alg 20.9.12.2.7
func.wrap.func.cap 20.9.12.2.3
func.wrap.func.con 20.9.12.2.1
func.wrap.func.inv 20.9.12.2.4
func.wrap.func.mod 20.9.12.2.2
func.wrap.func.nullptr 20.9.12.2.6
func.wrap.func.targ 20.9.12.2.5
function.objects 20.9
functions.within.classes 17.5.2.2
futures 30.6
futures.async 30.6.8
futures.errors 30.6.2
futures.future_error 30.6.3
futures.overview 30.6.1
futures.promise 30.6.5
futures.shared_future 30.6.7
futures.state 30.6.4
futures.task 30.6.9
futures.task.members 30.6.9.1
futures.task.nonmembers 30.6.9.2
futures.unique_future 30.6.6

G

get.new.handler 18.6.2.5
get.terminate 18.8.3.3
get.unexpected D.8.3
global.functions 17.6.5.4
gram A
gram.basic A.3
gram.class A.8
gram.cpp A.14
gram.dcl A.6
gram.decl A.7
gram.derived A.9
gram.except A.13
gram.expr A.4
gram.key A.1
gram.lex A.2
gram.over A.11
gram.special A.10
gram.stmt A.5
gram.temp A.12
gslice.access 26.6.6.3
gslice.array.assign 26.6.7.2
gslice.array.comp.assign 26.6.7.3
gslice.array.fill 26.6.7.4
gslice.cons 26.6.6.2

H

handler.functions 17.6.4.7
hash.requirements 17.6.3.4
headers 17.6.1.2

I

ifstream 27.9.1.6
ifstream.assign 27.9.1.8
ifstream.cons 27.9.1.7
ifstream.members 27.9.1.9
implimits B
includes 25.4.5.1
indirect.array.assign 26.6.9.2
indirect.array.comp.assign 26.6.9.3
indirect.array.fill 26.6.9.4
inner.product 26.7.3
input.iterators 24.2.3
input.output 27
input.output.general 27.1
input.streams 27.7.2
insert.iter.cons 24.5.2.6.1
insert.iter.op* 24.5.2.6.3
insert.iter.op++ 24.5.2.6.4
insert.iter.op= 24.5.2.6.2
insert.iter.ops 24.5.2.6
insert.iterator 24.5.2.5
insert.iterators 24.5.2
inserter 24.5.2.6.5
intro 1
intro.ack 1.11
intro.compliance 1.4
intro.defs 1.3
intro.execution 1.9
intro.memory 1.7
intro.multithread 1.10
intro.object 1.8
intro.refs 1.2
intro.scope 1.1
intro.structure 1.5
intseq 20.5
intseq.general 20.5.1
intseq.intseq 20.5.2
intseq.make 20.5.3
invalid.argument 19.2.3
ios 27.5.5
ios.base 27.5.3
ios.base.callback 27.5.3.6
ios.base.cons 27.5.3.7
ios.base.locales 27.5.3.3
ios.base.storage 27.5.3.5
ios.members.static 27.5.3.4
ios.overview 27.5.5.1
ios.types 27.5.3.1
ios::failure 27.5.3.1.1
ios::fmtflags 27.5.3.1.2
ios::Init 27.5.3.1.6
ios::iostate 27.5.3.1.3
ios::openmode 27.5.3.1.4
ios::seekdir 27.5.3.1.5
iostate.flags 27.5.5.4
iostream.assign 27.7.2.5.3
iostream.cons 27.7.2.5.1
iostream.dest 27.7.2.5.2
iostream.format 27.7
iostream.format.overview 27.7.1
iostream.forward 27.3
iostream.limits.imbue 27.2.1
iostream.objects 27.4
iostream.objects.overview 27.4.1
iostreamclass 27.7.2.5
iostreams.base 27.5
iostreams.base.overview 27.5.1
iostreams.limits.pos 27.2.2
iostreams.requirements 27.2
iostreams.threadsafety 27.2.3
is.heap 25.4.6.5
is.sorted 25.4.1.5
istream 27.7.2.1
istream.assign 27.7.2.1.2
istream.cons 27.7.2.1.1
istream.formatted 27.7.2.2
istream.formatted.arithmetic 27.7.2.2.2
istream.formatted.reqmts 27.7.2.2.1
istream.iterator 24.6.1
istream.iterator.cons 24.6.1.1
istream.iterator.ops 24.6.1.2
istream.manip 27.7.2.4
istream.rvalue 27.7.2.6
istream.unformatted 27.7.2.3
istream::extractors 27.7.2.2.3
istream::sentry 27.7.2.1.3
istreambuf.iterator 24.6.3
istreambuf.iterator.cons 24.6.3.2
istreambuf.iterator::equal 24.6.3.5
istreambuf.iterator::op!= 24.6.3.7
istreambuf.iterator::op* 24.6.3.3
istreambuf.iterator::op++ 24.6.3.4
istreambuf.iterator::op== 24.6.3.6
istreambuf.iterator::proxy 24.6.3.1
istringstream 27.8.3

istringstream.assign 27.8.3.2
istringstream.cons 27.8.3.1
istringstream.members 27.8.3.3
iterator.basic 24.4.2
iterator.container 24.8
iterator.iterators 24.2.2
iterator.operations 24.4.4
iterator.primitives 24.4
iterator.range 24.7
iterator.requirements 24.2
iterator.requirements.general 24.2.1
iterator.synopsis 24.3
iterator.traits 24.4.1
iterators 24
iterators.general 24.1

J

K

L

language.support 18
length.error 19.2.4
lex 2
lex.bool 2.13.6
lex.ccon 2.13.3
lex.charset 2.3
lex.comment 2.7
lex.digraph 2.5
lex.ext 2.13.8
lex.fcon 2.13.4
lex.header 2.8
lex.icon 2.13.2
lex.key 2.11
lex.literal 2.13
lex.literal.kinds 2.13.1
lex.name 2.10
lex.nullptr 2.13.7
lex.operators 2.12
lex.phases 2.2
lex.ppnumber 2.9
lex.pptoken 2.4
lex.separate 2.1
lex.string 2.13.5
lex.token 2.6
lib.types.movedfrom 17.6.5.15
library 17
library.c 17.2
library.general 17.1
limits 18.3.2
limits.numeric 18.3.2.1
limits.syn 18.3.2.2
list 23.3.5
list.capacity 23.3.5.3
list.cons 23.3.5.2
list.modifiers 23.3.5.4
list.ops 23.3.5.5
list.overview 23.3.5.1
list.special 23.3.5.6
locale 22.3.1
locale.categories 22.4
locale.category 22.3.1.1.1
locale.codecvt 22.4.1.4
locale.codecvt.byname 22.4.1.5
locale.codecvt.members 22.4.1.4.1
locale.codecvt.virtuals 22.4.1.4.2
locale.collate 22.4.4.1
locale.collate.byname 22.4.4.2
locale.collate.members 22.4.4.1.1
locale.collate.virtuals 22.4.4.1.2
locale.cons 22.3.1.2
locale.convenience 22.3.3
locale.ctype 22.4.1.1
locale.ctype.byname 22.4.1.2
locale.ctype.members 22.4.1.1.1
locale.ctype.virtuals 22.4.1.1.2
locale.facet 22.3.1.1.2
locale.global.templates 22.3.2
locale.id 22.3.1.1.3
locale.members 22.3.1.3
locale.messages 22.4.7.1
locale.messages.byname 22.4.7.2
locale.messages.members 22.4.7.1.1
locale.messages.virtuals 22.4.7.1.2
locale.money.get 22.4.6.1
locale.money.get.members 22.4.6.1.1
locale.money.get.virtuals 22.4.6.1.2
locale.money.put 22.4.6.2
locale.money.put.members 22.4.6.2.1
locale.money.put.virtuals 22.4.6.2.2
locale.moneypunct 22.4.6.3
locale.moneypunct.byname 22.4.6.4
locale.moneypunct.members 22.4.6.3.1
locale.moneypunct.virtuals 22.4.6.3.2
locale.nm.put 22.4.2.2
locale.num.get 22.4.2.1
locale.numpunct 22.4.3.1
locale.numpunct.byname 22.4.3.2
locale.operators 22.3.1.4
locale.statics 22.3.1.5
locale.stdcvt 22.5
locale.syn 22.2

locale.time.get 22.4.5.1
locale.time.get.byname 22.4.5.2
locale.time.get.members 22.4.5.1.1
locale.time.get.virtuals 22.4.5.1.2
locale.time.put 22.4.5.3
locale.time.put.byname 22.4.5.4
locale.time.put.members 22.4.5.3.1
locale.time.put.virtuals 22.4.5.3.2
locale.types 22.3.1.1
locales 22.3
localization 22
localization.general 22.1
logic.error 19.2.1
logical.operations 20.9.7
lower.bound 25.4.3.1

M

macro.names 17.6.4.3.1
make.heap 25.4.6.3
map 23.4.4
map.access 23.4.4.3
map.cons 23.4.4.2
map.modifiers 23.4.4.4
map.overview 23.4.4.1
map.special 23.4.4.5
mask.array.assign 26.6.8.2
mask.array.comp.assign 26.6.8.3
mask.array.fill 26.6.8.4
member.functions 17.6.5.5
memory 20.7
memory.general 20.7.1
memory.syn 20.7.2
meta 20.10
meta.help 20.10.3
meta.rel 20.10.6
meta.rqmts 20.10.1
meta.trans 20.10.7
meta.trans.arr 20.10.7.4
meta.trans.cv 20.10.7.1
meta.trans.other 20.10.7.6
meta.trans.ptr 20.10.7.5
meta.trans.ref 20.10.7.2
meta.trans.sign 20.10.7.3
meta.type.synop 20.10.2
meta.unary 20.10.4
meta.unary.cat 20.10.4.1
meta.unary.comp 20.10.4.2
meta.unary.prop 20.10.4.3
meta.unary.prop.query 20.10.5
mismatch 25.2.10
move.iter.nonmember 24.5.3.3.14
move.iter.op.+ 24.5.3.3.8
move.iter.op.+= 24.5.3.3.9
move.iter.op.- 24.5.3.3.10
move.iter.op.-= 24.5.3.3.11
move.iter.op.comp 24.5.3.3.13
move.iter.op.const 24.5.3.3.1
move.iter.op.conv 24.5.3.3.3
move.iter.op.decr 24.5.3.3.7
move.iter.op.incr 24.5.3.3.6
move.iter.op.index 24.5.3.3.12
move.iter.op.ref 24.5.3.3.5
move.iter.op.star 24.5.3.3.4
move.iter.op= 24.5.3.3.2
move.iter.ops 24.5.3.3
move.iter.requirements 24.5.3.2
move.iterator 24.5.3.1
move.iterators 24.5.3
multibyte.strings 17.5.2.1.4.2
multimap 23.4.5
multimap.cons 23.4.5.2
multimap.modifiers 23.4.5.3
multimap.overview 23.4.5.1
multimap.special 23.4.5.4
multiset 23.4.7
multiset.cons 23.4.7.2
multiset.overview 23.4.7.1
multiset.special 23.4.7.3

N

namespace.alias 7.3.2
namespace.constraints 17.6.4.2
namespace.def 7.3.1
namespace.memdef 7.3.1.2
namespace.posix 17.6.4.2.2
namespace.qual 3.4.3.2
namespace.std 17.6.4.2.1
namespace.udecl 7.3.3
namespace.udir 7.3.4
namespace.unnamed 7.3.1.1
narrow.stream.objects 27.4.2
negators 20.9.9
new.badlength 18.6.2.2
new.delete 18.6.1
new.delete.array 18.6.1.2
new.delete.dataraces 18.6.1.4
new.delete.placement 18.6.1.3
new.delete.single 18.6.1.1
new.handler 18.6.2.3
nullablepointer.requirements 17.6.3.3

numarray 26.6
numeric.iota 26.7.6
numeric.limits 18.3.2.3
numeric.limits.members 18.3.2.4
numeric.ops 26.7
numeric.ops.overview 26.7.1
numeric.requirements 26.2
numeric.special 18.3.2.7
numerics 26
numerics.general 26.1

O

objects.within.classes 17.5.2.3
ofstream 27.9.1.10
ofstream.assign 27.9.1.12
ofstream.cons 27.9.1.11
ofstream.members 27.9.1.13
operators 20.2.1
organization 17.6.1
ostream 27.7.3.1
ostream.assign 27.7.3.3
ostream.cons 27.7.3.2
ostream.formatted 27.7.3.6
ostream.formatted.reqmts 27.7.3.6.1
ostream.inserters 27.7.3.6.3
ostream.inserters.arithmetic 27.7.3.6.2
ostream.inserters.character 27.7.3.6.4
ostream.iterator 24.6.2
ostream.iterator.cons.des 24.6.2.1
ostream.iterator.ops 24.6.2.2
ostream.manip 27.7.3.8
ostream.rvalue 27.7.3.9
ostream.seeks 27.7.3.5
ostream.unformatted 27.7.3.7
ostream::sentry 27.7.3.4
ostreambuf.iter.cons 24.6.4.1
ostreambuf.iter.ops 24.6.4.2
ostreambuf.iterator 24.6.4
ostringstream 27.8.4
ostringstream.assign 27.8.4.2
ostringstream.cons 27.8.4.1
ostringstream.members 27.8.4.3
out.of.range 19.2.5
output.iterators 24.2.4
output.streams 27.7.3
over 13
over.ass 13.5.3
over.best.ics 13.3.3.1
over.binary 13.5.2
over.built 13.6
over.call 13.5.4
over.call.func 13.3.1.1.1
over.call.object 13.3.1.1.2
over.dcl 13.2
over.ics.ellipsis 13.3.3.1.3
over.ics.list 13.3.3.1.5
over.ics.rank 13.3.3.2
over.ics.ref 13.3.3.1.4
over.ics.scs 13.3.3.1.1
over.ics.user 13.3.3.1.2
over.inc 13.5.7
over.literal 13.5.8
over.load 13.1
over.match 13.3
over.match.best 13.3.3
over.match.call 13.3.1.1
over.match.conv 13.3.1.5
over.match.copy 13.3.1.4
over.match.ctor 13.3.1.3
over.match.funcs 13.3.1
over.match.list 13.3.1.7
over.match.oper 13.3.1.2
over.match.ref 13.3.1.6
over.match.viable 13.3.2
over.oper 13.5
over.over 13.4
over.ref 13.5.6
over.sub 13.5.5
over.unary 13.5.1
overflow.error 19.2.8

P

pair.astuple 20.3.4
pair.piecewise 20.3.5
pairs 20.3
pairs.general 20.3.1
pairs.pair 20.3.2
pairs.spec 20.3.3
partial.sort 25.4.1.3
partial.sort.copy 25.4.1.4
partial.sum 26.7.4
pointer.traits 20.7.3
pointer.traits.functions 20.7.3.2
pointer.traits.types 20.7.3.1
pop.heap 25.4.6.2
predef.iterators 24.5
priority.queue 23.6.4
priqueue.cons 23.6.4.1
priqueue.cons.alloc 23.6.4.2
priqueue.members 23.6.4.3

priqueue.special 23.6.4.4
propagation 18.8.5
protection.within.classes 17.6.5.10
ptr.align 20.7.5
push.heap 25.4.6.1

Q

queue 23.6.3
queue.cons 23.6.3.2
queue.cons.alloc 23.6.3.3
queue.defn 23.6.3.1
queue.ops 23.6.3.4
queue.special 23.6.3.5
queue.syn 23.6.2
quoted.manip 27.7.6

R

rand 26.5
rand.adapt 26.5.4
rand.adapt.disc 26.5.4.2
rand.adapt.general 26.5.4.1
rand.adapt.ibits 26.5.4.3
rand.adapt.shuf 26.5.4.4
rand.device 26.5.6
rand.dist 26.5.8
rand.dist.bern 26.5.8.3
rand.dist.bern.bernoulli 26.5.8.3.1
rand.dist.bern.bin 26.5.8.3.2
rand.dist.bern.geo 26.5.8.3.3
rand.dist.bern.negbin 26.5.8.3.4
rand.dist.general 26.5.8.1
rand.dist.norm 26.5.8.5
rand.dist.norm.cauchy 26.5.8.5.4
rand.dist.norm.chisq 26.5.8.5.3
rand.dist.norm.f 26.5.8.5.5
rand.dist.norm.lognormal 26.5.8.5.2
rand.dist.norm.normal 26.5.8.5.1
rand.dist.norm.t 26.5.8.5.6
rand.dist.pois 26.5.8.4
rand.dist.pois.exp 26.5.8.4.2
rand.dist.pois.extreme 26.5.8.4.5
rand.dist.pois.gamma 26.5.8.4.3
rand.dist.pois.poisson 26.5.8.4.1
rand.dist.pois.weibull 26.5.8.4.4
rand.dist.samp 26.5.8.6
rand.dist.samp.discrete 26.5.8.6.1
rand.dist.samp.pconst 26.5.8.6.2
rand.dist.samp.plinear 26.5.8.6.3
rand.dist.uni 26.5.8.2
rand.dist.uni.int 26.5.8.2.1
rand.dist.uni.real 26.5.8.2.2
rand.eng 26.5.3
rand.eng.lcong 26.5.3.1
rand.eng.mers 26.5.3.2
rand.eng.sub 26.5.3.3
rand.predef 26.5.5
rand.req 26.5.1
rand.req.adapt 26.5.1.5
rand.req.dist 26.5.1.6
rand.req.eng 26.5.1.4
rand.req.genl 26.5.1.1
rand.req.seedseq 26.5.1.2
rand.req.urng 26.5.1.3
rand.synopsis 26.5.2
rand.util 26.5.7
rand.util.canonical 26.5.7.2
rand.util.seedseq 26.5.7.1
random.access.iterators 24.2.7
range.error 19.2.7
ratio 20.11
ratio.arithmetic 20.11.4
ratio.comparison 20.11.5
ratio.general 20.11.1
ratio.ratio 20.11.3
ratio.si 20.11.6
ratio.syn 20.11.2
re 28
re.alg 28.11
re.alg.match 28.11.2
re.alg.replace 28.11.4
re.alg.search 28.11.3
re.badexp 28.6
re.const 28.5
re.def 28.2
re.err 28.5.3
re.except 28.11.1
re.general 28.1
re.grammar 28.13
re.iter 28.12
re.matchflag 28.5.2
re.regex 28.8
re.regex.assign 28.8.3
re.regex.const 28.8.1
re.regex.construct 28.8.2
re.regex.locale 28.8.5
re.regex.nmswap 28.8.7.1
re.regex.nonmemb 28.8.7
re.regex.operations 28.8.4
re.regex.swap 28.8.6
re.regiter 28.12.1

re.regiter.cnstr 28.12.1.1
re.regiter.comp 28.12.1.2
re.regiter.deref 28.12.1.3
re.regiter.incr 28.12.1.4
re.req 28.3
re.results 28.10
re.results.acc 28.10.4
re.results.all 28.10.6
re.results.const 28.10.1
re.results.form 28.10.5
re.results.nonmember 28.10.8
re.results.size 28.10.3
re.results.state 28.10.2
re.results.swap 28.10.7
re.submatch 28.9
re.submatch.members 28.9.1
re.submatch.op 28.9.2
re.syn 28.4
re.synopt 28.5.1
re.tokiter 28.12.2
re.tokiter.cnstr 28.12.2.1
re.tokiter.comp 28.12.2.2
re.tokiter.deref 28.12.2.3
re.tokiter.incr 28.12.2.4
re.traits 28.7
reentrancy 17.6.5.8
refwrap 20.9.4
refwrap.access 20.9.4.3
refwrap.assign 20.9.4.2
refwrap.const 20.9.4.1
refwrap.helpers 20.9.4.5
refwrap.invoke 20.9.4.4
replacement.functions 17.6.4.6
requirements 17.6
res.on.arguments 17.6.4.9
res.on.data.races 17.6.5.9
res.on.exception.handling 17.6.5.12
res.on.functions 17.6.4.8
res.on.headers 17.6.5.2
res.on.macro.definitions 17.6.5.3
res.on.objects 17.6.4.10
res.on.pointer.storage 17.6.5.13
res.on.required 17.6.4.11
reserved.names 17.6.4.3
reverse.iter.cons 24.5.1.3.1
reverse.iter.conv 24.5.1.3.3
reverse.iter.make 24.5.1.3.21
reverse.iter.op!= 24.5.1.3.15
reverse.iter.op+ 24.5.1.3.8
reverse.iter.op++ 24.5.1.3.6
reverse.iter.op+= 24.5.1.3.9
reverse.iter.op- 24.5.1.3.10
reverse.iter.op-= 24.5.1.3.11
reverse.iter.op.star 24.5.1.3.4
reverse.iter.op< 24.5.1.3.14
reverse.iter.op<= 24.5.1.3.18
reverse.iter.op= 24.5.1.3.2
reverse.iter.op== 24.5.1.3.13
reverse.iter.op> 24.5.1.3.16
reverse.iter.op>= 24.5.1.3.17
reverse.iter.opdiff 24.5.1.3.19
reverse.iter.opindex 24.5.1.3.12
reverse.iter.opref 24.5.1.3.5
reverse.iter.ops 24.5.1.3
reverse.iter.opsum 24.5.1.3.20
reverse.iter.op-- 24.5.1.3.7
reverse.iter.requirements 24.5.1.2
reverse.iterator 24.5.1.1
reverse.iterators 24.5.1
round.style 18.3.2.5
runtime.error 19.2.6

S

scoped.adaptor.operators 20.13.5
sequence.reqmts 23.2.3
sequences 23.3
sequences.general 23.3.1
set 23.4.6
set.cons 23.4.6.2
set.difference 25.4.5.4
set.intersection 25.4.5.3
set.new.handler 18.6.2.4
set.overview 23.4.6.1
set.special 23.4.6.3
set.symmetric.difference 25.4.5.5
set.terminate 18.8.3.2
set.unexpected D.8.2
set.union 25.4.5.2
slice.access 26.6.4.3
slice.arr.assign 26.6.5.2
slice.arr.comp.assign 26.6.5.3
slice.arr.fill 26.6.5.4
smartptr 20.8
sort 25.4.1.1
sort.heap 25.4.6.4
special 12
specialized.addressof 20.7.12.1
specialized.algorithms 20.7.12
stable.sort 25.4.1.2
stack 23.6.5
stack.cons 23.6.5.3

stack.cons.alloc 23.6.5.4
stack.defn 23.6.5.2
stack.ops 23.6.5.5
stack.special 23.6.5.6
stack.syn 23.6.5.1
std.exceptions 19.2
std.ios.manip 27.5.6
std.iterator.tags 24.4.3
std.manip 27.7.4
stmt.ambig 6.8
stmt.block 6.3
stmt.break 6.6.1
stmt.cont 6.6.2
stmt.dcl 6.7
stmt.do 6.5.2
stmt.expr 6.2
stmt.for 6.5.3
stmt.goto 6.6.4
stmt.if 6.4.1
stmt.iter 6.5
stmt.jump 6.6
stmt.label 6.1
stmt.ranged 6.5.4
stmt.return 6.6.3
stmt.select 6.4
stmt.stmt 6
stmt.switch 6.4.2
stmt.while 6.5.1
storage.iterator 20.7.10
stream.buffers 27.6
stream.buffers.overview 27.6.1
stream.iterators 24.6
stream.types 27.5.2
streambuf 27.6.3
streambuf.assign 27.6.3.3.1
streambuf.buffer 27.6.3.2.2
streambuf.cons 27.6.3.1
streambuf.get.area 27.6.3.3.2
streambuf.locales 27.6.3.2.1
streambuf.members 27.6.3.2
streambuf.protected 27.6.3.3
streambuf.pub.get 27.6.3.2.3
streambuf.pub.pback 27.6.3.2.4
streambuf.pub.put 27.6.3.2.5
streambuf.put.area 27.6.3.3.3
streambuf.reqts 27.6.2
streambuf.virt.buffer 27.6.3.4.2
streambuf.virt.get 27.6.3.4.3
streambuf.virt.locales 27.6.3.4.1
streambuf.virt.pback 27.6.3.4.4
streambuf.virt.put 27.6.3.4.5
streambuf.virtuals 27.6.3.4
string.access 21.4.5
string.accessors 21.4.7.1
string.capacity 21.4.4
string.classes 21.3
string.cons 21.4.2
string.conversions 21.5
string.io 21.4.8.9
string.iterators 21.4.3
string.modifiers 21.4.6
string.nonmembers 21.4.8
string.ops 21.4.7
string.require 21.4.1
string.special 21.4.8.8
string.streams 27.8
string.streams.overview 27.8.1
string::append 21.4.6.2
string::assign 21.4.6.3
string::compare 21.4.7.9
string::copy 21.4.6.7
string::erase 21.4.6.5
string::find 21.4.7.2
string::find.first.not.of 21.4.7.6
string::find.first.of 21.4.7.4
string::find.last.not.of 21.4.7.7
string::find.last.of 21.4.7.5
string::insert 21.4.6.4
string::op!= 21.4.8.3
string::op+ 21.4.8.1
string::op+= 21.4.6.1
string::op< 21.4.8.4
string::op<= 21.4.8.6
string::op> 21.4.8.5
string::op>= 21.4.8.7
string::operator== 21.4.8.2
string::replace 21.4.6.6
string::rfind 21.4.7.3
string::substr 21.4.7.8
string::swap 21.4.6.8
stringbuf 27.8.2
stringbuf.assign 27.8.2.2
stringbuf.cons 27.8.2.1
stringbuf.members 27.8.2.3
stringbuf.virtuals 27.8.2.4
strings 21
strings.general 21.1
stringstream 27.8.5
stringstream.assign 27.8.5.2
stringstream.cons 27.8.5.1
stringstream.members 27.8.5.3
structure 17.5.1

structure.elements 17.5.1.1
structure.requirements 17.5.1.3
structure.see.also 17.5.1.5
structure.specifications 17.5.1.4
structure.summary 17.5.1.2
support.dynamic 18.6
support.exception 18.8
support.general 18.1
support.initlist 18.9
support.initlist.access 18.9.2
support.initlist.cons 18.9.1
support.initlist.range 18.9.3
support.limits 18.3
support.limits.general 18.3.1
support.rtti 18.7
support.runtime 18.10
support.start.term 18.5
support.types 18.2
swappable.requirements 17.6.3.2
syntax 1.6
syserr 19.5
syserr.compare 19.5.4
syserr.errcat 19.5.1
syserr.errcat.derived 19.5.1.4
syserr.errcat.nonvirtuals 19.5.1.3
syserr.errcat.objects 19.5.1.5
syserr.errcat.overview 19.5.1.1
syserr.errcat.virtuals 19.5.1.2
syserr.errcode 19.5.2
syserr.errcode.constructors 19.5.2.2
syserr.errcode.modifiers 19.5.2.3
syserr.errcode.nonmembers 19.5.2.5
syserr.errcode.observers 19.5.2.4
syserr.errcode.overview 19.5.2.1
syserr.errcondition 19.5.3
syserr.errcondition.constructors 19.5.3.2
syserr.errcondition.modifiers 19.5.3.3
syserr.errcondition.nonmembers 19.5.3.5
syserr.errcondition.observers 19.5.3.4
syserr.errcondition.overview 19.5.3.1
syserr.hash 19.5.5
syserr.syserr 19.5.6
syserr.syserr.members 19.5.6.2
syserr.syserr.overview 19.5.6.1

T

temp 14
temp.alias 14.5.7
temp.arg 14.3
temp.arg.explicit 14.8.1
temp.arg.nontype 14.3.2
temp.arg.template 14.3.3
temp.arg.type 14.3.1
temp.class 14.5.1
temp.class.order 14.5.5.2
temp.class.spec 14.5.5
temp.class.spec.match 14.5.5.1
temp.class.spec.mfunc 14.5.5.3
temp.decls 14.5
temp.deduct 14.8.2
temp.deduct.call 14.8.2.1
temp.deduct.conv 14.8.2.3
temp.deduct.decl 14.8.2.6
temp.deduct.funcaddr 14.8.2.2
temp.deduct.partial 14.8.2.4
temp.deduct.type 14.8.2.5
temp.dep 14.6.2
temp.dep.candidate 14.6.4.2
temp.dep.constexpr 14.6.2.3
temp.dep.expr 14.6.2.2
temp.dep.res 14.6.4
temp.dep.temp 14.6.2.4
temp.dep.type 14.6.2.1
temp.expl.spec 14.7.3
temp.explicit 14.7.2
temp.fct 14.5.6
temp.fct.spec 14.8
temp.friend 14.5.4
temp.func.order 14.5.6.2
temp.inject 14.6.5
temp.inst 14.7.1
temp.local 14.6.1
temp.mem 14.5.2
temp.mem.class 14.5.1.2
temp.mem.enum 14.5.1.4
temp.mem.func 14.5.1.1
temp.names 14.2
temp.nondep 14.6.3
temp.over 14.8.3
temp.over.link 14.5.6.1
temp.param 14.1
temp.point 14.6.4.1
temp.res 14.6
temp.spec 14.7
temp.static 14.5.1.3
temp.type 14.4
temp.variadic 14.5.3
template.bitset 20.6
template.gslice.array 26.6.7
template.gslice.array.overview 26.6.7.1
template.indirect.array 26.6.9

template.indirect.array.overview 26.6.9.1
template.mask.array 26.6.8
template.mask.array.overview 26.6.8.1
template.slice.array 26.6.5
template.slice.array.overview 26.6.5.1
template.valarray 26.6.2
template.valarray.overview 26.6.2.1
temporary.buffer 20.7.11
terminate 18.8.3.4
terminate.handler 18.8.3.1
thread 30
thread.condition 30.5
thread.condition.condvar 30.5.1
thread.condition.condvarany 30.5.2
thread.decaycopy 30.2.6
thread.general 30.1
thread.lock 30.4.2
thread.lock.algorithm 30.4.3
thread.lock.guard 30.4.2.1
thread.lock.shared 30.4.2.3
thread.lock.shared.cons 30.4.2.3.1
thread.lock.shared.locking 30.4.2.3.2
thread.lock.shared.mod 30.4.2.3.3
thread.lock.shared.obs 30.4.2.3.4
thread.lock.unique 30.4.2.2
thread.lock.unique.cons 30.4.2.2.1
thread.lock.unique.locking 30.4.2.2.2
thread.lock.unique.mod 30.4.2.2.3
thread.lock.unique.obs 30.4.2.2.4
thread.mutex 30.4
thread.mutex.class 30.4.1.2.1
thread.mutex.recursive 30.4.1.2.2
thread.mutex.requirements 30.4.1
thread.mutex.requirements.general 30.4.1.1
thread.mutex.requirements.mutex 30.4.1.2
thread.once 30.4.4
thread.once.callonce 30.4.4.2
thread.once.onceflag 30.4.4.1
thread.req 30.2
thread.req.exception 30.2.2
thread.req.lockable 30.2.5
thread.req.lockable.basic 30.2.5.2
thread.req.lockable.general 30.2.5.1
thread.req.lockable.req 30.2.5.3
thread.req.lockable.timed 30.2.5.4
thread.req.native 30.2.3
thread.req.paramname 30.2.1
thread.req.timing 30.2.4
thread.sharedmutex.class 30.4.1.4.1
thread.sharedmutex.requirements 30.4.1.4
thread.sharedtimedmutex.class 30.4.1.5.1
thread.sharedtimedmutex.requirements 30.4.1.5
thread.thread.algorithm 30.3.1.7
thread.thread.assign 30.3.1.4
thread.thread.class 30.3.1
thread.thread.constr 30.3.1.2
thread.thread.destr 30.3.1.3
thread.thread.id 30.3.1.1
thread.thread.member 30.3.1.5
thread.thread.static 30.3.1.6
thread.thread.this 30.3.2
thread.threads 30.3
thread.timedmutex.class 30.4.1.3.1
thread.timedmutex.recursive 30.4.1.3.2
thread.timedmutex.requirements 30.4.1.3
time 20.12
time.clock 20.12.7
time.clock.hires 20.12.7.3
time.clock.req 20.12.3
time.clock.steady 20.12.7.2
time.clock.system 20.12.7.1
time.duration 20.12.5
time.duration.arithmetic 20.12.5.3
time.duration.cast 20.12.5.7
time.duration.comparisons 20.12.5.6
time.duration.cons 20.12.5.1
time.duration.literals 20.12.5.8
time.duration.nonmember 20.12.5.5
time.duration.observer 20.12.5.2
time.duration.special 20.12.5.4
time.general 20.12.1
time.point 20.12.6
time.point.arithmetic 20.12.6.3
time.point.cast 20.12.6.7
time.point.comparisons 20.12.6.6
time.point.cons 20.12.6.1
time.point.nonmember 20.12.6.5
time.point.observer 20.12.6.2
time.point.special 20.12.6.4
time.syn 20.12.2
time.traits 20.12.4
time.traits.duration_values 20.12.4.2
time.traits.is_fp 20.12.4.1
time.traits.specializations 20.12.4.3
tuple 20.4
tuple.assign 20.4.2.2
tuple.cnstr 20.4.2.1
tuple.creation 20.4.2.4
tuple.elem 20.4.2.6
tuple.general 20.4.1
tuple.helper 20.4.2.5
tuple.rel 20.4.2.7

tuple.special 20.4.2.9
tuple.swap 20.4.2.3
tuple.traits 20.4.2.8
tuple.tuple 20.4.2
type.descriptions 17.5.2.1
type.descriptions.general 17.5.2.1.1
type.index 20.14
type.index.hash 20.14.4
type.index.members 20.14.3
type.index.overview 20.14.2
type.index.synopsis 20.14.1
type.info 18.7.1

U

uncaught.exceptions 18.8.4
underflow.error 19.2.9
unexpected D.8.4
unexpected.handler D.8.1
uninitialized.copy 20.7.12.2
uninitialized.fill 20.7.12.3
uninitialized.fill.n 20.7.12.4
unique.ptr 20.8.1
unique.ptr.create 20.8.1.4
unique.ptr.dltr 20.8.1.1
unique.ptr.dltr.dflt 20.8.1.1.2
unique.ptr.dltr.dflt1 20.8.1.1.3
unique.ptr.dltr.general 20.8.1.1.1
unique.ptr.runtime 20.8.1.3
unique.ptr.runtime.asgn 20.8.1.3.2
unique.ptr.runtime.ctor 20.8.1.3.1
unique.ptr.runtime.modifiers 20.8.1.3.4
unique.ptr.runtime.observers 20.8.1.3.3
unique.ptr.single 20.8.1.2
unique.ptr.single.asgn 20.8.1.2.3
unique.ptr.single.ctor 20.8.1.2.1
unique.ptr.single.dtor 20.8.1.2.2
unique.ptr.single.modifiers 20.8.1.2.5
unique.ptr.single.observers 20.8.1.2.4
unique.ptr.special 20.8.1.5
unord 23.5
unord.general 23.5.1
unord.hash 20.9.13
unord.map 23.5.4
unord.map.cnstr 23.5.4.2
unord.map.elem 23.5.4.3
unord.map.modifiers 23.5.4.4
unord.map.overview 23.5.4.1
unord.map.swap 23.5.4.5
unord.map.syn 23.5.2
unord.multimap 23.5.5
unord.multimap.cnstr 23.5.5.2
unord.multimap.modifiers 23.5.5.3
unord.multimap.overview 23.5.5.1
unord.multimap.swap 23.5.5.4
unord.multiset 23.5.7
unord.multiset.cnstr 23.5.7.2
unord.multiset.overview 23.5.7.1
unord.multiset.swap 23.5.7.3
unord.req 23.2.5
unord.req.except 23.2.5.1
unord.set 23.5.6
unord.set.cnstr 23.5.6.2
unord.set.overview 23.5.6.1
unord.set.swap 23.5.6.3
unord.set.syn 23.5.3
upper.bound 25.4.3.2
using 17.6.2
using.headers 17.6.2.2
using.linkage 17.6.2.3
using.overview 17.6.2.1
usrlit.suffix 17.6.4.3.4
util.dynamic.safety 20.7.4
util.smartptr 20.8.2
util.smartptr.enab 20.8.2.5
util.smartptr.getdeleter 20.8.2.2.10
util.smartptr.hash 20.8.2.7
util.smartptr.ownerless 20.8.2.4
util.smartptr.shared 20.8.2.2
util.smartptr.shared.assign 20.8.2.2.3
util.smartptr.shared.atomic 20.8.2.6
util.smartptr.shared.cast 20.8.2.2.9
util.smartptr.shared.cmp 20.8.2.2.7
util.smartptr.shared.const 20.8.2.2.1
util.smartptr.shared.create 20.8.2.2.6
util.smartptr.shared.dest 20.8.2.2.2
util.smartptr.shared.io 20.8.2.2.11
util.smartptr.shared.mod 20.8.2.2.4
util.smartptr.shared.obs 20.8.2.2.5
util.smartptr.shared.spec 20.8.2.2.8
util.smartptr.weak 20.8.2.3
util.smartptr.weak.assign 20.8.2.3.3
util.smartptr.weak.const 20.8.2.3.1
util.smartptr.weak.dest 20.8.2.3.2
util.smartptr.weak.mod 20.8.2.3.4
util.smartptr.weak.obs 20.8.2.3.5
util.smartptr.weak.spec 20.8.2.3.6
util.smartptr.weakptr 20.8.2.1
utilities 20
utilities.general 20.1
utility 20.2
utility.arg.requirements 17.6.3.1

utility.exchange 20.2.3
utility.requirements 17.6.3
utility.swap 20.2.2

V

valarray.access 26.6.2.4
valarray.assign 26.6.2.3
valarray.binary 26.6.3.1
valarray.cassign 26.6.2.7
valarray.comparison 26.6.3.2
valarray.cons 26.6.2.2
valarray.members 26.6.2.8
valarray.nonmembers 26.6.3
valarray.range 26.6.10
valarray.special 26.6.3.4
valarray.sub 26.6.2.5
valarray.syn 26.6.1
valarray.transcend 26.6.3.3
valarray.unary 26.6.2.6
value.error.codes 17.6.5.14
vector 23.3.6
vector.bool 23.3.7
vector.capacity 23.3.6.3
vector.cons 23.3.6.2
vector.data 23.3.6.4
vector.modifiers 23.3.6.5
vector.overview 23.3.6.1
vector.special 23.3.6.6

W

wide.stream.objects 27.4.3

X

xref F

Y

Z

Index

`!`, *see* operator, logical negation
`!=`, *see* inequality operator
`()`, *see* operator, function call, *see* declarator, function
`*`, *see* operator, indirection, *see* multiplication operator, *see* declarator, pointer
`+`, *see* operator, unary plus, *see* addition operator
`++`, *see* operator, increment
`,`, *see* comma operator
`-`, *see* operator, unary minus, *see* subtraction operator
`->`, *see* operator, class member access
`->*`, *see* pointer to member operator
`--`, *see* operator, decrement
`.`, *see* operator, class member access
`.*`, *see* pointer to member operator
`...`, *see* ellipsis
`/`, *see* division operator
`:`
  field declaration, 239
  label specifier, 136
`::`, *see* scope resolution operator
`::*`, *see* declarator, pointer to member
`<`, *see* less than operator
  template and, 335, 337
`<...>`, *see* preprocessing directive, header
`<=`, *see* less than or equal to operator
`<<`, *see* left shift operator
`=`, *see* assignment operator
`==`, *see* equality operator
`>`, *see* greater than operator
`>=`, *see* greater than or equal operator
`>>`, *see* right shift operator
`?:`, *see* conditional expression operator
`[]`, *see* operator, subscripting, *see* declarator, array
`"..."`, *see* preprocessing directives, source-file inclusion
`#` operator, 429, 430
`##` operator, 430
`#define`, 429
`#elif`, 427
`#else`, 427
`#endif`, 427
`#error`, *see* preprocessing directives, error
`#if`, 427, 463
`#ifdef`, 427
`#ifndef`, 427
`#include`, 428, 448
`#line`, *see* preprocessing directives, line control
`#pragma`, *see* preprocessing directives, pragma
`#undef`, 432, 460
`%`, *see* remainder operator
`&`, *see* operator, address-of, *see* bitwise AND operator, *see* declarator, reference
`&&`, *see* logical AND operator
`^`, *see* bitwise exclusive OR operator
`__DATE__`, 435
`__FILE__`, 435
`__LINE__`, 435
`__STDC__`, 435
  implementation-defined, 435
`__STDCPP_STRICT_POINTER_SAFETY__`, 436
  implementation-defined, 436
`__STDCPP_THREADS__`, 436
  implementation-defined, 436
`__STDC_HOSTED__`, 435
  implementation-defined, 435
`__STDC_ISO_10646__`, 435
  implementation-defined, 435
`__STDC_MB_MIGHT_NEQ_WC__`, 435
  implementation-defined, 435
`__STDC_VERSION__`, 435
  implementation-defined, 435
`__TIME__`, 435
`__VA_ARGS__`, 429
`__cplusplus`, 435
`~`, *see* destructor
`\`, *see* backslash
`{}`
  block statement, 136
  class declaration, 225
  class definition, 225
  `enum` declaration, 165
  initializer list, 213
`_`, *see* character, underscore
`|`, *see* bitwise inclusive OR operator
`||`, *see* logical OR operator
`~`, *see* operator, one's complement

`0`, *see also* zero, null

null character, 29
string terminator, 29

`abort`, 64, 142
abstract-declarator, 191, 1222
abstract-pack-declarator, 191, 1222
access, 2
access control, 255–265
base class, 258
base class member, 242
class member, 104
default, 255
default argument, 256
friend function, 260
member name, 255
multiple access, 265
nested class, 265
overloading and, 302
`private`, 255
`protected`, 255, 263
`public`, 255
`union` default member, 226
using-declaration and, 177
virtual function, 264
access-specifier, 242, 1224
access control
anonymous `union`, 238
member function and, 266
overloading resolution and, 246
access specifier, 256, 258
addition operator, 123
additive-expression, 124, 1215
address, 76, 126
address of member function
unspecified, 464
aggregate, 213
aggregate initialization, 213
algorithm
stable, 440, 464
alias
namespace, 172
alias template, 362
alias-declaration, 146, 1217
alignment
extended, 79
fundamental, 79
alignment-specifier, 184, 1220
alignment requirement
implementation-defined, 79
allocation
alignment storage, 117
implementation defined bit-field, 239
unspecified, 231
allocation functions, 65
allowing all exceptions, *see* exception specification, allowing all exceptions
allowing an exception, *see* exception specification, allowing an exception
alternative token, *see* token, alternative
ambiguity
base class member, 245
class conversion, 248
declaration type, 148
declaration versus cast, 192
declaration versus expression, 144
function declaration, 211
member access, 245
overloaded function, 302
parentheses and, 115
Amendment 1, 460
and-expression, 127, 1215
anonymous union, 238
appertain, 185
`argc`, 61
argument, 2, 462–464, 498
access checking and default, 256
binding of default, 204
evaluation of default, 204, 205
example of default, 203, 204
function call expression, 2
function-like macro, 2
overloaded operator and default, 325
reference, 103
scope of default, 205
template, 338
template instantiation, 2
throw expression, 2
type checking of default, 204
arguments
implementation-defined order of evaluation of function, 205
argument and name hiding
default, 205
argument and virtual function
default, 205
argument list
empty, 200
variable, 200
argument passing, 103
reference and, 217
argument substitution, *see* macro, argument substitution

argument type
unknown, 200
`argv`, 61
arithmetic
pointer, 124
`unsigned`, 74
array, 200
bound, 198
`const`, 77
`delete`, 120
multidimensional, 199
`new`, 116
overloading and pointer versus, 300
`sizeof`, 114
storage of, 199
`array`
as aggregate, 775
contiguous storage, 775
initialization, 775, 776
tuple interface to, 777
zero sized, 777
array size
default, 198
arrow operator, *see* operator, class member access
as-if rule, 7
`asm`
implementation-defined, 181
asm-definition, 181, 1220
assembler, 181
`<assert.h>`, 449
assignment
and lvalue, 130
conversion by, 130
copy, *see* assignment operator, copy
move, *see* assignment operator, move, 439
reference, 217
assignment operator
copy, 266, 290–293
hidden, 292
implicitly declared, 290
implicitly defined, 292
inaccessible, 293
trivial, 292
virtual bases and, 293
move, 266, 290–293
hidden, 292
implicitly declared, 291
implicitly defined, 292
inaccessible, 293
trivial, 292
overloaded, 325
assignment-expression, 130, 1216
assignment-operator, 130, 1216
associative containers
exception safety, 762
requirements, 762
unordered, *see* unordered associative containers
asynchronous provider, 1194
asynchronous return object, 1194
`atexit`, 64
atomic operations, *see* operation, atomic
attribute, 184–188
alignment, 185
carries dependency, 187
deprecated, 188
noreturn, 187
syntax and semantics, 184
attribute, 184, 1220
attribute-argument-clause, 184, 1221
attribute-declaration, 146, 1218
attribute-list, 184, 1220
attribute-namespace, 184, 1221
attribute-scoped-token, 184, 1221
attribute-specifier, 184, 1220
attribute-specifier-seq, 184, 1220
attribute-token, 184, 1221
automatic storage duration, 65
`awk`, 1103

backslash character, 26
`bad_alloc`, 118
`bad_cast`, 106
`bad_exception`, 424
`bad_typeid`, 107
`bad_typeid::what`
implementation-defined, 488
balanced-token, 185, 1221
balanced-token-seq, 185, 1221
base class
dependent, 372
overloading and, 301
base class subobject, 7
base-clause, 242, 1223
base-specifier, 242, 1224
base-specifier-list, 242, 1223
base-type-specifier, 242, 1224
BaseCharacteristic, 601
base class, 242, 243
direct, 242
indirect, 242
`private`, 258

`protected`, 258
`public`, 258
base class virtual, *see* virtual base class
`basic_ios::failure` argument
implementation-defined, 1026
`begin`
unordered associative containers, 770
behavior
conditionally-supported, 2, 5
default, 439, 443
implementation-defined, 2, 7
locale-specific, 3
observable, 7, 8
on receipt of signal, 8
required, 440, 443
undefined, 4, 5, 8, 879
unspecified, 4, 8
Ben, 302
Bernoulli distributions, 960–963
`bernoulli_distribution`
discrete probability function, 960
binary fold, 100
binary function, 584
binary left fold, 101
binary operator
overloaded, 325
binary right fold, 101
binary-digit, 23, 1210
binary-literal, 23, 1210
BinaryTypeTrait, 601
binary operator
interpretation of, 325
bind directly, 219
binding
reference, 217
`binomial_distribution`
discrete probability function, 961
bit-field, 239
address of, 239
alignment of, 239
implementation-defined sign of, 1237
implementation defined alignment of, 239
type of, 239
unnamed, 239
zero width of, 239
bitmask
empty, 445
block, 438
initialization in, 143
block scope, 40
block statement, *see* statement, compound
block-declaration, 146, 1217
block structure, 143
body
function, 206
Boolean, 239
Boolean literal, 30
boolean literal, *see* literal, boolean
boolean-literal, 30, 1212
Boolean type, 75
bound arguments, 594
bound, of array, 198
brace-or-equal-initializer, 209, 1222
braced-init-list, 209, 1223
`bucket`
unordered associative containers, 770
`bucket_count`
unordered associative containers, 770
`bucket_size`
unordered associative containers, 770
buckets, 763
byte, 6, 114

C
linkage to, 181
standard, 1
standard library, 1
Unicode TR, 1
c-char, 25, 1211
c-char-sequence, 25, 1211
call
operator function, 324
pseudo destructor, 104
call signature, 583
call wrapper, 583, 584
forwarding, 584
simple, 584
type, 583
Callable, 597
callable object, 583, 597
callable type, 583
capture, 93, 1213
capture-default, 93, 1213
capture-list, 93, 1213
captured, 97
by copy, 98
by reference, 98
carries a dependency, 12
carry
`subtract_with_carry_engine`, 949
`<cassert>`, 449
cast

base class, 109
const, 111, 121
derived class, 109
dynamic, 106, 487
construction and, 287
destruction and, 287
integer to pointer, 110
lvalue, 108, 110
pointer-to-function, 110
pointer-to-member, 109, 111
pointer to integer, 110
reference, 108, 111
reinterpret, 110, 121
integer to pointer, 110
lvalue, 110
pointer to integer, 110
pointer-to-function, 110
pointer-to-member, 111
reference, 111
static, 108, 121
lvalue, 108
reference, 108
undefined pointer-to-function, 110
cast-expression, 121, 1215
casting, 104
casting away constness, 112
`catch`, 413
`cauchy_distribution`
probability density function, 970
`cbegin`
unordered associative containers, 771
`cend`
unordered associative containers, 771
`<cerrno>`, 460
`char`
implementation-defined sign of, 74
char-like object, 646
char-like type, 646
`char16_t`, 25
`char16_t` character, 25
`char32_t`, 25
`char32_t` character, 25
`char_class_type`
regular expression traits, 1093
character, 438
decimal-point, 445
multibyte, 3
`signed`, 74
source file, 16
underscore, 22
in identifier, 21
character literal, *see* literal, character
character set, 17–18
basic execution, 6
basic source, 16, 17
character string literal, 430
character-literal, 25, 1210
character string, 28
checking
point of error, 365
syntax, 365
`chi_squared_distribution`
probability density function, 970
class, 76, 225–241
abstract, 253
base, 460, 465
cast to incomplete, 122
constructor and abstract, 254
definition, 35
derived, 465
linkage of, 58
linkage specification, 182
member function, *see* member function, class
pointer to abstract, 253
polymorphic, 248
scope of enumerator, 167
standard-layout, 226
trivial, 226
unnamed, 153
class-head, 225, 1223
class-head-name, 225, 1223
class-key, 225, 1223
class-name, 225, 1223
class-or-decltype, 242, 1224
class-specifier, 225, 1223
class-virt-specifier, 225, 1223
class local, *see* local class
class name
elaborated, 161, 228, 229
point of declaration, 229
scope of, 228
`typedef`, 152, 153, 229
class nested, *see* nested class
class object
assignment to, 130
member, 231
`sizeof`, 114
class object copy, *see* copy constructor
class object initialization, *see* constructor
`clear`
unordered associative containers, 770
`<clocale>`, 445

closure object, 93
closure type, 93
collating element, 1092
comment, 18, 20
  `/* */`, 20
  `//`, 20
common initial sequence, 232
comparison
  pointer, 126
  pointer to function, 126
  undefined pointer, 124
compatible, *see* exception specification, compatible
compilation
  separate, 16
compiler control line, *see* preprocessing directives
complete object, 7
complete object of, 7
completely defined, 230
component, 439
compound-statement, 136, 1216
concatenation
  macro argument, *see* `##`
  string, 29
condition, 137, 1216
*condition*s
  rules for, 137
conditional-expression
  throw-expression in, 128
conditional-expression, 128, 1216
conditionally-supported behavior
  seebehavior, conditionally-supported, 1
conflict, 11
conformance requirements, 4–5, 8
  class templates, 5
  classes, 5
  general, 4–5
  library, 5
  method of description, 4
consistency
  linkage, 149
  linkage specification, 183
  type declaration, 61
`const`, 76
  cast away, 112
  constructor and, 235, 267
  destructor and, 235, 274
  linkage of, 58
  overloading and, 301
`const_cast`, *see* cast, const
`const_local_iterator`, 764
  unordered associative containers, 764
constant, 23, 90
  enumeration, 166
  null pointer, 85
constant expression
  permitted result of, 134
constant iterator, 850
constant-expression, 131, 1216
constexpr function, 153
construction, 285–287
  dynamic cast and, 287
  member access, 285
  move, 439
  pointer to member or base, 285
  `typeid` operator, 286
  virtual function call, 286
constructor, 266
  address of, 268
  array of class objects and, 279
  converting, 272
  copy, 266, 269, 287–290, 446
    elision, 293
    implicitly declared, 288
    implicitly defined, 290
    inaccessible, 293
    trivial, 289
  default, 266, 267
  exception handling, *see* exception handling, constructors and destructors
  explicit call, 268
  implicitly called, 268
  implicitly defined, 268
  inheritance of, 267
  inheriting, 295–298
  move, 266, 287–290
    elision, 293
    implicitly declared, 289
    implicitly defined, 290
    inaccessible, 293
    trivial, 289
  non-trivial, 267
  random number distribution requirement, 941
  random number engine requirement, 938
  `union`, 237
  unspecified argument to, 118
constructor, conversion by, *see* conversion, user-defined
constructor, default, *see* default constructor
`const`-object
  undefined change to, 157
container

contiguous, 747
context
non-deduced, 404
contextually converted to bool, *see* conversion, contextual
contiguous container, 747
contiguous iterators, 851
`continue`
and handler, 413
and try block, 413
control line, *see* preprocessing directives
control-line, 425, 1226
conventions, 443
lexical, 16–32
conversion
argument, 200
array-to-pointer, 82
`bool`, 84
boolean, 86
class, 271
contextual, 81
contextual to `bool`, 81
deduced return type of user-defined, 274
derived-to-base, 314
floating point, 84
floating to integral, 85
function-to-pointer, 82
implementation defined pointer integer, 110
implicit, 81, 271
implicit user-defined, 271
inheritance of user-defined, 274
integer rank, 86
integral, 84
integral to floating, 85
lvalue-to-rvalue, 82, 1233
narrowing, 223
overload resolution and pointer, 324
overload resolution and, 311
pointer, 85
pointer to member, 85
`void*`, 86
qualification, 83
return type, 142
standard, 81–86
static user-defined, 274
to signed, 84
to unsigned, 84
type of, 273
user-defined, 271, 272
usual arithmetic, 88
virtual user-defined, 274
conversion operator, *see* conversion, user defined
conversion rank, 315
conversion-declarator, 272, 1224
conversion-function-id, 272, 1224
conversion-type-id, 272, 1224
conversion explicit type, *see* casting
conversion function, *see* conversion, user-defined
copy
class object, *see* constructor, copy; assignment, copy
copy constructor
random number engine requirement, 938
copy elision, *see* constructor, copy, elision; constructor, move, elision
copy-initialization, 212
`CopyInsertable` into `X`, 748
`count`
unordered associative containers, 770
`<cstdarg>`, 200
`<cstddef>`, 114, 124
`<cstdint>`, 477
`<cstdlib>`, 64, 448
`<cstring>`, 446
ctor-initializer, 280, 1224
`<cuchar>`, 460
cv-decomposition, 83
cv-qualification signature, 83
cv-qualifier, 76
top-level, 77
cv-qualifier, 191, 1221
cv-qualifier-seq, 191, 1221
`<cwchar>`, 460
`<cwctype>`, 460

d-char, 28, 1212
d-char-sequence, 27, 1212
DAG
multiple inheritance, 244, 245
non-virtual base class, 245
virtual base class, 244, 245
data race, 14
data member, *see* member
data race, 14
deadlock, 439
deallocation, *see* `delete`
deallocation functions, 65
decay, *see* conversion, array to pointer; conversion, function to pointer
DECAY_COPY, 1159
decimal-literal, 23, 1210
decl-specifier, 148, 1218

decl-specifier-seq, 148, 1218
declaration, 33, 146–188
  array, 197
  `asm`, 181
  bit-field, 239
  class name, 34
  constant pointer, 194
  default argument, 203–206
  definition versus, 33
  ellipsis in function, 103, 200
  enumerator point of, 39
  `extern`, 34
  `extern` reference, 217
  forward, 150
  forward class, 228
  function, 34, 199
  local class, 241
  member, 229
  multiple, 61
  name, 33
  opaque enum, 34
  overloaded, 299
  overloaded name and `friend`, 262
  parameter, 34, 200
  parentheses in, 192, 194
  pointer, 194
  reference, 195
  `register`, 149
  `static member`, 34
  storage class, 148
  type, 193
  `typedef`, 34
  `typedef` as type, 151
declaration, 146, 1217
declaration-seq, 146, 1217
declaration-statement, 142, 1217
declaration hiding, *see* name hiding
declarative region, 38
declarator, 34, 147, 190–224
  array, 197
  function, 199–202
  meaning of, 193–206
  multidimensional array, 198
  pointer, 194
  pointer to member, 197
  reference, 195
declarator, 190, 1221
declarator-id, 191, 1222
decltype-specifier, 158, 1219
decrement operator
  overloaded, *see* overloading, decrement operator
default access control, *see* access control, default
default constructor
  random number distribution requirement, 941
  seed sequence requirement, 936
default-initialization, 210
default-inserted, 748
defaulted, 208
`DefaultInsertable` into `X`, 748
default argument
  overload resolution and, 311
default initializers
  overloading and, 301
deferred function, 1203
definition, 34
  alternate, 460
  class, 225, 230
  class name as type, 228
  constructor, 206
  declaration as, 147
  function, 206–209
    deleted, 208
    explicitly-defaulted, 207
  local class, 241
  member function, 232
  namespace, 168
  nested class, 239
  pure virtual function, 253
  scope of class, 228
  static member, 236
  virtual function, 251
definitions, 2–4
`delete`, 65, 119, 120, 277
  array, 120
  destructor and, 120, 275
  object, 119
  `operator`, 461
  overloading and, 67
  type of, 277
  undefined, 120
delete-expression, 119, 1215
deleter, 554
dependency-ordered before, 13
deprecated features, 105, 114
dereferencing, *see also* indirection
derivation, *see* inheritance
derived class
  most, *see* most derived class
derived object
  most, *see* most derived object

derived class, 242–254
destruction, 285–287
  dynamic cast and, 287
  member access, 285
  pointer to member or base, 285
  `typeid` operator, 286
  virtual function call, 286
destructor, 274, 446
  default, 274
  exception handling, *see* exception handling, constructors and destructors
  explicit call, 275
  implicit call, 275
  implicitly defined, 275
  non-trivial, 274
  program termination and, 275
  pure virtual, 275
  `union`, 237
  virtual, 275
diagnosable rules, 4
diagnostic message, *see* message, diagnostic
digit, 21, 1209
digit-sequence, 27, 1211
digraph, *see* token, alternative, 19
directed acyclic graph, *see* DAG
directive, preprocessing, *see* preprocessing directives
`discard`
  random number engine requirement, 938
`discard_block_engine`
  generation algorithm, 950
  state, 950
  textual representation, 951
  transition algorithm, 950
discarded-value expression, 89
discrete probability function
  `bernoulli_distribution`, 960
  `binomial_distribution`, 961
  `discrete_distribution`, 973
  `geometric_distribution`, 962
  `negative_binomial_distribution`, 963
  `poisson_distribution`, 963
  `uniform_int_distribution`, 958
`discrete_distribution`
  discrete probability function, 973
  weights, 973
distribution, *see* random number distribution
dominance
  virtual base class, 247
dot operator, *see* operator, class member access
dynamic binding, *see* virtual function
dynamic initialization, 62
dynamic type, *see* type, dynamic
dynamic-exception-specification, 418, 1225
`dynamic_cast`, *see* cast, dynamic

ECMA-262, 1
ECMAScript, 1103, 1136
`egrep`, 1103
elaborated-type-specifier, 161, 1219
elaborated type specifier, *see* class name, elaborated
elif-group, 425, 1226
elif-groups, 425, 1226
elision
  copy, *see* constructor, copy, elision; constructor, move, elision
  copy constructor, *see* constructor, copy, elision
  move constructor, *see* constructor, move, elision
ellipsis
  conversion sequence, 103, 316
  overload resolution and, 311
else-group, 425, 1226
`EmplaceConstructible` into `X` from `args`, 748
empty `future` object, 1199
empty `shared_future` object, 1201
empty-declaration, 146, 1218
enclosing-namespace-specifier, 168, 1220
encoding
  multibyte, 29
encoding-prefix, 25, 1210
`end`
  unordered associative containers, 771
end-of-file, 538
endif-line, 425, 1226
engine, *see* random number engine
engine adaptor, *see* random number engine adaptor
engines with predefined parameters
  `default_random_engine`, 954
  `knuth_b`, 954
  `minstd_rand`, 953
  `minstd_rand0`, 953
  `mt19937`, 953
  `mt19937_64`, 954
  `ranlux24`, 954
  `ranlux24_base`, 954
  `ranlux48`, 954
  `ranlux48_base`, 954
entity, 33

`enum`, 76
overloading and, 300
type of, 165, 166
underlying type, 166
enum-base, 165, 1219
enum-head, 165, 1219
enum-key, 165, 1219
enum-name, 165, 1219
enum-specifier, 165, 1219
enumeration, 165, 166
linkage of, 58
scoped, 166
unscoped, 166
enumeration scope, 42
enumeration type
conversion to, 109
`static_cast`
conversion to, 109
enumerator
definition, 35
value of, 166
enumerator, 165, 1220
enumerator-definition, 165, 1219
enumerator-list, 165, 1219
enum name
`typedef`, 153
environment
program, 61
epoch, 626
`equal_range`
unordered associative containers, 770
equality-expression, 126, 1215
equivalence
template type, 343
type, 151, 228
equivalent-key group, 763
equivalently-valued, 457
equivalent parameter declarations, 300
overloading and, 300
`Erasable` from `X`, 749
`erase`
unordered associative containers, 769, 770
escape-sequence, 25, 1211
escape character, *see* backslash
escape sequence
undefined, 26
Evaluation, 9
evaluation
order of argument, 103
unspecified order of, 10, 63
unspecified order of argument, 103
unspecified order of function call, 103
example
array, 198
class definition, 231
`const`, 194
constant pointer, 194
constructor, 268
constructor and initialization, 279
declaration, 34, 201
declarator, 191
definition, 34
`delete`, 277
derived class, 242
destructor and `delete`, 277
ellipsis, 200
enumeration, 167
explicit destructor call, 276
explicit qualification, 246
friend, 229
friend function, 260
function declaration, 201
function definition, 206
linkage consistency, 149
local class, 241
member function, 233, 260
nested type name, 241
nested class, 239
nested class definition, 240, 265
nested class forward declaration, 240
pointer to member, 197
pure virtual function, 253
scope of `delete`, 277
scope resolution operator, 246
`static` member, 236
subscripting, 198
`typedef`, 151
type name, 191
unnamed parameter, 206
variable parameter list, 200
virtual function, 250, 251
exception
arithmetic, 87
undefined arithmetic, 87
`<exception>`, 488
exception handling, 413–424
constructors and destructors, 416
exception object, 415
constructor, 415
destructor, 415
function try block, 414
`goto`, 413

handler, 413, 415–418, 465
array in, 416
incomplete type in, 416
match, 416–418
pointer to function in, 416
rvalue reference in, 416
memory, 415
nearest handler, 415
rethrow, 130, 415
rethrowing, 415
`switch`, 413
`terminate()` called, 130, 415, 420
throwing, 129, 414, 415
try block, 413
`unexpected()` called, 420
exception object, *see* exception handling, exception object
exception specification, 418–423
allowing all exceptions, 420
allowing an exception, 420
compatible, 419
incomplete type and, 418
noexcept
constant expression and, 418
non-throwing, 418
virtual function and, 419
exception-declaration, 413, 1225
exception-specification, 418, 1225
`exception::what` message
implementation-defined, 489
exclusive-or-expression, 127, 1215
execution agent, 1157
`exit`, 61, 63, 142
explicit-instantiation, 383, 1225
explicit-specialization, 385, 1225
explicitly captured, 96
explicit type conversion, *see* casting
exponent-part, 27, 1211
`exponential_distribution`
probability density function, 964
expression, 87–135
additive operators, 123
`alignof`, 121
assignment and compound assignment, 130
bitwise AND, 127
bitwise exclusive OR, 127
bitwise inclusive OR, 127
cast, 104, 121–122
class member access, 104
comma, 131
conditional operator, 128
constant, 131
const cast, 111
decrement, 105, 114
`delete`, 119
dynamic cast, 106
equality operators, 126
fold, 100–101
function call, 102
increment, 105, 114
lambda, 93–100
left-shift-operator, 125
logical AND, 128
logical OR, 128
multiplicative operators, 123
`new`, 114
`noexcept`, 121
order of evaluation of, 8
parenthesized, 91
pointer-to-member, 122
pointer to member constant, 113
postfix, 101–112
primary, 90–100
pseudo-destructor call, 104
reference, 87
reinterpret cast, 110
relational operators, 125
right-shift-operator, 125
rvalue reference, 87
`sizeof`, 114
static cast, 108
`throw`, 129
type identification, 107
unary, 112–121
unary operator, 112
expression, 131, 1216
expression-list, 101, 1214
expression-statement, 136, 1216
extend, *see* namespace, extend
extended alignment, 79
extended integer type, 74
extended signed integer type, 74
extended unsigned integer type, 74
`extern`, 148
linkage of, 149
`extern "C"`, 449, 460
`extern "C++"`, 449, 460
external linkage, 58
`extreme_value_distribution`
probability density function, 967

file, source, *see* source file

final overrider, 249
`find`
unordered associative containers, 770
finite state machine, 1092
`fisher_f_distribution`
probability density function, 971
floating literal, *see* literal, floating
floating-literal, 26, 1211
floating-point literal, *see* literal, floating
floating-suffix, 27, 1211
floating point type, 75
implementation-defined, 75
fold
binary, 100
unary, 100
fold-expression, 100, 1213
fold-operator, 100, 1214
`for`
scope of declaration in, 140
for-init-statement, 138, 1217
for-range-declaration, 139, 1217
for-range-initializer, 139, 1217
format specifier, 1092
forwarding call wrapper, 584
forwarding reference, 399
fractional-constant, 27, 1211
free store, 277
freestanding implementation, 5
free store, *see also* `new`, `delete`
friend
`virtual` and, 251
access specifier and, 262
class access and, 260
inheritance and, 262
local class and, 263
template and, 352
friend function
access and, 260
inline, 262
linkage of, 262
member function and, 260
friend function
nested class, 240
full-expression, 9
function, *see also* friend function; member function; inline function; virtual function, 200
allocation, 66, 116
comparison, 438
conversion, 272
deallocation, 66, 120, 277
definition, 35
global, 460, 463, 464
handler, 439
linkage specification overloaded, 183
modifier, 439
observer, 440
operator, 324
overload resolution and, 303
plain old, 494
pointer to member, 123
replacement, 440
reserved, 440
viable, 303
virtual member, 460, 464
function object, 580
binders, 593–595
`mem_fn`, 595
`reference_wrapper`, 584
type, 580
wrapper, 595–600
function pointer type, 76
function try block, *see* exception handling, function try block
function, overloaded, *see* overloading
function, virtual, *see* virtual function
function-definition, 206, 1222
function-like macro, *see* macro, function-like
function-specifier, 150, 1218
function-try-block, 413, 1225
functions
candidate, 377
function argument, *see* argument
function call, 103
recursive, 103
undefined, 110
function call operator
overloaded, 326
function parameter, *see* parameter
function prototype, 40
function return, *see* `return`
function return type, *see* return type
fundamental alignment, 79
fundamental type
destructor and, 276
fundamental type conversion, *see* conversion, user-defined
future
shared state, 1194

`gamma_distribution`
probability density function, 965
`generate`

seed sequence requirement, 936
generated destructor, *see* destructor, default
generation algorithm
`discard_block_engine`, 950
`independent_bits_engine`, 951
`linear_congruential_engine`, 946
`mersenne_twister_engine`, 947
`shuffle_order_engine`, 952
`subtract_with_carry_engine`, 949
generic lambda
definition of, 162
`geometric_distribution`
discrete probability function, 962
global, 41
global namespace, 41
global namespace scope, 41
global scope, 41
glvalue, 78
`goto`
and handler, 413
and try block, 413
initialization and, 143
grammar
regular expression, 1136
`grep`, 1103
group, 425, 1226
group-part, 425, 1226

h-char, 20, 1209
h-char-sequence, 20, 1209
handler, *see* exception handling, handler
handler, 413, 1225
handler-seq, 413, 1225
happens before, 13
`hash`
instantiation restrictions, 600
hash code, 763
hash function, 763
hash tables, *see* unordered associative containers
`hash_function`
unordered associative containers, 767
`hasher`
unordered associative containers, 764
header
C, 460, 463, 1251
C library, 449
C++ library, 447
name, 20
header-name, 20, 1209
hex-quad, 17, 1208
hexadecimal-digit, 23, 1210
hexadecimal-escape-sequence, 25, 1211
hexadecimal-literal, 23, 1210
hiding, *see* name hiding
high-order bit, 6
hosted implementation, 5

id
qualified, 92
id-expression, 91
id-expression, 90, 1213
identifier, 21, 91, 147
identifier, 21, 1209
identifier-list, 426, 1227
identifier-nondigit, 21, 1209
if-group, 425, 1226
if-section, 425, 1226
ill-formed program, *see* program, ill-formed
immediate subexpression, 420
immolation
self, 388
implementation
freestanding, 448
hosted, 448
implementation limits, *see* limits, implementation
implementation-defined, 460, 468, 479, 484, 489, 695, 1019, 1072, 1248
implementation-defined behavior, *see* behavior, implementation-defined
implementation-dependent, 1044
implementation-generated, 34
implicit capture
definition of, 97
implicit object parameter, 303
implicitly-declared default constructor, *see* constructor, default, 267
implicit conversion, *see* conversion, implicit
implied object argument, 303
implicit conversion sequences, 304
non-static member function and, 304
inclusion
conditional, *see* preprocessing directive, conditional inclusion
source file, *see* preprocessing directives, source-file inclusion
inclusive-or-expression, 127, 1215
incomplete, 124
incompletely-defined object type, 72
increment
`bool`, 105, 114
increment operator

overloaded, *see* overloading, increment operator
`independent_bits_engine`
generation algorithm, 951
state, 951
textual representation, 952
transition algorithm, 951
indeterminately sequenced, 10
indeterminate value, 211
indirection, 112
inheritance, 242
init-capture, 93, 1213
init-declarator, 190, 1221
init-declarator-list, 190, 1221
initialization, 61, 209–224
aggregate, 213
array, 213
array of class objects, 216, 279
automatic, 143
automatic object, 209
base class, 280, 281
character array, 216
character array, 216
class member, 211
class object, *see also* constructor, 213, 278–284
`const`, 157, 213
const member, 281
constant, 62
constructor and, 278, 279
copy, 212
default, 210
default constructor and, 278
definition and, 147
direct, 212
dynamic, 62
explicit, 279
jump past, 143
list-initialization, 219–224
local `static`, 143
member, 280
member function call during, 284
member object, 281
non-vacuous, 68
order of, 62, 243
order of base class, 283
order of member, 283
order of virtual base class, 282
overloaded assignment and, 279
parameter, 102
reference, 196, 217
reference member, 281
run-time, 62
static and thread, 61
static member, 236
`static object`, 62
`static` object, 209, 210
`union`, 216, 238
virtual base class, 290
initializer
base class, 206
member, 206
pack expansion, 284
scope of member, 283
temporary and declarator, 270
initializer, 209, 1222
initializer-clause, 209, 1222
initializer-list, 209, 1222
initializer-list constructor
seed sequence requirement, 936
`<initializer_list>`, 492
injected-class-name, 225
inline, 463
`inline`
linkage of, 58
inline function, 150
`insert`
unordered associative containers, 768, 769
instantiation
dependent member of the current, 372
explicit, 383
member of the current, 372
point of, 377
template implicit, 379
instantiation units, 17
integer literal, *see* literal, integer
integer representation, 68
integer-literal, 23, 1210
integer-suffix, 23, 1210
integer type, 75
integral type, 75
`sizeof`, 74
inter-thread happens before, 13
internal linkage, 58
interval boundaries
`piecewise_constant_distribution`, 975
`piecewise_linear_distribution`, 976
invocation
macro, 430
`isctype`
regular expression traits, 1094
iteration-statement, 138, 142, 1217

Jessie, 272
jump-statement, 141, 1217

`key_eq`
  unordered associative containers, 767
`key_equal`
  unordered associative containers, 764
`key_type`
  unordered associative containers, 764
keyword, 22

label, 142
  `case`, 136, 138
  `default`, 136, 138
  scope of, 41, 136
labeled-statement, 136, 1216
lambda-capture, 93, 1213
lambda-declarator, 93, 1213
lambda-expression, 93, 1213
lambda-introducer, 93, 158, 1213
lattice, *see* DAG, subobject
layout
  bit-field, 239
  class object, 231, 243
layout-compatible, 74, 167
layout-compatible type, 74
left shift
  undefined, 125
left shift operator, 125
lexical conventions, *see* conventions, lexical
library
  C standard, 438, 445, 447, 449, 1247, 1248, 1251
  C++ standard, 437, 460, 462, 465
library clauses, 5
lifetime, 68
limits
  implementation, 3
`<limits>`, 468
line splicing, 16
`linear_congruential_engine`
  generation algorithm, 946
  modulus, 947
  state, 946
  textual representation, 947
  transition algorithm, 946
linkage, 33, 58–61
  `const` and, 58
  external, 58, 449, 460
  implementation-defined object, 184
  `inline` and, 58
  internal, 58
  no, 58, 59
  `static` and, 58
linkage specification, *see* specification, linkage
linkage-specification, 181, 1220
literal, 23–32, 90
  base of integer, 24
  binary, 24
  boolean, 30
  `char16_t`, 25
  `char32_t`, 25
  character, 25
  constant, 23
  decimal, 24
  `double`, 27
  `float`, 27
  floating, 26, 27
  hexadecimal, 24
  `char`, 26
  integer, 23, 24
  `long`, 24
  `long double`, 27
  multicharacter, 25
    implementation-defined value of, 25
  narrow-character, 25
  octal, 24
  pointer, 30
  string, 27, 28
    `char16_t`, 28
    `char32_t`, 28, 29
    narrow, 28
    type of, 28
    undefined change to, 29
    wide, 28, 29
  type of character, 25
  type of floating point, 27
  type of integer, 24
  `unsigned`, 24
  user defined, 30
literal, 23, 1210
literal type, 73
literal-operator-id, 327, 1224
`load_factor`
  unordered associative containers, 771
local lambda expression, 96
local variable, 40
`local_iterator`, 764
  unordered associative containers, 764
locale, 1092, 1093, 1095, 1103
locale-specific behavior, *see* behavior, locale-specific
local class

friend, 263
member function in, 233
scope of, 241
local scope, *see* block scope
local variable
destruction of, 141, 143
lock-free executions, 11
logical-and-expression, 128, 1216
logical-or-expression, 128, 1216
`lognormal_distribution`
probability density function, 969
`long`
`typedef` and, 148
long-long-suffix, 24, 1210
long-suffix, 24, 1210
lookup
argument-dependent, 48
class member, 57
class member, 51
elaborated type specifier, 56
member name, 245
name, 33, 44–58
namespace aliases and, 58
namespace member, 52
qualified name, 50–55
template name, 363
unqualified name, 44
using-directives and, 58
`lookup_classname`
regular expression traits, 1138
`lookup_classname`
regular expression traits, 1094
`lookup_collatename`
regular expression traits, 1094
low-order bit, 6
lowercase, 445
lparen, 426, 1226
lvalue, 78, 1233
lvalue reference, 75, 195

macro
argument substitution, 430
function-like, 429
arguments, 430
masking, 463
name, 429
object-like, 429
pragma operator, 436
predefined, 435
replacement, 429–434
replacement list, 429
rescanning and replacement, 431
scope of definition, 431
`main()`, 61
implementation-defined linkage of, 61
implementation-defined parameters to, 61
parameters to, 61
return from, 61, 63
`match_results`
as sequence, 1120
matched, 1092
`max`
random number distribution requirement, 942
uniform random number generator requirement, 937
`max_bucket_count`
unordered associative containers, 770
`max_load_factor`
unordered associative containers, 771
mean
`normal_distribution`, 968
`poisson_distribution`, 964
mem-initializer, 280, 1224
mem-initializer-id, 280, 1224
mem-initializer-list, 280, 1224
member
`class static`, 65
enumerator, 168
static, 235
template and `static`, 346
member access operator
overloaded, 326
member function
call undefined, 233
class, 232
`const`, 234
constructor and, 268
destructor and, 275
friend, 262
inline, 232
local class, 241
nested class, 265
nonstatic, 233
overload resolution and, 303
static, 235, 236
`this`, 234
`union`, 237
`volatile`, 234
member names, 41
member subobject, 7
member-declaration, 230, 1223
member-declarator, 230, 1223

member-declarator-list, 230, 1223
member-specification, 229, 1223
members, 41
member data
  static, 236
member pointer to, *see* pointer to member
memory location, 6
memory model, 6
memory management, *see also* `new`, `delete`
`mersenne_twister_engine`
  generation algorithm, 947
  state, 947
  textual representation, 948
  transition algorithm, 947
message
  diagnostic, 2, 5
`min`
  random number distribution requirement, 942
  uniform random number generator requirement, 937
modification order, 12
more specialized
  function template, 403
most derived class, 7
most derived object, 7
  bit-field, 7
  zero size subobject, 7
move
  class object, *see* constructor, move; assignment, move
`MoveInsertable` into `X`, 748
multi-pass guarantee, 854
multibyte character, *see* character, multibyte
multicharacter literal, *see* literal, multicharacter
multiple threads, *see* threads, multiple
multiple inheritance, 242, 243
  `virtual` and, 251
multiplicative-expression, 123, 1215
`mutable`, 148
mutable iterator, 850
mutex types, 1165

name, 21, 33, 91
  address of cv-qualified, 113
  dependent, 369, 376
  elaborated
    `enum`, 161
  global, 41
  length of, 21
  macro, *see* macro, name
  point of declaration, 39
  predefined macro, *see* macro, predefined
  qualified, 50
  reserved, 459
  scope of, 38
  unqualified, 44
name hiding
  function, 302
  overloading versus, 302
  using-declaration and, 177
named-namespace-definition, 168, 1220
namespace, 447, 1251
  alias, 172
  definition, 168
  extend, 169
  global, 22
  member definition, 171
  unnamed, 170
namespace-alias, 172, 1220
namespace-alias-definition, 172, 1220
namespace-body, 169, 1220
namespace-definition, 168, 1220
namespace-name, 168, 1220
namespaces, 168–181
name class, *see* class name
name hiding, 39, 43, 44, 92, 143
  class definition, 228
  user-defined conversion and, 271
name space
  label, 136
narrowing conversion, 223
`NDEBUG`, 449
`negative_binomial_distribution`
  discrete probability function, 963
nested-name-specifier, 92, 1213
nested-namespace-definition, 168, 1220
nested class
  local class, 241
  scope of, 239
`<new>`, 480
`new`, 65, 114, 116
  array of class objects and, 118
  constructor and, 118
  default constructor and, 118
  exception and, 118
  initialization and, 118
  `operator`, 460, 461
  scoping and, 115
  storage allocation, 115
  type of, 277
  unspecified constructor and, 118
  unspecified order of evaluation, 118

new-declarator, 115, 1215
new-expression, 115, 1214
new-initializer, 115, 1215
new-line, 426, 1227
new-placement, 115, 1214
new-type-id, 115, 1214
`new_handler`, 66
no linkage, 58
noexcept-expression, 121, 1215
noexcept-specification, 418, 1226
non-directive, 426, 1226
non-throwing exception specification, 418
nondigit, 21, 1209
nonzero-digit, 23, 1210
noptr-abstract-declarator, 191, 1222
noptr-abstract-pack-declarator, 191, 1222
noptr-declarator, 190, 1221
noptr-new-declarator, 115, 1215
normal distributions, 968–973
`normal_distribution`
  mean, 968
  probability density function, 968
  standard deviation, 968
normative references, *see* references, normative
notation
  syntax, 5–6
NTBS, 446, 1080, 1259, 1260
  static, 446
NTCTS, 440
NTMBS, 446
  static, 446
number
  hex, 26
  octal, 26
`numeric_limits`, 468
  specializations for arithmetic types, 75

object, *see also* object model, 7, 33
  byte copying and, 72
  complete, 7
  definition, 35
  delete, 120
  destructor static, 63
  destructor and placement of, 276
  linkage specification, 184
  `local static`, 65
  undefined deleted, 67
  unnamed, 268
object expression, 104
object model, 7
object pointer type, 76
object representation, 72
object type, 7, 73
  incompletely-defined, 72
object, exception, *see* exception handling, exception object
object-like macro, *see* macro, object-like
object class, *see* also class object
object lifetime, 68–72
object temporary, *see* temporary
object type, 73
observable behavior, *see* behavior, observable
octal-digit, 23, 1210
octal-escape-sequence, 25, 1211
octal-literal, 23, 1210
odr-used, 35
one-definition rule, 35–38
opaque-enum-declaration, 165, 1219
operation
  atomic, 11–15
operator, 22, 325
  `*=`, 130
  `+=`, 114, 130
  `-=`, 130
  `/=`, 130
  `<<=`, 130
  `>>=`, 130
  `%=`, 130
  `&=`, 130
  `^=`, 130
  `|=`, 130
  additive, 123
  address-of, 112
  assignment, 130, 446
  bitwise, 127
  bitwise AND, 127
  bitwise exclusive OR, 127
  bitwise inclusive OR, 127
  cast, 112, 191
  class member access, 104
  comma, 131
  conditional expression, 128
  copy assignment, *see* assignment, copy
  decrement, 105, 112, 114
  division, 123
  equality, 126
  function call, 102, 324
  greater than, 125
  greater than or equal to, 125
  increment, 105, 112, 114
  indirection, 112
  inequality, 126

less than, 125
less than or equal to, 125
logical AND, 128
logical negation, 112, 113
logical OR, 128
move assignment, *see* assignment, move
multiplication, 123
multiplicative, 123
one's complement, 112, 113
overloaded, 87, 324
pointer to member, 122
pragma, *see* macro, pragma operator
precedence of, 8
relational, 125
remainder, 123
scope resolution, 92, 116, 232, 242, 252
side effects and comma, 131
side effects and logical AND, 128
side effects and logical OR, 128
`sizeof`, 112, 114
subscripting, 101, 324
unary, 112
unary minus, 112, 113
unary plus, 112, 113
operator, 324, 1224
`operator delete`, *see also* `delete`, 116, 120, 277
`operator new`, *see also* `new`, 116
operator overloading, *see* overloading, operator
`operator!=`
random number distribution requirement, 942
random number engine requirement, 938
`operator()`
random number distribution requirement, 941, 942
random number engine requirement, 938
uniform random number generator requirement, 937
operator-function-id, 324, 1224
`operator<<`
random number distribution requirement, 942
random number engine requirement, 939
`operator==`
random number distribution requirement, 942
random number engine requirement, 938
`operator>>`
random number distribution requirement, 942
random number engine requirement, 939
operator , *see* `delete`
operator left shift, *see* left shift operator
operator right shift, *see* right shift operator
operator use
scope resolution, 236
optimization of temporary, *see* elimination of temporary
order of evaluation in expression, *see* expression, order of evaluation of
ordered, 62
ordering
function template partial, 360
order of execution
base class constructor, 268
base class destructor, 275
constructor and `static` objects, 279
constructor and array, 278
destructor, 275
destructor and array, 275
member constructor, 268
member destructor, 275
ordinary character literal, 25
over-aligned type, 79
overflow, 87
undefined, 87
overloaded function, *see* overloading
overloaded operator, *see* overloading, operator
overloadedfunction
address of, 322
overloaded function
address of, 113
overloaded operator
inheritance of, 325
overloading, 200, 228, 299–331, 358
access control and, 302
address of overloaded function, 322
argument lists, 303–310
assignment operator, 325
binary operator, 325
built-in operators and, 328
candidate functions, 303–310
declaration matching, 301
declarations, 299
example of, 299
function call operator, 326
member access operator, 326
operator, 324–328
prohibited, 299
resolution, 302–322
best viable function, 311–325
contexts, 303
function call syntax, 305–306
function template, 410
implicit conversions and, 313–322
initialization, 309, 310

operators, 306
scoping ambiguity, 246
template, 360
template name, 363
viable functions, 311–325
subscripting operator, 326
unary operator, 325
user-defined literal, 327
using directive and, 180
using-declaration and, 177
overloads
floating point, 933
overrider
final, 249
own, 554

`pair`
tuple interface to, 517
`param`
random number distribution requirement, 941
seed sequence requirement, 936
`param_type`
random number distribution requirement, 941
parameter, 3
catch clause, 3
function, 3
function-like macro, 3
reference, 195
scope of, 40
template, 3, 34
`void`, 200
parameter declaration, 34
parameter-declaration, 200, 1222
parameter-declaration-clause, 199, 1222
parameter-declaration-list, 199, 1222
parameterized type, *see* template
parameters
macro, 429
parameters-and-qualifiers, 190, 1221
parameter list
variable, 103, 200
period, 445
phases of translation, *see* translation, phases
piecewise construction, 518
`piecewise_constant_distribution`
interval boundaries, 975
probability density function, 974
weights, 975
`piecewise_linear_distribution`
interval boundaries, 976
probability density function, 976
weights at boundaries, 976
placement *new-expression*, 118
placement syntax
`new`, 118
pm-expression, 122, 1215
POD class, 227
POD struct, 227
POD union, 227
POF, 494
point of declaration, 39
pointer, *see also* `void*`
safely-derived, 67–68
to traceable object, 466
to traceable object, 67
zero, 85
pointer literal, *see* literal, pointer
pointer, integer representation of safely-derived, 68
pointer-literal, 30, 1212
pointer to member, 76, 122
Poisson distributions, 963–968
`poisson_distribution`
discrete probability function, 963
mean, 964
POSIX, 1
extended regular expressions, 1103
regular expressions, 1103
postfix-expression, 101, 1214
postfix `++` and `--`
overloading, 326
postfix `++`, 105
postfix `--`, 105
potential exception, 420
potential results, 35
potential scope, 38
potentially evaluated, 35
potentially concurrent, 14
pp-number, 21, 1209
pp-tokens, 426, 1227
precedence of operator, *see* operator, precedence of
prefix
`L`, 25, 29
prefix `++` and `--`
overloading, 326
prefix `++`, 114
prefix `--`, 114
preprocessing directive, 425
conditional inclusion, 426
preprocessing directives, 425–436
error, 434

header inclusion, 428
line control, 434
macro replacement, *see* macro, replacement
null, 435
pragma, 434
source-file inclusion, 428
preprocessing-file, 425, 1226
preprocessing-op-or-punc, 22, 1209
preprocessing-token, 18, 1208
primary equivalence class, 1092
primary-expression, 90, 1212
`private`, *see* access control, `private`
probability density function
`cauchy_distribution`, 970
`chi_squared_distribution`, 970
`exponential_distribution`, 964
`extreme_value_distribution`, 967
`fisher_f_distribution`, 971
`gamma_distribution`, 965
`lognormal_distribution`, 969
`normal_distribution`, 968
`piecewise_constant_distribution`, 974
`piecewise_linear_distribution`, 976
`student_t_distribution`, 972
`uniform_real_distribution`, 959
`weibull_distribution`, 966
program, 58
ill-formed, 2
start, 61–63
termination, 63–64
well-formed, 4, 8
program execution, 7–11
abstract machine, 7
as-if rule, *see* as-if rule
promotion
bool to int, 84
floating point, 84
integral, 83
`protected`, *see* access control, `protected`
protection, *see* access control, 465
prvalue, 78
pseudo-destructor-name, 104
pseudo-destructor-name, 101, 1214
ptr-abstract-declarator, 191, 1222
ptr-declarator, 190, 1221
ptr-operator, 191, 1221
`ptrdiff_t`, 124
implementation defined type of, 124
`public`, *see* access control, `public`
punctuator, 22
pure-specifier, 230, 1223

q-char, 20, 1209
q-char-sequence, 20, 1209
qualification
explicit, 50
qualified-id, 92, 1213
qualified-namespace-specifier, 172, 1220

r-char, 27, 1212
r-char-sequence, 27, 1212
random number distribution
`bernoulli_distribution`, 960
`binomial_distribution`, 961
`chi_squared_distribution`, 970
`discrete_distribution`, 973
`exponential_distribution`, 964
`extreme_value_distribution`, 967
`fisher_f_distribution`, 971
`gamma_distribution`, 965
`geometric_distribution`, 962
`lognormal_distribution`, 969
`negative_binomial_distribution`, 963
`normal_distribution`, 968
`piecewise_constant_distribution`, 974
`piecewise_linear_distribution`, 976
`poisson_distribution`, 963
requirements, 940–943
`student_t_distribution`, 972
`uniform_int_distribution`, 958
`uniform_real_distribution`, 959
random number distributions
Bernoulli, 960–963
normal, 968–973
Poisson, 963–968
sampling, 973–978
uniform, 958–960
random number engine
`linear_congruential_engine`, 946
`mersenne_twister_engine`, 947
requirements, 937–939
`subtract_with_carry_engine`, 949
with predefined parameters, 953–954
random number engine adaptor
`discard_block_engine`, 950
`independent_bits_engine`, 951
`shuffle_order_engine`, 952
with predefined parameters, 953–954
random number generation, 934–978
distributions, 958–978
engines, 945–953
predefined engines and adaptors, 953–954
requirements, 935–943

synopsis, 943–945
utilities, 956–958
random number generator, *see* uniform random number generator
`random_device`
implementation leeway, 955
raw string literal, 28
raw-string, 27, 1211
reaching scope, 96
ready, 1120, 1195
redefinition
`typedef`, 151
ref-qualifier, 191, 1221
reference, 75
assignment to, 130
call by, 103
forwarding, 399
lvalue, 75
null, 196
rvalue, 75
`sizeof`, 114
reference collapsing, 196
reference-compatible, 217
reference-related, 217
references
normative, 1
`regex_iterator`
end-of-sequence, 1131
`regex_token_iterator`
end-of-sequence, 1133
`regex_traits`
specializations, 1106
region
declarative, 33, 38
`register`, 148
regular expression, 1092–1138
grammar, 1136
matched, 1092
requirements, 1093
regular expression traits, 1136
`char_class_type`, 1093
`isctype`, 1094
`lookup_classname`, 1138
`lookup_classname`, 1094
`lookup_collatename`, 1094
requirements, 1093, 1106
`transform`, 1138
`transform`, 1094
`transform_primary`, 1138
`transform_primary`, 1094
`translate`, 1138
`translate`, 1093
`translate_nocase`, 1138
`translate_nocase`, 1094
`rehash`
unordered associative containers, 771
`reinterpret_cast`, *see* cast, reinterpret
relational-expression, 125, 1215
relaxed pointer safety, 68
release sequence, 12
remainder operator, *see* remainder operator
replacement
macro, *see* macro, replacement
replacement-list, 426, 1227
representation
object, 72
value, 72
requirements, 442
`Allocator`, 453
container, 743, 763, 775, 776, 1120
not required for unordered associated containers, 762
`CopyAssignable`, 450
`CopyConstructible`, 450
`DefaultConstructible`, 450
`Destructible`, 450
`EqualityComparable`, 450
`Hash`, 453
iterator, 850
`LessThanComparable`, 450
`MoveAssignable`, 450
`MoveConstructible`, 450
`NullablePointer`, 453
numeric type, 922
random number distribution, 940–943
random number engine, 937–939
regular expression traits, 1093, 1106
seed sequence, 935–936
sequence, 1120
uniform random number generator, 937
unordered associative container, 763
reraise, *see* exception handling, rethrow
rescanning and replacement, *see* macro, rescanning and replacement
reserved identifier, 21
reset, 554
`reset`
random number distribution requirement, 941
resolution, *see* overloading, resolution
restriction, 462, 463, 465
address of bit-field, 239
anonymous `union`, 238

bit-field, 239
constructor, 267, 268
destructor, 274
`extern`, 149
local class, 241
operator overloading, 324
overloading, 324
pointer to bit-field, 239
reference, 196
`register`, 149
`static`, 149
`static` member local class, 237
`union`, 237
`result_type`
entity characterization based on, 934
`result_type`
random number distribution requirement, 941
seed sequence requirement, 936
uniform random number generator requirement, 937
rethrow, *see* exception handling, rethrow
`return`, 141, 142
and handler, 413
and try block, 413
constructor and, 142
reference and, 217
`return` statement, *see* `return`
return type, 201
overloading and, 299
right shift
implementation defined, 125
right shift operator, 125
rounding, 85
rvalue, 78
lvalue conversion to, *see* conversion, lvalue to rvalue
lvalue conversion to, 1233
rvalue reference, 75, 195

s-char, 27, 1211
s-char-sequence, 27, 1211
safely-derived pointer, 67
integer representation, 68
sampling distributions, 973–978
scalar type, 73
scope, 1, 33, 38–44, 147
anonymous `union` at namespace, 238
block, 40
class, 41
declarations and, 38–40
destructor and exit from, 141
enumeration, 42
exception declaration, 40
function, 41
function prototype, 40
global, 41
global namespace, 41
iteration-statement, 139
macro definition, *see* macro, scope of definition
namespace, 41
name lookup and, 44–58
overloading and, 301
potential, 38
selection-statement, 137
template parameter, 43
scope name hiding and, 43
scope resolution operator, 50
`seed`
random number engine requirement, 938
seed sequence, 935
requirements, 935–936
selection-statement, 137, 1216
semantics
class member, 104
separate compilation, *see* compilation, separate
separate translation, *see* compilation, separate
sequence
ambiguous conversion, 314
implicit conversion, 313
standard conversion, 81
sequence constructor
seed sequence requirement, 936
Sequenced before, 10
sequencing operator, *see* comma operator
set of potential exceptions of a function, function pointer, or member function pointer, 420
set of potential exceptions of an expression, 420
set of potential exceptions of the implicitly-declared member function, 421
`setlocale`, 445
shared lock, 1170
shared mutex types, 1170
shared state, *see* future, shared state
shared timed mutex type, 1172
shift-expression, 125, 1215
shift operator, *see* left shift operator, right shift operator
`short`
`typedef` and, 148
`shuffle_order_engine`
generation algorithm, 952

state, 952
textual representation, 953
transition algorithm, 952
side effects, 7, 9, 10, 12–14, 128, 136, 270, 282, 293, 432, 465
visible, 13
sign, 27, 1211
signal, 8
signature, 3, 4
`signed`
`typedef` and, 148
signed integer type, 74
similar types, 83
simple call wrapper, 584
simple-capture, 93, 1213
simple-declaration, 146, 1217
simple-escape-sequence, 25, 1211
simple-template-id, 336, 1225
simple-type-specifier, 158, 1219
`size`
seed sequence requirement, 936
`size_t`, 114
smart pointers, 565–580
source file, 16, 448, 460
source file character, *see* character, source file
space
white, 18
specialization
class template, 338
class template partial, 354
template, 378
template explicit, 385
special member function, *see* constructor, destructor, inline function, user-defined conversion, virtual function
specification
linkage, 181–184
`extern`, 181
implementation-defined, 181
nesting, 182
template argument, 391
specifications
C standard library exception, 465
C++, 466
implementation-defined exception, 466
specifier, 148–163
`friend`, 465
`constexpr`, 153
constructor, 153, 154
function, 153
cv-qualifier, 157
declaration, 148
`explicit`, 151
`friend`, 153
function, 150
`inline`, 150
`static`, 149
storage class, 148
type, *see* type specifier
`typedef`, 151
`virtual`, 151
specifier access, *see* access specifier
stable algorithm, 440, 464
stack unwinding, 416
see exception handling, constructors and destructors, 416
standard
structure of, 5
standard deviation
`normal_distribution`, 968
standard-layout types, 73
standard-layout class, 226
standard-layout struct, 227
standard-layout union, 227
standard integer type, 74
standard signed integer type, 74
standard unsigned integer type, 74
start
program, 62
startup
program, 449, 461
state
`discard_block_engine`, 950
`independent_bits_engine`, 951
`linear_congruential_engine`, 946
`mersenne_twister_engine`, 947
object, 439
`shuffle_order_engine`, 952
`subtract_with_carry_engine`, 949
statement, 136–145
`continue` in `for`, 140
`break`, 141, 142
compound, 136
`continue`, 141, 142
declaration, 142
declaration in `if`, 137
declaration in `switch`, 137
declaration in `for`, 140
declaration in `switch`, 138
declaration in `while`, 139
`do`, 138, 140
empty, 136

expression, 136
`for`, 138, 140
`goto`, 136, 141, 142
`if`, 137, 138
iteration, 138–141
jump, 141
labeled, 136
null, 136
`for`, 140
selection, 137–138
`switch`, 137, 138, 142
`while`, 138, 139
statement, 136, 1216
statement-seq, 136, 1216
`static`, 148
destruction of local, 143
linkage of, 58, 149
overloading and, 299
static initialization, 62
static storage duration, 65
static type, *see* type, static
static_assert, 147
static_assert-declaration, 146, 1217
`static_cast`, *see* cast, static
`<stddef.h>`, 25, 29
`<stdexcept>`, 496
storage-class-specifier, 148, 1218
storage class, 33
storage duration, 64–68
automatic, 64, 65
class member, 68
dynamic, 64–68, 115
local object, 65
`register`, 65
static, 64, 65
thread, 64, 65
storage management, *see* `new`, `delete`
stream
arbitrary-positional, 438
repositional, 440
`streambuf`
implementation-defined, 1008
strict pointer safety, 68
string
distinct, 29
null-terminated byte, 446
null-terminated character type, 440
null-terminated multibyte, 446
`sizeof`, 29
type of, 28
string literal, *see* literal, string
string-literal, 27, 1211
stringize, *see* `#`
struct
standard-layout, 227
`struct`
`class` versus, 226
structure, 226
structure tag, *see* class name
`student_t_distribution`
probability density function, 972
sub-expression, 1093
subobject, *see also* object model, 7
subscripting operator
overloaded, 326
subsequence rule
overloading, 320
`subtract_with_carry_engine`
carry, 949
generation algorithm, 949
state, 949
textual representation, 949
transition algorithm, 949
subtraction
implementation defined pointer, 124
subtraction operator, 123
suffix
`E`, 27
`e`, 27
`F`, 27
`f`, 27
`L`, 24, 27
`l`, 24, 27
`U`, 24
`u`, 24
summary
x C++ 2003, 1239
x C++ 2011, 1246
x C++ 2014, 1247
compatibility with ISO C, 1230
swappable, 451
swappable with, 451
`switch`
and handler, 413
and try block, 413
synchronize with, 12
synonym, 172
type name as, 151
syntax
class member, 104

target object, 583

template, 332–412
  definition of, 332
  function, 391
  member function, 345
  primary, 354
  static data member, 332
  variable, 332
`template`, 332
template parameter, 34
template-argument, 337, 1225
template-argument-list, 337, 1225
template-declaration, 332, 1224
template-id, 336, 1225
template-name, 336, 1225
template-parameter, 333, 1225
template-parameter-list, 332, 1224
template name
  linkage of, 333
template parameter scope, 43
temporary, 269
  constructor for, 269
  destruction of, 269
  destructor for, 269
  elimination of, 269, 293
  implementation-defined generation of, 269
  order of destruction of, 270
`terminate()`, 423, 424
  called, 130, 415, 420, 423
termination
  program, 61, 64
terminology
  pointer, 76
text-line, 426, 1226
textual representation
  `discard_block_engine`, 951
  `independent_bits_engine`, 952
  `shuffle_order_engine`, 953
  `subtract_with_carry_engine`, 949
`this`, 90, 234
  type of, 234
`this` pointer, *see* `this`
thread, 11
thread of execution, 11
thread storage duration, 65
thread, blocked, 438
`thread_local`, 148
threads
  multiple, 11–15
`throw`, 129
throw-expression, 129, 1216
throwing, *see* exception handling, throwing
timed mutex types, 1168
token, 20
  alternative, 19
  preprocessing, 18–19, 1208
token, 20, 1209
traceable pointer object, 67, 466
trailing-return-type, 191, 1221
trailing-type-specifier, 156, 1218
trailing-type-specifier-seq, 156, 1218
traits, 440
transfer ownership, 554
`transform`
  regular expression traits, 1138
`transform`
  regular expression traits, 1094
`transform_primaryl`
  regular expression traits, 1138
`transform_primary`
  regular expression traits, 1138
`transform_primaryl`
  regular expression traits, 1094
TransformationTrait, 601
transition algorithm
  `discard_block_engine`, 950
  `independent_bits_engine`, 951
  `linear_congruential_engine`, 946
  `mersenne_twister_engine`, 947
  `shuffle_order_engine`, 952
  `subtract_with_carry_engine`, 949
`translate`
  regular expression traits, 1138
`translate`
  regular expression traits, 1093
`translate_nocase`
  regular expression traits, 1138
`translate_nocase`
  regular expression traits, 1094
translation
  phases, 16–17
  separate, *see* compilation, separate
translation unit, 16
translation units, 58
translation-unit, 58, 1212
translation unit, 58
  name and, 33
trigraph sequence, 1247
trivial types, 73
trivially copyable class, 226
trivially copyable types, 73
trivial class, 226
trivial class type, 118

trivial type, 118
truncation, 85
`try`, 413
try block, *see* exception handling, try block
try-block, 413, 1225
`tuple`
  and `pair`, 517
type, 33, 72–77
  arithmetic, 75
  array, 75, 200
  bitmask, 444, 445
  Boolean, 74
  `char`, 74
  `char16_t`, 75
  `char32_t`, 75
  character, 74
  character container, 438
  class and, 225
  compound, 75
  `const`, 156
  destination, 212
  `double`, 75
  dynamic, 2
  enumerated, 76, 444
  example of incomplete, 73
  extended integer, 74
  extended signed integer, 74
  extended unsigned integer, 74
  `float`, 75
  floating point, 74
  function, 75, 199, 200
  fundamental, 74
  `sizeof`, 74
  incomplete, 35, 36, 39, 72, 82, 101, 102, 104, 106, 107, 112, 114, 120, 130, 242
  incompletely-defined object, 72
  `int`, 74
  integral, 74
  `long`, 74
  `long double`, 75
  `long long`, 74
  narrow character, 74
  over-aligned, 79
  POD, 73
  pointer, 75
  polymorphic, 248
  referenceable, 440
  `short`, 74
  `signed char`, 74
  signed integer, 74
  similar, *see* similar types
  standard integer, 74
  standard signed integer, 74
  standard unsigned integer, 74
  static, 4
  trivially copyable, 72
  underlying `wchar_t`, 75
  `unsigned`, 74
  `unsigned char`, 74
  `unsigned int`, 74
  `unsigned long`, 74
  `unsigned long long`, 74
  `unsigned short`, 74
  unsigned integer, 74
  `void`, 75
  `volatile`, 156
  `wchar_t`, 75
type generator, *see* template
type specifier
  `auto`, 161
  `const`, 157
  elaborated, 161
  simple, 158
  `volatile`, 157
type-id, 191, 1222
type-id-list, 418, 1226
type-name, 158, 1219
type-parameter, 333, 1225
type-parameter-key, 333, 1225
type-specifier
  `bool`, 158
  `wchar_t`, 158
type-specifier, 156, 1218
type-specifier-seq, 156, 1218
`type_info`, 107
typedef
  function, 201
`typedef`
  overloading and, 300
typedef-name, 151, 1218
`typeid`, 107
  construction and, 286
  destruction and, 286
`<typeinfo>`, 486
`typename`, 161
typename-specifier, 364, 1225
types
  implementation-defined, 444
  implementation-defined exception, 466
type checking
  argument, 103
type conversion, explicit, *see* casting

type name, 191
nested, 241
scope of, 241
type pun, 111
type specifier
`auto`, 158
`char`, 158
`char16_t`, 158
`char32_t`, 158
`decltype`, 158, 160
`double`, 158
elaborated, 56
`enum`, 161
`float`, 158
`int`, 158
`long`, 158
`short`, 158
`signed`, 158
`unsigned`, 158
`void`, 158
`volatile`, 158

ud-suffix, 30, 1212
unary fold, 100
unary function, 584
unary left fold, 100
unary operator
overloaded, 325
unary right fold, 100
unary-expression, 112, 1214
unary-operator, 112, 1214
UnaryTypeTrait, 601
unary operator
interpretation of, 325
unblock, 440
undefined, 440, 459, 460, 462, 983, 984, 987–990, 994, 998, 1022
undefined behavior, *see* behavior, undefined
underlying type, 75
unevaluated operand, 88
`unexpected()`, 424
called, 420
Unicode required set, 436
uniform distributions, 958–960
uniform random number generator
requirements, 937
`uniform_int_distribution`
discrete probability function, 958
`uniform_real_distribution`
probability density function, 959
union
standard-layout, 227
`union`, 76, 237
`class` versus, 226
anonymous, 238
global anonymous, 238
unique pointer, 554
unit
translation, 448, 449, 460
universal character name, 16
universal-character-name, 18, 1208
unnamed-namespace-definition, 168, 1220
unordered, 62
unordered associative containers, 762–841
`begin`, 770
`bucket`, 770
`bucket_count`, 770
`bucket_size`, 770
`cbegin`, 771
`cend`, 771
`clear`, 770
complexity, 762
`const_local_iterator`, 764
`count`, 770
`end`, 771
`equal_range`, 770
equality function, 763
equivalent keys, 763, 829, 837
`erase`, 769, 770
exception safety, 772
`find`, 770
hash function, 763
`hash_function`, 767
`hasher`, 764
`insert`, 768, 769
iterator invalidation, 772
iterators, 772
`key_eq`, 767
`key_equal`, 764
`key_type`, 764
lack of comparison operators, 762
`load_factor`, 771
`local_iterator`, 764
`max_bucket_count`, 770
`max_load_factor`, 771
`rehash`, 771
requirements, 762, 763, 772
unique keys, 763, 823, 833
`unordered_map`
element access, 827, 828
unique keys, 823
`unordered_multimap`

equivalent keys, 829
`unordered_multiset`
equivalent keys, 837
`unordered_set`
unique keys, 833
unqualified-id, 90, 1213
unsequenced, 10
`unsigned`
`typedef` and, 148
unsigned-suffix, 23, 1210
unsigned integer type, 74
unspecified, 480, 482, 486, 909, 1070, 1255, 1257
unspecified behavior, *see* behavior, unspecified, 988
unwinding
stack, 416
uppercase, 22, 445
user-defined literal, *see* literal, user defined
overloaded, 327
user-defined-character-literal, 30, 1212
user-defined-floating-literal, 30, 1212
user-defined-integer-literal, 30, 1212
user-defined-literal, 30, 1212
user-defined-string-literal, 30, 1212
user-provided, 208
Uses-allocator construction, 546
using-declaration, 173–178
using-declaration, 173, 1220
using-directive, 178–181
using-directive, 178, 1220
usual arithmetic conversions, *see* conversion, usual arithmetic
UTF-8 character literal, 25

valid, 38
valid but unspecified state, 441
value, 72
call by, 103
indeterminate, 211
null member pointer, 85
null pointer, 85
undefined unrepresentable integral, 85
value category, 78
value computation, 9–11, 14, 105, 118, 128, 130, 131, 270
value representation, 72
value-initialization, 210
variable, 33
indeterminate uninitialized, 210
variable template
definition of, 332
virt-specifier, 230, 1223
virt-specifier-seq, 230, 1223
virtual base class, 244
virtual function, 248–253
pure, 253
virtual function call, 252
constructor and, 286
destructor and, 286
undefined pure, 254
visibility, 44
visible, 44
`void*`
type, 76
`void&`, 195
`volatile`, 76
constructor and, 235, 267
destructor and, 235, 274
implementation-defined, 158
overloading and, 301

waiting function, 1194
`wchar_t`, 25, 29, 684
implementation-defined, 75
weak result type, 583
`weibull_distribution`
probability density function, 966
weights
`discrete_distribution`, 973
`piecewise_constant_distribution`, 975
weights at boundaries
`piecewise_linear_distribution`, 976
well-formed program, *see* program, well-formed
white space, 20
wide-character, 25

`X(X&)`, *see* copy constructor
xvalue, 78

zero
division by undefined, 87
remainder undefined, 87
undefined division by, 123
zero-initialization, 210

Index of grammar productions

The first page number for each entry is the page in the general text where the grammar production is defined. The second page number is the corresponding page in the Grammar summary (Annex A).

abstract-declarator, 191, 1222
abstract-pack-declarator, 191, 1222
access-specifier, 242, 1224
additive-expression, 124, 1215
alias-declaration, 146, 1217
alignment-specifier, 184, 1220
and-expression, 127, 1215
asm-definition, 181, 1220
assignment-expression, 130, 1216
assignment-operator, 130, 1216
attribute, 184, 1220
attribute-argument-clause, 184, 1221
attribute-declaration, 146, 1218
attribute-list, 184, 1220
attribute-namespace, 184, 1221
attribute-scoped-token, 184, 1221
attribute-specifier, 184, 1220
attribute-specifier-seq, 184, 1220
attribute-token, 184, 1221

balanced-token, 185, 1221
balanced-token-seq, 185, 1221
base-clause, 242, 1223
base-specifier, 242, 1224
base-specifier-list, 242, 1223
base-type-specifier, 242, 1224
binary-digit, 23, 1210
binary-literal, 23, 1210
block-declaration, 146, 1217
boolean-literal, 30, 1212
brace-or-equal-initializer, 209, 1222
braced-init-list, 209, 1223

c-char, 25, 1211
c-char-sequence, 25, 1211
capture, 93, 1213
capture-default, 93, 1213
capture-list, 93, 1213
cast-expression, 121, 1215
character-literal, 25, 1210
class-head, 225, 1223
class-head-name, 225, 1223
class-key, 225, 1223
class-name, 225, 1223
class-or-decltype, 242, 1224
class-specifier, 225, 1223
class-virt-specifier, 225, 1223
compound-statement, 136, 1216
condition, 137, 1216
conditional-expression, 128, 1216
constant-expression, 131, 1216
control-line, 425, 1226
conversion-declarator, 272, 1224
conversion-function-id, 272, 1224
conversion-type-id, 272, 1224
ctor-initializer, 280, 1224
cv-qualifier, 191, 1221
cv-qualifier-seq, 191, 1221

d-char, 28, 1212
d-char-sequence, 27, 1212
decimal-literal, 23, 1210
decl-specifier, 148, 1218
decl-specifier-seq, 148, 1218
declaration, 146, 1217
declaration-seq, 146, 1217
declaration-statement, 142, 1217
declarator, 190, 1221
declarator-id, 191, 1222
decltype-specifier, 158, 1219
delete-expression, 119, 1215
digit, 21, 1209
digit-sequence, 27, 1211
dynamic-exception-specification, 418, 1225

elaborated-type-specifier, 161, 1219
elif-group, 425, 1226
elif-groups, 425, 1226
else-group, 425, 1226
empty-declaration, 146, 1218
enclosing-namespace-specifier, 168, 1220
encoding-prefix, 25, 1210
endif-line, 425, 1226
enum-base, 165, 1219
enum-head, 165, 1219
enum-key, 165, 1219

enum-name, 165, 1219
enum-specifier, 165, 1219
enumerator, 165, 1220
enumerator-definition, 165, 1219
enumerator-list, 165, 1219
equality-expression, 126, 1215
escape-sequence, 25, 1211
exception-declaration, 413, 1225
exception-specification, 418, 1225
exclusive-or-expression, 127, 1215
explicit-instantiation, 383, 1225
explicit-specialization, 385, 1225
exponent-part, 27, 1211
expression, 131, 1216
expression-list, 101, 1214
expression-statement, 136, 1216

floating-literal, 26, 1211
floating-suffix, 27, 1211
fold-expression, 100, 1213
fold-operator, 100, 1214
for-init-statement, 138, 1217
for-range-declaration, 139, 1217
for-range-initializer, 139, 1217
fractional-constant, 27, 1211
function-definition, 206, 1222
function-specifier, 150, 1218
function-try-block, 413, 1225

group, 425, 1226
group-part, 425, 1226

h-char, 20, 1209
h-char-sequence, 20, 1209
handler, 413, 1225
handler-seq, 413, 1225
header-name, 20, 1209
hex-quad, 17, 1208
hexadecimal-digit, 23, 1210
hexadecimal-escape-sequence, 25, 1211
hexadecimal-literal, 23, 1210

id-expression, 90, 1213
identifier, 21, 1209
identifier-list, 426, 1227
identifier-nondigit, 21, 1209
if-group, 425, 1226
if-section, 425, 1226
inclusive-or-expression, 127, 1215
init-capture, 93, 1213
init-declarator, 190, 1221
init-declarator-list, 190, 1221
initializer, 209, 1222
initializer-clause, 209, 1222
initializer-list, 209, 1222
integer-literal, 23, 1210
integer-suffix, 23, 1210
iteration-statement, 138, 1217

jump-statement, 141, 1217

labeled-statement, 136, 1216
lambda-capture, 93, 1213
lambda-declarator, 93, 1213
lambda-expression, 93, 1213
lambda-introducer, 93, 1213
linkage-specification, 181, 1220
literal, 23, 1210
literal-operator-id, 327, 1224
logical-and-expression, 128, 1216
logical-or-expression, 128, 1216
long-long-suffix, 24, 1210
long-suffix, 24, 1210
lparen, 426, 1226

mem-initializer, 280, 1224
mem-initializer-id, 280, 1224
mem-initializer-list, 280, 1224
member-declaration, 230, 1223
member-declarator, 230, 1223
member-declarator-list, 230, 1223
member-specification, 229, 1223
multiplicative-expression, 123, 1215

named-namespace-definition, 168, 1220
namespace-alias, 172, 1220
namespace-alias-definition, 172, 1220
namespace-body, 169, 1220
namespace-definition, 168, 1220
namespace-name, 168, 1220
nested-name-specifier, 92, 1213
nested-namespace-definition, 168, 1220
new-declarator, 115, 1215
new-expression, 115, 1214
new-initializer, 115, 1215
new-line, 426, 1227
new-placement, 115, 1214
new-type-id, 115, 1214
noexcept-expression, 121, 1215
noexcept-specification, 418, 1226
non-directive, 426, 1226
nondigit, 21, 1209

nonzero-digit, 23, 1210
noptr-abstract-declarator, 191, 1222
noptr-abstract-pack-declarator, 191, 1222
noptr-declarator, 190, 1221
noptr-new-declarator, 115, 1215

octal-digit, 23, 1210
octal-escape-sequence, 25, 1211
octal-literal, 23, 1210
opaque-enum-declaration, 165, 1219
operator, 324, 1224
operator-function-id, 324, 1224

parameter-declaration, 200, 1222
parameter-declaration-clause, 199, 1222
parameter-declaration-list, 199, 1222
parameters-and-qualifiers, 190, 1221
pm-expression, 122, 1215
pointer-literal, 30, 1212
postfix-expression, 101, 1214
pp-number, 21, 1209
pp-tokens, 426, 1227
preprocessing-file, 425, 1226
preprocessing-op-or-punc, 22, 1209
preprocessing-token, 18, 1208
primary-expression, 90, 1212
pseudo-destructor-name, 101, 1214
ptr-abstract-declarator, 191, 1222
ptr-declarator, 190, 1221
ptr-operator, 191, 1221
pure-specifier, 230, 1223

q-char, 20, 1209
q-char-sequence, 20, 1209
qualified-id, 92, 1213
qualified-namespace-specifier, 172, 1220

r-char, 27, 1212
r-char-sequence, 27, 1212
raw-string, 27, 1211
ref-qualifier, 191, 1221
relational-expression, 125, 1215
replacement-list, 426, 1227

s-char, 27, 1211
s-char-sequence, 27, 1211
selection-statement, 137, 1216
shift-expression, 125, 1215
sign, 27, 1211
simple-capture, 93, 1213
simple-declaration, 146, 1217
simple-escape-sequence, 25, 1211
simple-template-id, 336, 1225
simple-type-specifier, 158, 1219
statement, 136, 1216
statement-seq, 136, 1216
static_assert-declaration, 146, 1217
storage-class-specifier, 148, 1218
string-literal, 27, 1211

template-argument, 337, 1225
template-argument-list, 337, 1225
template-declaration, 332, 1224
template-id, 336, 1225
template-name, 336, 1225
template-parameter, 333, 1225
template-parameter-list, 332, 1224
text-line, 426, 1226
throw-expression, 129, 1216
token, 20, 1209
trailing-return-type, 191, 1221
trailing-type-specifier, 156, 1218
trailing-type-specifier-seq, 156, 1218
translation-unit, 58, 1212
try-block, 413, 1225
type-id, 191, 1222
type-id-list, 418, 1226
type-name, 158, 1219
type-parameter, 333, 1225
type-parameter-key, 333, 1225
type-specifier, 156, 1218
type-specifier-seq, 156, 1218
typedef-name, 151, 1218
typename-specifier, 364, 1225

ud-suffix, 30, 1212
unary-expression, 112, 1214
unary-operator, 112, 1214
universal-character-name, 18, 1208
unnamed-namespace-definition, 168, 1220
unqualified-id, 90, 1213
unsigned-suffix, 23, 1210
user-defined-character-literal, 30, 1212
user-defined-floating-literal, 30, 1212
user-defined-integer-literal, 30, 1212
user-defined-literal, 30, 1212
user-defined-string-literal, 30, 1212
using-declaration, 173, 1220
using-directive, 178, 1220

virt-specifier, 230, 1223
virt-specifier-seq, 230, 1223

Index of library names

_Exit, 478
__alignas_is_defined, 494
__bool_true_false_are_defined, 493, 494

_1, 595

a
    cauchy_distribution, 971
    extreme_value_distribution, 968
    uniform_int_distribution, 959
    uniform_real_distribution, 960
    weibull_distribution, 967
abort, 64, 142, 448, 478, 485, 490
abs, 991, 1003
    complex, 931
accumulate, 1000
acos, 991, 1003
    complex, 932
acosh, 1003
    complex, 932
address
    allocator, 550
addressof, 552
adjacent_difference, 1001
adjacent_find, 897
adopt_lock, 1174
adopt_lock_t, 1174
advance, 861
<algorithm>, 884
align, 545
all
    bitset, 537
all_of, 895
allocate
    allocator, 550
    allocator_traits, 548
    scoped_allocator_adaptor, 642
allocate_shared, 571
allocator, 1124
allocator, 549
    address, 550
    allocate, 550
    constructor, 550
    deallocate, 550
    destructor, 550
    max_size, 550
    operator!=, 550
    operator==, 550
allocator_arg, 545
allocator_arg_t, 545
allocator_traits, 546
    allocate, 548
    const_pointer, 547
    const_void_pointer, 547
    constructor, 548
    deallocate, 548
    destructor, 548
    difference_type, 547
    is_always_equal, 548
    max_size, 548
    pointer, 547
    propagate_on_container_copy_assignment, 548
    propagate_on_container_move_assignment, 548
    propagate_on_container_swap, 548
    rebind_alloc, 548
    select_on_container_copy_construction, 549
    size_type, 547
    void_pointer, 547
alpha
    gamma_distribution, 966
always_noconv
    codecvt, 709
any
    bitset, 537
any_of, 895
append
    basic_string, 666, 667
apply
    valarray, 988
arg, 933
    complex, 931
<array>, 772
array, 775, 777
    begin, 775
    data, 776
    end, 775
    fill, 777
    get, 777

max_size, 775
size, 775, 776
swap, 776, 777
asin, 991, 1003
complex, 932
asinh, 1003
complex, 932
<assert.h>, 449
assign
basic_regex, 1112, 1113
basic_string, 667, 668
error_code, 506
error_condition, 508
function, 599
async, 1203
at
basic_string, 665
map, 810
unordered_map, 828
at_quick_exit, 478, 479
atan, 991, 1003
complex, 932
atan2, 991, 1003
atanh, 1003
complex, 932
atexit, 64, 448, 478
<atomic>, 1139
atomic type
atomic_compare_exchange_strong, 1151
atomic_compare_exchange_strong_explicit, 1151
atomic_compare_exchange_weak, 1151
atomic_compare_exchange_weak_explicit, 1151
atomic_exchange, 1151
atomic_exchange_explicit, 1151
atomic_fetch_, 1152
atomic_is_lock_free, 1150
atomic_load, 1150
atomic_load_explicit, 1150
atomic_store, 1150
atomic_store_explicit, 1150
compare_exchange_strong, 1151
compare_exchange_strong_explicit, 1151
compare_exchange_weak, 1151
compare_exchange_weak_explicit, 1151
constructor, 1149
exchange, 1151
fetch_, 1152
load, 1150
operator @=, 1153
operator *C*, 1150
operator++, 1153
operator--, 1153
operator=, 1150
store, 1150
atomic_compare_exchange_strong
atomic type, 1151
shared_ptr, 579
atomic_compare_exchange_strong_explicit
atomic type, 1151
shared_ptr, 579
atomic_compare_exchange_weak
atomic type, 1151
shared_ptr, 579
atomic_compare_exchange_weak_explicit
atomic type, 1151
shared_ptr, 579
atomic_exchange
atomic type, 1151
shared_ptr, 578
atomic_exchange_explicit
atomic type, 1151
shared_ptr, 579
atomic_fetch_
atomic type, 1152
atomic_flag
clear, 1154
atomic_flag_clear, 1154
atomic_flag_clear_explicit, 1154
atomic_flag_test_and_set, 1154
atomic_flag_test_and_set_explicit, 1154
atomic_is_lock_free
atomic type, 1150
shared_ptr, 578
atomic_load
atomic type, 1150
shared_ptr, 578
atomic_load_explicit
atomic type, 1150
shared_ptr, 578
atomic_signal_fence, 1155
atomic_store
atomic type, 1150
shared_ptr, 578
atomic_store_explicit
atomic type, 1150
shared_ptr, 578
atomic_thread_fence, 1155

b
cauchy_distribution, 971

extreme_value_distribution, 968
uniform_int_distribution, 959
uniform_real_distribution, 960
weibull_distribution, 967
back
basic_string, 665
back_insert_iterator, 867
back_insert_iterator, 867
back_inserter, 868
bad
basic_ios, 1027
bad_alloc, 118, 480, 484, 485
bad_alloc, 484
bad_alloc::what
implementation-defined, 485
bad_array_new_length, 485
bad_array_new_length, 485
bad_array_new_length::what
implementation-defined, 485
bad_cast, 106, 486, 487
bad_cast, 487
bad_cast::what
implementation-defined, 487
bad_exception, 489
bad_exception, 489
bad_exception::what
implementation-defined, 489
bad_function_call, 595
bad_function_call, 596
bad_typeid, 107, 486, 487
bad_typeid, 487
bad_weak_ptr, 565
bad_weak_ptr, 565
what, 565
base
move_iterator, 873
raw_storage_iterator, 551
reverse_iterator, 864
basic_filebuf, 1008, 1078
basic_filebuf, 1079
constructor, 1079
destructor, 1080
operator=, 1080
swap, 1080
basic_filebuf<char>, 1077
basic_filebuf<wchar_t>, 1077
basic_fstream, 1008, 1088
basic_fstream, 1089
constructor, 1089
operator=, 1089
swap, 1090
basic_ifstream, 1008, 1084
basic_ifstream, 1085
constructor, 1085
operator=, 1085
swap, 1085, 1086
basic_ifstream<char>, 1077
basic_ifstream<wchar_t>, 1077
basic_ios, 1008, 1022
basic_ios, 1023
constructor, 1024
destructor, 1024
exceptions, 1027
fill, 1025
init, 1024
move, 1025
rdbuf, 1024
set_rdbuf, 1026
swap, 1026
tie, 1024
basic_ios<char>, 1013
basic_ios<wchar_t>, 1013
basic_iostream, 1052
basic_iostream, 1053
constructor, 1053
destructor, 1053
operator=, 1053
swap, 1053
basic_istream, 1008, 1041
basic_istream, 1043
constructor, 1043
destructor, 1043, 1044
get, 1048, 1049, 1052
operator<<, 1053
operator=, 1043
seekg, 1052
swap, 1043
tellg, 1051
basic_istream<char>, 1040
basic_istream<wchar_t>, 1040
basic_istreambuf_iterator, 1008
basic_istringstream, 1008, 1072
basic_istringstream, 1073
constructor, 1073
operator=, 1073
str, 1073
swap, 1073
basic_istringstream<char>, 1067
basic_istringstream<wchar_t>, 1067
basic_ofstream, 1008, 1086
basic_ofstream, 1087
constructor, 1087

operator=, 1087
swap, 1088
basic_ofstream<char>, 1077
basic_ofstream<wchar_t>, 1077
basic_ostream, 1008, 1119
basic_ostream, 1056
constructor, 1056
destructor, 1056, 1057
operator<<, 1059, 1060, 1062
operator=, 1056
seekp, 1057
swap, 1056
basic_ostream<char>, 1040
basic_ostream<wchar_t>, 1040
basic_ostreambuf_iterator, 1008
basic_ostringstream, 1008, 1074
basic_ostringstream, 1074
constructor, 1074, 1075
operator=, 1075
str, 1075
swap, 1075
basic_ostringstream<char>, 1067
basic_ostringstream<wchar_t>, 1067
basic_regex, 1095, 1109, 1136
assign, 1112, 1113
basic_regex, 1111, 1112
constants, 1110, 1111
constructor, 1111, 1112
flag_type, 1113
getloc, 1113
imbue, 1113
mark_count, 1113
operator=, 1112
swap, 1114
basic_streambuf, 1008, 1031
basic_streambuf, 1033
constructor, 1033
destructor, 1033
operator=, 1035
setbuf, 1072
swap, 1035
basic_streambuf<char>, 1030
basic_streambuf<wchar_t>, 1030
basic_string, 656, 677, 1067
append, 666, 667
assign, 667, 668
at, 665
back, 665
begin, 664
capacity, 664
cbegin, 664
cend, 664
clear, 665
compare, 676
constructor, 660–663
copy, 672
crbegin, 664
crend, 664
empty, 665
end, 664
erase, 670
find, 673
find_first_not_of, 675
find_first_of, 674
find_last_not_of, 675
find_last_of, 674
front, 665
get_allocator, 673
getline, 681
insert, 668, 669
length, 664
max_size, 664
operator!=, 679
operator+, 677, 678
operator+=, 666
operator<, 679
operator<=, 680
operator<<, 681
operator=, 663
operator==, 678
operator>, 679
operator>=, 680
operator>>, 680
operator[], 665
pop_back, 670
push_back, 667
rbegin, 664
rend, 664
replace, 670–672
reserve, 665
resize, 664
rfind, 673, 674
shrink_to_fit, 665
size, 664
substr, 676
swap, 672, 680
basic_stringbuf, 1008, 1067
basic_stringbuf, 1068
constructor, 1069
operator=, 1069
str, 1070
swap, 1069

basic_stringbuf<char>, 1067
basic_stringbuf<wchar_t>, 1067
basic_stringstream, 1008, 1075
    basic_stringstream, 1076
    constructor, 1076
    operator=, 1077
    str, 1077
    swap, 1077
before
    type_info, 486
before_begin
    forward_list, 785
begin, 492
    array, 775
    basic_string, 664
    initializer_list, 493
    match_results, 1123
    valarray, 999
begin(C&), 882
begin(initializer_list<E>), 493
begin(T (&)[N]), 882
bernoulli_distribution, 960
    constructor, 961
    p, 961
beta
    gamma_distribution, 966
bidirectional_iterator_tag, 860
binary_negate, 592
binary_search, 912
bind, 593–595
binomial_distribution, 961
    constructor, 962
    p, 962
    t, 962
bit_and, 591
bit_and<>, 591
bit_not<>, 592
bit_or, 591
bit_or<>, 592
bit_xor, 591
bit_xor<>, 592
<bitset>, 532
bitset, 532
    bitset, 534, 535
    flip, 536
    operator[], 538
    reset, 536
    set, 536
boolalpha, 1027
byte_string
    wstring_convert, 698
c_str
    basic_string, 673
cacos
    complex, 932
cacosh
    complex, 932
call_once, 1184
calloc, 553, 1250
capacity
    basic_string, 664
    vector, 799
casin
    complex, 932
casinh
    complex, 932
<cassert>, 449
catan
    complex, 932
catanh
    complex, 932
category
    error_code, 507
    error_condition, 509
    locale, 691
cauchy_distribution, 970
    a, 971
    b, 971
    constructor, 971
cbefore_begin
    forward_list, 785
cbegin
    basic_string, 664
cbegin(const C&), 882
cbrt, 1003
<ccomplex>, 934
cend
    basic_string, 664
cend(const C&), 882
cerr, 1011
<cerrno>, 460
<cfenv>, 923
CHAR_BIT, 476
char_class_type
    regex_traits, 1106
CHAR_MAX, 476
char_traits, 648–651
    char_type, 648
    int_type, 648
    off_type, 648
    pos_type, 648
    state_type, 648

char_type
    char_traits, 648
chi_squared_distribution, 970
    constructor, 970
    n, 970
chrono, 623
cin, 1011
<ciso646>, 1248
classic
    locale, 696
classic_table
    ctype<char>, 707
clear
    atomic_flag, 1154
    basic_ios, 1026
    basic_string, 665
    error_code, 506
    error_condition, 508
    forward_list, 787
<climits>, 1256
<clocale>, 445, 1248
clock, 493, 494
clock_t, 494
CLOCKS_PER_SEC, 494
clog, 1011
close
    basic_filebuf, 1081, 1090
    basic_ifstream, 1086
    basic_ofstream, 1088
    messages, 736
code
    future_error, 1194
    system_error, 511
codecvt, 707, 740
    always_noconv, 709
    do_always_noconv, 711
    do_encoding, 711
    do_in, 709
    do_length, 711
    do_max_length, 711
    do_out, 709
    do_unshift, 710
    encoding, 709
    in, 709
    length, 709
    max_length, 709
    out, 709
    unshift, 709
codecvt_byname, 711
collate, 722
    compare, 723
    do_compare, 723
    do_hash, 723
    do_transform, 723
    hash, 723
    transform, 723
collate_byname, 724
combine
    locale, 695
common_type, 628, 631
compare
    basic_string, 676
    collate, 723
    sub_match, 1115
compare_exchange_strong
    atomic type, 1151
compare_exchange_strong_explicit
    atomic type, 1151
compare_exchange_weak
    atomic type, 1151
compare_exchange_weak_explicit
    atomic type, 1151
complex
    literals, 934
<complex>, 924
complex, 926
    complex, 928
    imag, 928
    operator-, 930
    operator/, 930
    real, 928
<condition_variable>, 1185
condition_variable
    constructor, 1186
    destructor, 1186
    notify_all, 1187
    notify_one, 1187
    wait, 1187
    wait_for, 1188, 1189
    wait_until, 1187, 1188
condition_variable_any
    constructor, 1190
    destructor, 1190
    notify_all, 1190
    notify_one, 1190
    wait, 1190, 1191
    wait_for, 1191, 1192
    wait_until, 1191
conj, 933
    complex, 931
const_pointer
    allocator_traits, 547

const_pointer_cast
shared_ptr, 573
const_void_pointer
allocator_traits, 547
construct
scoped_allocator_adaptor, 642, 643
converted
wstring_convert, 698
copy, 900
basic_string, 672
copy_backward, 901
copy_n, 900
copyfmt
basic_ios, 1025
copysign, 1003
cos, 991, 1003
complex, 932
cosh, 991, 1003
complex, 932
count, 897
bitset, 537
duration, 630
count_if, 897
cout, 1011
crbegin
basic_string, 664
crbegin(const C& c), 883
cref
reference_wrapper, 586
crend
basic_string, 664
crend(const C& c), 883
<csetjmp>, 460, 493, 494
cshift
valarray, 988
<csignal>, 493, 494
<cstdalign>, 494
<cstdarg>, 460, 493, 494
<cstdbool>, 493, 494
<cstddef>, 1248, 1250
<cstdint>, 477
<cstdio>, 1011, 1012, 1078, 1080, 1081, 1248
<cstdlib>, 448, 493, 494, 1248, 1251
<cstring>, 446, 1248, 1256, 1260
<ctgmath>, 1002
<ctime>, 493, 494, 689, 1248
ctype, 702
do_is, 703
do_narrow, 704
do_scan_not, 704
do_tolower, 704
do_toupper, 704
do_widen, 704
is, 703
narrow, 703
scan_is, 703
scan_not, 703
tolower, 703
toupper, 703
widen, 703
ctype<char>, 705
classic_table, 707
constructor, 706
ctype<char>, 706
destructor, 706
do_narrow, 707
do_tolower, 707
do_toupper, 707
do_widen, 707
is, 706
narrow, 707
scan_is, 706
scan_not, 707
table, 707
tolower, 707
toupper, 707
widen, 707
ctype_base, 701
do_scan_is, 703
ctype_byname, 705
<cuchar>, 460
curr_symbol
moneypunct, 734
current_exception, 491
<cwchar>, 460, 1248

data
basic_string, 673
array, 776
vector, 800
date_order
time_get, 725
DBL_DIG, 476
DBL_EPSILON, 476
DBL_MANT_DIG, 476
DBL_MAX, 476
DBL_MAX_10_EXP, 476
DBL_MAX_EXP, 476
DBL_MIN, 476
DBL_MIN_10_EXP, 476
DBL_MIN_EXP, 476
deallocate

allocator, 550
allocator_traits, 548
scoped_allocator_adaptor, 642
dec, 1029, 1059
DECIMAL_DIG, 476
decimal_point
moneypunct, 734
numpunct, 721
declare_no_pointers, 544
declare_reachable, 544
declval, 516
default_delete
default_delete, 555, 556
operator(), 556
default_error_condition
error_category, 504
error_code, 507
default_random_engine, 954
defaultfloat, 1029
defer_lock, 1174
defer_lock_t, 1174
delete
operator, 553
operator, 461, 481–483
denorm_absent, 474
denorm_indeterminate, 474
denorm_min
numeric_limits, 473
denorm_present, 474
densities
piecewise_constant_distribution, 976
piecewise_linear_distribution, 978
<deque>, 773
deque, 777
deque, 780
shrink_to_fit, 781
swap, 782
detach
thread, 1163
difference_type
allocator_traits, 547
pointer_traits, 544
digits
numeric_limits, 470
digits10
numeric_limits, 471
discard_block_engine, 950
constructor, 951
discrete_distribution, 973
constructor, 974
probabilities, 974
distance, 861
div, 1003
divides, 586
divides<>, 587
do_always_noconv
codecvt, 711
do_close
message, 737
do_compare
collate, 723
do_curr_symbol
moneypunct, 735
do_date_order
time_get, 726
do_decimal_point
moneypunct, 734
numpunct, 722
do_encoding
codecvt, 711
do_falsename
numpunct, 722
do_frac_digits
moneypunct, 735
do_get
messages, 737
money_get, 730
num_get, 714, 716
time_get, 727
do_get_date
time_get, 727
do_get_monthname
time_get, 727
do_get_time
time_get, 726
do_get_weekday
time_get, 727
do_get_year
time_get, 727
do_grouping
moneypunct, 735
numpunct, 722
do_hash
collate, 723
do_in
codecvt, 709
do_is
ctype, 703
do_length
codecvt, 711
do_max_length
codecvt, 711

do_narrow, 707
    ctype, 704
    ctype<char>, 707
do_neg_format
    moneypunct, 735
do_negative_sign
    moneypunct, 735
do_open
    messages, 736
do_out
    codecvt, 709
do_pos_format
    moneypunct, 735
do_positive_sign
    moneypunct, 735
do_put
    money_put, 732
    num_put, 717, 720
    time_put, 729
do_scan_is
    ctype_base, 703
do_scan_not
    ctype, 704
do_thousands_sep
    moneypunct, 735
    numpunct, 722
do_tolower
    ctype, 704
    ctype<char>, 707
do_toupper
    ctype, 704
    ctype<char>, 707
do_transform
    collate, 723
do_truename
    numpunct, 722
do_unshift
    codecvt, 710
do_widen, 707
    ctype, 704
    ctype<char>, 707
domain_error, 496, 497
    domain_error, 497
duration
    constructor, 629, 630
    count, 630
    max, 631
    min, 631
    operator!=, 632
    operator*, 632
    operator*=, 631
    operator+, 630, 636
    operator++, 630
    operator+=, 630
    operator-, 630, 636
    operator-=, 631
    operator--, 630
    operator/, 632
    operator/=, 631
    operator<, 633
    operator<=, 633
    operator==, 632
    operator>=, 633
    operator%, 632
    operator%=, 631
    zero, 631
duration_cast, 633
duration_values, 627
    max, 627
    min, 627
    zero, 627
dynamic_pointer_cast
    shared_ptr, 573

eback
    basic_streambuf, 1035
egptr
    basic_streambuf, 1035
element_type
    pointer_traits, 543
emplace
    priority_queue, 846
emplace_after
    forward_list, 786
emplace_front
    forward_list, 785
empty, 860
    basic_string, 665
    match_results, 1122
enable_shared_from_this, 576
    constructor, 577
    destructor, 577
    operator=, 577
    shared_from_this, 577
encoding
    codecvt, 709
end, 492
    array, 775
    basic_string, 664
    initializer_list, 493
    match_results, 1123
    valarray, 999

end(C&), 882
end(initializer_list<E>), 493
end(T (&)[N]), 882
endl, 1059, 1061
ends, 1061
entropy
    random_device, 955
eof
    basic_ios, 1027
epptr
    basic_streambuf, 1036
epsilon
    numeric_limits, 471
eq
    char_traits, 673–675
equal, 898
    istreambuf_iterator, 880
equal_range, 912
equal_to, 588
equal_to<>, 589
equivalent
    error_category, 504, 505
erase
    deque, 781
    list, 793
    basic_string, 670
    vector, 800
erase_after
    forward_list, 786
erased
    forward_list, 786
erf, 1003
erfc, 1003
errc, 500
error_category, 500, 503
    constructor, 504
    default_error_condition, 504
    destructor, 504
    equivalent, 504, 505
    message, 504
    name, 504
    operator!=, 504
    operator<, 504
    operator==, 504
error_code, 500, 505, 507
    assign, 506
    category, 507
    clear, 506
    default_error_condition, 507
    error_code, 506
    message, 507
    operator bool, 507
    operator!=, 509
    operator<, 507
    operator<<, 507
    operator=, 506
    operator==, 509
    value, 507
error_condition, 500
    assign, 508
    category, 509
    clear, 508
    constructor, 508
    message, 509
    operator bool, 509
    operator!=, 509
    operator<, 509
    operator=, 508
    operator==, 509
    value, 509
error_type, 1104, 1105
exception
    bad_function_call, 595
    bad_weak_ptr, 565
<exception>, 488
exception, 488
    constructor, 489
    destructor, 489
exception_ptr, 490
exceptions
    basic_ios, 1027
exchange
    atomic type, 1151
exit, 61, 63, 142, 448, 478, 479, 485
EXIT_FAILURE, 478
EXIT_SUCCESS, 478
exp, 991, 1003
    complex, 932
exp2, 1003
expired
    weak_ptr, 575
expm1, 1003
exponential_distribution, 964
    constructor, 965
    lambda, 965
extreme_value_distribution, 967
    a, 968
    b, 968
    constructor, 967

facet
    locale, 692

fail
    basic_ios, 1027
failed
    ostreambuf_iterator, 882
failure
    ios_base::failure, 1016
falsename
    numpunct, 721
fclose, 1081
fdim, 1003
FE_ALL_EXCEPT, 923
FE_DFL_ENV, 923
FE_DIVBYZERO, 923
FE_DOWNWARD, 923
FE_INEXACT, 923
FE_INVALID, 923
FE_OVERFLOW, 923
FE_TONEAREST, 923
FE_TOWARDZERO, 923
FE_UNDERFLOW, 923
FE_UPWARD, 923
feclearexcept, 923
fegetenv, 923
fegetexceptflag, 923
fegetround, 923
feholdexcept, 923
fenv_t, 923
feraiseexcept, 923
fesetenv, 923
fesetexceptflag, 923
fesetround, 923
fetch_
    atomic type, 1152
fetestexcept, 923
feupdateenv, 923
fexcept_t, 923
filebuf, 1008, 1077
fill, 903
    array, 777
    basic_ios, 1025
    gslice_array, 996
    indirect_array, 998
    mask_array, 997
    slice_array, 993
fill_n, 903
find, 896
    basic_string, 673
find_end, 896
find_first_not_of
    basic_string, 675
find_first_of, 896
    basic_string, 674
find_if, 896
find_if_not, 896
find_last_not_of
    basic_string, 675
find_last_of
    basic_string, 674
fisher_f_distribution, 971
    constructor, 972
    m, 972
    n, 972
fixed, 1029
flag_type
    basic_regex, 1113
flags
    ios_base, 701, 1018
flip
    bitset, 536
    bitset, 536
    vector<bool>, 803
float_denorm_style, 468, 474
    numeric_limits, 472
float_round_style, 468, 474
floor, 1003
FLT_DIG, 476
FLT_EPSILON, 476
FLT_EVAL_METHOD, 476
FLT_MANT_DIG, 476
FLT_MAX, 476
FLT_MAX_10_EXP, 476
FLT_MAX_EXP, 476
FLT_MIN, 476
FLT_MIN_10_EXP, 476
FLT_MIN_EXP, 476
FLT_RADIX, 476
FLT_ROUNDS, 476
flush, 1018, 1044, 1056, 1057, 1062
    basic_ostream, 1061
fma, 1003
fmax, 1003
fmin, 1003
fmtflags
    ios, 1062
    ios_base, 1016, 1018
fopen, 1080
for_each, 895
format
    match_results, 1123, 1124
format_default, 1102
format_default, 1104
format_first_only, 1102, 1129

format_first_only, 1104
format_no_copy, 1102, 1129
format_no_copy, 1104
format_sed, 1102
format_sed, 1104
forward, 515
forward_as_tuple, 528
forward_iterator_tag, 860
<forward_list>, 773
forward_list
    before_begin, 785
    cbefore_begin, 785
    clear, 787
    emplace_after, 786
    emplace_front, 785
    erase_after, 786
    erased, 786
    forward_list, 785
    front, 785
    insert_after, 786
    merge, 788
    pop, 786
    push_front, 786
    remove, 788
    remove_if, 788
    resize, 787
    reverse, 789
    sort, 788
    splice_after, 787
    swap, 789
    unique, 788
fpclassify, 1005
fpos, 1013, 1021, 1022
    state, 1021
frac_digits
    moneypunct, 734
free, 553
freeze
    ostrstream, 1260
    strstream, 1262
    strstreambuf, 1256
frexp, 1003
from_bytes
    wstring_convert, 698
from_time_t, 637
front
    basic_string, 665
    forward_list, 785
front_insert_iterator, 868
    front_insert_iterator, 869
front_inserter, 869
fseek, 1080
<fstream>, 1077
fstream, 1008
function, 596
    assign, 599
    bool conversion, 599
    destructor, 599
    function, 597, 598
    invocation, 599
    operator!=, 599
    operator(), 599
    operator=, 598, 599
    operator==, 599
    swap, 599, 600
    target, 599
    target_type, 599
<functional>, 580
future
    constructor, 1198, 1199
    get, 1199
    operator=, 1199
    share, 1199
    valid, 1199
    wait, 1200
    wait_for, 1200
    wait_until, 1200
future_category, 1193
future_errc
    make_error_code, 1193
    make_error_condition, 1194
future_error
    code, 1194
    what, 1194

gamma_distribution, 965
    alpha, 966
    beta, 966
    constructor, 966
gbump
    basic_streambuf, 1035
gcount
    basic_istream, 1048
generate, 903
    seed_seq, 956
generate_canonical, 958
generate_n, 903
generic_category, 503, 505
geometric_distribution, 962
    constructor, 962
    p, 962
get

array, 777
basic_istream, 1048, 1049, 1052
future, 1199
messages, 736
money_get, 730
num_get, 713
pair, 520, 521
reference_wrapper, 585
shared_future, 1202
shared_ptr, 570
time_get, 726
tuple, 529, 530
unique_ptr, 560
get_allocator
basic_string, 673
match_results, 1124
get_date
time_get, 725
get_deleter
shared_ptr, 573
unique_ptr, 560
get_future
packaged_task, 1206
promise, 1196
get_id
this_thread, 1164
thread, 1163
get_money, 1064
get_monthname
time_get, 725
get_new_handler, 461
get_pointer_safety, 545
get_temporary_buffer, 551
get_terminate, 461
get_time, 1065
time_get, 725
get_unexpected, 461
get_weekday
time_get, 725
get_year
time_get, 725
getenv, 493, 494
getline
basic_istream, 1049, 1050
basic_string, 681
getloc, 1108
basic_regex, 1113
basic_streambuf, 1033
ios_base, 1019
global
locale, 696
good
basic_ios, 1027
gptr
basic_streambuf, 1035
greater, 588
greater<>, 589
greater_equal, 588
greater_equal<>, 589
grouping
moneypunct, 734
numpunct, 721
gslice, 993
constructor, 994
gslice_array, 995

hardware_concurrency
thread, 1163
has_denorm_loss
numeric_limits, 472
has_facet
locale, 696
has_infinity
numeric_limits, 472
has_quiet_NaN
numeric_limits, 472
has_signaling_NaN
numeric_limits, 472
hash, 509, 580, 600, 645, 683
collate, 723
hash_code, 538
type_info, 486
type_index, 645
hex, 1029
hexfloat, 1029
hypot, 1003

id
locale, 693
idxl
operator>, 633
ifstream, 1008, 1077
ignore, 528
basic_istream, 1050
ilogb, 1003
imag, 933
complex, 928, 931
imbue, 1108
basic_filebuf, 1084
basic_ios, 1025
basic_regex, 1113
basic_streambuf, 1036

ios_base, 1019
in
codecvt, 709
in_avail
basic_streambuf, 1034
includes, 914
independent_bits_engine, 951
index_sequence, 514
index_sequence_for, 514
indirect_array, 997
operator[], 998
infinity
numeric_limits, 472
Init
ios_base::Init, 1018
init
basic_ios, 1024, 1043, 1056
<initializer_list>, 492
initializer_list, 492
begin, 493
end, 493
initializer_list, 493
size, 493
inner_allocator
scoped_allocator_adaptor, 642
inner_allocator_type
scoped_allocator_adaptor, 640
inner_product, 1000
inplace_merge, 913
input_iterator_tag, 860
emplace
deque, 781
insert
deque, 781
list, 793
basic_string, 668, 669
map, 810
multimap, 815
unordered_map, 828
unordered_multimap, 833
vector, 800
push_back
deque, 781
push_front
deque, 781
insert_after
forward_list, 786
insert_iterator, 869
insert_iterator, 870
insert_or_assign
map, 810, 811
unordered_map, 829
inserter, 870
int16_t, 477
int32_t, 477
int64_t, 477
int8_t, 477
int_fast16_t, 477
int_fast32_t, 477
int_fast64_t, 477
int_fast8_t, 477
int_least16_t, 477
int_least32_t, 477
int_least64_t, 477
int_least8_t, 477
INT_MAX, 476
INT_MIN, 476
int_type
char_traits, 648
wstring_convert, 699
integer_sequence, 532
internal, 1028
intervals
piecewise_constant_distribution, 976
piecewise_linear_distribution, 978
intmax_t, 477
intptr_t, 477
invalid_argument, 496, 497, 534
invalid_argument, 497
INVOKE, 583, 584
invoke, 584
<iomanip>, 1040
<ios>, 1012
ios, 1008, 1013
ios_base, 1013
destructor, 1021
fmtflags, 1018
ios_base, 1021
iostate, 1016
precision, 1019
setf, 1019
streamsize, 1019
ios_base::failure, 1016
ios_base::Init, 1018
destructor, 1018
<iosfwd>, 1008
iostate
ios_base, 1016
<iostream>, 1010
iostream_category, 1029
iota, 1002
is

ctype, 703
ctype<char>, 706
is_always_equal
allocator_traits, 548
scoped_allocator_adaptor, 641
is_bind_expression, 593
is_bounded
numeric_limits, 473
is_error_code_enum, 500
is_error_condition_enum, 500
is_exact
numeric_limits, 471
is_heap, 917
is_heap_until, 917
is_iec559
numeric_limits, 473
is_integer
numeric_limits, 471
is_modulo
numeric_limits, 473
is_open
basic_filebuf, 1080, 1090
basic_ifstream, 1086
basic_ofstream, 1088
is_partitioned, 907
is_permutation, 899
is_placeholder, 593
is_signed
numeric_limits, 471
is_sorted, 910
is_sorted_until, 910
isalnum, 696
isalpha, 696
isblank, 696
iscntrl, 696
isctype
regex_traits, 1107
regular expression traits, 1137
isdigit, 696
isfinite, 1005
isgraph, 696
isgreater, 1005
isgreaterequal, 1005
isinf, 1005
isless, 1005
islessequal, 1005
islessgreater, 1005
islower, 696
isnan, 1005
isnormal, 1005
<iso646.h>, 1248
isprint, 696
ispunct, 696
isspace, 696
<istream>, 1039
istream, 1008, 1040
istream_iterator, 875
constructor, 876
destructor, 876
operator!=, 877
operator*, 876
operator++, 877
operator->, 876
operator==, 877
istreambuf_iterator, 879
constructor, 880
operator++, 880
istringstream, 1008, 1067
istrstream, 1259
constructor, 1259
istrstream, 1259
isunordered, 1005
isupper, 696
isxdigit, 696
iter_swap, 902
<iterator>, 855
iword
ios_base, 1020

jmp_buf, 494
join
thread, 1163
joinable
thread, 1163

kill_dependency, 1144
knuth_b, 954

lambda
exponential_distribution, 965
LDBL_DIG, 476
LDBL_EPSILON, 476
LDBL_MANT_DIG, 476
LDBL_MAX, 476
LDBL_MAX_10_EXP, 476
LDBL_MAX_EXP, 476
LDBL_MIN, 476
LDBL_MIN_10_EXP, 476
LDBL_MIN_EXP, 476
left, 1028
length
char_traits, 662, 663, 668, 678

basic_string, 664
codecvt, 709
match_results, 1123
regex_traits, 1106
sub_match, 1114
valarray, 988
length_error, 496, 498, 656
length_error, 498
less, 588
less<>, 589
less_equal, 589
less_equal<>, 589
lexicographical_compare, 920
lgamma, 1003
<limits>, 468
linear_congruential_engine, 946
constructor, 947
<list>, 774
list, 789
list, 792
splice, 794
swap, 796
literals
complex, 934
LLONG_MAX, 476
LLONG_MIN, 476
llrint, 1003
llround, 1003
load
atomic type, 1150
<locale>, 688, 689
locale, 1108, 1113, 1136
category, 691
classic, 696
combine, 695
constructor, 694
destructor, 695
facet, 692
global, 696
has_facet, 696
id, 693
name, 695
operator!=, 695
operator(), 695
operator=, 694
operators==, 695
use_facet, 696
lock, 1183
shared_lock, 1181
unique_lock, 1178
weak_ptr, 575
lock_guard
constructor, 1175
destructor, 1175
log, 991, 1003
complex, 932
log10, 991, 1003
complex, 933
log1p, 1003
log2, 1003
logb, 1003
logic_error, 496
logic_error, 497
logical_and, 590
logical_and<>, 590
logical_not, 590
logical_not<>, 591
logical_or, 590
logical_or<>, 590
lognormal_distribution, 969
constructor, 969
m, 969
s, 969
LONG_MAX, 476
longjmp, 494
lookup_classname
regex_traits, 1107
regular expression traits, 1137
lookup_collatename
regex_traits, 1106
regular expression traits, 1137
lower_bound, 911
lowest
numeric_limits, 470
lrint, 1003
lround, 1003

m
fisher_f_distribution, 972
lognormal_distribution, 969
make_error_code, 500, 507, 1029
future_errc, 1193
make_error_condition, 500, 509, 1029
future_errc, 1194
make_exception_ptr, 491
make_heap, 917
make_index_sequence, 514
make_integer_sequence, 532
make_move_iterator, 875
make_pair, 520
make_ready_at_thread_exit
packaged_task, 1207

make_reverse_iterator, 867
make_shared, 571
make_tuple, 527
make_unique, 563
malloc, 553, 1250
<map>, 803
map, 805
    constructor, 809
    insert, 810
    insert_or_assign, 810, 811
    map, 809
    operator<, 809
    operator==, 809
    swap, 811
    try_emplace, 810
mark_count
    basic_regex, 1113
mask_array, 996
    operator[], 996
match_any, 1102
match_any, 1103
match_continuous, 1102, 1132
match_continuous, 1104
match_default, 1102
match_flag_type, 1102, 1138
match_not_bol, 1102
match_not_bol, 1103
match_not_bow, 1102
match_not_bow, 1103
match_not_eol, 1102
match_not_eol, 1103
match_not_eow, 1102
match_not_eow, 1103
match_not_null, 1102, 1132
match_not_null, 1103
match_prev_avail, 1102, 1132
match_prev_avail, 1104
match_results, 1120, 1130, 1133
    begin, 1123
    empty, 1122
    end, 1123
    format, 1123, 1124
    get_allocator, 1124
    length, 1123
    match_results, 1121, 1122
    matched, 1120
    max_size, 1122
    operator!=, 1125
    operator=, 1122
    operator==, 1125
    operator[], 1123
    position, 1123
    prefix, 1123
    size, 1122
    state, 1122
    str, 1123
    suffix, 1123
    swap, 1124
max, 918
    duration, 631
    duration_values, 627
    numeric_limits, 470
    time_point, 636
    valarray, 988
max_align_t, 467, 468
max_digits10
    numeric_limits, 471
max_element, 919
max_exponent
    numeric_limits, 472
max_exponent10
    numeric_limits, 472
max_length
    codecvt, 709
max_size
    allocator, 550
    allocator_traits, 548
    array, 775
    basic_string, 664
    match_results, 1122
    scoped_allocator_adaptor, 642
MB_LEN_MAX, 476
mean
    normal_distribution, 969
    poisson_distribution, 964
    student_t_distribution, 973
mem_fn, 595
memchr, 684
<memory>, 539
merge, 913
    list, 795
    forward_list, 788
mersenne_twister_engine, 947
    constructor, 948
message
    do_close, 737
    error_category, 504
    error_code, 507
    error_condition, 509
messages, 736
    close, 736
    do_get, 737

do_open, 736
get, 736
open, 736
messages_byname, 737
min, 918
duration, 631
duration_values, 627
numeric_limits, 470
time_point, 636
valarray, 988
min_element, 919
min_exponent
numeric_limits, 471
min_exponent10
numeric_limits, 471
minmax, 918, 919
minmax_element, 919
minstd_rand, 953
minstd_rand0, 953
minus, 586
minus<>, 587
mismatch, 897
mod, 1003
modf, 1003
modulus, 586
modulus<>, 587
money_get, 730
do_get, 730
get, 730
money_put, 732
do_put, 732
put, 732
moneypunct, 733
curr_symbol, 734
decimal_point, 734
do_curr_symbol, 735
do_decimal_point, 734
do_frac_digits, 735
do_grouping, 735
do_neg_format, 735
do_negative_sign, 735
do_pos_format, 735
do_positive_sign, 735
do_thousands_sep, 735
frac_digits, 734
grouping, 734
negative_sign, 734
positive_sign, 734
thousands_sep, 734
moneypunct_byname, 735
move, 516
basic_ios, 1025
movemove, 901
move_backward, 901
move_if_noexcept, 516
move_iterator, 871
base, 873
constructor, 872
move_iterator, 872
operator!=, 874
operator*, 873
operator+, 873, 875
operator++, 873
operator+=, 874
operator-, 874, 875
operator-=, 874
operator->, 873
operator--, 873
operator<, 874
operator<=, 874
operator=, 873
operator==, 874
operator>, 874
operator>=, 874
operator[], 874
mt19937, 953
mt19937_64, 954
multimap, 811
insert, 815
multimap, 814
operator<, 814
operator==, 814
swap, 815
multiplies, 586
multiplies<>, 587
multiset, 818
multiset, 821
operator<, 821
operator==, 821
swap, 822
<mutex>, 1164
mutex
shared_lock, 1183
unique_lock, 1179

n
chi_squared_distribution, 970
fisher_f_distribution, 972
name
type_info, 486
error_category, 504
locale, 695

type_index, 645
nan, 1003
narrow
basic_ios, 1025
ctype, 703
ctype<char>, 707
NDEBUG, 449
nearbyint, 1003
negate, 587
negate<>, 588
negative_binomial_distribution, 963
constructor, 963
p, 963
t, 963
negative_sign
moneypunct, 734
nested_exception, 491
nested_exception, 492
nested_ptr, 492
rethrow_if_nested, 492
rethrow_nested, 492
throw_with_nested, 492
nested_ptr
nested_exception, 492
<new>, 480
new
operator, 480, 484
operator, 460, 461, 481–484, 553
new_handler, 485
next, 862
next_permutation, 920
nextafter, 1003
nexttoward, 1003
noboolalpha, 1027
none
bitset, 537
none_of, 895
norm, 933
complex, 931
normal_distribution, 968
constructor, 968
mean, 969
stddev, 969
noshowbase, 1027
noshowpoint, 1027
noshowpos, 1028
noskipws, 1028
not1, 592
not2, 593
not_equal_to, 588
not_equal_to<>, 589
notify_all
condition_variable, 1187
condition_variable_any, 1190
notify_all_at_thread_exit, 1185
notify_one
condition_variable, 1187
condition_variable_any, 1190
nounitbuf, 1028
nouppercase, 1028
nth_element, 911
NULL, 467, 468
nullptr_t, 467, 468
num_get, 712
do_get, 714, 716
get, 713
num_put, 716
do_put, 717, 720
put, 717
<numeric>, 999
numeric_limits, 469
numeric_limits, 468
denorm_min, 473
digits, 470
digits10, 471
epsilon, 471
float_denorm_style, 472
has_denorm_loss, 472
has_infinity, 472
has_quiet_NaN, 472
has_signaling_NaN, 472
infinity, 472
is_bounded, 473
is_exact, 471
is_iec559, 473
is_integer, 471
is_modulo, 473
is_signed, 471
lowest, 470
max, 470
max_digits10, 471
max_exponent, 472
max_exponent10, 472
min, 470
min_exponent, 471
min_exponent10, 471
quiet_NaN, 473
radix, 471
round_error, 471
round_style, 474
signaling_NaN, 473
tinyness_before, 474

traps, 473
numeric_limits<bool>, 476
numpunct, 720
decimal_point, 721
do_decimal_point, 722
do_falsename, 722
do_grouping, 722
do_thousands_sep, 722
do_truename, 722
falsename, 721
grouping, 721
thousands_sep, 721
truename, 721
numpunct_byname, 722

oct, 1029
off_type
char_traits, 648
offsetof, 467, 468, 1250
ofstream, 1008, 1077
once_flag, 1183
open
basic_filebuf, 1080, 1081, 1090
basic_ifstream, 1086
basic_ofstream, 1088
messages, 736
openmode
ios_base, 1016
operator @=
atomic type, 1153
operator basic_string
sub_match, 1114
operator bool
basic_istream, 1044
basic_ios, 1026
basic_ostream, 1057
error_code, 507
error_condition, 509
shared_lock, 1183
shared_ptr, 571
unique_lock, 1179
unique_ptr, 560
operator *C*
atomic type, 1150
operator T&
reference_wrapper, 585
operator!
basic_ios, 1026
valarray, 986
operator!=, 514
pair, 519
type_info, 486
allocator, 550
basic_string, 679
bitset, 537
complex, 930
duration, 632
error_category, 504
error_code, 509
error_condition, 509
function, 599
istream_iterator, 877
istreambuf_iterator, 880
locale, 695
match_results, 1125
move_iterator, 874
queue, 843
regex_iterator, 1132
regex_token_iterator, 1136
reverse_iterator, 866
scoped_allocator_adaptor, 644
shared_ptr, 572
stack, 849
sub_match, 1115–1119
thread::id, 1161
time_point, 636
tuple, 530
type_index, 645
unique_ptr, 563, 564
valarray, 990
operator()
default_delete, 556
function, 599
locale, 695
packaged_task, 1206
random_device, 955
reference_wrapper, 585
operator*
back_insert_iterator, 868
complex, 930
duration, 632
front_insert_iterator, 869
insert_iterator, 870
istream_iterator, 876
istreambuf_iterator, 880
move_iterator, 873
ostream_iterator, 878
ostreambuf_iterator, 881
raw_storage_iterator, 551
regex_iterator, 1132
regex_token_iterator, 1136
reverse_iterator, 864

shared_ptr, 570
unique_ptr, 560
valarray, 989
operator*=
complex, 929
duration, 631
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator+
basic_string, 677, 678
complex, 930
duration, 630, 636
move_iterator, 873, 875
reverse_iterator, 865, 866
time_point, 636
valarray, 986, 989
operator++
atomic type, 1153
back_insert_iterator, 868
duration, 630
front_insert_iterator, 869
insert_iterator, 870
istream_iterator, 877
istreambuf_iterator, 880
move_iterator, 873
ostream_iterator, 878
ostreambuf_iterator, 881
raw_storage_iterator, 551
regex_iterator, 1132, 1133
regex_token_iterator, 1136
reverse_iterator, 864
operator+=
basic_string, 666
complex, 929
duration, 630
gslice_array, 996
indirect_array, 998
mask_array, 997
move_iterator, 874
reverse_iterator, 865
slice_array, 993
time_point, 635
valarray, 987
operator-
complex, 930
duration, 630, 636
move_iterator, 874, 875
reverse_iterator, 865, 866
time_point, 636
valarray, 986, 989
operator-=
complex, 929
duration, 631
gslice_array, 996
indirect_array, 998
mask_array, 997
move_iterator, 874
reverse_iterator, 865
slice_array, 993
time_point, 635
valarray, 987
operator->
istream_iterator, 876
move_iterator, 873
regex_iterator, 1132
regex_token_iterator, 1136
reverse_iterator, 864
shared_ptr, 570
unique_ptr, 560
operator--
atomic type, 1153
duration, 630
move_iterator, 873
reverse_iterator, 865
operator/
complex, 930
duration, 632
valarray, 989
operator/=
complex, 929
duration, 631
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator<
pair, 519
basic_string, 679
duration, 633
error_category, 504
error_code, 507
error_condition, 509
move_iterator, 874
queue, 843
reverse_iterator, 866
shared_ptr, 571, 572
stack, 849
sub_match, 1115–1119

thread::id, 1161
time_point, 636
tuple, 530
type_index, 645
unique_ptr, 563, 564
valarray, 990
operator«
shared_ptr, 573
sub_match, 1119
operator<<
bitset, 537, 538
complex, 931
operator<<=
bitset, 535
operator<=, 514
pair, 520
basic_string, 680
duration, 633
move_iterator, 874
queue, 843
reverse_iterator, 866
shared_ptr, 564, 572
stack, 849
sub_match, 1115–1119
thread::id, 1161
time_point, 636
tuple, 531
type_index, 645
unique_ptr, 564
valarray, 990
operator<<
basic_istream, 1053
basic_ostream, 1058–1060, 1062
basic_string, 681
error_code, 507
thread::id, 1161
valarray, 989
operator<<=
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator=
bad_alloc, 484
bad_cast, 487
bad_exception, 489
bad_typeid, 487
reverse_iterator, 864
atomic type, 1150
back_insert_iterator, 868
basic_filebuf, 1080
basic_fstream, 1089
basic_ifstream, 1085
basic_iostream, 1053
basic_istream, 1043
basic_istringstream, 1073
basic_ofstream, 1087
basic_ostream, 1056
basic_ostringstream, 1075
basic_regex, 1112
basic_streambuf, 1035
basic_string, 663
basic_stringbuf, 1069
basic_stringstream, 1077
enable_shared_from_this, 577
error_code, 506
error_condition, 508
exception, 489
front_insert_iterator, 869
function, 598, 599
future, 1199
gslice_array, 995
indirect_array, 998
insert_iterator, 870
locale, 694
mask_array, 997
match_results, 1122
move_iterator, 873
ostream_iterator, 878
ostreambuf_iterator, 881
packaged_task, 1206
pair, 518, 519
promise, 1196
raw_storage_iterator, 551
reference_wrapper, 585
shared_future, 1201, 1202
shared_ptr, 569, 570
slice_array, 992
thread, 1162
tuple, 526, 527
unique_lock, 1177
unique_ptr, 559, 560
valarray, 984, 989
weak_ptr, 575
operator==
pair, 519
type_info, 486
allocator, 550
basic_string, 678
bitset, 537
complex, 930

duration, 632
error_category, 504
error_code, 509
error_condition, 509
function, 599
istream_iterator, 877
istreambuf_iterator, 880
match_results, 1125
move_iterator, 874
queue, 843
regex_iterator, 1132
regex_token_iterator, 1133, 1136
reverse_iterator, 865
scoped_allocator_adaptor, 644
shared_ptr, 571, 572
stack, 849
sub_match, 1115–1119
thread::id, 1161
time_point, 636
tuple, 530
type_index, 645
unique_ptr, 563, 564
valarray, 990
operator>, 514
pair, 519
basic_string, 679
idxl, 633
move_iterator, 874
queue, 843
reverse_iterator, 866
shared_ptr, 572
stack, 849
sub_match, 1115–1119
thread::id, 1161
time_point, 637
tuple, 531
type_index, 645
unique_ptr, 564
valarray, 990
operator>=, 514
pair, 520
basic_string, 680
duration, 633
move_iterator, 874
queue, 843
reverse_iterator, 866
shared_ptr, 572
stack, 849
sub_match, 1115–1119
thread::id, 1161
time_point, 637
tuple, 531
type_index, 645
unique_ptr, 564, 565
valarray, 990
operator>>
bitset, 537, 538
complex, 930
operator>>=
bitset, 535
operator>>
basic_istream, 1046
basic_string, 680
istream, 1045–1047
valarray, 989
operator>>=
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator[]
basic_string, 665
bitset, 538
indirect_array, 998
map, 809
mask_array, 996
match_results, 1123
move_iterator, 874
reverse_iterator, 865
unique_ptr, 563
unordered_map, 827
valarray, 984–986
operator%
duration, 632
valarray, 989
operator%=
duration, 631
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator&
bitset, 538
valarray, 989
operator&=
bitset, 535
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993

valarray, 987
operator&&
valarray, 989, 990
operator^
bitset, 538
valarray, 989
operator^=
bitset, 535
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator~
bitset, 536
valarray, 986
operators==
locale, 695
operator|
bitset, 538
valarray, 989
operator|=
bitset, 535
gslice_array, 996
indirect_array, 998
mask_array, 997
slice_array, 993
valarray, 987
operator||
valarray, 989, 990
<ostream>, 1040
ostream, 1008, 1040
ostream_iterator, 877
constructor, 878
destructor, 878
operator*, 878
operator++, 878
operator=, 878
ostreambuf_iterator, 881
constructor, 881
ostringstream, 1008, 1067
ostrstream, 1260
constructor, 1260
ostrstream, 1260
out
codecvt, 709
out_of_range, 498, 534, 536, 537, 656
out_of_range, 498
out_of_range_error, 496
outer_allocator
scoped_allocator_adaptor, 642
output_iterator_tag, 860
overflow
basic_filebuf, 1082
basic_streambuf, 1039
basic_stringbuf, 1070
strstreambuf, 1256
overflow_error, 496, 499, 500, 534, 536, 537
overflow_error, 499
owner_before
shared_ptr, 571, 576
owns_lock
shared_lock, 1183
unique_lock, 1179

p
bernoulli_distribution, 961
binomial_distribution, 962
geometric_distribution, 962
negative_binomial_distribution, 963
packaged_task
constructor, 1205, 1206
destructor, 1206
get_future, 1206
make_ready_at_thread_exit, 1207
operator(), 1206
operator=, 1206
reset, 1207
swap, 1206, 1207
valid, 1206
pair, 517, 525, 527
get, 520, 521
operator=, 518, 519
pair, 517, 518
swap, 519
param
seed_seq, 957
partial_sort, 909
partial_sort_copy, 910
partial_sum, 1000
partition, 907
partition_copy, 907
partition_point, 908
pbackfail
basic_filebuf, 1082
basic_streambuf, 1038
basic_stringbuf, 1070
strstreambuf, 1257
pbase
basic_streambuf, 1036
pbump
basic_streambuf, 1036

pcount
ostrstream, 1260
strstream, 1262
strstreambuf, 1256
peek
basic_istream, 1050
piecewise_constant_distribution, 974
constructor, 975, 976
densities, 976
intervals, 976
piecewise_construct, 521
piecewise_construct_t, 521
piecewise_linear_distribution, 976
constructor, 977, 978
densities, 978
intervals, 978
placeholders, 595
plus, 586
plus<>, 587
pointer
allocator_traits, 547
pointer_to
pointer_traits, 544
pointer_traits, 543
difference_type, 544
element_type, 543
pointer_to, 544
rebind, 544
poisson_distribution, 963
constructor, 964
mean, 964
polar
complex, 932
pop
priority_queue, 846
forward_list, 786
pop_back
basic_string, 670
pop_heap, 916
pos_type
char_traits, 648
position
match_results, 1123
positive_sign
moneypunct, 734
pow, 934, 991, 1003
complex, 933
pptr
basic_streambuf, 1036
precision
ios_base, 701, 1019
prefix
match_results, 1123
prev, 862
prev_permutation, 920
priority_queue, 844
emplace, 846
priority_queue, 845
swap, 846
probabilities
discrete_distribution, 974
proj
complex, 931
promise
constructor, 1196
destructor, 1196
get_future, 1196
operator=, 1196
set_exception, 1197
set_exception_at_thread_exit, 1197
set_value, 1197
set_value_at_thread_exit, 1197
swap, 1196, 1198
propagate_on_container_copy_assignment
allocator_traits, 548
scoped_allocator_adaptor, 641
propagate_on_container_move_assignment
allocator_traits, 548
scoped_allocator_adaptor, 641
propagate_on_container_swap
allocator_traits, 548
scoped_allocator_adaptor, 641
proxy
istreambuf_iterator, 879
ptrdiff_t, 467
pubimbue
basic_streambuf, 1033
pubseekoff
basic_streambuf, 1034
pubseekpos
basic_streambuf, 1034
pubsetbuf
basic_streambuf, 1034
pubsync
basic_streambuf, 1034
push
priority_queue, 846
push_back
basic_string, 667
push_front
forward_list, 786
push_heap, 916

put
    basic_ostream, 1061
    money_put, 732
    num_put, 717
    time_put, 729
put_money, 1064
put_time, 1065
putback
    basic_istream, 1051
putenv, 494
pword
    ios_base, 1020

<queue>, 841
queue, 841
    swap, 844
quick_exit, 478, 479
quiet_NaN
    numeric_limits, 473
quoted, 1066

radix
    numeric_limits, 471
raise, 494
rand, 1002
<random>, 943–945
random_access_iterator_tag, 860
random_device, 955
    constructor, 955
    entropy, 955
    operator(), 955
range_error, 496, 499
    range_error, 499
ranlux24, 954
ranlux24_base, 954
ranlux48, 954
ranlux48_base, 954
ratio, 620
ratio_equal, 622
ratio_greater, 623
ratio_greater_equal, 623
ratio_less, 622
ratio_less_equal, 623
ratio_not_equal, 622
raw_storage_iterator
    base, 551
    constructor, 551
    operator*, 551
    operator++, 551
    operator=, 551
rbegin
    basic_string, 664
rbegin(C&), 882
rbegin(initializer_list<E>), 882
rbegin(T (&array)[N]), 882
rdbuf
    basic_filebuf, 1090
    basic_ifstream, 1086
    basic_ios, 1024
    basic_istringstream, 1073
    basic_ofstream, 1088
    basic_ostringstream, 1075
    basic_stringstream, 1077
    istrstream, 1259
    ostrstream, 1260
    strstream, 1262
    wbuffer_convert, 700, 701
rdstate
    basic_ios, 1026
read
    basic_istream, 1050
readsome
    basic_istream, 1051
real, 933
    complex, 928, 931
realloc, 553, 1250
rebind
    pointer_traits, 544
rebind_alloc
    allocator_traits, 548
ref
    reference_wrapper, 586
reference_wrapper, 584
    cref, 586
    get, 585
    operator T&, 585
    operator(), 585
    operator=, 585
    ref, 586
    reference_wrapper, 585
<regex>, 1095
regex, 1095
regex_constants, 1102
    error_type, 1104, 1105
    match_flag_type, 1102
    syntax_option_type, 1102
regex_error, 1105, 1109, 1138
    constructor, 1105
regex_iterator, 1130
    increment, 1132
    operator!=, 1132
    operator*, 1132

operator++, 1132, 1133
operator->, 1132
operator==, 1132
regex_iterator, 1131
regex_match, 1125–1127
regex_replace, 1129, 1130
regex_search, 1127, 1128
regex_token_iterator, 1133
end-of-sequence, 1133
operator!=, 1136
operator*, 1136
operator++, 1136
operator->, 1136
operator==, 1133, 1136
regex_token_iterator, 1135
regex_traits, 1105
char_class_type, 1106
isctype, 1107
length, 1106
lookup_classname, 1107
lookup_collatename, 1106
transform, 1106
transform_primary, 1106
translate, 1106
translate_nocase, 1106
value, 1108
register_callback
ios_base, 1021
regular expression traits
isctype, 1137
lookup_classname, 1137
lookup_collatename, 1137
transform_primary, 1138
rel_ops, 512
release
shared_lock, 1182
unique_lock, 1179
unique_ptr, 560
remainder, 1003
remove, 904
list, 794
forward_list, 788
remove_copy, 904
remove_copy_if, 904
remove_if, 904
forward_list, 788
remquo, 1003
rend
basic_string, 664
rend(const C&), 882
rend(initializer_list<E>), 883
rend(T (&array)[N]), 882
rep
system_clock, 637
replace, 902
basic_string, 670–672
replace_copy, 903
replace_copy_if, 903
replace_if, 902
reserve
basic_string, 665
vector, 799
reset
bitset, 536
packaged_task, 1207
shared_ptr, 570
unique_ptr, 560, 563
weak_ptr, 575
resetiosflags, 1062
resize
deque, 780
list, 792, 793
basic_string, 664
forward_list, 787
valarray, 988
vector, 799, 800
rethrow_exception, 491
rethrow_if_nested
nested_exception, 492
rethrow_nested
nested_exception, 492
return_temporary_buffer, 552
reverse, 905
list, 795
forward_list, 789
reverse_copy, 905
reverse_iterator, 862
reverse_iterator, 864
base, 864
constructor, 864
make_reverse_iterator non-member function, 867
operator++, 864
operator--, 865
rfind
basic_string, 673, 674
right, 1028
rint, 1003
rotate, 906
rotate_copy, 906
round, 1003
round_error

numeric_limits, 471
round_indeterminate, 474
round_style
numeric_limits, 474
round_to_nearest, 474
round_toward_infinity, 474
round_toward_neg_infinity, 474
round_toward_zero, 474
runtime_error, 496, 498
runtime_error, 498, 499

s
lognormal_distribution, 969
sbumpc
basic_streambuf, 1034
scalbln, 1003
scalbn, 1003
scan_is
ctype, 703
ctype<char>, 706
scan_not
ctype, 703
ctype<char>, 707
SCHAR_MAX, 476
SCHAR_MIN, 476
scientific, 1029
<scoped_allocator>, 638
scoped_allocator_adaptor
allocate, 642
construct, 642, 643
constructor, 641
deallocate, 642
destructor, 644
inner_allocator, 642
inner_allocator_type, 640
is_always_equal, 641
max_size, 642
operator!=, 644
operator==, 644
outer_allocator, 642
propagate_on_container_copy_assignment, 641
propagate_on_container_move_assignment, 641
propagate_on_container_swap, 641
select_on_container_copy_construction, 644
search, 899
search_n, 900
seed_seq
constructor, 956
generate, 956
param, 957
size, 957
seekdir
ios_base, 1018
seekg
basic_istream, 1052
seekoff
basic_filebuf, 1083
basic_streambuf, 1036
basic_stringbuf, 1071
strstreambuf, 1257
seekp
basic_ostream, 1057
seekpos
basic_filebuf, 1083
basic_streambuf, 1036
basic_stringbuf, 1072
strstreambuf, 1258
select_on_container_copy_construction
allocator_traits, 549
scoped_allocator_adaptor, 644
sentry
basic_istream, 1043
basic_ostream, 1056
constructor, 1044, 1056
<set>, 804
set, 815
bitset, 536
operator<, 818
operator==, 818
set, 818
swap, 818
set_difference, 915
set_exception
promise, 1197
set_exception_at_thread_exit
promise, 1197
set_intersection, 914
set_new_handler, 461, 485
set_rdbuf
basic_ios, 1026
set_symmetric_difference, 915
set_terminate, 461, 490
set_unexpected, 461, 1262
set_union, 914
set_value
promise, 1197
set_value_at_thread_exit
promise, 1197
setbase, 1063

setbuf
basic_filebuf, 1083
basic_streambuf, 1036, 1072
streambuf, 1259
strstreambuf, 1259
setenv, 494
setf
ios_base, 1018, 1019
setfill, 1063
setg
basic_streambuf, 1035
strstreambuf, 1256
setiosflags, 1062
setjmp, 460, 494
<setjmp.h>, 493
setlocale, 445
setp
basic_streambuf, 1036
setprecision, 1063
setstate
basic_ios, 1026
setw, 1063
sgetc
basic_streambuf, 1034
sgetn
basic_streambuf, 1034
share
future, 1199
shared_from_this
enable_shared_from_this, 577
shared_future
constructor, 1201
destructor, 1201
get, 1202
operator=, 1201, 1202
valid, 1202
wait, 1202
wait_for, 1202
wait_until, 1203
shared_lock
constructor, 1180, 1181
destructor, 1181
lock, 1181
mutex, 1183
operator bool, 1183
operator=, 1181
owns_lock, 1183
release, 1182
swap, 1182, 1183
try_lock, 1182
try_lock_for, 1182
try_lock_until, 1182
unlock, 1182
<shared_mutex>, 1165
shared_ptr, 565, 577
atomic_compare_exchange_strong, 579
atomic_compare_exchange_strong_explicit, 579
atomic_compare_exchange_weak, 579
atomic_compare_exchange_weak_explicit, 579
atomic_exchange, 578
atomic_exchange_explicit, 579
atomic_is_lock_free, 578
atomic_load, 578
atomic_load_explicit, 578
atomic_store, 578
atomic_store_explicit, 578
const_pointer_cast, 573
constructor, 568, 569
destructor, 569
dynamic_pointer_cast, 573
get, 570
get_deleter, 573
operator bool, 571
operator!=, 572
operator*, 570
operator->, 570
operator<, 571, 572
operator«, 573
operator<=, 564, 572
operator=, 569, 570
operator==, 571, 572
operator>, 572
operator>=, 572
owner_before, 571, 576
reset, 570
shared_ptr, 567
static_pointer_cast, 572
swap, 570, 572
unique, 571
use_count, 570
shift
valarray, 988
showbase, 1027
showmanyc
basic_filebuf, 1081
basic_streambuf, 1037, 1081
showpoint, 1027
showpos, 1028
shrink_to_fit
basic_string, 665

deque, 781
vector, 799
SHRT_MAX, 476
SHRT_MIN, 476
shuffle, 906
shuffle_order_engine, 952
constructor, 953
sig_atomic_t, 494
SIG_DFL, 494
SIG_ERR, 494
SIG_IGN, 494
SIGABRT, 494
SIGFPE, 494
SIGILL, 494
SIGINT, 494
signal, 494
<signal.h>, 493
signaling_NaN
numeric_limits, 473
signbit, 1005
SIGSEGV, 494
SIGTERM, 494
sin, 991, 1003
complex, 933
sinh, 991, 1003
complex, 933
size
array, 775, 776
basic_string, 664
bitset, 537
gslice, 995
initializer_list, 493
match_results, 1122
seed_seq, 957
slice, 992
size_t, 114, 467
size_type
allocator_traits, 547
skipws, 1028
sleep_for
this_thread, 1164
sleep_until
this_thread, 1164
slice, 991
slice, 991
slice_array, 992
snextc
basic_streambuf, 1034
sort, 909
list, 795
forward_list, 788
sort_heap, 917
splice
list, 794
list, 794
splice_after
forward_list, 787
sputbackc
basic_streambuf, 1034
sputc
basic_streambuf, 1035
sputn
basic_streambuf, 1035
sqrt, 991, 1003
complex, 933
<sstream>, 1067
<staarg.h>, 494
stable_partition, 907
stable_sort, 909
<stack>, 847
stack, 846
swap, 849
start
gslice, 995
slice, 992
state
fpos, 1021
match_results, 1122
wbuffer_convert, 700
wstring_convert, 699
state_type
char_traits, 648
wbuffer_convert, 701
wstring_convert, 699
static_pointer_cast
shared_ptr, 572
<stdalign.h>, 494
<stdarg.h>, 493
<stdbool.h>, 494
stddev
normal_distribution, 969
<stdexcept>, 496
<stdlib.h>, 493, 1251
stod, 683
stof, 682, 683
stoi, 682, 683
stol, 682, 683
stold, 682, 683
stoll, 682, 683
store
atomic type, 1150
stoul, 682, 683

stoull, 682, 683
str
    basic_istringstream, 1073
    basic_ostringstream, 1075
    basic_stringbuf, 1069, 1070
    basic_stringstream, 1077
    istrstream, 1259
    match_results, 1123
    ostrstream, 1260
    strstream, 1262
    strstreambuf, 1256
    sub_match, 1115
strchr, 684
<streambuf>, 1030
streambuf, 1008, 1030
streamoff, 1013, 1022, 1252
streamsize, 1013
    ios_base, 1019
strftime, 729
stride
    gslice, 995
    slice, 992
<string>, 652
stringbuf, 1008, 1067
stringstream, 1008
strlen, 1256, 1260
strpbrk, 684
strrchr, 684
strstr, 684
strstream, 1261
    destructor, 1261
    strstream, 1261
strstreambuf, 1253, 1255
    strstreambuf, 1255
    destructor, 1256
    setg, 1256
student_t_distribution, 972
    constructor, 973
    mean, 973
sub_match, 1114
    compare, 1115
    constructor, 1114
    length, 1114
    operator basic_string, 1114
    operator!=, 1115–1119
    operator<, 1115–1119
    operator«, 1119
    operator<=, 1115–1119
    operator==, 1115–1119
    operator>, 1115–1119
    operator>=, 1115–1119
    str, 1115
substr
    basic_string, 676
subtract_with_carry_engine, 949
    constructor, 949, 950
suffix
    match_results, 1123
sum
    valarray, 988
sungetc
    basic_streambuf, 1034
swap, 515, 531
    pair, 520
    array, 776, 777
    basic_filebuf, 1080
    basic_fstream, 1090
    basic_ifstream, 1085, 1086
    basic_ios, 1026
    basic_iostream, 1053
    basic_istream, 1043
    basic_istringstream, 1073
    basic_ofstream, 1088
    basic_ostream, 1056
    basic_ostringstream, 1075
    basic_regex, 1114
    basic_streambuf, 1035
    basic_string, 672, 680
    basic_stringbuf, 1069
    basic_stringstream, 1077
    deque, 782
    forward_list, 789
    function, 599, 600
    list, 796
    map, 811
    match_results, 1124
    multimap, 815
    multiset, 822
    packaged_task, 1206, 1207
    pair, 519
    priority_queue, 846
    promise, 1196, 1198
    queue, 844
    set, 818
    shared_lock, 1182, 1183
    shared_ptr, 570, 572
    stack, 849
    thread, 1162, 1163
    tuple, 527
    unique_lock, 1179
    unique_ptr, 561
    unordered_map, 829

unordered_multimap, 833
unordered_multiset, 840
unordered_set, 837
valarray, 987, 991
vector, 799, 801
vector<bool>, 803
weak_ptr, 575, 576
swap(unique_ptr&, unique_ptr&), 563
swap_ranges, 901
sync
basic_filebuf, 1084
basic_istream, 1051
basic_streambuf, 1036
sync_with_stdio
ios_base, 1019
syntax_option_type, 1102
awk, 1102
basic, 1102
collate, 1102, 1138
ECMAScript, 1102
egrep, 1102
extended, 1102
grep, 1102
icase, 1102
nosubs, 1102
optimize, 1102
syntax_option_type
awk, 1103
basic, 1103
collate, 1103
ECMAScript, 1103
egrep, 1103
extended, 1103
grep, 1103
icase, 1103
nosubs, 1103
optimize, 1103
system, 493, 494
system_category, 503, 505
system_clock
rep, 637
system_error, 500, 510
code, 511
system_error, 510, 511
what, 511

t
binomial_distribution, 962
negative_binomial_distribution, 963
table
ctype<char>, 707
tan, 991, 1003
complex, 933
tanh, 991, 1003
complex, 933
target
function, 599
target_type
function, 599
tellg
basic_istream, 1051
tellp
basic_ostream, 1057
terminate, 479, 490, 1262
terminate_handler, 461, 490
test
bitset, 537
tgamma, 1003
this_thread
get_id, 1164
sleep_for, 1164
sleep_until, 1164
yield, 1164
thousands_sep
moneypunct, 734
numpunct, 721
<thread>, 1159
thread
constructor, 1161, 1162
destructor, 1162
detach, 1163
get_id, 1163
hardware_concurrency, 1163
join, 1163
joinable, 1163
operator=, 1162
swap, 1162, 1163
thread::id
constructor, 1161
operator!=, 1161
operator<, 1161
operator<=, 1161
operator<<, 1161
operator==, 1161
operator>, 1161
operator>=, 1161
throw_with_nested
nested_exception, 492
tie, 528
basic_ios, 1024
time, 493
<time.h>, 493

time_get, 724
    date_order, 725
    do_date_order, 726
    do_get, 727
    do_get_date, 727
    do_get_monthname, 727
    do_get_time, 726
    do_get_weekday, 727
    do_get_year, 727
    get, 726
    get_date, 725
    get_monthname, 725
    get_time, 725
    get_weekday, 725
    get_year, 725
time_get_byname, 728
time_point
    constructor, 635
    max, 636
    min, 636
    operator!=, 636
    operator+, 636
    operator+=, 635
    operator-, 636
    operator-=, 635
    operator<, 636
    operator<=, 636
    operator==, 636
    operator>, 637
    operator>=, 637
    time_since_epoch, 635
time_point_cast, 637
time_put, 728
    do_put, 729
    put, 729
time_put_byname, 729
time_since_epoch
    time_point, 635
tinyness_before
    numeric_limits, 474
to_bytes
    wstring_convert, 699
to_string, 682
    bitset, 537
to_time_t, 637
to_ullong
    bitset, 536
to_ulong
    bitset, 536
to_wstring, 683
tolower, 697
    ctype, 703
    ctype<char>, 707
toupper, 697
    ctype, 703
    ctype<char>, 707
transform, 902
    collate, 723
    regex_traits, 1106
transform_primary
    regex_traits, 1106
translate
    regex_traits, 1106
translate_nocase
    regex_traits, 1106
traps
    numeric_limits, 473
treat_as_floating_point, 627
truename
    numpunct, 721
trunc, 1003
try_emplace
    map, 810
    unordered_map, 828
try_lock, 1183
    shared_lock, 1182
    unique_lock, 1178
try_lock_for
    shared_lock, 1182
    unique_lock, 1178
try_lock_until
    shared_lock, 1182
    unique_lock, 1178
try_to_lock, 1174
try_to_lock_t, 1174
<tuple>, 521
tuple, 521, 523, 777
    constructor, 524–526
    forward_as_tuple, 528
    get, 529, 530
    make_tuple, 527
    operator!=, 530
    operator<, 530
    operator<=, 531
    operator=, 526, 527
    operator==, 530
    operator>, 531
    operator>=, 531
    swap, 527
    tie, 528
    tuple, 524
tuple_cat, 528

tuple_element, 520, 529, 777
tuple_size, 520, 528, 777
    in general, 528
type_index
    constructor, 645
    hash_code, 645
    name, 645
    operator!=, 645
    operator<, 645
    operator<=, 645
    operator==, 645
    operator>, 645
    operator>=, 645
type_info, 107, 486
type_info::name
    implementation-defined, 487
<typeinfo>, 486, 644

UCHAR_MAX, 476
uflow
    basic_filebuf, 1082
    basic_streambuf, 1038
uint16_t, 477
uint32_t, 477
uint64_t, 477
uint8_t, 477
uint_fast16_t, 477
uint_fast32_t, 477
uint_fast64_t, 477
uint_fast8_t, 477
uint_least16_t, 477
uint_least32_t, 477
uint_least64_t, 477
uint_least8_t, 477
UINT_MAX, 476
uintmax_t, 477
uintptr_t, 477
ULLONG_MAX, 476
unary_negate, 592
uncaught_exception, 1262
uncaught_exceptions, 424, 490
undeclare_no_pointers, 545
undeclare_reachable, 544
underflow
    basic_filebuf, 1081
    basic_streambuf, 1037
    basic_stringbuf, 1070
    strstreambuf, 1257
underflow_error, 496
    underflow_error, 500
unexpected, 1262
unexpected_handler, 461, 1262
unget
    basic_istream, 1051
uniform_int_distribution, 958
    a, 959
    b, 959
    constructor, 959
uniform_real_distribution, 959
    a, 960
    b, 960
    constructor, 960
uninitialized_copy, 552
uninitialized_copy_n, 552
uninitialized_fill, 552
uninitialized_fill_n, 553
unique, 904
    list, 795
    forward_list, 788
    shared_ptr, 571
unique_copy, 905
unique_lock
    constructor, 1176, 1177
    destructor, 1177
    lock, 1178
    mutex, 1179
    operator bool, 1179
    operator=, 1177
    owns_lock, 1179
    release, 1179
    swap, 1179
    try_lock, 1178
    try_lock_for, 1178
    try_lock_until, 1178
    unlock, 1178
unique_ptr, 569
    constructor, 558, 559, 562
    destructor, 559
    get, 560
    get_deleter, 560
    operator bool, 560
    operator!=, 563, 564
    operator*, 560
    operator->, 560
    operator<, 563, 564
    operator<=, 564
    operator=, 559, 560, 562
    operator==, 563, 564
    operator>, 564
    operator>=, 564, 565
    operator[], 563
    release, 560

reset, 560, 563
swap, 561
unique_ptr, 557, 558
unitbuf, 1028
unlock
shared_lock, 1182
unique_lock, 1178
<unordered_map>, 822
unordered_map, 822–824
at, 828
insert, 828
insert_or_assign, 829
operator[], 827
swap, 829
try_emplace, 828
unordered_map, 827
unordered_multimap, 822, 829
insert, 833
swap, 833
unordered_multimap, 832
unordered_multiset, 823, 837
swap, 840
unordered_multiset, 840
<unordered_set>, 823
unordered_set, 823, 833
swap, 837
unordered_set, 836
unsetf
ios_base, 1019
unshift
codecvt, 709
upper_bound, 911
uppercase, 1028
use_count
shared_ptr, 570
weak_ptr, 575
use_facet
locale, 696
uses_allocator, 546, 1196, 1207
uses_allocator<tuple>, 531
USHRT_MAX, 476
<utility>, 512

va_arg, 494
va_copy, 494
va_end, 460, 494
va_list, 460, 494
va_start, 494
<valarray>, 978
valarray, 981, 995
begin, 999
constructor, 983
destructor, 983
end, 999
operator!=, 990
operator*, 989
operator*=, 987
operator+, 989
operator+=, 987
operator-=, 987
operator/, 989
operator/=, 987
operator<, 990
operator<=, 990
operator<<, 989
operator<<=, 987
operator=, 984, 989
operator==, 990
operator>, 990
operator>=, 990
operator>>, 989
operator>>=, 987
operator%, 989
operator%=, 987
operator&, 989
operator&=, 987
operator&&, 990
operator^, 989
operator^=, 987
operator|, 989
operator|=, 987
operator||, 990
swap, 987, 991
valarray, 983
valid
future, 1199
packaged_task, 1206
shared_future, 1202
value
error_code, 507
error_condition, 509
regex_traits, 1108
<vector>, 774
vector, 796
operator<, 798
operator==, 798
vector, 798, 799
swap, 801
vector<bool>, 801
flip, 803
swap, 803
void_pointer

allocator_traits, 547

wait
condition_variable, 1187
condition_variable_any, 1190, 1191
future, 1200
shared_future, 1202
wait_for
condition_variable, 1188, 1189
condition_variable_any, 1191, 1192
future, 1200
shared_future, 1202
wait_until
condition_variable, 1187, 1188
condition_variable_any, 1191
future, 1200
shared_future, 1203
wbuffer_convert, 700
constructor, 701
destructor, 701
rdbuf, 700, 701
state, 700
state_type, 701
wcerr, 1012
wcin, 1012
wclog, 1012
wcout, 1012
wcschr, 685
wcspbrk, 685
wcsrchr, 685
wcsstr, 685
weak_ptr, 569, 573
constructor, 574, 575
destructor, 575
expired, 575
lock, 575
operator=, 575
reset, 575
swap, 575, 576
use_count, 575
weibull_distribution, 966
a, 967
b, 967
constructor, 967
wfilebuf, 1008, 1077
wfstream, 1008
what
bad_alloc, 485
bad_array_new_length, 485
bad_cast, 487
bad_exception, 489
bad_typeid, 488
exception, 489
bad_weak_ptr, 565
future_error, 1194
system_error, 511
wide_string
wstring_convert, 699
widen
basic_ios, 1025
ctype, 703
ctype<char>, 707
width
ios_base, 701, 1019
wifstream, 1008, 1077
wios, 1013
wistream, 1008, 1040
wistringstream, 1008, 1067
wmemchr, 685
wofstream, 1008, 1077
wostream, 1008, 1040
wostringstream, 1008, 1067
wregex, 1095
write
basic_ostream, 1061
ws, 1046, 1052
wstreambuf, 1008, 1030
wstring_convert, 697
byte_string, 698
constructor, 699
converted, 698
destructor, 699
from_bytes, 698
int_type, 699
state, 699
state_type, 699
to_bytes, 699
wide_string, 699
wstringbuf, 1008, 1067
wstringstream, 1008

xalloc
ios_base, 1020
xsgetn
basic_streambuf, 1037
xsputn
basic_streambuf, 1039

yield
this_thread, 1164

zero

duration, 631
duration_values, 627

Index of implementation-defined behavior

The entries in this section are rough descriptions; exact specifications are at the indicated page in the general text.

`#pragma`, 435

additional formats for `time_get::do_get_date`, 727
alignment, 79
alignment additional values, 79
alignment of bit-fields within a class object, 239
allocation of bit-fields within a class object, 239
argument values to construct `basic_ios::failure`, 1026
assignability of placeholder objects, 595

behavior of attribute scoped token, 185
behavior of iostream classes when `traits::pos_type` is not `streampos` or when `traits::off_type` is not `streamoff`, 1008
behavior of non-standard attributes, 185
bits in a byte, 6

choice of larger or smaller value of floating literal, 27
concatenation of some types of string literals, 29
conversions between pointers and integers, 110
converting characters from source character set to execution character set, 17
converting function pointer to object pointer and vice versa, 110

default number of buckets in `unordered_map`, 827
default number of buckets in `unordered_multimap`, 832, 833
default number of buckets in `unordered_multiset`, 840
default number of buckets in `unordered_set`, 836, 837
defining `main` in freestanding environment, 61
definition and meaning of `__STDC__`, 435
definition and meaning of `__STDC_VERSION__`, 435
derived type for `typeid`, 107
diagnostic message, 2
dynamic initialization of static objects before `main`, 63
dynamic initialization of thread-local objects before entry, 63

effect of calling `basic_filebuf::setbuf` with non-zero arguments, 1083
effect of calling `basic_filebuf::sync` when a get area exists, 1084
effect of calling `basic_streambuf::setbuf` with non-zero arguments, 1072
effect of calling `ios_base::sync_with_stdio` after any input or output operation on standard streams, 1019
effect on C locale of calling `locale::global`, 696
encoding of universal character name not in execution character set, 26
`error_category` for errors originating outside the operating system, 466
exception type when `shared_ptr` constructor fails, 568
exceptions thrown by standard library functions that do not have an exception specification, 466
execution character-set and execution wide-character set, 18
exit status, 479
extended signed integer types, 74

formatted character sequence generated by `time_put::do_put` in C locale, 729

headers for freestanding implementation, 448

interactive device, 8

linkage of `main`, 61
linkage of names from Standard C library, 449
locale names, 694

manner of search for included source file, 428
mapping from name to catalog when calling `messages::do_open`, 736
mapping header name to header or external source file, 20
mapping physical source file characters to basic source character set, 16
mapping to message when calling `messages::do_get`, 737

meaning of `asm` declaration, 181
meaning of attribute declaration, 147

negative value of character literal in preprocessor, 427
nesting limit for `#include` directives, 428
number of threads in a program under a freestanding implementation, 11
numeric values of character literals in `#if` directives, 427

parameters to `main`, 61
passing argument of class type through ellipsis, 103
physical source file characters, 16
presence and meaning of `native_handle_type` and `native_handle`, 1156

rank of extended signed integer type, 86
representation of `char`, 74
required libraries for freestanding implementation, 5
result of `exception::what`, 489
result of inexact floating-point conversion, 84
result of right shift of negative value, 125
return value of `bad_alloc::what`, 485
return value of `bad_array_new_length::what`, 485
return value of `bad_cast::what`, 487
return value of `bad_exception::what`, 489
return value of `bad_typeid::what`, 488
return value of `char_traits<char16_t>::eof`, 650
return value of `char_traits<char32_t>::eof`, 651
return value of `type_info::name()`, 487

search locations for `""` header, 428
search locations for `<>` header, 428
semantics of linkage specification on templates, 333
semantics of linkage specifiers, 181
semantics of non-standard escape sequences, 26
sequence of places searched for a header, 428
signedness of `char`, 74
`sizeof` applied to fundamental types other than `char`, `signed char`, and `unsigned char`, 114
stack unwinding before call to `std::terminate()`, 417, 424
start-up and termination in freestanding environment, 61
string resulting from `__func__`, 207
string returned by `what()` for `bad_weak_ptr`, 565, 596
support for extended alignment, 620
support for over-aligned types, 114, 550, 552
supported multibyte character encoding rules, 649, 652

text of `__DATE__` when date of translation is not available, 435
text of `__TIME__` when time of translation is not available, 435
type of `ios_base::streamoff`, 1252
type of `ios_base::streampos`, 1252
type of `ptrdiff_t`, 124, 468
type of `regex_constants::error_type`, 1104
type of `size_t`, 468
type of `streamoff`, 649
type of `streampos`, 649
type of `u16streampos`, 650
type of `u32streampos`, 651
type of `wstreampos`, 651
type of `array::const_iterator`, 775
type of `array::iterator`, 775

underlying type for enumeration, 167
use of non-POF function as signal handler, 494

value of `ctype<char>::table_size`, 706
value of bit-field that cannot represent
  assigned value, 130
  incremented value, 105
  initializer, 213
value of character literal outside range of corresponding type, 26
value of multicharacter literal, 25
value of result of inexact integer to floating-point conversion, 85
value of result of unsigned to signed conversion, 84
value of wide-character literal containing multiple characters, 26
value of wide-character literal with single c-char that is not in execution wide-character set, 26
value representation of floating-point types, 75
value representation of pointer types, 76
values of a trivially copyable type, 72
values of various `ATOMIC_..._LOCK_FREE` macros, 1144

whether `get_pointer_safety` returns `pointer_safety::relaxed` or `pointer_safety::preferred` if the implementation has relaxed pointer safety, 545

whether `time_get::do_get_year` accepts two-digit year numbers, 727
whether an implementation has relaxed or strict pointer safety, 68
whether locale object is global or per-thread, 691
whether sequence pointers are copied by `basic_filebuf` move constructor, 1079
whether sequence pointers are copied by `basic_stringbuf` move constructor, 1069
whether source of translation units must be available to locate template definitions, 17
whether stack is unwound before calling `std::terminate()` when a `noexcept` specification is violated, 424
whether values are rounded or truncated to the required precision when converting between `time_t` values and `time_point` objects., 637
which functions in Standard C++ library may be recursively reentered, 464